ACERVATOR

User Manual and Feature Design Intention Guide

Turbator aequilibrii dissolvendus reformandusque.

—----------

“The disturber of the balance is to be dissolved and reformed.”

“You are either gaining more territory cheaper or you are selling for higher than you bought it in direct response to the market’s moment to moment volatility.” - Ekthelius

Contents

Part 1 Frontmatter & Overview **11**
Copyright and Attribution 11
Contact 11
Legal Disclaimers 11
Risk Factors Specific to Algorithmic Trading 12
Executive Summary 12
What Acervator Is and What It Is Aspiring To Be… 12
Product Manual Parts 14
Part 2 Novel Concepts and Patent Candidate Portfolio **16**
Novel Concepts and Patent Candidate Catalogue 16
1 - Harvest-Fold / Scrumming 16
2 - The Phantom Bot 24
3 - The Landing Strip 33
4 - Bollinger Band Travel Detection 36
5 - Position-Aware Technical Analysis 37
6 - Market Inspector 39
7 - Bot Swarm and Smart Wire Network w/ Provenance Tracking 40
8 - Hunger and Satiety Indices 42
9 - Accumulation Shadows 45
10 - Charge Up Permission Gate 47
11 - Fold, Stack, Extractor, and Shadow Tranches / Provenance Queue 48
12 - Technical Analysis Indicator Confidence Tiers 59
13 - Initial-Purchase-Price Floor on Folds 60
14 - Position Ceiling and Detonation 64
15 - Proof of Accumulation (PoA) 67
16 - Technical Analysis, Goodness of Fit, and the Confidence Index 68
17 - Tiered Strategy Validation and Workflow 69
Part 3 System Architecture and Features Catalogue **71**
System Architecture and Features Catalogue 71
The mark 72
Trading Tab 73
How to use the Trading tab 89
How to use the Bot Wizard 98
How to use a running bot 104
Indicator Voting Panel 108
The twelve voters 118
Indicator formulae 120
Departures from the published maths 132
2026-09-08 15:12 - #414 - the ratio the gate reads 133

- 2026-09-08 15:54 - #414 - when ADX first has something to say 134
- 2026-09-08 16:22 - #414 - the MACD reading at a millionth of a dollar 136
- 2026-09-08 16:52 - #414 - the money flow a halted market never traded 138
- 2026-09-08 17:45 - #414 - the Vortex reports the Vortex 140
- 2026-09-08 19:05 - #414 - the RSI label and the RSI vote read one number 142
- 2026-09-09 19:37 - #414 - the band a millionth of a dollar wide 145
- Trading gate logic chain 148
- 2026-09-24 22:50 - #202 - one admission rule for every panel reading 161
- 2026-10-24 - #665 - a gate row carries the panel it was decided on 162
- 2026-10-25 - #665 - the consolidation figure is declared once 162

Subsystem Tabs 163
- Main Window 167
- Inspector 174
- Charts 178
- Live Tab 183
- Converting the Interface to React 185
- 2026-10-28 - #1374 - where the owed rebuild now stands 233

Subsystem Detail 234
- How an update is written down 234
- Contents 235
- The live tab set 235
- How a screen reaches its renderer 237

Portfolio Information Panels 239
- What builds it 239
- Left: the spendable columns 240
- Right: the five counter cards 244
- Privacy 246
- Where the numbers come from 247
- Hiding 247
- Bridge 248

Simulator Tab (Hot Mess; Complete Rebuild In Progress) 248
- How to use the Simulator tab 249
- How to read a run 257
- How to use the Bot Wizard on this tab 259
- How to use the Bot Settings window on this tab 262

Paper Trader Tab 264
- The empty tab it replaced 264
- What the step is 265
- Where the platform once offered it 265
- The screen as it stands now 265
- The two presses 267
- Downstream only 267

- One gate chain ... 267
- The fake balance ... 268
- The fleet ledger ... 269
- The Paper Trader log ... 270
- Real time ... 271
- Paper Trade History ... 271
- 2026-09-08 08:17 - #19 - what the closed issues landed ... 271
- How the Paper tab reaches the bar ... 274
- The style sheet the Paper page loads ... 275
- 2026-09-19 - #19 - Live's tab code cloned under Paper names ... 276
- 2026-09-20 - #19 - The paper exchange adapter feeds the tab ... 286
- 2026-09-20 - #19 - The paper bot ticks and fills on the clone ... 291
- 2026-09-24 - #858 - the bot list upgrades reach the Paper Trader ... 302
- 2026-09-24 - #571 - the asset class group and Add Exchange reach the Paper Trader ... 304
- 2026-09-25 - #19 - a paper bot's settings reach its stored record ... 307
- 2026-09-26 - #19 - a paper fill draws its own Activity Log line ... 309
- 2026-09-26 - #19 - a paper bot is created from scratch ... 310
- 2026-10-04 - #1103 - a pushed candidate spawns paper bots ... 312
- A stored figure no box can hold draws a dash ... 315
- 2026-10-18 - #1203 - one resolver answers a market's sector ... 316
- 2026-10-20 - #937 - a Paper row's state sits on its Symbol cell ... 319
- 2026-10-25 - #937 - the Paper page draws the Asset column's mark ... 320
- 2026-10-25 - #665 - the paper record carries the wizard's lock figure ... 321
- 2026-10-25 - #665 - the Paper wizard reads the one Hedge Balance figure ... 322
- 2026-10-26 - #665 - the Paper row names the fold taper by size and shows the detonation bar ... 322
- 2026-10-27 - #1125 - a pair with no recorded row answers no sector at all ... 322
- CITED AS ABSENT ... 323

Proof of Accumulation ... 323

- The tab the window does not build ... 324
- What the screen declares ... 325
- Identity ... 326
- The trade log ... 327
- The competition lifecycle ... 327
- The token ledger ... 327
- The five rarity tiers ... 328
- The token contract ... 329
- The Quintessence contract ... 330
- Governance, the franchise and the halt council ... 331
- Trophies ... 332
- The chain ... 332
- Quintessence ... 334
- The certified fill ... 337

The capture bounds on one award 338
The rotating reward set 339
The six-month project age rule 339
The event modes and the turn 340
The action bands and the record store 341
The pot divides on performance 342
Head to head 342
The world grid 343
Movement, and the world turn's pool 344
Dungeons and the second clock 345
The Vessel 345
Stats, and the metrics they convert from 345
The seven character classes 346
The skill ladder and the transfer skill 346
Materials, items and crafting 347
Loot 347
Monsters 348
Alignment, prayer and consecrated places 349
Domination, and the two forms a taking takes 349
The PvP vote 350
Guilds and the treasury 350
Linking two nodes 351
Map marks 351
Every rate, and its provenance 352
The party row's mark 352
The two meters, and the reset 352
What the engine does on the Demo chain today 353
What is not built 353
Market Inspector Tab 354
What builds it 354
The six zones 354
Left half: the scanner 357
Where the analysis happens 377
Fetching 378
Right half: topology proposals 386
Dismissal 389
Adopt 389
Push to the Simulator, the Paper Trader and Live 390
ATA-SMP 392
Multi-Exchange Arbitrage 423
Phantom Bot HTF Signals 423
The asset logo library 425

- What the emitters write ... 431
- Bridge ... 432

Bot Swarm Tab ... 435

- What builds it ... 435
- The nodes ... 436
- The live rows ... 437
- Drawing a wire ... 437
- The flow along a wire ... 438
- Routing the profit ... 438
- Landing the profit ... 443
- The tranche book ... 443
- Where the fleet comes from ... 446
- Capital claims ... 446
- Bridge ... 447
- 2026-09-07 15:58 - no issue recorded - ten single-word tabs, each on its own ground ... 448
- 2026-09-09 22:12 - #128 - one view, growth stages, and wires that hang ... 453
- 2026-09-09 22:12 - #128 - the Swarm tab loses List View ... 458
- 2026-09-23 - #239 - Disconnect All clears the key the fleet is stored under ... 459
- 2026-09-23 - #239 - every control read on the running program ... 461
- CITED AS ABSENT ... 465

Asset Charts ... 465

- What builds it ... 466
- The chart widget ... 466
- Indicators on the chart ... 467
- Where the candles come from ... 468
- The other chart ... 469
- Bridge ... 469
- 2026-09-08 13:30 - #407 - a Live / ATA-SMP toggle over the arrows ... 469
- 2026-09-08 13:30 - #407 - the arrows now reach the renderer ... 471
- 2026-09-13 15:40 - #23 - the Charts tab draws in the desktop window ... 471
- 2026-09-15 10:38 - #55 - one painter draws the chart in both builds ... 474
- 2026-09-19 08:23 - #55 - every Voting Panel indicator on the chart ... 476
- 2026-09-19 17:40 - #55 - the painter's skin and the Acervator annotations ... 480
- 2026-09-19 20:10 - #55 - the ATA-SMP list draws the venue image's picture ... 485
- 2026-09-19 20:53 - #55 - the page's chart answers the pointer as the widget does ... 488
- 2026-09-20 05:40 - #55 - the frame: one control row, compact toggles, the value field on the A… ... 492
- 2026-09-20 06:10 - #55 - the trade events from the History; the floors gone ... 495
- 2026-09-20 09:45 - #55 - the oscillators: one readable sub-pane height, whole-pixel pens, tags… ... 498
- 2026-09-23 17:30 - #55 - the oscillators are a third taller, and the bottom bar drags both heights ... 502
- 2026-09-23 22:10 - #55 - the trade markers draw on every timeframe ... 504
- 2026-10-16 - #230 #223 - one unreadable price costs one candle, and a stored flag never print… ... 507
- 2026-10-16 - #230 #223 - the other five stored prices admit before the chart draws them ... 509

2026-10-19 - #230 - the header names the candle it left out, and the dead bullseye list goes 511
2026-10-20 - #230 - the grid line list, its setter and its published field go 513
History Tab 515
What builds it 515
Fetching 515
Where the trade record is kept 516
Filtering, paging, export 517
Where the cell values come from 518
Grading 519
Gate analysis 520
When the logs are read 521
The renderer 522
The Simulator front-load 522
Paper History 523
Bridge 523
2026-09-08 08:17 - no issue recorded - what the closed issues landed 523
2026-10-06 - #1133 - the trade historian's own fetch 527
Console 528
What builds it 529
Upper pane: the Python log 529
Pausing 531
Lower pane: the Emitter Network 531
Where the lower pane goes 532
Bridge 532
2026-09-07 15:58 - no issue recorded - ten single-word tabs, each on its own ground 533
2026-09-23 - #34 - the drained record carries three more fields 540
2026-10-20 - #1392 - a gate refusal reads REFUSED, once a candle 541
2026-10-21 - #1392 - the state lines take the same guard 542
2026-10-28 - #1374 - a locked launch starts no bot and the Console says what it waits for 544
2026-10-28 - #1374 - the screen that takes the password is built 546
2026-10-28 - #1374 - a locked launch reads the prices a screen draws for a market 547
2026-10-28 - #1374 - a sealed fleet makes no new bot 548
System Status Tab 550
The skeleton 550
Half one: the Emitter Network 551
Half two: the Watchdog 553
Until the tab lands 553
2026-09-08 08:17 - #34 - what the closed issues landed 553
Settings 573
The pages 574
What each page persists 575
How to use each page 576

The Promotion Pipeline ... 594
What each step contributes ... 595
One logic, three data sources ... 595
The dashed edge ... 595
Where the chain breaks today ... 596
Notifications and Alerts ... 596
What the Event Routing table is for ... 596
The columns come from one declaration ... 596
Which events reach the message channels ... 597
The dollar figure in a fill message ... 597
The state of this screen ... 598
Part 4 Live and Supporting Evidence ... 599
Part 5 Conceptual Hopscotch: The HOP Protocol, Rules Registry, and SADP v1 and v2 ... 600
Part 6 ADR Index & Glossary ... 601
Decision Records and Glossary ... 601
Where a decision is recorded ... 601
Glossary ... 601
The operator's own words ... 604
Part 7 Recent Updates and Version Tracking ... 605
Part 8 The Development Chronicle As Told By Claude ... 606
Part 9 Live Trade History and Supporting Evidence ... 607
VWAP Charts and Trade Grading ... 607
The record behind the charts ... 607
Subtotal is the price ... 607
How a VWAP is computed here ... 608
What the combined chart plots ... 608
The buy and sell ratio ... 610
Traps in the export ... 612
Where the figures live ... 612
The charts are tracked, and they name no asset ... 612
How to read a per-asset chart ... 613
The Bollinger band behind the price ... 613
The newest export, beside the one the figures above measure ... 617
The October export, and the base that makes 39 ... 617
What this repository holds for these charts ... 626
The builder draws every VWAP chart ... 627
The 38 per-asset charts ... 630
Trade grading ... 708
Gate logs behind each trade ... 708
Weekly candles behind the fills ... 710
The price at the finest resolution the record carries ... 711
Part 10 The Sigil Project ... 712

The Sigil Project 712
What a sigil is 712
The scope test 712
The three input tiers 713
Flags and follow-on rules 714
The shared modules 714
Portability 715
What a sigil cannot guarantee 715
Progress 716
The trial pilot 716
Application archetypes 717
The sigil pages 718
CITE.SYMBOL@3 718
What it reads 718
Outputs and flags 719
The command 719
Reasons on the card 719
Rules left off 720
CODE.OPTIMIZE@1 720
What it reads 720
Outputs and flags 721
The command 721
Rule families 721
Families left off 722
CODE.TYPE@1 722
What it reads 722
Outputs and flags 723
The command 723
Tools and rules 723
Rules left off 723
CODE.SECURE@1 723
What it reads 724
Outputs and flags 724
The command 724
Tools and rule sets 724
Rules left off 725
CODE.REACH@1 725
What it reads 725
Outputs and flags 725
The command 726
Tools and rules 726
Rules left off 726

CODE.LOOP@1 726
What it reads 726
Outputs and flags 727
The command 727
Rules 727
Rules left off 729
CODE.IMPORT@1 729
What it reads 729
Outputs and flags 730
The command 730
Rules 730
Rules left off 731
CODE.COLLECTION@1 732
What it reads 732
Outputs and flags 732
The command 733
Rules 733
Rules left off 735
CODE.FLOW@1 735
What it reads 735
Outputs and flags 736
The command 736
Rules 736
Rules left off 738
CODE.VARIABLE@1 739
What it reads 739
Outputs and flags 740
The command 740
Rules 740
Rules left off 743

Part 1

Frontmatter & Overview

Copyright and Attribution

Copyright © 2025–2026 Ekthelius the Accumulator. All rights reserved. Unauthorized reproduction, distribution, modification, or derivative work is prohibited.

This document describes systems, algorithms, and methods developed by the author over the course of a self-teaching period starting in 2017.

Produced using the SADP (Structured Assistant Development Protocol) discipline framework designed by Ekthelius and deployed within Claude Code as a Skills-based and python-driven harness.

Contact

Acervator is the personal project of Zavyrian Productions and Ekthelius the Accumulator. Commercial inquiries, licensing requests, and networking requests are available via the channels listed at publication time:

ekthelius_the_accumulator@protonmail.com

Legal Disclaimers

NOT INVESTMENT ADVICE Nothing in this document constitutes investment advice, a solicitation to buy or sell any security or financial instrument, a recommendation of any trading strategy, or an offer to engage in any financial transaction. Acervator is a software system, and software cannot give investment advice. Trading cryptocurrencies and digital assets involves substantial risk of loss. Acervator is a highly complex, professional, and stable tool for automating digital asset trades. It does not and cannot guarantee the ultimate state of a portfolio given any number of transactions executed through or by it based upon a given user's configuration.

NOT A REGISTERED INVESTMENT ADVISOR The author is not a registered investment advisor, broker-dealer, or fiduciary. No fiduciary relationship is created by distribution of this document. Consult a qualified financial professional before making any investment decision. Ekthelius is a self-taught Technical Analyst with no official documentation to certify or desire to certify his skillset beyond his own personal portfolio results. They have been developed and achieved the hard way.

SOFTWARE PROVIDED AS-IS

The Acervator software system is provided as-is without warranty of any kind, express or implied, including merchantability, fitness for a particular purpose, and noninfringement. Users assume all risk associated with deployment. The author is not liable for financial losses, system failures, or downstream damages resulting from use of the software. Training, guidance, and general set up assistance can be provided but the vast majority of help one needs will be found in the eight part documentation so read it and understand what you are trying to use.

JURISDICTIONAL COMPLIANCE

Users are solely responsible for compliance with all applicable laws and regulations in their jurisdiction, including but not limited to: securities regulations, cryptocurrency-specific rules, KYC/AML requirements, tax reporting obligations, and exchange terms of service. Acervator does not assess or enforce jurisdictional compliance.

PATENT AND INTELLECTUAL PROPERTY NOTICE

The mechanisms catalogued in Part 2 are claimed as novel inventions by the author. No patent has been granted or filed for any of them. Research, academic, and personal study use is permitted. Commercial or competitive use requires written license.

Risk Factors Specific to Algorithmic Trading

Algorithmic trading systems can fail in ways manual trading cannot. Known risk categories include: (1) exchange API failures and rate limits, (2) network latency and partial fills, (3) flash-crash and liquidity-gap events that bypass stop conditions, (4) stuck positions during exchange downtime, (5) adverse selection on thinly-traded pairs, (6) slippage exceeding model assumptions, (7) parameter drift as market microstructure evolves, (8) bugs in the trading code itself. Acervator includes explicit mitigations for these failure modes (see Part 4 · Risk Controls), but no mitigation is complete and Acervator is a platform that is being actively developed. A trader using any algorithmic system must understand these risks before deploying capital.

Executive Summary

What Acervator Is and What It Is Aspiring To Be…

Acervator is a desktop trading platform whose innovation is the platform itself. The breakthrough is not a feature, an indicator, or a bot — it is the integrated system. A harvest-fold accumulation engine, a twelve-voter Indicator Voting Panel across up to five timeframes, a seventeen-class declarative GateChain safety framework, multi-bot coordination, and a live evidence trail proving the whole operates on real capital — coherent as one machine that converts price oscillation into asset accumulation and monotonic target growth without forecasting direction.

The platform will ultimately be designed to trade both cryptocurrency and stocks but is being developed to focus on the blockchain first given its current and immediate future impact on global financial systems. The crypto trading side is being prototyped via Coinbase Advanced and is the production surface today — live operator capital, fourteen weeks of execution, 5,055 trades with all assets successfully fending off the current bear trend or flat out beating it.

Harvest-Fold is the method and a mechanism. A position opens against a base currency with an explicit USD target (or a target that is translated to a USD or other local equivalent) with the initial size of the position being used as a value set point about which the mechanism of the platform allow volatility to “orbit” by persistently re-zeroing to it in response to local market structures and price ranges. The method itself stands apart from traditional Technical Analysis trading strategy or systems by not speculating or attempting to guess what the price will be within any given amount of time but instead persistently remolds a portfolio as its positions fluctuate and refusing to leave profit on the table by saying “It might be worth more later.” .

The primary “bot” or strategy that executes the Harvest-Fold method is called a Scrumming Bot and can be supported by, and this is exclusive to the cryptocurrency side of the platform, an Extractor Bot which performs more traditional scalp-style trades using isolated chunks of the parent Scrumming Bot’s Target Asset with the Extractor’s primary limitation being that it only works for established Base Currencies on a given exchange that has multiple asset pairs to exchange them against. The platform is designed and optimized in such a manner to allow “swarms” of Scrumming Bots (with or without Extractors) to be deployed to support dozens of positions across multiple exchanges simultaneously and thereby allowing the average digital asset investor to experience full spectrum digital asset trading that has previously been inaccessible or inefficient to execute properly.

The term “scrum” was initially encountered while I was learning about how a barrel of oil’s profit is split or distributed. The scrum of the barrel, according to the tutorial, was what the gas station gets as its share of profit and its insultingly small, thus requiring huge sums of fuel to be sold for any respectable amount of money to be made by the facilities hosting the gas pumps and their infrastructure. The concepts of small, repeating, and accumulating were found to rest inside on one term and so it was readily adopted. The extractor, however, is more straightforward in origin and it rests on the idea that A15 and A14 trading became the primary means of acquisition after direct mining became unaffordable for average blockchain participants. It has always been apparent that the potential for faster wealth acquisition existed via asset trading but the skill and technology gaps are what prevent this door of opportunity from opening for just about everyone.

Prior to the invention of the Harvest-Fold mechanism, I had also conceptualized grid bots via a now deprecated cryptocurrency auto-trader called Margin. When this application first appeared and was used, it only featured a simple “ping-pong” and a more speculative / adaptive “margin” bot. It should also be noted that it was never a stable product throughout all of the versions attempted and this was what led to its failure while its company became a consultant firm of some sort. I am not sure how producing an unstable, feature-limited auto-trader qualifies you as a fintech advisor but maybe qualifications remain loose or have become looser. Regardless, and in spite of the instability experienced immediately during basic use, an attempt was made to stack dozens of ping bots of a large range on a given market. This may have been the first “grid bot” used in cryptocurrency trading. The resulting and constant crashing led to my seeking help from the company via their Telegram channel where he was told, basically, that he was using it incorrectly which was an unacceptable and stupid excuse for not fixing their product. Instead, they stole said concept and added a “stack bot” in a later version of Margin that performed the same basic behavior under one bot instance but because of the fundamentally buggy quality of the software, this was not used much before a boycott was necessary because the company chose to ban people criticizing the unending stability issues. After this, Margin supposedly partnered with KuCoin to bring their technology to said exchange but this seems to have dissolved at some point with KuCoin releasing its own Grid Bots with which I was able to win 3rd place and win three Lomen NFTs as a result. Said Lomens still reside on a certain Metamask to this day should this or any other part of this tale be doubted. Whether I am a co-inventor or discoverer of modern automated cryptocurrency trading is a point for irrelevant debate. What matters is that Acervator is my newest invention and it is infinitely more powerful than anything before it.

After the United States blocked access to KuCoin, I continued to develop my trading strategies further and executed them manually while learning more Technical Analysis. In the end, high volume trading methodologies were seen as the only method for maximizing profits in volatile markets. The entire idea of ‘HODL’ being pushed by people who were obviously continuing to accumulate was and remains disgusting but it is just how the game is played. No one is clicking anyone else’s mouse so it is permissible. This is just a variation of what is generally called “Wall Street Investment” as well or at least fits the meta-pattern well enough. The average person, and their precious money, is expected to trust and be exposed to dangerous, unstable market environments while gaining no real benefits beyond a promise that breaks every few years.

There is not an entry level for market volatility and, with the advent of Acervator, there does not have to be.

More than anything else perhaps, the creation of Acervator represents an explicit effort to essentially debunk the approach and hamstringed interaction methods most investors are expected to utilize or, more accurately, tolerate. The average person who puts money into the market surrenders most or all control of it based on a hope that increasingly has no rational basis due to the lack of protections or proportionate, timely reward. The investor gets a secondary value abstraction that they have to wait to sell for more later (and are attacked for doing so if it's anything other than a stock!) assuming the market moves favorably. I see this as an over-exposed “bait and switch” where too often people are left holding worthless bags of tokens or stocks or bonds. It is an upgraded lottery system in some regard but it is only gambling to those that attempt to play blindly or passively and accept the ‘HODL’ advice from whales who just need their next meal to sit still.

The highest goal is for Acervator to be a self-contained retirement platform that provides a powerful option for anyone seeking to take a direct, elegant, self-contained approach to their digital asset portfolio management. It aspires to be the lens, the manager, and the fiduciary while only requiring users to test it, suggest new features, and / or donate to its continued development. Regardless, this will be my last attempt to help humanity see past the wiggly chart illusions but, honestly, I do not think anything else will be required.

If Acervator has helped you accumulate or you wish to support development directly (I am a one man show…), consider sending a tip to the inventor.

USDT 0x6Eb5bc78f43b71F5f09B87ED54e7FF3bD8a3428B

A14 0x8b1022d836d19aA848Ebc9E09A6FC2921d9EB7d41

A15 3HutPgxaMTftWQY3KyRkC7Dckp6A1zdfdp0

PayPalUSD 0x52D75aD65a98f98fA2d1D3675Bd5781ee547426D

A15 2 bc1qn3mkssem0k2tyhwl5yeea4hj222k8j9rr6ruff

A14 2 0x56d1Ee9F683d7535D2637F242a1A2B338aC5aC12

LAST NOTE: This manual, like the software it discusses, is a work in progress and it will be updated on a regular basis as features are added and finalized..

Product Manual Parts

This manual is an attempt to clarify all parts and functions for what might be considered a somewhat esoteric product given its innovative approaches and the sheer number of novel features it contains. More cynical SaaS providers might disagree but I look forward to all feedback regardless. Currently there are ten such sections with each focusing on ever smaller components of the total system.

Part Title and Area of Focus

1 Frontmatter & Overview

2 Novel Concepts and Patent Candidate Portfolio

3 System Architecture and Features Catalogue

- Main Window

- Trading Tab - Active Bot Lists Under Exchange Tabs - Indicator Voter Panel - Trade Logic Activity - API Activity - Top-Level Portfolio Metrics
- Simulator Tab - Validation Runs vs. Historical Trade Data - Acervator Strategy Back Testing - Runs Historical Data Through Simulated Bots - Bot Swarm Simulation - Deploy As Paper Trading - Deploy As Live Trading
- Paper Trader Tab - Runs Live Exchange Data Through Simulated Bots - Executes and Documents Paper Trades - Collects Paper Trade Data - Pushes Trade Data to Paper Trade History Log - Paper Trade History Tab Reads and Displays
- Proof of Accumulation (PoA) Tab - PoA Blockchain and Trading Tournament Interface - Hosts User Competition Performance and Data - Displays Tournament Monsters and Targets
- Market Inspector Tab - Synthesizes and Proposes Bot Swarm Topologies and Oppositional Trades - Pushes Proposals to Sim, Paper, or Live As Directed
- Bot Swarm Tab - Displays and Allows Configuration of Capital Reinforcement Networks
- Asset Charts Tab - Displays Price Charts for All Active Bots
- History Tab - Loads and Displays Live Trade History Via Exchange API - Merges Live Trade History w/ Acervator Trade Log for Grading and Gate Analysis - Paper History Tab - Loads and Displays Paper Trade History w/ Grading and Gate Analysis
- Console Tab - Displays internal codebase activity, warnings and faults
- System Status Tab - Displays Platform Status by Subsystem - Fed by the Emitter Network
- Settings - System and High-Level User Settings

4 Live and Supporting Evidence

- YTD Exchange Data Covering the Entire Development Arc and Exchange Test Coverage

5 Conceptual Hopscotch: The HOP Protocol, Rules Registry, and SADP v1 and v2

6 ADR Index & Glossary

7 Recent Updates and Version Tracking

8 The Development Chronicle As Told By Claude

9 Live Trade History and Supporting Evidence

- VWAP Charts - YTD Data Analysis and Trade Grading - Deep Indicator Function Investigations Via Trade Logic Gate Logs

10 The Sigil Project

- Named Checks That Read Finished Code Against a Published Rule Set - Design and Progress - One Page per Built Sigil

Part 2

Novel Concepts and Patent Candidate Portfolio

Novel Concepts and Patent Candidate Catalogue

Active development on Acervator began in early April of 2026. It began after realizing that the AI-assisted software development tools had achieved an inflection point with their capabilities. Prior to this, efforts were being made to learn traditional coding with little success that was mainly due to an inability to get into a grinding effort for whatever reasons. This has been particularly frustrating since other subjects seem to grab and others deflect. With the advent of Claude Code however, it feels like some of us have been given a decently priced door man and well-informed guide through this and other labyrinths.

Each entry below carries an anchor. The anchor is the code that runs the concept today, quoted from the module that holds it. Where a concept has no code, the entry says so and gives the query that measured the absence, and then proposes the build. Anchors describe the present state of the platform, not the plan.

1 - Harvest-Fold / Scrumming

At the core of Acervator is what might not first appear as a multi-faceted concept designed to address or alleviate many of the mental pitfalls that can afflict a new trader. After having observed a number of people via Telegram and web approach the challenge of investment differently, I have come away convinced that all it only becomes as complex as the individual allows it to be. This is not to be reductive about economics or any other connected discipline but it is to say there are reliable, stable, consistent means of reading price charts and too many people are simply doing it too many different ways and this only compounds the complexity illusion further. Much of the uneasiness inflicted on the new trader surrounds the emotional desire or even personal need for the value of their positions to increase as soon as possible or show the midterm potential of becoming profitable. Those that adopt the HODL meme are, in my experience, quite crippled in their ability to sell positions even when the time is right, which is usually when most people are happiest about the market. The ironic truth is that however good a trade is, there is an equally bad one on the other side of it. Acervator attempts to respect the market by not taking profit all at once and absorbing or claiming ownership of its positions' variance strictly based on the market local, organic price ranges. In other words, its trade actions are always proportionate to whatever the market is actually doing at the time the trade is executed. Over time, this 'molds' the position to the long term truth of the market and avoids the need to speculate entirely. The more bold hope for this approach is that it might add another tool for stabilizing macro-economic phenomena such as underpinning currency exchange rates between countries.

Acervator addresses 'fundamental market unpredictability' by allowing volatility to "orbit" around a set point we call the Target Balance. This element is what performs the first mental trick. It forces the trader to say and establish that "my position is worth X and I will support X with an equal amount of liquidity" i.e. a Target Balance of $50 is set and a total mental position of $100 is established. Trades will be executed to maintain the truth of the Target Balance always being X. After the initial entry is executed and filled, which Acervator will only do in the lower extent of the Bollinger Band on the chosen timeframe, the Scrumming Bot will persistently shave off and "Fold" back in Y% of profits through repeating Harvest-Fold operations that continue until the bot is

deactivated or the market suffers a catastrophic collapse which can and do happen so don't not leave it unmonitored. It should also be noted that the term 'harvest' came to replace 'scrum' for the sake of clarity but they may be used interchangeably throughout this text. Essentially a Scrum or Harvest is a sale of a position's increased value over the Target Balance. The amount that the Scrumming Bot harvests or folds is pinned to the Target Delta or the amount of variation (%delta) from the most recent trade event. Much of the finer details of trade execution can be tuned by the user as expected for any professional application but the primary point here is that the amount of profit folded back is capped by the Maximum Target Growth Per Cycle setting while the ability of Acervator to prevent inverted (profit losing: buying higher than sold / selling lower than bought) trades we also have the Minimum Opposing Trade Distance rule which dictates the next trade must be Z distance from the most recent trade event. In addition, sells are not allowed to occur or be pushed below the Bollinger Band midline while buys are not allowed to occur or be pushed above the midline. As a result of just these few component features, we already have multiple soft, non-TA or non-traditional rules protecting the investor before we even start optimizing the trades themselves (pushing them as close to volatility extents as possible) via Technical Analysis and other inventions.

Now: a buy-back waits for this distance, wired or not, because a wire takes only the profit a sale leaves past the dollars that buy back its units and the room the growth cap leaves, and a tranche whose own dollars cannot restore its units at this distance waits until they can (`src/trading/scrumming/sizing.py, in tranche_rebuy_price`).

The harvest-fold method itself was manually executed and tested on a multiple cryptocurrency pairs throughout 2023~2026 but it has always been the aspiration to automate it and have it executed at a high rate of frequency with the operating theory that this approach to investment, being a hybrid a HODL and active trading, creates frontline area of support for whatever markets in which it participates long term. It does this by maintaining and, utilizing as directed, all earned profits. It turns any user into an active participant and liquidity provider.

After entering the space full time in 2017, I immediately began to parse and breakdown how price charts were being interpreted. This also allowed me to perform ever deeper introspections on my own response to how my mind was responding to the value of my portfolio fluctuating. I did myself absolutely no psychological favors in this pursuit but, as you will see, I had plenty of reasons to take this reckless approach. By this point, my undiagnosed and untreated military injuries had worn me down to the point where I realized I was running out of time to continue working as normal. I had, by then, been walking a torsioned pelvis for 17 years that had seized in place and, by 2020, would be causing enough pain to induce panic attacks. I could have joined the pill-mill club but I declined. In combination with the pain, the forced TA self-training, physical job, and broken marriage, I also assembled an entire GPU-based mining farm…alone. I had some business partners but their participation was basically nonexistent. The VA was providing no support and would not until 2022. The Navy and Pentagon had spit in my face by sending me PDBR paperwork that they had no intention of ever honoring no matter what I wrote down. My family more or less remained ignorant of all my turmoil while I kept the paychecks coming. So really the story of Acervator's and Harvest-Fold is one of betrayal to a degree that allowed me to see through things more clearly than would otherwise have been possible. It has been invented out of necessity and it is my intuition that Acervator can become that necessary tool for other people as well.

The target and the two sides

Functional. The bot holds one number, and that number is the dollar value you told it the position is worth. On every tick it compares the position against that number. Above it by more than a dust band, the bot has surplus to sell. Below it by more than the band, the bot has a deficit to buy. Inside the band it does nothing at all. One function answers that question for the whole engine, and the dust band is a tenth of a percent of the target with a floor of one cent. At a target of $50 the band is $0.05, so a position at $60 answers scrum and a position at $40 answers fold.

`src/trading/target_bands.py` — `target_territory`

```
delta = float(position_value) - float(target_balance)
band = at_target_dust_band(target_balance)
if delta > band:
    return "scrum"
if delta < -band:
    return "fold"
return "at_target"
```

Design intention. The Target Balance is the set point the whole method orbits. It is declared once in the bot's config and the tick re-zeroes to it. The bot brain holds the tick, the scrumming package fires the two halves of the cycle, and the execution module places the orders.

`src/trading/container/config.py` — `BotConfig`

```
target_balance: float = 200.0  # Balance the bot trades relative to
scrumming_interval_pct: float = 1.0  # % market move between actions
max_target_growth_pct: float = 1.0
```

The distance rule

Functional. An opposing trade has to travel far enough to clear both legs of the round trip. That is the operator's scrumming interval plus the exchange fee. A one percent interval against a six tenths fee gives 1.60 percent. One function owns the arithmetic, and the clamp is applied to the sum rather than to the interval, because the bound exists to keep the rebuy factor sane.

`src/trading/otd_math.py` — `minimum_opposing_trade_distance_pct`

```
total = float(interval_pct) + float(fee_pct)
return max(OTD_MIN_PCT, min(OTD_MAX_PCT, total))
```

Design intention. Without this rule the bot can sell low and buy back higher, which is the one outcome the method exists to prevent. The gate that enforces it refuses the trade until price has travelled that distance from the pivot, and it rebuilds the required price in its own refusal message so the operator can read why it held.

`src/trading/gate_chain.py` — `HysteresisGate.evaluate`

```
ok = ctx.hyst_ok_scrum_side if self.side == "scrum" else ctx.hyst_ok_fold_side
if ok:
    return GateResult(passed=True)
eff_pct = ctx.scrumming_interval_pct + ctx.trading_fee_pct
```

Current reading. The distance never sits under two fees, the round trip a scrum and its fold pay between them. Where the interval and the band extension together sit under one fee, the distance is two fees. At or above one fee, it is the sum above. The fee is the larger of the Trading Fee setting and the highest fee the venue has reported on this bot's fills.

`src/trading/otd_math.py` — `minimum_opposing_trade_distance_pct`

```
interval, fee, extension = terms
if interval + extension < fee:
    return clamped_otd_pct(ROUND_TRIP_FEES * fee)
return clamped_otd_pct(interval + fee + extension)
```

The midline rule and the growth cap

Functional. A sell is not allowed below the Bollinger midline and a buy is not allowed above it. The tick sets one flag for each side off the band position, and a gate refuses the wrong side of the band. The scrum flag also carries the phantom lock. Nothing sets that lock today, and on a ripe row the ripe-harvest override force-passes the gate that reads it.

`src/trading/scrumming_bot.py` — `ScrummingBot.tick`

```
scrum_ok = (bb_pos > 0.50) and not self._phantom_locked
fold_ok_midline = bb_pos < 0.50
```

Design intention. Profit folded back into the target is capped for one market cycle, so a single violent move cannot inflate the set point beyond what the next cycle can support. The cap is a percentage of the target at the moment the cycle opened, and it shrinks as the cycle consumes it.

`src/trading/scrumming_bot.py` — `ScrummingBot.cycle_growth_cap_usd`

```
_pct = float(getattr(self.config, "max_target_growth_pct", 1.0) or 0.0)
```

The same key answers differently in different places. Nine read sites fall back to 1.0 and three fall back to 0.0, so a bot whose stored config lacks the key compounds or freezes depending on which site read it first. Issue #409 carries it.

The issue #409 clause above is overtaken. It is not edited and not removed. It read:

> *Nine read sites fall back to 1.0 and three fall back to 0.0, so a bot whose stored config lacks the key compounds or freezes depending on which site read it first. Issue #409 carries it.*

Issue #409 closed on 2026-09-25. One declaration answers the key now, and no read site carries a fallback of its own. `src/trading/container/config.py` declares `TARGET_GROWTH_PCT_DEFAULT` from the field's own declared default, and `in target_growth_pct` is the single reader. Fourteen call sites read that function.

```
src/trading/container/config.py, in BotConfig    max_target_growth_pct: float = 1.0
src/trading/container/config.py                  TARGET_GROWTH_PCT_DEFAULT reads that default
src/trading/container/config.py, in target_growth_pct   the one reader
call sites reading target_growth_pct             14
read sites carrying their own fallback            0
```

The gate set the bot writes is named in one place: ten scrum labels and nine fold labels, nineteen lights per bot.

`src/trading/gate_vocabulary.py` — `_GATE_ORDER_SCRUM`

```
_GATE_ORDER_SCRUM: tuple[str, ...] = (
    "TGT",
    "INT",
    "BB",
    "FIRE",
    "TA",
    "LS",
    "TRND",
    "HTF",
    "CB",
    "OTD",
)
```

What the ripe-harvest override replaces at the midline

The entry above states this:

> *Nothing sets that lock today, and on a ripe row the ripe-harvest override force-passes the gate that reads it.*

The current reading. On a ripe row the override force-passes nothing at the midline, because the gate has already passed. A row is ripe only when the band position reaches the upper detect threshold, and that threshold never sits below the midline. A ripe row is therefore above the midline by construction, so the band half of the scrum flag is already true. The phantom lock is the only other way that flag can read false, and no method in the platform sets it.

`src/trading/scrumming/circuit_breakers.py` — `_bb_detect_thresholds`, the reason a ripe row is above the midline

```
detect_frac = max(0.0, min(1.0, detect_pct / 100.0))
half = detect_frac * 0.5
return (0.5 - half, 0.5 + half)
```

Driven. The shipped scrum chain ran over a swept window of hand-built candles. The last candle's close moved from 95.00 to 105.00 in steps of 0.0005, which is 20,001 rows, and the band position read from -0.3089 to 1.3165 off the real band reading. Both the midline and the upper detect threshold sit inside that span, and one step moves the band position by about 0.00008, so no change of verdict can fall between two rows.

```
20,001 rows, the phantom lock clear

the gate passed on its own merit          10,106
the gate refused and the refusal stood     9,895
the gate was passed regardless                 0
rows reaching a real verdict              20,001

6,832 rows were ripe, and the gate passed on its own on every one of them
```

The same override does replace a refusal, on a different gate. With the voting panel not bullish, the override passed the bullish-vote gate on exactly the 6,832 ripe rows, and its refusal stood on the other 13,169. The override acts, and the midline gate is not the gate it acts on.

```
20,001 rows, the phantom lock clear, the voting panel not bullish

ta_bullish passed regardless     6,832
ta_bullish refusal stood        13,169
midline passed regardless            0
```

What a recorded lock does. The Simulator can recover a lock from a stored record. Driven with the lock recovered, the midline gate was passed regardless on the 6,832 ripe rows and its refusal stood on the other 13,169, and the record named the override every time. That is the only state in which the override reaches the midline gate, and nothing in the running program produces it.

```
midline_scrum overridden by ripe_harvest_override: scrum_ok=False(bb_pos=1.23)
scrum_ok=False(bb_pos=-0.22) [override not applied: ripe_harvest_override]
```

What is still open, and it is his. The open decision on the midline rule is whether the band should hold a ripe scrum at all. It is not a decision about the higher timeframe. The reading above moves the ground under it: the band does not hold a ripe scrum today because a ripe row is already above the band midline, not because an override lifts the band. Two settings carry that behaviour and both are his, the BB Midline Gate tick box and the BB Detect threshold, and both are described in Part 6, the Trading tab.

Which gate each override covers is on the gate table in Part 7, Indicators.

The first buy reads three agreeing indicators

A bot has made its first buy once a tick reads its own lots at target, each at the ticker it was adopted at, the decision price of its buy, or else its cost. The read allows the largest of the dust band, one size step and the trading fee's share of target, so the price after the fill cannot matter, and a whole buy reads made on any market. It needs a weight above zero. This is a saved fact about the bot, and no fill sets it, so it holds however the coins reached the book. A bot that made its first buy and lost the position does not owe another. A saved record with no such field reads as made when it shows a fill of this bot's, a tranche, or, with a weight above zero, a holding of 1% of target or more.

While a bot owes a first buy, its Max Cartridge rebalance, its hedge top-up and its fold wait for one read. Price must sit in the lowest 30% of the band, past the Detect distance or within the Fire Threshold of the lower band. The bot's own vote must then be bearish and top what two indicators reach on that same vote.

A new bot holds twelve zeros, so it has no vote. It makes no first buy until you press Save on the TA Indicators page, and its line in the log says so. Twelve zeros never read as made, whatever the bot holds. After Save, a bot that finds its whole target at the venue reads as made at its next worked tick. One that finds less waits, and then the rebalance buys the rest. A bot saved as made keeps that reading.

When the read clears, the rebalance fires as it does at any other time: a market order sized at the Target Delta and clipped to the wallet. A first buy clipped to the wallet or part filled leaves the bot under target, so its next buy reads again. A press of Fire does not wait for the read.

With Smart Cartridge off, a Max Cartridge Size of 0 or above 100 fires no rebalance buy. Under 1% of target, a bot takes the first entry, with the same read, and buys the whole target. Its wallet must hold that target after what the other bots claim, and in Order Book mode its order is a limit. A bot that has made its first buy and buys back here votes with its own weights, and at twelve zeros it waits for Save. Only a first buy's entry fill restarts the opposing trade distance.

Both routes refuse a buy while the venue holds more of the coin than the book shows, past other bots on that venue and by more than 1% of target, or omits the coin from its balance reply. The holdings audit books those coins on its own count.

Each compare writes one line in the bot's log, once a stretch. A compare that clears writes VENUE COMPARE CLEAR. It and the refusal both name what the venue holds past other bots, what the book shows, and the gap in dollars against the band.

Each read writes one line in the bot's log, and the line ends with the Max Cartridge Size the tick ran under. A line the vote decided also names each weight above zero that the vote ran at. A read that clears in front of the rebalance, the hedge top-up or the fold writes FIRST BUY READ CLEARED. A read that clears on the first entry writes that all 4 gates pass.

When the app starts, each restored bot writes FIRST BUY MADE or FIRST BUY OWED in its log. The line says what decided it: the saved record, or, where the record saves no such field, its fills, tranches and holding.

The read:

`src/trading/scrumming/tick_phases.py` — `TickPhaseMixin._first_buy_read`

```
summary = self._voting_engine.compute_all(
    parsed, self.config.ta_timeframe, symbol=self.config.symbol
)
```

```
if not three_agree(summary.consensus_confidence, summary.signals):
```

The hold the rebalance, the hedge top-up and the fold ask:

`src/trading/scrumming/tick_phases.py` — `TickPhaseMixin._first_buy_clears`

```
  if self._first_buy_made:
      return True
  read = await self._first_buy_read(ticker, symbol)
  terms = self._first_buy_terms(read, cartridge_pct)
  if not read.refusal:
      if self._first_buy_line_is_new(FIRST_BUY_CLEARED, ticker.last):
          self._bus.emit(
              "bot.log",
              bot_id=self.bot_id,
              message=(
                  f"FIRST BUY READ CLEARED: TA=BEARISH at confidence "
                  f"{read.summary.consensus_confidence:.4f}. BB OK "
                  f"(pos={read.bb_pos:.2f}, dist={read.dist_down:.0%}). {terms}"
              ),
          )
      return True
```

The close of each line:

`src/trading/scrumming/tick_phases.py` — `TickPhaseMixin._first_buy_terms`

```
  size = f"Max Cartridge Size {cartridge_pct:g}%."
  if read.summary is None:
      return size
```

The one write of the field:

`src/trading/scrumming_bot.py` — `ScrummingBot.tick`

```
  if not self._first_buy_made and await self._book_reaches_target(symbol):
      self._first_buy_made = True
```

```
  src/trading/scrumming/tick_phases.py, in TickPhaseMixin._book_reaches_target
```

The read of a saved record:

`src/trading/scrumming/state_io.py` — `StateSerializerMixin.restore_first_buy_made`

```
  from_save = isinstance(saved, bool)
  self._first_buy_made = saved if from_save else self._shows_past_trading()
  self._bus.emit(
      "bot.log",
      bot_id=self.bot_id,
      message=first_buy_restore_line(self._first_buy_made, from_save),
  )
```

The line a restore writes:

`src/trading/scrumming/state_io.py` — `first_buy_restore_line`

```
if made:
    return f"FIRST BUY MADE on restore: {ground}."
```

The venue compare:

```
src/trading/scrumming/execution.py, in ExecutionEngineMixin._unbooked_coin_refusal
```

The line a compare that clears writes:

`src/trading/scrumming/execution.py` — `ExecutionEngineMixin._unbooked_coin_refusal`

```
if unbooked_usd <= band_usd:
    if self._refusal_is_new("buy_unbooked_clear", price):
        self._bus.emit(
            "bot.log",
            bot_id=self.bot_id,
            message=f"VENUE COMPARE CLEAR: {counts}",
        )
    return ""
```

Paper and the Simulator make no first buy. Each opens a bot at its target.

```
src/paper/paper_run.py, in opening_balance
src/simulator/back_test.py, in opening_position
```

2 - The Phantom Bot

While the Harvest-Fold method housed with the Scrumming Bot addresses several persistent and nagging questions around any given investment position (Should I buy more? Should I sell some? When? How much?), it does have the weakness of operating on a single timeframe. The Phantom Bot is another feature intended to optimize trade execution position. In this specific case, it allows a single Scrumming Bot to be inhibited by a higher timeframe signal with the intention of capturing alignments or disagreements so that trades are not being executed prematurely. The design intention here is that, for example, a 5m TF bot can be inhibited by a 15m, 30m or 1hr signal while maintaining a volatility watch on its primary TF. The intention is to build out this feature so that it obviates or reduces the need for multi-screen monitoring of individual assets.

Functional. A phantom is a copy of the position read on a slower chart. It computes technical analysis on its own timeframe, stores the result, and constructs no order. One timeframe is the default set, and a saved record names its own.

`src/trading/scrumming_bot.py` — `ScrummingBot.__init__`

```
self._phantom_timeframes = list(
    phantom_timeframes
    or default_phantom_timeframes(config.ta_timeframe, _offered)
)
```

Design intention. The inhibition the entry describes runs today. A bullish higher timeframe stops the scrum and a bearish one stops the fold, and each writes its own label into the blocker list so the operator can see which timeframe held the trade. The coordinator weights the phantom summaries by their rank before the gate reads them.

`src/trading/gate_chain.py` — `HTFDeferGate.evaluate`

```
blocks = (
    ctx.eff_htf_blocks_scrum
    if self.side == "scrum"
    else ctx.eff_htf_blocks_fold
)
if not blocks:
    return GateResult(passed=True)
label = "HTF-bullish" if self.side == "scrum" else "HTF-bearish"
return GateResult(passed=False, blocker_message=label)
```

One part of the feature does not work, and it is the phantom lock. No method on the coordinator creates a lock, so the flag stays clear and the lock list stays empty. The term in the scrum flag does read the flag, and a ripe row force-passes the gate it sits in. Whether a higher timeframe should hold a trade at all is the open decision on issue #1359. Replacing multi-screen monitoring is an intention, not current behaviour.

```
six ticks, three phantoms registered, one bot

the flag after every tick                       False
the coordinator's active locks                  []
the flag set by hand, band position 0.70        the midline gate refuses
the flag set by hand, band position 0.91        the ripe row force-passes it
```

Two sentences above are overtaken in part. Neither is edited and neither is removed. They read:

> *The scrum flag also carries the phantom lock. Nothing sets that lock today, and on a ripe row the ripe-harvest override force-passes the gate that reads it.*
>
> *The phantom lock is the only other way that flag can read false, and no method in the platform sets it.*

The current reading. A method on the bot now sets the flag. It runs on every tick, above the scrum flag, and it takes the state from the coordinator's own active lock list rather than from a field nobody wrote. Where the operator has phantom bots switched off it clears the flag instead.

`src/trading/scrumming_bot.py` — `ScrummingBot.tick`, the call above the flag

```
self._refresh_phantom_lock()
if self.config.bb_midline_gate:
    scrum_ok = (bb_pos > 0.50) and not self._phantom_locked
    fold_ok_midline = bb_pos < 0.50
```

The rest of both sentences still holds. No method on the coordinator creates a lock, so the lock list is empty on every running bot and the flag reads clear. The ripe-harvest override still force-passes the gate on a ripe row, which is why a held lock changes no trade. Driven across 3,201 band positions with a lock held, the scrum flag changed on 1,601 rows and the final scrum decision on none.

The full reading, the method, the coordinator's answer and the swept figures are on the Indicator Voting Panel page.

The operator's ruling, 2026-10-27. Asked what should create a phantom lock, he answered:

> *This is a strange question. A phantom bot is simply a higher timeframe read that is piped through a parallel trading gate logic chain. It is intended to improve rather than restrict ripe scrums or folds so it must work on both sides. It should be thought of as an optional guardian angel that may optimize trades.*

Two clauses above are overtaken. Neither is edited and neither is removed. They read:

> *The ripe-harvest override still force-passes the gate on a ripe row, which is why a held lock changes no trade.*
>
> *fold_ok_midline = bb_pos < 0.50*

The current reading. The fold flag carries the lock too, so a held lock does change a trade. The clause remains correct about the scrum side, where a ripe row is above the midline and the override force-passes the gate anyway. On the fold side there is no such override below the midline, so the refusal stands and the fold is held.

`src/trading/scrumming_bot.py` — `ScrummingBot.tick`, the fold flag

```
fold_ok_midline = (bb_pos < 0.50) and not self._phantom_locked
```

Driven. A bot was built by hand with a stand-in venue that refuses every order, and the real tick was driven over 801 window levels an arm, which moved the band position from -0.558 to 1.559. With phantom bots off, and with them on and no lock held, every recorded verdict is identical before and after. With them on and a lock held, the fold gate's refusal appears on the 398 rows below the midline, and the fold decision itself moves on 362 of them. No scrum reading moves on any row.

```
801 rows an arm, two trees compared row by row

phantoms off, lock held     every verdict identical
phantoms on,  no lock       every verdict identical
phantoms on,  lock held     fold decision differs on   362
                            scrum readings differ on     0
```

What should create a lock on the coordinator is still his, and nothing creates one today.

The operator's specification, 2026-10-27. On what a phantom bot is for, and the last change he made to its design:

> *Again, the entire function of a Phantom Bot is to provide high TF over rides and trade optimization. From this perspective, all such questions can be inferred. The design spec is established but simply not fully implemented with the last major modification to it being the decision to only allow one Phantom Bot TF to be active per bot or Target Asset.*

One phantom timeframe is active per bot. The limit is one declaration and every site that builds a reader reads it. The active name is the highest-ranked one the bot was given, which is the name the Phantom Bots picker already drew and the name the rank weighting already counted for most.

`src/trading/phantom_balance.py` — `one_active_phantom_timeframe`

```
    active = max(ranked, key=tf_rank)
    set_aside = sorted(
        [one for one in held if one != active],
        key=tf_rank,
    )
    return [active], set_aside
```

step	module	symbol
name the one	`src/trading/phantom_balance.py`	`one_active_phantom_timeframe`
hold the one	`src/trading/scrumming_bot.py`	`ScrummingBot.__init__`
build the one	`src/trading/phantom_balance.py`	`PhantomBalanceManager.create_phantom_set`
keep it on a live change	`src/trading/scrumming_bot.py`	`ScrummingBot.update_phantom_config`
name it on the page	`src/gui/main_tabs/phantom_bots_tab_surface.py`	`configured_timeframes`
name it in the Simulator	`src/simulator/back_test.py`	`phantom_timeframes_for`

Nothing is dropped in silence. A name the limit holds back is drawn on its own status row, and the bot writes the same sentence to its Activity Log on the tick that starts its phantom.

```
    a bot given 6h and 1d, read off the page and the log

    the picker                      1d
    Configured timeframes:          6h, 1d
    Set aside, one runs:            6h
    the Activity Log line           Phantom timeframes set aside: 6h
```

Two sentences below are overtaken. Neither is edited and neither is removed. They read:

> *A bot holding three names writes all three and reads all three back, so a set of several survives a restart.*
>
> *A bot holding 2h, 6h and 1d writes all three. The fleet load hands all three back, and the phantom manager builds three PhantomBots on them.*

The current reading. The record still carries every name and the fleet load still hands them all back, so both sentences hold for the record. The bot now keeps the highest of them as the one it runs and the rest as names set aside, and the manager builds one reader rather than three.

The save writes both halves back, so a save loses no name the operator chose.

`src/trading/bot_container.py` — `BotContainer.get_full_state`

```
    _phantom_held = list(getattr(self, "_phantom_timeframes", None) or []) + list(
        getattr(self, "_phantom_tf_set_aside", None) or []
    )
```

Driven on a bot given 6h and 1d, saved and restored three times in a row: the record reads 6h and 1d on every pass, the bot runs 1d on every pass, and the page names 6h as set aside on every pass.

Driven. A bot was built by hand with a stand-in venue that refuses every order, and the real tick was driven over 801 window levels an arm, which moved the band position from -0.292 to 1.283 in steps of 0.03 of a price unit. With phantom bots off, and with exactly one timeframe given, every recorded verdict is identical before and after. With two given, the lower-ranked reading no longer reaches the gate.

```
801 rows an arm, two trees compared row by row

phantoms off              every verdict identical
one timeframe             every verdict identical
one timeframe, lock held  every verdict identical
two timeframes            scrum decision differs on   256
                          fold decision differs on    265
```

One bot, one Target Asset. Each bot builds its own coordinator and no code hands one bot another bot's, so the limit at the bot is the limit at the Target Asset that bot holds. Two bots on one asset each run their own one, and neither changes the other.

The saved record carries the timeframe. The bot writes every name it holds under the key `phantom_timeframes`. The fleet load reads that key and gives the list to the bot it builds. The key, the change field and the bot attribute now use one spelling.

step	module	symbol
write the key	`src/trading/bot_container.py`	`BotContainer.get_full_state`
read the key	`src/trading/container/restore.py`	`StateRestoreMixin.restore_bots_from_state`
hold the list	`src/trading/scrumming_bot.py`	`ScrummingBot.__init__`
build the set	`src/trading/scrumming/tick_phases.py`	`TickPhaseMixin._tick_initialise`
run one timeframe	`src/trading/phantom_balance.py`	`PhantomBalanceManager.create_phantom_set`

A record written before the key took the plural spelling still restores. The fleet load reads the older singular key `phantom_timeframe` when the plural key is absent. A bot holding three names writes all three and reads all three back, so a set of several survives a restart.

The venue decides which names survive. The bot drops any stored timeframe its exchange does not serve. A record naming one timeframe the exchange refuses restores with no phantom at all, and the bot writes the dropped name to its own log line.

The record drops one key. It no longer carries `phantom_config`. The attribute behind it held an empty dictionary and no code read it. A record that still carries the key loads, and the fleet load steps past it.

The saved fleet on the trading machine, read once:

```
38 records, all scrumming        count
phantom_timeframes                   0
phantom_timeframe                    0
phantom_config                      38
phantoms_enabled false              38
```

No bot in that fleet runs a phantom. Every record holds the enable flag off.

Three phantom keys reach the record, and no fourth. A saved bot carries `phantoms_enabled`, `lock_candle_count`, and `phantom_timeframes` when the bot holds a name its venue serves. `src/trading/bot_container.py`, in `BotContainer.get_full_state`, writes all three. The Paper and Simulator records carry the same three names, from `wizard_record` in `src/paper/fleet_source.py` and in `src/simulator/fleet_source.py`. A bot built on any of those routes carries no key spelled `phantom_config`.

Three load paths step past the retired key. `src/trading/container/restore.py`, in `StateRestoreMixin.restore_bots_from_state`, builds the live bot. `src/paper/paper_bot_view.py` and `src/simulator/sim_bot_view.py` build the Paper and the Simulator views, each in `config_of`. All three read the stored declaration through `bot_config_kwargs` in `src/trading/container/config.py`, which keeps only the names the declaration holds. A record carrying `phantom_config` builds a bot on each of the three, with its Target Balance intact.

The retirement record holds no entry for it. `_DEPRECATED_KWARGS` in `src/trading/container/config.py` strips a retired name that a caller passes to the bot declaration. `phantom_config` was a top-level record key and never a field of the declaration. No load path hands it over, so the strip list needs no entry for it. Handed to `make_bot_config` directly the name raises, and a name the strip list does hold builds.

The cursor tick keeps a timestamp. `PhantomBot.tick_for_cursor` stores the cursor itself, not a bucket number. It divides the stored cursor by the candle length on every call, so both bucket numbers come from one divisor.

The cursor tick is gone. `PhantomBot` carries no such method now. No module in the tree called it, and no module passed the simulator flag beside it. The attribute that held the cursor went with it.

One phantom, above the parent. A bot whose record names no timeframe takes one name. The name is the lowest timeframe above the bot's own TA Timeframe that the venue serves. The class constant that held six names is gone.

`src/trading/phantom_balance.py` — `default_phantom_timeframes`

```
above = [
    one
    for one in TIMEFRAME_ORDER
    if one in allowed and is_higher_tf(one, parent_timeframe)
]
return above[:1]
```

Driven on a Coinbase bot with a 1h TA Timeframe, the bot holds 2h. It held five names before, and four of the five sat at or below that 1h parent.

The record keeps every name the bot holds. The save writes the key for any number of names. A bot holding 2h, 6h and 1d writes all three. The fleet load hands all three back, and the phantom manager builds three PhantomBots on them. The save wrote no key at all for that bot before.

step	module	symbol
write the key	`src/trading/bot_container.py`	`BotContainer.get_full_state`
read the key	`src/trading/container/restore.py`	`StateRestoreMixin.restore_bots_from_state`
pick a default	`src/trading/phantom_balance.py`	`default_phantom_timeframes`
build the set	`src/trading/phantom_balance.py`	`PhantomBalanceManager.create_phantom_set`

The eleven names are declared once. `src/exchange/timeframes.py` declares `ALL_TIMEFRAMES`. Five modules read that tuple. Each held its own copy of the same ranked list before, and one module held two copies.

reader	module
the rank order	`src/trading/phantom_balance.py`
the wizard page	`src/gui/main_tabs/bot_wizard_surface.py`
the Settings page	`src/gui/main_tabs/settings_dialog_surface.py`
the Phantom Bots tab	`src/gui/main_tabs/phantom_bots_tab_surface.py`
the Comp column	`src/gui/main_tabs/indicator_panel_surface.py`

A refused phantom is named on screen. The Phantom Bots tab carries six status rows. Two of them name the timeframes no phantom runs on. The first names the timeframes the venue dropped. The second names the timeframes at or below the bot's own TA Timeframe. The Comp column's cell carries that second list in its tool tip.

reading	module	symbol
the venue drop	`src/trading/scrumming_bot.py`	`ScrummingBot._phantom_tf_dropped`
the parent refusal	`src/gui/main_tabs/phantom_bots_tab_surface.py`	`below_parent_timeframes`
the Comp tool tip	`src/gui/main_tabs/indicator_panel_surface.py`	`comp_skipped_tooltip`
the engine record	`src/trading/phantom_balance.py`	`TimeframeCoordinator.get_higher_tf_bias`

A refused change is named too. A line under the timeframe picker names a stored timeframe the venue does not offer, and names a stored timeframe the tab does not list beside the one the picker shows instead. Both builds draw that line. In the React build a second line at the top of the tab names a change the tab would not take and the setting it came from, and it is replaced rather than repeated while the same change keeps arriving. The Qt build moves its own controls and carries no second line. A change the tab takes shows neither line.

reading	module	symbol
the stored timeframe	`src/gui/main_tabs/phantom_bots_tab_surface.py`	`stored_timeframe_notice`
the refused change	`src/gui/main_tabs/phantom_bots_tab_surface.py`	`step_refusal_text`
the one announcement	`src/gui/main_tabs/phantom_bots_tab_surface.py`	`PhantomBotsTabModel.note_steps`

reading	module	symbol
the drawn line	`src/gui/web/phantom_bots_tab.js`	`step-refusal`

The P&L column draws a dash. A phantom is a read-only observer. It runs no order on any path, it holds no position and it counts no cost, so it computes no profit and there is no figure for the column to print. Its answer to the tab carries no profit key, and the cell draws the dash the Comp column draws for a reading it cannot make. Both builds draw that one dash. The column printed `$+0.0000` on every row before, which an operator could not tell from a profit of nothing.

A figure the phantom does report is still printed. A profit of exactly zero prints `$+0.0000`, so the dash stands for an absent reading alone and never hides a real one. A figure of a kind the money admission refuses — a stored flag, a text number, a not-a-number, an endless number — draws the dash as well.

reading	module	symbol
the phantom's answer	`src/trading/phantom_balance.py`	`PhantomBot.get_status`
the profit figure	`src/gui/main_tabs/phantom_bots_tab_surface.py`	`phantom_pnl`
the drawn cell	`src/gui/main_tabs/phantom_bots_tab_surface.py`	`phantom_pnl_text`
the money admission	`src/trading/container/config.py`	`as_finite_float`
the one dash	`src/gui/main_tabs/indicator_panel_surface.py`	`ABSENT_CELL_TEXT`
the Qt cell	`src/gui/live_settings/phantom_bots_tab.py`	`PhantomBotsTabMixin._create_phantom_bots_tab`
the React cell	`src/gui/main_tabs/phantom_bots_tab_surface.py`	`phantom_row`

A phantom answering nothing still draws its row. The per-phantom table reads each phantom through one guarded call. A phantom that raises, and a phantom that hands back nothing at all, both reach the table as an empty reading, and the row draws its absent marks. The table ended the whole tab build on either one before.

reading	module	symbol
the guarded read	`src/gui/main_tabs/phantom_bots_tab_surface.py`	`phantom_status`
the Qt loop	`src/gui/live_settings/phantom_bots_tab.py`	`PhantomBotsTabMixin._create_phantom_bots_tab`
the React loop	`src/gui/main_tabs/phantom_bots_tab_surface.py`	`PhantomBotsTabModel.build`

The window's answer carries every row the tab draws. The Bot Settings window publishes one bot reading that each of its tabs asks for. That reading carried the enable switch, the timeframe list and the lock figure, and the tab read six more names the window never sent. On the running build the started row, the dropped row and the lock-state row drew their built-in words, and neither runtime table drew at all.

One declaration now names each status reading, its bot attribute and the value an absent attribute gets, so a row cannot reach the page alone.

```
BOT_REQUEST_ROW: tuple = (
    (REQUEST_ENABLED, ENABLED_ATTRIBUTE, False),
    (REQUEST_STARTED, STARTED_ATTRIBUTE, False),
    (REQUEST_TIMEFRAMES, TIMEFRAMES_ATTRIBUTE, None),
    (REQUEST_DROPPED, DROPPED_ATTRIBUTE, None),
    (REQUEST_LOCKED, LOCKED_ATTRIBUTE, False),
    (REQUEST_LOCK_TIMEFRAME, LOCK_TIMEFRAME_ATTRIBUTE, DEFAULT_LOCK_TIMEFRAME),
)
```

Driven on two bots differing in every row, the page and the Qt tab now report the same six rows and the same two tables.

```
                            before                after
Phantoms started            NO on both bots       YES then NO
Dropped by the venue        none on both bots     1w then 3m, 6h
SCRUM lock state            UNLOCKED on both      LOCKED then UNLOCKED
per-phantom table rows      0 and 0               1 and 2
active lock table rows      0 and 0               1 and 2
```

reading	module	symbol
the declaration	`src/gui/main_tabs/phantom_bots_tab_surface.py`	`BOT_REQUEST_ROW`
the bot read	`src/gui/main_tabs/phantom_bots_tab_surface.py`	`published_bot`
the per-phantom read	`src/gui/main_tabs/phantom_bots_tab_surface.py`	`phantom_statuses`
the lock read	`src/gui/main_tabs/phantom_bots_tab_surface.py`	`active_locks`
the window's answer	`src/gui/main_tabs/bot_live_settings_surface.py`	`tab_bot`
the page's reader	`src/gui/main_tabs/phantom_bots_tab_surface.py`	`view_model`

Paper and the Simulator read the same answer. Each builds its own window payload by calling Live's builder, so one repair reaches all three.

```
src/gui/paper/paper_bot_live_settings_surface.py, in build_view_model
src/gui/simulator/sim_bot_live_settings_surface.py, in build_view_model
```

A seventh status row names the timeframes the one-phantom limit holds back. Two sentences above are overtaken. Neither is edited and neither is removed. They read:

> *One declaration now names each status reading, its bot attribute and the value an absent attribute gets, so a row cannot reach the page alone.*
>
> *Driven on two bots differing in every row, the page and the Qt tab now report the same six rows and the same two tables.*

The current reading. The declaration still names each reading and a row still cannot reach the page alone. It holds seven entries rather than six, and the seventh is the set-aside row.

```
(REQUEST_SET_ASIDE, SET_ASIDE_ATTRIBUTE, None),
```

The row reads the same on a running bot and on a stored record. A running bot holds the names it set aside in its own attribute; a Paper or Simulator record holds the whole list, and one read on the page derives the same answer from either.

`src/gui/main_tabs/phantom_bots_tab_surface.py` — `configured_timeframes`

```
held = [str(one) for one in (getattr(bot, TIMEFRAMES_ATTRIBUTE, None) or ())]
aside = getattr(bot, SET_ASIDE_ATTRIBUTE, None)
if aside is None:
    return held
extra = [str(one) for one in (aside or ()) if str(one) not in held]
return sorted(held + extra, key=timeframe_order_key)
```

Driven offscreen on all three pages, with a bot given 6h and 1d, reading the strings each row reports:

```
                            Live       Paper      Simulator
the picker                  1d         1d         1d
Configured timeframes:      6h, 1d     6h, 1d     6h, 1d
Set aside, one runs:        6h         6h         6h
```

The phantom's own gate reading. On each pass of its loop, the phantom now runs the bot's two gate chains over the candles it read, at the bot's own settings, and keeps the gate states. This is the gate chain read on the higher timeframe, beside the bot's own read. It decides no trade. The Market Inspector draws both readings in the Phantom Bot HTF Signals zone.

`src/trading/ata_gate_scan.py` — `timeframe_gates`

```
held = settings if settings is not None else scan_settings(symbol)
return scan_gates(
    symbol, timeframe, candles, summary, band_reading(candles, held), None, held
)
```

The six gates that read the position do not run at the phantom read. No timeframe above the phantom's is weighed there.

3 - The Landing Strip

Perhaps the earliest assumption that hit me the most and really presented the catalyzing challenge to creating all of my alternate trading methods, is that you cannot predict what the market will do and after only a short while of actively trading I asked the questions: “But what if I don’t have to?” “What if I decide that I do not have to speculate at all?” I do not remember exactly when I first used the term but I think it was when I was trading A34 and A27 some years ago while participating in a Telegram group with some psytrance friends and associations called the Better Bitcoin Bureau. The name changed a few times but this is the one I recall.

The concept emerged from my persistent use of Heikin-Aishii candles. I had decided on this candle style early just by intuition. I could say it was because I “felt it looked nicer” but that does not quite capture the magnetism. To me, they just structurally make more sense and are less sloppy than traditional candles. What I began to observe over and over was the propensity for crypto markets to form long, tapering structures from these candles and that the likelihood of market reversal could be pinned to 1) the landing strip length and 2) its

proximity to the Bollinger Band. Further, I found that those markets that formed these on 1D or higher timeframes would often have explosive breakouts like a volcano that had charged up to blow out. These findings can be historically verified and back tested. I manually performed this observation all the way down to 1 minute trading and found that it was only at this lowest TF that it started to exhibit sub-50% reliability. Given this, the concept has been transposed into Acervator as a core component and trading logic gate with the number of candle strip candles being a user configurable setting.

This is also where I should mention that I am not at all a quant or mathematical trader. I consider myself a visual or tactical trader that focuses on market reversals. I have taught myself to visually interpret price chart structure using a given set of indicators used by countless other technical analysts. When looking at some of the labeling within Acervator, however, one will readily see military references and hints at how I view asset accumulation as something akin to progressive territory capture on a perpetual gameboard. The less skilled traders are surrendering it with every cycle of the market.

Functional. Two detectors build the Landing Strip and both run on every tick. The first places price inside the Bollinger channel and counts trailing Heikin Ashi bodies under a tightness threshold. It reads the operator's control directly.

`src/trading/scrumming_bot.py` — the first detector's call

```
min_pattern_candles=self.config.bb_landing_strip_candles,
```

Design intention. The second detector is the one that scores the taper. It counts a run of shrinking Heikin Ashi bodies, names the band the run reached, and returns a confidence boost the tick adds to its own. On a tapering rally into the upper band it answers detected, side upper, length three, boost 0.1437. The same tape without the taper answers not detected, and a five-candle tape answers not detected. A detected strip can force a direction, which is why the gate vocabulary marks it an override rather than a gate.

`src/trading/indicators/landing_strip.py` — `detect_landing_strip_v2`

```
def detect_landing_strip_v2(
    candles: list[Candle],
    min_consecutive: int = 3,
    shrink_threshold: float = 0.90,
    bb_tolerance_pct: float = 3.0,
    use_ha: bool = True,
    bb_period: int = 20,
    bb_std: float = 2.0,
) -> TighteningResult:
```

The second detector does not read the setting. The tick hands it a literal three, so the wizard's Landing Strip Candles control moves the first detector and leaves the second where it is. Both defaults sit at three, so the two agree until the operator changes the control.

`src/trading/scrumming_bot.py` — the second detector's call

```
tightening = detect_landing_strip_v2(
    candles,
    min_consecutive=3,
    shrink_threshold=0.90,
    bb_tolerance_pct=3.0,
)
```

Issue #432 carries it.

The issue #432 clause above is overtaken. It is not edited and not removed. It read:

> *The tick hands it a literal three, so the wizard's Landing Strip Candles control moves the first detector and leaves the second where it is. Issue #432 carries it.*

Issue #432 closed on 2026-10-23. The literal three is a named constant now, and the call states why the two counts are different things. `min_consecutive` counts shrinking Heikin Ashi bodies; the wizard's control counts the newest small-body candles the band-proximity check needs. The control is not meant to move the second detector.

```
src/trading/ata_gate_scan.py   TIGHTENING_MIN_CANDLES = 25
src/trading/ata_gate_scan.py   TIGHTENING_MIN_CONSECUTIVE = 3
src/trading/ata_gate_scan.py   TIGHTENING_SHRINK_THRESHOLD = 0.90
src/trading/ata_gate_scan.py   TIGHTENING_TOLERANCE_PCT = 3.0

src/trading/container/config.py, in BotConfig   bb_landing_strip_candles: int = 3
src/trading/scrumming_bot.py                    the call passes the four constants
```

Current reading, when a strip sets the side. A detected strip still names a side, but it sets the side of the vote only when three indicators agree. The vote must top what the two heaviest voters reach at full confidence, at the bot's weights on that tick. A bot with every weight at zero has no line, so its strip sets the side as before. Live, the Market Inspector scan, Paper and the Simulator all call the one function below.

`src/trading/ta_engine.py` — `vote_sides`

```
strip = band.landing_strip_side if band is not None and band.landing_strip else ""
if strip and not three_agree(confidence, signals):
    strip = ""
```

Driven on a stand-in exchange at a lower strip, with the twelve published weights. Two agreeing indicators read 0.1966 and the fold waited. Three read 0.2821 and the fold went through. The code before this change, taken from git, folded on both.

4 - Bollinger Band Travel Detection

Allowing a strategy to “see” where it is in relation to local width of the Bollinger Band is another aspect of treating the immediate market as a region on a map through which a stack of tokens is traveling. Along the route, we are able to offload surplus at the peaks and pick up additional supplies in the next valley. This is used in combination with Minimum Opposing Trade Distance. The complete question then becomes “Has the market traveled 70% (default), is current price X% distant from the most recent trade event, and is Target Delta not zero? When combining these gates with the (Technical Analysis) Indicator Voting Panel and Landing Strip Detection, you have a full market structure detection suite that is able to incrementally adjust a position’s size and generate recurring profits or growth with each market cycle.

Functional. The tick measures how far price has moved since the last trade and divides that by the current band width. When the fraction reaches the operator's percentage and the target delta is positive, the travel flag is set. The default is seventy percent and the wizard offers zero to a hundred. That is the compound question the entry states, and the code asks it in one expression.

`src/trading/scrumming_bot.py` — `ScrummingBot.tick`

```
_bb_width = max(bb_result.upper - bb_result.lower, 1e-12)
band_travel_frac = abs(ticker.last - self._last_trade_price) / _bb_width
if (
    abs(ticker.last - self._last_trade_price)
    >= self.config.band_travel_pct / 100.0 * _bb_width
    and delta > 0
):
    band_travel_triggered = True
```

Design intention. A fixed percentage means two different things in a quiet market and a violent one. Measuring the move against the band width makes the threshold follow the volatility instead. The flag then clears a standing trend hold and lets the harvest through. Travel is no gate of its own: the ten scrum labels carry no travel light.

`src/trading/scrumming_bot.py` — what the flag releases

```
trend_override = abs(delta) >= _interval_usd * 2.0 or band_travel_triggered
```

The clause above saying travel is no gate of its own is overtaken. It is not edited and not removed. It read:

> *Travel is no gate of its own: the ten scrum labels carry no travel light.*

The labels still carry no travel light. Travel is now a term inside Minimum Opposing Trade Distance, which is the combination the entry above names. `src/trading/otd_math.py`, in `band_extension_pct`, turns `band_travel_pct` and the band span into a percentage of the band midline, and `minimum_opposing_trade_distance_pct` adds that percentage to the interval and the fee. Zero keeps the term absent.

```
span = float(upper) - float(lower)
price = float(reference_price)
travel = float(travel_pct)
if travel <= 0.0 or span <= 0.0 or price <= 0.0:
    return BAND_EXTENSION_ABSENT_PCT
return span * (travel / 100.0) / price * 100.0
```

Current reading. The extension still adds to the interval and the fee. Where the interval and the extension together sit under one fee, the distance is two fees, so a narrow band cannot pull it under the round trip.

`src/trading/otd_math.py` — `DistanceSettings.distance_pct`

```
return minimum_opposing_trade_distance_pct(
    self.interval_pct,
    self.fee_pct,
    band_extension_pct(
        band_upper, band_lower, self.band_travel_pct, reference_price
    ),
)
```

5 - Position-Aware Technical Analysis

Traditional TA treats a signal (e.g., RSI > 70) as context-independent. Position-Aware TA encodes the current Bollinger Band position as context: RSI > 70 near the upper band confirms overbought (sell bias), but RSI > 70 near the lower band is a failed oversold — a trap for mean-reversion buyers. The system applies this inversion systematically across all oscillator signals. In other words, it attempts to strengthen the price to Bollinger Band relationship by reading additional indicators with relation to it. If you have the landing strip, band travel, opposing trade distance, and a bullish or bearish confidence score from the Indicator Voting Panel, it is likely a trading event will occur. In the end, many of these functions were layered in one after the other until I saw the platform take most trades that I would myself manually execute most of the time.

Functional. The band context enters at the bot, not inside the voters. The tick reads the band position once, marks the upper quarter and the middle band, and accumulates a skew from the votes that arrive. A bullish vortex adds 0.12 at the upper band and takes 0.05 back in the middle. A bullish MACD adds 0.08 and takes 0.03 on the same test. Ichimoku and Stochastic RSI move the skew on direction alone, with no band term at all.

`src/trading/scrumming_bot.py` — `ScrummingBot.tick`

```
at_upper_bb = bb_pos > 0.75
in_bb_middle = 0.35 <= bb_pos <= 0.65

if (
    sig.indicator == "vortex"
    and sig.direction == SignalDirection.BULLISH
):
    if at_upper_bb and sig.confidence > 0.5:
        position_boost += 0.12
    elif in_bb_middle:
        position_boost -= 0.05
```

Design intention. The skew relaxes or tightens the confidence floor the vote has to clear. A floor of 0.30 becomes 0.2679 at a skew of plus 0.12 and 0.3158 at a skew of minus 0.05. A skew of minus one returns infinity, which no confidence can clear, so a fully contradicted reading is refused outright rather than discounted.

`src/trading/scrumming_bot.py` — the two terms the floor reads

```
_ta_conf_skew = position_boost + bb_confidence_boost
```

The inversion does not reach the oscillator modules. Each indicator computes from candles alone, and the band position appears in six indicator modules only: Bollinger, band proximity, landing strip, M top, spring and W bottom. Neither the RSI module nor the Stochastic RSI module takes a band argument. Today the band context re-weights the vote after the fact. It does not re-read the oscillator.

Current reading, how far the skew relaxes the floor. The skew now relaxes the floor no further than the vote's own two-indicator line. The line is the confidence the two heaviest voters reach at full confidence, with every other voter at none, at the bot's weights on that tick. An eased floor stays above the line and never rises above 0.25. At equal weights the line is 0.1667, and at the published weights it is 0.1966. Where two indicators already reach 0.25, through voters that abstain or uneven weights, the favours ease nothing.

`src/trading/ta_engine.py` — `two_indicator_ceiling`

```
heaviest = sorted(
    (sig for sig in signals if not sig.abstained),
    key=lambda sig: sig.weight,
    reverse=True,
)[:2]
tally = [
    (
        replace(sig, direction=SignalDirection.BULLISH, confidence=1.0)
        if any(sig is one for one in heaviest)
        else replace(sig, direction=SignalDirection.NEUTRAL, confidence=0.0)
    )
    for sig in signals
]
return VotingEngine._aggregate(tally, "").consensus_confidence
```

Current reading, the line at the plain floor. The line now holds at every floor, the plain 0.25 included, so a floor can rise over 0.25. Where two indicators already reach 0.25, through voters that abstain or weights that lean on two, the floor sits just above what they reach. At equal weights that starts when four of the twelve abstain. At the published weights it starts when the three lightest abstain. A bot with every weight at zero reads a line of 0.0, so its floor does not move.

`src/trading/scrumming_bot.py` — `_skewed_confidence_floor`

```
denominator = 1.0 + skew
if denominator <= 0.0:
    return math.inf
eased = floor if math.isnan(denominator) else floor / denominator
return three_indicator_floor(eased, signals)
```

6 - Market Inspector

This addition was, like the Phantom Bot above, a move towards making Acervator “hyper-vigilant” of hypothetical opportunities that take the form of oppositional trades (assets that move inversely to one another) and bot swarm network topologies. The bot swarm itself will be covered in a moment but basically this feature allows for the creation of custom capital reinforcement networks where profits are configured to flow between positions in a manner that can potentially strengthen a portfolio faster than just focusing on single, isolated bot instances.

The Market Inspector is designed to analyze the broader market for groups of assets from which to build topologies and identify non-active assets that could be traded in opposition to active positions. Its findings can be pushed to the Simulator for historical back testing, to the Paper Trader for live market and real time validation, and, of course, Live Trading should the user choose to do so.

Functional. The Inspector walks daily, weekly and monthly candles, scores each market, and keeps the result. It then pairs a long-signal market with a short-signal market when the correlation of their returns over a rolling window sits inside a negative window. Those are the oppositional trades the entry describes, and the ranking puts the most negative correlation first.

`src/trading/market_inspector.py` — `MarketInspector._find_opposing_pairs`

```
corr = self._pearson(r_l[-m:], r_s[-m:])
if self._corr_min <= corr <= self._corr_max:
    pairs.append(
        OpposingPair(long_side=lo, short_side=sh, correlation_30d=corr)
    )
```

Design intention. The findings are meant to leave the screen. A separate module turns those signals into swarm proposals with the bots and the wires already written, and one push target exists today: the Simulator asks the Inspector for its current proposals at build time.

`src/gui/main_tabs/simulator_tab.py` — `_build_simulator_tab` The Simulator rebuild removed this file; it is not in the tree. The file is back in the tree: `src/gui/main_tabs/simulator_tab.py, in SimulatorTabMixin._build_simulator_tab` seats the Sim tab, or the note naming a caught build. The block below is not in it: `set_topology_getter` is in no file, so nothing wires a getter at build time. The Inspector still answers `src/gui/main_tabs/market_inspector_surface.py, in current_topology_proposals`, and `src/gui/main_window.py, in _push_candidate_to_sim` is the push the operator presses.

```
if hasattr(self._simulator, "set_topology_getter"):
    self._simulator.set_topology_getter(
        lambda: (
            self._market_inspector.current_topology_proposals()
            if hasattr(
                getattr(self, "_market_inspector", None),
                "current_topology_proposals",
            )
            else None
        )
    )
```

No Paper Trader exists to push to; entry 17 gives the measurement. Issue #23 tracks the tab's build-out and the topology and oppositional pushes, and issue #290 a live connector defect on Refresh.

The issue #290 clause above is overtaken. It is not edited and not removed. It read:

> *and issue #290 a live connector defect on Refresh.*

Issue #290 closed on 2026-10-17. The Inspector holds no connector. The window hands it a getter, and Refresh calls that getter and copies what it answers, so a wired source is read fresh on each press rather than kept from the build.

```
src/gui/main_tabs/market_inspector_tab.py, in _build_market_inspector_tab
    set_exchange_source(connectors_getter=..., scheduler=...)

src/gui/market_inspector.py, in set_exchange_source   stores the getter
src/gui/market_inspector.py                           dict(self._connectors_getter() or {})
```

7 - Bot Swarm and Smart Wire Network w/ Provenance Tracking

The Bot Swarm, which is how I presented the concept to Claude, is my version of a capital reinforcement network. At the Exchange level (swarms cannot be connected across multi-API boundaries) profit from individual bots can be routed to others with such outputs being termed Wire Credits since they are sent and received along Smart Wires which connect the bots. Each such wire can have its own flow rate. The network nodes are designed to protect a given bot from excessive outflows which would cripple its own growth capacity. The other piece of this is that Wire Credits are Provenance tracked which allows outflows to eventually start sending profits back to the originating bot with the over-arcing philosophy being long-term investment with accumulation and position strengthening rather than just sitting around waiting for a good outcome that might not arrive.

Now: only a fold spends wire credits, under the Maximum Target Growth Per Cycle, and a tranche whose own sale dollars are spent or sold ends and returns its credit to the pool for a later tranche (`src/trading/scrumming/wire_routing.py, in _return_wire_credits_to_pool`).

In combination with the Market Inspector, the Bot Swarm can receive and deploy groups of pre-configured and pre-connected bot clusters that immediately begin participating in the market. This layer, along with future portfolio-level algorithms, will enable the fully automated and persistent execution of complex trading strategies from a single terminal window.

Functional. The wire manager holds the topology and one ledger for each bot. A wire records a source, a target and a flow percentage, and a realised fold moves the share along it. The outflow is capped so a funder keeps its own growth capacity: a ten dollar scrum profit against a hundred dollar target releases ninety percent, and a zero profit releases nothing.

Now: a sale moves the share along a wire, and a fold moves nothing along it (`src/trading/scrumming/wire_routing.py, in WireRoutingMixin._route_scrum_proceeds_via_wires`).

`src/trading/smart_wire.py` — `compute_safe_outflow_pct`

```
    if scrum_profit_usd <= 0:
        return 0.0

    if band_upper <= band_lower:
        distance_to_fold_pct = 0.0
    else:
        distance_to_fold_pct = max(
            0.0, min(1.0, (current_price - band_lower) / (band_upper - band_lower))
        )
```

Design intention. Provenance is what makes the network more than a pipe. Each credit that lands on a tranche keeps the id of the bot that supplied it, so an outflow can eventually route profit back to the bot that funded it. The detail list is bounded at twenty entries per tranche and everything older folds into a rolled total, which keeps the sums exact without letting the record grow without limit.

`src/trading/scrumming/wire_routing.py` — `_WIRE_CREDIT_CAP`

```
    _WIRE_CREDIT_CAP = 20
    if not isinstance(wc, list) or len(wc) <= _WIRE_CREDIT_CAP:
        return 0
    overflow = wc[:-_WIRE_CREDIT_CAP]
    del wc[:-_WIRE_CREDIT_CAP]
```

How much of an arriving inflow a bot stacks onto an open position is its own setting, and it defaults to one percent.

`src/trading/container/config.py` — `BotConfig.wire_inflow_stack_pct`

```
    wire_inflow_stack_pct: float = 1.0
```

That setting is removed. An arriving inflow is added to the tranches the receiving bot already holds, or parked until a sell opens one, and no figure sizes it.

```
    if self._fold_tranches:
        share = self._spread_wire_usd_over_fold_queue(u, [credit])
```

Two mechanisms move capital between bots, and both are called throughout. The fold distribution sends a share of each realised fold to every wired target, and the scrum routing sends a share of each sale's proceeds along the same wires. No mechanism funds a new bot out of a parent's mature profit. Deploying a pre-connected cluster from the Market Inspector is an intention.

Now: the scrum routing sends each wire its share of a sale's surplus alone, the Scrum Fold Ratio's share of what the sale leaves past the dollars that buy back every unit it sold, less the room the growth cap leaves. A detonation sends each wire its percentage of the proceeds, because none of them can be folded back, and a fold sends nothing (`src/trading/scrumming/wire_routing.py, in _route_scrum_proceeds_via_wires`).

8 - Hunger and Satiety Indices

These elements were implemented as part of an effort to boost accumulation and scrumming performance in longer term bearish or bullish conditions that might not latch all logic gates to trigger a fold or scrum despite the Target Delta not being zero. This piece adds a factor to the bot that makes it more aggressively evaluate subsequent candles and will increase its likelihood of executing the re-zeroing action. This allows for trades to be executed in the hypothetical ‘dead zone’ between Minimum Opposing Trade Distance and the lower or upper Bollinger Band extent if the candle structure begins to float or cruise within it for too long.

Built, and switched off on every bot. Hunger is a count of the candles that pass while the position is off target and nothing fills. Satiety sets the count back to zero. The count becomes a skew on the TA confidence floor, so after a long idle stretch a weaker vote clears the floor.

Each bot carries its own two figures. The gain is zero on every bot, so no bot trades differently today. No screen sets the gain, and no measurement sets its figure.

Now: the gain is the 0.20 ceiling over the pace typed in each live Scrumming bot's Hunger Pace box, and none until its Hunger box is ticked (`src/trading/container/config.py, in BotConfig.hunger_gain_per_candle`).

`src/trading/container/config.py` — `BotConfig`

```
    # Floor skew each idle candle off target adds; 0.0 is off.
    hunger_gain_per_candle: float = 0.0
    # The most hunger_gain_per_candle may add to the floor skew.
    hunger_skew_max: float = 0.20
```

The tick adds one when the newest candle of the bot's TA timeframe carries a newer timestamp, the rule the circuit breaker uses for its cooldown. A hard circuit-breaker trip stops the count. A fill sets it to zero, and so does a tick inside the at-target dust band. The count lives in memory, so a restart starts it again at zero.

`src/trading/scrumming_bot.py` — `ScrummingBot._count_hunger_candle`

```
        if not math.isfinite(stamp) or stamp <= self._hunger_candle_ts:
            return
        anchored = self._hunger_candle_ts > 0.0
        self._hunger_candle_ts = stamp
        if not anchored:
            return
        before = self._hunger_floor_skew()
        self._hunger_candles += 1
```

Every fill the bot books reaches its clock figures once, so satiety sits there.

`src/trading/scrumming_bot.py` — `ScrummingBot._write_clock_figures`

```
    def _write_clock_figures(
        self, fill: SettledFill, order_id: str, filled: float, rests: bool
    ) -> None:
        """Run ``_sate_hunger`` for ``fill``, then write the mixin's clock figures."""
        self._sate_hunger(f"a {fill.type} fill")
        super()._write_clock_figures(fill, order_id, filled, rests)
```

The skew is the count times the gain, held at the ceiling. It joins the favours the floor already divides by, so the floor is 0.25 divided by one plus their sum. At the 0.20 ceiling alone the floor is 0.2083.

`src/trading/scrumming_bot.py` — `_hunger_skew`

```
def _hunger_skew(idle_candles: int, gain_per_candle: object, skew_max: object) -> float:
    """Return ``idle_candles`` times ``gain_per_candle``, capped at ``skew_max``.

    ``gain_per_candle`` and ``skew_max`` must be finite and above zero, else it is 0.0.
    """
    gain = as_finite_float(gain_per_candle)
    ceiling = as_finite_float(skew_max)
    if gain is None or ceiling is None or gain <= 0.0 or ceiling <= 0.0:
        return 0.0
    return min(ceiling, idle_candles * gain)
```

`src/trading/scrumming_bot.py` — `ScrummingBot.tick`

```
        _ta_conf_skew = position_boost + bb_confidence_boost
        if hunger_skew:
            _ta_conf_skew += hunger_skew
        if _bb_priority_arm:
            _ta_conf_skew += _BB_PRIORITY_SKEW
        _eff_conf_floor = _skewed_confidence_floor(_ta_conf_skew)
```

A lower floor changes three decisions and leaves a fourth alone. It releases a fold that every other gate passes and the TA confidence alone holds. That happens at the lower band while the delta is still inside the interval, where the deep-fold override does not reach. It lets a DIST sale through sooner. It refuses the hedge top-up more often, because that buy waits for a vote that is not bullish. It changes no scrum, because the ripe-harvest override already passes the TA gate wherever the band lets a scrum through. The band detect lines do not move, so hunger opens no trade short of them.

The bot log names hunger while it acts. A HUNGER line gives the count and the floor it sets alone, and a SATIETY line names the fill or the return to target that ended it.

Current reading, the DIST sale. The sentence above that hunger lets a DIST sale through sooner holds for the vote alone. A DIST sale now also waits for a price. Each fold that buys more units than its scrum sold queues those extra units beside the price it paid. They sell only at that price raised by the Minimum Opposing Trade Distance, or higher. Hunger can pass the vote sooner, and it does not move that price.

`src/trading/scrumming/tick_phases.py` — `TickPhaseMixin._tick_distribute`

```
clearing = dist_sets_clearing(self._dist_sets, price, otd_pct)
if not clearing:
    self._hold_dist_sets(price, otd_pct)
    return
```

Current reading, what hunger reaches and when it counts. Three statements in the readings above were wrong. "It changes no scrum" fails when the lower band sits at or under zero. The ripe-harvest override does not reach there, so the vote decides that scrum, and hunger can pass it. "Hunger opens no trade short of them" failed through the DIST sale, which has no band gate. The first reading counts "the candles that pass while the position is off target and nothing fills". That count ran inside the interval and while the Minimum Opposing Trade Distance blocked, and his entry names neither.

`src/trading/scrumming_bot.py` — `ScrummingBot.tick`, where the ripe-harvest override needs a lower band above zero

```
if (
    bb_result is not None
    and bb_result.upper > 0
    and bb_result.lower > 0
    and ticker.last > 0
):
    if delta > 0 and not below_interval and bb_pos >= _bb_upper_dt:
        _ripe_scrum = True
```

Hunger now reaches the scrum and fold chains only. The DIST sale and the hedge top-up read the same favours without hunger. So hunger no longer passes a DIST sale, and it no longer holds a hedge top-up. Both chains keep the band detect lines, and no override lifts them, so hunger now opens no trade short of those lines.

`src/trading/scrumming_bot.py` — `ScrummingBot.tick`

```
_ta_conf_skew = _favour_skew(
    position_boost,
    bb_confidence_boost,
    hunger_skew,
    priority_arm=_bb_priority_arm,
)
_eff_conf_floor = _skewed_confidence_floor(_ta_conf_skew, summary.signals)
_floor_no_hunger = _skewed_confidence_floor(
    _favour_skew(
        position_boost, bb_confidence_boost, 0.0, priority_arm=_bb_priority_arm
    ),
    summary.signals,
)
```

The count follows his entry. A new candle adds one only in the dead zone: the delta at the interval or past it, the opposing trade distance clear on its side, and the price still inside the band on that side. A candle inside the interval, short of the distance, or past the band adds nothing. It does not end the count either; a fill or a return to target still does that.

`src/trading/scrumming_bot.py` — `_in_hunger_zone`

```
    if not distance_clear or band is None:
        return False
    if delta > 0:
        return price < band.upper
    if delta < 0:
        return price > band.lower
    return False
```

No favour, hunger among them, now eases the floor to a level two indicators can reach. The floor stays above what the two heaviest voters reach at full confidence, at the bot's weights on that tick, and never rises above 0.25. At the published weights two indicators reach 0.1966, so the 0.20 ceiling alone still gives 0.2083.

Now: the floor stays just above what two indicators reach at every skew, no skew included, so it rises over 0.25 wherever two indicators reach more (`src/trading/scrumming_bot.py, in _skewed_confidence_floor`).

`src/trading/scrumming_bot.py` — `_skewed_confidence_floor`

```
    if denominator <= 1.0:
        return floor / denominator
    past_two = math.nextafter(two_indicator_ceiling(signals), math.inf)
    return min(floor, max(floor / denominator, past_two))
```

9 - Accumulation Shadows

The Accumulation Shadow was invented as a means of taking maximum advantage of long term and bearish conditions. It allows for a Scrumming to create a Shadow of itself that focuses on accumulating a smaller chunk of the target asset during a bearish regime. The shadow is allowed to accumulate downward but does not scrum. It was a bullish regime to be dedicated and then detonates thereby claiming all of its profits at once. Shadows operate in parallel with their parent bots and will be listed as Shadow Tranches once they are fully implemented.

Now: built, and off on every bot until its Accumulation Shadow box is ticked; a bearish fold's extra units then wait on a Shadow Tranche for a bullish regime (`src/trading/scrumming/shadow.py, in ShadowTrancheMixin._tick_shadow`).

Built, and switched off on every bot. A shadow is a tranche kind, not a second bot. The switch is one tick box on a live Scrumming bot's Settings tab. Paper and the Simulator open no shadow, so their tabs do not draw the box (`src/gui/live_settings/settings_tab.py, in SettingsTabMixin._add_shadow_row`).

The regime is a vote on 100 candles of the bot's Detonation Timeframe, read at most once an hour. A vote that leans bearish reads bearish, because that reading only marks a row and places no order. A vote reads bullish only when three indicators agree, because that reading sells.

`src/trading/scrumming/shadow.py` — `regime_of`

```
def regime_of(summary: Any) -> str:
    direction = summary.consensus_direction
    if direction == SignalDirection.BEARISH:
        return REGIME_BEARISH
    if direction == SignalDirection.BULLISH and three_agree(
        summary.consensus_confidence, summary.signals
    ):
        return REGIME_BULLISH
    return REGIME_NONE
```

A sale in a bearish regime marks its fold tranche with a kind key. The marked tranche is still a fold tranche. It buys back on the same chain, at the same Minimum Opposing Trade Distance. A merge, a despawn and a clear all skip it, so it survives a long regime. A row with no kind key behaves as it did.

`src/trading/scrumming/fold_tranches.py` — `_despawn_aged_tranches`

```
for _t in self._fold_tranches or []:
    if is_shadow(_t):
        report["shadow_kept"] = report.get("shadow_kept", 0) + 1
        _fold_keep.append(_t)
        continue
```

The fold gives the parent every unit its sale sold. The units past those move to a Shadow Tranche, off the parent's lots and holdings. No scrum, DIST sale or stack rung can sell them, and they grow no target. Each later fold in the regime adds a row at its own lower price. No setting sizes a row, because the fold measures it from the sale's own dollars.

`src/trading/scrumming/sizing.py` — `shadow_take_units`

```
take = math.fsum(
    bought * float(one.get(SHADOW_SALE_USD_KEY, 0.0) or 0.0) / spent
    - units_at_scrum(one) * share
    for one in marked
)
extra = bought - math.fsum(units_at_scrum(one) for one in eligible) * share
return max(0.0, min(take, extra))
```

A Shadow Tranche holds no wire credit. A credit may land on a marked fold tranche and is spent inside it. The units it buys stay with the parent, under the growth cap. The exit routes nothing down a wire (`src/trading/scrumming/sizing.py, in plan_fold_consumption`).

Now: the exit sends each wire its percentage of the sale, as a detonation does, because none of it can be folded back (`src/trading/scrumming/execution.py, in ExecutionEngineMixin._route_unfoldable_proceeds`).

A bullish regime sends every row at or over its floor through the detonation executor, in one order. The floor is the row's buy price plus the Minimum Opposing Trade Distance, and a row under it waits. The sale takes only what a row stands above its cost. The units left are worth what the row paid, and they join the lots at the fill price under the cycle growth cap (`src/trading/scrumming/execution.py, in _execute_detonation`).

A worked cycle, with room under the growth cap:

```
sale    1 unit at $100.00, fee 0.60%            $99.40 queued
fold    $99.40 at $95.00 buys 1.046316 units    the parent takes 1.000000
                                                the row takes 0.046316, cost $4.40
exit    bullish regime at $130.00               floor $96.52
        sells 0.012470 units                    $1.62 of profit
        keeps 0.033846 units                    worth $4.40, joins the lots
```

The whole cycle ends with more units than it began with, and profit in dollars.

Only a fold on the chain feeds a shadow. A Manual Fire or cartridge buy takes what the delta names and leaves no units past its tranche. A stack rung sale queues no fold tranche, so on a Stack Mode bot only a cartridge sale can mark a row (`src/trading/scrumming_bot.py, in ScrummingBot._book_stack_spend`).

A self-destruct or an expiry close returns every row to the lots first, so that exit sells them (`src/trading/scrumming/shadow.py, in ShadowTrancheMixin._return_shadows_to_lots`).

The bot log names each step: SHADOW REGIME, SHADOW OPENED, SHADOW ACCUMULATED, SHADOW HELD, SHADOW DETONATION and SHADOW DETONATED. The Stack Tranches tab lists each row with SHADOW in its Mode column, and the fill reads SHADOW_DETONATION (`src/trading/scrumming/shadow.py, in ShadowTrancheMixin._book_shadow_detonation`).

10 - Charge Up Permission Gate

This is client-side trade batching. It operates in conjunction with the Fold and Stack Tranches (explained later) to allow these individual positions to be combined into a single trade action in response to a given market trend.

Not built. The Charge Up Permission Gate has no code today. Fold and Stack Tranches do run, and entry 11 anchors them. Nothing combines them into one batched order.

The build this specifies. The fold chain already carries a gate that refuses when no tranche is queued, and the context it reads already counts them. Charge Up is the same test with a threshold: hold the fold until enough tranches are eligible, then release them together. It is a new gate class in the existing chain, and it needs no new context field.

Proposed, not present, in `src/trading/gate_chain.py`*:*

```
class ChargeUpGate(Gate):
    """FOLD-only: hold until ``min_tranches`` rows are queued, then release."""

    name = "charge_up"
    side = "fold"

    def __init__(self, min_tranches: int = 1) -> None:
        self.min_tranches = int(min_tranches)

    def evaluate(self, ctx: GateContext) -> GateResult:
        if ctx.n_fold_tranches >= self.min_tranches:
            return GateResult(passed=True)
        return GateResult(passed=False, blocker_message="charging")
```

The gate enters the fold chain before the hysteresis gate, and the gate vocabulary gains one fold label, taking the row from nineteen lights to twenty. A threshold of one reproduces present behaviour exactly. Issue #433 carries the build.

11 - Fold, Stack, Extractor, and Shadow Tranches / Provenance Queue

The term tranche emerged in Acervator as the concept of placing soft boundaries around where an opposing trade 1) can initially happen in terms of price level, 2) denote the amount of the position expected to be transacted once the price thresholds are met or exceeded, 3) allows wire credits to be fed into Fold Tranches (Buys) and Extractor Tranche profits to be distributed across Stack Tranches (Scrums / Sells) for additional accumulation and profit optimization. For the sake of clarity, Fold Tranches spawn after a Scrum (Sell) occurs; Stack Tranches spawn after a Fold (Buy) occurs; and Extractor Tranches spawn when an Extractor Bot finds a favorable Alternate Currency entry on its chosen timeframe and these tranches remain until the acquired Alternate Currency stack is sold by either the Extractor Bot (Sibling) or its corresponding Scrumming Bot (Parent). All Tranches will persist under a bot’s Details > Tranches Tab.

Now: Shadow Tranches are listed on the bot's Stack Tranches tab, each with SHADOW in its Mode column (`src/gui/main_tabs/stack_tranches_tab_surface.py, in compose_shadow_rows`).

Functional. All four tranche kinds run. Fold Tranches spawn on a scrum and hold the price of the sale that made them. Stack Tranches spawn on a fold and are priced as a ladder above the buy. A three-unit scrum at 100 with a one percent split and linear spacing returns rungs at 101, 102 and 103 carrying one unit each. Two rungs closer than a tenth of a percent merge, so 100, 100.05 and 105 come back as 100.05 carrying two units, then 105 carrying one.

`src/trading/stack_math.py` — `split_scrum_into_tranches`

```
def split_scrum_into_tranches(
    scrum_price: float,
    scrum_size: float,
    n_target: int,
    split_distance_pct: float = DEFAULT_INITIAL_GAP_PCT,
    spacing_mode: str = DEFAULT_SPACING_MODE,
    min_opposing_pct: float = 0.0,
    min_order_size: float = 0.0,
) -> list[Tranche]:
```

Design intention. Extractor Tranches are the sibling-and-parent relationship the entry describes. The Extractor opens a position on an alternate pair, names the tranche with an id that survives a restart, and hands the base currency back to the parent Scrumming Bot on exit. The parent raises its own target by the amount that arrived rather than selling it straight back out as surplus.

`src/trading/extractor_bot.py` — `ExtractorBot.tranche_id_for_position`

```
"""Build the stable Extractor Tranche id: ``bot_id|pair|opened_at``.

The row emitter and `set_tranche_arbiter` both format
`opened_at` through this one method, so they can never format
it differently and disagree on the same id.
"""
```

Shadow Tranches hold a bearish fold's extra units; entry 9 gives the build.

The queue sorts the fold tranches by their reference price, highest first, and drops any row whose reference or unit count is not a finite number. Wire credits ride on the tranche, which is the provenance the entry names.

Three defects stand against the Tranches tab and the rows it draws: spawned tranches do not match the scrum that spawned them (issue #367), the list is not visible and lifetime counters reset (issue #133), and the Units column prints a quantity that does not exist (issue #201). One more sits behind them. The capital reservation registry answers before a sale is placed, and of the five bot construction sites only the two Simulator controllers hand the bot a registry (issue #427).

The sentence above is overtaken. It is not edited and not removed. It read:

> *The capital reservation registry answers before a sale is placed, and of the five bot construction sites only the two Simulator controllers hand the bot a registry (issue #427).*

Three sites build a bot, and none of them hands it a registry. The constructor accepts one and every caller leaves it unset, so each bot resolves the process-wide registry instead.

```
src/gui/live_bot_window.py, in _run_bot                    builds a bot, passes no registry
src/gui/main_window.py, in _create_bot                     builds a bot, passes no registry
src/trading/container/restore.py, in restore_bots_from_state  builds a bot, passes no registry
src/trading/scrumming_bot.py, in ScrummingBot.__init__     capital_registry defaults to None
```

The engine already records the consequence at `src/trading/container/registry.py, in register`, which states that the setter has no caller. A sale is therefore admitted against the process-wide table rather than an injected one.

The first anchor in the block above named a line that had moved. The bot it means is built in `src/gui/live_bot_window.py`, in `_run_bot`, and that is the anchor the block now carries.

The issue #367 clause above is overtaken. It is not edited and not removed. It read:

> *spawned tranches do not match the scrum that spawned them (issue #367)*

Issue #367 closed on 2026-10-03. A spawned tranche carries the sale that made it. The sell loop writes the fill price onto each new record and carries the lot's own buy price with it, and the bounding pass then reads the sale's own net rate, which is the dollars the sale left with this bot divided by the units it sold.

```
src/trading/scrumming/execution.py
    each new record carries   usd, units, ref (the fill price), initial_buy_price

src/trading/scrumming/fold_tranches.py, in _bound_new_fold_tranches
    reads   the slice this sale appended, scrum_usd and scrum_asset
    rate    scrum_usd / scrum_asset, the sale's own net rate
```

The issue #133 clause above is overtaken. It is not edited and not removed. It read:

> *the list is not visible and lifetime counters reset (issue #133)*

The list is visible. A reading over a fleet shaped like the saved one drew every row. A queue of one tranche draws one row and a queue of twenty-three draws twenty-three, each row thirty pixels tall, each carrying its own

Fire button, none hidden. `src/gui/live_settings/fold_tranches_tab.py`, in `_create_fold_tranches_tab`, sets the row count from the fold rows plus the Extractor rows and fills each one.

No code resets the lifetime counters. The only writer of the reset stamp is `src/trading/scrumming/fold_tranches.py`, in `clear_lifetime_tranche_counters`, and its only callers are the Clear Lifetime Counters control on each Fold Tranches tab. One lifetime figure does fall without a control: `src/trading/scrumming/fold_tranches.py`, in `_bound_new_fold_tranches`, lowers the opened count each time one sell's per-lot records merge into one record, which the Lifetime tranches opened tooltip states.

The issue #201 clause above is overtaken. It is not edited and not removed. It read:

> *the Units column prints a quantity that does not exist (issue #201)*

Every figure the Fold Tranches table draws from a saved row passes one admission rule first. The rule takes a value only when it is exactly a whole number or a decimal number and finite, so it refuses a stored true-or-false value, a not-a-number, an endless value, text, and a whole number too large to hold. A refused figure draws an em dash, never a quantity, a price or an age.

The Extractor row's Age, Units and marked value read through that rule in `src/gui/live_settings/fold_tokens.py`, in `_compose_extractor_tranche_cells`, and its tooltip reads the leased amount and the alt holding the same way in `_compose_extractor_tranche_tooltip`. The other build draws the same row from `src/gui/main_tabs/fold_tranches_tab_surface.py`, in `extractor_row_cells` and in `guarded_extractor_units`.

The Clear fold tranches confirmation states the queued cash and the queued units before the queue is discarded. Both figures are totalled by `src/gui/live_settings/fold_tokens.py`, in `fold_queue_figure_total`, and by `src/gui/main_tabs/fold_tranches_tab_surface.py`, in `queue_figure_total`. A total is reported only when every queued row is readable; otherwise the figure draws an em dash rather than a partial sum offered as the whole queue. A missing key counts as nothing, which is how the table's own cells read it.

The same confirmation warns him when the bot also holds parked Smart Wire credit, because clearing the queue opens the absorb window. That figure reads through the same rule in `src/gui/live_settings/fold_tokens.py`, in `fold_parked_credit`, and in `src/gui/main_tabs/fold_tranches_tab_surface.py`, in `FoldTranchesTabModel.parked`. A credit that cannot be read counts as a credit that stands: `fold_parked_credit_stands` and `parked_credit_stands` keep the warning drawn and the Clear Wire control live, and the amount alone draws an em dash. A credit of nothing draws no warning and leaves that control off.

What the panel draws for a count it cannot work out

The cycle close ratio is the closed count over the opened count less the discarded count. A merge lowers the opened count while the discarded count holds, so the top of that sum can pass its bottom. `src/gui/live_settings/fold_tokens.py`, in `compose_cycle_close_ratio`, draws the dash for that pair and names the two counts beside it, never a share above one hundred percent. `src/gui/main_tabs/fold_tranches_tab_surface.py`, in `cycle_close_ratio`, draws the same thing for the React page.

How the ratio cell picks its colour

Red under a half, amber under four fifths, green above, and no colour at all while fewer than five tranches are openable. The allotment cell above it draws red when the queue claims more units than the bot holds.

The cell prints the share and decides the colour on the counts themselves. The share is worked out for the text alone, so the division has no part in the verdict.

`src/gui/main_tabs/fold_tranches_tab_surface.py`, in `cycle_close_ratio`

```
  if int(closed) < denominator * RATIO_RED_BELOW:
      return [text, FOLD_RATIO_RED_FG_HEX]
  if int(closed) < denominator * RATIO_AMBER_BELOW:
      return [text, FOLD_RATIO_AMBER_FG_HEX]
```

Both ways of asking give the same answer on every count either cell can be given. Swept over 1,399,894 whole-number pairs, with denominators up to one hundred thousand and every closed count within three of each threshold, no cell changes colour; 9,396 allotment readings stepped by the smallest amount a stored figure can move give the same result. The Fold Tranches screen on the Electron build, the Stack Tranches screen beside it and the Qt panel all draw the same colour for the same counts.

The Qt panel reads the same three names

The Qt Fold Tranches panel wrote the three figures in as plain numbers, so the arithmetic check could not read any of its three colour decisions. It now reads the names the Electron page declares, and it decides on the counts the way that page does.

The names are declared in `src/gui/main_tabs/fold_tokens_surface.py`.

`src/gui/live_settings/fold_tokens.py`, in `compose_cycle_close_ratio`

```
  if denominator < RATIO_MIN_DENOMINATOR:
      return (text, None)
  if int(closed) < denominator * RATIO_RED_BELOW:
      return (text, FOLD_RATIO_RED_FG_HEX)
  if int(closed) < denominator * RATIO_AMBER_BELOW:
      return (text, FOLD_RATIO_AMBER_FG_HEX)
```

Every colour is what it was. Swept over 131,302 whole-number pairs, with denominators from one to two hundred and three larger ones up to one hundred thousand, and every closed count from nothing up to the denominator, the panel draws the same colour on every pair and agrees with the Electron page on all 131,302 of them. Both counts are whole numbers, so a step of one is the finest step there is and no colour change can fall between two readings.

Moving the three declarations moved 77,340 of the 131,302 colours, on both sides together. Loosening the amber figure by a thousandth moved 151 of them. The same two moves against the old panel moved nothing on the Qt side, which left it disagreeing with the Electron page on 77,340 readings.

The arithmetic check now reads thirty-two of this panel's comparisons, where it read twenty-nine before. The three it gained are the three it could not see.

Standing tranches against the sells that opened them

The queue may never hold more tranches than the sells that opened them, outside a strong trend. `src/trading/scrumming/fold_tranches.py`, in `_bound_new_fold_tranches`, counts each such sell, and `src/gui/live_settings/fold_tokens.py`, in `compose_scrum_sells_row`, is the row on the panel that reports the pair. A standing count above the sells draws red. A bot whose save predates the sell counter draws the dash in place of a sell count.

One row number, not two

The # column is the tranche's place in the queue, and the Fire button quotes that number back in its confirmation. `src/gui/live_settings/fold_tokens.py`, in `configure_tranche_row_numbers`, hides the table's own row header, which drew the display position instead. The two disagreed on four of five rows under Oldest first and under Largest USD first. The React page draws only the declared columns, so it never held a second row number.

The Paper and Simulator Fold Tranches tabs call that same function on their own tables, so a tranche carries one number on all three screens. Both forks also draw the standing-against-sells row from `compose_scrum_sells_row`, the function named in the section above, rather than composing a row of their own.

What sets the Details window's opening width

The Fold Tranches tab is the widest tab, and the Fold-Tranche Cycle Health box is the widest thing on it. The despawn preview value is the longest line in that box. `src/gui/live_settings/fold_chrome.py`, in `install_despawn_rows`, lets that line wrap, which drops the tab's demand from 1938 to 1548 pixels. At a 1920 by 1080 display `src/gui/bot_live_settings.py`, in `open_at_content_size`, then opens the window at its full content width with a whole tranche row inside it. The screen clamp in `dialog_open_size_px` stays, because the operator cannot drag back a window wider than the display.

Every tranche names the opposing event that produced it

A fold tranche holds the tokens the scrum that made it sold. A scrum stack holds the base currency the fold that made it spent. Each record now carries the identity of that event and the two amounts the event produced, so the amount on the record can be read against the amount the event supplied.

`src/trading/scrumming/sizing.py`, in `fold_tranche_records`, is the one place a fold tranche is built. It takes the sale's own figures as its whole budget and hands them out across the lots the sale consumed, so a record carrying an amount the sale did not produce cannot be written there.

```
def fold_tranche_records(
    lots: list,
    fill_price: float,
    sold_units: float,
    proceeds_usd: float,
    event_id: str,
    created_ts: float,
) -> tuple[list, float]:
```

Three sites called their own copy of that loop before: the scrum phase, the distribution phase and the operator's Manual Fire. All three now call this one function. The second value it returns is the units a sale sold that no lot covered, and each site writes a FOLD ALIGNMENT SHORT line naming that figure.

`src/trading/scrumming_bot.py`, in `_open_stack_from_scrum`, is the one place a stack tranche is built, and it writes the same three fields from the fold fill that spawned the ladder.

The Units cell on the Fold Tranches row reads the tranche's units against the units its event sold, and its mouse-over names the event and both amounts. `src/gui/main_tabs/fold_tranches_tab_surface.py`, in `units_cell`, is the one declaration of that cell, and the Live, Paper and Simulator tables each call it.

The Stack-Tranche Cycle Health box gains a row reading the standing size against what the events that produced it acquired, from `src/gui/main_tabs/stack_tranches_tab_surface.py`, in `compose_event_alignment_row`. The row draws red while the standing size passes what was produced.

Marked units above held units, and what produces it

Driven on hand-built tranches with no venue reached, three scrums in a row at one price read 0.60, 2.20 and 5.40 times the holding. Every tranche's units equalled the units its own sale sold on all three. The ratio first passed one on the second scrum.

A bot that scrums three times without folding holds the units of three sales in its queue and only the remainder in its position, so the ratio climbs on every scrum and falls on every fold. The same six-event sequence read 2.20, 0.60 and 0.00 on the three folds that followed.

So a ratio above one is not by itself a tranche whose amount no event produced. The reading that separates the two is the amount on the record against the amount its event supplied, which is what the row now states.

A split against an unknown venue minimum is refused

A Stack ladder divides one fold's acquired tokens across spaced prices, and every piece must reach the smallest order size the venue published. Where the venue published no size rule the floor is unknown, and the split is refused rather than sized against a guess.

`src/trading/stack_math.py`, in `split_scrum_into_tranches`, refuses a minimum that is not a positive finite number, and `src/trading/scrumming_bot.py`, in `_open_stack_from_scrum`, passes the venue's own figure through unmasked so an absent rule arrives as the absence it is. The scrum still sells as one undivided order; only the split is refused.

The signature in the code block at the head of this entry is overtaken. It is not edited and not removed. It read:

```
    min_order_size: float = 0.0,
```

The parameter now defaults to `None`, and both `None` and zero are refused:

```
    min_order_size: Optional[float] = None,
```

Driven on a hand-built ladder of one unit over four rungs: a floor of 0.20 gives four pieces of 0.250000, a floor of 0.40 gives two of 0.500000, and both have every piece at or above the floor. A floor of 2.00 against one unit is refused. No floor at all is refused. In every arm that is not refused the pieces sum to the units the fold acquired.

`src/trading/stack_math.py`, in `_apply_min_order_size`, reads the smallest piece the ladder actually holds rather than a count of pieces. Two rungs that merge leave the ladder with pieces of unequal size, and a

count can fit while a piece sits under the floor. On a merged ladder of one unit the same arm read pieces of 0.500000, 0.250000 and 0.250000 against a floor of 0.300000 before, and three pieces of 0.333333 after.

The merge distance is one declaration both tranche types read

Two tranches within a tenth of a percent of each other in price combine. `src/trading/stack_math.py`, in `within_merge_distance`, is the one declaration of that distance, and `MERGE_THRESHOLD` beside it holds the figure.

The Stack ladder read that distance already, inside its own build. `src/trading/scrumming/sizing.py`, in `merge_fold_tranches_within_distance`, now reads the same declaration for the fold queue. The survivor keeps the lower reference price, which is the stricter fold threshold, sums the cash and the units, takes the units-weighted original cost and names both events.

```
def within_merge_distance(lower_price: float, higher_price: float) -> bool:
```

`src/trading/scrumming/fold_tranches.py`, in `_combine_fold_tranches_within_distance`, runs it on the standing queue each time a sale queues a tranche, and lowers the opened counter by what it combined so the panel's reconciliation still balances.

`src/trading/stack_math.py`, in `merge_into_standing_rungs`, moves a new rung into a standing rung inside the same distance. A standing rung holding a venue order is never a candidate, since combining it would need that order cancelled.

Driven in one run on both types: a pair 0.05 percent apart combined and a pair 0.5 percent apart did not. On the fold pair the survivor's cash and units were the sums of both, and its original cost was the units-weighted figure.

Current reading. A rung that a fold or a distribution opens carries the buy price of that fold as its floor reference. A new rung combines only with a standing rung holding the same floor reference, so no combine loses a floor. In Internal mode an activated rung with a floor reference is not spent while the price sits under that buy price raised by the Minimum Opposing Trade Distance. It stays activated, and the bot log names the price it waits for. A fold whose own ladder opens adds nothing to the distribution sale, because that ladder already offers its extra units.

`src/trading/scrumming_bot.py` — `ScrummingBot._rung_under_floor`

```
floor_ref = rung_floor_ref(rung)
if floor_ref <= 0.0:
    return False
try:
    floor = opposing_sale_price(floor_ref, self.opposing_trade_distance_pct())
```

What this changes for a queue that is already aligned

Three sales at prices a percent apart, driven through the real queueing path, produced the same three tranches before and after, to the digit. Sweeping the fold gate over 120,001 prices from $40.000 to $160.000 in steps of $0.001 gave 356,457 admitted rows, 3,352,311.201596 admitted units and $537,573,705.000000 of planned spend on both readings, and the same digest over every price.

Three sales at one price produced three tranches before and one after. The cash and the units the gate admits and plans over the same swept domain are identical on both readings; the row count falls from 352,323 to 117,441 because one record stands where three stood.

A queue that already exceeds its holding is not changed by any of this. What happens to an excess that already stands is the operator's decision.

The standing excess is retired on load

The decision above has been taken. The operator's ruling: drop the standing tranches whose amount no sale produced, keep the aligned ones, and bring the marked total back under the held total.

`src/trading/scrumming/fold_tranches.py`, in `retire_unaligned_tranches`, is the sweep. `src/trading/scrumming/state_io.py`, in `import_scrumming_state`, is the one caller. It runs once per bot, as that bot's state is restored, and after the parked wire credit has landed on the records. Every record queued after it is handed out of one sale's own figures, so the standing set is the only set that can hold an amount no sale produced, and that set does not grow.

No standing tranche carries the identity of the event that produced it, because the field did not exist when the tranche was queued. The sweep does not read it. One relation holds across every record the creation path makes: a record's dollars, less the wired-in dollars the record itself records, equal its units times the price its sale filled at, less the trading fee.

```
sale dollars / (units x sale rate)  =  1 - trading fee / 100
```

Now: a record that a wired sale queues carries the share the wire took, and the relation reads its dollars against units times price less the fee and that share. A record holding fewer dollars than that stays queued, because it waits until its dollars restore its units, so the sweep keeps both (`src/trading/scrumming/sizing.py, in unaligned_fold_tranches`).

`src/trading/scrumming/sizing.py`, in `unproduced_fold_amount`, reads that and answers why no sale produced a record's amount, or nothing where one could. `tranche_wire_usd` beside it takes off the wired-in dollars first, because wire money reaches a tranche with no units behind it.

The window carries one merge distance of slack at each edge, the distance at which `src/trading/stack_math.py`, in `within_merge_distance`, already calls two prices one price. For a bot at a 0.6 per cent fee a record is kept from 0.99301 to 1.00100 of price times units.

A record with no price, with no lot behind its units, or with no readable figure is retired at every fee. A drop counts as discarded and never as closed, so the panel's opened less closed less discarded still equals what stands, and the dropped record's price, its units, its dollars and the reason it went are written to the bot log.

Where the sweep spares a record. A Trading Fee % lowered by more than a tenth of a point moves every record the earlier sales queued below the window at once. `src/trading/scrumming/sizing.py`, in `unaligned_fold_tranches`, spares a record the fee window condemns where another standing record shares its implied fee, and where no record passed the window at all. Records agreeing on one fee are sales at one fee, which is what a changed setting leaves standing, and the log names the fee and how many were spared.

`src/trading/stack_math.py`, in `unproduced_stack_amount`, is the stack side. A stored rung carries no figure of the fold event that acquired its units, because `split_scrum_into_tranches` spreads one fold's

tokens over spaced rungs and the alignment is at their sum. Size and price are all a single stored rung can be tested against, and a rung holding a resting venue order is kept whatever it reads, because dropping the record would strand that order on the book.

Driven on constructed records in a redirected home, with no venue reached. On a queue holding one aligned record beside one carrying a quarter of its sale's dollars, the tree before this work kept both and the tree after kept the aligned one and retired the other. Over a swept domain of 120 records a sale produced, both trees left the same 120 records with the same digest, the same $468,656,385.72274005 and the same 27,528.615 units. Moving one of those records by one digit changed the digest and left all 120 standing. A bot whose fee setting read 0.00 per cent against five records queued at 0.6 per cent retired none of them and wrote the refusal.

The marked total is brought under the held total on load

The sweep above retires the records whose own figures no sale produced. The operator's second ruling covers what is left: keep the aligned tranches up to the held total and retire the oldest beyond it, so the units the queue marks never exceed the units the bot holds.

This is not a limit on how many tranches stand. Many may stand on either side, in any number. Only their sum is bounded.

`src/trading/scrumming/fold_tranches.py`, in `retire_tranches_above_held`, is the sweep. `src/trading/scrumming/state_io.py`, in `import_scrumming_state`, is the one caller, and it runs this sweep straight after the alignment sweep and before the fold queue's dollar total is re-derived. So it judges only what the alignment sweep left standing, and the re-derive reads the queue it leaves.

Diagram: flowchart, top to bottom

- a bot's state is restored -> **parked wire credit lands on the records**
- parked wire credit lands on the records -> **retire_unaligned_tranches — drops what no sale produced**
- retire_unaligned_tranches — drops what no sale produced -> **retire_tranches_above_held — drops the oldest beyond the held total**
- retire_tranches_above_held — drops the oldest beyond the held total -> **the fold queue's dollar total is re-derived**

Which figure is the held total. `import_scrumming_state` does not restore `_current_holdings`; the exchange supplies that on the next tick, and it still reads zero while the sweep runs. `src/trading/scrumming/fold_tranches.py`, in `_held_units_now`, therefore sums the units of this bot's restored lots, the same figure the restore drift warning in `import_scrumming_state` already sums. A lot holding no readable units figure is counted beside the total, never added as zero.

A held total of zero or less refuses the whole sweep and every tranche is kept. A bot whose lots were never saved would otherwise lose the lot. The refusal is written to the bot log and names the figure it read.

Which field dates a record. `src/trading/scrumming/sizing.py` declares `FOLD_AGE_KEY` as `created_ts`, written by `fold_tranche_records`, and `STACK_AGE_KEY` as `opened_ts`, written by `split_scrum_into_tranches`. The age sweep and this one read the same two names. `src/trading/scrumming/sizing.py`, in `tranches_beyond_held_total`, sorts the standing records oldest first on that field and drops from the front while the remainder still marks more than the held total, so the newest records are the ones that stay. A record whose stamp is not a positive finite number sorts as the oldest and is counted, and the log line states how many records the ordering could not place.

One sale stamps every record it queues with one `created_ts`, so a tie is the normal case. A tie keeps the queue's own order, which is the order those records were written in.

A queue that equals its held total exactly is left alone. Two things make that hold. `tranches_beyond_held_total` sums the marked units with `math.fsum`, which is exactly rounded, so the order the records are added in cannot push the total above the held figure; a running total of seven equal parts of ten thousand units reads 1.82 trillionths above it, which is near three times the real excess of 0.68 trillionths. And `src/trading/scrumming/sizing.py`, in `units_dust_at`, reads `UNITS_DUST` against the held figure and takes the larger of the absolute figure and that share of it, because one bit of a figure above ten thousand units is already wider than the absolute dust.

Measured at thirteen scales from a thousandth of a unit to a billion units, and at four part counts each: of 52 such queues the running total and the absolute dust dropped a record from 10, and the exact sum with the scaled dust dropped none. A queue genuinely above its held total still loses its oldest records at every one of those scales.

The two queues are measured separately and never added together. A fold tranche's units left the position when the scrum sold them; a stack rung's units are still in it, because `split_scrum_into_tranches` places a sell for each rung. One total across both is not a quantity. Each queue is measured against the one held total on its own.

A rung holding a resting venue order is kept whatever the arithmetic says, exactly as the alignment sweep and `src/trading/scrumming_bot.py`, in `clear_stack_tranches`, keep it, and its units are charged against the held total first. Where those kept rungs alone still mark more than the bot holds, the log line says so rather than claiming the invariant.

What this costs. Every record dropped here is units the bot sold and will now never buy back at that price. The operator chose this with that consequence stated. Each drop is written to the bot log with its price, its units, its dollars, its age in days and the reason it went, and counts as discarded and never as closed, so opened less closed less discarded still equals what stands. No order is placed or cancelled, and holdings, cost basis and target balance are unchanged.

Only a pending rung marks units. The clause above that says a stack rung's units are still in the position holds for a pending rung only. It is not edited and not removed. A rung reads pending until its sell fills or ends unfilled. A filled rung's units left the position when the sell filled. A cancelled rung's sell never reaches the book. Neither has units left to deliver.

So the sweep measures the stack queue on its pending rungs alone, and it retires only pending rungs. A filled or cancelled rung stays in the queue, and the log line counts how many stood outside the stack total. The sell for each rung, and the `opened_ts` stamp the ordering reads, come from `_open_stack_from_scrum`; the ladder arithmetic the clause names places nothing.

```
src/trading/stack_math.py, in rung_is_pending                          the one reading of a pending rung
src/trading/stack_math.py, in _rung_is_order_free                      reads rung_is_pending
src/trading/scrumming/fold_tranches.py, in retire_tranches_above_held  measures and retires those rungs only
src/trading/scrumming_bot.py, in _open_stack_from_scrum                places each rung's sell, writes opened_ts
```

`src/trading/stack_math.py` — `rung_is_pending`

```
def rung_is_pending(record: dict) -> bool:
    """Does `record` read `pending` as its `status`; a filled or cancelled rung
    marks no units."""
    return str(record.get(STACK_STATUS_KEY, "")) == STACK_STATUS_PENDING
```

What a dropped rung costs. The paragraph above says every record dropped here is units the bot sold and will never buy back at that price. That holds for a fold tranche. A dropped rung is units the bot holds and will not sell at that rung's price, and the log line states both.

Driven on constructed records in a redirected home, with no venue reached and no order placed. Two pending rungs of 4 units, inside 10 units held, stood beside a filled rung of 6 and a cancelled rung of 6. The tree before this work read 20 marked units and dropped both pending rungs and the filled one. This tree reads 8 and keeps all four. Fourteen status readings ran, each at 3 and at 30 units against 10 held. The tree before this work counted all fourteen. This tree counts only the three pending readings, and still retires a pending rung of 30 that holds no venue order. Over 600 bots holding only pending rungs, both trees left the same queues and dropped the same 1,130 rungs. Lowering one digit of one lot on one side only made 227 of the 600 differ.

```
at 100 units held, pending rungs inside the holding   pending rungs lost
                                                       before    after
60 pending in 6 rungs, 30 filled in 3, newer               0        0
60 pending in 6 rungs, 60 filled in 6, newer               2        0
60 pending in 6 rungs, 60 cancelled in 6, newer            2        0
90 pending in 9 rungs, 90 filled in 9, newer               8        0
100 pending in 10 rungs, 10 filled in 1, newer             1        0
60 pending in 6 rungs, 200 filled in 20, newer             6        0
60 pending in 6 rungs, 60 filled in 6, older               0        0
```

Where the filled rungs were the older ones, or the pending rungs held venue orders, the tree before this work dropped 2 filled records instead. This tree drops none.

What the screen reads afterwards. The allotment row described under *How the ratio cell picks its colour* draws its figure through `src/gui/live_settings/fold_tokens.py`, in `compose_units_marked_row`, on the Qt panel and through `src/gui/main_tabs/fold_tokens_surface.py`, in `compose_units_marked_row`, on the Electron page. After a load that sweep ran on, that row reads at or below `1.00x` and draws no red, unless a resting venue order pinned a rung or no held total was readable.

Driven on constructed records in a redirected home, with no venue reached and no order placed. Three fold records of 30 units each against 40 units held kept the newest and retired the two older ones; the tree before this work kept all three. Over 400 bots whose held figure sat just above their marked total, both trees left the same queues with the same digest on all 400 and dropped nothing. Lowering one digit of that held figure left 4 of the 400 digests matching and dropped 945 records. Over 1,200 pseudo-random bots, 645 stood above their held total on the tree before this work and 62 after it, and those 62 are exactly the bots whose remaining excess sits in rungs holding a resting venue order, each reported as such. The counter reconciliation held on all 1,200. Blinding the date stamp on all three records of the first reading made the sweep report three records it could not place.

The Product Manual PDF rebuild is owed for this page.

12 - Technical Analysis Indicator Confidence Tiers

Most TA systems stack indicators additively. The Tier-4 suite organizes signals into four confidence tiers with cumulative weighting: Tier-1 (structural, highest weight), Tier-2 (confirmation, medium weight), Tier-3 (momentum), Tier-4 (context). Entries require Tier-1 + Tier-2 alignment; Tier-3/4 scale confidence but cannot override. This tier system arose out of queries with Claude Code while it was roleplaying as a TA Archetype and is only one such example where most of the other such contributions deal with bug fixes.

Not built as tiers. The four-tier scheme is a design. The engine has no tier. The signal record carries an indicator name, a timeframe, a direction, a confidence, a weight, a details map, a timestamp and an abstention flag, and nothing else. No field names a tier, no code groups the voters into tiers, and no rule requires a Tier-1 and Tier-2 alignment before an entry.

`src/trading/indicators/types.py` — `Signal`

```
@dataclass
class Signal:
    """One indicator's output at a point in time."""

    indicator: str
    timeframe: str
    direction: SignalDirection
    confidence: float
    weight: float = 1.0
    details: dict = field(default_factory=dict)
    timestamp: float = field(default_factory=time.time)
    abstained: bool = False
```

What runs instead. A flat per-indicator weight. Each of the twelve voters carries one number between 0.8 and 1.2, and a signal's score is its direction times its confidence times that weight. Every vote can move the total, and none can veto another.

`src/trading/ta_engine.py` — `DEFAULT_WEIGHTS`

```
DEFAULT_WEIGHTS = {
    "bollinger_bands": 1.0,
    "vortex": 0.9,
    "macd": 1.2,
    "stochastic_rsi": 1.0,
    "ichimoku": 1.1,
    "volume": 0.8,
    "slingshot": 1.0,
    "adx": 1.0,
    "kaufman_er": 1.0,
    "supertrend": 1.0,
    "zscore": 0.9,
    "rsi": 0.8,
}
```

A new bot starts with all twelve at 0.0, written by `src/trading/ta_engine.py, in new_bot_weights`. It votes neutral and makes no first buy until Save on the Settings window hands it the TA Indicators figures. The published figures above are the start figures of those sliders.

The build this specifies. A tier is a second map beside the weights, and the veto is one test in the aggregator. The signal record grows one field, and it has to go last: the record is built positionally in this tree, so inserting a field earlier would re-bind an existing argument.

Proposed, not present, in `src/trading/ta_engine.py`*:*

```
DEFAULT_TIERS = {
    "bollinger_bands": 1,
    "ichimoku": 1,
    "supertrend": 1,
    "macd": 2,
    "vortex": 2,
    "adx": 2,
    "stochastic_rsi": 3,
    "rsi": 3,
    "slingshot": 3,
    "volume": 4,
    "kaufman_er": 4,
    "zscore": 4,
}
```

Proposed, not present, in `VotingEngine._aggregate`*:*

```
_lead = {s.tier for s in signals if not s.abstained and s.direction is not NEUTRAL}
if require_tier_alignment and not {1, 2} <= _lead:
    consensus_conf = 0.0
```

Issue #433 carries the build.

Current reading, what a trade needs from the vote today. The tiers are still not built. What a trade the vote decides needs today is three agreeing indicators: the vote must top what the two heaviest voters reach at full confidence. That holds at every floor, where a landing strip sets the side, and on a bot's first buy. A bot with every weight at zero is outside the rule on a later trade, except a buy back through the first entry. Neither that buy nor a first buy goes out at twelve zeros, because `src/trading/scrumming/tick_phases.py, in TickPhaseMixin._first_buy_read` refuses it before any vote.

`src/trading/ta_engine.py` — `three_agree`

```
if not any(sig.weight > 0.0 for sig in signals):
    return True
return confidence > two_indicator_ceiling(signals)
```

13 - Initial-Purchase-Price Floor on Folds

This rule asserts that a subsequent Fold occurring after a Scrum MUST be less than sell. This, of course, is asserted by Bollinger Band Travel, Minimum Opposing Trade Distance, Target Balance Not Zero, etc.

Now: every fold path, the cartridge included, pays a tranche back only at or under its own sale price less the distance, wired or not (`src/trading/scrumming/sizing.py, in tranche_rebuy_price`).

Functional. The floor runs, and it lives on the tranche. Every fold tranche stores the price of the scrum that spawned it. The row is released only when price has fallen to that reference multiplied by the rebuy factor, and the factor is one minus the opposing trade distance. A one percent interval with a six tenths fee gives a distance of 1.60 percent and a factor of 0.984, so a tranche spawned by a sale at $100.00 folds back at $98.40 or lower.

`src/trading/scrumming/fold_tranches.py` — `_fold_eligible_tranches`

```
return [
    t
    for t in self._fold_tranches
    if ticker_last <= float(t.get("ref", 0)) * otd_factor
]
```

A tranche can take money from more than one sale. When a sell's money moves into an older part-spent row instead of opening a new one, the row keeps the price its own scrum sold at, so the distance is still measured from a price a sale happened at. The dollars-per-unit rate across every sale that fed the row is held apart from that price as the row's blend rate, and only the units arithmetic reads it.

`src/trading/scrumming/sizing.py` — `tranche_sale_rate`

```
blended = float(record.get("blend_rate", 0) or 0)
if blended > 0:
    return blended
return float(record.get("ref", 0) or 0)
```

A row fed at one price alone carries no blend rate, and its rate is its scrum price. `src/trading/scrumming/fold_tranches.py`, in `_top_up_remnant_fold_tranches`, moves the money and writes a blend rate only where the two rates differ. `src/trading/scrumming/sizing.py`, in `units_at_scrum`, is the one place that turns a row's dollars back into the units it sold; the fold executor and `fold_surplus_usd` both read it, so the units a fold is measured against and the surplus it books cannot disagree.

Current reading. The function above no longer turns a row's dollars back into units. It reads the row's own units, the units its scrum sold, so a fold's extra is the units it bought less the units its scrum sold. The sell fee no longer counts as extra, and a cycle that ends with fewer units books no gain. The blend rate is now read only by the checks that hold a row's dollars against its sale, and by the top-up that writes it.

`src/trading/scrumming/sizing.py` — `units_at_scrum`

```
units = as_finite_float(record.get("units", 0.0))
if units is None or units <= 0.0:
    return 0.0
return units
```

Design intention. The same distance is held in three places so the rule cannot be true for the bot and false for a row. The bot-wide hysteresis gate holds it, the per-tranche filter above holds it, and each new ladder rung is priced against it.

`src/trading/otd_math.py` — `fold_rebuy_factor_from_pct`

```
def fold_rebuy_factor_from_pct(otd_pct: float) -> float:
    """Convert an already-clamped OTD percentage to a FOLD rebuy factor.

    Multiply a tranche's ``ref`` by this to get the HIGHEST price at
    which that tranche is still eligible to be re-bought:

        eligible  <=>  ticker.last <= ref * factor
    """
    return 1.0 - (float(otd_pct) / 100.0)
```

The lot carries its own cost basis as well as the scrum reference, and the tick counts the rows eligible against that basis.

`src/trading/scrumming/tick_phases.py` — the lot's own basis

```
_patent_only_eligible = sum(
    1
    for _t in self._fold_tranches
    if ticker.last
    <= float(_t.get("initial_buy_price", _t.get("ref", 0)))
)
```

One gap sits in the same path, and the function names it in its own docstring. The fold gate wraps the distance call and falls back to a distance of zero when a config value is not a number. At that fallback the rule is not active.

Current reading. No path falls back to a distance of zero. A distance setting that holds no number refuses the scrum and the fold, and the bot log names the setting.

`src/trading/scrumming_bot.py` — `ScrummingBot.tick`

```
try:
    _distance = self.opposing_distance_settings()
except DistanceSettingUnreadable as _unread:
    self._refuse_unreadable_distance(_unread, "SCRUM AND FOLD", ticker.last)
```

Where a tranche's rebuy guide is placed

The owner states the rule for where a tranche sits:

> *Placement of Tranches must first be a function of Minimum Opposing Trade Distance + Trading Fee + BB Extension (if present...).*

The guide is measured from the price the tranche's scrum sold at, the `ref` on the stored row, and never from the price first paid. A row shows both: the Sell ref column holds the first and the Original cost column holds the second. A guide above the cost basis is normal, because a bot that sold high can buy back above its own cost and still gain units.

`src/trading/otd_math.py` — the guide, from the sell reference

```
def fold_rebuy_price(sell_ref_price: float, otd_pct: float) -> float:
    return float(sell_ref_price) * fold_rebuy_factor_from_pct(otd_pct)
```

Two of the three terms reach the guide. The opposing trade distance is `scrumming_interval_pct` and the trading fee is `trading_fee_pct`, both declared in `src/trading/container/config.py`, and `minimum_opposing_trade_distance_pct_from_config` is the one place that reads either. Moving the declared interval from 1.0 to 7.3 moves the guide on a hundred-dollar sell reference from $98.40 to $92.10; moving the declared fee from 0.6 to 2.9 moves it to $96.10.

Current reading. The fee move reads differently. At a fee of 2.9 % the 1.0 % interval sits under one fee, so the distance is two fees, 5.80 %, and the guide reads $94.20. The interval move keeps its figure, since 7.3 % sits above the 0.6 % fee.

```
interval 1.0 %   fee 2.9 %   distance 5.80 %   guide $94.20
interval 7.3 %   fee 0.6 %   distance 7.90 %   guide $92.10
```

The band extension is the third term and no caller supplies it yet. `band_extension_pct` states it as a share of the reference price, taking the band's own span the way `src/trading/indicators/bb_proximity.py`, in `detect_bb_proximity`, takes it. Moving the declared `bb_tolerance_pct` from 1.0 to 4.7 moves that term from 0.200% to 0.940%, and a distance holding it would read 2.540% in place of 1.600%. A band makes the distance differ per tranche, and `src/trading/scrumming/sizing.py`, in `eligible_fold_tranches`, takes one factor for the whole queue, so wiring the term reaches that function, `src/trading/scrumming/tick_phases.py`, `src/paper/paper_run.py` and `src/simulator/back_test.py`.

Now: every tranche reads one distance, the bot's distance settings at the tick's band, the same for a wired tranche as for an unwired one (`src/trading/scrumming/sizing.py, in tranche_rebuy_price`).

A guide above the price it is measured from cannot be produced

An earlier reading of the factor trusted its caller to clamp the percentage first. Its own docstring said so:

> *Convert an already-clamped OTD percentage to a FOLD rebuy factor.*

The current reading is that the factor clamps its own argument, so a negative percentage cannot lift it above 1.0 and the guide cannot read above the price it is measured from.

`src/trading/otd_math.py` — the clamp inside the factor

```
def fold_rebuy_factor_from_pct(otd_pct: float) -> float:
    return 1.0 - (clamped_otd_pct(otd_pct) / 100.0)
```

Swept from -5.0% to 60.0% in steps of 0.001%, 5,000 of the 65,001 percentages produced a guide above its own reference before the clamp moved and none produce one now. The worst was 1.05 times the reference at -5.0%. The configured path never reached that domain: over 251,001 interval and fee pairs from -2.0 to 3.0 in steps of 0.01, the guide ratio stayed between 0.94 and 1.00 both before and after.

The Fold Tranches page model held the percentage and the factor as two numbers a host could set apart. Asked for a percentage of 1.60 and a factor of 1.04, the row drew a guide of $104.00000000 against a sell reference of $100.00000000. The model now derives the factor from the percentage and the same request

draws $98.40000000.

What the row's percentage means

The Status cell states the move the live price must still make to reach the guide, so the figure carries a minus while the price sits above it. With a hundred-dollar sell reference and a distance of 1.60%, a price of $110 reads `Need price ≤ OTD (-10.55%)` and a price of $90 reads `Price-OK (+9.33% vs OTD)`. The earlier reading stated where the price sat rather than the move it must make, so the same two prices read `+11.79%` and `-8.54%`.

`src/gui/main_tabs/fold_tranches_tab_surface.py` — the signed move

```
def status_move_pct(price: float, threshold: float) -> float:
    return (threshold - price) / price * 100.0
```

The Min rebuy tooltip names both prices the guide sits between: *A guide, not a trigger: sell ref $100.00000000 less 1.60% is $98.40000000. A fold-back still needs TA.* The Live, Paper and Simulator panels and both React pages draw `min_rebuy_cell` and `status_cell` from this one module, so no screen can carry a different figure.

Now: each row's guide and status read the bot's one distance, so a wired tranche shows the same buy-back price an unwired one does, and a tranche short of dollars shows the lower price at which its own dollars restore its units (`src/gui/main_tabs/fold_tranches_tab_surface.py, in rebuy_guide`).

14 - Position Ceiling and Detonation

This setting allows a total profit cap to be set for a given position before it sells its entire or a large portion of its total value.

Functional. The entry covers two mechanisms. They act on opposite sides of the cycle and both start disabled. The ceiling brakes buying. It is the anchor target multiplied by a factor clamped between one and ten, and the fold rate tapers as the position approaches it: full rate below half the ceiling, a linear taper from full to a tenth across the upper half, and a hard stop at the ceiling. The ceiling starts no sale.

`src/trading/scrumming_bot.py` — `ScrummingBot.fold_rate_taper`

```
ratio = self.ceiling_ratio
if ratio is None:
    return 1.0
if ratio >= 1.0:
    return 0.0
if ratio < 0.5:
    return 1.0
return 1.0 - (ratio - 0.5) / 0.5 * 0.9
```

Design intention. Detonation sells. It requires the feature to be enabled, a position above the anchor, and a bullish consensus at or above the confidence minimum on the detonation timeframe, which defaults to one day. The check is edge triggered, so one bull reversal fires it once, and a rate limit spaces the checks. The executor then market-sells the surplus above the anchor, clears the fold tranches and reseeds the lots. It harvests the excess above the anchor, not the whole position.

`src/trading/scrumming_bot.py` — `ScrummingBot._check_detonation_trigger`

```
conf_min = float(getattr(self.config, "detonation_confidence_min", 0.75))
is_bullish = (
    summary.consensus_direction == SignalDirection.BULLISH
    and summary.consensus_confidence >= conf_min
)

fired = is_bullish and not self._detonation_last_signal_bullish
self._detonation_last_signal_bullish = is_bullish
```

Both features default to off, and the ceiling multiple defaults to five.

`src/trading/container/config.py` — `BotConfig`

```
position_ceiling_enabled: bool = False
position_ceiling_multiple: float = 5.0  # Range [1.0, 10.0]
```

Issue #336 records both pairs among the wizard settings that never reach the bot. Issue #107 records a detonation that can fire twice across a restart. Any earlier description reading "bearish" is wrong: the code compares against the bullish direction.

The issue #336 and issue #107 clauses above are overtaken. They are not edited and not removed. They read:

> *Issue #336 records both pairs among the wizard settings that never reach the bot. Issue #107 records a detonation that can fire twice across a restart.*

Issue #336 closed on 2026-10-04 and issue #107 closed on 2026-10-17. Both pairs reach the bot, and the detonation flag survives a restart.

```
src/trading/container/config.py, in _BOT_CONFIG_SCRUMMING_ONLY_FIELDS
    position_ceiling_enabled, position_ceiling_multiple,
    detonation_enabled, detonation_timeframe, detonation_confidence_min

src/trading/scrumming/execution.py   reads position_ceiling_enabled and the multiple
src/trading/container/config.py, in ceiling_multiple   the one reader of the multiple
src/trading/bot_container.py         publishes both to the screen
src/trading/scrumming_bot.py         reads detonation_enabled before the bar fires
```

The flag the detonation bar reads is saved with the bot and read back on load, so a restart no longer presents a bull run the bar has already fired on as a fresh one.

```
src/trading/scrumming/state_io.py
    saved     detonation_last_signal_bullish
    restored  detonation_last_signal_bullish, False when the record has none
```

How the taper and the detonation bar read today

Two readings in section 14 above carry wording this unit changed. They stand as written, and these are the readings that replaced them.

written above	what it reads now
"the fold rate tapers as the position approaches it"	The fold size tapers. The figure multiplies the dollars a fold spends and never the interval between folds. The schedule itself is unchanged: full size below half the ceiling, a linear taper to a tenth across the upper half, a hard stop at the ceiling
"`src/trading/scrumming_bot.py` — `ScrummingBot.fold_rate_taper`"	`src/trading/scrumming_bot.py, in ScrummingBot.fold_size_taper`, reading the same schedule from `src/trading/scrumming/sizing.py, in fold_size_taper`
"a bullish consensus at or above the confidence minimum"	Unchanged, and the minimum now reaches the status record every screen reads, so the Fire tooltip on a Bot Swarm row names the figure a vote has to clear

`src/trading/scrumming_bot.py, in ScrummingBot.fold_size_taper` — the property

```
  ratio = self.ceiling_ratio
  if ratio is None:
      return 1.0
  return fold_size_taper(ratio)
```

The opening confidence figure is one declaration, read by the stored record and by the Min Confidence box.

`src/trading/container/config.py` — the figure

```
  DETONATION_CONFIDENCE_MIN_DEFAULT: float = 0.75
```

The detonation vote and its bar, 2026-10-28

Two readings in section 14 above carry wording this change moved. They stand as written, and these are the readings that replaced them.

written above	what it reads now
"`conf_min = float(getattr(self.config, "detonation_confidence_min", 0.75))`"	The fallback reads `DETONATION_CONFIDENCE_MIN_DEFAULT`, and the vote the bar is compared against reads the bot's own twelve weights
"a bullish consensus at or above the confidence minimum on the detonation timeframe"	Unchanged, and the minimum can now be set from 0.00 to 1.00, so a reading the vote reaches can fire it

`src/trading/scrumming_bot.py, in ScrummingBot._check_detonation_trigger`

```
  engine = VotingEngine(weights=self._ta_weights)
```

The Product Manual PDF rebuild is owed for this page.

The sentence above stands as written. This is the reading that replaced it.

written above	what it reads now
"The Product Manual PDF rebuild is owed for this page."	`docs/Acervator-Product-Manual.pdf` is rebuilt with this page

15 - Proof of Accumulation (PoA)

This is another one of the larger and more distinguishing features of the platform. At this point, it is a design proposal for a blockchain that provides certified trade performance between competitors without revealing their specific strategy tunings within Acervator or their identity. When this blockchain is initiated, ZERO tokens for it will exist in the world. I intend it to be, much like A15 to 1) have a fixed supply, 2) mean something through its generation and acquisition. Tokens are only created through the PoA tournaments and defeating Token Beasts or inflicting the most damage to them in a colosseum format. Additional functions would allow PoA tokens to be socketed into the platform to change its appearance and identify a user as a victor.

Functional. The competition package is the Proof of Accumulation package, and its engine runs without a screen. The engine walks a competition through registration, active trading, submission and adjudication, and hashes each entrant's config so a strategy proves consistency without publishing itself. A Merkle log produces the root that stands in for the trade record.

The token ledger is append-only and hard-capped. Balances come from replaying the log, and no operation edits one. An award that would cross the cap raises rather than trimming, and an award id repeats at most once.

`src/competition/token_ledger.py` — `TokenLedger.award`

```
event_id = self._event_id(bot_id, competition_id, tier.name)
if event_id in self._seen:
    return None

if self.total_minted() + tier.base_value > TOTAL_SUPPLY_CAP:
    raise OverflowError(
        f"Supply cap {TOTAL_SUPPLY_CAP:,} ACRV would be exceeded. "
        f"Only {self.remaining_ever()} tokens remain mintable."
    )
```

Design intention. The fixed supply is the whole point of the token, so the cap is a constant the ledger reads rather than a policy a caller can pass. It reads ten million.

`src/competition/season_schedule.py` — `TOTAL_SUPPLY_CAP`

```
TOTAL_SUPPLY_CAP = 10_000_000  # Hard cap — immutable
```

Nothing in the running platform starts a competition. Every module that would request one sits outside the running application, and the retired-tab sentinels assign nothing to the competition and testnet tabs. The canonical tab order now lists an Accumulation tab, and that tab is empty: it draws its name, one sentence saying it is not built, and issue #147, which covers the build-out.

`src/competition/local_testnet.py` — the in-platform chain, built every session

```
MAX_SUPPLY_WEI = TOTAL_SUPPLY_CAP * (10**TOKEN_DECIMALS)
```

Read on the current tree, no session builds the chain. The main window no longer installs the testnet bridge at launch, so the bridge and the eleven competition parts it installs stay unbuilt, and its 250 ms timer never starts. The competition package stays on disk while Proof of Accumulation is paused.

`src/gui/main_window.py` — `MainWindow._setup_ui`, the first two build steps

```
            main_layout = self._timed_build_step(
                "header strip", self._build_header_strip
            )

            self._main_tabs = _main_tab_book_class()()
```

The trophy generator draws the trophy as vector art and letters SOLVE ET COAGULA around its rim. No module imports it. Token Beasts and the colosseum appear in no Python file at all.

16 - Technical Analysis, Goodness of Fit, and the Confidence Index

The Confidence Index is the collated result of the 12-indicator Voting Panel and emulates the concept of a set of features (market structures / indicators) being read by a system having similar measurements for a given or expected or past action i.e. a Fold will always have a certain number of indicators voting bearish before its released for execution.

Functional. The Confidence Index runs. The voting engine builds twelve voters and holds twelve weights, and both count twelve when driven. The net is the bullish score minus the bearish score, and the confidence is the absolute net divided by the summed weight of the voters that did not abstain. Two bulls at weights one and two against one bear at weight one give a net of 2.0 and a confidence of 0.5. A single bull beside an abstainer of weight three gives 1.0, not 0.25, which shows the abstainer leaving the denominator.

`src/trading/ta_engine.py` — `VotingEngine._aggregate`

```
net = bull_score - bear_score

voted_weight = sum(s.weight for s in signals if not s.abstained)
consensus_conf = abs(net) / voted_weight if voted_weight > 0.0 else 0.0
```

Design intention. Confidence measures the breadth of agreement across the panel, not the strength of one reading. That is why an abstention has to be declared rather than inferred: an indicator with too little history returns a fabricated middle reading, and counting it would dilute a real consensus.

`src/trading/indicators/types.py` — `Signal.weighted_score`

```
@property
def weighted_score(self) -> float:
    """Signed score: direction * confidence * weight."""
    return self.direction.value * self.confidence * self.weight
```

Goodness of fit is not built. The phrase names the work of comparing the Simulator against the live platform, and the criterion for that comparison is gates latching identically on the same data. The measure has no settled definition yet and nothing here to attach it to, so nothing is proposed. Issue #117 carries it.

In development.

The issue #117 clause above is overtaken. It is not edited and not removed. It read:

The measure has no settled definition yet and nothing here to attach it to, so nothing is proposed. Issue #117 carries it.

Issue #117 closed on 2026-09-19. The criterion is settled and it is in the tree. A run reads identical when no light disagrees and both armed flags match, and every disagreeing light is charged to one of three causes.

```
src/simulator/validation.py, in latches_identically
    true while   no light disagreed, scrum armed matches, fold armed matches

src/simulator/validation.py, in causes   how many disagreeing lights per cause
src/simulator/validation.py              CAUSES = (TAPE_CAUSE, FIXTURE_CAUSE, CHAIN_CAUSE)
src/simulator/parity_report.py           one report per run, a page and a sidecar
```

No standing defect touches the panel. The eight indicator implementations that departed from their published formulae are all repaired, one indicator per unit, and issue #414 is closed. The API Interaction Log and the Stocks window's New Accumulation Bot box both name the twelve voters the engine builds.

17 - Tiered Strategy Validation and Workflow

Acervator has many unique characteristics and many of these are rooted in attempting to protect an investor from themselves. As such, I have designed a means of self-contained strategy testing that is meant to educate and validate before any attempt at using the platform against actual personal funds is ever attempted. Some may be confident or skilled enough to skip these protective steps. That is an individual user choice. I did not follow this workflow while developing it but I know exactly how my strategy works and when it is not. Given this, the intended workflow for a new user of Acervator should be Simulator > Paper Trader > Live. The system is configured so that the relevant operational elements, such as the Market Inspector, can inject Simulator and Paper equivalents into the appropriate Bot Swarm layers or bot fleet under the respective tab.

Functional. All four stages exist. The Inspector, Sim, Paper and Live tabs are all built and inserted. The Inspector no longer hands its proposals to the Sim tab: the rebuild removed that hook, and Sim and Paper each load a fleet from the stored bot record instead. Live is the tab the part list calls Trading, and it is third in the canonical order.

`src/gui/main_tabs/main_window_surface.py` — `CANONICAL_TAB_ORDER`

```
CANONICAL_TAB_ORDER = (
    SIM_TAB,
    PAPER_TAB,
    LIVE_TAB,
    CHARTS_TAB,
    INSPECTOR_TAB,
    SWARM_TAB,
    ACCUMULATION_TAB,
    HISTORY_TAB,
    STATUS_TAB,
    CONSOLE_TAB,
)
```

The Paper stage is built. Issue #19 added `src/paper/`, the Qt tab and the React panel. Before that no file named for it existed in the tree and none had ever been committed on any branch, which is what the query

below measured. One default still records the older state: the shelved stock window imports a module name that does not exist, so it adds no tab of its own.

`src/gui/main_tabs/stock_main_window_surface.py` — the declared default

```
paper_trader_available: bool = False,
paper_trader_error: Any = "No module named 'src.gui.paper_trader_tab'",
```

Design intention. Paper is the stage that proves a strategy against the real market in real time before real money reaches it, and its defining property is that it runs at the market's own pace. What it shares with Live and what it forks is settled: one trading logic, three data sources. What it does not do is import the live stateful shells. That was a build, and issue #19 landed it after the Simulator rebuild.

Issue #19 covers the build-out. Two surfaces that told the operator a Paper Trader was present were issues #422 and #426, and both folded into #19. Issue #117 tracks the Simulator rebuild that gated it.

Issue #117 closed on 2026-09-19 and issue #19 closed on 2026-09-26, so neither tracks anything now. Both screens are in the tree: `src/gui/paper/paper_trading_tab.py` and `src/gui/paper/paper_react_trading_tab.py` draw Paper, and `src/gui/main_tabs/simulator_tab_surface.py` with `src/gui/react_simulator_tab.py` draw the Simulator.

Today the workflow runs Market Inspector, then Simulator, then Paper, then Live. Every stage has a screen.

Part 3

System Architecture and Features Catalogue

- Main Window
- Trading Tab - Active Bot Lists Under Exchange Tabs - Indicator Voter Panel - Trade Logic Activity - API Activity - Top-Level Portfolio Metrics
- Simulator Tab - Validation Runs vs. Historical Trade Data - Acervator Strategy Back Testing - Runs Historical Data Through Simulated Bots - Bot Swarm Simulation - Deploy As Paper Trading - Deploy As Live Trading
- Paper Trader Tab - Runs Live Exchange Data Through Simulated Bots - Executes and Documents Paper Trades - Collects Paper Trade Data - Pushes Trade Data to Paper Trade History Log - Paper Trade History Tab Reads and Displays
- Proof of Accumulation (PoA) Tab - PoA Blockchain and Trading Tournament Interface - Hosts User Competition Performance and Data - Displays Tournament Monsters and Targets
- Market Inspector Tab - Synthesizes and Proposes Bot Swarm Topologies and Oppositional Trades - Pushes Proposals to Sim, Paper, or Live As Directed
- Bot Swarm Tab - Displays and Allows Configuration of Capital Reinforcement Networks
- Asset Charts Tab - Displays Price Charts for All Active Bots
- History Tab - Loads and Displays Live Trade History Via Exchange API - Merges Live Trade History w/ Acervator Trade Log for Grading and Gate Analysis - Paper History Tab - Loads and Displays Paper Trade History w/ Grading and Gate Analysis
- Console Tab - Displays internal codebase activity, warnings and faults
- System Status Tab - Displays Platform Status by Subsystem - Fed by the Emitter Network
- Settings - System and High-Level User Settings

System Architecture and Features Catalogue

Acervator features a semi-modular / layered design that nests a hyper-vigilant trading engine capable of operating in multiple investment domains simultaneously and all from one terminal. The Main Window contains all of the various subsections found in each subsystem tab. As it stands the existing and planned subsystem tabs are: Simulator, Paper Trading, Trading (Live), Market Inspector, Bot Swarm, Asset Charts, History, Console, Proof of Accumulation (PoA), and System Status. Most of these subsystems interact with each other to some degree with key isolations existing between the three trading tabs and their wiring to Market Inspector, Bot Swarm, and History.

Now: the window builds nine tabs in this order: Sim, Paper, Live, Charts, Inspector, Swarm, History, Status and Console, and it builds no Proof of Accumulation tab (`src/gui/main_tabs/main_window_surface.py, in BAR_TAB_ORDER`).

The mark



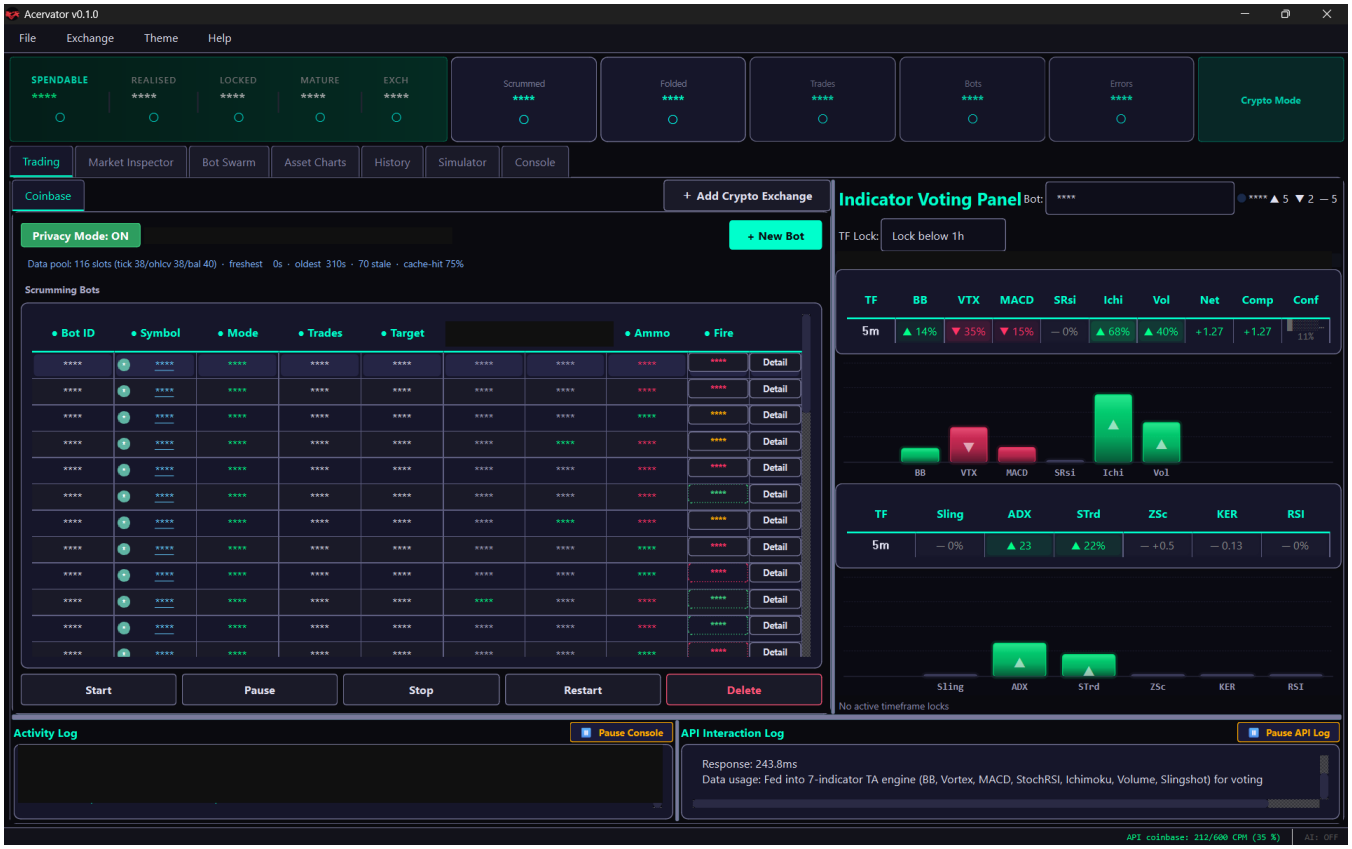
The Acervator mark, the sigil the splash screen paints at launch.

The splash screen paints this mark at launch (`src/gui/main_tabs/splash_screen_painter.py, in compose_still`).

Trading Tab

Here you see the first subsystem we are going to cover and this will primarily be due to familiarizing the prospective investor or user with the core trading philosophy and strategies that drive Acervator. To begin, the platform runs locally on whichever hardware is selected. A single authorization phase requiring a purchased license key will be the only non-exchange communication the application will ever need. The user's keys and secrets are stored locally and encrypted after API handshake verification passes which allows the chosen exchange to be initialized. At a later development stage, I have plans for dedicated hardware that uses a hardware key for quick boot into a given user's account and also allows the platform to run in isolation under Linux.

Now: no license check exists, and beyond the exchanges the app calls news feeds, CoinGecko and Yahoo Finance, plus AI, alert and posting services when you switch them on (`src/gui/crypto_news_ticker.py, in fetch_one`).



The Trading tab, with Privacy Mode on.

1 - Add Exchange - User provides valid API key and secret for target exchange - Platforms validates with an API handshake - Exchange Initializes

Now: most crypto exchanges must pass a live balance check before the key is saved, most broker keys save unchecked, and blank key rows add the exchange with no key. Its tab appears when the Settings window closes (`src/gui/settings_dialog.py, in SettingsDialog._add_exchange`, `src/gui/main_window.py, in MainWindow._sync_exchange_tabs`).

After an exchange is connected to the platform, bots can be added to target entire or portions of existing positions as well as using standing liquidity to enter positions for the first time. The two primary bots used by

Acervator are known as Scrumming and Extractor with the conceptual Modulus Bot to be added later. The first two are the active traders that continuously interact with the market whereas the Modulus Bot will be a macro-strategic interface designed to trigger portfolio-scale shifts in response to custom market signals and it can best be visualized as a modular synthesizer with investment and market-specific functions.

Scrumming Bot

If Acervator has a crown jewel, this is it. The scrumming bot is what houses and executes the harvest-fold method and the harvest-fold method is what leads to the creation of the platform itself. All else stems from this origin point and the simple revelation that allowed the method to be discovered (or probably rediscovered) by nonmathematical eyes. Instead of paying attention to what my entire portfolio was doing, I opted to start focusing on a single position until I had found a strategy that was more reliable than Grid Bots or standard speculation. I was convinced such a method existed and thought it absurd that price charts could not be played better when there was obviously so much room for improvement. You do not have to predict the market. You flow and mold your portfolio to it as time goes. Scrumming allows profits to be shaved off, held and re-investment in a cyclical, reliable manner that sizes its trades in direct proportion to actual market movement as it relates to position value drift. Trading in this manner allows a fixed balance for a position to be maintained and asserts the truth that “Position X is Y value. Any deviation from Y is a Target Delta and is subject to a Scrum (Sell) or Fold (Buy).” The primary weaknesses to this strategy are not have an equal amount of liquidity to the position value (a $500 Scrumming Bot should be supported by $500 of liquidity when initialized) and severe market downturns will little or no short term upside which can lock most liquidity for the position into the Target Asset. Scrumming is a survival strategy and is directionally agnostic but also depends on the market to cycle. I also refer to this simply as Accumulation Trading.

Scrumming Bot Terms

Target Balance - The initial value of the position to be taken or controlled by the Scrumming Bot. The bot will monitor the market for bullish or bearish conditions, check for Target Balance deviations (Target Delta), and re-zero back to the set point. It will repeat this until stopped by the user or some other market condition.

Target Delta - The amount by which a position value has drifted from the Target Balance. The Target Delta is denoted by the Ammo column under the Scrumming Bot list of the Trading Tab. This is the amount of value that will be fired during the appropriate market conditions.

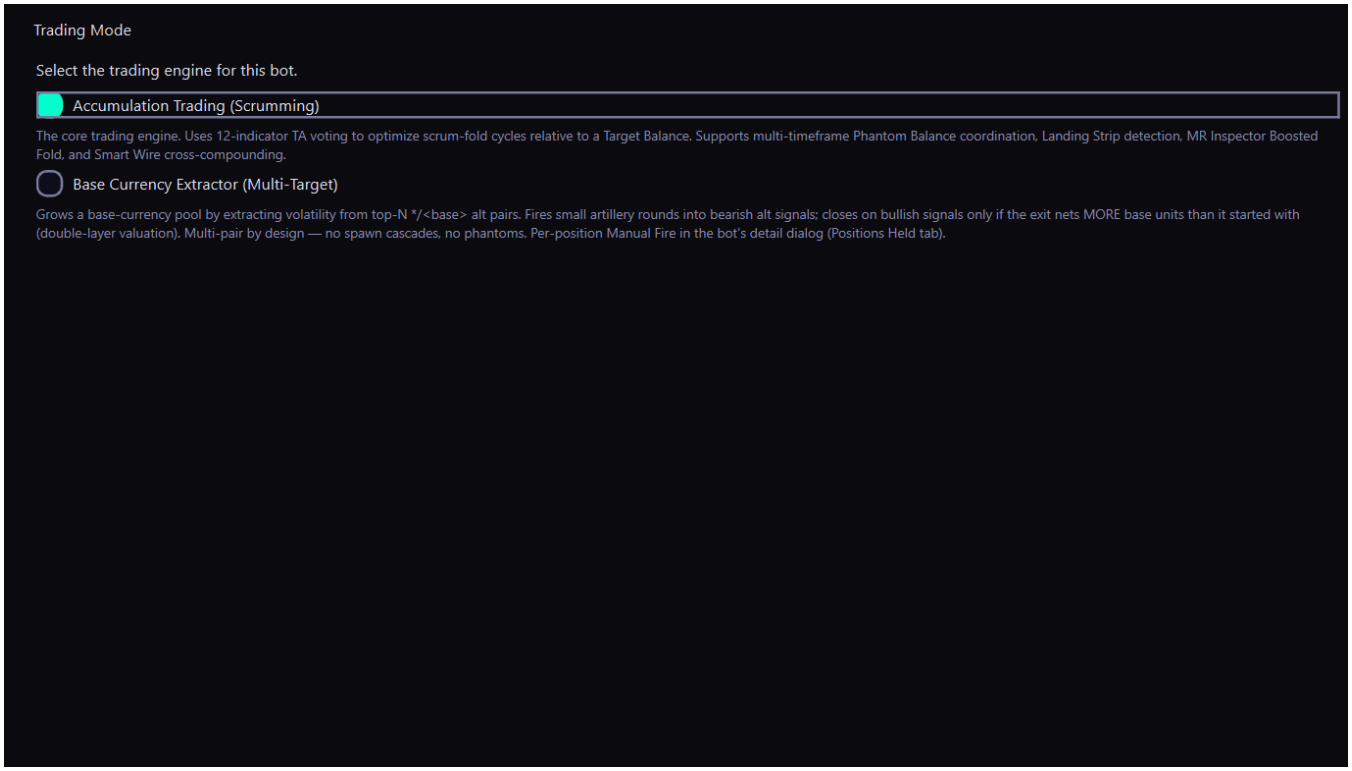
Now: a scrum and a cartridge fire the Target Delta, and an ordinary fold fires its queued tranche dollars instead (`src/trading/scrumming/tick_phases.py, in TickPhaseMixin._tick_execute_fold`).

Scrum - To sell an amount from an investment position that allows it to return to its initial price level. This never closes the position. This is the first half of the infinitely divisible circle that can persist for such positions in a healthy market.

Fold - To buy an amount for an investment position that allows it to return to its initial price level. This also drives Compounding Growth based on Local Volatility and is restricted by the Maximum Growth Per Cycle setting which has been set to a conservative 1% globally for Acervator’s live test and development run.

Now: each fold spends queued tranche dollars whose rebuy price the market has reached, up to the growth room left this cycle, so the Target Delta does not size it (`src/trading/scrumming/tick_phases.py, in TickPhaseMixin._tick_execute_fold`). A buy goes out only when the venue's free balance of its quote currency, read just before the send, covers the buy's price times its amount; otherwise nothing is sent and the log names both figures (`src/exchange/ccxt_connector.py, in CCXTConnector.place_order`).

The fee is not counted, so a buy that fits only without its fee still reaches the venue, which refuses it. Every venue call of a fold's buy, from its open-orders read to its send, waits its turn in the venue's call queue, and a full queue does not refuse it; a balance the venue does not confirm still refuses the buy (`src/trading/scrumming/execution.py, in waives_queue_cap_by_path`).

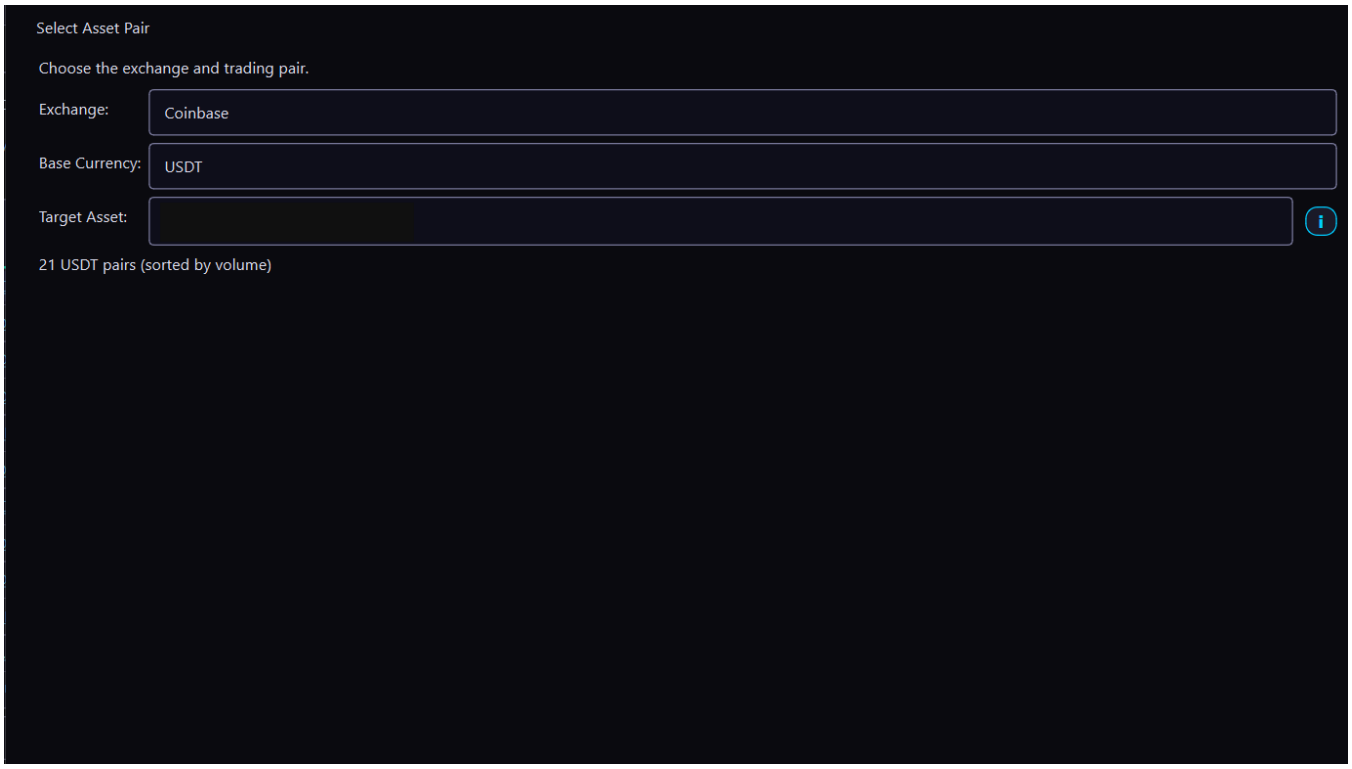


The wizard's first page: Trading Mode, with Scrumming selected.

When the +New Bot button is pressed, the above window appears. Currently there are two “trading mode”

options (this will be change to Strategies). We are only interested in the Scrumming Bot at this point.

Now: the first page offers two options on the Crypto class, and every other class offers only its own Scrumming option (`src/gui/main_tabs/bot_wizard_surface.py, in mode_offer`).



The wizard's asset page: exchange, base currency and target asset.

After Scrumming / Accumulation is selected, next we are presented with Exchange, Base Currency, and Target Asset options.

Exchange - A privately and federally licensed financial platform that allows API interfacing for remote or automated trade execution.

Base Currency - This will be a National Currency, Stable Coin, or High Volume Crypto (A15, A14, BNB) for which multiple trading pairs are available on the selected exchange.

Now: the drop-down lists seven declared currencies first, then every other currency the venue quotes a market in, even with one pair (`src/gui/main_tabs/bot_wizard_surface.py, in base_items_for`).

Target Asset - This is the asset in which the scrumming bot bases its position. If $50 and A05 are selected, it will strive to maintain and compound against a balance of $50 in A05 until it is shut down or some other adverse market condition occurs.

Trading Parameters

Next we come to the combined Scrumming Bot configuration page which has several sections for fine tuning how the specific instance behaves. Given that Acervator, at the time of this writing, is still in active development the description for each setting should be seen as a design intention should any issues be encountered.



The Trading Parameters group of the wizard's parameter page.

Order Visibility - Trades are listed on the order books or tracked internally to the platform.

Now: each order takes its own id, so the venue cannot hand back an order it has already answered. A lost send keeps its id for a retry of the same decision at any size or price, until an order comes back for it or the holdings audit finds its units and books them. A send the venue refuses gives its id back unless an earlier send under that id went unanswered, and the log names each id as it ends. A fold or a tranche fire names the tranches it buys, and an earlier order that comes back pays the queue by units (`src/trading/scrumming/reconciliation.py, in ReconciliationEngineMixin._settle_lost_order`). Another bot's audit leaves the units of an unanswered buy alone (`src/trading/container/aggregation.py, in FleetAggregationMixin.sum_sibling_tracked_units`). A venue that refuses a used id answers with that earlier order, read by client id. Where the read fails, the id stays held and each later try reads and sends nothing (`src/exchange/ccxt_connector.py, in CCXTConnector.place_order`).

Aggressive Trading - Trades are priced so that they fill immediately. Trading like this is a bit like guerilla warfare. In and out before anyone notices.

Now: only Stack tranches in Order Book mode change, each going out as an immediate-or-cancel limit at its ladder price, and every other order ignores this box (`src/trading/scrumming_bot.py, in ScrummingBot._open_stack_from_scrum`).

Stack Mode - Stack Mode enables Stack Tranches which operate on the Sell or Scrum side. This forms the “upside” of the organic ladder structure whereas Fold Tranches form its “downside”.

Split Distance - This setting determines the spacing between tranches if Tranche Spread (not available yet) is being used.

Now: Split Distance sets the percent gap between Stack tranches while Stack Mode is on, and no Tranche Spread row exists (`src/trading/stack_math.py, in ladder_offsets_pct`).

Tranche Spread - This allows a given Scrum or Fold to divide its result X# of times across multiple incremented (as dictated by Split Distance) positions.

Now: no Tranche Spread row exists; Stack Mode splits a scrum across rising prices, and a fold buy goes out as one order (`src/trading/stack_math.py, in split_scrum_into_tranches`).

Tranche Count - This can also be referred to as Spread Count. It determines how pieces a given Scrum or Fold is split into and distributed across incremented tranches as opposed to just one.

Now: Tranche Count is the sell tranches one Stack ladder aims for, 2 to 20, and a fold buys in one order (`src/trading/scrumming_bot.py, in ScrummingBot._spawn_stack_from_fold`).

Spacing - This adds a scaling factor Split Distance and works in conjunction with Tranche Spread and Tranche Count to induce curves and more aggressive growth within the ladder structure.

Now: Spacing multiplies Split Distance tranche by tranche: Linear 1, 2, 3; Quadratic 1, 4, 9; Exponential 1, 2, 4 (`src/trading/stack_math.py, in level_multipliers`).

Personal Hold - Setting to be removed.

Now: no Personal Hold row or config field exists, and the bot drops a saved value when it loads (`src/trading/container/config.py, in _sanitize_deprecated_kwargs`).



The Scrumming Settings group.

Scrolling down we next find the first block of Scrumming Settings. These are the core or basic metrics for a Scrumming Bot.

Opposing Trade Interval - Establishes the minimum travel distance required by price action from the point a given trade in order the next trade of the opposite type to occur.

Now: a scrum needs this share of target as Target Delta, and after a fold this interval plus the fee; a fold rebuy adds Band Travel, and the distance is at least two fees (`src/trading/scrumming/sizing.py, in`

`eligible_fold_tranches`).

BB Tolerance - Determines the minimum distance of the Bollinger Band extent price action must be in order for a trade action to occur.

Now: BB Tolerance sets how close to a band, as a percent of band width, price must sit to count for the Landing Strip, which can pass the TA gate (`src/trading/ta_engine.py, in vote_sides`).

Landing Strip Candles - Determines the strictness of Landing Strip detection. Minimum is three candles. Longer Landing Strips are historically more likely to indicate an impending market reversal than shorter ones assuming the taper remains intact or grows tighter.

Now: the box runs 2 to 10 and opens at 3; the band check reads it, and the shrinking-body check keeps its own fixed 3 (`src/trading/indicators/bb_proximity.py, in detect_bb_proximity`).

TA Timeframe - This is the timeframe at which the bot operates and denotes the price chart it will monitor for trade decisions.

Target Balance - This is the intended starting and locked value for the investment position that the Scrumming Bot is controlling.

Now: Target Balance is the starting value and the anchor; fold surplus grows the live target above it, and Apply Changes can move both (`src/trading/scrumming_bot.py, in ScrummingBot._apply_fold_target_growth`).

Max Entry Price - If the new bot does not detect the requisite amount (as dictated by Target Balance) of the Target Asset, this price threshold sets a limit at which it will attempt to perform the initiating Fold.

Now: every fold, hedge and entry buy refuses above this price; sells, a bot row's Fire and cartridge rebalances never read it, and 0 means no ceiling (`src/trading/scrumming/execution.py, in ExecutionEngineMixin._execute_buy`).

Min Entry (Should Be Exit) Price - If the new bot detects a requisite amount (as dictated by the Target Balance) of the Target Asset, this price threshold sets a limit at which it will attempt to perform the initiating Scrum.

Now: every fold, hedge and entry buy refuses below this price; no sell reads it, and 0 means no floor (`src/trading/scrumming/execution.py, in ExecutionEngineMixin._execute_buy`).

Trading Fee % - This allows the trading fee for the target exchange to be set. This is added to Minimum Opposing Trade Distance to further ensure buys / sells are properly distant and that a given bot is not losing an excessive amount to fee chop in volatile but overly tight market regimes.

Now: the distance adds the larger of this box and the highest fee the venue has charged this bot, a box of 0 reads as 0.6, and it never falls under two fees (`src/trading/otd_math.py, in trading_fee_pct_from_config`).

Max Target Growth % - This determines the maximum amount of growth the Target Balance can increase in a given Market Cycle with a cycle being a Fold / Scrum / Fold sequence. Essentially any Fold preceded by a Scrum will be allowed to Fold an amount of profit back in and, if Surplus remains after the Target Delta is re-zero’d, it can be used to increase Target Balance up to this hard limit for that cycle. This is the organic compounding mechanic.

Now: a cycle ends at a plain scrum fill or at the opposite band extreme; this figure caps target growth and each fold order, and only a fold spends wire credits (`src/trading/scrumming/tick_phases.py, in TickPhaseMixin._book_scrum_fill`).

Scrum Fold Ratio - This precedes Surplus calculation as described under Max Target Growth %. It determines how much of a given trade's profits will be redistributed to directly contribute to its own organic compounding. Note that this does not interfere with normal Target Delta re-zeroing and is intended to only serve as a "cushion" to slow runaway compounding when Wire Credits are being received from multiple sources.

Now: on a plain scrum or cartridge sale this share of proceeds, after wire routing, queues for buy-back and the rest stays cash; a Stack tranche fill queues nothing (`src/trading/scrumming/fold_tranches.py, in FoldTrancheAccountingMixin._apply_scrum_fold_pct`).



The Advanced Scrumming and Hedge Rebalance groups.

Next are the "advanced" Scrumming settings which primarily affect when the bot is allowed to fire a trade. These can be thought of as "calibrating the scope".

Detect Threshold - Intended as the point at which the bot "takes the safety off" and starts looking for a shot. To be re-evaluated.

Fire Threshold - The final Bollinger Band approach metric. Once satisfied, the bot can fire a trade.

Now: Fire Threshold sets how close to a band the ramp reaches FIRE; a scrum past the Detect line fires without it, and folds never read it (`src/trading/gate_chain.py, in RipeHarvestScrumOverride`).

BB Midline Gate - This is another, perhaps redundant layer, of Bollinger Band travel protection. It is different in that it is concerned with distance from the midline instead of the entire local width.

Read Rate - To be re-evaluated.

Band Travel - Previously described. To be re-evaluated.

BB Bullseye Check - If current price and Bollinger Band thresholds are equal, the user can opt to perform a double-sized trade.

Now: past Detect, or on a 0.5% band touch or 0.2% wick, a cleared interval and distance arm the BB priority skew, which lowers the vote floor, and no trade doubles (`src/trading/scrumming_bot.py, in ScrummingBot.tick`).

Wire Inflow Stack - To be re-evaluated.

Now: no Wire Inflow Stack row or config field exists; incoming wire dollars spread evenly over open fold tranches, or park until a sale opens one (`src/trading/scrumming/wire_routing.py, in WireRoutingMixin.apply_wire_income`).

Hedge Rebalance Active - Determines if Current Price drifting below Initial Entry Price will have a limited amount of funds that can be used to keep re-zeroing the Target Delta at key bearish thresholds or areas of possible reversal.

Now: while ticked, the reserve reads no entry price and buys from 1% under target, past the interval or with a tranche queued, with band position under 0.40 and no bullish vote, and a bot that has made no first buy waits for the band and its own vote first (`src/trading/scrumming_bot.py, in ScrummingBot.tick`).

Hedge Balance - Sets a limit on the amount of additional liquidity a given bot is allowed to absorb when Current Price drifts below Initial Entry Price.

Now: Hedge Balance caps a dollar reserve separate from Target Balance; each hedge buy spends up to half of it, and 8% of compound growth refills it (`src/trading/scrumming_bot.py, in ScrummingBot.tick`). A hedge or an entry holds its dollars on the in-flight ledger until its send ends, so a second buyer cannot pass on them (`src/trading/scrumming/execution.py, in ExecutionEngineMixin._execute_buy`). Every venue call of a hedge buy, from its open-orders read to its send, waits its turn in the venue's call queue, and a full queue does not refuse it (`src/trading/scrumming/execution.py, in waives_queue_cap_by_path`).



The Circuit Breakers group.

Now we arrive at some safety controls. Circuit Breakers are designed to fully inhibit trade actions for a given period should an extreme volatility (pump and dump) event occur. Soft Circuit Breakers have a candle-count based timer whereas Hard Circuit Breakers require the user to clear the bot to continue trading.

Now: an up candle stops scrums and a down candle stops folds for the cooldown, and a hard breaker stops every order until you press Reset All Breakers (`src/trading/bot_container.py, in BotContainer.guarded_place_order`).

Soft CB Threshold - The amount of instantaneous, single-candle price action required for the bot to pause trading for a number of candles denoted by Soft CB Cooldown.

Now: each breaker reads a candle once, when the shared candles first show it, and takes the high-to-low range it has reached by then against its open; a move later in that candle is not read, and the soft breaker shuts only that candle's side (`src/trading/scrumming/circuit_breakers.py, in CircuitBreakerMixin._check_circuit_breakers`).

Hard CB Threshold - The amount of instantaneous, single-candle price action required for the bot to be hard stopped at which point the user must re-authorize trading.

Now: a trip refuses every order until you press Reset All Breakers: the bot reads HELD and sends one alert, and an order already at the venue can still fill and books. Fire and Self-Destruct say they were refused, keep nothing, and must be pressed again after the reset, and a trip that lands while one of them waits refuses it even when the reset comes first. A cartridge, a detonation, an expiry close and a first entry decide again on the first tick after it (`src/trading/bot_container.py, in BotContainer.order_refusal`).

Soft CB Cooldown - This is the number of candles that must close before the Soft Circuit Breaker opens again.

Now: the count falls by one each time the shared candles show a new newest candle, which means the one before it has closed, and the side opens at zero; no count falls while the hard breaker holds (`src/trading/scrumming/circuit_breakers.py, in CircuitBreakerMixin._read_breaker_candle`).

Max Cartridge Size - This is the maximum amount of deviation allowed for the Target Delta. At this threshold the bot is actively and aggressively looking for a trade opportunity.

Now: at this share of Target Balance the bot fires a market rebalance past the band and vote gates, unless the opposing distance holds it or no queued tranche qualifies, and it spends only dollars no other bot's queued buy-backs or wire credits hold (`src/trading/scrumming/execution.py, in ExecutionEngineMixin._execute_manual_rebalance`); a bot that has made no first buy waits for the band and its own vote first (`src/trading/scrumming/tick_phases.py, in TickPhaseMixin._first_buy_clears`). A cartridge with no dollars it may spend sends nothing and writes one refusal line, and the tick goes on to the fold gates, so a fold still spends its own tranche (`src/trading/scrumming_bot.py, in ScrummingBot.tick`). The unfilled dollars of another bot's resting fold left the free balance, so they count once (`src/trading/container/aggregation.py, in FleetAggregationMixin._one_sibling_hold`). A fold's balance check and every venue call of its rebalance wait their turn in the venue's call queue, and a full queue does not refuse them; a balance the venue does not confirm still refuses the fold (`src/trading/scrumming/tick_phases.py, in TickPhaseMixin._tick_cartridge_fire`).

Smart Cartridge - This allows the Max Cartridge Size to organically resize in response to current price range as defined by the current-candle Bollinger Band reading.

Smart Ceiling - To be re-evaluated.



The Risk Controls and Strategy Gate Flags groups.

After Circuit Breakers, which help defend against extreme volatility, we come to Risk Controls. These are designed to cap the amount of profit or growth a given bot can earn before performing a full position exit.

Now: Risk Controls cap accumulation: folds taper toward the ceiling and stop, and a detonation sells what sits above the anchor while the bot keeps trading (`src/trading/scrumming/execution.py, in ExecutionEngineMixin._execute_detonation`).

Enable Position Ceiling - Enables a growth cap for a given position.

Ceiling Multiple - This setting caps the maximum amount of growth a position at a multiple of the Target Balance (anchor) and, once reached (and under higher timeframe bullish conditions with Detonation enabled) will allow the entire position to be sold and the corresponding bot will pause all further operations. Without Detonation enabled, this becomes a user notification.

Now: folds taper from half the ceiling and stop at it, which the log records, and a detonation ignores the ceiling while the bot runs on (`src/trading/scrumming_bot.py, in ScrummingBot._check_detonation_trigger`).

Enable Detonation - Enables an entire remaining position to be sold after the Ceiling Multiple growth threshold is crossed.

Now: a detonation sells only the value above the anchor, on a new bullish higher-timeframe turn, whether or not any ceiling is set or crossed (`src/trading/scrumming/execution.py, in ExecutionEngineMixin._execute_detonation`).

Detonation TF - Selects the timeframe for the chart that is being evaluated for bullish conditions that will allow the detonation to occur.

Min Confidence - This is the minimum technical analysis confidence index (via the Indicator Voting Panel) that will allow the detonation to occur.

Now: a detonation also needs three agreeing indicators, so a vote that only two indicators could reach waits at any bar, `src/trading/scrumming_bot.py, in ScrummingBot._check_detonation_trigger`.

Next we use the Strategy Gate Flags which allows top-level trade restrictions to be enabled or disabled thus relaxing or restricting the conditions under which a trade action can occur. The settings, in this case, are self-descriptive.

Now: on a running Scrumming bot this group also holds a Hunger tick box and a Hunger Pace box, which ease the scrum and fold vote floor over idle candles (`src/gui/live_settings/settings_tab.py, in SettingsTabMixin._add_hunger_rows`), and an Accumulation Shadow tick box, which sends a bearish fold's extra units to a Shadow Tranche (`src/gui/live_settings/settings_tab.py, in SettingsTabMixin._add_shadow_row`).



The Profit Routing group.

Moving onto the final section, we have Profit Routing which was intended to allow profits to be routed differently during initial set-up. This will be re-evaluated and potentially removed.

Now: the wizard carries no Profit Routing group, and you route profit with Smart Wires on the Swarm tab, by dragging between bot tiles or through Quick Routing (`src/gui/bot_wizard.py, in TradingParamsPage.set_mode`).

Route - Destination for profits.

Now: no Route row exists, and each Smart Wire names its target bot and the percent it carries (`src/trading/smart_wire.py, in SmartWireManager.register_wire`).

Target bot ID - Field for manually a bot ID which was intended to create a Smart Wire under the Bot Swarm tab.

Now: no Target bot ID row exists, and dragging from one bot tile to another opens the box that sets the wire's percent (`src/gui/bot_visualizer.py, in BotVisualizationTab._show_wire_config`).



The Phantom Bots page.

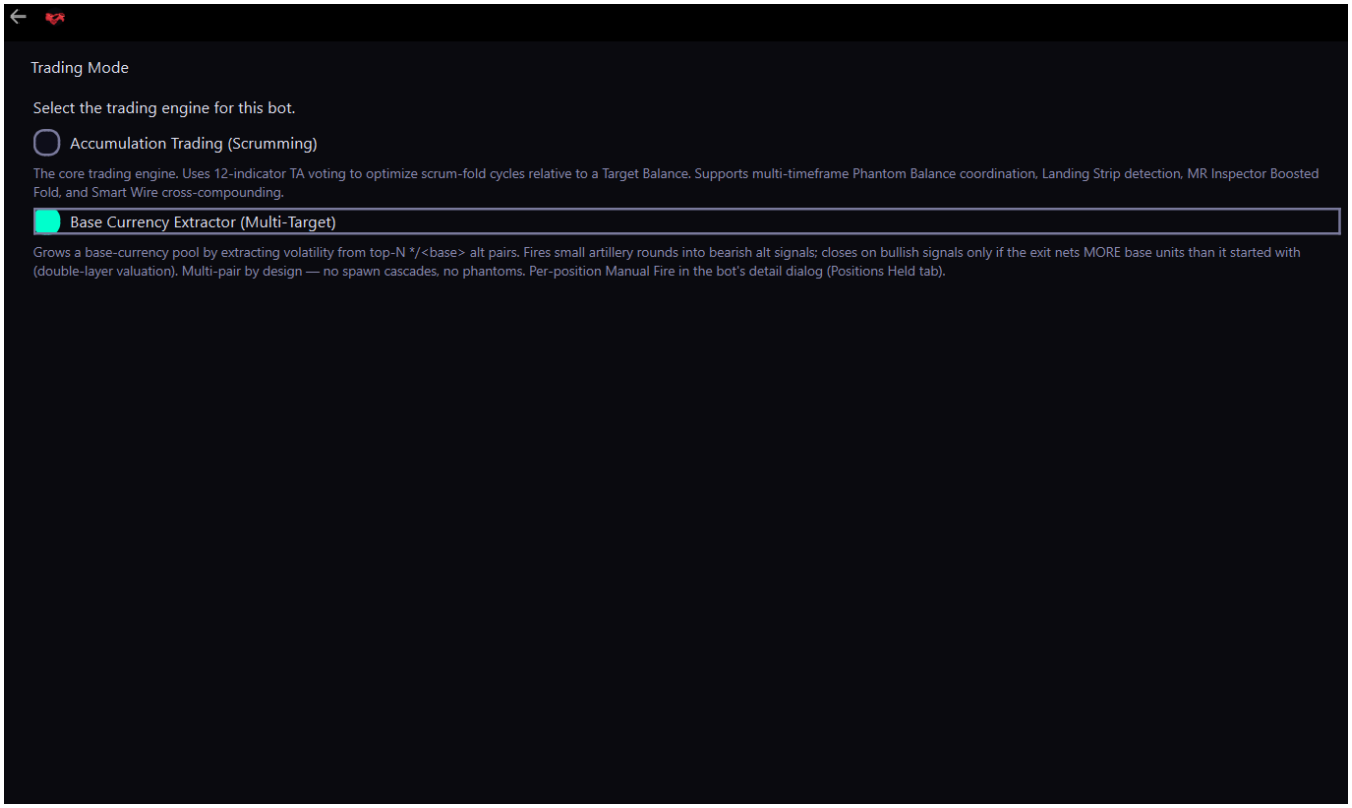
Lastly we have the selection for the Phantom (Balance) Bots. These are intended to provide trade action overrides from higher timeframe charts and indicator sets which, in turn, may result in an improved trade or prevent a premature one.

Now: a phantom is a read-only observer on one higher timeframe that feeds the Composite Net and the defer gates, so it can refuse a trade and never places one (`src/trading/scrumming_bot.py, in ScrummingBot.tick`).

Extractor Bot (Partially Built; Untested)

The second type of bot offered within Acervator is the Extractor. These operate quite differently from Scrumming Bots and actually operate as Siblings of them. In fact, an Extractor Bot cannot even be called unless a corresponding Base Currency Scrumming Bot (i.e. A15:USD or A14:USD) is already active. This is due to the core operating principle of the Extractor bot to acquire more of these base currencies by performing trades against available alternate currency pairings. It does this by using and blocking off a portion of the Parent's position within an Extractor Tranche that represents an active position taken against one of the available alternate pairs. The Extractor Tranche remains open until its opposing accumulating (or Short Position if preferred by the user) or profit taking trade is filled. Extractor Tranches can be of any size but should generally be a relatively small fraction of the Parent's total position which will allow the Extractor to take multiple positions if available and allowed by the specific user.

Now: creation stops unless exactly one Scrumming Bot on that exchange holds the base currency; the bot then claims its chunk, buys fixed Artillery rounds and never shorts (`src/gui/main_window.py, in MainWindow._extractor_parent_refusal`).



The Trading Mode page with the Extractor selected.



The Extractor Pool page.



The Extractor group of the parameter page, first nine rows.

Chunk size (USD) - This determines the maximum amount of the parent's pool that the Extractor can use.

Artillery size (USD) - This determines the individual size of Extractor Tranches.

Watch list top-N - The determines the number of Alternate Currency pairs the bot will scan for potential extraction.

Watch list refresh - This determines the rate at which the Extractor will scan its watched markets. This is the equivalent of a Timeframe for the Extractor but covers multiple pairs.

Now: this row sets how many candles pass before the bot ranks its list again, and the bot reads the listed pairs about every five seconds (`src/trading/extractor_bot.py, in ExtractorBot._refresh_interval_seconds`).

Pool Reserve - To be re-evaluated.

Now: no Pool Reserve row exists; a round fires while the free chunk, which exits refill with their gains, still holds its size (`src/trading/extractor_bot.py, in ExtractorBot._has_chunk_capacity`).

Exit % - To be re-evaluated.

Max compounding tier - Allows the Extractor to attempt a number of compounding Swing Trades with a given Extractor Tranche with subsequent re-entries based upon the Parent Scrumming Bot's Minimum Opposing Trade Distance + Trade Fee + Bollinger Band extension settings.

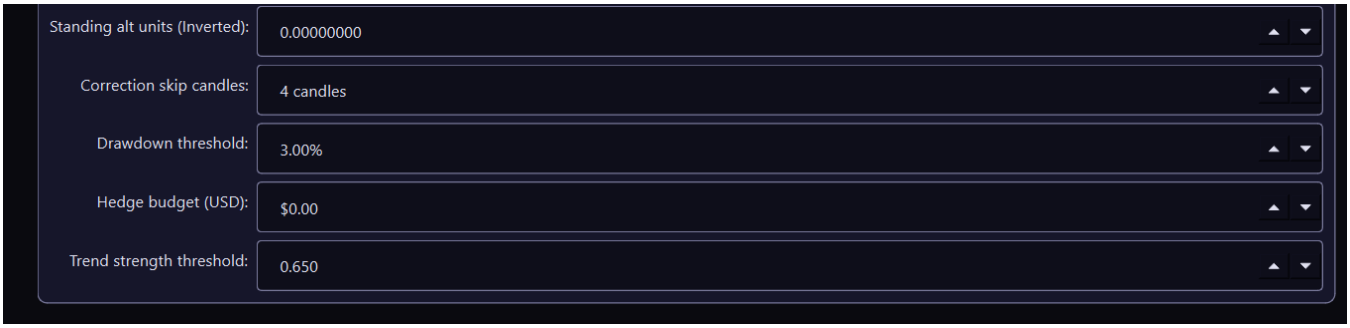
Now: no Max compounding tier row exists; each exit returns its base currency to the pool, and the pair enters again only on a later bearish signal (`src/trading/extractor_bot.py, in ExtractorBot._book_exit`).

Max cost-basis multiple - To be re-evaluated.

Now: no Max cost-basis multiple row exists; the bot holds one position per pair and adds no round while that pair stays open (`src/trading/extractor_bot.py, in ExtractorBot.tick`).

Direction - To be re-evaluated.

Now: no Direction row exists; the Extractor buys an alternate pair on a bearish signal and sells on a bullish one only when the sale nets more base after fees (`src/trading/extractor_bot.py, in ExtractorBot._evaluate_open_position`).



The Extractor group, remaining five rows.

Standing alt units (inverted) - To be re-evaluated.

Now: no Standing alt units row exists; every position starts from a bought round, and the bot keeps no standing alternate holding (`src/trading/extractor_bot.py, in ExtractorBot._book_artillery`).

Correction skip candles - To be re-evaluated.

Now: no Correction skip candles row exists; the bot adds no correction buys to an open position and only waits for its exit (`src/trading/extractor_bot.py, in ExtractorBot._evaluate_open_position`).

Drawdown threshold - To be re-evaluated.

Now: no Drawdown threshold row exists; a position reads drawdown whenever its dollar value sits below its entry value, by any amount (`src/trading/extractor_bot.py, in ExtractorBot._is_in_drawdown`).

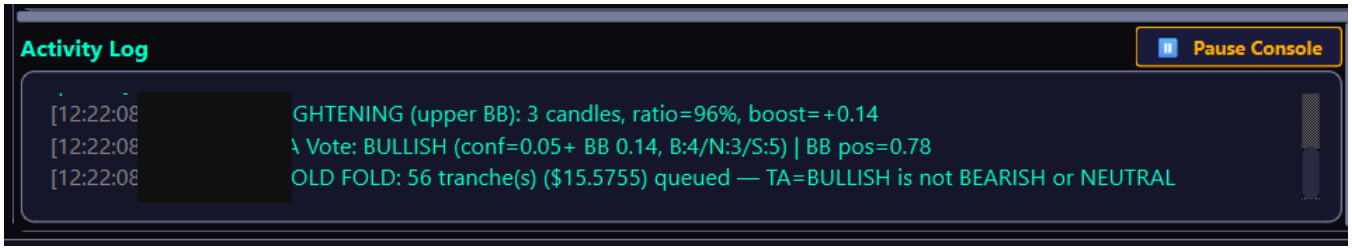
Trend Strength Threshold - To be re-evaluated.

Now: no Trend Strength Threshold row exists; the Extractor reads only the bullish or bearish consensus and ignores trend strength (`src/trading/extractor_bot.py, in ExtractorBot._entry_signal_matched`).

Additional Main Window > Trading Tab Features

Trade Logic and Gate Activity

This spool displays the trading logic and gate activity.



The Activity Log pane.

API Interaction Log

This spool displays API handshake information and data transfer speeds for these messages.

Now: this spool lists each API entry with its reason, endpoint, result, the venue's answer time in milliseconds, any wait for a worker and a data usage note (`src/gui/main_tabs/main_window_surface.py, in api_event_block`).



The API Interaction Log pane.

Add Crypto Exchange Button (to be changed - Add Exchange)

Now: the button reads + Add, the active class name, then Exchange or Broker, and the Exchange menu item reads Add Exchange (`src/gui/main_tabs/asset_class_surface.py, in add_exchange_label`).

While the initial set up of Acervator requires at least one valid exchange API, additional exchanges can be added and have their own bot swarms. The current upper operational limit of Acervator is unknown. Multi-exchange testing has yet to be attempted as of 8/25/26 with API compatibility work pending. This button also currently opens the Settings Panel but on the incorrect ‘User’ Tab when it should be ‘Exchanges’.

Now: the button, the Exchange menu and Start on a broker, or on Robinhood on the Crypto class, with no stored key all open Settings on the Exchanges tab, and a first launch opens with no exchange (`src/gui/main_window.py, in MainWindow._run_settings_window`).



The Add Crypto Exchange button.

How to use the Trading tab

This part is for lookup. Each row names one control, says what it does and what it reads, and cites the code behind it.

Two builds of this tab

A frozen build draws the variant stamped into it. A source run reads `ACERVATOR_VARIANT` and draws React when nothing names a variant (`src/_variant.py, in resolve_variant`).

Every press on this page runs the same handler on both builds. The React page sends each press to `src/gui/react_trading_tab.py, in TradingTabReact.run_press`, which calls the handler the Qt button holds.

The menu row and the status bar

Control	What it does and what it reads
File > Settings	Opens Settings on its User tab. `src/gui/main_window.py, in MainWindow._open_settings`
File > Reset All Settings	Asks Yes or No, then clears every setting, exchange and stored key. `src/gui/main_window.py, in MainWindow._reset_settings`
File > Exit	Closes the window. `src/gui/main_window.py, in MainWindow._setup_menu`
Exchange > Add Exchange	Opens Settings on the Exchanges tab, in both builds. A class with no venue logs a note instead. `src/gui/main_window.py, in MainWindow._add_exchange`
Theme	Repaints the window in one of five themes and remembers it. Reads the `theme` and `accent_color` settings. `src/gui/main_window.py, in MainWindow._switch_theme`
Help > About	Shows the version and a feature list. `src/gui/main_tabs/main_window_surface.py, in about_text`
API load pill	Names the busiest exchange, its calls per minute, its busy share and any refused calls. Amber above 50% and red above 75% of the worse of busy share and 600 calls a minute. Reads the API log for the last 60 seconds. `src/exchange/api_load_monitor.py, in APILoadMonitor.sample`
AI state label	Shows AI: OFF, READY or NO PACKAGE, then AUTH or UNAUTH once feedback arrives. Reads the `ai_monitor` settings and `ai.feedback` events. `src/gui/main_window.py, in MainWindow._on_ai_feedback`

The header strip

Every figure repaints each 2 seconds (`src/gui/main_window.py, in MainWindow._refresh_header_strip`). With Sim or Paper in front, the strip reads that tab's own fleet. Otherwise figures cover the whole fleet, except AMMO, which reads the active class, and EXCH, which reads the active layer.

Cell	What it shows and what it reads
SPENDABLE	Exchange wallet cash, the highest reading among the bots. A dash until cash or positions read above zero. `src/trading/container/aggregation.py, in FleetAggregationMixin._add_bot_stats`
P/L	Unrealised profit and loss, signed, as the exchange reports it. `src/gui/main_tabs/header_strip_surface.py, in unrealised_amount`
LOCKED	Dollar value of all positions: holdings times price times the quote's dollar rate, summed. A dash until cash or positions read above zero. `src/trading/container/aggregation.py, in FleetAggregationMixin._bot_position_usd`
MATURE	Position value above three times cost, summed, once an exchange answers. `src/trading/container/aggregation.py, in FleetAggregationMixin._add_bot_stats`
EXCH	Exchange sub-tabs on the active layer. `src/gui/main_window.py, in MainWindow._refresh_header_strip`
AMMO	Whole-dollar distance from target. Green when more bots sit above target, red when more sit below. `src/gui/main_tabs/header_strip_surface.py, in sector_ammo`

Cell	What it shows and what it reads
SCRUMMED, FOLDED	Dollars sold and bought this year, read from the exchange's trade history. A zero year shows the app's own running total. `src/trading/container/aggregation.py, in FleetAggregationMixin.get_aggregate_stats`
ACCUMULATED	Each bot's live target less its anchor, summed. `src/trading/container/aggregation.py, in FleetAggregationMixin.get_aggregate_stats`
TRADES	Orders filled this year, one per order, summed. `src/trading/scrumming/reconciliation.py, in ReconciliationEngineMixin.sync_ytd_trade_count`
BOTS	Running bots. `src/trading/container/aggregation.py, in FleetAggregationMixin._read_one_bot`
ERRORS	Bot errors since the last reset. A press opens the Error Log, with Recent events, Per-bot snapshot and Reset all errors. `src/gui/main_window.py, in MainWindow._build_error_log_dialog`
Round cell dots	Hide that cell behind asterisks. Filled shows and hollow hides. `src/gui/main_tabs/header_strip.py, in HeaderStripMixin._build_header_strip`

The asset class group

One square holds six segments: Crypto, Stock, Commodities, Forex, Indices and Futures / Perps. A press retitles the window, moves the Live layer and filters the tabs again (`src/gui/main_tabs/header_strip.py, in HeaderStripMixin.select_asset_class`). A class with no venue draws Get Started and a greyed button that says so (`src/gui/main_tabs/asset_class_surface.py, in class_state`).

The tab row

Tab	What it does	Follows the class	Page
Sim	Replays bots on stored history.	Yes	simulator.md
Paper	Runs bots on live prices with a pretend balance.	Yes	paper-trader.md
Live	Trades on real venues with real orders.	Yes	this page
Charts	Draws one chart per traded symbol.	Yes	asset-charts.md
Inspector	Scans markets and proposes topologies.	Yes	market-inspector.md
Swarm	Draws bots as nodes, with wires.	Yes	bot-swarm.md
History	Shows venue trades, paired and graded.	Yes	history.md
Status	Shows subsystem health.	No	system-status.md
Console	Shows every log line and signal check.	No	console.md

The class-following tabs are declared in `src/gui/main_tabs/class_filter_tab.py, in CLASS_TAB_ATTRS`, and each tab's mouse-over in `src/gui/main_tabs/main_window_surface.py, in TAB_TOOLTIPS`.

One sub-tab per exchange

Control	What it does and what it reads
Get Started tab	Shows until the class has an exchange, with a heading, an Add Exchange button and a hint. `src/gui/main_tabs/trading_tab.py, in TradingTabMixin._build_trading_tab`
Add Exchange corner button	Opens Settings on the Exchanges tab, and greys when the class has no venue. `src/gui/main_window.py, in MainWindow._add_exchange`
Exchange sub-tabs	One page per stored exchange, seated on every class it serves. Reads the stored exchange list, and the tabs change when a Settings window closes. A press changes the page. `src/gui/main_tabs/trading_tab.py, in TradingTabMixin._show_exchange_tab`, `src/gui/main_window.py, in MainWindow._sync_exchange_tabs`
Privacy Mode	Masks all registered fields, or shows them again when all are masked. It reads ON only when all are masked. `src/gui/main_tabs/exchange_tab_surface.py, in flip_privacy_masks`
+ New Bot	Opens the Create Auto Trader wizard on this class. It refuses while the fleet is sealed or no stored venue serves the class. `src/gui/main_window.py, in MainWindow._create_bot`
Data pool line	Shows cache slots, freshest and oldest ages, stale count and cache-hit share, each second. `src/gui/main_tabs/exchange_tab_surface.py, in pull_rate_text`
DATA POOL STOPPED banner	A red line. It shows when this venue made no calls and the newest reading is over 60 seconds old. `src/exchange/fetch_stall.py, in stall_seconds`
News line	One headline each 15 seconds from ten RSS feeds, fetched hourly. Hover pauses it, and a press opens the story. `src/gui/crypto_news_ticker.py, in open_story`

The bot tables

On the Scrumming table, a press on a row draws that bot on the Indicator Voting Panel, and a second press on the same row clears the panel (`src/gui/main_tabs/bot_status_table_surface.py, in selection_after_press`). A press on a column label sorts that column, and blanks sink (`src/gui/main_tabs/bot_status_table_surface.py, in order_statuses`). A press on an Extractor row only selects it for the command bar (`src/gui/react_exchange_tab.py, in ExchangeTabReact.select_extractor`). A held bot's Symbol cell turns red, and it and the Fire button name the hold in their mouse-over (`src/gui/main_tabs/bot_status_table_surface.py, in held_tooltip`).

Scrumming Bots column	What it shows and what it reads
Asset	The asset's kept logo, else its name. A press opens the asset's organisation site over https, and an asset with no address opens nothing. `src/gui/main_tabs/bot_status_table_surface.py, in organisation_address`
Symbol	The pair, coloured by bot state. A press opens the pair's chart on the exchange's site. `src/gui/main_tabs/bot_status_table_surface.py, in BotStatusTableModel._symbol_cell`
Current Position Value	Held units at a fresh exchange price. Blank when the price is missing or over 20 seconds old. `src/gui/main_tabs/table_cells_surface.py, in TableCellsModel.position_value_cell`
Trades	The bot's trade count. `src/gui/main_tabs/bot_status_table_surface.py, in BotStatusTableModel._write_row`

Scrumming Bots column	What it shows and what it reads
Target	The balance the bot returns to: the grown target, else the Target Balance. `src/gui/main_tabs/bot_status_table_surface.py, in target_value`
Target A15, Target A14	The target restated in that asset, with its 24-hour drift. Blank when the asset is that market, a dash when the pair is unlisted, and pending until the rate arrives. `src/gui/main_tabs/table_cells_surface.py, in TableCellsModel.target_denom_cell`
Ammo	Distance from target, unsigned. Green above target, red below, grey when stale. It reads the exchange's units where it has them, else the bot's. `src/gui/main_tabs/table_cells_surface.py, in TableCellsModel.ammo_cell`
Fire	One press on one bot. The next section covers it. While the hard breaker holds, a press is refused and keeps nothing, so press it again after Reset All Breakers. On a bot no tick runs, it is refused the same way, so press it again after Start. `src/trading/bot_container.py, in BotManager.force_fire`
Detail button	Opens the bot settings window and selects the row. `src/gui/main_window.py, in MainWindow._on_bot_clicked`
Round header dots	Hides that column behind asterisks. Filled shows and hollow hides. Target's three columns share one field, as do Position Value and Ammo. `src/gui/main_tabs/bot_status_table_surface.py, in BotStatusTableModel.on_privacy_toggled`

Extractor Bots column	What it shows and what it reads
Bot ID	The bot's id. `src/gui/main_tabs/extractor_bot_table_surface.py, in row_values`
Symbol	The pool's base currency, with a coin mark. `src/gui/main_tabs/extractor_bot_table_surface.py, in row_values`
Mode	The word extractor, coloured by state. `src/gui/main_tabs/extractor_bot_table_surface.py, in row_values`
Trades	The bot's trade count. `src/gui/main_tabs/extractor_bot_table_surface.py, in row_values`
Pool	The pool budget in dollars, from the configured chunk size. `src/gui/main_tabs/extractor_bot_table_surface.py, in row_values`
Liquid	The pool's share not in a position, in green, yellow or red. The mouse-over counts positions. `src/gui/main_tabs/extractor_bot_table_surface.py, in deployment_split`
Fire	Always off. Manual Fire sits per position in the Detail window. `src/gui/main_tabs/extractor_bot_table_surface.py, in buttons`
Detail button	Opens the bot settings window. `src/gui/main_window.py, in MainWindow._on_bot_clicked`

The Fire button

A press queues a market order to target on the next tick, past the gates, and it refuses inside 1% of target (`src/trading/scrumming/execution.py, in ExecutionEngineMixin._execute_manual_rebalance`). A press that buys also sends nothing while the venue lists an open buy on the market, or while that list cannot be read (`src/trading/scrumming/execution.py, in ExecutionEngineMixin._refuses_stacked_buy`).

While the hard breaker holds a bot, a press queues nothing and the status log names the breaker, and the button keeps its colour (`src/trading/bot_container.py, in BotManager.force_fire`).

Now: while the hold stands, the button reads Held, dotted, and its mouse-over names the hold (`src/gui/main_tabs/bot_status_table_surface.py, in refused_fire`).

Now: a press that buys spends only dollars no other bot's queued buy-backs or wire credits hold, and the log names those dollars when none are left (`src/trading/scrumming/execution.py, in ExecutionEngineMixin._held_for_other_bots`). A press on a tranche whose fold still waits on the venue's answer sends nothing and reads refused (`src/trading/bot_container.py, in BotContainer.guarded_place_order`).

Fire and Ammo use their colours differently. Ammo is green above target, and Fire is red to sell above target.

What the button shows	What it means
Solid red, glowing	Above target, every gate clear. A press sells to target. `src/gui/main_tabs/bot_status_table_surface.py, in BotStatusTableModel._armed_fire`
Dashed red	Above target, and a gate holds the sale. The mouse-over names it. A press sells anyway. `src/gui/main_tabs/bot_status_table_surface.py, in BotStatusTableModel._armed_fire`
Solid green, glowing	Below target, every gate clear. A press buys to target. `src/gui/main_tabs/bot_status_table_surface.py, in BotStatusTableModel._armed_fire`
Dashed green	Below target, and a gate holds the buy. A press buys anyway. `src/gui/main_tabs/bot_status_table_surface.py, in BotStatusTableModel._armed_fire`
Muted, dashed, glowing	The position ceiling stops automatic buys. A press buys past it. `src/gui/main_tabs/bot_status_table_surface.py, in BotStatusTableModel._armed_fire`
Amber fill, amber text or red text	Parked at target: at the band, tracking or searching. A press does nothing. `src/gui/main_tabs/bot_status_table_surface.py, in BotStatusTableModel._armed_fire`
No Trade, dotted	A press will not act. The distance sits inside 1% of target, or the exchange disputes the bot's unit count. `src/gui/main_tabs/bot_status_table_surface.py, in manual_fire_verdict`
Held, dotted	A press will not act. The hard breaker holds every order, and the mouse-over names the hold. `src/gui/main_tabs/bot_status_table_surface.py, in manual_fire_verdict`
Grey, off	The bot is neither running nor paused. `src/gui/main_tabs/bot_status_table_surface.py, in BotStatusTableModel._fire_button`

The command bar

Each command acts on the selected row. With no row selected it logs "Select a bot first." Hold SHIFT for the all-bots form.

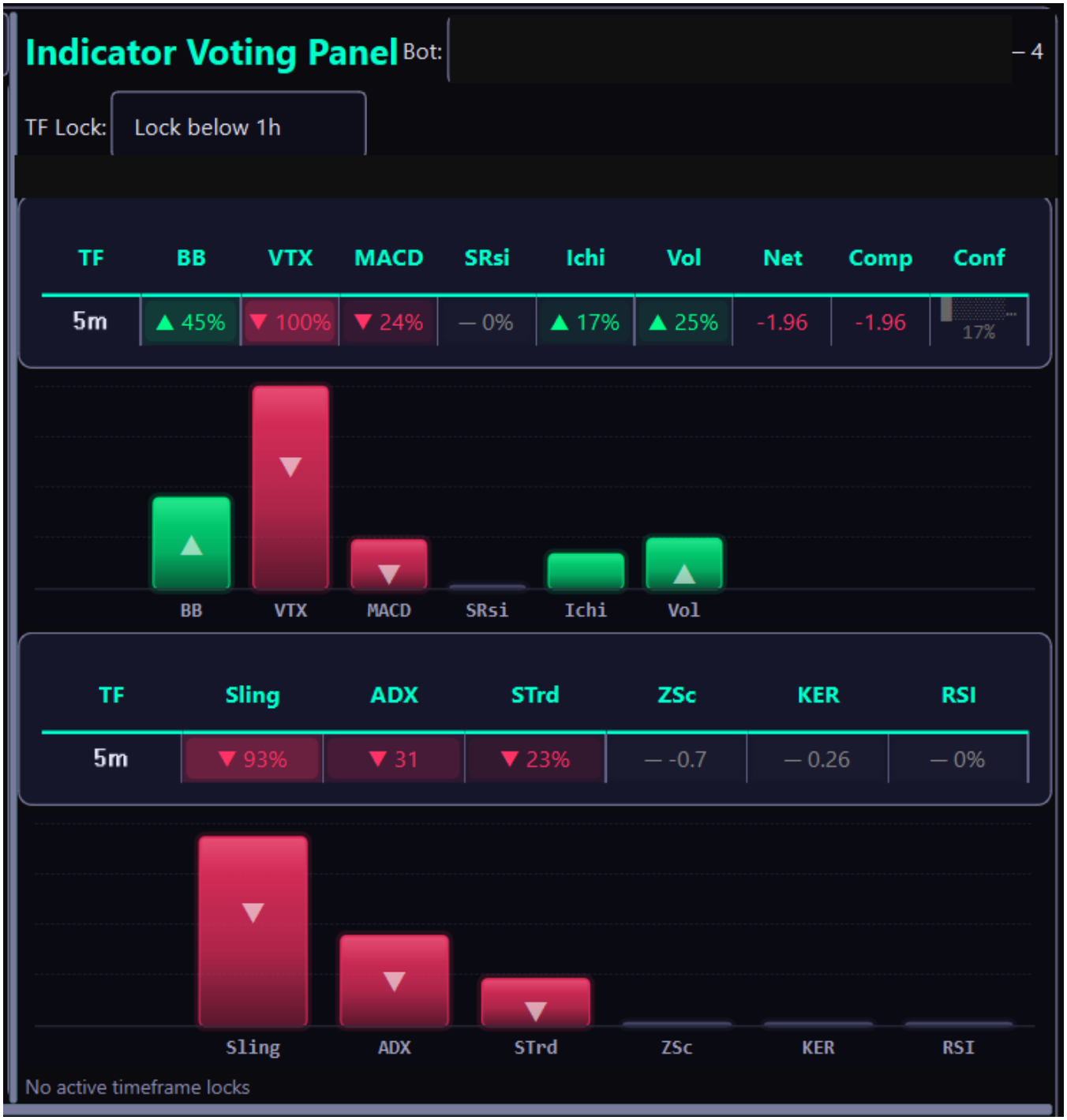
Button	What it does and what it reads
Start	Connects to the exchange, then starts the bot. `src/gui/main_window.py, in MainWindow._on_bot_command`. A bot keeps one tick loop in any state: Start resumes the loop a paused or errored bot already has. It does not end a hard hold: the log names the breaker and writes no RUNNING line. `src/trading/bot_container.py, in BotContainer.start`

Button	What it does and what it reads
Pause, Stop	Pauses or stops the bot. `src/gui/main_window.py, in MainWindow._on_bot_command`. A paused or stopped bot reads nothing and sends nothing on its own, and a fill books at the first worked tick after Start. `src/trading/scrumming_bot.py, in ScrummingBot.tick_stop_reason`
Restart	Stops the bot, connects again and starts it. `src/gui/main_window.py, in MainWindow._on_bot_command`
Delete	Asks Yes or No first. Yes stops the bot and removes it. SHIFT changes nothing. `src/gui/main_window.py, in MainWindow._on_bot_command`
SHIFT: Start All	Starts every idle or stopped bot, one at a time, with a progress window. A held bot is passed at once and listed with its hold. `src/gui/main_window.py, in MainWindow.run_fleet_sequence`
SHIFT: Restart All	Restarts every bot the manager holds, one at a time. `src/gui/main_window.py, in MainWindow.run_fleet_sequence`
SHIFT: Pause All, Stop All	Pause All takes every running bot and Stop All takes every bot, on every exchange and class. `src/gui/main_window.py, in MainWindow._global_bot_cmd`

The two log panes

Control	What it does and what it reads
Activity Log	Timestamped bot lines, gate lights and system messages. It keeps 5,000 lines. `src/gui/widgets/status_log.py, in StatusLog`
Pause Console	Holds new lines until Resume Console. The buffer holds 2,000, and a full buffer drops the newest. `src/gui/widgets/status_log.py, in StatusLog.log`
API Interaction Log	One block per API entry. It keeps 2,000 lines. `src/gui/main_tabs/trading_tab.py, in TradingTabMixin._build_trading_tab`
Pause API Log	Holds new entries until Resume API Log. The buffer holds 2,000, and a full buffer drops the oldest. `src/gui/main_window.py, in MainWindow._on_api_event`

The Indicator Voting Panel on this tab

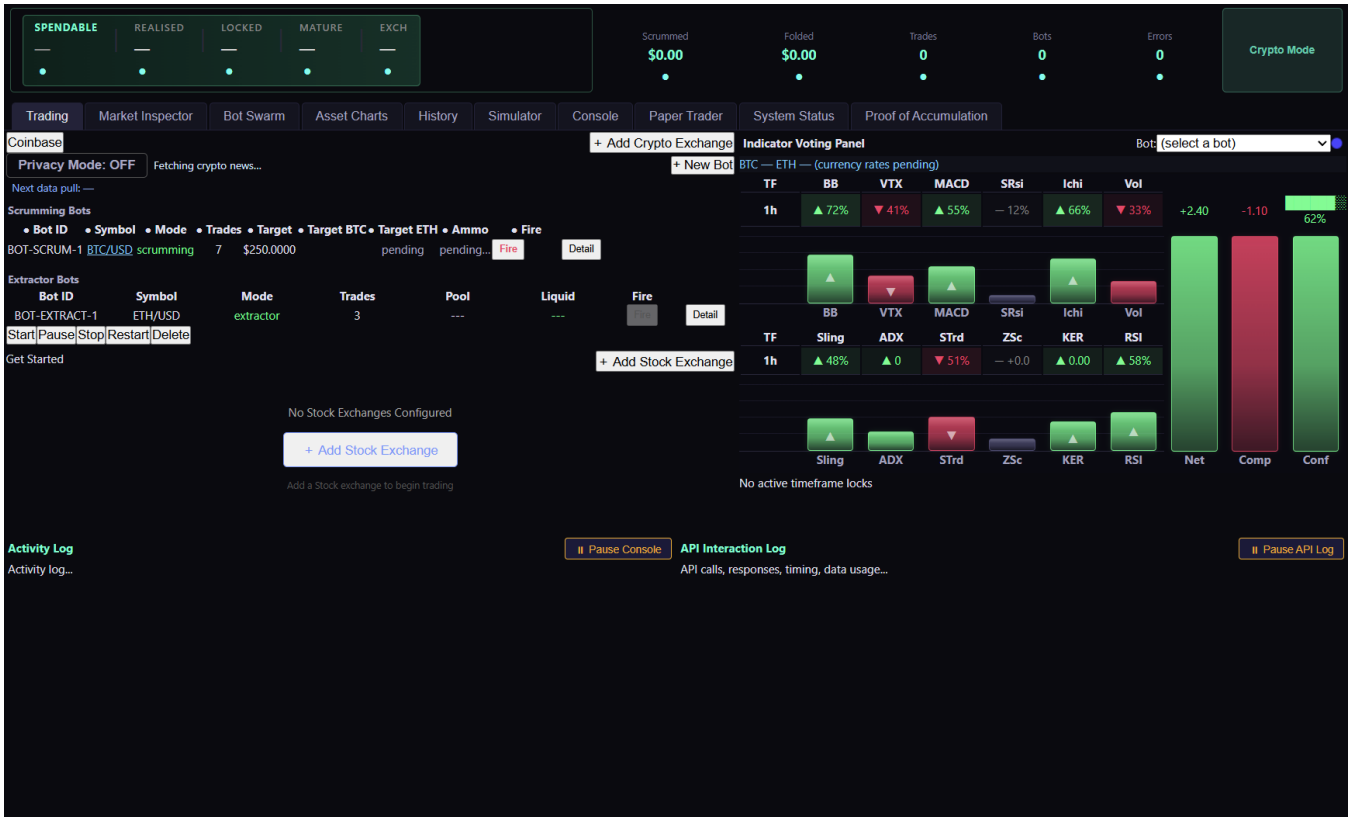


The Indicator Voting Panel, at the right of the Trading tab.

07-indicators.md holds the indicator formulae.

Control	What it does and what it reads
Bot selector	Picks the bot whose votes draw. It lists every Scrumming bot, on every class. `src/gui/indicator_panel.py, in IndicatorVotingPanel.update_bot_list`

Control	What it does and what it reads
Previous, next	Step the selector one bot, and stop at each end. `src/gui/indicator_panel.py, in IndicatorVotingPanel._step_bot`
Staleness label	An amber line naming stored votes and their age, or the cause when no votes exist. `src/gui/indicator_panel.py, in IndicatorVotingPanel.show_no_data`
Rate strip	A15 and A14 dollar prices, small units per dollar and per cent, and the source. `src/gui/indicator_panel.py, in IndicatorVotingPanel.update_currency_rates`
Privacy dot	Masks the selector text. The votes stay visible. `src/gui/indicator_panel.py, in IndicatorVotingPanel._apply_privacy_mask`
TF column	One row per timeframe, the bot's own plus its phantoms', shortest first. `src/gui/main_tabs/indicator_panel_surface.py, in timeframe_rows`
Twelve voter columns	BB, VTX, MACD, SRsi, Ichi, Vol, Sling, ADX, STrd, ZSc, KER and RSI. Each cell shows direction and confidence; ADX, ZSc and KER show raw readings. `src/gui/main_tabs/indicator_panel_surface.py, in indicator_cell_text`
Net	The signed bull-minus-bear tally for that row. `src/gui/main_tabs/indicator_panel_surface.py, in net_cell`
Comp	Net blended with the higher-timeframe phantoms, and a dash on phantom rows. `src/gui/main_tabs/indicator_panel_surface.py, in comp_cell`
Conf	A block bar and an agreement percentage. `src/gui/main_tabs/indicator_panel_surface.py, in conf_cell`
Bars	One bar per voter and one pillar each for Net, Comp and Conf, for the first-listed timeframe. `src/gui/main_tabs/indicator_panel_surface.py, in collated_pillars`
Locks line	Reads No active timeframe locks. Nothing creates a lock, so it stays. `src/gui/indicator_panel.py, in IndicatorVotingPanel._setup_ui`



How to use the Bot Wizard

The wizard's title reads Create Auto Trader. The Qt build draws `BotCreationWizard` and the React build draws `BotWizardReactDialog`, and a failed React import falls back to Qt (`src/gui/main_window.py, in MainWindow._create_bot`).

The walk through the wizard

Step	What it does and what it reads
Scrumming order	Trading Mode, Select Asset Pair, Trading Parameters, Phantom Bots, Finish. `src/gui/bot_wizard.py, in BotCreationWizard.nextId`
Extractor order	Trading Mode, Extractor Pool, Trading Parameters, Finish. `src/gui/bot_wizard.py, in BotCreationWizard.nextId`
Back, Next, Cancel	Back returns one page and keeps your entries. Cancel closes the wizard and makes no bot. `src/gui/main_tabs/bot_wizard_surface.py, in BotWizardModel.go_back`
Finish	Closes the wizard and builds the bot, which waits IDLE until Start. `src/gui/main_window.py, in MainWindow._create_bot`
A market no bot can trade	Finish keeps the wizard open and names why no built variant trades that market. `src/gui/bot_wizard.py, in BotCreationWizard.accept`
Pre-flight check	Scrumming only, after Finish. A missing venue, a missing symbol or a failed market read stops creation, and a failed price read does not. An inactive market, or a target under 3x the minimum cost, asks Yes or No. `src/gui/preflight_check.py, in check_symbol`
Parent check	Extractor only, after Finish. It stops creation unless exactly one Scrumming Bot holds the base currency. `src/gui/main_window.py, in MainWindow._refuse_extractor_without_parent`

Trading Mode page

Class	What it offers
Crypto	Crypto Scrumming, ticked, and Base Currency Extractor (Multi-Target). `src/gui/main_tabs/bot_wizard_surface.py, in mode_offer`
Stock, Commodities, Forex, Indices, Futures / Perps	One ticked row: that class's own Scrumming option. No Extractor. `src/gui/main_tabs/bot_wizard_surface.py, in mode_offer`

Select Asset Pair page

Row	What it does and what it reads
Exchange	Lists the stored venues that serve the class. Picking one loads its markets. `src/gui/bot_wizard.py, in AssetSelectionPage._fetch_markets`
Base Currency	Seven declared money legs, then any other leg the venue quotes. It opens on a leg the venue quotes. `src/gui/main_tabs/bot_wizard_surface.py, in opening_base_index`
Target Asset	Markets quoted in that base, highest volume first. A market no bot can trade says so and says why. `src/gui/bot_wizard.py, in AssetSelectionPage._filter_assets`
Information button	The Qt build opens the asset's description. The React build always answers that no description is available. `src/gui/main_tabs/bot_wizard_surface.py, in BotWizardModel.show_info`
Status line	Counts the pairs, and counts those that cannot carry a bot. `src/gui/bot_wizard.py, in AssetSelectionPage._filter_assets`

Trading Parameters rows

The first column gives each row's range and factory opening value. Order Visibility, Aggressive Trading, Target Balance, Enable Phantom Bots and Candles to lock open on your stored default (`src/gui/bot_wizard.py, in TradingParamsPage`). On a running bot the same rows wait for Apply Changes and reach the next tick.

Row	What it does and what it reads
Order Visibility (Order Book)	Order Book sends limit orders and rests tranches on the book. Internal sends market orders and tracks tranches in the app. Reads `visibility`. `src/trading/scrumming/execution.py, in ExecutionEngineMixin._execute_sell`
Aggressive Trading (off)	Sends Stack tranches in Order Book mode as immediate-or-cancel limits. A tranche above the market cancels unfilled. Reads `aggressive_trading`. `src/trading/scrumming_bot.py, in ScrummingBot._open_stack_from_scrum`
Stack Mode (on)	Turns a scrum into a rising sell ladder, and ladders a fold's bought units too. Reads `stack_mode`. `src/trading/scrumming/execution.py, in ExecutionEngineMixin._execute_sell`
Split Distance (0.1 to 20%, 1%)	Sets the percent gap between tranches. Reads `split_distance`. `src/trading/stack_math.py, in ladder_offsets_pct`
Tranche Count (2 to 20, 3)	Sets the tranches a ladder aims for. Fewer appear under the venue minimum, or where two prices merge within 0.1%. Reads `stack_tranche_count_target`. `src/trading/stack_math.py, in split_scrum_into_tranches`
Spacing (Linear)	Picks the multipliers: Linear, Quadratic or Exponential. Reads `stack_spacing_mode`. `src/trading/stack_math.py, in level_multipliers`

Scrumming Settings rows

Row	What it does and what it reads
Opposing Trade Interval (0.1 to 20%, 1%)	Sets the Target Delta a scrum needs, and the base of the opposing trade distance. Reads `scrumming_interval_pct`. `src/trading/otd_math.py, in minimum_opposing_trade_distance_pct`
BB Tolerance (0.25 to 5%, 1%)	Sets how close to a band, in percent of band width, price counts for the Landing Strip. Reads `bb_tolerance_pct`. `src/trading/indicators/bb_proximity.py, in detect_bb_proximity`
Landing Strip Candles (2 to 10, 3)	Sets the tight Heikin Ashi candles the band check needs. Reads `bb_landing_strip_candles`. `src/trading/indicators/bb_proximity.py, in detect_bb_proximity`
TA Timeframe (the venue's list, 1h)	Sets the candle length for votes, bands and breakers. Reads `ta_timeframe`. `src/trading/scrumming_bot.py, in ScrummingBot.tick`
Target Balance ($1 to $1,000,000, $200)	Sets the value the bot trades toward. On Apply, a figure above the anchor keeps accrued growth, and one at or below moves both. Reads `target_balance`. `src/trading/scrumming_bot.py, in ScrummingBot.set_target_balance_live`
Max Entry Price (0 to 10,000,000, 0 off)	Refuses fold, hedge, entry and tranche Fire buys above this price. Reads `max_entry_price`. `src/trading/scrumming/execution.py, in ExecutionEngineMixin._execute_buy`
Min Entry Price (0 to 10,000,000, 0 off)	Refuses the same buys below this price. Reads `min_entry_price`. `src/trading/scrumming/execution.py, in ExecutionEngineMixin._execute_buy`
Trading Fee % (0 to 5%, 0.6%)	Sets the fee inside the opposing distance. A higher fee the venue charged replaces it. Reads `trading_fee_pct`. `src/trading/otd_math.py, in trading_fee_pct_from_config`
Max Target Growth % (0 to 100%, 1%)	Caps target growth per cycle and the size of each fold order. 0% freezes the target and stops folds. Reads `max_target_growth_pct`. `src/trading/scrumming_bot.py, in ScrummingBot.cycle_growth_cap_usd`
Scrum Fold Ratio (1 to 100%, 100%)	On a plain scrum or cartridge sale, queues this share of proceeds for buy-back, and the rest stays cash. A Stack tranche fill queues nothing. Reads `scrum_fold_pct`. `src/trading/scrumming/fold_tranches.py, in FoldTrancheAccountingMixin._apply_scrum_fold_pct`
Unit lines under this group	Name the market's unit rule. A whole-unit market adds its minimum, interval and position notes. `src/gui/main_tabs/bot_wizard_surface.py, in unit_notes`

Advanced Scrumming rows

Row	What it does and what it reads
Detect Threshold (10 to 90%, 75%)	Starts TRACK and sets the hard line: no scrum below it, no fold above it. Reads `scrum_detect_pct`. `src/trading/scrumming/circuit_breakers.py, in CircuitBreakerMixin._bb_detect_thresholds`
Fire Threshold (0.1 to 10%, 0.5%)	Sets how close to a band FIRE starts. A scrum past Detect passes it, and the first buy reads it. Reads `scrum_fire_pct`. `src/trading/scrumming/tick_phases.py, in TickPhaseMixin._tick_target_ramp`
BB Midline Gate (on)	Keeps scrums above the midline and folds below it. Reads `bb_midline_gate`. `src/trading/scrumming_bot.py, in ScrummingBot.tick`
Read Rate (1 to 60 min, 5)	Sets the minutes between full reads in SEARCH. TRACK and FIRE read ten times faster, and a bot the hard breaker holds reads at the SEARCH rate. Reads `scrum_read_rate_min`. `src/trading/scrumming_bot.py, in ScrummingBot.tick`
Band Travel (0 to 100%, 70%)	Widens the fold distance by this share of band width, and lifts trend hold after that much travel. 0 is off. Reads `band_travel_pct`. `src/trading/otd_math.py, in band_extension_pct`
BB Bullseye Check (on)	Counts a band touch within 0.5%, or a wick within 0.2%, toward the BB priority skew. Reads `bb_bullseye_check`. `src/trading/scrumming_bot.py, in ScrummingBot.tick`

Hedge Rebalance rows

Row	What it does and what it reads
Hedge Rebalance Active (on)	Lets the reserve buy into a gap under target. Apply fills the reserve on tick and freezes it on untick. Reads `hedge_rebalance_active`. `src/trading/scrumming_bot.py, in ScrummingBot.set_hedge_rebalance_active_live`
Hedge Balance ($0 to $999,999,999, $200)	Sets the reserve ceiling. Apply leaves the current reserve alone. Reads `hedge_balance`. `src/trading/scrumming_bot.py, in ScrummingBot.set_hedge_balance_live`

Circuit Breakers rows

Row	What it does and what it reads
Soft CB Threshold (0 to 100%, 25%, 0 off)	Shuts one side when a candle's range reaches this share of its open at the one read of that candle. Reads `circuit_breaker_soft_pct`. `src/trading/scrumming/circuit_breakers.py, in breaker_move`
Hard CB Threshold (0 to 100%, 35%, 0 off)	Trips the hard breaker at this range, and every order stops until a reset. The bot reads HELD and sends one alert. Each candle is read once, when the shared candles first show it, so a move after that does not count. Reads `circuit_breaker_hard_pct`. `src/trading/scrumming/circuit_breakers.py, in CircuitBreakerMixin._check_circuit_breakers`
Soft CB Cooldown (1 to 100, 3)	Sets how many new candles the shared candles must show before the shut side opens again. Reopening the app reads the newest candle as new, so it spends one. Reads `circuit_breaker_cooldown_candles`. `src/trading/scrumming/circuit_breakers.py, in CircuitBreakerMixin._read_breaker_candle`
Max Cartridge Size (0 to 200%, 10%, 0 off)	Fires a market rebalance once the Target Delta reaches this share of target. A bot that has made no first buy holds this buy until the band and its own vote clear: `src/trading/scrumming/tick_phases.py, in TickPhaseMixin._first_buy_clears`. Reads `max_cartridge_size_pct`. `src/trading/scrumming/tick_phases.py, in TickPhaseMixin._tick_cartridge_fire`
Smart Cartridge (off)	Replaces that percent with the Bollinger range, floored at the interval. Reads `max_cartridge_smart`. `src/trading/scrumming/tick_phases.py, in TickPhaseMixin._tick_smart_cartridge_pct`
Smart Ceiling (1 to 100%, 30%)	Caps the Smart Cartridge figure. Reads `max_cartridge_smart_ceiling_pct`. `src/trading/container/config.py, in smart_cartridge_ceiling_pct`

Risk Controls rows

Row	What it does and what it reads
Enable Position Ceiling (off)	Tapers folds from half the ceiling and stops them at it. Scrums continue. Reads `position_ceiling_enabled`. `src/trading/scrumming_bot.py, in ScrummingBot.fold_size_taper`
Ceiling Multiple (1 to 10x, 5x)	Sets the ceiling as a multiple of the anchor. Reads `position_ceiling_multiple`. `src/trading/scrumming/sizing.py, in position_ceiling`
Enable Detonation (off)	Sells the value above the anchor on a new bullish turn, then resets the target, the fold queue and the lots. Reads `detonation_enabled`. `src/trading/scrumming/tick_phases.py, in TickPhaseMixin._tick_detonation`
Detonation TF (1d or 1w, 1d)	Picks the chart the hourly check reads. Reads `detonation_timeframe`. `src/trading/scrumming_bot.py, in ScrummingBot._check_detonation_trigger`
Min Confidence (0 to 1, 0.75)	Sets the bullish confidence bar. Confidence must also top what two indicators reach. Reads `detonation_confidence_min`. `src/trading/scrumming_bot.py, in ScrummingBot._check_detonation_trigger`

Strategy Gate Flags rows

Past the Detect line a ripe harvest passes the bullish TA and uptrend flags, and below it the band gate holds the scrum, so those two flags change no scrum (`src/trading/gate_chain.py, in RipeHarvestScrumOverride`).

Row	What it does and what it reads
SCRUM requires bullish TA (on)	Asks for a bullish vote, which a ripe harvest passes. Reads `scrum_require_ta_bullish`. `src/trading/gate_chain.py, in effective_gate_flags`
SCRUM holds in sustained uptrend (on)	Holds past 13 of 20 up candles, which a ripe harvest passes. Reads `scrum_hold_in_uptrend`. `src/trading/gate_chain.py, in TrendHoldGate`
SCRUM defers to higher-TF bullish (on)	Refuses a scrum while the phantom bias reads bullish. Reads `scrum_defer_to_htf`. `src/trading/gate_chain.py, in HTFDeferGate`
FOLD requires bearish TA (on)	Asks for a bearish vote. A deep fold, past the interval under target, passes it. Reads `fold_require_ta_bearish`. `src/trading/gate_chain.py, in DeepFoldOverride`
FOLD defers to higher-TF bearish (on)	Refuses a fold while the phantom bias reads bearish. Reads `fold_defer_to_htf`. `src/trading/gate_chain.py, in HTFDeferGate`

Phantom Bots rows

Row	What it does and what it reads
Enable Phantom Bots (on)	Starts one read-only observer when the bot initialises, and again on the first worked tick after a Stop or a Restart. Reads `enable_phantoms`. `src/trading/scrumming/tick_phases.py, in TickPhaseMixin._start_phantom_set`
Timeframe boxes	Picks one timeframe above the bot's own. Timeframes the venue lacks grey out. Reads `phantom_timeframes`. `src/gui/bot_wizard.py, in PhantomConfigPage._timeframe_refusal`
Candles to lock (1 to 10, 2)	Stores a lock length. Nothing creates a lock, so nothing uses it. Reads `lock_candle_count`. `src/trading/phantom_balance.py, in TimeframeCoordinator`

Extractor rows

The Extractor Pool page sets the venue and the base currency, and the parameter page sets the rest.

Row	What it does and what it reads
Exchange	The same venue list. Picking one fills the alternate pair list again. `src/gui/bot_wizard.py, in ExtractorPoolPage._on_exchange_changed`
Pool Base Currency	Five declared currencies, opening on the first. It sets what the pool grows and which pairs it may trade. `src/trading/extractor_bot.py, in ExtractorBot._pair_filter_matches`
Alternate pair list	A read-only view of pairs quoted in that base. The bot ranks its own. `src/gui/bot_wizard.py, in ExtractorPoolPage._refresh_alt_list`
Chunk size (USD) ($10 to $10,000,000, $100)	Base currency the bot may put to work, turned into base units on its first run. Reads `extractor_chunk_size_usd`. `src/trading/extractor_bot.py, in ExtractorBot.set_initial_chunk_rate`
Artillery size (USD) ($0.50 to $100,000, $5)	The dollar size of every round. A round larger than the free chunk does not fire. Reads `extractor_artillery_size_usd`. `src/trading/extractor_bot.py, in ExtractorBot._fire_artillery`
Watch list top-N (5 to 10, 8)	How many top-volume pairs it watches. A pair it holds stays listed on top of that. Reads `extractor_scan_top_n`. `src/trading/extractor_bot.py, in ExtractorBot._refresh_watch_list`
Watch list refresh (10 to 240 candles, 60)	Candles between rankings of the list. Reads `extractor_scan_refresh_candles`. `src/trading/extractor_bot.py, in ExtractorBot._refresh_interval_seconds`

Row	What it does and what it reads
Exit % (10 to 100%, 100%)	The share sold on a bullish exit. Below 100 the rest stays open. Reads `extractor_exit_pct`. `src/trading/extractor_bot.py, in ExtractorBot._execute_bullish_exit`

Whole-Unit Market and Contract Expiry rows

Each group shows only for a market that needs it.

Row	What it does and what it reads
Whole-Unit Market group	Shows for a Scrumming market sized in whole units, or one whose session takes whole units out of hours. `src/gui/bot_wizard.py, in TradingParamsPage.set_market`
Open Position At (0 to 1,000 units, 0)	The opening size. 0 or 1 leaves the two-unit minimum. Reads `whole_unit_opening_units`. `src/trading/container/config.py, in whole_unit_opening_units`
Contract Expiry group	Shows for a Scrumming market with an expiry date. `src/gui/main_tabs/bot_wizard_surface.py, in market_expires`
Expiry Close (Finish the sell ladder)	Finishes the sell ladder or sells the whole position inside the lead time. Buys stop there either way. Reads `expiry_close_action`. `src/trading/bot_container.py, in BotContainer.guarded_place_order`
Expiry Lead Mode (Share of the contract's life)	Picks a share of the contract's life or a number of days. Reads `expiry_lead_mode`. `src/trading/container/config.py, in expiry_lead_resolved_days`
Lead Fraction (0 to 1, 0.20)	The share of life left at bot start. 0 is off. Reads `expiry_lead_fraction`. `src/trading/container/config.py, in expiry_lead_resolved_days`
Lead Days (0 to 36,500, 11)	Days before expiry, read in days mode only. 0 is off. Reads `expiry_lead_days`. `src/trading/container/config.py, in expiry_lead_resolved_days`
Expiry Horizon (0 to 36,500 days, 365)	Treats a later contract as non-expiring. 0 turns the close off. Reads `expiry_horizon_days`. `src/trading/container/config.py, in expiry_close_decision`

How to use a running bot

The Detail button opens the bot settings window. A Scrumming bot's window carries Status, Settings, Fold Tranches, Stack Tranches, Bot Swarm, Market Inspector and Phantom Bots. An Extractor's carries Status, Settings and Positions Held (`src/gui/bot_live_settings.py, in BotLiveSettingsDialog._setup_ui`).

The window's own buttons

Button	What it does
Apply Changes	Writes every pending edit at once, with no restart. `src/gui/bot_live_settings.py, in BotLiveSettingsDialog._apply_changes`
Close	Closes the window and drops unapplied edits. `src/gui/bot_live_settings.py, in BotLiveSettingsDialog._setup_ui`
Prev, Next	Closes this window and opens the neighbouring bot at the same tab position, dropping unapplied edits. The walk covers every bot the manager holds and wraps at both ends. Both buttons hide below two bots. Ctrl+Left and Ctrl+Right do the same. `src/gui/bot_live_settings.py, in BotLiveSettingsDialog._navigate_to_sibling`
Pending line	Names the fields waiting for Apply, then counts what Apply changed. `src/gui/bot_live_settings.py, in BotLiveSettingsDialog._mark_changed`
Hold line	Shows under the header while a hard breaker holds the bot, and names the hold. `src/gui/main_tabs/bot_live_settings_surface.py, in hold_text`

Rows only a running bot shows

These sit on the Settings tab beside the wizard's rows.

Row	What it does and what it reads
Live target	The traded target, the anchor and accrued growth, as the tab opened. `src/gui/live_settings/settings_tab.py, in SettingsTabMixin._create_settings_tab`
Standing surplus	Fold surplus parked past the cycle cap, which later folds drain. `src/trading/scrumming_bot.py, in ScrummingBot._apply_fold_target_growth`
Cycle growth budget	This cycle's cap and what it has consumed. `src/trading/scrumming_bot.py, in ScrummingBot.cycle_growth_cap_usd`
Over-cap tranches	Shows when a tranche exceeds one cycle's budget. `src/gui/live_settings/settings_tab.py, in SettingsTabMixin._create_settings_tab`
Target A15, Target A14	The target in that asset and its 24-hour gap against the dollar pair, each 5 seconds. `src/gui/live_settings/settings_tab.py, in SettingsTabMixin._refresh_target_denom_rows`
Profit Folding Active (on)	Lets fold surplus grow the target. Reads `profit_folding_active`. `src/trading/scrumming_bot.py, in ScrummingBot._apply_fold_target_growth`
Tranche Despawn Timer (0 to 365 days, Off)	Removes fold and stack tranches this old on each tick. It places no order, and a tranche with a resting order stays. A hard hold removes nothing, and time under a hold does not count toward the age. Reads `tranche_despawn_days`. `src/trading/scrumming/fold_tranches.py, in FoldTrancheAccountingMixin._despawn_aged_tranches`
Reset All Breakers	Clears both breakers at once, and it is the only press that ends a hard hold. A bot the hard breaker paused runs again, a bot paused by hand stays paused, and a bot with no tick loop waits for Start. Hold SHIFT to clear every bot in the fleet, after Yes or No. `src/gui/live_settings/settings_tab.py, in SettingsTabMixin._on_cb_reset_all`

Row	What it does and what it reads
Self-Destruct	After you type SELF-DESTRUCT, it sells the asset's whole venue balance at market, clears the fold queue and pauses the bot. The pause lands once the sale has gone to the venue, whether it filled, rests or lost its reply, and it holds until you press Start. Reopening the app starts the bot with the fleet. A later piece books after Start and never pauses the bot again. A sale that lost its reply books at the first worked tick after Start, once the venue shows those units gone. A sale the venue refuses flat leaves the bot as it was, and the log says so. Shadow Tranches join the sale only once an order comes back. `src/trading/scrumming_bot.py, in ScrummingBot.self_destruct`. While the hard breaker holds, the press shows a box that names the hold, changes nothing and says to press it again after the reset. `src/gui/live_settings/settings_tab.py, in SettingsTabMixin._on_self_destruct_clicked`
Hunger (off)	Eases the vote floor after idle candles past the opposing distance. It never lets two indicators trade. Reads `hunger_enabled`. `src/trading/scrumming_bot.py, in ScrummingBot._count_hunger_candle`
Hunger Pace (a dash means none)	Sets the idle candles to full hunger. A dash does nothing. Reads `hunger_pace_candles`. `src/trading/container/config.py, in BotConfig.hunger_gain_per_candle`
Accumulation Shadow (off)	In a bearish regime, each fold's extra units go to a Shadow Tranche, held until a bullish regime. Reads `shadow_enabled`. `src/trading/scrumming/shadow.py, in ShadowTrancheMixin._mark_shadow_rows`

The window's other tabs

Tab	What it holds
Status	Read-only at open: realised profit and loss, trades, open orders, price, uptime and last error. `src/gui/live_settings/status_tab.py, in StatusTabMixin._create_status_tab`
Fold Tranches	The queued buy-backs and their counters. Fire buys one tranche with its own sale dollars, skipping the vote and Target Delta gates. While the hard breaker holds it refuses and leaves the tranche as it is, so press it again after the reset. On a paused bot it still buys, and its Stack rungs do not open. Each Clear button asks first and places no order. `src/trading/scrumming_bot.py, in ScrummingBot.manual_fire_tranche`
Fold Tranches: Order, Filter	Order sets the row order, and Filter hides rows that do not match. `src/gui/live_settings/fold_chrome.py, in build_fold_row_controls`
Fold Tranches: Arbiter button	Flips one label on an Extractor tranche and moves no money. `src/gui/live_settings/fold_tranches_tab.py, in FoldTranchesTabMixin._on_arbiter_toggle_clicked`
Stack Tranches	The rising sell levels and their counters. Clear keeps a tranche that holds a resting order, and Clear Lifetime Counters asks first. `src/trading/scrumming_bot.py, in ScrummingBot.clear_stack_tranches`
Stack Tranches: Shadow Tranches	A summary row and one detail row for each Shadow Tranche the bot holds. `src/gui/main_tabs/stack_tranches_tab_surface.py, in compose_shadow_rows`
Bot Swarm	Read-only: wires, lifetime totals, pending credits and recent transfers. Wires change on the main Swarm tab. `src/gui/live_settings/bot_swarm_tab.py, in BotSwarmTabMixin._create_bot_swarm_tab`
Market Inspector	Read-only: this bot's card from the Inspector's last scan. `src/gui/main_tabs/market_inspector_tab_surface.py, in per_bot_view`
Phantom Bots	Three controls that wait for Apply, and the phantoms' state. A running phantom keeps its timeframe until restart. `src/trading/scrumming_bot.py, in ScrummingBot.update_phantom_config`

A running Extractor

The Extractor's Settings tab draws the Scrumming rows except Opposing Trade Interval, BB Tolerance, Landing Strip Candles, TA Timeframe, Target Balance, Hunger and Hunger Pace, and Apply accepts them (`src/gui/live_settings/settings_tab.py, in SettingsTabMixin._create_settings_tab`). The engine reads only the five Extractor rows and Trading Fee % (`src/trading/extractor_bot.py, in ExtractorBot.tick`).

A started Extractor ranks its watch list and places no order, because its signal reader keeps the stand-in exchange the bot was built with (`src/trading/extractor_bot.py, in ExtractorBot.__init__`).

Control	What it does and what it reads
Pool color	Green with nothing open, yellow with positions open, red with one or more in drawdown. `src/trading/extractor_bot.py, in ExtractorBot.pool_color`
Chunk size, Chunk free	The chunk in base units and dollars, and the base units not in a position. `src/gui/live_settings/positions_held_tab.py, in PositionsHeldTabMixin._create_positions_held_tab`

Control	What it does and what it reads
Lifetime extracted	The running base-unit gain from sales. `src/trading/extractor_bot.py, in ExtractorBot._book_exit`
Position rows	Pair, state, units held and dollar size at entry. Current and its change are priced at the average buy price, so neither follows the market. `src/trading/extractor_bot.py, in ExtractorBot.positions_for_gui`
Fire, per position	After Yes, it sells the whole position at market and skips the profit check. `src/gui/live_settings/positions_held_tab.py, in PositionsHeldTabMixin._on_fire_position`
Reset All Breakers, Self-Destruct	An Extractor has neither. Self-Destruct shows a refusal, and Reset reads Reset failed. `src/gui/live_settings/settings_tab.py, in SettingsTabMixin._on_self_destruct_clicked`

Two mechanisms with no control on this tab

The Fair Value Gap reading has no control, and nothing builds it (`src/trading/indicators/fvg.py, in FVGIndicator`). Boost Fold has no control either, and the crypto engine sets its fields to zero and reads none of them (`src/trading/scrumming_bot.py, in ScrummingBot.__init__`).

Indicator Voting Panel

This panel shows the readings for each of the 12 as well the collated indices and confidence score.

TF Lock - To be re-evaluated.

The control is removed. `indicator_panel.py` builds no `_tf_lock_combo`, and `indicator_panel_surface.py` publishes no `tf_lock` block, so nothing puts a timeframe on `indicator.tf_lock_changed` any more. `main_window.py` still subscribes to that topic; with no publisher, `_lock_timeframe` keeps whatever the coordinator sets for itself.

`src/gui/indicator_panel.py` — the header, after the removal

```
header.addStretch()
header.addWidget(QLabel("Bot:"))
self._bot_selector = QComboBox()
```

The bot selector and its privacy dot close the header on the right. The pair symbol and the vote tally that stood there are gone: `_symbol_label` and `_summary_label` are not built, and the payload carries no `symbol_text` or `summary_text`.

A drop-down of eleven entries: None, then Lock below each of ten timeframes from 5m to 1w. It opens on the fifth entry, Lock below 1h. Choosing one writes a status line beside the box and puts the timeframe on the event bus.

The drop-down above is removed, so no such event reaches the main window, and nothing sets a lock name on the coordinator. Each bot reads the lock from its coordinator itself (`src/trading/scrumming_bot.py, in ScrummingBot._refresh_phantom_lock`). The tooltip on the box says the lock feeds directly into active Scrumming Bots. Issue #155 carries this.

The issue #155 clause above is overtaken. It is not edited and not removed. It read:

> *The tooltip on the box says the lock feeds directly into active Scrumming Bots. Issue #155 carries this.*

Issue #155 closed on 2026-10-22, so the pointer names no open work. The reading above it still holds: the write reaches no reader. The name the window sets on the coordinator and the name the Phantom Bots pages read are two different names, and only the second is declared on the bot.

```
src/trading/scrumming_bot.py, in ScrummingBot._refresh_phantom_lock
    writes   bot._phantom_lock_timeframe
    readers  the three Phantom Bots pages below

src/trading/scrumming_bot.py        declares _phantom_lock_timeframe on the bot
src/gui/main_tabs/phantom_bots_tab_surface.py   LOCK_TIMEFRAME_ATTRIBUTE names that one
src/gui/live_settings/phantom_bots_tab.py       reads it off the bot
src/gui/paper/paper_phantom_bots_tab.py         reads it off the bot
src/gui/simulator/sim_phantom_bots_tab.py       reads it off the bot
```

Which of the two names the drop-down should set is a product choice, so nothing here proposes one.

The handler below is gone from `src/gui/main_window.py`, and the bot reads the lock from its coordinator (`src/trading/scrumming_bot.py, in ScrummingBot._refresh_phantom_lock`).

```
for bot in self._bot_manager._bots.values():
    if isinstance(bot, ScrummingBot) and hasattr(bot, "_coordinator"):
        bot._coordinator._lock_timeframe = tf
```

The two clauses above are overtaken. Neither is edited and neither is removed. They read:

> *The reading above it still holds: the write reaches no reader. The name the window sets on the coordinator and the name the Phantom Bots pages read are two different names, and only the second is declared on the bot.*
>
> *Which of the two names the drop-down should set is a product choice, so nothing here proposes one.*

The choice is settled, and it is an implementation one. The name the three Phantom Bots pages already read is the one that survives. The handler sets that name on the bot and keeps setting the older name on the coordinator, so a reader of the older spelling still finds its value.

The later handler below is gone from `src/gui/main_window.py` too, and the same bot method reads the lock (`src/trading/scrumming_bot.py, in ScrummingBot._refresh_phantom_lock`).

```
for bot in self._bot_manager._bots.values():
    if not isinstance(bot, ScrummingBot):
        continue
    setattr(bot, LOCK_TIMEFRAME_ATTRIBUTE, tf)
    if hasattr(bot, "_coordinator"):
        bot._coordinator._lock_timeframe = tf
```

Driven on a hand-built record, with two different settings in one run, both names hold the chosen timeframe and the lock row on the page carries it.

lock set to	what the pages read	the page's SCRUM lock state row
nothing yet	empty	LOCKED — SCRUM suppressed by TF phantom
4h	4h	LOCKED — SCRUM suppressed by 4h TF phantom
1d	1d	LOCKED — SCRUM suppressed by 1d TF phantom

The first row is what the page drew for every setting before this change: a gap where the timeframe belongs.

Live, Paper and the Simulator read the one name, so all three draw the timeframe once a value arrives. Only the Live window subscribes to the topic, and nothing publishes it, so no press reaches any of the three today. Parity across the three surfaces is not closed while the control is absent.

Three clauses above are overtaken. None is edited and none is removed. They read:

> `main_window.py` *still subscribes to that topic; with no publisher,* `_lock_timeframe` *keeps whatever the coordinator sets for itself.*
>
> *The handler sets that name on the bot and keeps setting the older name on the coordinator, so a reader of the older spelling still finds its value.*
>
> *Only the Live window subscribes to the topic, and nothing publishes it, so no press reaches any of the three today.*

The current reading. The operator has ruled that the control is not coming back to this panel and that the setting stays with the Phantom Bots page. The handler and its subscription are therefore removed, and so is its write onto the coordinator. The window subscribes to nothing on that topic, and the register in `main_window_surface.py` no longer names either the topic or the handler.

The bot now reads the lock from the coordinator instead. One method on the bot writes both names every tick, from the coordinator's own active lock list, and clears both where the operator has phantom bots switched off.

Which ticks reach it, measured. The call sits on the main gate path, so it runs on every tick that reaches the scrum flag and on no tick that returns above it. A hand-built bot driven through the real tick ran eight full passes and reached the call on none of them, because its position value equals its target balance, which leaves no delta and returns at the interval guard. Twelve returns sit above the call. Calling the method directly on the same bot moved the flag to locked and the timeframe to the lock's own, so the zero is a fact about the path that tick took and not about the reading.

`src/trading/scrumming_bot.py` — `ScrummingBot._refresh_phantom_lock`

```
if not self._phantoms_enabled:
    self._phantom_locked = False
    self._phantom_lock_timeframe = ""
    return
timeframe = str(coordinator.active_lock_timeframe() or "")
self._phantom_locked = bool(timeframe)
self._phantom_lock_timeframe = timeframe
```

The timeframe comes from the lock itself, so the page names the timeframe that suppressed the scrum rather than leaving the gap the first table row shows.

`src/trading/phantom_balance.py` — `TimeframeCoordinator.active_lock_timeframe`

```
held = [str(one.get("source_tf") or "") for one in self.get_active_locks()]
if not held:
    return ""
return max(held, key=tf_rank)
```

Driven. A bot was built by hand with a stand-in venue that refuses every order, and a lock was placed on its coordinator for a timeframe above the parent. Four arms ran in one pass, and a second pair ran on a ripe row.

```
a lock on the coordinator, read at the bot and at the page

phantoms off, lock active      flag False   page UNLOCKED — SCRUM allowed
phantoms on, lock active       flag True    page LOCKED — suppressed by 1d
phantoms on, lock expired      flag False   page UNLOCKED — SCRUM allowed
phantoms on, no lock           flag False   page UNLOCKED — SCRUM allowed
```

The locking arm and the non-locking arm answer differently at the bot, at the scrum flag and on the page, so the reading can see the lock. Before this change all four arms answered alike, with the coordinator reporting the same active lock.

Swept across the band from -0.3 to 1.3 in steps of 0.0005, which is 3,201 rows, with phantom bots on and a lock held: the scrum flag changes on 1,601 rows and the midline refusal appears on 801 more, between 0.5 and 0.9. The final scrum decision changes on none of the 3,201 rows.

```
3,201 rows, phantoms on, a held lock against an expired one

scrum flag differs                     1,601
midline refusal differs                  801
final scrum decision differs               0
fold decision differs, as a control        0
```

Why no trade changes. On a ripe row the ripe-harvest override removes the midline refusal, and a row below the upper detect threshold is already refused by band proximity. Driven on a ripe row at 0.95 with the lock held, the scrum flag reads false, the blocked list is empty and the scrum fires. Whether the band should hold a ripe scrum is the open decision on issue #1359.

With phantom bots switched off, every one of the 3,201 rows is identical before and after the change, compared row by row on the scrum flag, the midline refusal, the final scrum decision and the fold decision.

Live, Paper and the Simulator draw from one page model, and all three were driven with the same locked bot. All three answer the identical sentence, and all three answer the unlocked sentence when the lock expires, so parity across the three is closed on that row. A Paper or Simulator page fed a stored record rather than a running bot draws the unlocked sentence, because a saved record carries the phantom settings and never a lock.

Nothing creates a lock on the coordinator yet, so the flag reads clear on every running bot today. What should create one changes which trades fire and is the operator's decision.

The operator's ruling, 2026-10-27. Asked what should create a phantom lock, he answered:

> *This is a strange question. A phantom bot is simply a higher timeframe read that is piped through a parallel trading gate logic chain. It is intended to improve rather than restrict ripe scrums or folds so it must work on both sides. It should be thought of as an optional guardian angel that may optimize trades.*

On precedence he answered that where the phantom does hold, it holds ahead of the ripe-harvest override.

Three readings above are overtaken. None is edited and none is removed. They read:

1d	1d	LOCKED — SCRUM suppressed by 1d TF phantom

> *fold decision differs, as a control 0*
>
> *Nothing creates a lock on the coordinator yet, so the flag reads clear on every running bot today. What should create one changes which trades fire and is the operator's decision.*

The phantom read reaches both sides. The fold flag now carries the same lock term the scrum flag carries, written by the same method on the bot, taken from the same active lock list, and read by the same gate on the other chain. The term is optional per bot through the tick box the operator already has: with phantom bots switched off, that method clears the flag and neither side reads anything.

`src/trading/scrumming_bot.py` — `ScrummingBot.tick`, both flags

```
self._refresh_phantom_lock()
if self.config.bb_midline_gate:
    scrum_ok = (bb_pos > 0.50) and not self._phantom_locked
    fold_ok_midline = (bb_pos < 0.50) and not self._phantom_locked
else:
    scrum_ok = not self._phantom_locked
    fold_ok_midline = not self._phantom_locked
```

Driven through the real tick. A bot was built by hand with a stand-in venue that refuses every order, one tranche queued and a position above target, so the tick reached both gate chains rather than returning above them. Four arms ran, each of them twice: once with the fold's own technical-analysis tick box on, and once with it cleared, which is a box the operator sets.

```
band position 0.11, the fold gate's own verdict, the TA box cleared

phantoms off, lock held        fold gate passes      fold fires
phantoms on,  lock held        fold gate refuses     fold held
phantoms on,  lock expired     fold gate passes      fold fires
phantoms on,  no lock          fold gate passes      fold fires
```

Swept, before against after. Each arm ran over 801 window levels from 0.800 to 1.200 in steps of 0.0005. That moved the real band position from -0.558 to 1.559 and put 398 of the 801 rows below the midline. Every row was compared on eleven recorded verdicts. A second sweep of 2,001 rows at a step of 0.000001 covered the midline crossing on its own, reading band positions from 0.4737 to 0.5181, and the fold refusal appeared on exactly the 1,185 rows below 0.50 and on none of the 816 at or above it.

```
801 rows an arm, eleven verdicts a row, two trees

phantoms off, lock held          every verdict identical
phantoms on,  no lock            every verdict identical
phantoms on,  lock expired       every verdict identical
phantoms on,  lock held          fold flag differs on            398
                                 fold gate refusal differs on    398
                                 fold decision differs on        362
                                 every scrum reading differs on    0
```

The 398 rows are the rows below the midline, where the band used to pass the fold on its own. The 362 are the rows where no other gate was holding the fold, so the lock is what changed the trade.

The control on that reading. Measured on the same rows after the change, a held lock against no lock differs on 801 lock readings, 403 scrum flag readings, 398 fold flag readings and 362 fold decisions, so the comparison can report a difference. The scrum side and three neighbouring fold gates were read in every arm the same way and all of them are identical before and after. No venue socket was attempted in either run, and the counter reached one on a single refused attempt.

Which surfaces it reaches. Live reads the lock on both sides. Paper and the Simulator compose both flags from the band alone and read no lock, before this change and after it, so neither moved. The Simulator's replay of a Live record does recover the lock, and it now applies it to both sides, so a replayed record answers what Live answered.

`src/simulator/validation.py` — `rerun_context`, the replay's two flags

```
if bot.bb_midline_gate:
    scrum_ok = (bb_pos > BB_MIDLINE) and not locked
    fold_ok_midline = (bb_pos < BB_MIDLINE) and not locked
else:
    scrum_ok = not locked
    fold_ok_midline = not locked
```

Still the operator's. What should create a lock on the coordinator. Nothing creates one today, so the flag reads clear on every running bot. The naming has not moved either: the flag, the page row quoted above and the log line all still describe restriction rather than the guardian his ruling names.

Bot - Drop down menu for selecting which bot's TA signals are displayed.

The list refills every tick and carries one entry per running bot, opening on a placeholder until one is chosen. Choosing a bot is what makes the twelve columns below read anything at all.

`src/gui/indicator_panel.py` — the selector

```
self._bot_selector = QComboBox()
self._bot_selector.setMinimumWidth(180)
self._bot_selector.addItem("(select a bot)", "")
self._bot_selector.currentIndexChanged.connect(self._on_bot_selected)
```

TF - Timeframe for the selected bot.

One row per timeframe, each row a separate verdict from the same twelve voters. The rows are the bot's own timeframes and any phantom timeframes it runs.

In development.

BB - Bollinger Bands - https://en.wikipedia.org/wiki/Bollinger_Bands

In Acervator, Bollinger Bands are used as the thresholds or boundaries at which trades are allowed to fire. Multiple market structure pieces are intertwined with our readings of the bands such as Landing Strip Detection (Heikin-Aishii Candle Consolidation Pattern) and Minimum Opposing Trade Distance. From a tactical standpoint, these represent the area or zone through which an investment position is passing.

The cell prints a direction arrow and a confidence percentage. Direction comes from the band position alone: under 0.15 or under 0.35 votes bullish, over 0.85 or over 0.65 votes bearish, and anything between the two middle figures votes neutral. Bollinger Bands carries the formula.

`src/trading/indicators/bollinger.py` — the vote

```
if bb_pos < 0.15:
    direction = SignalDirection.BULLISH
    confidence = max(0.0, min(1.0, (0.15 - bb_pos) / 0.15 * 0.8 + 0.3))
elif bb_pos > 0.85:
    direction = SignalDirection.BEARISH
    confidence = max(0.0, min(1.0, (bb_pos - 0.85) / 0.15 * 0.8 + 0.3))
elif bb_pos < 0.35:
    direction = SignalDirection.BULLISH
    confidence = 0.2
elif bb_pos > 0.65:
    direction = SignalDirection.BEARISH
    confidence = 0.2
```

VTX - Vortex Indicator - https://en.wikipedia.org/wiki/Vortex_indicator

In my experience (which is entirely subjective), this particular indicator is unusually strong at flagging early trend reversals. For Acervator, we are generally looking for signal saturation or near-saturation in either polarity which, under optimum conditions, will align with Bollinger Band thresholds being hit.

The cell prints an arrow and a percentage. The crossover of the two lines decides the vote, and then Acervator adds one reading of its own: the direction is overwritten to bullish whenever the downward line reaches 1.30, whatever the crossover said. Vortex Indicator carries the formula.

`src/trading/indicators/vortex.py` — the ceiling that overwrites the vote

```
VX_CEILING_PCT = 130.0
```

MACD - Moving Average Convergence Divergence - https://en.wikipedia.org/wiki/MACD

This is one of the oscillators used and, with it, we are looking for “top of the hill” formations in either polarity and these, under optimum conditions, will align with Bollinger Band thresholds being hit. Keep in mind that I do not approach any of these indicators from a mathematical standpoint as I am a purely visual trader without a quant or certified technical analysis background.

The cell prints an arrow and a percentage. A signal-line crossover decides the vote first. Failing that, a histogram growing away from zero votes with its own sign, and a histogram merely on one side of zero votes the same way with less conviction. MACD carries the formula.

`src/trading/indicators/macd.py` — the crossover, tested first

```
if prev_macd <= prev_signal and curr_macd > curr_signal:
    direction = SignalDirection.BULLISH
```

SRsi - Stochastic RSI - https://en.wikipedia.org/wiki/Stochastic_oscillator

Our second oscillator and, like the vortex indicator, we are looking at initial maximizing or prolonged periods of saturation of the signal at either polarity and this will, under optimum conditions, align with other indicators to trigger a trade.

The cell prints an arrow and a percentage. The vote needs a crossover of the two smoothed lines, and how deep in the range that crossover happens sets the confidence: a bullish cross under 30 scores 0.8, under 50 scores 0.5, and higher still scores less. Stochastic RSI carries the formula.

`src/trading/indicators/stochastic_rsi.py` — the bullish crossover

```
if prev_k <= prev_d and k > d:
    crossover = "bullish"
    if k < 30:
        direction = SignalDirection.BULLISH
        confidence = 0.8
```

Ichi - Ichimoku Cloud - https://en.wikipedia.org/wiki/Ichimoku_Kink%C5%8D_Hy%C5%8D

This particular indicator is one of the few ‘predicative’ indicators. In Acervator, we are focused on its additional ability to strengthen market reversal zone detection which is where we prefer to shave off or fold in funds.

The cell prints an arrow and a percentage. Several cloud readings add into one score, and only a score past 0.08 either way casts a vote. The score doubles as the confidence. Ichimoku Cloud carries the formula.

`src/trading/indicators/ichimoku.py` — the vote

```
if score > 0.08:
    direction = SignalDirection.BULLISH
    confidence = max(0.0, min(1.0, score))
elif score < -0.08:
    direction = SignalDirection.BEARISH
    confidence = max(0.0, min(1.0, abs(score)))
else:
    direction = SignalDirection.NEUTRAL
    confidence = 0.0
```

Net - Primary TF panel index collated from all 12 indicators.

The cell prints a signed figure to two decimals, green above zero and red below. It is the weighted bullish total less the weighted bearish total, and an abstaining voter adds nothing to either side. Net, Comp and Conf carries

the arithmetic.

`src/gui/indicator_panel.py` — the Net cell

```
net = tf_data.get("net_score", 0)
item = QTableWidgetItem(f"{net:+.2f}")
```

Comp - Composite TF panel index provided by the highest TF Phantom Bot that is active. Phantom bots are still in active development so any related features are not yet working.

The cell prints an em dash when no composite figure is present, which is what a fleet with no active phantom shows. Net, Comp and Conf carries the arithmetic.

In development.

Conf - Confidence reading for the panel.

This is the ‘breadth’ of agreement between all 12 indicators.

The cell draws a ten-block bar and the percentage beside it. The bar fills one block per tenth. Green from 0.6, amber from 0.3, grey below that.

`src/gui/indicator_panel.py` — the Conf cell

```
conf = tf_data.get("confidence", 0)
conf_bar = "■" * int(conf * 10) + "■" * (10 - int(conf * 10))
item = QTableWidgetItem(f"{conf_bar} {conf:.0%}")
```

Sling - CM (Chris Moody) Slingshot -

https://www.tradingview.com/script/GE7tSQK1-CM-Sling-Shot-System/

Another excellent indicator for reversal detection and here it adds to our laser precision detection of such zones and structures.

The cell prints an arrow and a percentage. A squeeze breaking either way votes first, and a band snapback votes when no squeeze is breaking. Slingshot carries the formula and the attribution, which is shared between three authors rather than one.

`src/trading/indicators/slingshot.py` — the squeeze, tested first

```
if squeeze_bull:
    direction = SignalDirection.BULLISH
    confidence = squeeze_conf
    active_type = "squeeze_bull"
```

ADX - https://en.wikipedia.org/wiki/Average_directional_movement_index

This is a trend strength indicator but, as with the other indicators, we are using it to detect reversal by way of signal decreasing.

The cell prints the raw ADX value, 0 to 100, not a percentage. Below 20 the cell reads Rng and the figure, with no arrow. The vote itself is decided by which directional line is on top, at any ADX, so a ranging market still

casts a directional vote even though the cell shows no arrow. ADX and DMI carries the formula.

`src/trading/indicators/adx.py` — the vote

```
if bull_dominant and strong_trend:
    direction = SignalDirection.BULLISH
    confidence = max(0.0, min(1.0, (adx - 35) / 30 + 0.5))
elif bear_dominant and strong_trend:
    direction = SignalDirection.BEARISH
    confidence = max(0.0, min(1.0, (adx - 35) / 30 + 0.5))
elif bull_dominant:
    direction = SignalDirection.BULLISH
    confidence = max(0.0, min(0.5, adx / 70))
elif bear_dominant:
    direction = SignalDirection.BEARISH
    confidence = max(0.0, min(0.5, adx / 70))
else:
    direction = SignalDirection.NEUTRAL
    confidence = 0.0
```

STrd - https://www.tradingview.com/support/solutions/43000634738-supertrend/

Supertrend is based upon Average True Range and, unsurprisingly further assists in detecting reversals.

The cell prints an arrow and a percentage. The vote is simply which side of the sticky band the price closed on, so this voter is never neutral once it reads at all. A band that has fallen to zero makes it abstain instead. Supertrend carries the formula.

`src/trading/indicators/supertrend.py` — the vote

```
direction = SignalDirection.BULLISH if curr_bull else SignalDirection.BEARISH
```

ZSc - Z Score - https://www.tradingview.com/script/KSMvIkvh-Z-Score-Predictive-Zones-AlgoPoint/

Our current indicator is the basic or classic Z Score indicator but this will be upgraded. But, as before, we are after more data to confirm reversals.

The cell prints a signed z-value to one decimal, not a percentage, and issue #316 made that value the smoothed one. Past the level the indicator remembers the vote is strong and contrarian: high votes bearish, low votes bullish. Until a turn is remembered that level is two deviations either way. Between 1.5 and 2 it votes the same way at a fixed 0.25 confidence, and inside 1.5 it casts no vote. Z-Score carries the formula.

`src/trading/indicators/zscore.py` — the thresholds

```
extreme_high = z > 3.0
strong_high = z > 2.0
mild_high = z > 1.5
extreme_low = z < -3.0
strong_low = z < -2.0
mild_low = z < -1.5
```

The cell now prints a smoothed z, and the vote compares it against levels the market itself set. The indicator remembers where the z-score last turned around, averages those turning points, and uses each average as the level that decides a vote. Until it has remembered a turn, the fixed level of two deviations decides, exactly as before. Each average is also projected back into a price, which is the upper and lower zone the hover text carries.

`src/trading/indicators/zscore.py` — the level that decides, and its price

```
target_z_high = self.reversal_threshold if peak_z is None else peak_z
target_z_low = -self.reversal_threshold if trough_z is None else trough_z

resistance_price = sma + target_z_high * std
support_price = sma + target_z_low * std
```

KER -

Kaufman Efficiency Ratio. It measures how much of the distance the price travelled became progress in one direction. A reading near 1.0 is a straight move and a reading near 0.0 is noise. A low ratio marks the market Acervator prefers. The Efficiency Ratio Regime gate refuses a SCRUM at 0.70 and above, and refuses one again at 0.05 and below.

RSI -

Relative Strength Index. It reports the share of recent movement that ran upward, on a 0 to 100 scale. Above 70 votes bearish, below 30 votes bullish, and between the two it casts no vote. It runs the same published Wilder recursion as the Stochastic RSI above, computed separately inside its own module. Its weight is 0.8, the lightest in the panel, which it shares with Volume.

The twelve voters

Reference. The operator's own reading of each column opens this part. What follows is the code behind those columns.

Functional. One weight map names the twelve indicators that vote, and one method builds exactly those twelve from it. The engine builds the list once and runs it in order on every tick. "Candles needed" in the table below is the tape length each indicator wants before it will read anything. Give it less and it abstains outright rather than guessing.

`src/trading/ta_engine.py` — `VotingEngine._create_indicators`

```
def _create_indicators(self) -> list:
    """Instantiate all indicator instances with configured weights."""
    return [
        BollingerBands(weight=self.weights.get("bollinger_bands", 1.0)),
        VortexIndicator(weight=self.weights.get("vortex", 0.9)),
        MACD(weight=self.weights.get("macd", 1.2)),
        StochasticRSI(weight=self.weights.get("stochastic_rsi", 1.0)),
        IchimokuCloud(weight=self.weights.get("ichimoku", 1.1)),
        VolumeAnalysis(weight=self.weights.get("volume", 0.8)),
        SlingshotIndicator(weight=self.weights.get("slingshot", 1.0)),
        ADXIndicator(weight=self.weights.get("adx", 1.0)),
        KaufmanERIndicator(weight=self.weights.get("kaufman_er", 1.0)),
        SupertrendIndicator(weight=self.weights.get("supertrend", 1.0)),
        ZScoreIndicator(weight=self.weights.get("zscore", 0.9)),
        RSIIndicator(weight=self.weights.get("rsi", 0.8)),
    ]
```

Design intention. The weights rank the voters by how much they are trusted. MACD is heaviest at 1.2 and Ichimoku next at 1.1. RSI and Volume are lightest at 0.8, RSI because it overlaps the Stochastic RSI and Volume because it is meant to confirm rather than lead. Hand the engine a different map and the panel retunes without an indicator being touched.

`src/trading/ta_engine.py` — `VotingEngine.__init__`

```
self.weights = weights or DEFAULT_WEIGHTS.copy()
self._indicators = self._create_indicators()
```

Panel column	Voter	Weight	Module under `src/trading/indicators/`	Candles needed
BB	`bollinger_bands`	1.0	`bollinger.py`	20
VTX	`vortex`	0.9	`vortex.py`	15
MACD	`macd`	1.2	`macd.py`	35
SRsi	`stochastic_rsi`	1.0	`stochastic_rsi.py`	36
Ichi	`ichimoku`	1.1	`ichimoku.py`	79
Vol	`volume`	0.8	`volume.py`	25
Sling	`slingshot`	1.0	`slingshot.py`	52
ADX	`adx`	1.0	`adx.py`	30
STrd	`supertrend`	1.0	`supertrend.py`	12
ZSc	`zscore`	0.9	`zscore.py`	51
KER	`kaufman_er`	1.0	`kaufman_er.py`	12
RSI	`rsi`	0.8	`rsi.py`	15

Functional. Count the files in the indicator folder and you get 23, not 12. Eleven of them cast no vote. Two carry the shared maths and the signal vocabulary. Three are measurements other code reads. Six are structural detectors that feed the bot's own pattern reads rather than the panel. Twelve is the count of voters, and twelve is the right count.

```
shared        helpers.py, types.py
measurements  atr.py, bb_proximity.py, heikin_ashi.py
detectors     fvg.py, landing_strip.py, m_top.py, w_bottom.py,
              spring.py, macd_taper.py
```

Design intention. One module holds one indicator, and a module that casts no vote still lives here so the voters can read it.

The per-module rule sentences, overtaken

True today: no check in the tree holds the rule. The repository tracks one Python module under `tests/`, its own conftest, so the per-module check and its two assertions are gone.

The package layout carries the rule instead. Each indicator is one module, and `src/trading/ta_engine.py` imports each indicator class or detector function by name from its own module.

Two imports cross between the indicator modules, and both reach a module in the measurements row above rather than a voter. `bb_proximity.py` and `landing_strip.py` each read the Heikin-Ashi measurement. Beside those, every module reads the shared helpers and types. No voter imports another voter.

Indicator formulae

Each block below carries the published formula for reference, then the code that computes it, then what the code does today and what the reading is meant to give the platform. The operator's own reading of each indicator opens this part and is not repeated here.

Every indicator computes its own maths from candles alone.

True today: no voter borrows another voter's arithmetic, and no check in the tree holds the rule. The twelve voters above names the module layout that holds it instead.

Where the code departs from the published formula, the code is the defect. Departures lists the ones found.

Shared terms

Every formula below uses these. All five live in `src/trading/indicators/helpers.py`.

```
SMA(v, n)     mean of the last n values                          _sma_tail
sigma(v, n)   population deviation of the last n values          _stdev_tail
EMA(v, n)     seed = SMA(v, n) at index n-1, then                _ema
              EMA_t = (v_t - EMA_prev) * k + EMA_prev
              k = 2 / (n + 1)
              no value exists before the seed
TR_t          max(H_t - L_t, |H_t - C_prev|, |L_t - C_prev|)     _true_range
              the first bar's TR is H - L
Wilder(v, n)  seed = mean of the first n, then                   (per module)
              W_t = (W_prev * (n - 1) + v_t) / n
```

Functional. True Range gives one bar per candle. The first bar is simply high minus low, because the other two ways of measuring it need a previous close and the first bar has none. Each indicator then takes the slice its own formula covers. ATR and Supertrend take the whole series. ADX and Vortex drop the first bar, because

directional movement needs a bar before it. The moving average helper answers nothing at all below its seed, rather than zero, so reading it too early fails loudly instead of returning a plausible wrong number.

`src/trading/indicators/helpers.py` — `_true_range`

```
tr = [candles[0].high - candles[0].low]
for i in range(1, len(candles)):
    c = candles[i]
    prev_close = candles[i - 1].close
    tr.append(
        max(c.high - c.low, abs(c.high - prev_close), abs(c.low - prev_close))
    )
return tr
```

Design intention. One definition per shared term, held in one place, so five indicators cannot quietly drift apart on the same arithmetic. Slingshot is the one deliberate exception. It computes its own true range, and its first bar agrees with this one.

`src/trading/indicators/slingshot.py` — the exception, written out

```
# Carter's Keltner leg needs True Range; `tr_all` computes it
# without `ATRIndicator`.
tr_all = [candles[0].high - candles[0].low]
```

Bollinger Bands

Published formula, for reference:

```
Middle    = SMA(close, 20)
Upper     = Middle + 2.0 * sigma(close, 20)
Lower     = Middle - 2.0 * sigma(close, 20)
%B        = (close - Lower) / (Upper - Lower)
BandWidth = (Upper - Lower) / Middle
```

Functional. The vote is a reading of where the last close sits between the two bands. Zero is the lower band, one is the upper, and the panel and the gates both call that number the band position. A second method returns all three bands for every candle so a chart can draw them, and answers nothing for the first nineteen. Give it fewer than twenty closes, or a window with no range at all, and it abstains.

`src/trading/indicators/bollinger.py` — `BollingerBands.compute`

```
sma = _sma_tail(closes, self.period, tail=self.period)
std = _stdev_tail(closes, self.period, tail=self.period)

mid = sma[-1]
upper = mid + self.std_dev * std[-1]
lower = mid - self.std_dev * std[-1]
price = closes[-1]
# BandWidth = (Upper - Lower) / Middle. `candles_from_raw` admits
# only positive closes, so `mid` cannot be zero.
band_width = (upper - lower) / mid
```

Design intention. The bands are the boundary a trade fires at. That is the operator's own reading, and the band position is the one number that carries it into the engine. Four gates read it directly and both overrides arm on it, so this reading has more reach into a trade decision than any other voter.

`src/trading/gate_chain.py` — two of the four, in the scrum chain

```
MidlineGate(side="scrum"),
TargetFiresGate(),
BBProximityGate(side="scrum"),
```

`src/trading/gate_chain.py` — and the other two, in the fold chain

```
MidlineGate(side="fold"),
SmartCeilingGate(),
BBProximityGate(side="fold"),
```

Vortex Indicator

Published formula, for reference:

```
VM+_t = |H_t - L_(t-1)|
VM-_t = |L_t - H_(t-1)|
VI+   = sum(VM+, 14) / sum(TR, 14)
VI-   = sum(VM-, 14) / sum(TR, 14)
```

Functional. Three totals are taken over each fourteen-bar window: upward movement, downward movement, and true range. A window whose true range totals zero, which is what a halted market gives, is skipped rather than divided by. The two movement totals are then divided by the true-range total, so both lines are dimensionless and 1.0 is dead parity. The vote reads the crossover of the two lines.

`src/trading/indicators/vortex.py` — `VortexIndicator.window_sums`

```
for i in range(1, n_c):
    vm_plus.append(abs(candles[i].high - candles[i - 1].low))
    vm_minus.append(abs(candles[i].low - candles[i - 1].high))
```

`src/trading/indicators/vortex.py` — `VortexIndicator.lines`

```
plus_series[i] = sum_vp / sum_tr_window
minus_series[i] = sum_vm / sum_tr_window
```

Design intention. The operator uses the Vortex to catch a reversal early, and watches for the signal to saturate at one polarity or the other. The vote follows the two published lines, and nothing overwrites it once the crossover has decided. A saturated upward line caps the vote's confidence at 0.45 and leaves the direction alone. The saturated downward line is published in the reading's own detail, where nothing reads it, so Acervator adds no reading of its own on top of the published lines here, and Departures no longer carries one.

`src/trading/indicators/vortex.py` — `VortexIndicator.compute`, the one cap the vote carries

```
if vip_at_ceiling:
    confidence = max(0.0, min(confidence, 0.45))
```

MACD

Published formula, for reference:

```
MACD Line = EMA(close, 12) - EMA(close, 26)
Signal    = EMA(MACD Line, 9)
Histogram = MACD Line - Signal
```

Functional. One method builds all three series and the vote reads the last two bars of them. What you are looking at is the gap between a fast and a slow average of the price, and how quickly that gap is changing. The signal line is an average of the MACD line rather than of the candles, so its first real value does not arrive until 33 bars into the tape. A series with a hole in the middle raises rather than being handed on.

`src/trading/indicators/macd.py` — `MACD._lines`

```
ema_fast = _ema(closes, self.fast)
ema_slow = _ema(closes, self.slow)
macd_line: _Line = [
    None if (f is None or s is None) else f - s
    for f, s in zip(ema_fast, ema_slow)
]
```

`src/trading/indicators/macd.py` — the signal line and the histogram

```
signal_line: _Line = [None] * first
signal_line.extend(_ema(real, self.signal_period))
histogram: _Line = [
    None if (m is None or g is None) else m - g
    for m, g in zip(macd_line, signal_line)
]
```

Design intention. The operator watches for a top-of-the-hill shape in either polarity, timed against a Bollinger band threshold. The histogram is the series that shape reads from, and the vote reads the last two of its bars, so a turn is caught on the bar it happens rather than a bar later.

`src/trading/indicators/macd.py` — the two bars the vote reads

```
macd_line, signal_line, histogram = self._lines(closes)

curr_hist = histogram[-1]
prev_hist = histogram[-2]
curr_macd = macd_line[-1]
```

The confidence divides the histogram by a share of the last close, so the same crossover scores the same on a four-figure asset and on one worth a few millionths of a dollar.

Stochastic RSI

Published formula, for reference:

```
StochRSI = (RSI - min(RSI, 14)) / (max(RSI, 14) - min(RSI, 14))
%K       = SMA(StochRSI * 100, 3)
%D       = SMA(%K, 3)
```

Functional. The module runs Wilder's recursion over the closes to build its own RSI series, then places each RSI value inside its own fourteen-value range. The vote reads the crossover of the two smoothed lines. The result saturates at 0 and at 100 far more often than plain RSI does, and that saturation is what the panel is watching for. A window whose RSI never moved is caught before the division, and the vote abstains on it rather than dividing by that spread.

`src/trading/indicators/stochastic_rsi.py` — `StochasticRSI.rsi_values`

```
avg_gain = (avg_gain * (self.rsi_period - 1) + gains[i]) / self.rsi_period
avg_loss = (avg_loss * (self.rsi_period - 1) + losses[i]) / self.rsi_period
```

`src/trading/indicators/stochastic_rsi.py` — `StochasticRSI.stoch_ratios`

```
window = rsi_values[i - self.stoch_period + 1 : i + 1]
low = min(window)
ratios.append((rsi_values[i] - low) / (max(window) - low))
```

Design intention. This is the operator's second oscillator, and he looks for a long run at one polarity to line up with the other voters. Rescaling RSI against its own recent range is what makes those runs stand out at all. A window that never moved is reported as flat rather than hidden, so the caller can tell a real zero from a market that stood still.

`src/trading/indicators/stochastic_rsi.py` — what the caller is handed

```
return start, ratios, flat
```

Ichimoku Cloud

Published formula, for reference:

```
Tenkan   = (highest high 9  + lowest low 9)  / 2
Kijun    = (highest high 26 + lowest low 26) / 2
Senkou A = (Tenkan + Kijun) / 2        plotted 26 bars forward
Senkou B = (highest high 52 + lowest low 52) / 2   plotted 26 forward
Chikou   = close                       plotted 26 bars back
```

Functional. Every line here is a midpoint: the highest high and the lowest low of a window, averaged. A window that has not closed yet answers nothing rather than zero. Five values come back for each candle. Two of them draw the cloud twenty-six bars ahead of the last candle, and that is the only forward-looking reading anywhere in the panel. The lines are returned at their own bar, so the twenty-six bar shift belongs to whatever draws them.

`src/trading/indicators/ichimoku.py` — `IchimokuCloud.lines`

```
tenkan = self._span_mid(candles, i, self.tenkan)
kijun = self._span_mid(candles, i, self.kijun)
senkou_b = self._span_mid(candles, i, self.senkou_b)
senkou_a = (
    (tenkan + kijun) / 2.0
    if (tenkan is not None and kijun is not None)
    else None
)
out.append((tenkan, kijun, senkou_a, senkou_b, candles[i].close))
```

Design intention. The operator calls this one of the few predictive indicators and uses it to sharpen reversal-zone detection. The forward cloud is the half of the indicator that does that work. A window that has not closed answers nothing rather than zero, so an unfinished cloud can never be mistaken for a flat one.

`src/trading/indicators/ichimoku.py` — `IchimokuCloud._span_mid`

```
if end < period - 1:
    return None
return self._mid(candles, end - period + 1, period)
```

The cloud thickness divides by the price bare. Every candle reaching this point has already been refused unless its price is above zero, so the division needs no guard of its own.

Volume

Published formulae, for reference:

```
OBV_t  = OBV_prev + V_t   if C_t > C_(t-1)
         OBV_prev - V_t   if C_t < C_(t-1)
         OBV_prev         otherwise
TP_t   = (H_t + L_t + C_t) / 3
MF_t   = TP_t * V_t
MFI    = 100 - 100 / (1 + sum(MF where TP rose, 14)
                        / sum(MF where TP fell, 14))
CLV_t  = ((C_t - L_t) - (H_t - C_t)) / (H_t - L_t)
CMF    = sum(CLV * V, 20) / sum(V, 20)
A/D_t  = A/D_prev + CLV_t * V_t
```

Functional. The Volume column is the one voter the operator does not describe in his list above, so this is its only description in the manual. Three published measures are computed separately and scored together. The vote answers one question: does the volume traded agree with the direction the price moved. A price low that On-Balance Volume refuses to match is the divergence this module scores highest. Where neither money-flow bucket was ever credited, the Money Flow Index answers 50 rather than 100, so a market that never moved does not read as maximum overbought.

`src/trading/indicators/volume.py` — `VolumeAnalysis._obv`

```
obv = [0.0]
for i in range(1, len(candles)):
    if candles[i].close > candles[i - 1].close:
        obv.append(obv[-1] + candles[i].volume)
    elif candles[i].close < candles[i - 1].close:
        obv.append(obv[-1] - candles[i].volume)
    else:
        obv.append(obv[-1])
```

`src/trading/indicators/volume.py` — `VolumeAnalysis._mfi`

```
if pos_mf <= 0.0 and neg_mf <= 0.0:
    return 50.0
if neg_mf <= 0.0:
    return 100.0
return 100.0 - 100.0 / (1.0 + pos_mf / neg_mf)
```

Design intention. Volume confirms or contradicts the other eleven. It does not lead them. Its weight is 0.8, the lightest in the panel alongside RSI, and that number is the intention written down.

`src/trading/ta_engine.py` — the weight that says so

```
"volume": 0.8,
```

Both tests in the block above compare a money flow against zero. Money flow is a price times a volume and carries no fixed scale, so only an exact zero can stand as the test.

Slingshot

Published formula, for reference:

```
UpperBB = SMA(close, 20) + 2.0 * sigma(close, 20)
LowerBB = SMA(close, 20) - 2.0 * sigma(close, 20)
UpperKC = SMA(close, 20) + 1.5 * SMA(TR, 20)
LowerKC = SMA(close, 20) - 1.5 * SMA(TR, 20)
sqzOn   = (LowerBB > LowerKC) and (UpperBB < UpperKC)
delta_t = C_t - (donchian_mid_20 + SMA(close, 20)) / 2
val     = linreg(delta, 20, 0)
```

Functional. The squeeze is on while the Bollinger band sits entirely inside the Keltner Channel. When it opens again, a fitted momentum line gives the sign of the release, and that sign is the vote. The module computes its own true range rather than calling the shared helper, and its first bar agrees with the shared one anyway.

`src/trading/indicators/slingshot.py` — `SlingshotIndicator.compute`

```
rangema = trma_v[i] if trma_v[i] is not None else 0.0
up_kc = mid + rangema * self.kc_mult
lo_kc = mid - rangema * self.kc_mult
sqz_on = (lo > lo_kc) and (up < up_kc)
seg = [
    deltas[k] for k in range(max(delta_lo, i - self.bb_period + 1), i + 1)
]
val = self._linreg_endpoint(seg)
```

Design intention. The operator wants a precise reading of a reversal zone, and he names Chris Moody's Slingshot system as the source. The code does not implement that system. What is in the module is John Carter's TTM Squeeze, with Bollinger's own rules 6 and 8 for the snapback. Moody's method is an EMA trend and pullback with no band, no channel and no squeeze at all. The module says so itself, in its own docstring. The column keeps the name he gave it, and the name and the code are two different indicators.

`src/trading/indicators/slingshot.py` — the module stating it

```
# LazyBear's KC shares the BB basis; rangema is the SMA of
# True Range over the same period.
```

In development. Which of the two indicators the column should hold is the operator's decision, not a coding one, so nothing is proposed here.

ADX and DMI

Published formula, for reference:

```
up_t   = H_t - H_(t-1)
down_t = L_(t-1) - L_t
+DM_t  = up_t   if up_t > down_t and up_t > 0     else 0
-DM_t  = down_t if down_t > up_t and down_t > 0   else 0
+DI    = 100 * Wilder(+DM, 14) / Wilder(TR, 14)
-DI    = 100 * Wilder(-DM, 14) / Wilder(TR, 14)
DX     = 100 * |+DI - -DI| / (+DI + -DI)
ADX    = Wilder(DX, 14)
```

Functional. One smoothing recursion carries every line in this indicator. The reading tells you how committed a move is, and says nothing about which way it points. Below 20 the market is ranging. Above 35 it is trending hard. The indicator abstains when its smoothed true range is not above zero, so a market that moved at all still gets a vote however little the asset costs.

`src/trading/indicators/adx.py` — `ADXIndicator._wilder_smooth`

```
    if len(values) < period:
        return [0.0] * len(values)
    result = [0.0] * (period - 1)
    result.append(sum(values[:period]) / period)
    for v in values[period:]:
        result.append((result[-1] * (period - 1) + v) / period)
    return result
```

Design intention. The operator reads ADX for trend strength, and he watches it fall rather than rise, because a falling reading marks a trend running out of road. The engine carries the other half of that: a strong trend holds a scrum back rather than letting it fire into a move that is still running.

`src/trading/gate_chain.py` — `ADXTrendSuppressionGate.evaluate`

```
    if ctx.adx < self.adx_threshold:
        return GateResult(passed=True)
```

Issue #414 owned the abstain threshold, the correction landed, and the issue is closed.

Supertrend

Published formula, for reference:

```
ATR     = Wilder(TR, 10)
hl2_t   = (H_t + L_t) / 2
rawUp   = hl2 + 3.0 * ATR
rawLow  = hl2 - 3.0 * ATR
Upper_t = rawUp  if rawUp < Upper_prev  or C_(t-1) > Upper_prev  else Upper_prev
Lower_t = rawLow if rawLow > Lower_prev or C_(t-1) < Lower_prev  else Lower_prev
bullish while C_t >= Lower_t; turns bullish again when C_t > Upper_t
```

Functional. A band here is sticky. It moves toward the price and refuses to move away from it, unless the price closed past it last bar. What comes out is a line that trails under the price in an uptrend and over it in a downtrend, and the bar where price crosses that line is the flip.

`src/trading/indicators/supertrend.py` — `SupertrendIndicator.compute`

```
    final_ub = (
        raw_ub
        if raw_ub < prev_ub or candles[i - 1].close > prev_ub
        else prev_ub
    )
    final_lb = (
        raw_lb
        if raw_lb > prev_lb or candles[i - 1].close < prev_lb
        else prev_lb
    )
```

Design intention. Supertrend rests on Average True Range and helps the operator spot a reversal. The flip is the event that does that work, and the line either side of it is context. The bullish test and the bearish test differ on purpose, so a trend has to be broken outright before the line turns.

`src/trading/indicators/supertrend.py` — the asymmetric flip test

```
prev_bull = st[-1]
if prev_bull:
    curr_bull = candles[i].close >= final_lb
else:
    curr_bull = candles[i].close > final_ub
```

The band this indicator measures distance from is derived rather than read off the tape, and it can genuinely reach zero once the average true range passes the midpoint over the multiplier. The indicator abstains outright on that bar instead of dividing by it.

`src/trading/indicators/supertrend.py` — the abstention

```
if st_line <= 0.0:
    return Signal(
        "supertrend",
        timeframe,
        SignalDirection.NEUTRAL,
        0.0,
        self.weight,
        abstained=True,
    )
```

Z-Score

Published formula, for reference:

```
mean  = SMA(close, 50)
sigma = population deviation of the same 50 closes
Z     = (close - mean) / sigma
```

Functional. The reading is how many standard deviations the price sits from its own fifty-bar mean, and the vote reads the extremes of that. Fifty bars is longer than the twenty the Bollinger Bands use, so a stretch those bands have already absorbed still shows up here. A window with almost no spread at all abstains rather than dividing by it.

`src/trading/indicators/zscore.py` — `ZScoreIndicator.compute`

```
closes = [c.close for c in candles[-self.period :]]
sma = sum(closes) / self.period
variance = sum((c - sma) ** 2 for c in closes) / self.period
std = variance**0.5
```

Design intention. The operator calls this the classic Z-Score, asked for the upgrade issue #316 landed, and uses it to confirm a reversal. The longer window is why it earns a column of its own beside the Bollinger Bands.

One formula, written once, for both bars. The current bar and the previous bar each divide by their own deviation bare, and a guard above each refuses a deviation under the same threshold before the division runs.

`src/trading/indicators/zscore.py` — both divisions

```
z = (candles[-1].close - sma) / std
```

```
z_prev = (candles[-2].close - s2) / std2
```

Published method. The source the operator named publishes two formulae. The first is the classic score above. The second turns a score back into a price, so a level the market has reversed at can be drawn as a zone that widens and narrows with the deviation.

```
Projected Price = Mean + (Target Z * Standard Deviation)
```

The same page names the noise reduction: the raw score is smoothed by a volume weighted average before anything reads it. Turning points are then found in that smoothed series, kept only when they reach a minimum score, and the last few of each side are averaged into the two levels the vote uses.

Both bars now read one window through one expression, so the two spellings that disagreed cannot come back. A window whose prices never moved returns nothing at all, which is how the reading abstains rather than shifting.

`src/trading/indicators/zscore.py` — one window, one expression, every reader

```
def _window_zscore(closes: list, close: float) -> Optional[tuple]:
    if _window_has_no_range(closes):
        return None
    count = len(closes)
    mean = sum(closes) / count
    deviation = (sum((c - mean) ** 2 for c in closes) / count) ** 0.5
    return mean, deviation, (close - mean) / deviation
```

What the page does not publish. It names four settings and gives a default value for none of them. Three take a number this file already used or the page itself quotes, and one is chosen here and marked as such.

Setting	Value	Where the value comes from
Z-Score Length	50	the period this file already used
Reversal Threshold	2.0	the strong level this file already used, and the level the page quotes as the classic one
Pivot bars	1	a turn against the bar each side; the page gives noise reduction to the smoothing instead
Lookback Depth	5	chosen here, because the page publishes no value and this file had none

Kaufman Efficiency Ratio

Published formula, for reference:

```
ER = |C_t - C_(t-10)| / sum(|C_i - C_(i-1)|, 10)
```

Functional. Net displacement divided by the distance travelled to get there. 1.0 is a straight line, 0.0 is pure noise, and both halves read the same eleven closes. A window where the price never moved travels exactly

zero distance, and the indicator abstains on that rather than reporting a regime it never measured.

`src/trading/indicators/kaufman_er.py` — `KaufmanERIndicator.compute`

```
closes = [c.close for c in candles[-(self.period + 1) :]]
net_change = abs(closes[-1] - closes[0])
price_travel = sum(
    abs(closes[i] - closes[i - 1]) for i in range(1, len(closes))
)
```

Design intention. A low ratio marks the market Acervator prefers, in the operator's own words above. The engine acts on that at both ends: a scrum is refused when the market is running too straight to mean-revert, and refused again when it is too noisy to hold an edge.

The division that produces the ratio is bare. A guard four lines above returns early when the distance travelled is not above zero, so the denominator is already proved positive by the time the division runs.

`src/trading/indicators/kaufman_er.py` — the division

```
er = net_change / price_travel
```

RSI

Published formula, for reference:

```
gain_t  = max(C_t - C_(t-1), 0)
loss_t  = max(C_(t-1) - C_t, 0)
AvgGain = Wilder(gain, 14)
AvgLoss = Wilder(loss, 14)
RS      = AvgGain / AvgLoss
RSI     = 100 - 100 / (1 + RS)

AvgLoss = 0 and AvgGain > 0   ->  RSI 100
AvgLoss = 0 and AvgGain = 0   ->  no reading, the indicator abstains
```

Functional. The reading is the share of recent movement that ran upward, on a 0 to 100 scale. Above 70 votes bearish, below 30 votes bullish, and anything between casts no vote. A bar with neither a gain nor a loss abstains outright.

`src/trading/indicators/rsi.py` — `RSIIndicator.compute`

```
if rsi > 70:
    direction = SignalDirection.BEARISH
elif rsi < 30:
    direction = SignalDirection.BULLISH
else:
    direction = SignalDirection.NEUTRAL
```

Design intention. RSI is the plainest voter in the panel, and it overlaps the Stochastic RSI on purpose. Its weight is 0.8, which stops the same evidence being counted twice at full strength.

One number is read one way here. The vote tests it, the overbought and oversold flags in the same dictionary test it, and nothing cuts it before either of them, so an RSI of 70.003 sets overbought and votes bearish on the same bar.

`src/trading/indicators/rsi.py` — the one index the dictionary carries

```
return {
    "rsi": rsi,
    "rs_indeterminate": rs_indeterminate,
    "overbought": rsi > 70,
    "oversold": rsi < 30,
```

Rounding belongs to display, not to a comparison, and it is done where a figure is drawn. Issue #414 carried this and is closed.

Departures from the published maths

Functional. No departure from the published maths remains. Every voter now reads the same on an asset worth four figures and on one worth a few millionths of a dollar, because no division and no threshold in the twelve carries an absolute constant against a quantity measured in the asset's own price units. Where a denominator can genuinely reach zero the indicator abstains on that bar rather than adding a constant to it.

- The Vortex direction follows the upward line against the downward one, and nothing overwrites it after the crossover has decided. The two lines themselves match Botes and Siepman exactly, and no reading of Acervator's own is laid on top of them.
- The RSI vote and the overbought and oversold flags beside it read one number. The index is not cut before any of them tests it, so the label and the vote cannot disagree about the same bar.

Supertrend is the one place an abstention stands in for a division. Its lower band is derived from the average true range rather than read off the tape, so it can reach zero on a real market, and the indicator refuses to vote on that bar.

Design intention. An indicator has to read the same on every asset, because one threshold serves the whole fleet.

The four scale-invariance files, overtaken

OVERTAKEN, quoted whole:

> *"Four test files hold that property for the twelve, one shape at a time, by running the same tape at several price scales and comparing the readings."*

```
test_bollinger_and_slingshot_are_scale_invariant
test_vortex_and_landing_strip_are_scale_invariant
test_macd_and_kaufman_er_are_scale_invariant
test_the_last_eight_indicators_are_scale_invariant
```

True today: no check in the tree holds the property. None of the four files is there, and the repository tracks one Python module under `tests/`, its own conftest. The runs recorded in the entries further down this page hold the property instead, each naming the scales it drove and the readings it compared. The bounds the readings must stay inside are in `src/trading/ta_invariants.py`.

Issue #414 owned the two readings above and is closed. Each one now reports the published maths with nothing added, so neither needs a replacement chosen for it. The entries dated below record what each repair changed and what it was measured on.

`src/trading/indicators/vortex.py` — `VortexIndicator.compute`, the crossover the direction follows

```
bullish_cross = prev_sep <= 0 and separation > 0
bearish_cross = prev_sep >= 0 and separation < 0
```

2026-09-08 15:12 - #414 - the ratio the gate reads

The published Efficiency Ratio is Kaufman's, unchanged here: the distance between the first and last close of the window, divided by the distance the price travelled to cover it. The Kaufman entry above states it and the code computes it.

```
ER = |C_t - C_(t-10)| / sum(|C_i - C_(i-1)|, 10)
```

The indicator gave three other parts of the platform that ratio rounded to four decimal places. One of them is the Efficiency Ratio Regime gate, which refuses a scrum at 0.05 and below and at 0.70 and above, so the gate compared a shortened number against its two thresholds. The indicator now publishes the ratio it computed.

`src/trading/indicators/kaufman_er.py` — what the reading carries

```
details={
    "er": er,
    "er_prev": er_prev,
```

Three parts of the platform read that number and each builds the same gate input from it: the live bot's tick, the Simulator's back test, and the ATA gate scan. Nothing on any screen changes, because the panel cell, the hover text and the ATA-SMP sentence each cut the number to two, three and four decimals of their own.

`src/trading/scrumming_bot.py` — the gate input, one of the three

```
efficiency_ratio=_extract_signal_detail(summary, "kaufman_er", "er", 0.0),
```

Measured over 71 recorded Coinbase tapes, run at a price of three millionths of a dollar and again at a four-figure price. The run drove 37,380 readings through the real voting engine and the real scrum chain. The gate changed its mind on five of them, in both directions: three where the rounding had pulled a ratio just above 0.05 down onto the threshold, and two where it had pulled a ratio near zero onto the sentinel that tells the gate no reading arrived.

```
A22 2025, bar 264, both price scales
    before   gate reads 0.05                  scrum blocked
    after    gate reads 0.05001050640890935   scrum allowed

A22 2025, bar 263, both price scales
    before   gate reads 0.2257                scrum allowed
    after    gate reads 0.2256565656565654    scrum allowed

A14 2025, bar 233, both price scales
    before   gate reads 0.0                   scrum allowed
    after    gate reads 2.983774235682565e-05 scrum blocked
```

The same run read the other eleven voters back over one tape at 88 bars of each price scale, and every reading is identical, which is what a repair to one indicator has to look like.

2026-09-08 15:54 - #414 - when ADX first has something to say

Wilder's ADX needs two runs of bars before it exists. The first is fourteen bars of true range and directional movement, which seed the smoothing. The second is fourteen DX values, which the ADX is the average of. Twenty-eight bars, and not one more. The ADX and DMI entry above states the formula and the code computes it.

```
+DI    = 100 * Wilder(+DM, 14) / Wilder(TR, 14)
-DI    = 100 * Wilder(-DM, 14) / Wilder(TR, 14)
DX     = 100 * |+DI - -DI| / (+DI + -DI)
ADX    = Wilder(DX, 14)          first value at bar 28
```

The indicator refused to vote until thirty bars had arrived. The extra two came from the first upload and never had a reason. On those two bars the market had moved, the ADX existed, and the panel counted eleven voters instead of twelve.

`src/trading/indicators/adx.py` — when the indicator speaks

```
n = len(candles)
# period bars seed the smoothed TR, and period DX values seed the ADX.
if n < self.period * 2:
```

The bar the ADX first exists on has no earlier ADX to compare against. The module read a zero from the padding the smoothing writes ahead of its first real value, so the rising flag on that bar was decided against a number that was never a reading. The readings now start where the padding ends.

`src/trading/indicators/adx.py` — the previous reading

```
adx_values = s_dx[self.period - 1 :]
adx = adx_values[-1] if adx_values else 0.0
p_adx = adx_values[-2] if len(adx_values) >= 2 else adx
```

The indicator also gave every reader its ADX cut to two decimal places, and the directional lines with it. The ADX Trend Suppression gate is one of those readers and compares the number against 30.0. The indicator

now publishes what it computed.

`src/trading/indicators/adx.py` — what the reading carries

```
  details={
      "adx": adx,
      "di_plus": di_plus,
      "di_minus": di_minus,
```

Eight parts of the platform read those three numbers. Three build the same gate input: the live bot's tick, the Simulator's back test and the ATA gate scan. Five display it and each cuts it themselves, so the panel cell still prints a whole number and the ATA-SMP sentence still prints two decimals.

reader	what it shows
`scrumming_bot.py` gate input	the full reading, `23.968874676972366`
`back_test.py` gate input	the full reading
`ata_gate_scan.py` gate input	the full reading
`ata_spm.py` sentence	`trend strength ADX 23.97`, unchanged
`indicator_panel.py` cell	`24`, unchanged
`indicator_panel_surface.py` cell	`24`, unchanged
`indicator_panel_surface.py` hover	`23.969`, was `23.970`
`snapshots.py` bot log line	`(adx 23.9689)`, was `(adx 23.97)`

The bot log printed whatever the indicator published, so it now cuts a decimal reading to four places of its own.

`src/trading/scrumming/snapshots.py` — the bot log cell

```
  reading = cell.get("detail")
  if reading is not None:
      shown = f"{reading:.4f}" if isinstance(reading, float) else reading
```

Measured over 71 recorded Coinbase tapes, run at a price of three millionths of a dollar and again at a four-figure price. The run drove 42,140 readings through the real voting engine and the real scrum chain, from a window of twenty-six bars upward.

```
  abstentions   before 280 per price scale    after 140 per price scale
  readings that moved
      the two recovered bars           280
      the published decimal places  41,580
  scrum verdict changed
      the two recovered bars           136
      the published decimal places       0
```

Every abstention that remains is a window under twenty-eight bars, where Wilder's ADX does not exist. Every one that ended is a window of twenty-eight or twenty-nine bars, where it did.

```
A34 2021, window 27 bars, both price scales
    before   abstained     13 DX values, one short of the fourteen
    after    abstained     the same reason

A34 2021, window 28 bars, both price scales
    before   abstained     the guard asked for 30 bars
    after    ADX 21.847870255087297  bullish at 0.3121

A34 2023, window 28 bars, both price scales
    before   gate reads 0.0                 scrum allowed
    after    gate reads 74.20924187088121   scrum blocked
```

The last row is what the repair changes about a live reading. A bot that has just started, or one whose exchange returned a short window, spent two bars firing scrums into a trend the ADX could already measure. The panel felt it too: on the A34 2021 bar the twelfth voter arriving moved the net score from 0.7437 to 1.0558 and the consensus confidence from 0.1352 to 0.1624, and across the sweep the consensus direction itself changed on 58 readings.

The reading carries no price scale. Every number above is the same at three millionths of a dollar and at a four-figure price, to the last few bits.

The same run read the other eleven voters back over one tape at 88 bars of each price scale, and every reading is identical.

2026-09-08 16:22 - #414 - the MACD reading at a millionth of a dollar

Appel's three numbers are unchanged and the MACD entry above states them. This repair touches what the indicator hands its readers, not how the lines are built.

```
MACD Line = EMA(close, 12) - EMA(close, 26)
Signal    = EMA(MACD Line, 9)
Histogram = MACD Line - Signal
```

All three are prices. The indicator published them cut to six decimal places, which is a fixed step of one millionth of a dollar. A market worth three millionths of a dollar has a whole MACD smaller than that step, so every reader was handed a zero. The same market shape at a four-figure price published real numbers. One threshold serves the whole fleet, so one shape has to read the same on both.

`src/trading/indicators/macd.py` — what the reading carries

```
details={
    "macd_line": curr_macd,
    "signal_line": curr_signal,
    "histogram": curr_hist,
```

Measured over 71 recorded Coinbase tapes, run at a price of three millionths of a dollar and again at a four-figure price. The run drove 3,750 readings through the real voting engine and both real gate chains, from a window of thirty-six bars upward.

```
readings publishing three zeros
    three millionths   before 1,834 of 1,875   after 0 of 1,875
    four figures       before     0 of 1,875   after 0 of 1,875
the MACD line over the histogram, the two scales agreeing
    before        0 of 1,875 pairs
    after     1,875 of 1,875 pairs
readings that moved             3,750 of 3,750
scrum and fold verdicts changed     0 of 3,750
```

One bar, at both prices, before and after:

```
A34 2021, bar 234, a bullish crossover
    before   three millionths   histogram  0.0
             four figures       histogram  1.651648
    after    three millionths   histogram  4.149197409777354e-09
             four figures       histogram  1.6516482592467057
the MACD line over the histogram
    before   three millionths   no value, the division is by zero
             four figures       -20.5541919344
    after    three millionths   -20.5541889248
             four figures       -20.5541889248
```

The confidence never carried a price scale and does not change here. It divides the histogram by a share of the last close, a price over a price, and the two shares now have names of their own.

`src/trading/indicators/macd.py` — the two shares the confidence divides by

```
crossover_scale = abs(closes[-1]) * 0.001
momentum_scale = abs(closes[-1]) * 0.002
```

A close of zero leaves nothing to divide by. The indicator used to raise on such a bar and take the whole voting run down with it, so the other eleven voters never answered either. It now says it has no reading, which is what every other voter does when its own denominator is empty.

`src/trading/indicators/macd.py` — the abstention that replaces the crash

```
if crossover_scale == 0.0 or momentum_scale == 0.0:
    # No scale to divide by, so macd publishes no confidence here.
    return Signal(
        "macd",
        timeframe,
        SignalDirection.NEUTRAL,
        0.0,
        self.weight,
        abstained=True,
    )
```

The bound that checks the reading against itself allowed a slack of two millionths. At three millionths of a dollar that slack is wider than any real MACD, so the bound passed a histogram with the wrong sign and a histogram of zero. It is exact now, and it holds on all 3,750 readings.

`src/trading/ta_invariants.py` — the bound on the reading

```
  lambda d: float(d["histogram"])
  == float(d["macd_line"]) - float(d["signal_line"]),
```

Nine parts of the platform read this indicator. Two read the vote and its strength rather than the numbers, and one builds its own series for the chart.

reader	at three millionths of a dollar
`ta_invariants.py` bound	refuses a wrong histogram, where it passed one
`ta_engine.py` recorder	records the reading, where it recorded three zeros
`ata_spm.py` sentence	`histogram +4.1491974e-09`, was `histogram +0.000000`
`scrumming_v3_sim.py` flags	flip on 1,006 of 1,875 readings, 0 of 1,875 at four figures
`indicator_panel.py` hover	`macd_line: -0.000`, unchanged
`indicator_panel_surface.py` hover	`macd_line: -0.000`, unchanged
`scrumming_bot.py` position boost	reads the vote, unchanged
`ata_gate_scan.py` favour	reads the vote, unchanged
`native_chart.py` chart series	builds its own series, unchanged

The ATA-SMP sentence printed six decimals of a price, so on a micro-priced market it printed nothing but zeros. It now prints eight significant figures, which reads the same at every price.

`src/trading/ata_spm.py` — the sentence one voter writes

```
  "macd": ("MACD", ("histogram",), "histogram {value:+.8g}"),
```

The two hover cells still show three zeros, because the panel cuts every voter's numbers to three decimal places of its own. That format is shared by all twelve and is not this repair.

No gate changes its mind. Neither chain reads these numbers; both read the panel's consensus, and the consensus, the vote, its strength, the abstention flag and the other eleven voters are identical on all 3,750 readings.

2026-09-08 16:52 - #414 - the money flow a halted market never traded

Quong and Soudack's index is published in two forms, and this repair needs both of them.

```
  TP_t = (H_t + L_t + C_t) / 3
  MF_t = TP_t * V_t

  positive money flow = sum of MF over the 14 bars where TP rose
  negative money flow = sum of MF over the 14 bars where TP fell

  MFI = 100 - 100 / (1 + positive / negative)
  MFI = 100 * positive / (positive + negative)
```

The two lines give the same number. The second is defined on one input more than the first. Where every falling bar in the window traded nothing, the negative flow is zero: the first line divides by that zero, and the second answers 100. One hundred is the right answer there, because every dollar that moved was buying.

One input is left over, and neither published line answers it. A window that credited neither bucket gives nothing over nothing. The index does not exist on that bar.

Functional. The module answered 50 on that bar. Fifty is the middle of the scale and it is a number, published to the voting panel, to the ATA-SMP post and to the emitter recorder, about a market the indicator had measured nothing on. Driven through the voting engine on a constructed seventy-minute halt at three millionths of a dollar, the Volume column voted NEUTRAL and still counted in the panel's denominator. On a window whose last fourteen bars traded nothing at all, it voted BULLISH at 0.37 strength.

The index now reports that it has no reading, and the column abstains on that bar. Abstaining is what this module already does when the window's average volume is zero, and what Supertrend, ADX, the Efficiency Ratio and the MACD each do at their own missing denominator.

`src/trading/indicators/volume.py` — `VolumeAnalysis._mfi`

```
    total_mf = pos_mf + neg_mf
    if total_mf <= 0.0:
        return math.nan
    return 100.0 * (pos_mf / total_mf)
```

`src/trading/indicators/volume.py` — `VolumeAnalysis.compute`

```
        mfi = self._mfi(candles, self.mfi_period)
        if math.isnan(mfi):
            # `_mfi` has no money ratio; `mfi_ob` and `mfi_os` read False on nan.
            return Signal(
                "volume",
                timeframe,
                SignalDirection.NEUTRAL,
                0.0,
                self.weight,
                abstained=True,
            )
```

The item cites a money flow below 1e-09 read as zero, on two tests that answered 100 on it. Commit `b27c6cdb` removed both tests on 2026-09-04. Driven through the voting engine on a window at three millionths of a dollar whose falling bars carry a volume of 1e-05, the tree that still holds those tests answers 100.0 and this tree answers 99.999001008.

A money flow can also reach zero by running out of double precision, and that is the one route by which an absent reading could still print as the top of the scale. Measured: at a price of three millionths of a dollar the product of price and volume is exactly zero for any volume at or below 7.9688e-319. The smallest volume in the 411 recorded tablets is 100. No order size the venue can express comes within three hundred orders of magnitude of that threshold, and where the falling side does underflow, the ratio puts the index at 100 to every digit a double carries, which is the correct answer rather than an absent one.

The window with no money flow is not in the recorded data either. All 411 tablets hold daily bars; across 99,582 fourteen-bar windows not one credited neither bucket, the typical price repeated on 127 of 104,925 bars, and the longest run of repeats is five. The fleet computes on five-minute bars, which the tablets do not contain, so the recorded data says nothing about whether a seventy-minute flat window occurs live.

The reading was also published cut to one decimal place. The vote itself always tested the computed number, so no gate ever read the short one, but the panel's hover cell prints three decimals and was handed a number carrying one. It now carries what the module computed.

`src/trading/indicators/volume.py` — the reading

```
  "mfi": mfi,
```

Six parts of the platform read this indicator.

reader	at three millionths of a dollar
`ta_invariants.py` bound	bounds the computed number; no bound applies on an abstention
`ta_engine.py` recorder	records the computed number, and an empty reading on an abstention
`ata_spm.py` sentence	`Money Flow Index 56.5`, unchanged on all 3,750 readings
`indicator_panel.py` hover	`mfi: 56.450`, was `mfi: 56.500`
`indicator_panel_surface.py` hover	`mfi: 56.450`, was `mfi: 56.500`
the panel's consensus	loses one voter from its denominator on an abstention

Neither gate chain is in that list. Both read the panel's consensus rather than any voter's numbers.

No gate changes its mind. Over 71 recorded tapes at two price scales, 3,750 readings and 15,000 gate verdicts, the vote, its strength, the abstention flag, the panel's consensus and the other eleven voters are identical, and both chains fire the same way on every one. Swapping the published form of the ratio moved the index on 1,907 of the 3,750 by at most 2.2e-14, which is the last bits of double arithmetic and vanishes at one decimal place. Publishing it uncut moved the hover cell on 3,710 of 3,750 and the ATA-SMP sentence on none of them.

2026-09-08 17:45 - #414 - the Vortex reports the Vortex

Botes and Siepman published two lines and one reading of them. The lines measure upward and downward movement over the same fourteen bars, each divided by the true range of those bars. The reading is the crossover: the upward line above the downward one is a rising market, and the other way round is a falling one. A downward line far above parity is the strongest falling reading the indicator has.

```
VM+_t = |H_t - L_(t-1)|
VM-_t = |L_t - H_(t-1)|
VI+   = sum(VM+, 14) / sum(TR, 14)
VI-   = sum(VM-, 14) / sum(TR, 14)
```

Functional. The two lines above are exactly as published, and this repair does not touch them. What came out is the four lines that ran after the vote had already been decided. They read the downward line reaching 1.30, called it a coming reversal, lifted the strength to at least 0.60 and rewrote the vote to bullish.

`src/trading/indicators/vortex.py` — `VortexIndicator.compute`, removed

```
if vim_at_ceiling:
    confidence = min(1.0, max(confidence, 0.60))
    direction = SignalDirection.BULLISH
```

Seventy-one recorded Coinbase tapes were run through the voting engine at three millionths of a dollar and again at 1234.0, 3,750 readings in all. The downward line reached 1.30 on 78 of them, spread over 30 tapes. On every one of those 78 the published lines said falling and the overwrite published rising. The strength was already at or above 0.60 on all 78, so the 0.60 lift changed nothing and only the vote moved.

```
3,750 readings, 71 tapes, two price scales
    VI- at or above 1.30                          78
    vote before the repair, all 78            BULLISH
    vote after the repair, all 78             BEARISH
    strength changed                                0
    votes moved the other way                       0
    VI- range on those rows          1.3014 to 1.4859
```

Design intention. An indicator carries one published set of sums and one published reading of them, and nothing else rides on its name. The ceiling that remains touches strength only and never the vote: an upward line at or above 1.30 caps the strength at 0.45. Both ceiling readings and both floor readings are still published beside the vote, so the panel can show that a line is saturated without the vote claiming the opposite of what the lines say.

`src/trading/indicators/vortex.py` — the ceiling that stayed

```
VX_CEILING_PCT = 130.0

if vip_at_ceiling:
    confidence = max(0.0, min(confidence, 0.45))
```

Seven parts of the platform read this indicator.

reader	on a VI- at or above 1.30
the panel's consensus	one vote moves from rising to falling; moved on 78
`indicator_panel.py` cell	the arrow turns from up to down
`indicator_panel_surface.py` cell	the arrow turns from up to down
`ata_spm.py` sentence	prints the gap between the lines; unchanged on all 78
`ta_invariants.py` bound	bounds both lines at or above zero; unchanged
`ta_engine.py` recorder	records the vote it now casts
`scrumming_bot.py` position boost	pays only at the upper band or the middle, and never reached these rows

`native_chart.py` is not in that list. It draws the two lines from `VortexIndicator.lines`, which this repair does not touch.

The position boost is the one reader the overwrite could have reached and never did. It pays a bullish Vortex 0.12 at the upper band and takes 0.05 back in the middle. A downward line saturating means the price has

fallen hard, and on all 78 rows the band position sat between -0.134 and 0.224, below both zones. The boost paid nothing before the repair and nothing after it.

The gate changes its mind on four windows. Over 71 tapes at two price scales, 3,750 readings and 30,000 gate verdicts, eight verdicts move and every one of them moves the same way: a fold that was refused now fires. No scrum verdict moves. Both windows sit at or below the lower band, where the price is at the bottom and the buy-back belongs.

```
30,000 gate verdicts, both chains, four flag states
    fold refused -> fold fires                      8
    fold fires -> fold refused                      0
    scrum verdicts changed                          0
    windows                                         4
        A22 2022, band position 0.1168, VI- 1.3276
        A23  2026, band position -0.1158, VI- 1.3014
    both windows identical at both price scales
```

Both moves need the fold's own bearish requirement switched on. With that switch off the fold does not consult the panel at all and nothing moves.

The reading was also published cut to four decimal places. No vote and no gate ever read the cut copy, because every branch inside the indicator tests the number it computed. The panel's hover cell prints three decimals and was handed a number carrying four, and the cell printed a different third decimal on 726 of 15,000 readings. It now carries what the indicator computed.

`src/trading/indicators/vortex.py` — the reading

```
"vi_plus": vi_plus,
"vi_minus": vi_minus,
"separation": separation,
"sep_acceleration": sep_acceleration,
```

One example, from A34 2021: the downward line computed 1.0835408442150014 and published 1.0835, so the cell printed `1.083` where the number is `1.084`. The ATA-SMP sentence prints four decimals and matched the cut copy exactly, so it reads the same on all 3,750.

2026-09-08 19:05 - #414 - the RSI label and the RSI vote read one number

Wilder publishes one quantity. It is the share of the last fourteen bars of movement that ran upward, on a nought to a hundred scale. The same number decides the label a reader sees and the vote the panel casts: above seventy is overbought and below thirty is oversold.

```
gain_t  = max(C_t - C_(t-1), 0)
loss_t  = max(C_(t-1) - C_t, 0)
AvgGain = Wilder(gain, 14)
AvgLoss = Wilder(loss, 14)
RS      = AvgGain / AvgLoss
RSI     = 100 - 100 / (1 + RS)
```

Functional. The indicator held two copies of that quantity. It cut one to two decimal places and published it, and it kept the whole one for the overbought and oversold labels. The vote read the cut copy. A reading of 70.003 cuts to seventy exactly, and seventy is not above seventy, so the label said overbought while the vote said neutral, and the two left the indicator together.

`src/trading/indicators/rsi.py` — the cut, removed

```
return {
    "rsi": round(rsi, 2),
    "overbought": rsi > 70,
    "oversold": rsi < 30,
```

One number now leaves the indicator, and the label, the vote, the strength and the published reading all read it. The code block in the RSI section above is the code in this tree. The rounding has gone back to the two places that display a number: the confirming sentence prints two decimals and the panel's hover cell prints three.

`src/trading/indicators/rsi.py` — the reading

```
return {
    "rsi": rsi,
    "rs_indeterminate": rs_indeterminate,
    "overbought": rsi > 70,
    "oversold": rsi < 30,
```

The gap the two copies can disagree in is five thousandths of a point wide, just above seventy and again just below thirty. Seventy-one recorded Coinbase tapes were run through the voting engine at three millionths of a dollar and again at 1234.0, 3,750 readings in all, and not one landed inside it. The closest reading to seventy was 70.06, which was already voting bearish.

```
3,750 readings, 71 tapes, two price scales
    readings inside 70.000 to 70.005                0
    readings inside 29.995 to 30.000                0
    closest reading to seventy                  70.06
    closest reading to thirty                   29.97
    the index ran from                 12.6 to 89.45
```

One bar puts a market inside it. Take any recorded run of forty bars or more and ask what final close lands the index on 70.003. Every one of the 2,901 runs admits such a close, because the index rises with the last price. Most of those bars are large, because most windows sit a long way from seventy: the middle one needs a move of nearly twenty-six per cent. Seventy-two of the runs need a bar of one per cent or less, and the smallest was half a basis point after 271 real days of A21.

```
2,901 recorded runs, the final bar each one needs
    smallest                                    +0.0049%
    middle                                       25.70%
    largest                                     334.53%
    runs needing one per cent or less                72
    runs needing five per cent or less              272
```

The smallest is the one to read. Half a basis point is an ordinary tick on a market that has already run up to the seventy line.

```
A21 2023, 271 recorded bars, then one bar of +0.0049%
    reading published, before                  70.00
    reading published, after                  70.003
    label, before and after                overbought
    vote, before                              NEUTRAL
    vote, after                               BEARISH
    identical at both price scales
```

Design intention. A number that decides something is the number that gets published, and a display rounds it at the moment it is shown. Two copies of one reading is the shape that lets a label and a vote say different things about the same bar while both look correct on their own.

Nothing about the vote moved on the recorded tapes. The reading published now differs from the cut copy on all 3,750, by at most five thousandths of a point; the direction is the same on every one; the strength moves on 180 of them, by at most one ten-thousandth. The panel's consensus moves on 24 readings and the other eleven voters do not move at all.

```
3,750 readings, both price scales
    reading published differs from computed     3,750
    vote direction moved                            0
    vote strength moved                           180
    overbought or oversold moved                    0
    panel consensus moved                          24
    the other eleven voters                         0
```

No gate changes its mind. Over 30,000 verdicts — both chains, four states of the two direction switches — 1,322 fire before the repair and the same 1,322 fire after it. The repair only bites inside a window five thousandths wide, the recorded tapes never enter it, and the largest strength move anywhere else is one ten-thousandth against a confidence floor near a quarter.

Inside the window the gate does change its mind. Every one of the 2,901 runs was walked in with its own final bar and put through both chains at both price scales: 5,802 readings, and on every one of them the label said overbought while the vote said neutral. After the repair not one of them disagrees. Of the 46,416 verdicts, 60 fire before the repair and six of those stop firing, all on the buy side and all the same way.

```
5,802 constructed readings, 46,416 gate verdicts
    label disagreeing with the vote, before     5,802
    label disagreeing with the vote, after          0
    vote moved from neutral to bearish          5,802
    verdicts that fire before                      60
    scrum fires -> scrum refused                    6
    scrum refused -> scrum fires                    0
    fold verdicts changed                           0
    windows                                         3
        DOT 2023, consensus 0.2002 -> 0.1728, floor 0.1786
        A14 2022, consensus 0.2087 -> 0.1813, floor 0.1923
        MATIC 2022, consensus 0.2286 -> 0.1910, floor 0.1923
```

All six need the scrum's own bullish requirement switched on. The mechanism is the panel: a bearish vote at 0.4001 in place of a neutral one at nought pulls the bullish consensus below the confidence floor, and the sell of the surplus is refused on a bar the RSI calls overbought.

Ten places read this indicator. Four read the vote and six read the published number.

reader	what its reading becomes
the panel's consensus	the vote and its strength, decided by the number the labels use; direction unchanged on all 3,750
`ta_engine.py` recorder	records the computed number in place of the cut copy
the trading gate chain	reads the consensus the vote feeds; no verdict moved
`indicator_panel.py` cell	the arrow and the strength percentage; unchanged on all 3,750
`indicator_panel.py` hover text	prints three decimals; moved on 3,356 of 3,750
`indicator_panel_surface.py` hover text	moved on the same 3,356
`ata_spm.py` sentence	prints two decimals; unchanged on all 3,750
`ta_invariants.py` bound	holds the reading between nought and a hundred; unchanged
`phantom_balance.py` record	stores what the indicator published
`snapshots.py` bot.log line	its headline readings are ADX, Z-Score and Kaufman, so it prints the RSI name and strength only

The bot's own position boost and the back test both ask a signal for a number, and both ask only for ADX, Kaufman and Z-Score. Neither has ever asked this indicator for one.

The reading was also published cut to two decimal places, and the panel's hover cell prints three. On 3,356 of the 3,750 readings the third decimal the cell printed was a nought that the indicator had not computed. The confirming sentence prints two decimals, which is exactly what the cut copy carried, so it reads the same on every one.

2026-09-09 19:37 - #414 - the band a millionth of a dollar wide

Item 6 of the issue names two epsilons, one in the Ichimoku and one in the Supertrend. Neither is in this tree. Both came out on 2026-09-04, in the commit that dropped the last eight absolute constants from price-scaled denominators.

```
b27c6cdb  fix(ta): drop the last eight absolute epsilons from scaled denominators
```

That makes eight items out of eight where the expression the issue quotes had already been repaired, and where the defect still standing underneath it was a rounded copy of the reading reaching a consumer. This entry closes that pattern for the whole folder.

John Bollinger's bands are the worst of the set, because their width is what the scrum ramp measures against. The published definition fixes no number of decimal places, and a band is an ordinary price in the asset's own units.

```
Middle Band = SMA(period)
Upper Band  = Middle + std_dev * sigma(period)
Lower Band  = Middle - std_dev * sigma(period)
BandWidth   = (Upper - Lower) / Middle
```

The indicator published all three cut to six decimal places, a step of one millionth of a dollar. A10 trades at three millionths, so the whole channel is narrower than the step and the three bands land on one number.

`src/trading/indicators/bollinger.py` — the reading, before and after

```
"upper": round(upper, 6),          "upper": upper,
"middle": round(mid, 6),           "middle": mid,
"lower": round(lower, 6),          "lower": lower,
```

Measured over 71 recorded Coinbase tapes, run at a price of three millionths of a dollar and again at a four-figure price. Every window from 36 bars in steps of 11 drove the real voting engine, and each reading built a gate context through the shipped scan. Over 1,875 windows at the small price the distance from the midline to the band read exactly zero on 1,014 of them, and on a further 20 the lower band reached zero and the scan refused the channel outright. At the four-figure price it read zero on none.

```
1,875 windows at 3.1e-06        1,875 windows at 1234.0
    half-span exactly 0.0           half-span exactly 0.0
        before  1,014                   before  0
        after   0                       after   0
    band refused outright           smallest half-span
        before  20                      before  7.76315
        after   0                       after   7.76315
```

The consequence is a decision, not a display. The ramp that arms a scrum divides the price's travel by that half-span, and a floor of a millionth of a millionth stands in when the span is zero. The ramp then walked a different path on a cheap asset than on an expensive one holding the same shape.

`src/trading/ata_gate_scan.py` — the step that decides whether a scrum is armed

```
span = max(abs(edge - middle), BAND_SPAN_FLOOR)
travel_frac = abs(price - middle) / span
```

The two price scales disagreed about that state on 162 of the 1,875 windows. They now agree on all of them, which is the property the fleet needs: one detect threshold serves every bot, whatever its asset costs.

```
target_fires compared between the two price scales
    before   agree 1,713   disagree 162
    after    agree 1,875   disagree 0
```

Both gate chains ran on every reading, in all four states of the two direction switches. Of 30,000 verdicts, 1,322 fire before the repair and 58 change. All 58 are on the buy side, all 58 are at the small price scale, and every one sits on a window where the ramp state moved. Nothing changed at four figures.

```
30,000 gate verdicts
    fire before the repair                1,322
    changed                                  58
        scrum refused -> scrum fires         50
        scrum fires -> scrum refused          8
        fold verdicts changed                 0
    changed at 1234.0                         0
    changed at 3.1e-06                       58
    rows carrying them                       27
        rows where the ramp state moved      27
```

The same cut sat on every other indicator in the folder. Fifty-three calls across thirteen files cut a reading where it was computed rather than where it was shown, and all fifty-three are gone. None was kept.

```
zscore 7   ichimoku 6   bollinger 5   landing_strip 5   fvg 4   macd_taper 4
slingshot 9   supertrend 3   bb_proximity 3   atr 2   stochastic_rsi 2
volume 2   rsi 1
```

No display loses its shape, because every one of them already carries its own format. The panel hover cell prints three decimals, the bot log line prints four, and each confirming sentence in the ATA-SMP names its own width.

`src/trading/ata_spm.py` — one voter's sentence, formatted where it is shown

```
"bollinger_bands": (
    "Bollinger Bands",
    (BAND_POSITION_KEY,),
    "band position {value:.4f}",
),
```

Item 6 also names two statements that compute a value and bind it to nothing. The Ichimoku one is the third cloud state. The code named the other two and then derived the third a second time where the reading is published, so one expression now decides it.

```
inside_cloud = not above_cloud and not below_cloud
...
"price_vs_cloud": (
    "inside" if inside_cloud else "above" if above_cloud else "below"
),
```

The volume one is the money-flow slope. It is published beside the balance-volume slope the same indicator already publishes, so the value it computes now reaches a reader.

```
mfi_rising = mfi > mfi_prev
```

Across the 3,750 readings, 339,170 non-numeric readings were compared. None moved and none was dropped. The cloud state is identical on every window, which is what a pure rebinding has to look like, and its inside branch runs on 499 of them rather than never. The money-flow slope takes both values.

```
price_vs_cloud   below 1,579   above 1,112   inside 499   absent 560
mfi_rising       True 2,025    False 1,725
```

Twelve places read these numbers, and the table below says what each one's reading becomes.

reader	what its reading becomes
the scan's scrum ramp	a real band width at any price; 58 of 30,000 verdicts move, all on the buy side
the live bot's scrum ramp	unchanged; it reads the proximity result, whose three band prices were never cut
the Z-Score gate	the computed z; it moved on 3,470 readings and crossed the two-deviation line on none
the ADX gate	unchanged; that reading was uncut already
the Efficiency Ratio gate	unchanged; that reading was uncut already
the confidence floor	the landing-strip boost, uncut; no consensus direction moved
the panel's twelve cells	direction and strength unchanged on all 3,750; strength moved on 358 votes
`indicator_panel.py` hover text	the computed number, printed to three decimals as before
`ata_spm.py` sentences	each keeps its own width, so every sentence reads as it did
`snapshots.py` bot log line	prints four decimals of the computed number
`phantom_balance.py` record	stores what the indicator published
`ta_invariants.py` bounds	no breach on any reading, before or after

Three figures the page already carries need restating against this run. The departures section above says two departures remain, the Vortex direction overwrite and the RSI rounding. Both are repaired: the Vortex on 2026-09-08 and the RSI on the same day, each in its own entry above.

The debug-report citation, overtaken

OVERTAKEN, quoted whole:

> *"The section also lists four scale-invariance test files. None is in the tree, which now holds its conftest and these debug reports and nothing else, so the property they held is held by the runs recorded in each entry instead."*

```
the 2026-09-09 indicator-rounding debug report
```

True today: the four files are still absent, and so is the report named above. The repository tracks one Python module under `tests/`, its own conftest, and no directory of debug reports beside it. The runs recorded in the entries on this page are the whole record of the property.

What changes for an operator: a bot on a market priced in millionths of a dollar now sees a Bollinger channel with a real width instead of a flat line, so its scrum arms on the same bars a bot on an expensive market would.

Trading gate logic chain

The epigraph on the title page reads *dissolvendus reformandusque*, dissolved and reformed. The chain below does exactly that, once per tick: twelve separate readings dissolve into one signed number, and the gates re-form that number into an order or a refusal.

The chain

Diagram: flowchart, top to bottom

- OHLCV candles -> **The 12 indicator modules**
- The 12 indicator modules -> **VotingEngine.compute_all** *(Signal: direction, confidence, weight)*
- VotingEngine.compute_all -> **VotingEngine._aggregate**
- VotingEngine._aggregate -> **VotingSummary.net_score**
- VotingEngine._aggregate -> **VotingSummary.consensus_confidence**
- VotingSummary.net_score -> **consensus_direction: deadband +/- 0.1**
- consensus_direction: deadband +/- 0.1 -> **BULLISH / BEARISH / NEUTRAL**
- BULLISH / BEARISH / NEUTRAL -> **ScrummingBot.tick: is_bullish, is_bearish**
- VotingSummary.consensus_confidence -> **ScrummingBot.tick: is_bullish, is_bearish**
- _skewed_confidence_floor: 0.25 relaxed by the skew -> **ScrummingBot.tick: is_bullish, is_bearish**
- OHLCV candles -> **detect_bb_proximity**
- detect_bb_proximity -> **bb_position**
- bb_position -> **bb_pos against _bb_detect_thresholds**
- ScrummingBot.tick: is_bullish, is_bearish -> **build_scrumming_scrum_chain**
- bb_pos against _bb_detect_thresholds -> **build_scrumming_scrum_chain**
- adx signal detail 'adx' -> **build_scrumming_scrum_chain**
- kaufman_er signal detail 'er' -> **build_scrumming_scrum_chain**
- zscore signal detail 'z' -> **build_scrumming_scrum_chain**
- build_scrumming_scrum_chain -> **ChainResult.should_fire**
- ChainResult.should_fire -> **_tick_execute_scrum, then _execute_sell** *(true)*
- ChainResult.should_fire -> **ChainResult.blocked, one message per gate** *(false)*
- _tick_execute_scrum, then _execute_sell -> **guarded_place_order**

Net, Comp and Conf

`VotingEngine._aggregate` in `src/trading/ta_engine.py` produces all three panel aggregates from the twelve signals.

```
weighted_score = direction * confidence * weight        one per voter
bull_score     = sum of |weighted_score| over the BULLISH voters
bear_score     = sum of |weighted_score| over the BEARISH voters
net_score      = bull_score - bear_score                 the Net column
voted_weight   = sum of weight over the voters that did not abstain
confidence     = min(1.0, |net_score| / voted_weight)    the Conf column
```

Functional. Net is a direction bias, not a reversal signal. Positive means the bullish voters outweighed the bearish ones. Conf is the breadth of that agreement, not its force. Six voters agreeing strongly while six abstain reads lower than twelve agreeing mildly, and the panel says the same thing in the Conf column's own tooltip. An abstaining indicator is left out of the divisor, so a short tape cannot water down the reading of the voters that did

read. Net is rounded to four decimals and Conf is capped at 1.0.

`src/trading/ta_engine.py` — `VotingEngine._aggregate`

```
net = bull_score - bear_score
```

On the panel. Net, Comp and Conf are drawn as one pillar each, standing in the last three columns of the shared grid. A pillar runs from the row-B plot floor to the row-A plot ceiling, so it passes both tables and both bar graphs and tops out on the line a voter at full confidence reaches. `collated_pillars` in `indicator_panel_surface.py` names each pillar and takes its direction from its own metric's sign, so Net follows `net_score`, Comp follows `composite_net` and Conf follows the summary's direction.

`src/gui/main_tabs/indicator_panel_surface.py` — `collated_pillars`

```
first = PANEL_COLUMN_COUNT - len(AGGREGATE_TITLES)
return [
    {"name": title, "direction": way, "column": first + at}
    for at, (title, way) in enumerate(zip(AGGREGATE_TITLES, directions))
]
```

A pillar carries no height reading. The value stays in the table cell above it, and `AGGREGATE_TITLES` names the pillar once, at its base. `RULED_COLUMNS` keeps the row partition, the plot baseline and the dotted increments on TF and the twelve, so none of them crosses a pillar.

When a cell cannot read its figure. `net_cell`, `comp_cell` and `conf_cell` admit a reading through `as_finite_float` before printing it. A reading that is not a finite number draws `ABSENT_CELL_TEXT` in `ABSENT_TEXT_COLOR`: the em dash the Comp column has always drawn where there is no composite. An integer too large to hold as a float is one such reading, and so is a NaN. No cell prints a figure it did not read, and no cell carries a notice in place of one.

`src/gui/main_tabs/indicator_panel_surface.py` — `absent_cell`

```
return {
    "text": ABSENT_CELL_TEXT,
    "text_color": ABSENT_TEXT_COLOR,
    "tooltip": tooltip,
}
```

The grid holds one reading. `timeframe_rows` settles every row — its timeframe, its reading and the signals keyed by indicator — before the row count is cut. The grid therefore holds the timeframes the fed reading holds and no others. A value a row cannot read draws the em dash in that row's own cell, so a refusal never leaves a row carrying the previous reading's numbers. `vote_totals` sums the three vote counts through the same admission step, and `lock_line` draws the em dash for a lock field it cannot read. The Paper and Simulator panels draw their rows and their aggregate cells through the same functions.

`src/gui/indicator_panel.py` — `IndicatorVotingPanel.update_data`

```
rows = ivp.timeframe_rows(multi_tf_summary)
self._vote_totals = ivp.vote_totals(multi_tf_summary)
self._table_a.setRowCount(len(rows))
```



The restyled panel

`src/trading/ta_engine.py` — and the divisor beneath it

```
voted_weight = sum(s.weight for s in signals if not s.abstained)
consensus_conf = abs(net) / voted_weight if voted_weight > 0.0 else 0.0
```

Design intention. Conf answers how many of the voters that could read agree, not how loud they were. Dividing by the weight of the voters that actually read, rather than by all twelve, is the arithmetic that says so. The panel repeats the same rule to the reader in its own tooltip, so the screen and the engine cannot tell two different stories.

`src/trading/ta_engine.py` — the rule, stated where it is applied

```
# A weighted mean divides by the weight of the points included, and an
# abstention is not one; it is NEUTRAL, so it adds nothing to `net`.
```

The deadband

`VotingSummary.consensus_direction` in `src/trading/indicators/types.py` turns Net into a direction, and it holds a deadband of plus and minus 0.1.

```
net_score > +0.1   ->  BULLISH
net_score < -0.1   ->  BEARISH
otherwise          ->  NEUTRAL
```

`ScrummingBot.tick` then folds the direction and the Conf reading into one boolean per side.

```
is_bullish = direction in (BULLISH, NEUTRAL) and confidence >= floor
is_bearish = direction in (BEARISH, NEUTRAL) and confidence >= floor
```

Functional. The confidence floor starts at 0.25 and the bot relaxes it when the vote is skewed. A landing strip at the upper band sets the bullish flag outright, and one at the lower band sets the bearish flag. A neutral direction passes both sides, so a panel with no consensus does not on its own block a trade. The floor and the gates below decide that.

`src/trading/indicators/types.py` — `VotingSummary.consensus_direction`

```
def consensus_direction(self) -> SignalDirection:
    if self.net_score > 0.1:
        return SignalDirection.BULLISH
    elif self.net_score < -0.1:
        return SignalDirection.BEARISH
    return SignalDirection.NEUTRAL
```

Design intention. The deadband stops a near-zero Net reading as a direction. Plus or minus 0.1 against a full voted weight of 11.7 is a very small bias, and the band treats it as none at all. The bot then folds the direction and the confidence into one boolean per side, and a neutral panel passes both.

`src/trading/scrumming_bot.py` — the two booleans, in plain terms

```
is_bullish = direction in (BULLISH, NEUTRAL) and confidence >= floor
is_bearish = direction in (BEARISH, NEUTRAL) and confidence >= floor
```

Current reading, how low the floor goes. The bot relaxes the floor no further than what two indicators reach. The engine runs the vote again with the two heaviest voters at full confidence and the rest at none, at the bot's weights on that tick. A relaxed floor stays above that figure and never rises above 0.25, so a vote needs a third agreeing indicator wherever the plain floor needs one. The DIST sale and the hedge top-up read a second floor, the same one without hunger.

`src/trading/scrumming_bot.py` — `_skewed_confidence_floor`

```
if denominator <= 1.0:
    return floor / denominator
past_two = math.nextafter(two_indicator_ceiling(signals), math.inf)
return min(floor, max(floor / denominator, past_two))
```

Current reading, three agreeing at the floor and at a strip. The floor now stays above the two-indicator line at every skew, so it can rise over 0.25. A landing strip sets its flag only when the vote tops the same line. A bot with every weight at zero keeps its strip as before, because its vote has no line. Live, the Market Inspector scan, Paper and the Simulator read one function for both booleans.

`src/trading/ta_engine.py` — `vote_sides`, the strip and the floor

```
if strip and not three_agree(confidence, signals):
    strip = ""
clears = confidence >= floor
```

The band position

Functional. The band position is %B: zero at the lower band, one at the upper. The two thresholds a scrum and a fold must clear come back in the same terms, so both sides of every comparison are positions and neither side is a price. The fraction is clamped to the 0 to 1 range, so a stored setting outside 0 to 100 cannot push a threshold outside the band. A missing setting falls back to 75 through an explicit test for absence, so a stored zero stays zero.

The relationship in plain terms:

```
detect_frac  = scrum_detect_pct / 100          75% by default
upper_detect = 0.5 + detect_frac * 0.5         0.875
lower_detect = 0.5 - detect_frac * 0.5         0.125

SCRUM requires bb_pos >= upper_detect
FOLD  requires bb_pos <= lower_detect
```

`src/trading/scrumming/circuit_breakers.py` — `_bb_detect_thresholds`

```
detect_frac = max(0.0, min(1.0, detect_pct / 100.0))
half = detect_frac * 0.5
return (0.5 - half, 0.5 + half)
```

Design intention. Working in band positions rather than in prices is what lets one threshold serve the whole fleet. A bot on a four-figure asset and a bot on an asset priced in millionths both read 0.875 and mean the same thing by it. This is the pattern the constants listed above break. A missing setting is tested for absence rather than for truthiness, so a deliberate zero survives instead of being turned back into 75.

`src/trading/scrumming/circuit_breakers.py` — the absence test

```
# None-sentinel default so a legitimate 0 isn't coerced to 75 by
# `or` truthiness.
_raw = getattr(self.config, "scrum_detect_pct", None)
```

The scrum chain gates

`src/trading/gate_chain.py` — `build_scrumming_scrum_chain`

```
return GateChain(
    gates=[
        DeltaPositiveGate(),
        IntervalGate(),
        TADirectionGate(side="scrum"),
        TrendHoldGate(),
        MidlineGate(side="scrum"),
        TargetFiresGate(),
        BBProximityGate(side="scrum"),
        CircuitBreakerGate(side="scrum"),
        HTFDeferGate(side="scrum"),
        HysteresisGate(side="scrum"),
        ADXTrendSuppressionGate(),
        EfficiencyRatioRegimeGate(),
        ZScoreExtremityGate(side="scrum"),
        RipeHarvestScrumOverride(),  # runs in pass 2 regardless of list position
    ],
    side="scrum",
)
```

Functional. Every gate in the list runs, and every refusal is collected. The trade fires only when the refusal list is empty, so one blocked gate is enough to hold it. The order of the list fixes the order the refusals appear in, which is the order the log prints them.

`src/trading/gate_chain.py` — the verdict

```
return ChainResult(
    should_fire=(not final_blocked),
    passed=passed,
    blocked=final_blocked,
    overrides_applied=overrides_applied,
)
```

Design intention. One gate holds one question, and each writes its own refusal in its own words. Reading back a held trade is then a matter of reading which name refused, rather than re-deriving the whole decision.

`src/trading/gate_chain.py` — `DeltaPositiveGate.evaluate`, the pattern every gate follows

```
def evaluate(self, ctx: GateContext) -> GateResult:
    if ctx.delta > 0:
        return GateResult(passed=True)
    return GateResult(passed=False, blocker_message="delta≤0")
```

Gate	Passes when	Reads	Overridden by
`delta_positive`	the holdings sit above target	the target delta	none
`interval`	the delta clears the scrumming interval	the delta	none
`ta_bullish`	`is_bullish`	Net, Conf and the floor	`ripe_harvest_override`

Gate	Passes when	Reads	Overridden by
`trend_hold`	no sustained uptrend	more than 13 of the last 20 candles closing above their open	`ripe_harvest_override`
`midline_scrum`	`bb_pos` above 0.50 and no phantom lock	`bb_position`	`ripe_harvest_override`
`target_fires`	the detect and fire ramp is in FIRE	the target ramp	`ripe_harvest_override`
`bb_proximity_scrum`	`bb_pos >= upper_detect`	`bb_position`	none
`circuit_breaker_scrum`	no soft trip on the scrum side	the breaker state	none
`htf_defer_scrum`	the higher timeframe is not bullish	the coordinator	none
`hysteresis_scrum`	the price cleared the last pivot plus fees	the pivot	none
`adx_trend_suppression`	ADX under 30.0	the `adx` signal detail	none
`efficiency_ratio_regime`	ER between 0.05 and 0.70	the `er` signal detail	none
`zscore_extremity`	Z above -2.0	the `z` signal detail	none

Functional. Three of those gates skip the collated vote and read one indicator's own number: ADX, the Efficiency Ratio and the Z-Score. The bot fills each field from that indicator's signal on the tick. A reading of zero or below is the not-populated marker, and all three pass on it.

`src/trading/gate_chain.py` — `EfficiencyRatioRegimeGate.evaluate`, one of the three

```
def evaluate(self, ctx: GateContext) -> GateResult:
    if ctx.efficiency_ratio <= 0.0:
        return GateResult(passed=True)
    if ctx.efficiency_ratio >= self.upper_threshold:
```

Design intention. These three describe the market regime rather than its direction, which is why they reach the chain on their own instead of through the vote. Passing on the marker keeps a bot that never filled the field from being frozen by it. The cost is real and worth stating: a genuine zero reads exactly the same as a missing one, and the gate cannot tell them apart.

In development. Separating the two would need a marker the field cannot produce, and no shape for that has been chosen.

The fold chain gates

`src/trading/gate_chain.py` — `build_scrumming_fold_chain`

```
return GateChain(
    gates=[
        TranchesQueuedGate(),
        TADirectionGate(side="fold"),
        MidlineGate(side="fold"),
        SmartCeilingGate(),
        BBProximityGate(side="fold"),
        CircuitBreakerGate(side="fold"),
        HTFDeferGate(side="fold"),
        HysteresisGate(side="fold"),
        ZScoreExtremityGate(side="fold"),
        DeepFoldOverride(),
    ],
    side="fold",
)
```

Functional. The buy side runs nine gates against the sell side's thirteen. Six of the sell-side gates have no twin here, and the buy side adds two of its own.

```
absent on the fold side   delta_positive, interval, trend_hold,
                          target_fires, adx_trend_suppression,
                          efficiency_ratio_regime
present only here         tranches_queued, smart_ceiling
```

Design intention. The two chains mirror each other, and two of the six absences carry a written reason. The interval gate says in its own source that it needs no fold twin, because the bot returns early on the same condition. The trend-hold gate is declared sell-side only, and the config field that would arm a buy-side twin is declared and unread. 06-trading-tab.md lists the flags the Strategy Gate Flags group holds.

Functional. That config field is now removed. The buy side holds no flag for a trend hold and no gate for one, so the asymmetry is a plain absence rather than a flag waiting on a gate. The sell-side gate is unchanged. On one tick with twenty candles all closing above their open, the sell side reported `trend_hold` and the buy side reported nothing.

`src/trading/gate_chain.py` — `IntervalGate`, stating its own case

```
# No FOLD twin; ScrummingBot.tick returns early on below_interval when both
# its fold queue and distribution accumulator are empty.
```

The other four absences carry no stated reason in the code at all. That is worth knowing before anyone assumes the asymmetry is deliberate.

Gate	Passes when	Overridden by
`tranches_queued`	at least one fold tranche is waiting	none
`ta_bearish`	`is_bearish`	`deep_fold_override`
`midline_fold`	`bb_pos` below 0.50	`deep_fold_override`
`smart_ceiling`	the position sits under the smart ceiling	none
`bb_proximity_fold`	`bb_pos <= lower_detect`	none

Gate	Passes when	Overridden by
`circuit_breaker_fold`	no soft trip on the fold side	none
`htf_defer_fold`	the higher timeframe is not bearish	none
`hysteresis_fold`	the price cleared the last pivot plus fees	none
`zscore_extremity`	Z below +2.0	none

The two overrides

Functional. An override runs in a second pass and force-passes gates that already refused. Its own verdict never appears in the pass list or the block list. Each refusal it lifts has its message rewritten to name the override, and the name is recorded separately, so the log tells you which gate was overridden and by what.

`src/trading/gate_chain.py` — `GateChain.evaluate`, the second pass

```
overrides_applied: list[str] = []
for gate in self._override_gates:
    result = gate.evaluate(ctx)
    if not result.passed:
        continue
    for target_name in result.override_gates:
        if target_name in blocked_index:
            idx = blocked_index.pop(target_name)
            _name, _msg = blocked[idx]
            blocked[idx] = (_name, f"OVERRIDDEN_BY:{gate.name}({_msg})")
            overrides_applied.append(
                OVERRIDE_APPLIED_FORMAT.format(
                    gate=target_name, override=gate.name, refusal=_msg
                )
            )
```

Functional. The record separates a refusal that stood from one an override replaced, and it names the override in both cases. A replaced refusal is one line naming the gate, the override that replaced it and the refusal text it replaced. A refusal that stood from a gate an override covers says which override did not apply, and a refusal from a gate no override covers names none.

`src/trading/gate_chain.py` — the two forms a reader reads

```
OVERRIDE_APPLIED_FORMAT = "{gate} overridden by {override}: {refusal}"
OVERRIDE_NOT_APPLIED_FORMAT = "{message} [override not applied: {overrides}]"
```

Override	Arms on	Force-passes
`ripe_harvest_override`	positive delta, interval cleared, `bb_pos >= upper_detect`	`midline_scrum`, `target_fires`, `trend_hold`, `ta_bullish`
`deep_fold_override`	negative delta, interval cleared, `bb_pos <= lower_detect`	`midline_fold`, `ta_bearish`

Design intention. At the band extreme, the band position outranks the vote. That comes straight from the operator's own reading of the Bollinger Bands: they are the boundary a trade fires at, and a position sitting on one is the condition he wants to trade. Neither override touches the circuit breaker, the hysteresis or the three

indicator gates, so the safety gates keep their veto.

`src/trading/gate_chain.py` — `RipeHarvestScrumOverride`, and what it may lift

```
name = "ripe_harvest_override"
side = "scrum"
overrides = ("midline_scrum", "target_fires", "trend_hold", "ta_bullish")
```

The trend-hold override

Functional. A third override clears the uptrend hold, and it is not a gate in either chain. It runs in `ScrummingBot.tick` before the chain is built, and it fires on either of two conditions: the position is at least twice the scrumming interval away from target in dollars, or Band Travel has raised its own trigger. When it fires, the bot sells into a sustained uptrend that the SCRUM holds in sustained uptrend box was set to hold through.

`src/trading/scrumming_bot.py` — the override in `ScrummingBot.tick`

```
trend_override = abs(delta) >= _interval_usd * 2.0 or band_travel_triggered
if trend_hold and trend_override:
```

Override	Arms on	What it clears
`trend_hold_override`	the delta at twice the scrumming interval in dollars, or the Band Travel trigger	the uptrend hold, before the chain is built

Functional. The operator sees it in three places. The bot log carries a TREND-HOLD OVERRIDE line naming the trend share, the condition that cleared it and the dollar figure behind that condition. The per-tick GATES line names TRND under its overridden clause, so the TRND light paints the landing strip's cyan rather than the passed green. The gate record carries one entry under `overrides_applied`, written only when the box was set, so a tick with the box clear reads apart from a tick the override acted on.

`src/trading/scrumming_bot.py` — the record entry, in `ScrummingBot.tick`

```
if _trend_hold_cleared_by and _flag_hold_in_uptrend:
    _scrum_replaced.append(
        OVERRIDE_APPLIED_FORMAT.format(
            gate=_GATE_TREND_HOLD,
            override=TREND_HOLD_OVERRIDE,
            refusal=(
                f"trend_hold({trend_strength:.0%}) cleared by "
                f"{_trend_hold_cleared_by}"
            ),
        )
    )
```

Design intention. The override stays as it is. A move of twice the interval is the profit the whole cycle exists to take, and holding it back to wait out a trend forfeits it. What was missing was the record, not a different rule: the hold was cleared and no screen said so.

Functional. The override is in the live engine alone. The back test and the paper fleet build their gate inputs from the raw trend reading and never clear it, so a Simulator tick and a Paper tick hold where a live tick sells. The checkbox tooltip says so on all three screens.

`src/trading/scrumming_bot.py` — the only module the override is written in

```
trend_override = abs(delta) >= _interval_usd * 2.0 or band_travel_triggered
```

Where the armed flag comes from

Functional. The gate row's two armed flags are the chain's own verdict. One chain decides per side, and the row carries that decision and the refusals that produced it, so a row and a decision cannot disagree. Every refusal the gate row names is a refusal a gate wrote, including the three indicator gates, which the row reports under their own names.

`src/trading/scrumming_bot.py` — the row, written in `ScrummingBot.tick`

```
self._last_gate_state["scrum_armed"] = bool(
    _scrum_chain_result.should_fire
)
self._last_gate_state["scrum_blockers"] = list(_scrum_blockers)
```

The one effective-flag mapping

Functional. Five raw readings become five effective readings through one function. A cleared require box passes its direction outright; a cleared hold or defer box drops its block. Four readers call it: the live tick, the Market Inspector's gate scan, the back test and the paper fleet.

`src/trading/gate_chain.py` — `effective_gate_flags`, the one mapping

```
return EffectiveGateFlags(
    eff_is_bullish=bool(is_bullish) if require_bullish else True,
    eff_is_bearish=bool(is_bearish) if require_bearish else True,
    eff_trend_hold=bool(trend_hold) if hold_in_uptrend else False,
    eff_htf_blocks_scrum=bool(htf_blocks_scrum) if defer_scrum else False,
    eff_htf_blocks_fold=bool(htf_blocks_fold) if defer_fold else False,
)
```

Reader	Where it calls the mapping
Live	`src/trading/scrumming_bot.py`, in `ScrummingBot.tick`
Market Inspector	`src/trading/ata_gate_scan.py`, in `build_context`
Simulator	`src/simulator/back_test.py`, in `tape_context`
Paper Trader	`src/paper/paper_run.py`, in `paper_tape_context`

Which records name the override

Functional. Three records carry the name of the override that replaced a refusal, one for each data source, and all three write it under the same key. The key is spelled once, in `OVERRIDES_APPLIED_KEY`.

Data source	The record	Where the name is written
Live	the gate log row `ScrummingBot.tick` writes	`scrum_fixture.overrides_applied`
Paper	the gate row `paper_log.paper_row` writes	`scrum_fixture.overrides_applied`
Simulator	the `bot.gate_decision` event `SimBus` emits	`scrum_fixture.overrides_applied`

Functional. The Paper Trader and the Simulator both read one record, the one the shipped chains are evaluated into. A row where no override acted carries an empty list; a row where one acted carries one entry per refusal it replaced. A row that passed outright and a row an override passed therefore read apart.

`src/simulator/validation.py` — `latch`, the record both of them read

```
return {
    "scrum_armed": bool(scrum.should_fire),
    "fold_armed": bool(fold.should_fire),
    "scrum_blockers": [message for _name, message in scrum.blocked],
    "fold_blockers": [message for _name, message in fold.blocked],
    "scrum_overrides_applied": [str(one) for one in scrum.overrides_applied],
    "fold_overrides_applied": [str(one) for one in fold.overrides_applied],
}
```

Not available. The nineteen gate lights the Simulator's parity comparison draws read the armed flag and the blocker list alone, so a replaced refusal still reads there as a plain pass. Its two sides are a recorded row and a rerun of the same inputs, and a row recorded before the key existed carries none, so reading the override into those lights would move the parity verdict on past rows.

`src/simulator/validation.py` — `light_states`, the four inputs it takes

```
def light_states(
    scrum_armed: bool,
    fold_armed: bool,
    scrum_blockers: Sequence[str],
    fold_blockers: Sequence[str],
) -> list[dict]:
```

Reading the chain after the fact

Not available. A trade already on the books keeps the reading that was recorded beside it. Nothing recomputes the direction, the Net, the Conf, the vote counts, an indicator's own vote or the band position for a past trade, so a gate row that was never written stays missing.

One writer. A gate row reaches `gate.log` only when `LogManager.log_gate_decision` writes it at the moment the gate decides.

`src/core/logging_engine.py` — the one call that records a decision

```
def log_gate_decision(
    self,
    exchange: str,
    bot_id: str,
    symbol: str,
    scrum_armed: bool,
    fold_armed: bool,
    scrum_blockers: Optional[list] = None,
    fold_blockers: Optional[list] = None,
```

2026-09-24 22:50 - #202 - one admission rule for every panel reading

Four rules read a stored number and they disagreed. A saved flag read as the number one on the panel and as nothing in the trading package. A 400-digit saved number ended the panel's refresh, which left one timeframe's label above the previous reading's numbers.

One rule reads every number now, and it is the trading package's own.

`src/trading/container/config.py` — the one admission rule

```
def as_finite_float(value) -> Optional[float]:
    if type(value) is float:
        return value if math.isfinite(value) else None
    if type(value) is int and -_FLOAT_SAFE_INT <= value <= _FLOAT_SAFE_INT:
        return float(value)
    return None
```

It answers nothing for a saved flag, for a not-a-number, for either infinity, for a number written as text and for an integer too large to be a float. It never raises, so a refusal cannot end a refresh.

What a refused reading draws. Each cell says for itself. The panel draws zero, so a refused Net reads `+0.00`, a refused Conf reads an empty bar at `0%`, and a refused indicator cell reads its direction beside `0%`.

`src/gui/main_tabs/indicator_panel_surface.py` — the figure a refusal draws

```
#: What a cell, bar, pillar or tint draws when as_finite_float refuses its reading.
REFUSED_READING: float = 0.0
```

The Qt panel and the React panel now build one cell from one pair of builders, `indicator_cell_text` and `indicator_cell_colors`, so the two draw the same text and the same tint from the same reading.

Three sentences the retired rules carried. Each is quoted whole, and each is overtaken by the sentence written under it.

> *"One published reading as a float, or* `fallback` *when it is not one."*

Overtaken. That rule read a saved flag as the number one and raised on a saved integer too large to be a float. `as_finite_float` answers nothing for either, and `REFUSED_READING` is the figure the panel draws in its place.

> *"*`value` *as a float, or* `fallback` *when it is not one."*

Overtaken, and this sentence stood in two files. Both read either infinity as a dollar figure. `as_finite_float` answers nothing for either, and `REFUSED_FIGURE` is the figure a Paper record and a Simulator record draw.

> *"The number one market row carries, nothing where it carries no number."*

Overtaken. That rule refused a saved flag and still raised on a saved number too large to be a float. `as_finite_float` refuses both, and `VOLUME_REFUSED_USD` is the volume a refused pair row sorts by.

2026-10-24 - #665 - a gate row carries the panel it was decided on

The gate log is how a past decision is read back. A row held the decision and none of the numbers behind it, so the reasoning was unrecoverable. A row now carries one cell per voter, and each cell holds that indicator's direction, its confidence, its weight and its headline reading.

`src/trading/scrumming/snapshots.py` — the panel a row carries

```
def _indicator_values(self) -> Optional[dict]:
    summary = getattr(self, "_last_summary", None)
    if summary is None:
        return None
    return _build_panel_snapshot(summary)
```

Both gate emitters pass it, so a fired trade and a refused tick are recorded the same way. `src/trading/scrumming/snapshots.py, in _emit_gate_decision_at_fire` writes the fired row, and `_emit_gate_decision_at_quiet_tick` writes the refused one.

An absent panel is not a row of zeros. A tick that computed no panel leaves the key off its row, so a reader can tell a panel that was never computed from a panel whose indicators all read zero. `src/core/logging_engine.py, in log_gate_decision` writes the key only when it is given one.

Three rows read back off a written gate log — one fired, one refused, one with no panel:

```
trade_action SCRUM    adx BULLISH 0.71 weight 1.4 adx 27.5
                      z_score BEARISH 0.33 weight 0.9 z -1.82
                      kaufman_er NEUTRAL 0.00 weight 1.0 er 0.18
tick_exit gate-chain  the same three cells
tick_exit gate-chain  no indicator key at all, because no panel was computed
```

No gate decision moved. Both scrumming chains were driven over 1024 gate input sets before and after the change: 32 scrum fires and 32 fold fires on each side, and 0 of 1024 whole decisions differ.

2026-10-25 - #665 - the consolidation figure is declared once

Row 280. Landing Strip detection asks whether a Heikin-Ashi body is tight. The figure it compares against is three percentage points of the candle's own high-low range, and that figure was written in five places: the detector's own parameter, a number typed at the live tick, a second number typed at initial entry, a named copy in the gate scan, and a third named copy in the Simulator.

No control writes this figure. It is not an operator setting, and the Product Manual named it under no spelling before this entry.

One declaration now serves every caller, and it sits beside the detector that reads it.

`src/trading/indicators/bb_proximity.py, in detect_bb_proximity`

```
CONSOLIDATION_THRESHOLD_PCT: float = 3.0


def detect_bb_proximity(
    candles: list[Candle],
    tolerance_pct: float = 1.0,
    consolidation_threshold: float = CONSOLIDATION_THRESHOLD_PCT,
    min_pattern_candles: int = 2,
```

Four callers read that one name. The live tick and the initial entry are the two that reach real money.

```
src/trading/scrumming_bot.py, in tick
src/trading/scrumming/tick_phases.py, in _first_buy_read
src/trading/ata_spm.py, in proximity_of
src/simulator/validation.py, in bb_reading
```

The last of the four is read by the Paper build, the Simulator and the back test, so Live, Paper and the Simulator now ask the same question of a candle.

Driven on one 222-candle tape, the reading is unchanged. The declaration was then moved to a figure nothing uses and all four callers followed it together, while the detector's other two opening figures did not move.

```
figure 3.0, the declaration
  gate scan reading     landing strip upper, 10 candles, strength 1.0000
  Simulator reading     landing strip upper, 10 candles, strength 1.0000

the one declaration moved 3.0 -> 0.77
  gate scan reading     no landing strip, 0 candles, strength 0.0000
  Simulator reading     no landing strip, 0 candles, strength 0.0000
  pattern candles       2      unmoved
  band tolerance        1.0    unmoved
```

Before the change the same move reached the detector's own parameter alone, and all four callers kept reading three, because each passed its own copy. The file is byte identical after the move was put back.

Subsystem Tabs

Reference. Main Window, Market Inspector and Asset Charts each have a heading below, and the manual's own text for the screen comes first with what the code does today under it. Every other screen has one section of its own, one file per screen in 08-tabs/, and this part points at it rather than repeating it.

Detail: 08-tabs/simulator.md. Detail: 08-tabs/paper-trader.md. Detail: 08-tabs/proof-of-accumulation.md. Detail: 08-tabs/bot-swarm.md. Detail: 08-tabs/history.md. Detail: 08-tabs/console.md. Detail: 08-tabs/system-status.md. Detail: 08-tabs/settings.md.

One list names the ten tabs the window builds, and every screen below that the window does not build says as much in its own entry. Two are skeletons now: Status and Accumulation each draw a name, one sentence saying the tab is not built, and the issue that carries the build-out. Sim and Paper were skeletons until issues #117 and #19 built them, and both are full screens today.

`src/gui/main_tabs/main_window_surface.py` — the order before issue #450

```
CANONICAL_TAB_ORDER = (
    TRADING_TAB,
    MARKET_INSPECTOR_TAB,
    BOT_SWARM_TAB,
    ASSET_CHARTS_TAB,
    HISTORY_TAB,
    SIMULATOR_TAB,
    CONSOLE_TAB,
    PAPER_TRADER_TAB,
    SYSTEM_STATUS_TAB,
    PROOF_OF_ACCUMULATION_TAB,
)
```

Each of those names is a constant holding the label the tab bar shows. The skeletons take their label from their own surface, so the bar and the screen cannot carry two spellings of one name. Issue #450 renamed every one of those constants and reordered the tuple, and the section under this one holds what the file carries today.

`src/gui/main_tabs/main_window_surface.py` — the labels before issue #450

```
TRADING_TAB = "Trading"
MARKET_INSPECTOR_TAB = "Market Inspector"
BOT_SWARM_TAB = "Bot Swarm"
ASSET_CHARTS_TAB = "Asset Charts"
HISTORY_TAB = "History"
SIMULATOR_TAB = "Simulator"
CONSOLE_TAB = "Console"

PAPER_TRADER_TAB = paper_trader_tab_surface.HEADING
SYSTEM_STATUS_TAB = system_status_tab_surface.HEADING
PROOF_OF_ACCUMULATION_TAB = proof_of_accumulation_tab_surface.HEADING
```

The module `paper_trader_tab_surface` named in that block is gone. `src/gui/paper/paper_trading_tab_surface.py, in HEADING` carries the Paper label today.

The order, the names and the colours today

The bar now opens on Sim and every tab carries a single word. The order below is what the window ends with, and it is decided in the one place named above.

`src/gui/main_tabs/main_window_surface.py` — `CANONICAL_TAB_ORDER`

```
CANONICAL_TAB_ORDER = (
    SIM_TAB,
    PAPER_TAB,
    LIVE_TAB,
    CHARTS_TAB,
    INSPECTOR_TAB,
    SWARM_TAB,
    ACCUMULATION_TAB,
    HISTORY_TAB,
    STATUS_TAB,
    CONSOLE_TAB,
)
```

Eight of the ten labels are new words for screens that already existed. History and Console were already one word and did not change. Each constant was renamed to match the word it now holds.

`src/gui/main_tabs/main_window_surface.py` — the labels

```
LIVE_TAB = "Live"
INSPECTOR_TAB = "Inspector"
SWARM_TAB = "Swarm"
CHARTS_TAB = "Charts"
HISTORY_TAB = "History"
SIM_TAB = "Sim"
CONSOLE_TAB = "Console"

PAPER_TAB = paper_trading_tab_surface.HEADING
STATUS_TAB = system_status_tab_surface.HEADING
ACCUMULATION_TAB = proof_of_accumulation_tab_surface.HEADING
```

The eight renames, in the order the bar reads them:

was	is now
Simulator	Sim
Paper Trader	Paper
Trading	Live
Asset Charts	Charts
Market Inspector	Inspector
Bot Swarm	Swarm
Proof of Accumulation	Accumulation
System Status	Status

Sim, Paper and Live carried the promotion pipeline on three different grounds, and the other seven tabs shared the gold ground. Those grounds are gone, and every tab now takes one look from the theme's style sheet.

The table that gave each tab its ground is gone from the window surface. It read:

```
TAB_GROUNDS = {
    SIM_TAB: BLACK_GROUND,
    PAPER_TAB: WHITE_GROUND,
    LIVE_TAB: GOLD_GROUND,
    CHARTS_TAB: GOLD_GROUND,
    INSPECTOR_TAB: GOLD_GROUND,
    SWARM_TAB: GOLD_GROUND,
    ACCUMULATION_TAB: GOLD_GROUND,
    HISTORY_TAB: GOLD_GROUND,
    STATUS_TAB: GOLD_GROUND,
    CONSOLE_TAB: GOLD_GROUND,
}
```

No tab holds a colour of its own. A ground named two theme tokens, and every theme filled those tokens with its own values, so gold was the yellow that suited each theme rather than one hex value everywhere.

The six tab tokens each theme carried are gone from the theme engine. They read:

```
theme             tab_gold_bg tab_gold_text tab_black_bg tab_black_text tab_white_bg tab_white_text
cyberpunk_dark    #fcee0a     #8c0018       #0a0a0f      #ff5577        #f5f5fa      #0a0a0f
neon_light        #f0cf1f     #99001f       #1a1a2e      #ff6b8a        #ffffff      #1a1a2e
classic_terminal  #ffff00     #990000       #0a0a0a      #ff3333        #e8e8e8      #0a0a0a
minimal_modern    #eab308     #7f1d1d       #1a1a1a      #ff6b6b        #ffffff      #1a1a1a
glass_metal       #e8b34a     #6b1020       #1c1c24      #ff6688        #e8e8f0      #1c1c24
```

Every pair above cleared WCAG 2.2 AA at 4.5 to 1. The narrowest margin was red on gold under Minimal Modern, at 5.22 to 1.

The Qt bar painted those colours itself, because a Qt style sheet cannot colour one tab by its position. The theme's own style sheet set the six colours on the bar, so switching theme repainted it.

The tab bar's own file and its paint method are deleted, and no module paints a tab ground by its position now. The bar takes its look from the theme's Qt style sheet, under its tab rules (`src/gui/theme_engine.py, in _theme_qss`). An asset class's tab book takes that class's accent on its selected tab (`src/gui/main_tabs/main_window_surface.py, in mode_tab_styles`). The block below records what the bar did while the file was in the tree.

```
painter.fillRect(self.tabRect(index), ground)
painter.setPen(text)
painter.drawText(self.tabRect(index), int(Qt.AlignCenter), self.tabText(index))
```

The React bar draws the same ten buttons. The window publishes their labels and methods and no colours, so the page holds no palette of its own.

`src/gui/main_tabs/main_window_surface.py` — `MainWindowModel.as_dict`, what the frontend reads

```
"tab_labels": list(self.tab_labels),
"tab_methods": dict(self.tab_methods),
"theme": self.theme,
"tabs_movable": self.tabs_movable,
```

The Live tab, which the manual's part list calls the Trading Tab, has its own section, 06-trading-tab.md, and the Indicator Voting Panel has 07-indicators.md.

A strategy reaches real money through four steps: Market Inspector, Simulator, Paper Trader, Live. Each step is a gate, and all four screens are built. 08-tabs/promotion-pipeline.md draws the chain and marks where a step still hands nothing to the next.

The sections below run in the order 04-manual-parts.md lists the tabs.

Main Window

Portfolio Information Panels

Found at the top of the Main Window at all times, this provides metrics for total performance and activity of the platform.

The window is built in three parts: the menu bar, the header strip, and the tab row. The panels above fill that strip, and each tab below sits in that row.

The strip is one row. Five labelled columns run down the left — SPENDABLE, REALISED, LOCKED, MATURE and EXCH — and five counter cards close it on the right.

`src/gui/main_tabs/header_strip.py` — `HeaderStripMixin._build_header_strip`

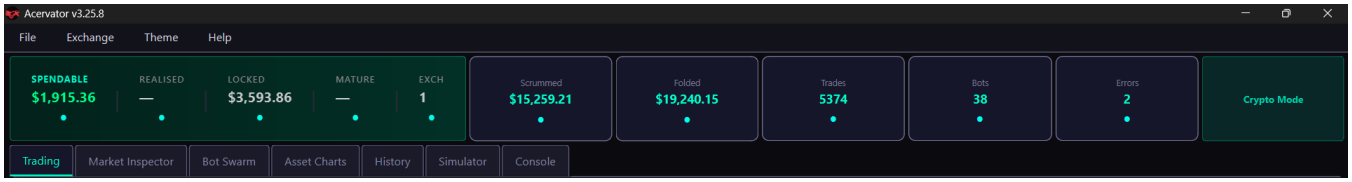
```
self._spendable_widget = SpendableProfitsWidget()
top_row.addWidget(self._spendable_widget, stretch=3)
```

The five cards are Scrummed, Folded, Trades, Bots and Errors. One aggregate call fills every field on the strip once a tick, which keeps two cards from disagreeing about the same fleet.

`src/gui/main_window.py` — `_refresh_dashboard`

```
agg = self._bot_manager.get_aggregate_stats()
_scr = float(agg.get("total_scrummed_usd", 0.0) or 0.0)
_fld = float(agg.get("total_folded_usd", 0.0) or 0.0)
self._stat_scrummed.set_value(f"${_scr:,.2f}")
self._stat_folded.set_value(f"${_fld:,.2f}")
```

Each field carries a privacy dot that masks the value through the registry the Swarm tab shares. The strip hides itself while the Sim tab or the Paper tab is active, and the Paper tab draws four figures of its own in place of it. The absent live strip is itself the signal that the screen is not live trading.



The header strip and the tab row, with Privacy Mode off.

The chrome above the strip carries four menus. File holds Settings, Reset All Settings and Exit. Exchange holds Add Exchange. Theme lists the five display names the theme engine declares, and Help holds About.

`src/gui/main_window.py` — the menu bar

```
file_menu = menu_bar.addMenu("&File")
file_menu.addAction("&Settings", self._open_settings)
file_menu.addAction("&Reset All Settings", self._reset_settings)
```

What the Theme menu does

The Theme menu paints and it stores. One press applies the theme to the Qt window, repaints the tab labels, repaints every React page already open, and writes the name under the `theme` key, so the choice is still there at the next start.

`src/gui/main_window.py` — `_switch_theme`

```
painted = repaint_pages(self, name)
self._remember_theme(name)
```

A page already up takes the new palette without being rebuilt. One statement per web view rewrites the six chrome colours on the document root, so every drawn row and the scroll position stay where they were.

```
the Market Inspector page, 900x600, 92,250 sampled pixels of ground

before the press   #0a0a0f
after the press    #f5f5fa    184,500 sampled pixels differing
pressed back       #0a0a0f    identical to before
the Activity Log   1 React pages repainted in neon_light
```

A page built later opens on the same theme. Every React page host named one theme in its own source; each now leaves the name unset, and the page reads back the theme the window is painted in.

`src/gui/react_history_panel.py` — `page_theme`

```
return stored_theme(applied_theme() if theme is None else theme)
```

Driven with nothing passed to the builder, the Alerts, Console, History and Market Inspector pages each painted a different ground under a stored Cyberpunk Dark and a stored Neon Light, at 184,500 to 187,500 sampled pixels differing per page. All four were identical before.

The window's theme table and the chart chrome table are one declaration now. The chart table is built from the names the theme table declares, so a name cannot sit in one and not the other, and the five colour bodies it holds are unchanged.

`src/gui/tradingview_chart.py` — `CHART_THEMES`

```
CHART_THEMES: dict[str, dict[str, str]] = {
    name: _CHART_CHROME.get(name, _CHART_CHROME[DEFAULT_THEME_NAME])
    for name in THEMES
}
```

The theme does not reach the contents of a page. A page's rows, panes and buttons arrive on the payload as a Qt stylesheet string built from the one dark token table, which carries no theme dimension. The Console tab, driven under three themes, painted the same five grounds every time, and each of the five is a token from that table.

`src/gui/design_system.py` — the grounds the Console tab painted under all three

```
SURFACE_CHART = "#0a0a12"  # Chart, console and group-box ground
SURFACE_CONSOLE = "#05050a"  # Console ground
SURFACE_CONSOLE_HEADER = "#0a0a14"  # Console header ground
```

Privacy Mode is off in the figure, so each field draws its own value rather than four asterisks. SPENDABLE and LOCKED carry dollar figures. EXCH counts the open exchange sub-tabs. Crypto Mode at the right swaps the window between the crypto layer and the stock layer.

REALISED and MATURE draw an em dash. Both take a literal absence at the one call site that fills them, on every tick.

`src/gui/main_window.py` — `_refresh_dashboard`

```
self._spendable_widget.update_profits(
    {
        "spendable": _wallet_cash,
        "total_realised": None,
        "locked": _crypto_value,
        "mature": None,
        "exchange_count": exchanges,
    }
)
```

Turning Privacy Mode on masks that em dash to asterisks, which reads as a hidden number rather than an empty column. 06-trading-tab.md lists what each cell reads. Issue #428 carries a second disagreement on this strip, between what the counter tooltips promise and what the cards draw.

The issue #428 clause above is overtaken. It is not edited and not removed. It read:

> *Issue #428 carries a second disagreement on this strip, between what the counter tooltips promise and what the cards draw.*

Issue #428 closed on 2026-10-03. The two counter tooltips name the same source the cards draw from, and neither promises a reset.

```
src/gui/main_tabs/header_strip_surface.py

SCRUMMED_TOOLTIP   "the dollars every bot has sold so far this year ...
                    comes from the exchange's own trade history, so the venue
                    and the card agree ... It survives a restart."
FOLDED_TOOLTIP     the same sentence for buys
```

Current reading, 2026-10-29. Both counter descriptions are shorter, within twenty words. They keep the source and the restart.

`src/gui/main_tabs/header_strip_surface.py` — `SCRUMMED_TOOLTIP`

```
SCRUMMED_TOOLTIP = (
    "Dollars every bot has sold this year, read from the exchange's own trade "
    "history. It survives a restart."
)
```

Both columns now carry an exchange figure. The window hands the strip the same payload the React strip receives, built by one function, so the two hosts cannot show different numbers.

`src/gui/main_window.py` — `_refresh_dashboard`

```
self._spendable_widget.update_profits(
    header_strip_surface.profits_payload(agg, exchanges)
)
```

REALISED is the venue's matched profit and loss across the fleet. MATURE is the profit held in positions worth more than three times what they cost. Where the venue has answered for no bot, both stay empty, and the mask now leaves an empty column empty instead of drawing four asterisks over it.

`src/gui/main_tabs/header_strip_surface.py` — `exchange_amount`

```
answered = int(data.get(EXCHANGE_FRESHNESS_KEY, 0) or 0)
if answered <= 0:
    return None
```

The venue's portfolio breakdown answers each position's cost basis, average entry price and unrealised profit, and each bot reads those three from it. The breakdown carries no lifetime realised figure for a spot position, so REALISED is the platform's first-in, first-out match over every fill the venue holds for each bot's symbol, one figure per bot, summed across the fleet; a bot pages the whole history once and then one page per refresh, and logs the venue's cost basis against the cost of the buys left open.

`src/trading/scrumming/reconciliation.py` — `fetch_fill_history`

```
if self._fill_history is None:
    self._fill_history = FillHistory(self.config.symbol)
return await self._fill_history.refresh(self.exchange)
```

Detail: 08-tabs/portfolio-panels.md.

2026-09-09 07:23 - #128 - the header strip's spendable panel is a panel, its dot is not

The header strip is one registered panel that draws two other modules inside itself. The shell mounts `header_strip` as chrome, above the tab bar and on show for every tab. Inside it, the spendable panel and the five counter cards each fill a space the strip leaves for them, and each of those spaces is a mount of its own.

`src/gui/web/header_strip.js` — the spendable space, as the strip draws it

```
  function renderSpendable(target, model) {
    var host = global.acervatorPanelHost;
    var space = spaceNamed(target, SPENDABLE);
    if (!host || space === null) {
      return null;
    }
    space.setAttribute(CHILD_ATTR, SPENDABLE_MODULE);
    return host.mount(SPENDABLE_MODULE, space, model) ? space : null;
  }
```

`spendable_profits.js` now registers, so the shell knows it by name and the strip draws it the way the Live tab draws its Activity Log and the Charts tab draws its chart. The registration names no bridge method, so the tab bar gives it no tab. Nineteen panels register where eighteen did before.

The privacy dot is a different kind of thing. It has no space and no host: two other modules render it as an element inside their own trees, so the panel host never draws it and a registration would name a panel with nowhere to go. Its Registers in Electron cell now reads `-`, the mark this table already uses for a column that does not apply to a row.

`src/gui/web/spendable_profits.js` — where the dot comes from

```
  function dotSpan() {
    var own = global.acervatorDot;
    if (own && own.Dot) {
      return own.Dot;
    }
    var api = global.acervatorHeader;
    return api && api.PrivacyDot ? api.PrivacyDot : null;
  }
```

Both builds were read on one fleet snapshot and one privacy state, with LOCKED and Folded masked and every other field revealed. Nothing was written into the page: the shell fetched every figure over the bridge. Every value, label, glyph, colour and tooltip on the strip agrees between them.

The strip's slots now share the row the way the Qt layout shares it. A Qt stretch divides the whole width; a CSS `flex-grow` divides only what is left over, so the counters drew at five different widths. With a zero flex basis the five cards measure 148 pixels each against Qt's 153, and the spendable panel takes 34.6 per cent of the

row against Qt's 34.8.

Qt file	React module	Uses React	Bridge	Manifest	Registers in Electron	Ships in the build	RENDERS	Scope
src/gui/widgets/spendable_profits.py	spendable_profits.js	yes	yes	yes	yes	yes	yes	in scope
src/gui/widgets/privacy_dot.py	privacy_dot.js	yes	yes	yes	-	yes	yes	in scope

The strip fetches its own figures. The shell asks for the strip with an empty request, so both surfaces now read the fleet the bridge is bound to, the way the History, Charts, Trading and Inspector surfaces already read theirs. The aggregate is `BotManager.get_aggregate_stats`, the same call the Qt window's own tick makes, and the exchange count is the length of `SettingsManager.list_exchanges`, which is the list the window builds one sub-tab from. Neither figure is derived in the surface or in the page.

`src/gui/main_tabs/header_strip_surface.py` — the request fills itself

```
def live_view_model(params: dict, live: Any) -> dict:
    asked = dict(params or {})
    if asked.get(STATS_PARAM) is None:
        aggregate = fleet_aggregate(live)
        if aggregate is not None:
            asked[STATS_PARAM] = aggregate
```

The spendable columns take the same aggregate through `header_strip_surface.profits_payload`, the one builder the Qt tick already calls, so the two hosts read one snapshot and cannot disagree.

Each privacy dot in the strip's columns now carries the pointing hand the Qt dot sets on itself. `privacy_dot_surface.dot_view` publishes `cursor_shape`, and the two surfaces that build a dot for this strip now keep it. `dashboard_stat_card_surface` still cuts it away, so the five counter dots draw with the ordinary pointer; that card is its own row.

Qt draws a one pixel panel edge around each counter card where the page draws none, because `dashboard_stat_card_surface.FRAME_STYLE` is empty and the `StyledPanel` shape reaches the page as an attribute no rule reads. That is the same row.

2026-09-09 08:42 - #128 - the counter cards and the bot table are components

The five counter cards and the Scrumming bot table are drawn inside a panel that has already registered, and neither registers in its own right. The strip leaves five spaces and builds a request for each one, carrying that counter's caption, amount, privacy field and whether it is pressable. The bot table is drawn once for the exchange the Trading tab is showing, from a payload the module keeps under that exchange's own name.

`src/gui/web/header_strip.js` — one request per counter

```
  function cardRequest(card) {
    var asked = {};
    asked[CARD_LABEL_PARAM] = text(card[LABEL]);
    asked[CARD_VALUE_PARAM] = text(card[TEXT]);
    if (text(card[FIELD_ID]) !== undefined) {
      asked[CARD_FIELD_ID_PARAM] = text(card[FIELD_ID]);
    }
    asked[CARD_CLICKABLE_PARAM] = card[CLICKABLE] === true;
    return asked;
  }
```

The panel host keeps one host element per panel name and asks each panel with one request of its own. Registering either module was driven in the running shell to see what it would give: the card drew an empty caption and the default amount, and the table drew its ten headers over no rows, each into a host of its own under the panel area, while the strip's five counters kept their figures. Both cells now read `-`, the mark this table already uses for a column that does not apply to a row, and the mark the privacy dot took for the same reason.

Each counter card now draws the one pixel edge, the rounded corner and the padding that Qt paints around it. Qt takes all three from the theme rule for a framed panel; the shell selects no theme, so the page had nothing to read and drew a plain box. The card's own skin carries them, built from the design tokens rather than from a colour written into the page.

`src/gui/main_tabs/dashboard_stat_card_surface.py` — the card's skin

```
FRAME_STYLE = (
    "border-style: solid; "
    f"border-width: {FRAME_LINE_WIDTH_PX}px; "
    f"border-color: {ds.OUTLINE}; "
    f"border-radius: {ds.RADIUS_SM}px; "
    f"padding: {ds.SPACE_CARD_PAD}px;"
)
```

The five counter dots now carry the pointing hand as well. The card cut its dot down to five fields and left out the cursor the dot publishes; it keeps all eight now, which is what the spendable columns already do.

Both builds were read on one fleet snapshot with every field revealed. Nothing was written into the page: the shell fetched the aggregate and the bot list over the bridge, and breaking each source in turn emptied the figures it feeds and left the other side drawing. Every label, amount, colour, glyph, tooltip, column header, row order and cell text agrees between the two.

Two differences remain, and both belong to other rows. Qt fills each card with the theme's card ground, which no design token carries. And the counter space is 141 pixels wide on all five while the card inside takes its content width, which is how the strip has placed its counters since before this change.

Qt file	React module	Uses React	Bridge	Manifest	Registers in Electron	Ships in the build	RENDERS	Scope
`src/gui/widgets/dashboard_stat_card.py`	`dashboard_stat_card.js`	yes	yes	yes	-	yes	yes	in scope
`src/gui/widgets/bot_status_table.py`	`bot_status_table.js`	yes	yes	yes	-	yes	yes	in scope

Inspector

Market Inspector Tab

The concept with the Market Inspector Tab is evaluate markets from a higher point of view and provide strategy proposals in three forms: Oppositional Trading Pairs (Trading Pairs w/ Opposing Trends), Bot Swarm Topologies (Bot Swarm Network Proposals), and Exchange Comparison Arbitrage.

The tab is now called Inspector. It sits fifth on the bar, on the gold ground with red text.

The tab splits in two. The left half holds the HTF Signals table and the Opposing Pairs table, both scored by one analyzer. One method drives the whole pipeline and keeps the results on that analyzer.

`src/trading/market_inspector.py` — `MarketInspector.scan_universe`

```
def scan_universe(
    self,
    candles_by_symbol_by_tf: dict,
    active_symbols: set,
    closes_by_symbol: dict,
) -> None:
```

The right half holds proposal cards from four detectors: momentum funnel, mean-reversion pair, sector cluster and distance to band. The engine takes a plain dictionary, unions the detectors, drops overlapping proposals by asset and caps the result.

`src/trading/topology_proposals.py` — `detect_all_topologies`

```
def detect_all_topologies(
    context: dict[str, Any],
    cap: int = PROPOSAL_CAP,
) -> list[dict[str, Any]]:
```

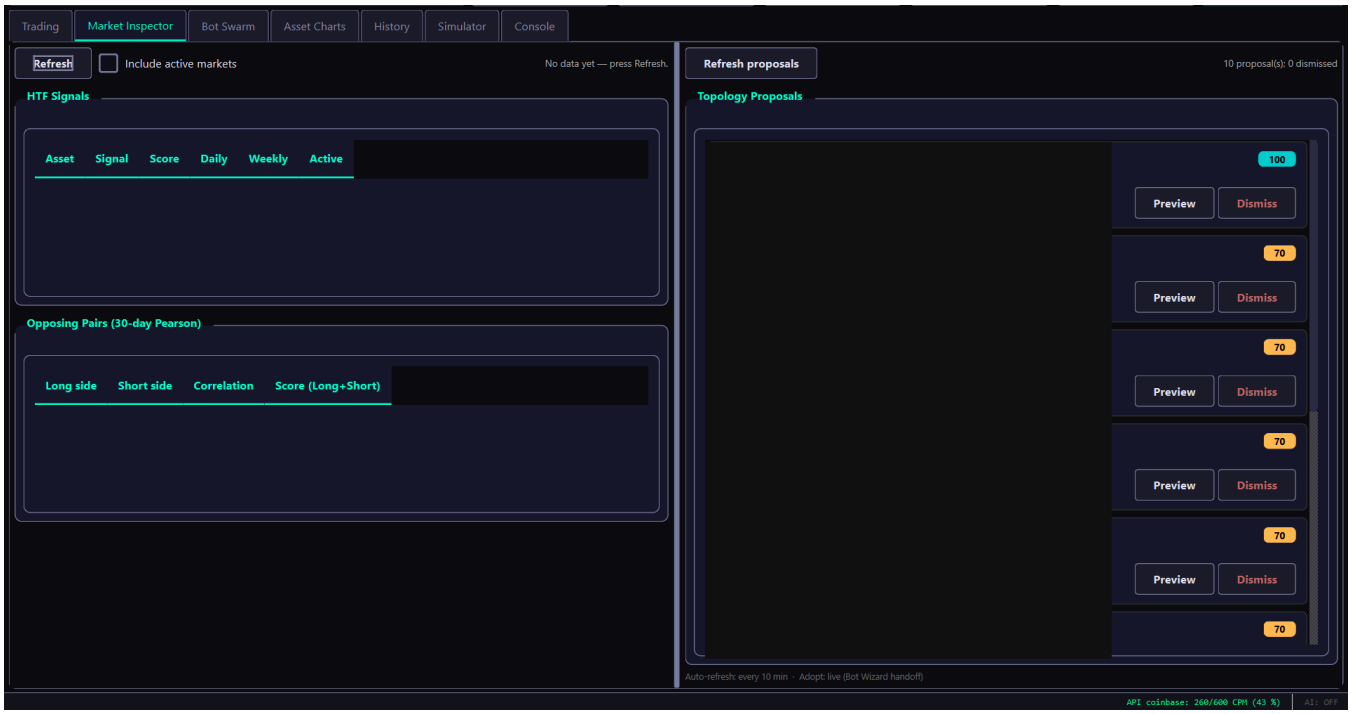
Adopting a card names the new bots, their combined budget and every existing wire the adopt would change before it creates anything.

`src/gui/main_window.py` — `_adopt_topology_proposal`

```
new_bots = [
    b for b in proposal.get("bots", []) if not b.get("existing_bot_id")
]
wires = list(proposal.get("wires", []))
new_count = len(new_bots)
```

Two of the three proposal forms reach the screen. Exchange comparison arbitrage has a module, `src/trading/arbitrage.py`, and no importer under `src/`; the tab draws no arbitrage panel.

In development.



The Market Inspector tab, before the first scan.

Refresh and the Include active markets checkbox sit above the two tables, and the status line reads `No data yet — press Refresh.` until a scan lands, which is the state the figure captures. Leaving the checkbox clear hides the markets a bot already holds.

HTF Signals carries six columns: Asset, Signal, Score, Daily, Weekly and Active. Opposing Pairs carries four: Long side, Short side, Correlation and Score. The pairing enumerates every long against every short and keeps the ones whose thirty-day return correlation sits in the configured negative window.

`src/trading/market_inspector.py` — `MarketInspector._find_opposing_pairs`

```
def _find_opposing_pairs(self, signals: list, closes_by_symbol: dict) -> list:
    """Enumerate long × short candidates; keep pairs whose 30-day
    return correlation sits in the configured negative window."""
    longs = [s for s in signals if s.direction == "long" and s.score >= 0.3]
    shorts = [s for s in signals if s.direction == "short" and s.score >= 0.3]
```

Refresh proposals runs the detectors, and the line beside it counts the proposals held and the cards dismissed. Each card names its archetype and the assets in it, then a line reading the score, the asset count, the wire count and the number of bots an adopt would create. The badge takes its colour from the score: teal at the high threshold and above, amber at the middle one, grey below. Every card in the figure comes from the sector-cluster detector. Preview lists the wires as Source, Target, Pct and Rationale.

Dismiss hides a card for a day, and the dismissal survives a restart. The pane writes the held ids into the operator's settings under `topology_dismissed_proposals` and reads them back on the next launch, so the count beside Refresh names what is held rather than zero.

`src/gui/main_tabs/market_inspector_topologies_surface.py` — `persist_dismissed`

```
def persist_dismissed(self) -> None:
    """Write the held dismissals to the store, recording a refusal."""
    if self.dismiss_store is None:
        return
    try:
        self.dismiss_store.set(DISMISS_SETTINGS_KEY, dict(self.dismissed))
```

A refused write puts `Dismissal NOT saved:` and the cause on that same line in place of the count, and the card stays hidden for the session.

The footer names the auto-refresh period and the adopt route. The status bar under it carries the API load pill, drawn green below half load, amber above it, and red past the monitor's safety percentage, then the `AI:` state label.

React draws this screen. Both halves are one web page inside a single view, and the build decides which class the tab builder makes. `src/gui/main_tabs/market_inspector_tab.py` — `_build_market_inspector_tab`

```
from ..variant_surface import MARKET_INSPECTOR, surface_class

self._market_inspector = surface_class(MARKET_INSPECTOR)()
```

`MarketInspectorReactTab` subclasses the Qt tab, so the scan cycle, the filtering and the analyzer writes are the same code on both sides. Five accessors are what the two sides answer differently: `_set_status`, `_set_refresh_enabled`, `_shown_signals`, `_fill_signal_rows` and `_fill_pair_rows`. The Qt tab writes them into its widgets and the React tab writes them into the page.

`src/gui/react_market_inspector_tab.py` — `MarketInspectorReactTab._build_ui`

```
self._web = QWebEngineView(self)
self._web_page = MarketInspectorPage(self)
self._web.setPage(self._web_page)
self._web.loadFinished.connect(self._on_load_finished)
self._web.setHtml(tab_html())
```

Refresh and the Include active markets checkbox report their press back to Python, and the right pane reports Refresh proposals, Preview, Dismiss, Cancel and Adopt the same way. Setting `ACERVATOR_VARIANT` to `qt` builds the Qt widgets instead, unchanged.

Detail: 08-tabs/market-inspector.md.

2026-09-09 09:42 - #128 - the HTF Signals and Opposing Pairs tables come out

Neither screen drew the two tables. The Qt tab built both group boxes and put neither into a layout, so no table ever reached the window. In the running shell, the React screen drew no table part at all, while its payload carried six signal columns and seven pair columns.

`src/gui/market_inspector.py` — `_fill_pair_rows`

```
def _fill_pair_rows(self, pairs: list) -> None:
    """Hold the opposing pairs the Opposing Trades stepper draws."""
    self._pairs = list(pairs)
    self._render_empty_notes()
```

The screen holds neither table by design. The six-zone layout moved HTF Signals into the reserved Phantom Bot zone, and it hands each opposing pair to the Opposing Trades stepper, one entry at a time. Both sides now drop the renderer, the column sets and the two empty sentences, and the tab keeps only the rows the zones read.

Nothing on the screen moved. The Qt picture and the React picture each match byte for byte before and after the change, and the drawn markup of the panel matches at 21,194 characters.

The earlier comparison that closed these two rows read its table items out of the payload. `signalRow` and `pairRow` handed back cells from the model, which agreed on both sides whatever the page drew. This change drops both.

OVERTAKEN, quoted whole:

> *"The run is in 2026-09-09_market_inspector_tables.md."*

True today: the run record is not in the tree. The repository tracks one Python module under `tests/`, its own conftest, and no directory of run records beside it. The two figures the run produced are the ones in this entry: both pictures match byte for byte, and the drawn markup matches at 21,194 characters.

2026-09-09 23:05 - #128 - the Electron shell draws the Inspector tab and its proposals pane

The Inspector tab draws in the Electron shell. The shell asks the application which tabs it builds, matches `market_inspector.state` to the module that serves it, and gives that module the Inspector tab.

`desktop/renderer/panel_host.js` — `register`

```
    panels[name] = spec;
    dropFault(name);
    kinds[name] = declaredKind(name, spec);
```

The panel host now draws the proposals pane into the right-hand slot, under the name `market_inspector_topologies.js` registers. The slot used to be filled by a call straight to the module, so the registration drew nothing and removing it changed no pixel.

`src/gui/web/market_inspector.js` — `fillSlot`

```
    host.mount(TOPOLOGY_PANEL, slot, null);
```

`desktop/renderer/index.html` links `market_inspector.css`, the stylesheet that carries the colour, border and type of every part the two modules stamp. The file shipped and nothing loaded it, so every button on the screen drew in the browser default. The button rule reads the design token that holds the ground the theme gives a Qt push button.

`src/gui/web/market_inspector.css` — the button ground

```
  background: var(--SURFACE_2, var(--btn-bg));
```

The check boxes on the tab drew as the browser's own control, white and 13 pixels, where Qt paints an 18 pixel box with a `#7a7a9c` edge on `#0e0e1a`. That ground was in no token, so the table gained one under the name of the role it fills, and the stylesheet names the token rather than the value.

`src/gui/main_tabs/design_system_surface.py` — the check-box ground

```
SURFACE_INPUT = "#0e0e1a"
```

Both screens were driven on one payload of 54 markets, 300 daily candles each and three ranked proposals. Twenty-seven of twenty-seven picture items match, so rows 878 and 879 read `yes` under Registers in Electron. The Opposing Pairs rows match at nothing on either side, so that item proves the column set and not the rows.

OVERTAKEN, quoted whole:

> *"The run is in 2026-09-09_market_inspector_register.md."*

True today: the run record is not in the tree. The figures it produced are the ones in this entry: one payload of 54 markets, 300 daily candles each, three ranked proposals, and twenty-seven of twenty-seven picture items matching.

Charts

Under the Asset Charts Tab, you will find our active bot (position) chart display. This will be upgraded to display only one chart at a time and will be able to display all indicators found in the Indicator Voting Panel.

The tab is now called Charts. It sits fourth on the bar, straight after Live, on the gold ground with red text.

The tab scrolls one panel per active bot, each painted with QPainter and no browser, carrying trade markers, position markers, grid lines, the standing tranche floors and the target balance line.

`src/gui/main_tabs/charts_tab.py` — `ChartsTabMixin._build_charts_tab`

```
def _build_charts_tab(self) -> None:
    """Build the Asset Charts tab and add it to the main tab widget."""
    # --- Tab 2: Charts ---
    self._charts_tab = TradeChartsTab()
    self._main_tabs.addTab(self._charts_tab, "Asset Charts")
```

The chart reads its overlays from the engine rather than computing any of them.

`src/gui/native_chart.py` — `CandlestickChart._compute_indicators`

```
candles = self._candles
self._bb_data = BollingerBands(20, 2.0).bands(candles)
vi_plus, vi_minus = VortexIndicator(14).lines(candles)
```

Candles arrive through a fetcher with three sources in order: exchange OHLCV first, a public API second, the last good fetch third.

`src/exchange/chart_data.py` — `ChartDataFetcher.fetch`

```
# --- Source 1: Exchange OHLCV via CCXT ---
if exchange is not None:
    try:
        candles, source = await self._fetch_exchange(
            exchange, symbol, timeframe, limit
        )
        if candles:
            self._set_cache(symbol, timeframe, candles)
            return candles, source
```



The Asset Charts tab, one panel per running bot.

The panel set rebuilds from the current bot roster, and the tab scrolls them.

The header line of a panel names the symbol, the last price, the bot state, the timeframe, the feed and the candle count. The line under it carries the open, high, low and close of the current candle, the change in price and in percent, and the volume. Rising candles draw teal and falling candles red.

The dashed lines across the plot are the standing fold-tranche floors, each labelled with its price. `TB-Ceiling` is the dollar-target line. The price axis runs down the right edge with the last price boxed on it, and the volume bars run along the foot.

The toolbar under each panel holds the TF picker and eight indicator toggles.

`src/gui/main_tabs/native_chart_surface.py` — `INDICATOR_TOGGLES`

```
INDICATOR_TOGGLES = (
    ("bb", "BB", "#50a0f0"),
    ("vortex", "Vortex", "#00ff88"),
    ("macd", "MACD", "#ffcc00"),
    ("stochrsi", "SRsi", "#ff9060"),
    ("ichimoku", "Ichi", "#c080ff"),
    ("volume", "Vol", "#80ffcc"),
    ("slingshot", "Sling", "#ff4488"),
    ("bbullseye", "BBull", "#ff00aa"),
)
```

Volume starts on and the other seven start off, which is the state the figure shows. Sling and BBull paint a placeholder shape rather than the indicator, and each says as much in its own tooltip. Issue #430 carries a tooltip on this toolbar that names a colour the chart does not paint.

The issue #430 clause above is overtaken. It is not edited and not removed. It read:

> *Issue #430 carries a tooltip on this toolbar that names a colour the chart does not paint.*

Issue #430 closed on 2026-09-09. No tooltip on this toolbar names a colour. The Vortex entry names its two lines, its period and its pane and stops there.

```
src/gui/native_chart.py
    tooltip="Vortex Indicator (VI+ / VI-, period 14, sub-pane)"

tooltips declared in that module   14
tooltips naming a colour            0
```

The legend at the right names the two position markers, Invisible and On Book, and the feed label closes the row.

One chart, chosen with the arrows

The tab now draws one chart at a time. A left arrow, a centre ticker list and a right arrow choose which traded asset it shows, and a readout beside them says which of how many is on screen. The list wraps, so neither arrow dead-ends, and the ticker readout is itself a drop-down. With no bots running the readout says zero of zero and both arrows are switched off.

`src/gui/widgets/trade_charts_tab.py` — the readout beside the arrows

```
POSITION_FORMAT = "{at} of {total}"
EMPTY_TICKER_TEXT = "No asset"
EMPTY_POSITION_TEXT = "0 of 0"
```

The toggles, and the two that start off

Every indicator the chart can draw now starts switched on, and a check box for each sits along the bottom of the panel. Two start off, because each paints a filled shape over the price where the Bollinger bands and the Ichimoku cloud are drawn. Slingshot paints solid marks at a squeeze release and a snapback. BB Bullseye paints four shaded envelopes directly on the bands themselves.

`src/gui/native_chart.py` — one entry per drawn indicator

```
class ChartOverlay:
    key: str
    label: str
    colour_field: str
    pane: str
    occludes: bool
    draw: str
    tooltip: str
```

The renderer is built to grow

`CHART_OVERLAYS` declares each indicator once, and the paint routine walks that list rather than naming any of them. Another overlay is one entry in the list and one draw method; no existing overlay is touched to add it.

The renderer holds no colour of its own. Every painted value resolves through `PALETTE_ROLES`, which names the theme field and the transparency each role reads, so a theme sets the look and the renderer does not.

2026-09-08 23:40 - #128 - the chart registers a panel in the shell

The chart now joins the Electron shell's roster of panels. Before this change the shell listed fifteen panels and said of this one that it registered nothing to draw. It now lists sixteen and says nothing.

`src/gui/web/native_chart.js` — the chart joins the roster

```
  if (global.acervatorPanelHost) {
    global.acervatorPanelHost.register({
      render: renderChart,
      load: loadChart,
      loadError: loadError
    });
  }
```

The chart takes no tab of its own. It names no bridge method the application serves a tab from, so the tab bar passes over it and the chart draws where it belongs, inside the Charts tab. The ten tabs the application reports are the same ten before and after.

The chart in the shell still draws its waiting line rather than a price. The Charts tab asks it for a symbol and a timeframe and sends none of the 180 candles the tab itself holds, and the symbol it sends is empty. Both values belong to the Charts tab file, not to the chart, so the row below keeps `no` under Registers in Electron until they are carried across.

2026-09-09 01:20 - #128 - the Charts tab hands the chart its data

The chart in the shell draws the price now. The tab passes it the asset it is following, the feed name, the candles it is holding and the size of the space the chart has, and the shell's panel host does the drawing rather than the tab reaching past it.

`src/gui/web/trade_charts_tab.js` — what the chart slot asks for

```
          return global.acervator.call(api.method, {
            reset: true,
            symbol: symbol,
            timeframe: mount.getAttribute(CHART_TF_ATTR),
            source: text(panel[SOURCE]),
            candles: candles,
            width: mount.clientWidth,
            height: mount.clientHeight
          });
```

The panel carries the asset it follows. It was built once with a blank name and renamed only its header line, so the name the chart was asked for was always blank.

`src/gui/main_tabs/trade_charts_tab_surface.py` — the panel's own asset

```
    def set_symbol_property(self, symbol: Any) -> None:
        """Rename the chart through its public setter, which repaints."""
        self.symbol = symbol
        self.label = symbol
```

Both builds now draw the same header, the same 180 candles, the same last price of 78,623.00, the same three axis prices and the same time axis. The two pictures still differ on the indicator lines. The Bollinger bands, the Ichimoku cloud and the Vortex, MACD and Stochastic RSI panes draw in the Qt build and in neither place in the shell, because nothing hands the chart those five readings yet. The eight indicator switches under the Qt chart are absent in the shell for the same reason. The row below keeps `no` under Registers in Electron until the readings arrive.

2026-09-09 03:05 - #128 - the indicators draw in both builds

The chart in the shell draws every indicator the Qt chart draws. The five readings now reach it: the Bollinger bands with their cloud, the Ichimoku cloud with its four lines and the lagging line, and the Vortex, MACD and Stochastic RSI panes under the price. The eight switches sit under the chart with Slingshot and BB Bullseye off, and the two position markers sit on the toolbar beside the feed name.

Each reading is the one the indicator itself published. The chart asks the five indicator classes for their values and paints what they answer.

`src/gui/main_tabs/native_chart_surface.py` — the chart takes its readings

```
        self.set_indicator_series(
            bb=BollingerBands(BOLLINGER_PERIOD, BOLLINGER_STD).bands(candles),
```

Both builds were driven from the same 38 running bots and the same 180 daily A15/USD candles. Both draw the same header, the same candles, the same last price of 78,623.00, the same four axis prices, the same time axis, and the same three panes reading Vortex 0.8761, MACD -245.82 and Stoch RSI 0.2318. The row below now reads `yes` under Registers in Electron.

Two figures moved to make that true. The Qt chart asked for 492 pixels and was given 250 until an indicator switch was pressed, so its MACD pane, its Stochastic RSI pane and its time axis were cut off; it now takes its full height as soon as the candles arrive. The panes were also listed as MACD, Vortex, Stochastic RSI in one

place and Vortex, MACD, Stochastic RSI in the other; the switch order decides, so both now read Vortex, MACD, Stochastic RSI.

The volume bars are empty in both pictures. The public feed serves no volume with its daily candles, so that one row of the comparison rests on two empty strips.

`src/gui/theme_engine.py` — the chart tokens each theme carries

```
chart_bg_top: str = "#08080e"
chart_up: str = "#00e5a0"
chart_down: str = "#ff2d6f"
chart_zone_fold: str = "#fcee0a"
```

Detail: 08-tabs/asset-charts.md.

Live Tab

The Live tab is where the bots trade real money on a real exchange. The exchange connections fill the left, the Indicator Voting Panel fills the right, and the Activity Log and the API Interaction Log close the foot.

The tab is now called Live. It sits third on the bar, straight after Paper, on the gold ground with red text.

2026-09-09 06:35 - #128 - the Live tab's two widgets register a panel

The Indicator Voting Panel and the Activity Log now register with the shell's panel host, and the Live tab draws both through it. Before the change the shell listed sixteen registered panels and named these two as modules that draw nothing; it lists eighteen now and names neither.

`src/gui/web/status_log.js` — what each of the two modules adds

```
  if (global.acervatorPanelHost) {
    global.acervatorPanelHost.register({
      render: renderLog,
      load: loadLog,
      loadError: loadError
    });
  }
```

Neither registration names a bridge method, so the tab bar gives neither a tab of its own. The Live tab keeps a slot for each and hands the slot to the host, which is how the Charts tab already draws the price chart.

`src/gui/web/trading_tab.js` — the Live tab's slot for the Activity Log

```
    slot.setAttribute(CHILD_ATTR, STATUS_LOG_MODULE);
    return host.mount(STATUS_LOG_MODULE, slot, model) ? slot : null;
```

The host is the draw path now, not a second name for it. With both registrations taken out and the same reading driven in, each slot fell from 43,264 and 4,532 characters of markup to 82 and 72, and each carried the sentence naming the panel that did not draw.

Both builds were driven with one reading: 180 daily A15/USD candles from CoinGecko through the voting engine, at 15m, 1h and 1d, and eight Activity Log messages covering a placed, a filled and a sent trade, a wire

flow, a wire stack, a warning and an error. The Qt tab was built at the size the shell gives the React tab, 1386 by 708.

The twelve indicator columns carry the same name, direction and bar in both. Each bar fills its share of a 70-pixel plot.

column	direction	confidence	Qt height	React height
BB	NEUTRAL	0.000	2.00	2.00
VTX	BEARISH	0.328	22.96	22.69
MACD	BEARISH	0.150	10.50	10.38
SRsi	BEARISH	0.200	14.00	13.84
Ichi	BEARISH	0.210	14.70	14.53
Vol	NEUTRAL	0.000	2.00	2.00
Sling	NEUTRAL	0.000	2.00	2.00
ADX	BEARISH	0.267	18.69	18.48
STrd	BULLISH	0.261	18.27	18.05
ZSc	NEUTRAL	0.000	2.00	2.00
KER	NEUTRAL	0.000	2.00	2.00
RSI	NEUTRAL	0.000	2.00	2.00

Net, Comp and Conf stand in the last three of the ten columns in both, take the same three directions, and run from the ceiling of the top row of bars to the floor of the bottom row. Only their bases carry a label, in a strip 22 pixels deep under that floor.

No horizontal line crosses the three. Every element in the React panel was read for a top border, a bottom border or a background gradient: ninety-three carry one, and every box ends at or before 1185.93, where the Net column starts. The 1 pixel each indicator column carries is that column's own base, which Qt paints as a line from the left margin to the x of the Net column. The dotted increments sit on those same cells and on no other.

	Qt	React
ceiling	171	267.2
floor	428	523.2
label strip	428 to 450	523.2 to 545.2
columns	8, 9, 10 of 10	8, 9, 10 of 10
rules end at	437, the Net column	1185.9, the Net column

The Activity Log draws the same eight lines in the same order with the same text. Each line takes the colour of its kind, and a trade line is raised to 14 pixels and bold.

line	colour	size	weight
Bot a15_core started	rgb(0,255,204)	12	normal
SCRUM PLACED	rgb(255,170,0)	14	bold
SCRUM FILLED	rgb(0,255,136)	14	bold
WIRE FLOW	rgb(255,102,221)	12	bold
WIRE STACK	rgb(255,204,68)	12	bold
Coinbase rate limit	rgb(255,170,0)	12	normal
Reconciliation refused	rgb(255,51,102)	12	normal
FOLD SENT	rgb(0,255,204)	14	bold

Each mini-panel table shows its header and two rows in both builds, and both cut the 1d row off the bottom. Qt fixes the table to that height and turns both scrollbars off; the React table read neither of the two numbers the payload publishes for it, stood two rows taller, and pushed the three pillar labels and the six lower bar labels below the panel. It reads them now.

`src/gui/web/indicator_panel.js` — the height the table body takes

```
    var bodyStyle = {
      display: BLOCK,
      maxHeight: length(slackHeight(table)),
      overflow: HIDDEN
    };
```

One line is drawn in one picture and not the other. The panel's locks line, `No active timeframe locks`, sits inside the Qt panel and two pixels below the bottom of the React one, which reaches it by scrolling. The Live tab's own splitter is what differs: Qt gives its top section 472 pixels of the 708 the tab has, the React tab gives it 411, and the panel cannot draw shorter than 415 because each bar graph carries a 100-pixel minimum in both.

Detail: 06-trading-tab.md. Detail: 07-indicators.md.

Converting the Interface to React

Every screen in this application is drawn by Qt. Issue #128 replaces them with React and keeps the backend in Python. This section is the running record of that work. One row per screen, one column per step, and a mark only where the step is finished.

One screen draws React in the application today: the History table. It runs inside the Chromium that the Qt toolkit already ships, not inside Electron.

`src/gui/react_history_panel.py` — `HistoryWebTable._setup_ui`

```
self._web = QWebEngineView()
self._web.loadFinished.connect(self._on_load_finished)
self._web.setHtml(panel_html(theme))
layout.addWidget(self._web, 1)
```

What the tree holds

Measured on 5 September 2026 over the tracked files.

```
React modules             70 files, 80,292 lines, 0 stub markers
                          65 of them call React.createElement
                          React and ReactDOM are vendored on disk
View-model modules        74 files, 74,207 lines
Parity tests              72 files, 170,244 lines
Electron shell             8 code files, 791 lines, plus a package manifest
Renderer manifest         70 names
Qt modules under src/gui  69, every one importing PySide6
Tabs drawn by React        0
```

No file in this repository has ever carried a JavaScript extension other than plain `.js`. A history search over every branch returns no commit that added, deleted or renamed one, and the same search for the plain extension returns 81 paths, so the search does find a file that existed.

```
git log --all --diff-filter=ADR --name-only -- "*.jsx" "*.tsx" "*.ts"   no commits
git log --all --diff-filter=ADR --name-only -- "*.js"                   81 paths
```

What the Electron shell drew

The shell has been started once, and what it drew is measured. All 70 renderer modules loaded and none failed. Two of them register a panel for the page to draw, so the page had nothing to ask the other 68 for. The window showed an empty History table, with no tab bar and no navigation.

```
React modules the page list names   70
modules that failed to load          0
modules that register a panel        2
panels the page asked to draw        2
panels that failed to draw           0
```

The two that register are the Console tab panel and the header strip. The full per-file record is in the 2026-09-05 file-verification audit, removed with the audit folders.

The tab bar the shell draws now

The shell builds the same ten tabs the application builds, with the same labels and in the same order. It holds no list of its own. `main_window.state` reports `tab_labels` and `tab_methods`; the bar keeps one tab per label, filled by the registered panel that calls that label's bridge method.

```
Sim  Paper  Live  Charts  Inspector
Swarm  Accumulation  History  Status  Console
```

A module that draws a screen inside a dialog registers no bridge method and takes no tab. `market_inspector_tab.js` and `bot_swarm_tab.js` are the two: both belong to the Live Bot Settings dialog.

Measured in one Electron launch, with the backend serving a two-bot fleet:

```
tabs on the bar                    10
panels that drew text              10
panels that failed to draw          0
child panels drawn inside a tab     7
```

The header strip stays above the bar for every tab.

How to read the table

Each row is a Qt module the running window builds. The columns are the steps of the conversion, in the order the work happens.

Column	What the mark means
React module	A renderer module in `src/gui/web` serves this screen
Surface	A Python view model of the screen exists, with no toolkit in it
Bridge	The view model is registered as a method the frontend may call
Manifest	The renderer module is named in the shell's module list
Ships in the build	The built React application carries the module inside it
RENDERS	What the operator sees when he opens that screen
Scope	Whether the row is work this item must do

The last column is the item. A row is finished only when it reads `yes`. Every other column can be marked while the operator sees no change, and that is the difference between wiring written and a screen replaced.

The RENDERS column takes three values, and none of them is derived from the bridge or the manifest. `yes` means React draws where Qt used to. `shell` means the module draws in the Electron shell and nowhere else. A dash means the Qt widget is still the screen.

The Scope column names the module that builds a screen the item has still to convert. Header strip and window chrome are the parts of the Main Window that sit outside the tab row.

The table carries nine columns and no tab column. The screen a module draws is named in that screen's own section above. Header strip and window chrome are the parts of the Main Window that sit outside the tab row.

The Scope column takes five values.

Value	What it means
in scope	The running window builds this screen, and the cell names what builds it
React side	The file exists because of this conversion, so a React module for it would be a module for a module
not a screen	The module defines no view
builds the shell	The module fills the tab bar rather than sitting in it
shelved	Nothing the window builds reaches it

The walk that answers the column starts at the main window. It follows an imported name only where the module calls that name, subclasses it or reads an attribute of it, so a re-export block reaches nothing. A row is in scope when some module on that walk builds one of the classes the row defines.

`tools/conversion_scope.py` writes the column, so the answers are re-measured rather than typed.

The conversion table

Qt file	React module	Uses React	Bridge	Manifest	Registers in Electron	Ships in the build	RENDERS	Scope
splash_screen.py	no	-	yes	no	no	-	no	shelved
src/gui/alerts_tab.py	alerts_tab.js	yes	yes	yes	-	yes	yes	in scope
src/gui/analytics_tab.py	analytics_tab.js	yes	yes	yes	no	yes	no	shelved
src/gui/audio_suite.py	audio_suite.js	yes	yes	yes	no	yes	no	shelved
src/gui/bot_live_settings.py	bot_live_settings.js	yes	yes	yes	yes	yes	yes	in scope
bot_swarm_list	removed	-	-	-	-	-	-	deleted
src/gui/bot_visualizer.py	bot_visualizer.js	yes	yes	yes	yes	yes	yes	in scope
src/gui/bot_wizard.py	bot_wizard.js	yes	yes	yes	-	yes	yes	in scope
src/gui/buy_confirmation_dialog.py	buy_confirmation.js	yes	yes	yes	-	yes	yes	in scope
src/gui/competition_tab.py	competition_tab.js	yes	yes	yes	no	yes	no	shelved
src/gui/crypto_news_ticker.py	crypto_news_ticker.js	yes	yes	yes	-	yes	shell	in scope
src/gui/history_qt_table.py	no	-	no	no	no	-	no	React side
src/gui/history_tab.py	history_tab.js	yes	yes	yes	yes	yes	yes	in scope
src/gui/indicator_panel.py	indicator_panel.js	yes	yes	yes	yes	yes	shell	in scope
src/gui/init_wizard.py	init_wizard.js	yes	yes	yes	no	yes	no	shelved
src/gui/instance_consent_dialog.py	instance_consent_dialog.js	yes	no	yes	no	yes	no	shelved
src/gui/journal_tab.py	journal_tab.js	yes	yes	yes	no	yes	no	shelved
src/gui/launcher.py	launcher.js	yes	yes	yes	no	yes	no	shelved
src/gui/live_bot_window.py	no	-	yes	no	no	-	no	shelved
src/gui/live_settings/bot_swarm_tab.py	bot_swarm_tab.js	yes	yes	yes	yes	yes	yes	in scope

Qt file	React module	Uses React	Bridge	Manifest	Registers in Electron	Ships in the build	RENDERS	Scope
src/gui/live_settings/fold_chrome.py	fold_chrome.js	yes	yes	yes	-	yes	yes	in scope
src/gui/live_settings/fold_tokens.py	fold_tokens.js	yes	yes	yes	no	yes	no	not a screen
src/gui/live_settings/fold_tranches_tab.py	fold_tranches_tab.js	yes	yes	yes	-	yes	yes	in scope
src/gui/live_settings/market_inspector_tab.py	market_inspector_tab.js	yes	yes	yes	yes	yes	yes	in scope
src/gui/live_settings/phantom_bots_tab.py	phantom_bots_tab.js	yes	yes	yes	-	yes	yes	in scope
src/gui/live_settings/positions_held_tab.py	no	-	no	no	no	-	-	in scope
src/gui/live_settings/settings_tab.py	live_settings_tab.js	yes	yes	yes	no	yes	yes	in scope
src/gui/live_settings/stack_tranches_tab.py	stack_tranches_tab.js	yes	yes	yes	-	yes	yes	in scope
src/gui/live_settings/status_tab.py	no	-	yes	no	no	-	yes	in scope
src/gui/main_tabs/audio_suite_surface.py	no	-	yes	no	no	-	no	React side
src/gui/main_tabs/buy_confirmation_surface.py	no	-	yes	no	no	-	no	React side
src/gui/main_tabs/console_log_handler.py	no	-	no	no	yes	-	-	in scope
src/gui/main_tabs/console_tab.py	console_tab.js	yes	yes	yes	yes	yes	yes	in scope
src/gui/main_tabs/empty_tabs.py	no	-	no	no	no	-	-	shelved
src/gui/main_tabs/header_strip.py	header_strip.js	yes	yes	yes	yes	yes	yes	in scope
src/gui/main_tabs/proof_of_accumulation_tab.py	proof_of_accumulation_tab.js	yes	yes	yes	yes	no	shell	in scope
src/gui/main_tabs/stock_main_window_surface.py	no	-	yes	no	no	-	no	React side
src/gui/main_tabs/system_status_tab.py	system_status_tab.js	yes	yes	yes	yes	no	yes	in scope
src/gui/main_tabs/trading_tab.py	trading_tab.js	yes	yes	yes	yes	yes	-	in scope
src/gui/main_tabs/tradingview_chart_surface.py	no	-	yes	no	no	-	no	React side
src/gui/main_window.py	main_window.js	yes	yes	yes	no	no	yes	in scope

Qt file	React module	Uses React	Bridge	Manifest	Registers in Electron	Ships in the build	RENDERS	Scope
src/gui/market_inspector.py	market_inspector.js	yes	yes	yes	yes	yes	yes	in scope
src/gui/market_inspector_topologies.py	market_inspector_topologies.js	yes	yes	yes	yes	yes	yes	in scope
src/gui/native_chart.py	native_chart.js	yes	yes	yes	yes	yes	yes	in scope
src/gui/qt_safe_events.py	no	-	yes	no	no	-	no	not a screen
src/gui/react_history_panel.py	no	-	yes	no	no	-	no	React side
src/gui/react_history_tab.py	no	-	no	no	no	-	no	React side
src/gui/react_proof_of_accumulation_tab.py	no	-	no	no	no	-	no	React side
src/gui/react_simulator_tab.py	no	-	no	no	no	-	no	React side
src/gui/react_system_status_tab.py	no	-	no	no	no	-	no	React side
src/gui/settings_dialog.py	settings_dialog.js	yes	yes	yes	-	yes	yes	in scope
src/gui/shared_testnet.py	shared_testnet.js	no	yes	yes	no	yes	no	shelved
src/gui/simulator_tab.py	simulator_tab.js	yes	yes	yes	yes	yes	yes	in scope
fleet_replay_panel	fleet_replay_panel.js	yes	yes	yes	no	yes	yes	shelved
sim_visuals	sim_visuals.js	yes	yes	yes	no	yes	yes	shelved
nuclear_mode_panel	nuclear_mode_panel.js	yes	yes	yes	no	yes	yes	shelved
sim_stat_strip	sim_stat_strip.js	yes	yes	yes	no	yes	yes	shelved
SimulatorTab	simulator_tab.js	yes	yes	yes	yes	yes	yes	shelved
src/gui/start_all_progress_dialog.py	start_all_progress.js	yes	yes	yes	-	yes	yes	in scope
src/gui/stock_main_window.py	no	-	yes	no	no	-	no	shelved
src/gui/testnet_tab.py	testnet_tab.js	yes	yes	yes	no	yes	no	shelved
src/gui/tradingview_chart.py	tradingview_chart.js	yes	yes	yes	no	yes	no	shelved

Qt file	React module	Uses React	Bridge	Manifest	Registers in Electron	Ships in the build	RENDERS	Scope
src/gui/usb_auth_widget.py	usb_auth_widget.js	yes	yes	yes	no	yes	no	shelved
src/gui/visualizer/bot_node.py	bot_node.js	yes	yes	yes	yes	yes	yes	in scope
src/gui/visualizer/quick_routing.py	quick_routing.js	yes	yes	yes	yes	yes	yes	in scope
src/gui/visualizer/themes.py	visualizer_themes.js	-	yes	yes	yes	yes	yes	in scope
src/gui/visualizer/wire_canvas.py	wire_canvas.js	yes	yes	yes	yes	yes	yes	in scope
src/gui/widgets/__init__.py	no	-	yes	no	no	-	no	not a screen
src/gui/widgets/api_tester_tab.py	api_tester_tab.js	yes	yes	yes	no	yes	no	shelved
src/gui/widgets/bot_selection.py	bot_selection.js	yes	yes	yes	no	yes	no	not a screen
src/gui/widgets/bot_status_table.py	bot_status_table.js	yes	yes	yes	-	yes	yes	in scope
capital_registry_panel	no	-	no	no	no	-	no	shelved
src/gui/widgets/dashboard_stat_card.py	dashboard_stat_card.js	yes	yes	yes	-	yes	yes	in scope
src/gui/widgets/exchange_tab.py	exchange_tab.js	yes	yes	yes	no	yes	-	in scope
src/gui/widgets/extractor_bot_table.py	extractor_bot_table.js	yes	yes	yes	no	yes	yes	in scope
src/gui/widgets/notification_spool.py	notification_spool.js	yes	yes	yes	no	yes	no	shelved
src/gui/widgets/privacy_dot.py	privacy_dot.js	yes	yes	yes	-	yes	yes	in scope
src/gui/widgets/pulse_manager.py	pulse_manager.js	yes	yes	yes	no	yes	no	shelved
src/gui/widgets/spendable_profits.py	spendable_profits.js	yes	yes	yes	yes	yes	yes	in scope
src/gui/widgets/status_log.py	status_log.js	yes	yes	yes	yes	yes	shell	in scope
src/gui/widgets/trade_charts_tab.py	trade_charts_tab.js	yes	yes	yes	yes	yes	yes	in scope

The Simulator rebuild removed the files above; they are not in the tree.

The Capital Registry row names a file the tree no longer holds. The registry it drew was built nowhere, so the table could draw no row. Four modules went together: `capital_registry_panel`, its view model, its renderer page, and the registry behind them. The bridge no longer offers the method, and the renderer manifest no longer names the page.

The table holds no row for the two Paper Trader hosts. The Qt host and the React host were removed together, and the Paper tab is drawn in Qt by `src/gui/paper/paper_trading_tab.py, in PaperTradingTab` today.

Totals across the 76 rows above, measured on 5 September 2026:

```
React module             63
Uses React               60
Bridge                   72
Manifest                 62
Registers in Electron    25
Ships in the build       58
RENDERS                  44
RENDERS, in scope        35 of 43
out of scope             41
```

Four columns are all but complete. RENDERS, the column that is the item, is not.

Re-measured on 6 September 2026, counted off the 76 rows above:

```
React module             55
Uses React               53
Bridge                   67
Manifest                 55
Registers in Electron    15
Ships in the build       55
RENDERS                  29
```

Only the last column moved. The Settings row already carried a React module, a bridge method and a manifest entry; what it gained is the screen.

Re-measured later on 6 September 2026, counted off the same 76 rows:

```
React module             55
Uses React               53
Bridge                   67
Manifest                 55
Registers in Electron    15
Ships in the build       55
RENDERS                  30
RENDERS, in scope        30 of 43
out of scope             33
```

The Create Auto Trader wizard is the row that moved. `bot_wizard.js` draws it into a space the React exchange screen keeps once + New Bot is pressed, so the wizard is a screen and no longer a module with nowhere to draw.

Counted once more the same day, after the Live Bot Settings window went over:

```
React module             55
Uses React               53
Bridge                   67
Manifest                 55
Registers in Electron    15
Ships in the build       55
RENDERS                  38
RENDERS, in scope        38 of 43
out of scope             33
```

Eight rows moved together: the window a bot row opens, six of the files its tabs are built from, and the delegate that draws the fold-tranche row borders. `fold_tokens.py` keeps its dash. Its colours are on the drawn rows, and the file names no screen of its own.

When that count was taken the Paper Trader, Proof of Accumulation and System Status tabs had no Qt module to replace and no React module to replace it with, so the table carried no row for them. `empty_tabs.py` drew all three empty, and its own row read `builds the shell`. Issue #19 has since built the Paper tab, which now has a Qt module and a React module of its own.

In development.

That is no longer the state. Each of the three has a renderer module in `src/gui/web`, a bridge method and a manifest entry, and `empty_tabs.py` carries the row for all three. Its Scope cell reads `in scope` and its RENDERS cell reads `yes`. The section below says what each tab draws.

Counted once more on 6 September 2026, after the header strip and the Start All dialog went over:

```
React module             58
Uses React               56
Bridge                   69
Manifest                 58
Registers in Electron    15
Ships in the build       58
RENDERS                  42
RENDERS, in scope        42 of 43
out of scope             33
```

Six rows moved. Three are the header strip parts. One is the Start All progress dialog. One is the Live Bot Settings screen tab, where the React module cell was the wrong one and now names the module that draws it. The last is the buy confirmation dialog, which gained a module, a bridge method and a build entry and still reads `no` under RENDERS.

Counted again on 6 September 2026, after the buy confirmation dialog went over:

```
React module             58
Uses React               56
Bridge                   69
Manifest                 58
Registers in Electron    15
Ships in the build       58
RENDERS                  43
RENDERS, in scope        43 of 43
out of scope             33
```

One row moved. It is the buy confirmation dialog, and it was the last in-scope row reading `no`. The count comes from `tools/conversion_table.py`, which reads the cells of the table above.

Counted again on 6 September 2026, after the window's own tab bar went over:

```
React module             59
Uses React               57
Bridge                   69
Manifest                 59
Registers in Electron    15
Ships in the build       58
RENDERS                  44
RENDERS, in scope        44 of 44
out of scope             32
```

One row moved, and it is `main_window.py`. The row read `builds the shell` because the module filled the tab bar rather than sitting in it; the bar it fills is now React, so the row is in scope and it renders. `Ships in the build` stays `no`: the newest bundle under `dist` carries 70 renderer modules and `main_window.js` is not one of them, because the module is newer than that build.

Counted again on 6 September 2026, after the three unbuilt tabs went over:

```
React module             60
Uses React               58
Bridge                   70
Manifest                 60
Registers in Electron    16
Ships in the build       58
RENDERS                  45
RENDERS, in scope        45 of 45
out of scope             31
```

One row moved, and it is `empty_tabs.py`. It is the second row that read `builds the shell`, and it is the only row naming three React modules: the Paper Trader, System Status and Proof of Accumulation skeletons each have a renderer module, a bridge method and a manifest entry, and all three register a panel with the Electron shell. `Ships in the build` reads `no` for the same reason as the row above: the newest bundle under `dist` carries 70 renderer modules and these three are not among them.

No row now reads `builds the shell`.

Counted again on 6 September 2026, after the Notifications and Alerts tab went over:

```
React module             60
Uses React               58
Bridge                   70
Manifest                 60
Registers in Electron    16
Ships in the build       58
RENDERS                  46
RENDERS, in scope        45 of 45
out of scope             31
```

One row moved, and it is `alerts_tab.py`. Its Scope cell still reads `shelved`, because nothing the running window builds reaches the screen. `variant_surface` now holds the row under the name `ALERTS`, so the React build makes `AlertsReactTab` and the Qt build makes `AlertsTab` unchanged. The React tab draws the whole screen in one web view from `src/gui/web/alerts_tab.js`, and `AlertsTab.refresh`, `AlertsTab._save_config`, `AlertsTab._test_telegram` and `AlertsTab._acknowledge_all` are the same methods on both sides.

2026-09-09 10:31 - #128 - the Live Bot Settings window fills its tab pages

The window a bot row opens already registered with the shell's panel host. Its cell read `no`, and the running shell named it among nineteen registered panels, so the cell was stale.

```
registered  bot_live_settings bot_swarm_tab bot_visualizer console_tab
            header_strip history_tab indicator_panel market_inspector
            market_inspector_tab market_inspector_topologies native_chart
            paper_trader_tab proof_of_accumulation_tab simulator_tab
            spendable_profits status_log system_status_tab trade_charts_tab
            trading_tab
```

Registering was never the whole of it. The window drew its title, its tab strip and its two footer buttons, and every one of its seven tab pages was an empty box. Read off the drawn page, the window held 28 elements and each page held none.

```
part                   children  characters
status-page                   0           0
settings-page                 0           0
fold-tranches-page            0           0
stack-tranches-page           0           0
bot-swarm-page                0           0
market-inspector-page         0           0
phantom-bots-page             0           0
```

Each tab module already carried a filler that looks for the empty space this window leaves. Nothing called it. The window now asks each module's own loader for its view model and hands it the page.

```
    var wait =
      typeof loader === "function"
        ? loader(tabRequest(model))
        : Promise.resolve(null);
    return Promise.resolve(wait).then(function (found) {
      return api.fill(space, found) === null ? null : space;
    });
```

A tab is asked for the bot the window built, never for one of its own. The window's answer now names that bot, and the request carries it under the name the window's own surface publishes.

```
    config = model.bot.config
    return {
        "bot_id": model.bot.bot_id,
        "symbol": config.symbol,
        "mode": config.mode,
        "state": model.bot.state,
    }
```

Three pages fill from it. Fold Tranches draws eleven health rows, three Clear buttons, an order picker of five choices and a filter box. Stack Tranches draws seven summary rows and two Clear buttons. Phantom Bots draws four summary rows, twelve timeframe boxes and the candles-to-lock number. Take the bot out of the window's answer and every one of those rows goes.

Now: a bot holding a Shadow Tranche draws one more Stack Tranches summary row, and one more detail row for each Shadow Tranche (`src/gui/main_tabs/stack_tranches_tab_surface.py, in compose_shadow_rows`).

```
page                  rows  order choices  elements   with the bot removed
fold-tranches-page      11              5        69    0,  5, 33
stack-tranches-page      7              0        33    0,  0, 12
phantom-bots-page        4              0        92    0,  0, 46
```

The order picker survives the removal because the fold chrome is asked for its own controls rather than for a bot, which is what proves the row counter was reading the page and not returning zero everywhere.

Every page drew at once before this. A page not on show was marked hidden and still carried an inline display, which beats the browser's own rule for a hidden element. The page not on show now takes `display: none`, so one tab is on screen at a time.

```
    var style = {
      display: props.current === true ? FLEX : DISPLAY_NONE,
```

The window's own stylesheet had never reached the shell page, so its tab buttons painted white on grey. The shell now links it. Its rules were written for a page holding this window alone, and the shell page holds every panel: 17 of the 20 tables on that page belong to other panels, as do 5 of the 12 tab buttons. Each rule is scoped to the window before the link, and five of six other panels render byte-identical to the run before it.

```
[data-part="bot-live-settings"] table {
  border-collapse: collapse;
  width: 100%;
}
```

The Settings page stays empty and its row stays `no`. Its surface builds from the bot's own settings values, and it stops on the first one the window cannot supply.

```
AttributeError: 'BotConfigSource' object has no attribute 'visibility'
```

The window answers a stand-in bot, not a running one. Giving it a running one would also point Apply Changes at that bot, which is a change to what a control writes and is not this unit's to make.

The exchange screen and the Extractor table are drawn inside the Live tab, not by the shell. Both rows stay `no`: the Live tab names an exchange only when the shell's backend is the trading program itself, and the backend reachable here names none, so neither child is built.

```
    slot.setAttribute(CHILD_ATTR, EXCHANGE_MODULE);
    return api.renderTab(slot, shown.model);
```

The Qt window could not be built beside the React one. It reads a mode and a state that carry a `value`, then dozens of settings fields with no fallback, so it needs a bot the running program holds.

```
AttributeError: 'str' object has no attribute 'value'
```

The window's own tab bar

The row of tab names across the top of the window is drawn by React under the React build. `MainWindow._setup_ui` no longer builds a `QTabWidget` by name. It asks the variant seam for the tab book, so one builder gives the operator the Qt bar he runs today or the React one.

```
            self._main_tabs = _main_tab_book_class()()
            self._main_tabs.setMovable(True)
```

Under the Qt build the seam answers with the widget the operator runs today and nothing changes. Under the React build it answers with a book that keeps that widget as the stack of pages, hides its own bar, and puts a web view above it. Every call the window makes is answered over the widget behind the bar, so the tab order and the pages are the same objects on both sides.

```
        def addTab(self, widget, label: str) -> int:
            """Add ``widget`` under ``label`` and redraw the bar."""
            index = self._book.addTab(widget, label)
            self._push_tabs()
            return index
```

The bar itself is the Electron shell's navigation, run inside the window. The page is built from the shell's own scripts, read off disk, and the buttons are drawn by one React module both sides reach. Neither host holds a

bar of its own, so the two cannot draw a different one.

```
  #: The shell's own navigation, read from ``desktop/renderer`` and inlined.
  SHELL_SCRIPTS: tuple[str, ...] = ("panel_host.js", "tab_bar.js")
```

The shell's navigation asks that React module for the buttons and passes it the tab names, the selected one, and what to do with a press and a drop. It holds no button of its own.

```
    drawn.renderTabBar(bar, {
      names: found,
      selected: current,
      onSelect: select,
      onMove: onMove
    });
```

A press on a button is reported back to Python, which moves the real tab book to that tab. Dragging a tab onto another reports the two slots the same way, and the same handler hands them to the hidden bar, so a drag reorders the pages as it did before.

```
            move = request.get("move") or {}
            if "from" in move and "to" in move:
                self._book.tabBar().moveTab(int(move["from"]), int(move["to"]))
```

The menu bar, the window title, the window icon and the status bar are still Qt. `MainWindow._setup_menu` builds the four menus and `_setup_status_bar` builds the status line; neither was changed, and every menu action the window offered before it is on it now.

The three tabs with nothing behind them yet

Paper Trader, System Status and Proof of Accumulation were named on the tab bar and were not built when that count was taken. Each one drew a heading, a sentence saying the tab is not built, and the issue that carries the build-out. Every word of it came from that tab's own Python surface, so the page invented nothing. Two of the three are still empty; issue #19 built the Paper tab.

```
    return {
        "accessible_name": HEADING,
        "built": BUILT,
        "heading": HEADING,
        "issue": ISSUE,
        "issue_text": ISSUE_TEXT,
        "method": METHOD,
        "state_text": STATE_TEXT,
    }
```

Under the Qt build the three sentences are Qt labels, as before. Under the React build the same view model goes to the tab's renderer module, which draws it in a web view. The builder asks the variant seam which of the two to make, and it hands both the same values.

```
        model = surface.view_model({})
        panel = _empty_tab_class()(model)
```

Neither side holds a colour or a size of its own. `SKIN` names the ground, the two text colours, the two text sizes, the padding and the gap; the Qt style sheets are built from it and the page reads it as custom properties.

```
SKIN = {
    "--empty-tab-ground": ds.SURFACE_0,
    "--empty-tab-heading-colour": ds.TEXT_MAX,
    "--empty-tab-body-colour": ds.TEXT_EMPTY_STATE,
```

The page runs the Electron shell's `panel_host.js`, so the module is drawn the same way the shell draws it: by name, into one host element, with the reason written onto the page when it does not draw.

The header strip

The strip and the five counters above the tab row are no longer built by name. `HeaderStripMixin._build_header_strip` asks the variant seam for each class, so one builder draws the Qt widgets or the React pages.

```
        self._spendable_widget = _spendable_profits_class()()
        top_row.addWidget(self._spendable_widget, stretch=3)

        card_class = _stat_card_class()
        self._stat_scrummed = card_class("Scrummed", "$0.00")
```

The spendable strip is drawn by `SpendableProfitsReact`, which hosts the strip module in one web view. It answers the two calls the main window makes, and writes both into the same view model the Qt strip is described by.

```
        def update_profits(self, data: dict) -> None:
            self._model.update_profits(data)
            self._push()
```

Each column carries a privacy dot drawn by the dot module. A press on one runs the model in `privacy_dot_surface`, which flips the field in the process privacy registry and repaints the amount as four stars.

```
    def clicked(self) -> None:
        self.calls.append(CLICKED)
        try:
            registry = get_privacy_mask_registry()
            registry.set_masked(self._field_id, not registry.is_masked(self._field_id))
        except Exception:
            self.calls.append(TOGGLE_FAILED)
        self.refresh()
```

One card is drawn by `StatCardReact`. A press on the card emits the card's one signal, and only while the header strip has armed it.

The glyph, the tooltip and the skin of a dot now have one home. Both header surfaces return what the dot surface paints, so the card and the strip cannot drift apart.

```
def privacy_dot(field_id: str, masked: Optional[bool] = None) -> dict:
    painted = privacy_dot_surface.dot_view(field_id, masked)
    return {name: painted[name] for name in _DOT_FIELDS}
```

The Start All progress dialog

The dialog the Start All button opens is drawn by `StartAllProgressReactDialog`. It replaces only the method that builds the controls. The progress handling, the Cancel press and the unsubscribe on close are the same code the Qt dialog runs.

```
        def store(self) -> surface.StartAllProgressModel:
            return self._model
```

Every control it needs is a small holder over that store. The list answers `addItem` and `count`; the buttons answer `setEnabled`. Both build sites, in the main window and in the launcher, now ask the variant seam for the class.

The buy confirmation dialog

This is the one in-scope row that still reads no, and the reason is reachability rather than missing work.

The main window builds the broker in this file. Nothing calls the broker's request method, which is the only sender of the signal that opens the dialog, so the running window has no path to the screen.

```
        async def request_confirmation(
            self,
            *,
            bot_id: str,
            symbol: str,
            reason: str,
```

Measured by walking every call in `src` and `main.py`: zero call sites, and no import of that name under another. The module, the bridge method and the build entry are all present, so the screen is one caller away.

Re-measured on 6 September 2026, by the same walk over 354 modules: still zero call sites. That count decides what the operator can reach, and it did not move. The row's RENDERS cell moved for a different measurement — the class the broker raises.

The dialog the broker raises is now `BuyConfirmationReactDialog`. The broker asks the variant seam for that class, so the React build opens the page and `ACERVATOR_VARIANT=qt` opens the same Qt widgets as before.

```
def dialog_class() -> type:
    """The confirmation dialog class the running build variant draws."""
    from .variant_surface import BUY_CONFIRMATION, surface_class

    return surface_class(BUY_CONFIRMATION)
```

Every figure the page shows is written in Python. `details_text` in `buy_confirmation_surface` writes the seven detail rows. The Qt label is set from it, and the browser draws the same seven lines sliced into their values. No arithmetic runs in the page.

```
def after_buy_usd(holdings_before: float, price: float, cost_usd: float) -> float:
    """Dollar value the position reaches once this buy fills."""
    return holdings_usd(holdings_before, price) + cost_usd
```

A press on the page names a button. Python turns that name into an answer and accepts the dialog through the same `_answer` the Qt buttons call. The page places no order and decides nothing.

```
        def run_action(self, name: str) -> bool:
            answer = surface.BUTTON_ANSWERS.get(name)
            if answer is None:
                return False
            self._answer(answer)
            return True
```

A window closed without a press is still no. A request nobody answers inside sixty seconds is refused too.

The gap

Issue #128 says this item replaces the Qt screens with React. Measured against that sentence:

```
screens the running window builds     52
screens that draw React                1
screens that draw in the shell only    2
tabs drawn by React                    0
```

The renderer manifest names 70 modules. The application the operator runs hosts no manifest, and loads exactly one of those modules: the History table's, which one Qt widget puts on a page of its own. The shell that does host the manifest draws two panels and offers no way to reach a third.

One call site carries the whole difference between the React build and the Qt build. `src/gui/history_table_variant.py` holds the only call to the variant resolver in the tree, and every other tab builds the same Qt widget either way.

`src/gui/history_table_variant.py` — `history_table_class`

```
chosen = resolve_variant() if variant is None else variant
if chosen == QT:
    from .history_qt_table import HistoryQtTable

    return HistoryQtTable
from .react_history_panel import HistoryWebTable

return HistoryWebTable
```

Why the count read as near-complete

Two instruments report on this work, and both answer a narrower question than the item asks.

The progress tool marks a screen paired when a bridge method reaches a renderer module the manifest names. A manifest entry declares that the shell would load the file. It does not put a screen in front of the operator.

`tools/conversion_state.py` — `served_methods`, the pairing rule

```
def served_methods(root: pathlib.Path) -> dict[str, str]:
    """Bridge method to the renderer module that speaks it, loaded ones only.

    A method absent from `registered_surfaces` is left out of the result.
    """
    loaded = renderer_modules(root)
```

The parity tests compare a Qt view model against the React view model of the same screen. Both sides are plain Python data and neither has to be displayed. A passing test proves the two descriptions agree with each other. It does not prove either one is on screen.

`parity_pins` reads only a parity test that imports exactly one surface module and exactly one Qt module. It collects the names on both sides and pairs them.

`tools/conversion_state.py` — `parity_pins`, what a pin reads

```
def parity_pins(root: pathlib.Path) -> dict[str, set[str]]:
    qt_by_name = {dotted_name(root, path) for path in qt_modules(root)}
    surfaces = surface_methods(root)
    pins: dict[str, set[str]] = {}
```

Both readings were true. Neither measured the item, and the RENDERS column is the one that does.

Not reached from a live tab

These Qt modules are not built by the live window or by any of its tab builders. They are outside this conversion. One row is a package marker rather than a screen, and it stays where it is.

Qt module	Why it is out of scope
`alerts_tab.py`	Not reachable from a live tab
`analytics_tab.py`	Not reachable from a live tab
`audio_suite.py`	Not reachable from a live tab
`competition_tab.py`	Not reachable from a live tab
`init_wizard.py`	Not reachable from a live tab
`journal_tab.py`	Not reachable from a live tab
`launcher.py`	Not reachable from a live tab
`live_bot_window.py`	Not reachable from a live tab
`stock_main_window.py`	Not reachable from a live tab
`testnet_tab.py`	Not reachable from a live tab

Qt module	Why it is out of scope
`usb_auth_widget.py`	Not reachable from a live tab
`widgets/__init__.py`	Package marker, no screen
`widgets/api_tester_tab.py`	Not reachable from a live tab
`widgets/capital_registry_panel.py`	Not reachable from a live tab
`widgets/notification_spool.py`	Not reachable from a live tab
`widgets/pulse_manager.py`	Not reachable from a live tab

One row in that table names a file the tree no longer holds. The Capital Registry panel is removed with the registry that fed it, so no conversion is owed for it.

The rows in that table are inside this item. The operator's 6 September 2026 directive puts the whole interface in scope, so a screen the window does not build is converted like any other. A converted row keeps `shelved` in the Scope cell of the table above, because that cell answers reachability. Its RENDERS cell reads `yes` once `variant_surface` holds the row and the React class draws the screen.

The reachability walk is rooted at the window and its tab builders, and an edge is a use rather than an import: a call, a base class, or a returned class. An import alone is not an edge, so the re-export block in the main window leaves a widget unreached. Ten screens known to be built and eleven known to be dead were run through the walk as a control, and all twenty-one came back on the expected side.

Detail on each live screen sits one file down, in 08-tabs/README.md.

2026-09-09 11:34 - #128 - the four modal dialogs are opened, not mounted

Four rows read no under Registers in Electron: the bot wizard, the buy confirmation, the Settings dialog and the Start All progress dialog. None of the four is a panel the shell should hold, so all four now take the dash the table already uses for a module that does not register in its own right.

The running shell was asked about each name before anything was changed. All four modules load and none registers a panel.

```
manifest names        69
panels registered     19
bot_wizard            bot_wizard.js registered no panel to draw
buy_confirmation      buy_confirmation.js registered no panel to draw
settings_dialog       settings_dialog.js registered no panel to draw
start_all_progress    start_all_progress.js registered no panel to draw
```

Each was then registered by hand and opened into a host of its own, which is the test that decides the cell. All four draw, so none is an empty shell; what decides them is that nothing in the shell opens them and each one's look belongs to the page its own host builds.

```
name                  elements  characters  where it belongs
bot_wizard                  27         906  a space exchange_tab.js keeps
buy_confirmation            29         254  a window the broker raises
settings_dialog            431        2465  a window the menu raises
start_all_progress          10         390  a window Start All raises
```

The wizard is the clearest of the four. The exchange screen keeps a space for it and fills that space once + New Bot is pressed, so its content reaches the shell through the tab above it.

```
    space.setAttribute(CHILD_ATTR, moduleName);
    return api.fill(space, null, { onClosed: closeBotWizard });
```

The Settings dialog drew all eleven tab pages at once. A page not on show was marked hidden and still carried an inline display, which beats the browser's own rule for a hidden element. Read off the drawn page, 320 parts sat outside the window; with the page not on show taking `none`, none do.

```
      display: props.current === true ? FLEX : DISPLAY_NONE,
```

Every drop-down on that dialog drew one letter per choice. The dialog turns each choice into plain data before it reaches the page, and listing a word gives its letters, so ten carriers drew as ten letters. A choice that is one word now stays that word.

```
before   sms_carrier   A  T  V  S  U  C  B  M  G  O
after    sms_carrier   AT&T  T-Mobile  Verizon  Sprint  US Cellular  Cricket
```

Thirty-eight tick boxes and eight number boxes drew with nothing beside them. The words were already in the payload under `text`, and the units under `suffix`, and neither was drawn. Both are drawn now, so the Sound page reads the same on both builds.

```
part            before   after
row-label           31      31
check-text           0      38
value-prefix         0       2
value-suffix         0       6
```

The Start All progress page did not fit the window it is drawn in. It is sized from the dialog's own width and height and then given padding, and the page carried a margin of its own, so the box measured 548 by 388 inside a window of 520 by 360 and both buttons fell below the edge. A stylesheet of two rules puts the padding and the margin inside that size.

```
before   window 520x360   page 548x388   parts outside the window 10
after    window 520x360   page 520x360   parts outside the window  0
```

The wizard's own walk row is not finished. It draws the four step names as words, where the Qt wizard draws Back, Next, Commit, Finish and Cancel and turns Back and Finish off on the first page. The payload names the four steps and carries no words and no on-or-off state for them.

```
WALK_STEPS = (WALK_NEXT, WALK_BACK, WALK_CANCEL, WALK_FINISH)
```

The RENDERS column asks what the operator sees when he opens the screen, so each of the four was asked of the running build rather than of the table. The variant seam holds sixteen screens and answers a class per build.

```
screen                      qt build                  react build
Settings dialog             SettingsDialog            SettingsDialogReact
Start All progress dialog   StartAllProgressDialog    StartAllProgressReactDialog
Buy confirmation dialog     BuyConfirmationDialog     BuyConfirmationReactDialog
bot wizard                  no entry: both builds open BotCreationWizard
```

Three of the four keep `yes`, and each was drawn under both builds and pictured. The wizard takes the dash. It has no entry on the seam, so + New Bot opens the Qt wizard whichever build is running, and its React module draws only inside the exchange screen the shell builds when its backend names an exchange.

The buy confirmation keeps `yes` for the class the broker raises, which is the React one. Nothing calls the broker's request method, so the operator reaches neither build of that screen; the section above records that count and it has not moved.

The wizard's walk row is a gap in what the payload carries, and the Python surface owns it. Qt's wizard supplies its own five buttons and works out which are live from the page it is on. The React module is handed four step names and nothing else, so it has no words to draw and no state to grey.

```
WALK_STEPS = (WALK_NEXT, WALK_BACK, WALK_CANCEL, WALK_FINISH)
```

Two things are missing from `bot_wizard_surface`: the words each step draws, and whether each step is live on the page now showing. Back is dead on the first page and Finish is dead until the last, and both follow from the step order the surface already holds.

```
PROPOSED, in bot_wizard_surface.py

WALK_WORDS = {"back": "< Back", "next": "Next >", "commit": "Commit",
              "finish": "Finish", "cancel": "Cancel"}

walk_live(page) -> {"back": page is not the first, "next": a page follows,
                    "finish": no page follows, "cancel": True}
```

2026-09-09 12:45 - #128 - the alerts tab is opened, the news strip is drawn inside the exchange screen

Two rows read `no` under Registers in Electron: the Notifications and Alerts tab and the crypto news strip. Neither is a panel the shell should hold, so both now take the dash the table already uses for a module that does not register in its own right.

The running shell was asked about each name before anything was changed. Both modules load and neither registers a panel.

```
manifest names        69
panels registered     19
alerts_tab            alerts_tab.js registered no panel to draw
crypto_news_ticker    crypto_news_ticker.js registered no panel to draw
module load errors    []      faults []
```

Each was then registered by hand and opened into a host of its own, which is the test that decides the cell. Both draw, so neither is an empty shell; what decides them is that the shell opens neither, and that a panel host has no way to give either one the values it needs.

```
name                  opened  elements  characters  what it drew
alerts_tab              true        54         233  both tables empty
crypto_news_ticker      true         2          21  Fetching crypto news
strips on the page                                  2, for one exchange space
```

The alerts tab is raised by a window rather than by a tab. The variant seam holds it under the name ALERTS, so the Qt build opens `AlertsTab` and the React build opens `AlertsReactTab`, which is one browser view filling the whole tab. Opened as a panel instead, it draws every control and no row, because the host asks with an empty request and no notification manager reaches it.

The news strip belongs to the exchange screen. That screen keeps a space for it and fills that space, once per exchange, the way it fills the two bot tables. A registration holds one host per name, so registering the strip put a second copy of it on the page.

```
    space.setAttribute(CHILD_ATTR, moduleName);
    return api.mount(target);
```

The two RENDERS cells part company. The variant seam answers a class per build for the alerts tab, so that cell keeps `yes`. It holds no entry for the news strip, so the exchange header builds the Qt strip whichever build is running, and the React module draws in the Electron shell and nowhere else. That cell now reads `shell`, the value this table already defines for exactly that.

```
screen                      qt build            react build
Notifications and Alerts    AlertsTab           AlertsReactTab
crypto news strip           no entry: both builds build CryptoNewsTicker
```

The strip was never fed. The exchange screen asked the backend for the strip's state with an empty request, so the strip was built and never started, and the line read "Fetching crypto news" for as long as the shell ran. Qt starts the strip as it builds it and the screen already reports that it did, so the request now carries the start it reports.

```
  function tickerRequest(model) {
    var started = model[NEWS_TICKER_STARTED];
    return typeof started === "number" && started > ZERO ? { start: true } : {};
  }
```

The backend had no fetch to run either. The strip's model takes its fetch as an argument and the bridge never gave it one, so the worker answered with no stories. The bridge now hands in the same transport the Qt strip uses, and runs the fetch away from the request that asked for it, so the answer arrives the way it arrives in Qt.

```
    crypto_news_ticker_surface.use_fetch(SafeRequest, safe_urlopen, time.monotonic)
```

With that in place the shell fetched 43 stories from the ten feeds and drew them, one at a time, in the exchange header. Refusing every feed drops them again, which is what proves the line carries what the fetch found.

```
live      [2/43] The Defiant - SEC Crypto Custody Rewrite Enters White House Review
broken    (no crypto news feeds reachable)
no start  Fetching crypto news
```

Both builds of each screen were then driven on one reading and compared item by item. The alerts tab was read on one notification manager filled the way the main window fills it: the risk rules over a three-bot fleet, the profit and loss milestone, one bot error and one bot start. The news strip was read on one answer from the shipped fetch, given to each side through its own receiver.

```
alerts tab       13 routing rows, 4 history rows      differences 0
news strip       6 stories, 8 steps, wraps 6 to 1     differences 0
```

The routing table now draws **14** rows. `AlertEvent.TRADE_EXECUTED` in `src/core/notifications.py` became `BUY_FILLED` and `SELL_FILLED`, one event for each of the two fill boxes on `Settings > SMS`. The table reads the rows off `NotificationManager.get_config`, so it draws whatever that answers and no count is written on the page. Eleven of the fourteen rows name the SMS channel, which `notifications.message_channels_for` gives every event `notifications.EVENT_MESSAGE_KIND` pairs with a switch.

Every alert row matches: its time, its priority word, its title, its message, its channels, and the colour of each. The four priorities draw in four colours, low grey through critical red, and the rows run newest first on both sides. Emptied of notifications, both sides draw no rows and both read `Unread: 0`.

The strip shows the same story at each place, moves one place forward on each step, and wraps from the last story to the first on both sides. Its words are `#cfe6ff` on both and its pointer is the hand on both.

Two colours were wrong on the page and are now right. The tab's group boxes carry a Qt style sheet whose colour paints the group's title alone; the page was painting the whole box with it, so the routing table's Event and Priority columns and the three field labels drew in the accent where Qt draws them in the body colour. The page also carried no stylesheet of its own, so with the group colour moved to the title it fell back to the browser's black.

```
                        before        after         qt
routing Event column    #00ffcc       #e0e0f0       #e0e0f0
field labels            #00ffcc       #e0e0f0       #e0e0f0
```

The two strips take the fetch at different moments. The Qt strip shows a story as soon as the fetch answers, because the worker reports to it. The page has no such report, so it shows the answer on its next 15 second step. Closing that needs a route from the backend to a module the shell holds by name, and the strip is drawn by its parent rather than held by name.

The Qt tables are drawn by the theme, not by the tab, so the page draws no grid lines, no alternating row bands and no header underline. Those live in the Qt style sheet for every table in the application and reach no payload.

2026-09-09 13:08 - #128 - the RENDERS column reads what the running build draws

The running program answers this column, not the table. The table marks 42 rows in scope. The reading covered 38 of them. 26 agree with the cell they carried. 12 do not, and every one of the 12 claims more than the operator gets. No row claims less. Four rows carry no reading, and each keeps the cell it carried.

```
rows marked in scope       42
measured                   38
  agree                    26
  disagree                 12
could not measure           4    bot_wizard, header_strip,
                                 market_inspector_topologies, visualizer/themes
```

Ten cells change here. Six drop to a dash and four drop to `shell`.

Qt file	before	after	what the running build draws
`src/gui/main_tabs/trading_tab.py`	yes	-	`_build_trading_tab` gives the Live tab a page with no web view
`src/gui/widgets/exchange_tab.py`	yes	-	`ExchangeTab` is that page
`src/gui/widgets/bot_status_table.py`	yes	-	`BotStatusTable` sits inside that page
`src/gui/widgets/extractor_bot_table.py`	yes	-	`ExtractorBotTable` sits inside that page
`src/gui/live_settings/positions_held_tab.py`	yes	-	`BotLiveSettingsReactDialog` carries seven tab labels and none of them is Positions Held
`src/gui/main_tabs/console_log_handler.py`	yes	-	the window installs `_QtLogHandler` on the root logger
`src/gui/widgets/trade_charts_tab.py`	yes	shell	`TradeChartsTab` is the Charts tab, and `trade_charts_tab.js` draws in the Electron shell alone
`src/gui/native_chart.py`	yes	shell	`ChartPanel` and `CandlestickChart` draw inside that tab, and `native_chart.js` draws in the shell alone
`src/gui/indicator_panel.py`	yes	shell	`IndicatorVotingPanel` is the voting panel, and `indicator_panel.js` draws in the shell alone
`src/gui/widgets/status_log.py`	yes	shell	`StatusLog` is the log pane, and `status_log.js` draws in the shell alone

Two of the twelve keep the cell they carry. The news strip cell already reads `shell`. The Main Window cell keeps `yes`, because none of the three values fits what that window draws.

The Main Window is a React tab book inside a Qt frame, with a Qt menu bar and a Qt status bar, and `MainWindow` in `src/gui/main_window.py` builds all three.

```
MainWindow._setup_ui           builds the tab book from the variant seam
MainWindow._setup_menu         builds the Qt menu bar
MainWindow._setup_status_bar   builds the Qt status bar
```

The 42 in-scope cells now read:

```
yes      30
shell     5
-         7
```

2026-09-09 14:30 - #34 - the Status tab leaves the empty-tab pair

Status is no longer a skeleton, so it leaves `empty_tabs.py` and takes a row of its own. Accumulation stays where it was and still draws its empty state.

`src/gui/main_tabs/system_status_tab.py` — what the window builds for the tab

```
    def _build_system_status_tab(self) -> None:
        """Append the Status tab, and hold None when its page cannot be built."""
        try:
            from ..react_system_status_tab import SystemStatusReactPanel

            self._system_status_tab = SystemStatusReactPanel()
            self._main_tabs.addTab(self._system_status_tab, HEADING)
        except Exception as exc:  # noqa: BLE001 - a missing tab is not a crash
            logger.warning("Status tab unavailable: %s", exc)
            self._system_status_tab = None
```

Three cells move. The `empty_tabs.py` row drops `system_status_tab.js` and now names one module. A new row carries the screen itself, and a second names the Qt-side file the conversion added.

Qt file	React module	RENDERS	Scope
`src/gui/main_tabs/empty_tabs.py`	`proof_of_accumulation_tab.js`	yes	in scope
`src/gui/main_tabs/system_status_tab.py`	`system_status_tab.js`	shell	in scope
`src/gui/react_system_status_tab.py`	no	-	React side

The new cell reads `shell`, and the reason is the seam. `variant_surface.py` records a Qt loader and a React loader for every screen that has both, and this screen has no entry there, because no Qt widget ever drew it. `yes` states that React draws where Qt used to, and nothing here used to. The module was read drawing under its own tab in the Electron shell, so `shell` is what was measured.

```
registered           system_status_tab is in acervatorPanelHost.registered()
tab bar              system_status_tab, ninth of ten
select               returns system_status_tab
drawn                17 subsystem panels, 11 tab groups, 78 emitter rows
```

The detail is in 08-tabs/system-status.md.

2026-09-09 15:12 - #435 - Start All waits 3.5 seconds and retries a bot once

The Start All dialog used to promise a pause of about 2.5 seconds with a two-second minimum. The engine waited 0.6. The pause is now 3.5 seconds, it is written down once, and the dialog quotes that figure rather than a number of its own.

```
#: Seconds `BotManager.start_all` waits between bots, and the figure the Start
#: All dialog quotes. 0.6 drew HTTP 429 on three of 38 starts.
START_ALL_GAP_SECONDS: float = 3.5
```

The boot of 8 September 2026 is in the operator's own log. Thirty-eight bots were brought up one at a time and three of them came back with the same refusal from Coinbase: too many requests, on the call that lists the accounts. Those three did not trade until they were started again eight minutes later. A wider pause is exactly what the dialog always said the gap was for.

```
2026-09-08 11:10:43,649 BOT_CONNECT_FAILED | RateLimitExceeded: coinbase GET
https://api.coinbase.com/api/v3/brokerage/accounts?limit=250
429 Too Many Requests | DIAGNOSIS: Rate limited.
```

A bot that does not report itself running inside ten seconds is now stopped and started one more time, and then the sequence moves on whatever happened. The stop is what makes the second attempt real, because a bot still in its starting state refuses a plain second start and the retry would do nothing.

```
            if not verified and not getattr(self, "_start_all_cancel", False):
                logger.warning(
                    "Bot %s did not reach RUNNING in %.1fs; retrying once",
                    bot.bot_id,
                    verify_timeout_seconds,
                )
                await asyncio.sleep(min_gap_seconds)
                await bot.stop()
                await bot.start()
                verified = await self._await_running(bot, verify_timeout_seconds)
```

The platform now writes a line for each of those events, so a boot that loses a bot says so in the log instead of leaving a silent hole. A bot that comes up on the retry gets one warning; a bot that fails both attempts gets two.

```
Bot LATEBOT starting on LATEBOT/USD
Bot LATEBOT did not reach RUNNING in 10.0s; retrying once
Bot LATEBOT starting on LATEBOT/USD
Bot DEADBOT did not reach RUNNING in 10.0s; retrying once
Bot DEADBOT is not RUNNING after 10.0s; start_all moved on
```

The cost is real and it is worth knowing before the next launch. Thirty-eight bots mean thirty-seven waits. The 8 September pass took 3 minutes 33 seconds for its 35 bots at a mean of 5.3 seconds each, so the same fleet at the new pause reaches about 5 minutes, and three retries add about 40 seconds on top.

```
gaps alone, 37 waits       0.6 s -> 22.2 s        3.5 s -> 129.5 s
measured pass, per bot     mean 5.318 s           mean 8.218 s
38 bots, three retried     3 min 17 s             about 5 min 45 s
```

2026-09-09 21:37 - #147 - the Accumulation tab leaves the empty-tab pair

Accumulation is no longer a skeleton, so it leaves `empty_tabs.py` and takes a row of its own. It draws three zones: the player window on the left, the square enemy screen upper right, and the party window across the lower half. Every zone draws a placeholder and no art, class or mode is built.

`src/gui/main_tabs/proof_of_accumulation_tab.py` — what the window builds

```
    def _build_proof_of_accumulation_tab(self) -> None:
        """Append the Accumulation tab, and hold None when its page cannot be built."""
        try:
            from ..react_proof_of_accumulation_tab import ProofOfAccumulationReactPanel

            self._proof_of_accumulation_tab = ProofOfAccumulationReactPanel()
            self._main_tabs.addTab(self._proof_of_accumulation_tab, HEADING)
        except Exception as exc:  # noqa: BLE001 - a missing tab is not a crash
            logger.warning("Accumulation tab unavailable: %s", exc)
            self._proof_of_accumulation_tab = None
```

Three cells move and two rows are new. The empty-tab row names no module now, so nothing the window builds reaches it.

Qt file	React module	RENDERS	Scope
`src/gui/main_tabs/empty_tabs.py`	no	-	shelved
`src/gui/main_tabs/proof_of_accumulation_tab.py`	`proof_of_accumulation_tab.js`	shell	in scope
`src/gui/react_proof_of_accumulation_tab.py`	no	-	React side

The new cell reads `shell`, and the reason is the seam. Sixteen screens are registered there and none of them is this one, because no Qt widget ever drew a proof-of-accumulation screen. The two shelved Qt classes are set aside by the operator's directive of 21 April 2026 and this tab is built on neither.

```
seam screens             16
a PoA, competition or testnet entry among them   none
registered               proof_of_accumulation_tab, with no fault
tab bar                  Accumulation, seventh of ten
zones drawn              player window 2, enemy screen 2, party window 56
party window             8 groups of 5, 40 slots on page 1 of 3
wallet                   0 panels closed, 1 panel and 3 sections open
```

Removing the registration is the control. The panel name leaves the register, the Accumulation tab leaves the bar, and every zone count falls to nought while the same counter still reads 154 elements elsewhere on the page.

```
registered               proof_of_accumulation_tab absent, 18 names
reasonFor                proof_of_accumulation_tab.js registered no panel to draw
tab bar                  nine tabs, Accumulation gone
zones drawn              0, 0, 0
whole page               154 elements, so the noughts are the zones
```

The chain the shell draws is a field of the payload, so a demo run on the TestNet asks the same bridge method with a different chain and reaches the same page. Nothing else in the request changes.

```
  def chain_of(params: dict) -> str:
      """The chain ``params`` names, or ``LIVE_CHAIN`` when it names none in ``CHAINS``."""
      asked = params.get(CHAIN_FIELD) if isinstance(params, dict) else None
      return asked if asked in CHAINS else LIVE_CHAIN
```

One tooling defect surfaced and is fixed here. The page carries three tables under the same header, and the Scope writer took the first, which is a two-row excerpt inside an earlier entry. It wrote a totals block into that excerpt. The span now picks the widest table.

```
      spans = []
      for start, line in enumerate(lines):
          if line.startswith(TABLE_HEAD):
              end = start + 2
              while end < len(lines) and lines[end].startswith("|"):
                  end += 1
              spans.append((start, end - 1))
      if not spans:
          raise LookupError("no line starts with " + TABLE_HEAD)
      return max(spans, key=lambda span: span[1] - span[0])
```

Pointed at the real table the writer disagrees with the committed Scope column on twenty rows and moves five totals. That disagreement is not this entry's to resolve, so the column was not rewritten and the three cells above were set by the rule the column states.

2026-09-12 - #665 - the Phantom Bots page takes one timeframe and keeps it

The page holds three settings. All three now reach the bot they are drawn for, and two of them survive a restart.

One phantom, not a set. The eleven timeframe boxes are one picker. It offers all eleven names, greys the ones the bot's venue does not serve, and opens on the timeframe the bot holds. A bot holding several, because it was restored before this, opens on the highest of them.

setting	label on the page	what it reaches
enable	Enable Phantom Bots	the tick gate that builds the phantom set
timeframe	Timeframe:	the timeframe list the phantom set is built from
lock	Candles to lock:	the coordinator's candle count

The constraint is the one the Comp field already stated. A timeframe at or below the bot's own TA Timeframe is refused, and the reason is written beside the picker. A timeframe the venue does not serve carries a second message. Each pick below was made on a page freshly opened on 1d, on a Coinbase bot whose TA Timeframe is 1h:

```
picked  kept   told to the window       reason written beside the picker
1d      1d     phantom_timeframes 1d    -
6h      6h     phantom_timeframes 6h    -
1h      1d     nothing                  1h is not above this bot's TA Timeframe 1h
15m     1d     nothing                  15m is not above this bot's TA Timeframe 1h
4h      1d     nothing                  4h is not offered by coinbase
```

The refusal is not new work for the engine. A phantom at or below the parent's own timeframe is already dropped twice over: the higher-timeframe bias keeps only a higher rank, and the Comp calculation skips a rank at or below the parent's. On the bot above, four of the five phantoms a restart used to build reached neither.

Two of the three now survive a restart. The saved record carries the one timeframe and the candle count beside the enable flag it already carried. Driven through the fleet-load path on a record written for the run:

```
record carries                 restored bot holds                    lock
phantom_timeframe 1d           1d                                       2
lock_candle_count 7            5m 15m 30m 1h 1d                         7
neither key                    5m 15m 30m 1h 1d                         2
```

A bot nobody has chosen for is left exactly as it was. The record takes a timeframe only when the bot holds one. A bot still on the engine's own six, which a Coinbase filter cuts to five, writes no timeframe key and restores to the same five. The narrowing to one happens when the operator picks one, and not before.

```
bot                saved keys                                  restored holds  lock
untouched          phantoms_enabled lock_candle_count                 5 names       2
one timeframe set  phantoms_enabled lock_candle_count                 1 name        7
                   phantom_timeframe
```

The window now publishes what the page reads. The answer a tab is asked for carried four values, and the page read five others off it that were never there, so every phantom reading on the React build drew at its own default. The answer carries them now.

```
published before   bot_id symbol mode state
published now      bot_id symbol mode state exchange_id ta_timeframe
                   enabled timeframes lock_candle_count
```

The candle count the page sets still sizes nothing. `TimeframeCoordinator` defines the one function that would read it and nothing calls that function, so the figure persists and waits.

2026-09-13 - #23 - the venue page draws in the second build

The venue page now has a host. `src/gui/react_exchange_tab.py` builds one web view for each venue and draws `exchange_tab.js` inside it.

The host answers the four calls the main window already makes on the Qt venue tab. Each call drives a surface model and pushes the answer into the page.

```
call the window makes             what the host runs
exchange_id                       the attribute the constructor sets
update_bots(statuses)             the screen model, then both bot tables
_refresh_privacy_mode_btn_style   the screen model, then the header dots
stop_feeds                        the page script that takes the bridge away
```

The page carries nine modules and one style sheet. It fetches nothing, because every script and every rule is written into the document.

```
design_tokens.js         theme_engine.js          shared_widgets.js
header_strip.js          table_cells.js           bot_status_table.js
extractor_bot_table.js   crypto_news_ticker.js    exchange_tab.js
one style sheet          exchange_tab.css         26 rules
```

The screen draws a header row, a data-pool line, two bot sections and a command bar of five buttons. `bot_status_table.js` draws the Scrumming rows and `extractor_bot_table.js` draws the Extractor rows, each into a space the screen keeps for it.

The page names what it loaded. A venue holding seven bots reads like this.

```
page modules       all nine, exchange_tab.js last
page style sheets  exchange_tab.css, 26 rules
scrum-table rows   5
extractor rows     2
```

The style sheet paints what the payload leaves unpainted. The Privacy Mode button, the data-pool line and the two section labels carry their own colours from the backend, so no rule touches them. The five command buttons and the New Bot button carry none, so the sheet gives each one a ground, an edge and a text colour.

An emptied style sheet reads as zero rules, and a withheld module drops out of the module list and draws no row. Both readings fall when the thing they measure is taken away.

```
run              module list       style sheet rules   scrum rows
whole page       nine modules                     26            5
style emptied    nine modules                      0            5
table withheld   eight modules                    26            0
```

The page reports every control press back to Python. `ExchangePage` reads the console line the page writes and hands it to `run_action`, which drives the surface that owns the method and then pushes the new payload.

```
pressed           reached Python             the screen recorded
Start on row 1    bot-a14-02, start          command.sent start scrumming
nothing pressed   no call                    no entry
```

The host starts no bot and places no order while it builds or draws. It reaches three names under the trading package, and none of them is a bot, a container or an exchange client.

```
src.trading                    the package
src.trading.gate_vocabulary    the gate names
src.trading.target_bands       the Target A15 and Target A14 arithmetic
```

The news strip is fed. `bind_news_transport` gives the strip the same reader the desktop bridge gives it, and the first ask carries the start the Qt strip performs as it builds.

2026-09-13 - #23 - the bot wizard gets a host and a stylesheet

`react_bot_wizard.py` builds the wizard page. `bot_wizard.css` paints it. The host answers the calls the window already makes on the Qt wizard, so one set of calls drives either build.

the window calls	the host answers with
the class, with the venue list and the stored defaults	`BotWizardReactDialog.__init__`
exec	`QDialog.exec`
DialogCode.Accepted	`QDialog.DialogCode`
get_bot_config	`BotWizardReactDialog.get_bot_config`

The page carries every asset it needs. `dialog_html` inlines React, the four style sources, the table cells module, the wizard module and the sheet. The page fetches nothing over the network. `MODULE_GLOBALS` names the global each asset defines once its script tag has run.

The page says what it loaded. `LOADED_MODULES_JS` reads each of those globals back. `LOADED_STYLES_JS` counts the sheets the browser parsed and the rules whose selector names the wizard class. One run of the built host read this:

```
modules   react  react-dom  design_tokens  theme_engine  shared_widgets
          header_strip  table_cells  bot_wizard        all true
styles    sheets 1    rules 56    wizard rules 50
rail      asset  mode  params  phantom  extractor_pool   params current
rows      47 field rows drawn
```

The same run with the sheet emptied on disk read `wizard rules 0`, and the browser drew all 47 rows in its own plain type. The same run with the module emptied on disk read `bot_wizard.js false`, no rail stops and no rows.

One model draws every page. `BotWizardReactDialog.model` asks `bot_wizard_surface.view_model` for the whole wizard as one payload. The venue list, the stored defaults, the market list and the offered timeframes go in around the steps the page sends.

A press on the page comes back to Python. The page writes one console line. `BotWizardPage` hands that line to `answer_call`, which builds a fresh payload and hands it back. A finished walk accepts the dialog, and a cancelled walk rejects it.

The sheet paints the parts the module names. Every rule sits under the `acervator-bot-wizard` class and selects one `data-part` value.

part	what the sheet paints
wizard-title	the window name, in the accent colour, over a rule

part	what the sheet paints
page-rail-stop	one pill per page, and the current page takes the accent
page-title, page-subtitle	the page heading and its line of help
field-group	a bordered block with its title in the accent
field-row-label, field	a right-aligned label against a growing control
target-combo, info-button	the pair box, its circle and the info button, on one line
timeframe-row	one chip per timeframe, and a chip the venue refuses dims
alt-list, alt-row	the pair list the pool page fills
warning-box	the call-budget message the phantom page raises
walk-step	the Next, Back, Cancel and Finish buttons

The host reaches no venue and creates no bot. The venue list and the market list are arguments, so the dialog opens no connection of its own. A run that built the host and drew all five pages at three widths imported no module under `src/trading` and wrote no file under the runtime tree.

The table row above still reads a dash. `variant_surface.py` records no loader pair for this screen, so the running window builds the Qt wizard. The row changes when that pair is recorded.

2026-09-13 - #23 - the window chooses the three new hosts

`variant_surface.py` records three more loader pairs. `register` holds one Qt loader and one React loader for each screen. The running build selects one side of each pair.

screen name	the Qt loader returns	the React loader returns
Charts	`TradeChartsTab`, in `widgets/trade_charts_tab.py`	`ChartsTabReact`, in `react_charts_tab.py`
Exchange page	`ExchangeTab`, in `widgets/exchange_tab.py`	`ExchangeTabReact`, in `react_exchange_tab.py`
Bot creation wizard	`BotCreationWizard`, in `bot_wizard.py`	`BotWizardReactDialog`, in `react_bot_wizard.py`

Three call sites ask `surface_class` for the class. `ChartsTabMixin` builds the Charts tab in `_build_charts_tab`. `MainWindow` builds one venue page in `add_exchange_tab` and opens the wizard in `_create_bot`. Each call site keeps the Qt class as its fallback, so a screen with no recorded pair still builds the original.

The window names what it built. One window ran under each build, and each class came off the live object.

```
screen          qt build            react build
Charts tab      TradeChartsTab      ChartsTabReact
venue page      ExchangeTab         ExchangeTabReact
wizard          BotCreationWizard   BotWizardReactDialog
```

Each React page names what it loaded, once the window reaches it.

```
Charts tab   header_strip, native_chart, trade_charts_tab, no faults
venue page   nine modules, exchange_tab.js last, exchange_tab.css, 26 rules
wizard       eight module globals, all true, one sheet, 56 rules
```

The reading falls when one pair goes away. A run with the Charts pair deleted built the Qt tab and logged the refusal. The venue page and the wizard still answered with their hosts.

```
recorded pairs        Charts tab       venue page         wizard
all three             ChartsTabReact   ExchangeTabReact   BotWizardReactDialog
Charts pair deleted   TradeChartsTab   ExchangeTabReact   BotWizardReactDialog
```

The venue page keeps a dash in the table above. `TradingTabMixin` holds the Qt layer page as a hidden child in `_react_trading_page`, and that page holds the venue tabs. The React venue page draws its nine modules as soon as a run shows that layer page, so the host is built and reachable, and the Live tab hides it.

2026-09-13 - #23 - the two strips get a host and a stylesheet

`react_status_log.py` builds the Activity Log page. `status_log.css` paints it. `react_crypto_news_ticker.py` builds the news strip page. `crypto_news_ticker.css` paints it. Each host answers the calls the window already makes on the Qt widget, so one set of calls drives either build.

the window calls on `StatusLog`	`StatusLogReact` **answers with**
log	`StatusLogReact.log`
force_log	`StatusLogReact.force_log`
pause, resume	`StatusLogReact.pause`, `StatusLogReact.resume`
is_paused, toggle_pause	`StatusLogReact.is_paused`, `StatusLogReact.toggle_pause`
health_stats	`StatusLogReact.health_stats`
setMaximumHeight	`QWidget.setMaximumHeight`

the header calls on `CryptoNewsTicker`	`CryptoNewsTickerReact` **answers with**
the class	`CryptoNewsTickerReact.__init__`
start	`CryptoNewsTickerReact.start`
stop	`CryptoNewsTickerReact.stop`
force_refresh	`CryptoNewsTickerReact.force_refresh`
current_headlines	`CryptoNewsTickerReact.current_headlines`

Each page carries every asset it needs. `panel_html` inlines React, the four style sources, the strip's own module and its sheet. Neither page fetches over the network. `MODULE_GLOBALS` names the global each asset defines once its script tag has run.

Each page says what it loaded. `LOADED_MODULES_JS` reads those globals back. `LOADED_STYLES_JS` names each sheet and counts the rules the browser parsed from it. One run of the built hosts read this:

```
status log   modules  design_tokens theme_engine shared_widgets header_strip status_log
             styles   status_log.css 12 rules             lines drawn 8
news strip   modules  design_tokens theme_engine shared_widgets header_strip crypto_news_ticker
             styles   crypto_news_ticker.css 8 rules      headlines drawn 1
```

The readings fall when an asset goes. The same run with each sheet emptied read no rules, and the browser drew both strips on white. The same run with each module emptied dropped that module from the roster and drew no line and no headline.

```
run         status log            news strip
live        12 rules, 8 lines     8 rules, 1 headline
no sheet     0 rules, 8 lines     0 rules, 1 headline
no module   12 rules, 0 lines     8 rules, 0 headlines
```

The pane paints a line by its kind. `status_log_surface.line_style` reads the message prefix first and the level second. A pixel census of each saved picture read these counts.

```
live        0a0a0f 238504   0e0e1a 193353   7a7a9c 2594   00ff88 478   ffaa00 458
no sheet    ffffff 430648   ffaa00 1759     00ff88 1365   ff66dd 531   ffcc44 461
no module   0a0a0f 450000
```

The sheet carries the page ground, the pane ground and the border. The payload carries every line colour, so a line keeps its colour with no sheet at all.

One payload draws the whole pane. `pane_payload` asks `status_log_surface.build_view_model` for the pane state, then puts every line the model holds under `document`. `build_view_model` drains `take_painted`, so its own `document` carries one call's batch. `document_blocks` counts the whole document, and the two agree once `pane_payload` has run.

The strip shows the story it holds. `CryptoNewsTickerReact._run_fetch` runs the strip's worker only when the strip carries a `fetcher`. `CryptoNewsTickerModel.on_headlines` writes `NO_FEEDS_TEXT` for an empty answer, so a strip with no reader would report no feeds over the stories it holds.

The strip reached no feed. A run built the host with four recorded stories and no `fetcher`. The strip drew the first story with its position and its feed name. The run refused every connect away from loopback and counted none.

```
fetcher              None
fetches started      1
headlines held       4
connects attempted   0
label                [1/4] The Defiant - SEC crypto custody rewrite enters White House review
```

Each sheet paints the parts its module names.

part	what the sheet paints
log	the read-only pane, its ground, its border and its scrollbar
line	one block, wrapped rather than widened

part	what the sheet paints
stamp	the leading time, in tabular figures
body	the message span, which takes its colour from the payload
placeholder	the line an empty pane shows
ticker	the frame around the strip
headline	one story on one line, clipped at the strip's edge

Both table rows above still read `shell`**.** `variant_surface.py` records no loader pair for either strip, so the running window builds the Qt pane and the Qt strip. Each row changes when its pair is recorded.

2026-09-13 - #23 - four modules already had a host, and the tables get their skin

Four rows read `-` or `shell`. All four already draw in the window the operator launches. No new host was written. The measurement is below, and four cells move.

The venue page host mounts both bot tables inside itself. The Charts host mounts the chart module inside itself. The Status panel is its own host.

```
react_exchange_tab.CHILD_MODULES   table_cells, bot_status_table,
                                   extractor_bot_table, crypto_news_ticker
react_charts_tab.CHILD_MODULES     native_chart
react_system_status_tab            PANEL_MODULE system_status_tab.js
```

The window reaches all three. `main_window.py` asks the seam for the venue page class, and `charts_tab.py` asks it for the Charts class. `SystemStatusTabMixin` builds the Status panel with no seam, because no Qt widget ever drew that tab.

```
main_window.py, in add_exchange_tab            page_class = surface_class(EXCHANGE)
main_tabs/charts_tab.py, in _build_charts_tab  built = surface_class(CHARTS)()
main_tabs/system_status_tab.py                 SystemStatusReactPanel()
```

Each page names itself off the live object. A run built each screen, drove content into it and read the class and the accessible name back.

```
ExchangeTabReact           React Exchange Tab   widget and web view
ChartsTabReact             React Charts Tab     widget and web view
SystemStatusReactPanel     Status               widget and web view
```

Each page reports what it loaded. The venue page already answered this. The Charts page and the Status page now answer it too, because `page_html` gives every style sheet a `data-asset` name of its own.

```
venue page   9 modules, exchange_tab.css 31 rules
Charts tab   3 modules, trade_charts_tab.css 20 rules, 0 panel faults
Status tab   1 module, system_status_tab.css 25 rules
```

Each page drew the content driven into it. Six bots went into the venue page, forty candles into the chart, and the emitter read-out into Status.

```
venue page   4 Scrumming rows, 2 Extractor rows, 56 cells, 6 coin badges
Charts tab   40 candles, 40 wicks, 40 volume bars, 3 indicator panels
Status tab   17 subsystems, 11 tab groups, 78 emitter rows
```

The readings can fail. Each page was built again with its style sheet emptied and again with its module absent. Every reading reported the loss.

```
sheet emptied    rule count 0, page paints white
venue module out modules drop both tables, rows 0 and 0, no cells
chart module out modules drop native_chart, fault "registered no panel to draw"
status module out modules empty, emitter rows 0
```

Both variants drew the same content, and five colours did not match. Each screen was built under the qt variant and the React variant with the same bots and the same candles. Every table cell agreed, header for header and row for row. Five colours read off the saved pictures did not.

element	qt variant	React before	React now
column header text	`#00ffcc`	`#e0e0f0`	`#00ffcc`
header underline	2px accent	1px grey	2px accent
Detail button face	`#1a1a28`	`#efefef`	`#1a1a28`
Fire button face	`#1a1a28`	`#efefef`	`#1a1a28`
coin badge	a coloured disc	nothing drawn	a coloured disc

The badge is the disc `_get_coin_icon` paints when no logo file is cached. Its colour comes from the characters of the name. One function now answers for both sides, so the two cannot drift.

```
def coin_disc_color(symbol: str) -> str:
    hue = sum(ord(char) for char in str(symbol)) % HUE_WHEEL
    red, green, blue = colorsys.hsv_to_rgb(
        hue / HUE_WHEEL, DISC_SATURATION / ALPHA_HIGHEST, DISC_VALUE / ALPHA_HIGHEST
    )
```

Four discs were read off the qt picture and the same four off the React picture. They agree to the digit.

```
A15-USD   #5fb479      A23-USD   #5fb496
A14-USD   #5fb484      A21      #b45fb1
```

Two differences stay, and each has a reason. The header ground reads `#16162a` under Qt and `#1a1a28` under React. No design token holds the first value, and the sheet may name no colour the design system does not hold. The column rule reads a lighter grey under Qt for the same reason.

The Charts tab chrome differs above the chart, and that belongs to another row. The qt arrows draw no glyph, the qt timeframe box carries a `TF:` label, and the qt indicator toggles paint filled squares where React draws check boxes. Every one of those sits in `trade_charts_tab.js`, whose row already reads `yes`.

Status draws the same under both variants. It has no Qt original, so the window builds the React panel either way. The two pictures hold the same colours in the same counts at all three widths.

```
status qt      #141420 60397   #0a0a0f 28812   #ffffff 4080
status react   #141420 60397   #0a0a0f 28812   #ffffff 4080
```

Four cells move to `yes`. The Status cell moves because the column reports what the operator sees, and he sees the React panel in the window.

Qt file	React module	before	after
`src/gui/widgets/bot_status_table.py`	`bot_status_table.js`	-	yes
`src/gui/widgets/extractor_bot_table.py`	`extractor_bot_table.js`	-	yes
`src/gui/native_chart.py`	`native_chart.js`	shell	yes
`src/gui/main_tabs/system_status_tab.py`	`system_status_tab.js`	shell	yes

2026-09-13 - #23 - the tab bar paints from the theme again

The ten grounds are gone. The bar paints no ground of its own. Every tab now takes the colours the running theme's own style sheet sets, which is what the bar did before the grounds were added.

`src/gui/variant_surface.py` — `_qt_main_tab_book`

```
def _qt_main_tab_book() -> type:
    """Import and return the Qt main tab book."""
    from PySide6.QtWidgets import QTabWidget

    return QTabWidget
```

Six theme tokens go with them. Each of the five themes carried a ground colour and a text colour for black, white and gold. None of the thirty values is declared now, and no style sheet names one.

CITED AS ABSENT. The file `main_tab_bar` is deleted, so the paint block earlier on this page records what the bar did between the seventh and the thirteenth of September 2026.

Read off the rendered bar, five themes, both builds. A bright pixel here is one reading at least 200 red, at least 150 green and at most 120 blue. The gold ground filled most of the bar before and none of it now.

```
theme             qt before  qt after   react before  react after
cyberpunk_dark    20,880     0          14,262        0
neon_light        20,771     0          14,178        0
classic_terminal  22,027     0          14,262        0
minimal_modern    20,643     0          14,091        0
glass_metal       20,604     0          14,078        0
```

2026-09-13 - #23 - Accumulation leaves the bar in one place

One tuple names every tab the window does not build. The window skips the builder for a name in it, and the tab list the frontend reads leaves that name out. The bar now reads Sim, Paper, Live, Charts, Inspector, Swarm, History, Status, Console.

`src/gui/main_tabs/main_window_surface.py` — `UNBUILT_TABS`

```
UNBUILT_TABS = (ACCUMULATION_TAB,)

BAR_TAB_ORDER = tuple(name for name in CANONICAL_TAB_ORDER if name not in UNBUILT_TABS)
```

Removing the name from that tuple puts the tab back. The Accumulation screen is unfinished, not unwanted. Its builder, its panel, its bridge method and its manual page are all unchanged.

2026-09-13 - #23 - a tab that fails to build no longer moves the tabs after it

The reorder advances its slot only over a tab the bar carries. A name the bar does not carry took a slot before, so every tab after a missing one stayed where the builders left it. The Qt build fails to build Sim and Paper, and its bar now reads the canonical order of the seven tabs it does have.

`src/gui/main_tabs/main_window_surface.py` — `reordered_tabs`

```
    order = list(labels)
    target_index = 0
    for name in desired:
        if target_index >= len(order):
            break
        for current_index in range(target_index, len(order)):
            if order[current_index] == name:
                if current_index != target_index:
                    order.insert(target_index, order.pop(current_index))
                target_index += 1
                break
```

Read off the two running bars. The React build carries all nine tabs it builds. The Qt build carries seven, in the same order.

```
react   Sim, Paper, Live, Charts, Inspector, Swarm, History, Status, Console
qt      Live, Charts, Inspector, Swarm, History, Status, Console
qt was  Live, Charts, Inspector, Console, Swarm, Accumulation, Status, History
```

The two missing tabs have one cause. The Qt Sim page and the Qt Paper page each read a panel key their own surface no longer publishes, so each builder raises and the window logs the tab as unavailable. That belongs to the Simulator rebuild and to the Paper screen, not to the bar.

2026-10-16 - #158 - the splitter edges take the theme and seven theme values go

All four handle edges were one cyan. A horizontal handle draws a left and a right edge, a vertical handle a top and a bottom. All four named the same fixed cyan at a tenth of full opacity, so both light themes drew that cyan over a pale ground. The four edges now read `border_accent`, which every palette already declared and no rule read.

`src/gui/theme_engine.py` — `_theme_qss`, the horizontal handle

```
QSplitter::handle:horizontal {{
    width: 5px;
    background-color: {t.border_primary};
    border-left:  1px solid {t.border_accent};
    border-right: 1px solid {t.border_accent};
}}
```

A pressed handle asked for an opacity Qt does not have. The rule set `opacity: 0.85` beside the accent. Qt drops a property it does not know, so a pressed handle drew the full accent a hovered one already drew. The rule now paints the accent under the alpha byte Qt does read, at the same 0.85 share. The colour comes from the palette's own accent, so an accent the operator sets reaches the pressed handle too.

`src/gui/theme_engine.py` — `SPLITTER_PRESSED_ALPHA` and the pressed handle

```
SPLITTER_PRESSED_ALPHA = 217

QSplitter::handle:pressed {{
    background-color: {rgba(t.accent_primary, SPLITTER_PRESSED_ALPHA)};
}}
```

Read off the rendered handle, five themes, before and after. The reading is the left edge at the handle's mid-height, and the handle's middle while the pointer holds it. Every edge now carries its own palette's accent hue, and every pressed handle now sits between that accent and the ground it stands on.

```
theme             edge before   edge after    pressed before  pressed after
cyberpunk_dark    47,72,84      38,106,104    0,255,204       1,218,176
neon_light        207,232,239   196,169,229   102,0,204       124,37,211
classic_terminal  5,48,28       4,86,4        0,255,0         1,218,1
minimal_modern    216,241,239   199,212,239   37,99,235       68,121,237
glass_metal       35,60,70      65,83,106     136,204,255     120,178,222
```

Seven palette values are gone. Each of the five palettes set `accent_success`, `accent_warning`, `accent_info` and `glow_color`, and the declared default held `font_mono`, `font_size_large` and `radius_lg`. No rule in the style-sheet text named one of the seven. A Qt style sheet has no glow property at all, and no widget marks itself as a success, a warning, an info, a monospace or a large thing for a rule to match. A palette carries 48 named values where it carried 55. `glow_color` also names a local colour inside three indicator panels, which is a different thing under the same name and stays.

One corner radius and one grey are repaired in the widget table. `RADIUS_FULL` held 9999 for a pill. Nothing imported it, and the drawing engine refuses a corner roundness above half the shape, so the value could draw no pill. The widget table holds 195 names where it held 196. `CARD_METRIC_LABEL` held a three-digit grey, the only one in the table, which is why a helper widens it before any comparison. It holds the same grey in six digits now, and the pixel does not change.

`src/gui/design_system.py` — the corner radii today

```
RADIUS_NONE = 0
RADIUS_XS = 4  # Chips, tight inputs
RADIUS_CARD = 6  # Analytics metric cards
RADIUS_SM = 8  # Default — buttons, inputs, cards
RADIUS_MD = 12  # Elevated cards, dialogs
RADIUS_LG = 16  # Large surfaces, bottom sheets (rare in desktop)
```

Both builds still write the same style sheet. The second build keeps its own copy of the palette table and of the template, in `src/gui/main_tabs/theme_engine_surface.py`. Each build's five style sheets match the other's byte for byte.

```
theme             characters before  characters after
cyberpunk_dark    7419               7404
neon_light        7415               7400
classic_terminal  7427               7402
minimal_modern    7422               7407
glass_metal       7415               7410
```

2026-10-26 - #1556 - the React tab book answers both tab-visibility questions

The Qt tab widget can hide a tab, and the React book could not be asked. The React book stands in for the Qt tab widget and forwards the calls the window makes over the hidden widget it holds. Two calls were missing: the one that asks whether a tab is on show, and the one that hides or shows it. Each raised an attribute error, so anything hiding a tab on the main book would have stopped the window.

`src/gui/react_main_window.py` — `MainTabBookReact.setTabVisible`

```
        def setTabVisible(self, index: int, visible: bool) -> None:
            """Show or hide the tab at ``index`` and redraw the bar without it."""
            self._book.setTabVisible(index, visible)
            self._push_tabs()
```

A hidden tab also leaves the bar. The bar is drawn from a list of labels the book sends it. That list was every label the book held, so a hidden tab would have stayed on the bar with no page behind it. The book now sends the visible labels alone, and keeps the full list for mapping a press back to a page, so a press still reaches the right tab.

Driven offscreen on a book holding three tabs. Nothing on this machine draws text offscreen, so every figure below is a value the widget reports or the list the page is sent, never a picture.

```
before   isTabVisible(1)         raised 'MainTabBookReact' object has no attribute
         setTabVisible(1, False) raised 'MainTabBookReact' object has no attribute
now      isTabVisible(1)         True, then False once hidden
         isTabVisible(0)         True throughout
         labels                  Trading, Charts, History, throughout
         the bar is sent         Trading, History
         shown again             Trading, Charts, History
```

The neighbouring tab is the control. Tab 0 reads True on every reading, and the full label list is unchanged, so the False on tab 1 is a reading and not a book that lost its tabs.

A press still reaches the page it names. The book maps a pressed label back to a page through the full list, so hiding a tab does not shift the pages after it. Driven on a book of four tabs with the second hidden, each press landing on the page whose label it carried.

```
hidden            Charts
pressed Trading   page 0, reads Trading
pressed History   page 2, reads History
pressed Console   page 3, reads Console
shown again       pressed Charts, page 1, reads Charts
pressed Nowhere   the page does not move
```

The last two rows are the controls. A label the book carries again is reachable again, and a label the book does not carry moves nothing.

2026-10-27 - #53 - one declaration carries the mouse-over box's size and its wait

A control's mouse-over description draws in a box, and four figures decide that box. Three are its size and one is its wait. All four now sit together in `src/gui/theme_engine.py`, and every rule that draws the box reads them from there.

`src/gui/theme_engine.py` — the one declaration

```
TOOLTIP_MAX_WIDTH_PX = 560
TOOLTIP_FONT_SIZE_PX = 12
TOOLTIP_PADDING_PX = 8
TOOLTIP_DELAY_MS = 3000
```

Four style sheets draw the box, and two of them used to carry their own numbers. `src/gui/main_tabs/theme_engine_surface.py` typed 560px, 12px and 8px into its template. `src/gui/main_window.py` read the width and set no text size and no spacing at all. Both read the declaration now.

```
rule set                                          width  size  padding
src/gui/theme_engine.py                             560    12        8
src/gui/main_tabs/theme_engine_surface.py           560    12        8
src/gui/main_window.py                              560    12        8
src/gui/main_tabs/main_window_surface.py            560    12        8
```

The wait is a style hint, not a timer. `src/gui/tooltips.py, in WakeDelayStyle` answers Qt's own wake-up hint with `TOOLTIP_DELAY_MS`, and `main.py` puts that style on the application at launch. Qt restarts the wait on every pointer movement, however small, so the pointer has to hold still for the full three seconds.

```
application without the style   the toolkit's own 700 ms
application with the style      3000 ms
```

The React History page carries its own declaration, read by its own rule and its own timer. `src/gui/web/history_panel.css` declares the two figures on `:root`. The `.tooltip` rule reads the width

and `src/gui/web/history_panel.js` reads the wait, which it used to hold as a number of its own.

`src/gui/web/history_panel.css` — the React declaration

```
:root {
  --tooltip-max-width: 560px;
  --tooltip-delay-ms: 3000;
}
```

Read off the running page. The page was asked what wait it will use and what width its box draws at, with a neighbouring element as the control.

```
--tooltip-delay-ms           3000
--tooltip-max-width          560px
the wait the page will use   3000
.tooltip max-width           560px
.pager max-width             none
```

The pager is the control. The rule reaches the box and not the pager, so the 560px is a reading of the rule rather than of every element on the page.

Move the declaration and every box follows it together. Set the four figures to numbers nothing else uses and read all four Qt rule sets and the running React page again.

```
                                          branch        before this change
src/gui/theme_engine.py                   377 17 3      377 17 3
src/gui/main_tabs/theme_engine_surface.py 377 17 3      560 12 8
src/gui/main_window.py                    377 17 3      377 -- --
src/gui/main_tabs/main_window_surface.py  377 17 3      377 17 3
the wake hint                             4321          4321
the React page's wait                     4747          not reachable
the React box's width                     393px         393px
```

What this does not change. No description's words change. 597 controls across the Live, Paper and Simulator screens set one, 365 of those sentences run over ten words, and rewriting them waits on the definition of `unavoidable` that requirement 3 asks for.

The rule sits on the application, so all three Qt screens draw from it. No `QToolTip` rule exists under `src/gui/paper/` or `src/gui/simulator/`. The Simulator's darker tone restates the whole block under its own selector, built from the same four figures, so a box on a Simulator screen reads the same.

```
plain tone      maximum width 578   text size 12   margin 9
Simulator tone  maximum width 578   text size 12   margin 9
```

The 578 is the 560 plus the box's own 9 pixel margin on each side.

Nothing else on a screen moves. All ten style sheets, five themes on each build, are byte-identical to the ones before this change.

```
theme             characters   unchanged
cyberpunk_dark    15605        yes
neon_light        15601        yes
classic_terminal  15599        yes
minimal_modern    15611        yes
glass_metal       15618        yes
```

One box no rule can reach. 124 React controls pass a `title` property, which draws the browser's own box. That box takes no stylesheet and no wait figure, so neither declaration reaches it. Changing those sites to the page's own box is a separate job, one screen at a time.

2026-10-27 - #53 - every mouse-over description fits twenty words

The operator set the limit:

> *A longer flat limit, no exceptions.*

The figure is twenty words. Every mouse-over description meets it, and no class of control is exempt. That figure replaces `ten words unless unavoidable`, which had no definition of `unavoidable` and so gave no sentence a verdict.

The same screen was written three times. The settings tab and the creation wizard each exist as a Live file, a Paper file and a Simulator file, and 113 of their controls carried a byte-identical description in all three copies. Each copy was its own sentence, so a repair to one left the other two behind.

```
                                on origin/current   on this change
controls with a description            597 calls          597 calls
descriptions over twenty words             228                  0
copies of a shared sentence                  3                  1
shared declarations                          0                112
```

`src/gui/descriptions.py` now holds one dict per screen, keyed by the control's own name, and the Live, Paper and Simulator copies of that screen all read it. One control is left out: the indicator panel sets its own window description twice, so it has no single declaration to carry.

```
SETTINGS_TAB_DESCRIPTIONS        50 controls
BOT_WIZARD_DESCRIPTIONS          49
FOLD_TRANCHES_TAB_DESCRIPTIONS    6
TRADING_TAB_DESCRIPTIONS          3
STACK_TRANCHES_TAB_DESCRIPTIONS   2
INDICATOR_PANEL_DESCRIPTIONS      1
PHANTOM_BOTS_TAB_DESCRIPTIONS     1
```

One control keeps its own sentence on each build. The indicator panel's staleness banner says more on Live than on Paper and the Simulator, because the Live panel also draws it when a bot has no stored reading at all. Forcing one sentence on all three would lose that, so the three stay separate.

Move the one declaration and all three screens follow. The Aggressive Trading box was set to wording nothing else uses, the three settings tabs were built, and each was asked what its box reports.

```
                    this change                       on origin/current
Live                the moved wording                 the moved wording
Paper               the moved wording                 its own old sentence
Simulator           the moved wording                 its own old sentence
control, the box's label    unchanged on all three  unchanged on all three
```

On `origin/current` there is no shared declaration, so the same move reaches the Live file alone. Both files were restored and read byte-identical by sha256.

Four descriptions were wrong about the code, and each is repaired.

```
control               where it sat                    what it said
Split Distance        the three settings tabs         a Logarithmic spacing mode
Standing Surplus      the three settings tabs         the pool never drains
Profit Folding        the three settings tabs         only this control turns it off
Privacy Mode          two exchange tabs               18 masked fields
```

The spacing modes are `linear`, `quadratic`, `fibonacci` and `exponential` in `src/trading/stack_math.py`, and `src/trading/container/config.py` migrates the old `logarithmic` value to `quadratic`. The pool does drain: `src/trading/scrumming_bot.py` reduces it by whatever Max Target Growth allows and names the drained figure in its own log line. The creation wizard turns Profit Folding off for every Extractor bot, so the settings control is not the only place it is set. `src/core/privacy_mask_registry.py` holds 21 fields, not 18, so the new sentence names no count and cannot drift again.

What a shorter sentence costs. A description capped at twenty words cannot carry every clause the long one did. Each rewrite keeps the fact the control is for and drops the rest: a worked example, a figure for a default that the box already shows, or a second mechanism named in passing. The longest loss is the data pool freshness line, which fell from 79 words and no longer spells out the cache-hit arithmetic or the per-entry lifetimes.

What is still open on this item. A description on every control that has none is unbuilt, and `src/gui/tooltips.py, in set_description` still has no callers. 178 of the 597 calls build their text at runtime, so no static count reaches them.

Eight descriptions are built while the screen runs, and their fixed words ran over too. A word count of a stored sentence cannot see them, so they were counted a second way: the fixed text inside the template, with the values left out. Eight came out over twenty words, and all eight are now under.

```
                                      fixed words before   after
the Fire button, five states                 22 to 26     15 to 19
the minimum rebuy guide, three tabs                41          17
```

The Fire button's five sentences lived twice — once in `src/gui/main_tabs/bot_status_table_surface.py` as a named value the React build reads, and once typed again inside `src/gui/widgets/bot_status_table.py`. The Qt table now reads the named value, so both builds draw one sentence.

```
                         this change        on origin/current
the five sentences       15 to 19 words     22 to 26 words
one declaration          yes                no, each build its own copy
```

Two runtime lines are appended to a Fire description and are not part of the sentence: a ceiling warning, and the list of gates blocking a fire.

2026-10-28 - #53 - the four live screens route every description through one helper

The operator ranks four screens as the ones he works: Trading, Charts, Bot Swarm and Market Inspector. On those screens every control now carries a description, every description fits the twenty-word figure, and every one of them is set by `src/gui/tooltips.py, in set_description`.

The one way in had no callers, so no rule could reach a single box. The helper existed and 84 calls on these four screens went straight to the toolkit instead. Each of those 84 is now a `set_description` call, so the next rule about a description is one change rather than 84.

```
                                 before   after
set_description calls in src          1     100
setToolTip calls in src/gui         603     520
on the four screens                  84       0
```

The twenty was prose, and prose reaches no screen. No module held the figure: `src/gui/theme_engine.py` declares the box's width, padding, text size and wait, and none of the four is a word count. `src/gui/descriptions.py, in DESCRIPTION_WORD_LIMIT` now holds it, `set_description` reads it on every call, and a description over it is still shown and is named in the GUI log with its word count. The screen never loses a sentence to the figure, and an over-long one stops being invisible.

What the built screens report. Each of the four was built with no window on screen, its widget tree walked, and `toolTip()` read off every control. A scrollbar and the toolkit's own tab arrows are left out; they are the toolkit's, not the product's.

```
screen                        controls   no description   over twenty words
                                         before  after    before   after
Trading                             44      10      3         7       0
Trading, one exchange layer         37       6      0        10       0
Charts                              19       1      0         3       0
Bot Swarm                           17       7      0         0       0
Market Inspector                    73      36      0         3       0
Market Inspector, topologies         3       0      0         0       0
Market Inspector, topology preview  14       9      0         1       0
total                              207      69      3        24       0
```

The three left are the indicator panel's blank padding columns. `src/gui/main_tabs/indicator_panel_surface.py, in column_titles` pads each table to a fixed column count with an empty title, so those three head nothing and describe nothing.

Where the 66 new descriptions went.

```
the Market Inspector's credential boxes      20, one format naming the field and venue
its venue buttons                            10, each naming its credential state
its ATA settings boxes                        6, one per setting a phase reads
the Bot Swarm's swarm buttons and theme menu  7
one exchange layer's fleet commands           6, each naming what SHIFT adds
the topology preview's two column sets        8
the six Add Exchange buttons on the empty-state cards
the indicator panel's bot menu, the Charts timeframe menu, New Bot, Cancel
```

Two sentence sets lived in several files, and a screen read the wrong copy. The Extractor table's eight column descriptions were declared four times — in `src/gui/main_tabs/extractor_bot_table_surface.py` and again in the Live, Paper and Simulator tables — and the topology preview typed four of its own beside the surface that already held them. Each now reads the one declaration, so the Qt screen and the browser screen draw one sentence.

```
                                        declarations before   after
Extractor column descriptions                            4       1
topology preview's bots, wires and adopt                 2       1
```

The three Extractor tables were built and each asked what its eight column headings report: all three read the same, and all three are under the figure. Control: changing one column in the surface alone makes that comparison report a difference.

The copies had already drifted. `src/gui/main_tabs/market_inspector_topologies_surface.py` still held a 31-word bots sentence and a 21-word adopt sentence while the Qt dialog beside it held the short ones, so the browser screen was over the figure and the Qt screen was not.

What a shorter sentence costs here. The indicator panel's column headings lost the most. Their long form named the author of each indicator, the raw value range, and the second threshold a gate reads where it differs from the voter's. Each heading keeps what the column is and the one threshold that acts:

```
heading   words before   after
Conf                86      19
KER                107      17
ADX                 78      19
ZSc                 73      19
Comp                56      17
Net                 49      19
```

A column heading also carries the line about pressing the label to sort, which is nine of the twenty. `src/gui/main_tabs/bot_status_table_surface.py, in COLUMN_TOOLTIPS` is therefore held to eleven words a column, and the browser build appends a privacy line of ten instead.

One defect found in the same files. `src/gui/widgets/exchange_tab.py, in ExchangeTab` installs an event filter on the application and never took it off. A layer taken off the bar stopped its feeds and left the filter in place, so every later key press reached a freed widget. `stop_feeds` now removes it, and the same teardown that raised in that filter ends clean.

The rebuild is owed. `docs/Acervator-Product-Manual.pdf` is not rebuilt here; another branch holds that file. This page, `docs/manual/08-tabs.md`, is the one that needs it.

2026-10-29 - every other screen describes every control in twenty words

Every screen the window builds now gives each control a description of twenty words or fewer. No description names a version, an issue or a pull request. The section above covers four screens in the Qt build. This section covers the rest of the window, and the browser build of every screen.

What the built screens report. Each screen was built with no window on screen, once in the Qt build and once in the browser build. Every control, table heading and tab was asked for its description. Each figure adds the two builds together. A planted description of twenty-one words reads as over in both builds, and a planted one of twenty reads as within.

screen	controls	no description		over twenty		a number	
		before	after	before	after	before	after
Window header strip	30	6	0	21	0	0	0
Window status bar	0	0	0	2	0	0	0
Main tab bar	18	18	0	0	0	0	0
Live tab	270	34	0	27	0	0	0
Charts tab	38	15	0	0	0	0	0
Inspector tab	101	0	0	3	0	0	0
Swarm tab	36	13	0	0	0	0	0
History tab	22	10	0	2	0	0	0
Status tab	80	80	0	0	0	0	0
Console tab	4	4	0	0	0	0	0
Paper tab	126	62	0	11	0	0	0
Sim tab	121	81	0	11	0	0	0
Settings, crypto	212	137	0	4	0	0	0
Settings, stocks	212	137	0	4	0	0	0
Live Bot Settings, Scrumming	194	83	0	53	0	1	0
Live Bot Settings, Extractor	118	28	0	32	0	1	0
Paper Bot Settings, Scrumming	163	54	0	53	0	1	0
Paper Bot Settings, Extractor	107	19	0	32	0	1	0
Sim Bot Settings, Scrumming	163	54	0	53	0	1	0
Sim Bot Settings, Extractor	107	19	0	32	0	1	0
Live bot wizard	151	29	0	34	0	0	0
Paper bot wizard	145	30	0	30	0	0	0
Sim bot wizard	145	30	0	30	0	0	0
Error Log	16	14	0	0	0	0	0
Start All	4	4	0	0	0	0	0
Buy confirmation	6	6	0	0	0	0	0
Fleet unlock	6	0	0	0	0	0	0
Instance consent	4	4	0	0	0	0	0
every screen	2599	971	0	434	0	6	0

The Qt build draws the Status tab's column heads as labels, so that build counts no control there. Those heads now carry the sentences the browser build draws.

One sentence, both builds. A sentence that each build typed for itself now sits once, in the screen's surface module or in `src/gui/descriptions.py`. Both builds read that one copy. The Bot Settings tables, the phantom and lock tables, the Positions Held headings, the Settings dialog and the creation wizard hold most of them.

```
declaration in src/gui/descriptions.py   on origin/current   this change
SETTINGS_TAB_DESCRIPTIONS                                50            59
BOT_WIZARD_DESCRIPTIONS                                  49            61
PHANTOM_BOTS_TAB_DESCRIPTIONS                             1             2
FOLD_TRANCHES_TAB_DESCRIPTIONS                            6             6
TRADING_TAB_DESCRIPTIONS                                  3             3
STACK_TRANCHES_TAB_DESCRIPTIONS                           2             2
INDICATOR_PANEL_DESCRIPTIONS                              2             2
```

Every description goes through one helper. The screens the window builds set each description with `src/gui/tooltips.py, in set_description`. The Settings dialog sets each control's sentence from that control's declaration, in `src/gui/settings_dialog.py, in _describe_controls`.

```
                                on origin/current   this change
set_description calls in src                  112           717
setToolTip calls in src/gui                   512            19
```

Two of the 19 are inside the helper itself, and one is the header strip card that the helper calls through. The other 16 are on screens the window does not build.

Descriptions built while the screen runs. Each description built from live values was built with the longest values its inputs can take. All 186 of them fit, and the longest is twenty words. Where a description adds lines to a sentence, `src/gui/descriptions.py, in fitted` keeps each line only while the whole stays within twenty words.

```
description                                       words before   after
Fire, held, with gates, ceiling and detonation              68      17
Fire, armed, with ceiling and detonation                    42      20
Fire, nothing to do                                         74      16
Fire, the exchange count disagrees                          82      18
bot table heading, browser build                            30      20
asset mark, no logo and no web address                      24      18
Bot Swarm node                                              21      20
Extractor tranche row                                       92      17
Extractor pool                                              27      14
Position Value, two reads                                   59      18
creation wizard information button                          71       6
```

What a shorter description costs.

- The Fire button names the gates that hold it by their gate label. It names three and counts the rest. When the gates fill the twenty words, the ceiling and detonation lines are left out. When there is room for one line, the ceiling line comes first.
- The Position Value cell says that Ammo uses the exchange's count, and gives that count. It no longer explains why the two dollar figures on the row differ.
- The Extractor tranche row names the Extractor that holds the lease and the amount leased. The alt units and the last mark follow only while they fit. The paragraph about why the row is a record and not inventory is gone.
- The Bot Swarm node reads `Growth` where it read `Realised growth`.

- The creation wizard's information button says what pressing it does. The box it opens still holds the asset's full description.

Where the readings above now differ

The readings above stand as written. This table gives what each one reads now.

written above	what it reads now
"Two runtime lines are appended to a Fire description and are not part of the sentence: a ceiling warning, and the list of gates blocking a fire."	Three lines can follow the sentence: the gates that hold it, a ceiling line and a detonation line. Each is kept only while the whole stays within twenty words.
"the five sentences 15 to 19 words"	The five sentences are 11 to 13 words, so a line can follow each one.
"A description on every control that has none is unbuilt"	Every control the window builds carries a description.
"178 of the 597 calls build their text at runtime, so no static count reaches them."	The 186 descriptions built at runtime were driven at their longest values, and none is over twenty words.
"`src/gui/descriptions.py, in DESCRIPTION_WORD_LIMIT` now holds it"	`src/gui/descriptions.py` holds the figure, and the helper imports it from there.
"The three left are the indicator panel's blank padding columns."	Each blank head sits over a pillar the other table titles, and carries that pillar's sentence.
"the browser build appends a privacy line of ten instead."	No heading carries a privacy line. The dot under the label carries the privacy state.
"SETTINGS_TAB_DESCRIPTIONS 50 controls" and the rows under it	The declaration table in this section gives the current counts.
"set_description calls in src 1 100"	717 calls, and 19 direct calls are left in `src/gui`.

The padded heads take their sentence from `src/gui/main_tabs/indicator_panel_surface.py, in column_tooltips`. A heading's sentence comes from `src/gui/main_tabs/bot_status_table_surface.py, in header_tooltip`.

The rebuild is owed. `docs/Acervator-Product-Manual.pdf` is not rebuilt here. This page, `docs/manual/08-tabs.md`, needs it.

2026-10-28 - #1374 - where the owed rebuild now stands

Where the sentence about the rebuild now reads differently

One reading above describes the state before this branch. It stands as written, and this is the reading that replaced it.

written above	what it reads now
"**The rebuild is owed.** `docs/Acervator-Product-Manual.pdf` is not rebuilt here; another branch holds that file. This page, `docs/manual/08-tabs.md`, is the one that needs it."	`docs/Acervator-Product-Manual.pdf` is rebuilt in this branch from the merged pages, and it carries this page

`tools/build_product_manual.py` is what builds it, and the page order it reads is the order of `docs/manual`.

Subsystem Detail

Reference. One file per subsystem tab, written from the source. Each file names the module that implements the screen, the symbols inside it, the bridge method that serves its renderer, and the screen's current state.

The parent part is 08-tabs.md, which points here for every screen but Main Window, Market Inspector and Asset Charts, and carries the manual's own text for those three. The rows below run in the order the tab list in 04-manual-parts.md sets, which is the order 08-tabs.md runs its sections in.

How an update is written down

Each tab has one section, and that section is its file below. An update to a tab takes a dated subsection under that one heading and never a second section named for the tab.

```
    ## 2026-09-08 14:20 - #407, #442 - the six zones and their arrows
       date       time    issues addressed   what the update did
```

The date leads so the subsections sort, and they run forward down the page. An update with no issue behind it writes `no issue recorded` in that field rather than a number nothing supports. `.claude/rules/documentation.md` bans an issue number in documentation, and the operator's directive of 2026-09-08 amends it for this header alone; version names stay banned everywhere.

`dev_harness/harness/docs_archetype.py` refuses both halves

```
rule_id="DOC011"   # two contents sections naming one tab
rule_id="DOC012"   # a dated update that runs backwards
```

Contents

File	Covers	State
portfolio-panels.md	The header strip: spendable columns, five counter cards, privacy dots	Live
simulator.md	Live's tab forked over Stone Tablets: Validation, Back Test, Portfolio Battery, the gate-latch criterion, the replay layer	Live, rebuilt under issue #117
paper-trader.md	The live feed, the fake balance, the paper log	Live — issue #19
proof-of-accumulation.md	Identity, Merkle log, competitions, ACRV, the local chain	Skeleton tab, engine only — issue #147
market-inspector.md	Higher-timeframe scanner, opposing pairs, topology proposals, adopt	Live
bot-swarm.md	Nodes, Smart Wires, wire credits, the fold-tranche book	Live
asset-charts.md	One candlestick panel per traded symbol	Live
history.md	Venue trade history, grading, gate analysis	Live
console.md	Python log tail and the emitter signal stream	Live
system-status.md	Emitter Network and Watchdog, the two halves that run today	Skeleton tab — issue #34
settings.md	Every control on the nine Settings pages, and what each page stores	Live
alerts.md	The two channel forms, the Event Routing table, the notification history	Both builds complete, the window opens neither
promotion-pipeline.md	How a strategy earns its way to real money	Four of four steps built

The live tab set

One list names the ten tabs the window builds, and each one is moved into that position after the builders run. Two of them are empty: Accumulation and Status.

`src/gui/main_tabs/main_window_surface.py` — `CANONICAL_TAB_ORDER`

```
CANONICAL_TAB_ORDER = (
    SIM_TAB,
    PAPER_TAB,
    LIVE_TAB,
    CHARTS_TAB,
    INSPECTOR_TAB,
    SWARM_TAB,
    ACCUMULATION_TAB,
    HISTORY_TAB,
    STATUS_TAB,
    CONSOLE_TAB,
)
```

The Live tab has its own section: 06-trading-tab.md. The Indicator Voting Panel has 07-indicators.md.

Every screen the window does not build gets a sentinel instead of a widget, so a legacy reader gets that sentinel rather than an error. The list covers the Competition and Local Testnet tabs, and the analytics, risk, journal, alerts and audio pages.

`src/gui/main_tabs/retired_tabs.py` — `RetiredTabsMixin._install_retired_tab_sentinels`

```
  def _install_retired_tab_sentinels(self) -> None:
      """Assign None to ``_competition_tab``, ``_testnet_tab`` and the rest.

      ``main_window`` tests ``_risk_tab`` and ``_alerts_tab`` before
      refreshing them.
      """
```

The underlying engines behind several of those keep running; only the surface is gone.

Read on the current tree, the engines that ran for nothing are stopped. Of the engines behind those pages, the window builds two: the risk engine and the notification manager. The risk engine still runs every 2 seconds, and its drawdown alerts still reach the Console and SMS. The audio suite's chime still plays from the Market Inspector.

`src/gui/main_window.py` — `MainWindow.__init__`, the two engines the window builds

```
              from ..trading.risk_manager import RiskManager
              from ..core.notifications import get_notification_manager

              self._risk_manager = RiskManager(bot_manager)
              self._notif_manager = get_notification_manager()
```

engine	before	now
analytics	built at launch, a snapshot about every 10 s	not built
testnet bridge	installed at launch, a 250 ms timer for the whole session	not installed
trade journal	built at launch, three day files read	built only when an AI note is written
reconciliation engine	built at launch, never called	not built
crash snapshot	a whole-fleet file written every 30 s	not built
risk engine	two fleet sweeps every 2 s	one sweep

When a risk rule raises its alert

Each of the seven risk rules raises its alert only for a reading strictly beyond its limit. A reading exactly on the limit raises none. Every alert text says the reading "exceeds" the limit, and every alert's reading does. The one test sits on the rule, and every rule calls it.

`src/trading/risk_manager.py` — `RiskRule.exceeded_by`

```
    def exceeded_by(self, reading: float) -> bool:
        """True only for a ``reading`` strictly above ``threshold``; ``evaluate``
        applies this one test to every rule.
        """
        return reading > self.threshold
```

Every alert writes one warning line to the Console and to `system.log`. A critical alert also goes to the notification manager, which sends it to the alert sound, Telegram and SMS. SMS sends only while the "P/L threshold alerts" box on `Settings > SMS` is ticked.

rule	limit	what it reads	severity
`max_drawdown`	10 %	the fall of total P/L from its peak	warning
`critical_drawdown`	25 %	the fall of total P/L from its peak	critical
`per_bot_loss`	$50	one bot's realised loss	warning
`asset_concentration`	40 %	one asset's share of the running target balance	warning
`exchange_concentration`	60 %	one venue's share of the running target balance	warning
`correlated_exposure`	50 %	the combined share of a pair at a correlation of 0.7 or more	warning
`rapid_loss`	5 %	the fall of total P/L inside 300 s	critical

Driven on constructed fleets in a redirected home, with every send stopped at its import. One step is 0.0001 of P/L, the place each bot's status rounds to, or one cent of target balance.

```
rule                    one step below   on the limit   one step above
max_drawdown            no alert         no alert       warning
critical_drawdown       no alert         no alert       critical, to SMS
per_bot_loss            no alert         no alert       warning
asset_concentration     no alert         no alert       warning
exchange_concentration  no alert         no alert       warning
correlated_exposure     no alert         no alert       warning
rapid_loss              no alert         no alert       critical, to SMS
```

A rule that alerts waits out its cooldown before it alerts again: 300 s, and 600 s for `rapid_loss`. A reading on the limit raises nothing, so it starts no cooldown. Another asset beyond the limit in the same sweep then raises the alert for that rule.

How a screen reaches its renderer

One registry maps a method name to a view-model function. A surface is a plain function that takes the request's parameters and returns a serialisable result, and a screen is reachable exactly when its method appears in that registry.

`src/core/desktop_bridge.py` — `build_registry`

```
def build_registry() -> Dict[str, Handler]:
    """Return the methods the frontend may call, keyed by method name.

    This is the wiring point for the whole frontend: a surface is
    reachable exactly when it appears here. Surfaces are imported inside
    the function so that the transport above carries no dependency on any
    one domain package.
    """
```

The seven surfaces behind the retired tabs are in that registry, and building the registry imports none of them. Each one is imported on the first request for its own method, so a session that never asks for a retired page never loads its surface.

`src/gui/main_tabs/retired_surfaces.py` — `on_demand_handlers`

```
def on_demand_handlers() -> dict[str, Callable[[dict], dict]]:
    """Return one handler per ``SURFACE_MODULES`` method, unimported until called."""
    return {method: _handler(module) for method, module in SURFACE_MODULES.items()}
```

Read on the current tree, a packaged build carries none of the seven surfaces. `src/gui/main_tabs/retired_surfaces.py, in on_demand_handlers` registers a page only when the running program can find its surface, so a packaged build's registry holds none of the seven methods. A request for one gets the bridge's `UnknownMethod` refusal, and no import runs. A run from the source folder still answers all seven.

`src/gui/main_tabs/retired_surfaces.py` — `on_demand_handlers` and `_is_carried`, as they read now

```
def on_demand_handlers() -> dict[str, Callable[[dict], dict]]:
    """Return one handler per ``SURFACE_MODULES`` method, unimported until called.

    A method whose module ``_is_carried`` cannot find gets no handler.
    """
    return {
        method: _handler(module)
        for method, module in SURFACE_MODULES.items()
        if _is_carried(module)
    }


def _is_carried(module_name: str) -> bool:
    """Return True when ``find_spec`` finds ``module_name`` beside this module."""
    return importlib.util.find_spec("." + module_name, __package__) is not None
```

The rule that leaves the seven out of the build is on 17-what-the-build-includes.md, under "A screen the window never builds".

The transport is one JSON object per line over the pipe the parent process already owns: no socket and no port.

Each surface module holds the screen as data. Nothing in one imports Qt, which is why the same view model serves the Qt tab and the renderer without a second description of the screen existing.

Thirty-three surface modules wrote their values out a second time rather than reading them from the module that declares them. Four of them now read that one declaration. The design system table and the theme engine import no Qt, so a surface that imports none can read either.

`src/gui/main_tabs/design_system_surface.py`, in the module body

```
TOKENS: dict[str, Any] = {
    name: getattr(design_system, name) for name in TOKEN_NAMES
}
```

The rest keep their own copy. Each names a Qt source, and a surface reaches no Qt source, so no read of the one declaration is available to them. Their docstrings state what the module does instead: nothing compares the two copies.

Back to the manual index.

Portfolio Information Panels

Reference. The strip above the tab bar, present on every tab except the two isolated ones.

What builds it

One method builds the strip. It makes the window's central widget, lays a single row across the top of it, and hands back the layout the tab row goes into. That row holds the five spendable columns on the left and the five counter cards on the right, and it does not move as you change tabs.

`src/gui/main_tabs/header_strip.py` — `HeaderStripMixin._build_header_strip`

```
self._spendable_widget = SpendableProfitsWidget()
top_row.addWidget(self._spendable_widget, stretch=3)
```

The row draws seven slots: the money strip, the five counter cards, and the asset class square that ends it. The first six divide the whole row between them, at three shares for the money strip and one for each card. The square takes no share, so its slot shrinks to the square and the square draws hard against the row's right edge. No background is left anywhere in the row.

`src/gui/main_tabs/header_strip_surface.py`

```
TOP_ROW_STRETCH = [3, 1, 1, 1, 1, 1, 0]
```

Overtaken, quoted whole: "TOP_ROW_STRETCH = [3, 1, 1, 1, 1, 1, 0]".

True today: `TOP_ROW_STRETCH` holds eight shares, `[6, 1, 1, 1, 1, 1, 1, 0]`, and is not written out. `header_strip_surface.py` builds it from `KPI_COLUMNS` and `COUNTER_CARDS` — one share per money column the strip holds, `COUNTER_STRETCH` for each card, and `CLASS_GROUP_STRETCH` for the class square. Six columns and six cards answer the six, the six ones and the nought, so the list carries one share per

TOP_ROW_ORDER slot by construction. The ACCUMULATED card is the sixth, added after this page was written.

Each of the six also carries a floor, the width its own whole amounts need, so a long figure is never cut.

`src/gui/main_tabs/header_strip.py` — `HeaderStripMixin._build_header_strip`

```
  card.setMinimumWidth(surface.slot_natural_w(slot))
```

Overtaken, quoted whole: "The first six divide the whole row between them, at three shares for the money strip and one for each card."

True today: the money strip's three shares come out below its own floor at every window up to about 1950 pixels wide, so it draws at that floor and the five cards take the whole surplus between them.

Both builds draw the row at the same geometry. The floor the Qt row sets on a widget is the number the React page sets on the same slot, and the surface serves that number to both.

`src/gui/main_tabs/header_strip_surface.py`

```
  "slots": {slot: slot_natural_w(slot) for slot in TOP_ROW_ORDER}
```

`src/gui/web/header_strip.js`

```
  style.minWidth = length(floor);
```

Measured at a 1920 pixel window, in both builds: the money strip 686, each card 217, the square 112.

Overtaken, quoted whole: the two code blocks above, which name `slot_natural_w` at both sites, and "Measured at a 1920 pixel window, in both builds: the money strip 686, each card 217, the square 112."

True today: both sites read `slot_floor_w`, which answers `slot_natural_w` for the money strip and `slot_min_w` for every other slot. `top_row_min_w` adds up the same `slot_floor_w`, so the narrowest window the budget names is the narrowest the row can draw at — 1384 pixels, which is what Qt reports for the real window.

`src/gui/main_tabs/header_strip.py` — `HeaderStripMixin._build_header_strip`

```
  card.setMinimumWidth(surface.slot_floor_w(model["key"]))
```

`src/gui/main_tabs/header_strip_surface.py` — `width_budget`

```
  "slots": {slot: slot_floor_w(slot) for slot in TOP_ROW_ORDER}
```

Read off the real row on the Qt build, with the machine's own fonts, at a 1920 pixel window: the money strip 866, each card 144 or 145, the square 148. At 1400 the strip reads 668, a card 90 or 91, and the square 148.

Left: the spendable columns

Five labelled columns fill the left half, divided by thin vertical rules.

Column	Meaning
SPENDABLE	Cash the exchange reports, across the bots sharing one wallet
REALISED	Realised profit and loss
LOCKED	Value the open positions hold
MATURE	Wire profit old enough to move
EXCH	Connected exchange count

The widget builds four of the five from one list, and each entry carries the label the operator reads and the payload key that fills it.

`src/gui/widgets/spendable_profits.py` — `SpendableProfitsWidget.__init__`

```
  for label_text, key in [
      ("REALISED", "total_realised"),
      ("LOCKED", "locked"),
      ("MATURE", "mature"),
      ("EXCH", "exchanges"),
  ]:
```

Overtaken, quoted whole: the five-row table above, and the four labels in the block above it.

True today: the strip draws six columns and REALISED is not one of them. `header_strip_surface.py, in KPI_COLUMNS` declares SPENDABLE, P/L, LOCKED, MATURE, EXCH and AMMO. `spendable_profits_surface.py, in COLUMNS` declares the same six, and `spendable_profits.py, in SpendableProfitsWidget._setup_ui` builds one column per entry of that list in place of the typed four. The name REALISED is spelled in neither file. P/L carries the only profit and loss figure the strip draws, and it is the unrealised figure the exchange answers.

One payload draws all five. A value the payload leaves out draws an em dash rather than a zero, which keeps "no reading" and "a reading of nothing" apart. That is the difference between a column with nothing behind it and a column reporting a genuine zero, and on this strip it matters: two of the five have nothing behind them today.

`src/gui/widgets/spendable_profits.py` — `SpendableProfitsWidget._money_text`

```
  @staticmethod
  def _money_text(value) -> str:
      """Render one amount as money text, or as the empty marker."""
      amount = SpendableProfitsWidget._amount_of(value)
      if amount is None:
          return "—"
      return f"${amount:,.2f}"
```

REALISED and MATURE are the two. Both are handed a literal absence at the one call site that fills the strip, so both draw the marker on every tick. The Trading tab section lists what each cell reads: 06-trading-tab.md.

Both now carry a figure, and the figure comes from the exchange. Realised is the venue's own matched profit and loss across the fleet. Mature is the profit held in positions worth more than three times what they cost, which is the growth threshold the platform applies everywhere it decides maturity.

Overtaken: "Realised is the venue's own matched profit and loss across the fleet."

The venue answers a cost basis, an average entry price and an unrealised profit per spot position, and no lifetime realised figure. Realised is the platform's own first in, first out match over the venue's own fills, one figure per bot, added across the fleet.

Overtaken: "Mature is the profit held in positions worth more than three times what they cost, which is the growth threshold the platform applies everywhere it decides maturity."

Mature is the part of a position's value that sits over three times its cost. A position worth $350 on a $100 cost basis holds $50 of mature profit. Three times the cost is still the threshold the platform applies everywhere it decides maturity.

`src/gui/main_tabs/header_strip_surface.py` — `profits_payload`

```
"total_realised": exchange_amount(data, "total_realized_exchange"),
"mature": exchange_amount(data, "total_mature_exchange"),
```

Absence is still a reading. When the venue has answered for no bot, both columns take nothing and draw the marker, and the mask leaves that marker alone. A hidden figure shows four asterisks; an empty column stays empty under Privacy Mode.

`src/core/privacy_mask_registry.py` — `mask_or`

```
text = str(value)
if field_id not in ALL_FIELD_IDS or text == ABSENT_TEXT:
    return text
```

What the venue answers, and what the platform derives

The venue's portfolio breakdown answers three figures per open spot position: the cost basis, the average entry price and the unrealised profit. Each bot reads those three from the venue on every refresh and holds them as its own. The breakdown carries no lifetime realised figure for a spot position, so Realised is derived: every fill the venue holds for the bot's symbol, matched first in, first out, one figure per bot, summed across the fleet.

`src/exchange/ccxt_connector.py` — `get_spot_positions`

```
portfolios = await self._call_sync(self._ex.fetch_portfolios)
rows = await self._call_sync(self._ex.fetch_portfolio_details, uuid)
basis = float(row.get("cost_basis", 0) or 0)
```

The venue's cost basis is the check on the derivation. After the fills are matched, the bot logs the venue's cost basis beside the cost of the buys FIFO left open and beside the average-cost basis, with each gap in per cent, and names the closer of the two. A venue whose account uses another matching method shows up there as a gap on both.

`src/trading/scrumming/reconciliation.py` — `_log_basis_check`

```
_fifo_gap = abs(_fifo - float(venue_basis)) / _scale
_avg_gap = abs(_avg - float(venue_basis)) / _scale
_closer = "fifo" if _fifo_gap <= _avg_gap else "average"
```

A venue without a portfolio breakdown leaves the three figures to the same derivation, and the check does not run.

`src/exchange/position_health.py` — `compute_position_health`

```
while sell_remaining > 1e-12 and buy_queue:
    buy_lot = buy_queue[0]
    take = min(buy_lot[0], sell_remaining)
    realized += (t.price - buy_lot[1]) * take
```

A sell is matched against the oldest open buy, so a sell placed above the latest fold and below the first buy of a market that has fallen since lowers the figure. Over the 2026 fill export, 1,302 of 2,653 sells lowered it and 66 were priced under the most recent buy.

Each bot holds the complete fill history for its symbol. The first refresh pages the venue newest to oldest until a page comes back short; every later refresh fetches one page and stops at the first fill it already holds.

`src/exchange/fill_history.py` — `FillHistory.refresh`

```
if len(fills) < FILL_PAGE_LIMIT:
    ended_short = True
    break
if joined:
    break
```

Before this, one call fetched the newest 500 fills and the figure was FIFO over that window alone. On the two symbols past 500 fills the window read $-137.38 and $119.91 where the complete history reads $-390.53 and $60.27.

`src/trading/scrumming/reconciliation.py` — `refresh_exchange_position_health`

```
_trades = await self.fetch_fill_history()
if _trades is None:
    return False
_asset_base = self.config.symbol.split("/")[0]
_ph = compute_position_health(_trades, _asset_base)
```

Mature applies one constant. A position counts once its value reaches its cost plus two hundred per cent of its cost, and the column sums the profit on the positions that qualify.

`src/trading/smart_wire.py` — `mature_profit_usd`

```
MATURE_GROWTH_PCT: float = 200.0
if value < basis * (1.0 + MATURE_GROWTH_PCT / 100.0):
    return 0.0
return value - basis
```

Overtaken: "the column sums the profit on the positions that qualify."

The column adds up what each qualifying position holds over the threshold. `mature_threshold_usd` is the cost plus two hundred per cent of the cost, and `mature_profit_usd` answers the value above it.

`src/trading/smart_wire.py` — `mature_profit_usd`

```
if not is_mature(cost_basis_usd, current_value_usd):
    return 0.0
return float(current_value_usd) - mature_threshold_usd(cost_basis_usd)
```

A position at three times its cost exactly is mature and holds nothing over the threshold. `is_mature` carries that test on its own, so the count of mature positions reads the position while the money column adds nothing for it.

`src/trading/smart_wire.py` — `is_mature`

```
threshold = mature_threshold_usd(cost_basis_usd)
if threshold <= 0.0:
    return False
return math.isfinite(value) and value >= threshold
```

Realised reads the same on every start

A bot writes its realised figure only when the walk reached the end of the venue's history. A walk that stops short keeps the last whole reading, writes `stats.fill_history_complete` False and says so in the log, so the same fills always answer the same figure.

`src/trading/scrumming/reconciliation.py` — `refresh_exchange_position_health`

```
_complete = bool(getattr(self._fill_history, "complete", False))
self.stats.fill_history_complete = _complete
if _complete:
    self.stats.realized_pnl_exchange = float(_ph.realized_pnl_usd)
```

The fleet aggregate answers `realised_history_complete` False while any bot the venue answered for reads False, and the column then draws the empty marker. An absent figure never looks like a present one.

`src/gui/main_tabs/header_strip_surface.py` — `realised_amount`

```
if not bool(data.get(REALISED_COMPLETE_KEY, True)):
    return None
```

The Paper and Simulator fleets walk no venue history, so their aggregates omit the key and their realised figure passes through whole.

Right: the five counter cards

Five cards close the row, and each one counts a single thing.

Card	What it counts
Scrummed	Cumulative USD sold across every bot since the process started
Folded	Cumulative USD bought across every bot since the process started
Trades	Executed buys and sells across the active bots
Bots	Bots in the RUNNING state
Errors	Errors across every bot since the last reset

Errors is the one card you can click. Arming it takes one call, and the click opens the rolling error buffer with its Reset button.

`src/gui/main_tabs/header_strip.py` — `HeaderStripMixin._build_header_strip`

```
self._stat_errors.set_clickable(True, "Click to open the error log.")
self._stat_errors.clicked.connect(self._show_error_log_dialog)
```

Current reading, 2026-10-29. The card takes no words of its own when it is made clickable. Its description and its click line are declared beside it.

`src/gui/main_tabs/header_strip_surface.py` — `ERRORS_CLICK_TOOLTIP`

```
ERRORS_TOOLTIP = "Errors across all bots since the last reset."
ERRORS_CLICK_TOOLTIP = "Press to open the Error Log, where Reset clears them."
```

Issue #428 carries a disagreement between what the tooltips on these cards promise and what the cards draw.

The issue #428 clause above is overtaken. It is not edited and not removed. It read:

> *Issue #428 carries a disagreement between what the tooltips on these cards promise and what the cards draw.*

Issue #428 closed on 2026-10-03. The two counter tooltips name the exchange's own trade history, which is the source the cards draw from, and both say the figure survives a restart.

```
src/gui/main_tabs/header_strip_surface.py

SCRUMMED_TOOLTIP   "the dollars every bot has sold so far this year ...
                    comes from the exchange's own trade history, so the venue
                    and the card agree ... It survives a restart."
FOLDED_TOOLTIP     the same sentence for buys
```

Current reading, 2026-10-29. Both counter descriptions are shorter, within twenty words. They keep the source and the restart.

`src/gui/main_tabs/header_strip_surface.py` — `SCRUMMED_TOOLTIP`

```
SCRUMMED_TOOLTIP = (
    "Dollars every bot has sold this year, read from the exchange's own trade "
    "history. It survives a restart."
)
```

A sixth card, P/L, is constructed and then hidden. Code that still writes to it keeps working and nothing appears on screen.

`src/gui/main_tabs/header_strip.py` — `HeaderStripMixin._build_header_strip`

```
self._stat_pnl = StatCard("P/L", "$0.00")
self._stat_pnl.setVisible(False)
```

The mode button on the right ends the row. `_toggle_trading_mode` swaps the window between the crypto and stock layers.

Overtaken, quoted whole: "The mode button on the right ends the row. `_toggle_trading_mode` swaps the window between the crypto and stock layers."

True today: one square ends the row. It is segmented two by two into one button per asset class, and a press makes that class active. The square keeps its own size, never stretches, and sits in the row's right corner.

`src/gui/main_tabs/header_strip.py` — `HeaderStripMixin._build_header_strip`

```
top_row.addWidget(
    class_group,
    surface.slot_stretch("mode_button"),
    Qt.AlignVCenter | Qt.AlignRight,
)
```

Privacy

Every field on the strip carries its own privacy dot. Attaching one registers the field, and from then on the card draws its value through the mask.

`src/gui/main_tabs/header_strip.py` — `HeaderStripMixin._build_header_strip`

```
self._stat_scrummed.attach_privacy_dot("counter.scrummed")
self._stat_folded.attach_privacy_dot("counter.folded")
self._stat_trades.attach_privacy_dot("counter.trades")
self._stat_bots.attach_privacy_dot("counter.bots")
self._stat_errors.attach_privacy_dot("counter.errors")
```

The registry behind those field ids is shared. Hide a field here and the same field hides on the Bot Swarm tab, because both surfaces ask the one registry.

`src/core/privacy_mask_registry.py` — `mask_or`

```
def mask_or(value, field_id: str, mask: str = "****") -> str:
    """Return ``mask`` when ``field_id`` is masked, else ``str(value)``.

    A ``field_id`` outside ``ALL_FIELD_IDS`` never masks, which covers every
    ``TA_FIELD_IDS_EXCLUDED`` entry.
    """
```

Where the numbers come from

One aggregate call fills the whole strip, once a tick. Every card and every column is written from that single answer, so two cards cannot disagree about the same fleet.

`src/gui/main_window.py` — `_refresh_dashboard`

```
agg = self._bot_manager.get_aggregate_stats()
_scr = float(agg.get("total_scrummed_usd", 0.0) or 0.0)
_fld = float(agg.get("total_folded_usd", 0.0) or 0.0)
self._stat_scrummed.set_value(f"${_scr:,.2f}")
self._stat_folded.set_value(f"${_fld:,.2f}")
```

The aggregate itself is `FleetAggregationMixin.get_aggregate_stats` in `src/trading/container/aggregation.py`.

Hiding

Two screens are isolated from live trading, and on those two the strip hides itself. The absent strip is the signal: no live numbers are on this screen.

`src/gui/main_window.py` — `_on_main_tab_changed`

```
isolated_tabs = set(ISOLATED_TABS)
container = getattr(self, "_header_strip_container", None)
if container is not None:
    container.setVisible(tab_name not in isolated_tabs)
```

The window surface carries the same pair as a constant, and issue #450 renamed both labels.

`src/gui/main_tabs/main_window_surface.py` — `ISOLATED_TABS`

```
ISOLATED_TABS = (SIM_TAB, PAPER_TAB)
```

The Simulator draws its own strip in place of this one. `SimStatStrip` in `sim_stat_strip` mirrors these ten fields against sim balances. The Simulator rebuild removed this file; it is not in the tree.

The strip stays on Sim and reads the sim fleet

The Simulator rebuild took the Sim tab out of the isolated pair, so the strip hides on Paper alone. On Sim the same ten fields carry the Simulator's figures, read from the Sim tab's fleet source, and the live fleet's numbers do not reach the strip while Sim is in front. The window picks the fleet on every tick and on every tab change.

`src/gui/main_tabs/main_window_surface.py` — the two tuples

```
ISOLATED_TABS = (PAPER_TAB,)

#: The tabs the header strip reads the Simulator's fleet on, not the live one.
SIM_FED_TABS = (SIM_TAB,)
```

`src/gui/main_window.py` — `_refresh_header_strip`

```
if self._header_strip_reads_sim():
    sim_tab = self._simulator_tab
    self._write_header_strip(
        sim_tab.fleet_source().aggregate(), sim_tab.exchange_count()
    )
    return
```

simulator.md covers what each field reads on Sim.

Bridge

The strip answers one bridge method, and one renderer module draws it.

`src/gui/main_tabs/header_strip_surface.py` — `METHOD`

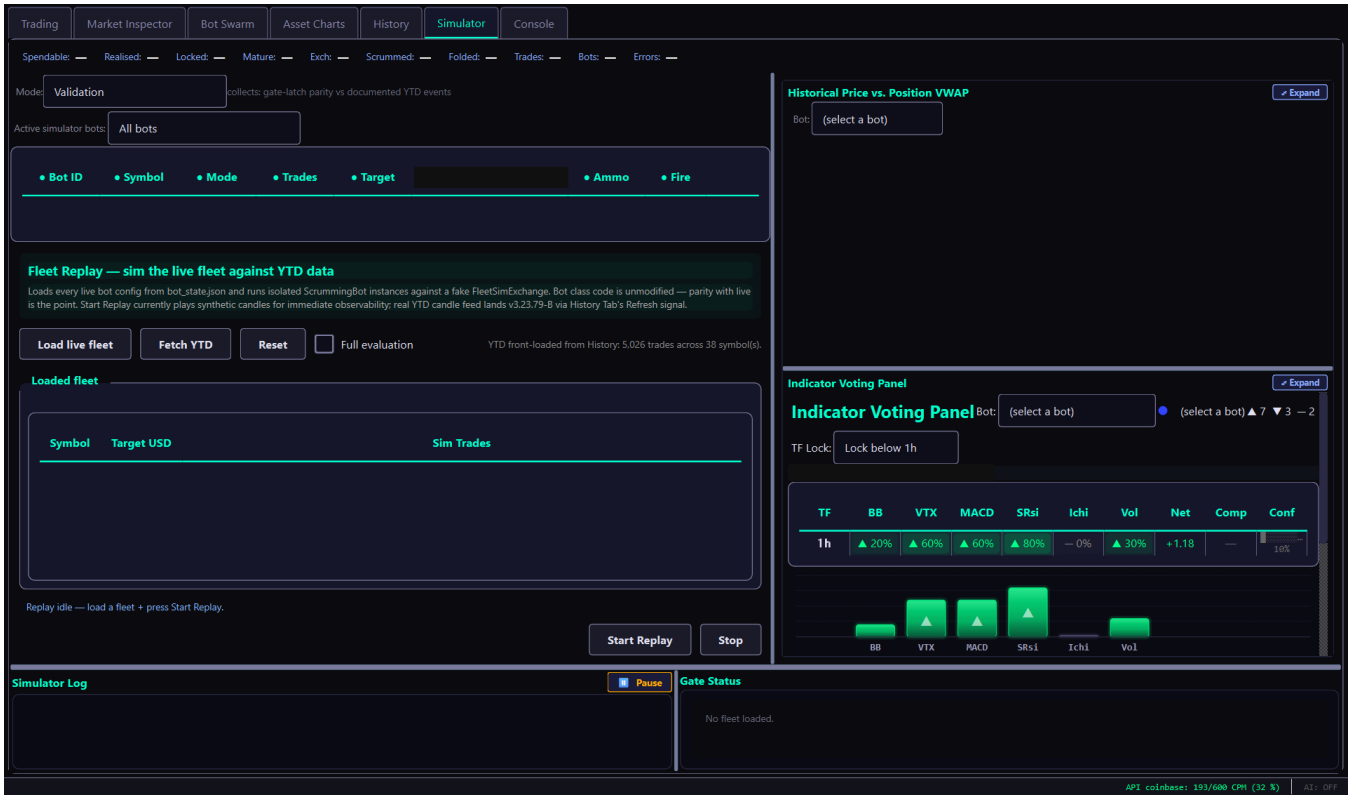
```
METHOD = "header.strip"
```

The renderer module is `header_strip.js`.

Back to the subsystem index.

Simulator Tab (Hot Mess; Complete Rebuild In Progress)

Now: the Sim tab draws the Live tab's layout over stored candles, with three run modes, and it sends no order (`src/gui/simulator/sim_trading_tab.py, in SimTradingTab`).



The Simulator tab, with no fleet loaded.

The figure shows the tab the rebuild replaced. Its Mode list, Load live fleet, Fetch YTD, Start Replay and Simulator Log are gone.

How to use the Simulator tab

This part is for lookup. Each row names a control or a group of them, says what it does, and cites the code that reads it. A row that holds a value gives its range and start value, or links to the Live row that holds them. Where Live's page holds a row for the same control, this page's row or section links there and says what differs.

The Sim tab holds its own fleet. A bot here is a stored record and not a live bot. The tab places no order, and it saves to the sim fleet file and not to the live fleet file (`src/simulator/fleet_source.py, in FleetSource.save`).

Two builds of this tab

A frozen build draws the variant stamped into it. A source run reads `ACERVATOR_VARIANT` and draws React when nothing names a variant (`src/_variant.py, in resolve_variant`).

The React page sends its presses to Python (`src/gui/simulator/sim_react_trading_tab.py, in SimTradingTabReact.run_action`). Two reach nothing there: the panel's Privacy dot and a press on an Extractor row. A press on an Asset or Symbol cell reaches Python and opens nothing. A control that works in one build alone is marked.

The three run modes

The tab opens on the mode the sim fleet file names, else Validation (`src/simulator/fleet_source.py, in _read_sim_mode`). Each mode holds its own fleet. A mode press draws that fleet and writes one line on the Activity Log (`src/gui/simulator/sim_trading_tab.py, in SimTradingTab.set_mode`).

Mode	What a run does and what it reads
Validation	Puts each trade in the YTD trade files on its Stone Tablet candle, runs the gate chains again on that candle, and sets the new gate lights beside the lights Live recorded. A trade needs a tablet and a recorded gate row within five minutes of it. A pair with files for two years reruns the newest file alone. One press reruns up to 400 trades. `src/simulator/validation.py, in _validate`, `src/gui/main_tabs/simulator_tab_surface.py, in VALIDATION_RERUN_LIMIT`
Back Test	Walks each bot along every stored bar of its asset on its venue, rolled up to its TA Timeframe. The bot opens worth its Target Balance at the 30th bar, and each fold spends that bot's own sale proceeds. `src/simulator/back_test.py, in walk`
Portfolio Battery	Walks a named historical portfolio over one span and sets each Harvest-Fold end beside the HODL end. No cash cap holds a fold, though a venue's settlement delay can. The budget, the sum of the Target Balances, is a reported figure. A crypto bot walks its own timeframe and 1d, and any other bot walks 1d, 1w and 1M. `src/simulator/portfolio_battery.py, in bot_timeframes`, `src/simulator/back_test.py, in apply_fold`

The tab saves the three fleets, their wires and the mode in force to the sim fleet file when the fleet changes. Run Portfolio's end figures wait for the next such save. Each bot loads idle at the next launch (`src/simulator/fleet_source.py, in loaded_idle`).

The fleet and run buttons

With no fleet held, the Get Started card offers the mode's two ways in. With a fleet held, a row above the exchange tabs offers Clear Fleet, the two ways in and Start Run (`src/gui/simulator/sim_trading_tab_surface.py, in way_in_buttons`).

Button	What it does and what it reads
Import Live Fleet (Validation, Back Test)	Copies the saved live bots of one exchange into this mode's fleet, under their own ids, each idle. It reads the live fleet file and writes nothing there. With two or more exchanges saved, a chooser asks which. A sealed fleet reads as holding no bot. `src/simulator/fleet_source.py, in FleetSource.import_live_fleet`
Generate From YTD (Validation)	Holds one idle bot per traded pair in one exchange's YTD trade files, on the bot config's built-in values with phantoms off. Its Target Balance is the units bought less the units sold, at the last fill's price. A pair that sold as much as it bought holds no Target Balance, and one line counts them. A pair with a listed file missing is not held, and one line names the file. `src/simulator/fleet_source.py, in FleetSource.generate_from_ytd`, `src/simulator/fleet_source.py, in ytd_record`
Create New Bots (Back Test)	Opens the Bot Wizard. `src/gui/simulator/sim_trading_tab.py, in SimTradingTab._create_bot`
Run Portfolio (Portfolio Battery)	Opens a chooser of one portfolio and one span, then walks that portfolio. The held fleet walks when it holds a Scrumming bot with a Target Balance for each symbol, on that symbol's venue. Otherwise the press generates one bot per symbol, at the portfolio's recorded size, else at $500 a symbol shared by its mix. Either way, a held record outside the run leaves the fleet. `src/simulator/portfolio_battery.py, in plan_run`, `src/simulator/fleet_source.py, in FleetSource.hold_battery_fleet`
Run Every Portfolio (Portfolio Battery)	Opens the chooser with the span alone, and the press empties the held fleet. The run holds one portfolio's bots, walks them, writes their report and clears them, then takes the next. `src/gui/simulator/sim_trading_tab.py, in SimTradingTab._run_battery`, `src/simulator/portfolio_battery.py, in run_battery`
Clear Fleet	Asks Yes or No. Yes empties this mode's fleet and its wires, and the other two fleets stay. `src/gui/simulator/sim_trading_tab.py, in SimTradingTab._clear_fleet`
Start Run	Runs the mode. Portfolio Battery opens Run Portfolio's chooser. Validation and Back Test cover the Scrumming bots of one venue. The Qt build takes the venue on the Crypto tab bar, whichever layer is on show, and with none seated there it writes one line and starts nothing. The React build takes the last venue pressed, else the first seated. `src/gui/simulator/sim_trading_tab.py, in SimTradingTab._start_run_pressed`, `src/gui/simulator/sim_react_trading_tab.py, in SimTradingTabReact._current_venue_id`
Push to Sim, on the Inspector tab	Holds one bot per pushed market in the Back Test fleet, registers the pushed wires, and shows Back Test. A market with no Stone Tablet, an unusable target or a wire that cannot be delivered refuses the whole push, and one line says why. `src/gui/simulator/sim_trading_tab.py, in SimTradingTab.push_to_back_test`

While a Validation or Back Test run is in flight, a mode press, Clear Fleet, Start Run and the command bar each write one line and do nothing, except Stop on a bot of that run (`src/gui/simulator/sim_trading_tab_surface.py, in run_in_flight_line`). While a Portfolio Battery walks, a mode press, Clear Fleet and Start Run do the same.

The choosers

Row	What it does and what it reads
Exchange (the exchanges the live fleet file or the YTD manifest names, first)	Picks the exchange for Import Live Fleet or Generate From YTD. It shows with two or more to pick from. `src/gui/simulator/sim_exchange_choice.py, in SimExchangeChoiceDialog`
Portfolio (35 portfolios, first by name)	Picks the portfolio, with its symbol count and description. Run Every Portfolio draws no such row. `src/gui/simulator/sim_exchange_choice.py, in SimPortfolioChoiceDialog`, `src/simulator/portfolios.py, in PORTFOLIOS`
Span (All and seven periods, All)	Picks the candles a walk covers. `src/simulator/portfolio_battery.py, in SPANS`
Cost line	States what the choice would fetch before OK: candles, calls and size, or that nothing needs fetching. `src/simulator/portfolio_battery.py, in cost_line`
OK, Cancel	OK takes the pick. Cancel writes one line and moves nothing. `src/gui/simulator/sim_trading_tab.py, in SimTradingTab._import_live_fleet`, `src/gui/simulator/sim_trading_tab.py, in SimTradingTab._run_battery`

The Get Started card

Part	What it does and what it reads
Get Started tab	Shows on a class layer that seats no venue. The title reads No Crypto Fleet Loaded and the hint Load a Crypto fleet to begin a run, each naming the layer's class. See Live's row. `src/gui/simulator/sim_trading_tab_surface.py, in placeholder_title_text`
Validation, Back Test, Portfolio Battery (the saved mode, else Validation)	The venue page's three mode buttons, so an empty mode can be left. `src/gui/simulator/sim_trading_tab_surface.py, in placeholder_mode_buttons`
Clear Fleet and the two ways in	The buttons of the row above the exchange tabs, at the card's size. Clear Fleet shows while this mode holds a bot. `src/gui/simulator/sim_trading_tab_surface.py, in placeholder_way_in_buttons`

The venue page

Each venue sub-tab holds one exchange's page. Live's Privacy Mode, news line, Add Exchange button and DATA POOL STOPPED banner are not on this tab.

`yahoo` serves no class, so it seats nothing. Portfolio Battery files a non-crypto bot there unless another venue alone holds its tablet. Such a bot walks with no sub-tab and no row, so a run of such bots alone shows no Stop (`src/gui/simulator/sim_trading_tab.py, in SimTradingTab.add_exchange_tab`, `src/simulator/portfolio_battery.py, in plan_venue`).

Control	What it does and what it reads
Venue sub-tabs (the exchanges the fleet names, the first seated)	A press shows that exchange, in both builds, and on the React page that can be an empty page, as the asset class group says. The Qt build seats a venue on one layer: Stock for a broker in `EQUITY_VENUES`, Crypto for the rest. The React build seats it on each layer it serves, so Coinbase draws on all six. See Live's row. `src/gui/main_tabs/asset_class_surface.py, in EQUITY_VENUES`, `src/gui/main_tabs/trading_tab_surface.py, in layer_exchanges`

Control	What it does and what it reads
Validation, Back Test, Portfolio Battery (the saved mode, else Validation)	Pick the run mode. The lit button names the mode in force. `src/gui/simulator/sim_exchange_tab_surface.py, in mode_style`
+ New Bot	Opens the Bot Wizard in any mode, in both builds. Finish holds one idle record in this mode's fleet. Neither of Live's refusals applies. `src/gui/simulator/sim_trading_tab.py, in SimTradingTab._create_bot`, `src/simulator/fleet_source.py, in FleetSource.create`
Data pool row	Holds nothing, and keeps the height of Live's line. `src/gui/simulator/sim_exchange_tab_surface.py, in DATA_POOL_ROW_TEXT`
Splitter handles	A drag shares the space between two panes, in both builds. `src/gui/simulator/sim_trading_tab.py, in SimTradingTab._build`, `src/gui/web/sim_trading_tab.js, in Splitter`

The asset class group on this tab

A press moves the tab to that class's layer and keeps that class's rows (`src/gui/simulator/sim_trading_tab.py, in SimTradingTab.set_asset_class`). While the class keeps no row, the tab draws a note that reads No Crypto simulated bot to show, with that class's name (`src/gui/main_tabs/class_filter_tab.py, in ClassFilterTabMixin._draw_class_notes`).

In the Qt build the kept rows go to the venue's own page, on the Crypto or Stock layer. A Stock pair on a Crypto-layer venue lands on a page that is not on show. The Commodities, Forex, Indices and Futures / Perps layers seat no venue, so they draw no row (`src/gui/simulator/sim_trading_tab.py, in SimTradingTab.refresh_bots`).

In the React build each layer shows one venue, and a venue's page fills on the first layer showing it. A later layer showing that venue has an empty page (`src/gui/web/sim_trading_tab.js, in renderExchangePane`).

A row carries no class of its own, so its class comes from the venue. The venue's market recording names the pair, or the venue serves one class. A pair on a venue that serves several classes, with no recording of the pair, draws no row. The bot stays in the fleet, and a Scrumming bot stays in the panel's selector. An Extractor's symbol is `*/` and its base, which a recording does not name, so an Extractor on such a venue draws no row (`src/gui/main_tabs/class_filter_surface.py, in bot_class`, `src/gui/main_tabs/asset_class_surface.py, in symbol_class`).

The bot tables

Both tables are Live's. The Scrumming table is Live's own class with three readings replaced (`src/gui/simulator/sim_bot_status_table.py, in SimBotStatusTable`).

Scrumming Bots column	What differs from Live
Row press, column sort	A row press that changes the panel's bot also moves the Tablet chooser to that bot's tablet. On the React page the sort mark sits at the top of the label, where Live's sits at the bottom, and a scroll leaves the highlight on a row out of view, where Live's drops it. See The bot tables. `src/gui/simulator/sim_trading_tab.py, in SimTradingTab._follow_bot_tablet`, `src/gui/web/sim_bot_status_table.js, in SortMark`

Scrumming Bots column	What differs from Live
Round header dots	Nothing. See The bot tables. `src/gui/main_tabs/bot_status_table_surface.py, in BotStatusTableModel.on_privacy_toggled`
Asset, Symbol	The Qt build opens the site, as Live's rows say. On the React page the press opens nothing. `src/gui/main_tabs/bot_status_table_surface.py, in BotStatusTableModel.on_cell_clicked`, `src/gui/simulator/sim_react_trading_tab.py, in SimVenue.answer`
Trades, Target	Nothing. See The bot tables. `src/gui/simulator/sim_bot_status_table.py, in SimBotStatusTable`
Current Position Value	Held units at the row's own last price, which a walk writes. No age check applies. See The bot tables. `src/gui/simulator/sim_bot_status_table.py, in SimBotStatusTable._display_price`
Target A15, Target A14	The target at the rate of the A15 or A14 row in this table, with no 24-hour drift. It reads pending with no such row or with no price on that row, and it never reads Live's unlisted dash. See The bot tables. `src/gui/simulator/sim_bot_status_table_surface.py, in sim_target_denom_cell`
Ammo	Live's cell, over the row's own last price. See The bot tables. `src/gui/main_tabs/table_cells_surface.py, in TableCellsModel.ammo_cell`
Fire	A press on a running or paused bot writes one refusal line on the Activity Log and moves nothing. A stored record arms nothing, so Live's solid, dashed and ceiling looks do not draw. See The Fire button. `src/gui/simulator/sim_trading_tab.py, in SimTradingTab._on_bot_fire`, `src/simulator/fleet_source.py, in row_status`
Detail button	Opens the Bot Settings window. See The bot tables. `src/gui/simulator/sim_trading_tab.py, in SimTradingTab._on_bot_detail`

Extractor Bots column	What differs from Live
Bot ID, Symbol, Mode, Trades, Pool, Liquid, Fire	Nothing, and a run walks no Extractor. See The bot tables. `src/gui/simulator/sim_extractor_bot_table.py, in SimExtractorBotTable`, `src/gui/simulator/sim_trading_tab.py, in SimTradingTab._run_bots`
Row press	The Qt build selects the row for the command bar, as Live's does. On the React page the press reaches nothing. `src/gui/web/sim_extractor_bot_table.js, in BotRow`
Detail button	Opens the Bot Settings window and selects the row for the command bar, in both builds. See The bot tables. `src/gui/simulator/sim_extractor_bot_table.py, in SimExtractorBotTable._on_detail`, `src/gui/web/sim_extractor_bot_table.js, in ActionButton`

The command bar

Each command acts on the selected row, as on The command bar. With no row selected it logs "Select a bot first." This bar has no SHIFT form, connects to no venue and walks nothing (`src/gui/simulator/sim_trading_tab.py, in SimTradingTab._on_bot_command`).

On the React page the bar sends its press to the venue the page drew last, which is the last layer's venue on show. With seated venues that serve different layers, that can be a venue other than the one on show (`src/gui/web/sim_exchange_tab.js, in sendCommand`, `src/gui/web/sim_trading_tab.js, in mountExchanges`). After an Extractor's Detail press, the React bar goes on acting on that Extractor, even after a Scrumming row press (`src/gui/simulator/sim_react_trading_tab.py, in SimVenue.answer`, `src/gui/main_tabs/exchange_tab_surface.py, in ExchangeTabModel.cmd`).

Button	What differs from Live
Start	Moves the record to running, which lets the panel read its tablet. See Live's row. `src/simulator/sim_bot_manager.py, in SimBotManager.start`
Pause	Moves the record to paused. See Live's row. `src/simulator/sim_bot_manager.py, in SimBotManager.pause`
Stop	Moves the record to stopped. On a bot of the run in flight, it ends the run, a walk at the bar reached and a Validation before its next trade, and the report reads as stopped. See Live's row. `src/gui/simulator/sim_trading_tab.py, in SimTradingTab._stop_run`, `src/gui/simulator/sim_trading_tab.py, in SimTradingTab._stop_battery`
Restart	Moves the record to stopped, then to running. See Live's row. `src/simulator/sim_bot_manager.py, in SimBotManager.restart`
Delete	Asks Yes or No, as Live's row does. Yes drops the record from this mode's fleet. `src/simulator/sim_bot_manager.py, in SimBotManager.unregister`

The Indicator Voting Panel on this tab

The panel is a copy of Live's, and fed one reading the two fill the same table cells (`src/gui/simulator/sim_indicator_panel.py, in SimIndicatorVotingPanel`).

Control	What differs from Live
Flip button: Replay, Indicators	Not on Live's panel. Replay shows the replay layer and Indicators brings the panel back. It sits first in the header on both layers. `src/gui/simulator/sim_trading_tab.py, in SimTradingTab.flip_layer`
Bot selector (the fleet's Scrumming bots, the first)	Lists the Scrumming bots of the mode's fleet. A pick also moves the Tablet chooser. In the Qt build it also selects that bot's table row, and on the React page it does not. No Previous or Next button draws, and a Bot: label stands before it. See Live's panel. `src/gui/simulator/sim_indicator_panel.py, in SimIndicatorVotingPanel.update_bot_list`
Votes	Read the last 100 candles of a tablet at the bot's own timeframe, while the bot is running or paused, at the weights your settings hold now. The repository's tablets are 5m, so a bot at the wizard's 1h start value reads none. `src/gui/simulator/sim_trading_tab_surface.py, in ivp_feed`
TF column, Comp	A phantom's row needs a tablet at the phantom's timeframe. With 5m tablets the bot's own row draws alone, and Comp reads no phantom. See Live's panel. `src/gui/simulator/sim_trading_tab_surface.py, in ivp_feed`
Bars	In the Qt build each pillar fills the panel's height and takes its direction's colour, where Live's takes its height and colour from its own reading. On the React page each pillar draws as an outline with no colour. See Live's panel. `src/gui/simulator/sim_indicator_panel.py, in CollatedPillarsWidget`, `src/gui/web/sim_indicator_panel.js, in Pillar`
Staleness label	Names the age of the votes, the tablet's last day and the weights note. With no votes, the React page names the cause and the Qt build hides the label. See Live's panel. `src/gui/simulator/sim_indicator_panel.py, in SimIndicatorVotingPanel.show_no_data`
Rate strip	A15 and A14 rates from the fleet's own rows, and a dash with no such row. See Live's panel. `src/gui/simulator/sim_trading_tab_surface.py, in rate_snapshot`

Control	What differs from Live
Privacy dot	The Qt build masks the selector text as Live's does, and its dot shows the state by colour where Live's shows a glyph. On the React page the press reaches nothing. See Live's panel. `src/gui/simulator/sim_indicator_panel.py, in SimIndicatorVotingPanel._apply_privacy_mask`, `src/gui/web/sim_indicator_panel.js, in renderPanel`
BB, VTX, MACD, SRsi, Ichi, Vol, Sling, ADX, STrd, ZSc, KER, RSI, Net, Conf, Locks line	Nothing. See Live's panel. `src/gui/simulator/sim_indicator_panel.py, in SimIndicatorVotingPanel`

The replay layer

The flip button's Replay face swaps the panel for this layer.

Control	What it does and what it reads
Tablet chooser (the tablets the mode reads, the selected bot's, else the newest)	Lists every tablet the mode reads, then each held market with no tablet, marked no tablet. A bot pick moves it to that bot's tablet. After a run it takes the tablet of the bot with the run's last fill. With no such bot it takes the selected bot's tablet, else the newest. `src/gui/main_tabs/simulator_tab_surface.py, in tablet_choices`, `src/gui/main_tabs/simulator_tab_surface.py, in default_tablet_key`
Retrieve Tablet, Update Tablet	Reads Retrieve for a market with no tablet and Update for one with a tablet. A press fetches the missing 5m candles from Coinbase's public candle endpoint and appends them to the tablet. Any other venue fetches nothing, and the line reads refused. Each call draws a block on the API Interaction Log, and the first refusal ends the fetch. With no item chosen, the Qt button greys, and on the React page a press writes one line. `src/simulator/back_test.py, in adapter_for`, `src/simulator/tablet_retrieval.py, in retrieve`
Figures line	Names the shown bot, its scrum and fold pairs, their mean and median distance and, after a Portfolio Battery, its improvement over HODL. After a Validation the pairs read 0 and the distances a dash. It reads no run while no fill of that bot matches the tablet's timeframe. After a walk a 1h bot over a 5m tablet reads no run, and so does the layer after Run Every Portfolio, which holds no bot. `src/gui/main_tabs/simulator_tab_surface.py, in fills_for`, `src/gui/main_tabs/simulator_tab_surface.py, in trade_pairs`, `src/gui/main_tabs/simulator_tab_surface.py, in improvement_for`
VWAP window	Draws the close and the VWAP over the tablet's last 100 candles. `src/gui/main_tabs/simulator_tab_surface.py, in vwap_payload`
Stone Tablet Playback window	Draws those candles and marks the shown bot's fills: the bot with the run's last fill, else the selected bot, else the lowest id on that market. A fill marks when its timeframe matches the tablet's and its time is a candle's stamp in the window. A Validation fill keeps its own time, so it marks on an exact stamp alone. `src/gui/main_tabs/simulator_tab_surface.py, in mark_shapes`, `src/simulator/validation.py, in rerun_trade`

The two log panes

Control	What differs from Live
Activity Log	The same pane as Live's, with no gate lights. A walk's trade line carries its candle's time and a Validation trade line its fill's time, and each run adds its own lines. `src/gui/simulator/sim_trading_tab.py, in SimTradingTab.log_trade`, `src/gui/simulator/sim_status_log.py, in SimStatusLog.log`
Pause Console, Resume Console	The same hold and buffer as Live's, and both presses work in both builds. On the React page a trade line held during the pause stops Resume Console at that line: it and the held lines after it are lost. `src/gui/simulator/sim_status_log.py, in SimStatusLogModel.log_at`, `src/gui/main_tabs/status_log_surface.py, in StatusLogModel.resume`
API Interaction Log	The same block as Live's, and it takes two entries: one per Stone Tablet call and one per Generate From YTD read. Any other entry is refused. `src/simulator/sim_api_log.py, in SimApiLog.record`
Pause API Log, Resume API Log	Both presses work in both builds. The Qt build holds as Live's does, and a full buffer drops the oldest. On the React page a full buffer drops the newest. `src/gui/simulator/sim_trading_tab.py, in SimTradingTab._on_api_event`, `src/gui/main_tabs/trading_tab_surface.py, in ApiPauseBuffer.hold`

The header strip with Sim in front

The strip reads the fleet of the mode in force and repaints as Live's strip does (`src/gui/main_window.py, in MainWindow._refresh_header_strip`). No walk stamps a record with a venue reading, so the cells that need one read a dash until a held record brings its stamp from Live.

Cell	What it reads with Sim in front
SPENDABLE	Under Validation and Portfolio Battery, the sum of the held Target Balances. Under Back Test, the highest cash a held record carries, and a walk writes none. A dash until cash or positions read above zero, as on Live. `src/gui/main_tabs/simulator_tab_surface.py, in fleet_aggregate`, `src/gui/main_tabs/header_strip_surface.py, in profits_payload`
P/L	The sum of the held records' unrealised figures, and a dash until a held record carries a venue stamp. `src/gui/main_tabs/header_strip_surface.py, in unrealised_amount`
LOCKED	Each bot's units at its own price, else its stored position value, summed. A dash until cash or positions read above zero, as on Live. `src/simulator/fleet_source.py, in aggregate_stats`, `src/gui/main_tabs/header_strip_surface.py, in profits_payload`
MATURE	A dash until a held record carries a venue stamp. `src/simulator/fleet_source.py, in aggregate_stats`
EXCH	Venues seated on the Crypto layer. `src/gui/simulator/sim_trading_tab.py, in SimTradingTab.exchange_count`
AMMO	A dash in both builds, because the strip reads no fleet rows from this tab. `src/gui/main_window.py, in MainWindow._fleet_rows`
SCRUMMED, FOLDED	The fleet's sum of this year's dollars, and the sum of lifetime dollars while that reads zero. A walk writes lifetime dollars. `src/simulator/fleet_source.py, in aggregate_stats`
ACCUMULATED	A dash. The sim fleet publishes no such figure. `src/gui/main_tabs/header_strip_surface.py, in COUNTER_CARDS`
TRADES	The sum over the held records. `src/simulator/fleet_source.py, in aggregate_stats`
ERRORS	The sum over the held records. A press opens the Error Log over the live fleet and not this one. `src/simulator/fleet_source.py, in aggregate_stats`, `src/gui/main_window.py, in MainWindow._build_error_log_dialog`

Cell	What it reads with Sim in front
BOTS	Held bots reading running. `src/simulator/fleet_source.py, in aggregate_stats`
Round cell dots	Live's dots. See Live's strip. `src/gui/main_tabs/header_strip.py, in HeaderStripMixin._build_header_strip`

How to read a run

The gate-latch criterion

Validation's criterion is that the rerun's gate lights and both armed flags read as the recorded row's did. Profit and a trade count play no part (`src/simulator/validation.py, in RowComparison.latches_identically`). The ADX, Efficiency Ratio and Z-score gates carry no light, and a rerun passes them (`src/simulator/validation.py, in rerun_context`). The run's verdict lines give two counts, the reruns that latched identically and the gate lights that agreed, and no word of pass or fail. With no row compared, one line says so (`src/simulator/validation.py, in verdict_lines`).

The recorded row is the one nearest the trade within five minutes, so it can sit on another candle (`src/simulator/validation.py, in nearest_gate_row`).

A bot imported from Live matches gate rows by its own id. Any other bot matches on exchange and symbol (`src/simulator/validation.py, in _validate`).

Which markets a run can walk

Each bot trades under a unit rule, fractional or whole, read from one table by class and venue. A bot whose class and venue hold no row walks nothing. Back Test names it on the Activity Log. Portfolio Battery names it in `diagnostics.log` and counts its asset on one Activity Log line (`src/simulator/back_test.py, in cited_rule_for`, `src/trading/scrumming/sizing.py, in CITED_UNIT_RULES`).

Where the market recording holds the venue's size step for the pair, a walk floors each order onto that step, and a recorded minimum refuses an order under it. A whole-unit rule overrides a fractional step, and a recorded minimum with no step leaves the sizing to the unit rule. With neither recorded, the unit rule sizes the order (`src/trading/scrumming/sizing.py, in sized_order`). A Back Test writes one line for each bot it walks, naming the recorded figures where either is held and the unit rule otherwise (`src/simulator/back_test.py, in rule_source_line`). A walk needs at least 30 bars, and a bot with no bar at its timeframe walks nothing (`src/simulator/back_test.py, in walk`).

What a run writes

Output	What it holds
Activity Log	One line per fill a walk makes or a Validation reruns, and the run's start and end lines. A walk adds progress lines, and a Validation adds none. `src/gui/simulator/sim_trading_tab_surface.py, in trade_line`
The bot rows	A walk writes each bot's figures into its record, once per two seconds on screen. Validation and Back Test leave their bots stopped. Under Portfolio Battery a walk's end stops its bot, a bot that walked nothing keeps its state, and Run Every Portfolio clears each fleet. `src/gui/simulator/sim_trading_tab.py, in SimTradingTab._take_bot_stats`, `src/gui/simulator/sim_trading_tab.py, in SimTradingTab._battery_portfolio_finished`
Trade parity report	One Markdown file and one JSON file under `reports/simulator` in the log folder, named for the mode, the subject, the span and the time. The Activity Log names it. Run Every Portfolio writes one pair per portfolio and one summary pair. A pass that raises writes a partial report where it can, and the Activity Log gets the failed line in place of the report's name. `src/simulator/parity_report.py, in report_name`, `src/simulator/parity_report.py, in write_partial`
The sim bucket	`trade.log`, `voting.log`, `diagnostics.log` and one `gate/<exchange>/<sector>/gate.log` under `sim` in the log folder. `src/simulator/sim_bus.py, in sim_log_paths`
The sim signal file	`signals/session.jsonl` under `sim`. A run's worker thread sends its signals here, the Activity Log names the path at the start, and the main signal file gains one row at the end. `src/simulator/sim_bus.py, in routed_run`
The sim fleet file	`sim_fleet.json` under `sim`, with each mode's fleet, wires and wire ledgers. `src/gui/simulator/sim_trading_tab.py, in SimTradingTab._hold_run_wires`, `src/simulator/fleet_source.py, in FleetSource.save`

Where the data comes from

Source	What it is
Stone Tablets	Stored 5m candles by asset, year and exchange, tracked in the repository's `stone_tablets` folder. A frozen build reads `.acervator/stone_tablets` in your home folder instead. Validation and Back Test read these. `src/trading/stone_tablets/storage.py, in _tablet_root`
RA-StoneTablets	The portfolios' own history in `.acervator_ra_tablets` in your home folder: 5m for crypto and 1d for the rest. Portfolio Battery reads these, and fetches missing crypto candles first. `src/trading/stone_tablets/ra_paths.py, in RA_STONE_TABLETS_DIR`, `src/simulator/portfolio_battery.py, in retrieve_tablets`
The recorded gate log	Live's gate rows, read only. Validation compares against them. `src/simulator/gate_log_source.py, in GateLogSource`
Saved live fleet	The live fleet file, read only, for Import Live Fleet. `src/simulator/fleet_source.py, in FleetSource.stored_records`

The YTD trade files

One file holds one exchange, one symbol and one year, and a manifest beside the files lists them. Each row holds a fill's id, time, side, amount, price, cost, fee and fee currency (`src/exchange/ytd_trade_store.py, in YtdTrade`).

A gap record marks a year the export covers with no trade for a symbol, and the covered days before that symbol's first row and after its last. A quiet stretch between two trades writes no gap (`src/exchange/ytd_csv_import.py, in _store_year`, `src/exchange/ytd_csv_import.py, in _edge_gaps`).

The import reads an exchange's export through that exchange's column map. A map exists for Coinbase, and an exchange with no map is refused by name (`src/exchange/ytd_csv_import.py, in export_map_for`). Nothing in the running program calls the import (`src/exchange/ytd_csv_import.py, in import_ytd_csv`). The Sim tab reads the files through one reader, which writes no trade file (`src/simulator/ytd_trade_source.py, in YtdTradeSource`).

How to use the Bot Wizard on this tab

The wizard's title reads Create Auto Trader. The Qt build draws `SimBotCreationWizard` and the React build draws `SimBotWizardReactDialog`, through `src/gui/variant_surface.py, in surface_class`. Finish holds a record, and the bot stays idle. Its walk matches Live's walk, apart from the rows below.

What differs from Live's wizard

Part	What differs from Live
Trading Mode page: Accumulation Trading (Scrumming), Base Currency Extractor (Multi-Target)	The Qt build offers these two on each class. The React build offers Crypto Scrumming and the same Extractor on each class. See Live's page. `src/gui/simulator/sim_bot_wizard.py, in SimModeSelectionPage`, `src/gui/simulator/sim_react_bot_wizard.py, in SimBotWizardReactDialog.model`
Select Asset Pair page: Exchange	Lists the venues seated on the Crypto layer, then each venue a tablet names. See Live's row. `src/gui/simulator/sim_bot_wizard_surface.py, in wizard_exchanges`, `src/gui/simulator/sim_trading_tab.py, in SimTradingTab._create_bot`
Select Asset Pair page: Base Currency	Both builds list the seven declared legs, where Live adds any other leg the venue quotes. See Live's row. `src/gui/simulator/sim_bot_wizard.py, in SimAssetSelectionPage`, `src/gui/web/sim_bot_wizard.js, in comboItems`
Select Asset Pair page: Target Asset	Lists each tablet's asset under each base currency, and no venue is called. A seated venue with no tablet lists nothing in the Qt build and its recorded crypto markets in the React build. A tablet row carries no volume, so the list keeps the tablets' order with no volume label. The Qt list marks no market as untradeable, and with no pair it reads No pairs found. See Live's row. `src/gui/simulator/sim_bot_wizard_surface.py, in tablet_markets`, `src/gui/main_tabs/bot_wizard_surface.py, in BotWizardModel.on_exchange_changed`
Select Asset Pair page: Status line	The Qt line counts the pairs, where Live's also counts those that cannot carry a bot. See Live's row. `src/gui/simulator/sim_bot_wizard.py, in SimAssetSelectionPage._filter_assets`
Information button	Nothing. See Live's row. `src/gui/simulator/sim_bot_wizard.py, in SimAssetSelectionPage._show_info`
Extractor Pool page: Exchange, Pool Base Currency, Alternate pair list	The same venue list. The alternate pair list reads the tablet's markets. In the React build a seated venue with no tablet lists its recorded crypto markets when the asset page loaded them, else none. See Extractor rows. `src/gui/simulator/sim_bot_wizard.py, in SimExtractorPoolPage`, `src/gui/main_tabs/bot_wizard_surface.py, in BotWizardModel.on_pool_exchange_changed`
Extractor Pool page: Status line	Counts the alternate pairs. `src/gui/simulator/sim_bot_wizard.py, in SimExtractorPoolPage`
Whole-Unit Market group, Contract Expiry group: Open Position At, Expiry Close, Expiry Lead Mode, Lead Fraction, Lead Days, Expiry Horizon	The Qt build draws neither group. The React build draws each on Live's rule, and Finish stores their values, which no walk reads. See Live's rows. `src/gui/simulator/sim_bot_wizard.py, in SimTradingParamsPage`, `src/gui/main_tabs/bot_wizard_surface.py, in BotWizardModel.set_market_groups`
Unit lines	Neither build draws them. See Live's row. `src/gui/simulator/sim_bot_wizard.py, in SimTradingParamsPage`
Finish on a market no bot can trade	The Qt build takes the market. The React build keeps the wizard open, and its line shows the bare word `market_untradeable`. See Live's row. `src/gui/simulator/sim_bot_wizard.py, in SimBotCreationWizard`, `src/gui/main_tabs/bot_wizard_surface.py, in BotWizardModel.finish`
Phantom Bots page, Next	The Qt build asks no API load box. `src/gui/simulator/sim_bot_wizard.py, in SimPhantomConfigPage.validatePage`
Pre-flight check	None runs. See Live's row. `src/gui/simulator/sim_trading_tab.py, in SimTradingTab._create_bot`

Part	What differs from Live
Parent check	Live's rule, over this mode's fleet. See Live's row. `src/gui/simulator/sim_bot_wizard_surface.py, in extractor_parent_refusal`

What a run reads from the wizard's rows

Each row below holds Live's range and start value, and its link goes to Live's rows. A walk reads a bot's record through a shorter list than a live bot does, so most rows store a value that no walk reads (`src/simulator/fleet_source.py, in SimBot`, `src/simulator/back_test.py, in tape_context`).

Rows	In a Back Test walk
Target Balance, TA Timeframe, Opposing Trade Interval, Trading Fee %. See Scrumming Settings rows	Read. The interval sets the Target Delta a scrum needs and, with the fee, the buy-back distance, which leaves out Band Travel. The opposing-distance gates pass on each bar, so a scrum has no such distance. The fee is the box's figure, and a box of 0 reads as 0.6. `src/simulator/back_test.py, in apply_fold`, `src/simulator/back_test.py, in tape_context`
Landing Strip Candles, BB Tolerance. See Scrumming Settings rows	Read for the band reading. `src/simulator/validation.py, in bb_reading`
Max Target Growth %. See Scrumming Settings rows	Read for the cycle cap. `src/simulator/back_test.py, in grow_target`
Detect Threshold, BB Midline Gate. See Advanced Scrumming rows	Read. `src/simulator/back_test.py, in tape_context`
SCRUM requires bullish TA, SCRUM holds in sustained uptrend, SCRUM defers to higher-TF bullish, FOLD requires bearish TA, FOLD defers to higher-TF bearish. See Strategy Gate Flags rows	Read. `src/simulator/back_test.py, in tape_context`
Enable Position Ceiling, Ceiling Multiple. See Risk Controls rows	Read for the fold taper. `src/simulator/back_test.py, in fold_taper`
Enable Phantom Bots, Timeframe boxes. See Phantom Bots rows	Read. A walk runs one phantom at most, on the highest stored timeframe, as a live bot does, or on the next length above its own with none stored. The phantom is the walk's own candles rolled up to that length. A length the walk cannot roll up to, such as 1w or one that is not a whole multiple, builds no phantom, and the gates read no bias from it. `src/simulator/back_test.py, in phantom_timeframes_for`, `src/simulator/back_test.py, in phantom_tapes`
Order Visibility, Aggressive Trading, Stack Mode, Split Distance, Tranche Count, Spacing. See Trading Parameters rows	Stored and not read. A walk sells in one piece, with no ladder. `src/simulator/back_test.py, in apply_scrum`
Max Entry Price, Min Entry Price, Scrum Fold Ratio. See Scrumming Settings rows	Stored and not read. A sale queues all its proceeds for buy-back. `src/simulator/back_test.py, in apply_scrum`

Rows	In a Back Test walk
Fire Threshold, Read Rate, Band Travel, BB Bullseye Check. See Advanced Scrumming rows	Stored and not read. A walk evaluates each bar from the 30th on. `src/simulator/back_test.py, in evaluations_expected`
Hedge Rebalance Active, Hedge Balance. See Hedge Rebalance rows	Stored and not read. `src/simulator/fleet_source.py, in SimBot`
Soft CB Threshold, Hard CB Threshold, Soft CB Cooldown. See Circuit Breakers rows	Stored and not read. Neither breaker blocks a gate in a walk. `src/simulator/back_test.py, in tape_context`
Max Cartridge Size, Smart Cartridge, Smart Ceiling. See Circuit Breakers rows	Stored and not read. `src/simulator/fleet_source.py, in SimBot`
Enable Detonation, Detonation TF, Min Confidence. See Risk Controls rows	No walk reads them. While Enable Detonation is on, the Fire button's mouse-over can name them. `src/simulator/fleet_source.py, in row_status`, `src/gui/main_tabs/bot_status_table_surface.py, in risk_lines`
Candles to lock. See Phantom Bots rows	Stored and not read. `src/simulator/sim_bot_view.py, in PhantomLock`
Chunk size (USD), Artillery size (USD), Watch list top-N, Watch list refresh, Exit %. See Extractor rows	Stored, and no run walks an Extractor. `src/gui/simulator/sim_trading_tab.py, in SimTradingTab._run_bots`

A Portfolio Battery walk reads the same rows of a held bot. A bot it generates walks on built-in values, and a non-crypto bot walks 1d, 1w and 1M whatever its TA Timeframe (`src/simulator/portfolio_battery.py, in generated_bot`, `src/simulator/portfolio_battery.py, in bot_timeframes`).

How to use the Bot Settings window on this tab

The Detail button opens this window over one stored record. A Scrumming bot's window carries Status, Settings, Fold Tranches, Stack Tranches, Bot Swarm, Market Inspector and Phantom Bots, and an Extractor's carries Status, Settings and Positions Held, as Live's window does (`src/gui/simulator/sim_bot_detail.py, in SimBotDetailDialog`). The window shows the record and changes nothing: each write it tries is refused (`src/simulator/sim_bot_view.py, in SimBotView`).

Control	What it does and what it reads
Apply Changes	Greys until a row moves. A press refuses each pending change, and the record keeps its values. See Live's row. `src/gui/simulator/sim_bot_detail.py, in SimBotDetailDialog._apply_changes`
Pending line	Names the fields waiting for Apply, then reads Refused, with the count. See Live's row. `src/gui/simulator/sim_bot_detail.py, in SimBotDetailDialog._mark_changed`
Close	Closes the window and drops unapplied edits, as Live's does. `src/gui/simulator/sim_bot_detail.py, in SimBotDetailDialog._setup_ui`

Control	What it does and what it reads
Prev, Next	Walk the bots of the same exchange and keep the tab, where Live's walk every bot. Ctrl+Left and Ctrl+Right do the same, Qt build only. On the React page Fold Tranches, Stack Tranches and Phantom Bots open empty after Prev or Next, until a tab press redraws them. `src/gui/simulator/sim_trading_tab.py, in SimTradingTab._on_bot_detail`, `src/gui/simulator/sim_bot_detail.py, in SimBotDetailDialog._setup_ui`, `src/gui/web/sim_bot_live_settings.js, in mountPage`
Status tab	Read-only: the record's trades, open orders, price, uptime and last error. Realised P/L reads a pending dash until the record brings a venue figure from Live, and Unrealised P/L, Avg Entry, Cost Basis Total and Fees Paid draw only then. See Live's row. `src/gui/simulator/sim_bot_detail.py, in SimBotDetailDialog._create_status_tab`
Settings tab, the wizard's rows	Show the record's values, and Apply refuses a change. A stored Trading Fee % of 0 draws 0.60. On the React page the TA Timeframe box lists ten fixed lengths, and a stored length outside them draws 1h. After a refused Apply, a box keeps the typed figure until the window reopens. Neither build draws the Expiry rows. The note that changes take effect at Apply is Live's text. `src/gui/simulator/sim_settings_tab.py, in SimSettingsTabMixin`, `src/gui/main_tabs/live_settings_tab_surface.py, in timeframe_index`, `src/gui/main_tabs/live_settings_tab_surface.py, in bot_market_expires`
Live target, Standing surplus, Cycle growth budget, Over-cap tranches, Target A15, Target A14	Show the record's figures, as Live's rows do. A record with no stored anchor reads anchor $0.00, with the whole target as accrued. The two target rows read the fleet's own rates, with no 24-hour gap and no 5-second repaint. `src/gui/simulator/sim_settings_tab.py, in SimSettingsTabMixin._refresh_target_denom_rows`
Profit Folding Active	Shows the stored value. A walk reads it, and Apply refuses a change. See Live's row. `src/simulator/back_test.py, in grow_target`
Tranche Despawn Timer	Shows the stored value. No walk reads it. See Live's row. `src/simulator/fleet_source.py, in SimBot`
TA Timeframe notice	Qt build only, as on Live's Qt screens. On this tab and on the wizard, a line names the candle the box shows when the venue does not serve the one chosen. `src/gui/simulator/sim_settings_tab.py, in SimSettingsTabMixin._create_settings_tab`, `src/gui/simulator/sim_bot_wizard.py, in SimTradingParamsPage.set_exchange_id`
Hunger, Hunger Pace, Accumulation Shadow	Not drawn. No walk reads them. See Live's rows. `src/gui/simulator/sim_live_settings_tab_surface.py, in SimLiveSettingsTabModel`
Reset All Breakers	Changes nothing. The Qt button reads Reset failed for two seconds, and the React page's line under the buttons names the refusal. See Live's row. `src/gui/simulator/sim_react_bot_detail.py, in press_method`
Self-Destruct	The Qt build asks for the typed word, then a box says nothing was sold. The React page refuses at once, and the line under the buttons names the refusal. See Live's row. `src/gui/simulator/sim_settings_tab.py, in SimSettingsTabMixin._on_self_destruct_clicked`, `src/gui/simulator/sim_react_bot_detail.py, in press_method`
Fold Tranches tab	Lists the record's queued buy-backs, their counters, the despawn timer and the cycle's fold budget. Its Arbiter column reads a dash. Fire, each Clear button and Clear Wire Credits end in a refusal. See Live's row. `src/gui/simulator/sim_fold_tranches_tab.py, in SimFoldTranchesTabMixin`

Control	What it does and what it reads
Fold Tranches: Order, Filter	Qt build only. They sort and hide rows, as Live's do. `src/gui/live_settings/fold_chrome.py, in build_fold_row_controls`
Stack Tranches tab	Lists the record's rising sell levels and counters. Its Clear buttons end in a refusal, and a walk opens no level. Neither build draws Live's Shadow Tranches rows. See Live's row. `src/gui/simulator/sim_stack_tranches_tab.py, in SimStackTranchesTabMixin`
Bot Swarm tab	Reads Bot Swarm not active, because the tab hands the window no wire manager. See Live's row. `src/simulator/sim_bot_view.py, in SimBotView`
Market Inspector tab	Reads No Market Inspector scan yet, because a stored record holds no scan. See Live's row. `src/gui/simulator/sim_market_inspector_tab_surface.py, in per_bot_view`
Phantom Bots tab	Enable Phantom Bots, Timeframe and Candles to lock show the stored set. In the Qt build Apply refuses a change, and a line under Timeframe names a refused pick. On the React page these controls take no edit and Apply stays grey. Phantoms started reads NO, Dropped by the venue reads none and SCRUM lock state reads UNLOCKED, whatever the record holds. The other status rows follow the record. With phantoms on, a line reads Phantoms enabled but not yet started, whatever a run did. See Live's row. `src/gui/simulator/sim_phantom_bots_tab.py, in SimPhantomBotsTabMixin`, `src/gui/web/sim_phantom_bots_tab.js, in renderTab`
Positions Held tab	Lists an Extractor's stored positions, and each Fire is refused. Pool color, Chunk size, Chunk free and Lifetime extracted read the record. See Live's section. `src/gui/simulator/sim_positions_held_tab.py, in SimPositionsHeldTabMixin`

Paper Trader Tab

Reference. The third step of the promotion pipeline. Issue #19 built the screen. The tab row carried an empty tab until that build landed, and The screen as it stands now describes what draws there today.

The empty tab it replaced

The tab drew three lines: its name, one sentence saying it was not built, and the issue that owned it. It read no bot, no price and no trade.

The placeholder carried its empty state in a surface module of its own, and the block below is that state.

```
HEADING = "Paper Trader"
ISSUE = 19
BUILT = False
STATE_TEXT = "This tab is not built."
ISSUE_TEXT = f"Issue #{ISSUE} carries the build-out."
```

That module is gone, and the empty state above went with it. No module on disk holds `BUILT` or `STATE_TEXT` for this tab.

Two frontends drew that one view model. `EmptyTabsMixin` in `src/gui/main_tabs/empty_tabs.py` built the Qt tab, and a renderer panel registered with the Electron shell's panel host. The bridge served the model under a method name that no module declares now.

What the step is

Real time is its defining property. The Simulator replays stored candles as fast as the machine allows. Paper takes the live feed at the speed the market delivers it, and that is the whole difference between them.

Live, Paper and the Simulator differ in one thing only: where the data comes from. The trading logic stays one body of pure code that all three call. Only the stateful shells fork, which is what keeps a paper run from holding a live object.

The paper budget is twice the dollar target, so a bot has room to fold without the exercise ending on the first dip.

`src/trading/container/config.py` — `BotConfig`, the field it doubles

```
target_balance: float = 200.0  # Balance the bot trades relative to
```

Paper sat behind the Simulator, which had to reproduce the gate latches first. Nuclear Mode was to be a second gate; the operator cancelled it, so one gate remains and issue #19 built the tab after the Simulator rebuild landed.

Where the platform once offered it

Two live surfaces named the step. The first is the Swarm tab's third sub-tab, labelled Paper Swarm. Its Start button flips a flag and relabels itself. No bot is constructed, no feed is attached and no order is recorded. Its caption is an empty string now, so the sub-tab promises nothing.

`src/gui/bot_visualizer.py` — the Start handler inside `_create_paper_bot_row`

```
def _toggle():
    if bot_data["running"]:
        bot_data["running"] = False
        start_btn.setText("■ Start")
```

The second is the wing toggle on the header strip. Pressing it swaps the whole wing. The Qt window no longer names a Paper Trader in the line it writes, and the React mirror still does, so the two hosts print different text.

`src/gui/main_tabs/main_window_surface.py` — the line the React mirror writes

```
"→ STOCK WING: equity exchanges + equity Paper Trader. (Crypto wing paused.)"
```

Issues #422 and #426 both folded into issue #19, which built the tab.

The screen as it stands now

The tab is built. It sits second on the bar and carries the Trading tab's panes over the live exchange feed. `src/gui/paper/paper_trading_tab.py, in PaperTradingTab` draws them in Qt and `src/gui/web/paper_trading_tab.js` draws them in React, from one view model.

The old Paper Trader surface declared the fields both hosts read. That module is gone and no module on disk holds `DECLARED_FIELDS` for this tab. The Paper tab's surface is `src/gui/paper/paper_trading_tab_surface.py`, `PaperTradingTab` in

src/gui/paper/paper_trading_tab.py draws it in Qt, and PaperTradingTabReact in src/gui/paper/paper_react_trading_tab.py draws src/gui/web/paper_trading_tab.js in React. The names PaperTraderTabQt and src/gui/web/paper_trader_tab.js are on no file.

```
DECLARED_FIELDS = (
    "accessible_name",
    "balance",
    "built",
    "feed",
    "fleet",
    "heading",
    "indicators",
    "issue",
    "method",
    "panes",
    "privacy_button",
    "reserved_rows",
    "run",
    "skin",
    "symbol",
    "symbols",
)
```

The block above is that old declaration. src/gui/paper/paper_trading_tab_surface.py, in view_model answers the tab today.

The left pane holds the Privacy Mode button, the market selector, the two button rows and the bot list. The bot list uses the Trading tab's own columns. The right pane holds the Indicator Voting Panel, computed from the live window. An empty selector draws the empty state and invents no reading.

The old Paper Trader surface built the voting panel's empty state through the function below. That module is gone, and indicator_payload is declared once, in src/gui/main_tabs/simulator_tab_surface.py, with its only caller in that same module. Nothing on the Paper side reads it. The Paper pane is src/gui/paper/paper_indicator_panel.py in Qt and src/gui/web/paper_indicator_panel.js in React.

```
def indicator_payload(
    candles: Sequence[Any], timeframe: str, symbol: str, refusal: str
) -> dict:
    model = ivp.IndicatorPanelModel()
    if refusal:
        model.show_no_data(refusal)
        return ivp.build_payload(model)
    summary = multi_tf_summary(candles, timeframe, symbol)
    if not summary:
        model.show_no_data(NO_SYMBOL_TEXT)
        return ivp.build_payload(model)
    model.set_summary(summary, symbol)
    return ivp.build_payload(model)
```

No Paper module calls `indicator_payload`. `src/gui/paper/paper_trading_tab_surface.py, in ivp_feed` reads the panel today.

The two presses

Import Live Fleet reads the stored bot record and lists what it finds. Start Paper Run opens the run and begins ticking. The same button then reads Stop Paper Run and ends it, leaving the balances and the trades on screen.

The old Paper Trader surface built the second row's two faces with the block below. That module is gone. The two faces are declared in `src/gui/paper/paper_trading_tab_surface.py` now, as its start and stop actions and their texts.

```
{
    "name": DATA_POOL_ROW,
    "height_px": 18,
    "action": STOP_RUN_ACTION if running else START_RUN_ACTION,
    "text": STOP_RUN_TEXT if running else START_RUN_TEXT,
    "button_name": button_name(
        STOP_RUN_ACTION if running else START_RUN_ACTION
    ),
}
```

No Paper module holds `DATA_POOL_ROW`. `src/gui/paper/paper_trading_tab_surface.py, in run_buttons` carries the row's two faces today.

Downstream only

The feed reader answers six names and refuses every other one, so a send has no spelling. It calls the venue's public market endpoints, which carry no key and no signature.

`src/paper/live_feed_source.py` — the refusal

```
def __getattr__(self, name: str):
    """Refuse every name outside ``READ_NAMES``."""
    raise SendRefused(
        f"LiveFeedSource answers {READ_NAMES} and cannot {name!r}. "
        "The Paper Trader receives and asks; it sends nothing."
    )
```

One gate chain

A tick builds the same `GateContext` a live bot builds and evaluates the shipped scrum and fold chains on it. The paper side adds no gate and changes none.

`src/paper/paper_run.py` — what a tick evaluates

```
window = list(candles[-WINDOW_CANDLES:])
reading = bb_reading(window, bot)
summary = VotingEngine().compute_all(window, bot.ta_timeframe, symbol=bot.symbol)
context = tape_context(bot, balance, window, reading, summary)
armed = latch(context)
```

The gate row names the override that replaced a refusal

A tick's gate row now says which refusals an override lifted, one entry per refusal, per side. A row that passed outright and a row an override passed used to read the same: both showed armed with an empty refusal list. They read apart now, and the refusal's own text is kept beside the override's name.

`src/paper/paper_log.py` — the entry written on the Paper Trader's gate row

```
def scrum_fixture(context: Any, overrides_applied: Sequence[str] = ()) -> dict:
    """The scrum half of a gate row, with each refusal an override replaced
    under ``OVERRIDES_APPLIED_KEY``."""
```

The Paper Trader reads the same record the Simulator reads, and the key is spelled once for all three data sources. Live's gate log, the Paper Trader's gate row and the Simulator's gate event therefore record an override the same way. Which gate each override covers, and the sentence that says what the overrides are for, are on the indicators page under The two overrides.

Two overrides exist and they cover six of the twenty-four gates the two shipped chains hold. Both chains were built and each gate asked which override covers it. The control is the same ask answering both ways in one run: it named an override for these six and nothing for the other eighteen.

Override	Side	Gates it covers
`ripe_harvest_override`	scrum	midline_scrum, ta_bullish, target_fires, trend_hold
`deep_fold_override`	fold	midline_fold, ta_bearish

Nothing about which trades fire changed. The armed verdict and the refusal list are untouched, and the row simply carries one more field.

The fake balance

Each bot opens a budget of twice its dollar target: units worth the target, and the rest in cash to fund a fold. The balances live in memory and are dropped when the tab goes away.

`src/paper/fake_balance.py` — the opening

```
def opening_balance(target_usd: float, price: float) -> FakeBalance:
    whole = budget_usd(target_usd)
    held_usd = min(whole / BUDGET_MULTIPLE, whole)
    units = held_usd / float(price) if float(price) > 0.0 else 0.0
    return FakeBalance(
        units=units,
        cash_usd=whole - units * float(price),
        tranches=0,
        budget_usd=whole,
    )
```

The fleet ledger

The fake balance is one figure for the whole fleet, not one figure per bot. Five paper bots with a $200 target open Paper Spendable at $1000 and Paper Locked at $1000. Each variable holds the full fleet sum, which is twice the fleet's dollar target in total.

`src/paper/fake_balance.py` — the opening

```
def opening_ledger(bots: Sequence[Any]) -> PaperLedger:
    total = fleet_target_usd(bots)
    return PaperLedger(
        fleet_target_usd=total,
        opening_spendable_usd=total,
        opening_locked_usd=total,
    )
```

The fleet changes when a bot is added or removed. Every press of Start Paper Run rebuilds the ledger from the fleet it is handed, so both opening figures move on the next press and never inside a run. Six $200 bots open $1200 in each figure and four open $800.

Four figures are tracked. Paper Spendable is the cash the fleet holds. Paper Locked is what its units are worth at the last price the feed answered. Paper Realized Profits moves when a fold buy closes the tranche a scrum sell opened. Paper Mature Profits is the profit on positions past 200 percent growth, the figure the live wing already reads.

`src/trading/smart_wire.py` — the maturity rule both wings call

```
MATURE_GROWTH_PCT: float = 200.0
"""A position is mature once its value exceeds its cost basis by this percent."""
```

The four figures are held apart from the live ones. They live in memory on the run, they are never written into the stored bot record, and no paper figure reaches the header strip's real-money columns.

`src/paper/fake_balance.py` — the four labels the strip prints

```
FIGURE_LABELS = (
    ("spendable_usd", SPENDABLE_LABEL),
    ("locked_usd", LOCKED_LABEL),
    ("realized_profit_usd", REALIZED_LABEL),
    ("mature_profit_usd", MATURE_LABEL),
)
```

The Qt window and the React page printed one line from one view model, so the two hosts could not report different money.

The old Paper Trader surface joined the four figures into that line with the call below.

```
"text": LEDGER_SEPARATOR.join(f"{one['label']} {one['text']}" for one in cells),
```

That module is gone, and no module on disk holds `LEDGER_SEPARATOR`. Both hosts now hand the header strip one figure set, built over the held fleet and the ledger's figures (`src/paper/fleet_source.py, in strip_aggregate`).

The Paper Trader log

Every paper action is written to one file. Each line carries two halves: the gate decision that produced the action and, when one filled, the pretend trade in the shape a Coinbase year-to-date row takes. Validation reads both of those shapes already, so a paper run is checked by the reader that checks the live wing.

`src/paper/paper_log.py` — the row

```
return {
    "timestamp": iso_stamp(tick.wall_ms),
    "category": CATEGORY,
    "exchange": str(exchange_id),
    "bot_id": str(tick.bot_id),
    "data": gate_half(tick),
    "trade": trade_half(tick.filled) if tick.filled is not None else None,
    "ledger": dict(figures or {}),
}
```

The log has a home of its own. It is a sibling of the live folders and never sits inside one, so nothing a paper run writes can land in a live tree.

`src/paper/paper_paths.py` — the root

```
PAPER_ROOT: Path = Path.home() / ".acervator_paper"
```

Gate names come from one place. The row records the blocker phrases the chain produced and the labels those phrases map to, and it spells no gate name of its own.

`src/trading/gate_vocabulary.py` — the mapping the log calls

```
def gate_for_blocker(blocker: str) -> str:
    """Map one blocker string to its gate label, or "" if unknown."""
```

A trade is never invented. A line carries a trade half only because the gate chain latched and the fill was applied to a fake balance.

`src/paper/paper_run.py` — the only writer

```
paper_log.append_row(
    paper_log.paper_row(seen, bot.exchange_id, run.figures())
)
```

Real time

A run ticks on the wall clock and asks the feed for one bot per fire, so a fleet of many bots never blocks the window on a single pass. One bar is shared across the open bots.

`PaperTraderTabQt` — the round robin

```
def advance_once(self) -> list:
    """Tick the next open bot against its newest live window."""
    if self._run is None or not self._run.running or not self._run.bots:
        return []
    chosen = self._run.bots[self._cursor % len(self._run.bots)].bot_id
    self._cursor += 1
    made = surface.advance_run(self._run, self._feed, chosen)
    self.refresh()
    return made
```

Paper Trade History

The manual's part list names a second History tab reading a paper trade log. No such module exists. `HistoryTab` in `src/gui/history_tab.py` reads live venue history alone.

The ledger such a tab would read now exists. Every paper action lands in the file the Paper Trader log names, and no screen reads that file yet.

In development.

2026-09-08 08:17 - #19 - what the closed issues landed

Real-time, API-fed trades against a fake budget. This is designed as the second tier of strategy validation within the platform.

The tab is now called Paper. It sits second on the bar, on a white ground with black text, and it is the only white tab.

The screen was empty until issue #19 built it. The tab row carried a Paper Trader placeholder, which drew its name, one sentence saying it was not built, and the issue that owned it. The section under this one holds what

the tab is now.

The placeholder carried its empty state in a surface module of its own, and the block below is that state.

```
HEADING = "Paper Trader"
ISSUE = 19
BUILT = False
STATE_TEXT = "This tab is not built."
ISSUE_TEXT = f"Issue #{ISSUE} carries the build-out."
```

That module is gone, and the empty state above went with it.

The Qt tab and a renderer panel both drew that one view model, so neither could drift from the other.

Live, Paper and the Simulator differ in one thing only, where the data comes from. The trading logic stays one body of pure code all three call, and only the stateful shells fork. Real time is Paper's defining property, and its budget is twice the dollar target.

`src/trading/container/config.py` — `BotConfig`, the field that budget doubles

```
target_balance: float = 200.0  # Balance the bot trades relative to
```

Two live surfaces once offered the step. The Swarm tab's Paper Swarm sub-tab is chrome: its Start button flips a flag and relabels itself, and no bot is constructed. Its caption is now an empty string, so it promises nothing. The wing toggle no longer names a Paper Trader in the Qt window, while the React mirror still does. Issues #422 and #426 folded into issue #19.

The tab as it stands now

The screen is built. It is the Trading tab's panes over the live exchange feed: the Privacy Mode row, the market selector, the two button rows, the bot list, the Indicator Voting Panel, the Live Feed pane, the Fake Balance table and the Paper Run table. The Qt window and the React page draw one view model.

The old Paper Trader surface answered the tab under the names in the block below. That module is gone and no bridge serves `paper_trader_tab.state`. The surface is `src/gui/paper/paper_trading_tab_surface.py`, which declares `METHOD = "paper_trading.tab"`, `HEADING = "Paper"` and `ISSUE = 19`. The block below records the older names.

```
METHOD = "paper_trader_tab.state"

HEADING = "Paper"
ISSUE = 19
BUILT = True
```

`src/gui/paper/paper_trading_tab_surface.py, in METHOD` answers `paper_trading.tab` today, and that file holds `HEADING` and `ISSUE`. No module on disk holds `BUILT` for this tab.

The data path is the venue's public market feed, and it goes one way. The reader names the six things it answers, and every other name raises. An order, a cancellation or a venue write cannot be written through it.

`src/paper/live_feed_source.py` — the read set and the refusal

```
READ_NAMES = (
    "venue",
    "product_id",
    "granularity",
    "candles",
    "ticker",
    "asked_at",
)


class SendRefused(AttributeError):
    """Raised when a name outside ``READ_NAMES`` is asked of ``LiveFeedSource``."""
```

Live candles enter the same gate chain a live bot runs. Nothing on the paper side defines a gate of its own.

`src/paper/paper_run.py` — the tick

```
context = tape_context(bot, balance, window, reading, summary)
armed = latch(context)
```

The money is fake and lives in memory alone. A run writes no file, and no figure it produces reaches the stored bot record or the header strip.

`src/paper/fake_balance.py` — the budget

```
BUDGET_MULTIPLE = 2.0
```

The two strips the Trading tab carries are not copied, and neither is their supporting code. Their rows carry Import Live Fleet and the Start or Stop press.

The fake balance is a fleet figure. Five paper bots with a $200 target open Paper Spendable at $1000 and Paper Locked at $1000, each holding the full fleet sum. Adding or removing a bot moves both openings on the next press of Start.

`src/paper/fake_balance.py` — the fleet total both openings take

```
def fleet_target_usd(bots: Sequence[Any]) -> float:
    """The sum of every bot's ``target_usd``, the figure both openings take."""
```

Two more figures are tracked beside them. Paper Realized Profits moves when a fold buy closes the tranche a scrum sell opened, and Paper Mature Profits is the profit on positions past 200 percent growth, which is the figure the live wing already reads. No paper figure is written into the stored bot record or the header strip.

`src/paper/fake_balance.py` — the strip's four figures

```
FIGURE_LABELS = (
    ("spendable_usd", SPENDABLE_LABEL),
    ("locked_usd", LOCKED_LABEL),
    ("realized_profit_usd", REALIZED_LABEL),
    ("mature_profit_usd", MATURE_LABEL),
)
```

Every paper action is recorded in one file of its own. Each line carries the gate decision and, when one filled, the pretend trade in the shape a Coinbase year-to-date row takes. The file sits beside the live folders and never inside one.

`src/paper/paper_paths.py` — where the log lives

```
PAPER_ROOT: Path = Path.home() / ".acervator_paper"
```

How the Paper tab reaches the bar

`PaperTraderTabMixin` builds the tab and inserts it into the window's tab book. The builder catches every error, writes one warning line, and leaves the tab off the bar. A tab that asks the panel for a value the panel does not publish is therefore a missing tab, not a crash the operator can see.

`src/gui/main_tabs/paper_trader_tab.py` — the builder

```
def _build_paper_trader_tab(self) -> None:
    """Insert the Paper tab at ``PAPER_BUILD_INDEX``."""
    try:
        from ..variant_surface import PAPER_TRADER, surface_class

        self._paper_trader_tab = surface_class(PAPER_TRADER)()
        self._main_tabs.insertTab(
            PAPER_BUILD_INDEX, self._paper_trader_tab, HEADING
        )
    except Exception as exc:  # noqa: BLE001 - a missing tab is not a crash
        logger.warning("Paper tab unavailable: %s", exc)
        self._paper_trader_tab = None
```

The window builds nine tabs. Paper takes the second slot, after Sim.

```
Sim  Paper  Live  Charts  Inspector  Swarm  History  Status  Console
```

The line beside the voting panel title

One label sits beside the Indicator Voting Panel title. It holds the panel's empty state and nothing else. The shared function `no_data_text` builds that line. The Simulator page says what its own panel draws in that state. The label is hidden whenever the panel has values to draw.

`PaperTraderTabQt` — the Qt clone draws the line

```
def _draw_indicators(self, panel: dict) -> None:
    self._indicator_title.setText(panel["title_text"])
    # no_data text is empty while the panel holds a reading, so the
    # summary label shows only the one reason there is nothing to draw.
    reason = panel["no_data"]["text"]
    self._indicator_summary.setText(reason)
    self._indicator_summary.setVisible(bool(reason))
    for table, spec in zip(self._indicator_tables, panel["tables"], strict=True):
        self._fill_indicator_table(table, spec)
```

With no paper fleet the label names the press that builds one, under both builds.

```
No TA data — No paper fleet yet. Press Import Live Fleet.
```

The style sheet the Paper page loads

The page carries Live's own two style sheets, and one function writes them into the page's head.

The old Paper page host declared one sheet of its own, in the line below. That host is gone, and so is its sheet. The Paper page carries Live's two sheets, named in `src/gui/paper/paper_react_trading_tab.py, in STYLE_ASSETS` and written into the page's head by `src/gui/react_history_panel.py, in page_html`.

```
TAB_STYLE_ASSETS: tuple[str, ...] = ("paper_trader_tab.css",)
```

`src/gui/paper/paper_react_trading_tab.py` — the sheets the page carries today

```
STYLE_ASSETS: tuple[str, ...] = ("trading_tab.css", "exchange_tab.css")
```

The old sheet set the page ground, the type, the pane grid and the table rules. Its grid repeated the two pane sizes the Qt clone gave its splitters, so the bot list, the voting panel, the feed pane and the run pane took the same share of the window in both builds.

The old sheet's pane grid read:

```
.paper-top,
.paper-bottom {
  display: grid;
  grid-template-columns: 600fr 500fr;
}
```

Without the sheet the page draws on the browser's own defaults: a white ground, serif type, one column and no grid.

The skin tokens the Paper sheet reads

The old sheet named no colour of its own. Every colour arrived as a custom property on the tab element, written from the `SKIN` dictionary the old surface built. A scrum row and a fold row in the Paper Run table took the two colours `SIDE_COLOURS` gave them in the Qt clone.

The old sheet's two trade rows read:

```
.paper-tab tr[data-side="scrum"] td {
  color: var(--paper-scrum-colour, var(--text));
}

.paper-tab tr[data-side="fold"] td {
  color: var(--paper-fold-colour, var(--text));
}
```

Start Paper Run redraws the pane

`start_run` draws the pane after it opens the run. With no paper fleet the first tick returns nothing, so without that draw the Paper Run pane kept the word it held before the press while the run was open.

`PaperTraderTabQt` — the press that opens a run

```
    def start_run(self) -> str:
        """Open the run and tick one bot per timer fire from now."""
        if not self._bots:
            self._bots = surface.live_fleet()
        self._run = surface.start_run(self._bots)
        self._cursor = 0
        self.advance_once()
        self.refresh()
        self._tick_timer.start(surface.tick_interval_ms(self._run, self._symbol))
        return self._run.state
```

Both builds carry the same draw.

2026-09-19 - #19 - Live's tab code cloned under Paper names

Unit Q1 of issue #19. The Paper tab is now Live's tab code, forked under Paper names in both builds, with Import Live Fleet as the fleet's one way in and the header strip reading the paper ledger while Paper is in front. The two hosts the sections above describe are removed, and no Python or JavaScript file holds either name; The old hosts are gone names them.

The budget rule today overtakes four sentences

Four sentences on this page carry a figure the operator's words of 2026-09-19 overtake. Each is quoted here and kept above as it stands.

- "The paper budget is twice the dollar target, so a bot has room to fold without the exercise ending on the first dip." (What the step is)
- "Each bot opens a budget of twice its dollar target: units worth the target, and the rest in cash to fund a fold." (The fake balance)
- "Each variable holds the full fleet sum, which is twice the fleet's dollar target in total." (The fleet ledger)
- "Real time is Paper's defining property, and its budget is twice the dollar target." (2026-09-08 08:17)

The rule now: the paper budget is unbounded, and Paper Spendable opens at the sum of the fleet's Target Balances. His words: *"a fake, unbounded (starts equal to bots total target balance) budget."* No fold is refused

for cash. `BUDGET_MULTIPLE` in `src/paper/fake_balance.py` still reads `2.0`; the unit that starts a paper run retires it, and no clone code reads it.

`src/paper/fake_balance.py` — the opening figure the rule keeps

```
def fleet_target_usd(bots: Sequence[Any]) -> float:
    """The sum of every bot's ``target_usd``, the figure both openings take."""
```

The Qt tab is Live's tab code, forked

The Qt build of the Paper tab is `PaperTradingTab`, a fork of the Live tab's own code under the Paper Trader's name. The seam's Qt loader answers it.

`src/gui/variant_surface.py` — the Qt loader

```
def _qt_paper_trader() -> type:
    """Import and return the Qt Paper tab, ``PaperTradingTab``."""
    from .paper.paper_trading_tab import PaperTradingTab

    return PaperTradingTab
```

Each module under `src/gui/paper/` is one Live module copied and renamed, or one of the Simulator's forks copied once more where that fork's only change was the feed. The copy keeps Live's layout, titles, sizes and design tokens.

Paper module	forked from
`paper_trading_tab.py` `PaperTradingTab`	`src/gui/main_tabs/trading_tab.py` `_build_trading_tab`, and the window's `add_exchange_tab`
`paper_exchange_tab.py` `PaperExchangeTab`	`src/gui/widgets/exchange_tab.py` `ExchangeTab`
`paper_bot_status_table.py` `PaperBotStatusTable`	`src/gui/simulator/sim_bot_status_table.py` `SimBotStatusTable`
`paper_extractor_bot_table.py` `PaperExtractorBotTable`	`src/gui/simulator/sim_extractor_bot_table.py` `SimExtractorBotTable`
`paper_indicator_panel.py` `PaperIndicatorVotingPanel`	`src/gui/simulator/sim_indicator_panel.py` `SimIndicatorVotingPanel`
`paper_status_log.py` `PaperStatusLog`	`src/gui/simulator/sim_status_log.py` `SimStatusLog`
`paper_bot_detail.py` `PaperBotDetailDialog` and its seven tab mixins	`src/gui/simulator/sim_bot_detail.py` `SimBotDetailDialog` and its seven
`paper_exchange_choice.py` `PaperExchangeChoiceDialog`	`src/gui/simulator/sim_exchange_choice.py` `SimExchangeChoiceDialog`
`src/paper/fleet_source.py` `PaperFleetSource`, `PaperBot`	`src/simulator/fleet_source.py` `FleetSource`, `SimBot`, one fleet and no run mode
`src/paper/paper_bot_manager.py` `PaperBotManager`	`src/simulator/sim_bot_manager.py` `SimBotManager`
`src/paper/paper_bot_view.py` `PaperBotView`	`src/simulator/sim_bot_view.py` `SimBotView`

Diagram: flowchart, left to right

- variant_surface PAPER_TRADER -> **PaperTradingTab**
- PaperTradingTab -> **exchange layer stack**
- exchange layer stack -> **PaperExchangeTab**
- PaperExchangeTab -> **PaperBotStatusTable**
- PaperExchangeTab -> **PaperExtractorBotTable**
- PaperTradingTab -> **PaperIndicatorVotingPanel**
- PaperTradingTab -> **PaperStatusLog**
- PaperTradingTab -> **API Interaction Log**
- PaperTradingTab -> **PaperFleetSource**
- PaperFleetSource -> **paper_fleet.json under PAPER_ROOT**

What the fork draws

The tab is Live's four splitters at Live's sizes: the exchange layer stack beside the panel on top, the Activity Log and the API Interaction Log side by side below. Every module carries Live's title. The venue page holds Privacy Mode where Live draws it, `+ New Bot` where Live draws it, the Scrumming Bots table and the Extractor Bots table, both hidden until a row arrives, and the command bar of Start, Pause, Stop, Restart and Delete. The panel holds its title, the Bot selector and its privacy dot, the currency rate strip, both indicator tables, both confidence bar graphs, the timeframe-lock line and the staleness banner.

Three positions differ from Live by ruling, the ruling unit 7 of issue #117 made for the Simulator. The corner Live gives `■ Add Crypto Exchange` holds Import Live Fleet and Start Paper Run, each at Live's corner-button width. The Get Started card holds the same two where Live's card holds its add button. The news line and the data-pool line are not forked; the data-pool row keeps Live's height and holds nothing.

`src/gui/paper/paper_trading_tab_surface.py` — the corner buttons

```
CORNER_BUTTONS = (
    (paper.IMPORT_LIVE_FLEET_ACTION, paper.IMPORT_LIVE_FLEET_TEXT),
    (paper.START_RUN_ACTION, paper.START_RUN_TEXT),
)
```

A press on `+ New Bot` writes one Activity Log line naming Import Live Fleet as the fleet's way in; the paper bot wizard is a later unit's. A press on Start Paper Run writes one line saying no paper run is built; the run is a later unit's too.

`src/gui/paper/paper_trading_tab_surface.py` — the two lines

```
NEW_BOT_FORMAT = (
    "+ New Bot on {exchange}: the Paper fleet is loaded through "
    "{way_in}; the paper bot wizard is not built."
)

START_RUN_TEXT = "Start Paper Run: no paper run is built; nothing started."
```

THE FORM THE WIZARD REPLACED. The block above stays here, because a reader who saw that line needs to find it. `NEW_BOT_FORMAT` left the tree, and the press opens the Bot Creation Wizard instead of writing a line. Its neighbour in that module now holds the button's caption rather than a refusal.

`src/gui/paper/paper_trading_tab_surface.py` — what stands there today

```
IMPORT_LIVE_FLEET_TEXT = "Import Live Fleet"
START_RUN_TEXT = "Start Paper Run"
STOP_RUN_TEXT = "Stop Paper Run"
```

What the fork does not carry

The copies hold no live bot manager, no connector and no event bus. Every send the Live code makes is cut, not stubbed: the signal-contract emits, the event-bus emit behind the bot selector, the API-log listener on the process-wide log, the watchdog over the live bot manager, the TA snapshot store, the data-pool read behind the display price, and the demo readings the Live panel invents from a random walk. Zero occurrences of `ScrummingBot`, `BotContainer`, `BotManager`, `EventBus` and `PhantomBalance` under `src/gui/paper/` and `src/paper/`.

The Simulator's run modes, its way-in row, its replay layer, its Stone Tablet source, its Portfolio Battery, its retrieval and its nigredo tone are not carried. A Paper module forked from a Simulator fork has those parts cut.

Asked for a send by name, `PaperFleetSource` raises `SendRefused`. The Fire button on a row reaches it and the refusal lands in the Activity Log.

`src/paper/fleet_source.py` — the refusal

```
    def __getattr__(self, name: str):
        """Refuse every name outside ``READ_NAMES``."""
        raise SendRefused(
            f"PaperFleetSource answers {READ_NAMES} and cannot {name!r}. "
            "The Paper Trader receives and asks; it sends nothing."
        )
```

The React page is Live's page modules, forked

The React build of the Paper tab is `PaperTradingTabReact`, a fork of the Live tab's React host under the Paper Trader's name, and it loads seven page modules that are Live's seven copied under Paper names. The seam's React loader answers it.

`src/gui/variant_surface.py` — the React loader

```
def _react_paper_trader() -> type:
    """Import and return the React Paper tab, ``PaperTradingTabReact``."""
    from .paper.paper_react_trading_tab import PaperTradingTabReact

    return PaperTradingTabReact
```

Paper module	forked from
`src/gui/web/paper_trading_tab.js`	`trading_tab.js`

Paper module	forked from
src/gui/web/paper_exchange_tab.js	exchange_tab.js
src/gui/web/paper_indicator_panel.js	sim_indicator_panel.js, the flip button cut
src/gui/web/paper_status_log.js	sim_status_log.js
src/gui/web/paper_bot_status_table.js	sim_bot_status_table.js
src/gui/web/paper_extractor_bot_table.js	sim_extractor_bot_table.js
src/gui/web/paper_table_cells.js	sim_table_cells.js
src/gui/paper/paper_react_trading_tab.py PaperTradingTabReact	src/gui/react_trading_tab.py TradingTabReact
src/gui/paper/paper_trading_tab_surface.py	src/gui/main_tabs/trading_tab_surface.py, the payload builder
src/gui/paper/paper_exchange_tab_surface.py	src/gui/main_tabs/exchange_tab_surface.py, the payload builder
src/gui/paper/paper_react_bot_detail.py PaperBotDetailReactDialog	src/gui/simulator/sim_react_bot_detail.py, over Live's nine window bundles

Diagram: flowchart, left to right

- variant_surface PAPER_TRADER -> **PaperTradingTabReact**
- PaperTradingTabReact -> **paper_trading_tab.js**
- paper_trading_tab.js -> **paper_exchange_tab.js**
- paper_exchange_tab.js -> **paper_bot_status_table.js**
- paper_exchange_tab.js -> **paper_extractor_bot_table.js**
- paper_bot_status_table.js -> **paper_table_cells.js**
- paper_extractor_bot_table.js -> **paper_table_cells.js**
- paper_trading_tab.js -> **paper_indicator_panel.js**
- paper_trading_tab.js -> **paper_status_log.js**
- paper_trading_tab.js -> **API Interaction Log pane**

What the page draws

The page is Live's four splitters at Live's sizes, read off Live's own `trading_tab_surface`. Every module carries Live's title and Live's part name, so the page and the Qt fork enumerate the same modules in the same order. The corner holds Import Live Fleet and Start Paper Run in Live's corner-button chrome, and the Get Started card holds the same two at Live's card-button size. The venue page holds Privacy Mode, plain space where Live draws the news line, `+ New Bot`, the data-pool row at Live's height holding a blank line, the two tables and the command bar.

`src/gui/web/paper_trading_tab.js` — the corner

```
  // The corner Live gives its add button: the payload's buttons in one row.
  function CornerButtons(props) {
    var listed = listField(props.layer, CORNER_BUTTONS);
```

The host builds one payload per bridge method from state it owns: the tab payload from its own `PaperTradingTabState`, the Activity Log from its own `PaperStatusLogModel`, the panel from its own `IndicatorPanelModel`, and each seated venue's three payloads from that venue's own `ExchangeTabModel`, its `PaperBotStatusTableModel` and Live's `ExtractorBotTableModel`. Every ask the page makes is written to the console line the host reads, and `run_action` answers it; nothing is sent anywhere else.

`src/gui/paper/paper_react_trading_tab.py` — the console line

```
ACTION_PREFIX = "acervator-paper:"
```

The header strip shows on Paper

The window's header strip stays on screen while the Paper tab is in front. `ISOLATED_TABS` names no tab now, so the strip hides on nothing. While Paper is in front the ten fields carry the Paper Trader's figures: SPENDABLE, REALISED, LOCKED and MATURE from the paper ledger, EXCH from the seated venues, and the five cards from the held records. On every other tab the strip carries what it carried before; the live fleet's figures do not appear while Paper is in front, and no paper figure appears on Live.

`src/gui/main_tabs/main_window_surface.py` — the tuples and the reader

```
ISOLATED_TABS: tuple = ()

#: The tabs the header strip reads the Paper Trader's ledger and records on.
PAPER_FED_TABS = (PAPER_TAB,)


def header_strip_reads_paper(tab_name: Any) -> bool:
    """Whether the stat strip reads the Paper Trader's ledger and records on the tab named."""
    return tab_name in PAPER_FED_TABS
```

`src/gui/main_window.py` — `_refresh_header_strip`, the branch beside the Sim's

```
            if self._header_strip_reads_paper():
                paper_tab = self._paper_trader_tab
                self._write_header_strip(
                    paper_tab.aggregate(), paper_tab.exchange_count()
                )
                return
```

The tab's `aggregate` is `strip_aggregate`: Live's arithmetic over the held records, with the ledger's four money figures laid over the four money keys. Before a run the ledger holds zeros, so the four money columns read an em dash, as Live's read with no bot.

`src/paper/fleet_source.py` — the ledger's keys over the aggregate's

```
LEDGER_TO_AGGREGATE = (
    ("spendable_usd", "wallet_cash_usd"),
    ("locked_usd", "crypto_position_value_usd"),
    ("realized_profit_usd", "total_realized_exchange"),
    ("mature_profit_usd", "total_mature_exchange"),
)
```

Diagram: flowchart, left to right

- dashboard tick, 2000 ms -> **tab in front**
- tab change -> **tab in front**
- pick -- Sim -> **SimTradingTab.aggregate**
- pick -- Paper -> **PaperTradingTab.aggregate**
- pick -- any other -> **BotManager.get_aggregate_stats**
- SimTradingTab.aggregate -> **the ten cells**
- PaperTradingTab.aggregate -> **the ten cells**
- BotManager.get_aggregate_stats -> **the ten cells**

AMMO reads the paper fleet's own total

AMMO on the strip is every paper bot's Target Delta added together, in whole dollars and with no sign, so a bot above its target cannot cancel one below. It draws green while more paper bots hold more than their target and red while more hold less, which is the rule the Trading tab page gives the column. The fleet source publishes the total and the two bot counts the colour reads.

`src/paper/fleet_source.py` — `aggregate_stats`

```
ammo_readings.append((position_value, bot.live_target_usd, bot.target_usd))
...
"total_target_delta_usd": ammo["total_target_delta_usd"],
"bots_scrum_territory": ammo["bots_scrum_territory"],
"bots_fold_territory": ammo["bots_fold_territory"],
```

One call sums those three figures for every fleet source, so the Paper Trader's total, the Simulator's and Live's cannot be worked out differently.

`src/trading/target_bands.py` — `fleet_ammo`

```
target_usd = ammo_target(live_target, configured_target)
if target_usd > 0:
    total_usd += abs(target_delta(position_value, target_usd))
    where = target_territory(position_value, target_usd)
```

A reading takes the grown target wherever a paper bot carries one, which is the target that bot's own Ammo cell is measured against. The strip's total is therefore the sum of the column above it. A paper bot with no target contributes nothing to either figure.

Import Live Fleet fills the tables and the strip

Import Live Fleet copies the stored bots of one exchange out of the live fleet file into the Paper Trader's own fleet file, in both builds. A press at the corner or on the Get Started card reads which exchanges the live file names. With more than one it opens the chooser; with exactly one it takes that one and opens nothing; with none it writes one Activity Log line naming the file and moves nothing. Each stored bot on the chosen exchange is copied whole, every key of the record, into the paper fleet under its own bot id, and the fleet signal fires: the paper fleet file is written, the venue seats, and the two tables draw one row per imported bot. The Activity Log reads `Imported 38 bot(s) from bot_state.json on coinbase.` on a fleet of thirty-eight. The live fleet file is only read, never written.

`src/paper/fleet_source.py` — the copy

```
    def import_live_fleet(self, exchange_id: str) -> list[PaperBot]:
```

`src/paper/paper_paths.py` — where the paper fleet file lives

```
#: The paper fleet file under ``PAPER_ROOT``, in ``bot_state.json``'s shape.
PAPER_FLEET_NAME = "paper_fleet.json"
```

The tab starts empty. The fleet source answers the records the paper fleet file holds and nothing else, so with no paper fleet there is no venue, no row and no figure until Import Live Fleet loads one. On the next launch the tab reads the paper fleet file once and draws the imported rows from it.

The file's top level carries the wires

Beside `bots`, the paper fleet file carries `smart_wires` and `smart_wire_ledgers`, the two keys `bot_state.json` carries for the live fleet. The Paper tab holds one fleet and no run mode, so both keys sit at the file's top level rather than inside a mode's entry the way the Simulator nests them.

`PaperFleetSource.hold_wire_manager` takes a `PaperWireManager`'s rows onto the fleet and `PaperFleetSource.wire_manager` builds the manager back at the next launch, so a paper wire survives a restart the way a live one does. Stop Paper Run holds the run's manager and saves the file, so what a wire carried during a run is on disk before the tab is closed.

A file written before these keys existed reads as a fleet with no wire, and a build that does not know the keys reads the same bots, the same venues and the same rows it read before. Both were driven: the unchanged tree's reader and this one gave an identical reading of a file carrying both keys, and the same comparison reported a difference the moment one value was changed by hand.

`src/paper/fleet_source.py` — the write

```
        payload = {
            "saved_at": time.time(),
            "saved_at_human": datetime.now().strftime(SAVED_AT_HUMAN_FORMAT),
            "bot_count": len(self._records),
            "bots": dict(self._records),
            WIRES_KEY: [dict(one) for one in self._wires],
            WIRE_LEDGERS_KEY: [dict(one) for one in self._wire_ledgers],
        }
```

Deleting a paper bot drops its ledger row and every wire row naming it, the way the live state file drops a removed bot's, so no route is left pointing at a bot the fleet no longer holds.

A fold's growth travels the wires to another paper bot

A paper bot can send part of a fold's growth to another paper bot. A wire names a source, a target and a percentage. The moment a paper fold compounds its growth into the source's target balance, the wire's share of that same figure is credited to the other bot. The source keeps its own growth whole and the share is new money at the destination, which is how the live fold route behaves.

Now: a paper fold sends no wire. A paper sale sends each wire its share of the sale's surplus, as a live sale does, and those dollars leave the source's cash for the target's. A share that parks leaves the source's cash at the sale and joins the target's when it lands (`src/paper/paper_run.py, in route_sale_surplus`).

The share lands in the target's standing fold tranches, split evenly, the way wire credits land on a live bot. A paper balance opens on the target's first worked tick, so a share arriving before that is parked on its ledger and spread over the tranches the target holds when it opens, or over the first tranche its own scrum builds. The ledger carries the lifetime sent, the lifetime received, the provenance per funder and the parked pool.

`PaperWireManager` in `src/paper/paper_wire.py` owns the topology and one ledger per paper bot.

verb	what it does
`register_wire`	sets the source-to-target percentage, from 0 up to 100
`unregister_wire`	drops one route
`attach_bot`	holds the fake balance of an open paper bot a share can reach
`release_bot`	gives that balance back when the run ends
`distribute_sale_surplus`	moves each wire's share of one sale's surplus
`land_parked_credits`	spreads a parked pool over standing tranches
`export_wires`, `import_wires`	the routes the paper fleet file carries
`export_ledgers`, `import_ledgers`	the ledgers the paper fleet file carries

The wire call in the fold code below is gone, and a paper sale hands its surplus on instead (`src/paper/paper_run.py, in route_sale_surplus`).

```
    growth = grow_target(bot, balance, units, slices, float(price), int(candle_ts_ms))
    if wires is not None and growth > 0.0:
        route_fold_growth(wires, bot.bot_id, growth, float(price))
```

One fold driven over an empty scratch home, two bots at a $200.0000 target, a scrum at $110.00 and a fold at $95.00, with one wire at 40 per cent between them:

```
the fold's growth on the source       6.7773
the wire's rate                         40.0 per cent
the share that leaves                    2.7109
the target's fold queue before          49.7000
the target's fold queue after           52.4109
the source's lifetime sent               2.7109
the target's lifetime received           2.7109
```

The same pair ticked over a whole price series sends four times. The source's own walk is unchanged by the wire, 5 scrums and 4 folds either way, and the target ends holding more of its asset because the wired money bought more on its folds.

```
                          no wire       a wire at 40 per cent
source scrums, folds      5, 4          5, 4
source end target       249.3815      249.3815
target end units          2.8997        3.1297
target lifetime received  0.0000       19.7526
```

A paper wire reaches paper money only. The manager holds one ledger per paper bot and delivers nothing to an id it has no ledger for, so a wire pointed at a live bot or at a simulated bot books no transfer and reports that the target has no ledger. That was driven against a real live bot id and a real simulated bot id: both routes answered `target bot has no ledger`, no transfer was booked, and both state files read the same digest before and after.

A loaded bot reads IDLE

Import Live Fleet copies each stored record and writes its state `idle`, whatever the live file saved, and the paper fleet file's records read `idle` at construction. That is the live restore's own rule, and the Simulator's. A row's state is the Bot ID cell's colour and its tooltip's `State:` line; Live's ten columns hold no State column.

`src/paper/fleet_source.py` — the one write

```
def loaded_idle(record: dict) -> dict:
    """``record`` with its ``state_when_saved`` written ``BotState.IDLE``, as
    ``restore_bots_from_state`` recreates every persisted bot in IDLE until
    the operator starts it; a loaded bot never reads ``running`` with no run."""
    record["state_when_saved"] = BotState.IDLE.value
    return record
```

The command bar moves a record's state through `PaperBotManager`: Start writes `running` and Live's `✓ Bot <id> RUNNING.` line, Pause `paused`, Stop `stopped`, Restart `running`, and Delete opens Live's Delete box. No paper bot ticks yet; the state is the record's. Detail opens the seven tabs over the record through `PaperBotDetailDialog`, and Apply is refused: the pending line reads `Refused 1 change(s) — the Paper Trader sends nothing`.

How the Paper tab reaches the bar today

The builder in `src/gui/main_tabs/paper_trader_tab.py` is unchanged. It asks the seam for the Paper class, and the seam now answers the clone in both builds. The window builds nine tabs and Paper takes the second slot, after Sim, in both.

`src/gui/variant_surface.py` — the registration

```
register(PAPER_TRADER, _qt_paper_trader, _react_paper_trader)
register(PAPER_BOT_DETAIL, _qt_paper_bot_detail, _react_paper_bot_detail)
```

The window's floor is 1400 by 900 pixels, set in `src/gui/main_window.py`, so no tab is reached narrower than 1400 through the window. Constructed alone, the Qt tab draws at 806 pixels wide and no narrower after a

fleet loads, the sum of Live's own minimum sizes; the React page draws at 700.

Overtaken, quoted whole: "The window's floor is 1400 by 900 pixels, set in `src/gui/main_window.py`".

True today: 1400 by 900 is still the figure, and the width is no longer typed out beside it. `main_window.py, in window_min_w_px` answers the wider of `WINDOW_MIN_W_PX`, the window's own figure, and `header_strip_surface.window_min_w`, which is what the header strip's own slots need. Both are read in one place, so the two cannot drift. Six counter cards answer 1400 px and a seventh answers 1476 px. The height stays `WINDOW_MIN_H_PX`, 900 px.

What the clone reading measured

Read off the real window in both builds over a scratch home holding a stand-in `bot_state.json` of the operator's record shape, thirty-eight scrumming records on coinbase at 5m, every socket but loopback refused:

reading	Qt	React
Paper on the bar	second of nine	second of nine
Live modules on the tab before the clone	0 of 6	0 of 10
Paper modules on the tab after the clone	6 of 6	10 of 10
strip on Paper at open	— — — — EXCH 0	— — — — EXCH 0
Import Live Fleet: rows	38, State: IDLE	38, State: IDLE
Import Live Fleet: Bot ID cell colour	#888888	rgb(136, 136, 136)
venue seated, EXCH	Coinbase, 1	Coinbase, 1
paper fleet file	38 records, all idle	38 records, all idle
bot_state.json hash after every press	unchanged	unchanged
a planted byte on a copy	hash moved	hash moved
a planted live figure on the Paper strip	did not appear	did not appear
a planted paper figure on the Paper strip	appeared	appeared
the same paper figure on the Live strip	did not appear	did not appear
Detail	7 tabs, Apply refused	7 tabs, Apply refused
old hosts in the import closure	neither	neither

2026-09-20 - #19 - The paper exchange adapter feeds the tab

Unit Q2 of issue #19. The Paper tab now holds one paper exchange adapter, `PaperExchange` in `src/paper/paper_exchange.py`, beside the feed reader the sections above describe. It answers every read a paper bot asks of the venue from the Coinbase Advanced Trade public market endpoints, and no write. Import Live Fleet reads the venue's product list once and each record's ticker once, and every one of those reads is a block on the API Interaction Log in both builds.

The feed reader sentences today overtake two passages

Two passages on this page describe the feed reader `LiveFeedSource`. Each is quoted here and kept above as it stands.

- "The feed reader answers six names and refuses every other one, so a send has no spelling." (Downstream only)
- "The data path is the venue's public market feed, and it goes one way. The reader names the six things it answers, and every other name raises." (The tab as it stands now)

The rule now: the tab's feed reader is `PaperExchange`, which answers the eleven names in its `READ_NAMES` and refuses every other public name. `LiveFeedSource` stays on disk for the two old hosts, which nothing

loads, and its GRANULARITY and BAR_SECONDS tables grew from eight rows to nine, the 4h row added, because the venue's candles page lists nine granularity names. A 1w ask of LiveFeedSource sent FIVE_MINUTE to the venue and answered 5m bars under the weekly name; the adapter rolls 1w up from daily bars instead.

src/paper/paper_exchange.py — the read set

```
READ_NAMES = (
    "venue",
    "product_id",
    "granularity",
    "timeframes",
    "products",
    "ticker",
    "candles",
    "windows",
    "quote_rate",
    "asked_at",
    "calls",
)
```

What the adapter answers, and from which endpoint

products reads GET market/products and keeps the products the venue trades, by the rule the Market Inspector's _product_trades states: status online and trading_disabled not set. ticker reads GET market/products/{id}/ticker?limit=1 and answers the last trade's price as last, with the book's best_bid and best_ask from the same answer; last is None when the venue answers no trade. candles reads GET market/products/{id}/candles at any of the nine granularity names, at most 350 bars a call, and answers [ts_ms, open, high, low, close, volume] rows oldest first. windows answers one candles window per higher timeframe a record's phantoms name. quote_rate answers 1.0 for USD with no call, and otherwise the {QUOTE}/USD ticker's last, the book's mid when the venue answers no trade; the thirteen USDC records share one USDC/USD read.

src/paper/paper_exchange.py — the ticker's answer

```
    def _ticker_view(self, symbol: str, entry: TickerEntry) -> dict:
        return {
            "symbol": symbol,
            "product_id": entry.symbol,
            "last": entry.last or None,
            "best_bid": entry.bid or None,
            "best_ask": entry.ask or None,
            "timestamp": entry.timestamp,
        }
```

One rollup, and only what the venue serves

The venue lists no weekly granularity. A `1w` ask reads daily bars in pages of 350, newest page first, until seven days per week asked are held, and rolls them through the stone tablets' `_rollup`, the one rollup every derived tablet timeframe reads through. Each timestamp is moved back by `MONDAY_OFFSET_MS` before the rollup and forward after it, the shift the Market Inspector's `weekly_from_daily` makes, so every weekly bucket starts on Monday 00:00 UTC. A 100-bar weekly window needs 700 daily bars, two calls. A granularity outside the nine is sent as spelled, so the venue's own 400 answers it and lands on the pane as an error block.

`src/paper/paper_exchange.py` — the rollup

```
def weekly_rows(daily: Sequence[Sequence[float]]) -> list[list[float]]:
    shifted = [[row[0] - MONDAY_OFFSET_MS, *row[1:6]] for row in daily]
    return [
        [float(int(row[0]) + MONDAY_OFFSET_MS), *row[1:6]]
        for row in _rollup(shifted, DAYS_PER_WEEK, WEEK_MS)
    ]
```

Cached with Live's two windows, paced at the connector's interval

Every ticker answer is held in a `TickerEntry` and every candle window in a `CacheEntry`, the two slot types Live's `MarketDataPool` holds in `src/exchange/data_pool.py`. A ticker asked again within 5 s is served from the slot with no call and no block. A candle window asked again within one bar of its timeframe, `TF_SECONDS`, is served the same way while the slot holds enough rows. Every venue call waits `PUBLIC_MIN_INTERVAL_S` after the call before it, the figure `src/exchange/market_inspector_fetcher.py` states for the public route and the exchange connector keeps as `_min_request_interval`, 0.1 s; one lock holds the calls in order, so two threads asking at once are spaced. A 429 is recorded as a warning block, waited out for the venue's `Retry-After` seconds when the answer carries one and one interval otherwise, and asked once more.

`src/paper/paper_exchange.py` — the cache and the pace

```
from ..exchange.data_pool import CacheEntry, TickerEntry
from ..exchange.market_inspector_fetcher import (
    API_LEVEL_ERROR,
    API_LEVEL_SUCCESS,
    API_LEVEL_WARNING,
    DAYS_PER_WEEK,
    MONDAY_OFFSET_MS,
    PUBLIC_MIN_INTERVAL_S,
    WEEK_MS,
    _product_trades,
)
```

Every read is a block on the API Interaction Log

The adapter records one entry per venue call on the `APIInteractionLog` its host hands it, the tab's own log, never the process-wide one Live's connector records on. The entry carries Live's fields: the exchange, the action (`FETCH_MARKETS`, `FETCH_TICKER` or `FETCH_OHLCV`, the words Live's connector records), the reason, the venue path as the endpoint, the params, the result, the response time, the level and the data usage. Each host now carries one signal, `apiEntryLogged`, and one receiver beside its `_on_api_event`, `_cross_api_event`, the crossing the main window carries for Live; the host registers the receiver as the log's listener, so an entry recorded on a worker thread lands in `_on_api_event` on the GUI thread through Qt's queued connection and draws as Live's block. The Live tab's pane is not written: the adapter never records on `get_api_log`.

`src/gui/paper/paper_trading_tab.py` — the crossing

```
    def _cross_api_event(self, entry: dict) -> None:
        """The ``api_log()`` listener: emit ``apiEntryLogged``, which Qt queues
        onto the GUI thread for ``_on_api_event`` from any other thread."""
        self.apiEntryLogged.emit(entry)
```

Import Live Fleet reads the venue once

After Import Live Fleet copies the records and fires `fleet_changed`, each host starts one worker thread, `paper-feed-import`, which runs `read_fleet` over the adapter: one `products` read, then one `ticker` read per imported record. Each read lands on the pane as it completes. When the worker ends, its summary crosses a second signal, `feedRead`, and one line lands on the Activity Log naming how many products the venue trades, how many of the fleet's products are among them, how many tickers answered, and how many calls it took; a fleet product the venue does not trade is named on the line, and a product list the venue did not answer is said so, naming no product. The GUI thread never waits on a read.

`src/gui/paper/paper_trading_tab_surface.py` — the line

```
FEED_LINE_FORMAT = (
    "Feed: {products} product(s) trade on {exchange}; {traded} of {records} fleet "
    "product(s) among them; {answered} of {records} ticker(s) answered in {calls} call(s)."
)
FEED_NO_PRODUCTS_FORMAT = (
    "Feed: {exchange} answered no product list; {answered} of {records} ticker(s) "
    "answered in {calls} call(s)."
)
FEED_UNTRADED_FORMAT = " Not traded: {symbols}."
FEED_THREAD_NAME = "paper-feed-import"
```

Every write name is refused

`PaperExchange.__getattribute__` raises `SendRefused` for every public name outside `READ_NAMES`, before any attribute is read, and `__getattr__` raises it for a name the class does not define. `place_order`, `cancel_order`, `create_order` and every other write name have no spelling on the adapter, and a method planted on the class without its name in `READ_NAMES` is refused too. No file under `src/paper/` or `src/gui/paper/` imports a credential loader, `ccxt`, `ScrummingBot`, `BotContainer` or `BotManager`.

`src/paper/paper_exchange.py` — the refusal

```
    def __getattribute__(self, name: str):
        """Answer a private name or one of ``READ_NAMES``; refuse the rest."""
        if not name.startswith("_") and name not in READ_NAMES:
            raise SendRefused(REFUSED_FORMAT.format(read_names=READ_NAMES, name=name))
        return object.__getattribute__(self, name)
```

What the feed reading measured

Read off the real window in both builds over a scratch home holding a copy of the operator's `bot_state.json`, thirty-eight scrumming records on coinbase at 5m, every socket but loopback refused, and a loopback stand-in shaped from the venue's pages: the four endpoints with their fields, 400 for a granularity outside the nine names, empty candles for a listed product with `trading_disabled` set, 350 candles a call at most, a ticker carrying `best_bid` and `best_ask`, and a 429 on a plant. On the commit before this unit, Import Live Fleet put nothing on the pane in either build, the feed reader's ticker answered one price with no bid and no ask, and a `1w` ask sent `FIVE_MINUTE` to the stand-in.

```
reading                                    Qt                      React
blocks on the pane after Import Live Fleet 39                      39 entries, 234 lines
actions                                    1 FETCH_MARKETS,        1 FETCH_MARKETS,
                                           38 FETCH_TICKER         38 FETCH_TICKER
thread the reads ran on                    paper-feed-import       paper-feed-import
thread violations written                  0                       0
least gap between two stamps               0.112 s                 0.112 s
gaps under 0.1 s                           0                       0
the Activity Log line                      38 of 38 tickers,       38 of 38 tickers,
                                           39 calls                39 calls
a ticker asked twice within 5 s            two asks, one call      two asks, one call
a 1w window against its daily rows         equal, Monday buckets   equal, Monday buckets
a 12h ask                                  HTTP 400 error block    HTTP 400 error block
a listed-not-traded product                [] and a No data block  [] and a No data block
a planted 429                              one wait of 1000 ms,    one wait of 1000 ms,
                                           then answered           then answered
place_order, cancel_order, create_order    SendRefused             SendRefused
a method planted without its name          SendRefused             SendRefused
a method planted with its name in READ_NAMES answered              answered
bot_state.json hash across the press       unchanged               unchanged
a planted byte on a copy                   hash moved              hash moved
sockets opened outside loopback            0                       0
```

The stand-in serves the 4h name and a `USDC-USD` ticker with trades because the venue's pages list them; whether the real venue answers `FOUR_HOUR` and whether `USDC-USD` carries trades are read at the real venue, not here.

One real reading followed, on both built bundles against the public host, no credential file present and the adapter's own pace: Import Live Fleet made 39 calls in each build, the product list answering 917 trading products and 8 not trading, all 38 fleet products among them, 38 of 38 tickers answered with their bid and ask, no 429, the least gap between two stamps 0.191 s in the Qt build and 0.168 s in the React build, the 39 calls spanning 11.4 s and 10.7 s. Launched in a Windows AppContainer with no network, the same press made 39 calls in each build, every one refused by the container and every one a block on the pane naming its cause, and

the Activity Log line read that the venue answered no product list and 0 of 38 tickers.

One more real reading settled the ninth name against the source. Through the built bundle's own copy of `PaperExchange`, against the public host, no credential file present and the adapter's own pace, one candles ask per name in `GRANULARITY` on `A15-USD`, nine calls, ten bars each:

```
name             status  rows  bar spacing
ONE_MINUTE       200     10    60 s
FIVE_MINUTE      200     10    300 s
FIFTEEN_MINUTE   200     10    900 s
THIRTY_MINUTE    200     10    1,800 s
ONE_HOUR         200     10    3,600 s
TWO_HOUR         200     10    7,200 s
FOUR_HOUR        200     10    14,400 s
SIX_HOUR         200     10    21,600 s
ONE_DAY          200     10    86,400 s
```

The nine calls were 0.271 s apart at least and spanned 2.3 s; no 400 and no 429. The venue answers `FOUR_HOUR`, so the 4h row stays. `src/exchange/timeframes.py` still says the venue ships eight granularities and no 4h; that file is Live's and this unit does not touch it.

2026-09-20 - #19 - The paper bot ticks and fills on the clone

Unit Q3 of issue #19. Start Paper Run now starts one worker thread that ticks every running paper bot at Live's cadence over `PaperExchange`, evaluates the shipped gate chains on each worked tick, sizes every scrum and fold with the pure functions Live calls, fills a scrum at the book's best bid and a fold at the best ask with the venue's taker fee, and moves the paper ledger the header strip reads. The old hosts left the tree.

The `apply_scrum` and `apply_fold` the earlier sections describe sized a fill with their own arithmetic; the two below are the Simulator's, forked over `src/trading/scrumming/sizing.py`.

Who runs a paper bot

`PaperRunner` in `src/paper/paper_run.py` is the forked shape of the Simulator's `_walk_bot`: one `threading.Thread` named `RUN_THREAD_NAME` (`paper-run`) per Start Paper Run. Its loop calls `advance` once, waits `TICK_INTERVAL_S` (5.0 s, the value `ScrummingBot.tick_interval` answers) through the one clock seam `WallClock.wait`, and reads the stop event before each pass. The tape the Simulator walks is replaced by the adapter's live reads.

`src/paper/paper_run.py` — the loop

```
    def _loop(self) -> None:
        try:
            while not self._stop.is_set():
                self.advance()
                if self._stop.is_set():
                    break
                self._clock.wait(self._stop, self._tick_interval_s)
```

Each pass reads every held record's state through the host's `_states` map, which the host rebuilds on the GUI thread at every `fleet_changed`. A record whose state reads `running` counts one tick; a `paused`, `stopped`

or `idle` record is skipped that pass and picked up the pass after the command bar moves it. A running bot is worked every `tick_skip` ticks, the rule `ScrummingBot.tick` applies: `scrum_read_rate_min` minutes over the 5 s tick, a tenth of that in track or fire, and the first tick after a bot starts running always worked, as Live's init tick is. `PaperBot` in `src/paper/fleet_source.py` carries `scrum_read_rate_min` from the record's config, 5 when the config names none.

`src/paper/paper_run.py` — the skip

```
def tick_skip(scrum_read_rate_min: int, scrum_target_mode: Optional[str]) -> int:
    rate = max(0, int(scrum_read_rate_min or 0))
    if rate <= 0:
        return 1
    base = max(1, int((rate * 60) / max(TICK_INTERVAL_S, 0.1)))
    if str(scrum_target_mode or "") in TRACK_MODES:
        return max(1, base // 10)
    return base
```

A worked tick reads `ticker` and `candles` on the runner's thread, never on the GUI thread. The ticker is cached 5 s and the candle window one bar, the two windows `src/exchange/data_pool.py` gives Live, so a 5m bot's gates read a new window every 300 s while its fill price follows the book every 5 s.

Who evaluates and who sizes

`tick` builds the `GateContext` through the Simulator's `tape_context` over the window's newest close, `bb_reading` and `VotingEngine.compute_all`, and evaluates it with `latch`, unchanged from the sections above. The higher-timeframe bias is None in this unit, so the two defer flags block nothing; unit Q5 widens it.

`apply_scrum` and `apply_fold` are the Simulator's two fills forked over a `FakeBalance`, and every figure comes from `src/trading/scrumming/sizing.py` by name: `scrum_units`, `priced_usd`, `estimated_fee_usd` and `sale_proceeds_usd` on a scrum; `fold_rebuy_factor`, `eligible_fold_tranches`, `cycle_growth_cap_usd`, `fold_cap_remaining_usd`, `plan_fold_consumption`, `fold_rate_taper`, `fold_spend_usd`, `fold_units`, `settle_fold_plan`, `plan_source_price` and `fold_surplus_usd` on a fold. A scrum sells from the highest-priced lot first and queues one fold tranche per lot sold from, in the dict shape Live's `_tick_execute_scrum` builds. A fold is planned under the cycle growth cap, as Live's `tick_phases` plans it, and the target's growth from the surplus is unit Q4's.

Now: the fold reads each tranche against the bot's distance settings, as Live's fold does, and the factor helper named above is gone (`src/paper/paper_run.py, in apply_fold`).

`src/paper/paper_run.py` — the scrum's size and proceeds

```
    units = scrum_units(float(delta), float(price), rule)
    ...
    notional = priced_usd(units, float(price))
    fee = estimated_fee_usd(notional, taker_fee_pct(bot))
    proceeds = sale_proceeds_usd(notional, fee)
```

The unit rule is read once per run through `run_rule`, which is `unit_rule(CLASS_CRYPTO, venue)`: `fractional` on coinbase. A venue with no cited rule starts nothing and writes `run_no_rule_line`.

Who fills, at what price and fee

A scrum fills at the tick's `best_bid` and a fold at the tick's `best_ask`, both from the one `PaperExchange.ticker` read of that tick; never at `last`. A fold's tranche eligibility is read at the tick's `last`, where Live's `fold_tranches.py` reads it. The fee is `estimated_fee_usd` of the notional at `taker_fee_pct`: the record's `trading_fee_pct` when set, else `TAKER_FEE_PCT`, the venue's taker fee at the default tier, 0.6 percent, the rule the Simulator's `DEFAULT_TRADING_FEE_PCT` applies. A scrum's cash gains the proceeds net of the fee; a fold's cash loses the spend plus the fee. Each `PaperTrade` carries the `bid`, `ask` and `last` of its read and the inputs its size came from, `delta_usd` on a scrum, `eligible_usd` and `taper` on a fold, so the paper log can be replayed through the same functions. The fill's stamp is the clock's seconds.

`src/paper/paper_run.py` — the two fills at the book

```
    if armed["scrum_armed"]:
        filled = apply_scrum(bot, balance, bid, now_s, stamp, context.delta, rule)
    elif armed["fold_armed"]:
        filled = apply_fold(bot, balance, ask, last, now_s, stamp, rule)
```

Who holds the money

`FakeBalance` in `src/paper/fake_balance.py` is the fork of the Simulator's `SimPosition`: `units`, `cash_usd`, `fold_tranches` as a list of Live's tranche dicts, `main_lots`, `target_usd`, `anchor_target_usd`, the growth cycle's figures and the trade counters, with `cost_basis_usd` over the lots and `mature_usd` through `mature_profit_usd`. It opens on the bot's first worked tick as the Simulator's `opening_position` opens: `fold_units` of the target at the read's `last` as one lot, plus `cash_usd` of the target, the bot's share of the unbounded budget.

`src/paper/paper_run.py` — the opening

```
def opening_balance(bot: PaperBot, price: float, rule: str) -> FakeBalance:
    target_usd = float(bot.target_usd or 0.0)
    units = fold_units(target_usd, float(price), rule) if price > 0.0 else 0.0
    lots = [{"units": units, "initial_buy_price": float(price)}] if units > 0 else []
    return FakeBalance(
        units=units,
        cash_usd=target_usd,
        opening_price=float(price),
        main_lots=lots,
        target_usd=target_usd,
        anchor_target_usd=target_usd,
    )
```

The ledger opens at the fleet total. `opening_ledger` sets Paper Spendable and Paper Locked to `fleet_target_usd` over every held record; on the operator's 38 records that is $3,698.46. `PaperLedger.figures` keeps both figures whole: each open balance replaces its `anchor_target_usd` share of the opening with its `cash_usd` for Spendable and its `value_usd` for Locked, so a record the runner has not opened still counts its target. No fold is refused for cash; the spend is not capped by the wallet, the `FUNDED_BY_TARGETS` rule the Simulator's Validation runs under, and Spendable reads negative rather than refuse. Paper Realized Profits is the sum of `fold_surplus_usd` over the run's folds, added through

`record_close` at each fold.

`src/paper/fake_balance.py` — the two figures over the open balances

```
        for bot_id, balance in held.items():
            price = float(at.get(bot_id, 0.0) or 0.0)
            spendable += balance.cash_usd - balance.anchor_target_usd
            locked += balance.value_usd(price) - balance.anchor_target_usd
```

What crosses to the GUI thread

Both hosts, `PaperTradingTab` in `src/gui/paper/paper_trading_tab.py` and `PaperTradingTabReact` in `src/gui/paper/paper_react_trading_tab.py`, carry four queued signals in the Simulator's shape: `run_line` for each runner line, `run_trade` for each `PaperTrade`, `run_finished` for the run at the thread's end, and `run_figures` for the ledger's figures, which the runner computes on its thread after each worked pass and each fill and the host holds as `_figures`. `aggregate` lays that snapshot over `strip_aggregate`, so the header strip's four money cells read a dict the GUI thread owns and never the runner's balances. The strip repaints from it on the window's 2 s dashboard tick.

`src/gui/paper/paper_trading_tab.py` — the crossing

```
        self._runner = PaperRunner(
            run,
            self._exchange,
            self._read_states,
            on_trade=self.run_trade.emit,
            on_figures=self.run_figures.emit,
            on_finished=self.run_finished.emit,
            say=lambda line: self.run_line.emit(line, "info"),
        )
```

Start Paper Run and Stop Paper Run

`_start_paper_run` is the Simulator's `_start_run` forked. A press with no run up opens `paper_run.start` over the held fleet under the run rule, starts the runner, writes `run_started_line` and turns the run button's face to `STOP_RUN_BUTTON`, "Stop Paper Run", on the corner and on the Get Started card. A fleet holding no record writes `RUN_NO_BOT_TEXT` and starts nothing. While the run is up the same seat's press reaches `_stop_paper_run`, which writes `RUN_STOPPING_TEXT` and sets the event the runner reads; the loop ends at the tick reached, `run_finished` crosses, and `_take_paper_run` writes `run_ended_line` and turns the button back to "Start Paper Run". The balances, the ledger and the trades stay readable on `run()` after the end. The Qt host relabels its four run buttons; the React host redraws the page through `show_tab` with `run_running` on `PaperTradingTabState`, and the page draws the seat's text and action from the payload as it did before.

`src/gui/paper/paper_trading_tab_surface.py` — the seat's two faces

```
def run_buttons(running: bool = False) -> tuple:
    if not running:
        return CORNER_BUTTONS
    return (CORNER_BUTTONS[0], STOP_RUN_BUTTON)
```

The command bar is unchanged: Start, Pause and Stop move one record's state through `PaperBotManager`, and the runner reads that state each pass. Start Paper Run starts the runner whatever the records read, so a bot started on the bar during a run ticks from the next pass.

The old hosts are gone

`PaperTraderTabQt`, `react_paper_trader_tab`, `paper_trader_tab.js`, `paper_trader_tab.css` and `paper_trader_tab_surface` are removed. The six names the clone read off the old surface, `HEADING`, `IMPORT_LIVE_FLEET_ACTION`, `IMPORT_LIVE_FLEET_TEXT`, `START_RUN_ACTION`, `START_RUN_TEXT` and `button_name`, live in `src/gui/paper/paper_trading_tab_surface.py` with `STOP_RUN_ACTION` and `STOP_RUN_TEXT`. `src/gui/main_tabs/main_window_surface.py` reads the tab's name and method there, `src/gui/main_tabs/paper_trader_tab.py` reads the name there, and `src/core/desktop_bridge.py` serves the clone's `view_model` under `paper_trading.tab` for the Electron shell, one call holding no host. `tools/sync_renderer_modules.py` rewrote `desktop/renderer/module_manifest.js`, which now lists the seven paper page modules and no old page.

`src/gui/main_tabs/main_window_surface.py` — the tab's name

```
from ..paper import paper_trading_tab_surface
...
PAPER_TAB = paper_trading_tab_surface.HEADING
```

The paper log row gains the book and the fill

`paper_row` in `src/paper/paper_log.py` carries a `book` block, the `last`, `bid` and `ask` of the tick's read, and a `fill` block, every field of the `PaperTrade`, beside the gate half and the trade half. The log stays the only file a run writes, under `PAPER_ROOT`. Unit Q6 rebuilds the log on the bus.

What the run reading measured

Read off the real window in both builds, under `pdb`, over a scratch home holding a copy of the operator's `bot_state.json`, thirty-eight scrumming records on coinbase at 5m, every socket but loopback refused, and Q2's loopback stand-in extended with a tape: every product's 5m candles follow one recorded shape whose newest bar advances one bar per stand-in minute from the first candles read, the ticker's last is that bar's close with the bid 0.05 percent under it and the ask 0.05 percent over it, and a 15 s delay can be planted on one ticker answer. The two records started on the bar read `scrumming_interval_pct` 5.0 and `scrum_read_rate_min` 1, so the tape rose about eight percent over five bars in a zig-zag and fell about fourteen over the four after. On the commit before this unit, in both builds, Start Paper Run wrote *"Start Paper Run: no paper run is built; nothing started."*, no thread started, `BUDGET_MULTIPLE` read 2.0, the fleet budget read $7,396.91, twice the target, and `apply_scrum` filled at the price it was handed with no bid and no ask in its signature.

```
reading                                    Qt                          React
Import Live Fleet, rows                    38                          38
Start on two rows through the command bar  both running                both running
thread after Start Paper Run               paper-run                   paper-run
the started line                           2 of 38 running,            2 of 38 running,
                                           target $3,698.46            target $3,698.46
ledger at the press, Spendable and Locked  3,698.4572 each             3,698.4572 each
the run button while up                    Stop Paper Run, 4 seats     Stop Paper Run, 3 seats
ticks in the first minute, two bots        24 (one per bot per 5 s)    24
worked ticks in the first minute           4 (one per bot per 60 s)    4
first scrum, A34/USDC, bar 51              at 725.294446 = the bid;    the same
                                           last 725.657275
first scrum, A32/USDC                     at 929.390286 = the bid     the same
scrum units against scrum_units on the     0.00241917, equal;          equal, equal
log row's delta, bid and rule              0.00188792, equal
scrum fee against 0.6% of the notional     0.01052768, equal, twice    equal, twice
strip after the scrums                     $3,701.95 / $0.00 /         the same
                                           $3,698.46 / $0.00 / 1
first fold, A34/USDC, bar 57               at 654.668976 = the ask;    the same
                                           last 654.341805
fold units against fold_units on the row's 0.00038187, equal;          equal, equal
usd, ask and rule                          0.00029801, equal
fold fee against 0.6% of the spend         0.0015, equal, twice        equal, twice
realized per fold, fold_surplus_usd        0.02434372, twice           the same
Realized on the ledger against the sum     0.04868745 = 0.04868745     the same
strip after the folds                      $3,701.44 / $0.05 /         the same
                                           $3,694.04 / $0.00 / 1
fills at last                              0 of 4                      0 of 4
a 15 s delay on one ticker answer          the answer waited 15.0 s;   15.0 s; 378 paints
                                           803 paints over 40 s        over 40 s
the control, the GUI thread held 3 s       0 paints                    0 paints
thread-violation lines                     0                           0
Stop Paper Run                             thread ended; 218 ticks,    the same
                                           20 worked, 2 scrums,
                                           2 folds, fees $0.0241,
                                           realized $0.0487
the run button after the stop              Start Paper Run             Start Paper Run
a planted zero-cash balance, a fold        filled $0.25; cash -0.2515  the same
a fill's price planted to last             read as not at the book     the same
bot_state.json across the four presses     unchanged                   unchanged
a planted byte on a copy                   hash moved                  hash moved
stand-in calls over the run                79: 1 products, 58 ticker,  79, 0.139 a second
                                           20 candles; 0.14 a second
sockets opened outside loopback            0                           0
```

The stand-in's tape holds fewer than one hundred bars, so the adapter's candle cache never held enough rows and every worked tick read a fresh window; on the real venue a 5m window holds one hundred bars and is served from the cache for 300 s. The scratch copy of `bot_state.json` was rewritten during the run by `main.py`'s `periodic_save`, the live application's own sixty-second state save under the scratch home; no file under `src/paper/` or `src/gui/paper/` writes it.

One line for another item: no element on the React page, Live's `src/gui/web/exchange_tab.js` or the forks, calls the page's own `selectScrumRow`, so a row press selects nothing for the command bar and only a Detail or Fire press selects the row on the model; the reading selected the row through the venue model's `select_scrum_row` and pressed the page's own Start.

What the bundle reading measured

Both builds were made from this unit's commit and launched twice each, `HOME` and the profile variables on a scratch home holding a copy of the operator's `bot_state.json` and no credential file, each press an accessibility Invoke or a button message posted to the launched tree's own window, the window captured by its handle. In a Windows AppContainer with no network, Import Live Fleet made 39 calls in each build, every one refused by the container; a Detail press on two rows, the window it opened closed, and Start on the bar moved the two records to running; Start Paper Run started the runner and turned the seat to Stop Paper Run; over about 97 s the runner made 8 refused reads, 4 tickers and 4 candle windows, and wrote 4 paper rows each carrying *"The live feed answered no candle."*; the strip read $3,698.46 for Spendable and Locked; Stop Paper Run ended the runner and the seat read Start Paper Run again.

The one real reading, both bundles against the public host alone, no credential file, the adapter's own pace, the two same records running:

reading	Qt bundle	React bundle
Import Live Fleet	39 calls, 917 products, 38 of 38 tickers	the same
ticks over the run	52 in 128 s	the same shape
worked ticks	6	6
venue calls during the run	8: 6 FETCH_TICKER, 2 FETCH_OHLCV of 100	8: 6 FETCH_TICKER, 2 FETCH_OHLCV of 100
least gap between two calls	0.258 s	0.257 s
calls a second over 122 s	0.066	0.066
the A34/USDC book on one read	last 0.221450, bid 0.221420, ask 0.221430	last 0.221840, bid 0.221760, ask 0.221840
paper rows written	6, 3 per bot	6, 3 per bot
fills	0, none expected	0
strip while running	$3,698.46 / $0.00 / $3,698.52 / $0.00 / 1	$3,698.46 / $0.00 / $3,698.46 / $0.00 / 1
Stop Paper Run	52 ticks, 6 worked, the seat read Start	the seat read Start Paper Run again
thread-violation lines	0	0

The 100-bar window the venue answers is served from the adapter's cache for 300 s, so the second worked tick of each bot read a fresh ticker and no candles. Locked moved with the venue's last price on the Qt run, from $3,698.46 to $3,698.52, and read $3,698.46 on the React run, whose two prices sat within a cent of their openings. The Activity Log's lines are not readable through UI Automation's text pattern on the React page, so the bundle's own `system.log` and the paper log carry the React reading. ## 2026-09-23 - #19 - The paper bot acts like its live equivalent

Unit Q4 of issue #19. A paper run now writes what it does into the held record: a scrum's tranches reach the record and Detail's Fold Tranches tab, a fold's surplus compounds into the target under the cycle cap, the Target Delta reads the ticker's last trade instead of the newest bar's close, and the row's Trades, Current Position Value and Target move within two seconds of a fill. A bot reads `RUNNING` while the runner holds it and `STOPPED` after Stop Paper Run.

The sentences unit Q4 overtakes

Three passages above describe the run as it was before this unit. Each is quoted here and kept above as it stands.

- "`tick` builds the `GateContext` through the Simulator's `tape_context` over the window's newest close" (Who evaluates and who sizes). `tick` now builds it through `paper_tape_context` in `src/paper/paper_run.py`, which prices the position at the ticker's last trade and leaves the candle window to the indicators, as Live's tick does.
- "A fold is planned under the cycle growth cap, as Live's `tick_phases` plans it, and the target's growth from the surplus is unit Q4's." (Who evaluates and who sizes). This is that unit: `apply_fold` calls `grow_target` after it settles the plan.
- "Q4 draws the tranches; this unit stores them" (What crosses to the GUI thread, and the Q3 comment on the issue). The tranches now reach the record on every snapshot, so Detail draws them mid-run.

The Target Delta reads the ticker

The Simulator's `tape_context` in `src/simulator/back_test.py` prices a position at `window[-1].close`, the only price a stone tablet carries. Live's `ScrummingBot.tick` prices it at the ticker's last trade every five seconds and reads the candle window for the indicators alone. Paper has both, so the Simulator's function is left untouched and `src/paper/paper_run.py` carries its own `paper_tape_context`.

`paper_tape_context` takes the tick's last trade for three fields only: `ticker_last`, the `delta` against the balance's grown `target_usd`, and `mem253_current_pos`. Every other field is unchanged: `bb_pos` comes from `bb_reading` over the window, the indicators from `VotingEngine.compute_all`, and `trend_reading` from the window itself. The same shared helpers size the figures, so only the price source differs.

`src/paper/paper_run.py` — the delta at the ticker

```
    last = float(ticker_last)
    ...
    position_usd = balance.value_usd(last)
    delta = target_delta_usd(position_usd, target_usd)
```

This matters on the real venue. `PaperExchange` caches a ticker for five seconds and a 5m candle window for three hundred, so before this unit the Target Delta could be five minutes old while the book moved. In one stand-in reading the gate priced a position at a bar close of 100.61803399 while the same tick's last trade read 101.83.

A fold compounds its surplus into the target

`apply_fold` settles its plan, books the bought units, and then calls `grow_target`, the Simulator's own function in `src/simulator/back_test.py`. It is imported, not forked: the growth step does not differ between a simulated bot and a paper bot by one figure, and the operator's rule for this issue is that trading logic is shared as pure code and only stateful shells are forked.

`grow_target` reads `fold_surplus_usd` over the consumed slices, holds it to what `cycle_growth_cap_usd` and `fold_cap_remaining_usd` leave of the cycle, applies `target_growth_applied`, adds the applied figure to `target_usd` and to `cycle_cap_consumed_usd`, parks the rest as standing surplus, sets the growth side to `lower`, and records the step on `target_path`. A record with `profit_folding_active` off grows nothing. The cap is `max_target_growth_pct` of the target per cycle, one per cent on the operator's records.

`reset_growth_cycle` runs at the top of every worked tick, before the context is built, so the growth cycle and the gates read the same bar. It is `growth_cycle_side` over the bar's band position: a cycle opened on the lower side zeroes its consumed cap when the band position reaches 0.75, and one opened on the upper side when it reaches 0.25.

`src/paper/paper_run.py` — the growth step inside the fold

```
    realized = fold_surplus_usd(units, slices, float(price))
    grow_target(bot, balance, units, slices, float(price), int(candle_ts_ms))
```

`FakeBalance` in `src/paper/fake_balance.py` gains the two fields `grow_target` writes, `growth_applied_usd` and `target_path`. Every `PaperTrade` now carries `target_usd_after`, the balance's target once the growth has been taken.

Current reading. The surplus a fold books is the units it bought less the units its scrum sold, priced at the fold. Live, Paper and the Simulator read it through one function, so all three book the same figure. A scrum of 0.1 units at $100 and a fold at $98, at a 0.6 per cent fee, book $0.14 of surplus in Paper and in the Simulator alike.

`src/trading/scrumming/sizing.py` — `fold_surplus_usd`

```
asset_at_scrum = sum(units_at_scrum(one) for one in slices)
return max(0.0, priced_usd(float(units_bought) - asset_at_scrum, float(fill_price)))
```

The run writes the record, and the row follows

`PaperRunner` gains one seam, `on_stats`, beside `on_trade` and `on_figures`. `post_stats` builds a `BotStatsSnapshot` through `stats_snapshot`, the Simulator's own builder, so a paper row and a simulated row carry the same keys. The snapshot goes out at three moments: once with `SNAPSHOT_START` and the state `running` on the tick a bot's balance opens, once per worked tick with `SNAPSHOT_TICK` or `SNAPSHOT_FILL`, and once per opened balance with `SNAPSHOT_END` and the state `stopped` when the loop ends.

Both hosts pass `on_stats=self.bot_stats.emit`, a fifth queued signal, so the write lands on the GUI thread in `_take_bot_stats`. That method calls `PaperFleetSource.write_stats`, which merges the snapshot's `stats` into the record's `stats` and its `scrumming_state` into the record's, then arms `_stats_redraw_timer`, a single-shot timer at `STATS_REDRAW_MS`, two thousand milliseconds, Live's own dashboard interval. A snapshot naming a record no longer held is dropped.

The row reads the record, so one write moves every cell that reads it: Current Position Value from `stats.position_value` and the lots, Trades from `stats.total_trades`, and Target from `scrumming_state.target_balance`, the grown target. The five cards on the header strip read the same records through `aggregate_stats`, so Trades, Scrummed and Folded move with the row.

`src/gui/paper/paper_trading_tab.py` — the record write

```
            self._fleet_source.write_stats(
                bot_id, snapshot.stats, snapshot.scrumming_state
            )
```

RUNNING while the runner holds the bot

A snapshot carrying the state `running` moves the record through `PaperBotManager.start` and redraws the rows at once; one carrying `stopped` moves it through `PaperBotManager.stop` and redraws at once. Each writes two Activity Log lines: Live's own ✓ `Bot <id> RUNNING.` or `Bot <id> stopped.`, and then `bot_opened_line` or `bot_ended_line` naming the units, the price, the fills and the target's two figures. The command bar keeps its own Start, Pause and Stop, and `PaperFleetSource.set_state` stays the only writer of a record's state underneath.

Detail draws the run's tranches

Detail's Fold Tranches tab is unchanged code. It draws `PaperBotView`, which reads `scrumming_state.fold_tranches` off the record, and the record now carries the tranches the run's scrums queued, so a Detail window opened during a run draws the open tranche. Live refreshes no open Bot Settings window on a timer and this adds none: the window draws what the record held when it opened.

What the live-equivalent reading measured

Both variants, the real `MainWindow` under `pdb` on a scratch home, every socket but loopback refused, against a loopback stand-in serving 1m bars and a ticker that moves inside each bar. Two records, targets $100 and $200, a 0.5 per cent scrumming interval, phantoms off.

reading	Qt	React
fills in one 215 s run	10	10
row Trades at +1 s after a fill	3	3
row Trades at +2 s after a fill	4	4
row Target before the first fold	$100.0000 / $200.0000	the same
row Target after two folds	$100.0454 / $200.0910	$100.0470 / $200.0942
Current Position Value after a fold	$97.7949 / $195.5608	$97.7313 / $195.4367
growth steps recomputed by hand	4 of 4 equal to 10 decimal places	4 of 4 equal
cycle cap on a $100 target	$1.00	$1.00
Detail Fold Tranches opened mid-run	2 rows	2 tranches
bar close held across two ticks	100.61803399	100.61803399
gate ticker_last across those two ticks	100.00780971 -> 101.90722988	100.00746202 -> 101.88644039
delta across those two ticks	0.00000000 -> 0.15219235	0.00000000 -> 0.10227326
cards: Trades, Scrummed, Folded	10, $5.1931, $5.1620	10, $5.2814, $5.2497
state while the runner held the bot	RUNNING	RUNNING
state after Stop Paper Run	STOPPED	STOPPED
venue calls, all loopback	93	93
non-loopback connections refused	0	0

The same run before this unit filled ten times and moved nothing: Trades stayed `0`, Target stayed `$100.0000`, the Target Delta read the bar's close while the ticker stood 1.8 per cent away, Detail drew no tranche, the cards read zero, and both rows still read `running` after Stop.

Two plants, each of which made a reading fail. With the redraw interval planted to sixty seconds the row held its old cells at +1, +2, +3 and +4 seconds after a fill, and a forced `_redraw_stats` then moved Trades from 3 to 5, so the instrument would have seen the move. With `profit_folding_active` planted off, four folds realised $0.022 to $0.047 of surplus each and the target stayed exactly $100.00000000 and $200.00000000 with an empty `target_path`.

What the bundle reading measured, against the venue

The Qt bundle, built from this branch and launched in isolation: a scratch home with no `bot_state.json` at launch and no credential file at any point, the window driven through the .NET UIAutomationClient assembly, every press an `InvokePattern` invoke or a `SelectionItemPattern` select on an element found under a window matched by its own process id. Two records on pairs the venue lists, `A15/USD` at a $100 target and `A14/USD` at $200, written to the scratch file after launch so no bot is ever restored.

```
moment                                    reading
-------------------------------------------------------------------
Import Live Fleet                         915 product(s) trade on coinbase;
                                          2 of 2 fleet products among them;
                                          2 of 2 tickers answered in 3 calls
row after Import                          paperlive1 A15/USD, Trades 0,
                                          Target $100.0000
Start on the first row                    "✓ Bot paperlive1 RUNNING."
Start Paper Run                           the runner thread up, Bots card 1
Current Position Value at 52 s            $99.9882
                       72 s               $99.9868
                       92 s               $99.9628
                      112 s               $99.9428
                      132 s               $99.9645
                      152 s               $99.9623
Ammo across the same marks                $0.0118 to $0.0377
LOCKED on the header strip                $299.99 down to $299.94, back to $299.97
SPENDABLE / REALISED / MATURE             $300.00 / $0.00 / $0.00
cards: Scrummed, Folded, Trades, Errors   $0.00, $0.00, 0, 0
the A14 row's Target A15 cell             0.002371 then 0.002372
after Stop Paper Run                      $99.9694, Trades 0
bot_state.json across every press         575DED46ED9CB7B7, unchanged
the planted-byte control                  B7821D9854A0EB0F, a different digest
```

No fill was expected inside two minutes and none came; the position value is the live book moving under a fixed holding, which is what a two-minute reading shows.

The call ledger, read off the API Interaction Log pane, carries 28 calls over 150 seconds:

```
13:03:23  FETCH_MARKETS                     Import Live Fleet
13:03:23  FETCH_TICKER  A15/USD             the same press
13:03:23  FETCH_TICKER  A14/USD             the same press
13:03:54  FETCH_TICKER                      the run's first worked tick
13:03:55  FETCH_OHLCV   100 bars at 5m      the same tick, the only one
13:04:00  FETCH_TICKER                      then one every five seconds
   ...                                      to 13:05:53, 24 of them
```

One candle read serves the whole run, because the adapter holds a 5m window for three hundred seconds; the ticker is read once per five-second tick, the cadence `ScrummingBot.tick_interval` sets. That is 0.19 calls a second, and the adapter's own `PUBLIC_MIN_INTERVAL_S` paces them.

Two controls could not be pressed, and the routes are named. The command bar starts the first row and no other: `PaperBotStatusTable.get_selected_bot_id` reads both `selectedItems()` and `currentRow()`, and only the first row's cell answers `HasKeyboardFocus=True`. Four attempts on the second row each reported `IsSelected=True HasKeyboardFocus=False`, and the Activity Log answered `Select a bot first.` or repeated the first bot's line. On the React bundle the Paper panel never reaches the accessibility tree at all: the Paper tab item answers `invoked` to an `InvokePattern` and `selected` to a `SelectionItemPattern`, and no child window contains the tab's own rect, so there is nothing to post a button message to. The window's descendant count stays at 225 through all three routes. Unit T1b of issue 34 read the same wall on the same bundle, and the React readings on this page stand on the source-tree React build.

2026-09-24 - #858 - the bot list upgrades reach the Paper Trader

His line closes the bot-list list on that item: "Transpose these upgrades to Sim and Paper once completed." Three upgrades are in it. Columns that sort. Labels that wrap, with the privacy button centred beneath each one. A row press that moves the Indicator Voting Panel to that bot. All three now draw on the Paper Trader's own Scrumming Bots table, in both builds, over the paper fleet's own figures.

The paper columns sort over the paper fleet's own price

A press on a column heading orders the rows by that column. A second press on the same heading reverses the order. The order is recomputed on every rewrite, so a figure that keeps moving does not unsort the list. The Paper Trader orders by the row's own reading and never by the live price pool, because the table and the page model each answer their own lookups.

`src/gui/paper/paper_bot_status_table.py` - the paper fleet's own sort readings

```
        def _sort_lookups(self) -> SortLookups:
            """``order_statuses``' two readings, over the Paper Trader's figures."""

            def price(status) -> tuple:
                stats = status.get("stats", {}) or {}
                return self._display_price(status, stats)

            def denom_text(
                quote: str, base: str, exchange_id: str, target: float
            ) -> str:
                return self._denom_cell(quote, base, exchange_id, target)[0]

            return SortLookups(price, denom_text)
```

Read off the running program, both builds, at 700, 900 and 1400 pixels wide, over the 38 rows the paper fleet file holds:

reading	window	page
columns that move on a press	9 of 10	9 of 10

reading	window	page
a second press reverses	7 of 9 exactly	yes
the order survives a fresh fleet	yes	yes

The Detail column holds one identical button a row, so it carries no value to order by and is the one column a press does not sort.

The paper labels wrap and the privacy dot sits beneath each one

Each of the ten labels sits inside its own column, over as many lines as it needs, with one blue dot centred under the nine that carry a mask. The label steps down one size when its longest word will not fit the column. A press on the dot masks that column; a press on the words sorts it.

width	Current Position Value, window	the page	dots	largest offset
700	3 lines in 67 px, size 10	46.2 px over a 15.4 px line	9	0.5 px
900	3 lines in 92 px, size 11	30.8 px over a 15.4 px line	9	0.5 px
1400	3 lines in 154 px, size 11	15.4 px over a 15.4 px line	9	0.008 px

No label is cut at any of the three widths, in either build. The tenth column draws no dot and keeps the same empty dot row, so every label sits on one line.

A row press moves the Paper Trader's own Voting Panel

A press on a row draws that bot on the Paper Trader's panel. A second press on the same row takes the panel to its empty state. A press on another row moves the panel to that bot. The panel's own dropdown still picks a bot, and the row it names highlights itself in the list. The panel is the Paper Trader's own, never Live's.

`src/gui/paper/paper_trading_tab.py` - the one bot both sides read

```
        self._bot_list_link = BotListPanelLink(
            self._indicator_panel, self._bot_list_hosts
        )
        self._indicator_panel.bot_selected.connect(self._bot_list_link.panel_moved)
```

Read on the running program, both builds, at all three widths: a first press puts the pressed row's bot on the panel, a second press on that row answers no bot, and a press on another row answers that row's bot.

The paper table is Live's table with three readings of its own

The Paper Trader's window table was a copy of Live's, taken before the header was rebuilt. It is now Live's table with the Paper Trader's three readings over it: the display price, the Target-denomination cell and the sort lookups. The page module carries the same relationship to Live's module, and the model already read the shared surface.

`src/gui/paper/paper_bot_status_table.py` - the row's own price

```
        def _display_price(self, status: dict, stats: dict) -> tuple:
            """The row's own ``current_price``, at ``PAPER_PRICE_AGE_S``."""
            del status
            return float(stats.get("current_price", 0.0) or 0.0), PAPER_PRICE_AGE_S
```

An empty paper fleet still draws the header

With no row at all the header still draws its ten labels and its nine dots, a press on a heading changes nothing and raises nothing, and a press where a row would be finds no row.

reading, no rows	window	page
labels drawn	10	10
dots drawn	9	9
a heading press	no error, still no row	no error, still no row
a row press	answers no bot	finds no row

2026-09-24 - #571 - the asset class group and Add Exchange reach the Paper Trader

The class group is the window's, and the Paper tab draws it

Functional. The segmented asset class group is part of the header strip at the top of the window, not part of any one tab. It is on screen while the Paper tab is in front, and a press on a class button reaches the Paper tab like every other.

Read off the running window with the Paper tab in front, in both builds, at 700, 900 and 1400 pixels wide:

```
Crypto   Stock   Commodities   Forex
```

All four buttons sit inside the window at every one of those three widths. The group holds 142 pixels whatever the window gives it, and each button shortens its own name to the room it has, so a narrow window elides a label and never pushes a button off the right edge.

One class for the whole window. The Paper tab holds no second class of its own. The window keeps the active class, the settings file stores it for the next launch, and the window tells the Paper tab which class that is.

`src/gui/main_tabs/class_filter_tab.py` — the list that reaches Paper

```
SELF_COUNTING_TABS = ("_history_tab", "_simulator_tab", "_paper_trader_tab")
```

OVERTAKEN, quoted whole:

`src/gui/main_tabs/class_filter_tab.py` — *the table that reaches Paper*

```
SELF_COUNTING_TABS = {
    "History": "_history_tab",
    "Sim": "_simulator_tab",
    "Paper": "_paper_trader_tab",
}
```

True today: the tab titles were keys only the removed note needed, so the table is a plain list of the three window attributes. `_refilter_class_tabs` reads it and hands the sector to each tab's own `set_asset_class`.

Paper's Add Exchange button follows the active class

Functional. The Paper tab now carries the Add Exchange button in the two seats Live gives it: the right of the venue sub-tab row, and the Get Started card. It reads the class the header strip has active, and it sits before Import Live Fleet and Start Paper Run in both seats.

`src/gui/paper/paper_trading_tab_surface.py` — the seat both builds read

```
def add_exchange_seat(asset_class: Any = None) -> dict:
    from ..main_tabs import asset_class_surface as acs

    key = acs.normalise(asset_class)
    return {
        "action": ADD_EXCHANGE_ACTION,
        "text": acs.add_exchange_label(key),
        "tooltip": acs.add_exchange_tooltip(key),
        "enabled": acs.add_exchange_enabled(key),
    }
```

Read for each class in turn, on the running program, in both builds:

class	the button reads	it can act
Crypto	■ `Add Crypto Exchange`	yes
Stock	■ `Add Stock Broker`	yes
Commodities	`Commodities — no venue yet`	no
Forex	`Forex — no venue yet`	no

A class with no venue says so here too. The button states the class, its tooltip names what is missing, and the press is refused. It never offers another class's venue list.

Both builds draw the same button. The words, the tooltip and whether the button can act come from one place, so the Qt widget and the page cannot drift. Read at 700, 900 and 1400, four classes, two seats: 24 readings a build, and every value the same on both sides.

What the Add Exchange press does on the Paper tab

Functional. Paper never opens the live venue settings. A venue page exists on Paper once the paper fleet holds a bot on that venue, so the press offers the venues of the active class that the live fleet file names, and seats the one chosen.

`src/gui/paper/paper_trading_tab.py` — the handler

```
key = acs.normalise(self._asset_class)
if not acs.add_exchange_enabled(key):
    self._status_log.log(acs.class_state(key)["note"], "warning")
    return
self._import_live_fleet(key)
```

Driven once per class on the running program, with the live fleet file naming Coinbase and nothing else. The Activity Log line and the venue count after the press:

class pressed	the Activity Log line	venues seated
Crypto	`Imported 38 bot(s) from bot_state.json on coinbase.`	1
Stock	`bot_state.json names no Stock venue to add.`	unchanged
Commodities	`No configured venue serves Commodities yet.`	unchanged
Forex	`No configured venue serves Forex yet.`	unchanged

Both builds write the same four lines. No venue was contacted: every non-loopback connection was refused during the run, and the live fleet file hashed the same before and after.

The Paper tab keeps its own card while no venue serves the class

Functional. The Paper tab keeps its own page under every sector, served or not. No sector replaces it, hides it or disables it. `src/gui/main_tabs/class_filter_tab.py, in _refilter_class_tabs` hands the sector to the tab's own `set_asset_class`, which narrows the rows the tab lists and nothing else. A sector Paper holds no venue for draws the same page with its Get Started card on it, and the Add Exchange button on that card names the sector.

Read on the running window, one bot standing in for a fleet on Crypto and none on the other five sectors. The cell counts the controls on the page the Paper tab holds, every one enabled and none hidden:

sector	controls on the Paper page
Crypto	20
Stock	20
Commodities	20
Forex	20
Indices	20
Futures / Perps	20

OVERTAKEN, quoted whole:

> ***Functional.*** *The class filter replaces a tab with a note when the tab holds nothing for the class. The Paper tab is left its own card instead while it holds no venue of that class, so the Add Exchange button on that card stays on screen and names the state. That is the rule the Live tab already follows.*
>
> `src/gui/main_tabs/class_filter_tab.py` — *the gate*

```
tab = getattr(self, attr, None)
ask = getattr(tab, "class_venues", None)
if not callable(ask):
    return False
return not ask(key)
```

> *Read on the running window, before and after one Coinbase venue is seated:*

class	venues Paper holds, none seated	keeps its card	after Coinbase	keeps its card
Crypto	none	yes	Coinbase	no
Stock	none	yes	none	yes
Commodities	none	yes	none	yes

class	venues Paper holds, none seated	keeps its card	after Coinbase	keeps its card
Forex	none	yes	none	yes

> *Once the class has a venue on Paper, the note takes over again. A tab that does not answer which venues it holds is untouched by this: the History tab and the Simulator tab draw their notes exactly as before, read on the running window at all three widths under all four classes, 134 values compared and none of them moved but Paper's own.*

True today: the gate is gone. `_tab_keeps_card` is removed from `src/gui/main_tabs/class_filter_tab.py`, and no tab needs an exemption, because no tab is ever replaced. Paper is no longer the one tab that keeps its page — the History tab and the Simulator tab keep theirs on the same rule, and so does every other filtered tab.

One venue list, read by every consumer

The nine broker ids that route a venue to the stock layer were written out four times. The Paper tab held the fourth copy; it now reads the one name the venue map declares, as the Live tab does.

`src/gui/paper/paper_trading_tab.py` — the venue ids

```
self._equity_exchange_ids = acs.EQUITY_VENUES
```

Three sentences this entry overtakes

The manual carries these sentences about the Paper Trader's corner. Each is quoted as it stands, with the sentence that is true today beneath it.

> *"The corner Live gives* ■ `Add Crypto Exchange` *holds Import Live Fleet and Start Paper Run, each at Live's corner-button width."*

The corner holds three buttons: Add Exchange, Import Live Fleet and Start Paper Run, each at Live's corner-button width.

> *"The Get Started card holds the same two where Live's card holds its add button."*

The Get Started card holds the same three, and Live's add button is among them.

> *"The corner holds Import Live Fleet and Start Paper Run in Live's corner-button chrome, and the Get Started card holds the same two at Live's card-button size."*

Both seats hold Add Exchange first, then those two, in the same chrome and at the same sizes.

2026-09-25 - #19 - a paper bot's settings reach its stored record

Apply Changes writes the paper record

The Paper Trader's Bot Settings window drew every field a live bot's window draws, and Apply Changes refused each one. A paper bot could therefore only ever be a copy of a live bot, which left no way to prove a variant before real money touched it. Apply Changes now writes each pending field into the bot's own stored record, and the paper fleet file keeps it.

`src/paper/fleet_source.py` — the write

```
def set_config_field(self, bot_id: str, field: str, value: Any) -> Any:
    wanted = str(bot_id)
    record = self._records.get(wanted)
    if not isinstance(record, dict):
        raise KeyError(f"no paper record is held under {wanted!r}")
    name = str(field)
    if name in RECORD_FIELDS:
        record[RECORD_FIELDS[name]] = value
        return value
```

A field the config does not declare is still refused

The write accepts the fifty-four fields the bot config declares, plus the three the record holds outside it — the phantom switch, its timeframes and the lock count. Any other name is refused, and the window says so on the same line it uses for an applied change.

`src/gui/paper/paper_bot_live_settings_surface.py` — the two lines

```
CHANGE_REFUSED_FORMAT = "Refused {count} change(s) — the Paper Trader sends nothing"
CHANGE_APPLIED_FORMAT = "Applied {count} change(s) to the paper record"
```

What the driven window reported

Both readings came from the real window, with the venue's market read stood in for by a recorded answer. The target balance was changed from twenty-five dollars to seventy-seven dollars fifty, and the row, the record and the file all followed it.

```
no writer   Refused 1 change(s) — the Paper Trader sends nothing
            record 25.0   row 25.0
writer      Applied 1 change(s) to the paper record
            record 77.5   row 77.5   paper fleet file 77.5
React page  Applied 1 change(s)   bot rebuilt at 77.5
```

Creating a paper bot is still the one way in

The venue page's new-bot button writes a line naming the way in, because no paper bot wizard is built. A variant is made by importing the live fleet and changing the imported bot's settings.

`src/gui/paper/paper_trading_tab_surface.py` — the line that button writes

```
NEW_BOT_FORMAT = (
    "+ New Bot on {exchange}: the Paper fleet is loaded through "
    "{way_in}; the paper bot wizard is not built."
)
```

THE FORM THE WIZARD REPLACED. The block above stays here, because a reader who saw that line needs to find it. That constant left the tree. The press now reaches a handler that opens the wizard and hands its settings to the paper fleet source.

`src/gui/paper/paper_trading_tab.py` — the handler that replaced it

```
        wizard_class = surface_class(PAPER_BOT_WIZARD)
        exchanges = wizard_surface.wizard_exchanges(self._exchange_tabs, self._exchange)
        defaults = wizard_surface.stored_defaults()
        markets = wizard_surface.venue_markets(self._exchange)
        wizard = wizard_class(exchanges, defaults, self, markets=markets)
```

2026-09-26 - #19 - a paper fill draws its own Activity Log line

A paper run held each fill and drew none of them. The pane beside the bot table carried the run's start and end and each bot's mark, and a reader watching a run fill ten times read nothing about any of the ten. A fill now draws its own line on the Paper Activity Log, on both hosts.

A fill draws the moment it happens

The line takes the same trade shape the Live Activity Log uses, and it carries the market, the side, the size and the price. The size reads to six decimals and the price to eight, which is what Live's own trade line reads.

`src/gui/paper/paper_trading_tab_surface.py` — the line one fill writes

```
FILL_LINE_FORMAT = (
    "{prefix} {role}{split}{symbol}{split}{stage}"
    "{extra_split}{units:.6f} @ ${price:.8f}"
)
```

The line carries the fill's own time

The stamp is the fill's own moment, never the moment the pane drew it. The runner stamps each fill on its own thread, and the line reads that stamp back instead of asking the clock.

`src/gui/paper/paper_trading_tab_surface.py` — the stamp

```
def fill_stamp(trade: Any) -> str:
    """The stamp a fill's Activity Log line carries: ``TIMESTAMP_FORMAT`` over the
    trade's ``wall_ms``, the fill's own time rather than the clock."""
```

Each record names the side in its own words

One fill leaves two records, and each names the side in its own vocabulary. The pane line says SCRUM or FOLD. The stored row says SELL or BUY. Neither is a drift, and the reason is a different reader at each end. The pane line has to match Live's trade line, because the glyph table keys on Live's two role words. The stored row has to match the reader that reads a Coinbase fill back, because Validation reads a paper row through the reader it reads a live row through.

The paper log module records that reason itself. Its own docstring says the two sides come from the stored-trade module, and the paper log spells neither of them.

`src/paper/paper_log.py` — the two sides, in the stored reader's words

```
#: A scrum sells and a fold buys, in the vocabulary ``YtdTrade.side`` uses.
SIDE_FOR = {SCRUM: SIDE_SELL, FOLD: SIDE_BUY}
```

The pane's two role words come from the table the glyph reads, so a line's role and its glyph cannot disagree.

`src/gui/paper/paper_trading_tab_surface.py` — the pane's two role words

```
FILL_ROLES = {SCRUM: SIDE_ROLES[SELL_SIDE], FOLD: SIDE_ROLES[BUY_SIDE]}
```

What the driven composer answered

One pass drove both functions over two paper trades, a scrum and a fold, with the two fill times five minutes and thirty-three seconds apart.

```
08:22:03 TRADE NOTIFICATION: SCRUM: A15: FILLED — 0.000153 @ $65432.10000000
08:27:36 TRADE NOTIFICATION: FOLD: A15: FILLED — 0.000154 @ $64980.55000000
```

The two stamps hold that gap although one pass drew them both. A stamp read off the clock would have put both lines on the same second, which is the reading that tells the two apart.

2026-09-26 - #19 - a paper bot is created from scratch

The Paper tab could hold a fleet only by copying the live one. Its new-bot button wrote one Activity Log line naming Import Live Fleet and opened nothing. That button now opens the Paper Trader's Bot Creation Wizard on both hosts, and the settings it collects reach `PaperFleetSource.create`, which had been in the tree with no caller at all.

The wizard's shape is the Simulator's, which already had one on both hosts against the same contract. One module holds Paper's own readings, and both hosts read it.

`src/gui/paper/paper_bot_wizard_surface.py` — the request the React fork asks

```
METHOD = "paper_bot_wizard.state"
```

The press that opens the wizard

The route is the same on both hosts and starts on the tab bar. Press Paper, press the venue sub-tab, then press the button beside the bot tables. The Qt tab opens the wizard at once. The React tab opens it one turn later, because a web view opened inside another page's own callback never finishes loading.

`src/gui/paper/paper_exchange_tab.py` — the button the route starts at

```
self._add_bot_btn = QPushButton("+ New Bot")
```

Each host draws the wizard from one registry line

One screen name carries both loaders, as every other forked screen does. The Qt loader answers a five-page window and the React loader answers the dialog that draws those pages in a browser.

`src/gui/variant_surface.py` — the screen and its two loaders

```
register(PAPER_BOT_WIZARD, _qt_paper_bot_wizard, _react_paper_bot_wizard)
```

The pair list carries the venue's own last price

The venue's product list is what the asset page offers. A product record holds a last price and carries neither a volume nor a volatility, and the page orders the list by name rather than by size. Each row carries that price in the currency the pair prices in, because a pair against Bitcoin prices in Bitcoin and not in dollars. A price at one unit or more reads to two decimal places and a smaller one reads to eight with its trailing zeros cut. A row the venue priced at nothing carries no price at all.

`src/gui/paper/paper_bot_wizard_surface.py` — the label one row carries

```
def price_label(price: Any, quote: str) -> str:
    """The last-price tail an asset row carries, empty when ``price`` is not a
    finite number.

    A price of ``WHOLE_UNIT_PRICE`` or more prints to two places and a smaller
    one to ``SMALL_PRICE_PLACES``, with the trailing zeros cut; the quote is
    named because a row on ``A14/A15`` prices in A15, not in dollars.
    """
```

The wizard's settings become the paper record

A finished wizard hands one settings map to the paper fleet source. The record built from it takes the shape a stored live record has, its state idle, and its own mark naming the wizard as its source rather than a copy. Every reader of that mark answers for a made bot as it answers for a copied one, and the command bar starts and stops it the same way.

`src/paper/fleet_source.py` — the call the wizard's Finish reaches

```
    def create(self, config: dict) -> PaperBot:
        """Hold one record built from the bot wizard's config through
        ``wizard_record`` and answer its ``PaperBot``, its ``bot_id`` the first
        ``BOT_ID_LENGTH`` characters of a ``uuid4`` as a live bot draws its own.
        """
```

What the driven wizard answered

One pass drove each host at three pane widths, with the venue's product read stood in for by a recorded answer, and read the made bot back off the real fleet source beside a copied bot the same pass had loaded.

```
Qt      5 pages    A14/USD made, marked as made here
                   A23/USD held, marked as copied
React   8 of 8 page scripts, 5 page stops, 2 mode rows
                   A14/USD made, marked as made here
both    49 of 49 settings agree, none differs
labels  A15  (64,231.07 USD)   A14  (2,410.55 USD)   A18  (0.00004312 USD)
status  3 USD pairs, 3 carrying a last price
widths  700   900   1400   every control inside the pane
```

Three sentences and one heading this entry overtakes

Each is quoted here and kept above as it stands, with the sentence that is true today beneath it.

> *"A press on + `New Bot` writes one Activity Log line naming Import Live Fleet as the fleet's way in; the paper bot wizard is a later unit's."*

A press on that button opens the Bot Creation Wizard, and its Finish creates the bot.

> *"The venue page's new-bot button writes a line naming the way in, because no paper bot wizard is built."*

That button opens the wizard, and the Activity Log carries the opening line and then the created bot's own line.

> *"A variant is made by importing the live fleet and changing the imported bot's settings."*

A variant is made either way: by changing an imported bot's settings, or by the wizard on a pair no live bot trades.

> *"Creating a paper bot is still the one way in"*

Creating a paper bot is a way in of its own, standing beside Import Live Fleet.

Two code blocks this entry retires

Both blocks name a constant the tree no longer holds. Each stays where it stands, marked as the form the wizard replaced, with today's code beneath it. The two sites are the corner-button passage and the settings-record entry above.

`src/gui/paper/paper_trading_tab_surface.py` — what neither block names any more

```
CORNER_BUTTONS = (
    (IMPORT_LIVE_FLEET_ACTION, IMPORT_LIVE_FLEET_TEXT),
    (START_RUN_ACTION, START_RUN_TEXT),
)
```

2026-10-04 - #1103 - a pushed candidate spawns paper bots

The Market Inspector draws a Push to Paper button beside Push to Sim, on an open Opposing Trades entry and in the topology preview. One press spawns one paper scrumming bot per market the candidate names and registers the wires between them.

The operator's ruling:

> *"For Paper Trader, the pushed topology must spawn the corresponding paper scrumming bots. Paper tab does not have multiple run modes. It pretends to trade against actual exchange data and records the result."*

"If the wiring does not travel then its not a complete or valid topology push."

Paper has no run mode, and none is added

The Simulator holds a candidate under its Back Test mode and leaves that mode in force. Paper has no mode to land a candidate under and gains none here. A pushed bot is simply held, idle, the way a bot the wizard created is held, and Start Paper Run works it against the venue's own data.

`src/paper/fleet_source.py` — the push, on the fleet source itself

```
def push_candidate(
    self,
    candidate: Any,
    exchange_id: str = "",
    ta_timeframe: str = "",
) -> dict:
```

The venue is Paper's own, and so is the timeframe

The Simulator resolves each pushed market to the venue its Stone Tablet was recorded on. Paper reads the venue live, so it needs no tablet: every pushed bot takes the venue `PaperExchange.venue` answers and the timeframe `BotConfig` declares as its own default.

`src/paper/fleet_source.py` — the timeframe is read off the dataclass

```
PUSHED_TIMEFRAME_DEFAULT = str(
    next(
        one.default
        for one in dataclass_fields(BotConfig)
        if one.name == "ta_timeframe"
    )
)
```

A pushed market keeps the quote its symbol names. `BTC/USD` becomes a bot on `BTC/USD` and `BTC/USDC` becomes a bot on `BTC/USDC`. A symbol naming no pair spawns nothing, and one market named twice spawns one bot.

The Target Balance is checked before a bot is held

A topology proposal that names no dollar figure reaches the push as a zero, and a zero takes the same 200 the Bot Wizard offers, read off `BotConfig`. Every other unusable figure refuses the whole push instead of holding a bot that cannot trade: a true or false in place of a dollar amount, an infinity, a not-a-number, a negative figure, and any value that is not a number at all.

`src/paper/fleet_source.py` — what one pushed market's Target Balance may be

```
def pushed_target_usd(value: Any) -> Optional[float]:
```

Driven on the real push, each of those five answered its own refusal with no bot held and no wire registered. Without the check an infinity and a not-a-number each became a bot whose Target Balance read zero, and a true became a bot trading against one dollar.

The wires travel with the bots

Each wire is registered on the fleet's own `PaperWireManager` between the bot ids the push created, and `hold_wire_manager` puts the rows on the fleet so the next `save` carries them in the paper fleet file.

`src/paper/fleet_source.py` — the wires that applied

```
def register_pushed_wires(manager: Any, wires: Any, bot_ids: Any) -> list:
```

A wire that cannot travel refuses the whole push. Five gaps do it: a wire end naming no market, both ends naming the one market, an end naming a market the push did not create, the one pair named twice, and a rate outside the range `register_wire` takes. Each refusal spawns nothing and registers nothing.

A wire row the push cannot read at all counts as naming no market, so it is refused rather than dropped. One pair named twice is refused for the same reason: a pair holds one rate, so keeping the second row would report two wires where the fleet holds one.

`src/paper/fleet_source.py` — the refusal for one pair named twice

```
PUSH_WIRE_TWICE_FORMAT = (
    "{source} to {target} is named twice, and one pair holds one rate"
)
```

The count the push reports is therefore the count the fleet holds.

`src/paper/fleet_source.py` — the four wire refusals

```
PUSH_WIRE_UNNAMED_TEXT = "a wire names no market at one of its ends"
PUSH_WIRE_UNHELD_FORMAT = "{source} to {target}: nothing was pushed for {asset}"
```

What the Activity Log carries

A push that lands writes the spawned line and, when the candidate carried wires, the wired line. A refusal writes its own line and nothing else moves. No line names a run mode.

`src/paper/fleet_source.py` — the line a landed push writes

```
PUSH_SPAWNED_FORMAT = (
    "Pushed {count} paper bot(s) on {venue}, trading the venue's own data: "
    "{assets}."
)
```

A press that reaches no Paper tab is refused by the window, which names the missing tab and spawns nothing.

`src/gui/paper/paper_trading_tab_surface.py` — the refusal with no tab

```
PUSH_NO_PAPER_TAB_TEXT = (
    "Push to Paper refused: the Paper tab did not build, so no paper bot was "
    "spawned."
)
```

A pushed bot is a paper record and nothing else

It reaches the paper fleet file alone. No live bot is created, no order is placed and the live engine and the live logs see nothing of it. A pushed wire is the same: it is registered on `PaperWireManager` and never on the live `SmartWireManager`.

What the driven push answered

The push was driven from the Inspector press through the window's handler to the Paper tab, with the venue read blocked at its edge. A two-market candidate carrying one wire between the two was the input.

read at the Paper tab	before this entry	after it
paper bots held afterwards	0	2
wire rows held afterwards	0	1

Both bots read `coinbase`, `1h`, `scrumming` and `idle`, and both were reached again through `PaperBotManager.get_bot`. The before column is the same script run against the tree as it stood, where the Inspector offers no Push to Paper at all.

One sentence this entry overtakes

It is quoted whole and kept where it stands, on the Market Inspector page, with the sentence that is true today beneath it.

> *"The Paper Trader has no mode a candidate could land under, so no push targets it."*

The Paper Trader still has no mode a candidate could land under, and a push targets it anyway: the candidate's bots are spawned on Paper's own venue rather than held under a mode.

A stored figure no box can hold draws a dash

The Paper Trader's Bot Settings tab and its creation wizard read every stored dollar figure through one admission before the control sees it. `src/trading/container/config.py, in as_box_figure` is that admission. It refuses six kinds: a stored flag, a text number, a not-a-number, an endless number, an integer too large for a decimal, and a finite figure outside the control's own span.

A refused figure leaves the box at the bottom of its span drawing the dash the indicator panel draws for a cell it cannot read. The log names the field. The dash clears the first time the operator moves the box. Never a zero.

`src/gui/paper/paper_settings_tab.py, in PaperSettingsTabMixin` takes `_show_admitted_figure` and `_clear_refused_dash` from Live's own `src/gui/live_settings/settings_tab.py, in SettingsTabMixin`, so Target Balance and Hedge Balance draw what the Live boxes draw. Three of the six kinds used to stop the whole tab from building; all six now build it.

`src/gui/paper/paper_settings_tab.py, in _refresh_target_denom_rows` reads the same admission, so a target the box draws as a dash reaches neither the Target-BTC row nor the Target-ETH row as money.

`src/gui/paper/paper_bot_wizard.py, in PaperTradingParamsPage` calls `src/gui/bot_wizard.py, in _show_stored_target` for the stored default target, and its `isComplete` answers False while that dash is drawn. Finish stays dark until the operator enters a figure, so no paper bot is built on a target nobody chose.

Apply writes nothing for a refused box. `src/paper/paper_bot_view.py, in set_config_field` is the only write path, and it is reached only by a control the operator moved.

The React build reads the same figures. `src/gui/paper/paper_react_bot_wizard.py` loads Live's own `src/gui/web/bot_wizard.js` and `src/gui/paper/paper_react_bot_detail.py` loads Live's own `src/gui/web/live_settings_tab.js`, and `src/gui/paper/paper_live_settings_tab_surface.py, in PaperLiveSettingsTabModel` is Live's model, so the page draws the dash inside the box as Qt draws it.

2026-10-18 - #1203 - one resolver answers a market's sector

Functional. Every tab that follows the asset class reads one chain, and the venue's own recording is the last word in it. A sector whose fleet holds a bot draws that bot. A sector whose fleet holds none draws the same tab with empty lists, and no tab replaces itself with a note.

Diagram: flowchart, left to right

- the class group press -> **set_active**
- set_active -> **bots_of_class**
- bots_of_class -> **bot_class**
- bot_class -> **market_class**
- market_class -> **symbol_class**
- symbol_class -> **the venue recording's asset_class**

`src/gui/main_tabs/class_filter_tab.py, in _apply_asset_class` starts the chain. `src/gui/main_tabs/class_filter_surface.py, in bots_of_class` narrows the fleet, and `src/gui/main_tabs/asset_class_surface.py, in symbol_class` answers the sector the recording holds for the symbol. A symbol the recording holds no sector for answers crypto. An unknown venue answers an empty string.

The sector each market answers

Driven on one venue serving all six sectors, with a recording holding one market per sector:

the recording's sector	`market_class` answers
crypto	crypto
stocks	stocks
commodities	commodities
forex	forex
indices	indices
futures_perps	futures_perps
no sector recorded	crypto

`bots_of_class` keeps 1 of 6 under each of the six sectors, both when the bot status declares its own `asset_class` and when the sector is resolved from the venue and the symbol.

What each tab draws under each sector

Driven through the real `src/gui/main_tabs/class_filter_tab.py, in _apply_asset_class`, with a fleet of six bots, one per sector:

tab	Crypto	Stock	Commodities	Forex	Indices	Futures / Perps
Sim	content	content	content	content	content	content
Paper	content	content	content	content	content	content
Live	content	content	content	content	content	content
Charts	content	content	content	content	content	content
Inspector	content	content	content	content	content	content
Swarm	content	content	content	content	content	content
History	content	content	content	content	content	content
Status	unfiltered	unfiltered	unfiltered	unfiltered	unfiltered	unfiltered
Console	unfiltered	unfiltered	unfiltered	unfiltered	unfiltered	unfiltered

The same six runs against an empty fleet draw nothing in every sector, so a `content` cell is a read and not a fixed answer.

The second resolver is gone

`src/gui/main_tabs/class_filter_surface.py, in market_class` carried a branch for a derivatives sector beside the recording. That branch could not run: `src/gui/main_tabs/asset_class_surface.py, in venue_classes` intersects its answer with the declared sectors, and `src/gui/main_tabs/asset_class_surface.py, in retired_onto` resolves `derivatives` onto crypto, so the name entered neither set. The branch, the `is_derivative` test behind it and its suffix list are removed, and `symbol_class` is the one resolver left.

The sentence above naming crypto is overtaken, and it is kept as written. `retired_onto` now resolves `derivatives` onto futures_perps.

A contract market still answers its own sector. The recording carries `futures_perps` on a perpetual row, which is the sector the taxonomy declares and the sector the Futures / Perps button filters on.

The Sim and the Paper tab build their pages per sector

`src/gui/paper/paper_trading_tab_surface.py, in build_view_model` and `src/gui/simulator/sim_trading_tab_surface.py, in build_view_model` each read `src/gui/main_tabs/trading_tab_surface.py, in layer_order` for their stack pages. Asked for all six sectors, both answer six pages, with the page on show running 0 through 5 as the sector moves. Both tabs are seated in both builds: the React build answers `SimTradingTabReact` and `PaperTradingTabReact`, and the Qt build answers `SimTradingTab` and `PaperTradingTab`. The bar holds nine tabs.

The Paper tab holds no sector list of its own, in either build. The browser build reads the one declared list at the moment the page is asked for, and writes two fields of its payload from that call: the page list, and the position of the page on show inside it.

`src/gui/paper/paper_trading_tab_surface.py` — both fields from one call

```
    key = live.layer_named(layer)
    order = live.layer_order()
```

The Qt build adds one stack page per name on the same list while the tab is built, and numbers the page on show from the same place, so the two builds cannot hold different sets of pages.

`src/gui/paper/paper_trading_tab.py` — one page per declared sector

```
        for layered in acs.layered_classes():
            page, bar, store, card = _make_layer(
                acs.display_name(layered), acs.accent(layered)
            )
```

The declared list is six sectors long, so the Paper tab answers six pages. Its own state object was asked for every sector in turn and the page list was read off the payload the tab serves. The page on show moves across all six asks, which is the control: the figure follows the sector asked and is not a fixed answer. A sector name no layer carries draws page 0, which is the crypto layer.

Sector asked	Pages	Page on show
Crypto	6	0
Stock	6	1
Commodities	6	2
Forex	6	3
Indices	6	4
Futures / Perps	6	5
a name no layer carries	6	0

The Simulator tab reads the same call and answered the same seven rows, figure for figure. The declaration is `src/gui/main_tabs/asset_class_surface.py, in layered_classes`, and the Simulator page says what a sector press does on its tab.

What a failed tab build still does

A tab whose build raises writes one warning line and leaves its window attribute unset. `src/gui/main_tabs/simulator_tab.py, in _build_simulator_tab` and `src/gui/main_tabs/paper_trader_tab.py, in _build_paper_trader_tab` each catch the refusal. Driven against a tab book that refuses the insert, beside the same book accepting as the control:

the tab book	warnings logged	tabs seated
refuses the insert	2	0
accepts the insert	0	2

Nothing reaches the screen for a refused build. The bar comes up short with no mark on it.

The Sim tab's own slot now carries the reason

The Sim tab answers differently today. `src/gui/main_tabs/simulator_tab.py, in SimulatorTabMixin._build_simulator_tab` seats `src/gui/main_tabs/simulator_tab.py, in BuildFailureNote` in the Sim slot when it catches, so the bar keeps nine tabs and the Sim page names the tab and what the build reported. The window attribute still stays unset, so the header strip, the Inspector's push and the sector filter all still read the tab as absent.

The Paper tab is unchanged. A refused Paper build still writes one warning line and leaves its slot empty.

2026-10-20 - #937 - a Paper row's state sits on its Symbol cell

The Paper Trader's Scrumming Bots table is a subclass of the window's own bot list, so its first column is Asset. It draws the target asset's own mark, and the asset's ticker where no mark is kept. It carries no bot id, no state colour and no state tooltip. The state colour and the mode-and-state tooltip sit on the Symbol cell. The Paper Extractor table keeps its own Bot ID column.

One sentence this page carried is overtaken

It is quoted whole, and the sentence under it is true today.

OVERTAKEN, quoted whole:

> *"A row's state is the Bot ID cell's colour and its tooltip's `State:` line; Live's ten columns hold no State column."*

True today: a row's state is the Symbol cell's colour and that cell's tooltip's `State:` line. Live's ten columns still hold no State column.

One reading above names the first column by its old name

The Import Live Fleet reading table holds the row `Import Live Fleet: Bot ID cell colour`, with `#888888` under Qt and `rgb(136, 136, 136)` under React. It is a recorded reading, not a sentence, so it stays exactly where it stands. The cell that carries that grey today is the Symbol cell.

Where the paper wizard's phantom defaults come from

The paper wizard's phantom page opens its tick box and its lock duration from the same two declarations the Live wizard reads. Neither figure is typed into the paper host.

`src/gui/paper/paper_bot_wizard.py` — the paper page falls back to the store's own declared figure

```
            self._stored_lock_candles = int(
                (defaults or {}).get("default_lock_candle_count")
                or AppSettings().default_lock_candle_count
            )
```

A figure set in Settings still wins, and a paper bot's own saved record wins over that, so an existing paper bot keeps the figure it was created with. The Trading tab page lists the Live wizard's rows under Phantom Bots rows.

All three wizard phantom pages were built and both boxes read off the built widgets. With nothing stored, each falls back to the declaration and opens on and 2. With off and 9 stored, each opens off and 9, so the boxes follow what is handed to them rather than holding a fixed answer.

Host	What was stored	Tick box	Lock box
Live	nothing	on	2
Paper	nothing	on	2
Simulator	nothing	on	2
Live	off and 9	off	9
Paper	off and 9	off	9
Simulator	off and 9	off	9

The three hosts therefore no longer need comparing.

2026-10-25 - #937 - the Paper page draws the Asset column's mark

The Paper Trader draws its Scrumming Bots table twice. The window draws one, and the page inside the window's browser draws the other. Both now draw the first column the same way. Where the logo library has put a file on disk for that asset, the cell draws that image at 32 pixels and holds no text. Where none is kept, the cell draws the asset's ticker, so the row still names what the bot accumulates. Where the asset database knows that asset's own organisation, the address rides on the cell and a press opens it.

`src/gui/web/paper_bot_status_table.js` — `AssetLogo`

```
  logoProps[PART_ATTR] = LOGO_PART;
  logoProps[LOGO_PATH_ATTR] = text(found[LOGO_PATH]);
  logoProps[LOGO_SIZE_ATTR] = text(found[LOGO_SIZE]);
```

Three invented bots, read off the drawn page

One asset has a kept file. One has none and is named by the asset database. One is named nowhere. Each reads differently from the other two.

```
  a file is kept             no text, one 32-pixel image, no address
  no file, the base is known the ticker, no image, the address on the cell
  the base is known nowhere  the ticker, no image, no address
  row height                 38 pixels, the same on all three pages
```

The page's own report of the payload

Each page checks the payload it was handed and lists what is missing. The three pages name the mark's four cell fields, so a payload that stops carrying one is reported rather than drawn as a blank square.

`src/gui/web/paper_bot_status_table.js` — the audited cell fields

```
  var CELL_FIELDS = [
    TEXT,
    COLOR,
    TOOLTIP,
    ALIGNMENT,
    ALIGNMENT_VALUE,
    LOGO_PATH,
    LOGO_IMAGE,
    LOGO_SIZE,
    LINK_URL,
    CHART_URL,
    UNDERLINE
  ];
```

Driven with every outward connection refused, the reading made no connection attempt and the page made no request. A control attempt afterwards was counted and refused, so the counter was able to report one.

The sentence above that this entry completes

It is quoted whole, and the sentence under it is true today.

Quoted whole:

> *"The Paper Trader's Scrumming Bots table is a subclass of the window's own bot list, so its first column is Asset. It draws the target asset's own mark, and the asset's ticker where no mark is kept."*

True today: that holds for the window's table and for the page, which draws the same mark from the same payload. Neither surface fetches anything while a row draws.

2026-10-25 - #665 - the paper record carries the wizard's lock figure

The paper wizard collected the Candles-to-lock figure and the record it saved held no key for it. The bot built from that record therefore had no lock holder at all, and the Phantom Bots tab fell back to the declared 2 whatever the operator typed.

`src/paper/fleet_source.py, in wizard_record` now writes the figure beside the two phantom keys it already wrote, clamped at 1, which is the clamp the live bot's own constructor applies.

```
    lock_candles = collected.get("lock_candle_count") if scrumming else None
    if isinstance(lock_candles, (int, float)) and not isinstance(lock_candles, bool):
        record["lock_candle_count"] = max(1, int(lock_candles))
```

`src/paper/paper_bot_view.py, in PaperBotView` reads that key to build the bot's lock holder, and the Candles-to-lock box on the Phantom Bots tab reads the holder.

```
wizard 7     record holds the key     tab box 7
wizard 2     record holds the key     tab box 2
no key       record holds no key      tab box 2
```

A paper record written before this change holds no such key, loads, and leaves the tab box on the declared 2.

2026-10-25 - #665 - the Paper wizard reads the one Hedge Balance figure

The Hedge Balance box on the Paper wizard used to open on a figure typed into the Paper wizard itself. It reads the bot's own declaration now, the same declaration the Live wizard, the Simulator wizard and the Electron wizard read.

`src/gui/paper/paper_bot_wizard.py, in PaperTradingParamsPage` — the opening figure

```
self._hedge_amount.setValue(BotConfig.hedge_balance)
```

The Paper despawn box takes the same route: the whole-days rule it reads is the one rule in `src/trading/container/config.py, in whole_despawn_days`, which the Paper Fold Tranches panel reads through the chrome it shares with Live.

Read before and after, the box opened on $200.00 both times. Moving the declaration to a figure nothing uses moved the Paper box with the other three; before the change the same move left all four at $200.00.

2026-10-26 - #665 - the Paper row names the fold taper by size and shows the detonation bar

The Paper fold reads the same taper schedule as Live, and that schedule's name changed. One statement above carries the old name. It stands as written, and this is the reading that replaced it.

written above	what it reads now
"`plan_fold_consumption`, `fold_rate_taper`, `fold_spend_usd`, `fold_units`"	The same four, with the schedule now named `fold_size_taper`. It multiplies the dollars a fold spends, which is what the Paper fold always used it for

`src/paper/paper_run.py, in fold_taper` — the Paper reading

```
return fold_size_taper(ratio_to_ceiling(value, ceiling))
```

The Paper row's status record also carries the detonation confidence minimum now, so the Fire tooltip on a Paper row names the figure a vote has to clear. The Paper table is the Live table with the Paper price feed, so one repair in the Live surface reaches both.

`src/paper/fleet_source.py, in row_status` — the key the Paper row carries

```
"detonation_confidence_min": bot.detonation_confidence_min,
```

Read on the real Paper page model and the real Paper widget, a record carrying 0.60 shows "at confidence 0.60 or above" and a record carrying the opening 0.75 shows 0.75. Before the change both records read the same line, with no figure in it.

2026-10-27 - #1125 - a pair with no recorded row answers no sector at all

The chain above ends at the venue's own recording, and its last step named a sector where the recording named none.

OVERTAKEN, and the sentence and the table row above are kept as written:

A symbol the recording holds no sector for answers crypto.

no sector recorded	crypto

Current reading: `src/gui/main_tabs/asset_class_surface.py, in symbol_class` answers the recorded sector, then the one sector the venue serves where it serves one, and an empty string where the venue serves several and the recording holds no row. A pair whose sector cannot be read is filed under no sector rather than under crypto.

the recording's sector	`market_class` answers
crypto	crypto
stocks	stocks
commodities	commodities
forex	forex
futures_perps	futures_perps
no row at all	nothing

Driven on one venue serving six sectors, with its own recording holding 2029 rows. Every recorded sector answers itself, before and after. The last row is what moved. A venue serving crypto alone and a venue serving stocks alone are the controls: each answers its own sector, and the two answers differ.

A Paper row carries no declared sector of its own, so `src/gui/main_tabs/class_filter_surface.py, in bot_class` resolves it from the venue and the symbol. A Paper bot on a venue whose recording holds its pair answers that pair's sector. A Paper bot on a pair no recording holds now appears under no sector button.

CITED AS ABSENT

The two paths below are named on this page only to record what the Paper tab drew before its rebuild. Neither opens on disk.

- `src/gui/web/paper_trader_tab.js` — the React panel the empty tab registered. The Paper page is `src/gui/web/paper_trading_tab.js` now, hosted by `src/gui/paper/paper_react_trading_tab.py`.
- `src/gui/web/paper_trader_tab.css` — the sheet that page loaded. The Paper page carries `trading_tab.css` and `exchange_tab.css`, named by STYLE_ASSETS in `src/gui/paper/paper_react_trading_tab.py`.

Back to the subsystem index.

Proof of Accumulation

Reference. The design is an on-chain competition layer where bots compete anonymously and the winners take a token on Base, which is Coinbase's layer two. Every instance holds a cryptographic identity, signs each trade into an append-only log, and publishes only a commitment to that log. The strategy stays private and the proof is public.

This is currently proposed as a concept but will likely require the building of a supporting blockchain team for proper / full implementation. This system is designed to enable users of Acervator to compete against each

anonymously via our own Proof of Accumulation blockchain. The idea is to convert trades executed into videogame metrics such as damage to a coliseum style monster or a fellow trader in a 1v1 face off. This further positions the platform as a surgical tool that can be finely tuned and customized to produce intense competition scenarios between entire groups of traders. This, of course, opens the door for actual tokenized Trading Guilds who may require their members to have a certain number of PoA tokens under their belt to join. There will be much more to follow on this as I do intend to scaffold it out for internal testing.

The tab the window does not build

The main window builds no Accumulation tab. The tab name sits in the canonical order and then in the skip list, so the reorder step drops it and the bar carries nine tabs, not ten. Dropping one name from that list is the one edit that puts the tab on the bar.

`src/gui/main_tabs/main_window_surface.py` — the skip list and what it leaves

```
UNBUILT_TABS = (ACCUMULATION_TAB,)

BAR_TAB_ORDER = tuple(name for name in CANONICAL_TAB_ORDER if name not in UNBUILT_TABS)
```

Read at runtime, `BAR_TAB_ORDER` answers Sim, Paper, Live, Charts, Inspector, Swarm, History, Status and Console. The window's own build step is guarded on the same list, so the method that would append the panel is never called.

`src/gui/main_window.py` — the guard, and the method it guards

```
            if ACCUMULATION_TAB not in UNBUILT_TABS:
                self._build_proof_of_accumulation_tab()
```

Read on the current tree, the guard hands the builder to the step timer, and the build reads the same guard. The window calls the builder only under it, so the build leaves the panel module the builder imports, `src.gui.react_proof_of_accumulation_tab`, out of a packaged build. The tab's own surface still ships, because the window's model imports it for the heading.

`src/gui/main_window.py, in MainWindow._setup_ui` — the guard as it reads now

```
            if ACCUMULATION_TAB not in UNBUILT_TABS:
                self._timed_build_step(
                    ACCUMULATION_TAB, self._build_proof_of_accumulation_tab
                )
```

Dropping the name from the skip list puts the panel back in the build as well. The rule is on 17-what-the-build-includes.md.

The Competition tab and the Local Testnet tab are not built either. One mixin assigns `None` to both attributes on every launch.

`src/gui/main_tabs/retired_tabs.py` — the two sentinels

```
        self._competition_tab = None

        self._testnet_tab = None
```

Three tab modules therefore exist and no tab draws. `src/competition/` is the engine behind all three: 47 modules, and the sections below describe what each one does today.

The package entry imports every module and lists its names, so each form is reachable by the package name alone. A name one module repeats from a module bound above it stays unexported, and reaching that one means importing its own module directly. Each module says in its own text which case it is in.

What the screen declares

One module declares the whole screen as data. It is registered in the bridge, so a renderer can ask for the shell by name even with no tab in front of it.

`src/gui/main_tabs/proof_of_accumulation_tab_surface.py` — the shell it declares

```
METHOD = "proof_of_accumulation_tab.state"

HEADING = "Accumulation"
BUILT = True
STATE_TEXT = (
    "The top row is halved: the current Vessel on the left, the Map or the "
    "Encounter on the right. The party window takes the lower half. No pixel art "
    "is drawn."
)
```

That `BUILT` flag reads true and the window still skips the tab, so the flag records an intention and the skip list records the behaviour. The skip list is what runs.

The declared shell halves its top row and takes the lower half for the party. Three zones, seven subtabs, two chains, eight event types and fifteen mechanism controls are declared. `src/gui/web/proof_of_accumulation_tab.js` is the renderer module for those zones.

Declared	Count	Names
Zones	3	Current Vessel, Enemy Screen, Party Window
Subtabs	7	Maps, Character Details, Gear, Resources, Quint, Skills, Guild
Subtab keys	7	Ctrl+1 to Ctrl+7, in that order
Chains	2	Live, Demo TestNet
Event types	8	four modes, each with a Standard and an Elite flag
Mechanism controls	15	identity and its confirm, distil, train, transfer, spend, grade, payout, close, drop, exclusion, reset and its confirm, world and its confirm
Wallet sections	4	Quintessence, Trophies, Loot, Vessels
Party rows	120	40 a page, 3 pages, 8 groups of 5 a page

A player picks a chain, Live or Demo TestNet, then picks one of the eight event types, and every mechanism control runs against that pair. The identity control, the reset control and the world control each ask first and act

on a second press. Every control prints what its mechanism did or the refusal that mechanism raised, and eleven exception types are caught as a refusal rather than a crash. Until this node writes its identity file, every other control refuses for want of a participant.

Each subtab draws off one or two engine modules, and the counts below are what each one puts on the screen.

Subtab	What it draws	Module behind it
Maps	5 grid bounds: 9 squares a side, 81 a layer, 20 layers, 79 participants a layer, 1 square in view	`world_grid`
Character Details	5 stats with their principle and effect, the 10 bands a stat climbs, and the 27 trading metrics	`entity_stats`, `rpg_metrics`
Gear	the loot the wallet holds, 4 item classes with their cohesion, 8 mechanisms	`loot_drop`, `items`
Resources	7 materials, each with the Quintessence band one unit embeds and where that band came from	`materials`
Quint	the chain, its two files, the smallest unit, the supply cap, the minted total	`quintessence_ledger`
Skills	two named pages and the one skill on the ladder, over its 10 levels	`skill_ladder`
Guild	0 guilds, 0 members, the two ranks and the treasury address shape	`guild_roster`

Clicking the current Vessel opens the Character Details subtab, which is also what its shortcut does. The Maps subtab opens only in an event whose mode carries a map. Two of the four modes do — the dungeon crawl and the raid — so four of the eight event types open it and the other four refuse and say why. A request that names no subtab, or names one that cannot open, opens Character Details.

`src/gui/main_tabs/proof_of_accumulation_tab_surface.py` — the refusals caught

```
CONTROL_FAULTS = (
    ActionSpendError,
    LootError,
    OSError,
    OverflowError,
    QuintessenceLedgerError,
    RedistributionError,
    RotationRefusedError,
    SkillLadderError,
    TypeError,
    ValueError,
    WorldGridError,
)
```

Identity

Each instance gets an Ed25519 keypair. The private key signs every trade in the log. The public key is the identity another party verifies against. Strategy parameters are never signed and never published.

`src/competition/bot_identity.py` — `BotIdentity`

```
class BotIdentity:
    """
    Manages a bot's Ed25519 keypair.  The private key never leaves this object
    unencrypted.  The public key is the bot's network-visible identity.
```

The trade log

Each signed trade appends to a Merkle tree. Leaves are the SHA-256 of a trade's canonical bytes, the root is the commitment, and a proof lets a third party confirm one trade sits in the log without receiving the log.

`src/competition/merkle_log.py` — `verify_proof`

```
def verify_proof(leaf_hash: str, proof: List[dict], root: str) -> bool:
    """Verify a Merkle inclusion proof."""
    current = leaf_hash
    for step in proof:
        if step["position"] == "left":
            current = _node_hash(step["hash"], current)
        else:
            current = _node_hash(current, step["hash"])
    return current == root
```

`merkle_root` returns the commitment and `merkle_proof` builds the inclusion proof one trade at a time. No subtab reads that root.

The competition lifecycle

Four phases run in order. A competition runs either locally across instances on one machine and one price feed, or between two machines exchanging signed submissions.

`src/competition/competition_engine.py` — the module's own summary

```
  1. REGISTRATION  — bots register with capital commitment + config hash
  2. ACTIVE        — bots trade; each trade appended to their Merkle log
  3. SUBMISSION    — trading closes; bots submit Merkle root + performance claim
  4. ADJUDICATION  — arbiter verifies submissions, ranks bots, awards tokens
```

The token ledger

The ledger is append-only and idempotent: settling the same result twice writes one record. Balances are replayed from the log and no operation edits one. Ten public methods answer, and the method reporting what is still mintable is `remaining_ever`.

`src/competition/token_ledger.py` — the ten methods `TokenLedger` declares

```
    def award(
    def balance(self, bot_id: str) -> int:
    def awards(self, bot_id: str) -> List[AwardRecord]:
    def total_minted(self) -> int:
    def remaining_ever(self) -> int:
    def season_minted(self, season: int) -> int:
    def leaderboard(self, top_n: int = 20) -> List[dict]:
    def supply_summary(self) -> dict:
    def save(self):
    def load(self) -> "TokenLedger":
```

The hard cap is ten million and the ledger refuses an award that would pass it. Each season awards seventeen twentieths of the season before, which is 85 per cent held as an exact ratio rather than a decimal, with a floor of a hundred.

`src/competition/season_schedule.py` — the supply constants

```
TOTAL_SUPPLY_CAP = 10_000_000  # Hard cap — immutable
GENESIS_SEASON = 1
INITIAL_REWARD = 500_000  # season 1 pool, in ACRV tokens
DECAY_NUMERATOR = 17
DECAY_DENOMINATOR = 20
MIN_SEASON_REWARD = 100
```

Nothing advances the season. The counter lives behind a gated registry function on the chain and no module under `src` reaches it.

The five rarity tiers

Five tiers are awarded on rank within the field. Four carry a lifetime ceiling on how many can ever exist and Harvest carries none, held in the field `max_ever`. Each tier is named for a stage of the alchemical path and pays a fixed number of tokens.

Tier	Stage	Rank	ACRV paid	Ever minted, at most
Harvest	NIGREDO	top 50%	10	no cap
Gold Fold	ALBEDO	top 10%	50	100,000
Bear Slayer	CITRINITAS	top 25% in a verified bear market	100	10,000
Grand Accumulator	RUBEDO	top 1% across three consecutive seasons	500	1,000
Ekthelius	UNIO MYSTICA	perfect score across every metric	10,000	21

The first tier and the last show both cases of the ceiling field.

`src/competition/season_schedule.py` — the uncapped tier, then the capped one

```
    RarityTier(
        name="Harvest",
        emoji="■",
        description="Top 50% of competition field",
        rank_pct_max=0.50,
        condition="Win rate > 50% of competing bots",
        max_ever=None,
        base_value=10,
    ),
RarityTier(
    name="Ekthelius",
    emoji="∞",
    description="Perfect score across all metrics, any season",
    rank_pct_max=0.001,
    condition="100% win rate + top Sharpe + max capital efficiency",
    max_ever=21,
    base_value=10_000,
),
```

The token contract

The ledger above is the platform's own record. On the chain the token is an ERC-20 called Acervator Token. Three properties are fixed at deployment and cannot change afterwards: the supply cap, the one address allowed to mint, and the absence of any way to burn.

Property	Value
Standard	ERC-20, named Acervator Token
Chain	Base, chain id 8453
Hard cap	10,000,000 ACRV, held on the chain
Minting	the CompetitionRegistry contract only
Burning	never

The token takes no address when it is built. One call afterwards names the registry as the minter, a second call to it is refused, and any target that is not itself a contract is refused, so a wallet cannot be named the minter by mistake.

`contracts/ACRV.sol` — the one wiring call

```
    function setRegistry(address registryAddress) external onlyOwner {
        require(registry == address(0),          "ACRV: registry already set");
        require(registryAddress.code.length > 0, "ACRV: registry not a contract");
        registry = registryAddress;
        emit RegistrySet(registryAddress);
    }
```

The contracts are not deployed. Five are deployed together — the token, the competition registry, the trophy, Quintessence and Governance. The registry and the trophy each need the other's address, and `setTrophy` is what cuts that cycle: the registry deploys first and points back at the trophy afterwards. Two npm packages

supply the libraries they build on, and the deploy script takes the network and the signing key. Its own instructions name the Base test network first.

`contracts/deploy.py` — how it is called

```
npm install @openzeppelin/contracts @chainlink/contracts
export ACERVATOR_PRIVATE_KEY=0x...
python deploy.py --network sepolia
```

Nothing needs an address that does not yet exist. The token takes no constructor argument, the registry takes the token's address, the trophy and Quintessence take the registry's, and Governance takes all four. Five wiring calls follow, each running once.

Diagram: flowchart, top to bottom

- 1. ACRV(-> **2. CompetitionRegistry(token, feeds**
- 2. CompetitionRegistry(token, feeds -> **3. ACRV.setRegistry(registry**
- 3. ACRV.setRegistry(registry -> **4. AcervatorTrophy(registry**
- 4. AcervatorTrophy(registry -> **5. CompetitionRegistry.setTrophy(trophy**
- 5. CompetitionRegistry.setTrophy(trophy -> **6. Governance(the four**
- registry

Computing an address before the contract exists does not solve the cycle. A computed address commits to the values handed to the constructor, so computing the registry's address still needs the token's address first, and the circle closes again.

```
address   = keccak256(0xff ++ deployer ++ salt ++ keccak256(init_code))[12:]
init_code = creation bytecode ++ the encoded constructor arguments
```

The build settings are fixed in `foundry.toml`, so the deployed bytecode is the bytecode the analyzers read. The deploy script refuses to run on an artifact `forge build` has not written.

The registry's award function writes every record and both log entries before it pays, so a token contract that called back during the payment cannot find the competition still marked open.

The Quintessence contract

A second contract holds Quintessence on the chain. It divides one unit into ten to the eighteenth parts, has no owner, no pause, no upgrade hook and no burn, and names its minter once when it is built.

Term	Value
Smallest unit	one Quintessence divided into 10^18 parts
Hard cap	33,000,000 Quintessence, as 33 followed by 24 zeros of those parts
Owner	none
Pause	none
Upgrade hook	none
Burn	never

Term	Value
Minting	the Proof-of-Accumulation registry only, named once when built

`contracts/Quintessence.sol` refuses a transfer too small for the recipient to receive one whole smallest unit, rather than paying it as nothing. `tests/contracts/QuintessenceConservation.t.sol` drives the conservation rule.

The buckets move on the chain as they do in the platform's ledger, with one difference: a wallet cannot pay another wallet out of the held bucket. It authorises the payment and the registry runs it.

Governance, the franchise and the halt council

A fifth contract governs the other four. A vote sits at one of four levels, and each level sets its own holding to propose, its own quorum, its own approval share and its own delay before the change takes effect.

Level	Holding to propose	Quorum	Approval	Delay
Informational	1 Quintessence	10%	50%	none
Patch	25 Quintessence	20%	50%	2 days
Interface	75 Quintessence	30%	60%	7 days
Core	150 Quintessence	40%	67%	30 days

The franchise is a second number beside the balance, and it moves no Quintessence. A balance that rises carries the franchise up with it in the same block. A balance that falls leaves the franchise at its remembered ceiling, and that ceiling stands for ninety days after the address's last action before it begins to fall back toward the balance, closing the gap at a ninetieth a day over the next ninety. The ramp is continuous rather than stepped, so the rate holds between days too.

`contracts/Governance.sol` — the franchise rule, in its own words

```
//   franchiseOf(a) >= Quintessence.balance(a), for every address, always
```

The halt council is five elected addresses, three of which halt one named mechanism for seven days. No release call exists: the halted state reads a timestamp, so a halt ends by itself.

`contracts/Governance.sol` — the council and the halt

```
    uint256 public constant COUNCIL_SIZE = 5;
    uint256 public constant HALT_SIGNALS_REQUIRED = 3;
    uint256 public constant HALT_SECONDS = 7 days;
    uint256 public constant HOLD_SECONDS = 90 days;
    uint256 public constant RESYNC_SECONDS = 90 days;
    uint256 public constant BPS_DENOMINATOR = 10_000;
```

Every deployment stays immutable. A change no vote can make is made by deploying a new contract and pointing migration at it on a Core vote. Holders then spend their own units into the migration held address, where Quintessence retires them. An address that never spends keeps its units on this contract and reaches nothing the new one governs.

Trophies

One SVG per tier, picked by name. An unknown tier raises rather than returning an empty drawing, and `generate_preview_html` lays the whole set out on one page.

`src/competition/trophy_generator.py` — `generate_trophy`

```
def generate_trophy(tier: str, data: TrophyData) -> str:
    fn = GENERATORS.get(tier)
    if not fn:
        raise ValueError(f"Unknown tier: {tier!r}")
    return fn(data)
```

Inside `harvest_svg` a text path letters the epigraph's own instruction around the trophy ring, in its usual form: SOLVE ET COAGULA. Each drawing also letters its own alchemical stage across the face, NIGREDO for Harvest through RUBEDO for Grand Accumulator.

Where a trophy's artwork is kept

The artwork and the metadata are held on the chain itself. The token's metadata call returns the JSON inline, and the picture inside that JSON is the tier's drawing, also inline. Nothing points at IPFS, the file-sharing network most NFT projects park their pictures on, and nothing points at any other host, so a trophy lasts as long as the chain does.

`contracts/AcervatorTrophy.sol` — the metadata call

```
            '","image":"', svgB64,

            "data:application/json; base64,",
```

The trophy contract holds the four ceilings, so a tier cannot be over-minted by a caller. `tests/contracts/TrophyTierCaps.t.sol` drives them and `contracts/MetadataLib.sol` builds the inline JSON.

Tier	Trophies ever	Where the ceiling is held
Harvest	no limit	nowhere, by design
Gold Fold	100,000	the trophy contract
Bear Slayer	10,000	the trophy contract
Grand Accumulator	1,000	the trophy contract
Ekthelius	21	the trophy contract

The chain

`local_testnet.py` simulates the whole Base environment in memory, with no wallet and no network. Seven classes answer: four are the chain and its contracts, three are the records those four store.

Class	Holds
`LocalChain`	The blocks

Class	Holds
`LocalACRV`	The ERC-20 balances and the mint history
`LocalRegistry`	Competitions, submissions and adjudications
`LocalTestnet`	The three above, plus a mock oracle and transaction receipts
`Block`	One block's number, hash, parent hash, timestamp and transactions
`TxRecord`	One transaction's hash, block, sender, function, arguments and gas
`ChainEvent`	One event a contract emitted, with its block and its arguments

A block and a transaction are each named by their own contents, so no two records of different content share a name. The two names read different fields.

value	names a transaction	names a block
sender and recipient	yes	no
the call and its arguments	yes	no
gas	yes	no
success flag	yes	no
block number	no	yes
timestamp	no	yes
parent block's name	no	yes
the transactions held	no	yes

One call runs a whole competition and returns the result table, writing nothing outside a temporary directory.

`src/competition/local_testnet.py` — a demo run

```
  from src.competition.local_testnet import LocalTestnet

  testnet = LocalTestnet()
  result = testnet.run_demo_competition(n_bots=3, season=1)
  print(result["results_table"])
```

The token carries eighteen decimals on that chain, the demo price series is drawn from a fixed seed so two runs agree, and a load that finds another schema reports at most five altered records rather than every one.

`src/competition/local_testnet.py` — the chain constants

```
  TOKEN_DECIMALS = 18
  TX_SUCCESS = 1
  GENESIS_BLOCK_ID = "0x" + "0" * 64
  TIMESTAMP_DIGITS = 3
  ALTERED_LOG_LIMIT = 5
  DEMO_PRICE_SEED = 42
```

A checkpoint file and an append log hold the chain between launches. The chain adds what changed rather than rewriting itself, and a checkpoint is written every fifty thousand records so a load does not replay the whole log. The reset control names both files and the bytes they hold before it deletes either.

The real targets sit ready for the day it deploys. The two ABIs are complete and the four address fields are empty, so the calls are already described the day an address arrives.

`src/competition/base_config.py` — the two chains and what they answer

```
Base Mainnet: chain_id=8453  — production
Base Sepolia: chain_id=84532 — testnet (deploy here first)

BASE_MAINNET   acrv_address = ''   registry_address = ''
BASE_SEPOLIA   acrv_address = ''   registry_address = ''
ACRV_ABI       12 entries
REGISTRY_ABI   16 entries
```

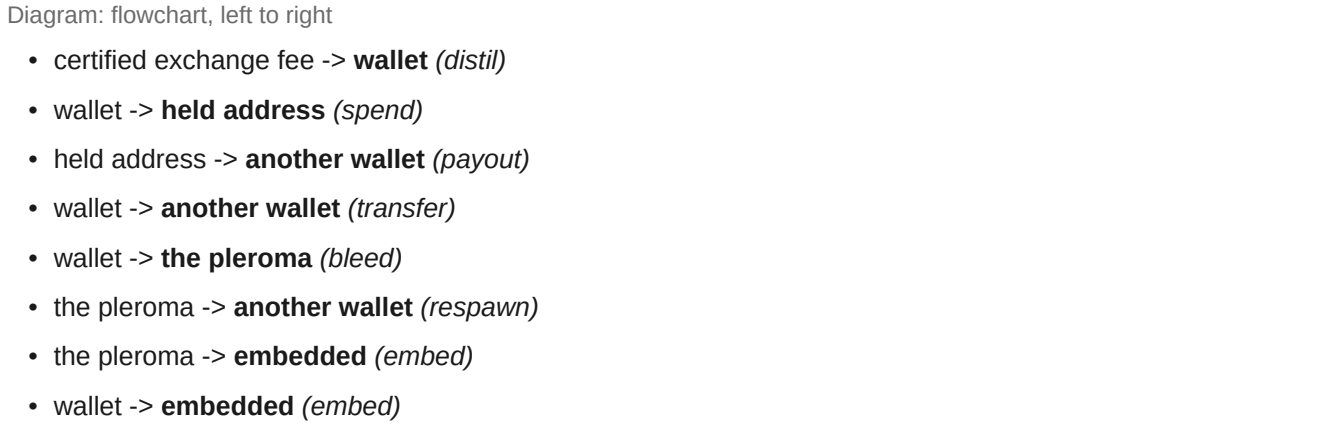
Quintessence

A certified trade distils the platform's second asset. Entering an event spends it, and nothing destroys it. It keeps its own ledger, apart from the token ledger above, because the two obey opposite rules: the token only ever moves outward into a balance, while Quintessence circulates.

`src/competition/quintessence_ledger.py` — the nine operations

```
def distil(self, address: str, fee_usd: object, trade_grade: object) -> Decimal:
def spend(self, address: str, amount: object, held_address: str) -> Decimal:
def transfer(self, sender, recipient, amount, skill_level) -> QuintessenceTransfer:
def payout(self, held_address: str, credits: dict) -> Decimal:
def respawn(self, address: str, amount: object) -> Decimal:
def embed_from_pleroma(self, amount: object) -> Decimal:
def embed_from_wallet(self, address, amount, embedded_amount) -> QuintessenceEmbed:
def release_from_embedded(self, address, amount, recovered_amount) -> QuintessenceRelease:
def release_all_to_pleroma(self, amount: object) -> Decimal:
```

Quintessence can be in exactly four places, and the four always add up to everything ever distilled. A wallet holds what a participant can spend. A held address holds what they have already spent, which rests there and funds later awards. The payout operation moves an award from the held address into the winner's wallet. The pleroma holds what bled out of a transfer and the ledger respawns that to other participants. The embedded bucket holds what a thing in the world carries in itself, drawn out of the pleroma and returned there when the thing is broken.



- embedded -> **another wallet** *(release)*
- embedded -> **the pleroma** *(release)*

Every operation checks that sum before it writes, and refuses the write when it does not balance.

`src/competition/quintessence_ledger.py` — the two conditions the check reads

```
  is_balanced=delta == 0 and not negatives,
  is_within_cap=self._total_ever_minted <= QUINTESSENCE_SUPPLY_CAP,
```

Term	Value
Hard cap	33,000,000 Quintessence, total ever distilled
Pre-ownership	none, for anyone
Distil rate	one Quintessence for one dollar of certified exchange fee
Grade curve	the trade's grade, zero to one, multiplies the award
Destruction	never
Transfer bleed	8% at skill level one, falling to 4% at level ten
Resolution	one Quintessence divides into 100,000,000 minimum units of 0.00000001

The module holds the cap and the rate as constants, and the write path refuses a mint past the cap rather than reporting it afterwards.

`src/competition/quintessence_ledger.py` — the recorded numbers

```
  QUINTESSENCE_SUPPLY_CAP = Decimal(33_000_000)
  QUINTESSENCE_MINIMUM_UNIT = Decimal("0.00000001")
  QUINTESSENCE_UNITS_PER_WHOLE = 100_000_000
  QUINTESSENCE_PER_FEE_USD = Decimal(1)
  BLEED_FRACTION_AT_LEVEL_1 = Decimal("0.08")
  BLEED_FRACTION_AT_LEVEL_10 = Decimal("0.04")
  MIN_TRANSFER_SKILL_LEVEL = 1
  MAX_TRANSFER_SKILL_LEVEL = 10
```

Ten movement kinds are named, one per operation plus the bleed, so every write carries the name of the movement that made it.

One whole Quintessence divides into a hundred million minimum units

The minimum unit is the smallest amount that can exist. A divine essence is potent at a minute amount, so one whole unit carries a hundred million places to hold power in, at the same resolution as the first cryptocurrency.

Every amount a bucket receives is a whole number of minimum units. The amount is rounded down onto that figure as it is written, so no wallet, held address, pleroma or embedded balance can carry a fraction the currency cannot express.

`src/competition/quintessence_ledger.py` — the rounding and the refusal

```
def quantize_quintessence(amount: Decimal) -> Decimal:
    return amount.quantize(QUINTESSENCE_MINIMUM_UNIT, rounding=ROUND_DOWN)


def is_on_quintessence_grid(amount: Decimal) -> bool:
    return amount == quantize_quintessence(amount)
```

The rounding leftover returns to the pleroma, because nothing is destroyed

A bleed of eight per cent down to four per cent does not divide evenly at eight of the ten skill levels. The amount the recipient receives is rounded down and the bleed takes the rest, so the fraction that cannot be paid joins the pleroma rather than vanishing. That keeps the four buckets equal to everything ever distilled, to the unit.

`src/competition/quintessence_ledger.py` — the transfer split

```
received = quantize_quintessence(sent - sent * fraction)
bled = sent - received
```

Skill level	Bleed on 100 Quintessence	Received
1	8	92
2	7.55555556	92.44444444
3	7.11111112	92.88888888
4	6.66666667	93.33333333
5	6.22222223	93.77777777
6	5.77777778	94.22222222
7	5.33333334	94.66666666
8	4.88888889	95.11111111
9	4.44444445	95.55555555
10	4	96

The five stat requirements are whole Quintessence already — one stat measures 550 at level 100 and five of them measure 2,750 — so the resolution changes nothing about the stat scale.

Where the ledger is built

Every launch builds the ledger and attaches it to the window beside the local chain, so a later panel finds it where it finds the chain. The same launch hands that ledger to the certification socket, so the socket and the panels move one ledger and not two.

`src/gui/shared_testnet.py` — the ledger, and the socket that takes it

```
        ledger = cls.install_quint_ledger(quint_ledger_path)
        main_win._quint_ledger = ledger
        main_win._market_rotation = bridge.install_market_rotation(rotation_path)
        main_win._capture_bounds = bridge.install_capture_bounds(
            main_win._market_rotation, bounds_path
        )
        main_win._certification_socket = bridge.install_certification_socket(
            ledger, socket_path, main_win._capture_bounds
        )
```

Fluor is the name Quintessence takes when it pays for a movement on the chain. Every game fee is Quintessence; only the chain movement is called Fluor.

> *"Everything is Quint and we can invent our version of gas using an alchemical term for 'flow' or 'move' or 'fuel'"*

The certified fill

A trade certifies through one method, which signs one fill, logs it, posts it and distils the venue's fee. It refuses a fill already certified.

`src/competition/certification_socket.py` — what the method contracts to do

```
    def certify(self, identity: BotIdentity, fill: CertifiedFill) -> CertificationReceipt:
        """Sign, log, post and distil one fill, and return its receipt.
```

Certification is assembled from parts that already existed and five the socket added.

Part of certification	Where it comes from
Signing one fill	`BotIdentity.sign_trade`
Refusing a wrong competition, a wrong bot, a bad signature	`MerkleTradeLog.append`
The commitment over every certified fill	`MerkleTradeLog.root`
Proof of one fill without the log	`MerkleTradeLog.proof_for`
A summary revealing no fill	`MerkleTradeLog.submission_summary`
The transaction and the event on chain	`LocalChain.send_tx`, `LocalChain.emit`
Minting the award	`QuintessenceLedger.distil`
One chain and one ledger per launch	`SharedTestnetBridge.install_on`
A fill offered for certification	`CertifiedFill`
What one certification produced	`CertificationReceipt`
The replay, stranger and cap refusals	`CertificationSocket.certify`
The lifetime fee total	`lifetime_certified_fee_usd`
The fill subscriber	`attach_to_bus`

Read on the current tree, no launch builds that chain or that ledger. The main window no longer calls the install, so the row above names the factory and not a launch step. Nothing in the running window calls it.

`src/gui/shared_testnet.py` — `SharedTestnetBridge.install_on`, the factory the row names

```
    @classmethod
    def install_on(
        cls,
        main_win,
```

Four refusals guard the mint, and the second column names what each one prevents.

Refused	Without its guard
A fill already certified	the fee total doubles, $4.00 to $8.00
A bot that has certified nothing	it enters an event
A forged signature	500 Quintessence mints for a bot that signed nothing
An award past the 33,000,000 cap	the log keeps a fill that never distilled

The fill event carries the venue's own fee, read back off the settled order rather than estimated from a configured percentage, so a distil mints on what the exchange actually charged. `src/trading/scrumming/execution.py` places the order and `src/exchange/ccxt_connector.py` reads it back; `src/trading/trade_grader.py` supplies the grade that multiplies the award.

The capture bounds on one award

An award is bounded four ways: a market must be in the rotating reward set, the award must name an activation, it must wait out a candle cooldown, and no participant may take more than a fixed share of a market's allotment.

`src/competition/capture_bounds.py` — the bounds and the cooldown

```
COOLDOWN_CANDLES = 3
COOLDOWN_FLOOR_S = 900
MIN_SCORED_AXES = 1
EMISSION_PER_ACTIVATION = None
EXECUTION_READABLE_BPS = 100.0
```

The cooldown is three candles of the bot's own timeframe with a floor of nine hundred seconds, so a fast bot waits the floor and a slow one waits its candles.

Bot timeframe	Three candles	The wait	What decides it
1m	180 s	900 s	the floor
3m	540 s	900 s	the floor
5m	900 s	900 s	they agree
15m	2,700 s	2,700 s	three candles
1h	10,800 s	10,800 s	three candles
1d	259,200 s	259,200 s	three candles

No emission figure exists. `EMISSION_PER_ACTIVATION` holds nothing, so an activation refuses for want of a figure nobody has set. Eleven refusal reasons are named, the missing emission and the missing grade

among them.

The rotating reward set

Which markets may reward Quintessence rotates. The set is drawn from the markets ranked by the dollar value traded in twenty-four hours, and the draw is committed before it is revealed, so no participant can learn the set while the window is open.

`src/competition/market_rotation.py` — the bounds on the draw

```
TOP_N_BY_VOLUME = 20
MIN_ELIGIBLE_POOL = 12
DRAW_DENOMINATOR = 4
MAX_PARTICIPANT_SHARE_PCT = Decimal("5")
AGE_LOOKUP_INTERVAL_S = 13.0
ROTATION_SALT_BYTES = 16
```

One market in four is drawn, the pool refuses to draw below twelve eligible markets, and no participant may take above five per cent of a market's allotment. The ranking reads the dollar value traded, not the coin count.

What the ranking reads	Where it comes from
The dollar value traded in 24 hours	`quoteVolume`, served by the venue
The same figure where a venue omits it	coins traded times the last price
The coin count, still recorded	`volume_24h`, read by the topology proposals

While the window is open, neither a participant, another node, nor the exchange can learn which markets pay; only the node that drew the set holds the answer, on its own disk. Seven states are named for one market: eligible, in rotation, outside the top volume, excluded by the exchange, pool below floor, not drawn, and no open window. An exclusion filed in a season binds at the next season boundary.

The six-month project age rule

A market rewards Quintessence only if its project is at least six calendar months old. The lookup reads the project's start date from the market data service, caches it outside the repository, and answers one of five verdicts.

`src/competition/project_age.py` — the rule and its verdicts

```
MIN_PROJECT_AGE_MONTHS = 6
HTTP_TIMEOUT_S = 15.0
GENESIS_CACHE_FILE_VERSION = 1
OLD_ENOUGH = "old_enough"
TOO_YOUNG = "too_young"
NO_COINGECKO_ID = "no_coingecko_id"
NO_GENESIS_DATE = "no_genesis_date"
LOOKUP_FAILED = "lookup_failed"
```

Six months means six calendar months, not a count of days. Four of the five verdicts are refusals: the project is under six months old, the catalogue has no identifier for it, the service answers with no date, or the call did not answer. The lookup is assembled from parts the platform already had.

Part of the lookup	Where it comes from
The catalogue identifier for an asset	`crypto_assets.ASSETS`
The project's own start date	the market service's coin detail address
A request that refuses any address but http and https	`safe_url.SafeRequest`
Opening that request	`safe_url.safe_urlopen`
Writing the cache file without a torn write	`io_utils.atomic_write_json`
Runtime data outside the repository	the same home folder the rest of the runtime uses
The six-month test	`months_after`
One market's answer	`ProjectAgeVerdict`
The four refusals	`ProjectAgeLookup.verdict_for`

The event modes and the turn

Four modes pair with one Elite flag, so there are eight event types and no mode is written twice. The code is what every other module keys on; the label is what the screen draws beside the Standard or Elite word.

Mode	Label on screen	Shape
`monster_smash`	Fixation	One participant against low to midlevel creatures
`team_monster_smash`	Coagulation	A certified guild, up to 120 against 1
`dungeon_crawl`	Descension	One participant, or a group of six, through a dungeon
`raid`	Cementation	A group of sixty, the most challenging and the most rewarding

The four labels are the names of alchemical operations, which he asked for.

> *"Please rename events to match Hermetic tests or similar..."*

One table indexed by the Elite flag carries every difference between a Standard and an Elite event, so the two cannot drift apart.

`src/competition/poa_modes.py` — the one table the Elite flag indexes

```
STANDARD_RULES = VariantRules(
    STANDARD_SUFFIX, STANDARD_LABEL, STANDARD_TIMEFRAME, 0, 0, 0, 0
)
ELITE_RULES = VariantRules(ELITE_SUFFIX, ELITE_LABEL, ELITE_TIMEFRAME, 1, -1, 1, 1)

VARIANT_RULES: dict[bool, VariantRules] = {False: STANDARD_RULES, True: ELITE_RULES}
```

An Elite event runs on the one-minute candle and a Standard event on the five-minute candle. Elite raises difficulty, loot rarity and drop rate by one step each and lowers the entry fee by one.

Event type	Turn	Difficulty	Entry fee	Loot rarity	Loot drop
`monster_smash`	5m	1	1	1	1
`monster_smash_elite`	1m	2	1	2	2
`team_monster_smash`	5m	2	2	2	2

Event type	Turn	Difficulty	Entry fee	Loot rarity	Loot drop
`team_monster_smash_elite`	1m	3	1	3	3
`dungeon_crawl`	5m	3	3	3	3
`dungeon_crawl_elite`	1m	4	2	4	4
`raid`	5m	4	4	4	4
`raid_elite`	1m	5	3	5	5

A turn is one candle of the market clock, and `turn_at` names the turn that clock is in. An Impetus pool holds the points one turn grants and loses what that turn does not spend.

`src/competition/poa_modes.py` — the Impetus curve

```
  WORLD_TURN_TIMEFRAME = "1h"
  WORLD_TURN_SECONDS = TF_SECONDS[WORLD_TURN_TIMEFRAME]
  IMPETUS_AT_FIRST_LEVEL = 4
  IMPETUS_LEVELS_PER_STEP = 20
  IMPETUS_SPEED_CAP_FACTOR = 2
  IMPETUS_FLOOR = 1
```

One world turn is one hour, read off the shared timeframe table, which is 3,600 seconds. A participant gains one Impetus point a turn every twenty levels.

Level	Impetus a turn
1	4
21	5
40	6
60	7
80	8
100	9

The action bands and the record store

Five bands price every action from the cheapest to the dearest, a hundred to one across the range. A charge may span turns, with a floor of one turn.

`src/competition/action_spend.py` — the bands and what each buys

```
BANDS: dict[str, Decimal] = {
    BAND_X1: Decimal("0.001"),
    BAND_X3: Decimal("0.003"),
    BAND_X10: Decimal("0.010"),
    BAND_X30: Decimal("0.030"),
    BAND_X100: Decimal("0.100"),
}

BAND_ACTIONS: dict[str, str] = {
    BAND_X1: "move, switch weapon, take an item from a bag",
    BAND_X3: "a basic attack, a basic heal",
    BAND_X10: "a class ability on a cooldown",
    BAND_X30: "a group-wide ability, a threat move across the field",
    BAND_X100: "a multi-turn spell, and the decisive tactics beside it",
}
```

A spend rests its cost at one address, the event pot, and a record store holds every action a participant paid for. The store's own file carries a version, so a load that finds another schema says so.

The pot divides on performance

An Elite event's pot returns three quarters of itself to the participants, divided on a normalised performance score. The remainder rests on the chain rather than being rounded away, and a participant with no scored axis takes no share.

`src/competition/event_redistribution.py` — the return share

```
RETURN_PERCENT = 75
SCORED = "scored"
NO_SCORE = "no_score"
```

The division reads performance and never spend: scaling one participant's spend by any factor leaves every payout identical, while halving that participant's score moves them all.

Head to head

`challenge_protocol.py` carries the Elo ladder. A challenger sends a signed challenge, the target accepts or declines, both trade the agreed asset for the agreed duration, and the shared engine adjudicates. The stake flows from loser to winner and both ratings move.

`src/competition/challenge_protocol.py` — the ladder constants

```
DEFAULT_ELO = 1200
ELO_K_FACTOR = 32
MIN_ELO = 100
```

The world grid

A world is a stack of layers, each a square grid. The grid side is not chosen: it is derived from how many records one layer's turn can hold on the chain, so the world is exactly as large as the chain can carry.

`src/competition/world_grid.py` — the derivation, and the figures it fixes

```
WORLD_RECORD_BYTES = 1473
TURN_BYTE_CAPACITY = 1_048_576
TREE_SPHERES = 10
SEPHIROT_LAYERS = TREE_SPHERES * 2
ARENA_LAYER = 0
BASE_VIEWRANGE_SQUARES = 1
SQUARE_STEPS = 100
WORLD_SEED_BYTES = 16
```

Read at runtime, one turn holds 711 records a layer, one participant writes 9 of them, so 79 participants fit a layer and the grid is 9 squares a side holding 81 squares. Twenty layers are declared, two for each of the ten spheres, and layer zero is the arena every participant starts on. One square is a hundred steps across.

Figure	Value
Bytes one world record writes	1,473
Bytes one layer's turn may hold	1,048,576
Records one layer's turn holds	711
Records one participant writes a turn	9
Participants a layer	79
Squares a side	9
Squares a layer	81
Layers declared	20
Squares in view	1
Steps across one square	100

Eight chain functions and seven events are named for the world: create, breach a layer, discover a zone, discover an asset, learn an asset, extract Quintessence and publish the seed. The seed is committed when the world is created and published later, so no participant can read the world ahead of discovering it. Discovery runs at three levels: the square, the zone region on it, and the fact found there.

A terrain grid is separate from the world grid and need not line up with it.

> *"Also, terrain grids do not need to overlap perfectly with the world grid. These grids are varied in size and shape while be linked together at their own borders."*

A zone region carries a boundary of at least three vertices and the participant who discovered it. It carries no terrain kind, so no terrain has a map mark.

Movement, and the world turn's pool

Movement is derived rather than stored. One record opens a journey leg, and where a participant stands is computed from that leg and the clock. A participant who has not moved stands at the arena.

`src/competition/world_movement.py` — what a leg declares

```
MOVEMENT_PRECISION = 28
BASE_STEPS_PER_TURN = SQUARE_STEPS
STANDING_FROM_LEG = "journey leg"
STANDING_FROM_ARENA = "arena"
JOURNEY_FILE_VERSION = 1
```

Where an address stands is answered one of two ways, and the answer names which one it came from. Both end in the locator text every place comparison reads.

Diagram: flowchart, top to bottom

- a consumer asks where an address stands -> **does a covering leg exist?**
- legs -- yes -> **progress_at derives the square and the two steps**
- legs -- no -> **PoaWorld.arena_position**
- progress_at derives the square and the two steps -> **MoverStanding source = journey leg**
- PoaWorld.arena_position -> **MoverStanding source = arena**
- MoverStanding source = journey leg -> **locator text every place comparison reads**
- MoverStanding source = arena -> **locator text every place comparison reads**

A turn grants a pool of steps, and the pool is counted in steps rather than in squares, so a part of a square is expressible.

> *"Just need to be able to say that character A traversed x% of a given square in a given turn and this will make it relatively easy to simulate varied terrain within one or across multiple squares. Should be simple enough to make this dynamic and per character or army or group with encumberance playing a role."*

Encumbrance is the rule that would spend more of the pool for a heavier load.

> *"Its a function of Strength (max weight) and Constitution (turn point penalty while carrying)."*
>
> *"Just need to be able to say that character A traversed x% of a given square in a given turn... Should be simple enough to make this dynamic and per character or army or group with encumberance playing a role."*

`src/competition/world_turn.py` — the pool's bounds

```
STEP_UNIT = "step"
MIN_GRANT_STEPS = 0
MIN_SPEND_STEPS = 1
FIRST_TURN_INDEX = 0
```

Nothing computes an encumbrance penalty. The strength and constitution stats exist and no module reads them for a movement cost.

Dungeons and the second clock

A participant on the world map is on the one-hour world turn. Entering a dungeon moves them to the event turn, which is one to five minutes, and leaving returns them. A dungeon entry refuses a party whose members are not all standing at the entrance.

> *"PoA - Dungeons - World Map - Another example of needed inference. What happens after a player enters a dungeon? These are the sorts of design questions I want you to ask while proceeding. Who. What Where. When. Why. How."*

`src/competition/dungeon_entry.py` — the clock it moves a participant to

```
EVENT_CLOCK = "event turn"
DUNGEON_STORE_VERSION = 1
```

A dungeon is addressed by a whole locator, not by a square alone, so two dungeons on different layers of one square are different places.

The Vessel

A Vessel is the body a participant occupies. Every Vessel a participant holds reads one wallet balance, so the wallet is the only bound on how many a participant may have. A Vessel is named by a 32-byte identity, and every record about a Vessel keys on that identity.

`src/competition/vessels.py` — what a Vessel's standing answers

```
VESSEL_ID_BYTES = 32
MERGED_WITH_PLEROMA = "fully merged with the pleroma"
HOLDS_A_BALANCE = "holds a Quintessence balance"
NEVER_HELD_A_BALANCE = "has never held a Quintessence balance"
VESSEL_COUNT_UNCAPPED = "the wallet balance is the only bound on Vessel count"
```

A level gained keeps the Vessel it was gained on, so a per-class level record is held for each. Nothing gives a Vessel an assignment, gear, equipping, destruction or permadeath.

Stats, and the metrics they convert from

Five stats are declared in one table, each measured in Quintessence, each climbing ten bands. A bot's trading profile converts into twenty-seven RPG metrics.

Declared	Count	Names
Stats	5	strength, dexterity, constitution, intelligence, wisdom
Bands a stat climbs	10	one a sphere
Trading metrics	27	read off the bot's own record
Metrics nothing holds	7	experience, level, character class, gear, enemy, threat, guild

Those seven read nothing because no field under `src` holds them. The stat table carries a principle and an effect for each stat.

`src/competition/entity_stats.py` — the potential scale a stat is read on

```
FIRST_SPHERE = 1
NO_POTENTIAL = Decimal(0)
FULL_POTENTIAL = Decimal(1)
POTENTIAL_PRECISION = 28
```

The seven character classes

Seven classes are declared. Each takes one of three principles and one of four roles, and five assignments cover the role combinations a healer or a support can take.

`src/competition/rpg_classes.py` — the principles, the roles and the arc

```
PRINCIPLES: tuple[str, ...] = (SALT, SULPHUR, MERCURY)
ROLES: tuple[str, ...] = (TANK, DAMAGE, HEALER, SUPPORT)
ARC_LEVELS = 100
FIRST_LEVEL = 1
```

The seven are Lead Ward, Tin Bulwark, Iron Edge, Solar Lance, Quicksilver Draught, Copper Conduit and Silver Mirror, each named for the metal of its planet. A class arc runs a hundred levels from level one.

The skill ladder and the transfer skill

One ladder of ten levels carries every skill. A level costs two and a half times the level below it, starting at one, and each level adds a tenth to the skill's effect. A use is weighted by its quality, and a use of quality zero advances nothing.

> *"All skills have levels that grow through use. Skills should be intelligently designed with growth curves similar to Eve Online."*

`src/competition/skill_ladder.py` — the cost and effect curves

```
UNTRAINED_LEVEL = 0
FIRST_LEVEL_COST = Decimal(1)
LEVEL_COST_MULTIPLIER = Decimal("2.5")
UNTRAINED_EFFECT = Decimal(1)
EFFECT_STEP_PER_LEVEL = Decimal("0.1")
TRANSFER_SKILL_NAME = "Quintessence Transfer"
TRANSFER_HOURS_DIVISOR = Decimal(10)
MIN_TRANSFER_HOURS = Decimal(1)
```

Exactly one skill sits on the ladder, the transfer skill, and it is the skill a Quintessence transfer is gated on.

> *"Quint is transferable between players via a specific skill isolated to common Guild members, takes significant time to complete based on amount and skill level, and has a negative effect of 'bleeding' quint back into the 'platonic' where it can be respawned and redistributed to other PoA participants."*

Three of the four properties he named are built and the fourth is not.

Term	State
gated by a skill	built - an untrained skill is refused

Term	State
lossy	built - 8 per cent at level one, 4 per cent at level ten
slow	the program answers the hours; no clock holds a transfer open
guild members only	nothing exists to check

A thousand Quintessence takes a hundred hours at level one and ten hours at level ten. No in-flight record is kept, so nothing holds a transfer in a queue. Two skill pages are named, one for a Vessel and one for a Reincarnate, and only the ladder carries a skill.

Materials, items and crafting

Seven materials are declared, all metal ores, each embedding a band of Quintessence in one unit. Four item classes are declared, and an item is a list of materials held together by cohesion.

Declared	Count	Names
Materials	7	lead, tin, iron, gold, quicksilver, copper and silver ore
Quality grades	5	Calx, Cauda Pavonis, Flores, Elixir, Magisterium
Item classes	4	armour, weapons, accessories, consumables
Storage classes	3	gear in counted slots, consumables and resources in stacks

`src/competition/conversion_rates.py` — the band one unit of the reference ore embeds

```
  IRON_ORE_QUINTESSENCE_LOW_QUALITY = Decimal("0.00000001")
  IRON_ORE_QUINTESSENCE_HIGH_QUALITY = Decimal("0.00000005")
```

An item's cohesion is the Quintessence that holds it together, and equipping is allowed only while the total Quintessence held covers that cohesion. A craft runs over several world turns, moves Quintessence out of a wallet and destroys none of it, and a finished craft puts its item in the crafting Vessel's store. A craft consumes the material list it names.

`src/competition/items.py` — the two equip answers

```
  EQUIP_ALLOWED = "the total Quintessence covers the item's cohesion"
  EQUIP_REFUSED = "the item's cohesion is above the total Quintessence held"
```

Eight mechanisms an item needs do not exist: a crafting recipe, salvage, equipping, inventory, slot count, encumbrance, loot generation and named instances. `src/competition/items.py` names all eight and says what each waits on. `src/competition/foraging.py` gathers a material from the square a Vessel stands on, and `src/competition/inventory.py` holds what one Vessel carries.

Loot

Five loot tiers share the quality grade names, and their weights sum to exactly a hundred. A qualifying market makes a drop, and the roll is drawn from a fixed seed so a run repeats.

Tier	Weight
Calx	60

Tier	Weight
Cauda Pavonis	25
Flores	11
Elixir	3.5
Magisterium	0.5

`src/competition/loot_drop.py` — the weights and the roll

```
WEIGHT_TOTAL_PCT = Decimal(100)
PERCENT_SCALE = Decimal(100)
LOOT_DROP_SEED = 1155
LOOT_FILE_VERSION = 2
UNITS_PER_DROP = 1
SHORT_FORM_CHARS = 12
```

A trimmed chart, with a tier cut, renormalises as exact fractions rather than decimals, so the remaining bands still total exactly a hundred and keep the odds they had. `tests/contracts/AcervatorLoot.t.sol` drives the loot contract and `contracts/AcervatorLoot.sol` holds it. A drop names the thing that dropped, so a loot record identifies its item rather than a tier alone.

No source names the item type a loot tier yields. A drop therefore takes that mapping from its caller and refuses a drop without one. A drop also names its holder by wallet address and names no Vessel, so the store is handed to the delivery rather than read off the drop.

Monsters

Twelve tiers run by signed depth, six ranks below the participant's level and six above. Every tier's own level is unset, so the Quintessence a tier embeds cannot be computed until a level is named.

`src/competition/monster_table.py` — the ranks and the art brief's counts

```
RANKS_PER_SIDE = 6
BRIEF_FRAMES_WITH_CAST = 26
BRIEF_FRAMES_WITHOUT_CAST = 21
BRIEF_ROSTER_FRAMES = 531
BRIEF_DESIGN_COUNT = 21
BRIEF_ATTESTED_NAMES = 21
BRIEF_TOTAL_NAMES = 23
```

Twenty-one designs are attested and twenty-three names are held in all. Nothing ties a tier to a world layer, because twelve does not divide the twenty declared layers. A monster's identity is derived from the committed world seed, the layer and the locator, so two nodes on one seed spawn the same monster in the same place. One sixty-four-digit leaf per square carries five draws, each read out of its own window of that leaf: three of sixteen digits and two of eight.

Diagram: flowchart, left to right

- world seed -> **discovery leaf of the square**
- square locator -> **discovery leaf of the square**

- discovery leaf of the square -> **digits 0 to 16: committed Quintessence**
- discovery leaf of the square -> **digits 16 to 32: monster tier**
- discovery leaf of the square -> **digits 32 to 48: monster design**
- discovery leaf of the square -> **digits 48 to 56: the material**
- discovery leaf of the square -> **digits 56 to 64: the grade**
- digits 48 to 56: the material -> **the Vessel store**
- digits 56 to 64: the grade -> **the Vessel store**

A world's tier sets the depth it reaches and the Quintessence a slaying imports into the world. `src/competition/world_tier.py` declares the first tier as one and holds no figure for the import.

Alignment, prayer and consecrated places

Alignment runs on one scale from full destruction through neutral to full creation, and it rolls up over three levels.

`src/competition/alignment.py` — the scale

```
CREATION = "creation"
DESTRUCTION = "destruction"
NEUTRAL = Decimal(0)
FULL_CREATION = Decimal(1)
FULL_DESTRUCTION = Decimal(-1)
```

A priest prays, and prayer is the first thing that writes another player's alignment. No record holds a target's consent and the prayer call asks for none. A priest also consecrates a place for a guild, and prayer holds that place blessed or lets it fall cursed. The consecration clock is the world turn.

`src/competition/consecration.py` — the two statuses and the clock

```
BLESSED = "blessed"
CURSED = "cursed"
FULL_CONSECRATION = Decimal(1)
LOST_CONSECRATION = Decimal(0)
CLOCK = "world turn"
```

Domination, and the two forms a taking takes

A Vessel must be incapacitated before it can be taken. Incapacitation comes from health at zero or from a mental effect, and a taking is either temporary control of another player's Vessel or theft of it.

`src/competition/domination.py` — the condition and the two forms

```
CONDITION_ABLE = "able"
CONDITION_INCAPACITATED = "incapacitated"
KIND_PHYSICAL = "physical"
KIND_MENTAL = "mental"
TAKING_CONTROL = "control"
TAKING_THEFT = "theft"
ACTION_CONTROL = "temporary control of another player's Vessel"
ACTION_THEFT = "theft of another player's Vessel"
```

Temporary control ends at a named world turn, so a driven Vessel returns to its owner rather than being held for ever.

The PvP vote

A world enters PvP mode on a player vote, and only in PvP mode may one player destroy another's Vessels.

> *"There will be PvP Worlds and Events. A world entering PvP mode is determined by an active Player Vote and that can put forth once every 24hrs. The voting window persists for 15m or three 5m candles. Only while in PvP mode or participating in PvP events can one player destroy another's Vessels. This will allow players to have specific Vessels they are willing to fight to the death with..."*
>
> *"51% or higher. Proper Democracy over here..."*
>
> *"Three successive pro-PVP votes over 72hrs will lock the World in PvP mode for an entire week starting from the third vote."*

`src/competition/pvp_vote.py` — the three figures he set

```
CASTING_WINDOW_TURNS = 3
VOTE_CADENCE_WORLD_TURNS = 24
RESOLUTION_WORLD_TURNS = 1
CARRY_NUMERATOR = 51
CARRY_DENOMINATOR = 100
CARRIES_TO_LOCK = 3
LOCK_LOOKBACK_WORLD_TURNS = 72
PVP_LOCK_WORLD_TURNS = 7 * 24
```

The casting window is three candles, a vote may be called once every twenty-four world turns, and a vote carries at fifty-one parts in a hundred. Three carries inside seventy-two world turns lock the world in PvP mode for seven days.

Guilds and the treasury

A guild roster holds each guild's members, its officers and the treasury it spends. Two ranks are declared, officer and member, and the treasury is a Quintessence address built from a fixed prefix and the guild's own name.

> *"only those holding PoA tokens can form a guild... guild membership locks and stakes PoA tokens, by guild rank... a guild may hold any number of members"*
>
> *"A guild holds a Quintessence treasury, filled two ways and spent one way. Filled by the transfer skill, from a member, paying the bleed. Filled by the guild's own event awards. Spent on action costs for guild members,*

inside an event. Never paid out to a member's personal wallet."

`src/competition/guild_roster.py` — the ranks and the treasury address

```
ROSTER_FILE_VERSION = 1
TREASURY_ADDRESS_PREFIX = "poa_guild_treasury_"
TREASURY_LEDGER_BUCKET = "wallets"
RANK_OFFICER = "officer"
RANK_MEMBER = "member"
```

The treasury is filled two ways and spent one way, and never pays into a member's personal wallet.

Diagram: graph, left to right

- a member's wallet -> **the guild treasury, wallets bucket** *(transfer skill)*
- a member's wallet -> **the pleroma** *(bleed)*
- the guild treasury, wallets bucket -> **the event pot, held bucket** *(an action's cost)*
- the event pot, held bucket -> **participants' wallets** *(payout by performance)*

The roster saves to a file and the Guild subtab reads it back, so the roster loads empty until something writes that file. **No control founds a guild.** `src/competition/army_command.py` links Raid parties into an army and makes a party leader a General, needing at least two linked Raids.

Linking two nodes

Two instances link over loopback so both chains hold the same records. The link refuses a message above eight mebibytes and gives up on a connection after five seconds.

`src/competition/node_link.py` — the link's bounds

```
LOOPBACK_HOST = "127.0.0.1"
DEFAULT_NETWORK = "acervator-poa"
MAX_MESSAGE_BYTES = 8 * 1024 * 1024
CONNECT_TIMEOUT_S = 5.0
```

A participant is one node holding a Quintessence wallet the ledger knows, and a bot is a source that feeds one. `src/competition/participant_node.py` holds that pairing.

Map marks

Every kind a map draws carries one glyph and one colour token. Three families carry marks and three carry none. The glyphs are stand-ins: no pixel art is drawn.

Family	Marked	Why
Vessel	yes	one planet glyph a class
Loot tier	yes	one glyph a tier
Event mode	yes	one glyph a mode
Monster kind	no	six of the twelve tiers carry no name, and no glyph set covers twelve positions
World fact kind	no	the kind is text a caller passes in, and no module under `src` names one

Family	Marked	Why
Zone terrain	no	a zone region carries no terrain kind

Each colour is a design style token rather than a literal colour, so the marks follow the application's own theme.

Every rate, and its provenance

One table carries every rate that turns one quantity into another, and each rate says where its figure came from. Thirty-eight rates are declared.

Provenance	Rates	What it means
measured	8	the figure came from running something and reading it
decided	3	he named the figure
working	27	a stand-in, and no source sets it

`src/competition/conversion_rates.py` — the three provenances

```
MEASURED = "measured"
DECIDED = "decided"
WORKING = "working"
RATE_ABSENT = None
```

Twenty-seven of the thirty-eight are working figures, so most of the conversion layer awaits a decision or a measurement.

The party row's mark

A party row carries one mark, ranked so the worst state shows. Six marks are ranked and four of the six have no source under `src`, so `participant_mark` answers none of them.

`src/gui/main_tabs/proof_of_accumulation_tab_surface.py` — the six ranked marks

```
MARK_RANKS: tuple[str, ...] = (
    MARK_DEAD,
    MARK_MISSED_WINDOW,
    MARK_OUT_OF_IMPETUS,
    MARK_AFFLICTED,
    MARK_NO_CLASS,
    MARK_ALIGNMENT_SKEW,
)
```

Dead and no class picked are answerable today. Missed the window, out of Impetus, afflicted and alignment skew wait on an action record and on the art brief's own figures.

The two meters, and the reset

Two meters sit beside each other on the screen: how full the current block is, and how much of the turn is left. Together they answer a question neither answers alone.

> *"A block fill and real time turn completion meter next to each other."*

"PoA - Demo Mode - ... Must be able to reset the testnet."

The reset control names both chain files and the bytes they hold, asks first, and acts on a second press. It refuses on the Live chain, as the world control does.

In development.

What the engine does on the Demo chain today

The controls work against the Demo TestNet chain. Pick that chain, press the identity control twice, then the distil control and the spend control. Each prints what its mechanism did, and each writes a file under the runtime directory.

```
the identity control writes   bot_identity.json
the distil control writes     quintessence_ledger_testnet.json
the spend control writes      poa_record_store_testnet.json
```

The wallet over the party window draws what the spend left.

```
distil   Distilled 0.1 Quint.
spend    Band x1 cost 0.001 Quint, resting at poa_elite_event_pot.
wallet   Quint reads -- before the distil and 0.099 after the spend
```

On a chain that has distilled, the Quint subtab names the file as held and reports the figure minted on it.

```
Ledger file      quintessence_ledger_testnet.json - held
Minted so far    0.1
Smallest unit    0.00000001
Supply cap       33000000
```

The skills subtab states the duration and the bleed, the train control records one use and reports the level it reached, and the Guild subtab reports the roster.

```
duration      1000 Quint takes 100 hours at level 1 and 10 hours at level 10.
train         Quintessence Transfer stands at level 1 on 1.0 weighted uses.
standing      level 1 - effect 1.1x - bleed 8.00% - transfers sent 0
guild         Guild, Ctrl+7 - 0 guilds, 0 members, ranks officer and member
return pool   Return pool, 75%
```

The Quint subtab also states what no subtab draws.

```
No block height and no merkle root is drawn. merkle_log writes the trade log
and no subtab reads its root.
```

What is not built

No tab draws any of it. The window skips the Accumulation tab, builds neither the Competition tab nor the Local Testnet tab, and nothing draws a block explorer.

Absent	Count	What is missing
Tabs	3	Accumulation, Competition, Local Testnet
Pixel art	all	every class, loot tier and event mode shows a stand-in glyph
Vessel mechanisms	8	assignments, lifeskilling, crafting, notifications, gear, equipping, destruction, permadeath
Item mechanisms	8	a crafting recipe, salvage, equipping, inventory, slot count, encumbrance, loot generation, named instances
Metrics with no field	7	experience, level, character class, gear, enemy, threat, guild
Party marks with no source	4 of 6	missed the window, out of Impetus, afflicted, alignment skew
Map mark families	3 of 6	monster kind, world fact kind, zone terrain
Unset figures	2	the emission an activation awards, the level each monster tier sits at
Working conversion rates	27 of 38	no source sets the figure

Nothing founds a guild, nothing advances the season, and no contract is deployed on either network. The emission figure is the sharpest gap: an activation refuses for want of it, so no Quintessence award can pass the capture bounds until it is set.

Market Inspector Tab

Reference. The higher-timeframe scanner, the ATA-SMP publisher and the topology proposal pane. First step of the promotion pipeline.

What builds it

`MarketInspectorTabMixin._build_market_inspector_tab` in `src/gui/main_tabs/market_inspector_tab.py` constructs the tab and wires its three injection points before adding it to the row.

The tab splits horizontally. The left half holds the scanner and the ATA-SMP publisher. The right half holds the approval bucket, the proposal cards and the Phantom Bot zone.

The six zones

The screen draws six zones, three down each side. ATA-SMP, Opposing Trades and Multi-Exchange Arbitrage run down the left. ATA-SMP Ready to Send, Bot Swarm Topologies and Phantom Bot HTF Signals run down the right. One function names each side in screen order, so a zone cannot appear on one host and go missing on the other.

`src/gui/main_tabs/market_inspector_surface.py` — the rows each side draws

```
def left_module_rows(
    run: Any,
    scan_state: Any,
    pair_count: Any,
    connectors: Any,
    sector_count: Any = 0,
) -> list:
    """The three left-side regions as key, title and status, in screen order."""
```

The left pane divides its height by a fixed share. ATA-SMP takes two shares and the other two zones take one each.

`src/gui/main_tabs/market_inspector_surface.py` — the heights the zones take

```
LEFT_MODULE_SHARES = (2, 1, 1)
```

Every zone works the same way. It shows one entry at a time, with a left arrow, a right arrow and a line saying which entry is on screen out of how many the zone holds. An arrow press wraps at both ends, so the last entry steps forward to the first.

`src/gui/main_tabs/market_inspector_surface.py` — what one arrow press does

```
def step_to(at: Any, total: Any, by: Any) -> int:
    """The entry index one arrow press moves to, wrapping at both ends."""
```

Clicking the entry opens it. The four expanded lines are the test, the window, the result, and why that answer let the entry through. Every zone's open entry takes one height whatever buttons it carries.

`src/gui/main_tabs/market_inspector_surface.py` — the four names an open entry carries

```
DETAIL_TEST_NAME = "Test"
DETAIL_WINDOW_NAME = "Window"
DETAIL_RESULT_NAME = "Result"
DETAIL_REASON_NAME = "Why it is here"
```

None of the four ever draws a name with nothing beside it. "Why it is here" reads `MethodResult.gate_text`, which names the threshold a verdict cleared, or, for one that ran and did not clear it, the one `failed_reason` names: the p-value above the significance, the trace statistic short of its critical value, or both. "Result" reads `statistic_text`. A test that did not run states no statistic and no reason to state, and both cells draw the dash this tab draws for every figure it cannot state.

`src/trading/pair_selection.py` — what a verdict that did not pass reads

```
GATE_P_ABOVE_FORMAT = "p {p_value:.4f} is above the {significance} required."
GATE_TRACE_SHORT_FORMAT = (
    "The trace statistic {trace:.1f} did not clear the {critical:.1f} required."
)
GATE_NO_REASON_TEXT = "—"
```

A zone holding nothing keeps its waiting sentence as the headline and reports a position of "0 of 0". An empty zone reads as a state rather than as a zone that failed to draw.

Zone	What its line says while it holds nothing
ATA-SMP	"No sector added. Name one and press Scan Now."
Opposing Trades	The scan state: not asked, running, or finished with its three counts
Multi-Exchange Arbitrage	The venues in reach, or that no exchange source is wired
ATA-SMP Ready to Send	"The last scan left no post. Nothing to approve."
Bot Swarm Topologies	One of three sentences, by scan state
Phantom Bot HTF Signals	"Phantom Bot source not wired."

The Phantom Bot HTF Signals row above is overtaken, and it is kept as written. The line now names the state of its source. With no bot lookup wired, it says the source is not wired. With the lookup wired and no bot running a phantom timeframe, it counts the bots it read.

`src/gui/main_tabs/market_inspector_surface.py` — `phantom_zone_text`

```
  if bots is None:
      return PHANTOM_HTF_UNWIRED_TEXT
  return PHANTOM_HTF_NONE_FORMAT.format(count=len(list(bots)))
```

The Qt build gives every zone a `ProposalStepper`. The two web hosts give every zone a `ZoneStepper`. Both are written from one description of the zone, so a zone's lines are decided in one place.

`src/gui/market_inspector.py` — the Qt widget every zone uses

```
  class ProposalStepper(QWidget):
      """One zone's entries shown one at a time, with arrows and a expansion.
```

`src/gui/main_tabs/market_inspector_surface.py` — the rest of the zone description

```
  LEFT_MODULE_KEYS = (ATA_SPM_MODULE, OPPOSING_TRADES_MODULE, ARBITRAGE_MODULE)

  def right_zone_rows(run: Any, bucket: Any = None) -> list:
      """The three right-side zones as key, title and status, in screen order."""

  def zone_view(run: Any, bucket: Any, board: Any) -> dict:
      """One zone's headline, position line, counter, entry rows and actions."""
```

The window sizes each group from `LEFT_MODULE_SHARES`, and the page turns the same tuple into a flex value, reading an equal share for anything that is not a positive number.

`src/gui/web/market_inspector.js` — the share one zone takes on the page

```
  const EQUAL_SHARE = 1;
  const SHARE_FLEX_TAIL = 0;
  function shareFlex(share) { ... }
```

Left half: the scanner

The signal tables

A filter row carries a Refresh button and an "Include active markets" checkbox. The default hides the markets already under a bot, which keeps the table on entry opportunities.

`HTF Signals` lists one row per market: Asset, Signal, Score, Daily, Weekly, Active.

`Opposing Pairs (cointegration, Engle-Granger + Johansen, p<=0.05)` lists Long side, Short side, Method, Window, Statistic, Correlation and the combined score.

The ATA-SMP control row

The zone draws three lines. The ticker field sits on the first line with the sector menu beside it. The four timeframe buttons sit under their own heading on the second. Scan Now, Scan All and Settings sit on the third. Every control sits inside the pane from 700 pixels of tab width upward, with no scroll bar.

`src/gui/main_tabs/market_inspector_surface.py` — the three line names

```
TIMEFRAME_TITLE = "Timeframe"
SECTOR_ROW_PART = "sector-row"
SCAN_ROW_PART = "scan-row"
```

Each group of buttons breaks at the count the pane's own width holds. The window asks the same function the page asks, so both hosts wrap at one count.

`src/gui/main_tabs/market_inspector_surface.py` — how many cells fit a line

```
def columns_for(
    available: int, cell: int, spacing: int = SETTINGS_ROW_SPACING_PX
) -> int:
```

Both hosts read the same pane width, because the window sets the splitter handle to the theme's own value rather than to the toolkit's default, and the page leaves the same gap.

Tab width	Venue and category buttons on a line	Settings rows on a line
700	2	1
900	3	1
1400	6	2
1960	8	2

`src/gui/main_tabs/market_inspector_surface.py` — the gap between the two panes

```
SPLITTER_HANDLE_PX = 7
BUTTON_COLUMNS = 4
```

The timeframe grid wraps after `BUTTON_COLUMNS` cells, as the Level 1 groups do. A ticked timeframe, a held venue credential and the chosen asset class are one state and take one look, and the field's own lines take one size in both hosts.

`src/gui/main_tabs/market_inspector_surface.py` — the look a pressed control takes

```
CHECKED_BUTTON_STYLE = (
    f"background:{ds.PRIMARY}; color:{ds.ON_PRIMARY}; border-color:{ds.PRIMARY};"
)
DETAIL_STYLE = "color: #ccc; font-size: 11px;"
```

Every row of the field takes that one size, in both hosts.

The asset classes and their sectors

Four classes ship. The published taxonomies name them: the S&P GSCI methodology makes commodities an asset class with sectors under it, and ISO 10962, the CFI standard, sorts each product by its own form rather than by a class of its own.

> *S&P GSCI groups energy as petroleum and natural gas*

`src/trading/ata_spm.py` — the class set every button, menu and scan reads

```
ASSET_CLASSES = (CLASS_CRYPTO, CLASS_STOCKS, CLASS_COMMODITIES, CLASS_FOREX)
```

OVERTAKEN, and the sentence and block above are kept as written. Six sectors ship. The two added are Indices and Futures / Perps, and the class box, the class table and every scan read all six.

`src/trading/ata_spm.py` — the sector set every button, menu and scan reads

```
ASSET_CLASSES = (
    CLASS_CRYPTO,
    CLASS_STOCKS,
    CLASS_COMMODITIES,
    CLASS_FOREX,
    CLASS_INDICES,
    CLASS_FUTURES_PERPS,
)
```

Neither added sector carries a listed market yet, so a scan of either walks no sector at all. The map store answers an empty set for a class it holds no entry for, which is what it already does for crypto.

The two sentences above are overtaken for Futures / Perps, and both are kept as written. A Futures / Perps scan now walks the venue's futures list, because every product on a crypto root is placed there. An Indices scan walks the same list for a product on an index root.

The operator asked for the published names rather than this project's own.

> *I just want to adopt the standard categorization and language so that the implementation is professional and accurate...*

A retired class name still resolves.

The name given	The class it answers
`metals`	commodities
`energy`	commodities

The name given	The class it answers
`derivatives`	crypto, because a futures product takes the class of its underlying
a name neither taxonomy holds	none; the press is refused

The `derivatives` row above is overtaken, and it is kept as written. The name now answers futures_perps, the class the contract rule files a contract on a crypto underlying under.

A refused name runs no scan, so no class is scanned with no timeframes behind it.

`src/trading/ata_spm.py` — the class one name resolves to

```
def asset_class_named(name: Any) -> str:
```

`src/trading/ata_asset_maps.py` — the sector one name resolves to

```
def sector_named(typed: Any) -> str:
```

The sectors each class holds, and how many markets the shipped map lists in each. A crypto list and a stocks list are read at press time instead, from the venue's own products list and the screener, so those two counts move with the venue.

Class	Sector	Markets the map lists
crypto	by sector tag	read at press time
stocks	portfolio	47
commodities	energy	4
commodities	industrial metals	2
commodities	precious metals	8
commodities	agriculture	0
commodities	livestock	0
forex	major	7
forex	minor	21
forex	exotic	0

A sector holding no market is not drawn as a sector the scan can walk. Its count is drawn under the ticker field with the reason it holds nothing.

`src/gui/main_tabs/market_inspector_surface.py` — the line an empty sector takes

```
TICKER_EMPTY_SECTORS_FORMAT = "{sectors}: 0 market(s) listed, {reason}."
```

Commodities reads "agriculture, livestock: 0 market(s) listed, no fund listed in dollars holds them, and this map carries no futures." Forex reads "exotic: 0 market(s) listed, no pair is named for this tier yet."

One function builds that line for any class, from the sectors the map declares and lists nothing for.

`src/gui/main_tabs/market_inspector_surface.py` — the line, and what it reads

```
  def empty_sector_line(asset_class: Any) -> str:
      """``TICKER_EMPTY_SECTORS_FORMAT`` over ``ata_asset_maps.unlisted_sectors``."""
```

`src/trading/ata_asset_maps.py` — the sectors a class holds, and the ones it lists nothing for

```
  def sectors_for(asset_class: Any) -> tuple:
  def unlisted_sectors(asset_class: Any) -> tuple:
```

`src/trading/ata_spm.py` — the name a typed sector's own rows carry

```
  def sector_key_of(rows: Any, typed: Any) -> str:
```

A typed sector takes the name its own rows carry, so a retired sector name draws the headline of the sector holding its markets.

What each class can reach, and what it can trade

Every class answers two questions apart. The first is whether a retail trader can reach the market from home. The second is whether a venue is wired here that can place an order on it. ATA-SMP scans a market either way, for its charts.

Class	Reachable from home	Venue wired
crypto	yes, a retail account on a crypto venue takes orders over its API	yes, the connector the live bots trade on
stocks	yes, a retail broker account takes equity orders over its API	no, no broker connector is reachable
commodities	yes, every listed row is an ETF share a retail broker account buys	no, the same missing broker connector
forex	yes, a retail account at a registered dealer takes spot orders	no, no dealer connector exists

The four commodity spot pairs carry no venue at all. A futures row in any class is reachable and unwired: the venue takes orders on its US futures products, and the futures adapter reads candles and places none.

The table above gains a row, and it is kept as written. Futures / Perps answers what a futures row answers, because `CLASS_DEPLOYABILITY` holds the futures form's own record for it.

Class	Reachable from home	Venue wired
futures_perps	yes, the venue takes orders on its US futures products over the same API	no, the futures adapter reads candles and places none

The form each listing carries

Every row draws the form its product takes. The form rides on the row; no control was added and no menu gained an entry.

Listing	Form drawn
a crypto pair on the exchange	`spot`
a commodity spot pair	`spot`
a fund share	`ETF`
a portfolio equity or a screener quote	`equity`

Listing	Form drawn
a dated venue contract	`future`
a perpetual venue contract	`perpetual`

`src/trading/ata_asset_maps.py` — where a futures product is placed

```
def futures_placement(product: Any) -> tuple[str, str, str]:
    """The underlying, the class and the sector one futures product takes."""
```

A root neither the record nor the root table resolves is held back, named on the order line and counted. Nothing is guessed.

The placement above now ends in the contract rule, and the block is kept as written. A crypto root answers Futures / Perps, and an index root answers Indices with no sector.

```
# src/trading/ata_asset_maps.py, in futures_placement
    root = _root_of(product)
    if not root:
        return "", NO_ASSET_CLASS, ""
    if not _non_crypto(product) and (root in CRYPTO_SYMBOLS or _crypto_sector(root)):
        return root, contract_class(CLASS_CRYPTO), _crypto_sector(root)
    held = NON_CRYPTO_ROOTS.get(root)
    if held is None:
        return root, NO_ASSET_CLASS, ""
    return held[2], contract_class(held[0]), held[1]
```

The ticker field

The field names one market, read on demand. Typing narrows a list the program already holds. Nothing is fetched on a keystroke and no venue is asked.

`src/gui/main_tabs/market_inspector_surface.py` — the field and what it says

```
TICKER_FIELD_PLACEHOLDER = "Ticker"
TICKER_FIELD_TOOLTIP = (
    "Name one market to read on demand. Typing offers the tickers the "
    "sector menu beside it holds."
)
```

The operator named the field, and said what it is for.

> *Sector Field - Incorrect naming... This entry field is for reading specific, individual markets on demand and will require TICKER recognition...*

Each offer names its class, its form, whether a bot can deploy there, and whether a bot can size a scrum on it. Offers come prefix matches first, the chosen class first inside each, then matches holding the text later in the name.

`src/gui/main_tabs/market_inspector_surface.py` — one offered row

```
TICKER_OFFER_FORMAT = "{symbol}  ({asset_class} · {form} · {deploy} · {tradeable})"
TICKER_MATCH_LIMIT = 8
```

Typed text is folded to upper case, outer spaces are stripped, and a hyphen, an underscore or an inner space reads as a slash. The folded text is matched against each class's names, the chosen class first.

`src/gui/main_tabs/market_inspector_surface.py` — the separators one name may carry

```
TICKER_SEPARATORS = ("-", "_", " ")
TICKER_JOIN = "/"
```

Typed	Resolves to	Class
`a15`, `A15/USD`, `a15-usd`, `A15USD`	A15 on the exchange	crypto
`gld`, `GLD/USD`	GLD on the chart venue	commodities
`eur/usd`, `eurusd`, `eur usd`	EUR/USD on the chart venue	forex
`A15/EUR`	nothing; that quote is not one the connector route reads	—
`ZZZQ`	nothing; no class lists it	—

A name the chosen class does not hold but another class does moves the class box to the class holding it, redraws the four timeframe buttons for that class, and writes one line to the Activity Log.

`src/trading/ata_spm.py` — the line a move writes

```
CLASS_MOVED_TEXT = (
    "ATA-SPM ticker {ticker} is listed under {placed}; "
    "the class box moves from {chosen} to {placed}"
)
```

`src/gui/main_tabs/market_inspector_surface.py` — the rows one keystroke offers

```
def ticker_offers(typed: Any, asset_class: Any, connectors: Any = None) -> TickerOffers:
    """The offered rows and the names left off them, together."""

def class_tickers(asset_class: Any) -> list:
def class_walk(asset_class: Any) -> list:
```

`class_walk` answers `ata_spm.ASSET_CLASSES` with the chosen class first, so a typed name is matched against that class before any other.

`src/gui/main_tabs/market_inspector_surface.py` — where one typed name lands

```
def market_listing(ticker: Any, asset_class: Any) -> Optional[Any]:
    """The ``ata_spm.TickerPlacement`` a typed ticker names, across every class."""

def placements_of(typed: Any, asset_class: Any) -> list:
```

`src/trading/ata_spm.py` — what a placement carries

```
@dataclass(frozen=True)
class TickerPlacement:
    """One typed name, the listing it resolved to, and the class holding it."""

    listing: Any
    asset_class: str
    typed: str = ""
```

A name no class holds is refused under the field, nothing runs, and no venue is asked.

`src/trading/ata_spm.py` — the refusal a name no class holds takes

```
TICKER_UNHELD_FORMAT = "No class lists ticker {ticker}. Pick one the field offers."
```

`src/gui/main_tabs/market_inspector_surface.py` — the line a sector with no list takes

```
TICKER_NO_LIST_FORMAT = "No ticker list for {sector}. A typed name still scans."
```

A sector whose list is read at press time says so instead, and a typed name still scans.

`src/gui/main_tabs/market_inspector_surface.py` — the line a read-at-press list carries

```
TICKER_PRESS_LIST_FORMAT = (
    "Scan Now reads the {sector} list from {source}. A typed name still scans."
)

def ticker_note(asset_class: Any, connectors: Any = None) -> str:
    """The one line under the field: the empty sectors, the press list, or a refusal."""

def ticker_offer(listing: Any, asset_class: Any, connectors: Any = None) -> str:
```

The Qt completer shows the offered row and writes the bare symbol into the field. The page's option carries the offer as its label and the symbol as its value.

Whether a bot can size a scrum

One function answers whether a bot can size a scrum on a market. A scrum sells the excess above a dollar target, and a venue accepts an order only at or above its own published minimum, stepped onto its own increment.

`src/trading/scrumming/sizing.py` — the three answers

```
def tradeable_answer(
    rules: Any, price: Any = None, excess_usd: float = REFERENCE_SCRUM_EXCESS_USD
) -> str:
```

Answer	Drawn as	What it means
yes	`can size a scrum`	the smallest order the venue accepts costs no more than the excess
no	`cannot size a scrum`	it costs more, so a scrum would have nothing to submit

Answer	Drawn as	What it means
unknown	`size rules not read`	no market record was obtained

An unknown is never read as either other, and such a market stays on the list. A market answering no is kept off the offered list, counted and named.

`src/gui/main_tabs/market_inspector_surface.py` — the line a refused market takes

```
TICKER_UNTRADEABLE_FORMAT = (
    "{count} market(s) left off, the smallest order costs more "
    "than a scrum's excess: {names}"
)
```

The excess the answer measures against is the largest one the saved fleet can produce. The highest Target Balance in the fleet is 350 dollars and every bot carries the same 5 per cent scrumming interval, so the largest excess is 17 dollars 50.

`src/trading/scrumming/sizing.py` — the reference excess

```
LARGEST_FLEET_TARGET_USD = 350.0
FLEET_SCRUMMING_INTERVAL_PCT = 5.0
REFERENCE_SCRUM_EXCESS_USD = scrumming_interval_usd(
    LARGEST_FLEET_TARGET_USD, FLEET_SCRUMMING_INTERVAL_PCT
)
```

OVERTAKEN, quoted whole:

> *The excess the answer measures against is the largest one the saved fleet can produce. The highest Target Balance in the fleet is 350 dollars and every bot carries the same 5 per cent scrumming interval, so the largest excess is 17 dollars 50.*

True today: the screen reads the fleet each time it labels a row, and it says on the screen which figure it used. The two constants were renamed, because neither one reads the fleet.

`src/gui/main_tabs/market_inspector_surface.py, in fleet_scrum_excess` reads `bot_state.json` read-only and answers the largest excess the saved records produce, with a count of how many gave one. It never writes that file. `src/gui/main_tabs/market_inspector_surface.py, in excess_note` then names the figure: the fleet sentence where the count is one or more, and the assumed sentence where the count is zero.

`src/gui/main_tabs/market_inspector_surface.py` — the two sentences the note carries

```
TICKER_UNTRADEABLE_MEASURED_FORMAT = (
    "{count} market(s) left off, the smallest order costs more than a scrum's "
    "excess, measured {note}: {names}"
)
```

A saved fleet reads, for example, `against $20.00, the largest scrum excess the 2 saved bot(s) produce`. No saved fleet reads `against an assumed scrum excess of $17.50, from a $350.00 Target Balance at a 5% interval`, which names the assumption as an assumption.

`src/trading/scrumming/sizing.py` — the assumed pair, which reads no fleet

```
ASSUMED_TARGET_BALANCE_USD = 350.0
ASSUMED_SCRUMMING_INTERVAL_PCT = 5.0
REFERENCE_SCRUM_EXCESS_USD = scrumming_interval_usd(
    ASSUMED_TARGET_BALANCE_USD, ASSUMED_SCRUMMING_INTERVAL_PCT
)
```

A row the ticker field labels is labelled before any bot exists, so the fleet's largest excess is the widest figure the screen can honestly use. A bot about to place an order is a different case: `src/trading/bot_container.py, in guarded_place_order` passes that bot's own excess, through `src/trading/scrumming/sizing.py, in bot_scrum_excess_usd`.

The venue's published minimum is ceiled onto its own increment, valued at the venue's own last price, and never falls under the venue's minimum order cost.

`src/trading/scrumming/sizing.py` — the smallest order the venue accepts

```
def smallest_order_usd(rules: Any, price: Any) -> Optional[float]:
```

`src/exchange/market_inspector_fetcher.py` — where its rules come from

```
def trading_rules(
    exchange_connectors: Any, accepted_quotes: Any = DEFAULT_QUOTES
) -> dict[str, tuple]:
```

The rules and the price come off the market table each connector has already loaded, as one `MarketRules` row per market. No venue is asked. The crypto rows the sector scan lists carry no size rules, so on that path the answer reads unknown for every crypto row; the ticker field reads rules off each connector's table and answers all three.

The timeframe row

The asset class sets which four timeframes the buttons offer. Each button draws on while its timeframe is ticked, and reads its key at the press from the row the class shows. Crypto is the fast set and every other class shares the slower one.

`src/trading/ata_spm.py` — the two sets

```
CRYPTO_TIMEFRAMES = ("5m", "1h", "1d", "1w")
SLOWER_TIMEFRAMES = ("1h", "1d", "1w", "1M")
```

A fresh board carries the whole set its class lists, so all four draw on until one is pressed off. A timeframe press made before any sector exists is held on the board and given to the sector the next press adds.

Scan Now

Scan Now reads one press. What it reads depends on the field.

The field	What the press reads
a ticker the class lists	that one market, on every ticked timeframe

The field	What the press reads
a sector name the map lists	that sector's own assets
empty	the class's markets in its own order, until the hit target lands

`src/gui/main_tabs/market_inspector_surface.py` — the button and what it says

```
SCAN_NOW_LABEL = "Scan Now"
SCAN_NOW_TOOLTIP = (
    "Scan this sector now on the timeframes ticked beside it, without "
    "waiting for a rotation."
)
```

The operator asked for the empty field to walk the sector by volume.

> *ATA-SMP - Scan Now - Seems to be expecting a Ticker field entry. This is incorrect. If Ticker field has no entry, this button must scan sector's markets (on each selected TF) in order of decreasing volume until x total number of "hits" (across all selected TFs for the sector; add under settings, default is 3) are found.*

He had asked for the button twice before it worked.

> *ATA-SMP - Please proceed back to ATA-SMP and get my Scan button working. Unitize and apply same work flow under the correct Issue. It still does nothing as of the build I am currently running.*
>
> *Simulator - Scan Now - I should not have to have anything in the Ticker field. Stocks is saying 'no ticker list'. This is scaffolded, unverified garbage. I watched you perform various operations while working on this. How the fuck is it that after I launch the build, not a god damn thing has improved with this fucking button.*

Every mention of the Simulator in those words names this tab.

> *Make sure you correct my incorrect verbatims because SImulator does not have Scan Now...Inspector and ATA-SMP do.*

The button is disabled and reads `Scanning…` from the press until the answer or the failure crosses back. A press made while a scan runs writes a busy line to the Activity Log at warning.

`src/trading/ata_spm.py` — the label a running scan draws

```
SCAN_BUSY_LABEL = "Scanning…"
SCAN_BUSY_TEXT = "ATA-SPM scan pressed while a scan is running; press ignored"
```

Scan All

A second button sits beside Scan Now at its size, in both builds. One press walks every class the sector menu holds, in menu order, on every timeframe that class scans, over every market its list holds, with no stop at a hit count.

`src/gui/main_tabs/market_inspector_surface.py` — the second button

```
SCAN_ALL_LABEL = "Scan All"
SCAN_ALL_PART = "scan-all"
```

The operator asked for it, and for the confirmation timers beside it.

> *ATA-SMP - Add a Scan All button which will scan every market in every TF for every market in every sector. Have noticed that scans run really fast which is good but just do not to risk overloading a free API or something. For the attached image, I want the Confirmation Timers for prior Hits to appear here. Can be just a small ticker pair with a theme-consistent timer which triggers the rescan on completion.*

A Scan Now pressed after a Scan All drops those sectors before it walks, so it walks its own class and not every class.

`src/trading/ata_spm.py` — what the counter says during a walk

```
SCAN_ALL_PROGRESS_FORMAT = (
    "Scanning {asset_class} ({at} of {sectors}) · {read} of {total} · {hits} hit(s)"
)
SCAN_ALL_LISTING_FORMAT = (
    "Scanning {asset_class} ({at} of {sectors}) · reading the market list"
)
```

The hits count over every sector, because the chime sounds when that count rises. A per-sector reset would leave the second sector's first hits silent.

The pace a walk holds

Scan All keeps every read to a host at least a published gap. The crypto venue publishes ten requests a second for its public endpoints; the chart venue publishes no limit, so a refusal is its only signal.

`src/trading/ata_spm.py` — the gaps and the waits

```
EXCHANGE_PACE_S = 0.2
YAHOO_PACE_S = 0.5
EXCHANGE_RATE_LIMIT_WAIT_S = 10.0
YAHOO_RATE_LIMIT_WAIT_S = 60.0
```

The gap is measured from the end of the last read. A refusal is read as a wait, never as a burst: the walk holds that host for the venue's own retry figure when the answer carried one, else the table's seconds, and reads that market once more. A second refusal stands. Scan Now reads through no pace.

`src/trading/ata_spm.py` — the read every walk goes through

```
def paced_read(pace: Any, host: Any, read: Any) -> tuple:
    """One read held to ``pace`` on ``host``, retried once on a rate refusal."""
```

A walk hands both its vote step and its chart step one market memo, so a venue is asked once per market per timeframe rather than twice.

`src/trading/ata_spm.py` — the memo both steps read

```
class MarketMemo:
    """One market's candles, read once and answered to every later caller."""
```

The order a walk reads

Every class is walked largest first, by the figure that class publishes. The open entry's first line names the source, the markets read in order, and the count not read.

Class	The figure the order uses
crypto with a connector	the venue's own 24-hour quote volume
crypto with no connector	name order on the public products list, no volume figure
stocks	the screener's own volume ranking
commodities	the last complete daily bar's volume times its close
forex	map order; every pair's bar carries volume 0, so no figure exists

`src/gui/main_tabs/market_inspector_surface.py` — the ranked list one class answers

```
  def class_markets(asset_class: Any, connectors: Any = None) -> list:
      """Every market a class holds as an ``ata_spm.MarketOrder``, largest first."""

  def class_volumes(exchange_connectors: Any) -> dict:
  def listing_volumes(rows: Any) -> dict:
```

A stablecoin is dropped from a crypto ordering. A market the venue no longer trades is placed on the order's own dead list rather than in the walk.

`src/trading/ata_asset_maps.py` — the figure a non-crypto class is ranked by

```
  VOLUME_TIMEFRAME = RA_TIMEFRAME
  VOLUME_WINDOW_DAYS = 7
  NO_VOLUME_FIGURE = 0.0

  def venue_quote_volume(listing: Any, read: Any = None) -> Optional[float]:
      """``complete_bar``'s volume times its close, or ``NO_VOLUME_FIGURE``."""
```

The bar measured is the newest one stamped before today, which is a closed session, so the figure does not move through the day. Volume times close puts it in dollars beside crypto's quote volume.

`src/trading/ata_spm.py` — the lines the order takes

```
  VOLUME_LINE_FORMAT = "{asset_class} by volume"
  MAP_ORDER_LINE_FORMAT = "{asset_class} in map order"
  ORDER_LINE_FORMAT = "Order by {source}: {markets}"
  ORDER_UNREAD_FORMAT = "{markets} ({unread} not read)"
  ORDER_UNFIGURED_FORMAT = "{source}, {unfigured} with no figure last by name"
  MAP_ORDER_SOURCE_TEXT = "map order, no volume figure"
```

A market the venue no longer trades never reaches the list and is never fetched. A futures product the venue answers no candle for on any granularity is dropped where the list is built, and both counts sit on the order line.

`src/trading/ata_spm.py` — the two tails the order line carries

```
ORDER_DEAD_FORMAT = "{line}; {count} not trading, never fetched: {names}"
ORDER_NO_CANDLE_FORMAT = (
    "{line}; {count} dropped, the venue answered no candle "
    "on any granularity: {names}"
)
```

A dropped product stays reachable by name: typed into the ticker field it still resolves and still scans. The candle route serves two granularities and the weekly line rolls up from the daily one, so the read asks the hourly granularity, then the daily one, and stops at the first that answers.

`src/trading/ata_asset_maps.py` — the two readings behind the rule

```
def venue_granularities(venue: Any) -> tuple:
def candle_served(symbol: Any, read: Any = None) -> bool:
```

A hit, and the target that stops a walk

A hit is a vote the live trade gates would fire on. The zone's count, the Activity Log's hit lines and the approval bucket count one thing.

`src/trading/ata_gate_scan.py` — the one verdict

```
    @property
    def would_fire(self) -> bool:
        """True while either chain would fire, which is what a post needs."""
        return self.firing_side != NO_SIDE
```

Each market is read on every ticked timeframe first, because the gate chains read a market's other timeframes through the panel rows. Votes are then judged in timeframe order. The hit that reaches the target ends the walk, and markets after it are not read.

`src/trading/ata_spm.py` — the target, and what an unreadable count reads as

```
DEFAULT_HITS_PER_SCAN = 3

def hits_target(asked: Any) -> int:
    """``asked`` as a whole number, or ``DEFAULT_HITS_PER_SCAN`` below one."""
```

An empty box, a zero and a negative number all read as three, so no scan can stop before it has read anything.

A reversal vote is a market on one timeframe whose twelve-voter consensus and its Bollinger voter name the same non-neutral direction. It is still read and still shown, in each sector's reversal-call count, and it no longer decides what publishes.

`src/trading/ata_spm.py` — what a reversal vote is

```
    @property
    def is_reversal(self) -> bool:
        """The consensus and the band voter naming one non-neutral direction."""
        if SignalDirection.NEUTRAL in (self.direction, self.band_direction):
            return False
        return self.band_direction == self.direction
```

`src/trading/ata_spm.py` — the lines each kind of entry takes

```
VOLUME_META_FORMAT = "{read} market(s) read · {hits} hit(s) · {stop}"
STOPPED_AT_TARGET_TEXT = "stopped at target"
SECTOR_EXHAUSTED_TEXT = "sector exhausted"
SECTOR_LINE_FORMAT = "{sector} ({asset_class})"
SECTOR_META_FORMAT = "{assets} asset(s) · {votes} vote(s) · {calls} reversal call(s)"
MARKET_LINE_FORMAT = "{ticker} in {asset_class}"
MARKET_META_FORMAT = "1 market on {venue} · {votes} vote(s) · {hits} hit(s)"
MARKET_TIMEFRAME_FORMAT = "{candles} candle(s) · {reading}"
MARKET_HIT_READING = "{direction} vote, hit"
MARKET_REFUSED_READING = "{direction} vote, refused by the gates"
MARKET_NO_VOTE_READING = "no vote"
```

The zone's status line counts sectors and markets apart, so a run holding both counts both.

`src/trading/ata_spm.py` — what one walk answers and how far it goes

```
NO_HIT_TARGET = 0

def walks_order(scan: Any) -> bool:
    """True for a ``walk_all`` sector and for a ``hit_target`` above ``NO_HIT_TARGET``."""
```

A board builds one walking sector per class, tells the counter which list it is reading, and runs them. The walk hands a callable one progress record after each market and each hit, and one builder writes the line each record carries.

`src/trading/ata_spm.py` — the walk and the records it hands back

```
def compute_all(self, ticked: Any, hits: Any) -> None:
def _scan_until_hits(sector: Any, sources: Any, target: Any, on_progress: Any) -> tuple:
def market_line(progress: Any) -> str:

class ScanProgress:
class WalkProgress:
```

What the field draws while a walk runs

The field draws the Indicator Voting Panel of the market under read, and nothing else. The panel is one row per timeframe that answered candles, one cell per voter, plus the Net, Comp and Conf columns, in the colours the Live tab's own panel uses. The title names the market, the word `reading`, and the tally of bullish, bearish and neutral votes.

`src/gui/main_tabs/market_inspector_surface.py` — one builder, two callers

```
def panel_grid(symbol: Any, label: Any, rows: Any, lines: Any = ()) -> Optional[dict]:
    """One voting grid, for a called asset or for the market under read."""
```

The operator asked for that panel and for the text to go.

> *ATA-SMP - The scanner freaking works but we do have some issues. Derivatives sector list is empty. Metals only has four in its list and I am pretty sure its a more complex sector than this but could be wrong. I see a series of floating windows rapidly appearing and disappearing over the ATA-SMP panel once the scan gets going. The field where the IVP fields are supposed to appear during the read initially displays text but this gets very small very fast before disappearing completely and the floating windows appear with some containing text before vanishing. I was able to get three hits and three charts generated. Very good progress.*
>
> *Observed a very strange issue with the lower information panel when attempting to scroll down to see more text. This seemed to trigger a chart re-read loop of some kind. During this I saw the rapidly spawning and despawning windows again. Need to get rid of the informational text during reads and only display the really nice looking IVP information that has only been visible in the larger floating windows up to this point. The IVP data keeps the symbology consistent and its really well designed from what I have seen of it. We have no use a for a repeating spool of identical data lines.*

The field holds where the operator left it. Neither host drags its scroll to the bottom on a progress. The grid asks 592 pixels: a 44-pixel timeframe column, six 58-pixel voter columns, two 52-pixel columns and a 96-pixel Conf column. It draws whole at a 1400-pixel tab. At 900 and 700 the last three columns sit past the pane edge with a horizontal scroll bar under them.

`src/gui/main_tabs/market_inspector_surface.py` — the grid, its widths and its two callers

```
PANEL_SUMMARY_FORMAT = "▲ {bullish}  ▼ {bearish}  ■ {neutral}"

def panel_column_widths() -> tuple:
def panel_table(rows: Any) -> list:
def panel_summary_text(rows: Any) -> str:
def panel_tally(row: Any, direction: Any) -> int:
def scan_panel(progress: Any) -> Optional[dict]:
```

`voting_panel` is the called asset's grid, drawn in an open entry with its gate chain lines under it. `scan_panel` is the same grid for the market under read, with no gate line, because no chain has run over that market.

`src/trading/ata_spm.py` — the rows one market answers

```
def panel_rows_for(symbol: Any, timeframes: Any) -> dict:
```

The per-market read line still exists, with the same four things, written to the Activity Log instead of the field.

`src/trading/ata_spm.py` — the line each market read writes

```
SCAN_MARKET_LINE_FORMAT = "{symbol} · {timeframes} · {votes} vote(s) · {verdict}"
SCAN_MARKET_HIT_FORMAT = "hit on {labels}"
SCAN_MARKET_BLOCKED_FORMAT = "no hit: the gates blocked {names}"
SCAN_MARKET_NO_VOTE_TEXT = "no hit: no vote"
SCAN_MARKET_UNREAD_TEXT = "no candles"
```

The counter

The counter sits at the right end of the stepper's position row, just above the field, in both builds. It is empty while no scan runs.

The operator placed it there.

> *Can see the scan counter in the upper left corner of the lower section. Would rather have this counter just above and at the right corner of the same field. Then have the field display the results of each scanned market.*

`src/trading/ata_spm.py` — what the counter says during one class's walk

```
SCAN_PROGRESS_FORMAT = "Scanning {asset_class} · {read} of {total} · {hits} hit(s)"
SCAN_LISTING_FORMAT = "Scanning {asset_class} · reading the market list"
```

What a scan reports

An empty table can mean three different things. The note under it says which: no scan has run yet, a scan is running, or a scan finished and found nothing.

`src/gui/main_tabs/market_inspector_surface.py` — the sentence an empty table carries

```
def empty_table_text(scan_state: Any, noun: Any) -> str:
    """The sentence an empty table carries for one scan state.

    ``SCAN_NOT_ASKED``, ``SCAN_RUNNING`` and ``SCAN_FINISHED`` each get
    their own wording, so the three never read alike.
    """
```

`scan_state` reports the same three states to both renderers, so the React side draws the note the way the window does.

A scan that stops part way reports no scan at all. `MarketInspector.scan_universe` clears the pairs and the scan second before it runs the pair test, and sets the second only once that test answers, so a scan the analyzer stopped leaves no timestamp and no pairs from the scan before it. `inspector_scan_age` reads a cleared second as `NO_SCAN_AGE`, which no freshness window accepts, so the sector charts refetch rather than draw from a scan nobody finished.

`src/gui/main_tabs/market_inspector_surface.py` — the age a cleared second reads

```
def inspector_scan_age(inspector: Any) -> float:
    """The seconds since the Market Inspector's last universe scan."""
```

A scan ends with one note on the Activity Log. The note names the markets no venue lists, the timeframes no venue serves, the markets that answered no candle, and each market no built bot variant trades with that market's own reason.

`src/trading/ata_spm.py` — three of the four sentences a note can carry

```
UNLISTED_TEXT = "No configured venue lists {symbols}."
UNSERVED_TEXT = "No venue serves {labels}."
UNTRADEABLE_TEXT = "Read and not traded, no bot variant trades {symbols}: {reason}"
```

`src/gui/main_tabs/market_inspector_surface.py` — the fourth

```
NO_CANDLE_TEXT = "No candles came back for {symbols}."
```

The note groups markets by reason and writes one sentence per group. A symbol is named as unread only when every ticked timeframe came back empty for it. No market leaves the scan: a named market keeps its place in the asset list, the charts and the reports.

`src/trading/ata_spm.py` — the separator between two reasons

```
UNTRADEABLE_SEPARATOR = ". "
```

Each timeframe keeps that market's last close beside the candle count it already keeps, and the note reads those closes rather than asking a venue.

`src/trading/ata_spm.py` — the closes the note reads

```
def scanned_closes(timeframes: Any) -> dict:
    """The last close each market answered, the first timeframe that read one."""
```

A timeframe holding too few candles is reported, never voted. The floor is thirty candles, the number the gate scan already requires.

`src/trading/ata_spm.py` — the floor, and the line it is reported on

```
MIN_CANDLES_TO_VOTE = ata_gate_scan.MIN_CANDLES_FOR_TA

TIMEFRAME_VOTE_FORMAT = (
    "{votes} vote(s), {unread} without candles, {short} under {floor} candles"
)
```

The floor earns its place. On twenty-five hourly candles seven of the twelve voters abstain for want of history, and the remaining five still produce a direction.

The scan also writes to the log, one line per phase. Every line is written at info on the `acervator` logger, so the system log and the Console pane carry it.

`src/trading/ata_spm.py` — the lines one press writes

```
SCAN_PRESSED_TEXT = (
    "ATA-SPM scan pressed: {asset_class} on {timeframes}, "
    "ticker '{ticker}', target {hits} hit(s)"
)
MARKET_READ_TEXT = "ATA-SPM read {symbol} on {label} from {venue}: {candles} candle(s)"
MARKET_EMPTY_TEXT = "ATA-SPM read {symbol} on {label} from {venue}: no candles"
MARKET_REFUSED_TEXT = "ATA-SPM read {symbol} on {label} from {venue}: refused, {reason}"
HIT_TEXT = "ATA-SPM hit: {call}"
SCAN_FINISHED_TEXT = "ATA-SPM scan finished: {headline} · {meta}. {method}"
SCAN_NOTE_TEXT = "ATA-SPM scan finished: {note}"
SCAN_EMPTY_TEXT = "ATA-SPM scan finished: no sector to scan"
SCAN_FAILED_TEXT = "ATA-SPM scan failed: {error}"
```

Each line crosses to the Activity Log on one signal carrying the message and its level. A read that answered candles draws at info. A read that answered none, the busy line, and a finished line with no vote draw at warning. A scan that raised draws at error.

`src/gui/market_inspector.py` — the signal that carries a phase line

```
    scanLogged = Signal(str, str)
```

The candle source names where it read, what it read, and why it read nothing.

`src/trading/ata_asset_maps.py` — the four refusals a read can name

```
UNMAPPED_TEXT = "no map lists {symbol}"
NO_VENUE_TEXT = "no configured venue lists {symbol}"
UNSERVED_TEXT = "{venue} does not serve {timeframe}"
NO_ADAPTER_TEXT = "no fetcher serves {venue}"
```

Each module names its own, so the scan note's sentence and one read's refusal are two constants and not one.

`src/gui/main_tabs/market_inspector_surface.py` — the source one read names

```
CANDLES_FROM_SCAN = "universe scan"

def sector_candle_read(symbol: Any, timeframe: Any, sources: Any) -> tuple:
    """Where one read went, what it read, and why it read nothing."""
```

A mapped name names its own venue. A crypto name served from the held scan names the scan, and one read through a connector names the exchange.

Opposing Pairs

The zone draws the pairs kept on the last scan. A market reaches long or short only when two or more of its timeframes sit at the same Bollinger extreme. A thirty-day return correlation in the configured negative window raises a candidate, and a test for a long-run equilibrium then decides it.

`src/trading/market_inspector.py` — `MarketInspector._find_opposing_pairs`

```
def _find_opposing_pairs(self, signals: list, closes_by_symbol: dict) -> list:
    """Enumerate long × short candidates; keep the cointegrated ones.

    The 30-day return correlation raises a candidate and no longer decides
    it: ``cointegration_test`` runs on the full close series and a pair it
    refuses never reaches the table.
    """
    longs = [s for s in signals if s.direction == "long" and s.score >= 0.3]
    shorts = [s for s in signals if s.direction == "short" and s.score >= 0.3]
```

Two markets can move opposite each other for a year with no relationship holding between them. The test is cointegration in both of its standard forms, taken from statsmodels, and a pair passes only when both reject the null of no cointegration: Engle-Granger at a significance of 0.05, and Johansen above its ninety-five per cent critical value for rank zero.

`src/trading/pair_selection.py` — what a pair has to clear

```
WINDOW_BARS = 365
MIN_OBSERVATIONS = 120
SIGNIFICANCE = 0.05
```

`src/trading/pair_selection.py` — the two published tests

```
def engle_granger_p_value(left: Any, right: Any) -> float:
    """The two-step p-value, from ``statsmodels.tsa.stattools.coint``."""

def johansen_trace(left: Any, right: Any) -> tuple:
    """The rank-zero trace statistic and its 95% critical value."""
```

A pair with fewer than 120 daily closes is not tested and not shown. A flat series is refused before the correlation package is imported, under a method and a reason of its own rather than as a test result.

`src/trading/pair_selection.py` — the refusal a flat series takes

```
METHOD_CORRELATION = "correlation"
FLAT_SERIES_DETAIL = "series holds one repeated price"

def is_flat(series: Any) -> bool:
```

The correlation test imports no package at all. `pearson_correlation` computes the coefficient, and from the Student t distribution its two-sided p-value, over numpy alone. The reading was swept against scipy 1.18.1 over 2,090 samples — sizes from three to four thousand, coefficients from minus one to plus one, and p-values inside a decade of the 0.05 the panes gate on — and both readings answer the same verdict in every one of them.

`src/trading/pair_selection.py` — the coefficient and its p-value

```
def pearson_correlation(left: Any, right: Any) -> tuple[float, float]:
    """Pearson's r over two samples, and its two-sided t-test p-value."""
```

`MarketInspector._pearson` reads that one function, so the thirty-day correlation that raises a candidate and the coefficient a card prints can never be two different formulae.

Both packages are imported inside the three functions that call them, and nowhere else, so the application starts before either is loaded. Both are declared, and the bundle carries both, so the test runs in the built application and in a source run alike. A missing package is re-raised as itself rather than recorded as a refused pair.

One package is named now, and the built application carries it. `scipy` is named by no module of the tree: the import set holds 20 outside packages where it held 21, and 17 of them are declared. `statsmodels` is the `cointegration` extra, and `CONSUMER_EXTRAS["build"]` in `tools/deps.py` installs that extra, so the shipped bundle carries the package and the Opposing Pairs table answers rows. The guard is what makes it an extra rather than a core requirement: an install from a consumer that does not name the extra runs without the cointegration test, and the scan's own status line names the package rather than the import error.

`src/gui/main_tabs/market_inspector_surface.py` — the line a stopped scan carries

```
ANALYZER_ERROR_FORMAT = "Analyzer error: {error}"
ANALYZER_MISSING_PACKAGE_FORMAT = (
    "Opposing Pairs needs {package}, which this build left out"
)
```

`analyzer_error_text` reads the second format only for an `ImportError` that names a package. Every other failure keeps the first.

`pyproject.toml` — the two packages the tests need

```
"scipy>=1.18.1",
"statsmodels>=0.15.0",
```

`pyproject.toml` — the package the cointegration test needs

```
[project.optional-dependencies]
cointegration = ["statsmodels>=0.15.0"]
```

`tools/spec_common.py` — the packages the bundle still leaves out

```
EXCLUDES: tuple[str, ...] = ("tkinter", "matplotlib", "PIL", ...)
```

A name in that list must not also sit in `dependencies`. The two statistics packages sit in `dependencies` and not in the list, which is what lets the Opposing Pairs table and the Bot Swarm Topologies zone draw rows.

Neither statistics package sits in that list, and the rule above still holds because neither sits in `dependencies` either. The list leaves them out so the bundle carries the cointegration test: measured with scipy refused, neither `statsmodels.tsa.stattools` nor `statsmodels.tsa.vector_ar.vecm` imports, so excluding scipy alone would strip that test as surely as excluding statsmodels. Every name the list does hold was read against the imports of the 498 modules under `src/` plus `main.py`. Seven of the nine are imported nowhere, and `matplotlib` and `PIL` are imported only where the built application never reaches. The Bot Swarm

Topologies zone draws rows either way, because three of its four detectors read the correlation test alone.

The zone's own line names the whole funnel: markets read, markets with a direction, pairs tested, and whether any held equilibrium.

`src/gui/main_tabs/market_inspector_surface.py` — the three counts and where each is read

```
def scan_counts(signals: Any, tested: Any) -> dict:
    """Markets read, markets with a direction, and pairs the test ran on."""
```

Where the analysis happens

`MarketInspector` in `src/trading/market_inspector.py` owns the maths. One method drives the whole pipeline and keeps the results on the analyzer.

`src/trading/market_inspector.py` — `MarketInspector.scan_universe`

```
def scan_universe(
    self,
    candles_by_symbol_by_tf: dict,
    active_symbols: set,
    closes_by_symbol: dict,
) -> None:
```

Two steps run under it.

Step	Produces
`_analyze_tf`	One `TimeframeAnalysis` per timeframe
`_score_market`	One `MarketSignal` per market

One analyzer instance serves two readers. `get_shared_inspector` returns it, the tab owns the fetch cycle and writes into it, and the per-bot Market Inspector page in the Bot Details dialog reads back out of it through `build_per_bot_view`. One analyzer, two readers, no second copy of the score.

`src/trading/market_inspector.py` — what one finished scan leaves on the analyzer

```
    last_signals: list
    last_tested: list
    last_closes: dict
```

`last_tested` holds one entry per candidate the equilibrium test ran on, whether it passed or not, and the Opposing Trades funnel line is the one reader of it.

The per-bot page is drawn from one description of the screen. It returns the rows, the groups, the words, the colours and the layout numbers as values. The Qt side draws them as widgets and the React side draws them as elements.

`src/gui/main_tabs/market_inspector_tab_surface.py` — `per_bot_view`

```
def per_bot_view(bot: Any) -> dict:
    """The per-bot Market Inspector screen as values, read off the shared
    analyzer's most recent scan.
```

The failure line draws as characters. Where the per-bot view cannot be built, the tab draws one amber line naming the failure. That line carries the caught error, which the application did not write, so the label is fixed to plain text and `FALLBACK_TEXT_FORMAT` carries no tag. An error holding <b> or an image address shows those characters, and the tab builds nothing from them. The Paper and Simulator forks draw the same line the same way, in `src/gui/paper/paper_market_inspector_tab.py` and `src/gui/simulator/sim_market_inspector_tab.py`.

`src/gui/live_settings/market_inspector_tab.py` — `MarketInspectorTabMixin._create_market_inspector_tab`

```
msg.setTextFormat(Qt.PlainText)
```

Fetching

`set_exchange_source` binds a connectors getter and the application scheduler, so the tab always sees the current connector dict rather than a snapshot taken at build time.

The fetch serves its last network result while that result stays young enough, and the Refresh button passes a flag that goes to the network regardless.

`src/exchange/market_inspector_fetcher.py` — `fetch_htf_universe`

```
async def fetch_htf_universe(
    exchange_connectors: dict,
    active_symbols: Optional[set] = None,
    top_n: int = DEFAULT_TOP_N,
    progress_cb=None,
    force_network: bool = False,
    min_refresh_s: float = DEFAULT_MIN_REFRESH_S,
) -> FetchResult:
```

Three helpers do the work inside it.

Helper	What it does
`_pick_universe`	Ranks each connector's bulk tickers by 24-hour quote volume
`_fetch_one_symbol`	Pulls per-symbol OHLCV on the connector's single-worker executor
`weekly_from_daily`	Derives the weekly series when the venue lists no weekly timeframe

The weekly series is one definition. It calls the stone tablets' own rollup with the week bucket moved onto Monday 00:00 UTC, the day the chart venue stamps its weekly bar with.

`src/exchange/market_inspector_fetcher.py` — the weekly line

```
def weekly_from_daily(daily: list[_Candle]) -> list[_Candle]:
    """One weekly _Candle per calendar week of daily, through weekly_rows_from_daily."""
```

The scan asks a venue only for what its own table lists.

`src/exchange/market_inspector_fetcher.py` — the timeframes one connector serves

```
DAILY_TIMEFRAME = "1d"
WEEKLY_TIMEFRAME = "1w"

def exchange_timeframes(connector: Any) -> tuple:
    """The timeframes one connector's own table lists, with 1w where 1d is."""
```

`src/trading/market_inspector.py` — the three the Refresh scan names

```
HTF_TIMEFRAMES = ("1d", "1w", "1M")
```

Every crypto row carries that table, and a listing answers whether it serves one timeframe. Where no table is found, the public route's own granularity set answers instead. A market must sit at the same Bollinger extreme on two of those timeframes to carry a direction, so the Refresh scan's two series decide it.

No code reads that tuple. `src/trading/market_inspector.py`, in `MarketInspector.scan_universe`, walks whatever timeframe keys the fetch put in the map, and `src/exchange/market_inspector_fetcher.py`, in `_fetch_one_symbol`, puts `1d` in and `1w` beside it. The tuple's `1M` entry names no rung the scan builds.

`src/trading/stone_tablets/ra_fetcher.py` — the granularities the public route serves

```
class CoinbasePublicCandles(ExchangeAdapter):
    GRANULARITY_S = {"1d": 86_400, "1h": 3_600, "5m": 300}
```

`src/exchange/market_inspector_fetcher.py` — the two product lists and the volume read

```
def public_products() -> tuple:
def trading_products(exchange_connectors: Any) -> tuple:
def public_candles(symbol: Any, timeframe: Any, bars: Any) -> list:
async def fetch_quote_volumes(
    exchange_connectors: dict, min_refresh_s: float = DEFAULT_MIN_REFRESH_S
) -> dict[str, float]:
```

`src/gui/main_tabs/market_inspector_surface.py` — the two readers the scan asks after the maps

```
def connector_candles(symbol: Any, timeframe: Any, connectors: Any) -> list:
def inspector_candles(symbol: Any, timeframe: Any) -> list:
def inspector_scan_age() -> float:
```

A product reaches a walk only while the venue says its status is online and its trading is not disabled. A name the map holds and the venue does not trade goes on the order's dead list.

`src/exchange/market_inspector_fetcher.py` — the week the rollup uses

```
DAY_MS = 86_400_000
MONDAY_OFFSET_MS = 4 * DAY_MS
DAYS_PER_WEEK = 7
WEEK_MS = DAYS_PER_WEEK * DAY_MS
```

A request to the public crypto route is held to 300 candles, and the bars a read asks for are held to that ceiling before the start is computed, so the window ends at the press.

`src/exchange/market_inspector_fetcher.py` — the window one read asks for

```
        count = min(int(bars), RA_CHUNK_DAYS)
```

The market-data cache is shared. A crypto name with a Refresh under fifteen minutes old uses the candles Refresh read; past that the scan reads again.

`src/gui/main_tabs/market_inspector_surface.py` — the window a held scan serves

```
CANDLES_FRESH_SECONDS = DEFAULT_MIN_REFRESH_S
```

The scan asks three questions in order and stops at the first that answers: a listing in the asset maps, a fresh inspector scan, then the connectors. A crypto scan with no exchange connected reads the products list and the candles through the application's own public route.

`src/gui/main_tabs/market_inspector_surface.py` — the three sources, in order

```
def sector_candles(symbol: Any, timeframe: Any, sources: Any) -> tuple:
    """The candles one market answers, from the first source that has them."""
```

`src/trading/ata_asset_maps.py` — the listing one name resolves to, and its candles

```
def listing_of(symbol: Any) -> Optional[AssetListing]:
def venue_candles(listing: Any, timeframe: Any) -> list:
def venue_candle_read(listing: Any, timeframe: Any) -> tuple:
```

`src/trading/ata_spm.py` — the markets one class and sector hold

```
def markets_of(asset_class: Any, sector: Any = "") -> tuple:
```

Both hosts hand the model its asset source and its candle source where they are built, so neither reads `None` for the life of the screen.

`src/gui/main_tabs/market_inspector_surface.py` and `src/gui/market_inspector.py` — the wiring

```
self.set_ata_sources(sector_assets, self.scanned_candles)
self.set_ata_sources(sector_assets, self._scanned_candles)
```

Three readers serve the sources: one answers the assets a named sector holds from the shipped sector map, one answers the four timeframes a class scans, and one names each market the note reports as read and not traded.

`src/gui/main_tabs/market_inspector_surface.py` — the assets one sector holds

```
def sector_assets(sector: Any, asset_class: Any) -> list:
```

`src/trading/ata_spm.py` — the timeframes a class scans, and the markets it did not trade

```
def timeframes_for(asset_class: Any) -> tuple:
def untradeable_markets(scan: Any) -> list:
```

The sector map itself is loaded by `load_sector_map` in `src/trading/topology_proposals.py`, which is the one copy the topology detectors read as well.

The venues and the timeframes each serves

The chart venue answers four intervals and spells two of them differently from the way the engine names them.

`src/trading/stone_tablets/ra_fetcher.py` — the engine's word, and the endpoint's

```
YAHOO_INTERVALS: dict[str, str] = {
    "1h": "1h",
    RA_TIMEFRAME: "1d",
    "1w": "1wk",
    "1M": "1mo",
}
```

It refuses an hourly window longer than 729 days: 729 answers and 730 returns an error. The adapter records that reach and moves the start of a longer request forward, so an over-long ask returns the candles that exist instead of nothing. The other three intervals answered a twenty-five year request in full, so no limit is recorded for them.

`src/trading/stone_tablets/ra_fetcher.py` — the days each interval reaches

```
YAHOO_REACH_DAYS: dict[str, int] = {
    "1h": 729,
    RA_TIMEFRAME: UNCAPPED_REACH_DAYS,
    "1w": UNCAPPED_REACH_DAYS,
    "1M": UNCAPPED_REACH_DAYS,
}
```

The futures route serves an hourly and a daily granularity at 350 candles a request, and its weekly line rolls up from its daily one.

`src/trading/stone_tablets/ra_fetcher.py` — the futures candle route

```
class CoinbaseFuturesCandles(ExchangeAdapter):
    exchange_id = "coinbase-futures"
    chunk_limit = FUTURES_CANDLE_CAP
    BASE_URL = "https://api.coinbase.com/api/v3/brokerage/market/products"
```

Every venue candle is stamped in seconds, the unit the connector's candles, the chart's time axis and the confirmation timer already read. The chart venue's futures list is read with its own timeout, and the crypto sector keeps its daily and weekly series on the exchange row.

`src/trading/ata_asset_maps.py` — the futures read, and the unit every row is stamped in

```
FUTURES_TIMEOUT_S = 20.0
MS_PER_S = 1000
VENUE_EXCHANGE: (RA_TIMEFRAME, "1w"),

def futures_listings(timeout_s: float = FUTURES_TIMEOUT_S) -> tuple:
    """The venue's futures rows, their figures, and a refusal where it answered none."""
```

The asset maps

One row describes one asset a sector holds, with the venue and ticker carrying it.

`src/trading/ata_asset_maps.py` — one asset, and where it is carried

```
@dataclass(frozen=True)
class AssetListing:
    """One asset a sector holds, with the venue and ticker carrying it.

    ``quote`` is the currency the venue prices ``ticker`` in, which
    ``YahooChartAdapter.fetch_chunk`` checks its answer against.
    """

    symbol: str
    quote: str = USD
    venue: str = NO_VENUE
    ticker: str = ""
```

Forex is the class built end to end, and its tiers are liquidity tiers. The major tier is the seven pairs that hold the dollar. The minor tier is every cross of two of those currencies with no dollar in it, twenty-one pairs. The exotic tier is defined and holds no pair, because no pair is named for it.

`src/trading/ata_asset_maps.py` — the two tiers that carry names

```
FOREX_MAJOR: tuple[AssetListing, ...] = tuple(
    _yahoo_fx(one)
    for one in (
        "EUR/USD",
        "USD/JPY",
        "GBP/USD",
        "USD/CHF",
        "AUD/USD",
        "NZD/USD",
        "USD/CAD",
    )
)

CROSS_ORDER: tuple[str, ...] = ("EUR", "GBP", "AUD", "NZD", "CAD", "CHF", "JPY")

FOREX_MINOR: tuple[AssetListing, ...] = tuple(
    _yahoo_fx(f"{base}/{quote}")
    for at, base in enumerate(CROSS_ORDER)
    for quote in CROSS_ORDER[at + 1 :]
)
```

Commodities carries funds and not futures. A futures chart longer than a few months joins two or more contracts together, and every join is a price step nobody traded. This platform sells against a dollar target, so an invented step moves every level behind it. A fund holds the metal, prices in dollars, and has no expiry and no roll.

`src/trading/ata_asset_maps.py` — the listed instrument for each metal

```
METALS_PHYSICAL: tuple[AssetListing, ...] = tuple(
    AssetListing(symbol=one, quote=USD, venue=VENUE_YAHOO, ticker=one)
    for one in ("GLD", "SLV", "PPLT", "PALL")
)
METALS_BASE: tuple[AssetListing, ...] = tuple(
    AssetListing(symbol=one, quote=USD, venue=VENUE_YAHOO, ticker=one)
    for one in ("CPER", "DBB")
)
```

The four commodity spot pairs stay in the map and stay unlisted. A metal spot quote is a dealer market with no public listing, so no free venue carries it, and all four spellings answer HTTP 404.

`src/trading/ata_asset_maps.py` — the map, its sector names and where each figure came from

```
SECTOR_MAJOR = "major"
SECTOR_MINOR = "minor"
SECTOR_EXOTIC = "exotic"
FOREX_EXOTIC: tuple[AssetListing, ...] = ()

METALS_SPOT: tuple[AssetListing, ...]
MAPS: dict[str, dict[str, tuple]]
MAP_SOURCES: dict[str, str]
```

MAPS is the one structure every ticker list, sector list and scan reads, and MAP_SOURCES names the published document each class's taxonomy came from.

Energy carries the two groups the published index names, petroleum and gas. Its four funds are USO for WTI crude, BNO for Brent crude, UGA for gasoline and UNG for natural gas. No fund holding heating oil or gasoil is listed, because both answered no rows.

src/trading/ata_asset_maps.py — the energy rows

```
ENERGY_PETROLEUM: tuple[AssetListing, ...] = tuple(
    AssetListing(symbol=one, quote=USD, venue=VENUE_YAHOO, ticker=one)
    for one in ("USO", "BNO", "UGA")
)
ENERGY_GAS: tuple[AssetListing, ...] = (
    AssetListing(symbol="UNG", quote=USD, venue=VENUE_YAHOO, ticker="UNG"),
)
```

The operator added the energy sector.

> *ATA-SMP - New Market Sector - Energy - Not sure how I forgot to add this...*
>
> *ATA-SMP - Energy sector also only has four entries in its list. Probably not complete...*

Every forex pair the map names sends volume 0 on every bar. Two of the twelve voters read volume, and the volume voter abstains on such a series rather than standing a number in. Spot currency trading has no single published volume anywhere, and a currency future's volume is not the pair's.

src/trading/indicators/zscore.py — which of the two values the reading holds

```
smoothed = _vwma(z_series, z_volumes, self.smoothing_period)
z_volume_weighted = smoothed is not None
z = z_raw if smoothed is None else smoothed
```

The daily open is kept in every formula that reads it. Three read it: the volume voter, Heikin Ashi and the landing strip. Each of the three is the published formula. A fund opens at an auction, so its open is a traded price beyond doubt, and a gap between an open and the previous close is what any daily bar does when the market stops trading between bars.

A venue row whose open or close sits outside its own high and low is not a candle. Each such row is refused on its own, the rest are kept, and nothing is clamped or invented.

src/trading/ata_asset_maps.py — the conversion, one row at a time

```
def _candles_of(ticker: str, rows: Any) -> list:
    """Every row ``candles_from_raw`` accepts, one row at a time.

    A venue row whose open or close sits outside its own high and low is not
    a candle, and it is counted into ``VENUE_ROWS_REFUSED_LOG``.
    """
```

The stocks list

At press time the stocks list is the chart venue's predefined most-active screener, unauthenticated, ranked by regular market volume, each quote carrying its sector.

`src/trading/ata_asset_maps.py` — the screener the list is read from

```
SCREENER_URL = "https://query1.finance.yahoo.com/v1/finance/screener/predefined/saved"
SCREENER_ID = "most_actives"
SCREENER_COUNT = 100
```

Where the screener refuses or answers no quote, the list is the portfolio equities the simulator holds: every non-crypto name the other maps do not carry, 47 names, in map order. The order line names which list was read.

`src/trading/ata_asset_maps.py` — the two lists, and the read behind the first

```
STOCKS_PORTFOLIO: tuple[AssetListing, ...] = tuple(
    AssetListing(symbol=one, quote=USD, venue=VENUE_YAHOO, ticker=one)
    for one in SYMBOLS
    if one not in CRYPTO_SYMBOLS and one not in _MAPPED_FUNDS
)

def screener_listings() -> tuple:
    """The screener's own quotes, through the request route the chart adapter uses."""
```

`src/gui/main_tabs/market_inspector_surface.py` — the two source lines

```
STOCKS_SCREENER_SOURCE_FORMAT = "{source}, {sectors} sector(s)"
STOCKS_PORTFOLIO_SOURCE_FORMAT = (
    "RA portfolio equities in map order, the screener refused: {refusal}"
)
```

The operator set the rule for a source that is not free.

> *If we are having difficulty sourcing the data for free we will need to implement an API driven solution or get clever...*

The futures list

Derivatives is not a class. Each futures product takes the class of its underlying, and the product list is read at press time from the venue's own public route, ranked by its own 24-hour volume times price.

The second sentence above is overtaken in part, and it is kept as written. A product on a crypto root now takes Futures / Perps. A press reads this list for the classes the placement can answer, and FUTURES_CLASSES derives that set from the placement's own tables.

```
# src/gui/main_tabs/market_inspector_surface.py, in FUTURES_CLASSES
FUTURES_CLASSES = tuple(
    sorted(
        {
            ata_spm.contract_class(one)
            for one in (
                ata_spm.CLASS_CRYPTO,
                *(row[0] for row in ata_asset_maps.NON_CRYPTO_ROOTS.values()),
            )
        }
    )
)
```

Driven over six product records written for the reading, with the home redirected and every network name refused, through `futures_listings` and `class_markets` on the tree before this change and on this one:

```
                                 scan before    scan now        recording
three contracts on a crypto root crypto         futures_perps   futures_perps
two contracts on a commodity root commodities   commodities     commodities
one contract on an index root    stocks         indices         indices
scan and recording agree         2 of 6         6 of 6
```

`src/trading/ata_asset_maps.py` — where the futures list comes from

```
FUTURES_PRODUCTS_URL = "https://api.coinbase.com/api/v3/brokerage/market/products"
FUTURES_PRODUCT_TYPE = "FUTURE"
FUTURES_SOURCE_TEXT = "coinbase futures and perpetuals, 24 h volume x price"
```

The operator reported the empty slots that rule closed.

> *Inspector -> ATA-SMP -> Market Sectors -> Several slots for derivatives are empty or were showing no candles returned.*

Right half: topology proposals

`MarketInspectorTopologies` in `src/gui/market_inspector_topologies.py` renders one card per proposal and opens a preview dialog on Preview.

The engine behind the cards takes a plain dictionary, which keeps it free of any exchange or bot-manager coupling. It unions the detectors, drops overlapping proposals by asset and caps the result.

`src/trading/topology_proposals.py` — `detect_all_topologies`

```
def detect_all_topologies(
    context: dict[str, Any],
    cap: int = PROPOSAL_CAP,
) -> list[dict[str, Any]]:
    """Union every detector's proposals, dedupe them and cap at ``cap``.
```

Four detectors feed it.

Detector	Shape	Members it needs
`detect_momentum_funnel`	Correlated cluster, leader into laggers	3
`detect_mean_reversion_pair`	Anti-correlated pair, wired both ways	2
`detect_sector_cluster`	Same-sector star, hub into spokes	4
`detect_distance_to_band`	One asset, scrum-deep into fold-deep	2

Three of the four read the closes a finished Inspector scan writes. The fourth reads the bot roster alone and pairs a scrum-deep bot with a fold-deep bot on the same asset.

Each detector runs in an arm of its own. `detect_mean_reversion_pair` is the only one that reads the cointegration test, so in a build without `statsmodels` it answers nothing and the other three still answer. Every proposal returned carries one sentence naming the contribution that did not run.

`src/trading/topology_proposals.py` — the sentence a proposal carries

```
UNAVAILABLE_KEY: str = "unavailable_tests"
UNAVAILABLE_FORMAT: str = "{detector} needs {package}, which this build left out"
```

The pane prints that sentence after its count, so a reading the application cannot take is named beside Refresh and not in the log alone. `unavailable_tests` reads the proposals on display and keeps the first of each repeated sentence, since one detector run stamps them all.

`src/gui/main_tabs/market_inspector_topologies_surface.py` — the count and what follows it

```
STATUS_COUNT_FORMAT = "{proposals} proposal(s); {dismissed} dismissed"
STATUS_UNAVAILABLE_FORMAT = "{count}  ·  {sentences}"
```

With `statsmodels` absent the line reads "2 proposal(s); 0 dismissed · Mean-reversion pair needs statsmodels, which this build left out". With every package present it reads the count alone.

Two sentences above describe a stamp the pane no longer depends on.

> *Every proposal returned carries one sentence naming the contribution that did not run.*
>
> `unavailable_tests` *reads the proposals on display and keeps the first of each repeated sentence, since one detector run stamps them all.*

A refresh that returns no proposal has nothing to carry the stamp. The pane now reads the import path whenever the stamp is absent. The count therefore carries the sentence with no proposal on display as well as with several.

```
def unavailable_sentences() -> list[str]:
    """Each unrun-contribution sentence read off the import path, importing nothing."""
```

`src/trading/topology_proposals.py, in unavailable_sentences` — the sentence with no proposal to ride on

Measured offscreen on both hosts, with the package refused at the import finder. With one proposal on display the line read "1 proposal(s); 0 dismissed · Mean-reversion pair needs statsmodels, which this build left out" before and after. With no proposal on display it read "0 proposal(s); 0 dismissed" before, and after it carries that

same sentence. The Qt pane and the React model report one string: both call the one helper.

Two helpers read from disk under `src/trading/`. `suggested_target_usd` sizes each proposed bot from the asset target defaults, and the sector map names each asset's sector.

The pane reads the detector again the moment a scan finishes, on its own Refresh press, and on its ten-minute timer. Both panes answer that read under one name, so the Qt widget and the React host take the same call. A pane whose read raises is asked once, the tab logs a warning, and the scan still finishes.

`src/gui/market_inspector.py` — the one refresh both hosts call

```
def _refresh_proposals(self) -> None:
    """Read the detectors again and redraw the pane, at most once per trigger."""
```

An empty pane says which of three states it is in.

`src/gui/main_tabs/market_inspector_topologies_surface.py` — the three sentences

```
def empty_proposals_text(scan_state: Any) -> str:
    """The sentence an empty proposals pane carries for one scan state."""
```

Before any scan it reads "No proposals yet. The detector reads the closes the scanner writes, so press Refresh on the left half first." While a scan runs it reads "A scan is running. Proposals are built from what it finds." Afterwards it reads "No proposals right now. Try Refresh, or wait for market state to shift."

The pane's button reads Refresh, because the zone title already says what is being refreshed.

`src/gui/main_tabs/market_inspector_topologies_surface.py` — the label and its width

```
REFRESH_TEXT = "Refresh"
REFRESH_TOOLTIP = "Rerun topology detectors on current market state."
REFRESH_WIDTH_PX = 96
HEADER_WORD_WRAP = True
```

`current_topology_proposals` answers two different things. It answers `None` when the right pane never built or refused the read, and a list when the pane answered, so an empty list means the pane holds no proposals. The reader in the Simulator tab passes both answers on rather than flattening them.

`src/gui/main_tabs/market_inspector_surface.py` — the two answers a reader gets

```
    def current_topology_proposals(self) -> Optional[list]:
        """The proposals on display, for a simulator to read and wire.

        ``None`` says the right pane never built or refused the read,
        and a list says the pane answered, so an empty pane and an absent
        one never look alike to a caller.
        """
```

Dismissal

Dismissing a card hides it for a day, and the dismissal survives a restart. The pane writes each held id against the second it lapses at into the operator's settings, under `topology_dismissed_proposals`.

`src/gui/market_inspector_topologies.py` — `_persist_dismissed`

```
def _persist_dismissed(self) -> None:
    """Write ``_dismissed`` to the store, recording a refusal."""
    if self._dismiss_store is None:
        return
    try:
        self._dismiss_store.set(DISMISS_SETTINGS_KEY, dict(self._dismissed))
```

A write the store refuses leaves the card hidden for the session and puts the cause on the line beside Refresh, which reads `Dismissal NOT saved:` in place of the count until the next write lands. The pane stays up either way.

`src/core/settings.py` — `AppSettings.topology_dismissed_proposals` is the field the write lands in. `set_dismiss_store` hands the pane the settings manager it persists through, and the pane never resolves settings itself. Entries already lapsed are dropped at load and written back out, so a day's suppression is not extended across a restart and the stored set does not grow without bound.

Adopt

The pane emits an adopt request and the window handles it. The handler counts the new bots and their combined budget, asks which of the proposed wires already exist and would change, and shows all of it before anything is created.

`src/gui/main_window.py` — `_adopt_topology_proposal`

```
def _adopt_topology_proposal(self, proposal: dict) -> None:
    """Confirm, open the wizard for each new bot, then emit `wire.created`."""
```

An adopt that aborts part way calls `_report_adopt_orphans`, which names the bot ids it created and left unwired. Adopt is the only control on this screen that creates anything.

While the saved fleet is sealed, Adopt shows none of that. The handler asks `src/gui/main_window.py, in _refuse_while_sealed` first, so the press puts one box over the window and one Console line saying to unlock first. No confirm box, no wizard, no wire and no snapshot file follow. The words and the measurement are on the Console page.

```
if self._refuse_while_sealed():
    return
```

Push to the Simulator, the Paper Trader and Live

Both candidate zones carry a Push to Sim button. An open Opposing Trades entry draws it under the pair's headline, and the topology preview draws it beside Adopt. One press sends the candidate's bots to the Simulator, which holds them under its Back Test mode and leaves that mode in force.

`src/gui/main_tabs/market_inspector_surface.py` — the one button both zones draw

```
def push_to_sim_actions() -> list:
    """Push to Sim, the one button an open candidate entry draws."""
    return [
        action_row(
            PUSH_TO_SIM_PART,
            PUSH_TO_SIM_LABEL,
            PUSH_TO_SIM_TOOLTIP,
            PUSH_TO_SIM_WIDTH_PX,
        )
    ]
```

A topology names several bots and the wires between them; an opposing pair names two assets and no wire. Both reduce to the same payload — rows of one market and one dollar target, plus the wires — and the Simulator turns each row into a held bot on the venue its Stone Tablet was recorded on.

`src/gui/main_tabs/market_inspector_topologies_surface.py` — a proposal's payload

```
def proposal_push_candidates(proposal: Any) -> dict:
    """One proposal as ``{"bots", "wires"}``, the payload a push takes.

    Each bot is a ``{"symbol", "target_usd"}`` row and each wire is a
    ``{"source_asset", "target_asset", "pct"}`` row, so the routing the
    proposal describes travels with the bots it names.
    """
```

An asset no Stone Tablet names has no tape for Back Test to walk, so the push is refused whole. The Activity Log names the asset, nothing is held, and the run mode does not move. The same refusal is written when no Simulator tab is built.

`src/gui/simulator/sim_trading_tab_surface.py` — the refusal the operator reads

```
PUSH_NO_TABLET_FORMAT = (
    "Push to Sim refused: no Stone Tablet names {assets}, so Back Test has no "
    "tape to walk. The run mode is unchanged."
)
```

A wire is refused the same way. Its two ends name markets, and the push creates one simulated bot per market, so a wire whose source or target names a market the push did not create has no bot to reach. A wire naming one market at both ends has only one bot to reach, which is what `detect_distance_to_band` builds: it pairs a scrum-deep live bot with a fold-deep live bot on the same market, and those two collapse to one simulated bot. Either gap refuses the whole push, names the wire and says why.

`src/gui/simulator/sim_trading_tab_surface.py` — the wire refusals

```
PUSH_WIRE_UNHELD_FORMAT = "{source} to {target}: nothing was pushed for {asset}"
PUSH_WIRE_SELF_FORMAT = (
    "{source} to {target}: both ends are the one bot pushed for {source}"
)
```

A Target Balance is refused the same way, and on both destinations by the same rule. Each candidate row names one market and one dollar target, and both pushes read that target through one admission: an endless figure, a figure that is not a number, a stored flag, a negative figure and text no dollar can be read from are each refused, the Activity Log names every market refused, nothing is held and the run mode does not move. A row naming no target, or naming zero, takes the configured default instead, which is the `target_balance` field default `BotConfig` itself declares, so a push and the Bot Wizard cannot name two different dollars.

`src/trading/container/config.py, in pushed_target_usd` — the one admission both destinations read

```
PUSHED_TARGET_DEFAULT: float = float(
    next(one.default for one in fields(BotConfig) if one.name == "target_balance")
)
```

A repeated name is refused too. One pair holds one rate, so a topology naming the same source and target twice refuses the whole push, and a candidate naming the same market twice holds one bot for it rather than two.

`src/gui/simulator/sim_trading_tab_surface.py` — the target and repeated-pair refusals

```
PUSH_TARGET_FORMAT = "{asset}: {target} is not a Target Balance a bot can trade"
PUSH_WIRE_TWICE_FORMAT = (
    "{source} to {target} is named twice, and one pair holds one rate"
)
```

A pushed bot is a simulated record, never a live bot and never an order. It reaches the Simulator's own fleet file and its own bus, and the live engine and the live logs see nothing of it. A pushed wire is the same: it is registered on the Simulator's own `SimWireManager` and never on the live `SmartWireManager`.

The Paper Trader has no mode a candidate could land under, so no push targets it. Live takes no push either. The two targets are named on the promotion pipeline page.

Push to Paper

That first sentence is overtaken, and it is kept above as it stands. The Paper Trader still has no mode a candidate could land under, and a push targets it anyway. Both candidate zones carry a Push to Paper button beside Push to Sim, and one press spawns one paper scrumming bot per market the candidate names.

`src/gui/main_tabs/market_inspector_surface.py` — the two buttons an open entry draws

```
def candidate_push_actions() -> list:
    """Both destination buttons an open candidate entry draws, Push to Sim then
    Push to Paper."""
    return [*push_to_sim_actions(), *push_to_paper_actions()]
```

The payload is the same one Push to Sim takes. Paper reads its venue live, so it needs no Stone Tablet: every pushed bot takes the venue the Paper Trader's own `PaperExchange` names. The wires travel with the bots, and a wire that cannot reach two different pushed bots refuses the whole push, exactly as it does on the Simulator.

`src/paper/fleet_source.py` — where a pushed candidate lands on Paper

```
def push_candidate(
    self,
    candidate: Any,
    exchange_id: str = "",
    ta_timeframe: str = "",
) -> dict:
```

No run mode is named anywhere on this press, and the Activity Log line names none. The Paper Trader page carries what the spawned bots read and what each refusal says. Live still takes no push.

ATA-SMP

The eight phases sit outside the screen. Phases one to three and phase eight run in `src/trading/ata_spm.py`, and phases four to seven run in `src/trading/ata_spm_push.py`. The screen reads what they produce and draws it; it computes none of it.

`src/trading/ata_spm.py` — the phases one press runs

```
def run(
    sectors: Any,
    asset_source: Optional[Callable] = None,
    candle_source: Optional[Callable] = None,
    engine: Optional[VotingEngine] = None,
    message_format: Optional[str] = None,
    clock: Optional[Callable] = None,
) -> AtaSpmRun:
    """Phases one, two, three and eight in order, as one ``AtaSpmRun``."""
```

Phase	Module	What it produces
1 Evaluate	`ata_spm.evaluate`	One scan per sector, per ticked timeframe
2 Identify	`ata_spm.identify`	The votes carrying a reversal, strongest first
3 Pull	`ata_spm.pull`	The chart, its bands, and one message per confirming voter
4 Format	`ata_spm_push.format_post`	One post per push target
5 Distribute	`ata_spm_push.distribute`	One delivery record per post
6 Ready to Send	`ata_spm_push.ReadyToSend`	The bucket the operator approves from
7 Follow-Up	`ata_spm_push.FollowUpWatch`	What the market did after a published call
8 Timeframes	`ata_spm.agreement_for`	Every timeframe the asset voted on, and their verdict

A run answers one readback line naming the sectors scanned, the calls found, the charts pulled and the markets the gates refused.

`src/trading/ata_spm.py` — the line a finished run leaves

```
PHASE_RUN_FORMAT = (
    "{phase}: {sectors} sector(s), {calls} call(s), {pulls} chart(s), "
    "{refused} refused by the gates"
)
```

Phase eight answers whether the other timeframes agree with the call. It walks every vote the same asset produced in the run, names the direction each one read, and lists the timeframes that read the opposite. A neutral vote is neither agreement nor contradiction.

`src/trading/ata_spm.py` — the three verdicts phase eight can write

```
AGREEMENT_AGREED_TEXT = "Every timeframe agrees."
AGREEMENT_SINGLE_TEXT = "Only one timeframe voted."
AGREEMENT_CONTRADICTED_FORMAT = "Contradicted on {labels}."
```

Phase eight also caps a scan. Each sector takes only as many timeframes as the rounds so far show will fit inside one candle of the shortest timeframe the class scans. The rest are recorded as deferred and named on the zone, so a shortened scan is visible rather than silent.

The gate chain over scanned prices

One judgement decides whether a market is charted and posted, and it is the live trade gate chain. If the chain says Acervator would take the trade, the chart is drawn and the post joins the approval bucket. If it does not, no chart is drawn and nothing reaches the bucket.

Both chains the bots build are evaluated, unchanged, against a context filled from one market's candles and its voting summary. Nothing on the live path changes: the bots build the only two contexts inside their own tick, and this is a third caller.

`src/trading/ata_gate_scan.py` — both live chains, over scanned prices

```
readings = read_chain(build_scrumming_scrum_chain(), context) + read_chain(
    build_scrumming_fold_chain(), context
)
```

Twenty-four gates run across the two chains, fourteen on the scrum side and ten on the fold side. Eighteen decide from price and settings alone. Six need a position the scan does not hold, and the scan never invents one. Four of the six stand down by name, and two publish a distance instead of a verdict.

`src/trading/ata_gate_scan.py` — the six, split by what a scan can honestly say

```
NOT_APPLICABLE_GATES = (
    "delta_positive",
    "interval",
    "tranches_queued",
    "smart_ceiling",
)
HYPOTHETICAL_GATES = ("hysteresis_scrum", "hysteresis_fold")
```

The four stood-down gates appear in the expansion by name, marked as not run, each with the reason. A post must never imply a gate latched when no gate ran.

`src/trading/ata_gate_scan.py` — the reason each stood-down gate publishes

```
NOT_APPLICABLE_REASONS = {
    "delta_positive": "a scan holds nothing, and no surplus exists to test",
    "interval": "the same surplus, and a scan holds none of it",
    "tranches_queued": "a scan has no fold queue",
    "smart_ceiling": "a scan holds nothing, and any ceiling test passes",
}
```

The two hysteresis gates run against a hypothetical entry at the scanned price and publish the price a reversal would have to reach, with no verdict attached. At the instant of the scan the pivot equals the price, so both sides would refuse; the number is the reading, and the refusal is not.

`src/trading/ata_gate_scan.py` — what a hypothetical row prints

```
HYPOTHETICAL_FORMAT = (
    "entry at ${price:.8f} would need ${required:.8f}, "
    "{distance:.2f}% away; no gate ran"
)
```

Opposing Trade Distance is one sum: the bot's scrumming interval percentage plus its trading fee percentage, clamped between 0 and 50. It is a formula over settings, so a scan computes it exactly as a bot does.

`src/trading/otd_math.py` — the whole formula

```
total = float(interval_pct) + float(fee_pct)
return max(OTD_MIN_PCT, min(OTD_MAX_PCT, total))
```

Current reading. The sum keeps a floor of two fees: where the interval sits under the fee, the distance is two fees. A scan holds no fills, so no venue fee raises the fee it reads from its settings.

`src/trading/ata_gate_scan.py` — `build_context`

```
held = settings if settings is not None else scan_settings(symbol)
distance_held = distance_settings(held)
```

Landing Strip is the second number. The band-proximity detector reads whether price has sat against a Bollinger band for a run of candles, and it answers a side. An upper strip is a sell reversal and a lower strip is a buy reversal. The side and the candle count are both published beside the distance.

`src/gui/main_tabs/market_inspector_surface.py` — the line carrying both numbers

```
GATE_DISTANCE_FORMAT = "opposing trade distance {pct:.2f}% · landing strip {strip}"
```

A second strip detector runs inside the gate scan and feeds the confidence floor rather than the direction. It lowers the floor the TA consensus must clear, the way the live tick lowers it, so the scan and a bot judge a

market at one bar. The one term a scan loses is the band-priority favour, which arms only on a signed delta, so a scan reads at a stricter floor than a bot holding a position. The floor is the chain's own; no second threshold is written on the posting path.

`src/trading/ata_gate_scan.py` — the scan's readings and the floor it reads

```
@dataclass(frozen=True)
class GateScan:
    """Every gate reading over one market, with the side that would fire."""

    confidence_floor: float = 0.0
```

Current reading, the line under the scan's floor. The scan's floor takes the line a bot's floor takes. The favours relax it no further than what the two heaviest voters of the scanned vote reach at full confidence, and never above 0.25. So a scan needs a third agreeing indicator wherever the plain floor needs one.

`src/trading/ata_gate_scan.py` — `confidence_floor`

```
return _skewed_confidence_floor(skew, getattr(summary, "signals", None) or [])
```

Current reading, the scan's floor and its strip. The scan's floor now keeps the line at every skew, so it can rise over 0.25. The scan's own copy of the strip rule is gone. It calls the function a bot calls, so a strip sets the scan's side only when three indicators agree. The scan's LS light reads cyan only while the TA light on its bank shows neither a refusal nor an override.

`src/trading/ata_gate_scan.py` — `build_context`

```
is_bullish, is_bearish = vote_sides(
    direction, confidence, floor, band, getattr(summary, "signals", None) or []
)
```

`src/trading/ta_engine.py` — the candle floor the engine sets

```
MIN_CANDLES_FOR_TA = 30
```

A chart with too few candles produces no gate scan at all. The floor is 30 candles, and the line says so rather than reporting an empty result as a clean one.

`src/gui/main_tabs/market_inspector_surface.py` — the two lines the expansion heads

```
NO_GATE_TEXT = "No gate ran. The chart carried too few candles."
GATE_COUNT_FORMAT = (
    "{ran} of {total} gate(s) ran · {latched} latched · {blocked} blocked "
    "· {stood_down} did not run"
)
```

A refused market is still on the record. Its gate scan is kept, so a reader sees that the market was judged, which gates blocked it and why.

`src/trading/ata_spm.py` — what is drawn and when

```
    image = (
        render_pull_image(vote, candles, max_supporting_indicators, messages)
        if gates.would_fire
        else ChartImage()
    )
```

What a scan shows of your circuit breakers

The scan reads both breaker thresholds and the soft cooldown off the settings it is handed, through the one declaration the trading engine reads, and closes its readings with a row named `circuit_breaker_hard`. The row carries no side, so no chain verdict reads it.

`src/trading/ata_gate_scan.py` — the reading that row carries

```
HARD_BREAKER_FORMAT = (
    "single-candle move {move:.2f}% against your hard {hard:.2f}% "
    "and soft {soft:.2f}% (cooldown {cooldown} candles, soft side {side})"
)
```

Driven on one bar spanning 12.00 % of its open, two scans in the same run:

```
stored soft / hard   state      the row's reading
5.0 / 10.0           blocked    move 12.00% against your hard 10.00% and
                                soft 5.00%, soft side scrum
25.0 / 35.0          latched    move 12.00% against your hard 35.00% and
                                soft 25.00%, soft side none
```

The two rows name different figures, so the reading follows the stored record rather than a figure held inside the scan. Before this the scan held its own copy of the move arithmetic and its own copy of the opening soft figure, named the hard field nowhere, and published no figure at all.

The verdict does not move. Over twenty scans across five tapes and four stored pairs, the firing side, the fire-or-refuse answer and every chain gate's state and wording are identical; the readings per scan go from 22 to 23 and the decided count from 16 to 17, which is the new row. A sweep of 63,168 candle and figure combinations reads the same soft-breaker sides before and after.

No store in the tree holds a breaker figure for a scan, so a scan started from this tab reads the declared 25.0 % and 35.0 % until a caller hands it a record carrying yours.

The voting panel a called asset carries

ATA-SMP carries its own Indicator Voting Panel. It is a clone of the live one, drawn inside an open entry, and it reads only the markets ATA-SMP scanned. No live bot's market reaches it, and it reaches no live bot.

`src/gui/main_tabs/market_inspector_surface.py` — the panel and what feeds it

```
def voting_panel(pull: Any) -> Optional[dict]:
    """The Indicator Voting Panel one scanned asset carries, as its rows.

    The rows are the timeframes ATA-SMP read for this asset alone, and
    every cell is drawn from ``indicator_panel_surface``.
    """
```

The settings page

The Settings button swaps the zone for a page of buttons that takes the whole left column, so nothing is squeezed into a strip and no scroll bar appears. That page is Level 1. A press on a venue button opens that venue's own sign-in page, which is Level 1A.

`src/gui/main_tabs/market_inspector_surface.py` — the two pages the zone shows

```
LEVEL_ONE = "level-1"
LEVEL_ONE_A = "level-1a"
```

Level 1 carries three groups and a Back button. SM Accounts holds one button per push target. Asset Category holds one button per asset class, and the pressed one is the class a scan uses. Settings holds the six values a phase reads.

`src/gui/main_tabs/market_inspector_surface.py` — the three group names

```
SM_ACCOUNTS_TITLE = "SM Accounts"
ASSET_CATEGORY_TITLE = "Asset Category"
ATA_SETTINGS_TITLE = "Settings"
```

Setting	Which phase reads it	Starts at
Max posts per hour	The send ceiling	unset, and an unset ceiling releases nothing
Max supporting indicators	Phase four, capping the evidence lines and the chart overlays	unset, and every confirming voter is drawn
Confirmation share %	Phase seven, sizing the target a call must reach	0, and a call reads open
Standardised message text	Phase three, wording each indicator message	the standard wording
Hits per scan	The walk's stop target	3
Confirmation read candles	Phase seven, spacing each read after the first	3

A held venue credential, the asset class a scan uses and a ticked timeframe are one state, painted in the theme's primary colour under its on-primary text in both builds.

`src/gui/main_tabs/market_inspector_surface.py` — the setting keys and the rows they draw

```
SETTING_CONFIRMATION_SHARE = "confirmation_share_pct"
SETTING_CONFIRMATION_CANDLES = "confirmation_candles"
SETTING_ROWS: tuple[tuple[str, str], ...] = (...)
COUNT_SETTINGS = (SETTING_HITS_PER_SCAN, SETTING_CONFIRMATION_CANDLES)

def setting_rows(settings: Any) -> list:
def set_setting(settings: Any, key: Any, typed: Any) -> None:
```

`SETTING_ROWS` names each row and `COUNT_SETTINGS` says which take a whole number, so both hosts draw the page and write a value with no host code of their own. A count read as text, or under one, takes its own default.

`src/trading/ata_spm_push.py` — the count read safely, and what persists

```
def read_candles(asked: Any) -> int:
    """``asked`` as a whole number, or ``DEFAULT_CONFIRMATION_CANDLES`` below one."""

PERSISTED_SETTINGS = ("hits_per_scan", "confirmation_share_pct", "confirmation_candles")
```

Three settings persist. Hits per scan, the confirmation share and the confirmation read candles are written on every change and read back when the page is built. The other three live for the life of the process.

`src/trading/ata_spm_push.py` — where the persisted settings sit

```
def settings_path() -> Path:
    """``Path.home() / STATE_DIR_NAME / ATA_SPM_SETTINGS_NAME``."""
```

A typed message format may carry four keys in braces. A format naming any other key, or one the formatter cannot parse, no longer stops the scan: the standard wording is written and one warning names the key.

`src/trading/ata_spm.py` — the keys a message format may carry

```
MESSAGE_FORMAT_KEYS = ("label", "reading", "direction", "confidence")
MESSAGE_FORMAT_REFUSED_LOG = (
    "ATA-SPM message format refused, %s; the standard wording is used. Keys: %s"
)
```

The push targets

Ten targets ship. Adding one is adding a row: no phase names a target, and every screen and folder writer reads the rows, so a new row needs no screen edit. Signal publishes no way for a program to post into a group and is not a row.

The operator asked for the three that joined last, and ruled on Signal.

> *ATA-SMP - Need to add post / message distribution for Discord, Telegram, WhatsApp, and Signal.*
>
> *ATA-SMP - For Signal, if its not being used for this sort of thing then we can skip it.*

Target	Body ceiling	Title ceiling	Counted in	Image
X	280	none	Weighted characters	1200 by 675

Target	Body ceiling	Title ceiling	Counted in	Image
Instagram	2200	none	Characters	1080 by 1350
LinkedIn	3000	none	Characters	1200 by 627
TikTok	4000	90	UTF-16 runes	1080 by 1920
Facebook	none published	none	Characters	1200 by 630
Threads	500	none	UTF-8 with emoji	1080 by 1350
Reddit	40000	300	Characters	1200 by 628
Discord	2000	none	Characters	1200 by 675
Telegram	1024	none	Characters	1280 by 720
WhatsApp	65536	none	Characters	1200 by 675

Every number came off the platform's own documentation and each one is cited on its own page. Where a platform publishes no number, the row records that rather than carrying a guess. TikTok's title is empty because the header alone is over 90.

The push target rules — one section per target, with the page each number came from

```
  Facebook   body ceiling: no number published
             the row carries NO_LIMIT_PUBLISHED, and phase four applies none
```

Two platforms named at the start are not rows, and both were left out for the same reason: a rendered chart has no route in. One publishes an idea through its website and states that it has no API for it. The other's upload endpoint accepts video only.

One row describes one target, and one tuple of rows is every screen's source.

`src/trading/ata_spm_push.py` — the row, and the names every screen reads

```
  @dataclass(frozen=True)
  class PushTarget:
      """One push target: its sections, its ceilings, its header and its image size."""

  PUSH_TARGETS: tuple[PushTarget, ...] = (...)
  TARGET_NAMES = tuple(one.name for one in PUSH_TARGETS)
  SECTION_CALL = "call"
  SECTION_INDICATORS = "indicators"
  COUNT_WEIGHTED = "weighted"
```

`src/trading/ata_spm_push.py` — the header one row takes, and the row that names its own

```
TARGET_X = "X"
TARGET_WHATSAPP = "WhatsApp"

PushTarget(
    TARGET_X,
    (SECTION_CALL, SECTION_INDICATORS),
    body_limit=280,
    count_unit=COUNT_WEIGHTED,
    header=X_HEADER,
    image_width_px=1200,
    image_height_px=675,
)

def target_header(target: Any) -> str:
    """The row's own header, ``FIXED_HEADER`` where the row names none."""
```

The post text

One routine writes every piece of post text. It puts the header on top, the evidence under it, and the organisation address at the bottom. No caller supplies the address and no caller can remove it. The body, the caption, the thread root and the title all carry it, on every target.

`src/trading/ata_spm_push.py` — the post, written once

```
def compose(lines: Any, header: Any = FIXED_HEADER) -> str:
    """``header`` over ``lines`` over ``ORGANIZATION_URL``.

    ``fit_to_target`` drops ``lines`` to reach a ceiling and reaches neither
    ``header`` nor ``ORGANIZATION_URL``.
    """
```

The header is a field on the venue's row, and every row takes the fixed header unless it names its own. Nine rows take it and the X row names a short header. A venue is changed by editing its row; nothing branches on a venue's name.

`src/trading/ata_spm_push.py` — the two headers and the address

```
FIXED_HEADER = (
    "This is not investment advice. It is a demonstration of Ekthelius's "
    "proprietary TA engine housed in the Acervator governance execution "
    "platform."
)
X_HEADER = "Not investment advice. Acervator TA engine demonstration."
ORGANIZATION_URL = "https://github.com/Acervator-LLC"
```

The fixed header measures 144 in every unit, because it is plain ASCII. On X, with the call line, the address at 23 weighted characters and two separators, the floor is 122 of 280, leaving 158 for evidence. The short header measures 57 weighted units and keeps three facts: that it is not investment advice, that it is Acervator, and that it is a TA engine demonstration. The words are the operator's to change; the three facts are not.

He asked for the shorter one.

Should shorten specifically for X and maintain a consistent level of valuable data...

Phase four holds each body under its target's ceiling. It drops evidence lines, longest first, and never the header or the address. A line drops whole, so no price or band value is cut mid-digit, and the post says how many sentences were left out.

`src/trading/ata_spm_push.py` — what the fitter drops, and what it will not

```
  def fit_to_target(ranked: Any, target: Any) -> tuple:
```

Where a venue holds a title, the title stands on the first line, then a blank line, then the body.

The chart a post carries

The picture is drawn by the Charts tab's own renderer. No second drawing engine was written for posts, because two engines drift apart and only one of them is the screen the operator watches. The renderer's drawing state and paint routine sit in a plain object. The Charts tab's chart is that object with a window around it, and a post image is that object with a picture file around it.

`src/gui/native_chart.py` — one paint routine, two places to send it

```
class ChartPainter:
    """The chart's drawing state and its paint routine, with no window around it."""

        def paint_to(self, p: QPainter, w: int, h: int) -> None:
            """Draw the whole chart onto ``p`` over a ``w`` by ``h`` area.

            The painter's device is the caller's: a widget from ``paintEvent``
            and a ``QImage`` from ``render_chart_png``.
            """
```

The operator asked for the two to be one renderer.

> *Refining our chart renderer (the one currently used under the Asset Charts Tab) will allow us to upgrade the visuals for one and align this to visuals of out going SM posts thus unifying direct usage with SM exposure.*
>
> *Refining our chart renderer ... will allow us to upgrade the visuals for one and align this to visuals of out going SM posts thus unifying direct usage with SM exposure.*
>
> *let's migrate to the Charts tab and do its visual upgrades. After its chart renderer is fully upgraded, it will need to be migrated to ATA-SMP.*
>
> *After its chart renderer is fully upgraded, it will need to be migrated to ATA-SMP.*

The split was forced by where a scan runs. Pressing Scan Now hands the work to a background worker so the screen keeps drawing, and the drawing toolkit will not build a window on a background worker. Driven, it does not raise an error that could be caught; it kills the program. A picture file has no such rule, so the post image is drawn straight onto the file and no window is made.

`src/gui/market_inspector.py` — the worker the scan runs on

```
            self._scan_thread = threading.Thread(
                target=self._compute_scan,
                args=(settings.message_format, settings.max_supporting_indicators),
                name=ATA_SCAN_THREAD_NAME,
                daemon=True,
            )
```

The indicators on the picture are the ones that voted for the call, not the ones the Charts tab happens to have switched on. Each overlay in the chart's registry names the voter it draws, so the picture is chosen by the vote. All twelve voters carry an overlay, and the header line names any voter the cap cut.

`src/gui/native_chart.py` — the overlays one vote draws

```
def overlays_for_voters(voters: Any, max_overlays: int = NO_OVERLAY_CAP) -> tuple:
    """The overlay keys drawing ``voters``, and the voters no overlay draws."""
```

`src/gui/native_chart.py` — the overlay names its voter

```
    ChartOverlay(
        key="bb",
        label="BB",
        colour_field="chart_band",
        pane=PRICE_PANE,
        occludes=False,
        draw="_draw_bollinger",
        tooltip="Bollinger Bands (20, 2σ) with cloud fill",
        voter="bollinger_bands",
    ),
```

Max supporting indicators decides how many are drawn. It was already the cap on how many confirming sentences a post carries, and it is the same cap on the chart. No second setting was added, and an unset value caps nothing.

`src/trading/ata_spm.py` — the unset cap, and the voters one call confirmed

```
NO_INDICATOR_CAP = 0

def confirming_signals(vote: Any) -> list:
def timeframe_label(timeframe: Any) -> str:
```

`src/gui/native_chart.py` — what reached the picture and what did not

```
@dataclass
class ChartImage:
    """One rendered chart on disk, and what the renderer could not draw.

    ``drawn`` and ``undrawn`` name voters: ``undrawn`` holds the ones no
    overlay draws and the ones ``max_overlays`` cut.
    """
```

The chart states the call. Three marks are drawn, and each one is earned. A badge in the top right names the direction. A dashed rule and a triangle mark the last bar, which is the bar the vote was made on. A strip under the time axis carries one row per confirming voter: a square in the colour of the line that drew that voter, and the sentence phase three wrote for it. A voter with no line still gets a row, in the dim colour. Nothing is drawn that a voter did not read.

`src/gui/native_chart.py` — the direction and the readings the picture takes

```
        def set_call(self, direction: str, readings=()) -> None:
            """Take one reversal direction and one reading line per voter.

            ``readings`` are ``(voter, text)`` pairs and ``_draw_call`` paints
            them under the time axis.
            """
```

A chart with no call set draws no badge, no bar mark and no strip, and keeps the height it had before. The Charts tab is unchanged by any of it.

Phase three answers one record per call, and that record carries the file.

`src/trading/ata_spm.py` — the picture the pull holds

```
@dataclass
class ChartPull:
    """The chart one reversal call was made on, and its confirming messages.

    ``image`` is that chart rendered to a PNG, which the post's caption
    captions.
    """
```

A venue image whose caller names no theme paints in the theme the window is in, and the message stamp at its foot takes that theme's own axis text colour. A venue image carries no resize grip; only a chart in a window draws one.

`src/gui/native_chart.py` — the theme one image paints in

```
painter.set_theme(tokens if tokens is not None else theme_in_force())
```

Each venue takes its image at the size that venue's own page publishes for a single image post. The two Meta feeds take the tallest shape one of them accepts, which gives the panes the most room.

`src/trading/ata_spm_push.py` — the two figures each row carries

```
    image_width_px: int = DEFAULT_IMAGE_WIDTH_PX
    image_height_px: int = DEFAULT_IMAGE_HEIGHT_PX
```

A height the natural layout fits draws the natural layout, and any height left over goes to the price pane. A height the layout does not fit compresses in one order: each sub-pane shrinks from its readable height toward its fold height; then the reading strip folds from one row per reading to one row of voter names with their swatches; then the price pane shrinks from its layout floor toward its paint floor. The strip folds because the caption

already carries every reading's sentence.

`src/gui/main_tabs/market_inspector_surface.py` — the grid's own column widths

```
PANEL_TF_WIDTH_PX = 44
PANEL_CELL_WIDTH_PX = 58
PANEL_NET_WIDTH_PX = 52
PANEL_CONF_WIDTH_PX = 96
```

`src/gui/main_tabs/native_chart_surface.py` — the four heights the fold moves between

```
TAG_HEIGHT_PX = 14
SUB_PANE_PLOT_BANDS = 7
SUB_PANE_LABEL_PX = TAG_HEIGHT_PX
SUB_PANE_READABLE_PX = SUB_PANE_LABEL_PX + SUB_PANE_PLOT_BANDS * TAG_HEIGHT_PX
SUB_PANE_FOLD_PX = 28
PRICE_PANE_LAYOUT_FLOOR_PX = 220
PRICE_PANE_PAINT_FLOOR_PX = 120
```

A sub-pane's readable height is 112 pixels, a label row over seven plot bands, and it folds to 28.

A height under the least layout is refused by name. The folder keeps its text file and the emitter row for that venue reads not ok.

`src/gui/native_chart.py` — the refusal a short image takes

```
NO_IMAGE_HEIGHT = 0

def strip_folded(width: int, height: int, readings: Any) -> bool:
def _least_height_for_panes(width_px: int) -> int:
def _minimum_height_for_panes(width_px: int) -> int:

IMAGE_TOO_SHORT_NOTE = (
    "{width}x{height} cannot hold the panes: {least} px is the least "
    "height at that width."
)
```

Each chart image carries its wording on it: the header, the call's own headline, then the address. The message takes 55 pixels at the foot of the image, 8 of padding above, 8 below and 13 for each of its three lines. A chart given no message is 55 pixels shorter and carries none.

`src/trading/ata_spm.py` — the wording one image carries

```
def post_caption(vote: AssetVote) -> str:
    """The header, the call's headline and the address, as the image's foot."""
```

Each bucket entry draws that picture scaled to the entry's width: a thumbnail closed and a larger view open, each at the height its own shape needs. An entry whose post names no file on disk draws a rectangle strip.

`src/gui/main_tabs/market_inspector_surface.py` — the two sizes one entry draws

```
THUMBNAIL_WIDTH_PX = 120
THUMBNAIL_HEIGHT_PX = 36
PREVIEW_WIDTH_PX = 320
PREVIEW_HEIGHT_PX = 160
```

`src/gui/main_tabs/market_inspector_surface.py` — the picture one entry draws

```
NO_IMAGE_SIZE = (0, 0)

def chart_image(post: Any) -> tuple:
    """That post's own PNG as a data address and its size, or ``NO_IMAGE_SIZE``."""
```

The Qt entry decodes that address into an image; the page places one image element with the same address in the same box. Both hosts read one `post_chart` payload.

The venue folders

The chart root holds one folder per venue, named for it. Each venue folder holds that venue's own version of every call: the chart with the venue's message at its foot, the same message as plain text beside it, and for X and WhatsApp a compose address as an Internet Shortcut.

`src/trading/ata_post_paths.py` — the root and one venue's folder

```
ATA_POST_ROOT: Path = Path.home() / ".acervator_ata_posts"

def venue_post_root(venue: Any, root: Any = None) -> Path:
    """One venue's own folder under the post root."""
```

The operator asked for the folders and for the message on each image.

> *Ready to Send Bucket - Want a folder button that opens the chart image directory. Want the standardized message marked on each image. Want this loading push candidates via a functioning market scanner...*
>
> *Should have a catered folder for each venue.*
>
> *Seems most of this stuff is price gated so will also need a folder where charts customed for each venue can be viewed and manually posted.*

The files are written inside phase three from the candles already held, in the same call that draws the root picture, so no venue is asked for a candle twice. A market the live gate chains refuse writes nothing anywhere.

`src/trading/ata_venue_folders.py` — the folder write

```
def write_venue_posts(vote: Any, candles: Any, posts: Any) -> dict:
def write_text(path: Path, body: str) -> None:
```

`src/gui/native_chart.py` — the wording the painter stamps at the foot

```
        def set_caption(self, text: str) -> None:
```

`FormattedPost.folder_path` names the venue folder each post's files sit in.

`src/trading/ata_post_paths.py` — the files one venue folder holds

```
POST_TEXT_SUFFIX = ".txt"

def get_ata_post_root() -> Path:
def post_image_path(symbol: Any, timeframe: Any, stamp: Any) -> Path:
def prune_post_images(path: Path) -> tuple:
```

`src/trading/ata_spm_push.py` — what one post names on disk

```
@dataclass(frozen=True)
class FormattedPost:
    """One venue's post: its body, its title, and the files its folder holds."""

def venue_files(pull: Any, target: Any) -> tuple:
```

`src/trading/ata_venue_folders.py` — the two compose addresses

```
X_INTENT_FORMAT = "https://x.com/intent/post?text={text}"
WHATSAPP_INTENT_FORMAT = "https://wa.me/?text={text}"
INTENT_SHORTCUT_FORMAT = "[InternetShortcut]\nURL={url}\n"
```

The folder keeps the newest picture of each market and removes the older ones. A market is one asset on one timeframe, so one asset on the hourly chart and the same asset on the daily chart are two markets and each keeps its own newest picture.

`src/trading/ata_post_paths.py` — how many stay, and why that number

```
#: ``ata_spm_push.format_run`` keys its pulls by symbol and timeframe, last
#: write winning, so one image per market is every image a post can name.
POST_IMAGES_KEPT_PER_MARKET = 1
```

Nothing in the application can name an older picture, so nothing loses one. Phase seven does not need one either: it reads the chart again from the market data, never from the folder, and the post it writes is text.

The trim happens on the same press that draws a picture, so the folder is never larger than one scan's worth of pictures plus the newest of every market scanned before. It reads one folder, the one its own file names, and goes no deeper. A file whose name a post picture could not have produced is not touched. A picture being written is never taken: the protection is the operating system's own refusal, and a refused file survives, is reported as refused, and goes on the next press.

`src/trading/ata_post_paths.py` — the line every trim writes

```
PRUNE_LOG = (
    "ATA post store: removed %d image(s), reclaimed %d byte(s), kept %d, refused %d"
)
```

The Chart Folder button opens the root, so every venue folder is in view, and the press writes one Activity Log line naming the folder it opened. The folder's address is handed to the operating system's own handler. No

browser is driven and nothing is typed into one.

`src/gui/main_tabs/market_inspector_surface.py` — the line and the hand-off

```
CHART_FOLDER_OPENED_LOG = "ATA chart folder opened: %s"

def chart_folder_line(root: Any) -> str:
def page_links(page: Any) -> tuple:
def link_segments(text: Any) -> list:
```

`src/trading/ata_spm_push.py` — the hand-off to the operating system

```
def open_path(path: Any) -> bool:
```

`page_links` is the whole list a press may name on the open page. Any other address opens nothing and the window records the refusal.

Sending

Every post waits in the approval bucket. The operator approves or declines each one, and no button sends a declined post.

`src/gui/main_tabs/market_inspector_surface.py` — the five buttons the bucket carries

```
APPROVE_LABEL = "Approve"
DECLINE_LABEL = "Decline"
POST_SELECTED_LABEL = "Post Selected"
POST_ALL_LABEL = "Post All"
FULL_AUTO_LABEL = "Send Bucket Full Auto"
```

The three send buttons share one width, the width that already held the longest wording, and wrap at the count the zone holds: one at a 700-pixel tab, two at 900, three at 1400 and above.

`src/gui/main_tabs/market_inspector_surface.py` — the shared button width

```
BUCKET_BUTTON_WIDTH_PX = 184
```

Each post takes one of three routes. The route is decided by the post's own venue and what is held for it.

`src/trading/ata_spm_push.py` — the three routes

```
ROUTE_API = "api"
ROUTE_INTENT = "intent"
ROUTE_FOLDER = "folder"
```

Route	Which posts take it	What a press does
api	Discord and Telegram, once held and the ceiling is set	one request carrying the text and the picture
intent	X and WhatsApp	hands the shortcut to the operating system's handler; the picture is attached by hand

Route	Which posts take it	What a press does
folder	every other venue	hands the venue folder to the operating system's handler

Discord and Telegram are the two API senders. Both take a typed credential rather than a browser sign-in.

`src/trading/ata_spm_send.py` — the two senders

```
SEND_ROUTES = {
    TARGET_DISCORD: send_discord,
    TARGET_TELEGRAM: send_telegram,
}

def build_sender(settings: Any, transport: Any = None) -> ApiSender:
```

The two target names those keys use are declared beside the rows, in `src/trading/ata_spm_push.py`.

`src/trading/ata_spm_push.py` — which route a post takes, and which targets a sender serves

```
def route_for(post: Any, settings: Any) -> str:
def deliver_one(post: Any, sender: Any, now: Any) -> Any:
def sender_takes(sender: Any, target: Any) -> bool:
```

A sender serves every target while it names none of its own, which is what a `RecordedDestination` does, and otherwise only the ones it names.

One request per post, as multipart form data, through the same address opener the sign-ins use. A venue answering 429 is read for its wait. A 401 or 403 is read as the credential refused, with what the venue said. A caption over its venue's ceiling is refused before any request, with the ceiling and the measure named. Nothing retries by itself; the post waits in the bucket for the next press.

`src/trading/ata_spm_send.py` — the three refusals a send reports

```
RATE_LIMITED_FORMAT = "429, retry after {seconds:g} s"
CREDENTIAL_REFUSED_FORMAT = "{http}, the credential was refused: {said}"
HTTP_REFUSED_FORMAT = "{http}, {said}"
```

The operator set the bound on how far this goes.

> *This will have to do. Do not want to build something that risks myself or others being banned.*

All three send routes obey one ceiling, counted over the hour ending now. The ceiling starts unset, and an unset ceiling releases nothing: a fresh install cannot publish until the operator sets a number.

`src/trading/ata_spm_push.py` — the ceiling every send route obeys

```
def allows(self, now: float, ceiling: Any) -> bool:
    """Whether one more post fits under ``ceiling`` at ``now``."""
    limit = int(ceiling or NO_CEILING_SET)
    if limit <= NO_CEILING_SET:
        return False
    return self.sent_within_hour(now) < limit
```

`src/trading/ata_spm_push.py` — the two refusals a held post reports

```
NO_CEILING_TEXT = "Max posts per hour is unset. Nothing leaves."
RATE_HELD_TEXT = "{sent} post(s) sent this hour, ceiling {ceiling}."
```

A second guard holds a repost back. A ticker reaches one push target once an hour, whatever composed the post, so a second call on the same ticker is refused exactly like a repeat of the first.

`src/trading/ata_spm_push.py` — the hour the guard measures

```
def allows(self, now: float, symbol: Any, target: Any) -> bool:
    """Whether an hour has passed since ``symbol`` last reached ``target``."""
    gap = self.since(now, symbol, target)
    return gap is NO_SEND_RECORDED or gap >= SECONDS_PER_HOUR
```

`src/trading/ata_spm_push.py` — what a held post reports

```
REPOST_HELD_FORMAT = (
    "{symbol} reached {target} {minutes:.0f} minute(s) ago; "
    "one post per ticker per hour."
)
```

Each bucket entry's status line carries its venue, its approval state, and then the last press's outcome.

`src/trading/ata_spm_push.py` — the five outcomes a status line can carry

```
STATUS_SENT_FORMAT = "sent · {destination}"
STATUS_NOT_SENT_FORMAT = "not sent · {detail}"
STATUS_IN_FOLDER_TEXT = "in folder"
STATUS_INTENT_OPENED_TEXT = "intent handed to the OS"
STATUS_INTENT_READY_TEXT = "intent ready"
```

Before any press the badge carries the confirmation timer's status instead.

`src/gui/main_tabs/market_inspector_surface.py` — the badge one entry's head row draws

```
def bucket_badge(held: Any) -> tuple:
    """The last press's delivery, else the timer's own status."""
```

`src/trading/ata_venue_folders.py` — the text one post writes beside its picture

```
def post_text(post: Any) -> str:
```

Every press runs on its own thread, one per press, and records cross back to the window thread. A press made while one runs is refused on the Activity Log.

`src/gui/market_inspector.py` — the hand-off thread

```
HAND_OFF_THREAD_NAME = "ata-smp-hand-off"
HAND_OFF_BUSY_TEXT = "ATA-SPM %s pressed while a hand-off is running; press ignored"
```

`src/trading/ata_spm_push.py` — the lines a press writes

```
PRESS_LINE_FORMAT = "ATA-SPM hand-off: {line}"
NOTHING_APPROVED_TEXT = (
    "ATA-SPM Post All: no post is approved. Approve one, then press again."
)
FULL_AUTO_TOGGLED_FORMAT = "ATA-SPM Send Bucket Full Auto: {state}."
```

Every send writes one block on the Live tab's API Interaction Log naming the venue, the action, the post, the text length, the image size and the message id or the refusal. The recorded endpoint names the route with the token left out.

`src/trading/ata_spm_send.py` — the two recorded endpoints

```
DISCORD_LOGGED_ENDPOINT = DISCORD_API_BASE + "/<id>/<token>?" + DISCORD_WAIT_QUERY
TELEGRAM_LOGGED_ENDPOINT = TELEGRAM_API_BASE + "/bot<token>/" + TELEGRAM_SEND_PHOTO
```

Every entry the connector records reaches that log from whatever thread made it. The main window carries one signal and one receiver, and the Live tab registers the receiver, so an entry from a scan thread crosses the signal and lands on the GUI thread. The receiver refuses any other thread and writes a violation file instead.

`src/gui/main_window.py` — the crossing every API entry takes

```
    apiEntryLogged = Signal(object)

    def _cross_api_event(self, entry: Any) -> None:
    def _on_api_event(self, entry: Any) -> None:
```

The scan's own progress and its failure cross the same way, and the tab hands the inspector the Live tab's Activity Log pane. Where that pane is absent the phase lines reach the log alone, and the tab says so.

`src/gui/market_inspector.py` — the three signals and the log the tab writes to

```
    scanProgressed = Signal(object)
    scanFailed = Signal(str)
    ACTIVITY_INFO = "info"

    def _say(self, message: str, level: str = ACTIVITY_INFO) -> None:
    def _on_scan_progress(self, progress: Any) -> None:
    def _set_scan_busy(self, busy: bool) -> None:
    def set_activity_log(self, log: Any) -> None:
```

The venue sign-in pages

Level 1A shows the venue name, the address it posts to, the permissions it asks for, and one box per value that venue needs. It shows no other venue's boxes.

`src/trading/ata_spm_push.py` — what one press opens

```
    def open_credentials(self, target: Any) -> Optional[str]:
        """Show one push target's Level 1A page, and answer which target it draws."""
        found = push_target(target)
        if found is None:
            return None
        self.settings_open = True
        self.credential_target = found.name
        self.connect_result = None
        return self.credential_target
```

Each venue row declares two sets. The fields are the values the operator types, which a venue hands him when he registers an application. The issued set is the values the sign-in obtains, which are never typed.

Venue	Boxes on its page	The sign-in issues	Posts to
X	Client ID, Client secret	Access token, Refresh token	`https://api.x.com/2/tweets`
Instagram	App ID, App secret	Instagram user id, Access token	`/<IG_ID>/media` then `/<IG_ID>/media_publish`
LinkedIn	Client ID, Linkedin-Version	Access token	`https://api.linkedin.com/rest/posts`
TikTok	Client key, Client secret, Verified URL prefix	Open id, Access token, Refresh token	`/v2/post/publish/content/init/`
Facebook	App ID, App secret	Page id, Page access token	`/<page_id>/feed` and `/<page_id>/photos`
Threads	App ID, App secret	Threads user id, Access token	`/<threads-user-id>/threads` then `/threads_publish`
Reddit	App ID, App secret, Subreddit, User agent	Access token, Refresh token	`https://www.reddit.com/api/v1/access_token` then `/api/submit`
Discord	Webhook URL	none	the channel's own webhook
Telegram	Bot token, Chat id	none	the bot's `sendPhoto`
WhatsApp	none	none	the Click-to-Chat address in the venue folder

Every value came off that platform's own published documentation, recorded in the platform rules audit.

Seven venues use three-legged OAuth: not one issues a working token from an app id and a secret alone. The operator sends himself to the venue in a browser, approves there, and the venue sends a code back to the program. A venue reads as held on Level 1 only once both sets are in the vault, so the button turns on when the sign-in has actually run.

Venue	Approval address	Token address
X	`https://x.com/i/oauth2/authorize`	`https://api.x.com/2/oauth2/token`
Instagram	`https://www.instagram.com/oauth/authorize`	`https://api.instagram.com/oauth/access_token`
LinkedIn	`https://www.linkedin.com/oauth/native-pkce/authorization`	`https://www.linkedin.com/oauth/v2/accessToken`

Venue	Approval address	Token address
TikTok	`https://www.tiktok.com/v2/auth/authorize/`	`https://open.tiktokapis.com/v2/oauth/token/`
Facebook	`https://www.facebook.com/v25.0/dialog/oauth`	`https://graph.facebook.com/v25.0/oauth/access_token`
Threads	`https://threads.com/oauth/authorize`	`https://graph.threads.com/oauth/access_token`
Reddit	`https://www.reddit.com/api/v1/authorize`	`https://www.reddit.com/api/v1/access_token`

X, LinkedIn and TikTok require PKCE for a desktop program, so those three send a `code_challenge` on the approval call and a code verifier on the token call. Every route sends its own `redirect_uri` on both calls.

Three venues take a second call after the exchange: Instagram and Threads trade the first token for one that lasts sixty days, and Facebook trades for a long-lived user token and then reads the Page token off the account. Reddit authenticates with HTTP Basic, refuses a generic user agent, and issues no refresh token unless the request asks for a permanent duration. TikTok names its client field `client_key`. LinkedIn's native address takes no client secret at all.

Venue	How it renews
X	by itself, refresh token grant, which needs `offline.access`
Reddit	by itself, refresh token grant
TikTok	by itself, refresh token grant, valid 365 days
Instagram	by itself, once the token is 24 hours old
Threads	by itself, once the token is 24 hours old
Facebook	no renewal; a long-lived Page token carries no expiry date
LinkedIn	does not renew; partner-only, and he approves again every 60 days

Six of the seven bind a loopback listener. It binds the loopback address and nothing else, serves exactly one request, refuses a reply carrying a state it did not generate, and closes. It never outlives one sign-in.

`src/trading/ata_spm_signin.py` — the listener, and the two ports

```
LOOPBACK_HOST = "127.0.0.1"
EPHEMERAL_PORT = 0
FIXED_CALLBACK_PORT = 8723
CALLBACK_TIMEOUT_SECONDS = 180.0
TRANSPORT_TIMEOUT_SECONDS = 30.0
```

Four venues check the redirect character for character and publish no wildcard, so no address a random port produces can ever match. Those four bind one declared port, and Level 1A prints it. LinkedIn asks a native client for a random port and TikTok publishes a wildcard port, so those two keep the operating system's port.

Venue	The redirect to register
X	`http://127.0.0.1:8723/callback`
Instagram	`http://127.0.0.1:8723/callback`

Venue	The redirect to register
LinkedIn	nothing; its native page asks for no registered address
TikTok	`http://127.0.0.1:*/callback`
Facebook	`https://www.facebook.com/connect/login_success.html`
Threads	`http://127.0.0.1:8723/callback`
Reddit	`http://127.0.0.1:8723/callback`

Facebook is the seventh, and it opens no listener. Meta requires HTTPS for an OAuth redirect and publishes one desktop address, and it answers a desktop app in the fragment of that address rather than with a code. A fragment never leaves the browser, so the program draws the browser itself, for one sign-in at a time.

`src/trading/ata_spm_signin.py` — the desktop reply Facebook sends

```
DESKTOP_RESPONSE_TYPE = "token"
FACEBOOK_DESKTOP_REDIRECT = "https://www.facebook.com/connect/login_success.html"
```

`src/gui/sign_in_view.py` — the browser the program draws

```
class SignInView(QDialog):
    """One modal sign-in view, one web view, no address bar and no context menu."""
```

The sign-in view reaches exactly two addresses: the venue's approval address and the venue's published redirect. Every other host ends the sign-in on the spot. It opens no second window, cancels every download the venue's page asks for, and empties the cookie jar of the off-the-record profile it held. Reaching the redirect ends the sign-in and the reply's fragment is read. The operator closing the window ends it, and so does 180 seconds passing.

`src/trading/ata_spm_signin.py` — the two hosts one sign-in may reach

```
def sign_in_hosts(authorize_url: Any, redirect_address: Any) -> tuple:
    """The only two hosts one sign-in view may load."""
```

Discord and Telegram issue no token, so Connect checks that the typed value reads as the form the venue's own page prints. It opens no browser and reaches no venue. A webhook address or a bot token in another form is refused on the page, with the form named. WhatsApp needs no sign-in at all.

`src/gui/main_tabs/market_inspector_surface.py` — the two wordings a typed venue carries

```
SIGN_IN_TYPED_TEXT = (
    "Connect checks what you typed and holds it. No browser opens and no "
    "venue is reached."
)
SIGN_IN_NONE_TEXT = (
    "No sign-in. The post goes through the venue folder and the compose address."
)
```

Level 1A returns to Level 1 by itself only when the venue accepts. An empty box, a missing route and a refusal from the venue each keep the page open and print what failed.

`src/trading/ata_spm_push.py` — the four wordings Connect can print

```
CONNECT_OK_FORMAT = "{target} accepted the credential."
CONNECT_FAILED_FORMAT = "{target} refused the sign-in: {error}"
MISSING_FIELD_FORMAT = "{label} is empty."
NO_CONNECTOR_FORMAT = "No sign-in route wired for {target}."
```

A refusal names the venue's own reason. The published standard puts that reason in two fields, and only those two are read out.

`src/trading/ata_spm_signin.py` — the three refusal wordings

```
HTTP_REFUSAL_FORMAT = "HTTP {status} {reason}"
VENUE_SAID_FORMAT = "{http}, and the venue said {said}"
VENUE_REASON_FORMAT = "{error}: {description}"
```

Every outcome writes one log line naming the target, whether it succeeded, and the detail. The message line takes one colour per outcome, read by both builds: no colour before a press, the theme's error colour on a refusal, and its success colour on an acceptance.

`src/gui/main_tabs/market_inspector_surface.py` — the colour one outcome takes

```
NO_COLOR = ""
MESSAGE_REFUSED_COLOUR = ds.ERROR
MESSAGE_ACCEPTED_COLOUR = ds.SUCCESS

def message_colour(answered: Any) -> str:
    """``NO_COLOR`` before a press, else the refused or the accepted colour."""
```

A pasted value is trimmed before it is judged and before it reaches a venue, and a box holding only spaces is still reported empty.

`src/trading/ata_spm_push.py` — what Connect reads, and what it answers

```
def connect_credentials(self, target: Any) -> ConnectResult:
def missing_field(self, target: Any) -> Optional[str]:
def typed_credential(self, target: Any, field: Any) -> str:
def set_credential_text(self, target: Any, field: Any, typed: Any) -> None:
def store_credential(self, target: Any, field: Any, value: Any) -> bool:
def set_connector(self, connector: Any) -> None:
def set_vault(self, vault: Any) -> None:
CONNECT_RESULT_LOG = "ATA-SPM sign-in on %s: accepted=%s, %s"
```

`src/trading/ata_spm_signin.py` — the two typed routes, and how a refusal is read

```
REFUSAL_BODY_LIMIT = 4096

class SignInError(Exception):
    """A venue's own refusal, carrying ``refusal_text``."""

def sign_in_discord(typed: Any, session: Any) -> Any:
def sign_in_telegram(typed: Any, session: Any) -> Any:
def urlopen_transport(request: Any) -> Any:
```

Each host builds the session those routes run in, through `sign_in_session` on its own side.

A refusal body is read up to that limit and no further.

`src/gui/sign_in_view.py` — what the sign-in view refuses

```
class SignInPage(QWebEnginePage):
    def createWindow(self, kind: Any) -> None:
        """None, so no second window opens."""

    def _refuse_download(self, item: Any) -> None:
    def _release(self) -> None:
    def reach(self, url: Any) -> bool:
```

The credential vault

The tokens are kept in one file outside the repository, encrypted under the same passphrase every other stored credential in the product uses.

`src/core/encryption.py` — where the tokens sit

```
DEFAULT_VAULT_PATH = Path.home() / ".acervator" / "ata_spm_credentials.json"
```

A keystroke writes memory and a finished box writes the vault. A box is finished when the operator leaves it. A held value stores under its own key and an emptied box deletes that key. A vault write runs the key derivation at the published iteration count, which is why a write on leaving the box costs one such write rather than one per keystroke.

`src/core/encryption.py` — the derivation cost

```
_KDF_ITERATIONS = 600_000  # OWASP PBKDF2-HMAC-SHA256 guidance
```

`src/trading/ata_spm_push.py` — the key one value is held under

```
VAULT_KEY_FORMAT = "{target}:{field}"

def vault_key(name: Any, field: Any) -> str:
```

`src/core/encryption.py` — the vault, and the two reads the page makes on every paint

```
class CredentialVault:
    def store(self, key: str, value: str, salt: str) -> None:
    def delete(self, key: str) -> None:

class FileVault(CredentialVault):
    """A ``CredentialVault`` reading ``DEFAULT_VAULT_PATH`` and writing it on store."""

def default_vault() -> FileVault:
def vault_phrase(username: str) -> str:
def has_exchange(key: str) -> bool:
def encrypt(value: str, phrase: str) -> str:
```

`has_exchange` answers whether a value is held without decrypting one, which is why Level 1A can ask it on every paint.

Nothing typed on the page reaches a view model, a render or a log. The page publishes each box as a name and a wording and no value, plus the keys the vault holds something for. A held box's wording says so, and typing into it replaces what is held when the operator leaves it.

`src/gui/main_tabs/market_inspector_surface.py` — what one page publishes

```
def credential_page(target: Any) -> dict:
    """One venue's page: its lines, its boxes as name and wording, and no value."""
```

`src/trading/ata_spm_push.py` — the keys the vault holds something for

```
def held_fields(self, target: Any) -> tuple:
```

`src/gui/main_tabs/market_inspector_surface.py` — the wording a held box carries

```
CREDENTIAL_HELD_PLACEHOLDER = "Held · type to replace"
```

`src/trading/ata_spm_push.py` — what a credential row on Level 1 publishes

```
CREDENTIAL_HELD_TEXT = "held"
CREDENTIAL_MISSING_TEXT = "not held"
```

A page the operator filled before a restart takes Connect without being typed again. There is no Save credentials control: Connect is what stores. The exchange keys sit in the settings file and the push-target tokens in the vault file, two files under one passphrase.

What each venue needs before its page can work

Every platform issues these values only to an application the operator has registered with that platform. The software cannot obtain a registration, so each page names what is needed and where to register.

Venue	Where to register	Review and cost
X	`https://console.x.com`	no review; a paid usage plan, about $0.015 a post and about $0.20 with a link

Venue	Where to register	Review and cost
Instagram	`developers.facebook.com/apps/creation/`	no review, free; a professional account on a Page, and a role on his own app
LinkedIn	`www.linkedin.com/developers/apps`	two tiers of review; a registered company, a verified Page, a screencast, and LinkedIn must switch on its native flow by hand
TikTok	`developers.tiktok.com/apps`	audit for a public post, free; until the audit, only he can see what it posts
Facebook	`developers.facebook.com/apps/creation/`	no review, free; a Page he administers and a role on his own app
Threads	`developers.facebook.com/apps/creation/`	no review, free; a Threads profile and a role on his own app
Reddit	`www.reddit.com/prefs/apps`	no review, free; register an app and pick a subreddit

Meta grants standard access to every permission automatically, and it reaches any account holding a role on the app, so Instagram, Facebook and Threads need no app review. TikTok restricts every post an unaudited client makes to private viewing. LinkedIn is the one venue that may refuse him outright, and its page states both gates rather than offering a Connect that looks like it will work.

A word on a page becomes a link only where it names a host and a path. Every address the open page publishes reaches the browser, and every address it does not publish opens nothing and is recorded as refused. Nothing the operator typed and nothing a venue answered is ever a link.

`src/gui/main_tabs/market_inspector_surface.py` — which words are links

```
def link_address(word: Any) -> str:
    """``word`` as an openable address where it names a host and a path, else empty."""
```

The hit chime

A hit sounds one chime the moment the window learns of it. A confirmation sounds one more. A failed call sounds nothing. The tone is the Audio Suite's own generator, one preset joining its table, rendered once per process to a short wav with a decay applied and played through one output on the tab.

`src/gui/audio_suite.py` — the chime

```
CHIME_PRESET = "Hit Chime"
CHIME_PARTIALS = [(1.0, 0.45), (2.0, 0.18), (3.0, 0.09), (4.16, 0.05)]
CHIME_SECONDS = 0.6
CHIME_BASE_HZ = 880.0
```

The chime plays at 0.5, the volume the suite's music player opens at. The Audio Suite holds no mute control, so the chime has no mute and no slider of its own.

The operator asked for the chime and the timer together.

> *Simulator - Scan Now - ... There should be a chime when there is a hit. Once a hit is made, an isolated confirmation read timer starts based on the original hit's TF. The confirmation read mechanic should be partially documented already if not implemented...*

The confirmation read timer

Each hit starts its own timer the moment it lands in the approval bucket. The first read waits for the candle the hit was read on to close. Every read after it waits a count of closed candles of the hit's own timeframe, and that count is a setting starting at three. A daily hit reads again three days on; a weekly hit three weeks on.

`src/trading/ata_spm_push.py` — the grid each read lands on

```
DEFAULT_CONFIRMATION_CANDLES = 3
ONE_CANDLE = 1

def next_read_ts(
    at_ts: Any, timeframe: Any, now: Any, candles: int = ONE_CANDLE
) -> float:
```

The operator set that spacing.

> *Confirmation timers are too short and should be a multiple of the timeframe on which the hit occurred i.e. a weekly chart confirmation read should not be happening again in a few days but in a few weeks and a daily chart should happen in a few days rather than a few hours.*

The grid is anchored at the first close, so a read the venue delayed and a retry both land back on it. A changed count re-anchors a running timer: it keeps the grid beginning at the hit's own candle close and takes the new step.

`src/trading/ata_spm_push.py` — the seconds one candle takes

```
TIMEFRAME_SECONDS = {"5m": 300, "1h": 3600, "1d": 86400, "1w": 604800}
MONTH_TIMEFRAME = "1M"
```

A weekly candle closes at Monday 00:00 UTC, the instant the fetcher's rollup and the chart venue both stamp a week with. A monthly candle closes at the first instant of the month.

A venue that has not published the awaited candle is read again after a short wait, a bounded number of times, then at the next close.

`src/trading/ata_spm_push.py` — the retry

```
FOLLOW_UP_RETRY_S = 60.0
FOLLOW_UP_RETRY_CAP = 5
```

The tab's own clock wakes once a second and hands every due timer to one worker thread. The worker reads the market's newest candles through the scan's own route and judges the call. The outcome crosses back to the window thread, which writes the entry, the field, the Activity Log and the emitter row.

The verdict counts closed candles and reads no clock; the timer reads the clock only to choose when to read. The candles counted are those opened at or after the call's bar open whose close time is at or before the read, so the call's own candle is candle one once it closes. A venue's partial bar is never read as a close.

`src/trading/ata_spm_push.py` — the candles one read counts

```
def closed_candles(candles: Any, timeframe: Any, closed_before: Any) -> list:
    """The candles of ``candles`` that opened at or after the call and have closed."""
```

Three outcomes exist.

`src/trading/ata_spm_push.py` — the three verdicts

```
OUTCOME_CONFIRMED = "confirmed"
OUTCOME_FAILED = "failed"
OUTCOME_OPEN = "open"
```

Price moving the favourable way as far as the confirmation target confirms the call. The target is a share of the run from the call's close to the Bollinger midline, and the midline is recomputed on every candle after the call rather than frozen at it. The share is the confirmation share setting, and a share of zero sets no target, so the read says so instead of claiming a result.

`src/trading/ata_spm.py` — the moving target phase seven measures against

```
def midline_after(candles: Any, at: Any) -> list:
    """The Bollinger middle band at every candle after index ``at``.

    ``BollingerBands`` computes its published band on each window, so the
    target phase seven measures against moves with the market.
    """
```

A target exists only while the midline lies on the side the call expects the run to go: above a bullish close, below a bearish one. While the midline lies behind the call, the read stays open and says so.

`src/trading/ata_spm_push.py` — whether a target exists yet

```
def midline_ahead(direction_text: Any, call_close: Any, midline: Any) -> bool:
    """Whether the midline lies on the side the call expects the run to go."""
```

The trend carrying on for three candles in a row fails the call. A continuation is evidence against a reversal call, so the two outcomes are not the same shape. The count is in the call's own timeframe, and no wall clock is read, so a daily call cannot fail on the afternoon it was made.

`src/trading/ata_spm_push.py` — the candles a failure needs

```
CONTINUATION_CANDLE_FLOOR = 3
```

A call short of that floor is not ready. The read then says how far the trend has run against the call instead of claiming a result, and no candle is invented to reach the floor.

`src/trading/ata_spm_push.py` — the wording a call under the floor carries

```
FOLLOW_UP_NOT_READY_FORMAT = (
    "the trend has held for {run} of the {floor} candles a failure needs, "
    "last close {close:g}, target {target:g} not reached"
)
```

A settled call stops being watched and an open one stays, so no call is posted on twice. A settled call's follow-up post is one per venue and lands in the bucket as the scan's own entries do. A timer stops when its read settles the call, or when its entry leaves the bucket.

`src/trading/ata_spm_push.py` — the status a timer publishes

```
BUCKET_META_FOLLOW_UP_FORMAT = "{target} · {state} · {follow_up}"
FOLLOW_UP_STATUS_OPEN_FORMAT = "open · next read {when}"
FOLLOW_UP_STATUS_SETTLED_FORMAT = "{state} · close {close:g}"
```

`src/trading/ata_spm_push.py` — the lines a timer and a read write

```
FOLLOW_UP_TIMER_LINE_FORMAT = (
    "ATA-SPM confirmation timer for {symbol} {label}: next read {when}"
)
FOLLOW_UP_READ_OPEN_LINE_FORMAT = (
    "ATA-SPM confirmation read for {symbol} {label}: {state}, next read {when}"
)
FOLLOW_UP_READ_SETTLED_LINE_FORMAT = (
    "ATA-SPM confirmation read for {symbol} {label}: {state}, close {close:g}"
)
```

The timer tiles

The region right of the Timeframe row, the scan line and the stepper holds one tile per watched call, in both builds. A tile reads the market and its timeframe on its first line, with a small red cross at the right, and the time to the call's next read on its second, counted down once a second on the tab's own clock. At zero the read fires and the tile reads `reading` until the outcome crosses back. An open call then counts down to its next close; a confirmed or failed call reads its outcome and its close. A settled call's tile stays until its entry leaves the bucket.

`src/trading/ata_spm_push.py` — what a tile reads

```
TIMER_PAIR_FORMAT = "{symbol} {label}"
TIMER_COUNTDOWN_FORMAT = "{hours:02d}:{minutes:02d}:{seconds:02d}"
TIMER_COUNTDOWN_DAYS_FORMAT = "{days}d {hours:02d}:{minutes:02d}:{seconds:02d}"
TIMER_READING_TEXT = "reading"
```

Tiles wrap by the region's width at 132 pixels each, the width the widest pair a class lists takes at the caption size. The region is 110 pixels tall, three tile rows, and scrolls past that, so the tiles never push the stepper and the field down the zone. While no call is watched the region reads "No confirmation timer running."

The cross opens one pop-up naming the pair. No leaves the tile, its countdown and its timer standing. Yes removes that tile and its timer and nothing else: the call keeps its entry in the bucket, the bucket keeps its count, and every other tile keeps its countdown. The Activity Log records the removal.

`src/gui/main_tabs/market_inspector_surface.py` — the cross and its question

```
  TIMER_CLOSE_PART = "timer-close"
  TIMER_CLOSE_TEXT = "×"
  TIMER_DROP_TITLE = "Delete confirmation timer"
```

The operator asked for the cross.

> *Confirmation Reading timers need a small red 'x' for their deletion and this provides a confirmation pop up.*

The pop-up comes from the host in both builds, the way the hit chime does, so both builds ask the same question.

`src/trading/ata_spm_push.py` — the rows the region draws, and what a removal takes

```
  @dataclass(frozen=True)
  class TimerTile:
      """One watched call's pair, its countdown and its outcome."""

  @dataclass
  class FollowUpTimer:
      """One call's next read, its read count and the count it steps by."""

  def timer_tiles(self) -> list:
  def drop_timer(self, value: Any) -> bool:
      """Drop one open timer from ``timers`` or one call from ``settled``."""
  def watch_run(self, run: Any, candles: Any) -> None:
  def take_outcome(self, outcome: Any, candles: Any) -> None:
  def after_scan(self, run: Any) -> None:
  def candle_close_ts(at_ts: Any, timeframe: Any) -> float:
```

The Qt widget draws those rows as frames and the page draws the same rows from its own skin, with the cross as its own element. The clock's tick hands the page the rows alone, so a countdown moves without the whole payload.

The tile takes its colours from the theme's own tokens, so a theme change moves them.

```
  tile ground     SURFACE_INPUT      border        OUTLINE
  the pair        TEXT_HIGH          countdown     PRIMARY
  reading         WARNING            confirmed     SUCCESS      failed  ERROR
```

The Qt widget draws those rows as frames and the page draws the same rows as its own elements. Each cross carries that row's own value, and both hosts reach one method on the tab, which opens the pop-up and drops the timer on a Yes.

`src/gui/market_inspector.py` — the Qt tiles and the question

```
class TimerTiles:
    def _build_tile(self, row: Any) -> QFrame:
    def show_tiles(self, rows: Any) -> None:

    def _ask_drop_timer(self, value: Any) -> None:
```

`src/gui/web/market_inspector.js` — the page's own cross

```
function TimerClose({ closeValue }) { ... }
```

The called markets the Charts tab keeps watching

Reaching the approval bucket does a second thing. The market joins a list the Charts tab keeps, so the operator can watch a called market long after the post about it has gone or been declined.

The board answers one row per market, never one per post. A market called on two timeframes is still one market being watched, and the row carries the timeframes it was called on and the vote of the most recent call.

`src/trading/ata_spm_push.py` — the row the Charts tab reads

```
@dataclass(frozen=True)
class WatchedMarket:
    """One market on the ATA-SMP chart list, and the call that queued it."""

    symbol: str
    vote: str = VOTE_NEITHER
    timeframes: tuple = ()
```

Three places hold a called market and the board reads all three, oldest first: the calls phase seven has settled, the calls it is still watching, and the posts sitting in the bucket now. Filling the bucket from a new scan therefore does not drop an older market from the chart list. The settled calls are read in key order, so the list comes back in the same order every time.

A watched call is told apart by its asset, its timeframe and its bar's open time, and by nothing else, so a hit recorded while a retired class name was chosen still resolves and draws its tile.

`src/gui/market_inspector.py` — what this screen offers the Charts tab

```
def watched_markets(self) -> list:
    """The markets ATA-SMP has called, for the Charts tab's second list.

    ``PushBoard.watched_markets`` is the one set phase seven also reads.
    """
    return self._push_board.watched_markets()
```

The screen hands that reader to the Charts tab when it is built. Nothing is copied across, so the two screens cannot disagree about which markets are called. What the Charts tab does with the list is on the Charts tab page.

Multi-Exchange Arbitrage

`src/trading/arbitrage.py` holds the cross-exchange price monitor and its spread tracking. No module under `src/` imports it. The zone names the venues in reach and stops there: it draws no arbitrage panel. The first two proposal forms are on screen; the third is not.

In development.

Phantom Bot HTF Signals

The zone is named, sized and drawn, and the row that builds it writes the same sentence on every call. Nothing feeds it.

`src/gui/main_tabs/market_inspector_surface.py` — the row that never varies

```
PHANTOM_HTF_ZONE = "phantom_htf"
PHANTOM_HTF_GROUP_TITLE = "Phantom Bot HTF Signals"
PHANTOM_HTF_UNWIRED_TEXT = "Phantom Bot source not wired."

[PHANTOM_HTF_ZONE, PHANTOM_HTF_GROUP_TITLE, PHANTOM_HTF_UNWIRED_TEXT],
```

In development.

The section above is overtaken, and it is kept as written. The zone now reads the fleet. It steps through one entry for each bot that runs a phantom timeframe, in fleet order. A bot whose phantom bots are off gets no entry. The Indicator Voting Panel does the same: it adds no phantom row for that bot.

`src/gui/main_tabs/market_inspector_surface.py` — `phantom_entries`

```
return [phantom_entry(one) for one in bots or [] if phantom_timeframe(one)]
```

The tab builder hands the zone one lookup into the bot manager. On each two-second refresh the zone looks up the bots the other tabs are fed, and draws them again.

`src/gui/main_tabs/market_inspector_tab.py` — `MarketInspectorTabMixin._build_market_inspector_tab`

```
self._market_inspector.set_phantom_source(self._fleet_bot)
```

One entry names the market, the bot's own timeframe and its phantom timeframe. The line under it gives Comp and Conf for each of the two. The next line gives the gate sentence for each, in the words of the bot's own GATES line.

```
ZZZ/USD 1h with phantom 1d
1h Comp +0.94 Conf 8% · 1d Comp +2.64 Conf 23%
1h gates: Scrum blocked adx_trend_suppression, efficiency_ratio_regime, overridden TA.
Fold blocked BB, MID, TA, TRNQ, zscore_extremity. · 1d gates: Scrum blocked FIRE,
adx_trend_suppression. Fold blocked BB, MID, TA.
```

Clicking the entry opens two grids. The first is the Indicator Voting Panel, with one row for each timeframe. The second is the nineteen gate lights, the scrum bank above the fold bank, with one row for each timeframe. Each light keeps the colour the History tab gives it.

The bot's row comes from its last tick. The phantom's row comes from the phantom's last read. On each pass of its loop, the phantom runs both of the bot's gate chains over the candles it read, at the bot's own settings. That reading changes no trade.

`src/trading/phantom_balance.py` — `PhantomBot._tick`

```
self.last_gates = timeframe_gates(
    self.symbol, self.timeframe, candles, summary, self.settings
)
```

A higher timeframe reads the market and not the position. The six gates that read the position do not run at the phantom read. Their lights stay grey, and the entry names them under the grid. The bot's own row holds those six.

```
1d position: Scrum TGT, INT, OTD and Fold TRNQ, CEIL, OTD did not run. They read
the bot's position, at 1h.
```

Piece	Module	Symbol
the bots the zone reads	`src/gui/main_tabs/market_inspector_surface.py`	`phantom_fleet`
the one active timeframe	`src/trading/phantom_balance.py`	`one_active_phantom_timeframe`
the bot's own lights	`src/trading/scrumming/snapshots.py`	`state_reading`
the phantom's lights	`src/trading/ata_gate_scan.py`	`scan_reading`
the gate sentence	`src/trading/scrumming/snapshots.py`	`bank_text`
Comp for each row	`src/trading/ata_gate_scan.py`	`composite_rows`
the gate light grid	`src/gui/main_tabs/market_inspector_surface.py`	`lights_panel`

Driven on a constructed bot over a stand-in venue that refuses every order, the zone's lights equal the lights the bot's own chains decide at both timeframes. One changed gate input moves one light, which shows the comparison can report.

```
bots in the fleet                          2, one running a phantom timeframe
entries in the zone                        1; 2 with the second bot's phantoms on
lights equal to the bot's own chains       19 of 19 at 1h, 19 of 19 at 1d
one gate input changed, lights that moved  1 at 1h, 1 at 1d
Qt and React fields compared               20 equal, 0 different
orders placed, venue sockets opened        0, 0
```

The asset logo library

An asset's mark is a library on disk, divided into a folder per asset class and a folder per sector inside it. The library is filled by a step the operator starts himself. Creating a position reads that library and never fetches.

`src/trading/logo_library.py` — where one asset's mark is filed

```
  def library_folder(asset_class: Any, sector: str = "") -> str:
      """``kept_folder(f"{asset_class}/{sector}")``, every segment sanitised."""
```

An asset with no sector tag sits in the class folder itself; no sector is guessed. Every folder segment is sanitised the way a file name already is, so a sector named with a space becomes one hyphenated folder.

The library covers every asset the connected venues list plus every asset the maps hold: 542 assets, filed into 25 folders.

Folder	Marks	Folder	Marks
crypto	361	crypto/interop	5
stocks/portfolio	47	crypto/oracle	5
forex/minor	21	commodities/energy	4
crypto/l1	18	crypto/depin	3
crypto/dex	9	crypto/identity	3
crypto/defi	8	crypto/wallet-infra	3
crypto/meme	8	commodities/industrial-metals	2
commodities/precious-metals	8	crypto/payments	2
crypto/ai	7	crypto/privacy	2
crypto/l2	7	crypto/stablecoin	2
forex/major	7	crypto/storage	2
crypto/gaming	6	crypto/rwa	1
		crypto/wrapped	1

The fill takes its asset list from the order rules already recorded for each venue, not from a fresh venue call. Nothing is asked of any exchange or broker while the library fills.

`src/trading/logo_library.py` — where the asset list comes from

```
  def venue_bases(document: Optional[dict] = None) -> tuple[str, ...]:
      """Every base asset the recorded venues list, sorted, each folded by ``underlying_base``."""
```

A venue lists some contracts on a multiple of an asset. The base is folded back onto the asset only when the remainder names an asset some source already lists, so no name is shortened on a guess.

```
1000A10   ->  A10      A10 is a recorded base
1000MOG   ->  MOG       MOG is a recorded base
1000PEPE  ->  PEPE      PEPE is a recorded base
1000SHIB  ->  SHIB      SHIB is a recorded base
00        ->  00        nothing is left after the digits
1INCH     ->  1INCH     INCH is not a recorded base
2Z        ->  2Z        Z is not a recorded base
```

Nothing in the running program starts the fill. It is started by hand, and it refuses outright while the application holds the lock on the runtime directory.

```
python -m src.trading.logo_library --reads 40
python -m src.trading.logo_library --list
```

`src/trading/logo_library.py` — the refusal while the fleet trades

```
TRADING_REFUSAL = (
    "the application is running and holds the instance lock; close it first"
)
```

The gap between two reads is 0.5 seconds, the figure the market scan already uses for a host that publishes no limit. The hold after a rate refusal is 60 seconds. A run is bounded by the number of addresses it may read, and a run that stops leaves the marks it kept on disk and the unresolved assets in a file beside them, so the next run reads only what is left.

`src/trading/logo_library.py` — where the unresolved assets are written

```
UNRESOLVED_NAME = "unresolved.json"
```

An asset written to that file is skipped by every later run, including one whose source has since come back.

`src/core/asset_logos.py` — the walk, its budget and what it answers

```
LOGO_TIMEOUT_S = 10.0

class LogoBudgetSpent(RuntimeError):
    """Raised where a run has read every address it was allowed."""

class LogoCache:
    def read_budget(self, count: int) -> None:
    def kept_path(self, symbol: str) -> Optional[Path]:

@dataclass(frozen=True)
class LogoAnswer:
    """One asset's kept file, or the reason no address served an image."""

def resolve(
    symbol: Any,
    candidates: Any,
    timeout_s: float = LOGO_TIMEOUT_S,
    *,
    page_url: str = "",
    no_source_reason: str = "",
    folder: str = "",
) -> LogoAnswer:
```

`src/trading/logo_library.py` — how a ticker is settled against the coin list

```
def choose_coin(ticker: Any, records: Any) -> Optional[dict]:
    """The coin a ticker names: by recorded name, by a lone rank, then by margin."""

def underlying_base(base: Any, known: Any) -> str:
```

Where a mark is looked for

A crypto asset's mark is looked up in two steps. One read names every coin and its id; a second read names the picture each id serves. A ticker is not a coin id, so an address built out of a ticker is a guess, and the address the ticker used to build serves a web page rather than an image.

`src/exchange/crypto_assets.py` — the retired guess

```
SYMBOL_ICON_URL = "https://www.cryptocompare.com/media/img/cc_icons/{symbol}.png"
```

`src/exchange/crypto_assets.py` — the two reads behind a coin's mark

```
COIN_LIST_URL = "https://api.coingecko.com/api/v3/coins/list"
COIN_MARKETS_URL = (
    "https://api.coingecko.com/api/v3/coins/markets"
    "?vs_currency=usd&per_page={size}&page=1&ids={ids}"
)
```

A ticker several coins carry is settled by the recorded name, then by a lone market rank, then by a rank far better than the next. A ticker it cannot settle is written down with its candidates and no address.

`src/trading/logo_library.py` — how far apart two ranks must sit

```
COIN_RANK_MARGIN = 10.0
```

Every other asset is read from the organisation's own domain, at the two standard locations a site serves its own icon at. Where neither answers an image, the organisation's own page is read and the icon that page declares is asked next, and last the share image the page declares. No third party is contacted, no key is held, and nothing has to appear on the screen.

`src/trading/ata_asset_maps.py` — the two standard locations, and the domains they are read from

```
ORGANISATION_ICON_FORMATS: tuple[str, ...] = (
    "https://{domain}/apple-touch-icon.png",
    "https://{domain}/favicon.ico",
)
ORGANISATION_SITES: dict[str, str]
```

`src/core/asset_logos.py` — the last location a mark is looked for

```
SHARE_IMAGE_TAG: re.Pattern[str]
```

`SHARE_IMAGE_TAG` matches a meta tag whose property or name is the page's own share image, folding case, so a page that answers its own document at every icon address still yields one picture.

`src/core/asset_logos.py` — the icons a page declares for itself

```
def declared_icons(page_url: Any, body: Any) -> tuple[str, ...]:
    """Every icon address one page declares, and its own share image last."""
```

A fund share and a currency pair answer their organisation's own site. Every other US-listed ticker answers the regulator's own company page, which is keyed on the ticker alone and needs no lookup, so an equity the map never carried still answers. A metal quote answers nothing, because a metal has no issuer.

`src/trading/ata_asset_maps.py` — the regulator route

```
REGISTRY_URL_FORMAT = (
    "https://www.sec.gov/cgi-bin/browse-edgar?action=getcompany"
    "&ticker={ticker}&type=10-K&dateb=&owner=include&count=10"
)
```

The regulator's own company page carries a web-address field and that field is empty, so the register names the company and not its site.

No logo read reaches a trading venue's own domain.

`src/trading/ata_asset_maps.py` — the domains no read reaches

```
VENUE_DOMAINS: frozenset[str] = frozenset({"coinbase.com", "sofi.com"})
```

A kept body must be an image, read from the body's own leading bytes rather than its length. A length floor alone kept a web page as a picture once; the signature does not.

`src/core/asset_logos.py` — the signatures a kept body must carry

```
IMAGE_SIGNATURES = (
    (b"\x89PNG\r\n\x1a\n", "png"),
    (b"\xff\xd8\xff", "jpg"),
    (b"GIF87a", "gif"),
    (b"GIF89a", "gif"),
    (b"\x00\x00\x01\x00", "ico"),
    (b"BM", "bmp"),
)
```

The file name is built from the symbol, with every character a file name may not carry replaced by one gap character, so a currency pair is kept under a hyphenated name.

`src/core/asset_logos.py` — the file name

```
def kept_name(symbol: str) -> str:
    """``symbol`` as the file name stem a kept logo takes, every other character ``KEPT_NAME_GAP``."""
    text = str(symbol).strip().upper()
    return "".join(
        one if one.isascii() and one.isalnum() else KEPT_NAME_GAP for one in text
    )
```

Reading a mark back by its symbol alone finds it at whatever depth it sits, so the bot list and the wizard need no knowledge of the folders.

What the library holds

Sector	Targets	Marks kept	Resolving nothing
crypto	453	362	91
stocks	47	38	9
currencies	28	24	4
commodities	14	10	4
total	542	434	108

The 434 marks take 7,455,763 bytes and carry 395 distinct pictures: 311 PNG, 82 JPEG, 37 ICO and 4 WebP, the smallest 660 bytes and the largest 175,010. Every picture two assets share belongs to one issuer, so seven fund shares share one asset manager's mark and each currency group shares its central bank's.

Seven reasons cover every asset that resolves nothing.

Reason	Assets
several coins carry the ticker at comparable market rank	67
no coin record carries the ticker, or its name	22
every address answered something that is not an image	7
the company no longer trades	4

Reason	Assets
a metal quote with no issuer	4
several coins carry the ticker and none carries a market rank	2
the domain names a trading venue	2

The 22 with no coin record are venue index and derivative products. Four organisations answer this platform's own user agent with a refusal, and one serves a certificate this machine cannot verify. Nothing was relaxed to reach any of them.

Two sentences this page used to carry are overtaken, and both are quoted whole.

> *"A stock, a fund share, a metal and a currency pair do not resolve the way a coin does."*
>
> *"the 47 stocks rows have no logo address a company's own domain is not a fact this repository holds for a ticker, and no keyless service keyed on a ticker was accepted"*

A stock and a fund share now resolve from the organisation's own domain, and 43 of the 47 stock rows carry a logo address. A metal quote still resolves nothing.

Opening an asset's own organisation

Every address offered as openable is checked for its scheme first, and a refused one is an empty address carrying the reason it is empty. Some schemes open a local file or run a script instead of visiting a site, and an address for an asset arrives from outside this repository. The check reads the same allowed set the guarded fetch reads, so the two cannot drift apart.

`src/core/safe_url.py` — the check every offered address passes

```
def openable_url(
    url: object,
    allowed_schemes: Optional[Iterable[str]] = None,
) -> tuple[str, str]:
```

The file, javascript, vbscript and data schemes open nothing. An empty address answers that no address is known.

Every caller now reads that reason instead of dropping it. A mark whose address was refused says so on its own tooltip and writes the same line to the log, and the coin index records the refusal against the ticker it belongs to. A refused address can therefore be told from an asset that never had one.

Where a mark is drawn

Six places draw an asset icon and all six read the library. A row draws the kept file or the lettered disc and never waits on a network read.

`src/gui/main_tabs/bot_status_table_surface.py` — the size a row draws

```
LOGO_SIZE_PX = 32
ROW_HEIGHT_PX = LOGO_SIZE_PX + 2 * ROW_LOGO_MARGIN_PX
```

Six of the 434 marks offer only 16 pixels, because their own source serves a 16-pixel icon, and those six draw at their own size in both builds. A row with no mark draws its symbol exactly as it does today. None of the kept

images enters the repository: the library directory is ignored.

`src/gui/bot_wizard.py` — the one reader every drawing place uses

```
KEPT_LOGOS = LogoCache()
```

`resolve` is never called from a wizard, and nothing there fetches. The six places that draw an asset icon are the three bot wizards and the three Extractor bot tables, one pair each under the live, the paper and the simulator directories.

```
src/gui/bot_wizard.py                       src/gui/widgets/extractor_bot_table.py
src/gui/paper/paper_bot_wizard.py           src/gui/paper/paper_extractor_bot_table.py
src/gui/simulator/sim_bot_wizard.py         src/gui/simulator/sim_extractor_bot_table.py
```

What the emitters write

Every row below goes through one wire and nothing else. A topic declared in `src/core/emit_contracts.py` is a tracked emitter, and a run that never fires it is reported.

The operator set what the network is for.

> *Entire idea for the Emitter Network is to provide us signals for verifying proper software function resulting from all user actions or automated sequences.*
>
> *Entire idea for the Emitter Network is to provide us signals for verifying proper software function resulting from all user actions or automated sequences...*
>
> *For this, will need to verify and / or wire in Emitter Network and update what the Inspector's subsystems (including ATA-SMP) write to the logs. Entire idea for the Emitter Network is to provide us signals for verifying proper software function resulting from all user actions or automated sequences...*

Topic	Written	Carries
`market_inspector.scan_started`	one per Refresh	forced, connector count, active symbols
`market_inspector.scan_finished`	one per Refresh	market count, duration, signals, pairs, source, error
`inspector.ata.scan_pressed`	one per press	whether a thread started, against True
`inspector.ata.scan_started`	one per press	the thread's name, against the scan thread's name
`inspector.ata.market_read`	one per market read	the candle count, against the vote floor
`inspector.ata.scan_finished`	one per press	markets read, against markets listed
`inspector.ata.volume_order`	one per class read	markets with a figure, against markets listed
`inspector.ata.hit`	one per hit	the running count, against the target
`inspector.ata.ticker_resolved`	one per press with text typed	the class the text landed in, against the class chosen
`inspector.scan.progress`	every ten markets and at the end	class, read, total, hits
`inspector.scan.list_source`	one per list read	class, source, count, the names not trading
`inspector.ata.candidate`	one per bucket entry	whether the image is on disk

Topic	Written	Carries
`inspector.ata.image_size`	one per venue image	the written size, against the row's size
`inspector.ata.handoff`	one per post a press took	venue, route, outcome, destination
`inspector.ata.sent`	one per request	venue, symbol, timeframe, status, message id, elapsed
`inspector.ata.chime`	one per chime	its cause, the hits it covers, whether the player took it
`inspector.ata.follow_up_read`	one per read	candles counted, against candles the clock says have closed

Each topic is named by one constant, so a topic and its writer cannot drift apart.

```
SCAN_PRESSED_PIN, SCAN_STARTED_PIN, MARKET_READ_PIN, SCAN_FINISHED_PIN
CANDIDATE_PIN, HANDOFF_PIN, SENT_PIN, IMAGE_SIZE_PIN
CHIME_PIN, FOLLOW_UP_READ_PIN
```

Bridge

Three methods serve this screen, and the renderer modules carry the matching names.

Bridge method	Serves
`market_inspector.state`	The scanner and its two tables
`market_inspector_tab.state`	The per-bot page in the Bot Details dialog
`market_inspector_topologies.state`	The proposal cards

Each bridge method reads one object, and the two hosts build one each.

Object	Where it is built	What it is
`MarketInspectorTab`	`src/gui/market_inspector.py`	the Qt widget tree, its steppers and its clock
`MarketInspectorScreenModel`	`src/gui/main_tabs/market_inspector_surface.py`	the same screen as values, which the React host draws
`AtaSpmSettings`	both of the above	the six settings, the vault and the sign-in route
`SectorScan`	`src/trading/ata_spm.py`	one sector's or one market's walk and its result
`SignInSession`	`src/trading/ata_spm_signin.py`	one sign-in's listener, its state value and its transport
`ScanNowButton`, `PostChart`, `TimerTiles`, `TimerClose`, `ZONE_COUNTER`	`src/gui/web/market_inspector.js`	the page's own elements, drawn from `ata_spm_skin`
`PaneWidthPage`	`src/gui/market_inspector.py`	the Qt page that relays its own width to its rows

The window relays that width on every splitter move, which is what keeps the button row and the settings rows inside their zone.

`src/gui/market_inspector.py` — the redraws a width change and a progress run

```
    def _size_zones(self) -> None:
    def _relay_bucket_row(self) -> None:
    def _render_ata_row(self) -> None:
    def _show_lines(self, rows: Any) -> None:
    def _show_panels(self, panels: Any) -> None:
    def _show_panel_lines(self, rows: Any) -> None:
    def _show_actions(self, actions: Any) -> None:
    def _drop_panel_box(self) -> None:
```

`src/gui/main_tabs/market_inspector_tab.py` — where the Activity Log pane is handed over

```
    def _wire_ata_activity_log(self) -> None:
```

Each of those hides a widget before it reparents it, so no parentless widget is ever shown and no window floats over the panel. The proposals pane builds its own detector context, and a scan writes one sector's votes at a time.

```
MainWindow._build_topology_proposals    the detector context
SectorBoard.load_run                    the bucket one finished run replaces
ata_spm._vote_one                       one market's vote on one timeframe
market_inspector_topologies.pane_view    the pane as values
registry._rollup                        the one calendar-bucket rollup
```

Which model answers, by host

Two hosts draw this screen, and each reads a model of its own.

host	the model it reads	wired by
the Qt window	the tab's own model	`set_exchange_source`
the Electron shell	`market_inspector_surface.pane_model`, the one the bridge keeps between calls	nothing

`src/gui/main_tabs/market_inspector_tab.py`, in `MarketInspectorTabMixin._build_market_inspector_tab`, is the only caller of `set_exchange_source`. `MarketInspectorReactTab` subclasses `MarketInspectorTab` without overriding that method, so the one call wires both Qt variants.

The React variant reaches no bridge method. `src/gui/react_market_inspector_tab.py`, in `HOST_SCRIPT`, replaces the page's own `acervatorMarketInspectorAction` with a console line carrying `SCREEN_PREFIX`, `MarketInspectorPage.javaScriptConsoleMessage` reads that line, and `run_action` calls the inherited `_start_fetch` on the tab's own getter and scheduler.

Diagram: flowchart, left to right

- Refresh on the page -> **HOST_SCRIPT console line**

- HOST_SCRIPT console line -> **javaScriptConsoleMessage**
- javaScriptConsoleMessage -> **run_action**
- run_action -> **_start_fetch, the wired getter**
- _start_fetch, the wired getter -> **_fill_signal_rows, _fill_pair_rows**
- _fill_signal_rows, _fill_pair_rows -> **push, screen_push_script**
- push, screen_push_script -> **setTab on the page**

The scan's rows come back as a push and never as a response. `_fill_signal_rows` and `_fill_pair_rows` write the tab's own model, and `push` sends `build_view_model` of that model through `screen_push_script`.

`src/gui/react_market_inspector_tab.py` — the three pushes the page is drawn by

```
def screen_push_script(model: dict) -> str:
def tile_push_script(rows: list) -> str:
def topology_push_script(model: dict, preview_at: Optional[int]) -> str:
```

Topology Proposals is a payload of its own. `TopologiesPaneHost.view_model` builds it from the pane's own `TopologiesPaneModel` and `topology_push_script` sends it, so that pane draws from neither model in the table above.

`market_inspector.state` is reached by `loadScreen` and by the page's own `acervatorMarketInspectorAction`, both of which need a host supplying `global.acervator`. The Electron shell supplies one and the Qt web view does not.

That function routes `scan-now`, `scan-all`, the zone steps, the ticker field, the class and timeframe boxes, the push parts, the credential fields and the settings. It carries no branch for `REFRESH_PART`, so a Refresh press on a host reading `global.acervator` answers null and asks the bridge for nothing.

What a Refresh publishes about the connectors

A Refresh hands the live `{exchange_id: connector}` dictionary to the scheduler, and `src/gui/main_tabs/market_inspector_surface.py`, in `MarketInspectorScreenModel.fetch_call`, holds that same dictionary on `scheduled`. The published payload carries values in its place.

`src/gui/main_tabs/market_inspector_surface.py` — what `scheduled` publishes

```
def connector_states(connectors: Any) -> list:
def scheduled_fetch(found: Any) -> list:
```

Each connector publishes one `[name, connected]` pair, sorted by name. The name is the connector's own `exchange_id`, or the key the main window filed it under where the connector answers no id, and `connected` is its `is_connected`. One scheduled fetch publishes as that pair list followed by its force flag, so a wired source draws as `[["binance", true]], false]`.

the screen reads	from
the venue name	`exchange_id`, declared abstract on `ExchangeInterface`

the screen reads	from
whether it is signed in	`is_connected`, True after `connect` and False after `disconnect`

No live object reaches the payload. `src/gui/web/market_inspector.js`, in `checkShapes`, needs `scheduled` to be a list and reads no entry inside it.

A payload the bridge cannot write

`src/core/desktop_bridge.py`, in `encode_frame`, writes one response as one line. A value JSON has no spelling for is reported in place rather than taken out on the whole frame.

`src/core/desktop_bridge.py` — the three the encoder handles

```
  def json_writable(payload: Any) -> Any:
  def unwritable_text(response: Any, exc: BaseException) -> str:
  def encode_frame(response: dict) -> bytes:
```

the frame holds	what is written
`nan`, `inf` or `-inf`	`null` in that field
a value whose type has no encoder	`<unwritable TypeName>` in that field
a circular reference	an error frame carrying the request's own id

`encode_frame` catches the `TypeError`, the `ValueError` and the `RecursionError` that `json.dumps` raises, logs the type and the message under `acervator.core.desktop_bridge`, and returns a line for each of the three. The frontend's parser has a frame to read for every one of them, and the screen answers in place of going silent.

The sentence the declared derivation cost replaces

The sentence below stands in an earlier section and no longer describes the code. The earlier text stays where it is.

Earlier sentence	What the code does now
"`src/core/encryption.py` — the derivation cost", and the `_KDF_ITERATIONS` line under it	The cost is declared as `KDF_ITERATIONS`, a public name carrying the OWASP figure in its own docstring. The figure is unchanged at 600,000 rounds, and a search for `_KDF_ITERATIONS` finds nothing.

The narrative for the whole screen is on the tabs page. The tab index is on the subsystem README.

Bot Swarm Tab

Reference. The capital reinforcement network: the nodes, the wires between them, and the accounting that lands routed profit.

What builds it

`BotSwarmTabMixin._build_bot_swarm_tab` in `src/gui/main_tabs/bot_swarm_tab.py` constructs the visualizer and adds it to the tab row. The tab holds three sub-tabs.

Sub-tab	State
Bot Swarm	The live fleet
Simulator Swarm	Rows the Simulator registers while a run plays
Paper Swarm	Chrome only — see paper-trader.md

The builder asks `variant_surface` for the tab class under the screen name `BOT_SWARM`. A build stamped for Qt gets the old widget. Every other build gets the React tab, which holds the whole screen in one web view. React is the running choice.

`src/gui/main_tabs/bot_swarm_tab.py` — `BotSwarmTabMixin._build_bot_swarm_tab`

```
from ..variant_surface import BOT_SWARM, surface_class

built = surface_class(BOT_SWARM)()
```

The nodes

Each bot draws as one locust card, painted with QPainter. A theme module supplies the palette and a particle module the drifting motes.

`src/gui/visualizer/bot_node.py` — `BotNodeWidget`

```
class BotNodeWidget(QWidget):
    """Paint one bot as a locust, fed by set_bot_data and animate.

    paintEvent draws the wings, abdomen, thorax and head, then the
    symbol, the P/L and the bot_id as text.
    """
```

In the React build the card is drawn by `bot_node.js` from the payload `bot_node_surface` publishes. The tab page carries that module beside the wire canvas, the quick routing matrix and the theme palettes.

`src/gui/react_bot_swarm_tab.py` — `TAB_SCRIPT_ASSETS`

```
TAB_SCRIPT_ASSETS: tuple[str, ...] = (
    "vendor/react.production.min.js",
    "vendor/react-dom.production.min.js",
    "design_tokens.js",
    "shared_widgets.js",
    "header_strip.js",
    "visualizer_themes.js",
    "bot_node.js",
    "wire_canvas.js",
    "quick_routing.js",
    "bot_visualizer.js",
)
```

The grid of locust cards is the whole screen. The wire overlay covers it, because the grid is the only surface whose coordinates a wire can be drawn against.

The live rows

A strip above the locust grid holds one dense row per live bot: the id, the pair, the mode, the timeframe, the capital, the state, the price, the profit and the trade count. The strip is hidden while the fleet is empty. The same fleet load that draws the locusts fills it.

`src/gui/bot_visualizer.py` — `BotVisualizationTab._sync_live_rows`

```
def _sync_live_rows(self, bot_statuses: list[dict]) -> None:
    """Hold one ``_live_bot_rows`` row per bot in one fleet load.

    A row is rebuilt when the identifier mask changes the id it
    shows, and a bot no longer in the load loses its row.
    """
```

The React build fills the same rows through `BotVisualizerModel.sync_live_rows` and draws them in `LiveRows`.

`src/gui/main_tabs/bot_visualizer_surface.py` — `BotVisualizerModel.sync_live_rows`

```
def sync_live_rows(self, bot_statuses: list) -> None:
    """Hold one ``self.live`` row per bot in one fleet load.

    A row is rebuilt when ``masked`` changes the id it shows, and a
    bot no longer in the load loses its row.
    """
```

A stored reading that is text, a bool, not finite or too large to hold reads zero in a money or count column, so no stored value stops a row drawing. The identifier privacy dot masks the id column in both builds.

Drawing a wire

Drag from one node to another. Wires that arrive on the bus draw through the same handler, which is the path the startup restore and the topology adopt both take, so one subscription covers every source.

`src/gui/bot_visualizer.py` — `_on_external_wire_created`

```
def _on_external_wire_created(self, event) -> None:
    """Add the ``wire.created`` event's wire to ``_wires`` when new.

    A repeated pair updates its pct and adds no second entry.
    """
```

The quick routing matrix is the bulk alternative: tick source bots, tick destination bots, and every pair lands at once. Before that runs, the handler refuses four things and says which.

`src/gui/visualizer/quick_routing.py` — `QuickRoutingMatrix._on_connect_clicked`

```
    if not (0.0 <= pct <= 100.0):
        self._reject(f"Rate {pct} is outside 0–100%.")
        return
    if not sources:
        self._reject("No SOURCE bots are checked.")
        return
    if not dests:
        self._reject("No DESTINATION bots are checked.")
        return
    if pct <= 0:
        self._reject(
            "Rate is 0% — that would create wires that route " "nothing."
        )
        return
```

`_confirm_mass` puts a count and a description in front of the operator before a bulk connect or disconnect runs.

The flow along a wire

One `QTimer` on the Bot Swarm tab drives the whole animation. `BotVisualizationTab._animate` in `src/gui/bot_visualizer.py` runs on every tick: it moves each locust card on, adds the elapsed time to every wire's phase, and asks the wire overlay to repaint. `_WireCanvas.paintEvent` in `src/gui/visualizer/wire_canvas.py` draws the frame, and the travelling dot sits at `pulse_percent(phase)` along the wire.

The timer's interval is `FRAME_INTERVAL_MS` in `src/gui/main_tabs/bot_visualizer_surface.py`, worked out rather than written down: the largest whole number of Windows scheduler ticks that fits inside one frame at `TARGET_FRAMES_PER_S`. A wait is rounded up to the next `SCHEDULER_TICK_MS`, so asking for the 33 ms that `1000 / 30` reads as delivers 46.875 ms — 21 frames a second, not 30. Two ticks is 31.25 ms, so the timer asks for 31 ms and the flow runs at 30 frames a second.

Every frame reports itself. `_WireCanvas.paintEvent` hands its start time and its own duration to `FrameCadence`, and once a `CADENCE_WINDOW_S` window is folded `BotVisualizationTab._report_frame_cadence` emits `swarm.11.003.invariant.frame_cadence`. The record carries the window's frame count, the mean, median, 95th and worst gap between paints, the median and worst paint duration, and how many gaps ran past `FRAME_BUDGET_MS`. A stutter is one long gap among short ones, so `FrameCadence.reading` reports the median and the 95th apart — a mean alone hides it.

The React Bot Swarm page draws no flow. Its `wire-canvas` element is `display: none`, `WireOverlay` in `src/gui/web/bot_visualizer.js` redraws only when the wire list, the opacity or the theme changes, and no caller on the page sends the `animate` action that `apply_action` in `src/gui/main_tabs/bot_visualizer_surface.py` provides. Read on the running page with 38 bots and 33 wires loaded, it calls `requestAnimationFrame` zero times and `setInterval` zero times.

Routing the profit

`SmartWireManager` in `src/trading/smart_wire.py` owns the topology and one ledger per bot.

Method	What it does
`register_wire`	Adds an edge carrying its own percentage
`unregister_wire`	Drops one edge
`get_outgoing_wires`	Answers one bot's targets and percentages, which the bot reads on each sale to move that percentage of the sale's surplus (`src/trading/scrumming/wire_routing.py, in WireRoutingMixin._route_scrum_proceeds_via_wires`)
`book_transfer`	Books one transfer onto both ledgers — the two wire totals and the target's provenance line for the funder
`export_wires`, `import_wires`	Carry the topology through a save and a restart
`export_ledgers`, `import_ledgers`	Carry the provenance through the same

Now: a sale and a detonation move a share along a wire, and no fold does. A wire moves dollars only between two bots on one exchange and one currency (`src/trading/scrumming/wire_routing.py, in WireRoutingMixin._shares_wallet`).

The percent a wire is allowed to carry

One rule decides whether a stored percent is a number, and every reader of the topology reads it through that rule, so a percent the Bot Swarm tab draws as an em dash can never route a share.

`src/trading/smart_wire.py` — `routable_pct` and `pct_kind_refusal`

```
  def routable_pct(value) -> Optional[float]:
      """One stored wire percent as a float, or None when no reading admits it."""
```

A bool is refused by name. `float(True)` answers 1.0, so a saved `true` would otherwise stand as a one percent wire nobody configured. `register_wire` and `import_wires` refuse that kind before they store anything, `src/trading/scrumming/wire_routing.py, in WireRoutingMixin._route_scrum_proceeds_via_wires` routes a share only from a percent those two stored, and `export_wires` drops such a row from the save and logs which route is gone. `PaperWireManager` and `SimWireManager` read the same two functions, so the three engines admit and refuse the same values.

Three readings on the per-bot ledger answer where a bot's capital came from and how much of it has aged enough to move on.

Reading on `BotLedger`	Answers
`predominant_source`	Which bot sent the most of what this one holds
`mature_profit_total`	Profit above the bot's maturity threshold
`mature_profit_available`	The mature figure less what a saved ledger recorded as allocated

Maturity is a growth threshold, not an age and not a share. A bot's capital is mature once it is worth more than three times what it started with, and the mature figure is then the whole profit above that starting capital. Below the threshold the figure is zero.

`src/trading/smart_wire.py` — `mature_profit_usd`

```
MATURE_GROWTH_PCT: float = 200.0

if value < basis * (1.0 + MATURE_GROWTH_PCT / 100.0):
    return 0.0
return value - basis
```

One function decides maturity everywhere. The per-bot ledger reads it against the seed capital, and the header strip reads it against the exchange cost basis of each bot's holdings, so the two readings cannot disagree about what mature means.

`src/trading/smart_wire.py` — `BotLedger.mature_profit_total`

```
return mature_profit_usd(
    self.starting_balance, self.starting_balance + self.total_profit
)
```

What the provenance breakdown counts

The breakdown is one line per funder, biggest amount first. A line is the running total that funder has supplied to this bot: the seed the bot was created with, plus every wire transfer that funder has sent it since. The seed sits under the key `SEED` when no bot funded the creation, and under the funding bot's own id when one did.

One method writes the map. Every live transfer books through `src/trading/smart_wire.py`, in `book_transfer`, which moves three figures together — the source ledger's outgoing total, the target ledger's incoming total, and the target's breakdown line for the source. Both live transfer paths call it: `src/trading/scrumming/execution.py`, in `_route_unfoldable_proceeds` on a detonation and a Shadow exit, and `src/trading/scrumming/wire_routing.py`, in `_route_scrum_proceeds_via_wires` on a scrum and a manual scrum. A transfer cannot reach the totals without reaching the breakdown.

Now: a scrum or a manual scrum books each wire's share of its surplus alone, a detonation books each wire's share of its proceeds, and a fold books no transfer (`src/trading/scrumming/wire_routing.py, in _route_scrum_proceeds_via_wires`).

`src/trading/smart_wire.py` — `book_transfer`

```
def book_transfer(self, source_id: str, target_id: str, usd: float) -> bool:
    """Book one wire transfer onto the two ledgers it moves money between."""
```

An amount no reading admits books nothing and answers `False`, so the totals and the breakdown a ledger already holds stay readable. A breakdown line already unreadable keeps the figure it has, because adding to it would turn a line the tab draws as an em dash into one it draws as money.

The predominant funder row reads the same map through `src/trading/smart_wire.py`, in `BotLedger.predominant_source`, which answers the largest line other than `SEED`. A bot no wire has funded has no such line, so the row reads `— (SEED-funded only)`.

Paper and the Simulator book the same three figures in their own managers, `src/paper/paper_wire.py`, in `PaperWireManager._book`, and `src/simulator/sim_wire.py`, in `SimWireManager._book`. The

same wiring reads the same breakdown on all three screens.

What the group draws now

The Provenance & Mature-Profit Spawn State group draws three rows: the starting balance, the predominant funder and the provenance breakdown. It drew three more — the mature profit total, the mature profit allocated to spawns and the mature profit available — and those three came off every screen that shows the group.

The heading is written once, and the three screens in the table below read it from there, so changing the heading is one edit. A single ampersand in a box title is read as a keyboard shortcut mark, so the doubled form is what the three boxes are given and one ampersand is what each screen draws. The Electron panel reads the same words out of the shared model.

`src/gui/main_tabs/bot_swarm_tab_surface.py` — the heading and the three rows

```
PROVENANCE_GROUP_TITLE = "Provenance & Mature-Profit Spawn State"
PROVENANCE_GROUP_TITLE_QT = PROVENANCE_GROUP_TITLE.replace("&", "&&")

STARTING_ROW_LABEL = "Starting balance (seed):"
PREDOMINANT_ROW_LABEL = "Predominant funder (PPS):"
PROVENANCE_ROW_LABEL = "Provenance breakdown:"
```

Each of the three could only ever read `$0.0000`. Nothing in the program works out the allocated figure; three places write it and each only copies it back out of a saved row, so a zero saved comes back as a zero. The total and the available figure are worked out from a ledger's `total_profit`, and the only method that adds to `total_profit` is called from one place that nothing calls. A money figure that can only read zero tells the operator nothing, because he cannot tell a true zero from a reading that was never fed.

The sum itself is sound and stays where it is. `mature_profit_usd`, `BotLedger.mature_profit_total` and `BotLedger.mature_profit_available` are unchanged, and the header strip still reads maturity against the exchange cost basis of each bot's holdings. Only the three rows on the tab came off.

Screen	Where
Live	`src/gui/live_settings/bot_swarm_tab.py`
Paper	`src/gui/paper/paper_bot_swarm_tab.py`
Simulator	`src/gui/simulator/sim_bot_swarm_tab.py`
The shared model the panel reads	`src/gui/main_tabs/bot_swarm_tab_surface.py`

The sentences this overtakes, from the start of this section and from the block above:

> *The Provenance & Mature-Profit Spawn State group draws three rows: the starting balance, the predominant funder and the provenance breakdown.*
>
> *A single ampersand in a box title is read as a keyboard shortcut mark, so the doubled form is what the three boxes are given and one ampersand is what each screen draws.*
>
> `PROVENANCE_GROUP_TITLE = "Provenance & Mature-Profit Spawn State"`

The true sentences. The heading is `Provenance`. It names the three rows the group holds and names nothing else, because the clause that named mature-profit spawn state named three readings that came off the box.

`src/gui/main_tabs/bot_swarm_tab_surface.py` — the heading and the three rows

```
PROVENANCE_GROUP_TITLE = "Provenance"
PROVENANCE_GROUP_TITLE_QT = PROVENANCE_GROUP_TITLE.replace("&", "&&")

STARTING_ROW_LABEL = "Starting balance (seed):"
PREDOMINANT_ROW_LABEL = "Predominant funder (PPS):"
PROVENANCE_ROW_LABEL = "Provenance breakdown:"
```

The heading carries no ampersand, so the doubled form and the plain form are the same word and every screen that shows the group draws `Provenance`. The three Qt screens read `PROVENANCE_GROUP_TITLE_QT` and the two panel scripts read the title out of the payload, so none of them spells the words and the three builds cannot disagree about them.

The three rows are unchanged. Each keeps the label above, the breakdown row keeps `PROVENANCE_STYLE` and its word wrap, the funder row draws `— (SEED-funded only)` for a bot that only ever held its seed, and an em dash for a ledger that refuses to name a funder.

The summary box at the top of the tab is headed `Swarm Connections & Capital Flow`, and its words are declared in the same place as the provenance heading.

`src/gui/main_tabs/bot_swarm_tab_surface.py` — the two headings and the forms Qt is given

```
SUMMARY_GROUP_TITLE = "Swarm Connections & Capital Flow"
PROVENANCE_GROUP_TITLE = "Provenance"
# A QGroupBox title reads a single ampersand as a mnemonic mark, so Qt takes two.
SUMMARY_GROUP_TITLE_QT = SUMMARY_GROUP_TITLE.replace("&", "&&")
PROVENANCE_GROUP_TITLE_QT = PROVENANCE_GROUP_TITLE.replace("&", "&&")
```

The three Qt screens read `SUMMARY_GROUP_TITLE_QT`, and each panel script reads its title out of the payload, so no drawing surface spells either heading. The derived form is `Swarm Connections && Capital Flow`, so each box draws the same words it has always drawn.

The sentence this overtakes. The page carried no sentence naming where the summary heading is spelled. Changing either heading is one edit in one file, and the doubled form stays correct for whatever wording the declaration holds.

One function bounds what a bot may send away. It reads the target balance, the band edges, the ammunition the next fold needs and the cash on hand, and returns the exportable share as a percentage. A bot never exports the capital it is holding to buy the dip it is waiting for.

`src/trading/smart_wire.py` — `compute_safe_outflow_pct`

```
def compute_safe_outflow_pct(
    scrum_profit_usd: float,
    target_balance_usd: float,
    current_price: float,
    band_lower: float,
    band_upper: float,
    next_fold_ammo_usd: float,
    current_cash_usd: float,
    compound_growth_pct: float,
    retained_this_cycle_usd: float,
) -> float:
```

Landing the profit

The USD arrives at the destination and spreads evenly across the open fold tranches. Where no tranche stands, it parks instead, and the parked pool drains into the next tranche that opens. Parked credits are routed income, never a tranche.

`src/trading/scrumming/wire_routing.py` — `apply_wire_income`

```
def apply_wire_income(self, usd: float, source: str, ref: str = "") -> dict:
    """Apply incoming Smart Wire USD to this bot's fold queue.
```

Each tranche keeps its wire-credit provenance, held by three methods.

Method	What it does
`_add_wire_credits`	Appends one detail entry per arrival
`_compact_wire_credits`	Bounds how many entries a tranche holds
`_roll_wire_credit_overflow`	Folds everything past the cap into a rolled total that keeps the sums exact

The tranche book

`FoldTrancheAccountingMixin` in `src/trading/scrumming/fold_tranches.py` owns the queued fold tranches, the discharge order the buy path reads, and the lifetime counters. It places no orders.

Four methods carry the fold through, in this order.

Method	Step
`_fold_eligible_tranches`	Selects what may fold at the current price
`_fold_discharge_order`	Fixes the order they discharge in
`_plan_fold_consumption`	Builds the plan
`_settle_fold_plan`	Books it

Merge, despawn, clear

Three functions collapse or remove a tranche. No other code path does.

Merge. `_top_up_remnant_fold_tranches` folds a new sell's USD into part-spent tranches instead of opening more records, and `_bound_new_fold_tranches` merges one sell's fresh slice back to a single record when the fill arrived as many lots. Both reduce the record count and move USD; neither loses any.

A fourth function combines a standing pair within a tenth of a percent

The sentence at the head of this subsection is overtaken. It is not edited and not removed. It read:

> *Three functions collapse or remove a tranche. No other code path does.*

A fourth now does. `src/trading/scrumming/fold_tranches.py`, in `_combine_fold_tranches_within_distance`, combines two standing fold tranches whose reference prices sit inside a tenth of a percent of each other. It runs each time a sale queues a tranche, after the top-up.

`src/trading/scrumming/sizing.py`, in `merge_fold_tranches_within_distance`, holds the arithmetic, and `src/trading/stack_math.py`, in `within_merge_distance`, holds the figure. The Stack ladder read that same declaration already.

The survivor keeps the lower reference price, which is the stricter fold threshold, sums the cash and the units, takes the units-weighted original cost and names both events. It moves USD and loses none, as the two merges above it do, and it brings the opened counter down by what it combined so `created - closed - discarded == standing` still holds.

Driven on a hand-built pair in one run: refs 0.05 percent apart combined, two records into one carrying $100.0000 and 1.000000 units at a cost basis of 85.00000000; refs 0.5 percent apart stayed as two.

A standing Stack rung also combines with a new rung inside the same distance, through `src/trading/stack_math.py`, in `merge_into_standing_rungs`. A rung holding a venue order is never a candidate, since combining it would need that order cancelled.

Despawn. Aged tranches are removed outright. The record leaves the book, the queue total recomputes from what is left, and the count lands in the discarded ledger rather than the closed one, because a closed tranche is one that folded. That split holds one invariant true.

`src/trading/scrumming/fold_tranches.py` — `_despawn_aged_tranches`

```
def _despawn_aged_tranches(self, now: Optional[float] = None) -> dict:
    """Remove tranches older than ``config.tranche_despawn_days``.

    Removing a fold tranche recomputes ``_fold_queue_usd``, the
    derived aggregate behind the tick's ``_fold_queue_usd == 0``
    short-circuit and the panel's "Parked USD", and counts the
    record in ``_tranches_discarded_lifetime``, never in
    ``_tranches_closed_lifetime``, because a closed tranche is one
    that FOLDED. That split keeps
    ``created - closed - discarded == standing`` true.
```

An aged stack tranche still holding a resting limit order is kept and counted separately, since removing it would leave that order on the book with nothing tracking it. The threshold reads the same shared helper the settings spinbox reads, so the sweep and the control cannot disagree about what a stored value means.

The sweep's report now names its counts the way the preview beside it already did, so the two agree on the word as well as on the number. The key set below replaces the one in the block above, and the line the

operator reads says removed where it used to say delisted. Nothing outside the sweep read the old names.

```
  report = {
      "threshold_days": _days,
      "fold_removed": 0,
      "stack_removed": 0,
      "stack_kept_live_order": 0,
      "ageless_kept": 0,
      "usd_removed": 0.0,
  }
```

Two removers touch the stack ledger, not one. The age-driven sweep is one and the operator's own clear is the other, and both move the same discarded counter.

```
  def clear_stack_tranches(self, reason: str = "operator") -> dict:
      """Discard this bot's standing Stack tranches, trading nothing."""
```

Clear. `clear_fold_tranches` discards every queued tranche and trades nothing. Holdings, the main lots, the target balance and the parked wire credits all survive it, and the report warns when parked credits remain.

One admission rule for every lifetime counter

The invariant above holds only while every term of it is read the same way. One rule reads them all: `as_whole_count`, in `src/trading/container/config.py`. It admits a whole number and nothing else. A stored flag, a text number, a not-a-number, an endless number, a fraction and a whole number too large for a float each answer "no usable count", and none of them raises.

Seven counters read it: the four fold counters, the scrum-sells counter and the two stack counters.

Reader	What a refused counter does there
`src/trading/scrumming/state_io.py`, in `admitted_counter`	Lands zero and names the bot, the field and the stored type in the log
`src/gui/live_settings/fold_tokens.py`, in `fold_lifetime_counter`	Returns no count and logs the same three facts
`src/gui/live_settings/stack_tranches_tab.py`, in `_stack_lifetime_counter`	The same, for the two stack counters
`src/gui/main_tabs/fold_tranches_tab_surface.py`, in `guarded_count`	The same reading, for the second build
`src/gui/main_tabs/stack_tranches_tab_surface.py`, in `lifetime_counter`	The same reading, for the second build

The Paper and Simulator panels are forks of the Live ones and read the same rule. Both fold forks call `fold_lifetime_counter` in `src/gui/live_settings/fold_tokens.py`, the one module all three fold panels share. Each stack fork carries `_stack_lifetime_counter`, matching the Live panel's own, so no screen reaches across from the Simulator into Live to read a count.

Panel	Reader
`src/gui/paper/paper_fold_tranches_tab.py`	`fold_lifetime_counter`
`src/gui/simulator/sim_fold_tranches_tab.py`	`fold_lifetime_counter`

Panel	Reader
src/gui/paper/paper_stack_tranches_tab.py	_stack_lifetime_counter
src/gui/simulator/sim_stack_tranches_tab.py	_stack_lifetime_counter

src/gui/main_tabs/fold_tokens_surface.py holds a third copy of compose_cycle_close_ratio, and no counter reaches it. That module reads no bot and no stored field: src/core/desktop_bridge.py registers its view_model as the handler for the fold_tokens.state method, and src/gui/web/fold_tokens.js draws that payload's colours, sort orders, column tooltips, row heights and step names. Its ratio copy answers only a request that names the step and carries its own three numbers.

admitted_counter sits on both halves of the save path, so a counter the panel will not print is also a counter the file will not store. A stored flag used to be written back as the count one, which made the fault permanent; it is now written back as zero, with the field named in the log.

On screen a refused counter draws an em dash and never a number. The Cycle close ratio row and the Fill ratio row each say a counter is unreadable rather than work out a share from the terms they can still read. The Clear Lifetime Counters button stays live and its confirmation prints the dash, so an unreadable count is never the reason an operator cannot zero the ledger.

Where the fleet comes from

One file is the only source of the bots, and of their wires, ledgers and tranches with them. StateManager in src/core/state_manager.py reads and writes it. Two restore methods rebuild the fleet and its topology from it, and one save method writes it back. A restored bot connects to nothing and places no order until the operator starts it.

src/trading/container/restore.py — StateRestoreMixin.restore_bots_from_state

```
def restore_bots_from_state(self, state: dict) -> list[str]:
    """Recreate every persisted bot in IDLE state; return the ids restored.

    Connects to no exchange and places no order. Each restored bot carries
    a ``_restored`` flag and stays IDLE until the operator starts it, which
    is what triggers the exchange sync before any trading.
    """
```

restore_smart_wires_from_state in the same mixin rebuilds the topology. delete_bot is the only path that removes a record.

Capital claims

One reservation per bot stands against a wallet. A total above the wallet figure is refused whole, and the free figure reports what is left. The claims settle against the venue, because the exchange is the authority.

The dollar registry's file and its reservation method are gone. The claims table lives in one registry now, and src/trading/capital_reservation.py, in CapitalReservationRegistry.reserve takes a claim. It stores a reservation record and returns its token rather than a granted figure, and src/trading/capital_reservation.py, in CapitalReservationRegistry.effective_available answers what is left. The block below records

the older contract.

```
  """Create or replace ``bot_id``'s ``Reservation`` and return
  ``(granted, reason, granted_usd)``.

  A total above the ``_wallet_usd`` figure is refused whole with
  ``granted_usd`` 0, and a ``_wallet_usd`` of None grants unchecked.
  """
```

The dollar registry's table reached the frontend through its own bridge method, named in the table at the foot of this file. A Qt panel beside it was declared and no screen built it, and the section below records that both are gone.

The dollar registry and its table are gone

The paragraph above describes a subsystem the tree no longer holds. Nothing imported the dollar registry, so nothing built one, and the table it fed could draw no row.

Four modules are removed: the registry in `capital_registry`, the Qt table in `capital_registry_panel`, the view model in `capital_registry_surface`, and the renderer page in `src/gui/web/capital_registry.js`.

The bridge no longer offers the method. `build_registry` in `src/core/desktop_bridge.py` returns 72 methods, and none of them starts with `capital_registry`. The renderer manifest lists 68 modules, and every one has a file on disk.

The slot that held the dollar registry is removed with it. `BotManager` carried a `_capital_registry` attribute, a setter with no caller, and five methods that read it. Every method they called on it — `request_reservation`, `release_reservation`, `grow_reservation`, `get_reservations` and `reconcile_with_exchange` — has no definition anywhere in the tree, so filling the slot would have raised on the first call. The attribute, the setter and the five readers are gone.

Claims in asset units are a different mechanism and they stay. `CapitalReservationRegistry` in `src/trading/capital_reservation.py` holds one claim per bot in target-asset units. The saved claim state held 38 of them on 2026-09-13. A bot resolves that registry itself, through `CapitalReservationMixin._crr`, and claims on its first tick.

Bridge

Seven methods serve this screen, and the renderer modules carry the matching names.

Bridge method	Serves
`bot_visualizer.state`	The whole tab
`bot_swarm_tab.state`	The three sub-tabs and the header row
`bot_node.state`	One locust card
`wire_canvas.state`	The wire overlay on the grid
`quick_routing.state`	The quick routing matrix
`capital_registry.rows`	The reservation table
`fold_tranches_tab.state`	The tranche book

The row for `capital_registry.rows` names a method the bridge no longer offers. Six methods serve this screen now. `build_registry` in `src/core/desktop_bridge.py` is the table that decides, and it registers the other six.

2026-09-07 15:58 - no issue recorded - ten single-word tabs, each on its own ground

The Bot Swarm is a proprietary capital reinforcement network system that allows profit to be dynamically and strategically routed between positions with the primary intention of this being to accelerate accumulation curves. Smart Wires are dragged between active bots or the quick connection matrix can be used to route multiple streams to different destinations. Each wire can carry a different percentage amount of profit. Profit sent over Smart Wires are registered at the destination as Wire Credits and these are then distributed into standing Fold Tranches which allow them to have a Surplus that will be spent to increase the Target Balance up to the Maximum Growth Per Cycle. Yes, that is probably a mouth full but hopefully the settings and names are, for the most part, self-explanatory.

The tab is now called Swarm. It sits sixth on the bar, on the gold ground with red text.

Each bot draws as one locust card on the grid, and wires drag between nodes.

`src/gui/visualizer/bot_node.py` — `BotNodeWidget`

```
class BotNodeWidget(QWidget):
    """Paint one bot as a locust, fed by set_bot_data and animate.

    paintEvent draws the wings, abdomen, thorax and head, then the
    symbol, the P/L and the bot_id as text.
    """
```

`SmartWireManager` in `src/trading/smart_wire.py` owns the topology and one ledger per bot. One function bounds what a bot may export, reading the target balance, the band edges and the ammunition the next fold needs, which stops a bot sending away the capital it holds for its own dip.

`src/trading/smart_wire.py` — `compute_safe_outflow_pct`

```
def compute_safe_outflow_pct(
    scrum_profit_usd: float,
    target_balance_usd: float,
    current_price: float,
    band_lower: float,
    band_upper: float,
    next_fold_ammo_usd: float,
    current_cash_usd: float,
    compound_growth_pct: float,
    retained_this_cycle_usd: float,
) -> float:
```

At the far end, arriving USD spreads across the open fold tranches, or parks as a pending wire credit while no tranche stands.

`src/trading/scrumming/wire_routing.py` — `apply_wire_income`

```
  def apply_wire_income(self, usd: float, source: str, ref: str = "") -> dict:
      """Apply incoming Smart Wire USD to this bot's fold queue.
```

Three functions collapse or remove a tranche and no others: merge, despawn and clear. Despawn removes an aged record outright rather than holding it in a delisted state, and the count lands in the discarded ledger rather than the closed one.

`src/trading/scrumming/fold_tranches.py` — `_despawn_aged_tranches`

```
  _days = self._despawn_threshold_days()
  report = {
      "threshold_days": _days,
      "fold_delisted": 0,
      "stack_delisted": 0,
      "stack_kept_live_order": 0,
      "ageless_kept": 0,
      "usd_delisted": 0.0,
  }
```

The fleet, and its wires, ledgers and tranches with it, comes from one state file alone. A restored bot connects to nothing and places no order until the operator starts it.

`src/trading/container/restore.py` — `StateRestoreMixin.restore_bots_from_state`

```
  def restore_bots_from_state(self, state: dict) -> list[str]:
      """Recreate every persisted bot in IDLE state; return the ids restored.
```

Three sub-tabs open the screen: Bot Swarm, Simulator Swarm and Paper Swarm. The header row holds the drag hint, an identifier privacy dot that masks the bot hashes and the symbol labels together, the Privacy Mode button, and three controls. Exchange filters both the swarm and the quick routing scope. Theme lists the four palettes. The Wires slider sets wire opacity from 0 to 100 %.

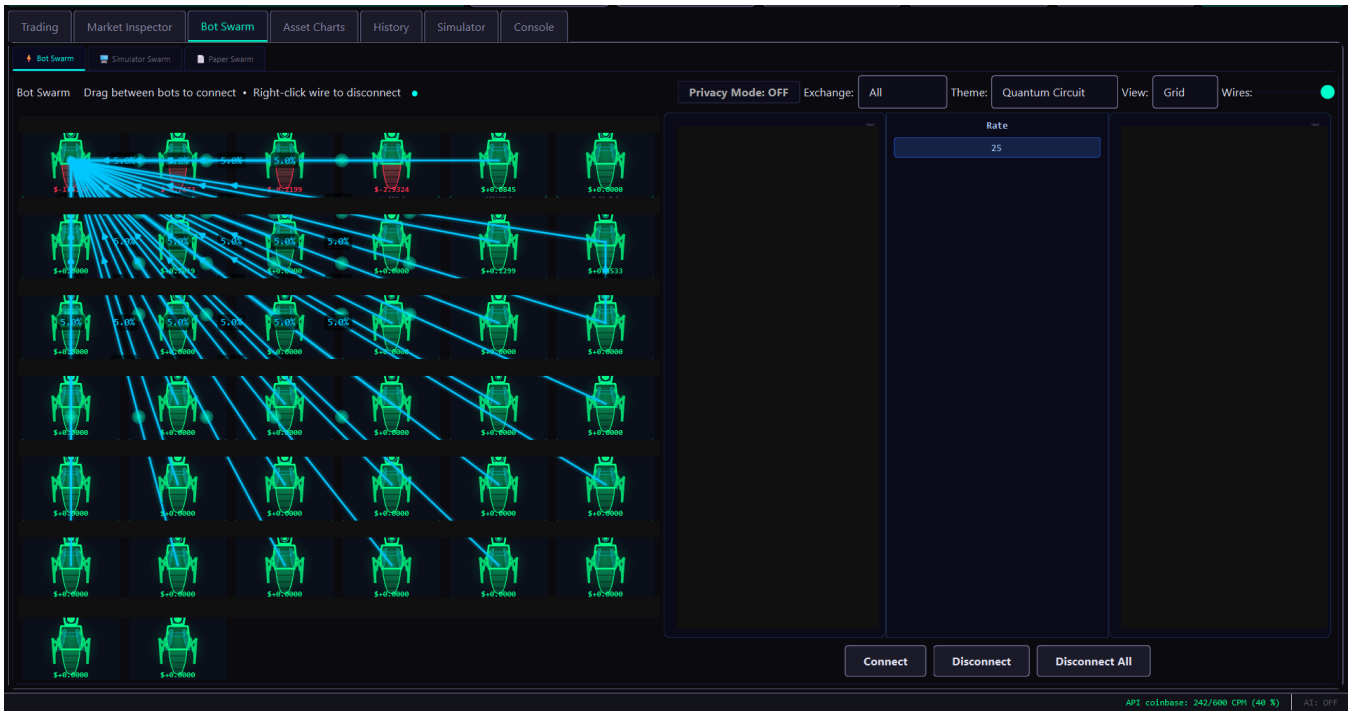
The quick routing matrix fills the right half in three zones: the source list, the Rate field between them, and the destination list. Every checked source wires to every checked destination at that rate, and four conditions are refused out loud before anything is created.

`src/gui/visualizer/quick_routing.py` — `QuickRoutingMatrix._on_connect_clicked`

```
if not (0.0 <= pct <= 100.0):
    self._reject(f"Rate {pct} is outside 0–100%.")
    return
if not sources:
    self._reject("No SOURCE bots are checked.")
    return
if not dests:
    self._reject("No DESTINATION bots are checked.")
    return
if pct <= 0:
    self._reject(
        "Rate is 0% — that would create wires that route " "nothing."
    )
    return
```

A confirmation puts the count in front of the operator before Connect, Disconnect or Disconnect All runs.

(Grid View)



Each card carries the symbol above, the realised profit below it, and the first eight characters of the bot id at the foot. The profit colour is the theme's success colour at zero and above and the error colour below, and the abdomen gradient takes that same colour with its alpha scaled by the size of the figure.

Privacy Mode is off in the figure, so the symbol and the id both draw plain. One field id covers all three readings on a card, the symbol, the short bot id and the tooltip, so a card is never half masked.

`src/gui/visualizer/bot_node.py` — the symbol, drawn through the one mask

```
p.drawText(
    QRectF(0, 2, w, 11),
    Qt.AlignCenter,
    _mask_or(symbol, "bot_swarm.identifiers"),
)
```

Theme names the palette the canvas paints in. The module declares four, and the figure sits on Quantum Circuit. The palette decides the locust body, the success and error colours the profit line takes, and the four corner brackets around each card, which draw in the second accent at low alpha.

`src/gui/visualizer/themes.py` — the palette the figure is on

```
"quantum": {
    "name": "Quantum Circuit",
    "bg": QColor(10, 15, 25),
    "accent": QColor(0, 200, 255),
    "accent2": QColor(0, 255, 200),
    "success": QColor(0, 255, 136),
    "warning": QColor(255, 180, 0),
    "error": QColor(255, 50, 80),
```

The wires cover the grid, because the grid is the only surface whose coordinates a wire can be drawn against. Each wire carries its own percentage, drawn beside it. Drag between two nodes to create a wire, and right-click a wire to remove it.

A wire's percentage sits in a rounded dark badge at the midpoint of the curve, so a wire crossing a card still reads. The Wires slider at the top right sets the opacity of the whole overlay rather than of any one wire.

`src/gui/visualizer/wire_canvas.py` — the badge under the label

```
label = f"{pct}%"
font = QFont("Consolas", 8, QFont.Bold)
p.setFont(font)
fm = p.fontMetrics()
tw = fm.horizontalAdvance(label) + 8
badge = QRectF(label_pt.x() - tw / 2, label_pt.y() - 9, tw, 18)
```

React draws this tab. The choice is made in one place, under the screen name `BOT_SWARM`. A build stamped for Qt gets the old widget. Every other build gets the web one, which holds the tab's page in a single view and answers the calls the main window already made.

`src/gui/main_tabs/bot_swarm_tab.py` — `BotSwarmTabMixin._build_bot_swarm_tab`

```
    def _build_bot_swarm_tab(self) -> None:
        """Build the Bot Swarm tab and add it to the main tab widget."""
        from ..variant_surface import BOT_SWARM, surface_class

        try:
            built = surface_class(BOT_SWARM)()
```

Every header control reports back to Python. The privacy dot, the Privacy Mode button, the Exchange and Theme pickers and the Wires slider each send one action. The screen then redraws from the answer.

`src/gui/react_bot_swarm_tab.py` — `BotSwarmReactTab.take`

```
        def take(self, params: dict) -> dict:
            """Apply one action to the model, push the payload and return it."""
            answer = surface.apply_action(self._state, params)
            self._last_model = answer
            if self._page_ready:
                self._run(push_script(answer))
            return answer
```

A drag between two bots opens the rate box. A drag between two bots already wired opens the disconnect confirmation. A drag released on empty space offers the source bot's outgoing wires. One method decides which of the three it is, so both screens ask the same question in the same words.

`src/gui/main_tabs/bot_visualizer_surface.py` — `BotVisualizerModel.finish_drag`

```
    def finish_drag(self, source_id: str, target_id: str) -> Optional[dict]:
        """Set ``panel`` to what a wire drag from ``source_id`` asks for next.

        ``WireBoard.finish_drag`` decides the outcome; this turns it into
        the rate box, the disconnect confirmation or the wire picker the
        screen shows.
        """
```

The window a bot row opens

Clicking a bot opens its own settings window. Which window opens depends on the build. The React build draws the header, the tab strip and every tab inside one embedded browser; the Qt build draws the widget tree those tabs describe. The Simulator opens the same window from its own bot rows.

`src/gui/main_window.py` — `_on_bot_clicked`

```
from .variant_surface import BOT_LIVE_SETTINGS, surface_class

_cls = surface_class(BOT_LIVE_SETTINGS)
dlg = _cls(current_bot, self._bot_manager, self)
```

The React side is a subclass, not a rewrite. It replaces only the method that builds the widgets, so an edit still travels the same route into the bot. `_mark_changed` records the field against what the bot holds now, and `_apply_changes` writes it. The window carries no second copy of that logic.

Now: each box on the React Settings tab keeps the operator's edit until Apply, and the window records it only where the Qt control could hold it, a list pick as its item (`src/gui/main_tabs/live_settings_tab_surface.py, in admitted_edit`).

`src/gui/react_bot_live_settings.py` — `BotLiveSettingsReactDialog.apply_edit`

```
field = field_for(str(asked.get("name") or ""))
if not field:
    return
self._mark_changed(field, asked.get("value"))
```

Each tab is drawn by its own renderer module into a named empty space the window leaves for it. `TAB_PLAN` pairs the tab with that space, its module and the surface that builds its payload from the bot. A tab whose surface refuses the bot draws the reason in that space rather than nothing.

`src/gui/react_bot_live_settings.py` — `TAB_PLAN`, the first two rows

```
("Status", "status-page", "acervatorLiveStatus", "live_status_tab_surface"),
(
    "Settings",
    "settings-page",
    "acervatorLiveSettingsTab",
    "live_settings_tab_surface",
),
```

A refusing tab costs one tab, never the window. On the Qt side `_add_guarded_tab` and `_install_guarded_tab` build each tab and, on a refusal, add a page naming the kind that refused and the value, so the other six tabs still open and the operator's click still reaches a window. On the React side `tab_payload` answers a fault payload for that one space, with its import and its model lookup inside the same guard, because `redraw` fills every space in one loop.

`src/gui/bot_live_settings.py` — `_add_guarded_tab`

```
try:
    page = self._wrap_scrollable(build())
except Exception as tab_exc:
```

2026-09-09 22:12 - #128 - one view, growth stages, and wires that hang

List View is gone. The header row now holds the drag hint, the identifier privacy dot, the Privacy Mode button, Exchange, Theme and the Wires slider. The View picker between Theme and Wires is removed, and the grid of locust cards is the whole left pane.

`src/gui/bot_visualizer.py` — the grid goes straight into the row

```
            inner_hbox = QHBoxLayout()
            # No gap: _quick_routing_matrix takes every pixel the grid leaves.
            inner_hbox.setSpacing(0)
            inner_hbox.addWidget(self._grid_widget)
            inner_hbox.addWidget(self._quick_routing_matrix, stretch=1)
```

The card is smaller and the grid is wider

A card asks for 88 by 78 pixels, down from 112 by 98. That is 6,864 pixels of area against 10,976, a drop of 37 per cent, and the grid fits eight columns where it fitted six. The three text rows are measured in points rather than in card units, so they survive the smaller card unchanged; the insect body scales by the same ratio as the card.

`src/gui/visualizer/bot_node.py` — the card size

```
CARD_WIDTH_PX = 88
CARD_HEIGHT_PX = 78
```

Four growth stages, keyed on realised profit

Each card sits in one of four stages. The stage comes from the bot's realised profit measured against its cost basis, both pulled from the exchange, and the 200 per cent floor is the same number the Smart Wire maturity test reads.

Stage	Realised growth	Dominant colour	Decoration
Hopper	0 to 99 %	grey and black	wing buds, two tergites
Fledgling	100 to 199 %	red and white	three tergites
Immature adult	200 to 299 %	silver	full grown, four tergites
Mature adult	300 % and over	gold	three crown spines

The stage names are the published ones. They come from the FAO Desert Locust Guidelines, part one, Biology and behaviour, by Symmons and Cressman, which names the hopper, the fledgling, the immature adult and the mature adult in that order. The module records that source beside the names.

`src/gui/visualizer/growth_stage.py` — the source and the floors

```
STAGE_SOURCE = (
    "FAO Desert Locust Guidelines 1. Biology and behaviour, Symmons and Cressman"
)

HOPPER_FLOOR_PCT = 0.0
FLEDGLING_FLOOR_PCT = 100.0
IMMATURE_FLOOR_PCT = MATURE_GROWTH_PCT
MATURE_FLOOR_PCT = 300.0
```

One function turns the two exchange figures into the percentage. It returns nothing at all when the basis is absent or not above zero, which is how a bot the venue has not answered for is kept out of a stage it has not earned.

`src/gui/visualizer/growth_stage.py` — `realised_growth_pct`

```
    if not math.isfinite(basis) or not math.isfinite(profit) or basis <= 0.0:
        return None
    return profit / basis * PCT_SCALE
```

A second function reads the stats dict the fleet load carries and refuses a bot whose exchange reading is stale. A card with no reading draws as a Hopper and prints an em dash where the percentage goes, never a zero.

`src/gui/visualizer/growth_stage.py` — `growth_pct_from_stats`

```
    if fresh <= 0.0:
        return None
    return realised_growth_pct(stats.get(BASIS_KEY), stats.get(REALISED_KEY))
```

The two figures reach the screen through the bot status snapshot, which now carries them beside the year-to-date sums.

`src/trading/bot_container.py` — `get_status`, the three added stats

```
                "realized_pnl_exchange": round(
                    float(getattr(self.stats, "realized_pnl_exchange", 0.0) or 0.0), 4
                ),
                "cost_basis_total_exchange": round(
                    float(getattr(self.stats, "cost_basis_total_exchange", 0.0) or 0.0),
                    4,
                ),
                "exchange_data_fresh_ts": float(
                    getattr(self.stats, "exchange_data_fresh_ts", 0.0) or 0.0
                ),
```

Every stage colour is a theme token

No card holds a colour of its own. Each theme carries a body tone and a trim tone for each stage, and the trim is also the colour the growth figure is printed in. The bot id line reads its colour from the same place.

`src/gui/theme_engine.py` — the stage tokens on `ThemeTokens`

```
    locust_hopper_body: str = "#4a4a52"
    locust_hopper_trim: str = "#9a9aa6"
    locust_fledgling_body: str = "#c4303f"
    locust_fledgling_trim: str = "#f2f2f7"
    locust_immature_body: str = "#9ea4ad"
    locust_immature_trim: str = "#e2e6ec"
    locust_mature_body: str = "#b8912c"
    locust_mature_trim: str = "#f0c64a"
    locust_id_text: str = "#8c8ca8"
```

Every text colour on a card was measured against every canvas ground. One hundred and sixty readings were taken, five app themes by four canvas palettes by eight text roles, and all one hundred and sixty clear the WCAG AA floor of 4.5 to 1. On the Quantum Circuit ground the symbol reads 13.35 to 1, the positive P/L 14.30 to 1, the negative P/L 5.32 to 1 and the bot id 5.87 to 1. The four growth figures read 6.89, 17.18, 15.31 and 11.76 to 1.

`src/design_system.py` — the measure used

```
def validate_contrast(fg: str, bg: str, large_text: bool = False) -> tuple[bool, float]:
```

What the card says without a hover

Four readings, in the order they are drawn. The symbol names the market. The P/L is the realised figure in dollars. The growth figure beneath it is the percentage that placed the card in its stage. The short bot id at the foot separates two bots on one market. A one-letter stage badge sits in the top right corner.

`src/gui/visualizer/bot_node.py` — the growth figure and the badge

```
            p.drawText(
                QRectF(0, h - 31, w, 9),
                Qt.AlignCenter,
                growth_text(growth_pct),
            )
```

The tooltip carries the stage in words and the same percentage, so the colour on the card is never the only place the stage is stated.

The animation repaints eight times less often

Every coroutine in this application runs on the window's own thread, so a card that repaints on every frame of the 33 millisecond timer spends that thread. A still card now repaints every fourth frame. Measured over 300 frames, both builds repaint a still card 75 times, which is 7.58 times a second. A card holding a trade ring or a speck still repaints on every frame until they fade. Across a fleet of 38 cards that is 288 repaints a second where it was 1,152.

`src/gui/visualizer/bot_node.py` — `animate`

```
            self._since_repaint += dt
            busy = bool(self._trade_pulses or self._particles)
            if not busy and self._since_repaint < REPAINT_INTERVAL_S:
                return False
            self._since_repaint = 0.0
            self.update()
            return True
```

A wire hangs as a catenary

A wire is no longer a curve bent through a midpoint. It is the curve a cable makes when it hangs between two points, which has a published equation. The wire carries 18 per cent more cable than the straight distance between the two bots, and the two wires of a bidirectional pair hang at different depths.

`src/gui/main_tabs/wire_canvas_surface.py` — `catenary_parameter`

```
    wanted_ratio = math.sqrt(length * length - drop * drop) / span
    ceiling_ratio = math.sinh(CATENARY_U_MAX) / CATENARY_U_MAX
    if wanted_ratio >= ceiling_ratio:
        return None
```

Driven on a 200 pixel span, the sampled curve measures 1.17972 times its chord against the 1.18 asked for, sags 55.03 pixels, and lands exactly on both bots. A wire between two bots in the same column has no span, and falls back to the straight line between them.

`src/gui/main_tabs/wire_canvas_surface.py` — `catenary_curve`, the sample loop

```
    for at in range(count):
        x_at = x_from + span * at / (count - 1)
        points.append([x_at, lift - a * math.cosh((x_at - vertex_x) / a)])
```

No two flow-rate labels overlap

The percentage badge goes where the curve is flattest, which is the lowest point of the hanging wire. Where that place is already taken, the badge walks outward along the curve and then downward in 20 pixel steps until it is clear.

`src/gui/main_tabs/wire_canvas_surface.py` — `place_badge`

```
    for at in badge_order(points, flattest):
        box = badge_rect(points[at], advance_px)
        if not any(rects_overlap(box, one) for one in placed):
            return box
```

Measured on a grid of 38 bots carrying 152 wires, with the real 33 pixel width of a printed rate label: 114 pairs of badges sit on each other at the flattest point, and none does after the walk. Both numbers come from the same run, so the zero is a result rather than an empty instrument.

One painter each side reads one set of geometry. The Qt sheet imports the curve and the placement from the same module the React payload is built from, so the two cannot drift.

`src/gui/visualizer/wire_canvas.py` — what the Qt sheet imports

```
from ..main_tabs.wire_canvas_surface import (
    arrow_points,
    catenary_curve,
    label_text,
    place_badge,
    point_at,
    pulse_percent,
    slack_for_offset,
)
```

Right-clicking a wire measures against the same curve, so the wire the pointer finds is the wire on the screen.

Qt against React

Twenty-two measured values were read off the Qt tab and out of the React payload and compared: the card size, the grid columns and spacing, the repaint interval, the theme count, the opacity slider, the absence of a View control, the tooltip, the four stage names, labels, initials and floors, the tergite counts, the crown spines, the wing spreads, the body scales, the stage colour table, the bot id colour and the absent-growth text. Twenty-two of twenty-two matched. The comparison reported one real difference on its first run, a grid of eight columns on the Qt side against six on the React side, which is what it exists to catch.

2026-09-09 22:12 - #128 - the Swarm tab loses List View

The Swarm tab no longer offers a choice of view. The grid of locust cards is the whole screen, and the View picker that used to sit between Theme and Wires is gone from both builds. The table below carries the row for every file the change touched; the rows in the table above are the state before it.

Qt file	React module	Uses React	Bridge	Manifest	Registers in Electron	Ships in the build	RENDERS	Scope
`bot_swarm_list`	removed	-	-	-	-	-	-	deleted
`src/gui/bot_visualizer.py`	`bot_visualizer.js`	yes	yes	yes	yes	yes	yes	in scope
`src/gui/visualizer/bot_node.py`	`bot_node.js`	yes	yes	yes	yes	yes	yes	in scope
`src/gui/visualizer/wire_canvas.py`	`wire_canvas.js`	yes	yes	yes	yes	yes	yes	in scope
`src/gui/visualizer/growth_stage.py`	shared, no module of its own	-	-	-	-	yes	yes	in scope
`src/gui/react_bot_swarm_tab.py`	host for `bot_visualizer.js`	yes	yes	yes	yes	yes	yes	in scope
`src/gui/theme_engine.py`	`design_tokens.js`	-	yes	yes	yes	yes	yes	in scope

The Qt widget file went. `BotListView` and `LaneWireCanvas` had one caller, the Swarm tab, and nothing else built either of them. The live bot settings window draws its own wire table from `QTableWidget`, so it lost nothing.

The List View module is deleted, and these are the three classes it held.

```
  class BotListView(QTableWidget):
  class LaneWireCanvas(QWidget):
  class BotSwarmLaneAllocator:
```

List View is removed whole. Its Qt surface and its renderer module are deleted with it. The bridge method, the manifest line and the renderer page's script tag that reached them are gone. The grid of locust cards is the whole tab (`src/gui/bot_visualizer.py, in BotVisualizationTab`).

`src/gui/web/bot_swarm_tab.js` — the shell that still names the list globals

```
  var LIST_API = "acervatorSwarmList";
  var LIST_LOADER = "acervatorLoadBotSwarmList";
```

Nothing defines those two globals now. The bot settings shell reads them in four places and guards every read, so the list region draws nothing and raises nothing.

`src/gui/react_bot_swarm_tab.py` — `TAB_SCRIPT_ASSETS`, the tail

```
    "quick_routing.js",
    "bot_visualizer.js",
)
```

One control in `tools/conversion_state.py` named the deleted file. A control pointed at a file that no longer exists reports `born React` rather than `paired`, which reads as a pass and proves nothing, so it now names a Qt file that is still paired.

`tools/conversion_state.py` — `CONTROLS`

```
CONTROLS = ("bot_visualizer", "theme_engine", "design_tokens")
```

Both builds were opened with the change in place. Qt draws eight main tabs and React draws ten, the same counts as before, and the Swarm tab in each carries no View picker, no row list and no lane sheet.

2026-09-23 - #239 - Disconnect All clears the key the fleet is stored under

Every control the tab carries

The tab holds three sub-tabs, six header controls and three routing buttons. A wire also carries two menus of its own.

Control	What it does	Behind it
Bot Swarm, Simulator Swarm, Paper Swarm	Picks the layer on show	the sub-tab bar
Identifier privacy dot	Masks the bot hashes and the symbol labels together	`_toggle_bot_swarm_identifier_mask`
Privacy Mode	Turns every mask on, or every mask off	`_on_privacy_mode_btn_clicked`
Exchange	Filters the swarm and the routing scope	`_on_exchange_filter_changed`
Theme	Picks one of the four palettes	`_on_theme_changed`
Wires	Sets the overlay opacity, 0 to 100 %	`_on_wire_opacity_changed`
Connect	Wires every ticked source to every ticked destination	`connect_clicked`
Disconnect	Removes the wire on every ticked pair	`disconnect_clicked`
Disconnect All	Removes every wire in the swarm	`disconnect_all_clicked`
Right-click a wire	Offers that wire, and both directions when both are wired	`_show_disconnect_menu`
Drag onto empty space	Offers the bot's own wires, one entry each	`_show_disconnect_picker`

Every one of the three routing buttons refuses out loud rather than doing nothing. With no bot ticked, Connect and Disconnect both answer **"No SOURCE bots are checked."**

What Disconnect All clears

A wire is stored once, at the top level of the fleet file, under `smart_wires`. `StateManager.save_state` writes that list from `SmartWireManager.export_wires`, and `restore_smart_wires_from_state` reads it back at the next launch.

`src/gui/main_tabs/bot_visualizer_surface.py` — the key a wire is stored under

```
WIRES_KEY = "smart_wires"
SOURCE_KEY = "source_id"
TARGET_KEY = "target_id"
```

`clear_all_routes` empties that list and answers the pairs that were in it. Both builds call it: the Qt tab through `_clear_all_routes_in_state`, and the React page through `LiveRoutingTab.clear_all_routes`. Each answered pair becomes one `wire.removed` on the bus, which `BotManager._on_wire_removed_mgr` turns into an `unregister_wire` on the topology's owner. The canvas empties as those messages arrive, and the next launch reads an empty list and paints nothing back.

`src/gui/main_tabs/bot_visualizer_surface.py` — `clear_all_routes`

```
def clear_all_routes(state: dict) -> list:
    """Empty ``WIRES_KEY`` and report the pairs ``stored_wires`` found.

    Each reported pair is one ``wire.removed`` the caller emits.
    """
```

The sentence this overtakes. The page carried no sentence naming the key Disconnect All cleared. The older wording that describes the routes a wire is stored as lives on in `apply_routes`, which Connect and Disconnect still use; only the clear and the paint read `smart_wires`.

The question, on both screens

The Qt build puts the question in a message box. The React page now draws the same two sentences itself, from the words the surface already published to it, and sends the operator's reply back with the press. A press that carries no reply is refused, because a bulk change nobody confirmed must not go through.

`src/gui/main_tabs/quick_routing_surface.py` — `LiveRoutingTab.ask`

```
def ask(self, title: Any, body: Any) -> bool:
    """Take the next reply in ``answers``, or no once they run out."""
```

The React page takes the wires onto its board

The React host subscribes to `wire.created` and `wire.removed` at build, the two messages every wire reaches a screen as, and retracts both subscriptions when the tab closes. Before that it held none, so its wire board drew nothing however many wires the fleet carried.

`src/gui/react_bot_swarm_tab.py` — `BotSwarmReactTab._subscribe_to_wires`

```
  for topic, action in (
      (surface.WIRE_CREATED_TOPIC, "wire_created"),
      (surface.WIRE_REMOVED_TOPIC, "wire_removed"),
  ):
```

2026-09-23 - #239 - every control read on the running program

What each control does, in both builds

The Qt tab carries **33** controls and the React page carries **21**. Every one was pressed or read on the running program, against a read-only copy of the saved fleet: 38 bots, 33 wires, 51 ledgers.

Control	What it does	Qt	React
Bot Swarm, Simulator Swarm, Paper Swarm	Picks the layer on show	yes	yes
Identifier privacy dot	Masks the bot hashes and the symbol labels together, in the shared register	yes	yes
Privacy Mode	Turns every mask in the shared register on, or every one off	yes	yes
Exchange	Narrows the swarm and the routing scope to one exchange	yes	yes
Theme	Picks one of the four palettes	yes	yes
Wires	Sets the overlay opacity, 0 to 100 %	yes	yes
Source ticks, Destination ticks	Pick the bots a routing press acts on	yes	yes
Rate	The share a new wire carries, 0 to 100 %	yes	yes
Connect	Writes one wire per ticked pair into the stored wire list	yes	yes
Disconnect	Removes the stored wire on every ticked pair	yes	yes
Disconnect All	Empties the stored wire list	yes	yes
Drag between two bots	Asks for a share, then stores that wire	yes	yes
Right-click a wire	Removes that wire, and offers both directions when both are wired	yes	**absent**
Drag onto empty space	Offers the bot's own wires, one entry each	yes	yes
+ Add Sim Bot, Run All, Stop All	Adds a Simulator bench row, and flips every row's own button	yes	**draws, acts on nothing**
+ Add Paper Bot, Start All, Stop All	Adds a Paper bench row, and flips every row's own button	yes	**draws, acts on nothing**
A bench row's asset, preset and capital	The values that row carries	yes	**absent**
A bench row's Run or Start	Flips that row's own button and status	yes	**absent**
A bench row's cross	Drops that row	yes	**absent**

The three routing buttons refuse out loud rather than doing nothing. Read on the running program: no source answers **"No SOURCE bots are checked."**, no destination answers **"No DESTINATION bots are checked."**, a rate that is not a number answers **"Rate 'abc' is not a number. Enter a percentage between 0 and 100."**, a rate of 0 answers **"Rate is 0% — that would create wires that route nothing."**, and a rate of 140 answers **"Rate 140.0 is outside 0–100%."**

A `yes` in the React column means the control acts when it is pressed. It does not mean the operator can reach it: only the sub-tab bar and the header strip paint on that page today. The section on how wide the tab has to be carries the measurement.

The key every connectivity press writes

The sentence this overtakes, from the section above:

> *The older wording that describes the routes a wire is stored as lives on in `apply_routes`, which Connect and Disconnect still use; only the clear and the paint read `smart_wires`.*

The true sentence. `apply_routes` now writes `WIRES_KEY`, so every connectivity press reads and writes the one list `stored_wires`, `clear_all_routes` and `hydration_plan` already read. Connect adds a record, a second Connect on the same pair replaces its share rather than adding a second record, and Disconnect drops it. Read on the running program with 33 wires stored: a Connect takes the list to **34** and a Disconnect takes it to **32**.

`src/gui/main_tabs/bot_visualizer_surface.py` — `apply_routes`

```
def apply_routes(state: dict, add: list, remove: list) -> dict:
    """Write each ``add`` pair into ``WIRES_KEY`` and drop each ``remove`` pair.

    ``stored_wires``, ``clear_all_routes`` and ``hydration_plan`` read
    that same list.
    """
```

The Qt tab calls the same function, so the widget and the page store a wire the same way. The right-click menu and the drag picker both land in `remove_wire`, which now drops the pair from the store as well as from the canvas, and the drag that makes a wire stores it when the operator accepts the share.

A direct write is refused when the load it came from carries no fleet, because `_load_bot_state_dict` answers an empty dict when the read fails. Read on the running program: a write from an empty load leaves the file byte-identical and logs the refusal; the same call from a real load moves the file.

The dot and the Privacy Mode button read one register

`PrivacyMaskRegistry` owns all 19 masks and the Trading tab writes the same one. Both builds now write it and both read it back.

- The dot flips `bot_swarm.identifiers`, the glyph turns from ● to ■, and the routing row labels turn from the symbol and short id to `**** [********]`.
- Privacy Mode sets all 19 masks on, or all 19 off, and its own word follows.
- **The dot also rewrites the Privacy Mode button.** Before this unit the button read `Privacy Mode: OFF` while one mask was on.

`src/gui/main_tabs/bot_visualizer_surface.py` — `any_mask_on`

```
def any_mask_on(self) -> bool:
    """Whether ``mask_registry`` holds any mask on.

    Answers ``masked`` or ``any_masked`` with no registry.
    """
```

What the page does not carry yet

The page draws the six bench layer buttons and no bench row. Each button carries no handler, so pressing all six leaves the row count at 0. The page has no right-click menu on a wire either: nothing under `src/gui/web/` binds a context menu. The Simulator bench belongs to #117 and the Paper bench to #19; a bench row's own Run and Start buttons flip a label and a status and start no session in either build.

The issue #117 and issue #19 clause above is overtaken. It is not edited and not removed. It read:

> *The Simulator bench belongs to #117 and the Paper bench to #19; a bench row's own Run and Start buttons flip a label and a status and start no session in either build.*

Issue #117 closed on 2026-09-19 and issue #19 closed on 2026-09-26, so neither pointer names open work. One surface declares both benches now, and the React page names the bench eleven times.

```
src/gui/main_tabs/bot_visualizer_surface.py
    SIM_BENCH_ID_FORMAT      "SIM-{idx:02d}"
    PAPER_BENCH_ID_FORMAT    "PAP-{idx:02d}"
    in sim_summary           the three lines under the Simulator bench rows
    in paper_summary         the same for Paper, with the bench capital
    in bench_id_text         a row's own id
    in bench_toggled         what a row's Run press answers
    in bench_status_style_sheet   the status colour

src/gui/web/bot_visualizer.js   names the bench on 11 lines
```

Whether a bench row starts a session is a product choice, so nothing here proposes one.

How wide the Swarm tab has to be

The Qt tab refuses to draw narrower than **1086 px**: the locust grid asks 786 px across its eight columns and the Quick Routing panel asks 278. Asked for 700 or for 900 the tab is 1086 wide both times, so in a pane under 1086 px the Wires slider and the three routing buttons sit past the right edge. At 1400 the whole tab draws — 38 locusts, 33 wires with their share labels, the two tick columns, the Rate box and the three buttons.

The React page fits every width sideways and **none of it downwards**. At 700, 900 and 1400 the body needs no sideways scroll and every header control and routing button reports its right edge inside the viewport. Read down the page instead: `tab-body` is 873 px tall and stacks all three layer panes in the flow — 287 + 293 + 293 — because a pane's own `display: flex` beats the `hidden` attribute. Inside the 287 px that leaves, `header-row` measures 900 px tall over children of 17 to 24 px, so `inner-row` starts at y = 931 with a height of 0, `quick-routing` has a height of 0, a routing button sits at y = 1754, and `wire-canvas` is `display: none`. The captured page shows the tab bar and the header strip and nothing under them. The locust grid, the wires and the routing panel are in the document and are not on the screen.

Where the wire list sits once a password is set

The sentence this overtakes, from the section on what Disconnect All clears:

> *A wire is stored once, at the top level of the fleet file, under* `smart_wires`*.*

The true sentence. That is where the wire list sits on an install carrying no password. With a password set, `smart_wires` moves inside one sealed field and the top level carries a plain-text latch in its place. `src/core/state_manager.py, in _sealed_for_disk` moves `bots`, `bot_count`, `smart_wires` and `smart_wire_ledgers` into that field on every save, and `in unseal` puts all four back for a session holding the key. Read on a state file saved with a password: the top level holds five fields — `fleet_sealed`, `sealed_fleet`, `saved_at`, `saved_at_human` and `version` — and not one of the four fleet fields. Read on the same material saved with no password: the same seven fields the page describes, and no latch.

What a routing row reads while the fleet is sealed

The sentence this stands beside, from the section on the privacy dot. It is not edited and not removed:

> *The dot flips* `bot_swarm.identifiers`*, the glyph turns from ● to ■, and the routing row labels turn from the symbol and short id to* `**** [********]`*.*

The current reading. A row label is the pair and the short id while the fleet is open. While the fleet is sealed there is no pair to read, and the row says so in a word instead of drawing a question mark. `src/gui/main_tabs/quick_routing_surface.py, in SEALED_SYMBOL` holds that word and `in LiveRoutingTab.state_symbol` returns it when `src/core/state_manager.py, in fleet_sealed` answers True. `in QuickRoutingModel.symbol_for` carries it into the label, and `src/gui/visualizer/quick_routing.py, in QuickRoutingMatrix._symbol_for` answers the same word on the Qt panel.

Read on a hand-built fleet of two, once sealed with a password and once saved without one:

```
the file   privacy dot   symbol_for    the drawn row label
sealed     off           SEALED        SEALED [UNIT-A1]
sealed     on            SEALED        **** [********]
plain      off           AAA/BBB       AAA/BBB [UNIT-A1]
plain      on            AAA/BBB       **** [********]
```

The privacy dot still masks a sealed row, so the sentence above holds unchanged for what the dot does.

? is what the row reads for a bot the stored settings cannot answer for, and that case is still a question mark. `src/gui/main_tabs/quick_routing_surface.py, in UNKNOWN_SYMBOL` holds it. A sealed fleet and a bot with no stored symbol are two different readings and the row no longer draws them alike.

Both builds draw the word. The label is formatted before the panel is handed its values, so the React page draws `SEALED [UNIT-A1]` from the same string the Qt panel draws.

Owed, one line. `docs/Acervator-Product-Manual.pdf` needs a rebuild for this page, `docs/manual/08-tabs/bot-swarm.md`. Another branch holds that binary and this change does not touch it.

The grid is the only surface that draws a bot's figures

The sentences this overtakes, from the section on the live rows:

A strip above the locust grid holds one dense row per live bot: the id, the pair, the mode, the timeframe, the capital, the state, the price, the profit and the trade count. The strip is hidden while the fleet is empty. The same fleet load that draws the locusts fills it.

The true sentences. No swarm tab draws a one-line bot row. The locust grid is the only surface that draws a bot's figures and it carries them under the icon. Driven with a six-bot fleet, Bot Swarm, Simulator Swarm and Paper Swarm build no row between them.

Each locust draws eight strings. The pair sits above the icon. Under the icon sit the realised growth figure, the profit and loss, the short id, the run state, the trade count and the last price. The mode, the candle length and the budget are not on the card, because the running-bot window already shows them.

The card is 88 by 108 pixels. It grew downward by three lines and the locust did not move: `BODY_BOX_HEIGHT_PX` pins the body box at the former card height, so the body centre and the body scale read what they read before.

`src/gui/main_tabs/bot_node_surface.py, in BotNodeModel.paint`

```
cx, cy = width / 2, BODY_BOX_HEIGHT_PX / 2
scale = min(width, BODY_BOX_HEIGHT_PX) / SCALE_DIVISOR * BODY_SCALE_RATIO[stage]
```

The three new lines are drawn at five point Consolas, each in a rectangle the full width of the card and ten pixels tall. The widest string each field can produce, measured with the machine's own Consolas loaded:

line	longest string	width
pair	thirteen characters	65 px
profit and loss	sign, seven digits, four decimals	80 px
run state	`COOLDOWN`	32 px
trade count	four digits and the word	44 px
last price	eight decimals, or seven digits	52 px

Every one fits the 88-pixel card. The Qt card and the React card take the same sizes from `bot_node_surface`, so the two builds draw the same eight strings in the same places.

CITED AS ABSENT

The path below is named on this page only to record which renderer page the dollar registry used before its removal. It does not open on disk.

- `src/gui/web/capital_registry.js` — the renderer page the dollar registry fed. Commit `d72f69cf` removed the registry, its Qt table, its view model and this page together. The renderer manifest names 68 modules and every one has a file on disk.

Back to the subsystem index.

Asset Charts

Reference. One candlestick panel per traded symbol.

What builds it

The builder mixin makes the tab and adds it to the row. The tab is a scroll area holding one panel per active bot.

`src/gui/main_tabs/charts_tab.py` — `ChartsTabMixin._build_charts_tab`

```
def _build_charts_tab(self) -> None:
    """Build the Charts tab and add it to the main tab widget."""
    self._charts_tab = TradeChartsTab()
    self._main_tabs.addTab(self._charts_tab, CHARTS_TAB)
```

Issue #450 renamed the tab from Asset Charts to Charts, and `CHARTS_TAB` holds that one word.

One method rebuilds that panel set from the current roster.

`src/gui/widgets/trade_charts_tab.py` — `TradeChartsTab.update_charts`

```
def update_charts(
    self,
    bot_statuses: list[dict],
    bot_manager=None,
    exchange_connectors: dict = None,
) -> None:
    """Create or update chart panels for each bot.
```

Three more methods keep a panel current.

Method	What it does
`_on_tf_changed`	Changes one panel's timeframe, leaving the others alone
`log_trade`	Adds a fill marker to the panel that owns the symbol
`push_synthetic_candles`	Feeds a panel whose venue returned no history

The chart widget

`CandlestickChart` in `src/gui/native_chart.py` paints each panel with QPainter. No browser and no WebEngine dependency.

Four record types carry everything drawn over the candles.

Three do now. `GridLine` is gone, and no painter in either build ever drew from it.

Record	Draws
`Candle`	One open, high, low, close and volume bar
`TradeMarker`	An executed buy or sell
`PositionMarker`	A holding, with a distinct icon for invisible and visible

The position record is the one that carries the visibility mode, which is what tells an invisible holding from one resting on the book.

`src/gui/native_chart.py` — `PositionMarker`

```
@dataclass
class PositionMarker:
    """Active position shown on chart with visibility mode icon."""

    price: float
    side: str  # "buy" or "sell"
    visibility: str  # "internal" (invisible) or "orderbook" (visible)
    level: int = 0  # Grid level index
    filled: bool = False
    asset_held: float = 0.0
```

The setters name what they place.

Setter	Places
`set_candles`	The candle series
`add_marker`	One executed fill
`set_trade_history_markers`	The fills already on record
`set_positions`	The holdings, invisible and on book
`set_tranche_floors`	The standing fold-tranche floors
`set_target_balance_lines`	The dollar target
`set_fire_armed_state`	An armed manual fire
`set_source_label`	The feed the candles arrived on
`set_error`	A failure message in place of the panel body

The mouse handlers give the panel a crosshair with a price and time readout, drag panning and a wheel zoom bounded by `_effective_visible_start` and `_effective_visible_count`.

Indicators on the chart

The chart calls the engine rather than computing anything of its own. Five series arrive that way, and an entry with no value is skipped when the panel paints.

`src/gui/native_chart.py` — `CandlestickChart._compute_indicators`

```
def _compute_indicators(self):
    """Fill ``_bb_data``, ``_vortex_data``, ``_macd_data``,
    ``_stochrsi_data`` and ``_ichimoku_data`` from the engine
    classes. A ``None`` entry marks a candle with no value, and
    ``paintEvent`` skips it.
    """
```

The toolbar under each panel holds the timeframe picker and eight toggles.

`src/gui/main_tabs/native_chart_surface.py` — `INDICATOR_TOGGLES`

```
INDICATOR_TOGGLES = (
    ("bb", "BB", "#50a0f0"),
    ("vortex", "Vortex", "#00ff88"),
    ("macd", "MACD", "#ffcc00"),
    ("stochrsi", "SRsi", "#ff9060"),
    ("ichimoku", "Ichi", "#c080ff"),
    ("volume", "Vol", "#80ffcc"),
    ("slingshot", "Sling", "#ff4488"),
    ("bbullseye", "BBull", "#ff00aa"),
)
```

Volume starts on and the other seven start off.

`src/gui/main_tabs/native_chart_surface.py` — `INDICATOR_DEFAULTS`

```
INDICATOR_DEFAULTS = {
    "bb": False,
    "vortex": False,
    "macd": False,
    "stochrsi": False,
    "ichimoku": False,
    "volume": True,
    "slingshot": False,
    "bbullseye": False,
}
```

Sling and BBull paint a placeholder shape rather than the indicator, and each says as much in its own tooltip. Issue #430 carries one further disagreement between a tooltip on this toolbar and what the chart paints.

The issue #430 clause above is overtaken. It is not edited and not removed. It read:

> *Issue #430 carries one further disagreement between a tooltip on this toolbar and what the chart paints.*

Issue #430 closed on 2026-09-09. No tooltip on this toolbar names a colour, so no tooltip can disagree with what the chart paints. The Vortex entry names its two lines, its period and its pane.

```
src/gui/native_chart.py
    tooltip="Vortex Indicator (VI+ / VI-, period 14, sub-pane)"

tooltips declared in that module   14
tooltips naming a colour            0
```

The complete voter set and the maths behind each indicator belong to the Indicator Voting Panel.

Where the candles come from

Three sources in order: the connected exchange first, a public API second, and the last successful fetch third. Each source that fails is recorded and the next one runs.

`src/exchange/chart_data.py` — `ChartDataFetcher.fetch`

```
# --- Source 1: Exchange OHLCV via CCXT ---
if exchange is not None:
    try:
        candles, source = await self._fetch_exchange(
            exchange, symbol, timeframe, limit
        )
        if candles:
            self._set_cache(symbol, timeframe, candles)
            return candles, source
```

With every source exhausted the fetcher returns an empty list and a label naming what failed, and the panel draws that label rather than an empty grid.

`src/exchange/chart_data.py` — `ChartDataFetcher.fetch`

```
# All sources failed
error_detail = " | ".join(errors) if errors else "No sources available"
```

Each row arrives as an `OHLCVCandle`.

The other chart

`src/gui/tradingview_chart.py` holds the QWebEngineView chart, which runs HTML and JavaScript inside the Chromium that PySide6 already ships. The Asset Charts tab does not use it; the panels above are painted natively. Its one construction site is the stock window, and the application builds no stock window, so no screen draws it today.

`src/gui/stock_main_window.py` — the one construction site

```
from .tradingview_chart import TradingViewChart

self._chart = TradingViewChart(symbol="AAPL", theme="dark")
```

Bridge

Three methods serve this screen.

Bridge method	Renderer module
`native_chart.state`	`native_chart.js`
`trade_charts_tab.state`	`trade_charts_tab.js`
`tradingview.chart`	`tradingview_chart.js`

2026-09-08 13:30 - #407 - a Live / ATA-SMP toggle over the arrows

The tab now walks two lists of markets. Live is the markets the bots are trading. ATA-SMP is the markets the TA engine has called and queued. One selector serves both: the arrows, the ticker menu and the readout all follow whichever list the toggle names.

The toggle is a button on its own row, above the blue arrows and the ticker menu. It reads the list on screen, so the button says Live while the Live list is up and ATA-SMP while the called markets are up.

`src/gui/main_tabs/trade_charts_tab_surface.py` — the two lists and their words

```
LIST_LIVE = "live"
LIST_ATA = "ata_smp"
LIST_MODES = (LIST_LIVE, LIST_ATA)
LIST_LIVE_TEXT = "Live"
LIST_ATA_TEXT = "ATA-SMP"
```

Pressing the toggle swaps the list, points the chart at that list's own selected market, and writes its header. Each list remembers where it was, so coming back to Live returns to the asset that was on screen.

`src/gui/main_tabs/trade_charts_tab_surface.py` — pressing the toggle

```
def toggle_list(self) -> str:
    """Move the arrows to the other list and answer the mode on screen."""
    self.list_mode = LIST_ATA if self.list_mode == LIST_LIVE else LIST_LIVE
    self.calls.append([SELECT_LIST_TOGGLED, self.list_mode, len(self.list_order())])
    self._follow_shown()
    self._label_shown()
    self._feed_shown()
    return self.list_mode
```

A market joins the ATA-SMP list when it reaches the Ready to Send bucket. It does not have to be posted. A call that is queued and then declined is still tracked, because the trigger is the bucket rather than the delivery.

The tab holds no register of its own. It reads the markets through a callable the Market Inspector hands it, so the list the operator walks and the calls phase seven is watching are one set of markets read twice.

`src/gui/main_tabs/market_inspector_tab.py` — where the two are joined

```
def _wire_ata_chart_list(self, inspector: Any) -> None:
    """Point the Charts tab's ATA-SMP list at ``inspector``'s ``PushBoard``.

    The Charts tab is built first, so it reads the board through a
    callable instead of holding a second copy of the markets.
    """
```

No entry is ever invented. With nothing bound, or with an empty bucket, the ticker offers one line reading No called market, the readout says 0 of 0, the arrows go grey, and a note beside the toggle says what the list is waiting for.

`src/gui/main_tabs/trade_charts_tab_surface.py` — the empty state's own words

```
ATA_EMPTY_TICKER_TEXT = "No called market"
ATA_EMPTY_HINT = "A market joins this list when it reaches Ready to Send."
```

An ATA-SMP market has no bot behind it, so the chart draws its candles without the trade markers, the target lines, the tranche floors and the fire glow that a traded asset carries. Nuclear Mode returns the tab to the Live

list before it draws, because a Nuclear scenario belongs to a bot.

Figures. This page carries no figure. The screen changed on this date and no capture of the toggle row exists yet, so the paragraphs above are the only record of it.

2026-09-08 13:30 - #407 - the arrows now reach the renderer

The React half of this tab drew the arrows and the ticker menu, and pressing either one changed nothing. The buttons called the bridge with the step it wanted, and the handler read every other request field and dropped that one, so the answer that came back was the state the tab was already in.

`src/gui/main_tabs/trade_charts_tab_surface.py` — the three the handler now reads

```
        params.get("step_by"),
        params.get("pick_at"),
        params.get("toggle_list", False),
```

The same tab also refused to draw at all under the renderer. One of its own checks read a name the file never declared, which stopped the whole payload before a single row was placed. The check now compares the number of rows the Python side declares against the number the renderer draws, which is the disagreement it was written to catch.

`src/gui/web/trade_charts_tab.js` — the check, repaired

```
  // The surface and this module must name the same number of tab rows.
  function checkSlots(model) {
    var content = objectField(model, CONTENT);
    if (!owns(content, LAYOUT_SLOTS)) {
      return;
    }
    if (content[LAYOUT_SLOTS] !== LAYOUT_SLOT_COUNT) {
      chartFaults.push(
        fault(CONTENT, LAYOUT_SLOTS, DISAGREES_FAULT, LAYOUT_SLOT_COUNT)
      );
    }
  }
```

2026-09-13 15:40 - #23 - the Charts tab draws in the desktop window

The tab draws under the window the operator launches. `ChartsTabReact` in `src/gui/react_charts_tab.py` builds one web view and loads one page. The page carries every script and every rule inside it, so it fetches nothing.

`src/gui/react_charts_tab.py` — the modules the page carries, in load order

```
def roster() -> tuple[str, ...]:
    """Every module the page carries, in load order.

    ``STYLE_SOURCE_ASSETS`` leads: ``trade_charts_tab.js`` reads a payload
    colour back to its token through ``shared_widgets.variableFor``.
    """
    return STYLE_SOURCE_ASSETS + CHILD_MODULES + (PANEL_MODULE,)
```

`trade_charts_tab.js` draws the tab. `native_chart.js` draws the candles into the slot that module keeps. Both register with the shell's own panel host, so the page in the window and the page in the shell run the same two modules.

The page reaches the model over one console channel. Each ask leaves the page as a line under ASK_PREFIX, and `ChartsTabPage` hands that line to `run_ask`. The answer returns by the ask's own number. The arrows, the ticker menu and the TF picker each move the asset the window holds.

`src/gui/react_charts_tab.py` — the call the dashboard pass makes

```
        def update_charts(
            self,
            bot_statuses: list,
            bot_manager=None,
            exchange_connectors: Optional[dict] = None,
        ) -> None:
            """Rebuild the asset list for one pass of bot statuses and redraw."""
            self._model.update_charts(bot_statuses, bot_manager, exchange_connectors)
            self.redraw()
```

`fetch_chart_data` refetches the asset on screen. `set_ata_source` takes the callable behind the ATA-SMP list. Both answer the calls the main window and the Market Inspector tab already make on the Qt tab.

The tab's own style sheet

`src/gui/web/trade_charts_tab.css` paints the chrome around the chart. It holds no colour of its own. Every rule names a design token, and `design_tokens.js` writes each token onto the page as a CSS variable.

`src/gui/web/trade_charts_tab.css` — the arrows and the list toggle

```
[data-part="asset-prev"],
[data-part="asset-next"],
[data-part="chart-list-toggle"] {
  background: var(--SURFACE_CONTROL, var(--btn-bg));
  color: var(--PRIMARY, var(--accent));
  border: 1px solid var(--GLOW_PRIMARY_EDGE, var(--border));
  border-radius: 4px;
  font-family: inherit;
  font-size: 12px;
  font-weight: bold;
  cursor: pointer;
}
```

An arrow with one asset to walk goes grey. The panel takes a border and a rounded ground. The header line, the toolbar and the toggle row each take a rule between them. The readout beside the arrows and the empty-list hint take the low text colour at ten pixels.

The chart keeps a host of its own

`native_chart.js` opens a React root of its own. It draws into `chart-host`, a child element the tab declares and never fills. One element under two roots lost the whole tab the moment the waiting line cleared.

`src/gui/web/trade_charts_tab.js` — the slot the chart draws into

```
    var hostProps = { key: CHART_HOST_PART, style: { flex: AUTO, overflow: HIDDEN } };
    hostProps[PART_ATTR] = CHART_HOST_PART;
    hostProps[BOT_ATTR] = text(props.botId);
```

A missing chart module says so on the slot. `renderChartMounts` asks the panel host to mount the slot with no model, and the host writes the reason it refused where the candles belong.

The tab redraws when its width changes. `resizeEvent` re-arms a short timer, and the timer pushes the payload again. The chart then takes the width the tab has.

What the page draws

The page draws the Live toggle, the two arrows, the ticker menu and the readout saying which asset of how many. Under them one panel header names the symbol, the price and the bot state. The chart draws the candles, the Bollinger cloud, the Ichimoku cloud, the last price on the right axis, the volume strip and the Vortex, MACD and Stochastic RSI panes. Eight indicator switches close the panel, with Sling and BBull off.

The legend beside the feed name carries its own colour. `native_chart_surface` holds the two Qt style sheets for it, and the tab surface sends them as `legend_styles`.

Back to the subsystem index.

Where the chart draws

The Charts tab host mounts the chart module inside itself. The tab draws in the window the operator launches, not only in the shell. `CHILD_MODULES` in `react_charts_tab.py` names the one module the tab slot takes.

```
modules the page registered   header_strip, native_chart, trade_charts_tab
panel faults                  none
drawn from forty candles      40 candles, 40 wicks, 40 volume bars
indicator panes               Vortex, MACD, Stochastic RSI
```

With the chart module absent the page still draws its own chrome, and the panel host records the loss rather than drawing an empty slot.

```
native_chart.js registered no panel to draw
```

2026-09-15 10:38 - #55 - one painter draws the chart in both builds

The chart area of the Charts tab is the image one painter paints, in both builds. The Qt build's chart widget already painted through that routine; the React build now shows the same image where its own drawing used to be. The ATA-SPM post images come from the same routine, so one chart looks one way in the Qt tab, in the React tab and in a post.

`src/gui/native_chart.py` — the routine both builds and ATA-SPM paint through

```
    def paint_image(
        painter: ChartPainter,
        width_px: int,
        height_px: int,
        device_pixel_ratio: float = 1.0,
    ) -> QImage:
        ratio = float(device_pixel_ratio) if float(device_pixel_ratio) > 0 else 1.0
        image = QImage(
            int(round(int(width_px) * ratio)),
            int(round(int(height_px) * ratio)),
            QImage.Format_ARGB32,
        )
        image.setDevicePixelRatio(ratio)
        image.fill(painter.BG_TOP)
        image_painter = QPainter(image)
        painter.paint_to(image_painter, int(width_px), int(height_px))
        image_painter.end()
        return image
```

Fed the same hundred candles, the same label and the same six overlays, the three agree byte for byte at 1200 pixels wide: the Qt widget, the React image and the ATA-SPM image all read `d8932829dcf760c1`. A planted change to one candle's close moves the Qt widget and the React image to the same new digest, `4dbcb29e360c363e`, so the agreement is a reading and not an artefact. The ATA-SPM image for its own inputs is unchanged by this work.

How the React page gets the image

The tab holds one painter beside its model. Each time the page draws, the chart slot asks the tab for the image at the slot's own width and height and at the page's device pixel ratio. The tab feeds the painter what the panel holds and answers a PNG as a data URI, which the slot shows as one image element.

`src/gui/react_charts_tab.py` — what the painter is fed, as the Qt tab feeds its chart

```
        def _feed_painter(self) -> None:
            """Give ``painter`` what ``PanelSink`` holds, as the Qt tab gives its chart."""
            panel = self._model.panel
            painter = self._painter
            painter.symbol = str(panel.label)
            painter.set_timeframe(str(panel.chart_timeframe or panel.timeframe))
            painter.set_candles(
```

The image carries the display's pixels. At a device pixel ratio of 1.25 the page asked for 1201 by 731 CSS pixels and received a 1501 by 914 pixel image; at 2x, 600 by 492 CSS pixels received 1200 by 984. In the

header band the 2x image holds 395 distinct colours where the 1x image scaled up holds 305, and the thinnest vertical run in the price pane is one device pixel where the scaled 1x image needs two.

The page no longer carries `native_chart.js`. The tab's list of child modules is empty, the page registers two modules, and the file stays in the tree unmounted.

`src/gui/react_charts_tab.py` — the modules the chart slot mounts

```
  CHILD_MODULES: tuple[str, ...] = ()
```

The eight toggles reach the painter

Each check box under the chart now asks the tab to switch that overlay on the painter, and reads the painter's own state back off the answer. The Qt build's boxes drive the same switch on the same painter.

`src/gui/web/trade_charts_tab.js` — a click on one box

```
  function overlayChosen(key, on) {
    if (!global.acervator || typeof global.acervator.call !== "function") {
      loadFault = NO_BRIDGE;
      return Promise.resolve(null);
    }
    var params = {};
    params[TOGGLE_KEY_PARAM] = key;
    params[TOGGLE_ON_PARAM] = Boolean(on);
    return global.acervator
      .call(METHOD, params)
```

Each of the eight, flipped from its starting state, moves the image to a new digest in both builds, and the two builds' digests agree on every flip; flipped back, each returns to the starting digest. A toggle left alone answers the last image with no repaint. The starting states are six on and two off: BB, Vortex, MACD, SRsi, Ichi and Vol on; Sling and BBull off.

What a repaint costs

The tab answers the last image again when nothing it reads has changed. When something has, the painter repaints and the PNG is encoded on the window's own thread. Measured ten runs each, six overlays on, one hundred candles:

width x height	ratio	image	paint	PNG + base64
1200 x 492	1x	1200x492	7.6 ms	21.6 ms
1920 x 492	1x	1920x492	9.0 ms	31.5 ms
1200 x 492	2x	2400x984	18.0 ms	76.9 ms
1920 x 492	2x	3840x984	22.1 ms	112.9 ms

The dashboard pass calls `update_charts` every two seconds, and the header label carries the price, so a moving price repaints on most passes. Candles refetch at most every 30 seconds.

Live candles reach the React chart

The React tab's fetch handed the fetcher's rows to `list`, and the fetcher's row type is not iterable, so every live fetch ended in an error line and no candles. The rows now pass through `candle_row`, which reads an object, a dict or a six-number row.

`src/gui/main_tabs/trade_charts_tab_surface.py` — the fetch, repaired

```
            if rows:
                self.panel.set_candles([candle_row(one) for one in rows])
                self.panel.set_source(answered)
```

Figures. `artifacts/issue-55/` holds the Qt tab, the React tab and the ATA-SPM image before and after, for the same hundred candles. The React page draws the tab header line and the painter draws its own header line under it, so the label reads twice on that build.

2026-09-19 08:23 - #55 - every Voting Panel indicator on the chart

His words, 2026-08-21: *"Asset Charts Tab - Restructure - Must now display only one chart at a time with all indicators from the Indicator Voting Panel loaded and properly visualized."* And 2026-09-08: *"Want charts tab to only display one richly detailed and Acervator-annonated chart at a time with all Indicators visible by default excluding that might visually oclude BB or ICHI as Sling or Z-Score Algo Point. All indicators will need toggle check boxes at the bottom."*

The five series the indicator classes now answer

ADX, Supertrend, Z-Score, KER and RSI each gain one method beside their vote, named as the Vortex and Ichimoku classes name theirs. The vote reads that method, so the line on the chart and the vote in the Voting Panel are one arithmetic.

`src/trading/indicators/adx.py` - the series and the vote reading it

```
    def lines(self, candles: list) -> tuple[_Line, _Line, _Line]:
        """+DI, -DI and ADX, one entry per candle, for a chart to draw.
```

```
        di_plus_series, di_minus_series, adx_series = self.lines(candles)
```

class	`lines` answers, one entry per candle
`ADXIndicator`	+DI, -DI and ADX
`SupertrendIndicator`	the line, its side and the ATR under it
`ZScoreIndicator`	a `ZScoreBar`: the smoothed score, the two projected prices and the window figures
`KaufmanERIndicator`	the ratio
`RSIIndicator`	Wilder's RSI
`SlingshotIndicator`	a `SlingshotBar`: the squeeze state, the release, the snapback and the momentum

The vote is unchanged. `compute` from the commit before and `compute` from this one ran over every prefix of 600 recorded A15 1h candles, six classes, every Signal field compared: 0 of 600 prefixes differ for any class. The control: shifting one class's series by one candle makes the same comparison report 549 to 589 differing

prefixes.

Fourteen overlays, seven sub-panes

The overlay registry now holds the twelve Voting Panel indicators in the panel's own order, then the two overlays that draw no voter: the Z-Score algo point and BB Bullseye.

`src/gui/native_chart.py` - one of the six new entries

```
    ChartOverlay(
        key="adx",
        label="ADX",
        colour_field="chart_last_price",
        pane=SUB_PANE,
        occludes=False,
        draw="_draw_adx",
        tooltip="ADX (14) with +DI and -DI, 0..100, 20 and 25 ruled, sub-pane",
        voter="adx",
    ),
```

overlay	pane	what it paints	ruled
ADX	sub-pane	+DI, -DI and ADX on 0..100	20 and 25
STrd	price pane	the Supertrend line, green under price while bullish, red over it while bearish	
ZSc	sub-pane	the smoothed score on a symmetric scale of at least 3	+2 and -2
KER	sub-pane	the Efficiency Ratio on 0..1	
RSI	sub-pane	Wilder's RSI on 0..100	30 and 70
ZPt	price pane	the resistance and support prices the averaged reversals project, dashed	

Slingshot now draws the indicator's own marks. Before, the painter re-derived a bandwidth squeeze of its own and drew marks the indicator never fired; now each diamond is a bar the class read as a squeeze release, signed by its momentum, and each circle a bar it read as a Bollinger snapback.

`src/gui/native_chart.py` - the slingshot marks

```
                if bar.released and bar.momentum != 0.0:
                    marks.append(("release", bar.momentum > 0.0))
                if bar.snapback:
                    marks.append(("snapback", "bull" in bar.snapback))
```

The sub-panes sit under the volume strip in registry order: Vortex, MACD, Stoch RSI, ADX, Z-Score, KER, RSI. Each takes 60 px at the chart's natural height. When the window gives less, every sub-pane shrinks alike, down to 28 px, so the price pane keeps its 220 px and every pane stays on screen.

`src/gui/native_chart.py` - the pane heights

```
SUB_PANE_H = 60
SUB_PANE_MIN_H = 28
PRICE_PANE_LAYOUT_H = 220
PRICE_PANE_MIN_H = 120
```

The legend

A band under the OHLC row names every overlay with the last candle's value of its series, in the overlay's colour. An overlay that is off reads `off` in grey. The band wraps to the chart's width. It is painted by the painter, so the Qt widget, the React image and the ATA-SPM post images all carry it.

`src/gui/native_chart.py` - what one legend entry carries

```
                found.append(
                    {
                        "key": overlay.key,
                        "label": overlay.label,
                        "text": text,
                        "colour": self.overlay_colour(overlay).name(),
                        "on": on,
                        "has_value": value is not None,
                    }
                )
```

Fourteen toggles

The toggle row under the chart holds one box per overlay, in the same order. The three occluders start off: Sling, ZPt and BBull, each a mark or a fill over the price pane. Every other indicator starts on.

`src/gui/main_tabs/native_chart_surface.py` - the occluders

```
INDICATOR_OCCLUDES = {
    "bb": False,
    "vortex": False,
    "macd": False,
    "stochrsi": False,
    "ichimoku": False,
    "volume": False,
    "slingshot": True,
    "adx": False,
    "supertrend": False,
    "zscore": False,
    "ker": False,
    "rsi": False,
    "zscore_point": True,
    "bbullseye": True,
}
```

Both hosts flip an overlay through one method on the painter. The Qt check box calls it, and the React box asks the tab, which calls it.

`src/gui/native_chart.py` - the Qt press

```
            self._chart.set_overlay(name, on)
```

The two emitters

`charts.indicator.toggled` is written once per press, with the key, the state asked and the variant; `ok` is true when the painter's state after the press is the state asked. `charts.indicators.drawn` is written once the candles arrive and the series recompute, with the symbol, the timeframe, the keys shown and each one's legend value; `ok` is true when every shown overlay holds a value on the last candle. Both go through the emitter network's one wire and nothing else.

`src/gui/native_chart.py` - the two names

```
TOGGLED_PIN = "charts.indicator.toggled"

DRAWN_PIN = "charts.indicators.drawn"
```

Read off the running program

The real window in each build, the home on a scratch directory, every socket but loopback refused, a two-bot fleet read off a copy of the operator's `bot_state.json`, and the real fetcher fed 100 recorded A15 1h candles by a stand-in connector:

reading	Qt	React
toggles under the chart, in order	14: BB, Vortex, MACD, SRsi, Ichi, Vol, Sling, ADX, STrd, ZSc, KER, RSI, ZPt, BBull	the same 14 on the page
starting states	11 on, Sling, ZPt and BBull off	the same
legend entries	14, 11 with a value, 3 reading `off`	the same 14 in the image answer
ADX on the last candle, by hand and on the painter	20.16, +DI 10.91, -DI 40.24	the same
Supertrend	63,003.61, bearish	the same
Z-Score	-1.24	the same
KER	0.7617	the same
RSI	25.44	the same
each new toggle pressed off then on	the line gone and back, the image digest moved and returned, one emitter record per press	the same through the page's box
sub-pane rules	RSI 30 and 70, ADX 20 and 25, Z-Score +2 and -2: 692 to 727 pixels on each ruled row differ from a repaint with no rules, 0 on the row above and 0 on a control row	the same
`bot_state.json` after every press	byte-identical	byte-identical
the planted legend value	the legend reads 99.99 where the series reads 25.44, and the comparison reports it	the same

The two builds paint one image: the painter's image for the same candles reads one digest in both.

Sentences this entry overtakes

They were not reworded. They are quoted here.

`docs/manual/08-tabs/asset-charts.md, under 'Indicators on the chart'` - "The chart calls the engine rather than computing anything of its own. Five series arrive that way, and an entry with no value is skipped when the panel paints." Fourteen series arrive that way now.

`docs/manual/08-tabs/asset-charts.md, under 'Indicators on the chart'` - "The toolbar under each panel holds the timeframe picker and eight toggles." Fourteen toggles.

`docs/manual/08-tabs/asset-charts.md, under 'Indicators on the chart'` - "Volume starts on and the other seven start off." Eleven start on and three start off.

`docs/manual/08-tabs/asset-charts.md, under 'Indicators on the chart'` - "Sling and BBull paint a placeholder shape rather than the indicator, and each says as much in its own tooltip." Both paint the indicator, and neither tooltip says placeholder.

`docs/manual/08-tabs/asset-charts.md, under 'The eight toggles reach the painter'` - "The starting states are six on and two off: BB, Vortex, MACD, SRsi, Ichi and Vol on; Sling and BBull off." Eleven on and three off.

Figures. `artifacts/u55/C2/` holds the Qt window and the React page at 1400 by 900 and at 1960 by 1200, the painter's image, and one venue PNG, before and after.

2026-09-19 17:40 - #55 - the painter's skin and the Acervator annotations

His words, 2026-08-21: *"Charts did get a slight visual over all during the last past some instances ago but I am still not completely satisfied."* And 2026-09-08: *"I want our chart renderer to be top notch. I want to be able to add modern features and graphics and styling to it so that we can help asset trading migrate away from the pastel and bland."*

Every colour is a theme token

The painter holds no colour of its own. Each colour it paints is one role in its palette table, and each role names a theme field and an alpha. A theme press on the Theme menu now reaches the painter in both builds: the window's theme switch calls the Charts tab, the tab calls the painter, and the painter resolves every role again. The toggle boxes under the chart and the two position labels above it read their colours from the painter, so they follow the press too. The toggle table in the chart's surface carries a theme field name per entry, not a hex string. A theme that lacks a field keeps the default theme's value for that role and the emitter row names how many roles it did resolve.

`src/gui/native_chart.py` - the roles added this day

```
        PALETTE_ROLES: dict[str, tuple[str, int]] = {
            "PANEL_SOURCE_TEXT": (PANEL_SOURCE_FIELD, 255),
            "TAG_TEXT": ("chart_bg_top", 255),
            "STRIP_FILL": ("chart_band", 40),
            "STRIP_EDGE": ("chart_band", 210),
            "FILL_TAG_SURFACE": ("chart_bg_top", 215),
```

`src/gui/main_window.py` - the one line the theme switch gained

```
                self._charts_tab.set_theme(tm.current)
```

At build the painter starts in the theme the window is in, and the venue image paints in that theme too when its caller names none.

`src/gui/native_chart.py` - the theme in force

```
    def theme_in_force():
        """The ``ThemeTokens`` ``ThemeManager.apply_theme`` last painted."""
        return THEMES.get(applied_theme(), DEFAULT_THEME_TOKENS)
```

Crisp at every device pixel ratio

The paint pass reads the device pixel ratio of the surface it paints and holds one device pixel as a logical length. The grid, the time ticks and the pane separators draw with anti-aliasing off, one device pixel wide, on coordinates moved to the centre of a device pixel. The wick is a share of one candle column and never under one device pixel; the candle border is one device pixel; the indicator lines keep their logical width and stay anti-aliased. Read on a 1200 pixel image at ratio 1.0 the pens are 0.8, 1.0, 1.2, 1.4, 1.5 and 1.56 logical pixels; at ratio 2.0 the set gains 0.5, the one-device-pixel pen.

`src/gui/native_chart.py` - the device pixel and the snap

```
            ratio = self._device_ratio(p)
            px = 1.0 / ratio

            def snap(value: float) -> float:
                """``value`` moved to the centre of the device pixel it falls in."""
                return (int(value * ratio) + 0.5) / ratio
```

The type

The header symbol is set in the design system's UI family at the card-title size, the OHLC row in the mono family at the label size, and the legend, the axes, the tags and the caption in the mono family at the caption size. The legend wraps at the chart's width as before; at 1400, 1366 and 1200 pixels it takes two rows and its widest row ends before the price scale.

`src/gui/native_chart.py` - the fonts

```
    def caption_font() -> QFont:
        """The mono font the legend, the axes, the tags and the caption are set in."""
        return design_font(ds.FONT_FAMILY_MONO, CAPTION_FONT_PX)

    def header_font() -> QFont:
        """The UI font the header symbol is set in."""
        return design_font(ds.FONT_FAMILY_UI, HEADER_FONT_PX, bold=True)
```

Every fill as the two glyphs

A fill is drawn as one of the two hermetic glyphs the Simulator's playback draws, from one definition both read: a sell as `dissolve`, a triangle pointing down as an outline, and a buy as `reform`, a triangle pointing up filled. The definition moved from the Simulator's surface to the chart's surface, and the Simulator imports it from there. The glyph sits on the candle whose interval holds the fill's stamp, one candle column wide and a twentieth of the price pane tall, with a tag naming the role and the price beside it. A fill whose stamp falls before the window or past its end is counted and not drawn.

`src/gui/main_tabs/native_chart_surface.py` - the one definition

```
MARK_GLYPHS = {
    SCRUM_SIDE: {
        "name": DISSOLVE_GLYPH,
        "points": [[-0.5, -0.5], [0.5, -0.5], [0.0, 0.5]],
        "filled": False,
    },
    FOLD_SIDE: {
        "name": REFORM_GLYPH,
        "points": [[-0.5, 0.5], [0.5, 0.5], [0.0, -0.5]],
        "filled": True,
    },
}
```

The two glyphs are hermetic characters

Two sentences above are overtaken. Each is quoted whole, with the true sentence under it.

Overtaken: *"A fill is drawn as one of the two hermetic glyphs the Simulator's playback draws, from one definition both read: a sell as* `dissolve`*, a triangle pointing down as an outline, and a buy as* `reform`*, a triangle pointing up filled."*

True now: a fill is drawn as one of two hermetic characters the Simulator's playback draws, from one definition both read. A sell draws U+1F761, named ALCHEMICAL SYMBOL FOR DISSOLVE, in gold. A buy draws U+1F75F, named ALCHEMICAL SYMBOL FOR PRECIPITATE, in blue. The alchemical block carries no COAGULATE character, and precipitate is the standard's name for a substance leaving solution and re-forming solid, which is what a fold does.

Overtaken: *"The glyph sits on the candle whose interval holds the fill's stamp, one candle column wide and a twentieth of the price pane tall, with a tag naming the role and the price beside it."*

True now: the character sits on the candle whose interval holds the fill's stamp, set at a twentieth of the price pane in the symbol family, with a tag naming the role and the price beside it. One candle column still sets the mark's footprint for placing that tag. The corner list and the fill flag are gone with the triangles they described, and so is the outline width, because a drawn character takes no outline.

```
# U+1F761 is named ALCHEMICAL SYMBOL FOR DISSOLVE, the scrum's operation.
DISSOLVE_GLYPH = "\U0001f761"
# U+1F75F is named ALCHEMICAL SYMBOL FOR PRECIPITATE, a substance leaving
# solution and re-forming solid, which is the fold's operation.
REFORM_GLYPH = "\U0001f75f"
MARK_GLYPHS = {
    SCRUM_SIDE: {"name": DISSOLVE_GLYPH},
    FOLD_SIDE: {"name": REFORM_GLYPH},
}
MARK_GLYPH_FAMILY = ds.FONT_FAMILY_GLYPH
```

The family resolves to Segoe UI Symbol, and both characters resolve on it. The Simulator's two playback windows draw the same two characters from the same object.

The fills come from two figures of the bot. Each tab subscribes to the bus topic every fill crosses and records it, so a fill made after launch draws on the next status tick. And each standing fold tranche the bot holds was made at a scrum: its reference price is that sell and its stamp is that sell's time, so the tab draws each one as a scrum glyph. Before this day nothing called the tab's trade recorder, so no fill had ever reached the chart.

`src/gui/widgets/trade_charts_tab.py` - the subscription and the handler

```
            get_event_bus().subscribe(FILLED_TOPIC, self._on_trade_filled)
```

```
        def _on_trade_filled(self, event) -> None:
            """Record one ``trade.filled`` bus event through ``log_trade``.

            The next ``update_charts`` tick draws it as a glyph on its candle.
            """
```

The target, the floors, the strip, the glow and the position

Each annotation carries a tag at the right edge, a badge filled in the line's colour with the theme's ground colour for text. The target line and the ceiling line read `TARGET` and `CEILING` with their prices. Each tranche floor keeps its dashed line; floors whose tags would overlap share one tag naming their count and their price span, so 16 floors in one pane read as seven tags. The landing strip is a band between the side's Bollinger band and its tolerance edge, over the strip's candles at the bot's own timeframe, tagged with the side and the count; it draws from the bot's own proximity reading. The fire glow keeps its gradient and gains `SCRUM ARMED` or `FOLD ARMED`. The position is the venue's average entry with the units held, drawn as its line, its icon and a `POSITION` tag.

`src/gui/native_chart.py` - the strip's bounds

```
        def _strip_bounds(self) -> Optional[tuple]:
            """The landing strip's two prices, ``(top, bottom)``, or None with no strip set.

            The band lies between the side's Bollinger band and its tolerance
            edge, ``(upper - lower) * tolerance_pct / 100`` inside it.
            """
```

`src/gui/main_tabs/trade_charts_tab_surface.py` - the readers both tabs share

```
  def landing_strip(bb: Any, timeframe: Any = "") -> Optional[dict]:
```

```
  def position_reading(bot: Any) -> Optional[dict]:
```

```
  def tranche_scrums(tranches: Any, bot_id: Any, symbol: Any) -> list:
```

The theme and annotation emitters

`charts.theme.applied` is written once per theme applied, with the theme, the variant and the symbol; `ok` is true when every palette role resolved from that theme. `charts.annotations.drawn` is written when the set of annotations the paint pass drew changes, with the counts drawn against the counts fed that fall inside the pane, and the fills fed and the fills off the window in its context. Both go through the emitter network's one wire.

`src/gui/native_chart.py` - the two names

```
  THEME_PIN = "charts.theme.applied"

  ANNOTATIONS_PIN = "charts.annotations.drawn"
```

The skin readings, off the running program

Both builds, the real window, the home on a scratch directory, every socket but loopback refused, one A15/USD bot's figures off a scratch copy of the operator's fleet file, and 100 recorded A15 1h candles rolled from the 5m tablet by the registry's own rollup:

reading	Qt	React
each of the five Theme menu entries pressed	79 of 79 roles move on the four non-default themes, 0 on the default; the toggle boxes and the two labels restyle	the same 79; the image digest moves on each press and the page's boxes recolour
literal colours in the painter and its surface	3 and 17 before, 0 and 0 after; a literal planted into one draw call reads 1	the same files
two fills handed to the tab's handler, one live-shaped fill on the bus, three standing scrums off the record	6 fills fed, 2 drawn as triangles on candles 60 and 80 of 100, 4 off the window and counted	the same 6, the same 2 triangles at the same vertices
the glyph corners against the Simulator's	one object, `MARK_GLYPHS`	the same object
55 floors off the record	55 lines fed, 16 inside the pane, 7 tags	the same
the target and ceiling prices	equal to the anchor and the cycle ceiling over the units held and the quote rate; outside the pane on these candles	the same
the position	the venue's average entry off the record, one `POSITION` tag	the same

reading	Qt	React
the strip	the bot's own detector reads none on any window of this tablet; a chosen 3-candle upper strip draws its band between the detector's own band and tolerance edge, tagged `STRIP upper 3c`	the same
the glow	`SCRUM ARMED` tagged	the same
pens at ratio 1.0 and 2.0	0.8 to 1.56 logical, then 0.5 joins at 2.0; anti-aliasing off eight times for the grid and the separators, on again after each	the same
a theme with one role	no crash; that role takes the planted colour, every other keeps the default, the emitter row reads 5 of 79 and false	the same
the legend at 1400, 1366 and 1200	two rows, none clipped	the same
`charts.theme.applied`	7 rows, 79 of 79 on each	7 rows
`charts.annotations.drawn`	2 rows, drawn equal to fed inside the pane	2 rows
`bot_state.json` after every press	byte-identical; a copy with one byte appended reads a different digest	the same
the venue image under Neon Light and Cyberpunk Dark	the top-left pixel equals each theme's ground and the foot pixel its lower ground; the two files differ; the message stamp is drawn in the theme's own axis text colour	one painter

One sentence this entry overtakes

It was not reworded. It is quoted here.

`docs/manual/08-tabs/asset-charts.md, under 'The chart widget'` - "The setters name what they place." The table under it lacks `set_landing_strip`, which places the landing strip band.

Figures. `artifacts/u55/C3/` holds the Qt window and the React page at 1400 by 900 and at 1960 by 1200, the painter's image at ratio 1.0 and 2.0 with a crop of each, and the venue image under two themes, before and after.

2026-09-19 20:10 - #55 - the ATA-SMP list draws the venue image's picture

His words, 2026-09-08: *"After a market qualifies, has its chart rendered with the appropriate confirming indicators and test, and is to be sent to the Ready to Send bucket the Charts tabs will continue to track said market under a ATA-SMP chart list."* And 2026-09-19: *"After its chart renderer is fully upgraded, it will need to be migrated to ATA-SMP."* A called market on the ATA-SMP list now draws the picture its venue images carry: the candles the call was made on, at the call's timeframe, the overlays of its confirming voters and no other, the reversal badge, the reading strip and the caption band. The Live list is unchanged.

What a called market's chart draws now

The chart takes the call's own candles, so it draws the same bars the venue image drew, and it fetches nothing for that market. The overlays switched on are exactly the ones the venue image switched on, the badge names the same direction, the strip carries the same readings and the foot carries the root image's caption. The bot annotations stay off: no fill glyph, no floor, no target line, no landing strip, no fire glow, no position marker.

`src/gui/widgets/trade_charts_tab.py` - what one call puts on the panel

```
        def _draw_call(self, entry: dict, call) -> None:
            """Draw ``call`` on the panel: its candles at its timeframe, its
            overlays, its badge and its caption, with no bot annotation.

            The overlay set the Live list showed is held in ``_live_overlays``
            until ``_leave_call`` restores it.
            """
            chart = self._panel.chart
            if not self._live_overlays:
                self._live_overlays = [
                    key for key, on in chart.overlays_shown().items() if on
                ]
            self._clear_annotations()
            chart.set_timeframe(str(call.timeframe))
            self._panel.show_only(list(call.overlays))
            chart.set_candles(list(call.candles))
            chart.set_call(call.direction, call.readings)
            chart.set_caption(call.caption)
```

The toggle boxes at the bottom follow the picture. The panel's own `show_only` flips the painter and sets each box to match, with the boxes' signals blocked so no press is counted, and the React page reads the same set off the payload it is drawn from.

Where the call comes from

Phase three keeps, on the chart it pulled, the candles the images drew and the overlay keys they switched on. The Inspector's sector board turns each pulled call into one record per symbol when the run lands, and the Charts tab reads that record through a second callable beside the one that lists the markets.

`src/trading/ata_spm.py` - one call as the tab draws it

```
@dataclass(frozen=True)
class ChartCall:
    """One call as the Charts tab draws it: the picture its venue images carry.

    ``overlays`` are the keys the images switched on, ``readings`` the
    ``(voter, message)`` pairs ``set_call`` takes, ``caption`` the root
    image's ``post_caption`` and ``candles`` the ``chart_candles`` rows.
    """
```

`src/gui/main_tabs/market_inspector_tab.py` - the second callable

```
        charts.set_ata_source(inspector.watched_markets)
        # MarketInspectorTab keeps its SectorBoard as _ata_board and offers no accessor.
        board = getattr(inspector, "_ata_board", None)
        if board is None or not hasattr(charts, "set_ata_call_source"):
            logger.debug("no SectorBoard reachable; ATA-SMP charts draw no call")
            return
        charts.set_ata_call_source(board.chart_call)
```

The newest call per symbol is the one drawn, which is the vote the list shows. A market on the list with no call held, because no run in this session called it, draws as before: its candles fetched, no badge.

The Live set held and given back

The overlay set the Live list showed is held the moment a call is drawn and put back when the list returns to Live or moves to a market with no call. The badge, the strip and the caption clear at the same moment.

`src/gui/widgets/trade_charts_tab.py` - leaving a call

```
        def _leave_call(self) -> None:
            """Clear the call's badge and caption and restore the Live overlay set."""
            chart = self._panel.chart
            chart.set_call("", ())
            chart.set_caption("")
            if self._live_overlays:
                self._panel.show_only(self._live_overlays)
                self._live_overlays = []
            if self._showing_ata():
                self._clear_annotations()
```

The tab's ATA-SMP readings, off the running program, both variants

The real window in each variant, the home on a scratch directory, every socket but loopback refused, a connector shaped like Coinbase Exchange serving the operator's own tablet copies, his scan shape (crypto, nothing typed, 1hr 1d 1wk, target 3), then the Charts tab's toggle pressed:

reading	Qt	React
the list after the press	A27 bear 1d, AXS bear 1h	the same
the chart	A27, 1d, 213 candles	the same
overlays on	adx, ichimoku, ker, macd, slingshot, supertrend, vortex; no other	the same
badge and strip	bearish, 7 readings; caption 3 lines	the same
floors, target line, glow, strip, position	none	none
boxes checked at the bottom	the same seven	the same seven off the page
X, Instagram, TikTok images against the chart	overlays equal, badge equal, candles 213 and 213	the same
a market with no voter, planted	50 candles, no overlay, the badge and nothing else	the same
`charts.ata.rendered` rows	one per call drawn, `ok` True	the same
the scratch `bot_state.json`	the same bytes before and after every press	the same

The rendered emitter

`charts.ata.rendered` writes when a called market's chart draws: the overlays the painter shows against the call's, with the symbol, the timeframe, the list, the widget's size, the direction, the candle count and the variant.

`src/gui/widgets/trade_charts_tab.py` - the row

```
                _emit(
                    ATA_RENDERED_PIN,
                    actual=shown,
                    expected=sorted(call.overlays),
                    context={
                        "symbol": call.symbol,
                        "timeframe": call.timeframe,
                        "list": LIST_ATA,
                        "width": chart.width(),
                        "height": chart.height(),
                        "direction": call.direction,
                        "candles": len(call.candles),
                        "variant": "qt",
                    },
                )
```

The sentence the ATA-SMP picture overtakes

It was not reworded. It is quoted here.

`docs/manual/08-tabs/asset-charts.md, under 'a Live / ATA-SMP toggle over the arrows'` - "An ATA-SMP market has no bot behind it, so the chart draws its candles without the trade markers, the target lines, the tranche floors and the fire glow that a traded asset carries." The sentence holds, and holds on screen now: on `5819ced6` the last bot's 55 floors, its target line and its glow stayed drawn when the list moved to a called market, and the candles were fetched. A called market's candles are the call's own, and its picture carries the call's overlays, badge, strip and caption, which the sentence does not name.

Figures. `artifacts/u55/C5/` holds the tab's ATA-SMP chart in both variants, the three comparisons side by side and the seven venue images.

2026-09-19 20:53 - #55 - the page's chart answers the pointer as the widget does

The React page's chart takes the same five pointer inputs the Qt widget takes, and answers each one the same way to the candle: a hover draws a crosshair with its readout, a wheel tick zooms about the pointer, a drag pans, a double-click fits every candle again, and a leave takes the crosshair off. The mechanism differs by input. A hover repaints nothing: the page draws the crosshair itself on a canvas over the image, from the geometry the painter exports beside every image. A zoom, a pan and a reset move the painter's window and repaint the image, as the widget repaints itself.

One arithmetic, called by both

The widget's handlers and the page's asks reach the same methods on `ChartPainter`, and those methods call the pure functions `native_chart_surface` already held: `zoom_window`, `pan_start`, `zoom_factor`, `clamp_y_zoom`, `effective_visible_start`, `effective_visible_count` and `fmt_price`. The drag state moved from the Qt widget onto the painter, so both variants keep it in one place.

`src/gui/native_chart.py` - the wheel, on the painter both variants hold

```
        def wheel_turned(
            self, x: int, wheel_delta: float, width_px: int, control_held: bool = False
        ) -> bool:
            if not self._candles:
                return False
            if control_held:
                self._y_zoom_pct = clamp_y_zoom(
                    self._y_zoom_pct * zoom_factor(wheel_delta)
                )
                self._emit_view(
                    VIEW_CAUSE_Y_ZOOM, self._visible_start, self._visible_count
                )
                self._repaint()
                return True
            window = zoom_window(
                wheel_delta,
                int(x),
                int(width_px),
                len(self._candles),
                self._visible_start,
                self._visible_count,
            )
            if window is None:
                return False
            self._visible_start = window["start"]
            self._visible_count = window["count"]
            self._emit_view(VIEW_CAUSE_ZOOM, window["start"], window["count"])
            self._repaint()
            return True
```

`CandlestickChart.wheelEvent` calls `wheel_turned` with the event's x, its angle delta, the widget's width and whether Ctrl is held. `CandlestickChart.mouseMoveEvent` calls `pointer_moved`, which places the crosshair and pans through `pan_to` while a drag is active; `mousePressEvent` and `mouseReleaseEvent` call `pointer_pressed` and `pointer_released`; `mouseDoubleClickEvent` calls `view_reset`; `leaveEvent` calls `pointer_left`. The grip drag along the bottom strip stays on the widget, which is the only variant that draws a grip.

The page's asks

The page sends each input that moves the window as one ask on the bridge method `trade_charts_tab.view`, with the pointer's x and y, the host's width and height and the page's device pixel ratio. `ChartsTabReact.chart_view` answers it on the tab's painter through the same methods.

`src/gui/react_charts_tab.py` - the ask answered

```
            if action == surface.VIEW_ACTION_WHEEL:
                moved = painter.wheel_turned(
                    x,
                    float(asked.get(surface.VIEW_DELTA_PARAM) or 0.0),
                    width,
                    bool(asked.get(surface.VIEW_CONTROL_PARAM, False)),
                )
            elif action == surface.VIEW_ACTION_PRESS:
                painter.pointer_pressed(x)
            elif action == surface.VIEW_ACTION_DRAG:
                moved = painter.pan_to(x, width)
            elif action == surface.VIEW_ACTION_RELEASE:
                painter.pointer_released()
            elif action == surface.VIEW_ACTION_RESET:
                painter.view_reset()
                moved = True
```

The answer carries the window's start and count and, when the window moved, the repainted image with its geometry, so a zoom costs one round trip. The image key now carries the window and the price padding, so the two-second `update_charts` pass answers the zoomed image again instead of the fitted one. A drag sends one move at a time; a move that arrives while one is in flight waits, and the latest goes when the answer lands.

The crosshair drawn by the page

Every image answer carries `geometry`: the pane rect, the price scale, the candle column width, the window, and for each visible candle its time label and the six readout lines the painter formats with `readout_lines` - the same function `paint_to` draws the widget's readout from - with the theme's colours as CSS text. The page's `pointermove` handler draws the dotted crosshair, the price badge, the time badge and the readout box on the canvas from those numbers, at the positions `paint_to` uses, and writes what it drew on the chart host as `data-crosshair-x`, `data-crosshair-y`, `data-crosshair-candle` and `data-readout`. A `pointerleave` clears the canvas and the attributes. The price badge at the pointer's row is the one text the page formats itself, from `PRICE_FORMAT_BANDS`, the bands `fmt_price` reads.

`src/gui/main_tabs/native_chart_surface.py` - the readout both draw from

```
def readout_lines(candle: CandleLike) -> list[list[str]]:
    accent = READOUT_ROLE_UP if candle.close >= candle.open else READOUT_ROLE_DOWN
    change = candle.close - candle.open
    return [
        ["O", fmt_price(candle.open), READOUT_ROLE_LIGHT],
        ["H", fmt_price(candle.high), READOUT_ROLE_LIGHT],
        ["L", fmt_price(candle.low), READOUT_ROLE_LIGHT],
        ["C", fmt_price(candle.close), accent],
```

The two pointer emitters

Two pins through `signal_contract.emit`. `charts.view.changed` writes each time the window moves, with `actual` the window the painter holds, `expected` the window the surface function answered, and the cause: `zoom`, `y_zoom`, `pan` or `reset`. `charts.crosshair.shown` writes at most once per quarter second per painter, with `actual` the candle `candle_at_x` places under the pointer and `expected` the candle the drawing pass placed: the widget's `paint_to` on the Qt build, the page's throttled `crosshair` ask on the React build.

The five inputs read off the running program

Both variants built the real `MainWindow` with the home on scratch, every socket but loopback refused, one A15/USD bot off a scratch copy of `bot_state.json`, 100 A15 1h candles rolled from a copy of the operator's 5m tablet, at a device pixel ratio of 1.25, the widget and the page's chart host both 1358 pixels wide.

input	Qt widget	React page
pointer at x 679 in the price pane	candle 52; readout O 64,634.01, H 64,846.02, L 64,632.71, C 64,741.54, delta +107.53 (+0.17%), V 445	candle 52; the same six lines to the character
one wheel tick in at x 679	window None, None to 7, 85	7, 85; the image digest moved
a drag of 120 px left	start 7 to 15	7 to 15
a double-click	None, None; padding 1.0	None, None; padding 1.0
a leave	`_mouse_x` None; the crosshair pixels gone	the canvas holds 0 painted pixels; the attributes gone
a pointer in the header band	no readout	no readout, no canvas pixels

With `ZOOM_IN_FACTOR` set to 0.5 in the React process only, the same wheel tick reads 26, 50 on the page against 7, 85 on the widget, so the equality reading can fail. The scratch `bot_state.json` and the tablet copy read the same digest after every input in both variants.

What a hover and a zoom cost

Measured on the page at 1.25, the chart host 1358 by 561 CSS pixels: a hover draws in 0.3 to 0.4 ms per move over 60 moves, the widget's repaint reading 13.9 ms per move over the same 60. A wheel tick on the page runs 66 to 83 ms from the ask to the image placed - 14.3 ms to paint, 41.1 ms to encode a 222 KB PNG at 1698 by 701 pixels, the rest transport and the 30 KB geometry - against 13.4 ms on the widget. A hover on the page repaints nothing.

Figures. `artifacts/u55/C4/` holds the crosshair and readout in both variants on the same candle, and both after the same wheel tick.

2026-09-20 05:40 - #55 - the frame: one control row, compact toggles, the value field on the ATA-SMP picture

His words, 2026-09-20: *"it does not appear that the GUI Archetype organized all of the elements properly using any kind of rules as we still have a significant void space and poorly placed or incorrect icons"*; *"TA values are smashed together in a long row above the chart when they should be stacked in the lower left corner in their own field with a dark background and these only need to be present for ATA-SMP posting"*; *"Check boxes for indicators are over large and taking up too much real estate."* The tab now lays its controls by one rule, in both builds.

The layout rule

One control row sits at the top of the tab. From the left: the list toggle, the previous arrow, the ticker menu, the next arrow, the `n of N` counter, the empty-list hint when the ATA-SMP list is empty, the `TF:` word and the timeframe menu, then a stretch, then the two legend labels and the source label at the right edge. Every control in the row is `CONTROL_HEIGHT_PX` tall and the gap between two controls is `SELECTOR_SPACING_PX`. The type is the design system's: the buttons and the menus at `TYPE_SMALL` in `FONT_FAMILY_UI`, the counter and the labels at `TYPE_CAPTION`, the two arrow glyphs at `ARROW_GLYPH_PX` in `FONT_FAMILY_GLYPH`. The chart fills the tab's width from the row's bottom edge to the toggle row. The toggle row is one line of fourteen boxes in `CHART_OVERLAYS` order, left-aligned, with no stretch between them; each box is `TOGGLE_BOX_PX` square, under the caption line height, and its label is at `TYPE_CAPTION` in the overlay's colour. The tab draws no legend band. The ATA-SMP picture draws the value field at the lower-left corner of the price pane. Nothing else moves.

`src/gui/main_tabs/native_chart_surface.py` - the numbers both builds read

```
CONTROL_HEIGHT_PX = 26
ARROW_GLYPH_PX = 16
ARROW_GLYPH_FAMILY = ds.FONT_FAMILY_GLYPH
CONTROL_FONT_FAMILY = ds.FONT_FAMILY_UI
CONTROL_FONT_PX = ds.TYPE_SMALL
CAPTION_PX = ds.TYPE_CAPTION
TOGGLE_BOX_PX = 12
```

Who lays the row

The Qt tab lays one `QHBoxLayout`. `ChartPanel` builds the `TF:` label, the timeframe menu, the two legend labels and the source label as before, and the tab places them in its row through `timeframe_widgets` and `legend_widgets`. The panel keeps the chart and the toggle row.

`src/gui/widgets/trade_charts_tab.py` - the panel's controls join the row

```
            for widget in self._panel.timeframe_widgets():
                widget.setFixedHeight(CONTROL_HEIGHT_PX)
                control_row.addWidget(widget)
            control_row.addStretch()
            for widget in self._panel.legend_widgets():
                widget.setFixedHeight(CONTROL_HEIGHT_PX)
                control_row.addWidget(widget)
            layout.addLayout(control_row)
```

The React page draws the same row in `ControlRow`, from the sizes and the families `selector_values` carries, and the surface's `LAYOUT_SLOTS` reads three: the control row, the chart and the toggle row. The page no longer draws a header line of its own over the image; the painter's header line inside the image is the one both builds show.

The arrows draw their glyphs

The arrows carry U+25C0 and U+25B6. In the UI family those glyphs are 6 by 7 pixels at 12 pixels, a mark under a quarter of the button's height, which reads as an empty button. The buttons now set `FONT_FAMILY_GLYPH` at `ARROW_GLYPH_PX`, and the glyph reads 12 by 13 pixels in a 34 by 26 button.

`src/gui/design_system.py` - the family for symbol glyphs

```
FONT_FAMILY_GLYPH = "'Segoe UI Symbol', 'DejaVu Sans', 'Apple Symbols', sans-serif"
```

The toggle boxes

Each Qt box takes its own style sheet: the label in the overlay's colour at the caption size, the indicator `TOGGLE_BOX_PX` square with a one-pixel border. The page sizes each box's input from the same number, carried as `toggle_box_px` in the panel chrome.

`src/gui/native_chart.py` - one box's style sheet

```
TOGGLE_STYLE_FORMAT = (
    "QCheckBox {{ color: {color}; font-size: {font_px}px; spacing: {gap}px; }}"
    "QCheckBox::indicator {{ width: {box}px; height: {box}px; "
    "border-width: 1px; border-radius: 2px; }}"
)
```

The value field on the ATA-SMP picture

The legend band under the OHLC row is gone from every surface: the OHLC row sits directly over the price pane and the chart's natural height loses the band's rows. The values now draw as a field at the lower-left corner of the price pane, only on a chart that carries a call. Every venue PNG and the tab's ATA-SMP list call `set_call` with a direction; the Live list never does, so the tab shows no field. The field holds one caption-size line per overlay that is on, the label in the overlay's colour and the value in the axis text colour, on the theme's `chart_bg_top` at alpha 205 so the candles under it stay faint, inside a one-pixel border in `chart_grid`, `FIELD_PAD_PX` in from the pane's left edge and bottom edge. Lines past the pane's room are not drawn.

`src/gui/native_chart.py` - what makes a chart the ATA-SMP picture

```
        def draws_value_field(self) -> bool:
            """True when the chart carries a call, which makes it the ATA-SMP picture.

            Every venue PNG and the tab's ATA-SMP list call ``set_call`` with a
            direction; the Live list never does, so the tab draws no field.
            """
            return CALL_DIRECTION_ROLES.get(self._call_direction) is not None
```

A market on both lists keeps one symbol and draws two pictures, the bot's and the call's, so a list press now clears the followed symbol before the chart follows the shown market. Before this, a market that was both traded and called kept the bot's picture on the ATA-SMP list.

Read off the running program, both builds

Both builds built the real `MainWindow` with the home on scratch, every socket but loopback refused, one A15/USD bot off a scratch copy of `bot_state.json`, 100 A15 1h candles rolled from a copy of the 5m tablet, at the display's ratio of 1.25, the window 1920, 1400 and 900 pixels wide, each under `python -m pdb` with a breakpoint on the first statement of `_draw_legend`.

reading	before, Qt / React	after, Qt / React
distinct y of the controls	8, 42, 78 / 12, 50, 115	8 / 12
the chart's top edge against the controls' bottom	108 against 104 / 136.4 against 134.4	42 against 34 / 50.8 against 38
the arrow glyph	Segoe UI 12 px, 59 pixels of ink in the grabbed button / the browser's fallback	Segoe UI Symbol 16 px, 138 pixels of ink / the same family and size on the page
the box	indicator 16 by 16 in a 24 px box, label 9 px / input 13 by 13, label 9 px	indicator 12 by 12 in a 14 px box, label 10 px / input 12 by 12, label 10 px; the caption line height 12
`_draw_legend` on the Live tab, the presses and ten repaints	19 hits / 14 to 16 hits	0 hits / 0 hits
the natural chart height at 1886 wide	764 / 764	732 / 732
the field on one planted call, on the tab	none	191 by 51, three lines, at the pane's lower-left corner in both builds; gone again on the Live list
the field on the six venue sizes	none	191 by 51 with three voters, 191 by 181 with thirteen overlays on, inside the price pane on every size

With the row's spacing planted to zero in the driver's process only, the gaps between the controls read 0 in both builds against 8 unplanted, so the reading can fail.

Figures. `artifacts/u55/C6/` holds the tab, the arrow buttons, one box, the window and the page at each width before and after, the six venue PNGs with three voters and with all twelve, and the tab on the planted call.

Five sentences the frame overtakes

"The toggle is a button on its own row, above the blue arrows and the ticker menu." - the toggle is now the first control of the one row.

"A band under the OHLC row names every overlay with the last candle's value of its series, in the overlay's colour. An overlay that is off reads `off` in grey. The band wraps to the chart's width. It is painted by the painter, so the Qt widget, the React image and the ATA-SPM post images all carry it." - no surface carries the band; the ATA-SMP picture carries the field, and an overlay that is off has no line in it.

"The legend wraps at the chart's width as before; at 1400, 1366 and 1200 pixels it takes two rows and its widest row ends before the price scale." - the field does not wrap; each value is one line.

"The React page draws the tab header line and the painter draws its own header line under it, so the label reads twice on that build." - the page draws no header line of its own.

"The header line, the toolbar and the toggle row each take a rule between them." - the page has no toolbar and no header line; the toggle row keeps its rule.

2026-09-20 06:10 - #55 - the trade events from the History; the floors gone

His words, 2026-09-20: *"Dotted yellow lines for prior orders are supposed to be gone. Supposed to have hermetic symbols for trade events based on the History."* The chart draws no tranche floor on any surface, and every trade event of the shown market is drawn as its hermetic glyph from the History tab's rows, in both builds.

The rule

No dashed floor line, no `FLOOR` tag and no `N FLOORS` tag on the Qt widget, the React page's image or any venue PNG. For each row the History tab holds for the shown symbol, one glyph: a sell as `dissolve`, a buy as `reform`, on the candle whose interval holds the row's stamp, tagged with the role and the price, `SCRUM` for a sell and `FOLD` for a buy. A row outside the window is counted and not drawn. The rows are the venue's own fills, and the History tab is the one place that fetches them; the Charts tab never calls a venue. A fill the bus carries after launch draws at once and is not doubled when the History's next fetch carries it. The ATA-SMP picture and every venue PNG carry no glyph.

Where the trade events come from

The History tab holds the venue's fills in `_all_trades`, the time of its last fetch in `_last_fetched_ts` and whether a fetch is in flight in `_fetch_in_flight`; see the History tab. The window builds it after the Charts tab, so the Charts tab reads it at each `update_charts` tick under the name the window gives it, `_history_tab`. One surface reader takes the three names, one reader turns the rows of the shown symbol into fills, and one join keeps each bus fill the venue has not answered for.

`src/gui/main_tabs/trade_charts_tab_surface.py` - the rows to fills

```
def history_fills(rows: Any, symbol: Any) -> list:
    """One fill per History row of ``symbol``: the venue's id, side, price and stamp, the role from the side.

    A row whose side is neither buy nor sell, or whose price or stamp is
    not positive, gives no fill.
    """
```

`src/gui/main_tabs/trade_charts_tab_surface.py` - the join

```
def merged_fills(venue: Any, logged: Any) -> list:
    """``venue`` fills, then each ``logged`` fill no venue fill names.

    A venue fill names a logged fill when both carry the same venue id, or
    when their prices sit within ``FILL_MATCH_PRICE`` and their stamps
    within ``FILL_MATCH_WINDOW_S`` of each other.
    """
```

The bus fill carries no venue id today, so every join is by stamp and price. A fill the venue answers as several rows at several prices keeps its bus glyph beside them until the next launch.

The one ask, and the landing

When the History has never fetched and no fetch is in flight, the Charts tab asks `refresh()` once in its life, the same call the window makes when the History tab is activated. The Charts tab also connects the History's `history_refreshed` signal once, so the rows landing redraw the shown market without waiting for the two-second tick. The window's own rule keeps the History fresh after that.

`src/gui/main_tabs/trade_charts_tab_surface.py` - the one time the tab asks

```
def history_never_fetched(state: dict) -> bool:
    """True when the History has never landed a fetch and none is in flight: the one time the Charts tab asks.
"""
    return state["fetched_ts"] == NEVER_FETCHED_TS and not state["in_flight"]
```

`src/gui/widgets/trade_charts_tab.py` - the Qt tab's draw

```
        def _draw_history(self) -> None:
            """Put the shown market's trade events on the chart: the History's fills
            for its symbol, then each bus fill since launch the venue has not
            answered for.

            A History that has never fetched is asked to ``refresh`` once, the
            call the window makes on activation; ``history_refreshed`` is
            connected once so the landing redraws without waiting for the tick.
            A called market on the ATA-SMP list gets none.
            """
```

The React widget hands the same three names to its model through `set_history`, and the model's `feed_history` runs in `update_charts` before the bot's own annotations. Both builds read the same glyph set on the same candles from one painter.

The floors gone

`ChartPainter` no longer holds `_tranche_floors`, `set_tranche_floors` or `_draw_floors`; `FLOOR_LINE` is no longer a palette role; the two floor tag formats are gone. The Qt tab no longer reads the bot's lots, the React model no longer carries floors, and the page no longer stamps a floor count. The annotation emitter `charts.annotations.drawn` names fills, target lines, the strip, the glow and positions, and no floors; it now also fires when the fed count or the off-window count moves, so a row landing outside the window writes a row. The standing scrums off the bot's fold tranches are no longer a source of markers: the History's rows are the venue's answer for the same sells.

The History readings, off the running program, both builds

Both builds, the real window, the home on a scratch directory, every socket but loopback refused, a two-bot fleet off a scratch copy of one seed record, 100 recorded A15 1h candles rolled from the 5m tablet by the registry's own rollup and shifted by whole hours so the newest candle opens at the current hour, and a History venue on loopback answering `get_my_trades` with 40 A15/USD rows (26 on distinct candles inside the window, 14 older than it) and 6 A14/USD rows, through the History tab's own fetch path:

reading	Qt	React
`_draw_floors` and `set_tranche_floors` on the painter	absent; 0 floors held; the emitter names no floor	the same

reading	Qt	React
the History at the tab's first tick	never fetched; the tab asks once; the venue is asked once per market, 2 of 2; 46 rows land in 0.21 s	the same
the shown market's rows against its glyphs	40 rows for A15/USD, 40 markers held, 26 drawn on 26 candles, 14 counted off the window	the same 40, 26 and 14
a six-notch zoom to 36 candles, then a reset	5 drawn and 35 off; back to 26 and 14	the same
one bus fill on the newest candle	drawn at once, 27 of 41; the History's next fetch carrying it leaves 27 of 41	the same
a row planted inside the window, then one outside	28 drawn of 42; then 28 drawn of 43 with 15 off	the same
A14/USD shown, then A15/USD again	6 rows, 4 drawn, 2 off; back to 28 of 43	the same
the History tab after these fetches	49 of 49 trades shown, page 1 of 1	the same
the venue image with a call	no glyph, no floor	one painter

Figures. `artifacts/u55/C7/` holds the tab, the chart and the window at 1920 and 1400 in both builds, before and after.

Six sentences the History glyphs overtake

They were not reworded. They are quoted here.

"`set_tranche_floors` | The standing fold-tranche floors" - the setter table row; the setter no longer exists and nothing places a floor.

"The fills come from two figures of the bot. Each tab subscribes to the bus topic every fill crosses and records it, so a fill made after launch draws on the next status tick. And each standing fold tranche the bot holds was made at a scrum: its reference price is that sell and its stamp is that sell's time, so the tab draws each one as a scrum glyph." - the fills come from the History tab's rows for the shown symbol, then the bus fills since launch the venue has not answered for; the fold tranches are no longer a source.

"Each tranche floor keeps its dashed line; floors whose tags would overlap share one tag naming their count and their price span, so 16 floors in one pane read as seven tags." - no floor draws on any surface.

"`charts.annotations.drawn` is written when the set of annotations the paint pass drew changes, with the counts drawn against the counts fed that fall inside the pane, and the fills fed and the fills off the window in its context." - it is also written when the fills fed or the fills off the window change, and it names no floor.

"`55 floors off the record | 55 lines fed, 16 inside the pane, 7 tags | the same`"-a reading of the floors, which no longer draw.

"An ATA-SMP market has no bot behind it, so the chart draws its candles without the trade markers, the target lines, the tranche floors and the fire glow that a traded asset carries." - the sentence holds; there are no tranche floors on any market now.

2026-09-20 09:45 - #55 - the oscillators: one readable sub-pane height, whole-pixel pens, tags that never cross

His words, 2026-09-20: *"Oscillators are not being drawn at the highest quality and are too compressed vertically to be read clearly."* And: *"it does not appear that the GUI Archetype organized all of the elements properly using any kind of rules."* Three rules now govern the seven sub-panes, the pens and the fill tags, and each is proved on the running program in both builds. The GUI Archetype carries no layout or draw-quality rule, so the rules are stated here and read off the widget and the page.

The height rule

Every sub-pane on the Charts tab draws at one readable height. The figure is derived from what a sub-pane holds: a label row one tag tall over a plot of five bands one tag tall each - room over the upper ruled level, the upper level, the value tag at the middle, the lower level, and room under the lower level. A tag is 14 px, so a sub-pane is 14 + 5 x 14 = 84 px. The lines, the histogram and the ruled levels plot under the label row and never through it. The value tag sits at the plot's middle. The price pane never falls under 220 px.

`src/gui/main_tabs/native_chart_surface.py` - the figure beside the pane numbers

```
TAG_HEIGHT_PX = 14
#: A sub-pane's label row, one tag tall, over its plot.
SUB_PANE_LABEL_PX = TAG_HEIGHT_PX
#: The tag-tall bands a sub-pane's plot holds: over the upper ruled level, the
#: upper level, the value tag at the middle, the lower level, under the lower level.
SUB_PANE_PLOT_BANDS = 5
#: The height every sub-pane draws at on the Charts tab: the label row over
#: the plot's bands. A host shorter than the panes' height scrolls.
SUB_PANE_READABLE_PX = SUB_PANE_LABEL_PX + SUB_PANE_PLOT_BANDS * TAG_HEIGHT_PX
#: The least height a sub-pane folds to on a fixed-height venue image.
SUB_PANE_FOLD_PX = 28
```

The chart's height is the natural height of the panes on. With the volume strip and the seven sub-panes it is 28 + 18 + 220 + 28 + 7 x 84 + 18 = 900 px. A panel taller than that gives the rest to the price pane. A panel shorter than that keeps the chart at 900 and scrolls it.

`src/gui/native_chart.py` - the widget's height

```
        def _apply_height_for_panes(self) -> None:
            """Set the minimum height to ``_natural_height_for_panes`` at the widget's width.

            ``_height_override`` from a grip drag wins when it is taller. The
            scroll area holding the chart scrolls when its viewport is shorter
            than this height; a taller viewport's extra height goes to the price pane.
            """
            target = self._natural_height_for_panes(self.width())
```

The scroll hosts

On the Qt panel the chart sits in a scroll area with a vertical bar as needed and no horizontal bar. A wheel over the chart zooms the window, as before, and never scrolls the area, because the chart takes the wheel. The bar scrolls. The toggle row sits under the scroll area at the tab's foot.

`src/gui/native_chart.py` - the scroll area in the panel

```
            self._scroll = QScrollArea(self)
            self._scroll.setAccessibleName(CHART_SCROLL_NAME)
            self._scroll.setWidgetResizable(True)
            self._scroll.setFrameShape(QFrame.NoFrame)
            self._scroll.setHorizontalScrollBarPolicy(Qt.ScrollBarAlwaysOff)
            self._scroll.setVerticalScrollBarPolicy(Qt.ScrollBarAsNeeded)
            self._scroll.setWidget(self._chart)
            layout.addWidget(self._scroll)
```

On the page the chart mount scrolls the same way. The image host inside it is never under the natural height, so a shorter mount scrolls and the toggle row under the mount stays at the panel's foot. The wheel listener on the host already takes the wheel for the zoom.

`src/gui/web/trade_charts_tab.js` - the mount and the host

```
      style: {
        flex: AUTO,
        minHeight: ZERO_PX,
        overflowX: HIDDEN,
        overflowY: AUTO,
        display: FLEX,
        flexDirection: COLUMN
      }
```

```
    into.style.minHeight = height(
      answer[IMAGE_NATURAL_HEIGHT] === undefined
        ? answer[IMAGE_HEIGHT]
        : answer[IMAGE_NATURAL_HEIGHT]
    );
```

The image the page asks for is never under the natural height either.

`src/gui/react_charts_tab.py` - the image's height

```
            natural = int(self._painter._natural_height_for_panes(width))
            minimum = int(self._painter._minimum_height_for_panes(width))
            height = max(natural, asked_height)
```

A fixed-height venue image keeps the fold of 2026-09-19: each sub-pane shrinks from the readable height toward 28 px, then the reading strip folds, then the price pane shrinks toward 120 px, and a height under the least layout is refused by name.

The quality rule

Every stroke pen's width is its logical width scaled by the device pixel ratio to whole device pixels, never under one. The five logical widths 0.8, 1.0, 1.2, 1.4 and 1.5 draw 1, 1, 1, 1 and 2 device pixels at ratio 1.0 and 1, 1, 2, 2 and 2 at 1.25. Lines draw with anti-aliasing on. Axis rules - the grid, the pane separators, the ruled levels, the reference lines and the MACD zero line - draw with anti-aliasing off, one device pixel wide, on coordinates snapped to the centre of a device pixel. Bars - the volume bars and the MACD histogram - draw with anti-aliasing off on whole device pixels. Text stays at the caption size in the mono family.

`src/gui/main_tabs/native_chart_surface.py` - the pen width

```
def device_pen_width(width_px: float, ratio: float) -> float:
    """``width_px`` logical pixels scaled by ``ratio`` to whole device pixels, never
    under one, given back in logical pixels."""
    scale = float(ratio) if float(ratio) > 0 else 1.0
    return max(1, round(float(width_px) * scale)) / scale
```

`src/gui/native_chart.py` - one ruled level

```
                def _rule_line(y: float, colour=None, dashed: bool = True) -> None:
                    """One ruled level across the pane at ``y``: one device pixel, aliased, snapped."""
                    pen = QPen(colour if colour is not None else self.GRID_MINOR, px)
                    if dashed:
                        pen.setStyle(Qt.DashLine)
                    p.setRenderHint(QPainter.Antialiasing, False)
                    p.setPen(pen)
                    p.drawLine(QPointF(ML, snap(y)), QPointF(w - MR, snap(y)))
                    p.setRenderHint(QPainter.Antialiasing, True)
```

The tag rule and the readout

Every glyph draws first. A fill tag is then placed at the right of its glyph, or at the left when the right would cross the price scale. A tag that would cross a glyph or a tag already drawn is not drawn. The glyph count does not move. Merging two tags into one naming a count was not chosen, because a count names no price and no role.

`src/gui/native_chart.py` - where a tag goes

```
        def _tag_place(
            self,
            right_x: float,
            left_x: float,
            y: float,
            width: float,
            drawn: list,
            scale_x: float,
        ) -> Optional[QRectF]:
            """The rect a fill tag draws in: ``TAG_H_PX`` tall at ``right_x`` when
            ``_tag_clear``, else at ``left_x`` when clear, else None and no tag."""
            for x in (right_x, left_x):
                rect = QRectF(x, y, width, TAG_H_PX)
                if self._tag_clear(rect, drawn, scale_x):
                    return rect
            return None
```

The crosshair's readout names every fill on the candle under the pointer. One row per fill follows the six candle rows: its label is the side's glyph character, a sell as a filled triangle pointing down and a buy as one pointing up, and its text is the tag's text in the marker's colour. The painter's own tip and the page's canvas tip both draw the rows, because the page reads them off the geometry the image answer carries.

`src/gui/main_tabs/native_chart_surface.py` - one readout row for a fill

```
def fill_row(fill: FillLike) -> list[str]:
    """One readout row for ``fill``: the side's glyph, the tag's text and its role."""
    is_buy = str(fill.side).lower() == FILL_BUY_SIDE
    return [
        READOUT_BUY_GLYPH if is_buy else READOUT_SELL_GLYPH,
        FILL_TAG_FORMAT.format(label=fill_label(fill), price=fmt_price(fill.price)),
        fill_role(fill),
    ]
```

The oscillator readings, off the running program, both builds

Each build ran the real window with the home on a scratch directory, every socket but loopback refused, two bots off a scratch copy of one record, 100 A15 1h candles and a loopback History venue answering 40 fills, 26 inside the window. The chart's host was resized to 690, 900 and 1100 px. Each run was driven under the debugger with a breakpoint on the paint pass, ignored on every crossing and counted at the end.

reading	before, Qt / React	after, Qt / React
seven sub-panes at a 690 px host	54.0 px each, the price pane 220, no scroll	84.0 px each, the price pane 220, the chart 900 tall, the bar's range 210
at 900	60.0 each, 388	84.0 each, 220, range 0
at 1100	60.0 each, 588	84.0 each, 420, range 0
the toggle row	under the chart	under the scroll host at the tab's foot: y 738 on Qt, 740.8 on the page, at 690
the wheel over the chart	zooms 100 to 85 candles	zooms 100 to 85; the bar stays at 0

reading	before, Qt / React	after, Qt / React
the stroke pens in device pixels, ratio 1.0	0.8, 1.0, 1.2, 1.4, 1.5, 1.7	1, 2
ratio 1.25	1.0, 1.25, 1.5, 1.75, 1.875, 2.12	1, 2
the fills at 100 candles	26 glyphs, 26 tags, 24 overlapping pairs	26 glyphs, 15 tags, 0 overlapping pairs
at 36 candles	5 glyphs, 5 tags, 0 pairs	5 glyphs, 4 tags, 0 pairs
the squat venue, Facebook 1200 by 630, eight voters	seven sub-panes at 40 px, the strip folded, the price pane 226	the same
the readable figure planted to 20	-	seven sub-panes at 20.0, the chart 452 tall; 84.0 restored
the tag rule planted off	-	26 tags, 24 overlapping pairs; 15 and 0 restored

The crops at 1.25, four times enlarged: before, the RSI line smeared over two to three device rows and the 30 and 70 rules over two faint rows; after, the line two device pixels and each rule one crisp dashed row, the label in its own row over the plot, the value tag legible. Renders sit under `artifacts/u55/C8/` before and after.

Four sentences the oscillators overtake

They were not reworded. They are quoted here.

"Each takes 60 px at the chart's natural height. When the window gives less, every sub-pane shrinks alike, down to 28 px, so the price pane keeps its 220 px and every pane stays on screen." - each takes 84 px on the tab at every window height, and the panel scrolls when the window gives less; only a fixed-height venue image shrinks a sub-pane, toward 28 px.

"`SUB_PANE_H = 60`" and "`SUB_PANE_MIN_H = 28`" - the two names are gone; the painter reads the surface's readable figure and its fold floor.

"The wick is a share of one candle column and never under one device pixel; the candle border is one device pixel; the indicator lines keep their logical width and stay anti-aliased. Read on a 1200 pixel image at ratio 1.0 the pens are 0.8, 1.0, 1.2, 1.4, 1.5 and 1.56 logical pixels; at ratio 2.0 the set gains 0.5, the one-device-pixel pen." - every stroke pen is now a whole number of device pixels; at ratio 1.0 the set is 1 and 2, at 1.25 it is 0.8 and 1.6 logical, which are 1 and 2 device pixels.

"The glyph sits on the candle whose interval holds the fill's stamp, one candle column wide and a twentieth of the price pane tall, with a tag naming the role and the price beside it." - the glyph holds; its tag draws only where it crosses no glyph and no other tag, and the readout names the fill under the pointer.

2026-09-23 17:30 - #55 - the oscillators are a third taller, and the bottom bar drags both heights

His words, 2026-09-23: *"Oscillator default height should be increased by 30% - Oscillator and chart height needs to be adjustable by dragging the bottom bar. Said adjustments must move all subsequent elements down and not overlap them"*.

The default height

A sub-pane's plot now holds seven tag-tall bands, not five: two over the upper ruled level, the upper level, the value tag at the middle, the lower level, and two under it. A tag is 14 px, so a sub-pane is 14 + 7 x 14 = 112 px. Read off the running chart before the change it was 84.0 px. His figure, 84 x 1.3, is 109.2 px, which is not a whole number of bands; seven bands is the least count that reaches it, and it gives the plot the same room above the upper level as below the lower one. The rise is 33.3%.

`src/gui/main_tabs/native_chart_surface.py` - the band count

```
#: The tag-tall bands a sub-pane's plot holds: two over the upper ruled level,
#: the upper level, the value tag at the middle, the lower level, two under the
#: lower level. Seven is the least count whose height clears 84 px by the 30%
#: the operator asked for: 14 + 7 x 14 = 112, a rise of 33.3%.
SUB_PANE_PLOT_BANDS = 7
```

The chart's natural height follows. With the volume strip and the seven sub-panes it is 28 + 18 + 220 + 28 + 7 x 112 + 18 = 1096 px, where it was 900. A panel shorter than that scrolls to it.

The bottom bar

Three short dashes draw across the chart's bottom edge, in a strip 8 px tall. Pressing in that strip and dragging down sets the chart's height, and every sub-pane keeps the share of the chart it holds at the natural height. One drag therefore raises the price pane and every oscillator together. Dragged up, the height stops at the natural 1096 and the sub-panes return to 112 px. The pointer turns to the vertical resize arrows over the strip on the Qt widget.

`src/gui/main_tabs/native_chart_surface.py` - the share the drag gives a sub-pane

```
def sub_pane_height(dragged_height_px: int, readable_height_px: int) -> int:
    scaled = SUB_PANE_READABLE_PX * int(dragged_height_px) // int(readable_height_px)
    return max(SUB_PANE_READABLE_PX, scaled)
```

`CandlestickChart.mouseMoveEvent` drives it on the widget, and `chart_view` drives it on the page from the same press, drag and release the page already sends. Both call `set_pane_drag_height`, so both builds answer one grab of the bar the same way.

Nothing below the drag overlaps

Inside the chart the panes are laid one under the next: each pane's top is the pane above's bottom, so a drag moves every pane under it down by the height it won. Outside the chart the panes sit in a scroll host, with the toggle row under the host, so the host's own height never changes and the toggle row never moves. Read on the running program at 700, 900 and 1400 px wide:

drag	sub-pane	price pane	vortex	rsi	time axis	chart bottom
at rest	112.0	46.0 - 266.0	294.0 - 406.0	966.0 - 1078.0	1078.0 - 1096.0	1096
+260	138.0	46.0 - 344.0	372.0 - 510.0	1200.0 - 1338.0	1338.0 - 1356.0	1356
+4000	520.0	46.0 - 1410.0	1438.0 - 1958.0	4558.0 - 5078.0	5078.0 - 5096.0	5096
dragged up	112.0	46.0 - 266.0	294.0 - 406.0	966.0 - 1078.0	1078.0 - 1096.0	1096

On the Qt panel at 700 px wide the scroll host reads 0.0 - 668.0 and the toggle row 674.0 - 686.0 at every one of those positions; at 900 the host reads 0.0 - 878.0 and the row 884.0 - 896.0; at 1400 the host reads 0.0 - 1078.0 and the row 1084.0 - 1096.0. On the page at 700 the chart mount reads 50.8 - 656.8 and the toggle row 660.6 - 674.6 at every position, and the image inside the mount grows from 1146.8 to 4002.8 while the mount scrolls.

The height and drag readings, off the running program, both builds

Each build ran with the home on a scratch directory and every socket but loopback refused, 100 A15 1h candles, seven sub-panes on, the panel at 700, 900 and 1400 px wide. The Qt numbers come from `ChartPainter.geometry_payload`, which reports what the last paint laid out; the page's numbers come from the same payload carried on the image answer, and its element rectangles from the page's own `getBoundingClientRect`.

reading	before, Qt / React	after, Qt / React
each sub-pane at rest, every width	84.0 / 84.0	112.0 / 112.0
the chart's natural height	900 / 900	1096 / 1096
the bar dragged +260	84.0, the chart 1160 / no answer, the image 900	138.0, the chart 1356 / 138.0, the image 1356
the bar dragged +4000	84.0, the chart 4900 / no answer, the image 900	520.0, the chart 5096 / 520.0, the image 5096
the bar dragged up	84.0, the chart 900 / 900	112.0, the chart 1096 / 1096
the panes' tops and bottoms, every position	each top the one above's bottom	the same
the toggle row against the scroll host	674.0 under 668.0 / 660.6 under 656.8	the same at every drag position
the band count planted back to 5	-	84.0 at rest and the natural height 900, in both builds; 112.0 restored

Renders sit under `artifacts/u55/C9/`, one per width and drag position in each build.

Three sentences the drag overtakes

They were not reworded. They are quoted here.

"A tag is 14 px, so a sub-pane is 14 + 5 x 14 = 84 px." - a sub-pane's plot holds seven bands, so a sub-pane is 14 + 7 x 14 = 112 px.

"With the volume strip and the seven sub-panes it is 28 + 18 + 220 + 28 + 7 x 84 + 18 = 900 px." - it is 28 + 18 + 220 + 28 + 7 x 112 + 18 = 1096 px.

"each takes 84 px on the tab at every window height" - each takes 112 px at every window height, and a drag of the bottom bar raises that figure with the chart.

2026-09-23 22:10 - #55 - the trade markers draw on every timeframe

His words, 2026-09-23: *"Historical trade markers ... do not consistently appear and I could only get them to do so on the weekly timeframe during the previous test."* The chart now draws a fill's glyph at 5m, 1h, 1d and 1w, and where a timeframe cannot reach a fill it says so in the header.

The window rule and what each timeframe reaches

One fetch asks for `FETCH_LIMIT` candles, and a fill can only be drawn on a candle the fetch brought back. The window a chart draws is therefore the candle count times the timeframe's own length, and a fill older than the first candle is counted, named in the header, and not drawn.

`FETCH_LIMIT` is 300. One request carries at most 300 candles on the Coinbase candles route, which the tree records as `RA_CHUNK_DAYS`, and the Advanced Trade route's own ceiling is 350. The lower of the two is the figure the chart asks for, so no fetch asks a route for more than it serves. The fetch is one request either way, because the count is a parameter of that one call.

`src/gui/main_tabs/trade_charts_tab_surface.py` - the one declaration

```
#: Candles one fetch asks for, and so the span a fill's glyph can land in;
#: 300 is the Coinbase candles route's one-request ceiling, recorded as
#: ``RA_CHUNK_DAYS``. ``src.gui.widgets.trade_charts_tab`` imports this name.
FETCH_LIMIT = 300
```

What 300 candles reach, from `TIMEFRAME_SECONDS`:

timeframe	window span	reaches a fill this old
5m	1.042 days	one day
1h	12.500 days	twelve and a half days
1d	300.000 days	ten months
1w	2100.000 days	five years nine months

A five-minute chart cannot reach a fill five months old. Reaching 152 days at 5m needs 44,011 candles, which is 147 requests and 0.014 px a candle in the pane; at 1h it needs 3,668 candles and 12 requests. The daily and the weekly reach every fill a bot of this fleet holds, and the two short timeframes draw what they reach and name the rest.

One declaration of the candle count

`FETCH_LIMIT` was declared in two files with the same value. Nothing compares the two, so the two copies could disagree without anything reporting it. The surface declares the name now, and the Qt tab imports it.

`src/gui/widgets/trade_charts_tab.py` - the import

```
from ..main_tabs.trade_charts_tab_surface import (
    FETCH_LIMIT,
    bot_timeframe,
    fill_record,
```

The header names the fills it could not draw

The paint pass counts each fill it refused, under the reason it refused it: older than the first candle, past the last one, or at a price outside the drawn scale. The header carries one phrase per reason that counted anything, beside the symbol, the timeframe, the source and the candle count. A chart with nothing refused carries none of them, and its picture is unchanged.

`src/gui/native_chart.py` - the three phrases

```
FILLS_OLDER_HEADER_FORMAT = "{count} {fills} older than this chart"
FILLS_NEWER_HEADER_FORMAT = "{count} {fills} newer than this chart"
FILLS_OFF_SCALE_HEADER_FORMAT = "{count} {fills} off the price scale"
```

`charts.annotations.drawn` carries the same counts split by reason: `fills_older_than_window`, `fills_newer_than_window` and `fills_off_price_scale`, with `fills_off_window` the sum of the first two. It previously folded a fill at a price off the drawn scale into `fills_off_window`, which named the window for an absence the window did not cause.

The marker readings per timeframe

One market, 37 fills fed, read off the running program in both builds at 700, 900 and 1400 px wide. The candles come from a read-only copy of one five-minute stone tablet on disk, folded to each timeframe by the registry's own rollup; the fills carry the fill ages of the running fleet, read from a read-only copy of its state file. No venue was contacted, every socket but loopback was refused, and no bot was started and no order placed, priced or cancelled.

timeframe	candles before	candles after	window before	window after	drawn before	drawn after	header after
5m	100	300	0.347 days	1.042 days	**0**	**6**	31 fills older than this chart
1h	100	300	4.167 days	12.500 days	12	**18**	19 fills older than this chart
1d	100	213	100.0 days	213.0 days	31	**37**	none
1w	31	31	217.0 days	217.0 days	37	37	none

The daily and the weekly answered fewer candles than the 300 asked for, because the tablet on disk holds 212.771 days; a venue serving the whole 300 reaches 300 days at 1d.

Every marker sat on the candle holding its own stamp at every timeframe and every width, with no fault in any reading. One fill planted at 400 days, outside every window including the weekly's, raised every timeframe's refused count by exactly one and turned the daily and the weekly from no phrase to `1 fill older than this chart`.

Read off the saved renders rather than off the painter: at the weekly, before and after differ by **0 pixels** in both builds and in the image the page is handed, and the planted phrase differs by 720 pixels inside rows 14 to 22, the header's own band. At 700 px wide the longest header this draws ends 89 px clear of the right edge, so no phrase is clipped at the narrowest width.

A paint pass costs 40.58 ms at 300 candles against 27.64 ms at 100, at 700 px; 28.10 against 15.82 at 900; 30.51 against 19.43 at 1400. The tick that redraws the chart runs every two seconds.

Renders sit under `artifacts/u55/C11a/`, one per build, stage, width and timeframe.

Sentences the window rule overtakes

They were not reworded. They are quoted here.

"A fill whose stamp falls before the window or past its end is counted and not drawn." - it is counted, named in the header under the reason it was refused, and not drawn.

"A row outside the window is counted and not drawn." - a row outside the window is counted, named in the header, and not drawn.

"`charts.annotations.drawn` is written when the set of annotations the paint pass drew changes, with the counts drawn against the counts fed that fall inside the pane, and the fills fed and the fills off the window in its context." - the context now carries the fills fed, the fills off the window, and that figure split into the fills older than the window, the fills newer than it and the fills at a price off the drawn scale.

2026-10-16 - #230 #223 - one unreadable price costs one candle, and a stored flag never prints a price

The admission rule

Every price the chart reads goes through one admission step before anything is drawn with it. `as_finite_float` admits a value only when its exact type is int or float and the number is finite, so a flag, a word, a missing value, a not-a-number, an infinity and an integer too large to hold are all refused. A refused candle is left out of the series and counted. A refused price in the header is named rather than printed. Nothing refused is ever turned into a zero.

Where the admission sits

`src/gui/main_tabs/native_chart_surface.py`, in `admit_candles`

```
def admit_candles(candles: Sequence[Any]) -> tuple:
    kept = [one for one in candles if candle_prices(one) is not None]
    return kept, len(candles) - len(kept)
```

`candle_prices` reads a candle's open, high, low and close through `as_finite_float` and answers None when any one of them is refused. Both builds take their series through `admit_candles`: the Qt widget in `src/gui/native_chart.py`, in `ChartPainter.set_candles`, and the React page through `src/gui/main_tabs/native_chart_surface.py`, in `ChartModel.set_candles`. `price_range` admits again before it fits the axis, so a caller that reaches it directly cannot hand it a price it would refuse.

`price_range` answers None when no candle carries a readable high and low. The Qt widget then draws `NO_PRICE_RANGE_TEXT` through `_paint_no_range`, and `build_view_model` publishes the same phrase as the chart's empty message.

`row_prices` admits one venue row for the React page's feed in `src/gui/react_charts_tab.py`, in `_feed_painter`, so one unreadable row costs that row alone. A row too short to carry the four prices is refused with the rest. `_feed_painter` counts the rows it dropped and hands that count to `set_candles` as `refused`, which adds it to the candles `admit_candles` refuses again, so the count line names every candle missing from the picture. A series that arrives empty rewrites the count line rather than leaving the last one standing.

What the chart says about a candle it could not read

`candle_count_text` writes the count line. A series taken whole reads exactly as it always has.

`src/gui/main_tabs/native_chart_surface.py` - the three phrases

```
HEADER_COUNT_FORMAT = "{count} candles"
REFUSED_COUNT_FORMAT = "{count} candles, {refused} unreadable"
ALL_CANDLES_REFUSED_TEXT = "No candle carried a readable price"
```

`build_view_model` publishes the same figure as `refused_candles`, beside `candle_count`.

What the header says about a price it could not read

`src/gui/main_tabs/trade_charts_tab_surface.py`, in `header_label`, is the one header step both builds call: the Qt tab from `src/gui/widgets/trade_charts_tab.py`, in `_label_current`, and the React page from `_label_shown` in the surface. It answers three different things.

the saved price	the header
no price saved yet	the symbol alone, as it stands
a number `as_finite_float` admits	`{symbol} • ${price:.8f} • {state}`
anything `as_finite_float` refuses	`{symbol} • price unreadable • {state}`

A saved zero is a number, so it draws as `$0.00000000`. A price that was never saved draws the symbol alone. The two are no longer the same picture.

`PRICE_LABEL_FORMAT` was declared twice, once in the surface and once in `src/gui/widgets/trade_charts_tab.py`. The tab's copy is gone and both builds read the surface's.

The header step runs after the draw

`update_charts` places the header step below the candle draw in both builds: after `_draw_history` and `_decorate_current` in the Qt tab, and after `feed_history` and `_feed_shown` in the surface. A header the tab cannot write costs the operator nothing he can see.

The readings, off the running program, both builds

One chart built inside a shown parent, fed 30 candles, painted over a fixed 900 by 700 area. No venue was contacted, no bot was started, and no order was placed, priced, previewed or cancelled. The runtime tree was read with a redirected home.

A clean series: 42638 drawn pixels, `30 candles`, price axis 98.34 to 116.16. Before and after differ by **0 pixels** across all 630000 of them, and every published reading is unchanged.

One candle of thirty carrying an unreadable high:

the value in one candle's high	candles drawn before	candles drawn after	the chart before	the chart after
a text number	0	**29**	113 px of 42638	`29 candles, 1 unreadable`
a word	0	**29**	113 px of 42638	`29 candles, 1 unreadable`
a stored flag	30 at $1.00	**29**	a wrong scale, drawn	`29 candles, 1 unreadable`

the value in one candle's high	candles drawn before	candles drawn after	the chart before	the chart after
a missing price	0	**29**	113 px of 42638	`29 candles, 1 unreadable`
an integer too large to hold	0	**29**	113 px of 42638	`29 candles, 1 unreadable`
a not-a-number	0	**29**	895 px of 42638	`29 candles, 1 unreadable`
an infinity	0	**29**	113 px of 42638	`29 candles, 1 unreadable`
a zero	30	30	drawn	drawn, a readable zero
a negative	30	30	drawn	drawn, a readable negative

Three of thirty refused reads `27 candles, 3 unreadable`, and thirty of thirty reads `No candle carried a readable price`. The two builds answer the same kept series candle for candle at every count.

The saved price in the header, read in both builds:

the saved price	the header before	the header after
a text number	`$123.45000000` in Qt, the draw lost in React	`price unreadable`
a word	the draw lost in both	`price unreadable`
a stored flag	`$1.00000000` in both	`price unreadable`
a missing price	the symbol alone	the symbol alone
an integer too large to hold	the draw lost in both	`price unreadable`
a not-a-number	`$nan` in Qt, the symbol alone in React	`price unreadable`
an infinity	`$inf` in both	`price unreadable`
a zero	the symbol alone	`$0.00000000`
a readable price	`$101.25000000`	`$101.25000000`

The two builds disagreed on three of those kinds before this entry and agree on all of them after.

Sentences the admission rule overtakes

They were not reworded. They are quoted here.

"The tab is a scroll area holding one panel per active bot." - the tab holds one panel and one shown market, stepped between two lists by the arrows.

"One method rebuilds that panel set from the current roster." - `update_charts` rebuilds the two market lists and redraws the one panel.

2026-10-16 - #230 #223 - the other five stored prices admit before the chart draws them

What the candle admission did not reach

The candle series and the header price were admitted first. Five more stored prices still reached the paint, or the row builder, with a bare conversion. Each is read off the running program below.

the stored price	before	after
a position price that is a word, a missing value, a huge integer or a not-a-number	232 px of 42638 drawn	42638, chart intact
a target anchor or ceiling that is a word or a huge integer	7397 px of 42638 drawn	42638, chart intact
a marker price that is a text number, a not-a-number or an infinity	a marker drawn at a nonsense price	dropped
a fed candle whose field is a word, a missing value or a huge integer	**0 of 30** rows reached the chart	**29 of 30**
a synthetic candle with the same	the whole push refused	29 of 30, `29 candles, 1 unreadable`

A text number, a flag and a not-a-number in a fed candle built thirty rows before, carrying the wrong value into the picture.

Where each admission sits

`src/gui/native_chart.py`, in `set_positions`, keeps the positions whose price `as_finite_float` admits, so one refused position costs its own marker. `ChartModel.set_positions` in `src/gui/main_tabs/native_chart_surface.py` does the same for the React page.

`set_target_balance_lines` reads both prices through `as_finite_float` in both builds. A refused price becomes None, which is what None has always meant for that line: the line is hidden.

`set_trade_history_markers` in the widget and `set_markers` in the model read the marker price through `as_finite_float`. A marker without a readable price is dropped, as a marker without a positive price already was.

`src/gui/main_tabs/trade_charts_tab_surface.py`, in `candle_row`, answers None for a candle it cannot read, and `candle_rows` drops it and keeps the rest. The three feeds that built rows one by one call `candle_rows` now: the ATA-SMP call, the history feed and the synthetic push.

`src/gui/main_tabs/native_chart_surface.py`, in `synthetic_candle`, reads a candle by attribute or by key, so a candle built without a fetch is admitted the same way a fetched one is. `push_synthetic_candles` in `src/gui/widgets/trade_charts_tab.py` counts what it dropped and hands that count to `set_candles`, so the count line names it.

`CANDLE_FIELDS` and `CANDLE_PRICE_FIELDS` name the six fields and the four prices once, so the two builds cannot disagree on which fields are read.

The readings

One chart built inside a shown parent, fed thirty candles, painted over a fixed 900 by 700 area. A clean chart draws 42638 pixels; the reading's control reports 0 for a planted empty chart.

A clean series still draws identically: **0 pixels differ** across all 630000 of them against the tree before this work began, and every published reading is unchanged.

No venue was contacted, no bot was started, and no order was placed, priced, previewed or cancelled. The runtime tree was read with a redirected home.

Sentences the five admissions overtake

They were not reworded. They are quoted here.

"One candle as six numbers, from an `OHLCVCandle`, a dict or a six-number row." - `candle_row` answers six numbers for a candle it can read, and None for one it cannot, and `candle_rows` keeps those it could.

"Store the grid levels. Nothing paints them." - still true. The grid level list is written and read zero times, and `GridLine` is constructed zero times, in both builds. Layer 2 of #230 is unbuilt.

Layer 2 is built now, and the setter that sentence described is gone. The section below quotes what went with it.

The zero a bot carries before its first fetch

`src/trading/container/config.py` declares `current_price: float = 0.0`, so a bot that has not fetched a price yet reports zero rather than nothing. Printing that as a dollar figure would hand the operator a price he would believe.

`header_label` answers four different things, and the two builds answer the same.

the saved price	the header
no price saved at all	the symbol alone
a zero	`{symbol} • no price yet • {state}`
anything `as_finite_float` refuses, or a negative	`{symbol} • price unreadable • {state}`
a readable price above zero	`{symbol} • ${price:.8f} • {state}`

A negative reads as unreadable rather than as a price, because a market price below zero is not a price the chart can place.

Two sentences the four header answers overtake

They were not reworded. They are quoted here.

"A saved zero is a number, so it draws as `$0.00000000`." - a saved zero names `PRICE_NOT_YET_NOTE`, because the zero a bot carries before its first fetch is not a price it measured.

"| a readable zero | `$0.00000000` |" - a readable zero reads `no price yet`.

2026-10-19 - #230 - the header names the candle it left out, and the dead bullseye list goes

What the count line reached, and what the operator saw

`candle_count_text` wrote the refusal into the chart's status text, and the status text is only drawn when the chart holds no candles at all. A series with twenty-nine readable candles and one unreadable one therefore drew the plain count and said nothing about the candle missing from the picture. The refusal was counted, published and never painted.

`header_parts` is now the one step that writes the count piece, and it writes it through `candle_count_text`.

`src/gui/main_tabs/native_chart_surface.py`, in `header_parts`

```
  if candle_count:
      parts.append(candle_count_text(candle_count, refused_count))
```

`build_view_model` hands it `model.refused_candles`, and `src/gui/native_chart.py`, in `ChartPainter._draw_header`, calls `header_parts` instead of building the same list from its own format strings. The Qt header and the React page's `header_text` cannot drift apart on the count, because one function writes it for both.

What the widget says, read off the pixels

One chart rendered through its own `paintEvent` at 1000 by 630, offscreen, with the home directory redirected. Each figure below is the count of pixels that differ from the background gradient `paint_to` fills first. The drawn string is read by matching a candidate's own render against the header band's pixels, so the reading comes from the picture and not from the source.

the reading	before	after
a clean series of thirty readable candles	801085 px, `30 candles`	801085 px, `30 candles`
one candle's high is a text value	801904 px, `29 candles`	802148 px, `29 candles, 1 unreadable`
one candle's high is a missing value	801904 px, `29 candles`	802148 px, `29 candles, 1 unreadable`
one candle's high is a stored flag	801904 px, `29 candles`	802148 px, `29 candles, 1 unreadable`
one candle's high is a not-a-number	801904 px, `29 candles`	802148 px, `29 candles, 1 unreadable`
one candle's high is an endless value	801904 px, `29 candles`	802148 px, `29 candles, 1 unreadable`
one candle's high is a huge integer	801904 px, `29 candles`	802148 px, `29 candles, 1 unreadable`
every candle's high is a stored flag	1544 px, `No candle carried a readable price`	1544 px, the same

The match scores name the reading. `29 candles, 1 unreadable` scores 0.526 against the header before the change and **0.993** after it. A string the chart never drew scores 0.531, and a near miss of one digit scores 0.959, so a match above 0.98 is the string and nothing else is.

The positive control

The clean series draws **0 differing pixels** between the two trees, across all 630000 of them. A chart with nothing refused is the same picture it was.

The counter itself reports **0 px** for the background gradient read against itself and **1112 px** for a chart holding no candles, so it can report both a zero and a figure.

The bullseye list

`_bbullseye_data` was set to an empty list in `ChartPainter.__init__`, written nowhere else and read nowhere. `_draw_bullseye` draws the four BB Bullseye envelopes from `_bb_data`, the live Bollinger series, which is written in `_compute_indicators` and read in three places. The empty list is gone. The overlay, its key, its toggle and its four envelopes are untouched.

The grid line list stays, and here is why

`_grid_lines` is written by `set_grid_lines` and read zero times, and `GridLine` is constructed zero times. The React page's `GridLines` component draws `price_axis.grid.ticks`, the price axis ruling, not this list.

Removing it does not stop at the widget. `set_grid_lines` is named in `STEP_NAMES`, `grid_lines_held` is published by `build_view_model`, and `src/gui/web/native_chart.js` names `grid_lines_held` in `DECLARED_FIELDS`. Taking the list out takes a field out of the payload the Electron renderer is handed, which is a published contract rather than a price admission. That row stays open.

Readings this entry declares

The run named the host's own Consolas and Segoe UI font files to the Qt font database. The offscreen platform ships no font family at all, so without them every glyph renders as a box and the pixels carry a string's length rather than its letters. `pip install -e ".[dev]"` installs neither.

The match reading holds only where the band it reads is sparse. On a full chart the header band carries 820 ink pixels and a candidate either fits or does not. On a band crowded with candles every candidate fits, so the body message is read on a chart that drew no candles at all.

No venue was contacted, no bot was started, and no order was placed, priced, previewed or cancelled. The runtime tree was read with a redirected home.

A sentence this entry overtakes

It was not reworded. It is quoted here.

"Layer 2 of #230 is unbuilt." - the bullseye list is removed and the grid line list stays, named above with the published fields that hold it.

2026-10-20 - #230 - the grid line list, its setter and its published field go

What went, and what read it first

Nothing read the list. `_grid_lines` carried two writes and no reads. `grid_lines` on the page's model carried two writes and one read, and that read took the list's own length for `grid_lines_held`. `set_grid_lines` had no caller in either build, and `GridLine` was constructed nowhere.

The list, the setter, the record type and the published field are gone from both builds. `native_chart.js` no longer names `grid_lines_held` among its declared fields.

Why both sides had to change in one step

`native_chart.js`, in `checkFields`, walks every declared field and raises a missing-field fault on one the frame does not carry. Driven in node against the real served frame, the page before this change reports no fault on the frame as served and **one** fault, `grid_lines_held: missing`, on a frame with the field stripped. The page after this change reports no fault either way. Removing the field from the surface alone would have faulted on every frame.

What the operator sees, read off the pixels

One chart rendered through its own `paintEvent` at 1000 by 630, offscreen, with the home directory redirected. A clean series of thirty readable candles draws **0 differing pixels** across all 630000 of them, with 298608 ink pixels before and after.

the control	pixels differing
the same series rendered twice	0
a chart holding no candles	583542
twenty candles instead of thirty	77705
a different symbol in the header	1181

The offscreen platform reports no font family, so the run names six of the host's own font files to the Qt font database and the database then reports five families. The different-symbol row is what proves the comparison reads letters and not boxes.

What the frame carries

One real request answered by `serve`, over the real registry, written by `encode_frame`. The frame still says ok and still ends in one newline. It carried 47 fields and now carries 46. `grid_lines_held` is the only field gone, no field arrived, and no other field's value moved.

The ruling stays

The dotted price ruling is not this list. `src/gui/native_chart.py`, in `paint_to`, steps it from the fitted price range, and `src/gui/main_tabs/native_chart_surface.py`, in `price_grid`, answers the same steps for the page, which draws them in `src/gui/web/native_chart.js`, in `GridLines`. The page reads seven ticks at a step of five before and after.

Readings this removal declares

`STEP_NAMES` in `src/gui/main_tabs/native_chart_surface.py` is read zero times. Only the one entry naming the deleted setter went with it.

`refused_candles` is in the frame and is not in the page's declared fields, so `checkFields` does not guard it.

The docs archetype's hallucination rule checks a cited path, never a cited symbol. A page citing four symbols absent from disk reports no hallucination finding while its paths resolve.

No venue was contacted, no bot was started, and no order was placed, priced, previewed, amended or cancelled.

Two rows and two sentences this entry overtakes

They were not reworded. They are quoted here.

"| `GridLine` | A buy or sell level |" - the buy and sell levels the chart draws are `PositionMarker` records, placed by `set_positions`.

"| `set_grid_lines` | The buy and sell levels |" - `set_grid_lines` had no caller in either build.

"That row stays open." - the operator authorised the removal, and the list, its setter and the published field went together.

"the bullseye list is removed and the grid line list stays, named above with the published fields that hold it." - the grid line list is removed too, and `grid_lines_held` is gone from the frame and from the declared fields.

History Tab

Reference. Trade history pulled from the venues, paired with the platform's own trading logic and graded.

What builds it

One builder makes the tab, adds it to the row, and hands it the bot manager.

`src/gui/main_tabs/history_tab.py` — `HistoryTabMixin._build_history_tab`

```
def _build_history_tab(self) -> None:
    """Build the History tab and add it to the main tab widget."""
```

Fetching

Refresh starts an asynchronous fetch that asks every active exchange for the operator's own fills. The helper walks each exchange and symbol pair the bot manager exposes, and turns each venue trade into one row.

`src/exchange/history_helpers.py` — `fetch_all_history_chunked`

```
async def fetch_all_history_chunked(
```

The exchange is the authority here. The rows on screen are the venue's answer, and the platform's own log joins to them rather than replacing them.

The tab refreshes itself on activation when the last fetch is older than five minutes and no fetch is already in flight.

`src/gui/main_window.py` — `_on_main_tab_changed`

```
is_stale = last_ts == 0 or _t.time() - last_ts > 300
if is_stale and not in_flight:
    hist.refresh()
```

The venue call queue a refresh shares

One connector serves every bot on a venue and runs a single worker, so every caller on that venue shares one call queue eight deep. A refresh asks for one symbol at a time, and before each ask it waits for that queue to empty. The balance, ticker, portfolio and OHLCV reads the running platform polls therefore take the worker first. Past its budget the ask goes through behind two reserved slots instead, so a refresh is never the submission that fills the queue, and a busy venue changes when a refresh runs and never which rows it answers.

`src/exchange/ccxt_connector.py` — `CCXTConnector.await_bulk_read_slot`

```
    if depth <= BULK_READ_ADMIT_DEPTH:
        break
    if time.monotonic() - start >= budget_sec and depth <= admit_past_budget:
        break
    await asyncio.sleep(BULK_READ_WAIT_SLICE_SEC)
```

`src/exchange/history_helpers.py` — `_await_slot`

Measured on a fleet of thirty-nine bots on one venue, with two hundred platform reads issued while a refresh ran: forty-seven of those reads were refused at the queue's cap while the refresh held the worker back to back, and six once the refresh waited its turn. The same two hundred reads with no refresh running at all were refused five times, so the refresh now costs the platform about one read where it used to cost forty-two. Both runs returned the same 8,558 rows.

A refusal at the cap costs a symbol its rows, so the wait never ends while the queue is full and one refused submission earns a second wait.

`src/exchange/history_helpers.py` — `BULK_READ_ATTEMPTS`

```
    BULK_READ_ATTEMPTS = 2
```

A venue that does not serialise its calls needs no turn-taking, and the default waits for nothing.

`src/exchange/base.py` — `ExchangeInterface.await_bulk_read_slot`

```
    del budget_sec
    return 0.0
```

Where the trade record is kept

Refresh reads the venue live and keeps nothing. The year of trades the Simulator needs is kept separately, on disk, under the exchange history bucket. One file holds one exchange, one symbol and one year, and every file names its exchange inside itself.

`src/core/log_paths.py` — the bucket

```
    def get_exchange_history_dir() -> Path:
        """``exchange_history/`` bucket — the YTD trade files.
```

The files are filled from a transactions CSV exported from the exchange. The import keeps buys and sells, counts and drops everything that is not a trade, and refuses a file that is missing a column it needs. Re-importing an overlapping export adds only the rows whose id is new.

`src/exchange/ytd_csv_import.py` — the import

```
def import_ytd_csv(
    csv_path: Path,
    exchange_id: str,
    root: Optional[Path] = None,
) -> ImportResult:
    """Read ``csv_path`` and write ``exchange_id``'s trade files under
    ``get_ytd_root(root)``."""
```

Where the export covers a period and carries no trade for a symbol in it, that period is written to a gap record rather than filled. See the Simulator tab for the file format and the reader.

The import reads the export through a column map chosen by the exchange id it is given. One map exists, for Coinbase, and a Coinbase export writes the same files it always did. An exchange with no map is refused by name, with a line saying a sample export is needed, and nothing is written. The Simulator tab says how the Simulator reads what the map writes.

`src/exchange/ytd_csv_import.py` — the map chosen at import

```
def import_ytd_csv(
    csv_path: Path,
    exchange_id: str,
    root: Optional[Path] = None,
) -> ImportResult:
    """Read ``csv_path`` and write ``exchange_id``'s trade files under
    ``get_ytd_root(root)``."""
    column_map = export_map_for(exchange_id)
```

Filtering, paging, export

Eight methods carry the controls under the table.

Method	What it does
`_populate_filter_options`	Fills the exchange, symbol and side pickers from the rows in hand
`_apply_filters`	Narrows the set
`_reset_filters`	Restores the full set
`_current_filters`	Reports the active state
`_remember_to_bound`, `_to_bound_ts`	Hold the upper date bound across a refresh
`_render_page`, `_prev_page`, `_next_page`	Page the result
`_export_csv`	Writes the current selection out

Where the cell values come from

One function returns the rendered page: the rows plus the pager state. It is the only disk read on that path.

`src/exchange/history_read_contract.py` — `build_page`

```
def build_page(
    filtered: list[dict],
    page: int = 0,
    bot_manager: Any = None,
    gate_index: Optional[dict] = None,
    voting_index: Optional[dict] = None,
) -> HistoryPage:
    """One rendered page: the ``HistoryRow`` list plus the pager state.

    ``build_join_indexes`` runs when neither index is supplied, and it is the
    only disk read on this path.
    """
```

One sentence above is overtaken. Quoted whole:

> *One function returns the rendered page: the rows plus the pager state. It is the only disk read on that path.*

The page builder still returns the rows and the pager state. It makes no disk read when the tab draws, because the tab hands it both join indexes. The tab reads the logs once per fetch, on its own thread, and the page builder's own read only runs for a caller that supplies neither index.

Now: the tab reads the logs once per fetch, in steps of about 8 ms on the window thread, after the first page draws (`src/gui/history_tab.py, in join_steps`).

Thirteen columns each yield one cell carrying a value, its text, its colour and its tooltip.

`src/exchange/history_read_contract.py` — `COLUMNS`, the first ten

```
COLUMNS: tuple[HistoryColumn, ...] = (
    HistoryColumn(0, "timestamp", "Timestamp (UTC)"),
    HistoryColumn(1, "exchange", "Exchange"),
    HistoryColumn(2, "symbol", "Symbol"),
    HistoryColumn(3, "bot", "Bot"),
    HistoryColumn(4, "side", "Side"),
    HistoryColumn(5, "amount", "Amount"),
    HistoryColumn(6, "price", "Price"),
    HistoryColumn(7, "cost", "Cost USD"),
    HistoryColumn(8, "fee", "Fee"),
    HistoryColumn(9, "trade_id", "Trade ID"),
```

Grade, Gates and Voting close the set. The ordering is fixed newest-first and nothing re-sorts.

`src/exchange/history_read_contract.py` — `ROW_ORDER`

```
ROW_ORDER = "timestamp_desc"
"""Newest first. ``fetch_all_history_chunked`` sorts ``reverse=True``,
``HistoryTab`` calls no ``setSortingEnabled``, and nothing re-sorts."""
```

The table renders those fields and derives none of them. A cost, a grade, a colour or a gate light on screen is always the contract's answer, which is what keeps a second implementation of History from growing behind the renderer.

The footer's two strings follow the same rule. The contract declares the page counter and the summary line, and the tab calls the contract for both, so one arithmetic produces each string for every surface that draws it.

`src/exchange/history_read_contract.py` — `page_label`, the counter the tab calls

```
def page_label(page: int, total: int) -> str:
    """The footer's page counter, or "No matches" when ``total`` is zero."""
    if total == 0:
        return "No matches"
    page = clamp_page(page, total)
    return f"Page {page + 1} / {page_count(total)} ({total} trades)"
```

Grading

One record and its price context go in, and an unweighted mean comes out. An axis with no input is skipped, so it neither credits nor penalises.

`src/trading/trade_grader.py` — `grade_trade`

```
def grade_trade(record: TradeRecord, ctx: PriceContext) -> TradeGrade:
    """Grade one ``record`` against its ``ctx``.

    ``TradeGrade.overall_numeric`` is the unweighted mean of the sub-scores
    whose inputs were present, or 0.5 when no axis could be scored.
    """
    exec_score, exec_bps = _score_execution(record, ctx)
    timing_score, mfe, mae = _score_timing(record, ctx)
    strategic_score, sb_delta = _score_strategic(record, ctx)
    outcome_score, realized = _score_outcome(record, ctx)
```

The mean becomes a letter at fixed boundaries.

`src/trading/trade_grader.py` — `_letter_from_numeric`

```
def _letter_from_numeric(num: float) -> str:
    """Map ``num`` in [0.0, 1.0] to ``A+``, ``A``, ``B``, ``C``, ``D`` or ``F``."""
    if num >= 0.93:
        return "A+"
    elif num >= 0.85:
        return "A"
    elif num >= 0.70:
        return "B"
    elif num >= 0.55:
        return "C"
    elif num >= 0.40:
        return "D"
    else:
        return "F"
```

`grade_trades` runs the batch. The regime tag reaches the grade's regime and its rationale without a sub-score of its own.

Gate analysis

Four helpers in `src/exchange/history_helpers.py` join the gate log to a trade.

Helper	What it does
`build_page_gate_index`	Buckets the gate log entries by bot id and minute
`lookup_gate_entry`	Picks the closest entry to a trade
`gate_cell_text`	Renders the cell text
`gate_cell_tooltip`	Renders its tooltip

The lights themselves come from one vocabulary. Nineteen of them, the scrum bank then the fold, each carrying its bank marker, its label, its state and its colour. No Qt type crosses that boundary.

`src/trading/gate_vocabulary.py` — `gate_light_row`

```
def gate_light_row(
    scrum_armed: bool,
    fold_armed: bool,
    scrum_blockers: Optional[list] = None,
    fold_blockers: Optional[list] = None,
    landing_strip_side: str = "",
    evaluated: bool = True,
) -> list[dict]:
    """The nineteen lights, in draw order: the scrum bank then the fold.

    Each entry carries ``bank`` ("S" or "F"), ``label``, ``state`` and
    ``color``. Serialisable: no Qt type crosses this boundary.
    """
```

Five states, five colours.

`src/trading/gate_vocabulary.py` — `LIGHT_COLORS`

```
LIGHT_COLORS: dict[str, str] = {
    "override": "#22d3ee",
    "not_evaluated": "#333340",
    "blocked": "#ff3366",
    "passed": "#00cc55",
    "not_the_blocker": "#c8901e",
}
```

That vocabulary is shared on purpose. The Simulator's gate cell and this table read the same labels and the same blocker prefixes, so a gate added on one surface cannot be missing from the other. That shared seam is what makes the Simulator's gate comparison meaningful.

Each trade is joined to its gate entry by `lookup_gate_entry` in `src/exchange/history_helpers.py`. One threshold decides what counts as a match, and a trade with no entry inside it reads "no record" in the cell.

`src/exchange/history_helpers.py` — the join threshold

```
JOIN_TOLERANCE_SECONDS = 60.0
"""Largest gap ``_best_entry_within_window`` accepts between a trade and a log
entry, and the backdate the index builders apply to ``since``."""
```

Nothing measures coverage across a window.

`build_page_voting_index` and `lookup_voting_entry` do the same for the indicator voting snapshot behind each trade.

When the logs are read

The tab reads the gate log and the voting log once, when a fetch lands, on a thread of its own. Drawing a page and pressing Prev or Next read the indexes already in hand and touch no file.

Now: the read waits until the first page draws, then runs in steps of about 8 ms on the window thread (`src/gui/history_tab.py, in HistoryTab._start_join_index_build`).

`src/gui/history_tab.py` — `HistoryTab._start_join_index_build`

```
def _start_join_index_build(self) -> None:
    """Read the gate and voting indexes for ``_all_trades`` off this thread.

    ``_render_page`` then reads the held indexes, so no page draw and no
    pager press touches disk; the newest build wins and an older
    build's answer is dropped.
    """
```

The read keeps only the minutes a cell can ask for. A lookup reads a trade's own minute and the two beside it, so an entry in any other minute can never reach a cell and is not held.

`src/gui/history_tab.py` — `join_bucket_keys`

```
def join_bucket_keys(rows: list, bot_manager) -> set:
    """The ``(bot_id, minute)`` keys the gate and voting lookups can reach.

    ``lookup_gate_entry`` reads a row's own minute and the two beside it, so an
    index holding only these keys answers every lookup identically.
    """
```

Measured on the operator's own logs, with a six-megabyte active gate log, five rotated copies of fifty-two megabytes each and a sixteen-megabyte voting log: 3,494 rows over 35 pages. The gate read kept 2,895 minutes and the voting read 2,955, against 167,944 gate entries parsed before. The longest stretch in which the window could not repaint fell from 4.2 seconds to 0.06, which is what an idle window reads on the same instrument. Every Gates cell and every light row on four sampled pages matched the earlier values exactly.

Nothing paces the fetch itself. One tab activation asks the venue for fills once per bot symbol, and those calls share one worker thread and a queue of eight with every other venue reader on the platform.

`src/exchange/ccxt_connector.py` — the shared queue

```
SYNC_QUEUE_CAP: int = 8
```

The renderer

`HistoryWebTable` in `src/gui/react_history_panel.py` draws the table with React inside the Chromium PySide6 ships, and sits where the Qt table used to. One History tab, one renderer, one set of rows.

The Qt table takes no theme. The React table takes one because it builds its page in that theme, and the Qt table's chrome comes from the application stylesheet instead, so a theme handed to it reached no colour. The tab passes neither.

`src/gui/history_qt_table.py` — `HistoryQtTable`, what construction takes

```
        def __init__(self, parent=None) -> None:
            super().__init__(parent)
            self.setAccessibleName("Qt History Table")
            self._last_model: dict = {}
```

Driven with one row and two columns, 26 values read back off the real table under Cyberpunk Dark and then Neon Light: every colour and every string was identical and the only value that moved was the theme name the table had stored. The React table, driven the same two ways, moves all six of its page colours.

The Simulator front-load

Refreshing History hands the Simulator the year of live trades it needs before a parity run. The tab's signal connects to the Simulator's panel, and the builder makes that connection once both tabs exist.

`src/gui/history_tab.py` — `HistoryTab.history_refreshed`

```
history_refreshed = Signal(list)
```

Now: the signal is a `Signal(object)`, so it hands the rows over without converting them (`src/gui/history_tab.py, in HistoryTab.history_refreshed`).

Paper History

The manual's part list names a Paper History Tab beside this one. No such module exists. Issue #19 built the Paper tab and its log, so the ledger such a tab would read is on disk now, and no screen reads it. See paper-trader.md.

Bridge

Three methods serve this screen.

Bridge method	Serves
`history.view_model`	The whole screen, assembled from the read contract
`history_tab.chrome`	The surrounding controls
`react_history_panel.state`	The table

Three renderer files draw it: `history_tab.js` for the chrome, then `history_panel.js` and its stylesheet for the table.

2026-09-08 08:17 - no issue recorded - what the closed issues landed

The History Tab is able to pull trade history from all active exchanges via their respective APIs. It also pairs each imported trade with its in-platform trading logic and applies our trade grading system.

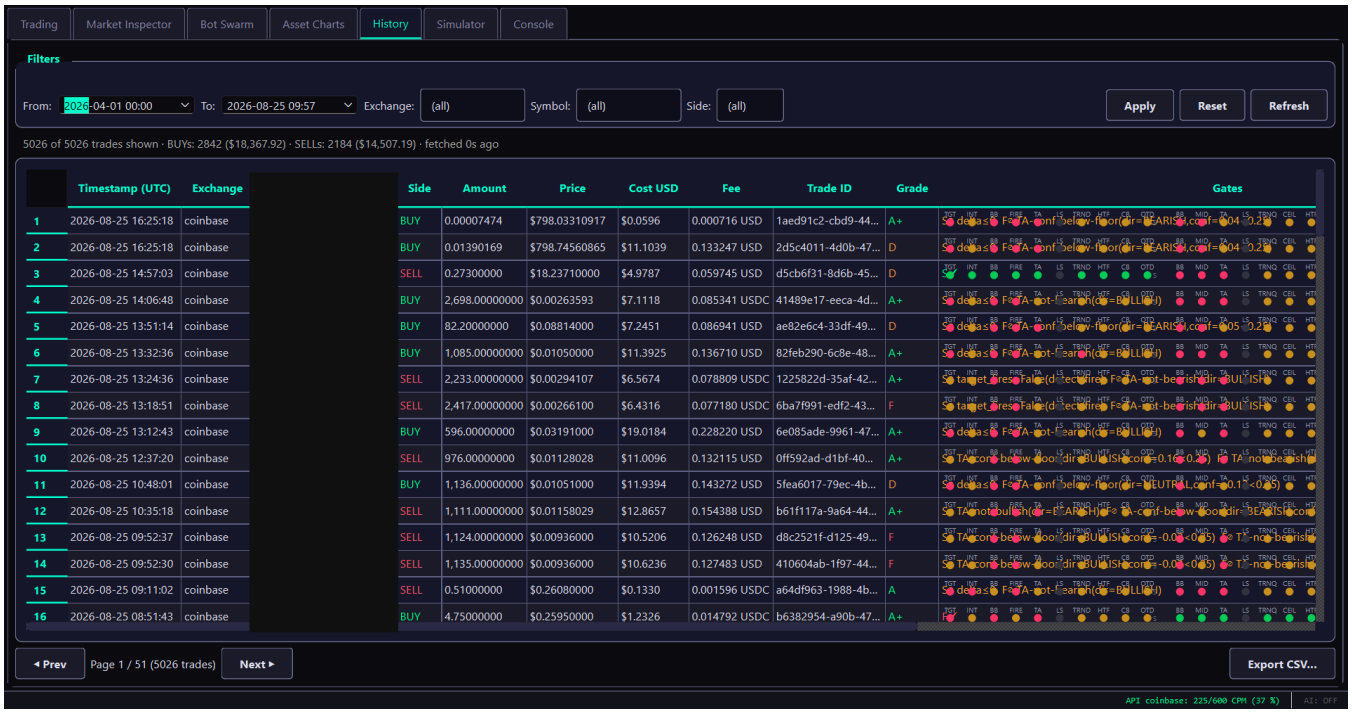
The tab keeps the name History. It sits eighth on the bar, on the gold ground with red text.

The tab fetches from every active venue and draws the rows with React. Every cell value comes from one read contract; the table renders those fields and derives none, which stops a second History growing behind the renderer.

`src/exchange/history_read_contract.py` — `build_page`

```
  def build_page(
      filtered: list[dict],
      page: int = 0,
      bot_manager: Any = None,
      gate_index: Optional[dict] = None,
      voting_index: Optional[dict] = None,
  ) -> HistoryPage:
```

The grade comes from the trade grader and the gate lights come from the same vocabulary the Simulator's gate cell reads. A refresh also hands the Simulator its year of live trades, so the tab's signal front-loads the parity run.



The History tab: filters, graded rows and the gate lights.

The Filters group carries From and To date pickers, then Exchange, Symbol and Side, each filled from the rows in hand. Apply narrows the set, Reset restores it, and Refresh goes back to the venues. The line under the group counts the retained rows against the loaded rows, both sides with their dollar totals, and the age of the last fetch.

Thirteen columns carry a row, newest first, and nothing re-sorts. BUY draws green and SELL red.

`src/exchange/history_read_contract.py` — `COLUMNS`, the first ten

```
COLUMNS: tuple[HistoryColumn, ...] = (
    HistoryColumn(0, "timestamp", "Timestamp (UTC)"),
    HistoryColumn(1, "exchange", "Exchange"),
    HistoryColumn(2, "symbol", "Symbol"),
    HistoryColumn(3, "bot", "Bot"),
    HistoryColumn(4, "side", "Side"),
    HistoryColumn(5, "amount", "Amount"),
    HistoryColumn(6, "price", "Price"),
    HistoryColumn(7, "cost", "Cost USD"),
    HistoryColumn(8, "fee", "Fee"),
    HistoryColumn(9, "trade_id", "Trade ID"),
```

Grade, Gates and Voting close the set.

The Gates cell opens with the blocker text, then the lights. Nineteen of them: the ten scrum gates TGT, INT, BB, FIRE, TA, LS, TRND, HTF, CB and OTD, then the nine fold gates BB, MID, TA, LS, TRNQ, CEIL, HTF, CB and OTD. Each light draws under its own label, and each bank ends with an S or an F marker. Five states, five colours.

`src/trading/gate_vocabulary.py` — `LIGHT_COLORS`

```
LIGHT_COLORS: dict[str, str] = {
    "override": "#22d3ee",
    "not_evaluated": "#333340",
    "blocked": "#ff3366",
    "passed": "#00cc55",
    "not_the_blocker": "#c8901e",
}
```

Green passed, red the gate that blocked, amber not armed for another reason, grey not evaluated at this candle, and cyan a landing-strip override. The blocker text and the light banks overlap in the figure; both occupy the one cell.

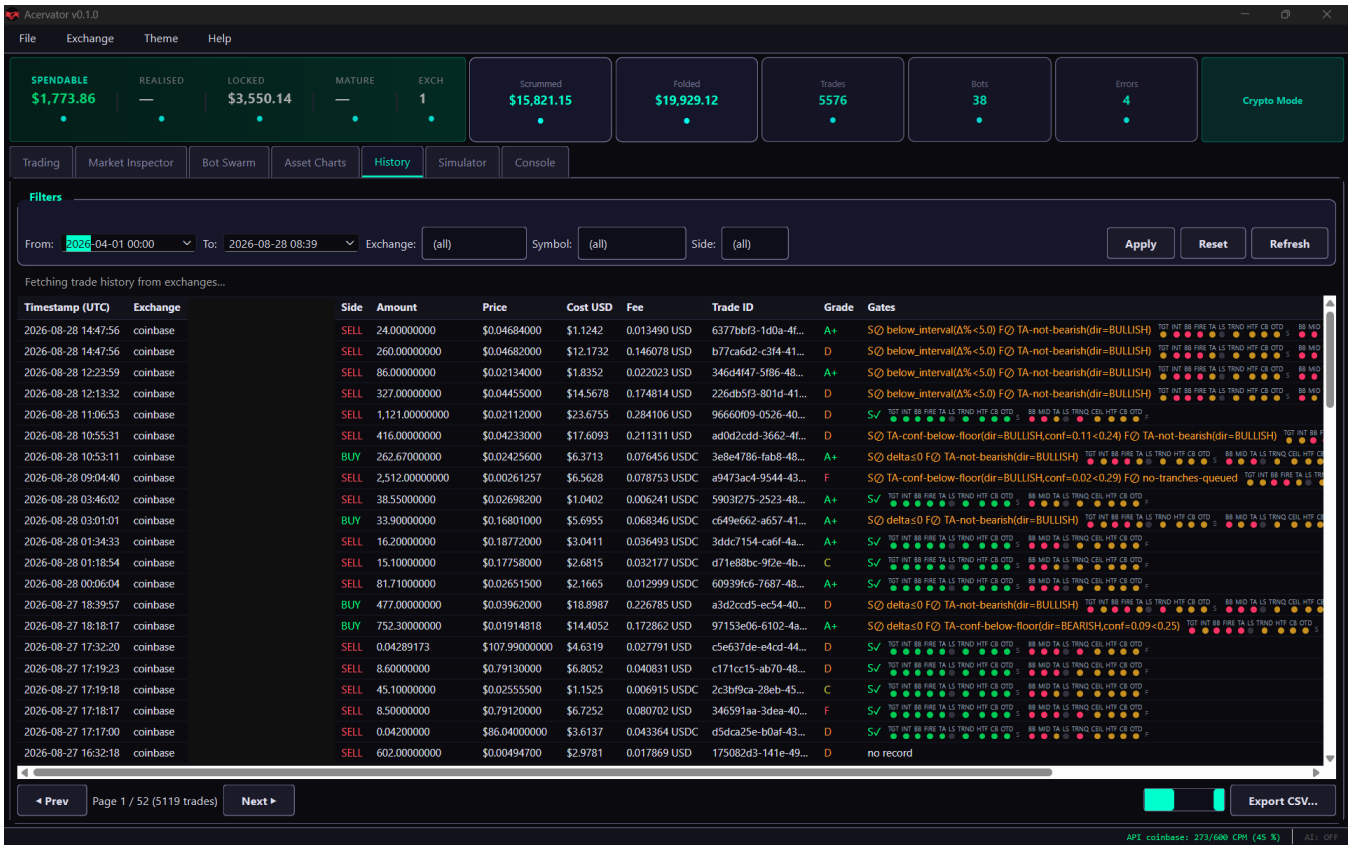
One sentence above is overtaken. Quoted whole:

> *The blocker text and the light banks overlap in the figure; both occupy the one cell.*

The blocker text and the light banks share the one cell and no longer overlap. The Qt table writes the text once, into the lights cell's own label, and leaves the table item under that cell blank. A row whose gate log holds no entry has no lights cell, and there the table item carries the `no record` text itself.

Prev, the page counter and Next page the result at the foot, and Export CSV writes the current selection out. The read contract declares both footer strings, and the tab calls it for both, so the counter and the summary read the same on this tab as on the React panel.

History Tab (React):



The same tab, captured while the asynchronous fetch runs. Five fixed status strings cover the states before any row exists, and the one on screen is the fetching entry.

`src/exchange/history_read_contract.py` — `STATUS_TEXT`

```
STATUS_TEXT = {
    "idle": "No history loaded yet — click Refresh.",
    "no_bot_manager": "Bot manager unavailable — cannot fetch history.",
    "no_async_loop": "Async loop not ready — try again after platform starts.",
    "fetching": "Fetching trade history from exchanges…",
    "timeout": "Fetch timeout (300s). Exchange may be rate-limited; try again.",
}
```

Two constants bound the fetch, in the same module: the poll runs on a 400 ms timer, so an observed latency is the true latency plus up to one interval, and the fetch is abandoned after three hundred seconds.

`src/exchange/history_read_contract.py` — the fetch bounds

```
FETCH_POLL_INTERVAL_S = 0.4
```

The header strip is visible here, which marks the screen as a live-trading tab. REALISED and MATURE draw the exchange figures the Live tab draws, which issue #418 fed. A row whose gate log holds no entry inside the join window reads `no record`.

Both columns draw an exchange figure here too, because one strip serves every tab that shows it. Where the venue has answered for no bot they stay empty, and Privacy Mode leaves an empty column empty.

Trade grading

The grader scores a trade that has already happened. It never feeds a trading decision, it never tunes a parameter, and no gate consults it. The same inputs always give the same grade.

Four axes are scored. An axis with no input is skipped: it neither credits nor penalises, and the overall number is the unweighted mean of the rest.

`src/trading/trade_grader.py` — `grade_trade`

```
exec_score, exec_bps = _score_execution(record, ctx)
timing_score, mfe, mae = _score_timing(record, ctx)
strategic_score, sb_delta = _score_strategic(record, ctx)
outcome_score, realized = _score_outcome(record, ctx)
```

Axis	What it reads	What a 1.0 means
Execution	fill price against the reference price at decision time	no slippage
Timing	the favourable excursion against the adverse one over the following candles	the trade went far more right than wrong before it closed
Strategic	the asset's discipline ratio before the trade against after it	the trade moved the ratio the operator's way
Outcome	realised profit per unit	a clear gain

The regime tag reaches the grade and the rationale string and gets no score of its own. Grading the same record with a regime tag and without one returns the same number, which is the control that proves it.

The scale is 0.0 to 1.0, not 0 to 100, and the mean becomes a letter at fixed boundaries.

`src/trading/trade_grader.py` — `_letter_from_numeric`

```
    if num >= 0.93:
        return "A+"
    elif num >= 0.85:
        return "A"
    elif num >= 0.70:
        return "B"
    elif num >= 0.55:
        return "C"
    elif num >= 0.40:
        return "D"
    else:
        return "F"
```

Driving the mapper across each boundary and just under it flips the letter exactly where the code says, in both directions.

One trap worth knowing. A trade with no gradeable input at all scores 0.5 and therefore reads as a **D**, not as ungraded. A fully specified trade scores 1.0 and reads A+. A column of Ds may mean the trades were poor, or it may mean the surrounding price context never arrived.

2026-10-06 - #1133 - the trade historian's own fetch

Refresh is not the only thing that asks the venue for fills. A launch also runs the trade historian, once for every market in the fleet, and the historian calls the venue directly rather than through the connector's paced read path.

`src/core/trade_historian.py` — the call

```
trades = self._exchange.fetch_my_trades(symbol, limit=limit) or []
```

The published limit

Coinbase publishes the limit for the private fills endpoint as ten requests per second per profile, with bursts up to twenty. One fetch every hundred milliseconds sits on that published figure, and the platform derives the interval from it rather than picking a number.

`src/core/trade_historian.py` — the interval

```
PUBLISHED_FILLS_RPS = 10.0
HISTORY_FETCH_INTERVAL_S = 1.0 / PUBLISHED_FILLS_RPS
```

The gate is one object for the whole program, so every market's fetch queues behind the last one. A second connector's threads wait their turn in the same queue, which is what keeps the request rate at the published figure no matter how many connectors a launch builds.

`src/core/trade_historian.py` — the gate

```
_FETCH_GATE = _HistoryFetchGate(HISTORY_FETCH_INTERVAL_S)
```

A refused fetch

A venue can refuse for reasons the platform does not control, so a refusal is tried again. The historian uses the retry the exchange paths already share: three attempts, doubling the wait each time, and only for the error classes that name a transient condition.

`src/core/trade_historian.py` — the attempt budget

```
HISTORY_FETCH_ATTEMPTS = 3
HISTORY_RETRY_BASE_DELAY_S = 1.0
```

A refusal is the venue answering normally, so it records nothing about the connection. Nothing on this path writes the lost-connection state that turns a venue's row red in Settings, and that state keeps its one writer in the credential validator.

`src/exchange/credential_state.py` — the lost-connection writer

```
def record_connection_lost(
    exchange_id: Any, when: Any = None, path: Any = None
) -> bool:
```

Once every attempt is spent the market gets no history, and the line above the table says so with the count.

`src/core/trade_historian.py` — the line

```
MISSING_HISTORY_FORMAT = (
    "Trade history missing for {refused} of {scanned} markets — "
    "the venue refused the fetch."
)
```

One sentence above is overtaken. Quoted whole:

> *The read contract declares both footer strings, and the tab calls it for both, so the counter and the summary read the same on this tab as on the React panel.*

The read contract still declares both footer strings and the tab still calls it for both. When a history fetch has been refused the Qt tab appends the missing-history line to the summary it got back, so on that one state the summary reads longer here than on the React panel.

`src/gui/history_tab.py` — the append

```
missing = self.missing_history_status()
self._set_status(f"{summary}  {missing}" if missing else summary)
```

Back to the subsystem index.

Console

Reference. A raw log tail in the upper pane and the emitter signal stream in the lower one.

What builds it

`ConsoleTabMixin._build_console_tab` in `src/gui/main_tabs/console_tab.py` builds both panes into a vertical splitter, weighted three to two, and starts three timers.

The builder does not hold the panes. It asks `variant_surface` for the tab class, takes the parts off it, and adds it to the tab row.

`src/gui/main_tabs/console_tab.py` — `ConsoleTabMixin._build_console_tab`

```
tab = surface_class(CONSOLE)()
self._console_tab = tab
self._console = tab.log_pane
self._signal_view = tab.signal_view
self._console_pause_btn = tab.pause_button
self._console_pause_indicator = tab.pause_indicator
```

Two classes answer that call. `ConsoleQtTab` builds the Qt panes, the control bar and the vertical splitter. `ConsoleReactTab` holds the same six parts and draws all of them in one web view. React is the running choice.

`src/gui/variant_surface.py` — the two loaders for the Console

```
def _qt_console() -> type:
    """Import and return the Qt Console tab."""
    from .qt_console_tab import ConsoleQtTab

    return ConsoleQtTab


def _react_console() -> type:
    """Import and return the React Console tab."""
    from .react_console_tab import ConsoleReactTab

    return ConsoleReactTab
```

`_start_console_timers` starts two timers, the signal drain and the console health. The pause refresh timer is built with the tab and started by the pause action.

`src/gui/main_tabs/console_tab.py` — `ConsoleTabMixin._start_console_timers`

```
self._signal_timer = QTimer(self)
self._signal_timer.setInterval(surface.DRAIN_INTERVAL_MS)
self._signal_timer.timeout.connect(self._drain_signals)
self._signal_timer.start()
```

Upper pane: the Python log

The upper pane is a read-only text view capped at two thousand blocks, which is what bounds its memory.

`src/gui/main_tabs/console_tab.py` — `ConsoleTabMixin._build_console_tab`

```
self._console = QPlainTextEdit()
self._console.setReadOnly(True)
self._console.setFont(QFont("Consolas", 9))
```

`ConsoleQtTab` builds that view. Under React the same calls land on `PagePane`, which holds the blocks and pushes them to the page.

The pushes of one event-loop turn coalesce into one. `redraw` starts a zero-interval single-shot timer and `_push_now` builds the payload and pushes it once when the timer fires, so a signals drain that appends 200 rows pushes the page once rather than 200 times. Measured on a Portfolio Battery walk, whose gate chain fires the TA engine's thirteen postcondition pins per evaluation into the drain: with a push per append the walk thread reached 5,000 bars in 257 seconds while the GUI thread rebuilt the page per row; with the pushes coalesced, 14 seconds.

`src/gui/react_console_tab.py` — the coalesced push

```
        def redraw(self) -> None:
            """Ask for one push of both panes on the next event-loop turn; the
            asks of one turn coalesce into one ``_push_now``."""
            if not self._redraw_timer.isActive():
                self._redraw_timer.start()
```

`src/gui/qt_console_tab.py` — `ConsoleQtTab.__init__`

```
self.log_pane = QPlainTextEdit()
self.log_pane.setReadOnly(True)
self.log_pane.setFont(
    QFont(surface.PANE_FONT_FAMILY, surface.PANE_FONT_POINT_SIZE)
)
self.log_pane.setMaximumBlockCount(surface.PANE_MAX_BLOCKS)
```

A log handler formats each record and hands the line to a relay. The relay carries it over a Qt signal to the thread that constructed it and paints it there, so a log call from any thread reaches the pane without touching the widget from that thread. Colour follows the level, and a message holding the phrase `INDICATOR PANEL` paints in the accent colour.

The handler attaches at two points, and both are needed because the logging engine clears propagation on the application logger.

`src/gui/main_tabs/console_tab.py` — `ConsoleTabMixin._build_console_tab`

```
logging.getLogger().addHandler(qt_handler)
logging.getLogger("acervator").addHandler(qt_handler)
```

The builder reads those two names from the surface and adds the tab's own handler to each.

`src/gui/main_tabs/console_tab.py` — `ConsoleTabMixin._build_console_tab`

```
for name in surface.HANDLER_LOGGERS:
    logging.getLogger(name).addHandler(tab.log_handler)
```

The handler level

The tab sets its own handler to debug and changes no logger's level. An earlier build raised the root logger and never restored it, which left the whole application logging at debug volume for the life of the process. Setting the handler instead keeps the verbosity local to this pane.

`src/gui/main_tabs/console_tab.py` — `ConsoleTabMixin._build_console_tab`

```
# Two disjoint attach points: `logging_engine` clears `acervator.propagate`.
for name in surface.HANDLER_LOGGERS:
    logging.getLogger(name).addHandler(tab.log_handler)
```

Each tab class sets that level on its own handler, from `HANDLER_LEVEL` in the surface.

`src/gui/qt_console_tab.py` — `ConsoleQtTab.__init__`

```
self.log_handler.setFormatter(
    logging.Formatter(surface.LOG_FORMAT, datefmt=surface.LOG_DATEFMT)
)
self.log_handler.setLevel(surface.HANDLER_LEVEL)
```

Pausing

Pause holds new lines in a bounded buffer. On resume the buffer replays and one notice names anything the cap dropped, so a gap is never silent.

`src/gui/main_tabs/console_log_handler.py` — `_QtLogHandler.set_paused`

```
def set_paused(self, paused: bool) -> None:
    """Hold new lines in ``_buffer`` while ``paused``, and drain them on resume.

    A drain that followed a ``_buffer_max`` overflow appends one notice line.
    """
```

`buffered_count` feeds the pause indicator, and Clear empties the pane.

Lower pane: the Emitter Network

The signal tail drains twice a second and renders the newest two hundred records. Each line carries a pass marker, a fail marker or a neutral one, then the signal name, the site, the actual value and the expected one. Records the slice drops are counted and announced with a gap marker drawn above the slice.

`src/gui/main_window.py` — `_drain_signals`

```
def _drain_signals(self) -> None:
    """Render sink records past `_signal_seq`, keeping the newest 200."""
```

The drain reads the process sink, the one `get_sink` answers on the GUI thread. A Simulator run routes its worker thread to the Simulator's own sink under the sim bucket, so the walk's pins, the TA engine's thirteen postcondition pins per evaluation among them, no longer reach this pane; the Simulator's own presses on the GUI thread and the one `sim.sink.routed` row each run leaves at its end do. See simulator.md.

`src/core/signal_contract.py` — the sink the drain reads

```
def get_sink() -> Optional[SignalSink]:
    """The sink the calling thread emits into: its `route_thread` sink when one
    is set, else the process sink."""
```

Seven counters ride along, published every five seconds: the sequence watermark, drain ticks, records read, records rendered, records the slice dropped, gap markers drawn, and health ticks seen. Ticks count invocations rather than records, which is what separates a quiet sink from a stopped timer.

`src/gui/main_window.py` — `_drain_signals`

```
# Counts invocations, not records: a quiet sink is not a stopped timer.
self._signal_drain_ticks = getattr(self, "_signal_drain_ticks", 0) + 1
```

Where the lower pane goes

The signal pane belongs to the Status tab once that screen exists. See system-status.md.

Bridge

Two methods serve this screen.

Bridge method	Serves	Renderer module
`console.tab`	Both panes	`console_tab.js`
`console.log_lines`	The log tail	`console_log.js`

The surface holds the two behaviours the tab owns rather than describes, so neither is written twice.

In the surface	Holds
`ConsolePane`	The same capped block buffer, at the same two-thousand-block limit
`SignalLedger`	The same seven counters
`format_record`	The same formatter the tab attaches to its handler

2026-09-07 15:58 - no issue recorded - ten single-word tabs, each on its own ground

This tab is focused on displaying Python activity and errors. The lower half, which is displaying the Emitter Network activity, will be migrated to the System Status Tab (under the Watchdog) which is to be built in the near future.

The tab keeps the name Console. It sits last on the bar, on the gold ground with red text.

The tab builds both panes into a vertical splitter. The upper one is a raw log tail: a handler formats each record and a relay paints it on the GUI thread, which lets a log call from any thread reach the pane safely.

`src/gui/qt_console_tab.py` — `ConsoleQtTab.__init__`

```
self.log_pane = QPlainTextEdit()
self.log_pane.setReadOnly(True)
self.log_pane.setFont(
    QFont(surface.PANE_FONT_FAMILY, surface.PANE_FONT_POINT_SIZE)
)
```

The tab sets its own handler to debug and changes no logger's level; an earlier build raised the root logger and never restored it.

`src/gui/qt_console_tab.py` — `ConsoleQtTab.__init__`

```
# The tab sets its own handler's level and leaves every logger level alone.
self.log_handler.setLevel(surface.HANDLER_LEVEL)
```

Pause holds lines in a bounded buffer and announces anything it had to drop.

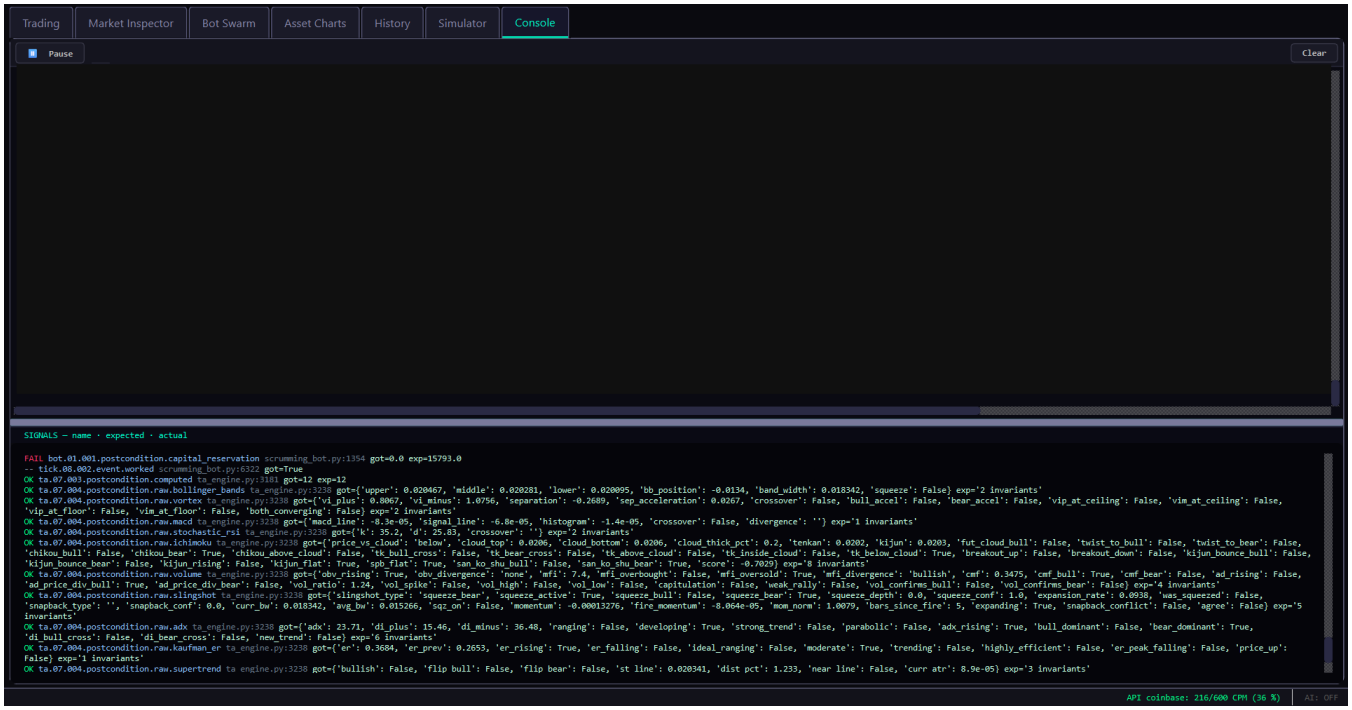
`src/gui/main_tabs/console_log_handler.py` — `_QtLogHandler.set_paused`

```
self._paused = bool(paused)
if not self._paused and self._buffer:
    for msg, r, g, b in self._buffer:
        self._append_signal.emit(msg, r, g, b)
    self._buffer.clear()
```

The lower pane is the Emitter Network. It reads the sink twice a second and renders the newest 200 records with their expected and actual values, keeping seven counters, which lets a quiet sink and a stopped timer read differently.

`src/gui/main_window.py` — `_drain_signals`

```
sink = get_sink()
if sink is None:
    return
new = sink.since(getattr(self, "_signal_seq", 0))
```



The Console tab: the log tail above, the Emitter Network below.

Pause and Clear sit above the upper pane. Pause holds lines in the bounded buffer and announces anything it had to drop when it resumes. Clear empties the pane.

Each line in the upper pane carries the time, the level, the logger name and the message, coloured by level. Three logger names appear in the figure.

React draws this tab. The choice is made in one place, under the screen name `CONSOLE`. A build stamped for Qt gets the pane set above. Every other build gets the web one, which holds the whole tab in a single view.

`src/gui/main_tabs/console_tab.py` — `ConsoleTabMixin._build_console_tab`

```
        from ..variant_surface import CONSOLE, surface_class

        tab = surface_class(CONSOLE)()
        self._console_tab = tab
        self._console = tab.log_pane
        self._signal_view = tab.signal_view
        self._console_pause_btn = tab.pause_button
        self._console_pause_indicator = tab.pause_indicator
```

Both tabs name the same six parts, so the pause, the drain and the health timer call one set of methods whichever side is built. The pause buffer, the level colours and the drop notice stay in the one handler both sides attach.

`src/gui/main_tabs/console_log_handler.py` — `_QtLogHandler.__init__`

```
    def __init__(self, text_edit, painter: Callable[[str, int, int, int], None] = None):
        """Paint into ``text_edit``, or into ``painter`` when one is given."""
```

A line reaches the page as markup the surface writes, with the message escaped, so a log message holding a tag is shown rather than read as one.

`src/gui/main_tabs/console_tab_surface.py` — `line_html`

```
def line_html(text: Any, red: int, green: int, blue: int) -> str:
    """One console line as the markup a pane paints it with.

    The text is escaped, so a message holding a tag is shown rather than
    read as markup.
    """
```

Logger	Writes
`acervator.scrumming`	The bot warnings
`acervator.api`	The venue calls
`acervator.gui`	The panel feed

Five levels each carry a colour, and the figure shows three of them: warning, info and debug, with debug the most muted of the three. A sixth colour follows the text of the message rather than its level, and it marks the indicator panel feed.

`src/gui/main_tabs/console_log_handler.py` — the level colours

```
COLORS = {
    "DEBUG": QColor(ds.TEXT_MUTED),
    "INFO": QColor(ds.TEXT_INACTIVE),
    "WARNING": QColor(ds.WARNING),
    "ERROR": QColor(ds.ERROR),
    "CRITICAL": QColor(ds.MAIN_LOG_CRITICAL),
}
```

Two of those lines are worth reading against the code. The repeated warning naming a capital-reservation over-commit comes from `src/trading/capital_reservation.py`, where a claim whose total would pass the holdings is refused and the existing claims, the request and the holdings are named. The `FETCH_OHLCV` line names a twelve-indicator engine and lists all twelve indicators, which is the number the voting engine builds and the number the Indicator Voting Panel shows.

The lower pane opens with the header `SIGNALS — name · expected · actual`. Each record draws a pass marker, a fail marker or a neutral one, then the signal name, the site that emitted it, the actual value and the expected one. A satisfied expectation is recorded the same as a violated one, which keeps a call site that never ran distinct from one that always passed.

`src/gui/main_window.py` — `_drain_signals`, the three markers

```
if r.ok is True:
    mark, colour = "OK  ", ds.SUCCESS
elif r.ok is False:
    mark, colour = "FAIL", ds.ERROR
else:
    mark, colour = "--  ", ds.STATUS_NEUTRAL
```

The site is the file and the line of the frame that made the call, read off the stack at the moment of emission. It moves whenever the code moves. The numbers in the figure are the numbers those call sites held on the day of the capture, not the numbers they hold now.

`src/core/signal_contract.py` — `_caller_site`

```
def _caller_site(depth: int = 2) -> str:
    """Return `file:line` of the calling frame at the given stack `depth`."""
    try:
        f = sys._getframe(depth)
        return f"{Path(f.f_code.co_filename).name}:{f.f_lineno}"
```

The two snapshot emitters

Two symmetric records go to the bot log, and the Console tab is where an operator reads them. A blocked trade and a fired trade write the same shape, so the log holds both halves of the decision and not only the half that acted.

`src/trading/scrumming/snapshots.py` — `SnapshotEmitterMixin._emit_risk_gate_snapshot`

```
f"RISK GATE SNAPSHOT [{side.upper()}] "
f"risk_blockers={risk_blockers_sorted} "
f"ticker_last={ticker_last:.6g} "
f"panel={snapshot}"
```

The fired-side companion carries the same prefix shape.

`src/trading/scrumming/snapshots.py` — `SnapshotEmitterMixin._emit_trade_fire_snapshot`

```
f"TRADE FIRED SNAPSHOT [{side.upper()}] "
f"ticker_last={ticker_last:.6g} "
f"panel={snapshot}{extra_str}"
```

Method	Line prefix	Written when
`_emit_risk_gate_snapshot`	`RISK GATE SNAPSHOT [SIDE]`	a risk gate blocks a trade
`_emit_trade_fire_snapshot`	`TRADE FIRED SNAPSHOT [SIDE]`	a scrum or a fold actually fires

Both lines carry the last ticker price and a panel dictionary holding every voter's direction, confidence, weight and detail at that moment. The fire record adds the list of overrides that engaged, so a reader can tell an override-driven fire from a consensus-driven one without opening anything else.

The two code blocks above and the prefixes in the table are the shape these lines had before the tick-message rewrite. The current prefixes are `RISK GATE [SIDE]` and `TRADE FIRED [SIDE]`, and both write the same panel through one renderer.

Method	Line prefix	Written when
`_emit_risk_gate_snapshot`	`RISK GATE [SIDE]`	a risk gate blocks a trade
`_emit_trade_fire_snapshot`	`TRADE FIRED [SIDE]`	a scrum or a fold actually fires

The panel prints as a count and three direction groups rather than as a dictionary. A voter's weight is a configuration value that never moves between ticks, and a neutral voter's confidence is fixed at zero in the indicator code, so neither reaches the line. Direction, confidence and an indicator's own raw detail all remain.

`src/trading/scrumming/snapshots.py` — the grouped panel

```
  counts = ", ".join(f"{len(grouped[name])} {name.lower()}" for name in _PANEL_GROUPS)
  rows = [f"Panel {counts}."]
  for direction, voters in grouped.items():
      voters.sort(key=lambda one: (-one[0], one[1]))
```

True today: the measurement record is not in the tree, and the two figures it carried are kept here. The dictionary form measured 1,005 to 1,017 characters across 1,143 recorded snapshots, and the grouped form of the same panel measures 322.

Three further emitters sit in the same mixin: a trade notification carrying its own text prefix, a voting-panel snapshot at fire time, and a gate decision at fire time. The last two emit events rather than text lines, and the Console tab's signal pane reads them.

A missing capital reservation in the log

A bot claims the units it is allowed to work with so that a second bot on the same asset cannot sell them. Four places let a bot carry on when that claim is not there, and all four now write a warning naming what was lost. They appear in the upper pane at warning level.

`src/trading/scrumming/execution.py` — the one that bears on a sale

```
  logger.warning(
      "Bot %s sell of %.6f %s is NOT bounded by any other bot's "
      "capital reservation: the pre-check raised %s: %s. A sibling "
      "bot's claim on this asset is invisible to this sell.",
```

The other three name an allocation that is not held aside at admission, an Extractor whose base currency is empty, and a dollar grant made without an over-allocation check.

```
  src/trading/container/registry.py      at admission
  src/trading/extractor_bot.py           at the chunk-rate claim
  capital_registry                       at the dollar grant
```

All three of those lines have changed. `capital_registry` is removed, so no dollar grant is made and that warning can no longer be written. The admission warning in `src/trading/container/registry.py` is removed as well. It was never true. A bot claims its own capital on its first tick, through

`CapitalReservationMixin._crr`, so admission writes no claim and nothing is lost there. The Extractor writes a second warning now, described under the next heading but one.

The sale stops when the claim cannot be read

One sentence above is overtaken for the first of the four places, and it stays as written:

> *"Four places let a bot carry on when that claim is not there, and all four now write a warning naming what was lost."*

The true sentence for the place that bears on a sale: the warning is written and then the sale is refused. A check that cannot answer is not an allowance, which is what the open-orders read one screen below already does with a failure of its own.

`src/trading/scrumming/execution.py` — the refusal that follows the warning

```
self._emit_trade_notification(
    "SCRUM", "CANCELLED", "capital reservation unreadable"
)
return None
```

Driven on the real sell path with the claim read raising, twice: once on a claim table holding a quantity that is not a number, and once on a registry that could not be resolved. Both readings are the record the bot itself emitted.

```
before   SELL signal: 20.000000 @ $100.00000000 (VH:clean, MARKET)
         SCRUM SENT, venue order calls 1
after    SELL REFUSED (capital reservation unreadable)
         SCRUM CANCELLED, venue order calls 0
```

The operator sees a cancelled scrum with its reason in the upper pane, where a completed sale used to appear.

A claim file that will not read

A read can fail in two shapes, and only one of them raises. The claim file is read once, at start-up, by `_load` in `src/trading/capital_reservation.py`. A file that will not parse is caught there. The table is then empty because nothing was read, which is not the same as a file holding no claims.

The sale used to go through on that shape. The table read as empty, so `effective_available` reported the bot's whole holding as free, the pre-check passed it, and the operator saw a scrum sent.

`effective_available` now refuses to answer instead. It raises `ClaimsUnreadable`, which the pre-check one screen away already turns into a refusal. Both shapes of a failed read now end the same way.

Driven on the real sell path, with the claim file on disk holding bytes that will not parse. Both readings are the record the bot itself emitted.

```
before   SELL signal: 4.000000 @ $100.00000000 (VH:clean, LIMIT)
         SCRUM SENT, venue order calls 1
after    SELL REFUSED (capital reservation unreadable)
         SCRUM CANCELLED, venue order calls 0
```

The refusal names the file and what to do:

> *"the claim file reservation_state.json did not read at start-up, so no other bot's claim on the asset is known and an empty table here means nothing. Repair or move the file and restart"*

The writer holds back on the same shape. `_save` will not write a file it could not read, because writing it would replace the operator's claims with the few this run happens to know about. It logs an error naming the file and the count of changes that are not on the disk.

A claim that never reached the disk

The registry keeps the claim table in memory and writes it to a file. The writer is `_save` in `src/trading/capital_reservation.py`. A failed write does not stop the bot. It logs an error that names the file, the reason, and the count of changes that are not on the disk.

The line that records the claim now carries the answer as well. `CapitalReservationRegistry.reserve` logs the field `on_disk`. The line is an error when the value is False, and information when the value is True. Read the last line, not the first: a claim with `on_disk=False` exists for this run only, and the next start loses it.

A claim the bot refuses to place

The bot reads its exchange balance before it claims. The reader is `_get_cached_exchange_balance` in `src/trading/scrumming_bot.py`. The reader answers None when the venue call fails.

A balance that did not read is not a balance of zero, and it is not a licence to claim. `CapitalReservationMixin._ensure_capital_reservation` in `src/trading/scrumming/capital_reservation_mixin.py` stops on that answer. It places no new claim. It resizes no standing claim. It writes a warning that names the bot and the asset, and it tries again on the next call.

The Extractor refuses on the same answer

The Extractor reads its base currency the same way. The reader is `_read_base_holdings` in `src/trading/extractor_bot.py`, and it answers nothing when the venue call fails.

`set_initial_chunk_rate` stops on that answer. It places no claim, and it writes one warning at the upper pane that names the bot and the asset. The line reads:

```
Bot <id> could not read its <asset> balance; placing no claim, because no
holdings figure bounds it.
```

The Extractor claims once for each run, so it does not try again on a later call. A start reads the balance again.

The reader names the venue and the error

The refusal above tells you a claim was not placed. It does not tell you which venue would not answer, or what the venue said. The reader writes that itself, at warning level, so the whole failure sits on one line in this pane.

`src/trading/scrumming_bot.py` — `_get_cached_exchange_balance`

```
Bot <id> could not read its <asset> balance on <venue> (TimeoutError: the
venue balance endpoint did not answer). No holdings figure bounds a claim, so
no new claim is placed and no standing one is resized. Retrying next call.
```

Warning is the level this pane reads. The program runs its logging at information level, so a line written below that is dropped before any pane sees it. A venue that answered zero and a venue that refused used to look the same here; the line above tells them apart.

Driven with a venue stand-in whose balance call raises, the line was absent from a reading taken at warning level and present in a reading taken one level below it. It is now present in both. A neighbouring warning the same bot writes was present in all four readings, so the reading itself reports.

The bot named in a refused sale

A sale stops when another bot claims the units. The check is `effective_available` in `src/trading/capital_reservation.py`, and the caller is `src/trading/scrumming/execution.py`.

The refusal names the bots that hold the claim and the units each one holds. Stop a named bot to release its claim: `ScrummingBot.stop` calls `_release_capital_reservation`. The refusal names no page, because no page edits this table.

A claim that expires on its own clock

A bot pulses a heartbeat while it runs. A bot that stops pulsing leaves a claim that blocks every other bot on the same asset.

`prune_expired` in `src/trading/capital_reservation.py` drops a claim after HEARTBEAT_TTL seconds of silence, which is 120. `_prune_on_schedule` runs it once every HEARTBEAT_INTERVAL seconds, which is 30. Two methods call it: `effective_available`, before it answers, and `heartbeat`, after it stamps. A running bot stamps first, so it never drops its own claim. No setting changes either interval.

2026-09-23 - #34 - the drained record carries three more fields

`SignalSink.emit` in `src/core/signal_contract.py` stamps `cadence`, `budget_s` and `tab` on every record. The drain reads the same records, so every record this pane renders carries them. See system-status.md for what each field holds.

What the pane draws after the three fields land

The rendered line does not change. `_drain_signals` in `src/gui/main_window.py` reads five attributes off each record: `ok`, `name`, `site`, `actual` and `expected`. None of the three new fields is one of them.

`src/gui/main_window.py` - `_drain_signals`, the five it reads

```
view.appendHtml(
    f'<span style="color:{colour}">{mark}</span> '
    f'<span style="color:{ds.MAIN_LOG_NAME}">{_esc(r.name)}</span>'
    f'<span style="color:{ds.MAIN_LOG_SITE}"> {_esc(r.site)}</span>'
)
```

The seven counters do not change either. They count drain ticks, records read and records rendered, and no counter reads a field of the record.

2026-10-20 - #1392 - a gate refusal reads REFUSED, once a candle

The stage word

A trade notification carries a stage. `_emit_trade_notification` in `src/trading/scrumming/snapshots.py` writes it after the pair, and `stage_color` in `src/gui/main_tabs/status_log_surface.py` paints the line by reading it back out.

Three stages are passed anywhere in the tree.

Stage	What the bot did
`SENT`	an order is about to reach the venue
`FILLED`	the venue reported the fill
`REFUSED`	a gate stopped the trade and no order was sent

Ten sites used to pass `CANCELLED`. Every one of them is reached before `guarded_place_order`, so no order existed at any of them and nothing was pulled. `grep -rn '"CANCELLED"' src/trading/` now returns nothing.

`STAGE_COLORS` in `src/gui/main_tabs/status_log_surface.py` names `REFUSED`, so the pane paints it from the table rather than from `STAGE_DEFAULT_COLOR`. The colour is the same colour `CANCELLED` drew.

Two sentences and two readings above are overtaken. Quoted whole:

> *"The operator sees a cancelled scrum with its reason in the upper pane, where a completed sale used to appear."*

```
after    SELL REFUSED (capital reservation unreadable)
         SCRUM CANCELLED, venue order calls 0
```

True today: the operator sees a refused scrum with its reason in the upper pane, and the second line of both readings is `SCRUM REFUSED, venue order calls 0`. The code block one screen above, quoted whole:

```
self._emit_trade_notification(
    "SCRUM", "CANCELLED", "capital reservation unreadable"
)
```

True today, the same call:

```
self._emit_trade_notification(
    "SCRUM", "REFUSED", "capital reservation unreadable"
)
```

One line per reason per candle

`tick` evaluates every gate on every pass, and a pass happens every few seconds. On a five-minute timeframe the band position cannot move between most of those passes, so a refusal used to be announced again with identical numbers each time.

`_refusal_is_new` in `src/trading/scrumming/execution.py` decides whether an evaluation is news. It answers yes on the first call for a reason, on a candle different from the one the last call saw, when no candle is known, and when the price differs by `REFUSAL_HOLD_MOVE_FRACTION` from the price the previous call carried. Every verdict in `tick`'s scrum and fold chain now asks it before it writes anything.

The refusal itself is untouched. The gate refuses on the same inputs at the same threshold, and only the announcement is held.

Driven on the real `tick`, a bot built from invented configuration, 20 passes with the candle tape held still.

```
                                    before   after
refusal lines in bot.log                20       1
trade notifications                     20       1
venue order calls                        0       0
```

The bot sitting in a refusal stays visible. Each new candle carries one line, and a price move inside a candle re-opens the announcement on the next pass.

```
20 passes, nothing moving              before 20 lines    after 1 line
the candle rolls, nothing else moving  before 1 line      after 1 line
the band moves inside one candle       before 1 line      after 1 line
the band crosses the detect line       before 0 lines     after 0 lines, the trade fires
```

The gate's own reading was taken on a fresh bot at each of 30 band positions from 0.39 to 0.92, before and after: `_last_bb.bb_position`, `_bb_detect_thresholds` in `src/trading/scrumming/circuit_breakers.py`, whether the position is at or above the upper threshold, and the count of venue order calls. All 30 rows read the same both times. The gate closes from 0.390 through 0.873 with no order call, and opens from 0.889 through 0.920.

What is not held back

Four lines a pass are written with no guard: `DELTA`, `TA VOTE`, `BB PRIORITY SKEW` and `BULLSEYE UPPER`. They report the tick's state rather than a verdict. `FOLD HOLD (Position Ceiling)` is bounded by a counter of its own, one line per sixty passes, and `SCRUM SERVED BY STACK` and `HEDGE REBALANCE` each follow an action that happened once.

2026-10-21 - #1392 - the state lines take the same guard

One sentence above is overtaken. Quoted whole:

> *"Four lines a pass are written with no guard:* `DELTA`, `TA VOTE`, `BB PRIORITY SKEW` *and* `BULLSEYE UPPER`*."*

True today: those four lines ask `_refusal_is_new` before they write, as the verdicts do. Two more lines of the same kind went in with them. `READ` is the same report on the other side of the scrumming interval, written by the same branch of `tick`. `BULLSEYE LOWER` is the same line on the other band, written by `_tick_bullseye_notices` beside its upper twin. The rest of that sentence's paragraph stands: `FOLD HOLD (Position Ceiling)` is bounded by a counter of its own, and `SCRUM SERVED BY STACK` and `HEDGE REBALANCE` each follow an action that happened once.

What each line keys on

A refusal has one reason. A state line reports a reading that moves on every pass, so a key carrying the reading would be a new key each pass, the guard would answer new every time, and nothing would change. No key here carries a price, a confidence, a delta or a band position.

Each key is a fixed prefix plus the categorical state the line's own words already name, so a flip of that state re-announces at once.

Line	Key	Values it can take
DELTA	state_delta_<pos|neg>	2
READ	state_read	1
TA VOTE	state_ta_vote_<direction>	3
BB PRIORITY SKEW	state_bb_skew_<scrum|fold>	2
BULLSEYE UPPER	state_bullseye_upper_<wick|close>	2
BULLSEYE LOWER	state_bullseye_lower_<wick|close>	2

Twelve keys, every one from a bounded set. A key space that cannot grow with the reading cannot fall back to announcing every pass.

What it reads like now

Driven on the real `tick`, a bot built from invented configuration, with the candle tape held still.

```
                          20 passes      8 passes       8 passes
                          upper band     under interval lower band
                          before after   before after   before after
DELTA                        20     1
TA VOTE                      20     1
BB PRIORITY SKEW             20     1
BULLSEYE UPPER               20     1
READ                                        8     1
BULLSEYE LOWER                                          8     1
records per pass        [7,4,4,…] [7,0,0,…]
ta.voting events          1 each    1 each
venue order calls             0     0        0     0        0     0
```

A parked bot writes its state once and then writes nothing until something moves. The `ta.voting` event is untouched: it is a structured event the Indicator Voting Panel reads, it carries a silence budget in `src/core/emit_contracts.py`, and the run above reads it at one per pass both before and after. `bot.log` has no entry in that file and so no silence budget, which is why a quiet pass raises no fault.

The state still reaches him

```
                          before  after
the candle rolls             1       1      every line, every new candle
the next pass, same candle   1       0      the quiet one
the price moves              1       1      every line, at once
```

The gate did not move. The gate's own reading was taken on a fresh bot at each of 30 band positions, before and after: all 30 rows read the same both times, the gate closed from 0.390 through 0.873 with no venue order call and open from 0.889 through 0.920. The 35 message templates in `tick` and the 2 in `_tick_bullseye_notices` are unchanged, none lost and none gained. The skew itself still applies on every pass, because `_skewed_confidence_floor` is read above the line that announces it.

2026-10-28 - #1374 - a locked launch starts no bot and the Console says what it waits for

An optional password can seal the saved fleet. A launch that finds that password set, and holds no key for it, now draws every screen, loads no bot, and writes one Console line saying the fleet waits on the unlock. A locked launch is not an error and it writes no fault.

The three launches

`src/core/state_manager.py, in launch_classification` is the one answer every launch site reads, so no two sites can decide the lock differently. It reads the plain-text latch first, through `in latch_on`, and classifies the file through `in probe` only when that latch is set.

the saved file	the answer	what the launch does
no latch	`open`	restores the fleet and arms the start, as before
latch, key held	`unlocked`	the same, with the sealed payload opened
latch, no key	`locked`	draws every screen, loads no bot, records why

Driven on a hand-built fleet of two bots and one wire, the three launches give three different answers. Pointed at one file instead, all three give `open` and the reading reports them alike, so the three answers are a reading of the file and not a fixed result.

Where the latch is read

`main.py, in main` reads the classification before the fleet-start callback and before the fallback timer. Both the callback and the timer sit inside one branch a locked launch never takes, so no bot starts and nothing is scheduled.

Diagram: flowchart, top to bottom

- MainWindow built, every screen drawn -> **launch_classification**
- launch_classification -> **probe reads has_bots** *(open or unlocked)*
- probe reads has_bots -> **restore_bots_from_state**
- restore_bots_from_state -> **eligible bots**
- eligible bots -> **splash callback, with a 9500 ms timer behind it** *(more than none)*
- splash callback, with a 9500 ms timer behind it -> **run_fleet_sequence start**
- launch_classification -> **hold_fleet_for_unlock** *(locked)*
- hold_fleet_for_unlock -> **one Console line, no bot loaded, no timer armed**

The launch with no latch is unchanged. Measured over six saved-file shapes — no bots, one bot, two bots and a wire, nested lots and tranches, unicode and floats and nulls, and an unknown top-level field — `in latch_on`, `in probe`, `in has_saved_state` and `in load_state` answer the same through the files

before this section and the files after it. The control: one wire rate moved from 25.0 to 26.0 and the same comparison named `load_state` as different, so the six readings are a comparison that can fail.

What the Console says

`src/gui/main_tabs/main_window_surface.py` declares every line below, so the Qt build and the React build draw the same words.

when	the line
a locked launch	`The saved fleet is sealed. Every screen is open and no bot is loaded. The fleet starts when the unlock password is given.`
Start All while locked	the same line, and no bot is taken as eligible
a password that does not open	`That password did not open the saved fleet. The fleet stays sealed and no bot is loaded.`
a password that opens	`The saved fleet is open: N bot(s) loaded.`
the wires that came with it	`N Smart Wire(s) loaded with the fleet.`

`src/gui/main_window.py, in hold_fleet_for_unlock` writes the first line and latches the window, and `in run_fleet_sequence` asks `in fleet_locked` before it reads the fleet at all. Driven with one eligible bot and the window latched, the sequence answered 0, never read the bot map, and left that line in the Console. With the same bot and the window not latched it read the bot map and went on into the start path, so the latch is what stops it.

The unlock given mid-session

`src/gui/main_window.py, in unlock_fleet` takes the password, and no restart is needed. The order is fixed: `in unlock` holds the key, `in load_state` opens the payload, `in restore_bots_from_state` and `in restore_smart_wires_from_state` put the fleet in the process, and the callback `in set_fleet_start_callback` holds then runs. `main.py` registers that callback, which asks `InstanceGuard.evaluate` again with the bots the unlock loaded, because the launch asked it with a fleet of none.

Driven on the sealed fleet of two:

the password	the answer	still locked	handed over	the fleet start
wrong	False	yes	nothing	did not run
right	True	no	both bots and the wire	ran once

An unattended restart writes no fault

`src/core/instance_guard.py, in authorise_auto_start` answers `WITHHELD_FLEET_SEALED` before it reads the ownership decision and before it can ask for consent, so a machine with nobody at it is never shown a question it cannot answer. Measured with every log record of the run counted:

the run	records	faults	consent asked	files written
the fleet is sealed	1 at INFO	0	0	none
the control: consent given, no exclusive handle	1 at ERROR	1	1	—

The control is a condition that genuinely is a fault, and it reads as one through the same instrument, so the zero above is a reading.

What a locked launch does not read

A locked launch reads no price. `src/exchange/data_pool.py, in register` has one caller, `src/trading/bot_container.py, in start`, and `in refresh_all_tickers` has one caller that walks the fleet's own connectors. `src/gui/main_window.py, in _pump_currency_rates` and `in _pump_market_pairs_scout` both read `_exchange_connectors`, and every line that writes that map sits inside `in _connect_broker_for_bot`, `in _connect_written_crypto_for_bot` or `in _connect_exchange_for_bot` — each of which takes a bot. No bot, no connector, no price. A launch with an empty saved file reads no price for the same reason, so a locked launch reads what a fleet of none reads.

Not drawn yet

No screen takes the password. `src/gui/settings_dialog.py` owns that screen and it is not built, so `in unlock_fleet` is reachable from code and not yet from a keystroke.

The Product Manual PDF rebuild is owed for this page.

2026-10-28 - #1374 - the screen that takes the password is built

Where the sentences about the unlock now read differently

Two readings above describe the build before the screens landed. They stand as written, and these are the readings that replaced them.

written above	what it reads now
"No screen takes the password. `src/gui/settings_dialog.py` owns that screen and it is not built, so `in unlock_fleet` is reachable from code and not yet from a keystroke."	Two screens take it. `src/gui/settings_dialog.py, in _create_fleet_lock_group` draws the Password and Confirm Password rows that set it, and `src/gui/fleet_unlock_dialog.py, in FleetUnlockDialog` takes it back. `in unlock_fleet` is reached from a keystroke
"The Product Manual PDF rebuild is owed for this page."	`docs/Acervator-Product-Manual.pdf` is rebuilt in this branch and carries this page

Where the unlock screen appears

`main.py, in main` hands `_ask_for_unlock` to the splash's finished hook inside the branch a locked launch takes, beside `in hold_fleet_for_unlock`. The screen therefore opens over a window that already drew every tab. Dismissing it leaves the fleet sealed and the Console line in place, and the Unlock the fleet now button on the Settings User tab raises the same screen again.

Diagram: flowchart, top to bottom

- launch_classification reads locked -> **hold_fleet_for_unlock**
- hold_fleet_for_unlock -> **one Console line, no bot loaded**
- one Console line, no bot loaded -> **the splash closes**
- the splash closes -> **ask_for_unlock**
- ask_for_unlock -> **unlock_fleet, the fleet loads** *(password given)*
- ask_for_unlock -> **the fleet stays sealed** *(Not now)*

- the fleet stays sealed -> **Settings, User tab, Unlock the fleet now**
- Settings, User tab, Unlock the fleet now -> **ask_for_unlock**

What the screen was measured doing

Driven as a built widget against a sealed fleet of two bots and one wire: the Unlock button is dead on an empty row; a wrong password answered `refused`, put the Console's own refusal sentence on the screen, left the fleet sealed and changed no file; the right password answered `opened` and both bots and the wire came out of the payload in the same session.

The control for that reading: the same screen pointed at a window answering no `unlock_fleet` answered `no_window`, reported no unlock, and said so on the screen. So a reading that reports a reachable screen can also report that no screen accepts the password.

The words on the screen are the Console's words. `src/gui/main_tabs/fleet_unlock_surface.py` takes its detail line from `src/gui/main_tabs/main_window_surface.py, FLEET_LOCKED_TEXT` and its refusal line from `FLEET_UNLOCK_REFUSED_TEXT`, so the screen and the Console cannot disagree.

The rows that set the password are on the Settings page. See the Settings page.

2026-10-28 - #1374 - a locked launch reads the prices a screen draws for a market

A locked launch now reads every price a screen draws for a market, not for a bot. The venue keys are sealed with the fleet, so each of these reads goes through a route that carries no key.

Where the sentence about prices now reads differently

The reading above stands as written. This is the reading that replaced it.

written above	what it reads now
"A locked launch reads no price."	A locked launch reads three prices: the rate strip, the Charts ATA-SMP list and the Market Inspector scans. The prices that come through a bot or a venue connection wait for the unlock

The prices a locked launch reads

The dashboard tick runs every two seconds. While the window is latched, it feeds the two price screens it owns, and no other launch takes that branch.

```
if self.fleet_locked():
    self._feed_locked_price_screens()
```

`src/gui/main_window.py, in _feed_locked_price_screens` rebuilds the Charts ATA-SMP list, fetches the market on screen with no connector, and asks `in _pump_currency_rates` for the rates.

the screen	the read	the route, no key
Live tab, the rate strip, on both builds	BTC/USD and ETH/USD	`src/exchange/currency_rate_monitor.py, in refresh_from_feed`, through `src/paper/live_feed_source.py, in LiveFeedSource.ticker`
Charts, the ATA-SMP list	the candles of the market on screen	`src/exchange/chart_data.py, in ChartDataFetcher.fetch`, which reads CoinGecko with no connector

the screen	the read	the route, no key
Market Inspector, Scan Now and Scan All	each scanned market's candles	`src/gui/main_tabs/market_inspector_surface.py, in sector_candle_read`, the public Coinbase route

The Market Inspector scans read on a locked launch before this change. The rate strip and the Charts ATA-SMP list did not.

The prices that wait for the unlock

the read	why it waits
the venue page bot rows, the bulk ticker refresher, the Charts live list, the header strip, the voting panel's votes	each one reads a bot, and no bot is loaded
the market pairs scout	it reads each venue's ticker list through a venue connection, and only a bot's start opens one
Market Inspector, Refresh	its markets are the connected venue's own list, read through a venue connection

On a locked launch the Refresh line names the seal instead of asking for an exchange. `src/gui/main_tabs/market_inspector_surface.py, SEALED_NO_CONNECTORS_TEXT` holds it:

```
The saved fleet is sealed, so no bot has opened a venue connection. Give the password: Settings, User tab, Unlock the fleet now.
```

The Paper and Sim tabs read their own public routes for their own bots, and the lock seals neither of those fleets.

What the price readings were measured doing

Driven through the real dashboard tick, with every venue answered by a stand-in shaped like the public replies the code parses:

```
latched, no bot       rate strip   both prices, on the Qt panel and the React line
                      Charts       96 candles from CoinGecko
                      Scan Now     300 candles from the public route
                      Refresh      the sealed line
not latched, no bot   rate strip   pending, Charts 0 candles, Refresh asks for an exchange
before this change    latched      rate strip pending, Charts 0 candles
```

The unlatched run is the control: the same reading reports the reads that did not run. A launch with no password reads the same through the files before this section and the files after it: with a fleet and a connector the tick schedules the same rate read, and with neither it schedules nothing.

The Product Manual PDF rebuild is owed for this page.

2026-10-28 - #1374 - a sealed fleet makes no new bot

While the saved fleet is sealed, New Bot and Adopt make no bot. Each press puts one box over the window and one line in the Console, and nothing opens and nothing is written. A bot made then could not be saved, so it would trade with no record that the next launch could load.

Where the press stops

`src/gui/main_window.py, in _refuse_while_sealed` is the one place every press stops. It reads `src/core/state_manager.py, in launch_classification`, which is the answer every launch site reads and the answer under which each save is refused.

```
def _create_bot(self, exchange_id="", defaults_override=None):
    """Open the Bot Creation Wizard; ``defaults_override`` merges in."""
    if self._refuse_while_sealed(exchange_id):
        return
```

the press	how it reaches the stop
+ New Bot, Qt build	`src/gui/widgets/exchange_tab.py, in ExchangeTab.__init__` hands it to `in _create_bot`
+ New Bot, React build	`src/gui/react_exchange_tab.py, in ExchangeTabReact.new_bot` hands it to `in _create_bot`
Adopt, both builds	`in _adopt_topology_proposal` asks the stop before its confirm box and before its wire snapshot

These are the only presses that make a live bot. The restore at launch and the restore at the unlock bring back the saved fleet, and neither runs while it is sealed. Paper and Sim bots live in their own fleets, and the lock seals neither.

What he sees

Both builds draw the same box, because one window holds every page in both. `src/gui/main_tabs/main_window_surface.py` declares the words.

where	the words
the box title	Unlock the fleet first
the box and the Console line	The saved fleet is sealed. A bot made now could not be saved, so no bot is made. Unlock the fleet first: Settings, User tab, Unlock the fleet now.

The last sentence takes the button name from `src/gui/main_tabs/settings_dialog_surface.py, FLEET_LOCK_UNLOCK_BUTTON_TEXT`, so the route and the button cannot disagree. The API log records one `BOT_CREATE_REFUSED` line.

What was measured

Driven with the home redirected, against a sealed fleet of two bots, through the files before this section and the files after it. The wizard, the pre-flight check and the venue connect were stand-ins, and every other part was the real one. No socket left the machine.

the press	before	after
+ New Bot, Qt build	the wizard built, 1 bot made and not saved, and its Start reached the venue connect	the box and the line; no wizard, no bot, the file unchanged
+ New Bot, React build	the same	the same
Adopt, 2 bots and 1 wire	the confirm box, a snapshot file, 2 bots and 1 wire made and not saved	the box and the line; no confirm box, no snapshot, no bot, no wire

With no password set, and after an unlock, all three presses made the same bots through both sets of files, on both builds, and saved them. The control: one target balance moved from 200 to 201, and the comparison named the bot that differed.

An unlock that opens the key and not the fleet

`src/gui/main_window.py, in unlock_fleet` now answers False when the password opens the key file and the saved fleet does not open with that key. That happens when the two files come from two different seals, or the fleet file is damaged. The window stays held, no key is held, and the Console says:

```
That password opened the key file, but the saved fleet did not open with that key. The fleet stays sealed and no bot is loaded.
```

Measured both ways on a key file from another seal. Before, the unlock answered True, the window let go of the lock, the Console said the fleet was open with 0 bots, and the fleet start ran, while a save still left the file unchanged. After, the unlock answered False, the window stayed held, and the fleet start did not run.

Where an earlier sentence now reads differently

The sentence above stands as written. This is the reading that replaced it.

written above	what it reads now
"The Product Manual PDF rebuild is owed for this page."	`docs/Acervator-Product-Manual.pdf` is rebuilt with this section and carries this page

Back to the subsystem index.

System Status Tab

Reference. The screen is not built. The tab row carries an empty tab labelled Status, and issue #34 carries the build-out. Both halves the operator's text describes run today, and this file is those two halves.

The skeleton

The tab exists and draws three lines: its name, one sentence saying it is not built, and the issue that owns it. It reads no emitter, no heartbeat and no post-mortem.

`src/gui/main_tabs/system_status_tab_surface.py` — the whole empty state

```
HEADING = "Status"
ISSUE = 34
BUILT = False
STATE_TEXT = "This tab is not built."
ISSUE_TEXT = f"Issue #{ISSUE} carries the build-out."
```

Two frontends draw that one view model. `EmptyTabsMixin` in `src/gui/main_tabs/empty_tabs.py` builds the Qt tab, and `src/gui/web/system_status_tab.js` registers a panel with the Electron shell's panel host. `src.core.desktop_bridge` serves the model under `system_status_tab.state`.

Half one: the Emitter Network

One plain function is the whole connection point. A call site says what it expected and what it actually saw, and the record goes to the sink. It is a function rather than a bus subscription, so a call site needs no reference to anything, and with no sink installed the call is a dictionary lookup and a return.

`src/core/signal_contract.py` — `emit`

```
def emit(
    name: str,
    actual: Any,
    expected: Any = None,
    ok: Optional[bool] = None,
    context: Optional[dict] = None,
    every: float = 0.0,
    instance: Optional[str] = None,
    module: Optional[str] = None,
    duration: Optional[float] = None,
) -> Optional[Signal]:
```

A satisfied expectation records the same as a violated one. That is the design point. In a log that only records failures, a call site that never ran and a call site that always passed look identical. Here they do not.

Three constants bound what the sink holds: how many rows buffer before a write, how many stay in memory, and the size at which the file rotates.

`src/core/signal_contract.py` — the sink's bounds

```
DEFAULT_FLUSH_EVERY = 200
"""Rows buffered in memory before `flush()` writes them to disk."""

RETAIN_ROWS = 350_000
"""Signal records kept in memory at once; older ones are evicted from memory but remain
in the file on disk.
"""

MAX_FILE_BYTES = 50 * 1024 * 1024
"""Byte size at which the sink's file rotates to `.1`."""
```

The function stays one; the sinks are two. A thread that calls `route_thread` sends its own emits to another sink of the same class until it calls `unroute_thread`, and `get_sink` answers per thread. The Simulator routes each run's worker thread to its own sink under the sim bucket, so a run's rows never rotate the process sink; every other thread's emits reach the process sink as before. See simulator.md.

`src/core/signal_contract.py` — the routing

```
def route_thread(sink: SignalSink) -> None:
    """Send every `emit` raised on the calling thread to `sink` until `unroute_thread`."""
```

Tracked emitters

A second module declares every emitter. A contract names a topic, the fields a payload must carry, the fields it may carry, and the vocabulary a field is allowed to draw from.

`src/core/emit_contracts.py` — the gate-decision contract

```
  EmitContract(
      topic="bot.gate_decision",
      required=("symbol",),
      optional=(
          "scrum_armed",
          "fold_armed",
          "scrum_blockers",
          "fold_blockers",
          "scrum_fixture",
          "fold_fixture",
      ),
      description="Gate evaluation snapshot at fire time.",
  ),
```

`EmitObserver` records what actually arrived and reports each violation in one of three classes: a topic that never fired, a required field that was missing, and a value outside its vocabulary.

Topic	Required	Notes
`trade.filled`	side, amount, price	The trade kind is `type` here; the persisted log row calls it `action`
`bot.gate_decision`	symbol	Scrum and fold arm state, blockers and fixture at fire time
`pnl.event`	none	Realised, unrealised, symbol
`ta.voting`	none	Voting-engine snapshot

The note on the first row is the reason the module exists. A consumer read one key from a payload that carried another. Both sides succeeded, the consumer got nothing, and a coverage report read zero while hundreds of trades flowed past it. A key-name mismatch is invisible to the producer and to the consumer; only a declared contract makes it visible.

The current reading, 2026-10-29. The fill row of the table above now requires every field of the one fill shape, with exactly one of the two fee fields, and it names the trade kinds a fill may carry. The payload is read flat, because every fill is now sent flat. A payload that carries neither fee field reads as a missing field, and one that carries both reads as a bad value.

`src/core/emit_contracts.py, in CONTRACTS` — the fill contract

```
      EmitContract(
          topic=FILL_TOPIC,
          required=SETTLED_FILL_FIELDS,
          optional=FILL_FEE_FIELDS,
          exclusive=FILL_FEE_FIELDS,
          vocab={"side": FILL_SIDES, "type": TRADE_TYPES},
          nested_key=None,
```

The observer already has a formatter that turns its counts into operator-facing lines. Its one caller was the Nuclear fleet controller writing a run report; the Simulator rebuild deleted that file, so nothing calls it and no screen reads it.

`src/core/emit_contracts.py` — `format_observer_lines`, what it returns

```
d = obs.to_dict()
out = [
    f"Emit contracts: {d['topics_seen']}/{d['topics_declared']} "
    "topic(s) observed",
]
```

Half two: the Watchdog

The Watchdog runs outside the process that can crash. It launches the application as a subprocess, tees both output streams to the console and to its own log, and polls a heartbeat file. A heartbeat whose modification time stops advancing past the stall threshold counts as a hang. When the child dies the Watchdog writes a post-mortem, which is the record an in-process hook cannot produce for a hard abort or an operating-system kill.

`acervator_watchdog.py` — the stall threshold and the poll

```
DEFAULT_STALL_SECONDS = 60  # must match or exceed Acervator's longest sync call (CCXT: ~15s typical, 30s timeout)
HEARTBEAT_POLL_INTERVAL = 2.0
```

Every post-mortem is a bundle of logs, and bundles once grew without limit. Three constants bound them now. Two govern the rotation and the third only warns.

`acervator_watchdog.py` — the rotation bounds

```
POSTMORTEM_KEEP_LATEST: int = 20  # most-recent N bundles preserved
POSTMORTEM_MAX_AGE_DAYS: int = 30  # anything older is dropped
POSTMORTEM_SIZE_WARN_BYTES: int = 5 * 1024 * 1024 * 1024  # 5 GB warning threshold
```

The Watchdog writes to files under the log root, and nothing in the application reaches into it. Surviving the application dying is the whole point of it.

Until the tab lands

The Console tab's lower pane is the Emitter Network readout. It renders the same sink records, capped at the newest 200 per drain.

2026-09-08 08:17 - #34 - what the closed issues landed

The manual marks this screen System Status Tab (To Be Built) and it is a skeleton today.

This tab consists of two distinct but closely related parts. The Emitter Network is an embedded system of data activity detectors intended to allow for detailed subsystem performance monitoring. The Watchdog is the raw signal capture for the Emitter Network’s output.

The tab is now called Status. It sits ninth on the bar, on the gold ground with red text.

The screen is not built. The tab row now carries a System Status skeleton, which draws its name, one sentence saying it is not built, and the issue that owns it. Issue #34 carries the build-out. Both halves run today, and the Console tab shows the first of them.

`src/gui/main_tabs/system_status_tab_surface.py` — the whole empty state

```
HEADING = "Status"
ISSUE = 34
BUILT = False
STATE_TEXT = "This tab is not built."
ISSUE_TEXT = f"Issue #{ISSUE} carries the build-out."
```

The Emitter Network is one plain function and one sink. A call site says what it expected and what it actually saw, and a satisfied expectation is recorded the same as a violated one, which keeps a call site that never ran distinct from one that always passed.

`src/core/signal_contract.py` — `emit`

```
def emit(
    name: str,
    actual: Any,
    expected: Any = None,
    ok: Optional[bool] = None,
    context: Optional[dict] = None,
    every: float = 0.0,
    instance: Optional[str] = None,
    module: Optional[str] = None,
    duration: Optional[float] = None,
) -> Optional[Signal]:
```

A second module declares every emitter. A contract names a topic's required fields and its vocabulary, and the observer reports a topic that never fired, a missing field or a value outside that vocabulary.

`src/core/emit_contracts.py` — the gate-decision contract

```
EmitContract(
    topic="bot.gate_decision",
    required=("symbol",),
    optional=(
        "scrum_armed",
        "fold_armed",
        "scrum_blockers",
        "fold_blockers",
        "scrum_fixture",
        "fold_fixture",
    ),
    description="Gate evaluation snapshot at fire time.",
),
```

The Watchdog is `acervator_watchdog.py` at the repository root. It launches the application as a child process, tees both output streams to its own log, polls the heartbeat file, and writes a post-mortem after the child dies.

`acervator_watchdog.py` — the stall threshold and the poll

```
DEFAULT_STALL_SECONDS = 60  # must match or exceed Acervator's longest sync call (CCXT: ~15s typical, 30s timeout)
HEARTBEAT_POLL_INTERVAL = 2.0
```

Post-mortem bundle rotation

The Watchdog writes a post-mortem bundle every time the application dies. Each bundle copies the runner's console log, the crash log, the fault-handler log and the thread-violation log. The console log runs to hundreds of megabytes. A hang writes two bundles: one while the child is still alive, and one after the Watchdog has stopped it. Per-run log rotation capped the size of one run's logs and capped nothing about the number of bundles, so weeks of restarts grew without limit until the log directory took the host down.

`acervator_watchdog.py` — `prune_postmortem_bundles`

```
def prune_postmortem_bundles(
    keep_latest: int = POSTMORTEM_KEEP_LATEST,
    max_age_days: int = POSTMORTEM_MAX_AGE_DAYS,
    log_dir: Path | None = None,
) -> tuple[int, int]:
```

It keeps the most recent bundles, drops anything past a maximum age whatever its position, and runs at two sites: once at Watchdog startup, which clears what earlier runs left behind, and once after each post-mortem is written, which holds the ceiling during a long session.

`acervator_watchdog.py` — the three constants

```
POSTMORTEM_KEEP_LATEST: int = 20  # most-recent N bundles preserved
POSTMORTEM_MAX_AGE_DAYS: int = 30  # anything older is dropped
POSTMORTEM_SIZE_WARN_BYTES: int = 5 * 1024 * 1024 * 1024  # 5 GB warning threshold
```

The third is a warning threshold read by `report_log_dir_footprint` and it deletes nothing, so two constants govern the rotation and one reports on it. The only other inputs are the log directory itself, derived from the home directory, and the literal bundle-name prefix inside the function. Neither is a rotation constant.

No test in this repository references any of the three names. The cap runs and nothing holds it in place.

2026-09-09 14:30 - #34 - the tab draws the emitter network by subsystem and by tab

The screen is built. The Status tab no longer draws the empty state; it draws the emitter network, divided two ways.

The top of the screen names the feed: the file the sink writes, how many records it has taken this run, how many are still in memory, how many were evicted and how many were dropped. Under it, one strip of totals across the whole network. With no sink in the process the feed line reads one sentence instead, and every count falls to zero.

`src/gui/main_tabs/system_status_tab_surface.py` — the tab a subsystem's emitters serve

```
TAB_BY_SUBSYSTEM = {
    "apitest": ENGINE_GROUP,
    "bot": ENGINE_GROUP,
    "charts": "Charts",
    "console": "Console",
    "exchange": "Live",
    "extractor": ENGINE_GROUP,
    "fleet": "Sim",
    "gui": ENGINE_GROUP,
    "history": "History",
    "instance": ENGINE_GROUP,
    "sim": "Sim",
    "swarm": "Swarm",
    "ta": ENGINE_GROUP,
    "tick": ENGINE_GROUP,
    "topology": ENGINE_GROUP,
    "trading": "Live",
    "ytd": ENGINE_GROUP,
}
```

By subsystem is one panel per prefix, seventeen of them. Each panel carries a light, the tab it serves, six counts, and a table of that subsystem's own emitters with the cadence each one declares, how many times it fired, how many of its retained records failed, and the last value it reported.

By tab is one row per tab on the bar, ten of them, and an eleventh named Engine for the emitters that serve no single screen. A row names the subsystems it holds and adds their counts together. Paper, Inspector, Accumulation and Status hold no subsystem at all, and each says `no emitters` rather than drawing a light. That absence is the reading, not a gap.

Green, Yellow, and why Red cannot show

Two lights, and the screen says so in its own legend.

`src/core/signal_contract.py` — the two states the report returns

```
HEALTH_STATES = (HEALTH_GREEN, HEALTH_YELLOW)
"""The states `subsystem_health` returns; `HEALTH_RED` is not one of them."""
```

Green means every always-on emitter the subsystem declares has fired and no retained record of it failed. Yellow means a record failed, or an always-on emitter never fired at all. A subsystem that has recorded nothing carries no light and reads `no records this run`, which keeps a quiet subsystem separate from a healthy one.

Red waits on issue #14. A hung subsystem sends nothing, so silence is the only sign of it, and silence can only be read against a rhythm each emitter is expected to keep. No emitter declares one, so nothing returns Red and no light on this screen can show it.

The issue #14 clause above is overtaken. It is not edited and not removed. It read:

Red waits on issue #14. No emitter declares one, so nothing returns Red and no light on this screen can show it.

Issue #14 closed on 2026-09-01, and the rhythm it asked for is declared. Every emitter carries a cadence category, and an always-on pin has an age above which it reads late.

```
src/core/signal_contract.py

CADENCE_ALWAYS_ON          "always_on"
CADENCE_TOGGLE             "toggle"
CADENCE_CATEGORIES         the two above
ALWAYS_ON_STALE_AFTER      10.0 seconds
ALWAYS_ON_WORST_HEALTHY_GAP   4.736 seconds
in cadence_verdict         answers on_time, stale or late per pin
```

The second half of the clause still holds. `HEALTH_RED` is declared and is not one of the states `subsystem_health` returns, so no light on this screen shows Red. Whether a late always-on pin should turn a subsystem Red is a product choice, so nothing here proposes one.

What the log record carries

The persisted row now names the subsystem it belongs to, so the report and the log divide the network the same way.

`src/core/signal_contract.py` — `Signal.to_json`

```
        payload = {
            "ts": self.ts,
            "seq": self.seq,
            "module": self.module,
            "name": self.name,
            "subsystem": subsystem_of(self.name),
            "kind": self.kind,
```

`subsystem_of` is the one function that splits a name, and `SignalSink.by_subsystem` calls it too, so the row on disk and the panel on screen cannot disagree.

Measured on this build

The registry declares 78 emitters across 17 subsystems. The issue body's figure of 70 is older than the registry.

```
declared emitters          78
  always-on                16
  toggle                   62
subsystems                 17
no call site in src/       27    fleet 8, sim 14, ta 2, ytd 3
```

The twenty-seven with no call site can never fire, so their subsystems read silent whatever the platform does. Two of them are always-on, which is why Sim cannot reach Green today. Those emitters belong to the Simulator rebuild and the emitter network item, not to this screen.

What is still open

The Console tab keeps its Emitter Network pane; moving it here is not done. The Watchdog is not on this screen yet. Both are named in issue #34 and neither is built.

Back to the subsystem index.

Where the Status tab draws

The window builds this panel under both builds. `SystemStatusTabMixin` appends it with no variant seam, because no Qt widget ever drew this tab. The two builds draw the same picture at every width.

```
class            SystemStatusReactPanel
accessible name  Status
module loaded    system_status_tab
style sheet      system_status_tab.css, 25 rules
drawn            17 subsystems, 11 tab groups, 78 emitter rows
```

2026-09-20 06:18 - #34 - the register equals the code

The register in `src/core/signal_contract.py` now names every emitter the code calls, and only those. Every name under `src/` that reaches `emit` has an entry with its kind, and every entry has a call site. The tab map lives in the same module.

`src/core/signal_contract.py` - one entry per group, with its reason

```
_ALWAYS_ON_PINS: dict[str, str] = {
    "charts.indicators.drawn": (
        "the same `fetch_chart_data` pass calls `set_candles` on every fetch "
        "that returns candles"
    ),
```

```
_TOGGLE_PINS: dict[str, str] = {
    "inspector.ata.scan_pressed": "the operator presses Scan Now or Scan All",
```

An entry is a name and one line. The line is the reason the kind was read at the call site, in the form the retired roster used. The two groups keep their kinds: a loop or a timer reaches an always-on pin on every pass; a toggle pin needs a trigger, and silence from it is normal. `CADENCE_BY_NAME` is built from the two groups as before.

The tab map in the register

The tab a subsystem serves is read from `TAB_BY_SUBSYSTEM` and `tab_of` in `src/core/signal_contract.py`, beside `subsystem_of`. The map names `inspector` to Inspector and `status` to Status. The surface imports the map and the function; it holds no copy.

`src/core/signal_contract.py` - `tab_of`

```
def tab_of(subsystem: str) -> str:
    """Return the tab `subsystem`'s emitters serve, or `ENGINE_GROUP` when none does."""
    return TAB_BY_SUBSYSTEM.get(subsystem, ENGINE_GROUP)
```

`src/gui/main_tabs/system_status_tab_surface.py` - the import

```
from src.core.signal_contract import (
    ENGINE_GROUP,
    HEALTH_GREEN,
    HEALTH_STATES,
    HEALTH_YELLOW,
    TAB_BY_SUBSYSTEM,
    get_sink,
    subsystem_health,
    tab_of,
)
```

What entered and what left

Thirty-six names entered: 7 `charts.*`, 15 `inspector.*`, 14 `sim.*`. One of them, `charts.indicators.drawn`, is always-on: the 2000 ms dashboard timer schedules `fetch_chart_data`, and that pass calls `set_candles` on every fetch that returns candles. The other thirty-five need a press, a pointer, a call, or a run the operator starts.

Twenty-seven entries left, each with no call site under `src/`: every `fleet.03.*` entry (8), every `sim.06.*` entry (14), `ta.07.001` and `ta.07.002`, and every `ytd.10.*` entry (3). Two of them were always-on, `sim.06.011.postcondition.price_chart.fed` and `sim.06.012.postcondition.gate_status.rendered`, and they held the Sim panel at Yellow with `always-on silent 2`.

Overtaken sentences on this page

Each sentence below stands as written above. The sentence under it is the current reading.

> `src/gui/main_tabs/system_status_tab_surface.py` — *the tab a subsystem's emitters serve*

The map is in `src/core/signal_contract.py`, as `TAB_BY_SUBSYSTEM`, with `inspector` and `status` added.

> ***By subsystem*** *is one panel per prefix, seventeen of them.*

Sixteen. The `fleet` and `ytd` prefixes hold no entry after this change, and `inspector` holds fifteen.

> *Paper, Inspector, Accumulation and Status hold no subsystem at all, and each says* `no emitters` *rather than drawing a light.*

Inspector holds `inspector`. Paper, Accumulation and Status still say `no emitters`.

> *The registry declares 78 emitters across 17 subsystems.*

The register declares 87 emitters across 16 subsystems.

> *The twenty-seven with no call site can never fire, so their subsystems read silent whatever the platform does. Two of them are always-on, which is why Sim cannot reach Green today.*

Those twenty-seven are gone. No entry lacks a call site, and the Sim panel's `always-on silent` reads 0.

> *drawn 17 subsystems, 11 tab groups, 78 emitter rows*

Drawn: 16 subsystems, 11 tab groups, 87 emitter rows.

Measured on this change

Read off the running program in both variants, on a scratch home with every socket but loopback refused, the tab shown, the page read at the widget.

```
declared emitters          87
  always-on                15
  toggle                   72
subsystems                 16
names in code not declared  0
entries with no call site   0
drawn                      16 subsystems, 11 tab groups, 87 emitter rows
Inspector tab row          inspector
Sim panel always-on silent 0
```

Before the change the same reading gave 17 subsystems, 80 rows on a page where two undeclared names had already fired at build, `always-on silent 2` on Sim, and `no emitters` on the Inspector row. One Scan Now on the loopback stand-in then raised an `inspector` panel with 11 undeclared rows under Engine. After the change the same press fills the Inspector panel: 15 declared, 11 fired, 0 undeclared, and the panel sits under the Inspector row.

The page still draws once, when the window builds. A Status tab opened after a scan shows what the sink held at build. The redraw is a later unit of #34.

2026-09-23 - #34 - the record carries its kind, its budget and its tab

`SignalSink.emit` in `src/core/signal_contract.py` stamps three more fields on every record it builds. `Signal.to_json` writes them to the line on disk. `read_records` restores them. A line on `session.jsonl` is eighteen fields. A reader can now judge a silence from the line alone. Before this change a reader had to import the register of the build that wrote the line.

field	what it holds	the function that gives it
`cadence`	`always_on` or `toggle`, empty for a name the register does not hold	`cadence_of`
`budget_s`	the seconds an always-on emitter may stay silent, empty for every other emitter	`always_on_stale_after`
`tab`	the tab the emitter's subsystem serves	`tab_of`

`src/core/signal_contract.py` - `SignalSink.emit`, the stamp

```
                _cadence, _tab = self._declared_for(_name)
                sig = Signal(
                    cadence=_cadence,
                    budget_s=(
                        always_on_stale_after(_window)
                        if _cadence == CADENCE_ALWAYS_ON
                        else None
                    ),
                    tab=_tab,
                )
```

`_window` is the widest `every=` window the sink holds for that emitter's identity. An emitter with no window keeps the flat budget, `ALWAYS_ON_STALE_AFTER`, ten seconds. An emitter with a window keeps

ALWAYS_ON_SLACK times that window, which is 2.11 times. A toggle keeps no budget. _declared_for reads the register once per name and keeps the answer, so the register is not walked per record.

What a restored record answers

read_records reads each of the three through its own check: _as_cadence admits only a member of CADENCE_CATEGORIES, _as_budget admits only a finite number above zero, and _as_tab admits only a string with text in it. A value outside a check restores empty and the record is kept, the way a bad dt or a bad duration is kept today. A line written before this change carries none of the three, so all three restore empty and every other field restores as before.

src/core/signal_contract.py - read_records, the restore

```
                        cadence=_as_cadence(d.get("cadence")),
                        budget_s=_as_budget(d.get("budget_s")),
                        tab=_as_tab(d.get("tab")),
```

Overtaken by the record's three new fields

The sentence below stands as written above. The sentence under it is the current reading.

> *The persisted row now names the subsystem it belongs to, so the report and the log divide the network the same way.*

The row names the subsystem, the emitter's kind, the emitter's budget in seconds and the tab the subsystem serves. Four of the row's eighteen fields come from the register rather than from the call, and the Signal.to_json block above carries three more keys after context.

Read on this change

Read off the running program on a scratch home, with every socket but loopback refused, and on a read-only copy of a rotated file from the operator's own log root.

```
a line on session.jsonl          18 fields, 25 of 25 lines
a line on session.digest.jsonl   19 fields, the three plus folded
an always-on emitter, no window  ta.07.003.postcondition.computed  budget_s 10.0
an always-on emitter, 30 s window console.14.001.invariant.records_rendered
                                                    budget_s 63.34459459459459
a toggle                         trading.12.001.postcondition.tab_assembled
                                                    budget_s empty, tab Live
the operator's rotated file      92,500 lines, 92,500 records restored,
                                 all three empty on every record
```

Before the change the same reading gave fifteen fields on every line and no cadence, budget_s or tab on any of them.

The panels do not read the three fields yet. subsystem_health still counts silence since launch, so the light is still Green or Yellow and never Red. Reading a budget off the record is a later unit of #34.

2026-09-23 - #34 - the qt bundle ships the shell renderer

This tab draws through a React page, and the page host sits in the renderer folder beside the application. A qt build shipped no copy of that folder. The tab then raised a missing shell asset and drew an empty pane. Every build now ships the folder.

`tools/spec_common.py` - the pair every build copies

```
def renderer_candidate(project_root: str) -> tuple[str, str]:
    """The (source, destination) pair of ``SHELL_RENDERER``, which every variant ships."""
```

`shell_candidates` now holds what the Electron shell alone needs: the three shell files and the Electron runtime. A React build ships those, and no other variant does.

The overtaken sentence about the bundle

The sentence below stands as written above. The sentence under it is the current reading.

> *The window builds this panel under both builds.*

The window builds this panel under both builds. Before this change only a React bundle carried the page host, so only a React bundle could draw the panel. Both bundles carry it now.

Read off the launched bundles

Both variants were built from one commit and launched with no network: a scratch home, an application container with no capability, and the name resolver mapped away. The Status tab was pressed through its accessibility action. The page was read off the accessibility tree.

```
qt bundle before     no desktop folder in the bundle
                     1 missing-asset line in system.log
                     0 panels, 0 emitter rows, empty pane

qt bundle after      desktop/renderer, 8 files, 38,592 bytes
                     0 missing-asset lines in system.log
                     16 panels, 87 emitter rows, 11 tab rows

react bundle after   desktop, 11 files: 3 shell files and the renderer
```

A build that names a renderer folder which is not on disk ships no renderer, and its Status tab raises the missing asset again. That is how the reading is known to be able to fail.

The crash log and the five writers that fill it

`~/.acervator_logs/crash_<date>_<time>.log` is the file that holds a fault. One file per run. The Watchdog copies it into every post-mortem bundle, beside the console log and the fault-handler log.

Five writers reach it. `main.py` installs all five at import, before the window exists.

writer	what reaches it	installed at
`sys.excepthook`	an exception no `except` caught	`main.py`
`threading.excepthook`	the same, on a worker thread	`main.py`
`sys.unraisablehook`	a failure the interpreter cannot propagate	`main.py`

writer	what reaches it	installed at
the Qt message handler	every message Qt emits	`main.py`
`_CrashLogHandler`	every log record at ERROR and above	`main.py`

The fifth is the one that sees a fault the other four cannot. A broad `except` that reports through `logger.error` and returns reaches no excepthook at all. `logging_engine` clears `acervator.propagate`, so the handler attaches to two logger names.

`main.py` — the two attach points

```
# Two disjoint attach points: `logging_engine` clears `acervator.propagate`.
CRASH_LOG_HANDLER_LOGGERS = ("", "acervator")
```

A record that carries no exception of its own is written with the exception live at the call site, labelled `exception live at log time`. A record that carries one is labelled `exception declared on the record`.

A fault that cannot be printed is still named

`traceback.format_exception` reads each frame's source line. It raises when the module it needs is gone, and an exception whose own `__str__` raises defeats it as well. Both happened on the operator's build, and the handler died without writing.

`_crash_record` names the exception type and its message first, through `_safe_text`, and adds the frames second. The type and the message therefore reach the file when no frame can be read. A frame that cannot be read is written as `<frame unreadable>` and the walk continues past it.

A repeating entry is counted, not repeated

An entry equal to the one before it is counted. The count lands as a `REPEAT` entry when a different entry arrives, at `CRASH_REPEAT_RELEASE_AT`, or at exit. The first copy of the entry is always written immediately.

```
[..] [QT_WARNING] [thread=MainThread] DirectWrite: CreateFontFaceFromHDC() failed ...
[..] [REPEAT]     [thread=MainThread] previous QT_WARNING entry repeated 58 more times
```

No category is silenced and no distinct entry is dropped. One font warning that repeats for the life of the process costs two lines instead of hundreds.

Read off the crash log

One crash log per run, read back off disk. A fault whose message cannot be rendered, raised through the installed `sys.excepthook`:

```
                       crash log lines    the exception named in it
before                        2           no
after                        16           yes
```

The same fault caught by a broad `except` and reported with `logger.error`:

```
before                        2          no
after                        15          yes
```

Fifty-nine copies of the font warning, then the fault:

```
before                       63          no    (59 of the 63 are the warning)
after                        19          yes   (1 warning, 1 REPEAT line)
```

The `before` column is the operator's own file: 63 lines, 59 of them the font warning, three boot lines, one `QFont::setPointSize` warning, and no crash.

The hardest case, with the frame source cache broken and the message unrenderable at once, so nothing about the exception can be formatted:

```
before                        2          no
after                         9          yes   (all three frames named)
```

A fault reported below ERROR reaches none of the five

`_CrashLogHandler` is attached at ERROR, so it is the fifth writer only for a record at ERROR or above. A broad `except` that reports the same fault with `logger.debug` builds a record no writer reads, and the crash log gains nothing.

The second place an exchange tab's bot rows are refreshed is the privacy-widget sweep, which the privacy-mode button on each exchange tab runs. Its own `except` reported at DEBUG. It now hands the failure to a throttled fault, which reports the first failure at ERROR and names what the operator loses.

`src/gui/main_window.py` — the two faults the sweep reports through

```
        _tab_bots_fault = ThrottledFault(
            "the exchange tab bot list",
            "Every bot row on each exchange tab keeps its last figures and "
            "stops refreshing.",
        )
        _tab_privacy_fault = ThrottledFault(
            "the exchange tab privacy button",
            "The privacy button on each exchange tab stops following the "
            "privacy mode it is set to.",
        )
```

The throttle is the same one the window already uses for its currency-rate and market-pairs pumps. The first failure reports and opens a thirty-second window; further failures inside that window are counted and not written, so a sweep that keeps failing costs one line rather than one per press.

Read off the crash log, the privacy sweep

The shipped method driven five times with one exchange tab whose bot-row refresh raises, and a second tab beside it whose refresh succeeds:

```
                        crash log lines    the fault named in it
before                         0           no
after                          9           yes
reports emitted                 1          from five failures
neighbour tab refreshed         5 times     in both readings
```

The fault reaches the file as a `LOG_ERROR` entry naming the feature, what is lost, the retry window and the cause, followed by the frame inside `refresh_all_privacy_widgets`.

A stopped work loop names the restart it needs

The platform runs its venue reads on one background work loop. A fifty- millisecond timer advances that loop, and the slot that drives it is `main.py, in _make_async_pump_timer`. When the call that queues the loop's next step raises, the loop is not advanced, every bot stops reading its venue, and nothing a running platform does recovers it.

`main.py, in PumpStallReport` is what the slot now reports through. The first raising pump writes one line naming what stopped and what is lost. Further raising pumps inside the thirty-second window are silent. Once the run of raising pumps has lasted the venue-stall span, one further line names the restart.

```
BACKGROUND WORK STOPPED - the loop that runs every bot's venue reads was not
advanced. Every bot stops reading its venue and every figure on screen keeps
its last value. Cause: TypeError: __init__() should return None, not 'NoneType'

RESTART NEEDED - the loop that runs every bot's venue reads has not advanced
for 1 minute, over 1193 pump(s). Nothing recovers it while the platform runs.
Close the platform and start it again. Cause: TypeError: __init__() should
return None, not 'NoneType'
```

The span is the one already in the tree. `STALL_THRESHOLD_SECONDS` is the same sixty seconds the Exchange sub-tab's red line reads, so the Console and that line cannot disagree about when a stop has begun. The window between reports is `DEFAULT_BASE_COOLOFF_S`, the thirty seconds every other feature fault uses.

Read off the crash log, the background work loop

The real pump slot fired every fifty milliseconds for seventy seconds, in a failing arm and a healthy arm of the same run, with an ordinary queued callback in both arms read the same way.

```
                                  before      after
crash log lines                    37912        126
Console lines                       1354          7
lines naming a restart                 0          1
queued callbacks that ran, failing     0          0
queued callbacks that ran, healthy  1390       1390
```

The healthy arm is the control. It ran for the same seventy seconds, past the sixty-second span, and wrote nothing in either reading, so the lines above are a reading of the stop and not of the pump's ordinary work. The failing arm's queued callback never runs in either reading: this names the stop, it does not prevent it.

The two signatures one state produces

The crash record for the recorded run holds two different failures. Driven on this interpreter, both come from one state: an object whose type is named `NoneType` and which an `is None` test refuses.

```
TypeError: __init__() should return None, not 'NoneType'
    raised where CPython compares what __init__ answered against its own None

AttributeError: 'NoneType' object has no attribute 'partition'
    raised in the standard library's traceback reader, one line below its own
    is-None guard
```

With the interpreter's own None both sites behave: a constructor raises nothing and the traceback reader answers None cleanly. So the two signatures are one fault seen twice, and the exit dialog's missing-callable message is that same state at a third site.

Overtaken by the pump's own report

The sentence below stands as written above. The sentence under it is the current reading.

> *One font warning that repeats for the life of the process costs two lines instead of hundreds.*

That holds for a warning that repeats with nothing between it. A raising pump wrote an `ASYNCIO` entry and a `LOG_ERROR` entry alternately, so no entry ever equalled the one before it and the count never fired: 1,354 raising pumps wrote 37,912 lines in seventy seconds. The pump now reports once per window, so the same seventy seconds cost 126 lines.

What this does not name

The state itself is not produced by any Python in this repository. Counted across the source tree and the entry point: classes named `NoneType`, nothing. Application imports of the two installed packages that define one, nothing, against sixteen for the exchange library as a control. Calls that hand raw object pointers to a C library, nothing. Across the 699 installed extension modules the only interpreter runtime named is `python314.dll`, on 591 of them, with a name that must appear reading 699 and a name that must not reading zero.

So nothing below the report is repaired here, and the report is what a restart is now made for.

The heartbeat file and the root that holds it

The Watchdog half above says what the heartbeat is for. It polls the file, and a modification time that stops advancing past the stall threshold counts as a hang. The file is `heartbeat.txt`, one per log directory. It holds a single line: a timestamp and the process id of the run that wrote it.

`main.py` — the line written on every tick

```
f.write(f"{datetime.now().isoformat()} pid={os.getpid()}\n")
```

The line is rewritten every two seconds for the life of the process. The modification time is the signal the Watchdog reads. The contents only name which run stamped it.

`main.py` — the timer that drives the write

```
heartbeat_timer.start(2000)
```

The directory is `~/.acervator_logs/`, the same root that holds the crash log, the fault-handler log and the stale-binary marker. One environment variable moves all four. Set `ACERVATOR_CRASH_LOG_ROOT` to a directory and every one of them is written there instead.

`main.py` — the resolver the four writers share

```
_override = os.environ.get(CRASH_LOG_ROOT_ENV)
return _P(_override) if _override else _P.home() / ".acervator_logs"
```

An empty value counts as unset, so the root falls back to the home directory. A run that redirects the home directory carries the heartbeat with it, whether the variable is set or not.

Read off the resolved path

The path the shipped resolver returns, read in three environments:

```
home            variable        heartbeat written to
scratch A       unset           scratch A / .acervator_logs
scratch A       scratch B       scratch B
real home       scratch B       scratch B
```

Every log directory the override moves

The four writers named above are not the whole set. The resolver those two lines show now lives in `src/core/log_paths.py`, under the name `resolve_log_root`, and the entry point calls it there. Thirteen things follow it.

The four entry-point files:

```
crash_<timestamp>.log            the crash log
faulthandler_<timestamp>.log     the fault-handler log
STALE_DIST_WARNING.txt           the stale-build marker
heartbeat.txt                    the file the Watchdog polls
```

The nine bucket directories, one per bucket function:

```
activity/           get_activity_dir
api/                get_api_dir
console/            get_console_dir
trade/              get_trade_dir
trade/pnl/          get_pnl_dir
exchange_history/   get_exchange_history_dir
_meta/              get_meta_dir
reports/            get_reports_dir
sim/                get_sim_dir
```

The variable is `ACERVATOR_CRASH_LOG_ROOT`. Set it to a directory and all thirteen are written there. An empty value counts as unset.

With the variable unset every one of the thirteen resolves under the home tree, at the same place it resolved before the buckets were wired to the resolver. The trade log, the gate log and the daily profit files do not move unless the variable is set.

The root is read on each call, not once when the module loads. A run that sets the variable after the module has been imported still moves all nine buckets.

Read off the resolved paths, all nine buckets, in two environments:

```
variable        where the nine resolved         entry point agreed
unset           home / .acervator_logs / *      yes, 9 of 9
scratch B       scratch B / *                   yes, 9 of 9
```

2026-10-04 - #410 - the periodic collection reads the young objects on eleven ticks in twelve

Python's own automatic memory collector is switched off at startup. A timer on the thread that draws the window collects in its place, every five seconds. That has not changed, and the switch-off has not changed. What changed is how deep each collection reads.

A collection has three depths. The shallowest reads only the objects made since the last collection. The deepest reads every live object the program holds, and so touches every page of memory the program occupies. Until this change every tick ran the deepest one, twelve times a minute.

Now eleven ticks in twelve run the shallowest read, and the twelfth runs the deepest. A collection still happens every five seconds, so nothing is collected less often than before. One full read a minute replaces twelve.

`main.py` - the depth each tick reads

```
def periodic_gc_generation(tick: int) -> int:
    """Return the gc generation the periodic collection reads on tick, counted from 1."""
    return 2 if tick % GC_FULL_EVERY_TICKS == 0 else 0
```

The timer still belongs to the thread that draws the window, so the collector still never runs on an exchange worker thread. That is the whole reason the automatic collector is switched off, and it is untouched.

Measured on the collection depth

Both sides were driven on one machine, on a heap holding 343,283 tracked objects built by importing all 487 modules under `src`, with the collector's own `gc.DEBUG_STATS` report supplying the object counts and `time.perf_counter` the clock. Each side ran one twelve-tick round, which is one minute of the real timer.

```
                    seconds per collection      objects read per collection
before              0.079518 mean               352,491 mean
                    0.090910 worst              352,502 worst

after               0.015458 mean                37,630 mean
                    0.081259 worst              352,503 worst

per minute          0.954211s  ->  0.185494s    4,229,892  ->  451,568
```

The deepest read costs the same as it always did; the worst single tick is that read, and it is unchanged. What falls is how many times a minute the program pays for it. Objects read per minute falls by 9.37 times, and that figure is the one the page-fault rate follows. The seconds column moves with whatever else the machine is doing - the same pair of rounds read a ratio of 4.05, 4.64 and 5.14 across three runs with the live platform sharing the machine - while the objects column read 9.37 every time.

The timer was then driven through a real Qt event loop for twenty-four ticks. Twenty-two shallow reads and two deep ones ran, `gc.get_stats()` reported twenty-four collections, every one of them ran on the thread that draws the window, and the automatic collector stayed off throughout.

A third depth, read every fourth tick, was measured and left out. It held the same number of objects as leaving it out did - a peak of 126,004 above a quiet heap either way - and cost more time, so it earns nothing.

`gc.set_threshold` cannot be part of this. While the automatic collector is switched off the thresholds are never read. Measured: the most aggressive threshold the interpreter accepts, then 20,000 unreachable reference cycles made, gives **0** collections with the collector off and **858** with it on.

What this does not prove

The crash the switch-off guards against cannot be shown to stay away by any reading taken here. That needs his own machine, his own fleet, and hours of running. What to watch for: a hard exit with no traceback, and a new file under `~/.acervator_logs/` named `faulthandler_*.log` naming a thread that is not the one drawing the window.

The deepest read now runs once a minute, so reference cycles that outlive one shallow read wait up to a minute instead of five seconds. Measured on a heap driven at 9,000 such objects every five seconds, the peak held 126,004 objects above a quiet heap against 26,998 before. Objects not in a reference cycle are freed the instant nothing points at them, and that is untouched. What to watch for: a resident set that climbs through a minute and does not fall back.

2026-10-17 - #62 - the ladder weighs rarity, and a topic's silence has a reading

The signal ladder evicted by position alone, so the newest routine reading pushed out the oldest rare one. Three bounds now sit beside the size bound, and a bus topic's silence has a declared reading.

The loudest writer declares a window

`src/trading/ta_engine.py`, in `compute_all`, holds the two loudest emitters on the ladder. The first fires once per call. The second fires once per indicator per call. Both now pass a window and a market key.

`src/trading/ta_engine.py` — the window

```
TA_EMIT_EVERY = DIGEST_MIN_INTERVAL
```

The window is the digest ladder's own admission window, imported rather than copied, so one figure cannot become two. `instance` keys the window per market and timeframe, so one market's reading never suppresses another's. A suppressed observation is folded into the next admitted record's `count`, and `emit` never suppresses a failing check.

Driven on a 260-candle tape with a sink under a temporary home: one compute admitted 13 records, the same market computed again admitted 0, a second market admitted 13 of its own, and 13 returned after `reset_throttle`.

An age bound beside the size bound

`src/core/signal_contract.py`, in `SignalSink._retire_aged`, unlinks a rotated backup whose last write is older than the bound.

```
MAX_FILE_AGE_SECONDS = 14.0 * 24.0 * 3600.0
```

Fourteen days. The sweep runs on every `_roll` call, so a ladder below its size bound still retires its tail instead of holding it for ever. It reaches backups only. The live file is never unlinked, renamed or truncated, so no handle the sink holds is disturbed, and a backup that cannot be unlinked is left alone and not counted.

This bound does not widen the ladder's time window. It bounds the footprint in time. The window is widened by the rate limit and the share bound.

Driven with five planted backups, three of them past the bound: the three went and `age_retired` read 3. With every backup inside the bound, none went and the counter read 0. With the bound set to `0.0`, the same three aged backups stayed and the counter read 0.

A per-emitter share of one file

`src/core/signal_contract.py`, in `SignalSink._admitted_lines`, charges each record's bytes to its identity and refuses a passing record from an identity already at its share.

```
MAX_IDENTITY_SHARE = 0.10
```

Ten distinct identities are needed to fill one file at this share, so no single writer can crowd a file alone. The charge resets when the file rotates. Two records are never refused: one whose check failed, which is the reading worth finding, and one from a new identity past `max_identities`. A refused record's identity stands on the digest ladder, where its next admitted line carries the `folded` count of what was suppressed.

Driven on a 20,000-byte file, which gives one identity 2,000 bytes: a loud identity emitting 300 passing records put 6 of them on the live file and had 294 refused, the one rare identity's record was on the live file, and the loud identity's failing record was on the live file after its share was spent. With the share set to `0.0`, all 300 were on the live file and nothing was refused.

A reading for silence on a topic

A file of written records cannot report silence, because a record is only ever written when something happened. `src/core/emit_contracts.py`, in `EmitObserver.silence`, reads each declared topic against the budget its contract declares.

reading	what it means
`unbounded`	the topic declares no budget, so silence from it is not evidence
`never`	a budget is declared and the topic has not emitted at all; its producer is absent, which is not the same as stopped
`within`	the last emission is inside the budget
`overdue`	the topic emitted and then stopped past its budget

`EmitContract` carries the budget as a declared field, `silence_budget`. Two topics declare one: `bot.gate_decision` and `ta.voting`, the two a bot's own evaluation loop drives. `trade.filled`,

pnl.event, market_inspector.scan_started and market_inspector.scan_finished declare none, because a fill, a P&L movement and a scan each need a discrete trigger and silence from them says nothing. Those four read unbounded for ever.

```
LOOP_DRIVEN_SILENCE_BUDGET = STALE_AFTER
```

STALE_AFTER is the age at which the network already reads a pin stale, imported rather than copied, so the bus reading and the pin reading cannot drift to two ages. silence takes its clock from the caller and falls back to time.monotonic(), so a reader replaying compressed time supplies its own.

Driven on a fresh observer: the two budgeted topics read never before any emission and the four unbudgeted read unbounded. One ta.voting emission later, that topic read within a second on and overdue a second past its budget. trade.filled read unbounded a million seconds on.

Where the readings land

No new reader was built. Six readings joined SignalSink.health(), which src/gui/main_tabs/system_status_tab_surface.py already reads, and one key joined EmitObserver.to_dict(), which src/simulator/sim_bus.py already reads at a run's close.

reading	what it carries
age_retired	backups the age bound unlinked from the main ladder
digest_age_retired	the same for the digest ladder
max_file_age	the age bound in force
share_refused	passing records the share bound kept off the live file
share_budget	bytes one identity may fill between rotations
share_identities	identities holding a charge since the last rotation

format_observer_lines prints one tally line for the silence readings, then one line per topic reading never or overdue. A topic reading unbounded is counted and not listed, because a list of such topics would read as a list of faults.

Overtaken by the two new ladder bounds

Each sentence below stands as written above. The sentence under it is the current reading.

> *Three constants bound what the sink holds: how many rows buffer before a write, how many stay in memory, and the size at which the file rotates.*

Five. MAX_FILE_AGE_SECONDS bounds the age of a rotated backup, and MAX_IDENTITY_SHARE bounds the share of one live file a single identity may fill with passing records.

What the new bounds do not prove

Every figure above was read from a drive under a temporary home, with HOME, USERPROFILE, LOCALAPPDATA and APPDATA redirected before the first import and Path.home() printed to prove it. None of it was read from a running platform. What the window and the share bound do to a real session's megabytes per hour is unmeasured until a launch writes a real signals/session.jsonl. What to watch for: share_refused climbing while digest_folded climbs with it, and age_retired holding at 0 until a ladder's tail is fourteen days old.

2026-10-18 - #143 - the Watchdog names one encoding for the child, the pipe and the log

The Watchdog launches the platform, reads back everything it prints, and writes that into `console_<timestamp>.log`. Three ends carry the same text: what the child writes, what the pipe hands back, and what the file holds. A text stream that names no encoding falls back to the machine's locale, so each end picked its own and the operator's machine decided which.

The rule: one encoding, named at every end. `acervator_watchdog.py` holds it as two values.

`acervator_watchdog.py` — the one encoding and its error policy

```
CHILD_IO_ENCODING = "utf-8"
CHILD_IO_ERRORS = "replace"
```

Four sites read them. `ChildRunner.child_env` sets `PYTHONIOENCODING` for the child, so the child writes UTF-8. `ChildRunner._start_internal` names the same encoding on the pipe it reads and on the console log it opens. `ChildRunner._rotate_if_needed` names it again on the two files it reopens. `_set_relay_encoding` sets this process's own `sys.stdout`, which `ChildRunner._reader_loop` relays each child line to.

Both launch branches, frozen and source tree, build their environment in `ChildRunner.child_env`, so the two ends cannot drift apart.

What a failure cost before this

Driven on a Windows host whose locale reads `cp1252`, with a child made to print an arrow, an em dash and a zero-width joiner:

what the ends named	what happened
neither end, the shape before this	**the child died**, return code 1, and the log held its traceback instead of the line
the pipe only	the reader thread raised and the whole stream was lost
the child only	the reader thread raised and the whole stream was lost
both ends	return code 0, all three glyphs on disk and in the tail

Only the last row works. Naming the encoding at one end alone makes it worse, not better.

Why the error policy is not strict

`CHILD_IO_ERRORS` is `replace` at both ends, and each half earns its place.

where	strict instead	`replace`
the child's own output	a lone surrogate, which a Windows filename can carry, kills the child	the child survives and the glyph is substituted
the pipe the parent reads	a byte that is not UTF-8 loses the whole stream, 0 characters read	47 characters read, nothing lost

The machine identity read at launch

`src/core/instance_guard.py` decides whether another copy already holds the lock. It reads the machine identity before the window opens. `_machine_id_darwin` runs `ioreg` and `_machine_id_linux` reads `/etc/machine-id`. Both now name the encoding and both use `replace`, because a `UnicodeDecodeError` is not one of the faults either handler catches, and an uncaught one there ends the launch.

Measured on the one encoding

Two instruments ran over `git ls-files '*.py'`, 615 files, each proved on a planted fixture pair first.

```
shape                       before      after     the control
file read and file write    3 sites     3 sites   250 calls name the encoding
the pipe read              14 sites    11 sites    20 -> 23 calls name it
```

The three sites that moved are the two Watchdog launches and the `ioreg` call. The count is conserved: three left the target column and three entered the control column. Of the 3 file sites, two are not file objects at all, and the third is in the paused USB key reader.

The remaining 11 pipe sites sit outside the launch path: the paused USB key reader holds 3, the build and gate tooling holds 4, and the review harness holds 4.

The drive ran on a scratch home with `HOME`, `USERPROFILE`, `LOCALAPPDATA` and `APPDATA` redirected before the first import and `Path.home()` printed to prove it. It wrote one file, under that scratch home. What a real launch does to a real console log is unmeasured until one runs. What to watch for: a `console_<timestamp>.log` that holds an arrow or an em dash where it used to hold a `UnicodeEncodeError` traceback, and a `watchdog_errors.log` with no `_reader_loop: could not relay` line.

Settings

This window sets who you are, which venues the app can reach, and how new bots start. It holds nine pages above one Cancel button and one Save button.

This page is for lookup. Each row of a control table names a control or a group of them, says what it does, and cites the code behind it. A row with a stored key names it after Reads. The page lists and the route and vote tables are summaries. A row that holds a value gives its range and start value.

The pages

Page	What it sets
User	Your name and the Fleet Lock password.
Exchanges	Each venue's keys, for the asset class active when the window opened, or the bot's own class when Start opened it.
Trading	Where a new bot's wizard opens: target, order visibility and aggressive mode.
TA Indicators	How far each of the twelve indicator votes counts.
Phantom Bots	Where a new bot's phantom choices open.
Theme	The colour theme and the accent colour.
Sound	Which events play a sound, and how loud.
SMS	Text message alerts: how they go, which events send them, and how often.
AI Monitor	The API key, the handshake phrases and the monitor's behaviour.

Only the Exchanges page changes with the asset class. The title bar reads Settings — Crypto Wing on the Crypto class and Settings on every other class (`src/gui/settings_dialog.py, in SettingsDialog.__init__`).

The React page draws a second title line. It reads Settings — Crypto Wing on Crypto, Settings — Stock Wing on Stock, and Settings — Crypto Wing on every other class (`src/gui/main_tabs/settings_dialog_surface.py, in wing_or_default`, `src/gui/main_tabs/settings_dialog_surface.py, in title_for`).

How the window opens

Route	What opens and what follows
File > Settings	Opens on the User page. `src/gui/main_window.py, in MainWindow._open_settings`
Exchange > Add Exchange, or an Add button on the Trading tab	Opens on the Exchanges page. `src/gui/main_window.py, in MainWindow._add_exchange`
Start on a broker, or on Robinhood on the Crypto class, with no stored key	Opens on the Exchanges page with that venue pressed, for the bot's asset class. A class that does not list the venue gives way to the first class that does. A venue no class lists stays unpressed unless the store holds it, and a stored one is pressed on its not-offered row. In both cases the feedback line says why. Start tries again when the window closes. `src/gui/main_window.py, in MainWindow._ask_for_venue_connection`, `src/gui/main_tabs/settings_dialog_surface.py, in wing_showing_venue`

One method builds the window on all three routes (`src/gui/main_window.py, in MainWindow._run_settings_window`). On each one, a Save reaches the running window at once.

An exchange you add or remove changes the tabs when the window closes, on both builds. A new tab draws its bot table at that moment (`src/gui/main_window.py, in MainWindow._sync_exchange_tabs`). A removed venue leaves the tab bars, the React Live page and the list of connectors (`src/gui/main_window.py, in MainWindow._forget_venue`).

A venue the store no longer lists keeps its tab and its connection while a bot on it is neither idle nor stopped, or while the fleet is sealed. The Activity Log names the venue and says why, at each close

(`src/gui/main_window.py, in MainWindow._keep_unlisted_venues`). Start refuses a broker the store no longer holds, even while its connection is open (`src/gui/main_window.py, in MainWindow._open_broker_session`).

A key that changes in the window releases that venue's connection when the window closes, and the next Start connects with the new key. A live bot keeps its connection, and the Activity Log says the new key takes effect at that bot's next Start (`src/gui/main_window.py, in MainWindow._release_changed_keys`).

Two builds of this window

A frozen build draws the variant stamped into it. A source run reads `ACERVATOR_VARIANT` and draws React when nothing names a variant (`src/_variant.py, in resolve_variant`).

The React window inherits Save, Test, Add, Remove and Fleet Lock from the Qt window, and replaces only the methods that draw (`src/gui/react_settings_dialog.py, in SettingsDialogReact`).

Seven things differ on screen:

- The React build hides what you type in API Key and API Secret.
- A form row the page hides loses its box in React and keeps its words (`src/gui/web/settings_dialog.js, in ControlRow`).
- React draws no dash for a stored value a control cannot hold.
- A typed accent paints the Qt window alone.
- React draws a title line inside the page (`src/gui/web/settings_dialog.js, in Dialog`).
- The React SFX Volume figure reads 70% until the slider moves, whatever is stored (`src/gui/react_settings_dialog.py, in SettingsDialogReact.apply_edit`).
- React number boxes print no fixed decimals (`src/gui/web/settings_dialog.js, in Control`).

What each page persists

Save writes the settings file in your home folder. The Exchanges page writes at each press instead. The store refuses a key it does not declare and a value of the wrong type (`src/core/settings.py, in SettingsManager.set`).

The Twilio Auth Token, SMTP Password and Anthropic API Key rows hide only the screen. Those three go into the file as plain text. Passphrase and venue keys go in encrypted, and the two Fleet Lock rows and Verification Code are never stored (`src/gui/settings_dialog.py, in SettingsDialog._stored_groups`).

Page	Stored as	Takes effect
User	`username`	At Save. Fleet Lock writes its own key file.
Exchanges	`exchanges`	At each press, with no Save.
Trading	`default_target_balance`, `bot_visibility`, `aggressive_trading`	At the next bot wizard.
TA Indicators	`ta_indicator_weights`	At Save, on every Scrumming bot.
Phantom Bots	`default_enable_phantoms`, `default_phantom_timeframe`, `default_lock_candle_count`	At the next bot wizard.
Theme	`theme`, `accent_color`	At Save.

Page	Stored as	Takes effect
Sound	`sound`	At each tick, and Save keeps it.
SMS	`message_channels`	At Save.
AI Monitor	`ai_monitor`	At Save.

Control	What it does and what it reads
Page tabs	Move between pages. Edits stay held until Save, Cancel or a passing Add, which closes the window and drops every unsaved edit. Each tab's mouse-over says what the page holds. `src/gui/main_tabs/settings_dialog_surface.py, in TAB_TOOLTIPS`
Save	Asks the Username and Fleet Lock questions first, then writes every page but Exchanges and closes the window. A key typed with no Add press is dropped. A group the store refuses opens a box that names it, and the rest still save. `src/gui/settings_dialog.py, in SettingsDialog._save`
Cancel	Closes the window and writes nothing. A Sound box you ticked has already reached the engine. `src/gui/settings_dialog.py, in SettingsDialog._setup_ui`
A stored value a control cannot hold	A Qt number box shows a dash. A Qt slider and every React control keep a figure, and a mouse-over says the stored value is unusable. Save keeps the stored value until you set another. A top-level value of the wrong type falls back to the default at load, shows no mark, and Save writes the default over it. A stored Carrier the list lacks shows AT&T with no mark, and Save writes AT&T. A Save replaces a stored theme name no theme holds. `src/gui/settings_dialog.py, in SettingsDialog._mark_refused`, `src/core/settings.py, in SettingsManager._apply_dict`, `src/gui/settings_dialog.py, in SettingsDialog._sms_rows`, `src/gui/main_window.py, in MainWindow._remember_theme`

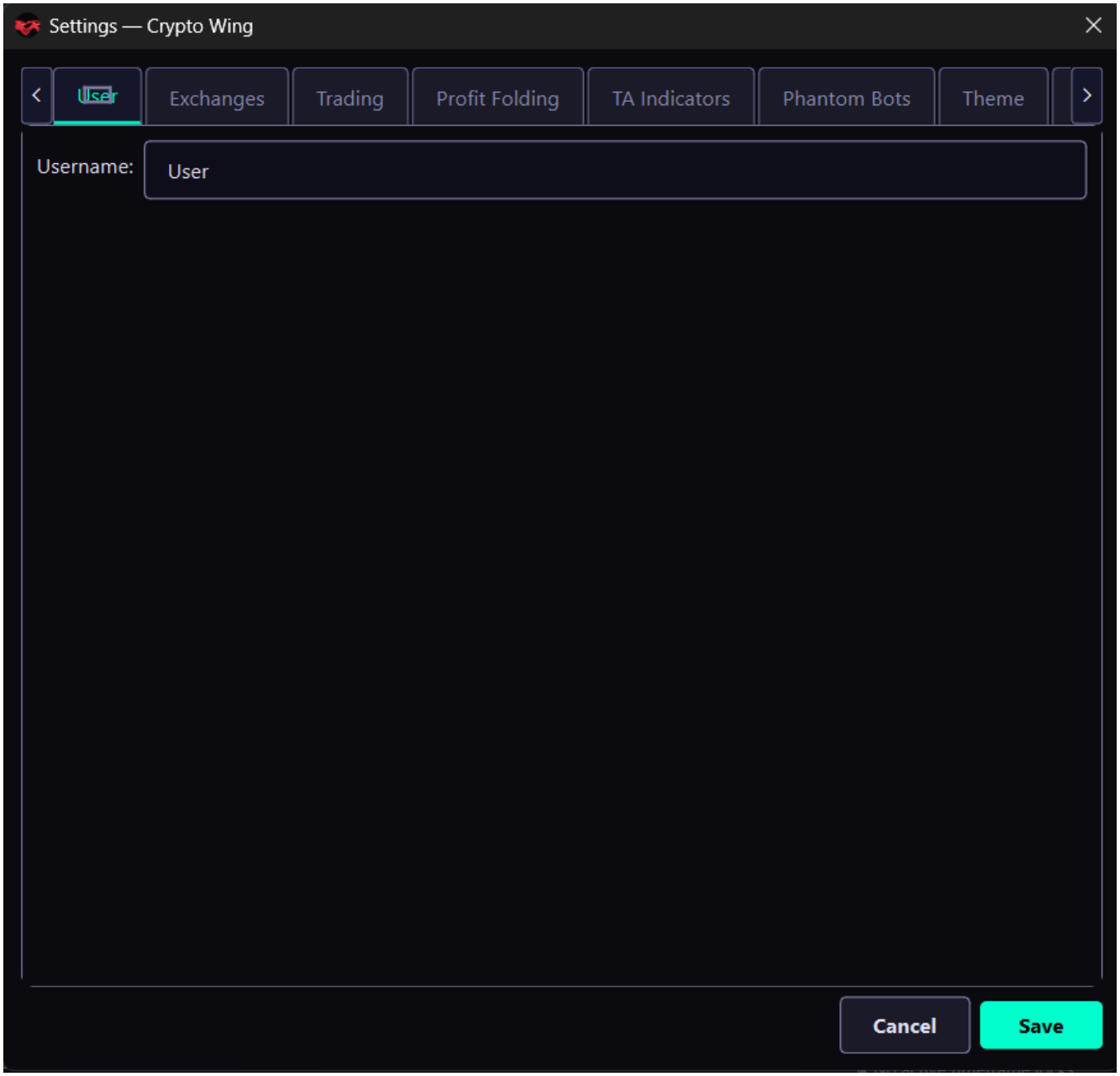
How to use each page

A row that holds a value gives its range and start value in the first column. The twelve TA sliders give their start value there and share the range in the line above their table. Nine of the ten figures show an earlier capture, so their tab row lists Profit Folding or Logging pages the window does not draw. The Phantom Bots figure matches the drawn page.

Settings > User Tab

This needs to be built out to accept and contain individual end user credentials. This will also be where a license is entered or a keyfile is imported depending on how we design the authentication piece.

Now: this page holds the Username row and the Fleet Lock group, venue keys go on the Exchanges page, and no license or keyfile row exists (`src/gui/settings_dialog.py, in SettingsDialog._create_user_tab`).



Settings, the User page.

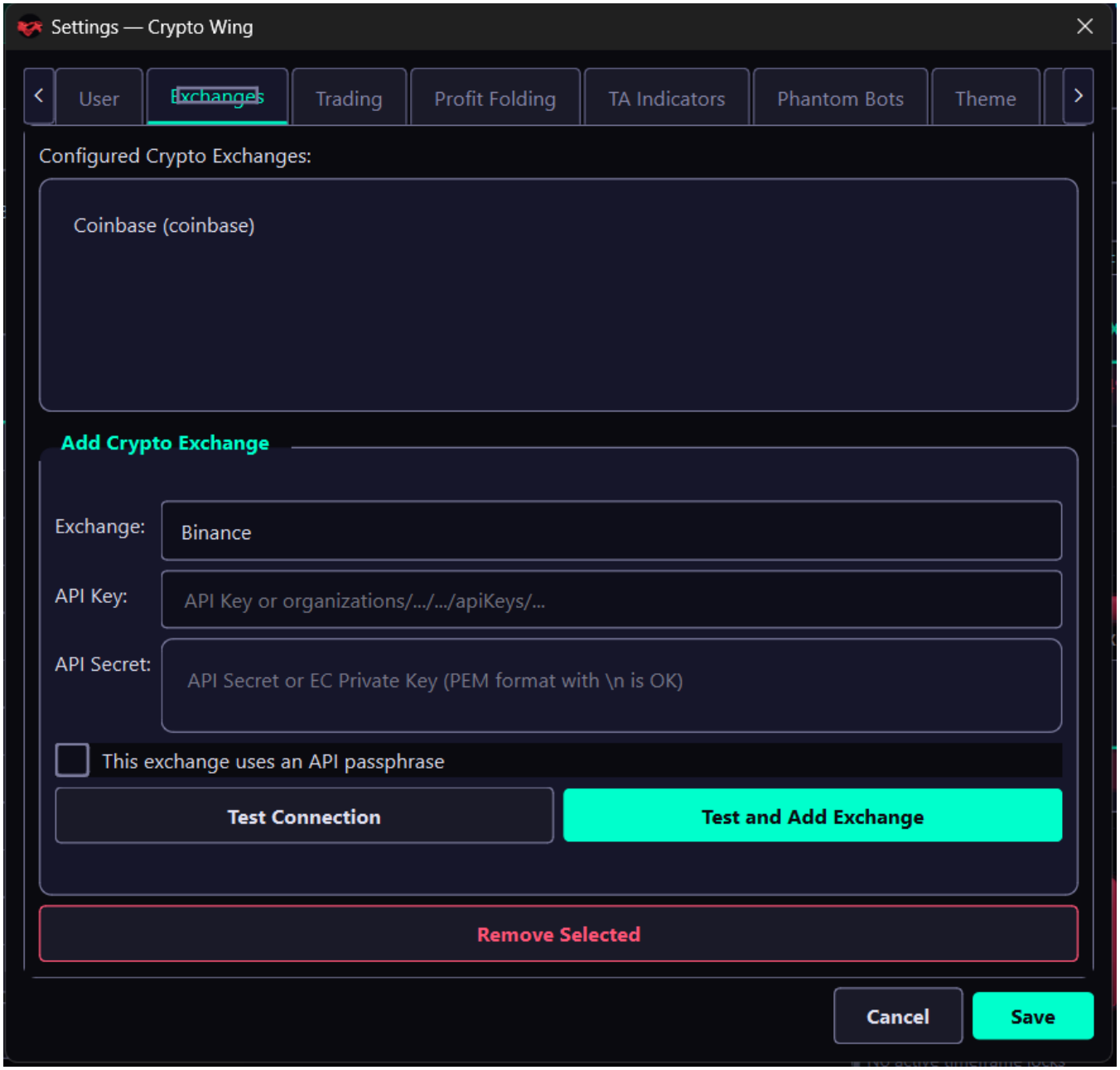
The figure shows the Username row alone. The drawn page adds the Fleet Lock group under it.

Row	What it does and what it reads
Username (text, empty)	Names you. A change over keys on the Exchanges page asks Yes or No at Save. With no lock, those keys are sealed under this name, so Yes reseals them under the new name and No puts the old name back. With a lock, Yes changes the name alone. With no key on that page no question comes, and Market Inspector sign-ins stop opening. Reads `username`. `src/gui/settings_dialog.py, in SettingsDialog._settle_username_rename`, `src/gui/main_tabs/settings_dialog_surface.py, in username_rename_consequence`
Fleet Lock state line	Says whether a password is set, read off the key file. A set password seals the saved fleet and the stored keys, and a launch starts no bot until you give it. While Clear the lock is pressed in, this line shows the clear question. `src/gui/settings_dialog.py, in SettingsDialog._show_fleet_lock_state`

Row	What it does and what it reads
Password (hidden, empty)	Sets a lock, or changes one this run has opened. A change before you unlock is refused, and the log says why. Both rows empty changes nothing. Save asks Yes or No first, because nothing can recover a lost password. `src/gui/main_tabs/settings_dialog_surface.py, in fleet_lock_intent`
Confirm Password (hidden, empty)	Must match Password, or Save takes neither and logs why. It greys while Clear the lock is pressed in. `src/gui/main_tabs/settings_dialog_surface.py, in fleet_lock_intent`
Unlock the fleet now	Opens the unlock screen. The right password loads the sealed fleet, runs the launch check and starts every idle or stopped bot. It works only while a lock is set and this run has not opened it. Press it before you add a venue key, because a key added first does not open afterwards. `src/gui/fleet_unlock_dialog.py, in ask_for_unlock`, `src/gui/main_window.py, in MainWindow.unlock_fleet`
Clear the lock	Greys with no lock set. It stays pressed in until you press it again or Save. Save needs the current password in the Password row and asks Yes or No. With the right password, Yes writes the saved fleet back as plain text, reseals stored keys under your Username and deletes the key file. A wrong password logs a refusal and keeps the lock. The question on screen says the keys are lost, and the code keeps them. `src/gui/settings_dialog.py, in SettingsDialog._clear_fleet_lock`

Change Username and the lock in separate Saves. One Save that renames and also sets or clears the lock leaves stored keys that do not open (`src/gui/settings_dialog.py, in SettingsDialog._username_phrase`).

Settings > Exchanges



Settings, the Exchanges page.

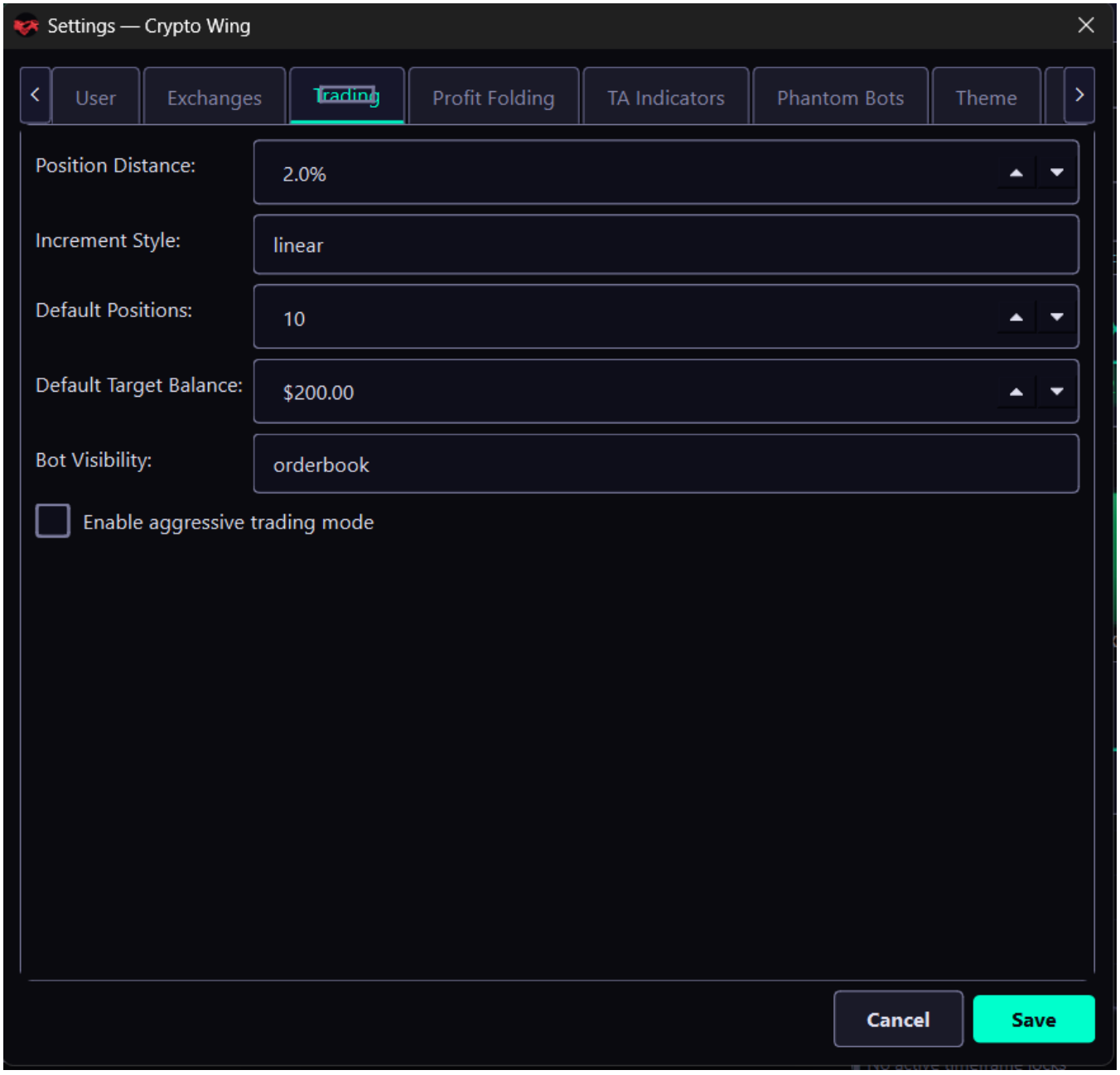
The figure shows a drop-down and a list of stored exchanges. The drawn page puts the Exchange Status buttons where the list sat, and the Exchange row is plain words.

Row	What it does and what it reads
Stock banner	Stock class only. It names the brokers the app builds a connector for, and says how each signs in. `src/gui/main_tabs/settings_dialog_surface.py, in stock_banner_text`
Exchange Status buttons	One button per venue the class serves, 16 to a page and four across. A stored venue no class lists follows in amber. A press binds the form below, and the mouse-over names the venue's state. An amber press greys Test and Add and hides the key rows, so only Remove Selected acts. A redraw reads a file and calls no venue. `src/gui/main_tabs/settings_dialog_surface.py, in exchange_status_page`

Row	What it does and what it reads
Button colour	Green: the venue answered an authenticated call. Red: the connection was lost. Grey: never asked, the venue refused a key, or a key whose shape alone was checked. Amber: stored, and listed under no class. `src/gui/main_tabs/settings_dialog_surface.py, in VENUE_STATE_COLOURS`
Page button	Shows past 16 venues. A press moves to the next page, and the last wraps to the first. `src/gui/main_tabs/settings_dialog_surface.py, in venue_pages_button`
Exchange row	Names the pressed venue, and you cannot type in it. `src/gui/main_tabs/settings_dialog_surface.py, in chosen_venue_text`
API Key (text, empty)	The venue's key, or a Coinbase key name. The React build hides what you type, and the Qt build shows it. `src/gui/main_tabs/settings_dialog_surface.py, in CONTROL_SPECS`, `src/gui/settings_dialog.py, in SettingsDialog._create_exchange_tab`
API Secret (text, empty)	The venue's secret or private key. Escaped newlines in a pasted PEM key turn into real ones. The React build hides what you type, and the Qt build shows it. `src/gui/main_tabs/settings_dialog_surface.py, in CONTROL_SPECS`, `src/gui/settings_dialog.py, in SettingsDialog._typed_secret`
This exchange uses an API passphrase (off)	Shows the passphrase row. A press on a venue that needs one ticks it for you. `src/gui/settings_dialog.py, in SettingsDialog._on_exchange_changed`
Passphrase (hidden, empty)	The passphrase set on the API key. Test refuses a venue that needs one when this row is empty. `src/exchange/api_validator.py, in validate_credentials`
Verification Code (hidden, empty)	Shows for a venue that grants access only on a pasted code. The first Add press returns an address. Open it, approve, paste the code and press Add again. The code is used once and never stored. `src/gui/main_tabs/settings_dialog_surface.py, in connect_verification_code_venue`
Test Connection	Checks the typed key and secret and shows the balances. It stores no key, and it colours the button by the venue's answer. A hand-written connector checks the key's shape alone and asks no venue. It greys for a venue with no check path, and it refuses with no venue pressed or a blank row. `src/gui/settings_dialog.py, in SettingsDialog._test_api_connection`
Test and Add Exchange	A typed key and secret on a venue Test can check must pass first, or nothing is stored. A venue with no check path stores them unchecked, except E*TRADE, which stores them only after the venue accepts the pasted code. With either row blank, Add runs no check and stores no typed key, and a stored venue keeps its key. A verification-code venue refuses a blank row and stores nothing. Keys are stored encrypted. A passing Add shows Exchange Added and closes the window. The box says the tab will appear for a venue the store did not hold, and says it is already in the main window for a stored one. `src/gui/settings_dialog.py, in SettingsDialog._add_exchange`, `src/gui/settings_dialog.py, in SettingsDialog._connect_verification_code_venue`
Connect with a Browser	Replaces Test and Add for a venue that signs in at a browser, on the classes that sign in that way. The key rows hide and Test greys. Approve at the venue's page, and the feedback line names its trade-approval setting. The window and every bot wait until the venue answers, up to 180 seconds, and then this window stays open. On Robinhood's Crypto class the key rows show, and the feedback line asks for the API key and secret. `src/gui/settings_dialog.py, in SettingsDialog._connect_browser_venue`, `src/trading/ata_spm_signin.py, in LoopbackReceiver.wait`, `src/gui/main_tabs/settings_dialog_surface.py, in venue_form_bound`
Feedback line	Shows the last answer: blue for a note, green for success, amber for a warning and red for an error. `src/gui/settings_dialog.py, in SettingsDialog._set_feedback`

Row	What it does and what it reads
Remove Selected	Removes the pressed venue's stored entry and its colour at once, with no Save. The press then lets go of the venue, on both builds, and the form returns to its opening shape. With no venue pressed it asks you to press one. It refuses while a bot on that venue is neither idle nor stopped, while a Start is taking a bot there through its connect, and while the fleet is sealed. The line names each bot, or says to unlock first. With orders or Stack rungs of a stopped bot resting at the venue it asks Yes or No. The box names how many rest, and says they stay at the venue and that nothing will read their fills. Yes reads the fleet again and then removes the venue, and No keeps it. `src/gui/settings_dialog.py, in SettingsDialog._remove_exchange`, `src/gui/main_tabs/settings_dialog_surface.py, in remove_refusal`, `src/gui/main_tabs/settings_dialog_surface.py, in remove_question`

Settings > Trading



Settings, the Trading page.

The figure shows Position Distance, Increment Style and Default Positions rows the page does not draw.

Each row sets where a bot wizard opens. A running bot keeps its own value.

Row	What it does and what it reads
Default Target Balance ($1 to $1,000,000, $200)	Opens the wizard's Target Balance. A wizard opened by a Market Inspector topology Adopt press opens on the proposal's suggested target instead. Reads `default_target_balance`. `src/gui/bot_wizard.py, in _show_stored_target`, `src/gui/main_tabs/bot_wizard_surface.py, in BotWizardModel._apply_stored_target_balance`, `src/gui/main_window.py, in MainWindow._adopt_topology_proposal`
Bot Visibility (orderbook or internal, orderbook)	Opens the wizard's Order Visibility. Reads `bot_visibility`. `src/gui/bot_wizard.py, in TradingParamsPage`, `src/gui/main_tabs/bot_wizard_surface.py, in BotWizardModel._apply_stored_visibility`
Enable aggressive trading mode (off)	Opens the wizard's Aggressive Trading. The engine changes only Stack tranches in Order Book mode, which go out as immediate-or-cancel limits. A tranche above the market cancels unfilled, and Robinhood on the Crypto class refuses each one. The mouse-over says it changes every buy and sell. Reads `aggressive_trading`. `src/trading/scrumming_bot.py, in ScrummingBot._open_stack_from_scrum`

Settings > TA Indicators



Settings, the TA Indicators page.

Each slider runs 0.00 to 2.00 and sets how far that indicator's vote counts. Save hands all twelve to every Scrumming bot at once, and a bot at target shows its old reading until the next candle (`src/trading/bot_container.py, in BotManager.push_ta_weights`). Reads `ta_indicator_weights`.

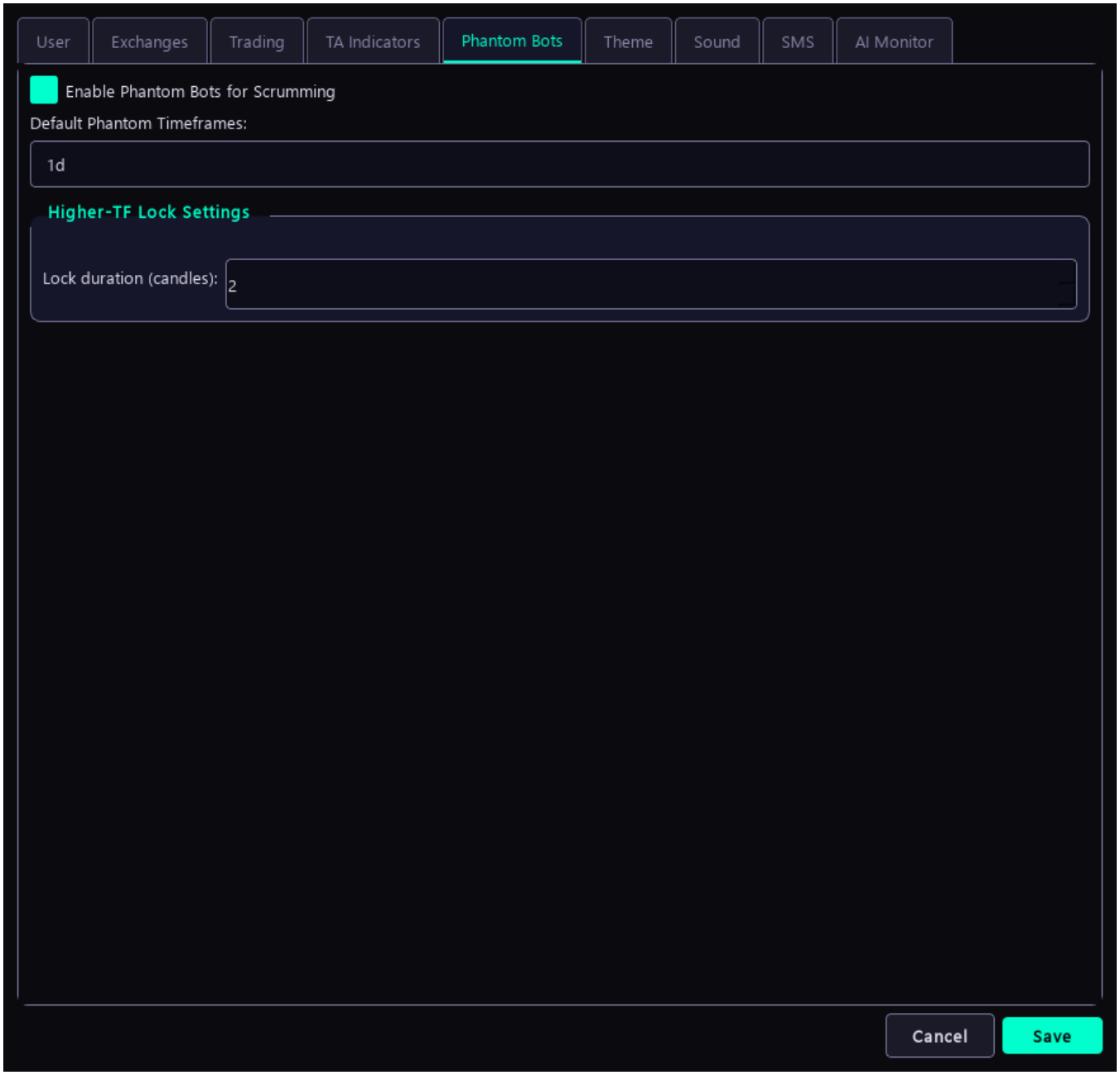
Slider (start)	What it weighs
Bollinger Bands (1.00)	The Bollinger Bands vote. `src/trading/indicators/bollinger.py, in BollingerBands`
Vortex (0.90)	The Vortex vote. `src/trading/indicators/vortex.py, in VortexIndicator`

Slider (start)	What it weighs
Macd (1.20)	The MACD vote. `src/trading/indicators/macd.py, in MACD`
Stochastic Rsi (1.00)	The Stochastic RSI vote. `src/trading/indicators/stochastic_rsi.py, in StochasticRSI`
Ichimoku (1.10)	The Ichimoku Cloud vote. `src/trading/indicators/ichimoku.py, in IchimokuCloud`
Volume (0.80)	The volume vote. `src/trading/indicators/volume.py, in VolumeAnalysis`
Slingshot (1.00)	The Slingshot vote. `src/trading/indicators/slingshot.py, in SlingshotIndicator`
Adx (1.00)	The ADX vote. `src/trading/indicators/adx.py, in ADXIndicator`
Kaufman Er (1.00)	The Kaufman Efficiency Ratio vote. `src/trading/indicators/kaufman_er.py, in KaufmanERIndicator`
Supertrend (1.00)	The Supertrend vote. `src/trading/indicators/supertrend.py, in SupertrendIndicator`
Zscore (0.90)	The Z-Score vote. `src/trading/indicators/zscore.py, in ZScoreIndicator`
Rsi (0.80)	The RSI vote. `src/trading/indicators/rsi.py, in RSIIndicator`

Vote	Which figures it reads
A running Scrumming bot	These sliders. `src/trading/scrumming_bot.py, in ScrummingBot.set_ta_weights`
A new Scrumming bot from the Live wizard	Twelve zeros until the next Save on this window. It has no vote, so it makes no first buy. If it holds a position, a landing strip still sets its side. `src/gui/main_tabs/bot_wizard_surface.py, in with_new_bot_weights`
A running phantom	The figures its set was built with, so a wizard bot started before that Save keeps a twelve-zero phantom until it stops. `src/trading/scrumming_bot.py, in ScrummingBot.set_ta_weights`
A bot's first-entry check	These sliders, through the same vote as its later trades, at every Max Cartridge Size. It waits for a third indicator. A bot at twelve zeros waits for Save, and its line says so. `src/trading/scrumming/tick_phases.py, in TickPhaseMixin._first_buy_read`
A Simulator run	These sliders, read from the file when the run starts. `src/simulator/back_test.py, in walk`
A Paper run	The start figures. `src/paper/paper_run.py, in PaperRunner.__init__`
An Extractor	The start figures. `src/trading/extractor_bot.py, in ExtractorBot.__init__`
A Market Inspector scan	The start figures. `src/trading/ata_spm.py, in evaluate`

07-indicators.md holds the indicator formulae.

Settings > Phantom Bots



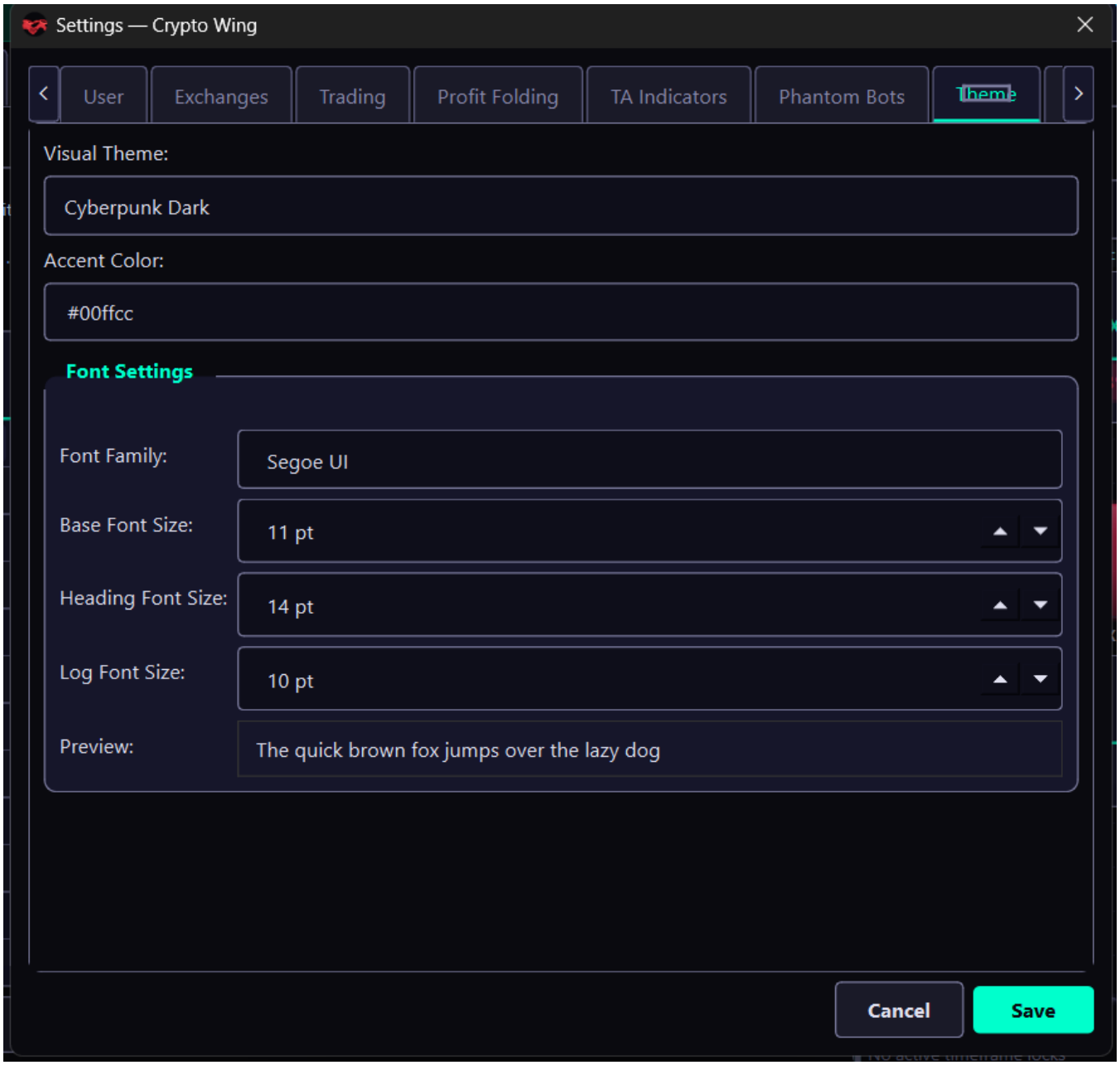
Settings, the Phantom Bots page.

Each row sets where a bot wizard's phantom choices open.

Row	What it does and what it reads
Enable Phantom Bots for Scrumming (on)	Opens the wizard's Enable Phantom Bots box. Reads `default_enable_phantoms`. `src/gui/bot_wizard.py, in PhantomConfigPage`

Row	What it does and what it reads
Default Phantom Timeframes (1m to 1w, 1d)	Names the one timeframe the wizard ticks. One at or below the bot's own, or one the venue lacks, stays clear and its box says why. With Enable on and no box ticked, the bot runs one phantom on the next timeframe the venue offers, and none where it offers none higher. Reads `default_phantom_timeframe`. `src/gui/bot_wizard.py, in PhantomConfigPage._timeframe_refusal`, `src/trading/phantom_balance.py, in default_phantom_timeframes`
Lock duration (candles) (1 to 10, 2)	Opens the wizard's Candles to lock. Nothing creates a lock, so nothing uses it. Reads `default_lock_candle_count`. `src/trading/phantom_balance.py, in TimeframeCoordinator`

Settings > Theme



Settings, the Theme page.

The figure shows a Font Settings group the page does not draw, and #00ffcc in Accent Color, where the drawn box opens empty.

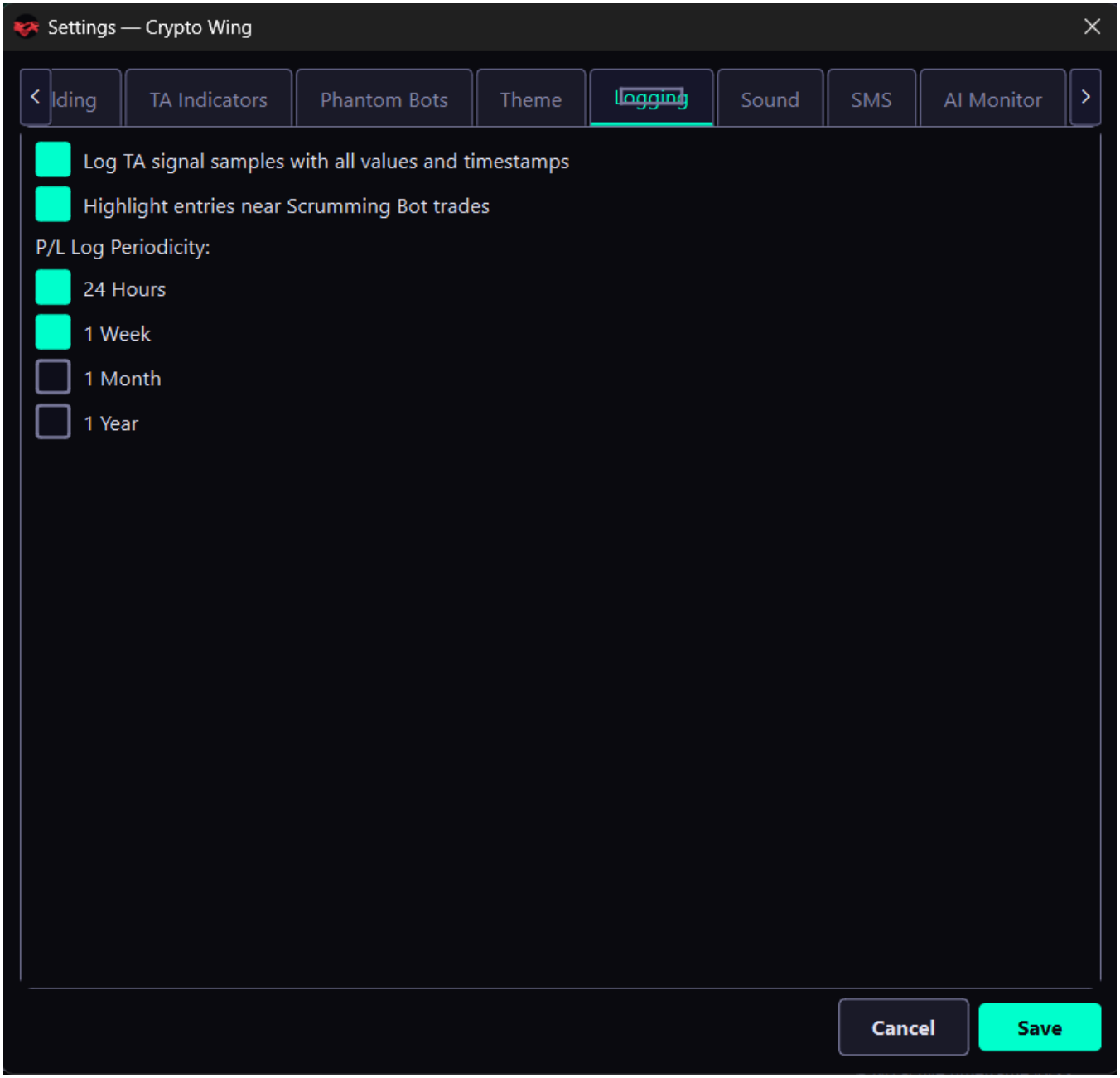
Row	What it does and what it reads
Visual Theme (five themes, Cyberpunk Dark)	Picks Cyberpunk Dark, Neon Light, Classic Terminal, Minimal Modern or Glass & Metal. Save repaints the Qt window and the chrome colours of each open React page. The Theme menu writes the same setting. A stored name no theme holds opens on Cyberpunk Dark, and a Save writes that name over it. Reads `theme`. `src/gui/main_window.py, in MainWindow._switch_theme`, `src/gui/main_window.py, in MainWindow._remember_theme`
Accent Color (text, empty)	A hex colour of three or six digits replaces the accent on Qt parts and the chart, and empty keeps the theme's own. React pages keep the theme's own either way. Other text is refused, and so is a colour under 4.5 to 1 against the theme's ground. The log names a refusal. Reads `accent_color`. `src/gui/theme_engine.py, in accented`, `src/gui/react_history_panel.py, in _palette`

The Simulator's tone, nigredo

On the Sim tab, Qt windows move each theme's seven ground colours a quarter of the way to black. The accent, the text, the borders and the chart colours stay the theme's, so a Sim screen reads as the same theme, darker (`src/gui/theme_engine.py, in nigredo`).

The React Sim page darkens its two page grounds and the design system's grounds by the same share (`src/gui/simulator/sim_react_trading_tab.py, in nigredo_design`). No control sets the tone.

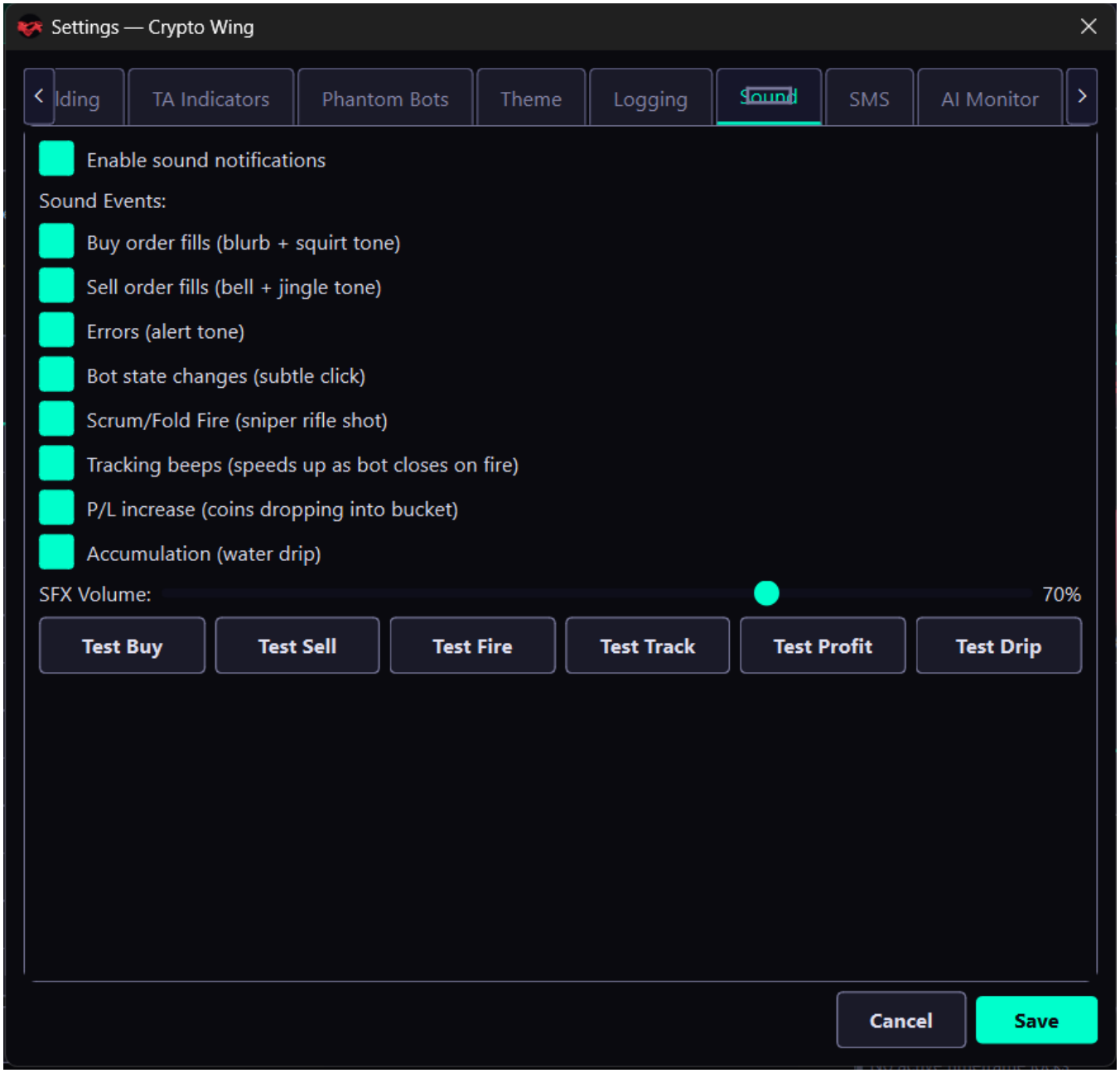
Settings > Logging



Settings, the Logging page.

The window draws no Logging page, and no row in Settings sets logging.

Settings > Sound



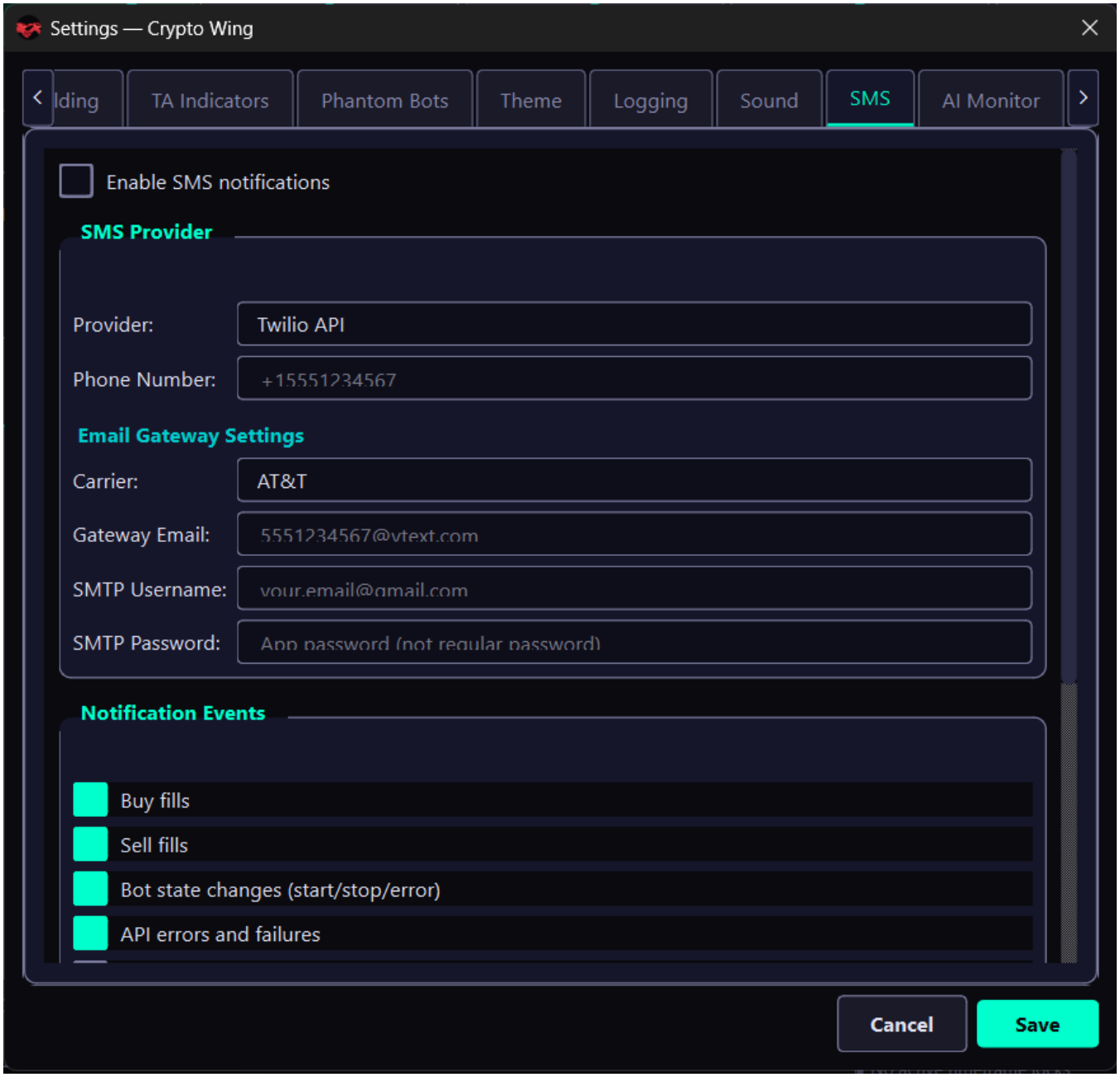
Settings, the Sound page.

Each box reaches the sound engine the moment you tick it, before Save (`src/gui/settings_dialog.py, in SettingsDialog._push_sound_config`). Save keeps the page. Cancel writes nothing, and the engine holds your tick until the window next opens or the app restarts.

These clips play on Windows alone, and each clip cuts off the one before, so a FOLD fill ends on the drip (`src/core/sound_engine.py, in SoundEngine.play`). The Market Inspector chime has no switch here (`src/gui/market_inspector.py, in MarketInspectorTab._chime_for`).

Row	What it does and what it reads
Enable sound notifications (on)	The master switch. Off silences every clip on this page. Reads `enabled`. `src/core/sound_engine.py, in SoundEngine.play`
Buy order fills (on)	Plays a rising tone on a buy fill. Reads `buy_sound`. `src/gui/main_tabs/main_window_surface.py, in trade_filled_sounds`
Sell order fills (on)	Plays a bell tone on a sell fill. Reads `sell_sound`. `src/gui/main_tabs/main_window_surface.py, in trade_filled_sounds`
Errors (on)	Plays an alert tone when Start or Restart fails, and on a critical risk alert. Reads `error_sound`. `src/gui/main_window.py, in MainWindow._on_bot_command`
Bot state changes (on)	Plays a click when Start, Pause, Stop, Restart or Delete works on one bot, when a bot is built, when Apply changes a running bot, and when a venue connection drops. Pause All and Stop All on the Live tab play nothing. On the Paper tab each plays this click. Reads `bot_state_sound`. `src/gui/main_window.py, in MainWindow._on_bot_command`, `src/gui/main_window.py, in MainWindow._global_bot_cmd`, `src/gui/paper/paper_trading_tab.py, in PaperTradingTab._global_bot_cmd`, `src/gui/paper/paper_react_trading_tab.py, in PaperTradingTabReact._global_bot_cmd`
Scrum/Fold Fire (on)	Plays a rifle shot on a SCRUM, FOLD or DIST fill, and on a Manual Fire press. Reads `fire_sound`. `src/gui/main_window.py, in MainWindow._on_bot_fire`
Tracking beeps (on)	Beeps once per two-second refresh for the hottest Scrumming bot of the active asset class, running or paused, in TRACK or FIRE. SEARCH and a bot the hard breaker holds stay silent, and the rate does not rise in FIRE. Reads `track_sound`. `src/gui/main_window.py, in MainWindow._dispatch_tracking_beep`
P/L increase (on)	Plays coins when a fill reports a profit above zero. The growth a cartridge, Manual Fire or manual tranche fire reports reads each tranche's sale price less its wire share. Reads `profit_sound`. `src/gui/main_tabs/main_window_surface.py, in trade_filled_sounds`, `src/trading/scrumming/sizing.py, in discharge_gain_usd`, `src/trading/scrumming_bot.py, in ScrummingBot._book_manual_tranche_fill`
Accumulation (on)	Plays a water drip on a FOLD fill. Reads `drip_sound`. `src/gui/main_tabs/main_window_surface.py, in trade_filled_sounds`
SFX Volume (0 to 100%, 70%)	Sets the loudness of every clip. A move reaches the engine at once, and the engine remakes each clip at that level. Reads `volume`. `src/core/sound_engine.py, in SoundEngine.update_config`
Test Buy, Test Sell, Test Fire, Test Track, Test Profit, Test Drip	Each hands the engine the page's boxes, then plays its clip. The master box and that clip's own box still apply. `src/gui/settings_dialog.py, in SettingsDialog._test_sound`

Settings > SMS



Settings, the SMS page.

The figure shows Twilio API picked, with no Twilio rows and no Mail rows. The page opens on Email-to-SMS Gateway and draws every row below, for both providers.

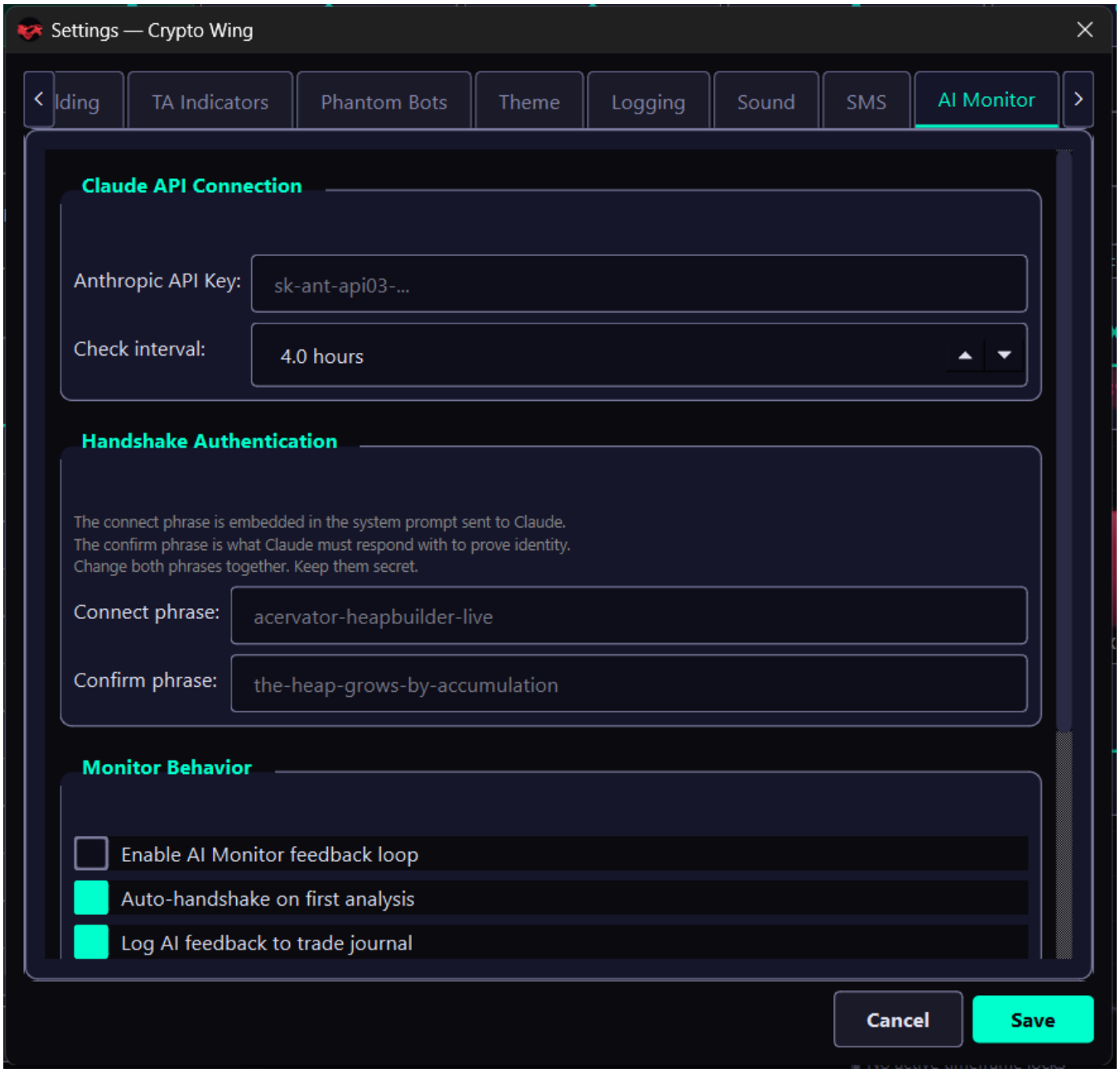
Save hands the page to the message engine at once. A message goes only while the master box and that event's box are ticked, and a message inside a rate limit is dropped (`src/core/sms_engine.py, in SMSEngine.send`).

Alerts add their own waits: 10 seconds a side on fills, 60 seconds on a connection change, one hour on P/L and ten minutes on a critical risk alert (`src/core/notifications.py, in NotificationManager.DEFAULT_RULES`).

Row	What it does and what it reads
Enable SMS notifications (off)	The master switch. Off sends nothing. Reads `enabled`. `src/core/sms_engine.py, in SMSEngine.send`
Provider (Email-to-SMS Gateway or Twilio API, Email-to-SMS Gateway)	Picks how messages go. Reads `provider`. `src/core/sms_engine.py, in SMSEngine.send`
Phone Number (text, empty)	The number Twilio sends to, and the number Carrier builds the gateway address from. Reads `phone_number`. `src/core/sms_engine.py, in SMSEngine._send_twilio`
Twilio Account SID (text, empty)	Your Twilio account. Reads `twilio_sid`. `src/core/sms_engine.py, in SMSEngine._send_twilio`
Twilio Auth Token (hidden, empty)	Your Twilio token. Reads `twilio_auth_token`. `src/core/sms_engine.py, in SMSEngine._send_twilio`
Twilio From Number (text, empty)	The Twilio number messages come from. Reads `twilio_from_number`. `src/core/sms_engine.py, in SMSEngine._send_twilio`
Carrier (ten carriers, AT&T)	A pick fills Gateway Email from a ten-digit number, or eleven with a leading 1, so type the number first. AT&T shows at the start, so to use it pick another carrier and return, or type Gateway Email yourself. Other (Manual) leaves that row as it is. Reads `carrier`. `src/core/sms_engine.py, in gateway_address`, `src/gui/settings_dialog.py, in SettingsDialog._fill_gateway_email`
Gateway Email (text, empty)	The address the email route sends to. The route sends nothing while it is empty. Reads `gateway_email`. `src/core/sms_engine.py, in SMSEngine._send_email_gateway`
SMTP Username (text, empty)	The email account that sends. The route sends nothing while it is empty. Reads `smtp_username`. `src/core/sms_engine.py, in SMSEngine._send_email_gateway`
SMTP Password (hidden, empty)	That account's app password. Reads `smtp_password`. `src/core/sms_engine.py, in SMSEngine._send_email_gateway`
Mail Server (text, smtp.gmail.com)	The outgoing mail server. Reads `smtp_server`. `src/core/sms_engine.py, in SMSEngine._send_email_gateway`
Mail Port (1 to 65535, 587)	The outgoing mail port. Reads `smtp_port`. `src/core/sms_engine.py, in SMSEngine._send_email_gateway`
Buy fills (on)	Sends a message when a buy fills. Reads `notify_buy_fills`. `src/core/notifications.py, in NotificationManager._on_trade_filled`
Sell fills (on)	Sends a message when a sell fills. Reads `notify_sell_fills`. `src/core/notifications.py, in NotificationManager._on_trade_filled`
Bot state changes (start/stop/error) (on)	Nothing raises a bot start, stop, error or risk-pause alert, so this box sends no message. Reads `notify_bot_state_changes`. `src/core/notifications.py, in EVENT_MESSAGE_KIND`
API errors and failures (on)	No alert event reaches this box, so it sends no message. Reads `notify_errors`. `src/core/notifications.py, in EVENT_MESSAGE_KIND`
P/L threshold alerts (off)	Sends a message when realised profit passes a new multiple of P/L threshold, at most one an hour, and a multiple passed inside that hour never sends. A critical risk alert sends too. Reads `notify_pl_threshold`. `src/core/notifications.py, in NotificationManager.check_pnl_milestone`
P/L threshold ($1 to $100,000, $100)	Sets that multiple. Reads `pl_threshold_amount`. `src/core/notifications.py, in NotificationManager._pnl_threshold`
Low balance warnings (off)	No alert event reaches this box, so it sends no message. Reads `notify_balance_warning`. `src/core/notifications.py, in EVENT_MESSAGE_KIND`

Row	What it does and what it reads
Exchange connection status (off)	Sends a message when a venue connection drops and when it returns. Reads `notify_connection_status`. `src/exchange/ccxt_connector.py, in _alert_venue`
Max messages per hour (1 to 100, 20)	The most messages sent in one hour. Reads `max_messages_per_hour`. `src/core/sms_engine.py, in SMSEngine.send`
Min time between messages (5 to 300 sec, 30)	The shortest wait between two messages. Reads `cooldown_seconds`. `src/core/sms_engine.py, in SMSEngine.send`

Settings > AI Monitor



Settings, the AI Monitor page.

The figure stops above the Connection Status group.

The monitor runs only with its Enable box ticked and an API key stored. Save rebuilds it at once (`src/trading/bot_container.py, in BotManager.configure_live_monitor`).

A review sends the fleet total and the bot count. Its trade lines read zero, because nothing writes the monitor's journal (`src/trading/live_monitor.py, in LiveMonitor.analyze`).

Row	What it does and what it reads
Anthropic API Key (hidden, empty)	The key the monitor calls Claude with. Reads `api_key`. `src/trading/live_monitor.py, in LiveMonitor._call`
Check interval (0.5 to 24 hours, 4)	Hours between two reviews. The first falls due at once, and each Save makes one due. A failed handshake or a call that raises is asked again each two seconds. An error reply with no text counts as a passing review. Reads `interval_hours`. `src/trading/live_monitor.py, in LiveMonitor.should_check`, `src/trading/live_monitor.py, in LiveMonitor.analyze`
Connect phrase (text, empty)	The phrase the prompt carries. Empty uses the built-in phrase. Reads `connect_phrase`. `src/trading/live_monitor.py, in LiveMonitor.__init__`
Confirm phrase (text, empty)	The reply must be this phrase alone to pass the handshake. Empty uses the built-in phrase. Reads `confirm_phrase`. `src/trading/live_monitor.py, in LiveMonitor.handshake`
Enable AI Monitor feedback loop (off)	Turns the monitor on. Reads `enabled`. `src/trading/bot_container.py, in BotManager.configure_live_monitor`
Auto-handshake on first analysis (on)	Stored as `auto_handshake`, and nothing reads it. The monitor always shakes hands before its first review. `src/trading/live_monitor.py, in LiveMonitor.analyze`
Log AI feedback to trade journal (on)	Writes nothing. The window misses each answer, because the answer arrives one level down. Reads `log_feedback`. `src/gui/main_window.py, in MainWindow._on_ai_feedback`, `src/trading/bot_container.py, in BotManager.check_live_monitor`
Status	Reads Not connected. Only Test Handshake writes it. The status bar's AI label reads the settings: a passing review turns it to UNAUTH, and a failed handshake leaves it on READY. `src/gui/main_window.py, in MainWindow._draw_ai_monitor_pill`
Journal hash	Shows a dash. Nothing writes it. `src/gui/settings_dialog.py, in SettingsDialog._create_ai_monitor_tab`
Checks completed	Shows 0. Nothing writes it. `src/gui/settings_dialog.py, in SettingsDialog._create_ai_monitor_tab`
Test Handshake	Calls nothing. It checks that the key and both phrases are filled, then writes one line to Status, and that line says Settings saved though no Save has run. `src/gui/settings_dialog.py, in SettingsDialog._test_ai_handshake`

The Promotion Pipeline

Reference. A strategy earns its way to real money in four steps. Each step is a gate: it feeds the next one, and the next one refuses what the previous one could not produce.

Diagram: flowchart, left to right

- Market Inspector — scans markets, proposes topologies -> **Simulator — Stone Tablet history**
- Simulator — Stone Tablet history -> **Paper Trader — live feed, fake budget**
- Paper Trader — live feed, fake budget -> **Live — real orders on the venue**

- Market Inspector — scans markets, proposes topologies ..> **Live — real orders on the venue** *(adopt)*

What each step contributes

Step	Data source	What it must produce
Market Inspector	Higher-timeframe OHLCV from the connected venues	A signal table, an opposing-pair table and topology proposals
Simulator	Stone Tablets, read-only	The same gates latching on the same candles as live
Paper Trader	The live feed, in real time, against a budget that is not real	A trade record the History surface can grade
Live	The venue	Filled orders, and the exchange values every ledger reconciles to

One logic, three data sources

Live, Paper and the Simulator differ in one thing only: where the candles and the balances come from. The trading logic stays one body of pure code that all three call. Only the stateful shells fork, which keeps a simulated run from touching a live object.

The Simulator's fleet was the worked example. It built real bots against a fake exchange rather than a second bot class, and that fake exchange was a subclass of the same interface the live connector implements.

`FleetSimExchange`, in the Simulator fleet package the rebuild deleted

```
class FleetSimExchange(ExchangeInterface):
    """Real-symbol candle-driven fake exchange for Fleet Replay.
```

The rebuild replaced it. The Sim tab now reads Stone Tablet files through `TabletSource`, which holds no venue and defines no write.

The dashed edge

One edge skips the two middle steps. The Market Inspector's adopt path creates real bots and draws real wires on the live fleet, which is why the handler opens a confirmation naming the new bot count, the new budget and every wire the adopt would change.

`src/gui/main_window.py` — `_topology_wire_collisions`

```
def _topology_wire_collisions(
    self, wires: list, asset_to_bot: dict
) -> list[dict]:
    """Return one dict per proposal wire whose bot pair is already wired."""
```

The read-only edge into the Simulator was a different hook, and it said so in its own words. It let the sim replay a proposal's shape across sim bots and created nothing. The rebuild removed that hook, so no proposal reaches the Sim tab today.

`set_topology_getter`, in the Simulator tab the rebuild deleted

```
    READ ONLY. This is not the adopt path — adopting a proposal
    creates real bots and wires on the live fleet, while this only
    lets the simulator replay a proposal's shape across sim bots.
    """
```

Where the chain breaks today

The Paper Trader step has a module now, and issue #19 built it. No step hands its result to the next one: the Simulator hands nothing forward, Paper loads its own fleet from the stored bot record, and Live receives from the Market Inspector's adopt path instead. See paper-trader.md.

Back to the subsystem index.

Notifications and Alerts

Reference. Two channel forms and a Save button on the left. The Event Routing table and the Notification History table on the right.

What the Event Routing table is for

The table is the map from an event to the channels that carry it. One row per event. One column per channel. A cell reads `Yes` when the event reaches that channel and `No` when it does not.

The operator clicks a routing cell to change it. The click writes the channel list the notification manager reads, so the next alert of that event follows the new reading.

Diagram: flowchart, left to right

- click a routing cell -> **alerts_tab_surface, in toggle_route**
- alerts_tab_surface, in toggle_route -> **notifications, in set_route**
- notifications, in set_route -> **notifications, in set_rule**
- notifications, in set_rule -> **AlertRule.channels**
- a filled trade -> **notifications, in send**
- AlertRule.channels -> **notifications, in send**
- notifications, in send -> **the channel handler**

`src/gui/alerts_tab.py, in _toggle_route` is the one method both builds enter. The Qt table reaches it from `cellClicked`. The drawn page reaches it from `src/gui/react_alerts_tab.py, in _apply_request`, which reads the `route` the page sends. `src/gui/main_tabs/alerts_tab_surface.py, in toggle_route` holds the flip itself, so one gesture writes one rule.

The columns come from one declaration

`src/gui/main_tabs/alerts_tab_surface.py`, `ROUTING_COLUMNS` names one channel per column with the heading above it. `RULES_COLUMNS`, `RULES_COLUMN_COUNT` and `ROUTED_CHANNELS` are all read from it.

Column	Channel	What carries the message
Event	—	the event name, titled
Priority	—	the rule's priority word
In-App	`IN_APP`	the log spool the window reads
Telegram	`TELEGRAM`	the Telegram Bot API
SMS	`SMS`	`src/core/sms_engine.py, in send`
Sound	`SOUND`	the alert sound engine

The Qt table takes its count and its headings from those names. The drawn page takes the same headings from the `rules_columns` field that `src/gui/main_tabs/alerts_tab_surface.py, in build_view_model` writes. Removing one entry from `ROUTING_COLUMNS` removes that column from both builds together, so the two cannot disagree on a channel.

Which events reach the message channels

`src/core/notifications.py`, `EVENT_MESSAGE_KIND` pairs an event with the `src/core/sms_engine.py` switch that gates it. An event listed there is given the Telegram and SMS channels by `src/core/notifications.py, in message_channels_for`. An event left out reaches the in-app spool and the sound engine alone.

Eleven of the fourteen events are paired today. `Buy filled` and `Sell filled` are among them, so a filled trade reaches in-app, Telegram and SMS.

The switch on `Settings > SMS` still gates the send. `src/core/sms_engine.py, in send` refuses and writes the refusal to the log when the group is off, when no switch names the event kind, or when the rate caps are reached. A ticked SMS cell therefore routes the event; it does not override the switch.

The dollar figure in a fill message

`src/core/notifications.py, in fill_message` prints the kind, the units, the price and the dollars of one fill. The dollars are read by `src/core/emit_contracts.py, in read_settled_fill`, which uses the field names `SETTLED_FILL_FIELDS` declares. All ten emitters of the fill event write that one name, and the `trade.filled` contract takes its optional dollar fields from the same tuple.

The current reading, 2026-10-29. The message prints the `usd` field of the fill that `read_settled_fill` returns, and the two names above are current. Every place in the bot that books a fill sends it in one shape, and the contract for the fill event requires every field of that shape, the dollars among them. A fill that is not in that shape raises no alert, and the refusal is written to the log.

`src/core/notifications.py, in fill_message` — the dollars of the one shape

```
    usd = _as_number(fill.usd)
```

The state of this screen

`src/gui/variant_surface.py` registers both builders under the name `ALERTS`. Nothing calls `surface_class(ALERTS)`, and `src/gui/main_tabs/retired_tabs.py, in _install_retired_tab_sentinels` sets `_alerts_tab` to `None`. The running window therefore builds no Alerts page, and `src/gui/main_window.py, in _refresh_dashboard` skips it.

Both builds of the screen are complete and both draw the same six columns. The window does not open either one.

The engine behind this screen keeps running. The window builds the notification manager at launch, and every 2 seconds the refresh tick hands it the fleet P/L for the milestone check. The screen's own surface loads only when a request for this page arrives, so a session that never asks for it never imports it.

`src/gui/main_tabs/alerts_tab_surface.py` — `METHOD`, read from the one module that names every retired page's method

```
METHOD = retired_surfaces.ALERTS_TAB_METHOD
```

The Product Manual PDF rebuild is owed for this page.

Where an earlier sentence now reads differently

It stands as written. This is the reading that replaced it.

written above	what it reads now
"The Product Manual PDF rebuild is owed for this page."	`docs/Acervator-Product-Manual.pdf` is rebuilt with this page
"Both builds of the screen are complete and both draw the same six columns. The window does not open either one."	Both stay in the source folder. A packaged build carries neither: no call asks the screen registry for `ALERTS`, so the build leaves `src.gui.alerts_tab` and `src.gui.react_alerts_tab` out. The rule is on 17-what-the-build-includes.md
"The screen's own surface loads only when a request for this page arrives, so a session that never asks for it never imports it."	A packaged build does not carry the surface either, and its registry holds no handler for this page's method. A request for the page gets `UnknownMethod`, and no import runs. A run from the source folder still answers it

Back to the subsystem index.

Part 4

Live and Supporting Evidence

- YTD Exchange Data Covering the Entire Development Arc and Exchange Test Coverage

Part 5

Conceptual Hopscotch: The HOP Protocol, Rules Registry, and SADP v1 and v2

ADR Index & Glossary

Decision Records and Glossary

Reference. Where this repository records a decision, and the words the manual uses, each anchored to the module that implements it.

Where a decision is recorded

Two places hold the decisions this repository has made. Each answers a different question.

- `.claude/rules/` — three binding standards, on code comments, on documentation and on tests. `CLAUDE.md` at the root holds the repository rules above them: one role per folder, and no machine-specific path anywhere committed.
- `CHANGELOG.md` and the commit history. See 09-updates-and-versioning.md for what the changelog holds today.

Beside those, `docs/audits/` holds ten audit directories and one audit page. An audit records one measurement, and the issue it answers cites it by name.

The four places, overtaken

True today: two of the four places are gone, so two remain. The repository tracks no engineering-notes directory and no ledger file inside one. It tracks one Python module under `tests/`, its own conftest, so no decision in this repository is held as an executable guard.

Glossary

Every term below names something in the code, and carries the module or symbol that implements it. The technical-analysis vocabulary is in 07-indicators.md with the twelve published formulae, and is not repeated here.

The accumulation cycle

- **Target** — the dollar value a bot aims to hold in an asset. The scrum sells above it and the fold buys back beneath it.
- **Scrum** — a sell of the excess above the target. `ScrummingBot` in `src/trading/scrumming_bot.py` decides and places it.
- **Fold** — the buy back on the dip that follows a scrum, carried by `apply_profit_fold` in `profit_fold`.
- **Fold, corrected 2026-09-13** — the sentence above names a module that no code ever called, and it is removed. A fold runs on the tranche book: `_fold_eligible_tranches` picks the tranches the fold may reach, and `_apply_fold_target_growth` in `src/trading/scrumming_bot.py` grows the target from the surplus, gated by the bot's own Profit Folding Active row.
- **Interval** — the smallest price move between two fires, in percent. `scrumming_interval_pct` in `src/trading/container/config.py`, default 1.0.

- **Scrum/Fold cycle** — the pair, run against volatility rather than against a price forecast. A completed cycle ends holding more of the asset than it started with.
- **Price floor** — every unit bought back after a fold costs at or under the lot's original price. `_fold_eligible_tranches` in `src/trading/scrumming/tick_phases.py` enforces it.
- **Fold tranche** — one queued slice of scrummed dollars waiting for its buy back. `src/trading/scrumming/fold_tranches.py` holds the book.
- **Merge** — `_top_up_remnant_fold_tranches` folds a new sell's dollars into part-spent tranches, and `_bound_new_fold_tranches` collapses one sell's fresh slice into a single record when the fill arrived as many lots. Both reduce the record count without losing a dollar.
- **Despawn** — `_despawn_aged_tranches` removes a tranche older than the bot's threshold. It removes the record; it does not delist it, and the count lands in the discarded lifetime counter rather than the closed one.
- **Clear** — `clear_fold_tranches` discards every queued tranche and trades nothing.

Merge, despawn and clear are the only three functions that collapse or remove a tranche. 08-tabs/bot-swarm.md describes each in full.

Bots and what they hold

- **Scrumming Bot** — the accumulation bot, `ScrummingBot` in `src/trading/scrumming_bot.py`. One per traded symbol.
- **Extractor Bot** — `ExtractorBot` in `src/trading/extractor_bot.py`, a chunked state machine rather than a scrum/fold engine.
- **Smart Wire** — a capital reinforcement link between two bots, in `src/trading/smart_wire.py`.
- **Phantom balance** — a bot's view of holdings it may not sell yet, in `src/trading/phantom_balance.py`.
- **Capital claim** — one bot's hold on a quantity of an asset. `effective_available` in `src/trading/capital_reservation.py` answers holdings minus every other bot's claim on the same asset, never the caller's own.
- **Fleet** — the bots a run holds, with their wires, ledgers and tranches. All of it loads from `bot_state.json` through `StateManager`, which is the only source.

OVERTAKEN, and the Phantom balance entry above is kept as written. The module holds no view of holdings and no selling rule. It holds one read-only observer per timeframe. Each observer holds its own voting engine, runs it on its own candle length, keeps the summary, and builds no order and holds no position. The coordinator weights those summaries by timeframe rank and answers the parent bot with them. A bot's hold on a quantity of an asset is the Capital claim entry above, in a different module.

`src/trading/phantom_balance.py` — the module's own first line

```
"""``PhantomBot`` read-only TA observers, one per timeframe.
```

The classes behind the wire, the phantom balance and the fleet load:

```
class SmartWireManager: ...     # src/trading/smart_wire.py
class BotLedger: ...
class WireTransaction: ...

class PhantomBalanceManager: ...    # src/trading/phantom_balance.py
class PhantomBot: ...

class StateManager: ...     # src/core/state_manager.py
```

Deciding a trade

- **Voter** — one indicator's contribution to a direction and a confidence. `VotingEngine` in `src/trading/ta_engine.py` runs the twelve.
- **Gate chain** — the ordered checks a candidate trade passes before it fires, in `src/trading/gate_chain.py`. The sell chain and the buy chain differ, and 07-indicators.md lists both.
- **Heikin-Ashi candle** — a smoothed price bar. `detect_landing_strip_v2` in `src/trading/indicators/landing_strip.py` reads them.
- **Landing Strip** — a tightening detector, `detect_landing_strip_v2` in `src/trading/indicators/landing_strip.py`.
- **Band travel** — the mean-reversion read, `MRInspector` in `src/trading/mr_inspector.py`.
- **Trade grade** — a letter per completed trade, from four scored axes in `src/trading/trade_grader.py`. The market regime is recorded beside them and never scored. See 10-live-trade-history.md.

The gate chain's two classes and the grader's five entry points:

```
class GateChain: ...    # src/trading/gate_chain.py
class GateContext: ...

def grade_trade(record, ctx) -> TradeGrade: ...     # src/trading/trade_grader.py
def _score_execution(...): ...
def _score_timing(...): ...
def _score_strategic(...): ...
def _score_outcome(...): ...
```

Getting a strategy to real money

- **Promotion pipeline** — Market Inspector, then Simulator, then Paper Trader, then Live. Each step is a gate rather than a mode, and 08-tabs/promotion-pipeline.md holds the entry and exit criteria for each.
- **Gate-latch criterion** — the Simulator's own criterion. It passes when the gates latch identically on the same candles as live, not when profit or trade count matches. See 08-tabs/simulator.md.
- **Stone Tablet** — a stored candle history a Simulator run reads. `src/simulator/tablet_source.py` serves the candles, and `src/simulator/back_test.py` walks them through the live gate chain.
- **Paper Trader** — the step between Simulator and Live, defined by running the same logic against a real-time feed and a fake budget. Built under issue #19, in `src/paper/`. The shelved stock window still defaults to `paper_trader_available: bool = False`. See 08-tabs/paper-trader.md.

- **Proof of Accumulation** — the competition package, `src/competition/`. It holds the bot identity, the Merkle log, the challenge protocol, the token ledger and the local chain. Its two screens are shelved by the operator's own direction. See 08-tabs/proof-of-accumulation.md. The Simulator rebuild removed the fleet-replay files this glossary once named.

The development harness

- **Archetype** — a domain quality gate under `dev_harness/harness/`, reporting a `passed` boolean.
- **Handoff** — `ACERVATOR_HOP9.md`, measured for drift by `tools/hop_check.py`.
- **Rule id** — an RN name in `RULE_META`, in `unreached/core/rule_registry.py`. The registry has no production importer and the ids bind nothing. `RULE_META` now lives at `unreached/core/rule_registry.py`, moved out of `src/` because no file in the tree imports it.

11-hop-protocol-and-rules-registry.md describes all three.

The title page

- **Solve et coagula** — dissolve and reform, the sense the epigraph *Turbator aequilibrii dissolvendus reformandusque* carries on 01-title.md. One function letters SOLVE · ET · COAGULA around the outer arc of the harvest trophy, and that is the one place the phrase appears in the code.

```
def harvest_svg(d: TrophyData) -> str: ...      # src/competition/trophy_generator.py
```

The operator's own words

These name a posture rather than a component. They appear in the manual because the operator uses them, and no module implements any of them.

- **Accumulation trading** — the product category he places Acervator in: harvesting volatility to end holding more of the asset, rather than predicting a direction.
- **Conceptual hopscotch** — his name for a session hopping across a context boundary, which the handoff file exists to carry.
- **Ekthelius the Accumulator** — his handle.

Part 7

Recent Updates and Version Tracking

Part 8

The Development Chronicle As Told By Claude

Part 9

Live Trade History and Supporting Evidence

- VWAP Charts - YTD Data Analysis and Trade Grading - Deep Indicator Function Investigations Via Trade Logic Gate Logs

VWAP Charts and Trade Grading

Reference. What a year of live fills measures, how the volume-weighted average price is drawn from it, and what the platform's own grading and gate logs add to a fill.

The fill record is the operator's own venue export. It is not committed to this repository and no row of it is reproduced anywhere in this manual. Every figure below is an aggregate over that record.

The record behind the charts

The export holds 5,709 physical rows and 5,705 body rows. 5,661 of the body rows are fills.

Body rows by transaction type: Advanced Trade Buy 3,189; Advanced Trade Sell 2,472; Reward Income 16; Deposit 15; Buy 8; Sell 4; Withdrawal 1.

44 body rows carry no fill, and each one is accounted for by name: 16 Reward Income, 15 Deposit, 12 USDC cash legs of a trade priced elsewhere, and 1 Withdrawal. Nothing is dropped without a reason that says what it was.

40 distinct assets appear in the record. USD and USDC are quote currencies, which leaves 38 charted bases, one for each bot in the fleet, and each of the 38 has fills of its own.

Fills per asset, most to fewest: A01 900, A02 495, A03 459, A04 322, A05 261, A06 230, A07 226, A08 220, A09 196, A10 191, A11 178, A12 159, A13 147, A14 141, A15 124, A16 124, A17 119, A18 119, A19 113, A20 87, A21 83, A22 80, A23 77, A24 75, A25 74, A26 73, A27 73, A28 66, A29 63, A30 58, A31 25, A32 24, A33 23, A34 23, A35 16, A36 6, A37 6, A38 5. Those 38 counts sum to 5,661, the fill total above.

Subtotal is the price

A row carries both a Subtotal and a Total, and only one of them belongs in a VWAP. On every row the absolute quantity times the price equals the absolute Subtotal to within 5e-6. Total is that same figure plus the fee.

A fee is a transaction cost, not a price. Fee drag measures between 0.92 and 1.20 percent per asset, so a VWAP built on Total would sit roughly that far above the price the venue actually filled at, on every buy in the record.

How a VWAP is computed here

The running figure is a streaming sum of price times quantity over a streaming sum of quantity, both carried as floats. The price is the export's Price at Transaction column. The generator never reads Subtotal, and it does not need to. The section above measures quantity times price against Subtotal and finds the two equal to within 5e-6.

Run against the same rows with Python's exact `Fraction`, the two paths agree to 3.63e-15 across 26 assets, so the float path loses nothing a chart can show.

One control separates a weighted mean from an unweighted one. One unit bought at $10 and three at $20 give 17.50 by hand, and both the float path and the `Fraction` path return 17.50. An unweighted mean of the two prices returns 15.00. An implementation that forgot to weight by quantity would open a gap of 2.50 on that case rather than pass unnoticed.

What the combined chart plots

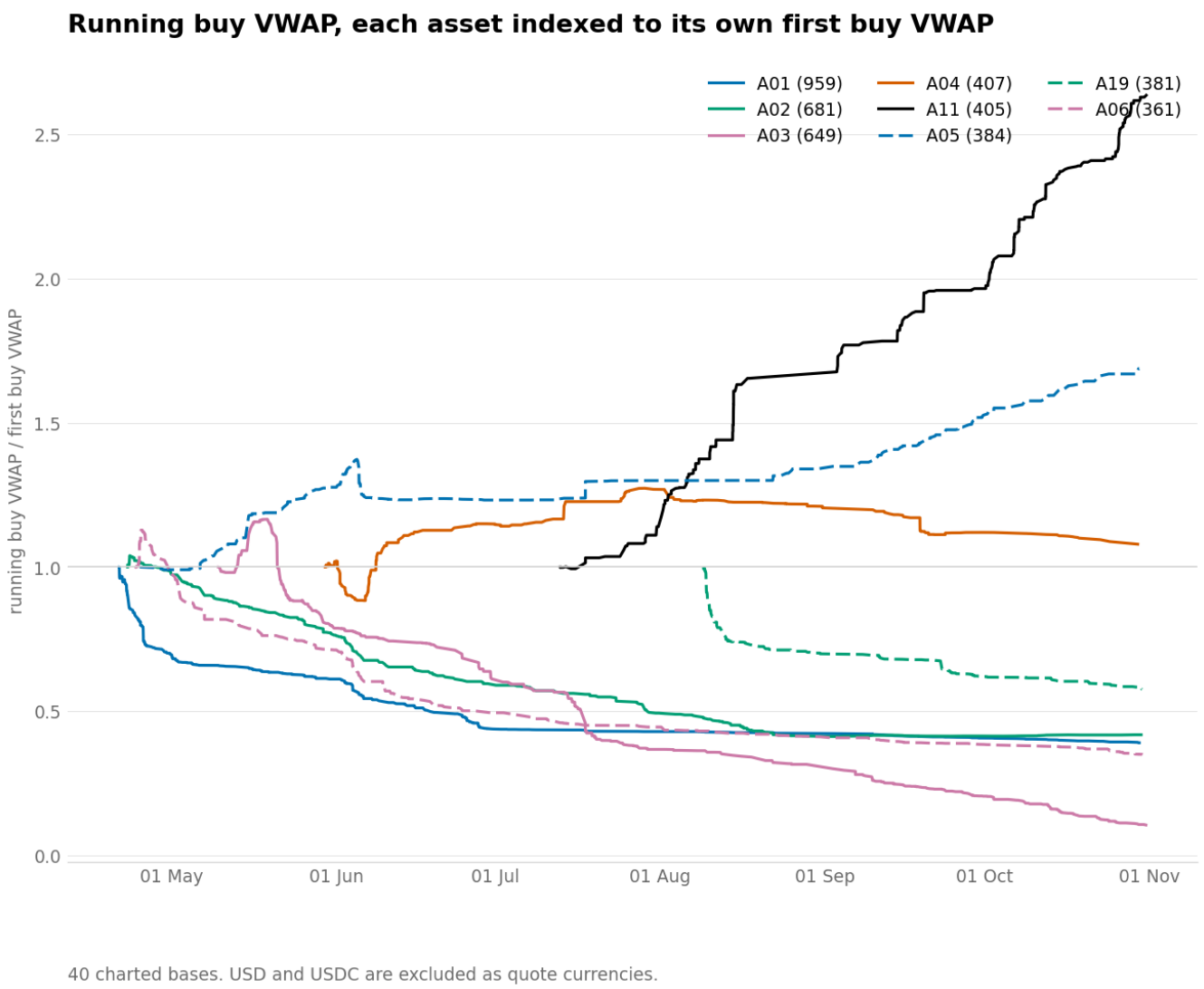
Each asset has a chart of its running buy VWAP against time. The line is the trajectory of the average as accumulation proceeds, and the shape of that trajectory is the reading.

Two metrics run through this part and they answer different questions. The trajectory answers what the units now held cost on average, and it keeps moving while the position is open. The sell-over-buy ratio answers whether the trades already closed sold above what they cost, and it is one settled figure per asset across the whole record. One reads the position, the other reads the realised trades. This part carries both, and neither stands in for the other.

The combined chart indexes every asset to its own first buy VWAP and plots the running buy VWAP divided by that first value, so an asset priced in cents and an asset priced in thousands share one axis and one scale.

Of the eight assets with the most fills, six trend down to between 0.4 and 0.6. A05 and A04 trend up.

Read the line as what it is. A falling running buy VWAP says the average cost of the base units now held fell as more were bought. It is not profit. It does not become profit until units are sold, and the index carries no sale in it at all.



The running buy VWAP of the eight assets with the most fills, each divided by its own first buy VWAP.

The eight lines are the eight assets with the most fills. Each one is that asset's running buy VWAP divided by its own first buy VWAP, which puts A10 at 5.71e-06 and A05 at 359.67 on one axis and one scale. The rule at 1.0 marks the first value. The legend names the asset and its fill count in three columns. The note under the axis reads:

> `38 charted bases. USD and USDC are excluded as quote currencies.`

Five pens carry the eight lines. The chart takes the series palette from `src/design_system.py` and keeps only the colours reaching a contrast ratio of 3.0 against the page background. Five of the eight qualify. The five run solid for the first five assets and dashed for the last three.

Series colour	Contrast on the page background	In the chart
#000000	21.00	yes
#0072B2	5.19	yes
#D55E00	3.87	yes

Series colour	Contrast on the page background	In the chart
#009E73	3.42	yes
#CC79A7	3.06	yes
#56B4E9	2.31	no
#E69F00	2.25	no
#F0E442	1.32	no

Measured over the same 5,661 fills, the eight lines end here:

Asset	Fills	First buy VWAP	Final buy VWAP	Final index
A01	900	1.5915	0.672605	0.423
A02	495	0.10842	0.044733	0.413
A03	459	0.11868	0.035741	0.301
A04	322	0.2478	0.298528	1.205
A05	261	359.67	485.048	1.349
A06	230	0.01581	0.006453	0.408
A07	226	0.05537	0.032580	0.588
A08	220	0.0508	0.022611	0.445

Six of the eight end below 1.0 and two end above it. Of the six, five land between 0.408 and 0.588, and A03 lands at 0.301.

The buy and sell ratio

The second metric is one figure per asset: the sell VWAP divided by the buy VWAP, across every fill in the record. Above 1.000 the average sale price beat the average buy price. Below it the average sale came in under the average cost. The figure reads how the closed trades performed, and nothing else.

No chart needs redrawing for it. Every per-asset chart already carries both lines, and the ratio is where the two of them finish: the sell line's last value over the buy line's last value. Both averages come from the same two columns as the charts, the export's Price at Transaction and its Quantity Transacted. Each side keeps its own pair of sums, so neither average pulls on the other.

Of the 38 charted bases, 35 have enough fills on both sides for the figure to mean anything. 23 of those finish above 1.000 and 12 below, and none lands exactly on it. A29 is the highest at 1.351 and A20 the lowest at 0.879.

Asset	Buys	Sells	Buy VWAP	Sell VWAP	Ratio
A01	538	362	0.672605	0.610801	0.908
A02	265	230	0.044733	0.0453964	1.015
A03	275	184	0.0357413	0.040867	1.143
A04	143	179	0.298528	0.328486	1.100
A05	140	121	485.048	536.675	1.106
A06	138	92	0.00645325	0.00627872	0.973

Asset	Buys	Sells	Buy VWAP	Sell VWAP	Ratio
A07	155	71	0.0325805	0.0317443	0.974
A08	140	80	0.0226113	0.0212018	0.938
A09	113	83	1.38181	1.34298	0.972
A10	104	87	5.98261e-06	6.10717e-06	1.021
A11	75	103	0.0297901	0.036577	1.228
A12	98	61	0.00762374	0.00758363	0.995
A13	69	78	14.6904	15.4413	1.051
A14	95	46	1996.6	2047.62	1.026
A15	95	29	68918	68393.6	0.992
A16	69	55	242.103	238.855	0.987
A17	66	53	0.36133	0.38167	1.056
A18	82	37	1.39501	1.40129	1.005
A19	57	56	0.00276124	0.00283892	1.028
A20	32	55	0.0336835	0.0296231	0.879
A21	43	40	9.06192	9.27909	1.024
A22	43	37	0.0881513	0.0882367	1.001
A23	43	34	82.9528	83.4024	1.005
A24	47	28	0.0116845	0.0116532	0.997
A25	41	33	0.857863	0.824958	0.962
A26	41	32	0.475448	0.497449	1.046
A27	45	28	0.198142	0.21519	1.086
A28	29	37	61.99	66.8959	1.079
A29	23	40	0.00205112	0.00277049	1.351
A30	27	31	2.03969	2.09155	1.025
A31	16	9	0.0828047	0.0787786	0.951
A32	8	16	0.431039	0.480832	1.116
A33	8	15	0.145768	0.164984	1.132
A34	10	13	0.174843	0.202243	1.157
A35	9	7	0.160515	0.164726	1.026
A36	3	3	46.4508	51.1833	not read
A37	3	3	0.0589505	0.0591535	not read
A38	1	4	2148.63	2576.19	not read

Three rows carry no ratio and the reason is the fill counts beside them. A38 has one buy, so its buy figure is a single price and not an average of anything. A36 and A37 have three buys and three sells each. A mean over three fills moves with any one of them, so a ratio built on it says more about which trade landed last than about how the asset performed. The table keeps both averages, since the export supports each one, and leaves the division out, since a number there would read as a result.

Every other row rests on at least seven fills a side, and 30 of the 38 rest on at least 23 a side.

The two metrics leave out different things and both gaps matter. The ratio counts only what completed on both sides, so it says nothing about units still held — and on most of these assets the held position is the larger part of the story. The trajectory counts every buy and no sale at all, so it says nothing about whether a sale ever cleared its cost. A cycle needs both readings. Either one alone overstates what it knows.

Traps in the export

Three shapes in the record break a reader that takes every row the same way.

A plain `Sell` row carries a negative quantity against a positive Subtotal. An `Advanced Trade Sell` row pairs those signs the other way round. Taking the sign from one column and the magnitude from the other turns four trades backwards.

One row carries a negative fee.

Three A27 rows carry a Subtotal that rounds to $0.00 against a quantity that is real. A filter that drops zero-value rows drops those base units with them.

Where the figures live

The charts are generated output, so they are not tracked. They belong beside the manual's other images in `artifacts/manual-figures/` under the repository root, a path `.gitignore` excludes. FIGURES.md inventories the images the manual itself embeds. No chart image is committed to this repository.

The 39 charts of this part live one level down, in a directory of their own beside the manual figures and under the same ignore rule. Keeping the two sets apart keeps the manual's own 38 images and this part's 39 charts from mixing. The combined view above sits in the first file below, and each charted base has one chart of its own.

```
docs/manual/figures/
    p<page>-i<index>.png        the manual's own 38 images
    vwap_combined.png           the eight-line combined chart
    vwap_a<NN>.png              one per charted base, 38 of them
    weekly_a<NN>.png            one weekly chart per charted base, 38 of them
    trace_a<NN>.png             one price trace per charted base, 38 of them
```

The charts are tracked, and they name no asset

Three sentences above are overtaken. They are quoted whole, and the sentence that replaces each one follows it.

> *The charts are generated output, so they are not tracked.*

The 39 charts are tracked, under `docs/manual/figures/`, so a clone can rebuild the manual without the venue export.

> *They belong beside the manual's other images in* `artifacts/manual-figures/` *under the repository root, a path* `.gitignore` *excludes.*

Both sets sit together in `docs/manual/figures/`, which `.gitignore` does not exclude.

> *No chart image is committed to this repository.*

Every chart is committed, and every one of them has the asset name painted out of its title, out of its price-axis label, and out of the combined view's legend. The file name carries the label this part uses, `vwap_a01.png` through `vwap_a38.png`, and never the asset.

The repository carries one manual, and that manual names no traded asset. The build that names them runs only from a checkout holding the unobscured charts, which this repository does not carry.

How to read a per-asset chart

Every chart carries two panels over one time axis.

The upper panel draws every fill as a dot at its price, buys in `COLORS["series"][0]` and sells in `COLORS["series"][5]`. Two lines cross it: the running buy VWAP and the running sell VWAP, in those same two colours. Each line steps at a fill of its own side and holds flat across a fill of the other side. The price axis turns logarithmic when the highest fill price exceeds the lowest more than eightfold, which covers A01 at 10.30 and A03 at 13.25 and no other asset. A chart of 30 fills or fewer marks each step with a dot: A31, A32, A33, A34, A35, A36, A37 and A38.

The lower panel draws net units accumulated — every bought quantity minus every sold quantity, running — in `COLORS["accent"]`, filled down to zero. It answers what the price panel cannot: whether the base position grew.

Every chart in this part draws its colours, its type sizes and its page size from `src/design_system.py`, which holds them under five names.

```
COLORS = {"bg": ..., "ink": ..., "accent": ..., "series": [...]}
TYPE = {"h1": ..., "body": ..., "cap": ...}
GRID = {"unit": 8, "page_w_in": 11.0, "page_h_in": 8.5}


def apply_rcparams(): ...


def contrast_ratio(fg: str, bg: str) -> float: ...
```

Read the buy line as a trajectory of average cost. A falling line says the average cost of the units bought so far fell. That fall is not profit, for the same reason the combined chart above carries none: no sale enters that line.

The Bollinger band behind the price

A grey channel sits behind the fills on the upper panel. It is a Bollinger band, and it is there so a trade at a volatility extreme can be seen rather than asserted. A scrum sells into the top of the channel and a fold buys into the bottom, and the channel puts both on the picture.

The band is the published one. A middle line is a simple moving average over a fixed number of bars, and the outer lines sit a fixed number of population standard deviations above and below it, taken over the same bars.

```
Middle = SMA(period)
Upper  = Middle + std_dev * sigma(period)
Lower  = Middle - std_dev * sigma(period)
```

The setting is the conventional one: period 20, width 2 standard deviations.

Where the band maths comes from

The platform already computes this band for its own trading decisions, and the charts read the same code rather than a second copy of the formula. The method returns one upper, middle and lower value per bar, and nothing before the twentieth bar, because no window has closed yet.

```
# src/trading/indicators/bollinger.py
BollingerBands(period=20, std_dev=2.0).bands(candles)
```

The deviation is the population one, divided by the window length, which is what Bollinger publishes. It is computed in `src/trading/indicators/helpers.py` and shared by every indicator that needs a rolling deviation.

What a fill at a band edge means

The upper line is two standard deviations above the recent average price, so a sell dot sitting on it or above it is a sale made while the price was at the top of its own recent range. The lower line is the mirror of that, so a buy dot on it or below it is a purchase made into the bottom of that range. Those two readings are the whole reason the band is drawn.

A dot inside the channel is an ordinary trade. Nothing about a fill at an edge says it was profitable. It says where in the recent price distribution the trade landed.

Which bar length the band uses

The export carries a timestamp and a price per fill and no candles, so the bars are built from the fills: the last fill in a bar closes it, and a bar with no fill closes where the one before it closed. That construction decides whether a short bar can carry a band at all. Twenty consecutive bars with no trade have no spread between them, so the deviation is zero and the band collapses onto the price line.

Measured over all 38 charted bases, counting every band the method returned and how many of them had zero width:

Bar length	Bands drawn	Zero width	Share	At or above upper	At or below lower	Fills in a banded bar
5 minutes	1,288,981	1,064,990	82.6%	2,921	3,134	6,868
15 minutes	429,202	315,054	73.4%	2,578	2,609	6,842
1 hour	106,790	52,205	48.9%	1,972	1,854	6,702
4 hours	26,182	4,994	19.1%	1,167	939	6,272
1 day	3,792	35	0.9%	605	460	4,955

At five minutes, 82.6 percent of the channel has no width. The band traces the price line as a hairline and shows no extreme at all. At one day, 0.9 percent has no width and the channel is wide enough to read across the whole window. The charts use one-day bars.

The cost of the coarse bar is the warm-up. Twenty one-day bars must close before the first band appears, so the first twenty days of each asset carry no channel, and 4,955 of the 6,997 fills land in a bar that has one.

The band edges, counted over every base

One row per charted base, in fill-count order. The two edge columns count fills against the band of the bar holding them, on one-day bars at period 20 and width 2.

Asset	Fills	Buy	Sell	At or above upper	At or below lower	In a banded bar	First buy VWAP	Final buy VWAP	Index	Window
A01	925	551	374	20	56	400	1.5915	0.652338	0.410	21 Apr to 25 Sep
A02	589	314	275	57	10	454	0.10842	0.0449093	0.414	22 Apr to 26 Sep
A03	537	315	222	13	28	409	0.11868	0.0257786	0.217	9 May to 27 Sep
A04	378	170	208	21	13	225	0.2478	0.277471	1.120	29 May to 27 Sep
A05	323	172	151	35	28	283	359.67	534.024	1.485	24 Apr to 26 Sep
A06	292	165	127	21	11	218	0.01581	0.00614089	0.388	24 Apr to 26 Sep
A08	274	165	109	20	19	209	0.0508	0.0223169	0.439	23 Apr to 27 Sep
A07	261	171	90	10	14	198	0.05537	0.0320055	0.578	3 May to 27 Sep
A10	237	122	115	25	24	167	5.71e-06	5.84811e-06	1.024	12 Apr to 27 Sep
A11	220	95	125	49	7	166	0.01722	0.033729	1.959	13 Jul to 25 Sep
A09	215	118	97	18	9	138	1.4622	1.38509	0.947	27 Apr to 27 Sep
A12	210	118	92	26	20	181	0.009814	0.00778479	0.793	27 Apr to 27 Sep
A13	205	93	112	21	7	170	15.7253	16.3105	1.037	9 May to 27 Sep
A14	169	108	61	18	31	149	2308.68	2037.61	0.883	23 Apr to 24 Sep
A16	157	80	77	13	20	141	273.4	243.486	0.891	2 May to 27 Sep
A17	155	82	73	10	4	132	0.35136	0.364156	1.036	7 May to 27 Sep
A15	152	107	45	11	39	143	77782.1	70016.9	0.900	23 Apr to 24 Sep
A18	148	97	51	18	22	123	1.4278	1.39586	0.978	23 Apr to 26 Sep

Asset	Fills	Buy	Sell	At or above upper	At or below lower	In a banded bar	First buy VWAP	Final buy VWAP	Index	Window
A19	147	79	68	2	3	40	0.00395	0.0024861	0.629	9 Aug to 27 Sep
A25	121	55	66	19	5	101	1.0673	0.850641	0.797	9 May to 27 Sep
A20	116	44	72	0	1	42	0.0697757	0.0309713	0.444	9 Aug to 27 Sep
A22	112	56	56	15	22	103	0.09814	0.0884351	0.901	25 Apr to 26 Sep
A24	108	62	46	4	3	73	0.02809	0.0104166	0.371	8 Jul to 27 Sep
A21	107	53	54	10	18	102	9.296	9.38981	1.010	24 Apr to 25 Sep
A23	101	52	49	11	24	88	85.32	84.0803	0.985	23 Apr to 25 Sep
A29	98	40	58	15	5	85	0.001603	0.00239143	1.492	7 Jul to 27 Sep
A30	98	40	58	21	2	69	2.411	2.2074	0.916	29 May to 27 Sep
A27	87	51	36	9	4	44	0.205737	0.197296	0.959	29 May to 25 Sep
A33	86	36	50	20	0	60	0.14222	0.155404	1.093	22 Aug to 26 Sep
A26	85	46	39	6	1	42	0.5153	0.471505	0.915	13 Jul to 27 Sep
A28	81	35	46	9	0	46	64.42	63.7886	0.990	29 May to 27 Sep
A32	49	15	34	15	0	37	0.42307	0.465448	1.100	23 Jul to 26 Sep
A31	42	21	21	9	4	28	0.09457	0.0825252	0.873	29 May to 24 Sep
A34	36	15	21	7	0	26	0.1696	0.179876	1.061	23 Jul to 26 Sep
A35	35	15	20	13	0	27	0.1695	0.166708	0.984	13 Jul to 25 Sep
A36	18	6	12	8	0	17	46.33	47.883	1.034	23 Jul to 25 Sep
A37	15	7	8	1	5	12	0.05997	0.057538	0.959	23 Jul to 25 Sep
A38	8	2	6	5	1	7	2148.63	2168.26	1.009	23 Jul to 21 Sep

Across the 38, 605 fills sit at or above the upper band and 460 at or below the lower one. A02 and A11 sell into the top most often, at 57 and 49. A01 buys into the bottom most often, at 56.

The newest export, beside the one the figures above measure

The figures and tables above were measured on an earlier export. A later one covers the same account over a longer window. Both sets of numbers are below, so a reader holding either can tell which is which.

The following sentence is overtaken:

> *The export holds 5,709 physical rows and 5,705 body rows. 5,661 of the body rows are fills.*

The later export holds 7,049 physical rows and 7,045 body rows, and 6,997 of the body rows are fills on a charted base.

The following sentence is overtaken:

> *Of the eight assets with the most fills, six trend down to between 0.4 and 0.6. A05 and A04 trend up.*

On the later export, five of the eight trend down to between 0.388 and 0.578, A03 falls further to 0.217, and A05 and A04 still trend up, to 1.485 and 1.120.

Side by side:

Reading	Earlier export	Later export
Physical rows	5,709	7,049
Body rows	5,705	7,045
Fills on a charted base	5,661	6,997
Charted bases	38	38
First fill	12 Apr	12 Apr
Last fill	3 Sep	27 Sep
Rows carrying no fill	44	48

The charted bases are the same 38 assets in both. The later export adds 1,336 fills and 24 days, and its extra rows carrying no fill are three more reward payments and one more cash leg.

One check separates a longer record from a different calculation. Of the 38 first buy VWAP values the table above publishes, 37 come out of the later export identical to the digit, and the one that differs is A20, published rounded to 0.069776 against a computed 0.0697757. A different price column or a different weighting would have moved all 38.

The October export, and the base that makes 39

A third export covers the same account to 19 October. The two readings above stand, and this one sits beside them. Every figure in this section is the output of `tools/build_product_manual.py`, in `measurement_lines`, which reads the exports `export_files` finds under `artifacts/evidence-input/transactions/`. That directory is excluded from the repository, so the record stays out of it and the figures drawn from it ship.

The following sentence is overtaken:

> *40 distinct assets appear in the record. USD and USDC are quote currencies, which leaves 38 charted bases, one for each bot in the fleet, and each of the 38 has fills of its own.*

41 distinct assets appear in the October export. USD and USDC are still the quote currencies, which leaves 39 charted bases, and each of the 39 has fills of its own. The 39th base opened on 4 October.

The following sentence is overtaken:

> *The later export holds 7,049 physical rows and 7,045 body rows, and 6,997 of the body rows are fills on a charted base.*

The October export holds 8,365 physical rows and 8,361 body rows, and 8,310 of the body rows are fills on a charted base.

The following sentence is overtaken:

> *Of the 38 charted bases, 35 have enough fills on both sides for the figure to mean anything. 23 of those finish above 1.000 and 12 below, and none lands exactly on it. A29 is the highest at 1.351 and A20 the lowest at 0.879.*

All 39 charted bases now carry fills on both sides. 32 finish above 1.000 and 7 below, and none lands exactly on it. A29 is the highest at 1.228 and A20 the lowest at 0.911.

The following sentence is overtaken:

> *Of the eight assets with the most fills, six trend down to between 0.4 and 0.6. A05 and A04 trend up.*

The eight assets with the most fills are no longer the same eight. A11 has more fills than A07 and takes its place. Five of the eight trend down, A03 as far as 0.137, and A04, A05 and A11 trend up, to 1.100, 1.639 and 2.403.

Three readings side by side:

Reading	Earlier export	Later export	October export
Physical rows	5,709	7,049	8,365
Body rows	5,705	7,045	8,361
Fills on a charted base	5,661	6,997	8,310
Charted bases	38	38	39
First fill	12 Apr	12 Apr	12 Apr
Last fill	3 Sep	27 Sep	19 Oct
Rows carrying no fill	44	48	51

The October export adds 1,313 fills and 22 days over the later one. Its three extra rows carrying no fill are all reward payments.

One check separates a longer record from a different calculation. The producer rebuilt the later export's table from that same export, and every cell it shares with the published one came back identical:

```
python -m tools.build_product_manual --measure
    rows compared                        38
    cells compared                      266
    cells differing                       0
control, the same table against the earlier export's
    cells differing                     229 of 266
```

The 38 first buy VWAP values the table above publishes are unchanged across the October export, every one of them. A first buy VWAP is the first buy's price, and no later fill can move it.

Every base, measured on the October export

One row per charted base, most fills first. The columns are the ones the export alone fixes.

Asset	Fills	Buy	Sell	First buy VWAP	Final buy VWAP	Index	Window
A01	950	567	383	1.5915	0.632117	0.397	21 Apr to 18 Oct
A02	657	351	306	0.10842	0.0452924	0.418	22 Apr to 19 Oct
A03	609	366	243	0.11868	0.0162426	0.137	9 May to 18 Oct
A04	404	190	214	0.2478	0.272606	1.100	29 May to 18 Oct
A05	371	204	167	359.67	589.423	1.639	24 Apr to 19 Oct
A11	336	151	185	0.01722	0.0413817	2.403	13 Jul to 19 Oct
A06	327	186	141	0.01581	0.00589118	0.373	24 Apr to 19 Oct
A08	313	190	123	0.0508	0.0223261	0.439	23 Apr to 18 Oct
A07	300	197	103	0.05537	0.0313047	0.565	3 May to 18 Oct
A10	269	142	127	5.71e-06	5.75974e-06	1.009	12 Apr to 18 Oct
A09	263	143	120	1.4622	1.43666	0.983	27 Apr to 18 Oct
A13	248	119	129	15.7253	17.411	1.107	9 May to 19 Oct
A12	242	139	103	0.009814	0.00790726	0.806	27 Apr to 19 Oct
A19	226	148	78	0.00395	0.0023844	0.604	9 Aug to 15 Oct
A17	191	104	87	0.35136	0.384832	1.095	7 May to 18 Oct
A14	190	121	69	2308.68	2075.55	0.899	23 Apr to 18 Oct
A16	186	97	89	273.4	248.764	0.910	2 May to 19 Oct
A15	181	123	58	77782.1	71198.7	0.915	23 Apr to 18 Oct
A18	172	114	58	1.4278	1.39522	0.977	23 Apr to 18 Oct
A25	160	81	79	1.0673	0.890555	0.834	9 May to 19 Oct
A29	155	74	81	0.001603	0.00293077	1.828	7 Jul to 19 Oct
A20	146	65	81	0.0697757	0.0281437	0.403	9 Aug to 19 Oct
A30	141	63	78	2.411	2.59862	1.078	29 May to 18 Oct
A24	140	85	55	0.02809	0.00931148	0.331	8 Jul to 17 Oct
A33	138	72	66	0.14222	0.171733	1.208	22 Aug to 18 Oct
A21	137	73	64	9.296	9.88602	1.063	24 Apr to 18 Oct
A22	132	70	62	0.09814	0.0881565	0.898	25 Apr to 18 Oct
A23	115	62	53	85.32	85.572	1.003	23 Apr to 17 Oct
A27	113	69	44	0.205737	0.198163	0.963	29 May to 18 Oct
A26	108	61	47	0.5153	0.466622	0.906	13 Jul to 18 Oct
A28	95	44	51	64.42	65.9488	1.024	29 May to 18 Oct
A32	69	26	43	0.42307	0.520594	1.231	23 Jul to 18 Oct

Asset	Fills	Buy	Sell	First buy VWAP	Final buy VWAP	Index	Window
A31	62	33	29	0.09457	0.0856605	0.906	29 May to 18 Oct
A35	49	25	24	0.1695	0.171694	1.013	13 Jul to 18 Oct
A34	45	21	24	0.1696	0.187527	1.106	23 Jul to 18 Oct
A36	25	11	14	46.33	50.3828	1.087	23 Jul to 14 Oct
A37	18	9	9	0.05997	0.0572927	0.955	23 Jul to 19 Oct
A38	17	8	9	2148.63	2294.25	1.068	23 Jul to 14 Oct
A39	10	7	3	2.0423	1.96971	0.964	4 Oct to 18 Oct

The buy and sell ratio on the October export

The sell VWAP over the buy VWAP, across every fill in the October export.

Asset	Buys	Sells	Buy VWAP	Sell VWAP	Ratio
A01	567	383	0.632117	0.587099	0.929
A02	351	306	0.0452924	0.0463992	1.024
A03	366	243	0.0162426	0.0198769	1.224
A04	190	214	0.272606	0.300622	1.103
A05	204	167	589.423	660.061	1.120
A11	151	185	0.0413817	0.0490044	1.184
A06	186	141	0.00589118	0.00576189	0.978
A08	190	123	0.0223261	0.0216727	0.971
A07	197	103	0.0313047	0.031177	0.996
A10	142	127	5.75974e-06	5.89461e-06	1.023
A09	143	120	1.43666	1.42274	0.990
A13	119	129	17.411	18.9641	1.089
A12	139	103	0.00790726	0.00806262	1.020
A19	148	78	0.0023844	0.00254737	1.068
A17	104	87	0.384832	0.412881	1.073
A14	121	69	2075.55	2155.22	1.038
A16	97	89	248.764	252.5	1.015
A15	123	58	71198.7	73581.2	1.033
A18	114	58	1.39522	1.40659	1.008
A25	81	79	0.890555	0.910984	1.023
A29	74	81	0.00293077	0.00359865	1.228
A20	65	81	0.0281437	0.0256386	0.911
A30	63	78	2.59862	3.01716	1.161
A24	85	55	0.00931148	0.00962266	1.033
A33	72	66	0.171733	0.194725	1.134

Asset	Buys	Sells	Buy VWAP	Sell VWAP	Ratio
A21	73	64	9.88602	10.4953	1.062
A22	70	62	0.0881565	0.0901305	1.022
A23	62	53	85.572	87.5001	1.023
A27	69	44	0.198163	0.21289	1.074
A26	61	47	0.466622	0.49167	1.054
A28	44	51	65.9488	72.1458	1.094
A32	26	43	0.520594	0.629924	1.210
A31	33	29	0.0856605	0.0897963	1.048
A35	25	24	0.171694	0.188452	1.098
A34	21	24	0.187527	0.218207	1.164
A36	11	14	50.3828	60.2623	1.196
A37	9	9	0.0572927	0.0574807	1.003
A38	8	9	2294.25	2724.79	1.188
A39	7	3	1.96971	1.89008	0.960

Three columns are not carried forward

The band table above counts fills against the band of the bar holding them, and those bars are built from the fills rather than from candles. No committed file builds them. The closest reconstruction returns 3,797 bands against the 3,792 that table counts, and 653 fills at or above the upper band against its 605. A figure that cannot be reproduced is not republished here, so this section carries the columns the export alone fixes and leaves the three band columns to the reading above.

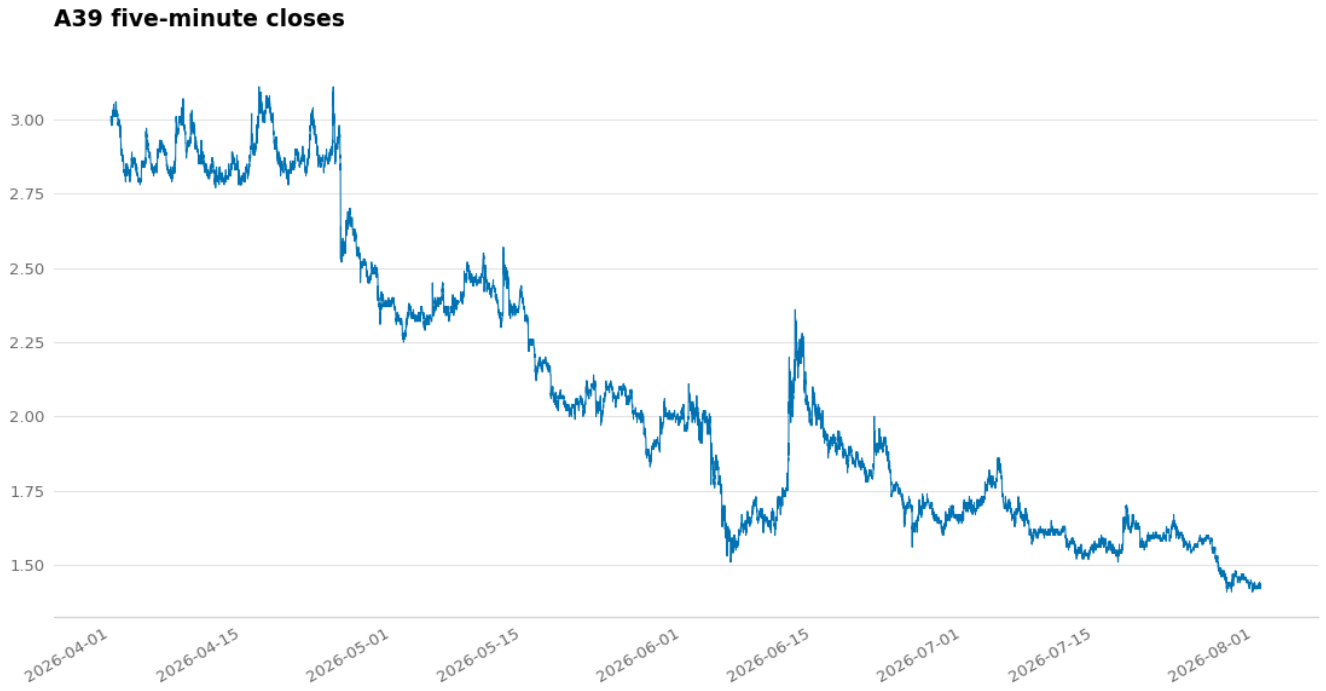
The combined chart near the top of this part draws the eight assets with the most fills of the earlier export. Those are no longer the eight with the most fills, and nothing in this repository redraws that chart.

A39, the base the October export adds

A39 has 10 fills, seven buys and three sells, between 4 October and 18 October. Net units rise from 12.128 to 14.895. The buy line runs from 2.0423 down to 1.96971 and the sell line ends at 1.89008, which puts the ratio at 0.960.



A39 weekly chart, 18 weekly candles built from 18,525 five-minute rows.



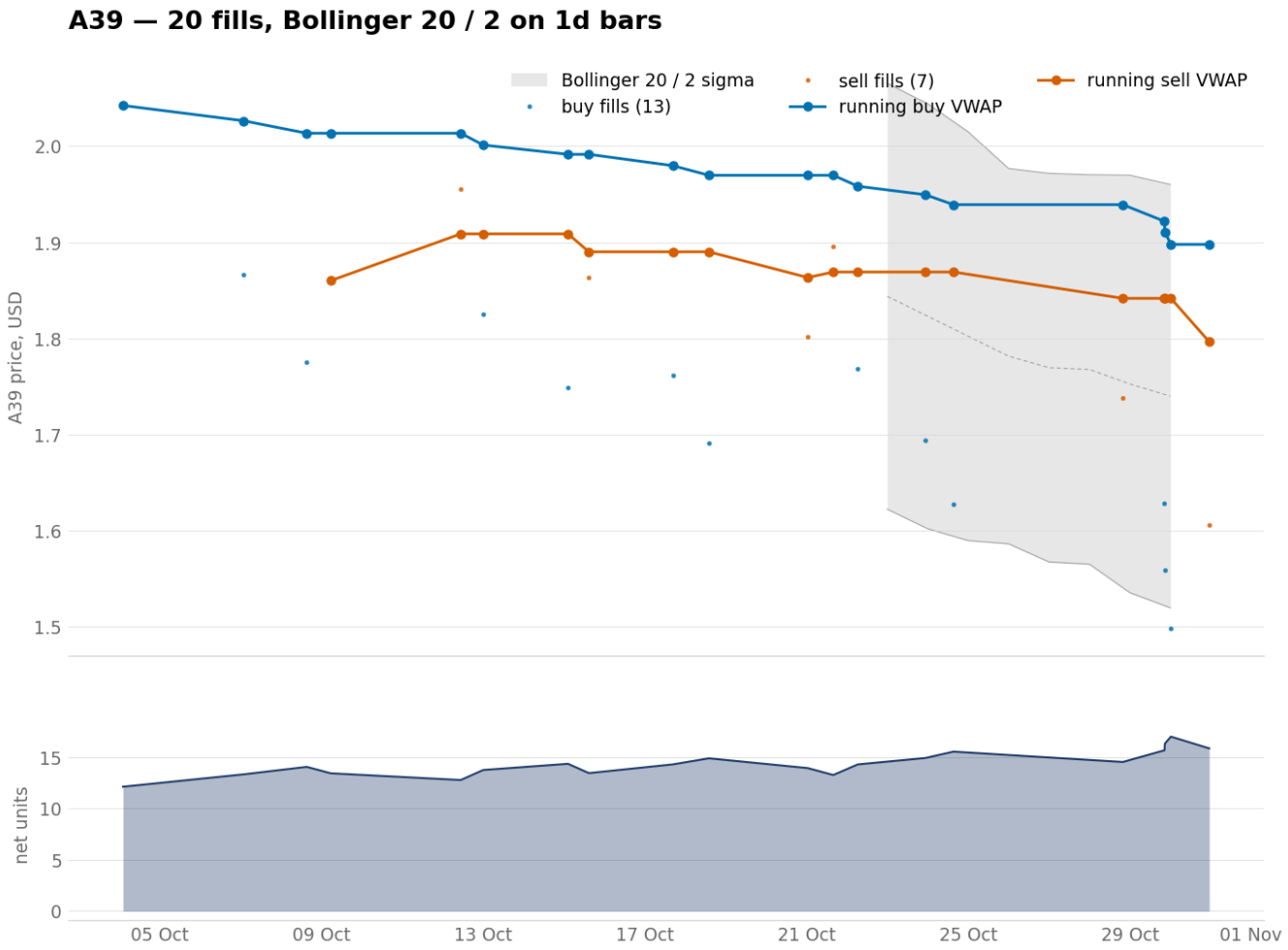
A39 price trace, 18,525 five-minute closes across the recorded window.

A39 has no VWAP chart. The 39 charts above arrived with the record, and no committed file draws a fortieth, so the weekly chart and the price trace carry this base. `tools/build_product_manual.py`, in `labels_across`, numbers a base that arrives late after the labels already published, so A01 through A38 keep the bases they have carried throughout and a new base takes the next number.

The following two sentences are overtaken:

> *A39 has no VWAP chart. The 39 charts above arrived with the record, and no committed file draws a fortieth, so the weekly chart and the price trace carry this base.*

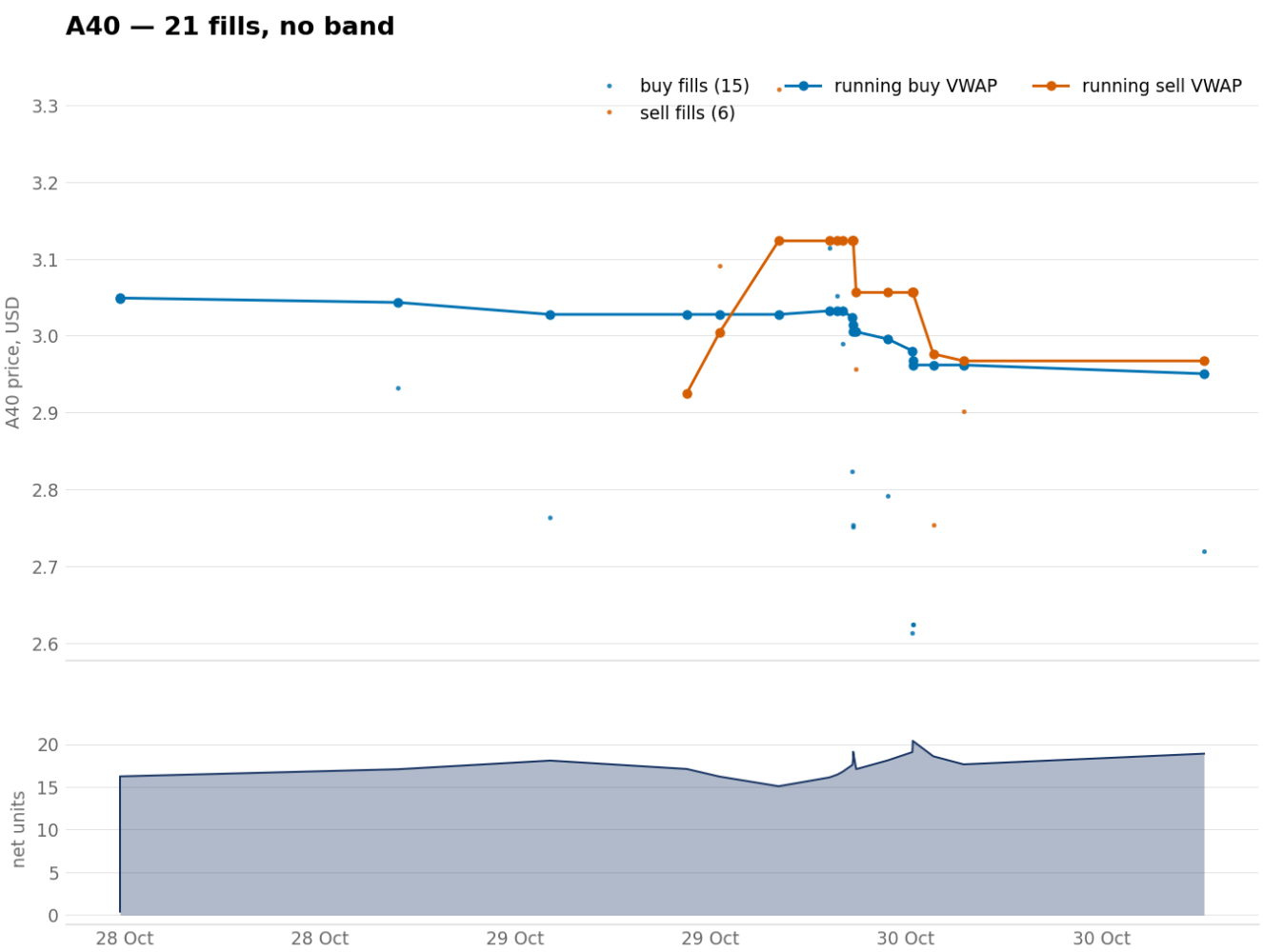
A39 has a VWAP chart. `_write_asset_vwap` in `tools/build_product_manual.py` draws one for every label the newest export carries, and that export carries A39.



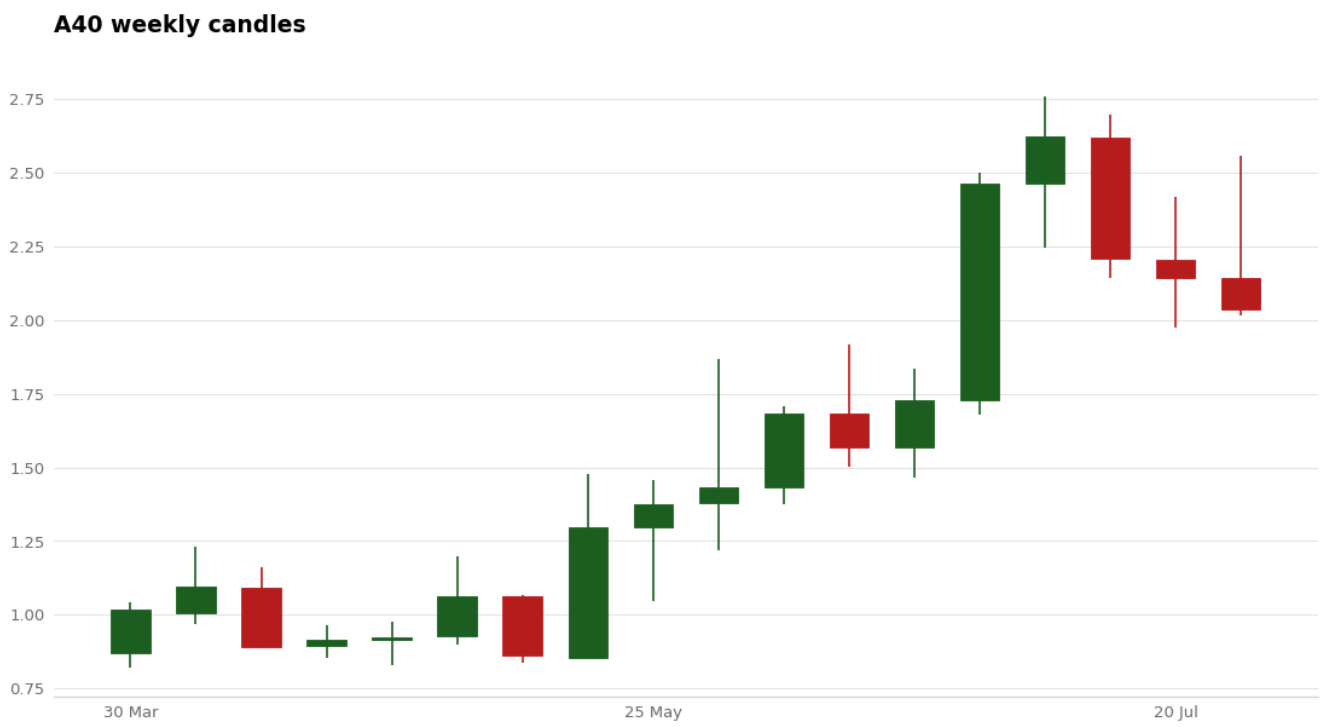
A39, 10 fills with every step dotted, and no band: its 15 one-day bars are fewer than the 20 the band needs.

Now: the A39 chart draws 20 fills, dots every step, and carries a band over the last 8 of its 27 one-day bars (`tools/build_product_manual.py, in fill_band`).

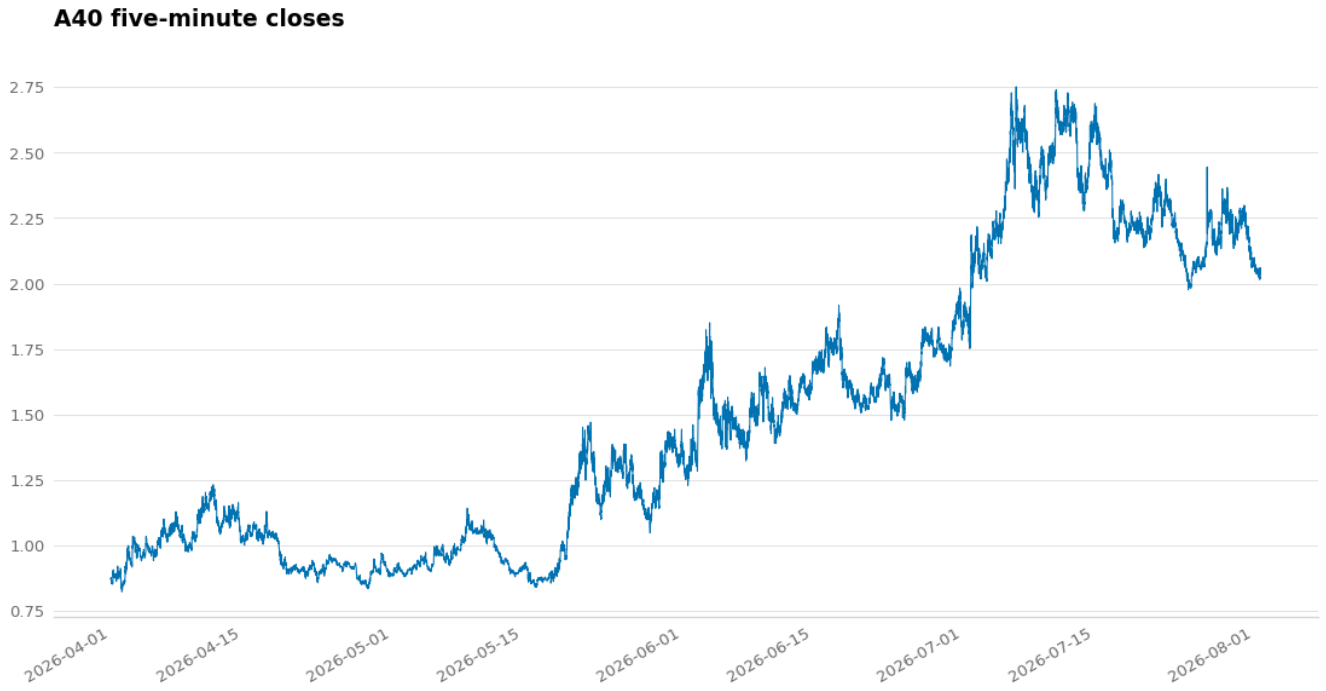
A40



A40, 21 fills with every step dotted, and no band: its 4 one-day bars are fewer than the 20 the band needs.



A40 weekly chart, 18 weekly candles built from 28,776 five-minute rows.



A40 price trace, 28,776 five-minute closes across the recorded window.

A40 has 21 fills, 15 buys and 6 sells, between 27 October and 30 October. Its Stone Tablet rows end on 1 August, so its weekly chart and price trace stop before its first fill (`tools/build_product_manual.py, in tablet_candles`).

What this repository holds for these charts

The rows behind the charts take the shape `fetch_all_history_chunked` in `src/exchange/history_helpers.py` returns for the History tab, one dict per fill.

The closest committed figure to the buy line is the average entry price `compute_position_health` derives in `src/exchange/position_health.py` from the records a venue returns for its own trade history. The two differ, and the difference matters to a reader holding a chart beside the platform. The average entry follows the open position only: a buy re-weights it, a sell leaves it alone, and a full close resets it to zero. The buy line here counts every buy in the record and never resets.

```
if t.side == OrderSide.BUY:     # t is one get_my_trades record
    new_qty = qty + t.amount
    if new_qty > 0:
        avg_entry = (qty * avg_entry + t.amount * t.price) / new_qty
    qty = new_qty
elif t.side == OrderSide.SELL:
    qty -= t.amount
    if qty < 1e-12:
        qty = 0.0
        avg_entry = 0.0
```

No committed module computes an average sell price, and none computes a ratio of one average over the other. Five names were searched across the source, the harness and the tools, ignoring case, and every one of them returns no file. The control returns files for both of its terms and none for a coined one.

```
searched in src/, dev_harness/ and tools/, ignoring case
    avg_sell             0 files
    average_sell         0 files
    sell_vwap            0 files
    avg_sell_price       0 files
    sb_ratio             0 files
control
    avg_entry           13 files
    vwap                 6 files
    a coined term        0 files
```

The chart module computes no ratio either. It draws the two lines and stops there. The ratio table above comes from the export directly, on the same two columns the charts read.

No committed file produces these charts. A walk of every path any commit ever added, renamed or deleted reaches 1,626 distinct paths, and exactly one of them carries the letters vwap: a Simulator test for the price band. A pickaxe over every Python file in every commit returns nothing for the three names a chart builder would carry. The charts and the record behind them belong to the operator. This repository cannot regenerate either.

```
git log --all --diff-filter=ADR --name-only
    1,626 distinct paths
    test_vwap_band_scales_to_price             the one vwap name
                                                deleted; not in the tree

git log -S<name> -- '*.py'
    draw_combined         0 commits
    vwap_combined         0 commits
    buy_vwap              0 commits
control
    avg_entry            16 commits
    sync_ytd_trade_count  7 commits
    a coined term         0 commits
```

The builder draws every VWAP chart

`tools/build_product_manual.py` draws all 40 VWAP charts this part embeds: one for each of the 39 labelled bases, and the combined view. Below, each overtaken sentence appears whole, and what the code does today follows it.

Now: the builder draws 41 VWAP charts, one for each of the 40 labelled bases in the newest export and the combined view (`tools/build_product_manual.py, in _write_asset_vwap`).

No committed file produces these charts.

`write_evidence_figures` draws them first among the evidence figures, before the build lays out a page.

```
out = _write_asset_vwap(figures_dir, labels, fills)
out += _write_asset_weekly(figures_dir, labels, candles)
```

The charts and the record behind them belong to the operator. This repository cannot regenerate either.

The record still belongs to the operator and stays out of the repository. A checkout holding it under `artifacts/evidence-input/transactions/` regenerates every chart. The `--evidence-dir` flag points the builder at a copy held in any other folder.

```
python -m tools.build_product_manual --evidence-dir <a folder holding transactions/>
```

The 39 charts of this part, the combined view above included, come from one module, `vwap_charts`. It reads the export once, groups the fills by asset, and writes one PNG per charted base.

The module is not committed to this repository. Neither is the export it reads.

`_write_asset_vwap` takes the newest export's fills and groups them by label, never by asset.

Each group goes to `_draw_vwap`, and the whole set goes to `_draw_combined`. Every string a drawing function writes comes from the label, a count or a fixed word, so no title, axis label or legend carries an asset name. The repository holds the drawing. It does not hold the export.

```
groups: dict[str, list[Fill]] = {}
for fill in fills:
    groups.setdefault(labels[fill.asset], []).append(fill)
```

Every chart is committed, and every one of them has the asset name painted out of its title, out of its price-axis label, and out of the combined view's legend.

The repository holds every chart, and no chart has anything painted out. The label sits in the title and the price-axis label of each per-asset chart, and in the legend of the combined view.

```
price.set_ylabel(f"{label} price, USD")
price.set_title(f"{label} — {len(fills)} fills, {note}")
```

The file name carries the label this part uses, `vwap_a01.png` *through* `vwap_a38.png`*, and never the asset.*

The file names run from `vwap_a01.png` to `vwap_a39.png`, and none names the asset.

Now: the file names run to `vwap_a40.png`, one for each label (`tools/build_product_manual.py, in asset_figure_name`).

The combined chart near the top of this part draws the eight assets with the most fills of the earlier export. Those are no longer the eight with the most fills, and nothing in this repository redraws that chart.

`_draw_combined` redraws it from the newest export. Its eight lines are the eight bases with the most fills there: A01 to A06, A08 and A11. The note under its axis counts the bases that export carries:

```
39 charted bases. USD and USDC are excluded as quote currencies.
```

Now: the eight lines are A01 to A06, A11 and A19, and the note counts 40 charted bases (`tools/build_product_manual.py, in _draw_combined`).

No committed module computes an average sell price, and none computes a ratio of one average over the other.

`trajectory` in `tools/build_product_manual.py` computes the running sell VWAP each chart draws.

`measurement_lines` divides the final sell VWAP by the final buy VWAP for the ratio table on the October export.

```
ratio = row.sell_vwap / row.final_buy_vwap if row.final_buy_vwap else 0.0
```

The price axis turns logarithmic when the highest fill price exceeds the lowest more than eightfold, which covers A01 at 10.30 and A03 at 13.25 and no other asset. A chart of 30 fills or fewer marks each step with a dot: A31, A32, A33, A34, A35, A36, A37 and A38.

On the newest export the axis turns logarithmic for A01 at 14.82 and A03 at 38.35, and for no other base. Four charts have 30 fills or fewer, and each marks every step with a dot: A36, A37, A38 and A39.

```
marker = "o" if len(fills) <= STEP_DOT_MAX_FILLS else "None"
```

Now: the axis turns logarithmic for A01 at 16.72 and A03 at 43.35, and the four charts that dot every step are A37, A38, A39 and A40 (`tools/build_product_manual.py, in _draw_vwap`).

What a redrawn chart reads

Every chart reads the newest export in the input folder: 8,310 fills on 39 bases, the last on 19 October. The fill count in each chart's title matches the table under "Every base, measured on the October export".

Now: every chart reads 9,052 fills on 40 bases, the last on 31 October, and each title's fill count matches the table `python -m tools.build_product_manual --measure` prints (`tools/build_product_manual.py, in measurement_lines`).

The band table, the table under "The 38 per-asset charts" and the description under each base's heading take their numbers from the earlier exports. They stand as published.

Where a one-day bar opens

No committed file builds them.

`fill_bars` builds the one-day bars the band reads. Every bar opens at 00:00 UTC. The last fill in a bar closes it, and a bar with no fill closes where the bar before it closed.

`fill_band` hands those bars to the platform's own band, at period 20 and width 2.

```
CHART_BAND = BollingerBands(period=20, std_dev=2.0)
BAND_BAR_MS = DAY_MS
```

The band table above counts one-day bars that open at 17:00 UTC. A recount of the same 6,997 fills at that boundary returns every row of that table exactly. At 00:00 UTC the recount returns the reconstruction the section on the three band columns quotes:

```
the 27 September export, period 20, width 2, one-day bars
                   bands   zero width   at or above upper   at or below lower   in a banded bar
bars at 17:00 UTC  3,792       35             605                 460              4,955
bars at 00:00 UTC  3,797       36             653                 455              4,975
```

The channel on a redrawn chart sits on bars that open seven hours later than the bars that table counts. The builder opens every bar at 00:00 UTC, the boundary `week_start_ms` also uses, on any machine that runs it.

The evidence slides

The same run writes the evidence slide show, Prototyping-Trading-Evidence.pptx, after the charts and before the PDF. Every fill figure on a slide comes from the newest export, each market caption counts the Stone Tablet rows its two charts draw (`tools/build_product_manual.py, in _deck_market`), and every chart on a slide is one this run drew.

```
evidence = read_evidence(args.evidence_dir)
deck = write_evidence_deck(args.deck, args.figures_dir, evidence)
```

The 38 per-asset charts

One chart per charted base, in fill-count order. The table gives what each chart's two lines start and end at, and the window its axis spans.

Asset	Fills	Buy	Sell	First buy VWAP	Final buy VWAP	Index	Window
A01	900	538	362	1.5915	0.672605	0.423	21 Apr to 2 Sep
A02	495	265	230	0.10842	0.044733	0.413	22 Apr to 3 Sep
A03	459	275	184	0.11868	0.035741	0.301	9 May to 2 Sep
A04	322	143	179	0.2478	0.298528	1.205	29 May to 31 Aug
A05	261	140	121	359.67	485.048	1.349	24 Apr to 3 Sep
A06	230	138	92	0.01581	0.006453	0.408	24 Apr to 3 Sep
A07	226	155	71	0.05537	0.032580	0.588	3 May to 3 Sep
A08	220	140	80	0.0508	0.022611	0.445	23 Apr to 3 Sep
A09	196	113	83	1.4622	1.381809	0.945	27 Apr to 3 Sep
A10	191	104	87	5.71e-06	5.98261e-06	1.048	12 Apr to 3 Sep
A11	178	75	103	0.01722	0.029790	1.730	13 Jul to 3 Sep
A12	159	98	61	0.009814	0.007624	0.777	27 Apr to 3 Sep
A13	147	69	78	15.7253	14.690407	0.934	9 May to 1 Sep
A14	141	95	46	2308.68	1996.595	0.865	23 Apr to 21 Aug
A15	124	95	29	77782.1	68917.999	0.886	23 Apr to 3 Sep
A16	124	69	55	273.4	242.103	0.886	2 May to 1 Sep
A17	119	66	53	0.35136	0.361330	1.028	7 May to 3 Sep
A18	119	82	37	1.4278	1.395015	0.977	23 Apr to 3 Sep
A19	113	57	56	0.00395	0.002761	0.699	9 Aug to 31 Aug
A20	87	32	55	0.069776	0.033684	0.483	9 Aug to 2 Sep
A21	83	43	40	9.296	9.061918	0.975	24 Apr to 3 Sep
A22	80	43	37	0.09814	0.088151	0.898	25 Apr to 3 Sep
A23	77	43	34	85.32	82.952844	0.972	23 Apr to 1 Sep
A24	75	47	28	0.02809	0.011685	0.416	8 Jul to 31 Aug
A25	74	41	33	1.0673	0.857863	0.804	9 May to 3 Sep
A26	73	41	32	0.5153	0.475448	0.923	13 Jul to 30 Aug
A27	73	45	28	0.205737	0.198142	0.963	29 May to 28 Aug
A28	66	29	37	64.42	61.990016	0.962	29 May to 30 Aug
A29	63	23	40	0.001603	0.002051	1.280	7 Jul to 30 Aug
A30	58	27	31	2.411	2.039694	0.846	29 May to 3 Sep
A31	25	16	9	0.09457	0.082805	0.876	29 May to 22 Aug
A32	24	8	16	0.42307	0.431039	1.019	23 Jul to 3 Sep
A33	23	8	15	0.14222	0.145768	1.025	22 Aug to 3 Sep

Asset	Fills	Buy	Sell	First buy VWAP	Final buy VWAP	Index	Window
A34	23	10	13	0.1696	0.174843	1.031	23 Jul to 3 Sep
A35	16	9	7	0.1695	0.160515	0.947	13 Jul to 2 Sep
A36	6	3	3	46.33	46.450791	1.003	23 Jul to 3 Sep
A37	6	3	3	0.05997	0.058950	0.983	23 Jul to 16 Aug
A38	5	1	4	2148.63	2148.63	1.000	23 Jul to 25 Aug

Those 38 fill counts sum to 5,661, the fill total this part opens with. Eleven of the 38 end at or above 1.000 and 27 end below it.

The 39 charts of this part, the combined view above included, come from one module, `vwap_charts`. It reads the export once, groups the fills by asset, and writes one PNG per charted base.

The line each chart draws is a running mean of price weighted by quantity. The price is the export's Price at Transaction column and the quantity is its Quantity Transacted column. Buys and sells accumulate in two separate pairs of sums, so a fill on one side steps that side's line and leaves the other flat. The lower panel is the same walk of quantities, added on a buy and taken away on a sell.

The module is not committed to this repository. Neither is the export it reads.

```
def load_fills(src: Path):
    """Return the Fill list and a Counter keyed by drop cause."""
    ...
    price = parse_money(r[idx["Price at Transaction"]])
    qty = abs(parse_money(r[idx["Quantity Transacted"]]))
    ...
    fills.append(Fill(asset, when, side, price, qty))

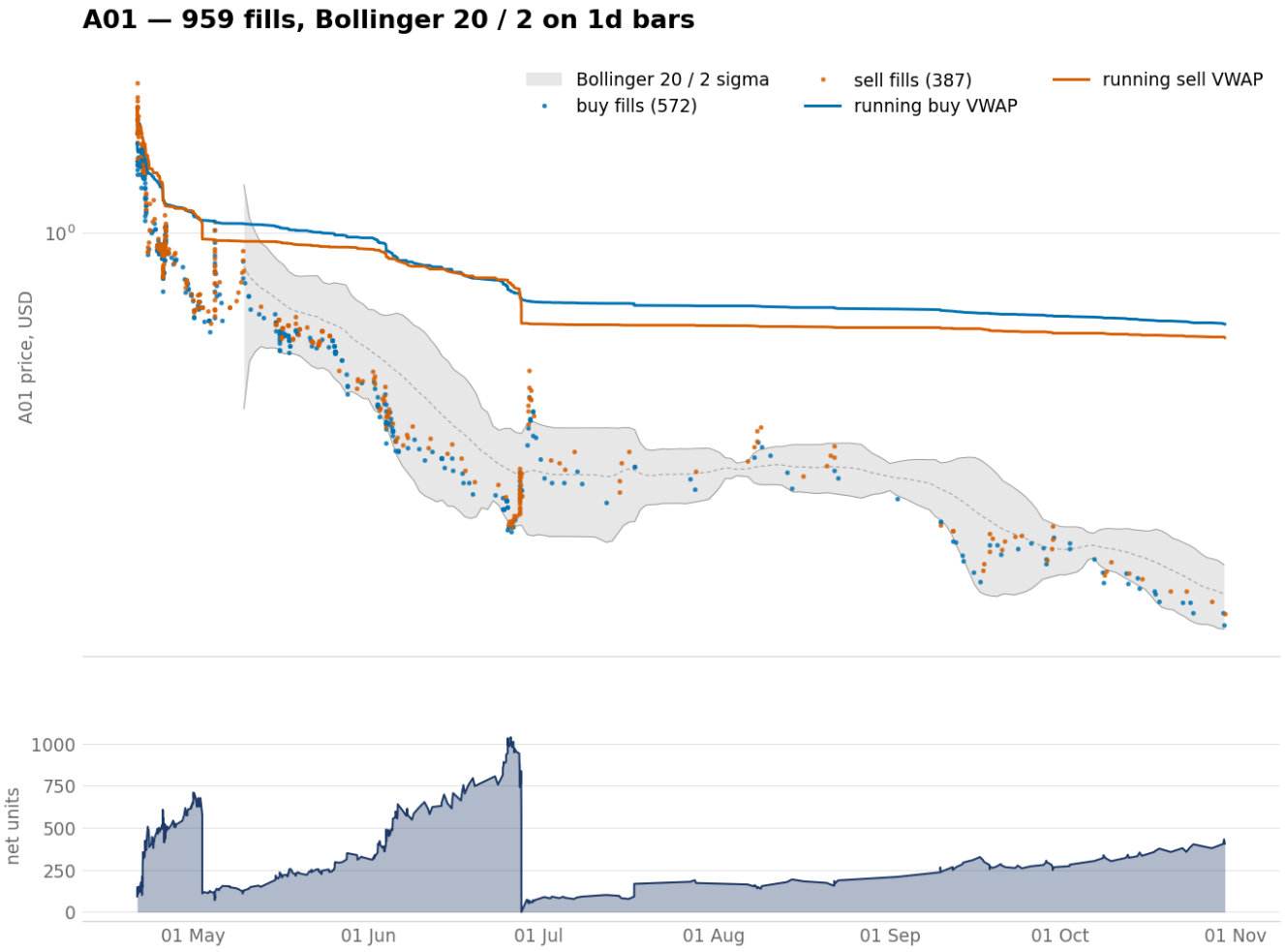

def trajectory(fills):
    """Return times and the running buy VWAP, sell VWAP and net units."""
    ordered = sorted(fills, key=lambda f: f.when)
    t, buy_v, sell_v, held = [], [], [], []
    bn = bd = sn = sd = pos = 0.0
    for f in ordered:
        if f.side == "buy":
            bn += f.price * f.qty
            bd += f.qty
            pos += f.qty
        else:
            sn += f.price * f.qty
            sd += f.qty
            pos -= f.qty
        t.append(f.when)
        buy_v.append(bn / bd if bd > 0 else float("nan"))
        sell_v.append(sn / sd if sd > 0 else float("nan"))
        held.append(pos)
    return t, buy_v, sell_v, held
```

Four of the 38 charts below carry a block of their own. Each one is a chart where the module takes a turn it does not take for the rest: a logarithmic price axis, the minor ticks that axis needs, a tick label small enough to

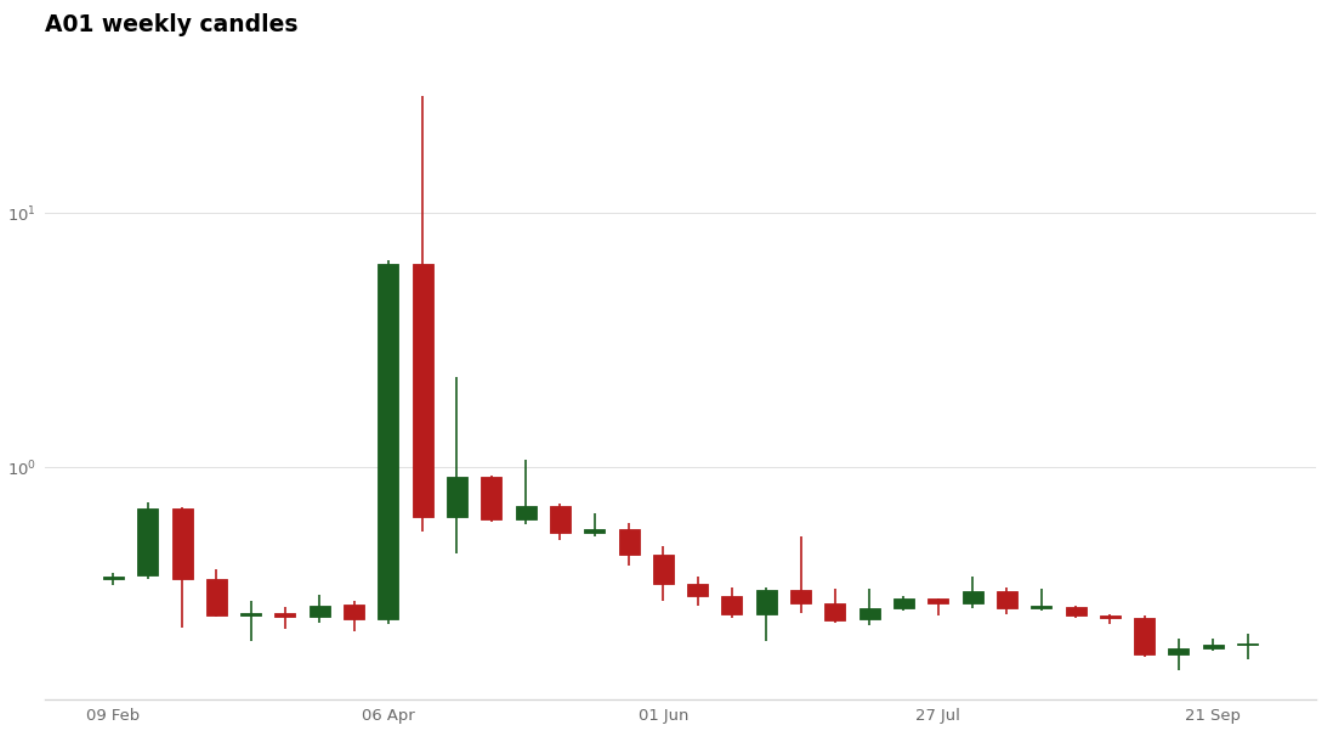
go scientific, and a fill count low enough to mark every step. The other 34 are drawn by the two functions above and nothing else.

Each of the 38 headings below holds three charts for one base, under one label: its VWAP chart, its weekly chart and its price trace. The VWAP chart draws the fills. The weekly chart and the price trace draw the market those fills ran in, from the Stone Tablet rows the next two sections set out.

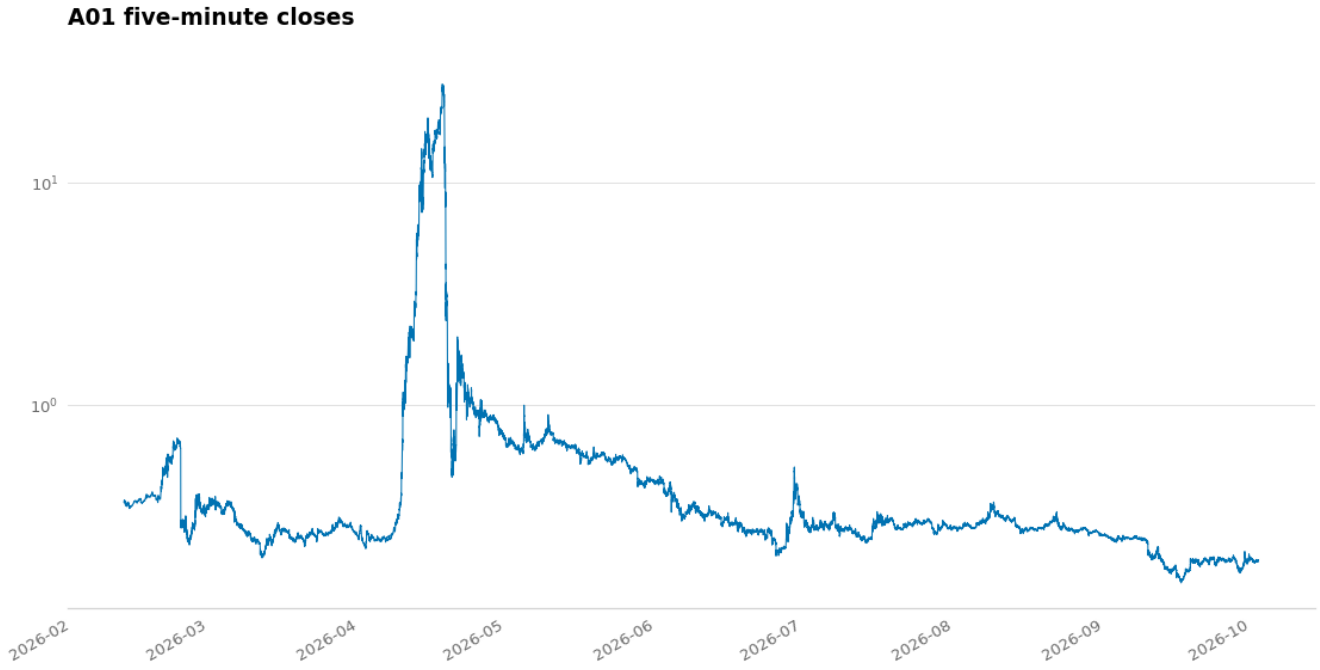
A01



A01, 900 fills, a logarithmic price axis, and a net-units panel that empties twice.



A01 weekly chart, 34 weekly candles built from 50,380 five-minute rows.



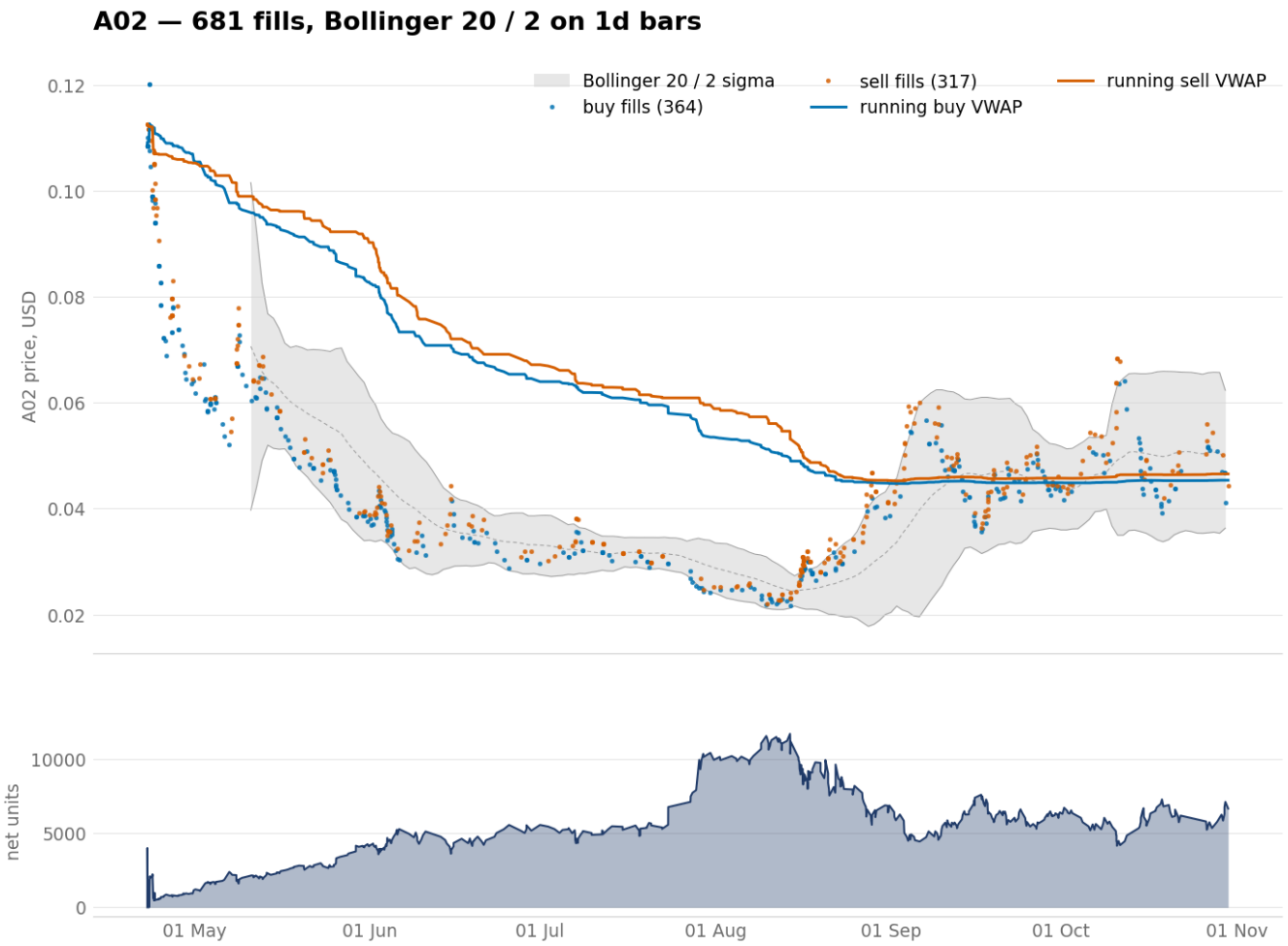
A01 price trace, 50,380 five-minute closes across the recorded window.

The most-traded base of the 38, and one of the two charts whose price axis runs logarithmic. Both lines step down together to about 1.00 by early May, then to about 0.67 at the end of June, where the sell line settles below the buy line and stays there. The net-units panel peaks at 1,037 units in late June and ends at 209, with two near-vertical drops.

The axis is a measurement, not a choice. A01's fills run from 0.2118 to 2.1805, a span of 10.30, and the module turns the axis logarithmic above eight. A03 is the only other chart over that line.

```
  lo = min(f.price for f in fills)
  hi = max(f.price for f in fills)
  if hi / lo > 8:
      ax.set_yscale("log")
```

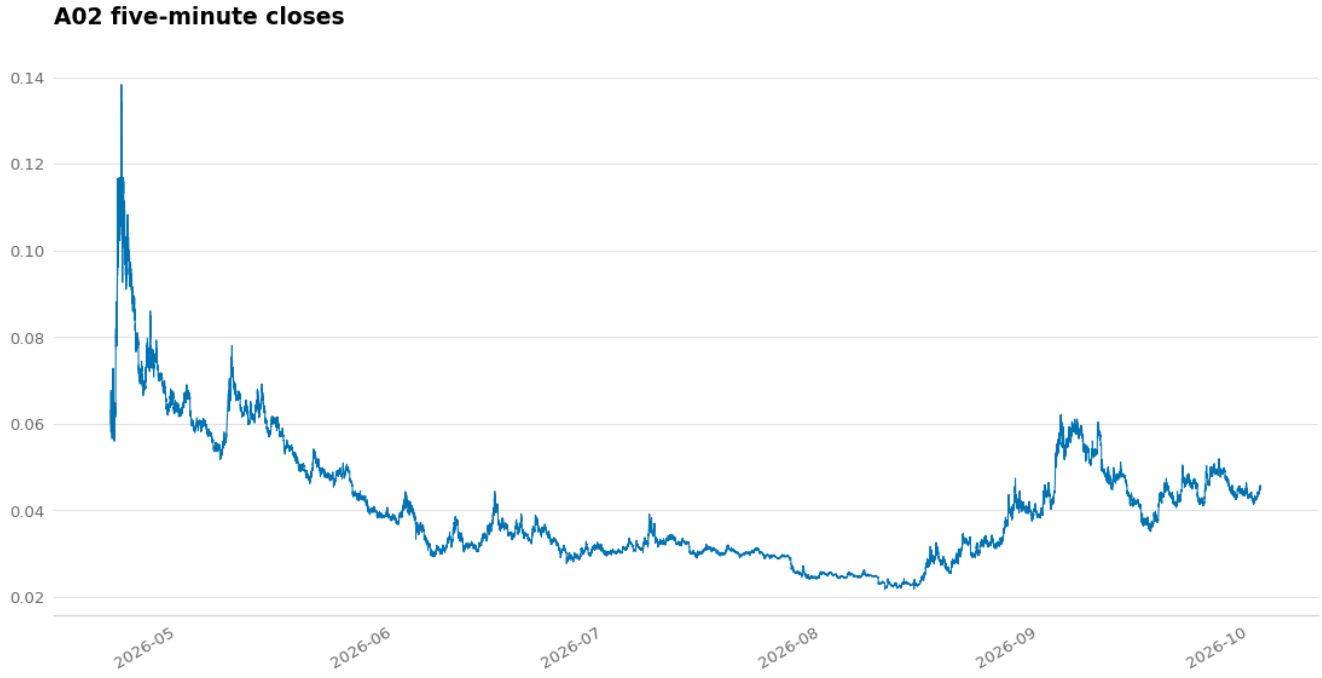
A02



A02, 495 fills, with the sell line above the buy line for the whole window.



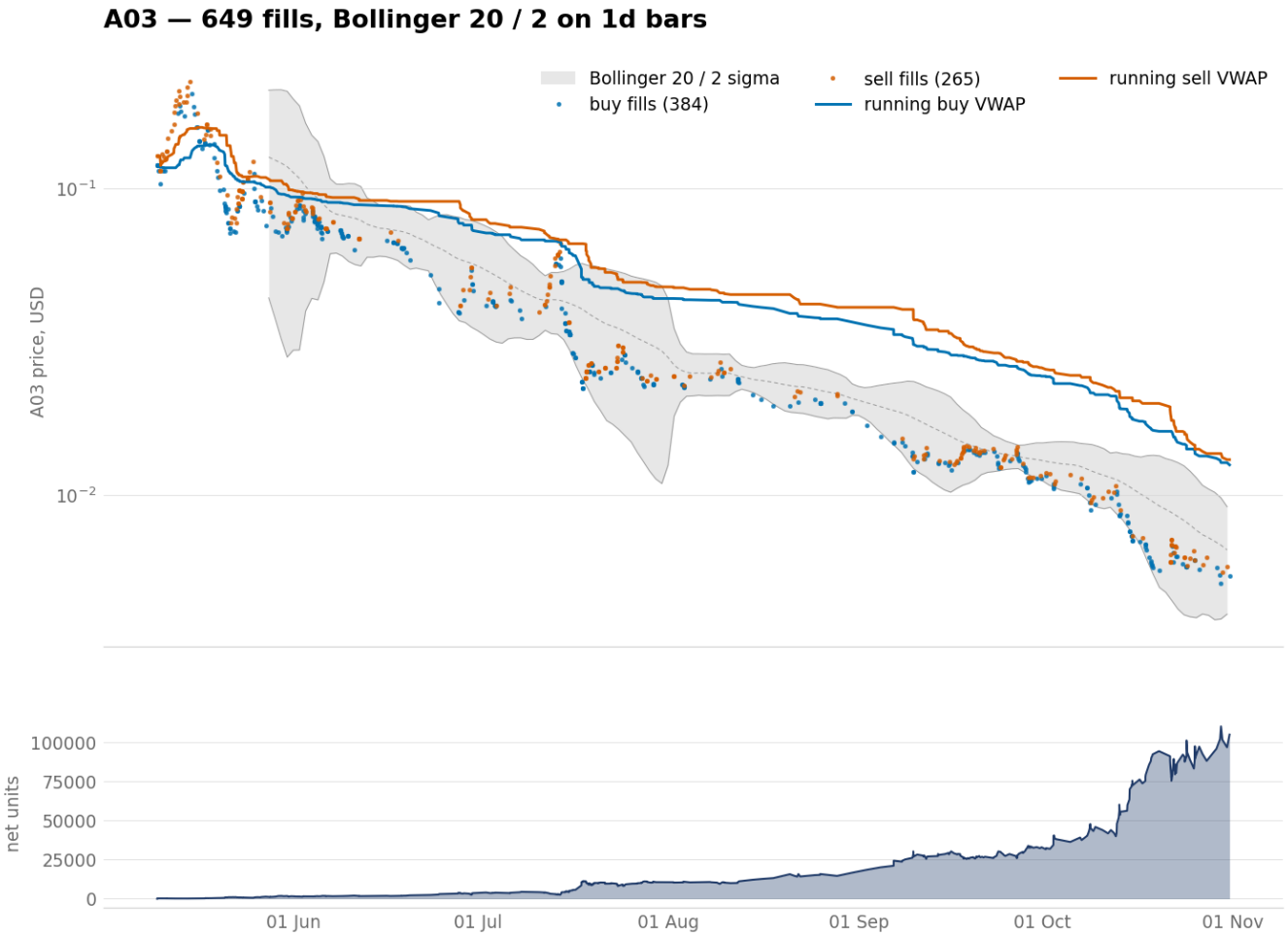
A02 weekly chart, 24 weekly candles built from 38,463 five-minute rows.



A02 price trace, 38,463 five-minute closes across the recorded window.

The sell line runs above the buy line across the whole window and the two meet in September. Net units climb to 11,699 in mid-August and end at 4,765.

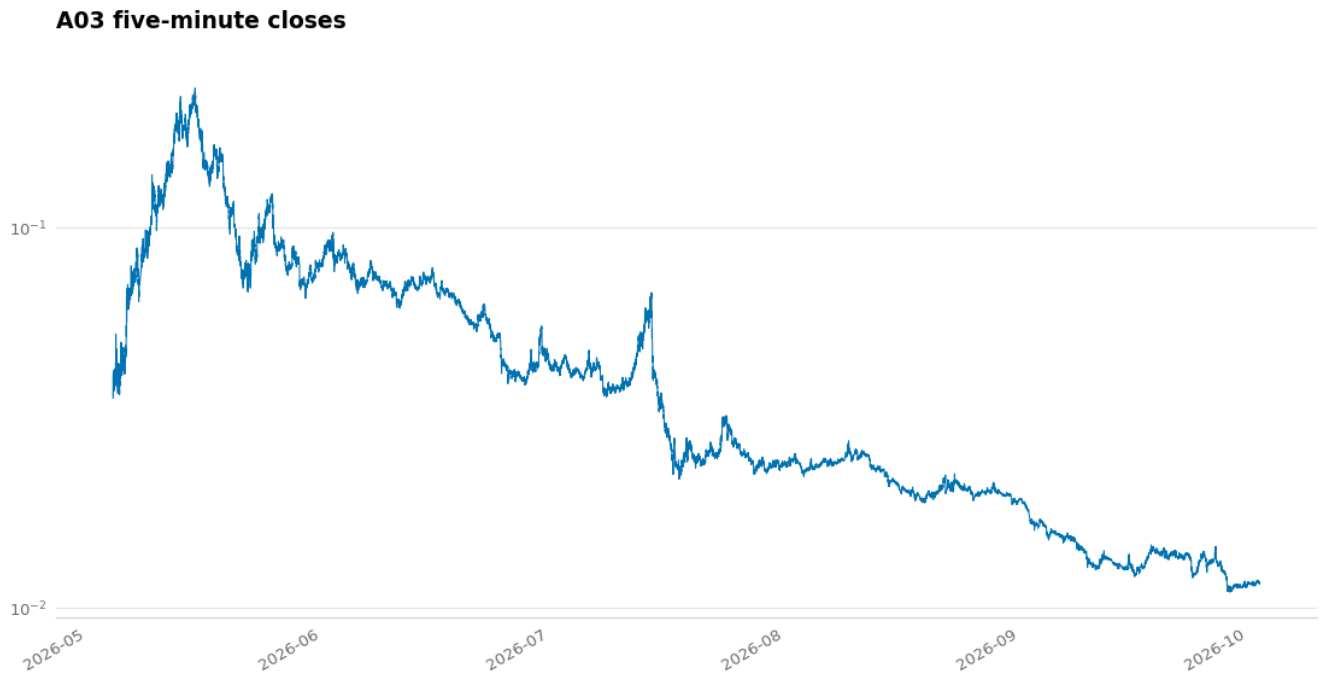
A03



A03, 459 fills, ending at the lowest index of the eight most-traded assets.



A03 weekly chart, 22 weekly candles built from 39,186 five-minute rows.



A03 price trace, 39,186 five-minute closes across the recorded window.

The second logarithmic axis, at a span of 13.25. Both lines hold near 0.09 through June, then break down in mid-July. Net units end at the highest point of the window, 18,510.

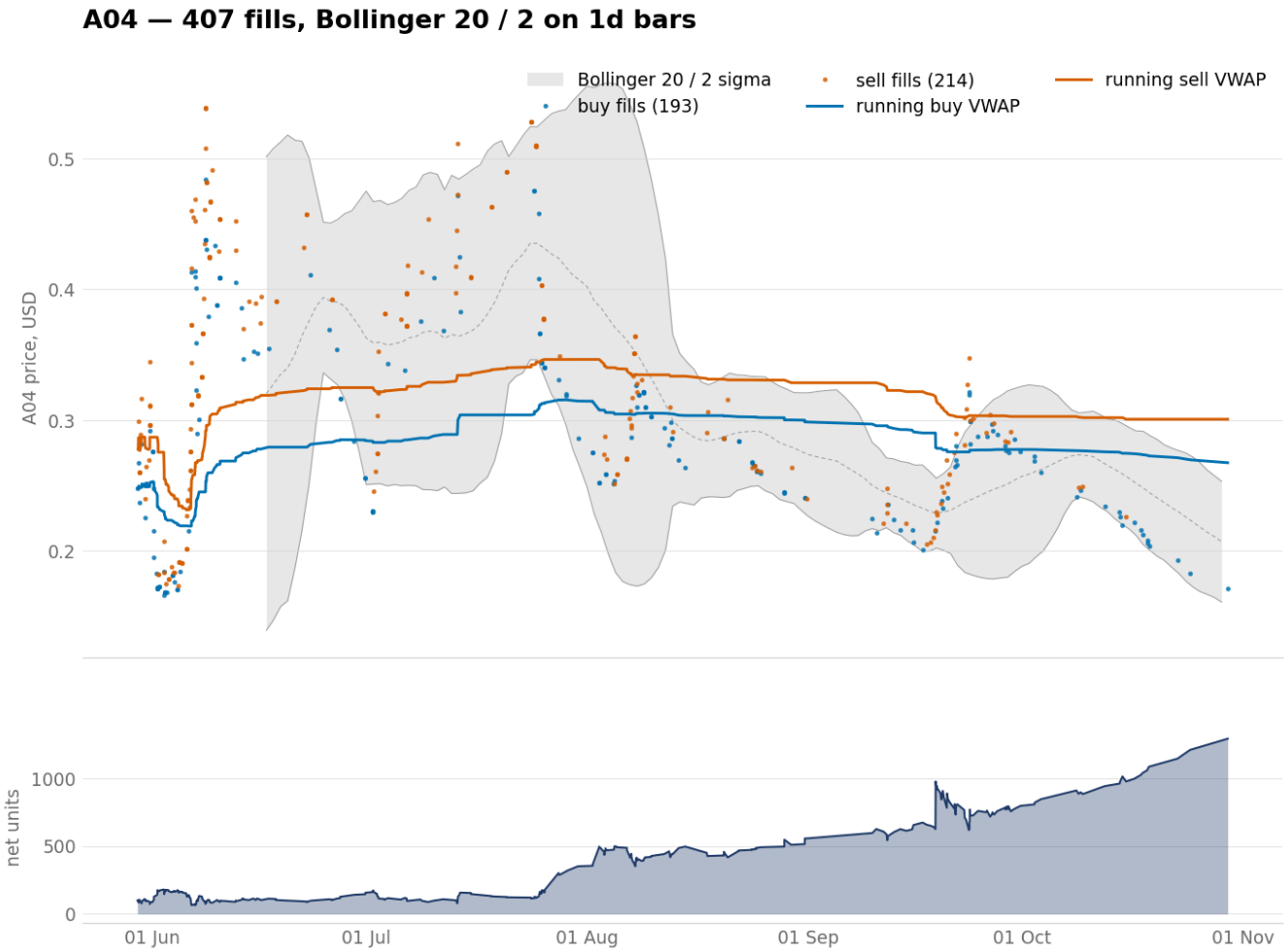
A logarithmic axis labels the powers of ten. A03's fills run from 0.01682 to 0.22281, a window holding one of them, and A01's window holds one as well. The module adds five minor ticks per decade and gives them the

same price formatter as the major ones, which is what puts 0.2000 down to 0.0200 on this axis.

```
SUBS = (1.0, 2.0, 3.0, 5.0, 7.0)

ax.yaxis.set_minor_locator(LogLocator(base=10.0, subs=SUBS, numticks=99))
ax.yaxis.set_minor_formatter(FuncFormatter(fmt_price))
ax.tick_params(axis="y", which="minor", labelsize=TYPE["cap"]["size"])
```

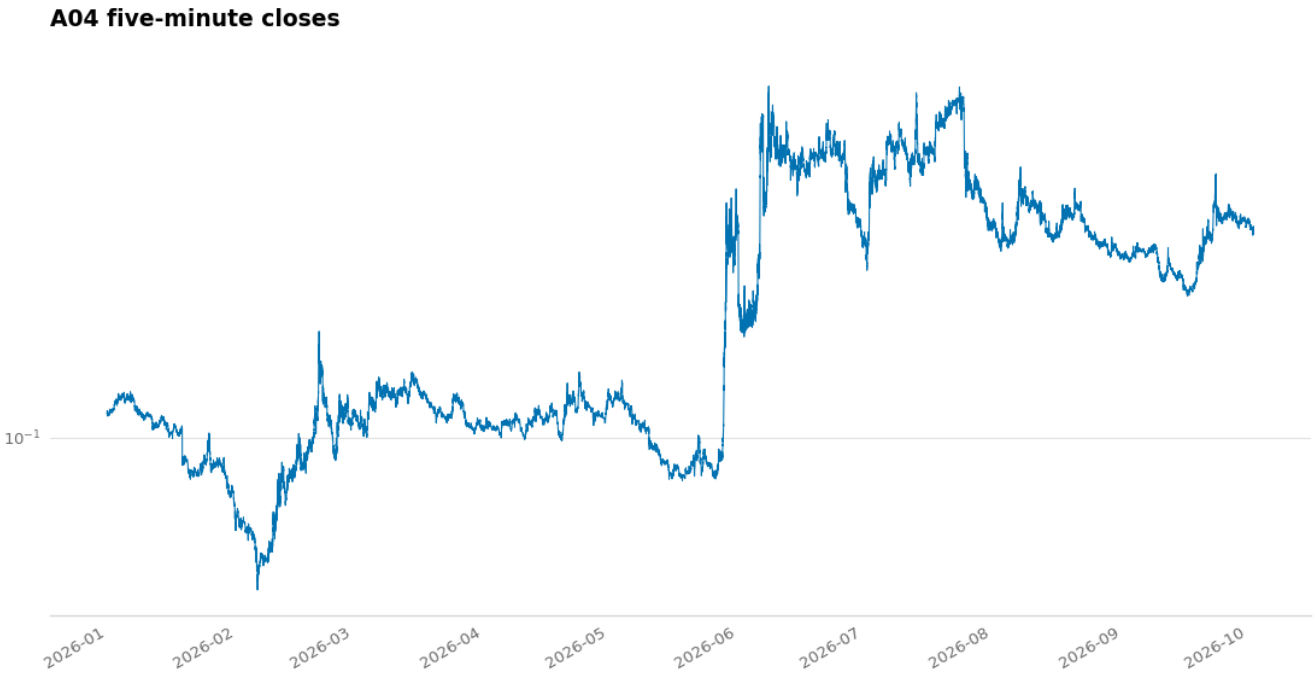
A04



A04, 322 fills, with more sells than buys and both lines rising.



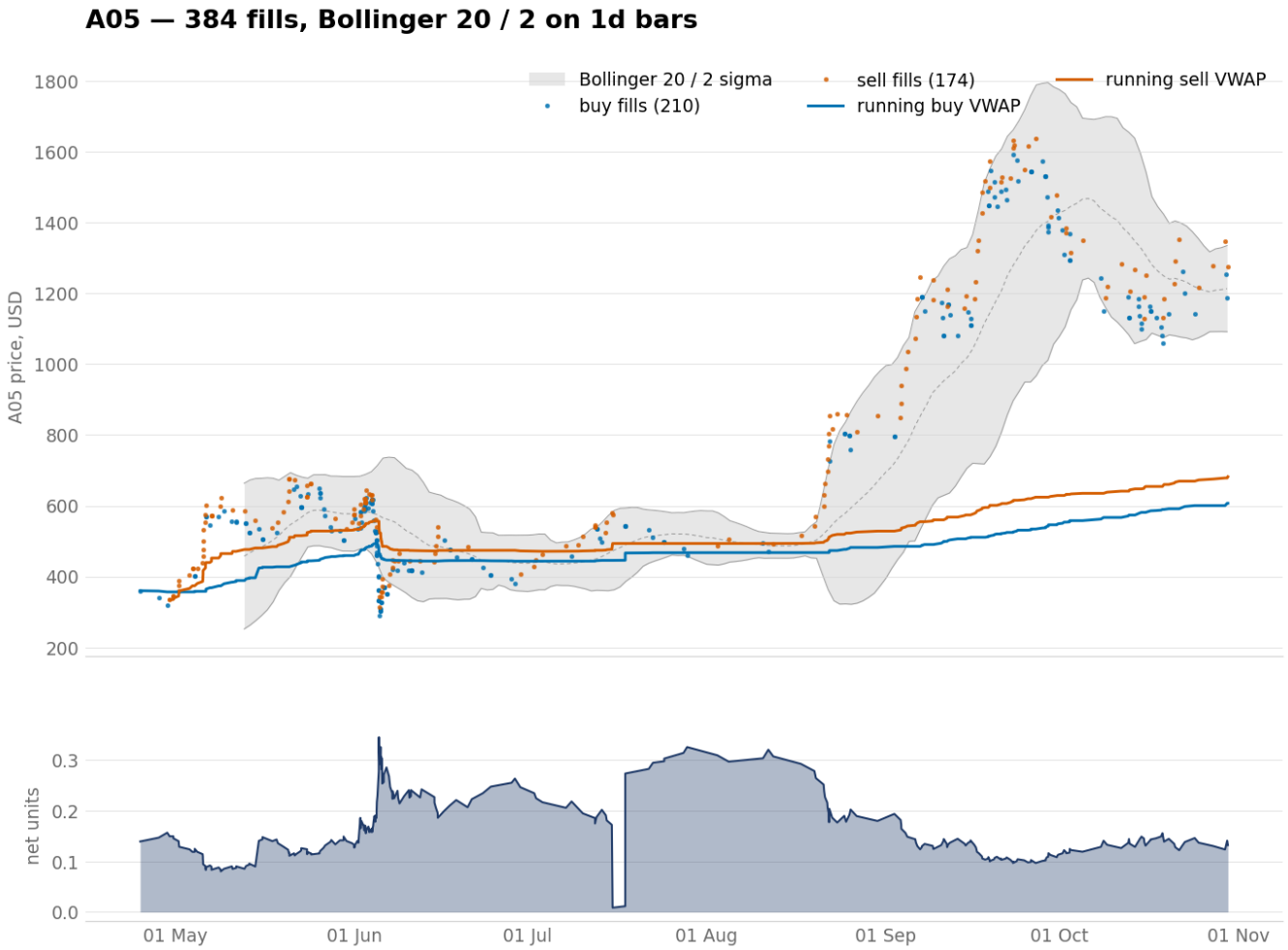
A04 weekly chart, 40 weekly candles built from 47,201 five-minute rows.



A04 price trace, 47,201 five-minute closes across the recorded window.

One of eleven charts with more sells than buys. Both lines dip to about 0.22 in early June and climb after it. Net units step from about 120 to about 500 through late July.

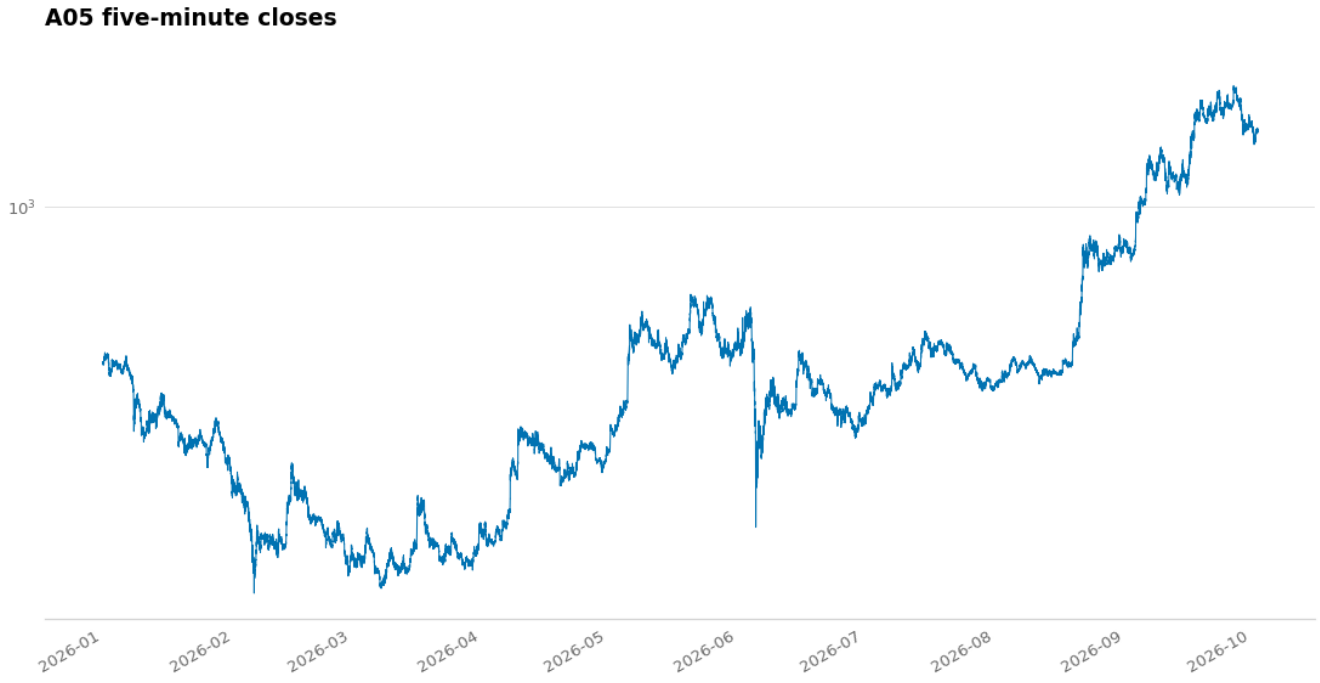
A05



A05, 261 fills, the highest final index of the eight most-traded assets.



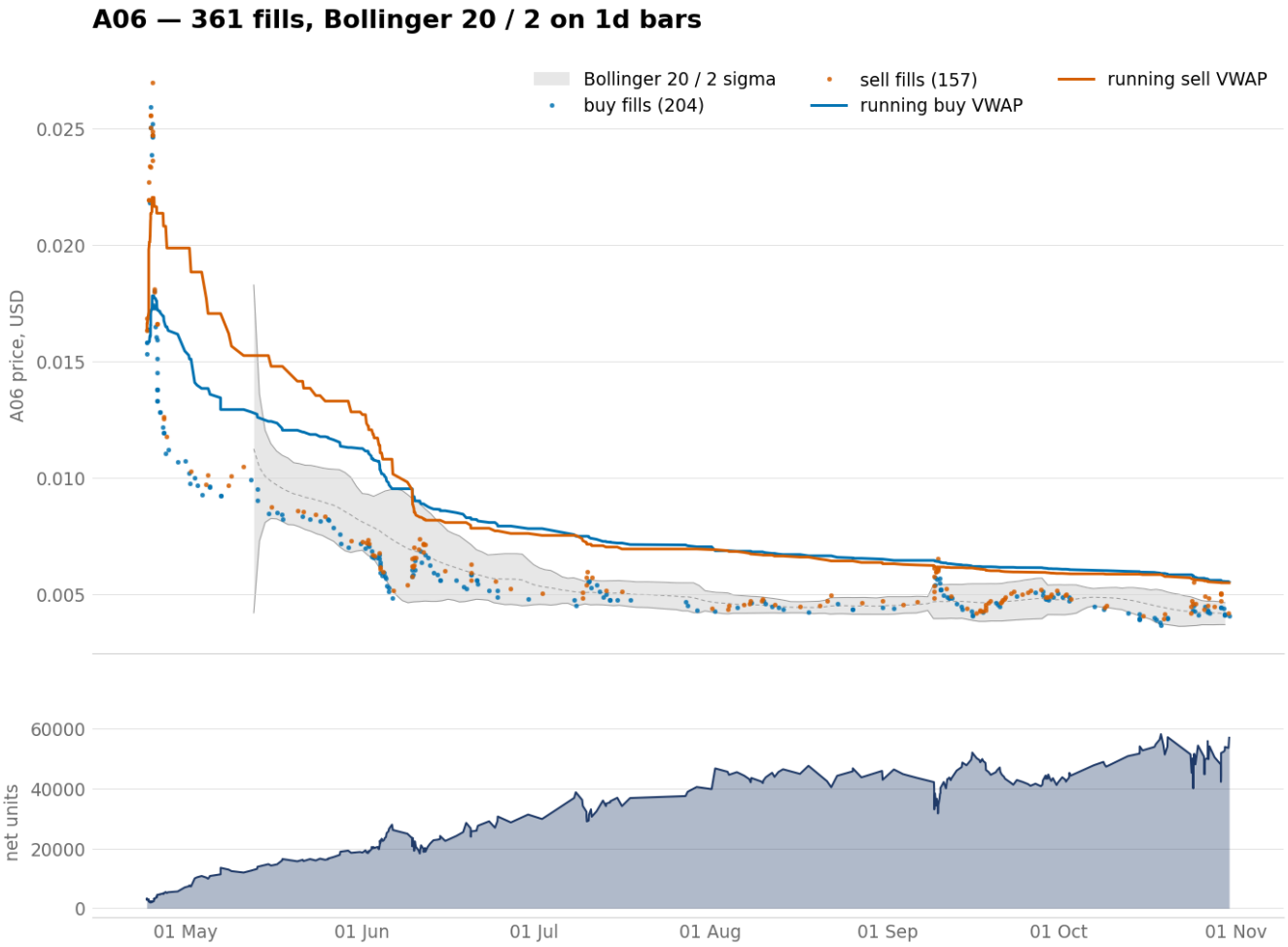
A05 weekly chart, 40 weekly candles built from 79,004 five-minute rows.



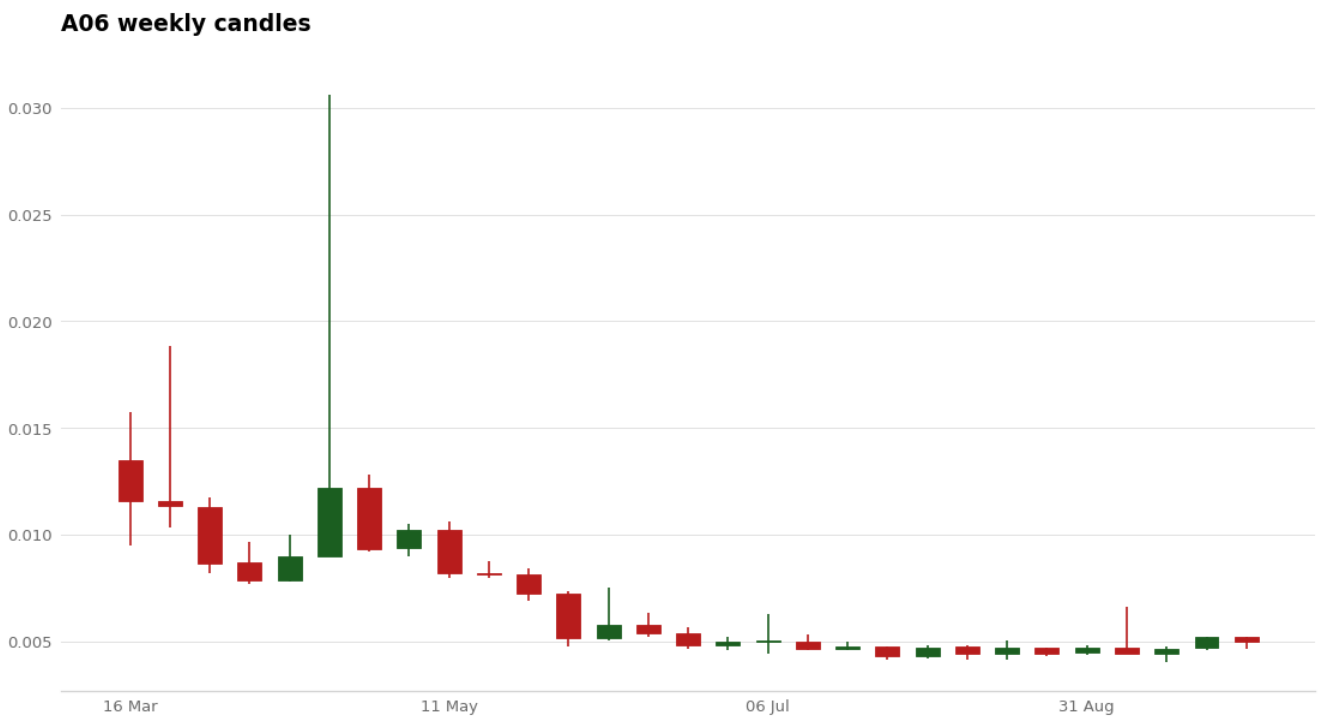
A05 price trace, 79,004 five-minute closes across the recorded window.

The buy line rises from 359.67 to 485.05, the largest climb among the eight in the combined chart. Net units fall to near zero in mid-July, then rebuild to about 0.20.

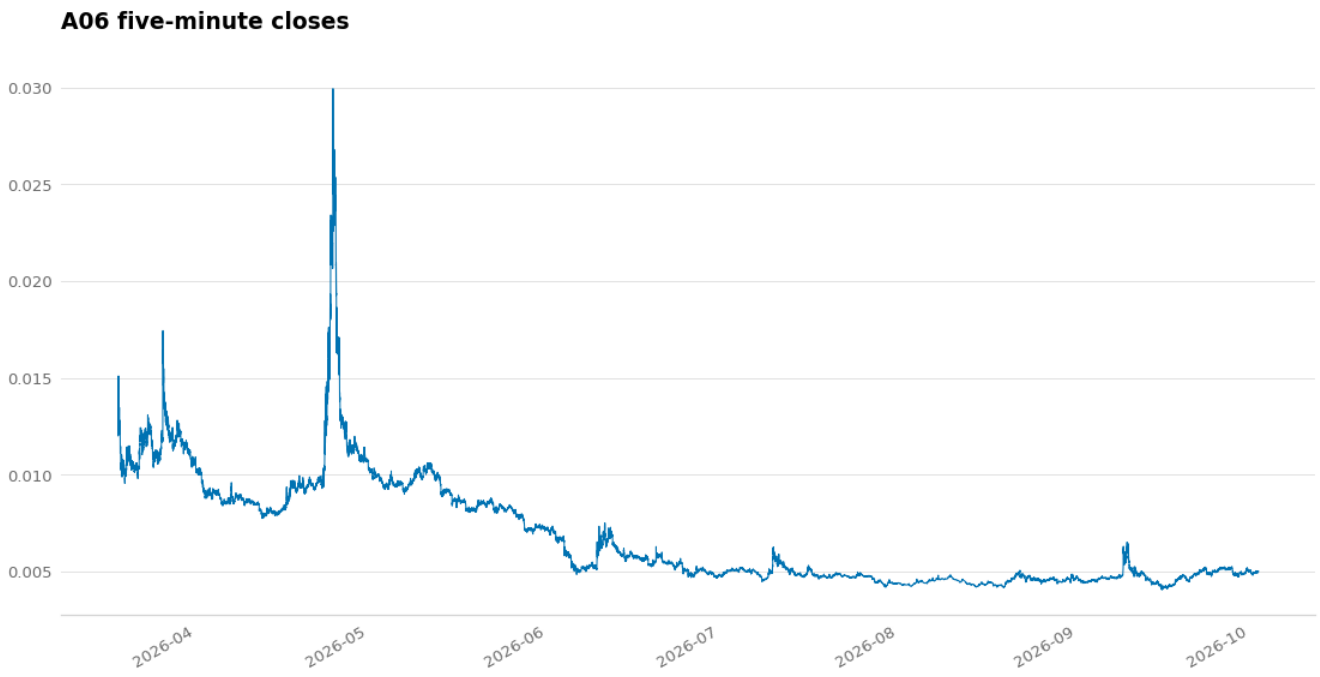
A06



A06, 230 fills, with the two lines converging from mid-June.



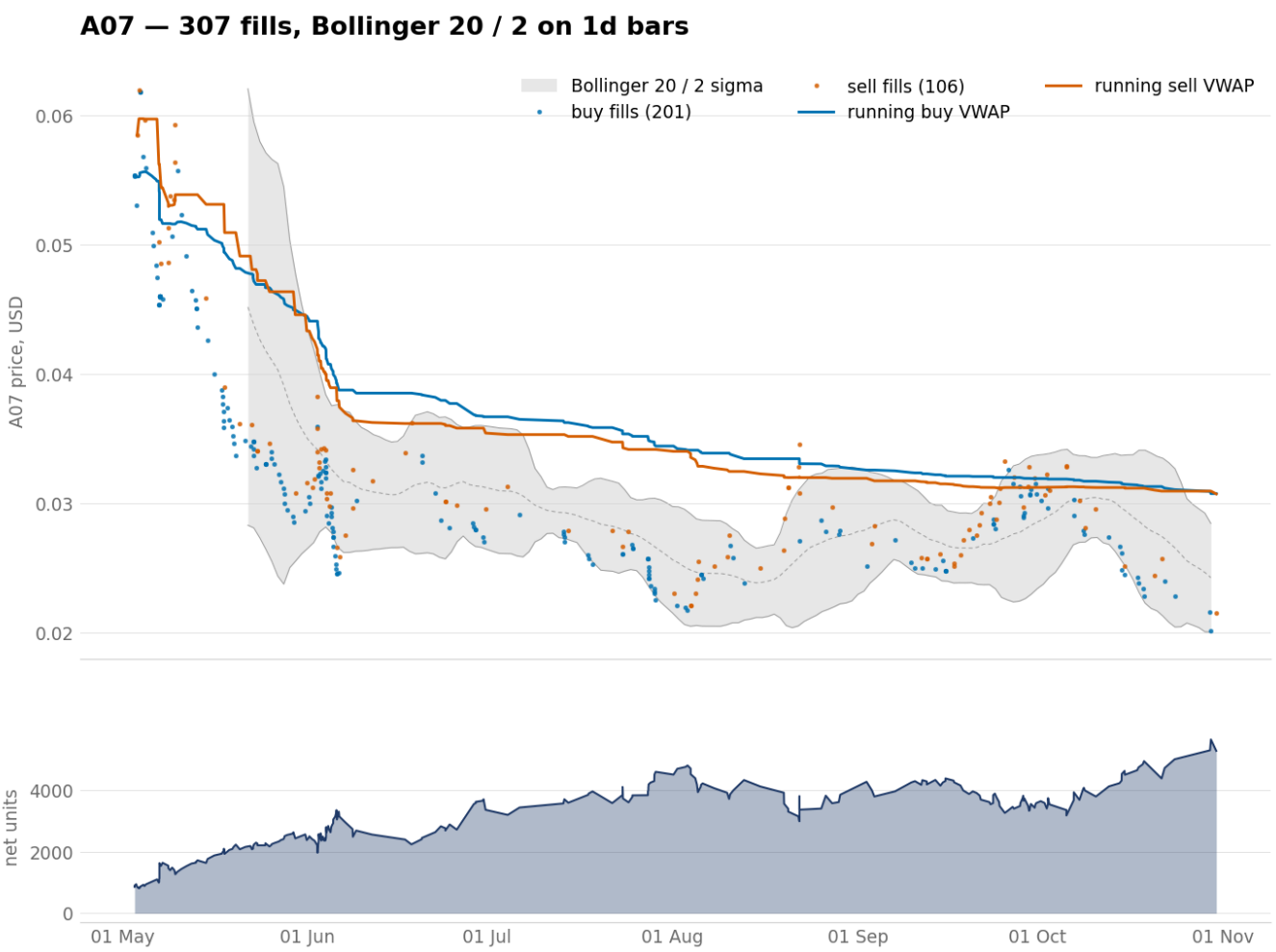
A06 weekly chart, 29 weekly candles built from 33,344 five-minute rows.



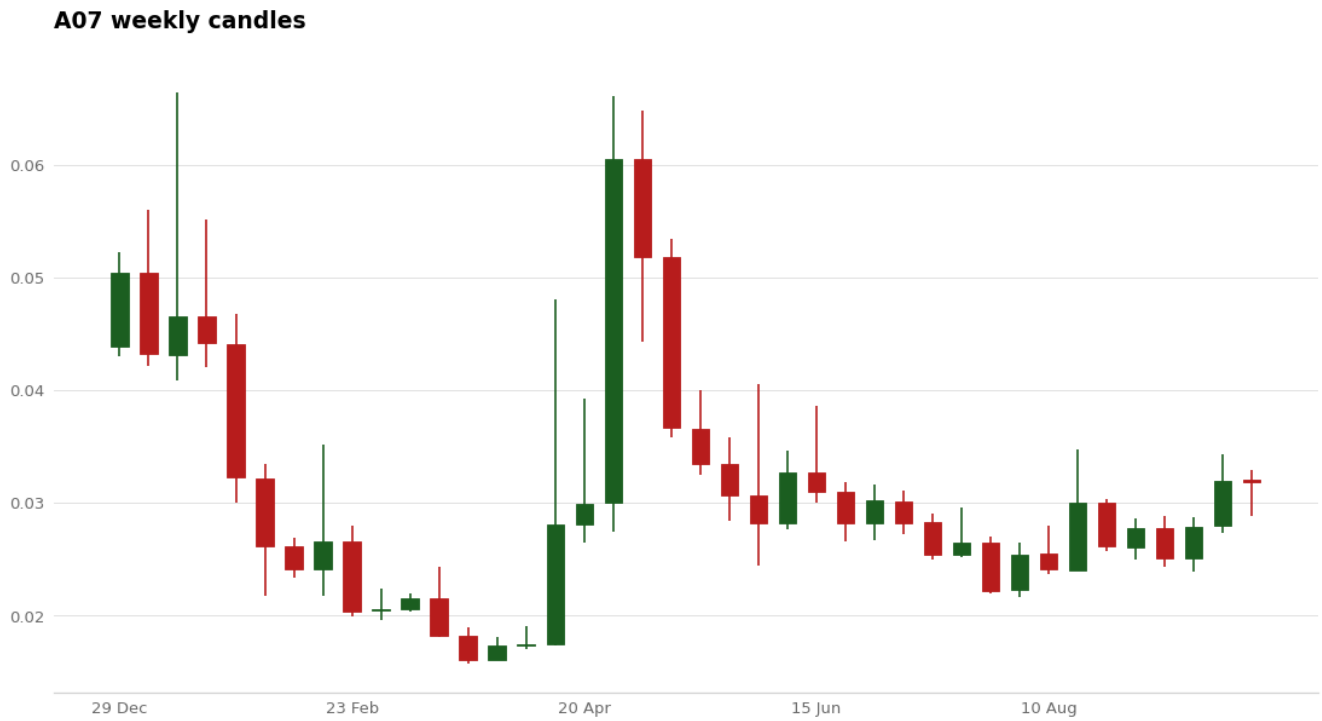
A06 price trace, 33,344 five-minute closes across the recorded window.

The sell line runs above the buy line until mid-June, then the two converge and the sell line finishes just below. Net units grow to 47,574 and end at 44,795.

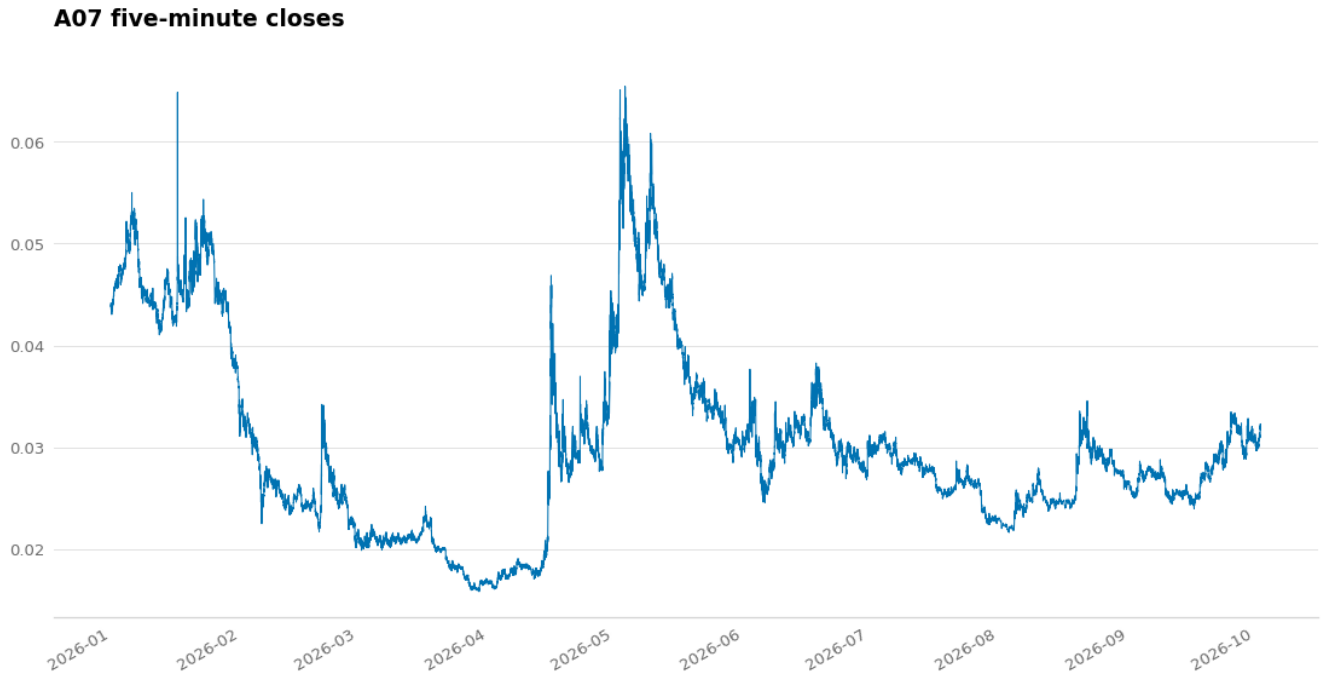
A07



A07, 226 fills, with a step down at the start of June.



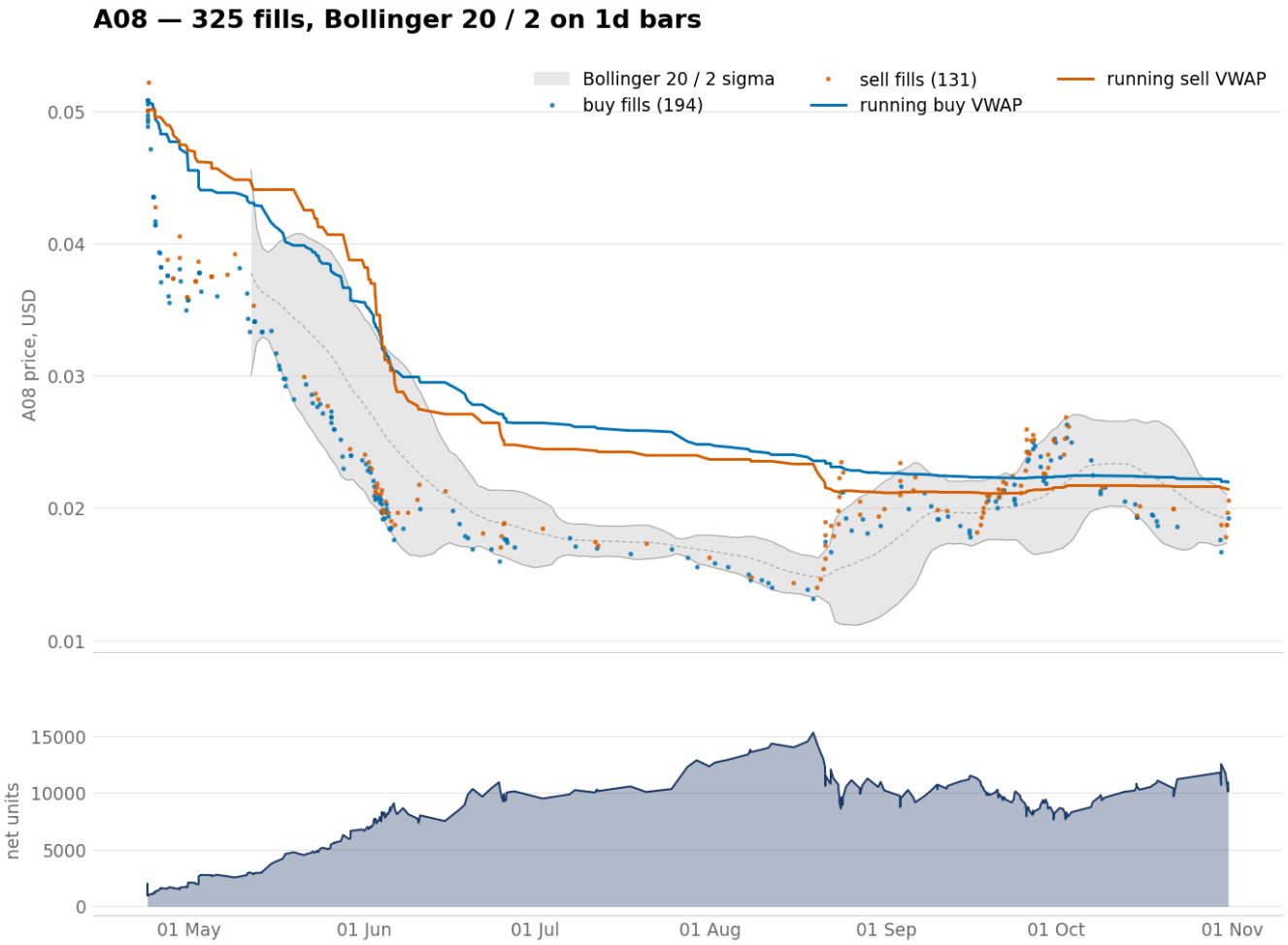
A07 weekly chart, 40 weekly candles built from 49,638 five-minute rows.



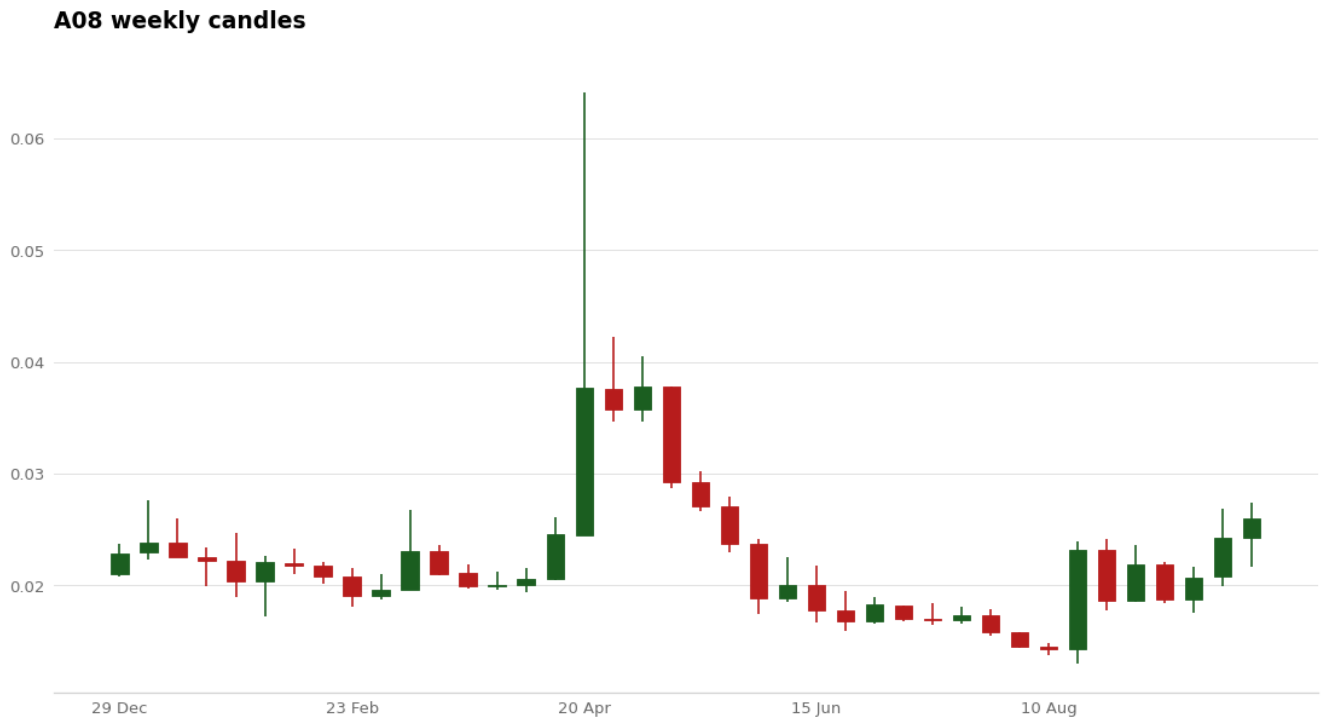
A07 price trace, 49,638 five-minute closes across the recorded window.

One step down at the start of June takes the buy line from about 0.044 to about 0.039, and a slow decline follows. The sell line crosses below the buy line at that same step.

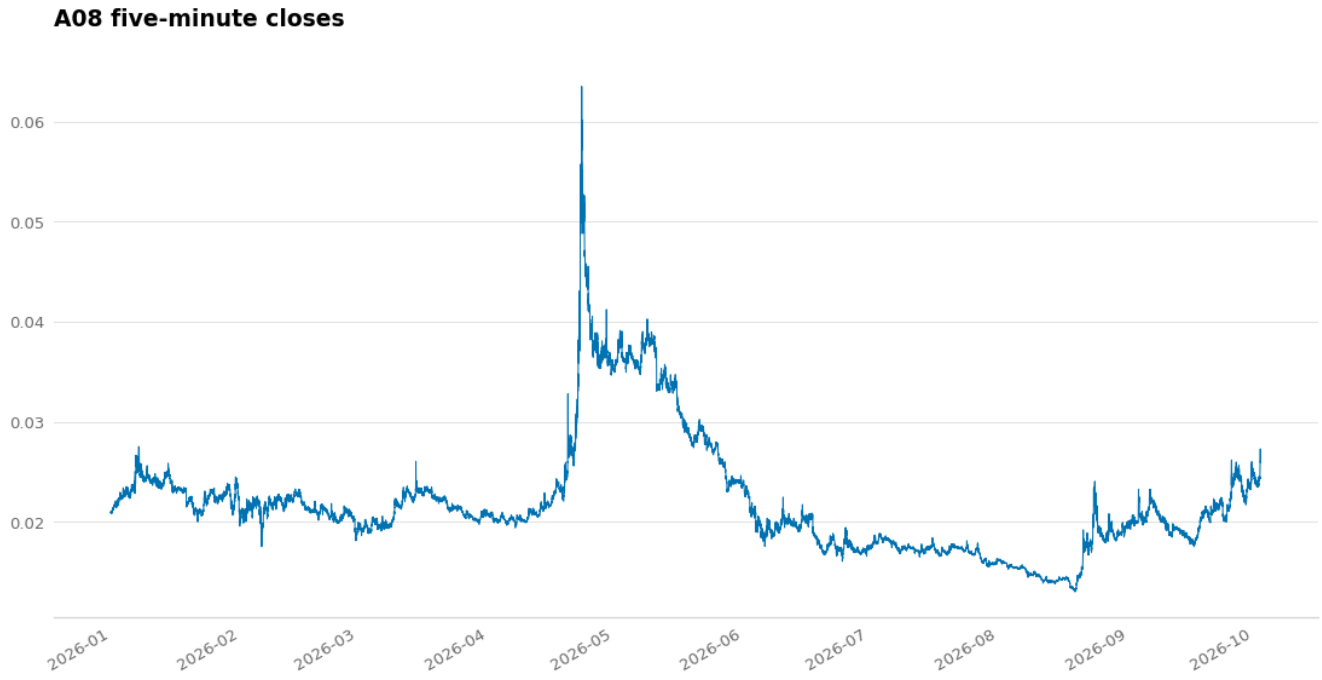
A08



A08, 220 fills, with the sell line crossing below the buy line in mid-June.



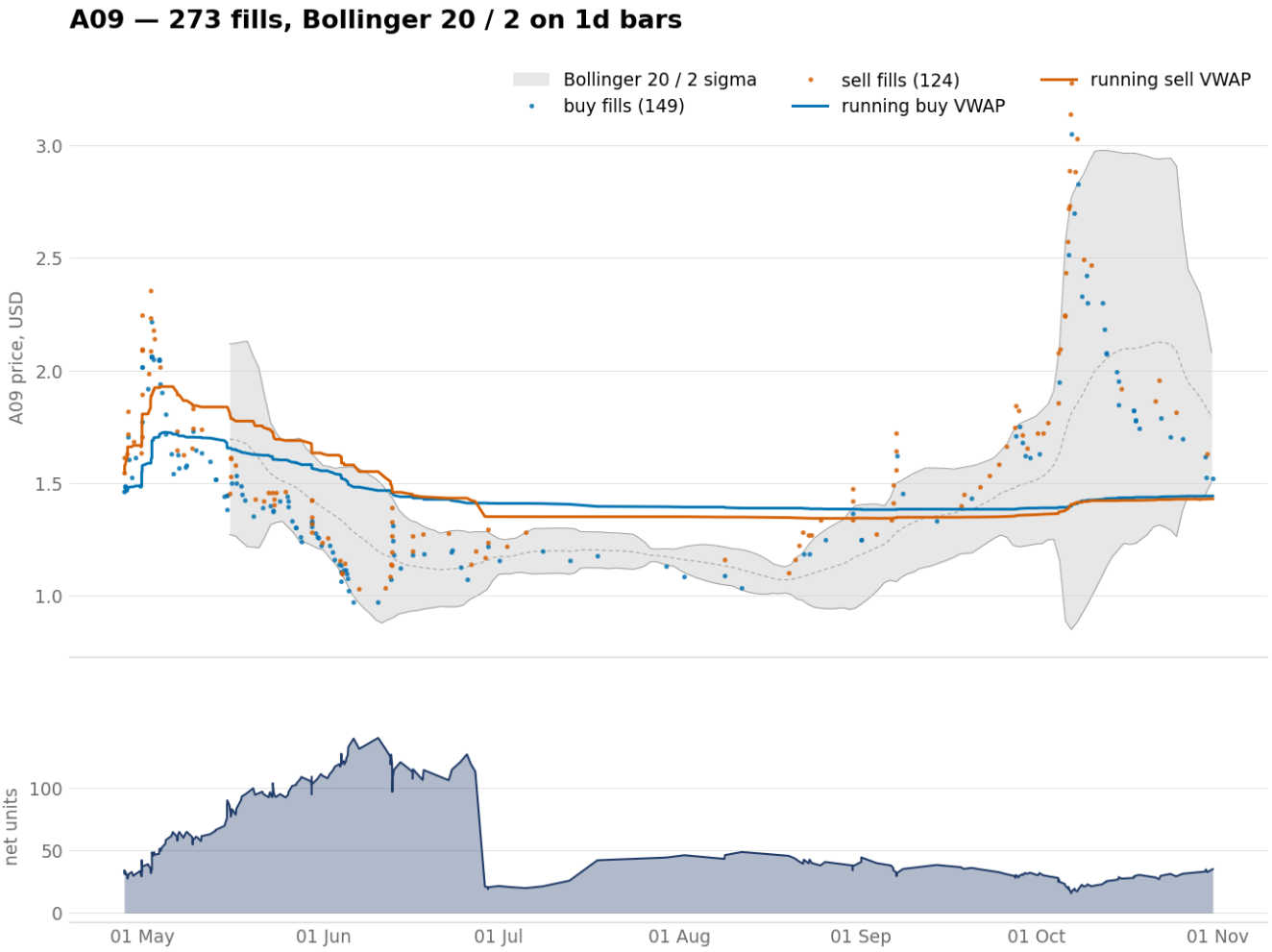
A08 weekly chart, 40 weekly candles built from 52,751 five-minute rows.



A08 price trace, 52,751 five-minute closes across the recorded window.

The crossing holds for the rest of the window. Net units peak at 15,301 in mid-August, then fall to 9,483.

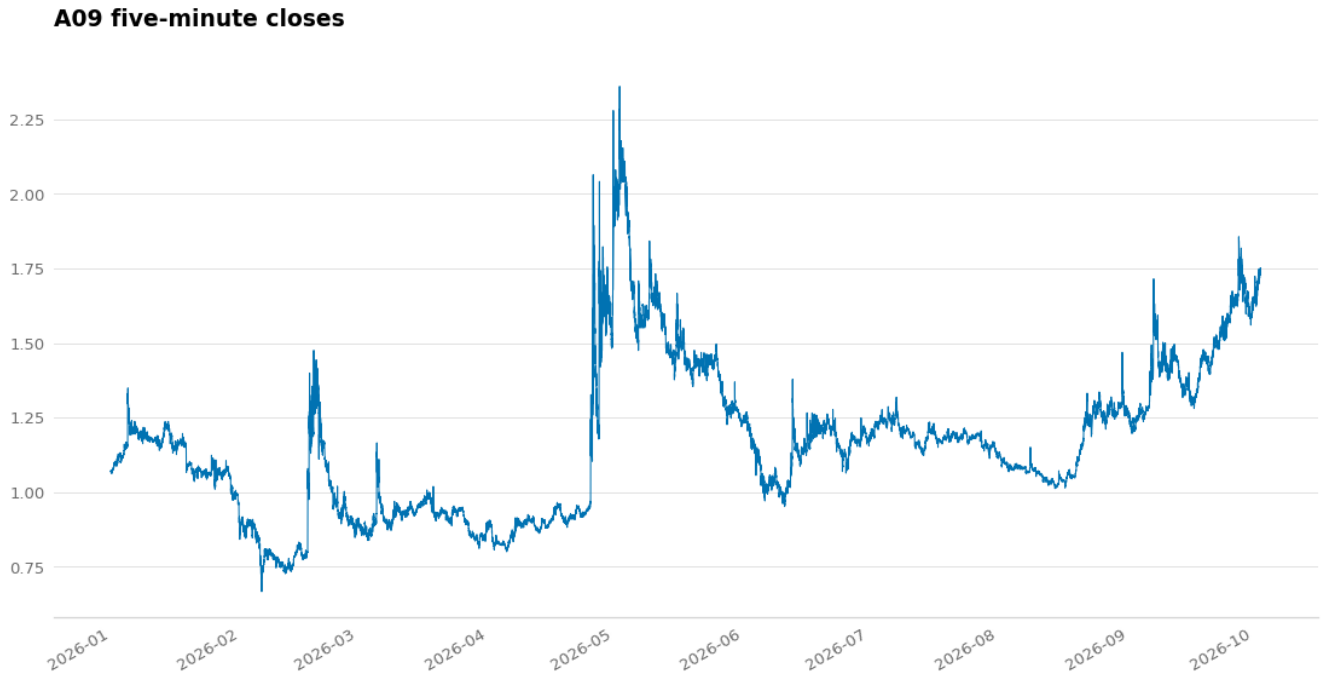
A09



A09, 196 fills, with both lines flat from July onward.



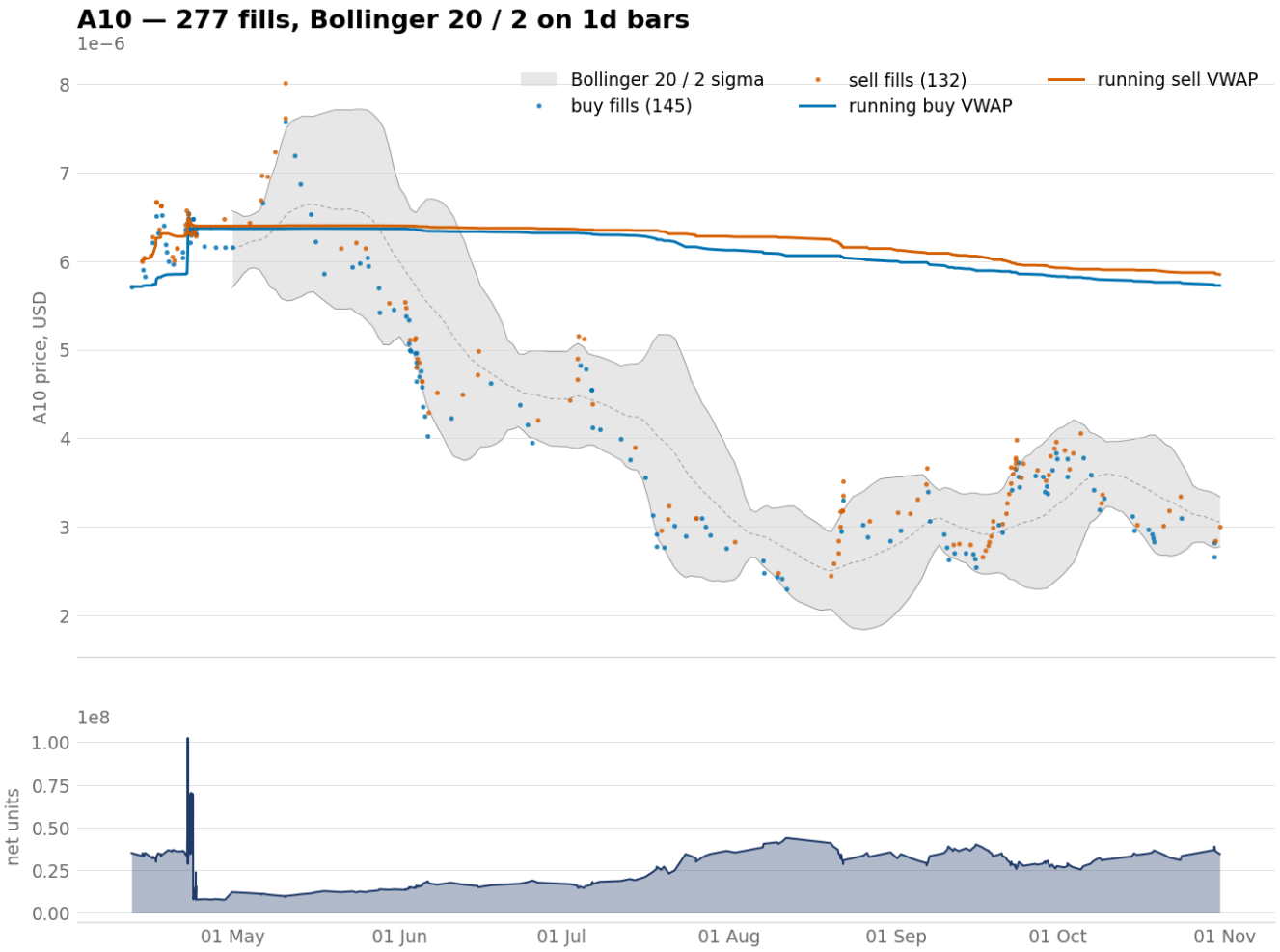
A09 weekly chart, 40 weekly candles built from 53,988 five-minute rows.



A09 price trace, 53,988 five-minute closes across the recorded window.

Both lines rise to a peak near 1 May, then fall to a flat run from July. Net units drop from 140 to about 20 at the end of June and hold near 40 after that.

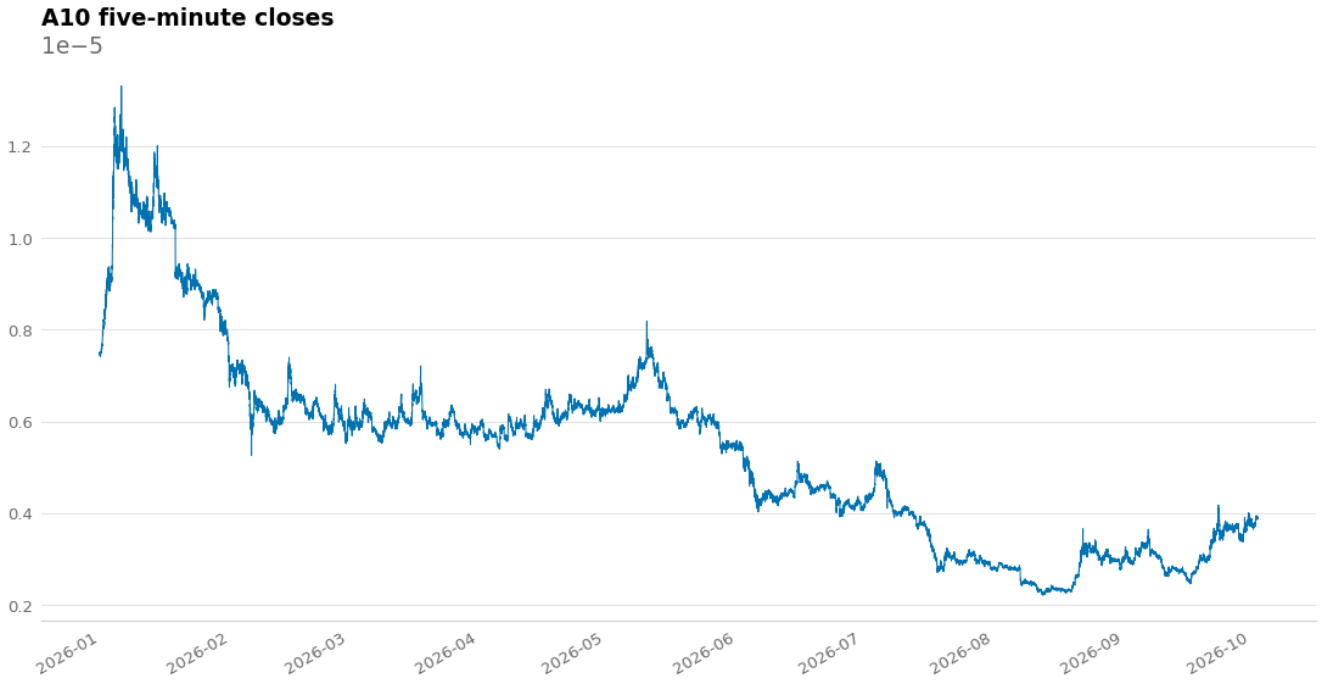
A10



A10, 191 fills, priced near six millionths of a dollar.



A10 weekly chart, 40 weekly candles built from 72,287 five-minute rows.



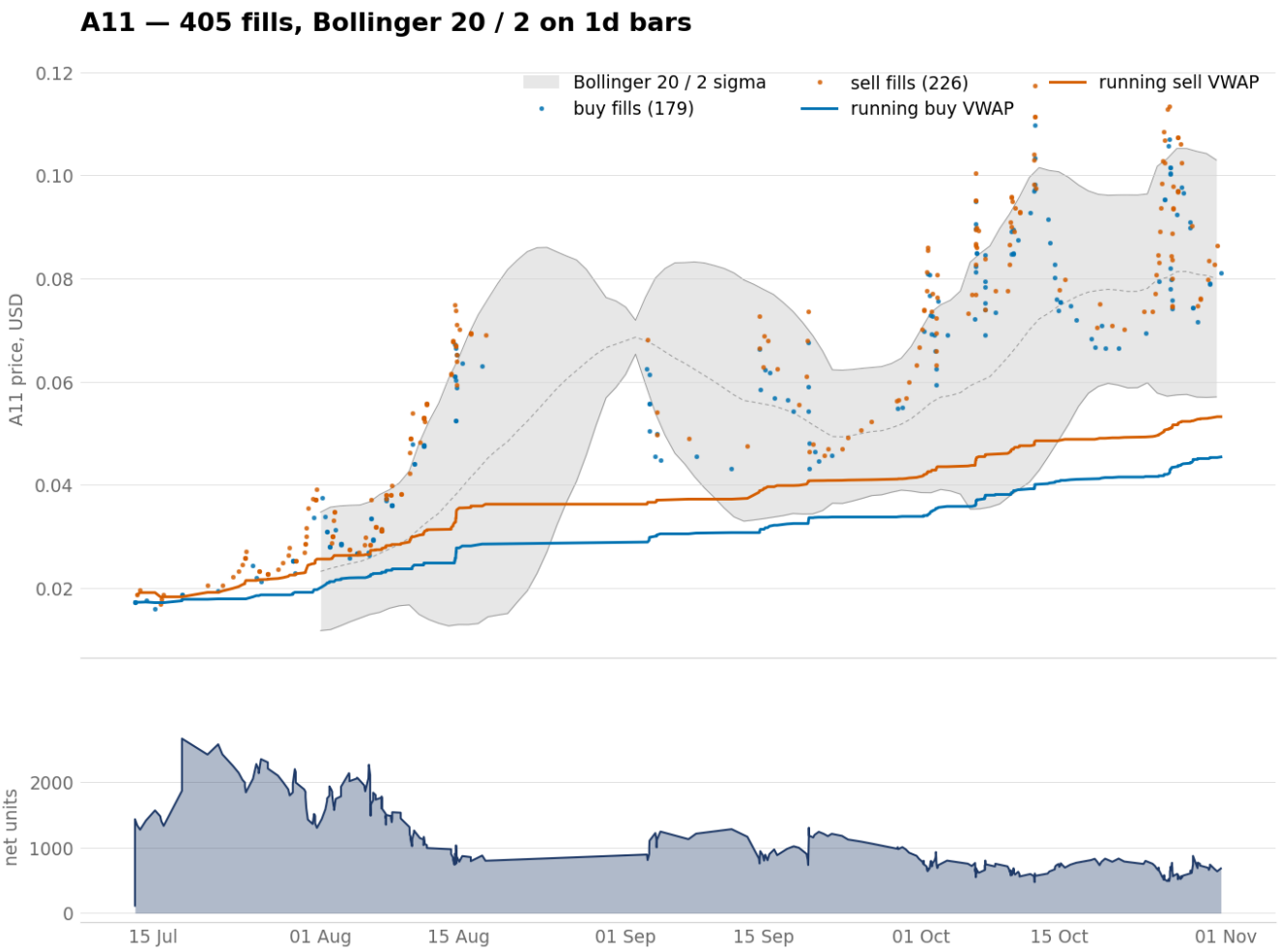
A10 price trace, 72,287 five-minute closes across the recorded window.

The lowest-priced base in the record. The buy line holds near 6.3e-06 from late April while the fill dots fall from 8.0e-06 to 2.3e-06. Net units spike to 1.02e+08 in late April and end at 3.22e+07.

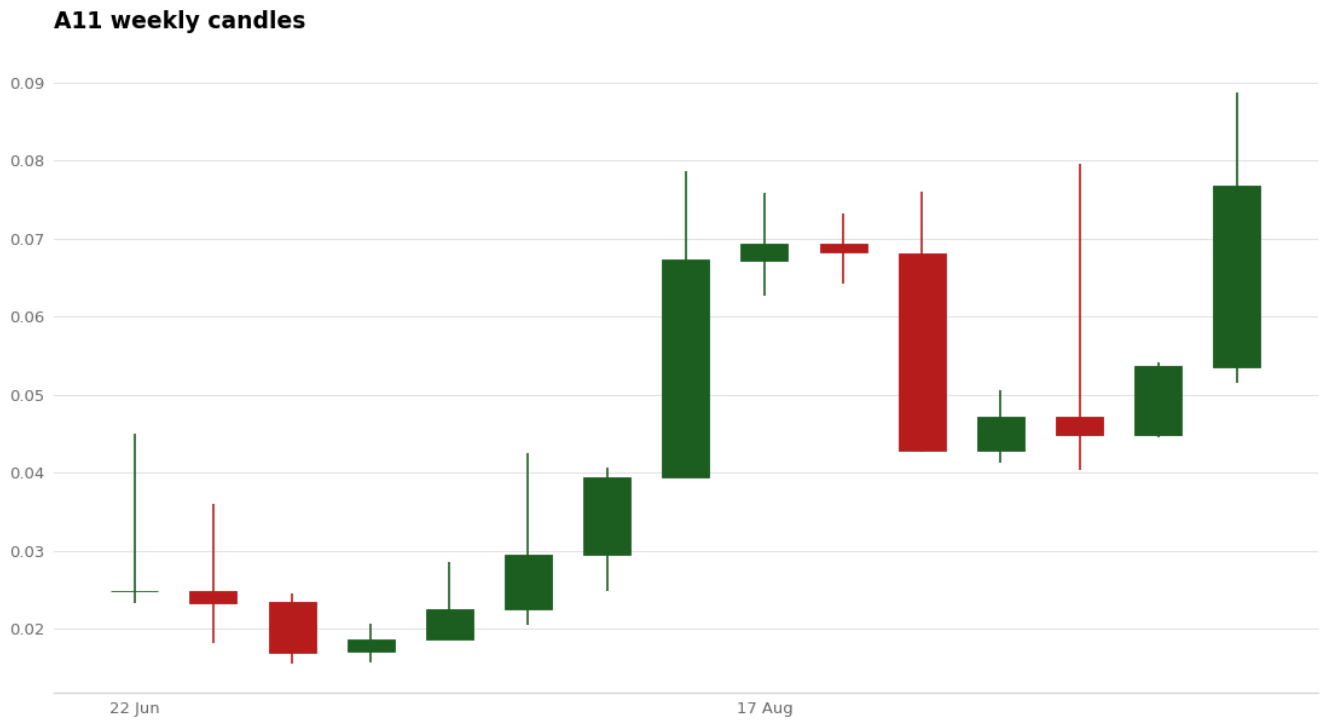
A10 is the only base whose fills fall under a tenth of a cent. The next lowest is A29 at 0.001479, so A10 alone reaches the last branch of the tick formatter, and this is the only chart of the 38 labelled in scientific notation.

```
def fmt_price(v, _pos=None) -> str:
    """Format a y-axis price tick, keeping a micro-cap significand visible."""
    if v == 0:
        return "0"
    if abs(v) >= 1000:
        return f"{v:,.0f}"
    if abs(v) >= 1:
        return f"{v:,.2f}"
    if abs(v) >= 0.001:
        return f"{v:.4f}"
    return f"{v:.2e}"
```

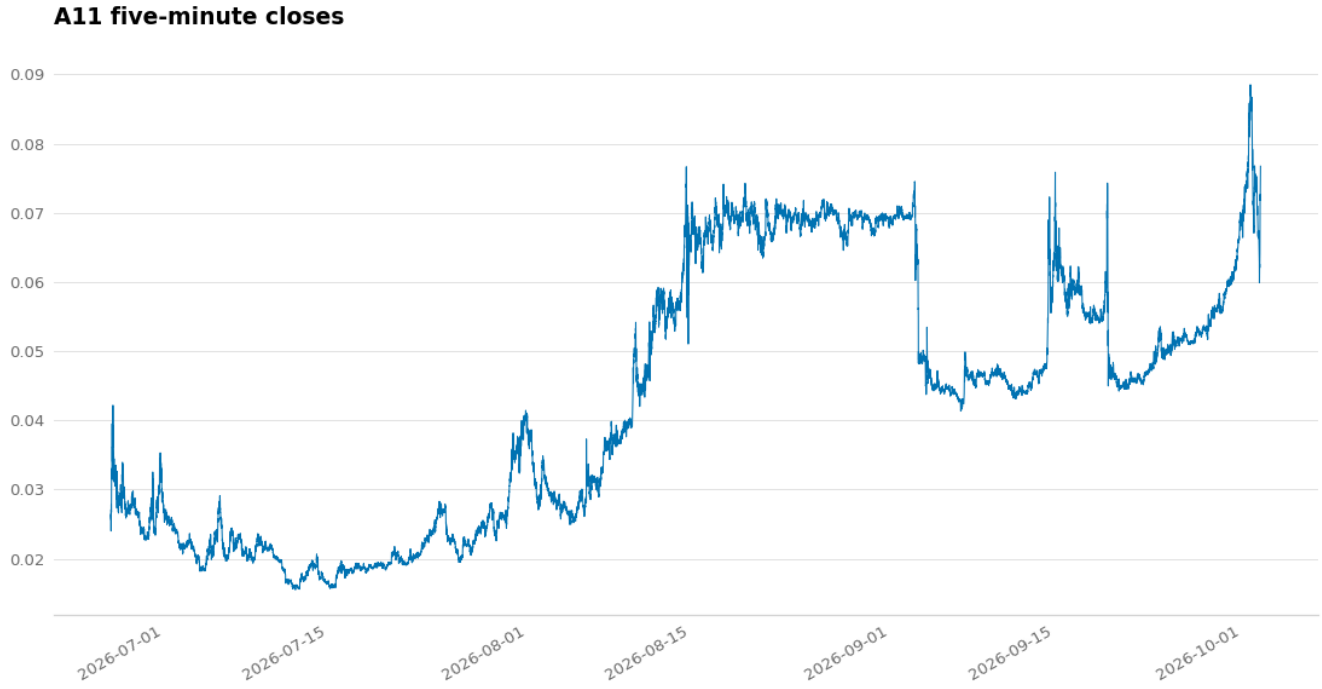
A11



A11, 178 fills, and the largest rise of the 38.



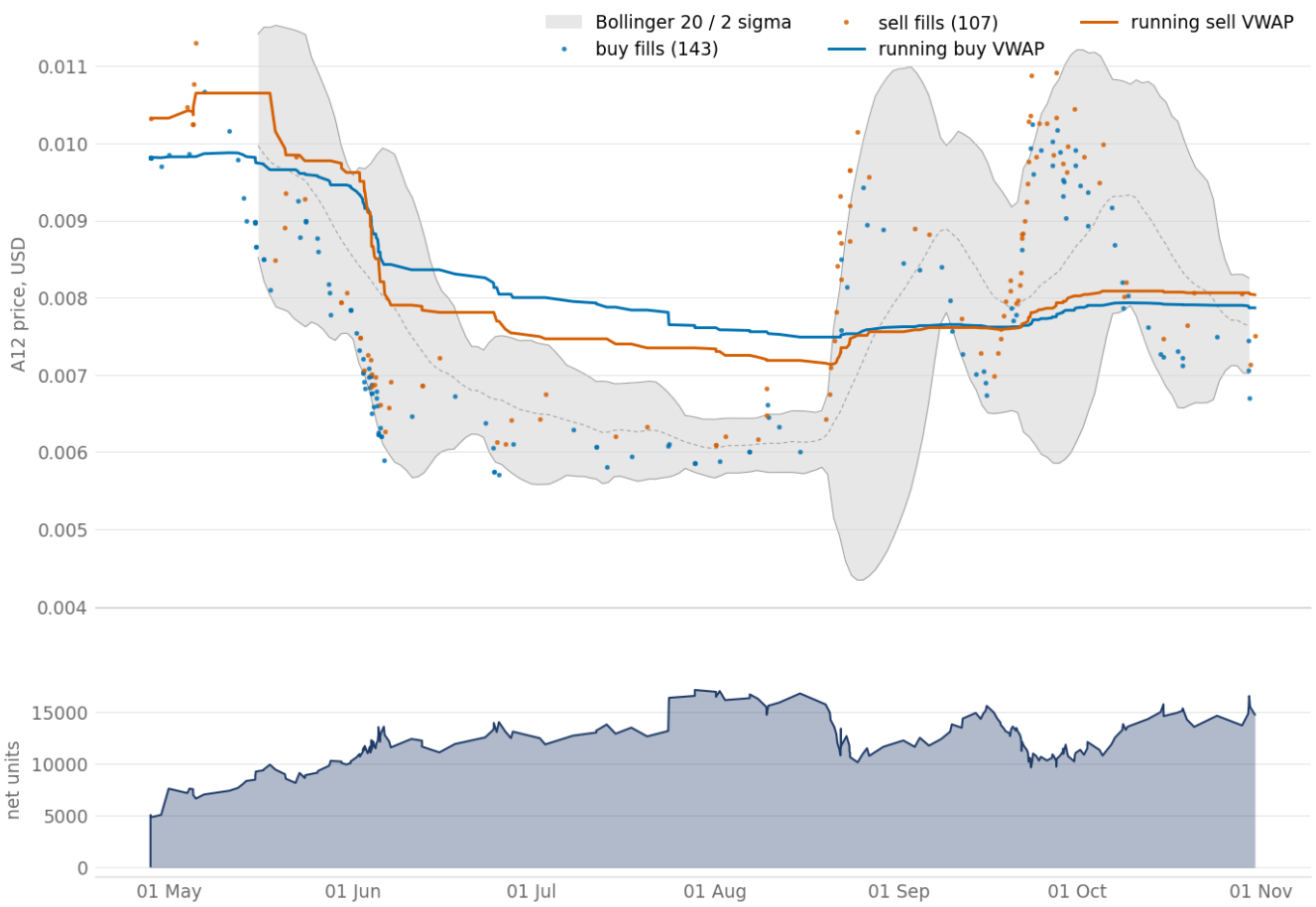
A11 weekly chart, 15 weekly candles built from 26,750 five-minute rows.



A11 price trace, 26,750 five-minute closes across the recorded window.

The buy line rises from 0.01722 to 0.02979, an index of 1.730 and the largest of the 38. Both lines climb through August. Net units drop across that same climb, 2,669 down to 1,107.

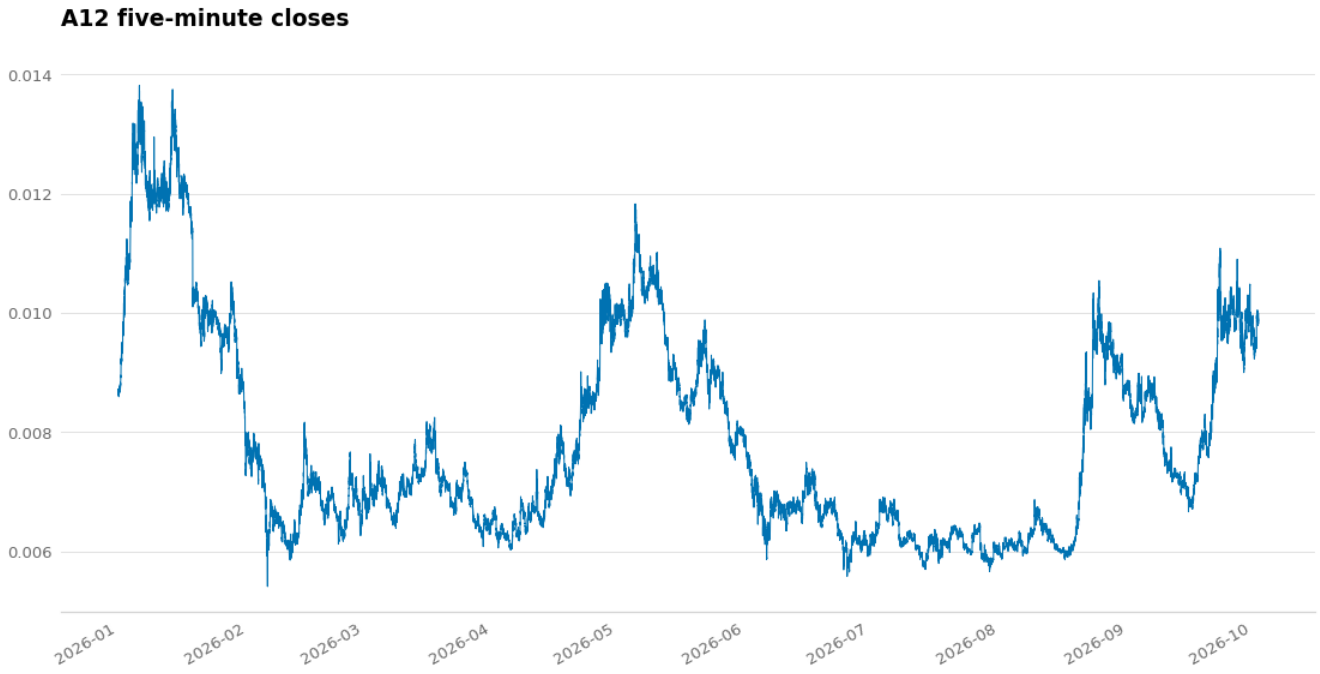
A12



A12, 159 fills, with the sell line dropping below the buy line at the start of June.



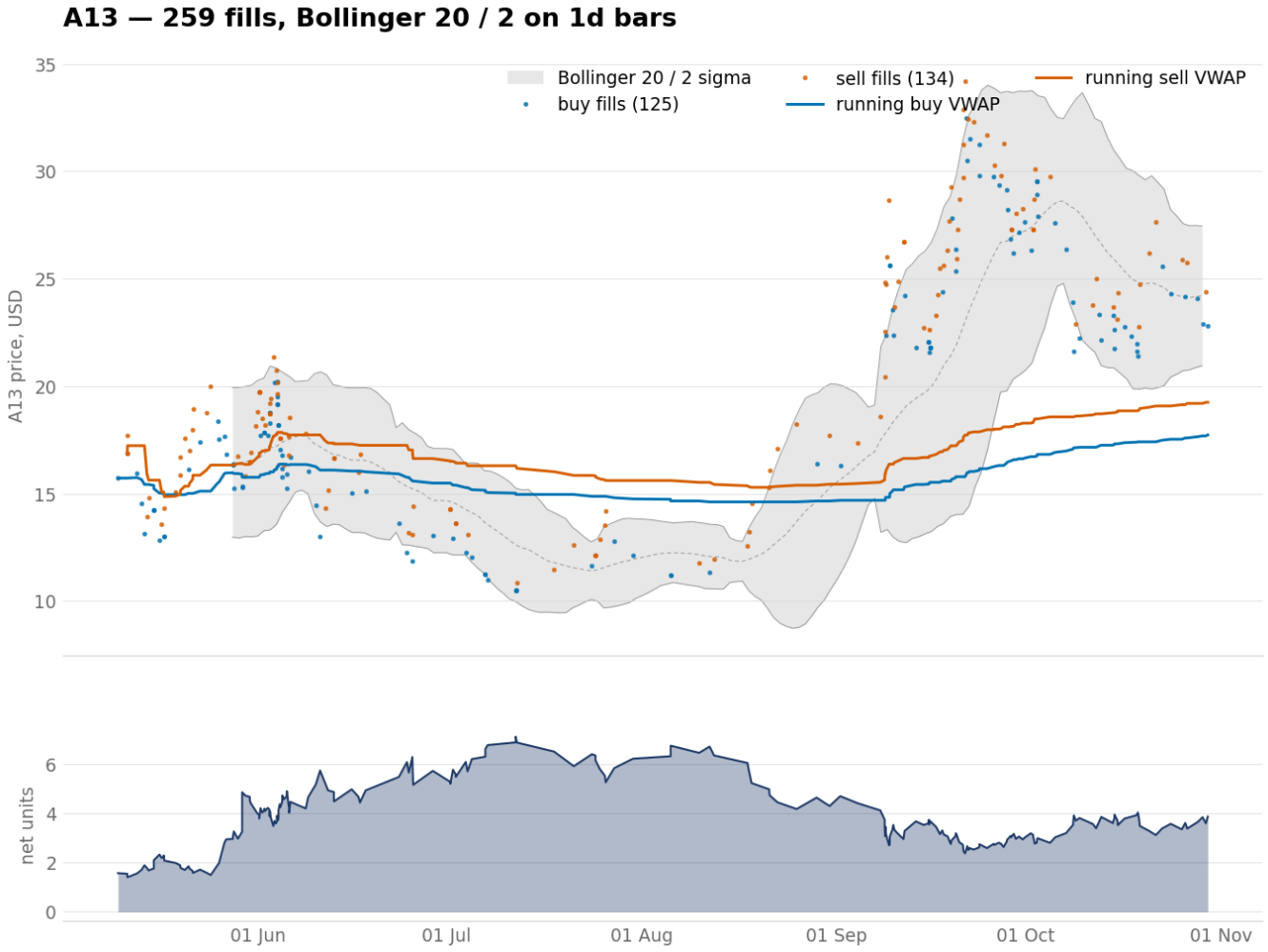
A12 weekly chart, 40 weekly candles built from 78,332 five-minute rows.



A12 price trace, 78,332 five-minute closes across the recorded window.

The buy line holds near 0.0098 through May and the sell line near 0.0105, then both step down at the start of June. Net units reach 17,089 in early August and end at 11,612.

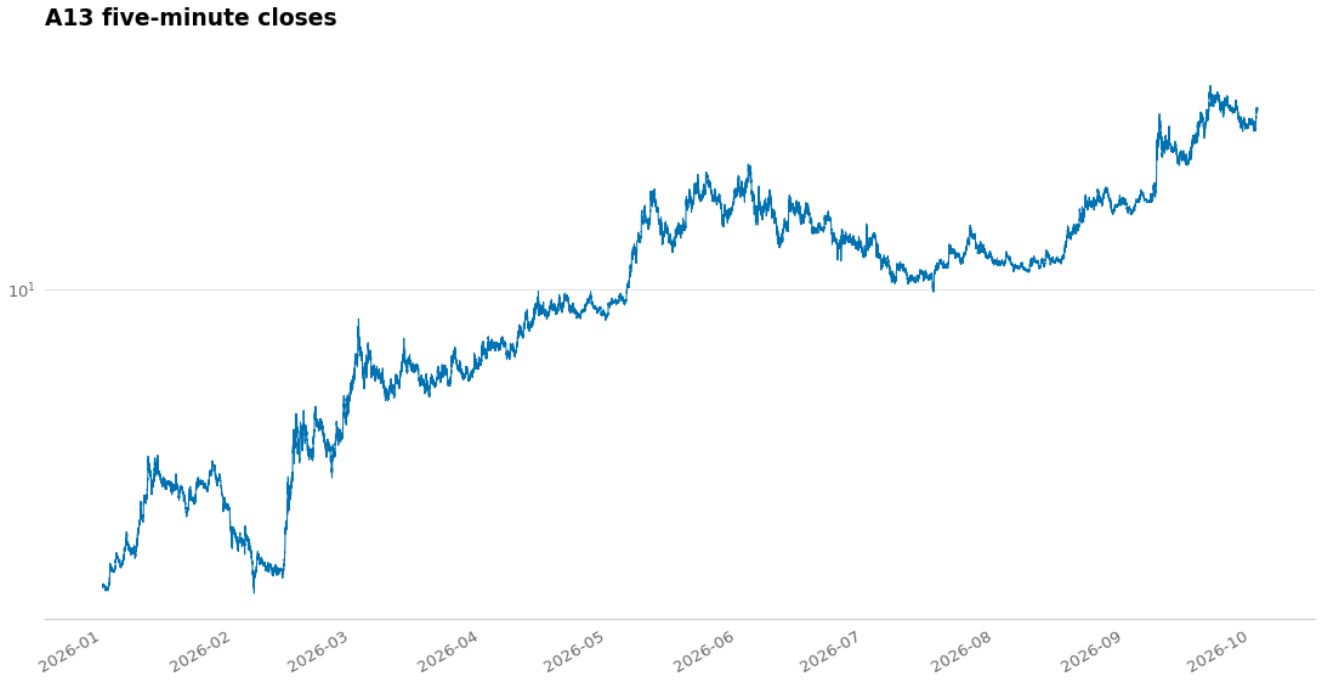
A13



A13, 147 fills, with the sell line above the buy line throughout.



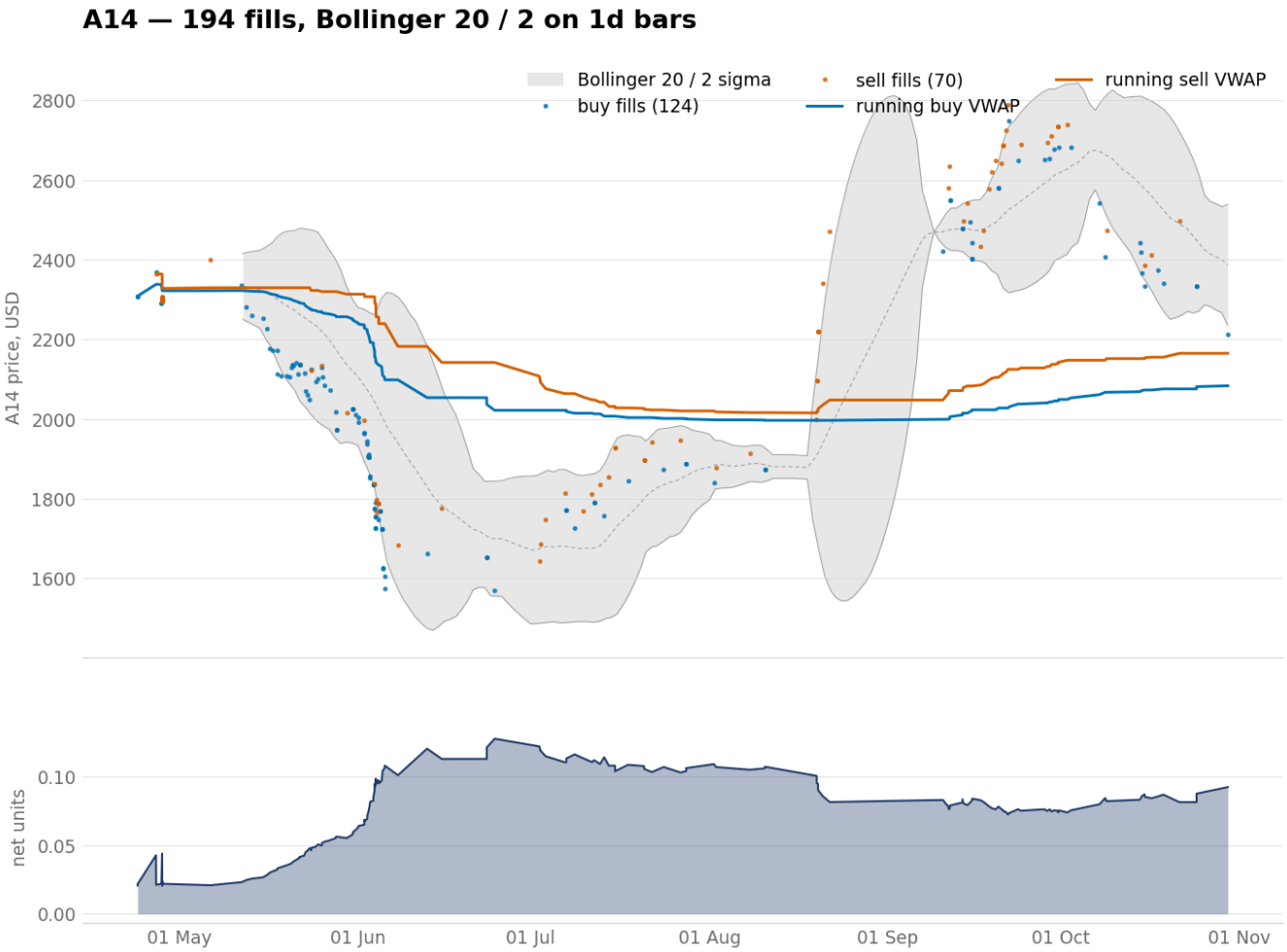
A13 weekly chart, 40 weekly candles built from 76,386 five-minute rows.



A13 price trace, 76,386 five-minute closes across the recorded window.

The sell line stays above the buy line for the whole window, ending near 15.4 against 14.7. Net units peak at 7.13 and end at 4.71.

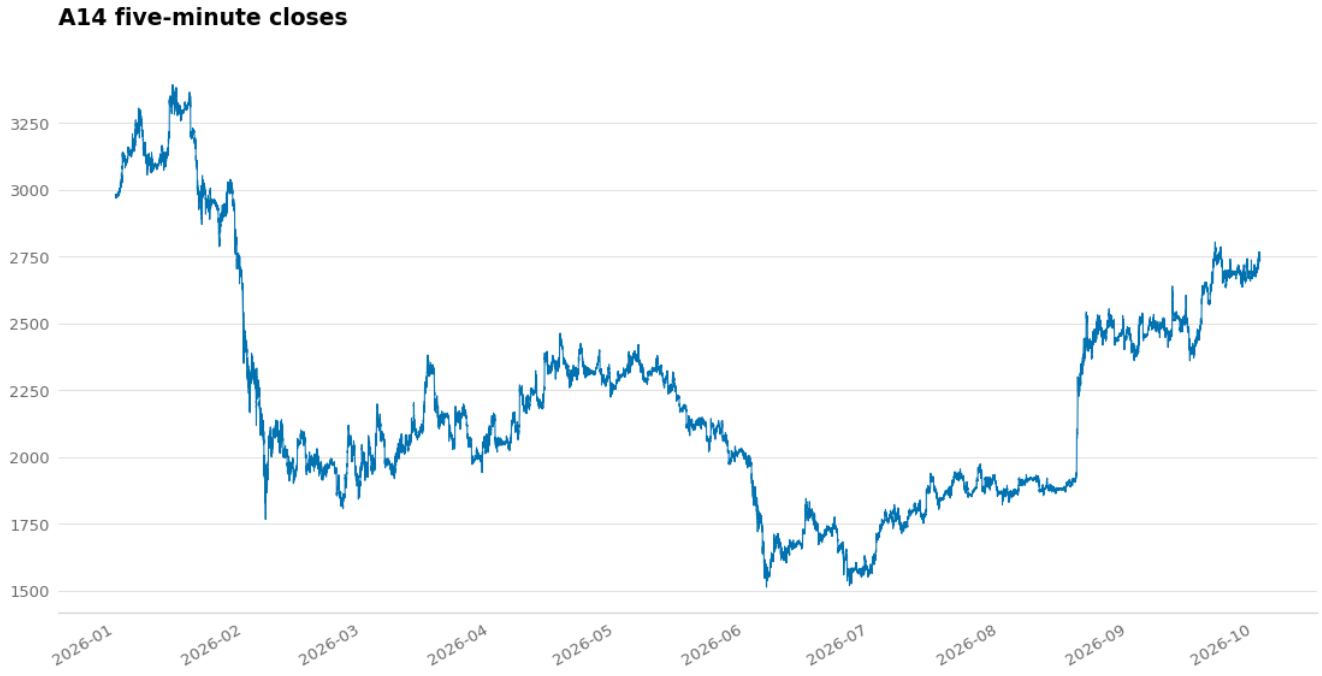
A14



A14, 141 fills, ending 21 August.



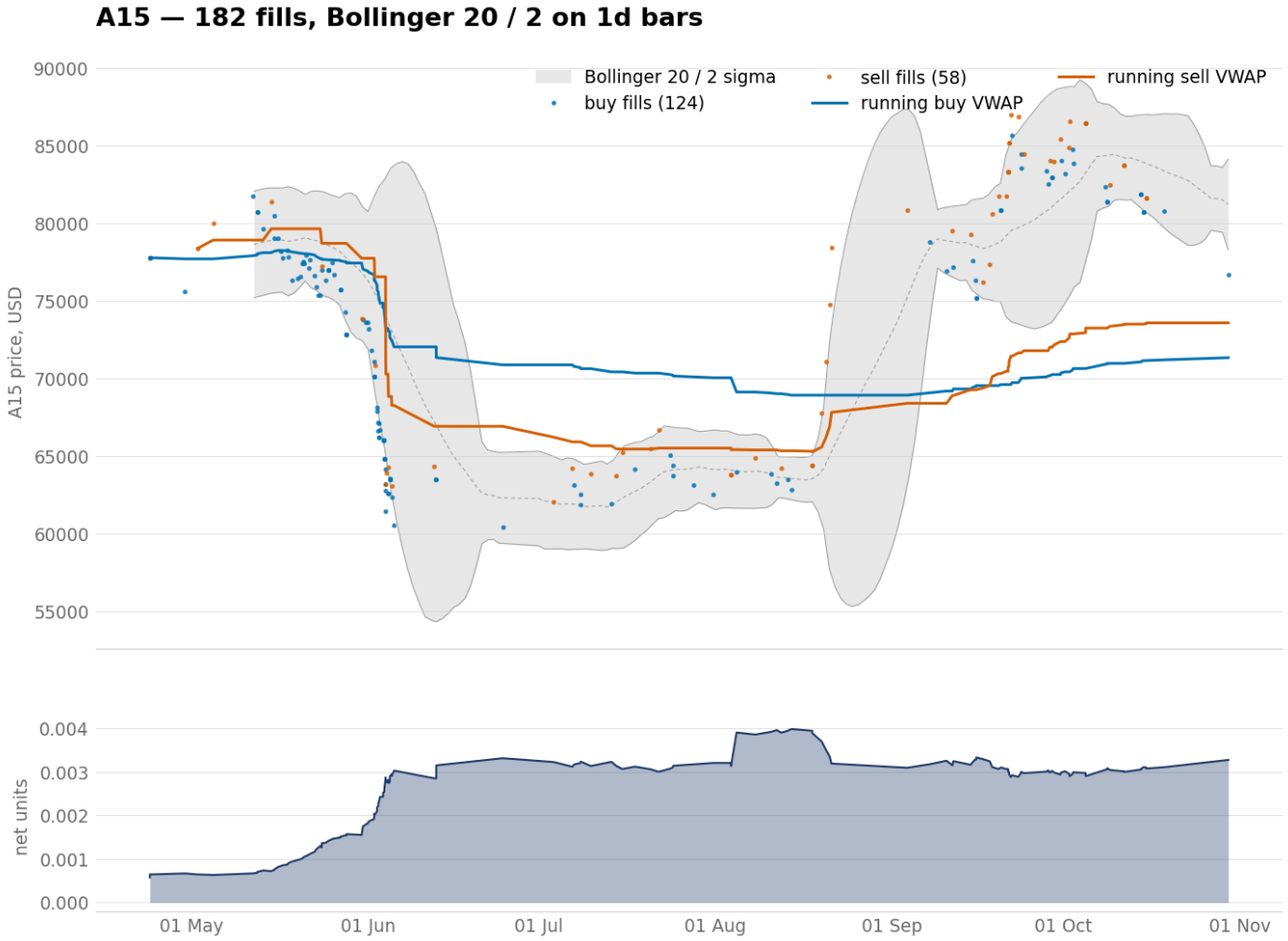
A14 weekly chart, 40 weekly candles built from 79,004 five-minute rows.



A14 price trace, 79,004 five-minute closes across the recorded window.

The window closes on 21 August, the earliest close of the 38 apart from A37. The two lines run together until the start of June, then split, with the sell line about 100 dollars above through June and July. They meet again in August.

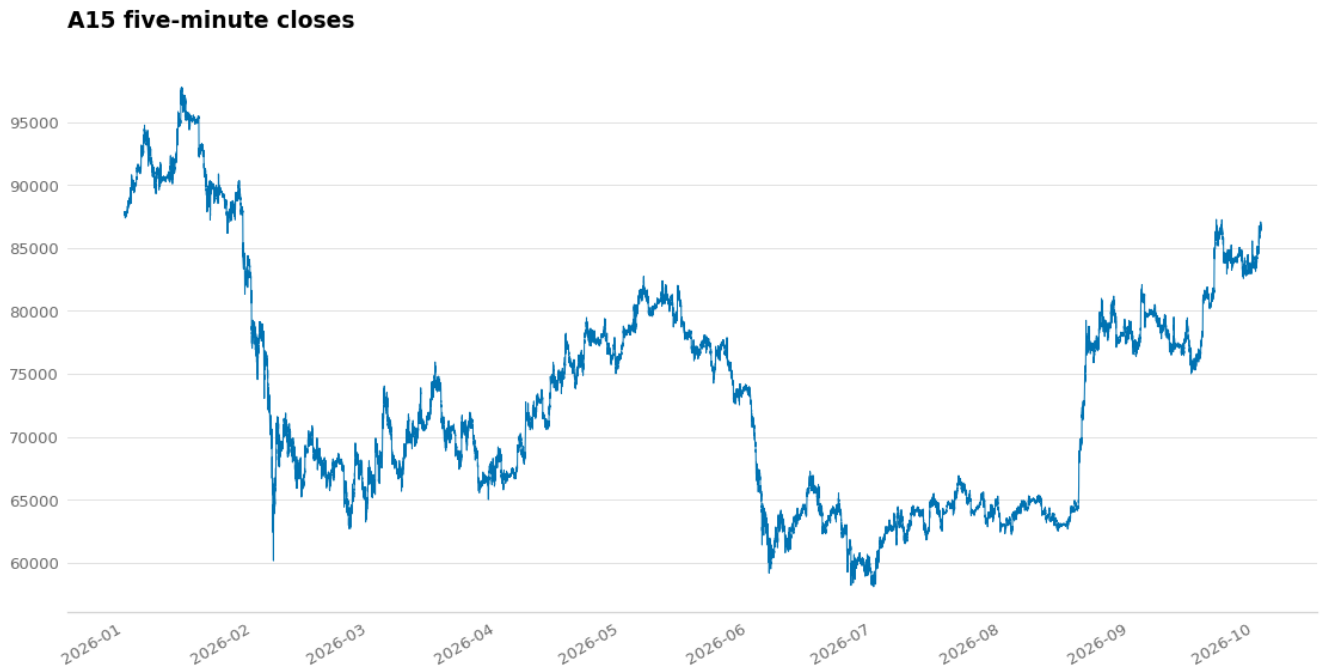
A15



A15, 124 fills, with the two lines splitting at the start of June.



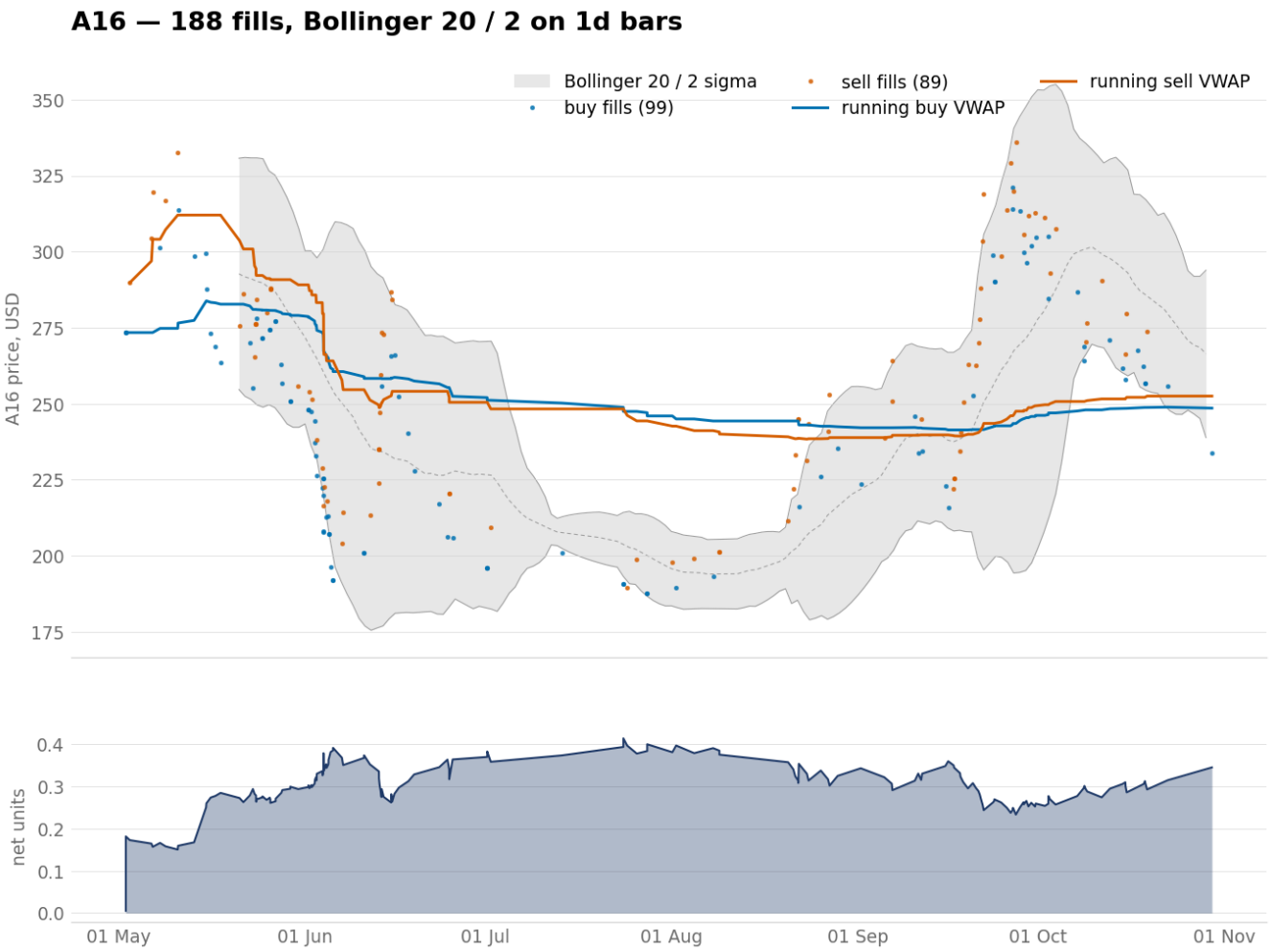
A15 weekly chart, 40 weekly candles built from 79,005 five-minute rows.



A15 price trace, 79,005 five-minute closes across the recorded window.

The highest-priced base in the record. Both lines hold near 78,000 through May. At the start of June the sell line drops to about 68,000 and the buy line to about 72,000, and the gap holds until late August. Net units end at 0.0031.

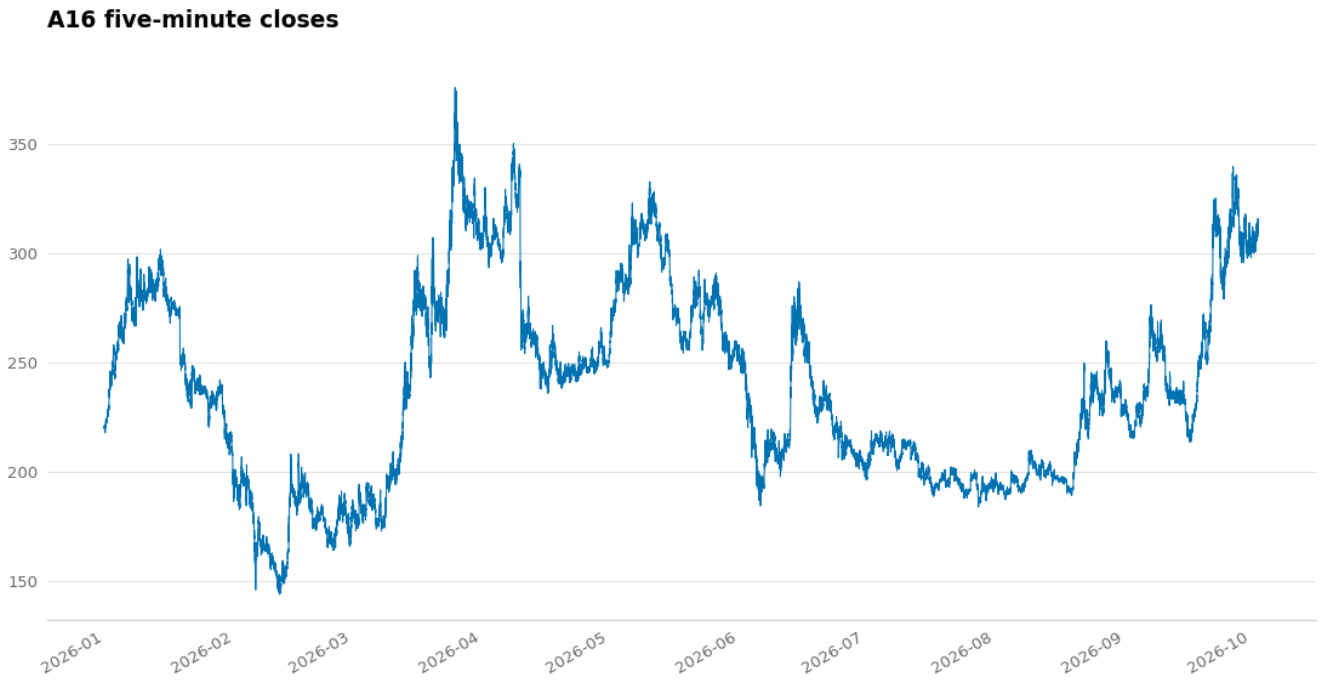
A16



A16, 124 fills, with the sell line above the buy line until mid-June.



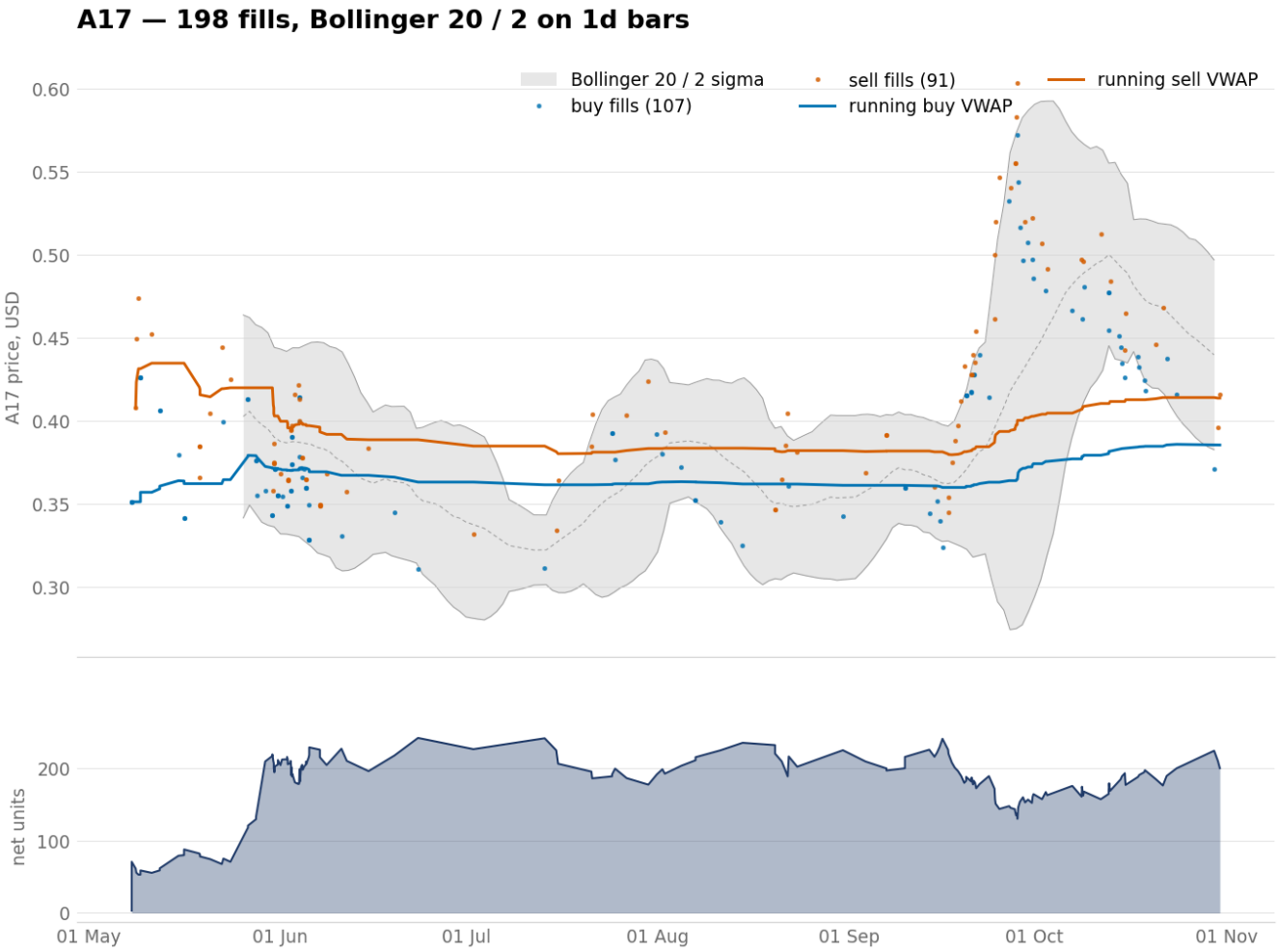
A16 weekly chart, 40 weekly candles built from 78,992 five-minute rows.



A16 price trace, 78,992 five-minute closes across the recorded window.

The sell line peaks near 312 in mid-May, about 30 dollars above the buy line, and crosses below it in mid-June. Net units end at 0.34.

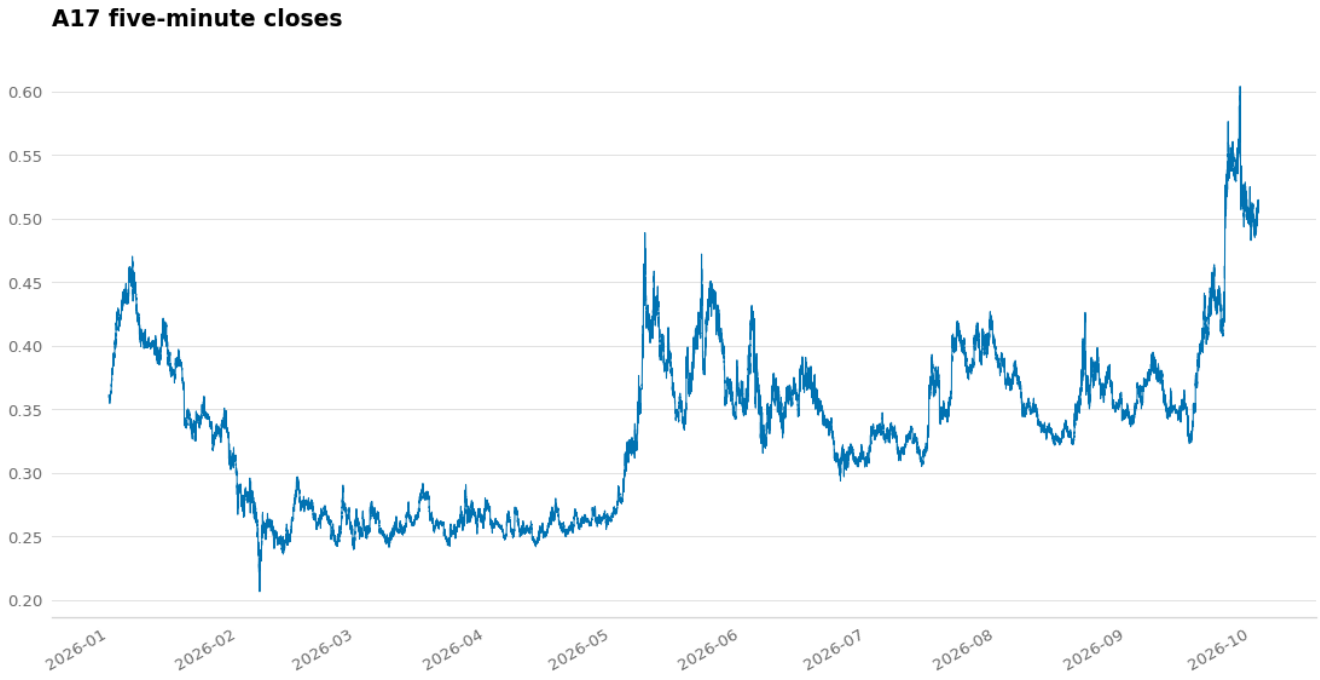
A17



A17, 119 fills, with both lines nearly flat after June.



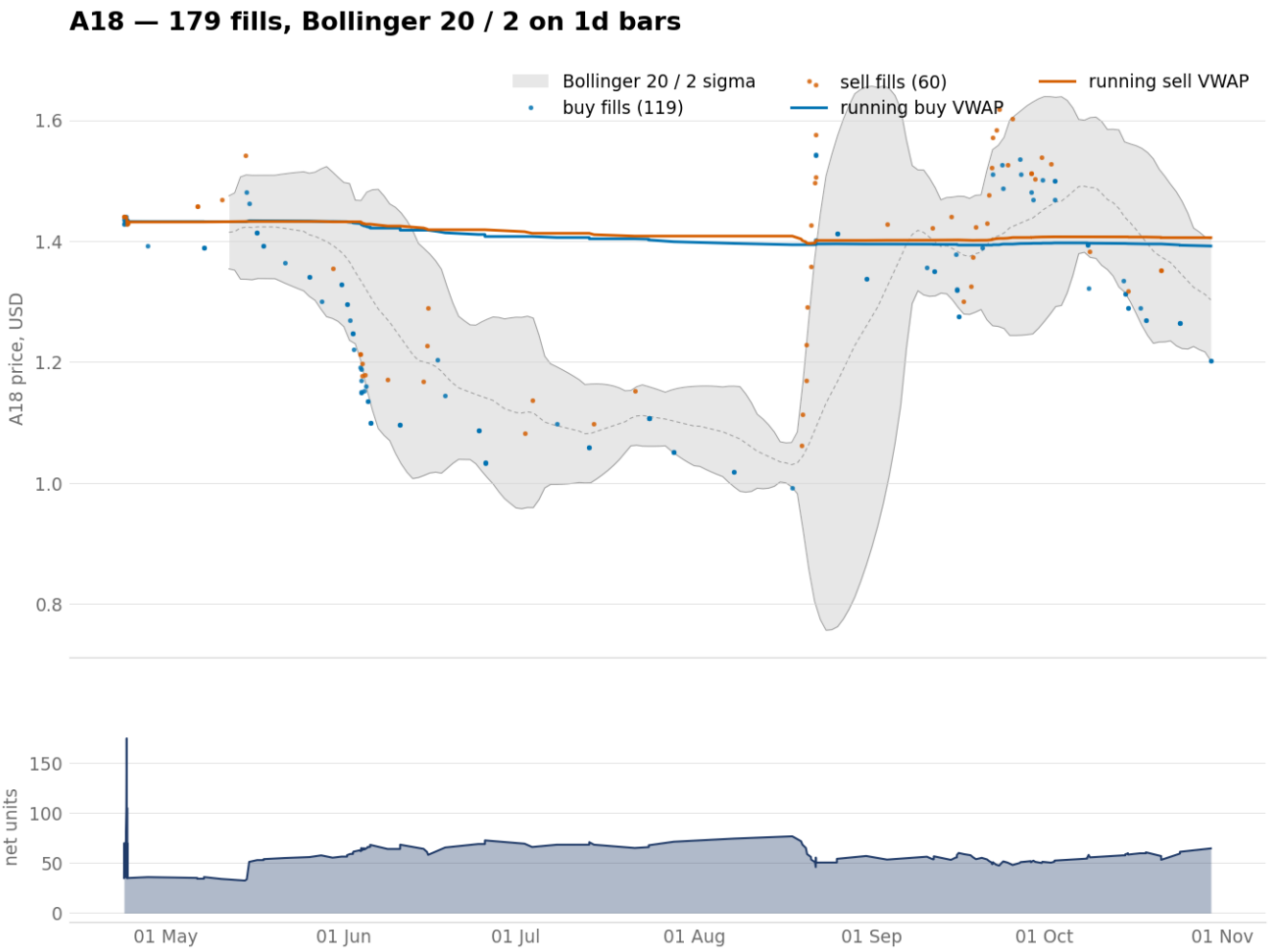
A17 weekly chart, 40 weekly candles built from 78,992 five-minute rows.



A17 price trace, 78,992 five-minute closes across the recorded window.

The buy line sits near 0.362 and the sell line near 0.382 from mid-June to the close, a gap of about 0.02 held across eleven weeks.

A18



A18, 119 fills, with the two lines almost touching across the window.



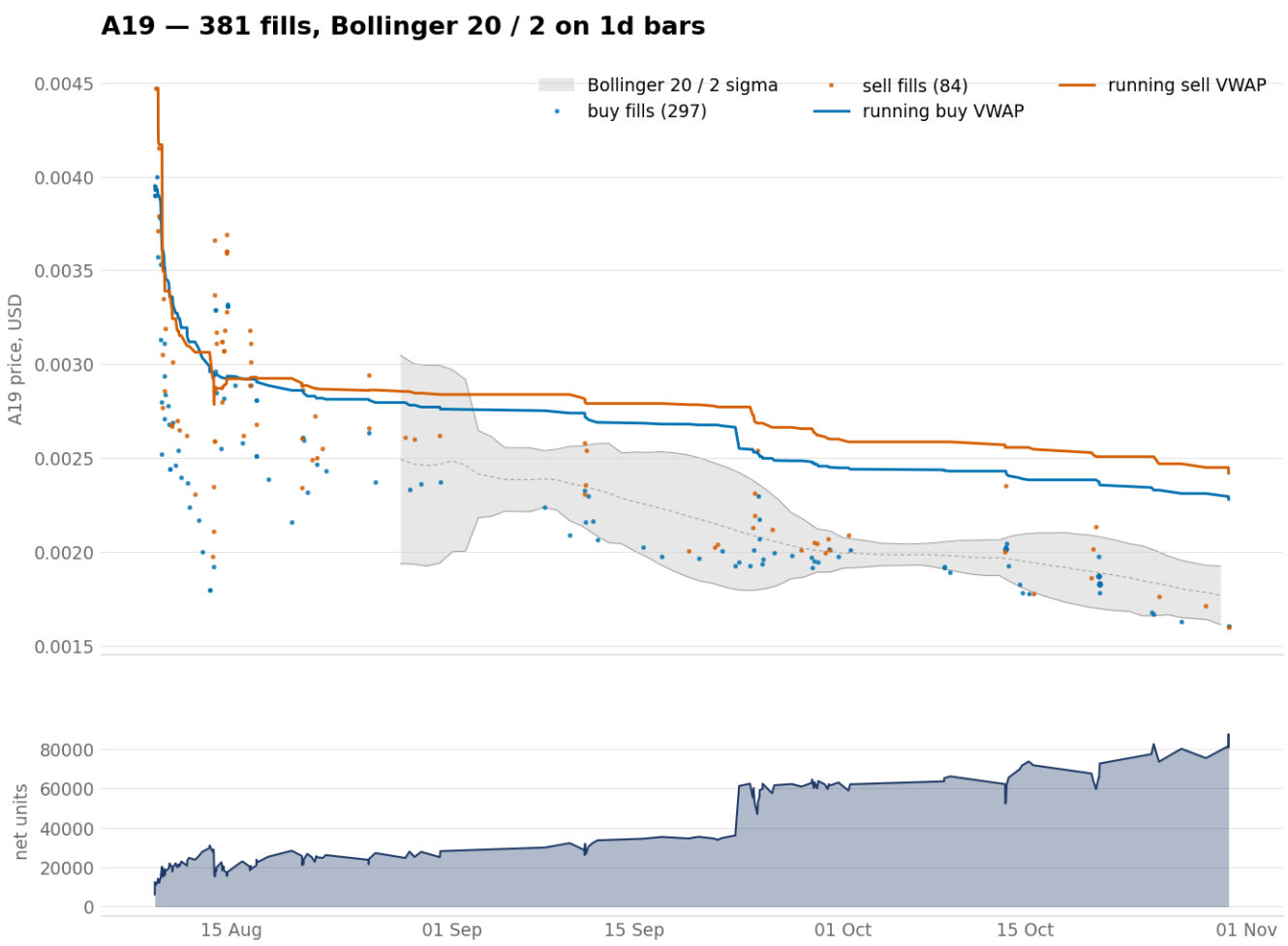
A18 weekly chart, 40 weekly candles built from 79,004 five-minute rows.



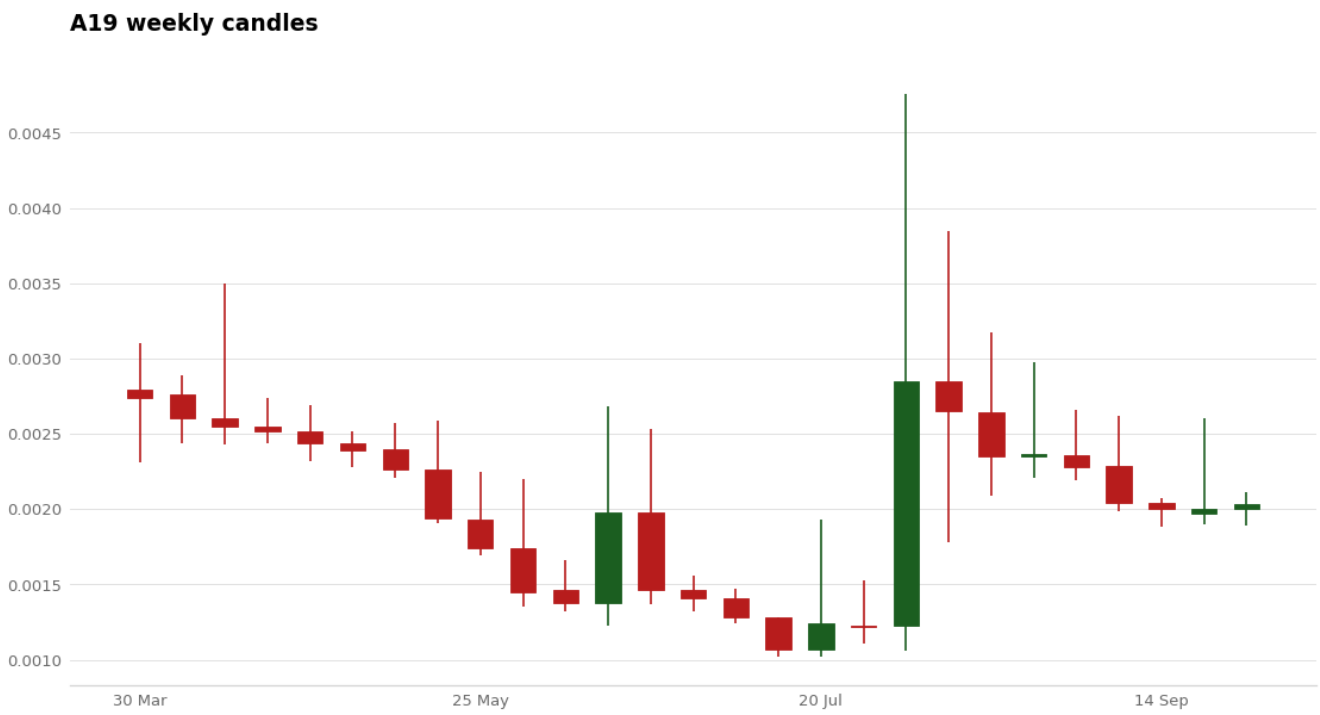
A18 price trace, 79,004 five-minute closes across the recorded window.

The two lines run within about 0.01 of each other for the whole window. Net units spike to 175 at the start, then hold near 55.

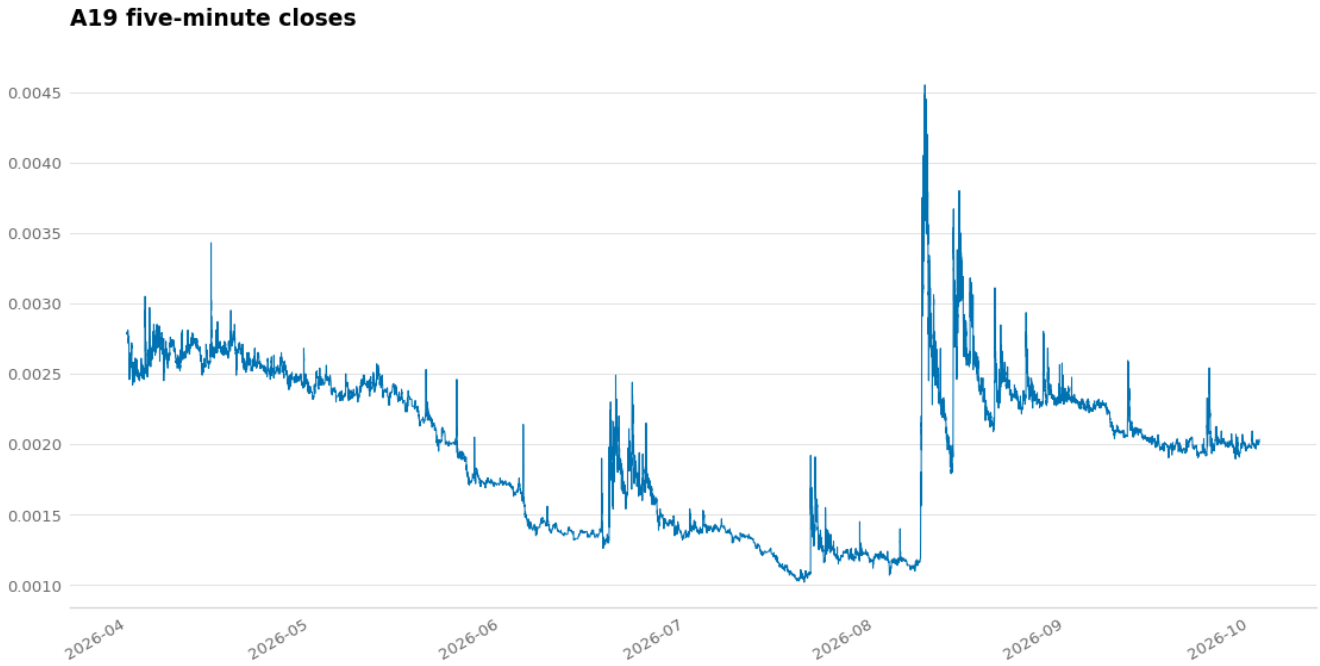
A19



A19, 113 fills across 22 days in August.



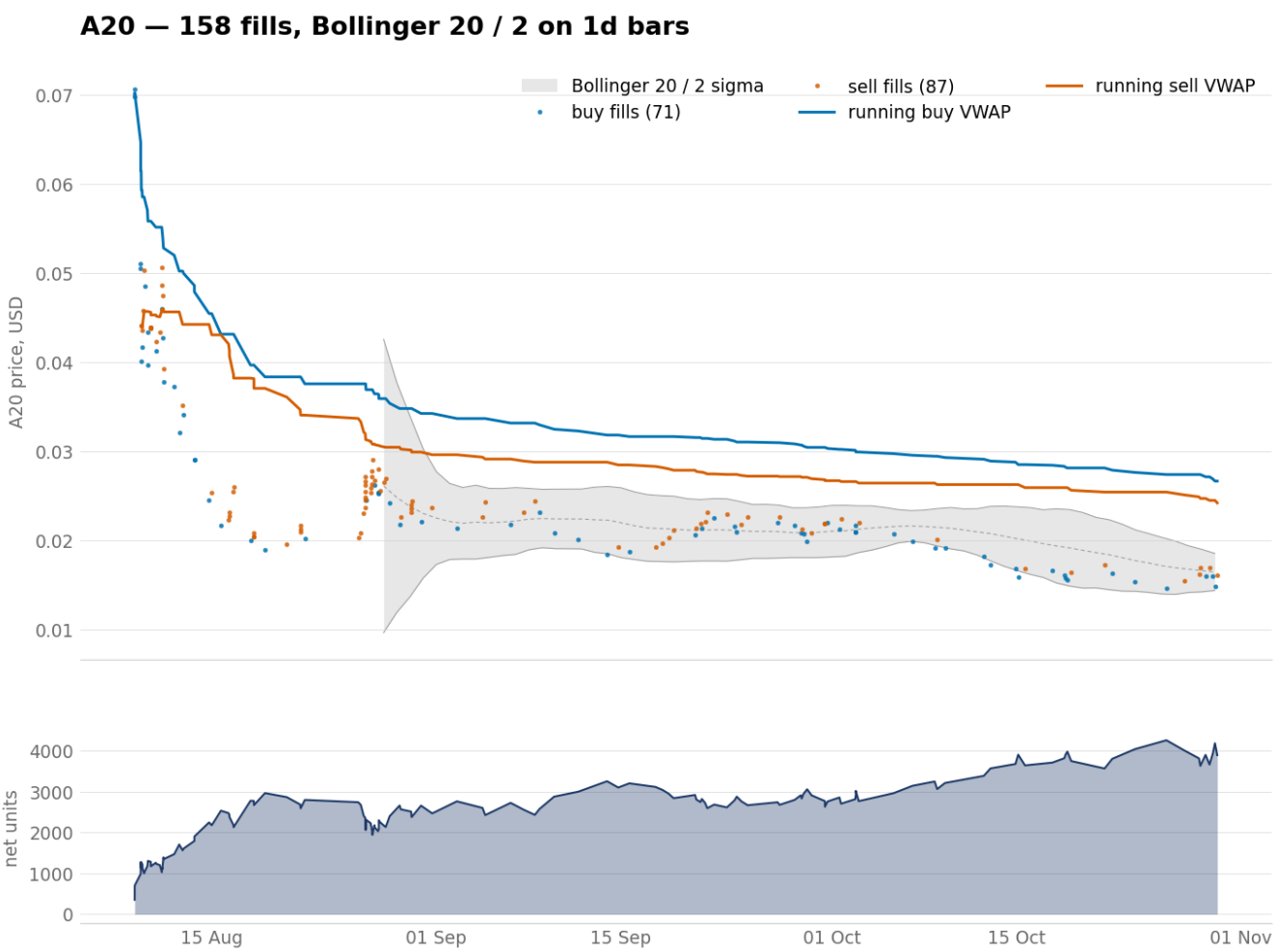
A19 weekly chart, 27 weekly candles built from 15,736 five-minute rows.



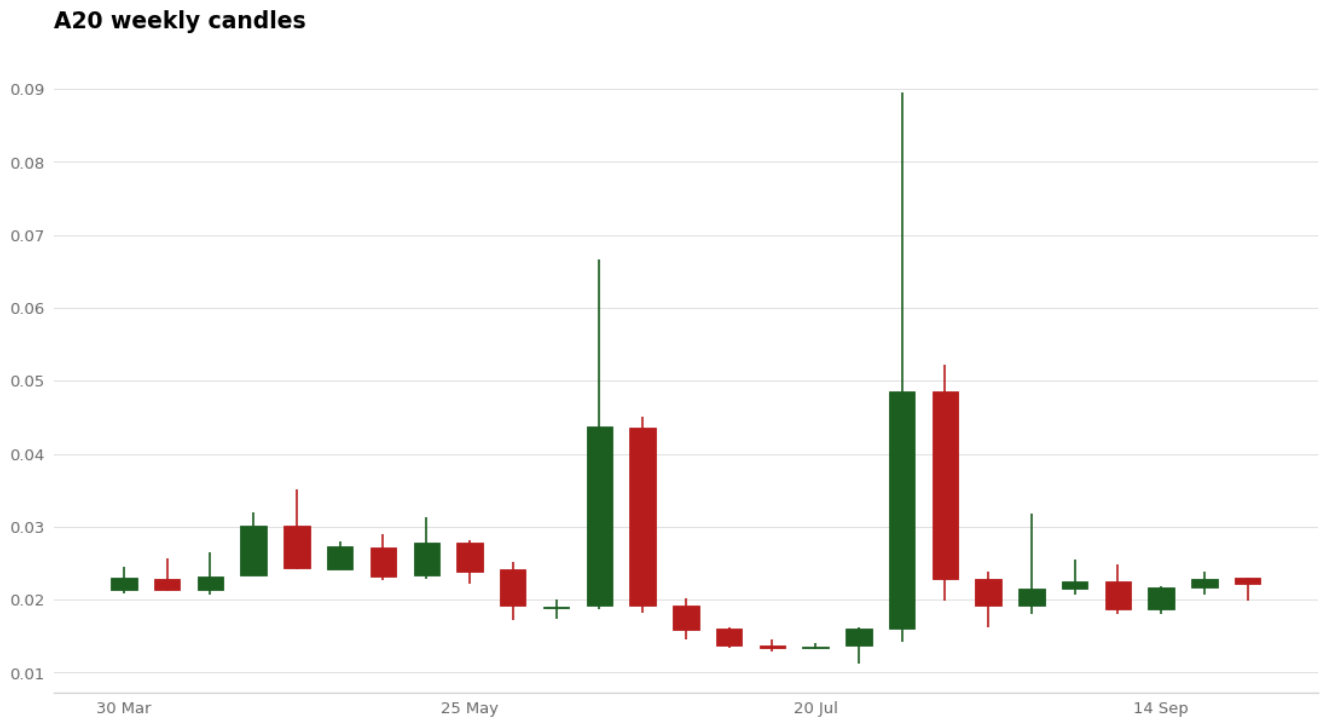
A19 price trace, 15,736 five-minute closes across the recorded window.

The second-shortest window of the 38. The buy line falls from 0.00395 to about 0.0030 in the first four days, then flattens. The two lines nearly overlap after 17 August.

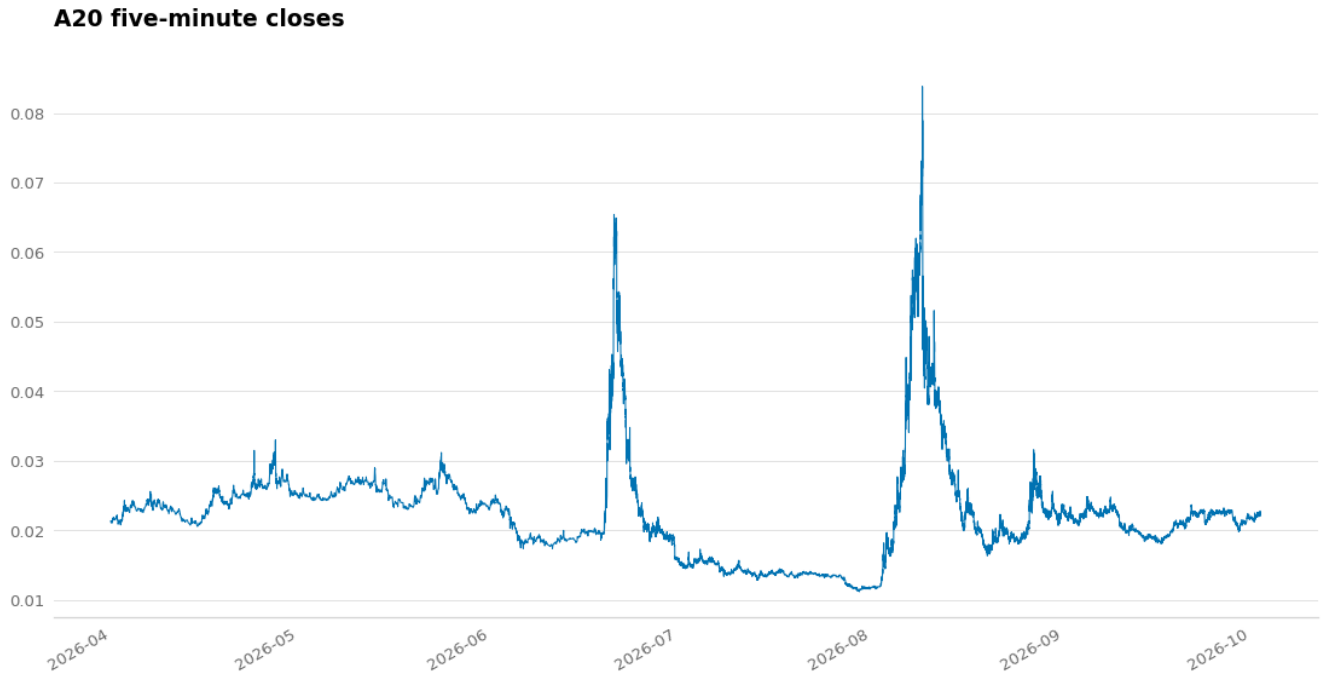
A20



A20, 87 fills, with the sell line crossing below the buy line in mid-August.



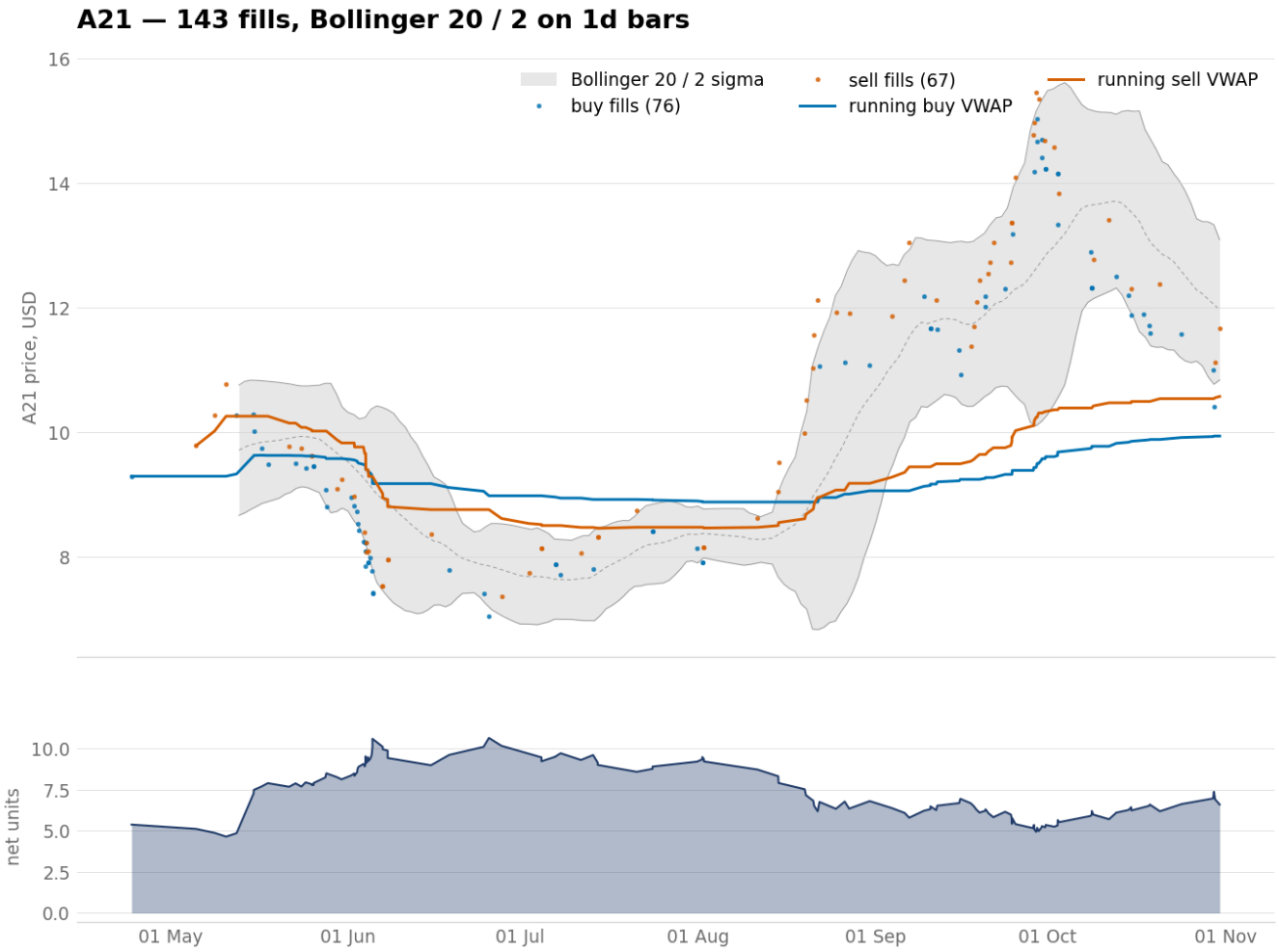
A20 weekly chart, 27 weekly candles built from 21,070 five-minute rows.



A20 price trace, 21,070 five-minute closes across the recorded window.

Buys number 32 against 55 sells. The buy line starts at 0.0698 and falls to 0.0337 across 24 days, the steepest fall of the three assets that start in August.

A21



A21, 83 fills, with both lines turning up in the last two weeks.



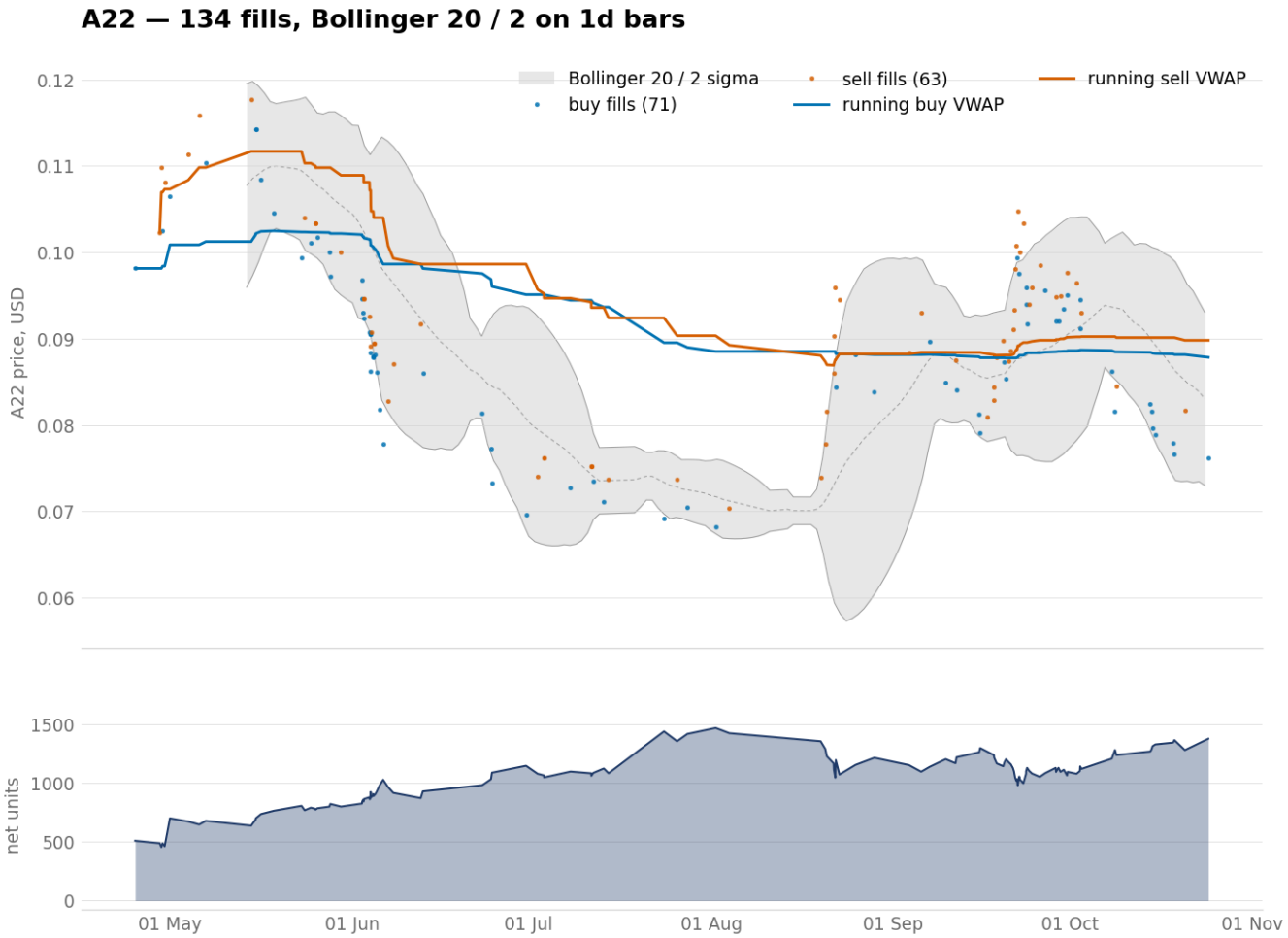
A21 weekly chart, 40 weekly candles built from 79,003 five-minute rows.



A21 price trace, 79,003 five-minute closes across the recorded window.

The sell line falls below the buy line at the start of June and stays there until late August, when both turn up and cross again.

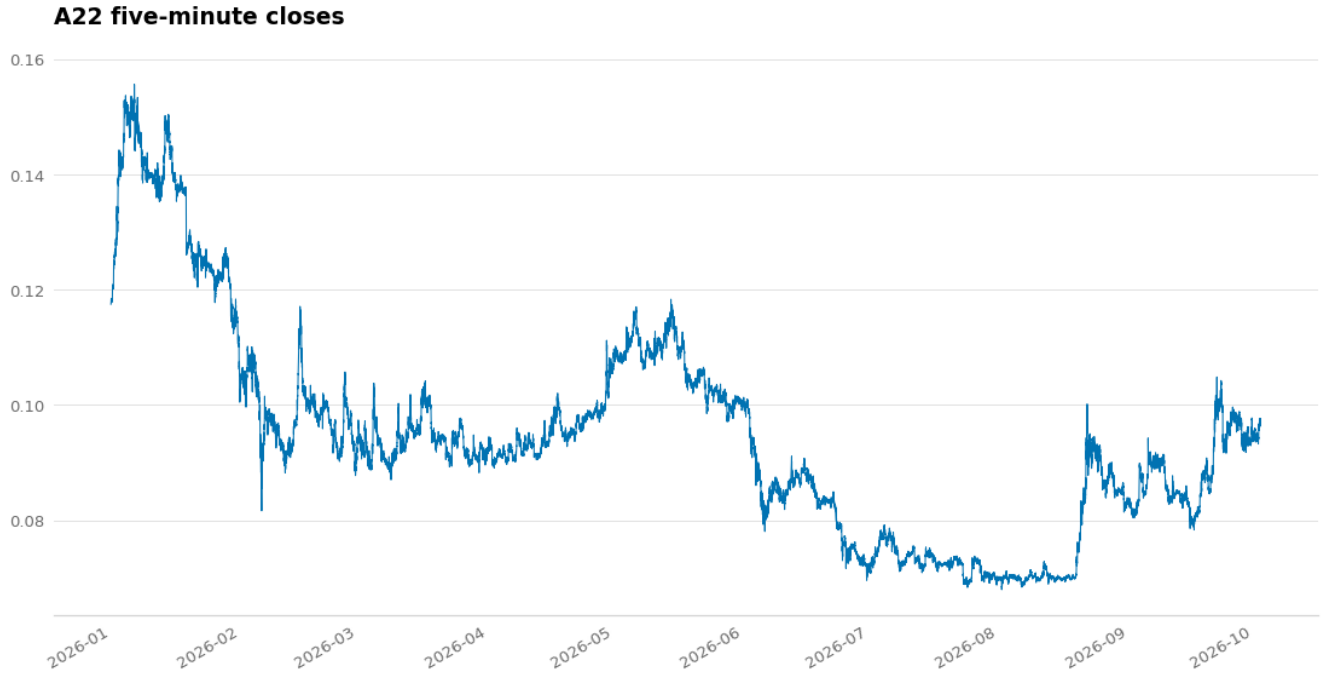
A22



A22, 80 fills, with the two lines meeting in mid-August.



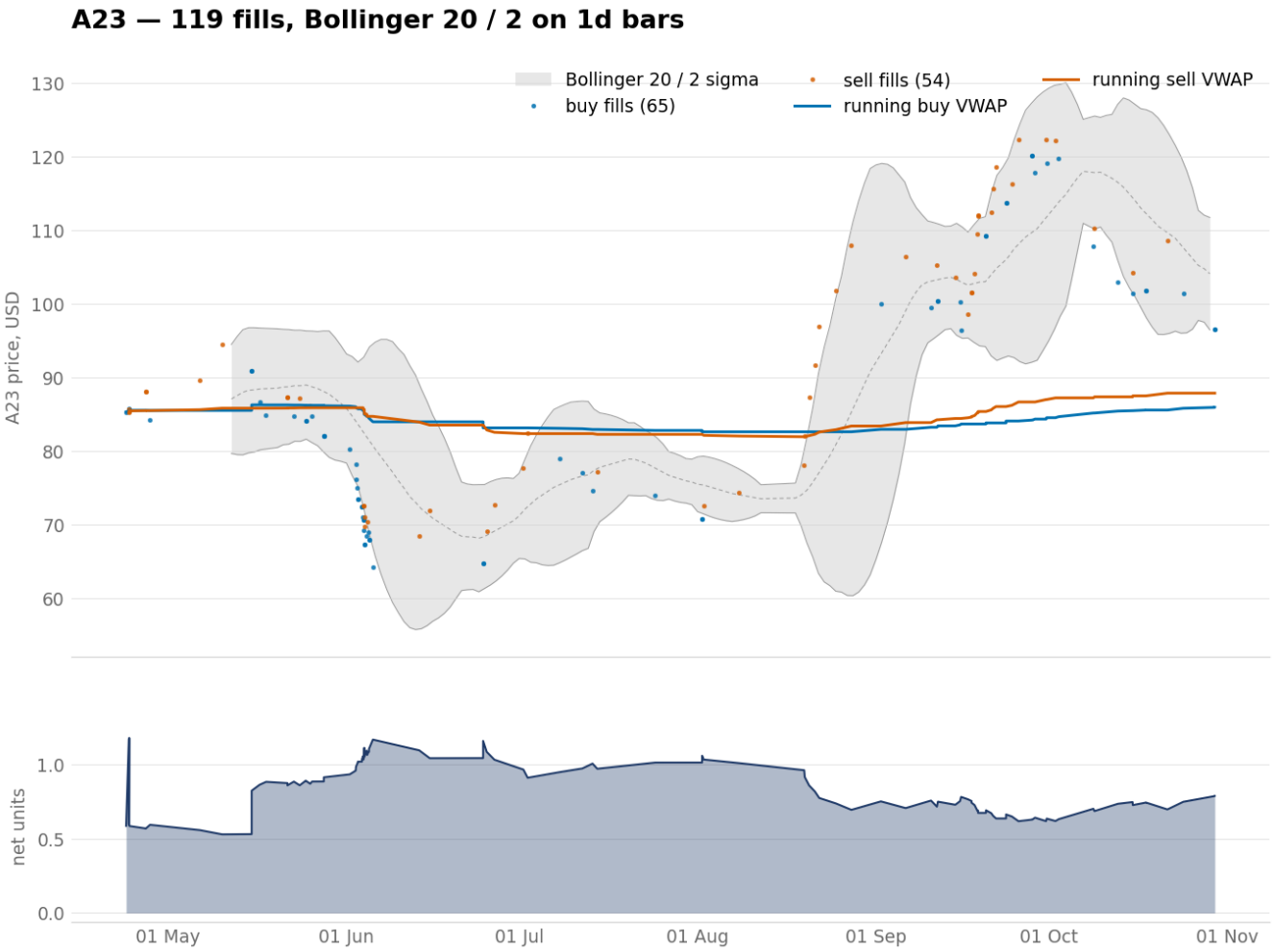
A22 weekly chart, 40 weekly candles built from 79,000 five-minute rows.



A22 price trace, 79,000 five-minute closes across the recorded window.

The sell line runs above the buy line from May to mid-August, then the two converge near 0.088 and finish together. Net units end at 1,153.

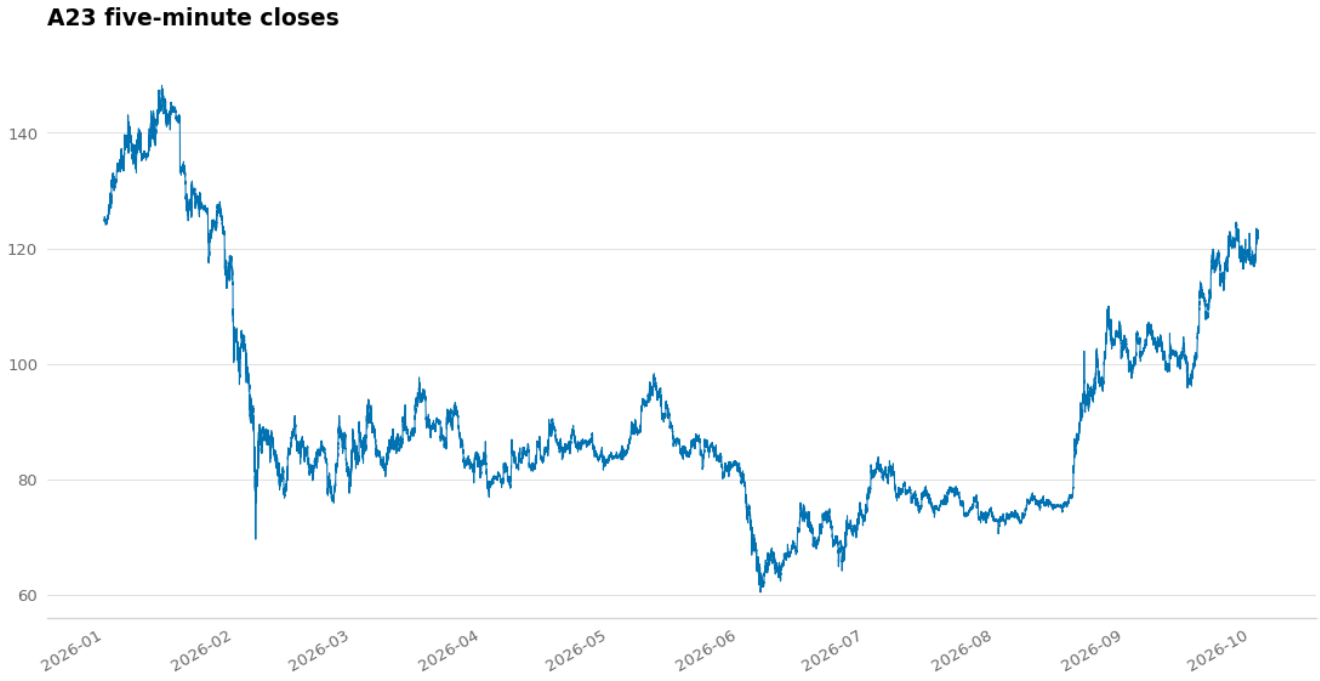
A23



A23, 77 fills, with the two lines within about a dollar of each other.



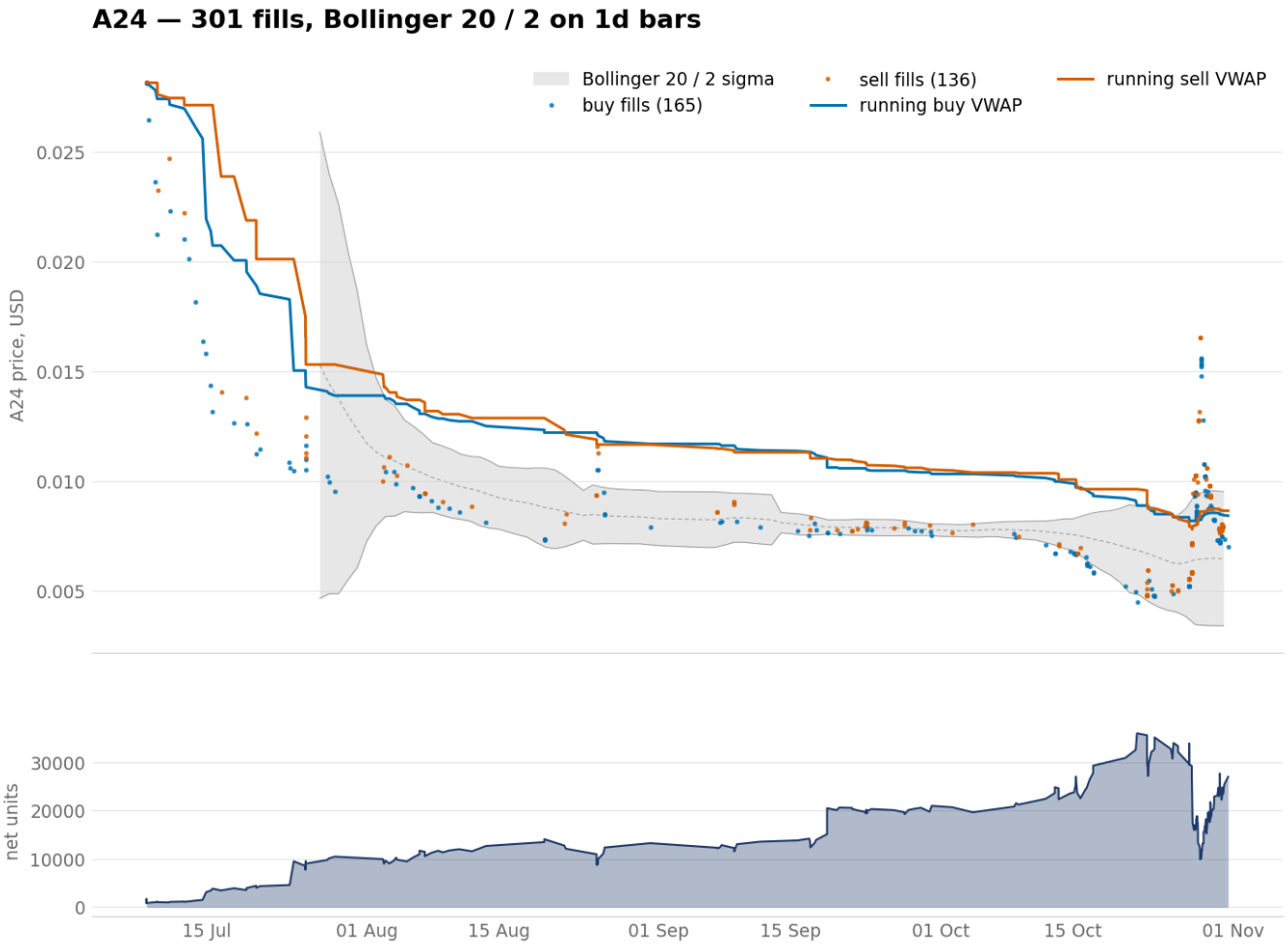
A23 weekly chart, 40 weekly candles built from 79,004 five-minute rows.



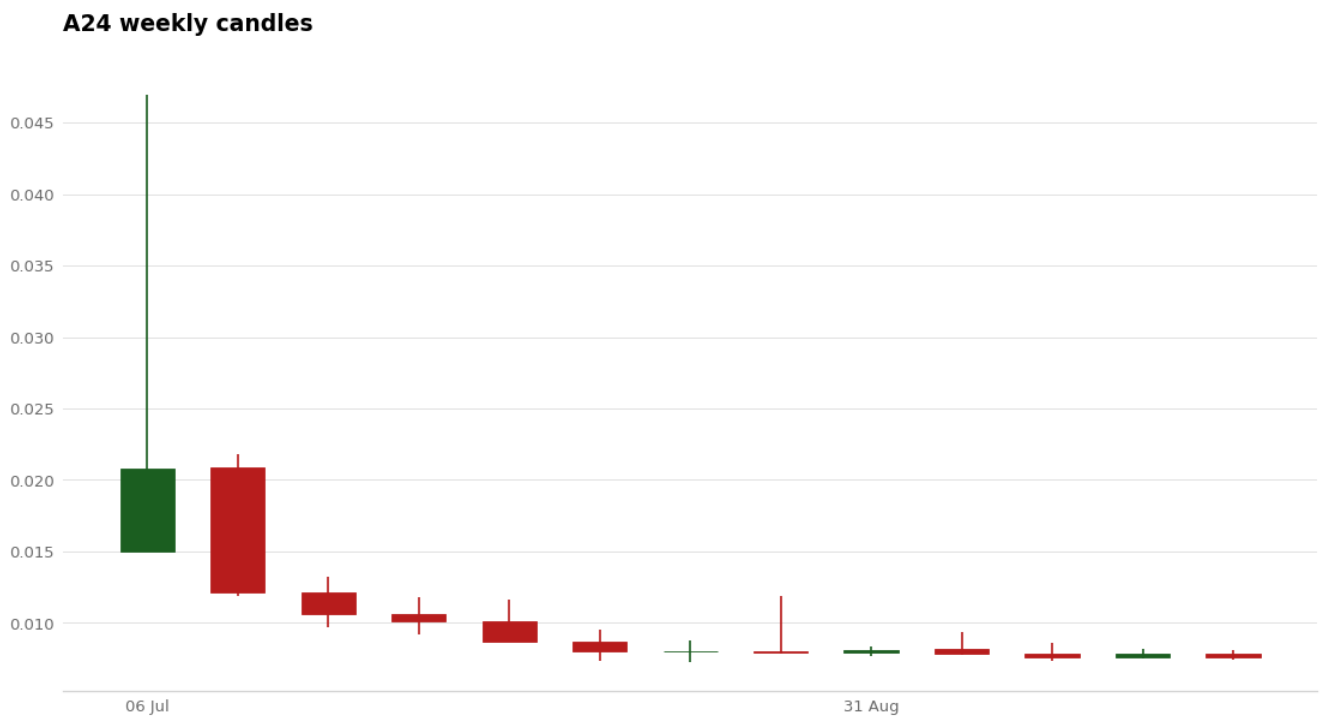
A23 price trace, 79,004 five-minute closes across the recorded window.

Both lines hold near 85 through May, step to about 83 at the start of June, and end within a dollar of each other. Net units end below one whole unit, at 0.75.

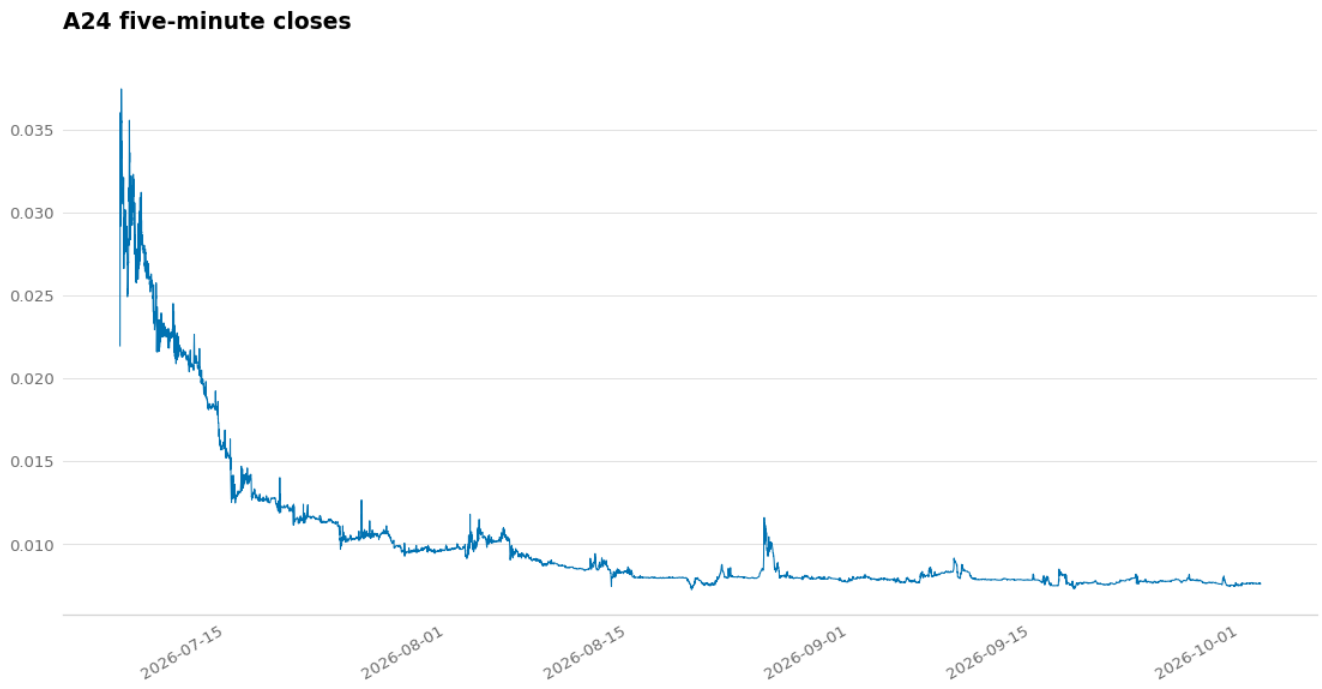
A24



A24, 75 fills, with both lines falling by more than half.



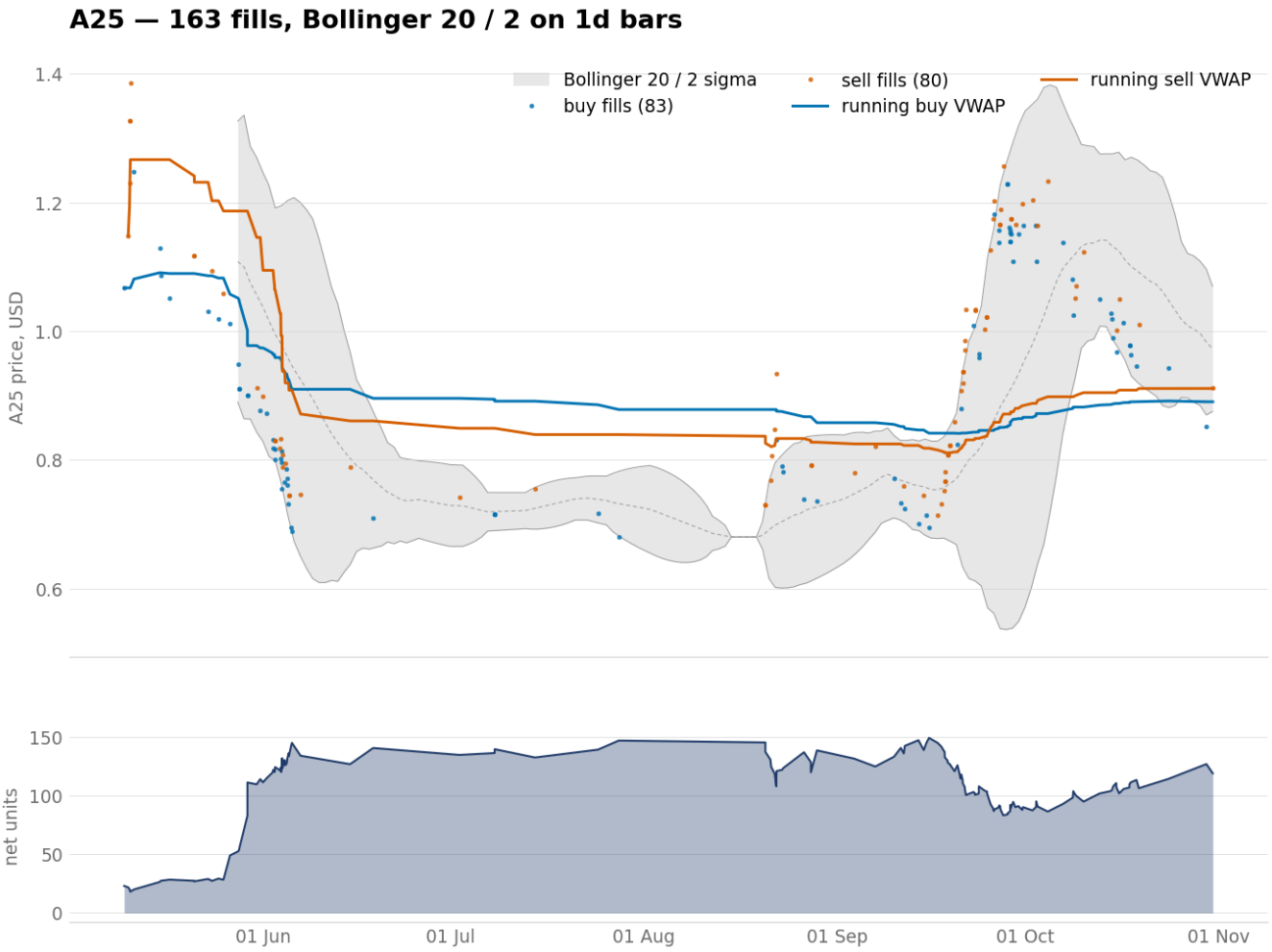
A24 weekly chart, 13 weekly candles built from 8,734 five-minute rows.



A24 price trace, 8,734 five-minute closes across the recorded window.

A July starter. The buy line falls from 0.02809 to 0.01168 in eight weeks, and the sell line converges onto it by the close. Net units end at 13,268.

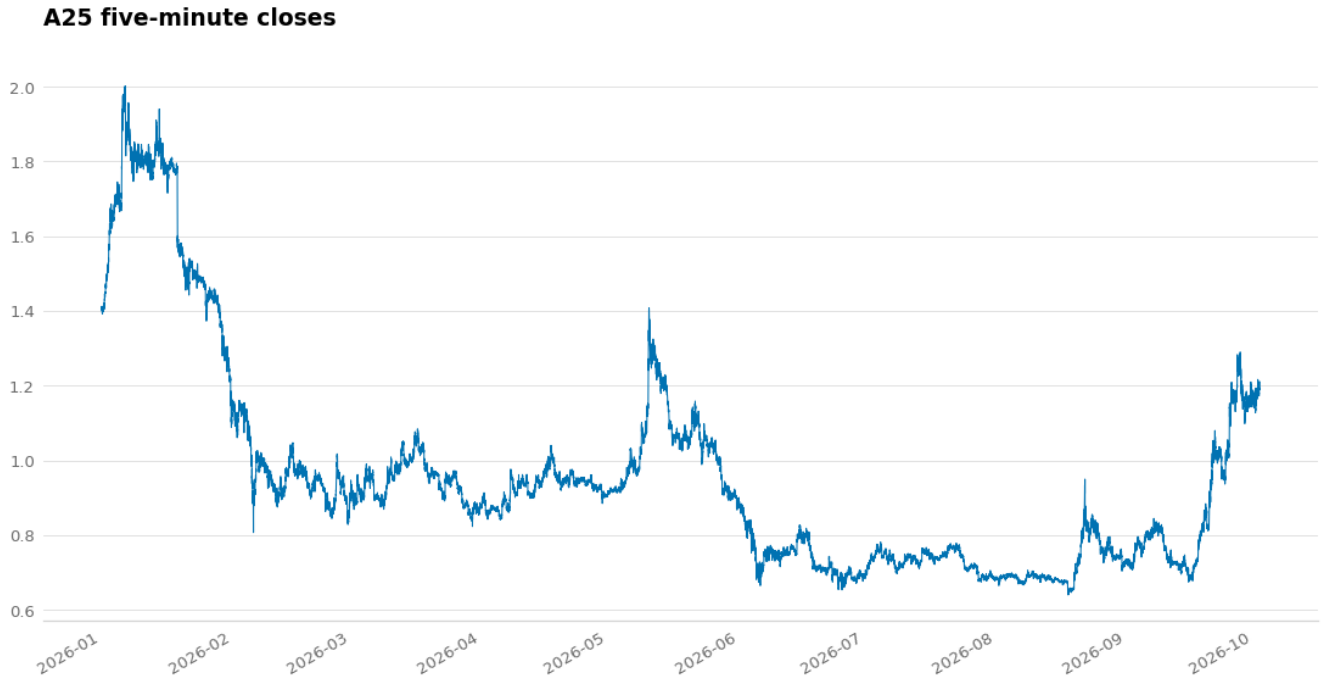
A25



A25, 74 fills, with a step down at the start of June.



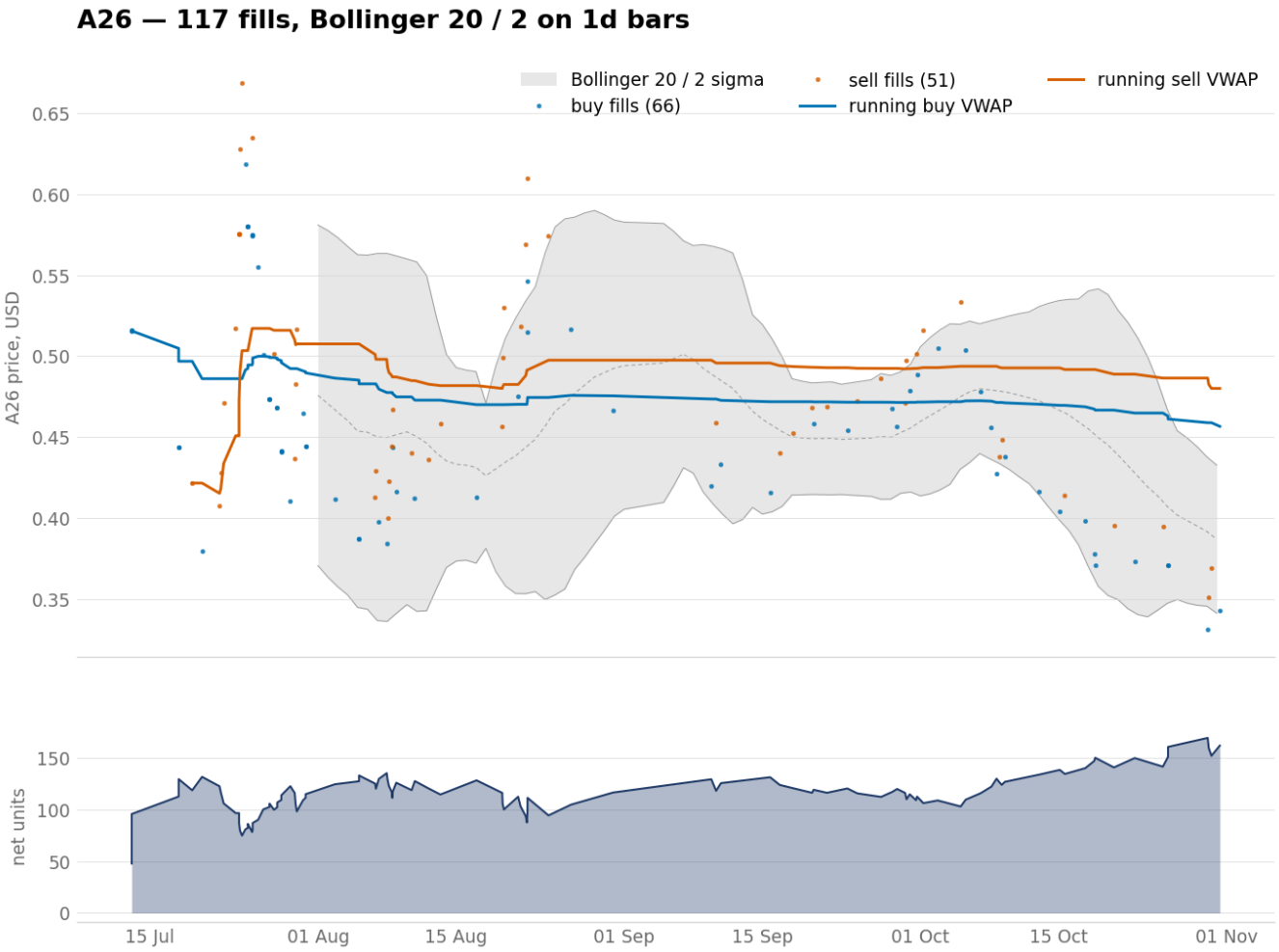
A25 weekly chart, 40 weekly candles built from 78,996 five-minute rows.



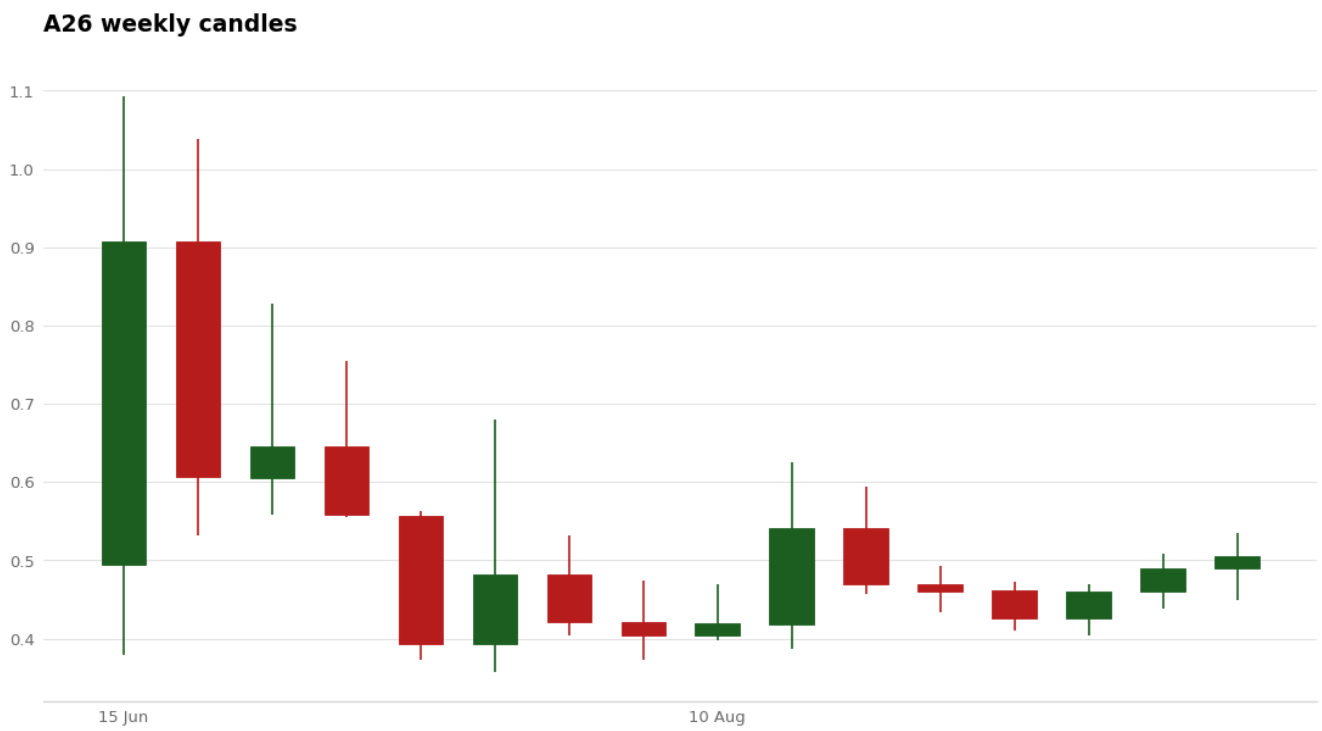
A25 price trace, 78,996 five-minute closes across the recorded window.

The sell line falls from 1.27 to 0.87 across late May and early June, crossing below the buy line, and the two hold about 0.04 apart afterwards.

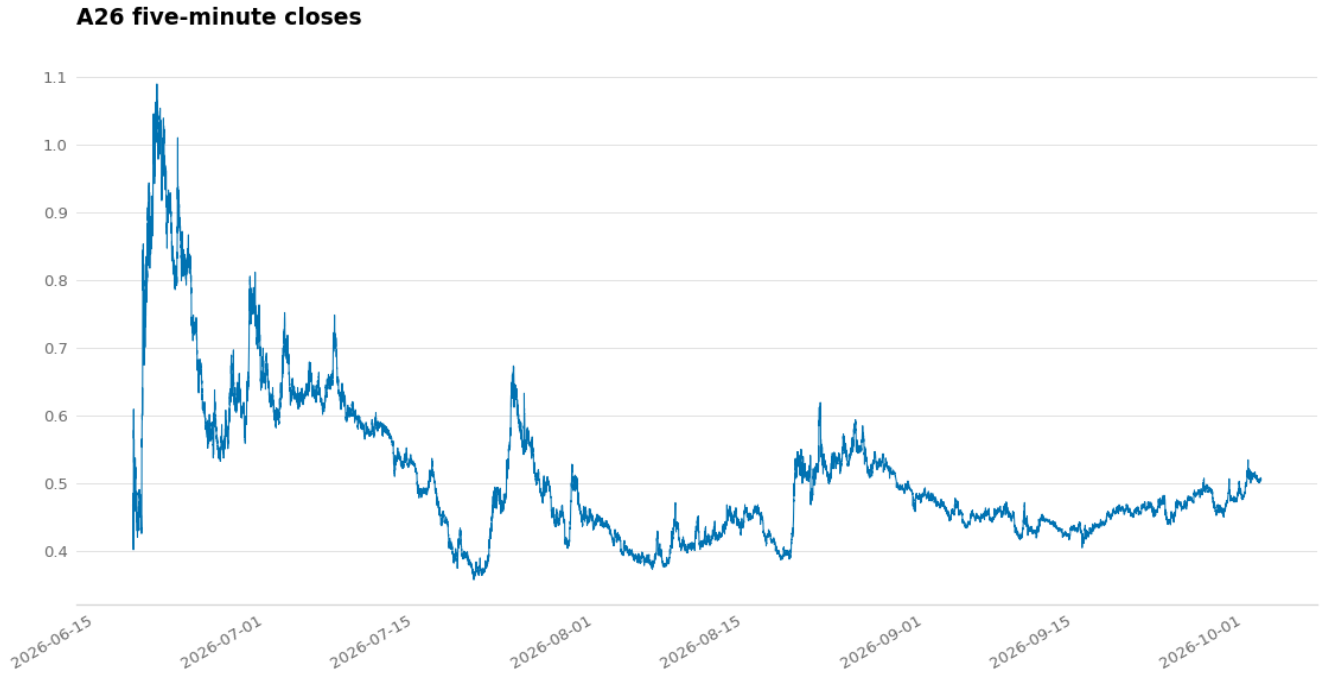
A26



A26, 73 fills, with the sell line above the buy line from late July.



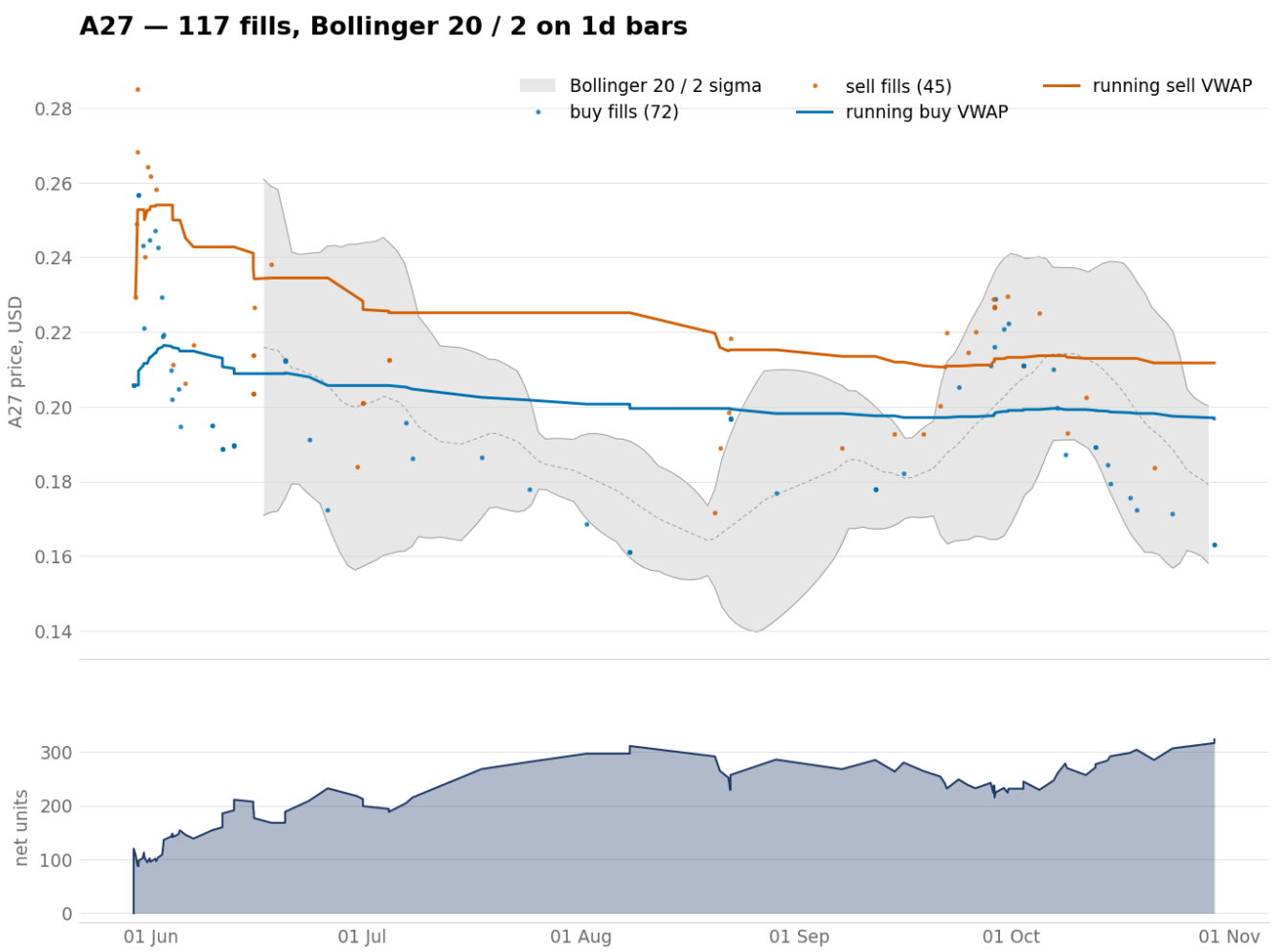
A26 weekly chart, 16 weekly candles built from 25,946 five-minute rows.



A26 price trace, 25,946 five-minute closes across the recorded window.

The sell line jumps from 0.42 to 0.52 in the last week of July, then settles near 0.50 against a buy line near 0.476.

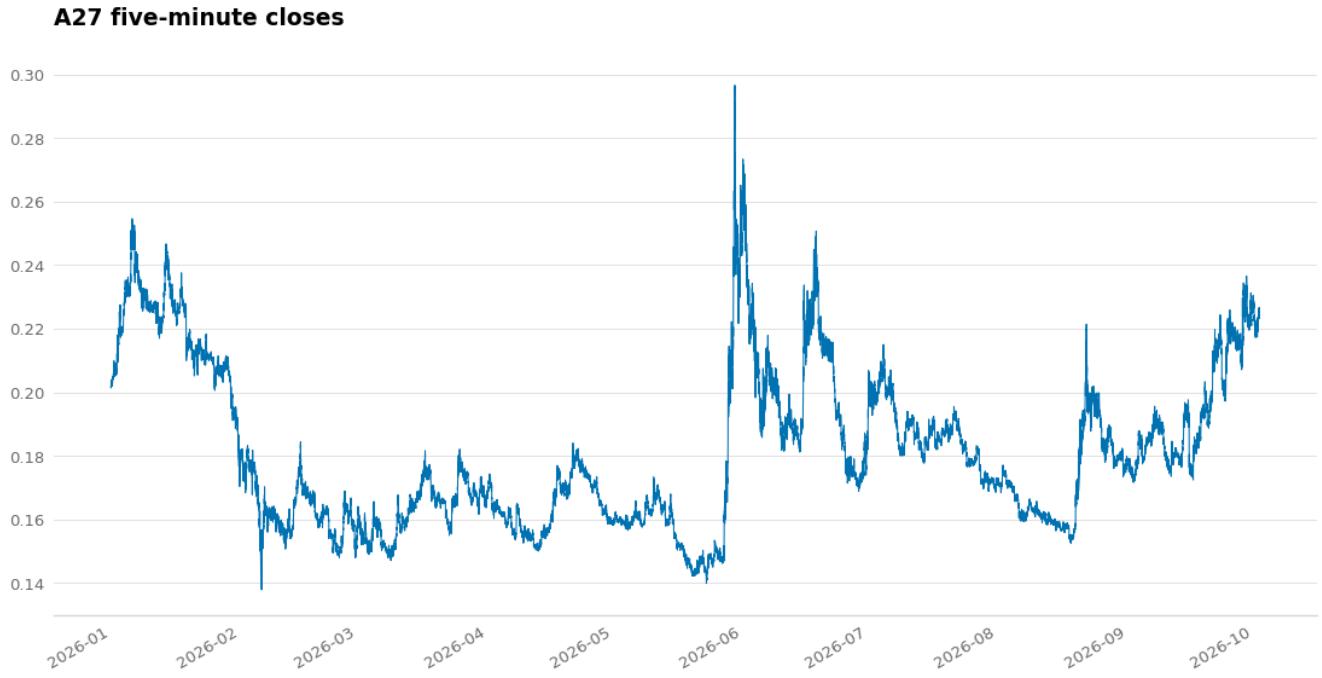
A27



A27, 73 fills, with the sell line about 0.02 above the buy line throughout.



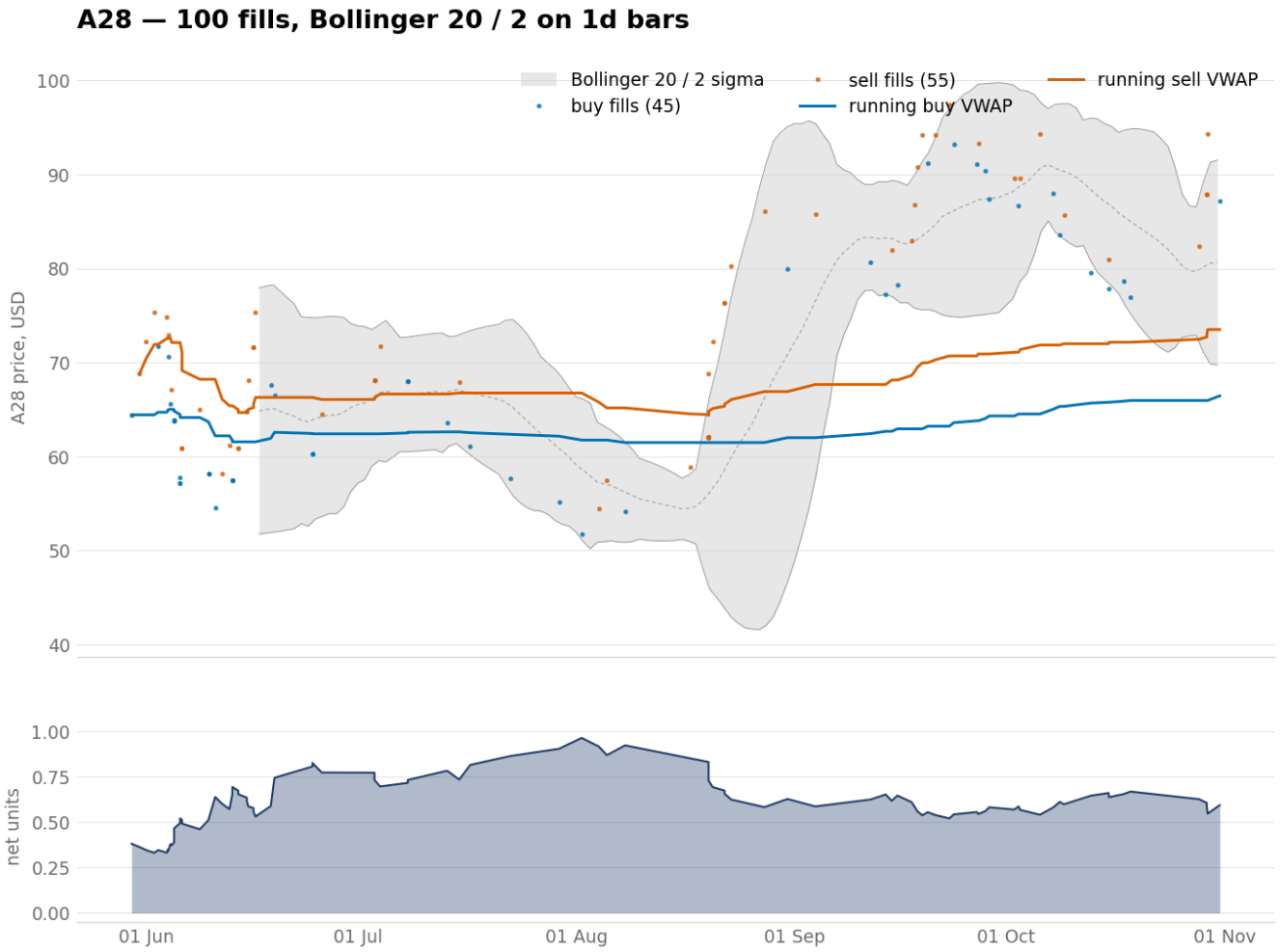
A27 weekly chart, 40 weekly candles built from 79,003 five-minute rows.



A27 price trace, 79,003 five-minute closes across the recorded window.

The gap between the two lines holds across the whole window, and both drift down. Net units grow to 311 and end at 285.

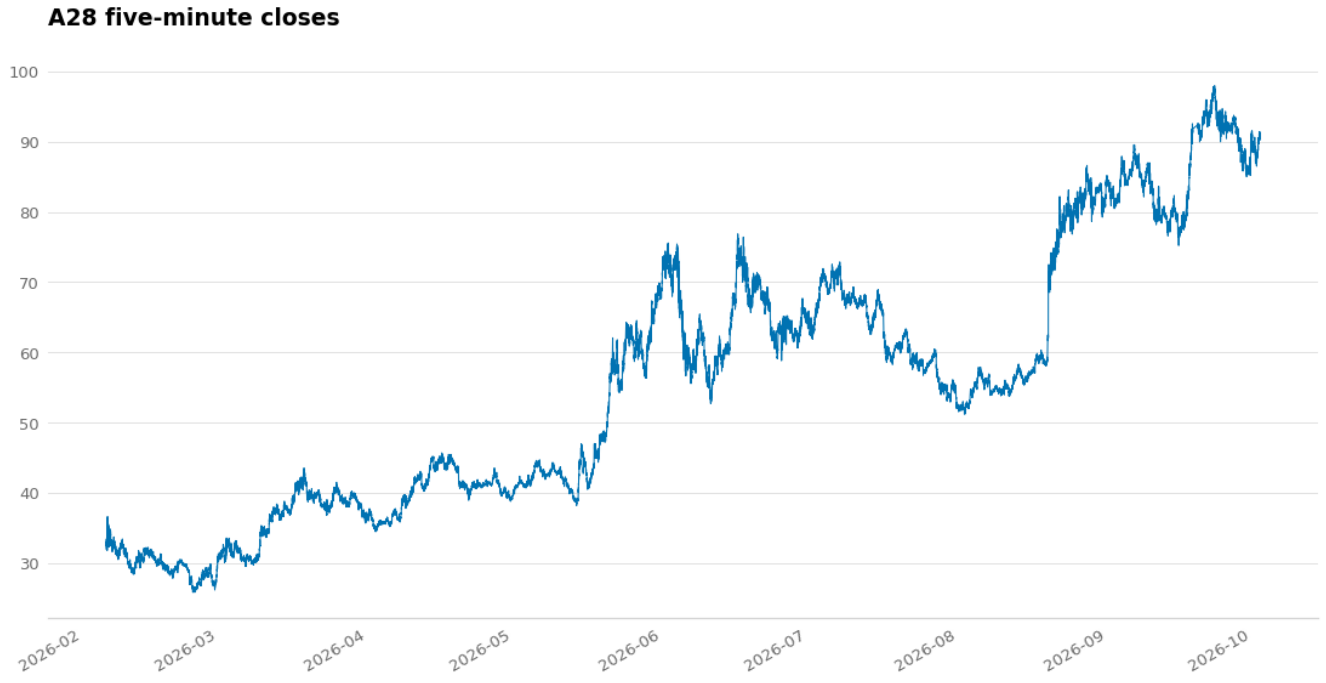
A28



A28, 66 fills, with more sells than buys.



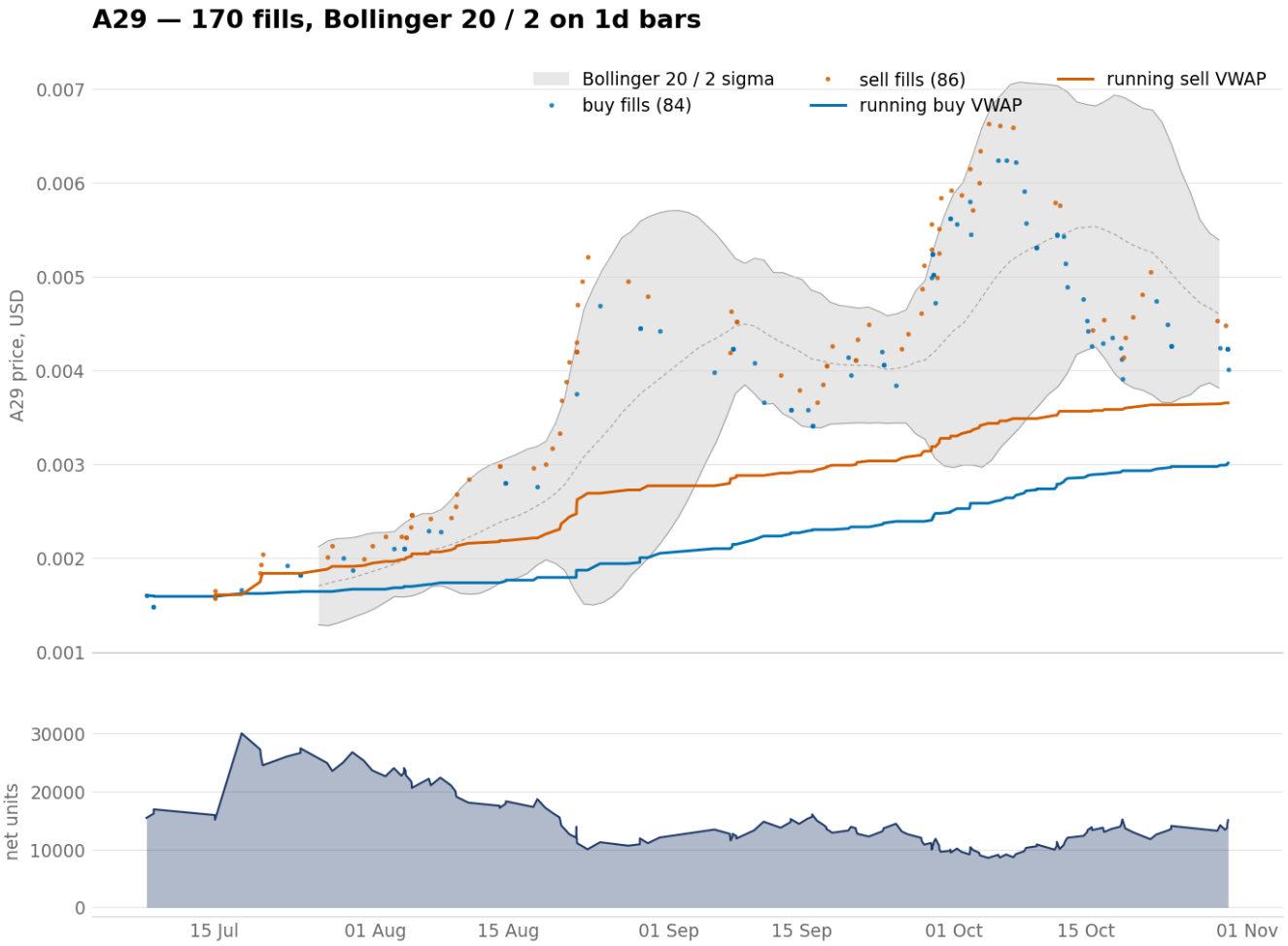
A28 weekly chart, 35 weekly candles built from 68,265 five-minute rows.



A28 price trace, 68,265 five-minute closes across the recorded window.

29 buys against 37 sells. The sell line peaks near 72.6 in early June and ends near 67, and the buy line holds between 61 and 65 across the window.

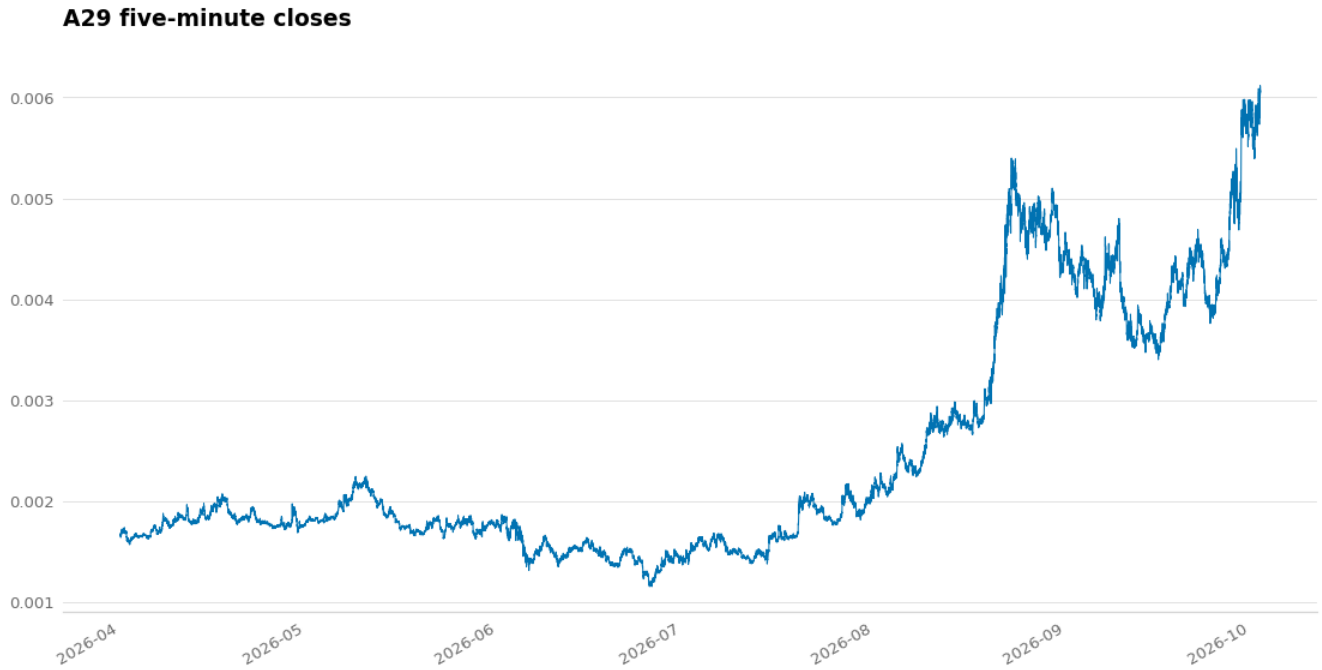
A29



A29, 63 fills, with both lines rising through August.



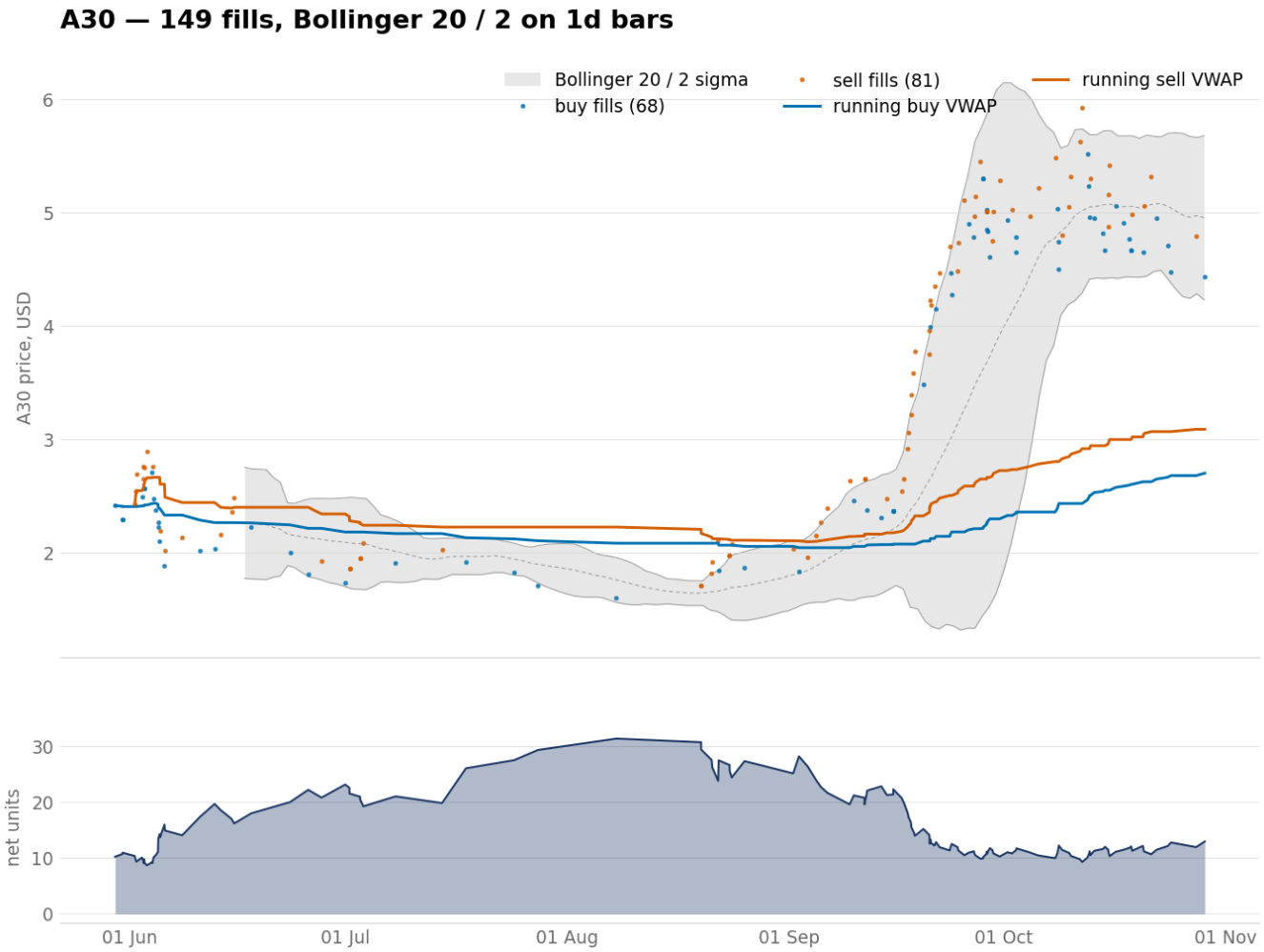
A29 weekly chart, 27 weekly candles built from 51,228 five-minute rows.



A29 price trace, 51,228 five-minute closes across the recorded window.

23 buys against 40 sells. The buy line rises from 0.001603 to 0.002051, and the sell line rises faster, ending about 0.0007 above it. Net units drop across the window, 29,987 down to 12,032.

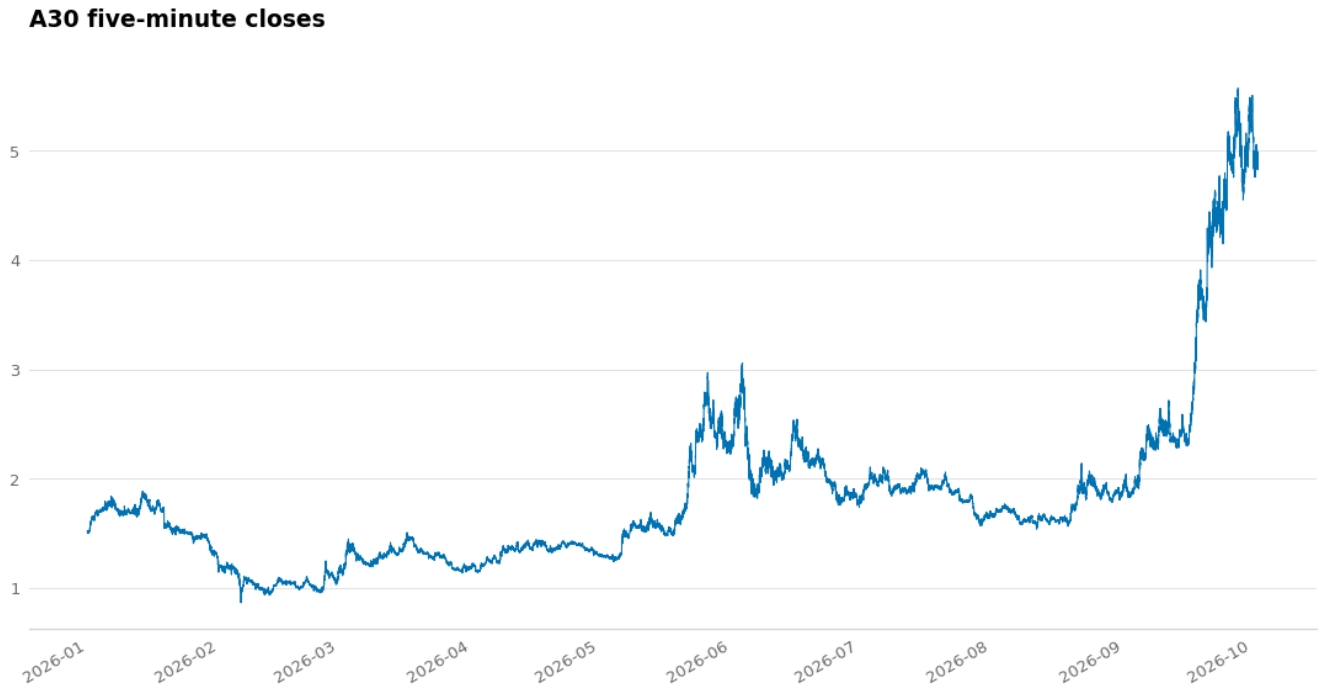
A30



A30, 58 fills, with both lines falling steadily.



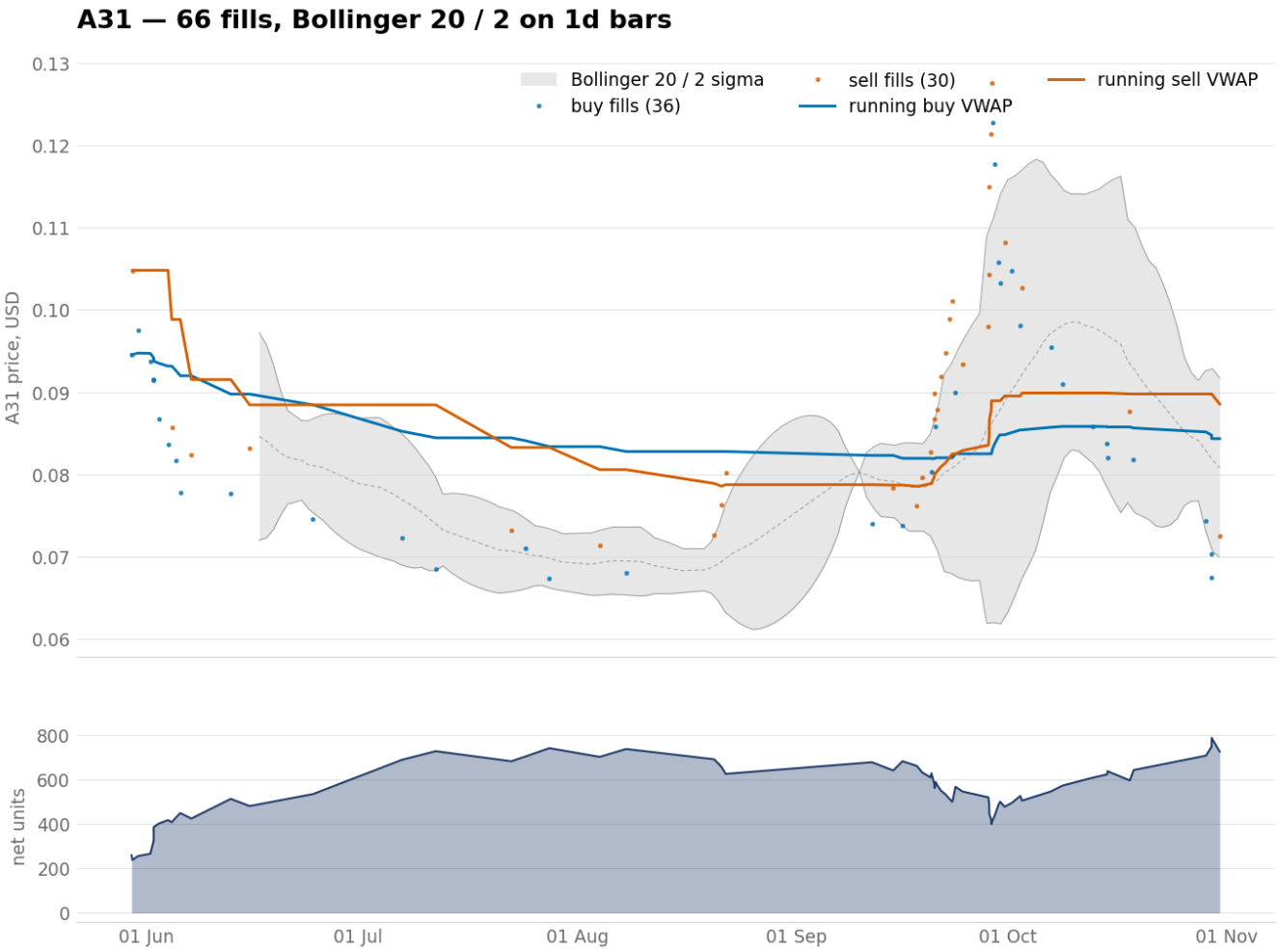
A30 weekly chart, 40 weekly candles built from 78,491 five-minute rows.



A30 price trace, 78,491 five-minute closes across the recorded window.

The sell line runs about 0.1 above the buy line from June to late August, then the two converge near 2.05. Net units end at 26.

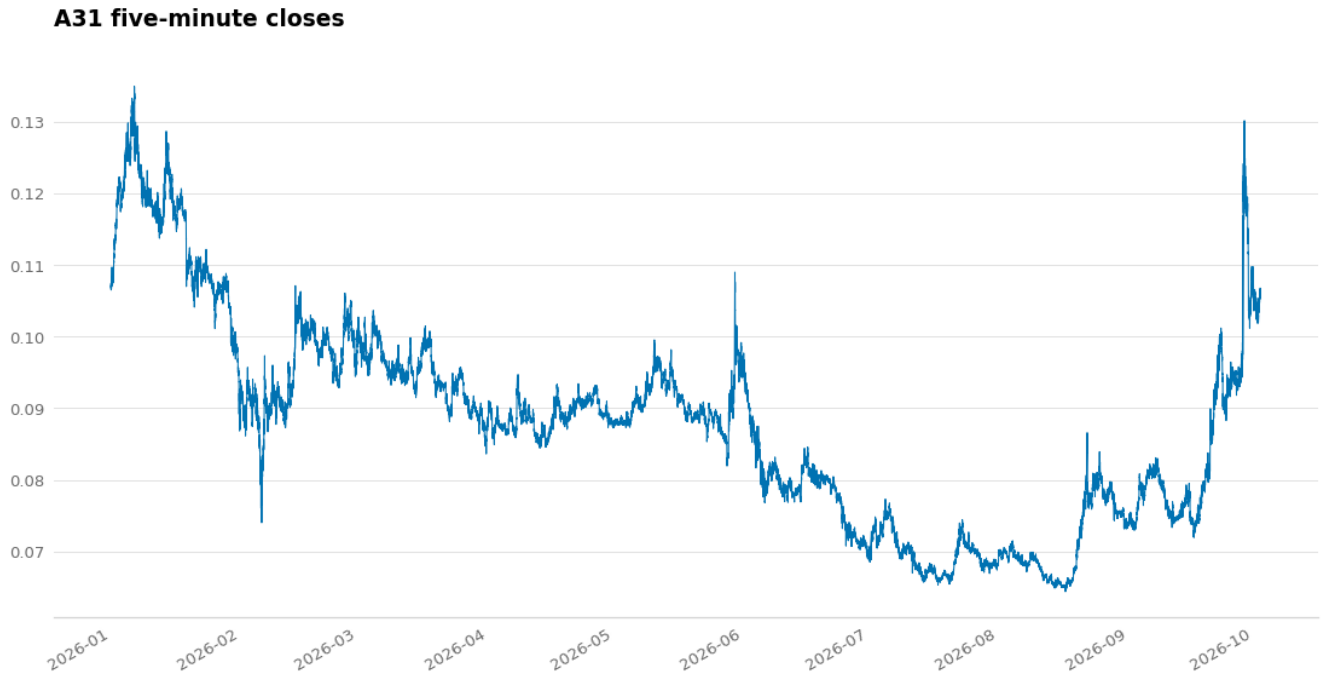
A31



A31, 25 fills, thin enough for the two lines to carry a dot per step.



A31 weekly chart, 40 weekly candles built from 78,993 five-minute rows.



A31 price trace, 78,993 five-minute closes across the recorded window.

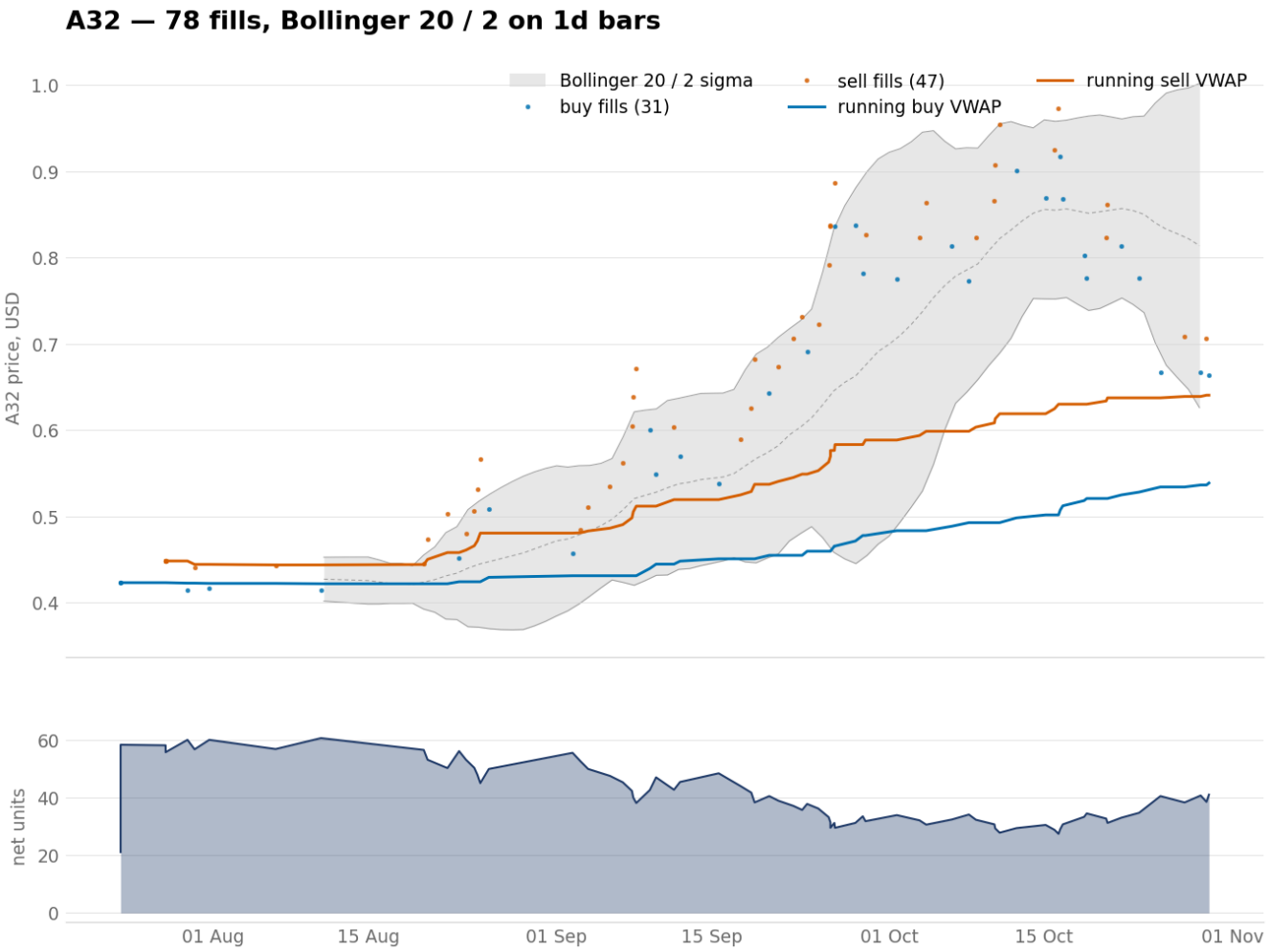
The first of the eight thin charts. The sell line starts at 0.1048, steps down through June and July, and crosses below the buy line in late July. Net units end at 626.

25 fills is under the module's threshold of 30, so both lines carry a marker at every step. Sixteen dots sit on the buy line and nine on the sell line. The seven charts after this one are drawn the same way.

```
THIN_FILLS = 30

dot = "o" if len(fills) <= THIN_FILLS else None
ax.plot(t, buy_v, color=COLORS["series"][0], lw=2.0, marker=dot,
        markersize=4, label="running buy VWAP")
```

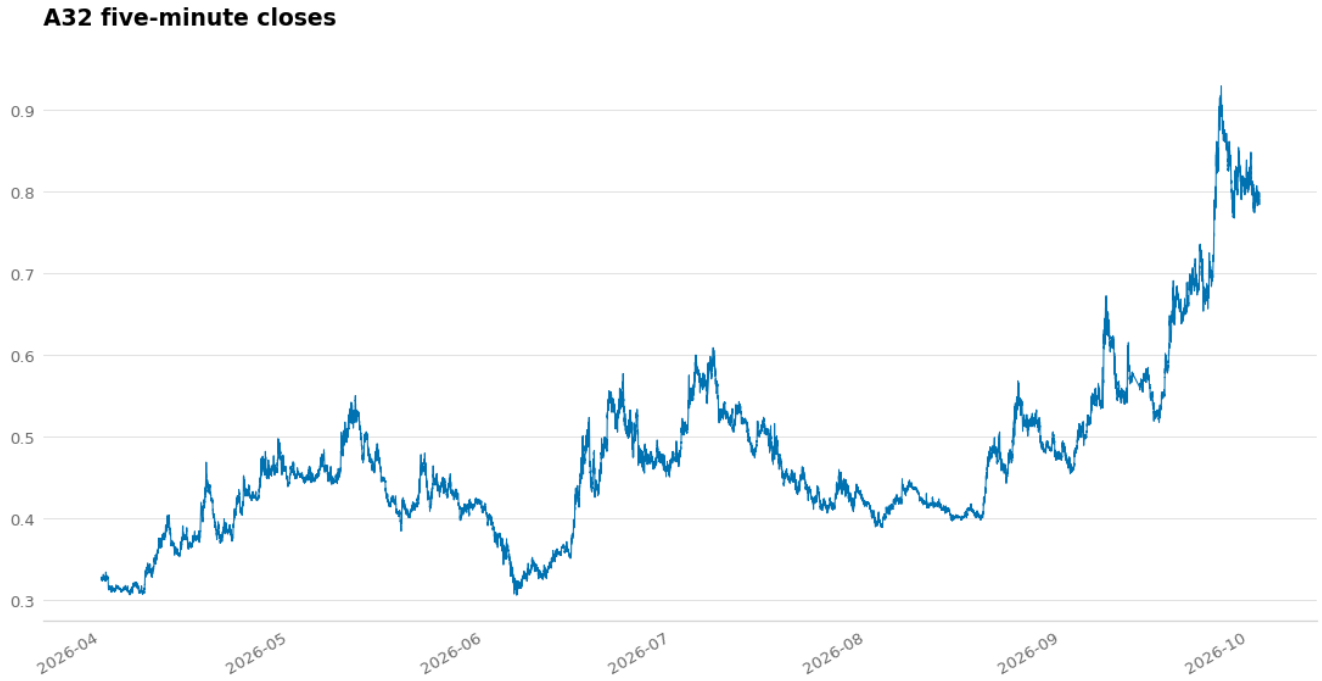
A32



A32, 24 fills, with eight buys and sixteen sells.



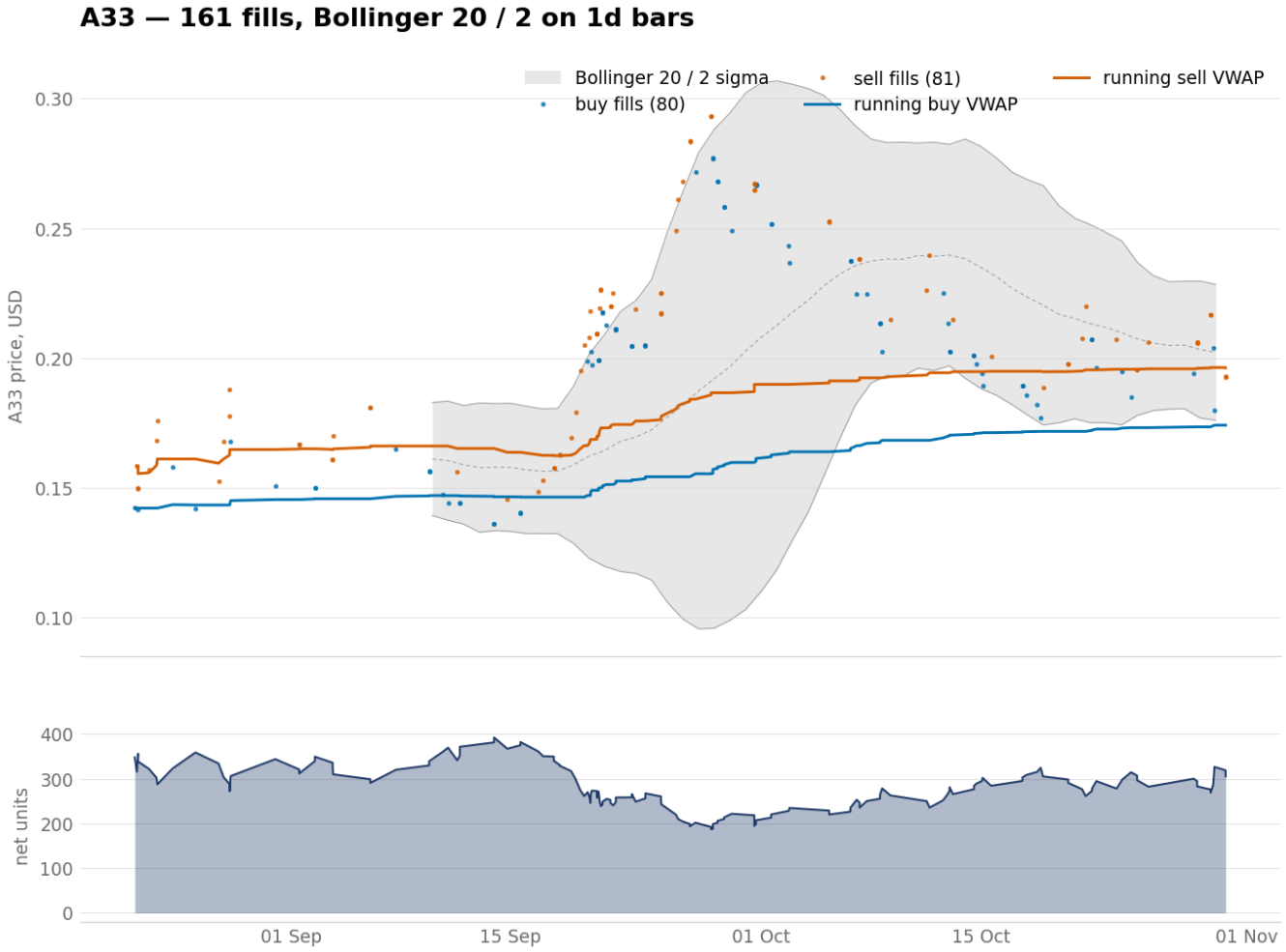
A32 weekly chart, 27 weekly candles built from 52,521 five-minute rows.



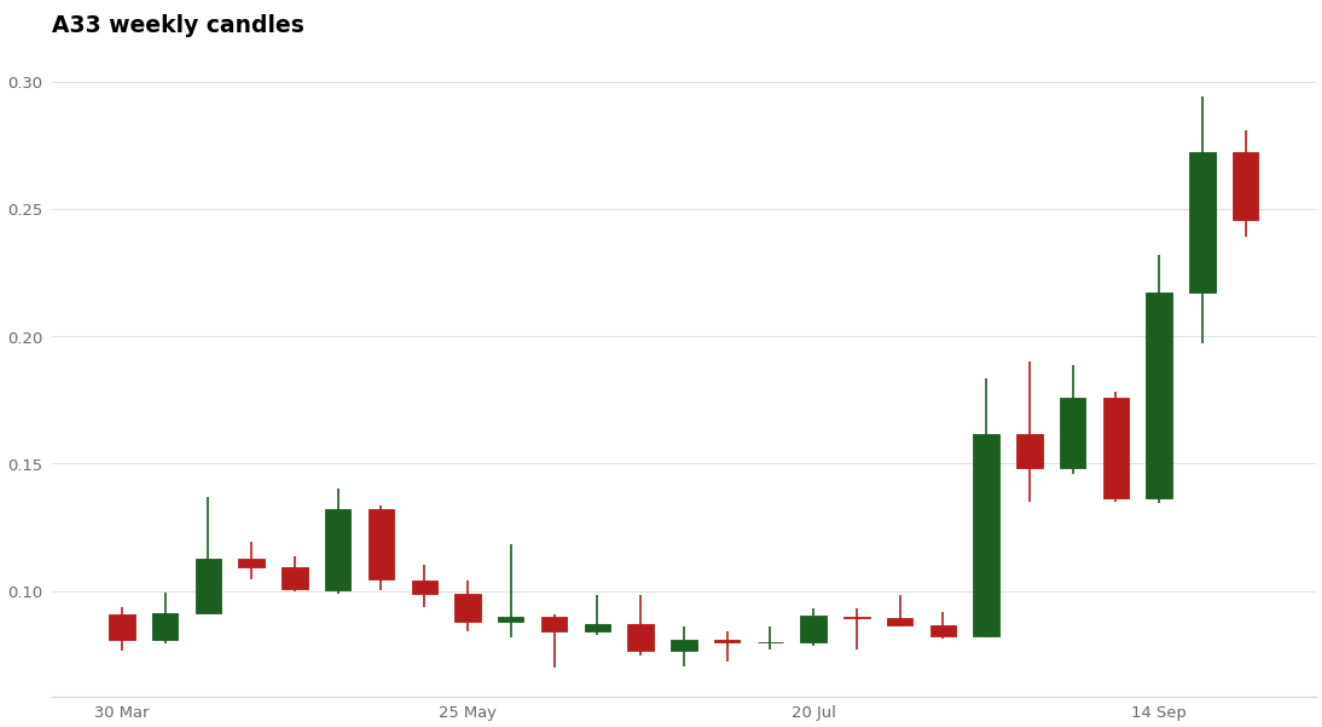
A32 price trace, 52,521 five-minute closes across the recorded window.

A thin chart. The buy line holds near 0.4225 until late August, then rises to 0.4310. The sell line jumps from 0.444 to 0.481 in the same week.

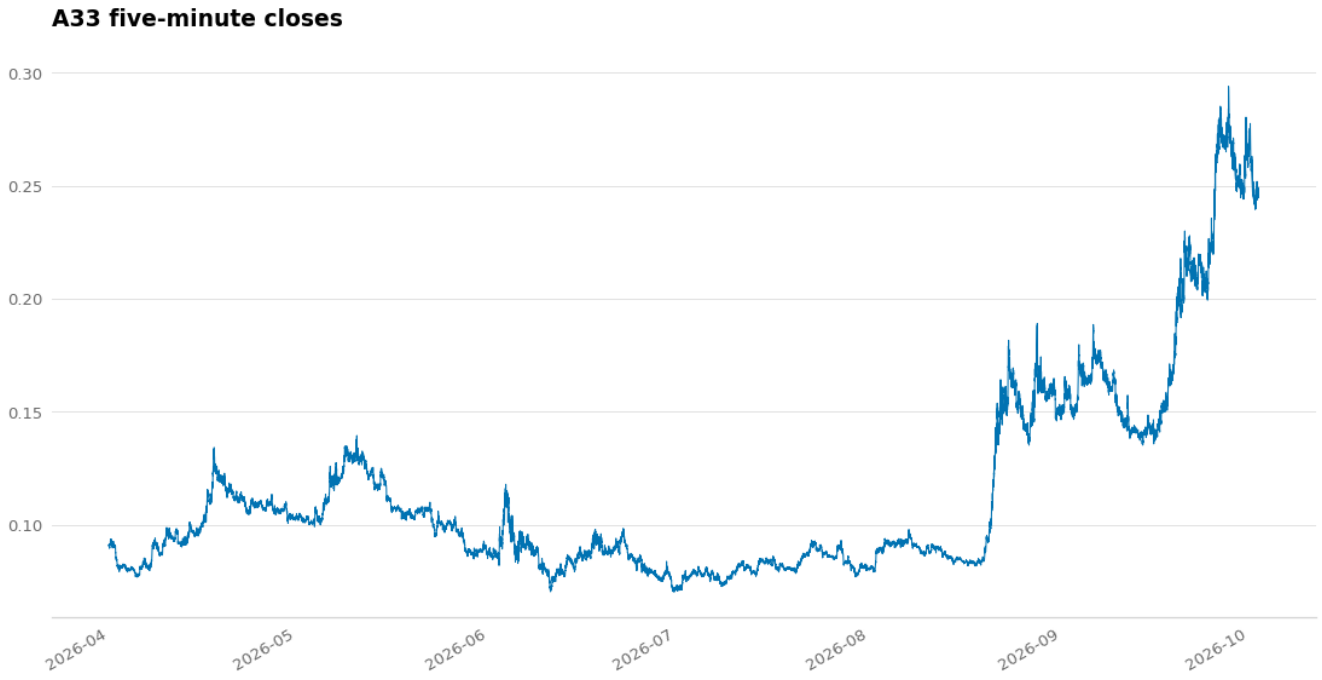
A33



A33, 23 fills across 12 days, the shortest window of the 38.



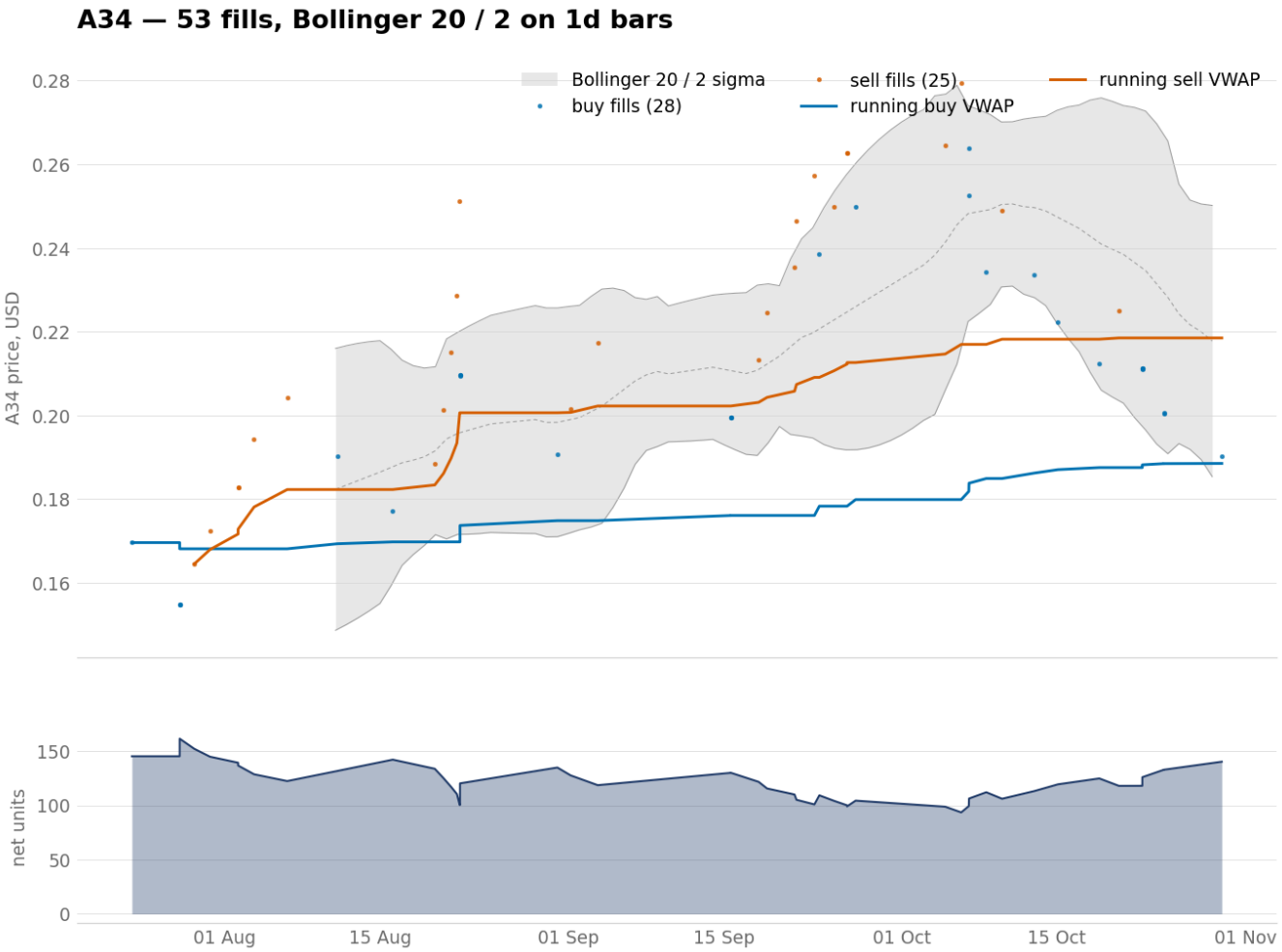
A33 weekly chart, 27 weekly candles built from 50,503 five-minute rows.



A33 price trace, 50,503 five-minute closes across the recorded window.

A thin chart, and the shortest window here. Eight buys against fifteen sells. Both lines step up once, on 28 August.

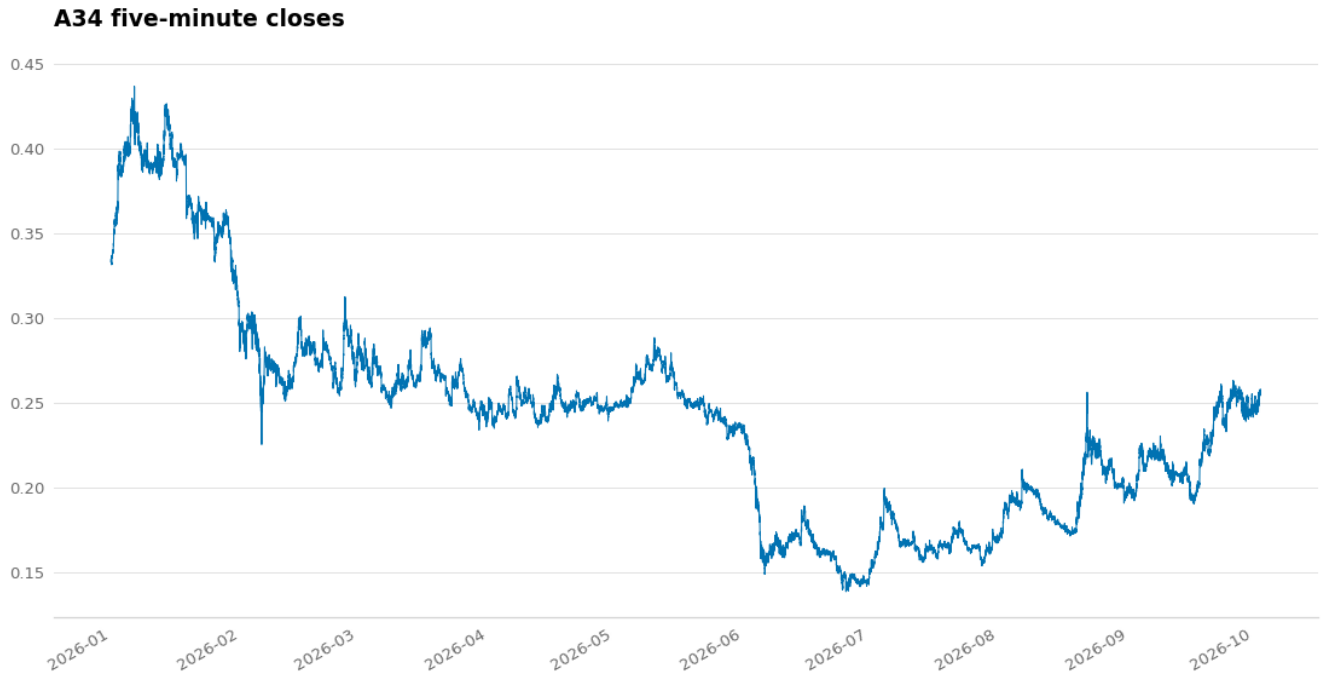
A34



A34, 23 fills, with the sell line stepping up sharply on 22 August.



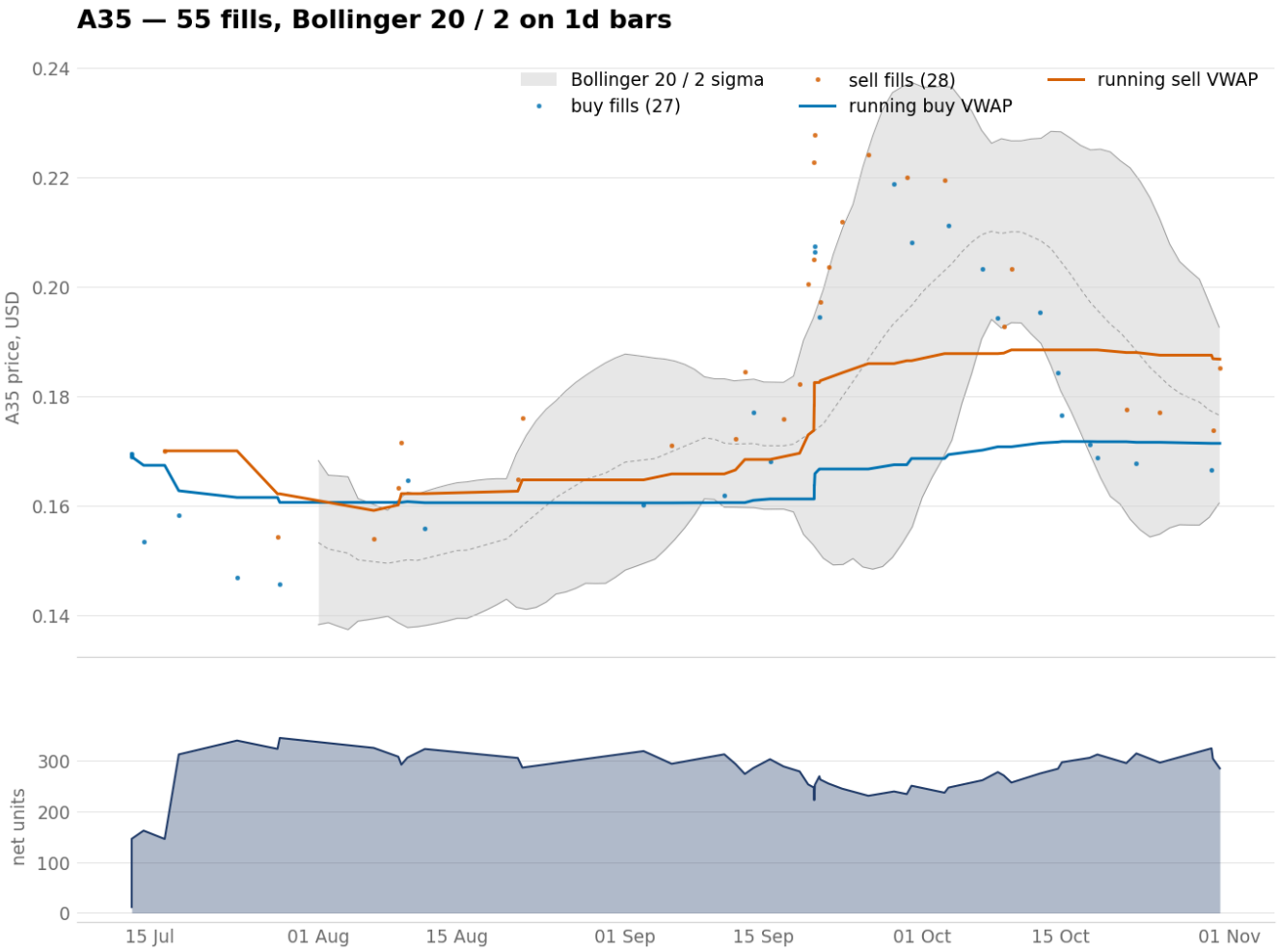
A34 weekly chart, 40 weekly candles built from 78,986 five-minute rows.



A34 price trace, 78,986 five-minute closes across the recorded window.

A thin chart. The sell line climbs from 0.1645 to about 0.202, while the buy line moves from 0.1696 to 0.1748 across the same six weeks.

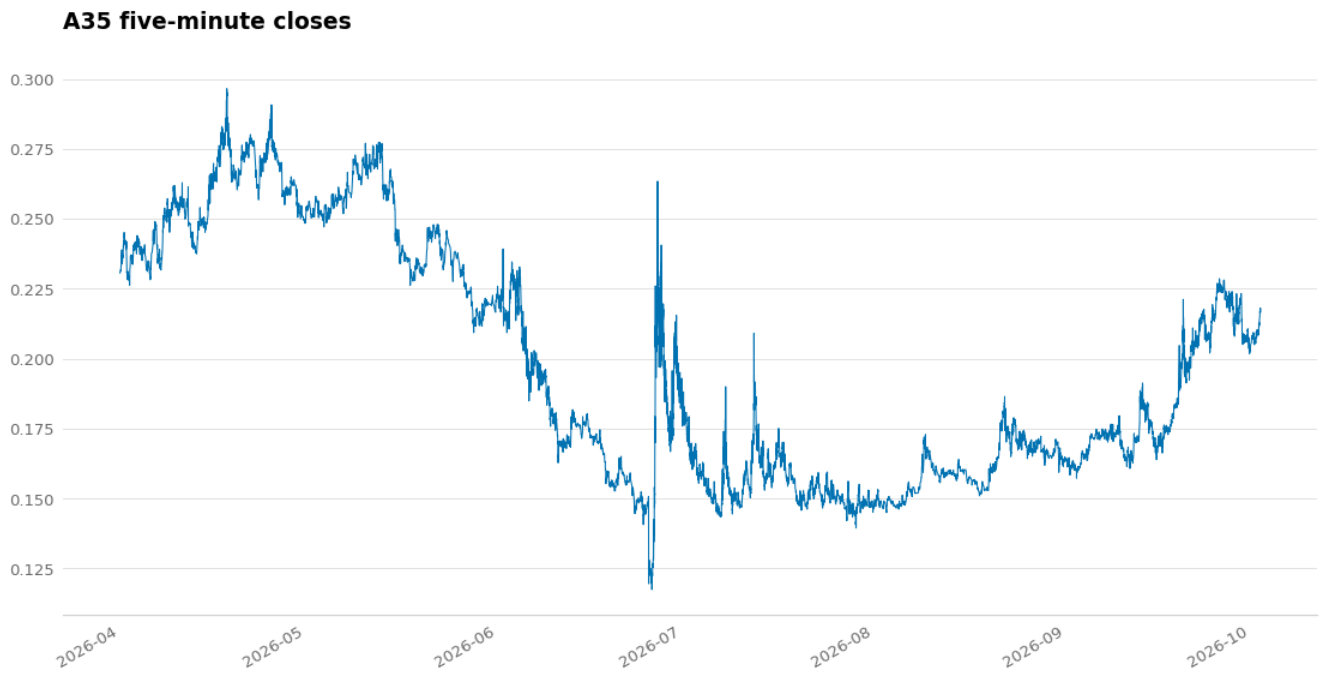
A35



A35, 16 fills, with the buy line flat from late July.



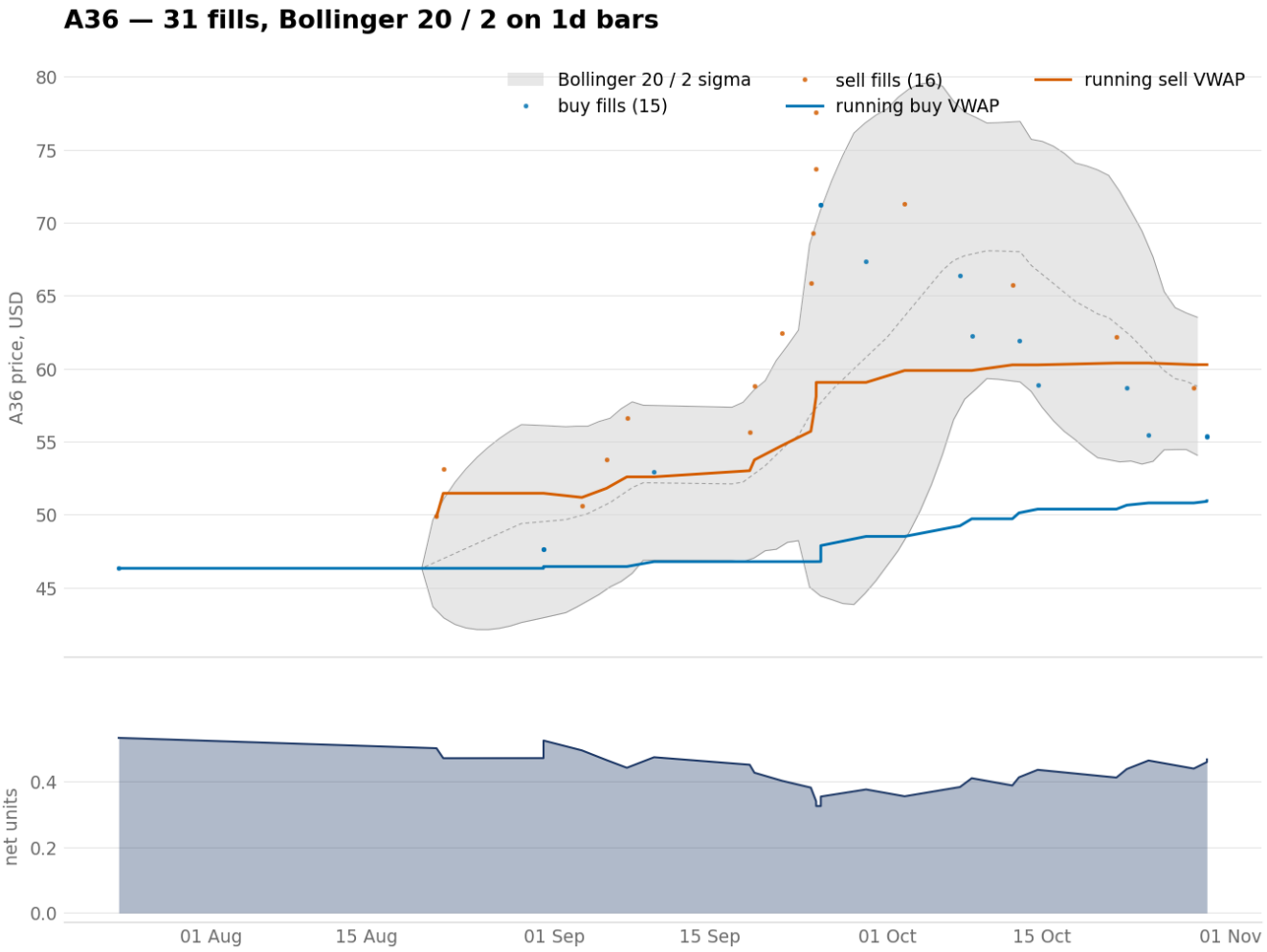
A35 weekly chart, 27 weekly candles built from 10,940 five-minute rows.



A35 price trace, 10,940 five-minute closes across the recorded window.

A thin chart. The buy line settles at 0.1605 in late July and holds. The sell line dips to 0.159 in early August, then steps back to 0.1648.

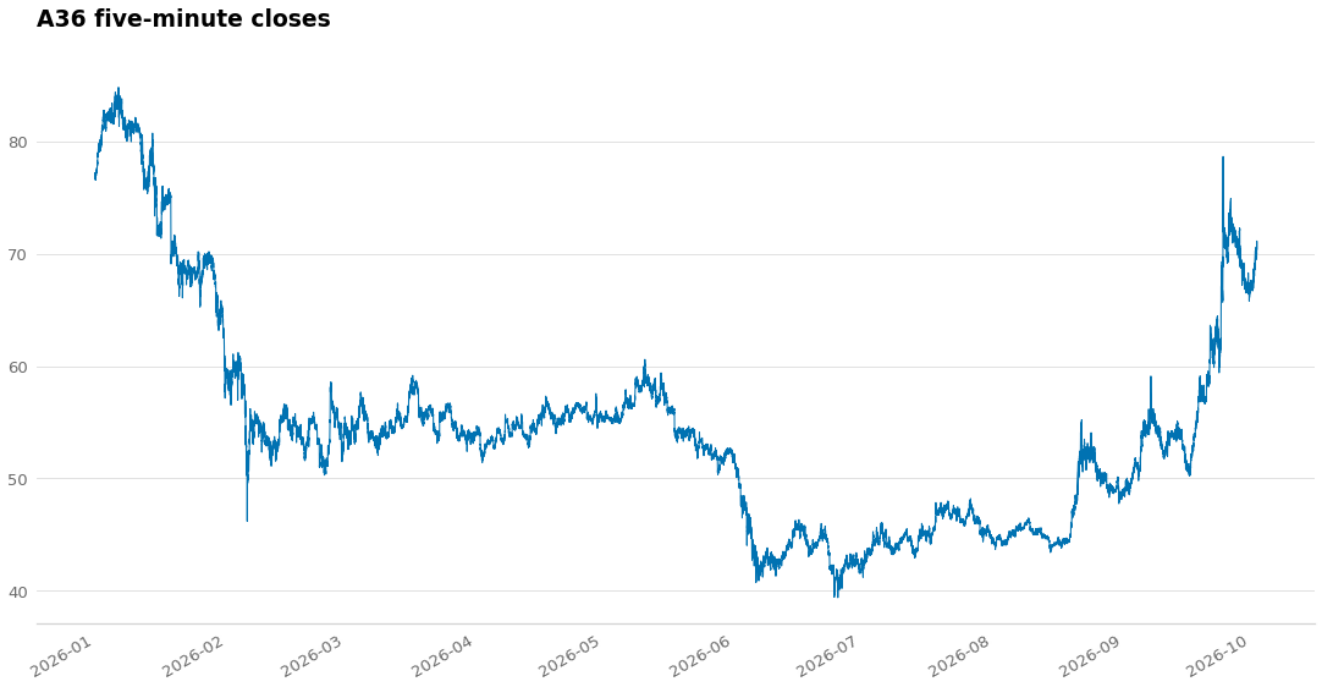
A36



A36, 6 fills, three buys and three sells.



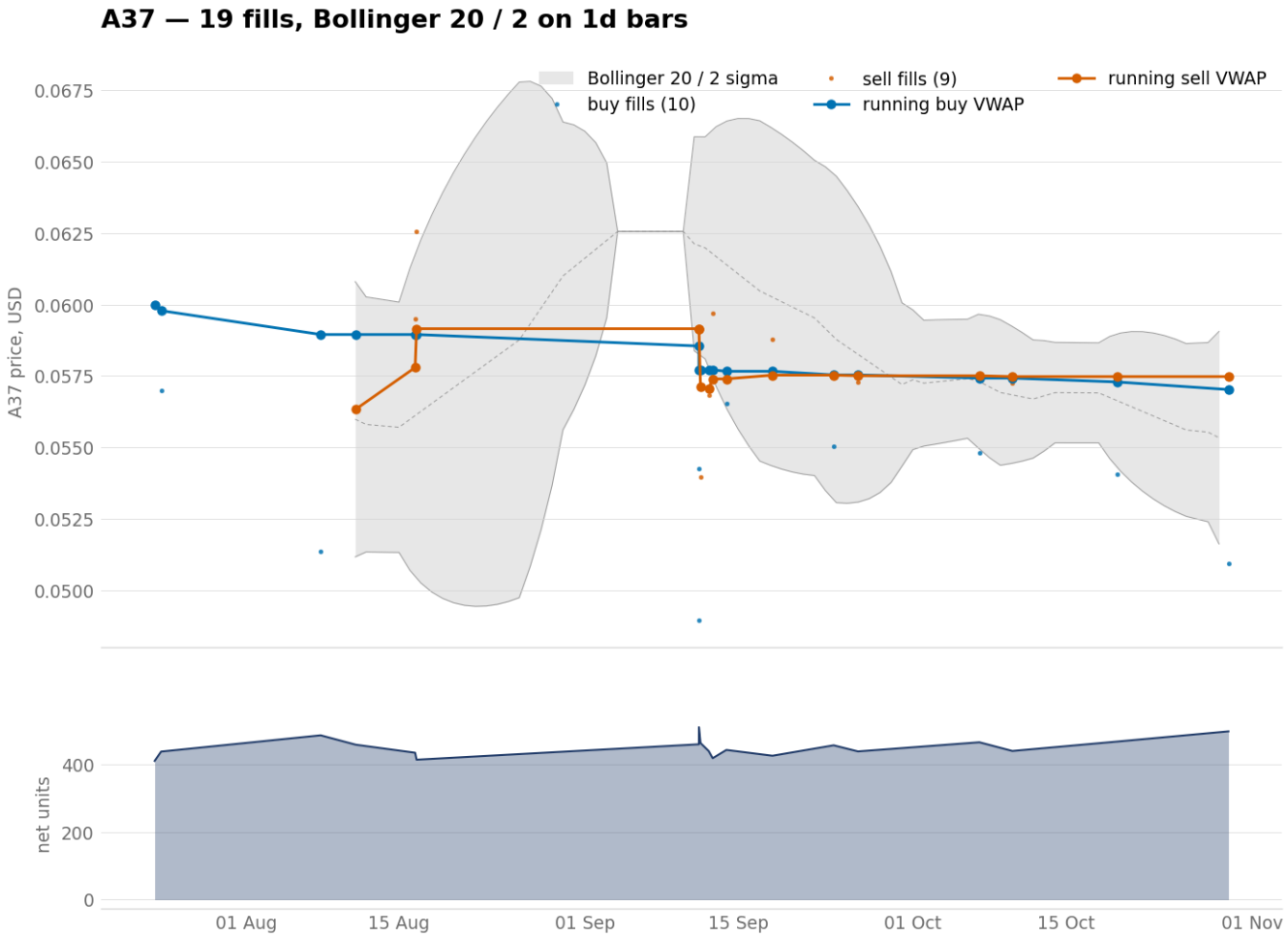
A36 weekly chart, 40 weekly candles built from 79,004 five-minute rows.



A36 price trace, 79,004 five-minute closes across the recorded window.

The buy line is nearly flat at 46.33, since all three buys land near one price. The sell line runs about 5 dollars above it. Net units end at 0.49.

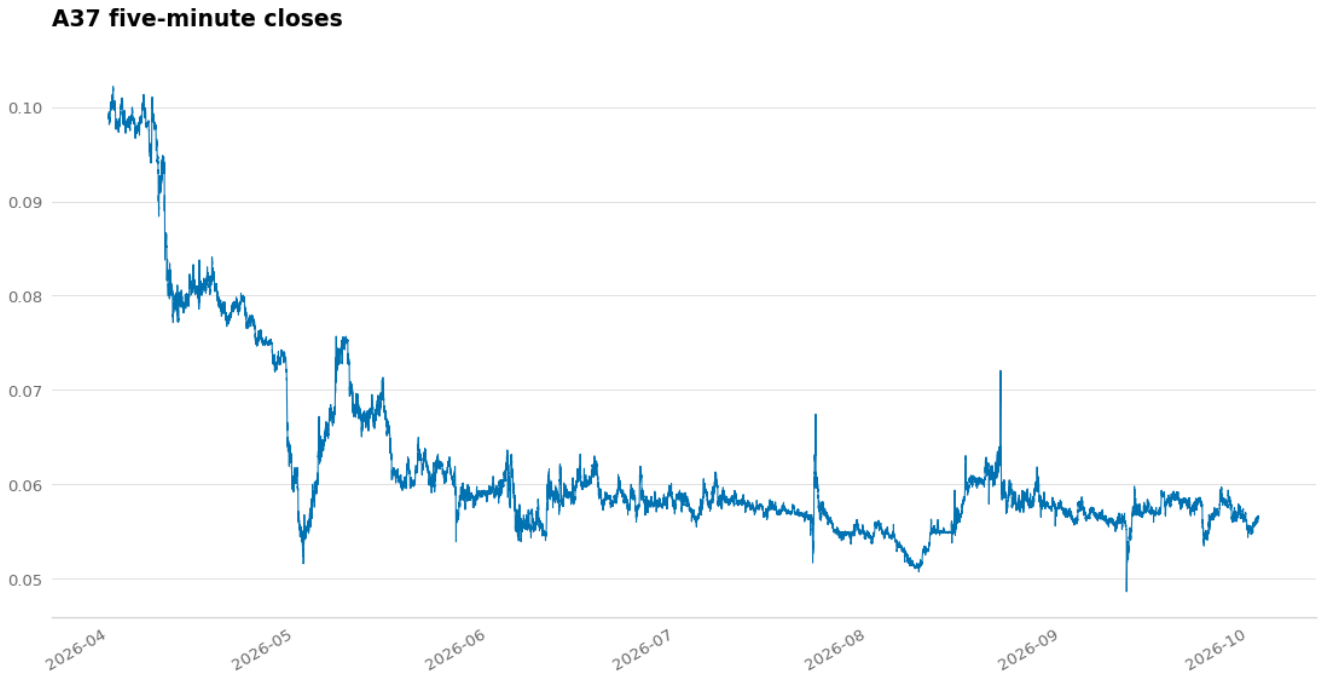
A37



A37, 6 fills, closing 16 August.



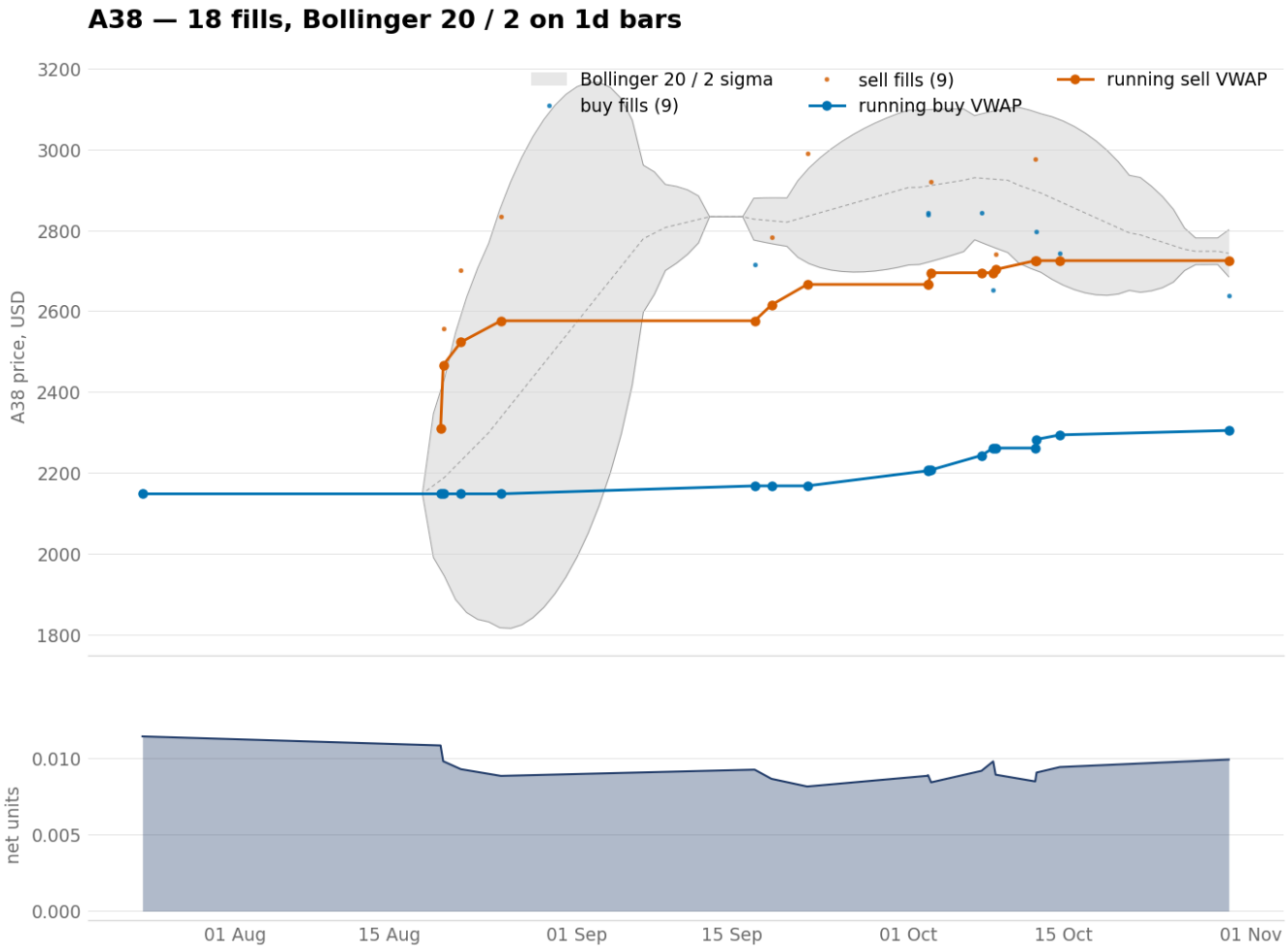
A37 weekly chart, 27 weekly candles built from 49,654 five-minute rows.



A37 price trace, 49,654 five-minute closes across the recorded window.

The earliest close of the 38. Three buys and three sells, all inside four weeks. The buy line drifts from 0.0600 to 0.0590.

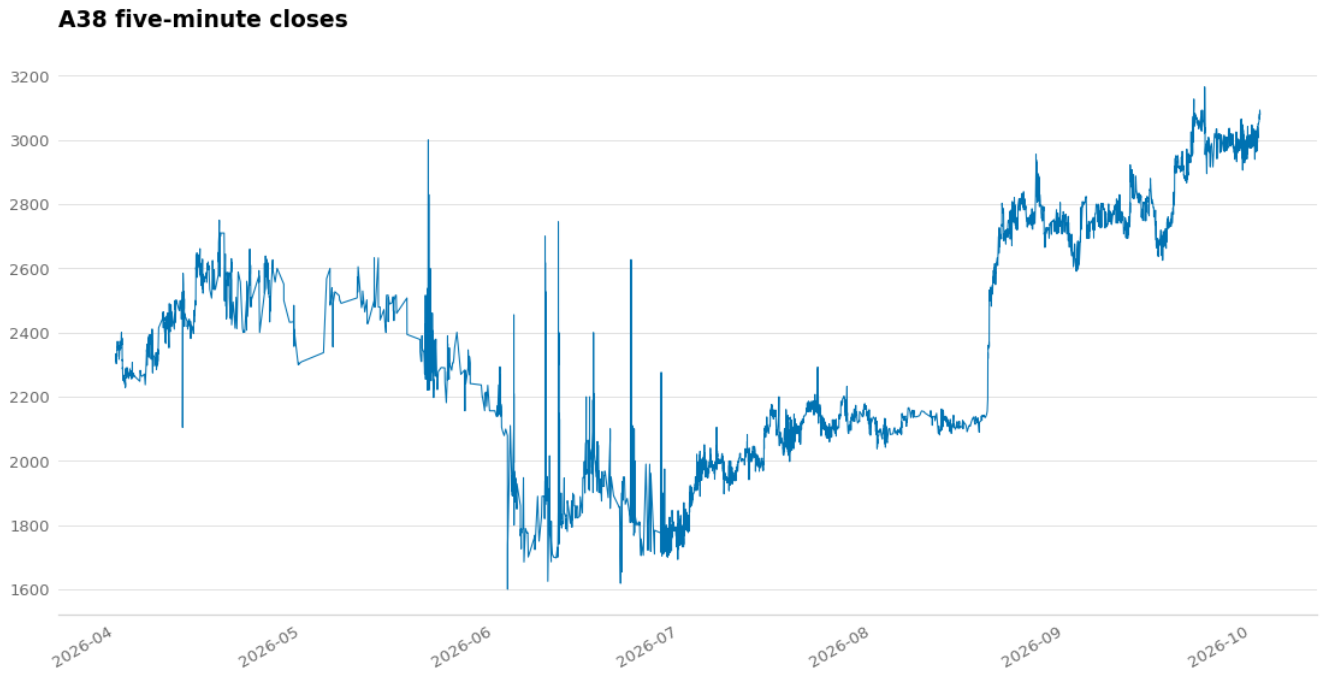
A38



A38, 5 fills, one buy and four sells.



A38 weekly chart, 27 weekly candles built from 6,257 five-minute rows.



A38 price trace, 6,257 five-minute closes across the recorded window.

The only base in the record with a single buy. One buy makes the buy line a single horizontal line at 2,148.63 and its index exactly 1.000. The sell line rises across the four sells, 2,307 up to 2,577. Net units drop across the window, 0.0114 down to 0.0088.

Trade grading

One function in `src/trading/trade_grader.py` grades a completed fill, on four axes and no more. Execution scores against a reference price at the decision, timing across the prices after the trade, strategy across the shift in the rolling S/B figure, and outcome against realised profit per unit. An axis whose price inputs are absent is left out, and the overall figure is the unweighted mean of the axes that scored — 0.5 when none of them could.

```
def grade_trade(record: TradeRecord, ctx: PriceContext) -> TradeGrade: ...


def _score_execution(...): ...
def _score_timing(...): ...
def _score_strategic(...): ...
def _score_outcome(...): ...
```

A second function maps that mean to a letter.

```
def _letter_from_numeric(num: float) -> str:
    if num >= 0.93:
        return "A+"
    elif num >= 0.85:
        return "A"
    elif num >= 0.70:
        return "B"
    elif num >= 0.55:
        return "C"
    elif num >= 0.40:
        return "D"
    else:
        return "F"
```

The market regime rides beside the grade rather than inside it. `PriceContext.regime_tag` reaches `TradeGrade.regime` and the rationale text, and it carries no sub-score of its own, which is why a rationale can read five terms while the grade rests on four.

`grade_trades` runs the batch.

The screen these grades reach is in the History tab reference.

Gate logs behind each trade

A fill says what happened. The gate log says which decision produced it.

`gate_light_row` in `src/trading/gate_vocabulary.py` returns nineteen lights for one decision, the scrum bank first and the fold bank second, each entry carrying its bank, label, state and colour. The History table and the Simulator draw their lights from that one function, so both surfaces name the same gates in the same order. The labels themselves, and the indicator readings behind them, are in the indicator reference.

`lookup_gate_entry` in `src/exchange/history_helpers.py` pairs each fill with the nearest gate entry for the same bot. The tolerance is 60 seconds either side of the fill, and a fill that finds an entry inside that

window carries its gate. A fill that finds none carries no gate.

```
JOIN_TOLERANCE_SECONDS = 60.0


def gate_cell_text(entry: Optional[dict]) -> str:
    if not entry:
        return "no record"
```

A fill with no entry reads "no record" in the Gates column, and its tooltip says no gate record joined to the trade. Nothing measures coverage over a set of fills, and nothing names a reason a fill has no gate.

One gate log per exchange and per sector

A decision is written to the venue it was taken on and the sector of the market it was taken in. The path is `trade/gate/<exchange>/<sector>/gate.log` under the log root, and the row carries its own sector in `data.asset_class`, so a row names the file it is in.

```
~/.acervator_logs/trade/gate/coinbase/crypto/gate.log
~/.acervator_logs/trade/gate/robinhood/crypto/gate.log
~/.acervator_logs/trade/gate/robinhood/stocks/gate.log
```

`src/core/log_paths.py, in gate_log_path` composes it, and `path_segment` folds each name to one lowercase directory, so a venue id cannot reach a parent directory through the path. `src/core/logging_engine.py, in LogManager._gate_writer_for` holds one writer per pair and opens it on the first decision, so every bot on one venue trading one sector appends to a single file.

The sector reaches the writer from the bot. `src/trading/scrumming/snapshots.py, in _emit_gate_decision_at_fire` reads `BotContainer._asset_class` for the traded symbol, which answers the sector the venue's market recording holds, then the sector the bot's own config declares, then crypto. The Simulator answers the same question through `src/simulator/sim_bus.py, in sim_asset_class`, so a sim decision is filed where its live twin is filed.

Each pair's active file rotates at 50 MB and keeps five backups, so one pair bounds at 300 MB and each further pair adds 300 MB of its own.

Reading every decision, wherever it was written

`src/core/log_paths.py, in gate_log_files` lists every gate log once: the pre-split file while it is still there, then each pair's archive members, rotations and active file. Every reader goes through it.

reader	what it draws
`src/trading/live_log_reader.py, in live_gate_decisions`	the entries the History tab and the parity tools join against
`src/exchange/history_helpers.py, in build_page_gate_index`	the per-page bot-and-minute index
`src/gui/history_tab.py, in read_join_indexes`	the same index, read off the GUI thread
`src/simulator/gate_log_source.py, in GateLogSource.files`	the Simulator's read-only `GateRow` records
`src/simulator/sim_bus.py, in sim_log_paths`	the paths the parity report prints

reader	what it draws
`tools/capture_live_baseline.py, in gate_members`	the baseline snapshot's latch distribution

Now: the History tab reads that index in steps of about 8 ms on the GUI thread, not off it (`src/gui/history_tab.py, in read_join_indexes`).

What became of the records written before the split

`src/core/logging_engine.py, in migrate_legacy_gate_logs` carries them, once, when the first `LogManager` of a build with this layout is built. It runs before any gate writer opens, so no handle is held on a file it moves.

Each pre-split file leaves every reader's view in one rename before any bucket file appears, under a `.consumed-` name the archive member will take. A file whose records all belong to one pair is then renamed again into that pair's archive, with no byte copied. A file holding more than one pair is streamed into a `.pending-` file per pair, and each is promoted in turn. A carry cut short leaves the `.consumed-` name in place and the next `LogManager` finishes it, so a decision is never recorded twice.

An archive member never rotates and nothing prunes it. The records are decisions behind real fills, not a running stream.

Measured over a full-size copy of the operator's own gate records, with no write to his tree:

```
in     6 files   167,947 decisions   268,798,267 bytes
out    7 files   167,947 decisions   268,798,267 bytes   1.81 seconds

gate/coinbase/crypto/archive/     6 files, the five rotations and the active file
gate/fleet_sim/crypto/archive/    1 file, 136 decisions an older simulator wrote
```

The byte totals are equal because the carry copies each line as the bytes on disk hold it. The two destinations are the two values the `exchange` field carries across those records: `coinbase` on 167,811 of them and `fleet_sim` on 136. No record carried a sector, so every one is filed under the sector `src/core/logging_engine.py, in default_gate_sector` answers, which is the sector `BotContainer._asset_class` answers for a market the recording labels none.

Two controls were read on the same copy. A comparison of the line sets before and against after reports `lost=1` when one decision is removed from the result and `duplicated=1` when one is repeated, so the equal reading above is a measurement. A carry interrupted after its first pair was promoted reads 3,245 of 3,381 decisions readable, 136 lost and 0 duplicated; the next `LogManager` reads 3,381, 0 lost and 0 duplicated.

Weekly candles behind the fills

The 38 charts above draw fills. This set draws the market those fills ran in. Each panel is one charted base and carries the same label, ten panels to a page.

The bars are built here, not fetched. `weekly_bars` in `tools/build_product_manual.py` groups a base's five-minute Stone Tablet rows by the Monday 00:00 UTC that `week_start_ms` returns for each stamp. Each group gives one bar: the first row's open, the highest high across the group, the lowest low across the group, and the last row's close. A week the venue recorded no trade in produces no bar.

One week of one series checks that rule against hand arithmetic. On A05, the week opening 2026-05-18 holds 2,016 five-minute rows.

```
weekly_bars   open 535.96  high 688.52  low 515.24  close 662.04
by hand       open 535.96  high 688.52  low 515.24  close 662.04
control       a last-open rule returns 662.10 for the open
```

The control matters because three of the four figures survive a wrong grouping. A rule taking the last open rather than the first moves only the open, and the comparison reports it.

A panel's price axis turns logarithmic when its highest high exceeds its lowest low more than eightfold, the same rule the per-asset charts above use.

One sentence above is overtaken. It is quoted whole, and the sentence that replaces it follows.

> *Each panel is one charted base and carries the same label, ten panels to a page.*

Each charted base has a weekly chart of its own, and it sits in that base's group above, beside that base's VWAP chart.

`_write_asset_weekly` in `tools/build_product_manual.py` writes one figure per base, named for the base's label and titled with it.

The price at the finest resolution the record carries

Five minutes is the finest resolution the Stone Tablets hold. `StoneTabletsRegistry` in `src/trading/stone_tablets/registry.py` stores every tablet at that step and rolls every longer timeframe up from it, so no finer price exists in this repository. These panels plot the close of every five-minute candle across the recorded window, one line per charted base.

The weekly bars above and these lines read the same rows and answer different questions. A weekly bar hides what happens inside its week, and inside the week is where the Scrum and Fold cycle works. A week that opens and closes at the same price can still hold the swings the engine sells into and buys back from.

Each charted base has a price trace of its own, and it sits in that base's group above, under the same label as that base's VWAP chart and weekly chart.

`_write_asset_traces` in `tools/build_product_manual.py` writes one figure per base, from the same five-minute rows the weekly bars are built from.

Part 10

The Sigil Project

- Named Checks That Read Finished Code Against a Published Rule Set - Design and Progress - One Page per Built Sigil

The Sigil Project

Reference. A sigil vernacular is a working language for an occupation. Each sigil names a professional workflow or a canonized, restricted output. The project is named Sigil-based Vernacular Condensate.

In this repository, a sigil is a named check. It reads finished code against a published rule set and answers from a fixed list. Its mark holds its name and its edition, such as CODE.LOOP@1. Nothing here changes the trading platform.

What a sigil is

Each sigil holds its card as data in its own module. The card names the sigil, its edition, the archetype that owns it, its input form, its three input tiers, its outputs and the flag of each output. The command prints the card as JSON with `--card`, so any harness can read the card without this repository's skills.

`dev_harness/sigils/core.py, in Sigil`, the card as data:

```
@dataclass(frozen=True)
class Sigil:
    """A card as data, with tiers 1, 2 and 3 as `data_type`, `content`, `functions`."""

    name: str
    edition: int
    owner: str
    input_form: str
    settings: Mapping[str, str]
    tiers: tuple[TierCard, TierCard, TierCard]
    outputs: Mapping[str, FlagRow]
    refusal: str
    follows: Mapping[Flag, str]
    readers: Mapping[str, Callable[..., object]]
    data_type: DataType
    content: Content
    functions: Functions
    fault_file: FaultFile | None = None
```

The scope test

A sigil applies only to work with a known set of formatted results. Everything else stays in prose. Work qualifies for a sigil only when all five lines read yes. The test is written in `dev_harness/skills/unit-decomposition/SKILL.md`, in the section headed THE SCOPE TEST.

```
input     its shape is declared, and input out of range has a defined answer
outputs   every allowed output is listed in advance
ground    every output can be checked against code or a document
owner     an archetype reads the result without asking the model
decision  the model makes one named choice; with no choice, it is an ordinary function
```

The three input tiers

A sigil filters each input in three tiers, in this order. A tier runs only after the tier before it accepts. A refusal names the tier that refused it, and its output is REJECTED with the flag BLOCK.

```
tier 1  data type                    the input has the declared shape and characters
tier 2  content compatibility        the content belongs to the owning archetype's type and work
tier 3  required processing functions  the function set that does the sigil's work and gives its output
```

Tier 2 comes from the owning archetype. A sigil with no owning archetype has no tier 2 and is not admitted. Every Coding sigil takes tiers 1 and 2 from one module. Each Coding sigil built after the first four narrows tier 2: Python's ast module must find the construct the sigil reads, or the input is refused.

`dev_harness/sigils/core.py, in judge`, the three tiers in order:

```
def judge(sigil: Sigil, text: str, root: Path, *, in_source: bool) -> Verdict:
    """Run `text` through tiers 1, 2 and 3 of `sigil`, stopping at the first refusal."""
    fault = sigil.data_type(text)
    if fault:
        return Verdict(sigil.refusal, f"{DATA_TYPE}: {fault}", 1, Flag.BLOCK)
    fault = sigil.content(text, root)
    if fault:
        return Verdict(sigil.refusal, f"{CONTENT}: {fault}", 2, Flag.BLOCK)
    reading = sigil.functions(text, root)
    holds = not isinstance(reading, Reading) or reading.holds
    output, reason = reading[0], reading[1]
    flag = flag_of(sigil, output, in_source=in_source and holds)
    return Verdict(output, f"{FUNCTIONS}: {reason}", 3, flag)
```

The path of one input line, from the tiers to its flag:

Diagram: flowchart, top to bottom

- one input line -> **tier 1: data type**
- tier 1: data type -> **BLOCK: the work does not start** *(refuses)*
- tier 1: data type -> **tier 2: content compatibility** *(accepts)*
- tier 2: content compatibility -> **BLOCK: the work does not start** *(refuses)*
- tier 2: content compatibility -> **tier 3: required processing functions** *(accepts)*
- tier 3: required processing functions -> **an output the card lists**
- an output the card lists -> **PASS: nothing further** *(valid)*
- an output the card lists -> **OPTIMIZE: the merge proceeds** *(a named optimization rule)*

- an output the card lists -> **FIX: the merge proceeds, the fault is fixed at once** *(a fault outside application source)*
- an output the card lists -> **HOLD: the merge waits until it is fixed** *(a fault in application source)*

Flags and follow-on rules

Each output maps to one flag on its card. Each flag names the rule that follows it. The card holds that rule as data, and nothing runs a follow-on rule yet.

`dev_harness/sigils/core.py, in FLAGS`:

Flag	Side	When	What follows
BLOCK	input	a tier 1 or tier 2 refusal	the work does not start
HOLD	output	a fault in application source code	the merge waits until it is fixed
FIX	output	a fault anywhere else, such as a document	the merge proceeds, and the fault is fixed at once
OPTIMIZE	output	a named optimization rule fires	the merge proceeds
PASS	output	the output is valid	nothing further

Only application source code holds a merge. Application source is a setting of the card: src, desktop and main.py in this repository. The command's `--source` option names another tree's own. With `--gate`, the command exits 1 only when an input reads BLOCK or HOLD. Without it, the command exits 1 when any input reads a flag other than PASS.

`dev_harness/sigils/core.py, in GATED` and `in APPLICATION_SOURCE`:

```
GATED = frozenset({Flag.BLOCK, Flag.HOLD})

APPLICATION_SOURCE = ("src", "desktop", "main.py")
```

A finding from ruff carries the fix class that ruff states for its rule. The fix class names the follow-on rule. Each sigil page states which findings read HOLD.

`dev_harness/sigils/fix_class.py, in FOLLOW_ON`:

```
FOLLOW_ON = {
    "always": "apply ruff's fix for that rule and read the file again",
    "sometimes": "the same, where ruff offers it for that line",
    "none": "a unit of work",
}
```

The shared modules

Four modules hold the steps that every sigil shares. Each step has one body. No sigil module imports from another sigil module.

Module	What it holds
`dev_harness/sigils/core.py`	the card as data in `Sigil`, the tiers in `judge`, the flags in `FLAGS`, and the command in `main`
`dev_harness/sigils/coding_input.py`	the Coding Archetype's input: tier 1 in `DATA_TYPE_TIER`, tier 2 in `CONTENT_TIER`, and the file readers in `READERS`
`dev_harness/sigils/ruff_rules.py`	one ruff run for a rule list in `check`, the fix class in `fix_class`, and a rule's published faulty and corrected forms in `rule_page`
`dev_harness/sigils/fix_class.py`	the follow-on rule of each fix class, in `FOLLOW_ON`

CITE.SYMBOL@3 keeps its own tiers 1 and 2. Its input form differs, and the Truth Archetype owns it.

Portability

Every sigil is built to work in any repository, not only this one. Acervator is the first place it does a real job.

```
imports    the Python standard library or a declared third-party package; nothing from the host application
target     the tree a sigil reads is an input, never derived from where the sigil file sits
card       the sigil's name, edition, input form, outputs and reject reasons are data in its module,
           and its command prints them, so any harness can read the card without this repository's skills
languages  each file type has one reader behind one interface; a type with no reader is rejected, never guessed
settings   a house convention, such as the citation form, is a named setting of the card, not a literal in code
fixtures   each sigil ships a small sample tree, so its test pair runs in any repository
proof      a sigil is called universal only after it runs on a second repository and its results are stated
```

Where the code holds each row:

Row	Where it is in the code
target	`dev_harness/sigils/core.py, in main`: the `--root` option
card	`dev_harness/sigils/core.py, in render_card`, printed by `--card`
languages	`dev_harness/sigils/coding_input.py, in READERS`: `.py` is the only reader
settings	`dev_harness/sigils/core.py, in APPLICATION_SOURCE`, which `--source` replaces
fixtures	one folder per sigil under `harness_fixtures/`, named `sigil_` and the module name

What a sigil cannot guarantee

Layer	The sigil's check	What it stops	What it cannot stop
Form	The output fits the restricted shape	A malformed result	A well-formed wrong answer
Reference	Every cited name resolves in the published canon	A fabricated code, case or formula	A real citation that does not support the claim
Computation	The function set runs outside the model	A miscalculated result	A wrong choice of function
Judgement	The owning archetype, or a licensed reviewer	What that owner can read	Whatever that owner cannot read

A real citation that does not support the claim passes every check above. That layer stays with the owning archetype or a licensed reviewer.

Progress

Every sigil, its owner and where it stands. A built sigil's count comes from the card its command prints. A sigil that is not built has no card yet.

Sigil	Owner	Rules on the card	Status	Module
CITE.SYMBOL@3	truth	no rule list: its tiers decide	built	`dev_harness/sigils/cite_symbol.py, in SIGIL`
CODE.OPTIMIZE@1	coding	7 ruff rule families	built	`dev_harness/sigils/code_optimize.py, in SIGIL`
CODE.TYPE@1	coding	no rule list: every mypy and pyright error	built	`dev_harness/sigils/code_type.py, in SIGIL`
CODE.SECURE@1	coding	no rule list: every bandit and semgrep finding	built	`dev_harness/sigils/code_secure.py, in SIGIL`
CODE.REACH@1	coding	no rule list: every vulture item	built	`dev_harness/sigils/code_reach.py, in SIGIL`
CODE.LOOP@1	coding	19 ruff rules	built	`dev_harness/sigils/code_loop.py, in SIGIL`
CODE.IMPORT@1	coding	8 of 28 ruff rules	built	`dev_harness/sigils/code_import.py, in SIGIL`
CODE.COLLECTION@1	coding	34 of 39 ruff rules	built	`dev_harness/sigils/code_collection.py, in SIGIL`
CODE.FLOW@1	coding	39 of 56 ruff rules	built	`dev_harness/sigils/code_flow.py, in SIGIL`
CODE.VARIABLE@1	coding	54 of 89 ruff rules	built	`dev_harness/sigils/code_variable.py, in SIGIL`
TA.AUDIT@1	ta	no card yet	unbuilt	not in the tree
ROW.CLOSE@1	not named yet	no card yet	unbuilt	not in the tree

CODE.FLOW@1 reads if, try, with, return, raise, assert, pass and del. VARIABLE reads names, assignments and function arguments. TA.AUDIT@1 ranks one departure in one indicator module against its published formula. ROW.CLOSE@1 answers verified, fixed, completed, removed, it waits or NOT_GROUNDED for one row of an issue.

The trial pilot

The pilot asked one question: does a unit that loads the Coding sigils before it writes need less rework than the same unit without them? It ran three tasks, once without the sigils and once with them. It found no benefit from loading the sigils before writing. Rework split one task each way, the loaded arm cost 21 to 37 percent more tokens, and no replayed fault appeared in either arm. The sigils work as readers of finished code.

Task	Arm	Writes refused	Edits after the first accepted write	Final module	Tokens
T1	without	0	3	passed	93,625
T1	with	0	0	passed	113,483
T2	without	1	0	no module	110,167

Task	Arm	Writes refused	Edits after the first accepted write	Final module	Tokens
T2	with	2	0	no module	150,925
T3	without	0	0	passed	78,960
T3	with	0	2	passed	101,412

The full trial is not built. It would score four arms on drift, hallucination and validity.

Proposed, not present:

```
arms     Plain   the task alone
         Cited   the task and the name of its canon
         Bound   the task and a Sigil line, resolved, held and refused by the harness
         Dummy   the task and a mark with no bloc
scores   drift          the share of a task's repeats that give its most common rank
         hallucination  checkable claims per run that fail against the tree or the canon
         validity       the rank matches the key and the result passes the owning archetype
falls    the binding claim falls if Bound does not beat Cited on validity
```

Application archetypes

An archetype per professional program, such as Blender, Unity, Unreal or FreeCAD. Its sigils are professional approaches built from that program's own scripting calls. A prompt becomes a run of those approaches instead of a generated picture.

Each discipline of a game team is a mode. A discipline's practice is the same in every program: a lighting artist lights a scene by the same rules in Unreal, Unity or Blender. A mode states each discipline once, and every application archetype carries it out with its own program's calls. A mode is a family of sigils, such as `LIGHT.<approach>@1`, grounded in that discipline's published practice. A prompt names the role, that mode's sigils load, and the program's archetype carries each one out.

Proposed, not present:

```
ground    the scripting calls the program lists from inside itself, for a pinned version
modes     one per discipline of a game team, written once and carried out by every
          application archetype: WORLD, LIGHT, LOOK, ANIMATE, VFX, CINEMA, AUDIO, UI,
          QA, GAMEPLAY, each a family of sigils grounded in that discipline's published
          practice
owner     the archetype reopens the saved file with no window and reads the scene back
waits     built only if a bound sigil beats a cited one on validity in the full trial
status    unbuilt
```

The table lists the first ten modes. Each mode maps to the section of the Unreal Engine documentation that covers that work. Unity and Blender map the same modes to their own manuals' sections once their archetypes exist. FreeCAD is an engineering program, so its modes come from its own manual, not from this list.

Mode	The role on a game team	Unreal Engine documentation section
WORLD	level designer, environment artist	Building Virtual Worlds
LIGHT	lighting artist	Lighting the Environment

Mode	The role on a game team	Unreal Engine documentation section
LOOK	technical artist: materials and rendering	Designing Visuals, Rendering and Graphics
ANIMATE	animator	Animating Characters and Objects
VFX	visual effects artist	Creating Visual Effects
CINEMA	cinematic artist	Cinematics and Movie Making
AUDIO	sound designer	Working with Audio
UI	user interface designer	Creating User Interfaces
QA	quality and performance tester	Testing and Optimizing Your Content
GAMEPLAY	gameplay programmer	Programming with C++

Sources:

- Unreal Engine documentation
- Cinematics and Movie Making
- What is an environment artist
- Jobs in Unreal Engine: technical artist

The sigil pages

One page per built sigil. Each page states what the sigil reads, its tier 2 condition, its outputs and flags, the command that runs it, and its rules.

- CITE.SYMBOL@3
- CODE.OPTIMIZE@1
- CODE.TYPE@1
- CODE.SECURE@1
- CODE.REACH@1
- CODE.LOOP@1
- CODE.IMPORT@1
- CODE.COLLECTION@1
- CODE.FLOW@1
- CODE.VARIABLE@1

CITE.SYMBOL@3

Reference. CITE.SYMBOL@3 reads one code citation in a document and answers whether the cited file defines the cited name. A citation is a claim, so the Truth Archetype owns it. It is one sigil of the sigil project.

What it reads

One input line is one citation in the house form: a path, the separator ", in ", and a symbol. The path is relative to the root of the target tree and uses forward slashes. No model takes part.

Tier 1 accepts a relative path in forward slashes and name characters, and a dotted symbol of identifiers. Tier 2 accepts a path inside the target tree, of a file type that has a reader. Python is the only reader today. Tier 3 walks the path in exact letter case, parses the file with Python's ast module, and searches it for the definition.

A bare name resolves to a function or class at any depth, or to a module-level assignment. A dotted name resolves to that exact chain of nested definitions, and its last part may be an assignment in a class body. A comment, a docstring or any other string never counts as a definition.

`dev_harness/sigils/cite_symbol.py, in _defines`:

```
def _defines(tree: ast.Module, parts: list[str]) -> bool:
    """Report whether `tree` defines the symbol split into `parts`.

    A bare `name` needs a `_SCOPES` node at any depth, or a module-level assignment.
    """
    if len(parts) > 1:
        return _nested(tree, parts)
    name = parts[0]
    for node in ast.walk(tree):
        if isinstance(node, _SCOPES) and node.name == name:
            return True
    return any(name in _bound_names(stmt) for stmt in _scope_statements(tree))
```

Outputs and flags

Output	Flag elsewhere	Flag in application source
RESOLVED	PASS	PASS
NOT_GROUNDED	FIX	HOLD
REJECTED	BLOCK	BLOCK

RESOLVED means the file defines the name. NOT_GROUNDED reads HOLD when the file that holds the citation sits in application source, and FIX anywhere else. The follow-on rule for FIX is to find where the symbol lives now, rewrite the citation and read it again.

The command

The input file holds one citation per line. Its test pairs and sample tree sit in `harness_fixtures/sigil_cite_symbol/`.

```
python -m dev_harness.sigils.cite_symbol <file>
python -m dev_harness.sigils.cite_symbol --root <folder> <file>
python -m dev_harness.sigils.cite_symbol --gate <file>
python -m dev_harness.sigils.cite_symbol --card
```

Reasons on the card

The card holds no rule list. Its tiers decide every verdict, and each verdict carries one of the reasons below. One row per reason the card lists.

Tier	Verdict	Reason on the card
tier 1 data type	REJECTED	backslash
tier 1 data type	REJECTED	line number
tier 1 data type	REJECTED	not the form `<path>, in <symbol>`
tier 1 data type	REJECTED	drive letter
tier 1 data type	REJECTED	absolute path
tier 1 data type	REJECTED	path holds `..`
tier 2 content compatibility	REJECTED	path outside the target tree
tier 2 content compatibility	REJECTED	file type other than `.py`
tier 3 required processing functions	NOT_GROUNDED	file missing
tier 3 required processing functions	NOT_GROUNDED	letter case differs
tier 3 required processing functions	NOT_GROUNDED	file unreadable
tier 3 required processing functions	NOT_GROUNDED	file does not parse
tier 3 required processing functions	NOT_GROUNDED	nothing in the file defines it
tier 3 required processing functions	RESOLVED	the file defines it

Rules left off

The card holds no rule list, so it leaves no rule off.

CODE.OPTIMIZE@1

Reference. CODE.OPTIMIZE@1 reads one Python file against seven ruff rule families that name a faster or simpler form of the same code. The Coding Archetype owns it. It is one sigil of the sigil project.

What it reads

One input line names a Python file, or one function or class in that file. Tiers 1 and 2 come from the Coding Archetype's input module, and this sigil adds no condition to tier 2. Tier 3 runs ruff with the seven rule families on the card. The file goes to ruff on standard input, from the folder of the file. For a function or class, only the findings inside its lines count.

The ruff settings the tree keeps beside the file set the Python version, the preview rules and the complexity limit. The card records ruff 0.16.0, and the command prints the ruff version it ran.

`dev_harness/sigils/code_optimize.py, in FAMILIES` and `in _ruff_findings`:

```
FAMILIES = ("PERF", "SIM", "C4", "UP", "FURB", "PLR17", "C901")


def _ruff_findings(source: bytes, folder: Path) -> list[Finding]:
    """Return each `FAMILIES` `Finding` for `source` on stdin, run from `folder`."""
    return [
        Finding(row.code, row.name, row.line, fix_class(row.code))
        for row in check(source, folder, FAMILIES, preview=False)
    ]
```

Outputs and flags

Output	Flag elsewhere	Flag in application source
CLEAN	PASS	PASS
OPTIMIZABLE	OPTIMIZE	OPTIMIZE
REJECTED	BLOCK	BLOCK

CLEAN means no rule in the seven families fires. OPTIMIZABLE carries, for each finding, the rule id, the rule name, the line and the fix class ruff states: always, sometimes or none. OPTIMIZE never holds a merge. The fix class names what follows: an automatic fix is applied and the file read again, and a rule with no automatic fix becomes a unit of work.

The command

The input file holds one input per line. Its test pair and sample tree sit in `harness_fixtures/sigil_code_optimize/`.

```
python -m dev_harness.sigils.code_optimize <file>
python -m dev_harness.sigils.code_optimize --root <folder> <file>
python -m dev_harness.sigils.code_optimize --gate <file>
python -m dev_harness.sigils.code_optimize --card
```

Rule families

The card lists rule families, not single rules. One row per family. The linter name comes from `python -m ruff linter`. The two counts come from the rule set ruff resolves for the seven families with preview rules off and on, read with `python -m ruff check --show-settings` for ruff 0.16.0. Each finding carries its own rule's fix class.

Family	ruff linter	Rules, preview off	Rules, preview on
PERF	Perflint	6	6
SIM	flake8-simplify	30	30
C4	flake8-comprehensions	19	19
UP	pyupgrade	45	46
FURB	refurb	22	36

Family	ruff linter	Rules, preview off	Rules, preview on
PLR17	Pylint, refactor rules PLR1700 to PLR1799	9	11
C901	mccabe	1	1

The seven families hold 132 rules with preview rules off and 149 with them on. C901 is one rule, complex-structure, which ruff states has no automatic fix.

Families left off

The card holds no list of rules left off, so no family and no rule in one is left off.

CODE.TYPE@1

Reference. CODE.TYPE@1 reads one Python file with two type checkers, mypy and pyright. The Coding Archetype owns it. It is one sigil of the sigil project.

What it reads

One input line names a Python file, or one function or class in that file. Tiers 1 and 2 come from the Coding Archetype's input module, and this sigil adds no condition to tier 2. Tier 3 runs both type checkers from the root of the target tree, with the settings each tool finds there. Only errors count, and only those in the input file. For a function or class, only the errors inside its lines count.

A plugin or interpreter that the tree's own mypy or pyright settings name runs as code. Each tool finds imports from the root of the target tree and from the installed packages, never from the tree's own list of files to check.

`dev_harness/sigils/code_type.py, in _mypy_findings`:

```
def _mypy_findings(path: str, root: Path, file: Path) -> list[Finding]:
    """Return each mypy error in `file`, run on `path` from `root` with no cache."""
    with contextlib.chdir(root):
        stdout, stderr, status = mypy.api.run(
            ["--output", "json", "--cache-dir", os.devnull, path]
        )
    if status not in MYPY_FINISHED:
        raise ToolFault(f"mypy exited {status}: {tool_text(stderr or stdout)}")
    try:
        rows = [json.loads(line) for line in stdout.splitlines() if line.strip()]
        return [
            Finding("mypy", row["code"] or NO_CODE, row["line"])
            for row in rows
            if row["severity"] == "error" and _same_file(root / row["file"], file)
        ]
    except (ValueError, KeyError, TypeError) as exc:
        raise ToolFault(f"mypy printed an unreadable row: {exc}") from exc
```

Outputs and flags

Output	Flag elsewhere	Flag in application source
SOUND	PASS	PASS
UNSOUND	FIX	HOLD
REJECTED	BLOCK	BLOCK

SOUND means neither tool reports an error. UNSOUND carries, for each error, the tool, its error code and the line. UNSOUND reads HOLD when the file the input line names sits in application source. Anywhere else it reads FIX, and the merge proceeds.

The command

The input file holds one input per line. Its test pair and sample tree sit in `harness_fixtures/sigil_code_type/`. The sample tree holds the published examples of six mypy error codes: arg-type, return-value, attr-defined, union-attr, call-arg and index.

```
python -m dev_harness.sigils.code_type <file>
python -m dev_harness.sigils.code_type --root <folder> <file>
python -m dev_harness.sigils.code_type --gate <file>
python -m dev_harness.sigils.code_type --card
```

Tools and rules

The card holds no rule list. It names two tools, and every error either tool reports in the input file is an output. One row per tool, from the card's settings and tier 3. The command prints the versions it ran.

Tool	Version on the card	How the card runs it	What counts
mypy	2.1.0	`mypy.api.run` from the root of the target tree, with `--output json` and the null device as its cache folder	each error in the input file, with mypy's error code
pyright	1.1.411	`pyright.run` from the root of the target tree, with `--outputjson`	each error in the input file, with pyright's rule name

Rules left off

The card holds no rule list, so it leaves no rule off.

CODE.SECURE@1

Reference. CODE.SECURE@1 reads one Python file with two security scanners, bandit and semgrep, and reads each finding against the weakness id the scanner reports for it. The Coding Archetype owns it. It is one sigil of the sigil project.

What it reads

One input line names a Python file, or one function or class in that file. Tiers 1 and 2 come from the Coding Archetype's input module, and this sigil adds no condition to tier 2. Tier 3 runs both scanners with the rule sets the coding archetype runs. For a function or class, only the findings inside its lines count.

Each finding carries the CWE id that its own tool reports, or no CWE when the tool reports none. semgrep downloads both of its rule sets from its registry on each run. A tool that exits with any other code, reports an error, or did not read the file stops the run.

`dev_harness/sigils/code_secure.py, in BANDIT_SEVERITY` and `in SEMGREP_SEVERITY`, the grades each tool's severity takes:

```
SEVERITIES = ("low", "medium", "high")
BANDIT_SEVERITY = {"LOW": "low", "MEDIUM": "medium", "HIGH": "high"}
SEMGREP_SEVERITY = {"INFO": "low", "WARNING": "medium", "ERROR": "high"}
```

Outputs and flags

Output	Flag elsewhere	Flag in application source
CLEAN	PASS	PASS
WEAK	FIX	HOLD
REJECTED	BLOCK	BLOCK

CLEAN means neither tool reports a finding. WEAK carries, for each finding, the tool, its rule id, its CWE id, its severity and the line. WEAK reads HOLD only when the file the input line names sits in application source and a finding reaches the severity floor on the card, medium. A file whose findings are all low reads FIX, and the merge proceeds.

The command

The input file holds one input per line. Its test pair and sample tree sit in `harness_fixtures/sigil_code_secure/`. The sample tree holds bandit's own example files for six tests: B602, B506, B307, B324, B105 and B608.

```
python -m dev_harness.sigils.code_secure <file>
python -m dev_harness.sigils.code_secure --root <folder> <file>
python -m dev_harness.sigils.code_secure --gate <file>
python -m dev_harness.sigils.code_secure --card
```

Tools and rule sets

The card holds no rule list. It names two tools and their rule sets, and every finding either tool reports in the input file is an output. One row per tool, from the card's settings and tier 3. The command prints the versions it ran.

Tool	Version on the card	How the card runs it	Rule set
bandit	1.9.4	`python -P -m bandit -f json -q -`, with the file's bytes on standard input	bandit's default tests
semgrep	1.171.0	`semgrep scan` with `--json --quiet --no-git-ignore`, in an empty temporary folder that holds a copy of the file under its own name	`p/python` and `p/security-audit`

Rules left off

The card holds no rule list, so it leaves no rule off.

CODE.REACH@1

Reference. CODE.REACH@1 reads one Python file for code that nothing reaches, with vulture. The Coding Archetype owns it. It is one sigil of the sigil project.

What it reads

One input line names a Python file, or one function or class in that file. Tiers 1 and 2 come from the Coding Archetype's input module, and this sigil adds no condition to tier 2. Tier 3 runs vulture over every Python file in the target tree, from its root, and keeps only the items in the input file. A name that another file of the tree uses is therefore not called unused. For a function or class, only the items inside its lines count.

The sigil does not read the vulture table in the tree's own pyproject file, so no name, decorator or path listed there is ignored. A vulture run that exits with any other code, or cannot read a Python file in the tree, stops the run.

`dev_harness/sigils/code_reach.py, in SETTINGS`:

```
SETTINGS: dict[str, str] = {
    "separator": SEPARATOR,
    "vulture_version": VULTURE_BUILT,
    "confidence_floor": "100",
    "fault_file": "the file the input line names: UNREACHED reads HOLD when it sits "
    "in application_source and an item reaches confidence_floor",
}
```

Outputs and flags

Output	Flag elsewhere	Flag in application source
REACHED	PASS	PASS
UNREACHED	FIX	HOLD
REJECTED	BLOCK	BLOCK

REACHED means vulture reports no item in the file. UNREACHED carries, for each item, its kind, its name, vulture's confidence and the line. UNREACHED reads HOLD only when the file sits in application source and

an item reaches the confidence floor on the card, 100. Any other item reads FIX.

The follow-on rule for FIX is quarantine first: the unreached code moves aside in one commit and is never deleted.

The command

The input file holds one input per line. Its test pair and sample tree sit in `harness_fixtures/sigil_code_reach/`. The sample tree holds the dead-code example from vulture's own README.

```
python -m dev_harness.sigils.code_reach <file>
python -m dev_harness.sigils.code_reach --root <folder> <file>
python -m dev_harness.sigils.code_reach --gate <file>
python -m dev_harness.sigils.code_reach --card
```

Tools and rules

The card holds no rule list. It names one tool, and every item that tool reports in the input file is an output. One row, from the card's settings and tier 3. The command prints the version it ran.

Tool	Version on the card	How the card runs it	Floor that holds a merge
vulture	2.16	`Vulture().scavenge(['.'])` then `get_unused_code()`, as vulture's README shows it, from the root of the target tree	confidence 100

Rules left off

The card holds no rule list, so it leaves no rule off.

CODE.LOOP@1

Reference. CODE.LOOP@1 reads the loops in one Python file against 19 ruff rules. The Coding Archetype owns it. It is one sigil of the sigil project.

What it reads

One input line names a Python file, or one function or class in that file. Tiers 1 and 2 come from the Coding Archetype's input module. Tier 3 runs ruff with the 19 rules on the card and with preview rules on. The file goes to ruff on standard input, from the folder of the file. For a function or class, only the findings inside its lines count.

For each rule that fires, ruff's own rule page gives the rule's name, its fix class, and its published faulty and corrected forms. The card records ruff 0.16.0, and the command prints the ruff version it ran.

`dev_harness/sigils/code_loop.py, in _ruff_findings`:

```
def _ruff_findings(source: bytes, folder: Path) -> list[tuple[str, int]]:
    """Return each `RULES` code and line for `source` on stdin, run from `folder`."""
    return [(row.code, row.line) for row in check(source, folder, RULES, preview=True)]
```

Tier 2 accepts an input only when Python's ast module finds a for, async for, while, list, set or dictionary builder, or generator expression in the input's lines. An input that holds none is refused at tier 2.

`dev_harness/sigils/code_loop.py, in LOOP_NODES`:

```
LOOP_NODES = (
    ast.For,
    ast.AsyncFor,
    ast.While,
    ast.ListComp,
    ast.SetComp,
    ast.DictComp,
    ast.GeneratorExp,
)
```

Outputs and flags

Output	Flag elsewhere	Flag in application source
CLEAN	PASS	PASS
FINDINGS	FIX	HOLD
REJECTED	BLOCK	BLOCK

CLEAN means no rule on the list fires. FINDINGS carries, for each finding, the rule id, the rule name, the line, the fix class, and both published forms. FINDINGS reads HOLD only when the file sits in application source and PLE4703 fires. PLE4703 is the one pylint error-class rule on the list. Every other finding reads FIX, and the merge proceeds.

The command

The input file holds one input per line. Its test pair and sample tree sit in `harness_fixtures/sigil_code_loop/`.

```
python -m dev_harness.sigils.code_loop <file>
python -m dev_harness.sigils.code_loop --root <folder> <file>
python -m dev_harness.sigils.code_loop --gate <file>
python -m dev_harness.sigils.code_loop --card
```

Rules

Each line comes from `python -m ruff rule <code>` for ruff 0.16.0. The fix column is the fix class ruff states. A preview rule fires only with preview rules on.

Rule	ruff name	What it catches	Fix	Preview
ASYNC110	async-busy-wait	Checks for the use of an async sleep function in a `while` loop.	none	no
B035	static-key-dict-comprehension	Checks for dictionary comprehensions that use a static key, like a string literal or a variable defined outside the comprehension.	none	no
C416	unnecessary-comprehension	Checks for unnecessary dict, list, and set comprehension.	always	no
C419	unnecessary-comprehension-in-call	Checks for unnecessary list or set comprehensions passed to builtin functions that take an iterable.	sometimes	no
C420	unnecessary-dict-comprehension-for-iterable	Checks for unnecessary dict comprehension when creating a dictionary from an iterable.	sometimes	no
SIM911	zip-dict-keys-and-values	Checks for use of `zip()` to iterate over keys and values of a dictionary at once.	always	no
PERF102	incorrect-dict-iterator	Checks for uses of `dict.items()` that discard either the key or the value when iterating over the dictionary.	always	no
PERF402	manual-list-copy	Checks for `for` loops that append every item of an iterable to a list, which can be replaced with a call to `list`.	none	no
PLC0206	dict-index-missing-items	Checks for dictionary iterations that extract the dictionary value via explicit indexing, instead of using `.items()`.	none	no
PLE4703	modified-iterating-set	Checks for loops in which a `set` is modified during iteration.	always	yes
PLR1733	unnecessary-dict-index-lookup	Checks for key-based dict accesses during `.items()` iterations.	always	no
PLR1736	unnecessary-list-index-lookup	Checks for index-based list accesses during `enumerate` iterations.	always	no
PLW0120	useless-else-on-loop	Checks for `else` clauses on loops without a `break` statement.	sometimes	no
UP028	yield-in-for-loop	Checks for `for` loops that can be replaced with `yield from` expressions.	sometimes	no
FURB122	for-loop-writes	Checks for the use of `IOBase.write` in a for loop.	always	no
FURB129	readlines-in-for	Checks for uses of `readlines()` when iterating over a file line-by-line.	always	no
FURB140	reimplemented-starmap	Checks for generator expressions, list and set comprehensions that can be replaced with `itertools.starmap`.	sometimes	yes

Rule	ruff name	What it catches	Fix	Preview
FURB142	for-loop-set-mutations	Checks for code that updates a set with the contents of an iterable by using a `for` loop to call `.add()` or `.discard()` on each element separately.	always	yes
RUF006	asyncio-dangling-task	Checks for `asyncio.create_task` and `asyncio.ensure_future` calls that do not store a reference to the returned result.	none	no

Rules left off

The card holds no list of rules left off, so every rule this sigil reads is in the table above.

CODE.IMPORT@1

Reference. CODE.IMPORT@1 reads the import statements in one Python file against 8 ruff rules. The Coding Archetype owns it. It is one sigil of the sigil project.

What it reads

One input line names a Python file, or one function or class in that file. Tiers 1 and 2 come from the Coding Archetype's input module. Tier 3 runs ruff with the 8 rules on the card. No rule on the list is a preview rule, so ruff runs with preview rules off. The file goes to ruff on standard input, from the folder of the file. For a function or class, only the findings inside its lines count.

For each rule that fires, ruff's own rule page gives the rule's name, its fix class, and its published faulty and corrected forms. The card records ruff 0.16.0, and the command prints the ruff version it ran.

`dev_harness/sigils/code_import.py, in _ruff_findings`:

```
def _ruff_findings(source: bytes, folder: Path) -> list[tuple[str, int]]:
    """Return each `RULES` code and line for `source` on stdin, run from `folder`."""
    return [(row.code, row.line) for row in check(source, folder, RULES, preview=False)]
```

Tier 2 accepts an input only when Python's ast module finds an import or from-import statement in the input's lines. An input that holds none is refused at tier 2.

`dev_harness/sigils/code_import.py, in IMPORT_NODES`:

```
IMPORT_NODES = (ast.Import, ast.ImportFrom)
NO_IMPORT = "no import or from-import statement"
```

Outputs and flags

Output	Flag elsewhere	Flag in application source
CLEAN	PASS	PASS
FINDINGS	FIX	FIX
REJECTED	BLOCK	BLOCK

CLEAN means no rule on the list fires. FINDINGS carries, for each finding, the rule id, the rule name, the line, the fix class, and both published forms. A finding holds a merge only when it is a pylint error-class rule or the pyflakes undefined-name rule F821. No rule on the list is either, so the holding list on the card is empty and every finding reads FIX.

`dev_harness/sigils/code_import.py, in HOLDING`:

```
HOLDING = tuple(
    code for code in RULES if code.startswith(ERROR_CLASS) or code == UNDEFINED_NAME
)
```

The command

The input file holds one input per line. Its test pair and sample tree sit in `harness_fixtures/sigil_code_import/`.

```
python -m dev_harness.sigils.code_import <file>
python -m dev_harness.sigils.code_import --root <folder> <file>
python -m dev_harness.sigils.code_import --gate <file>
python -m dev_harness.sigils.code_import --card
```

Rules

Each line comes from `python -m ruff rule <code>` for ruff 0.16.0. The fix column is the fix class ruff states.

Rule	ruff name	What it catches	Fix	Preview
A004	builtin-import-shadowing	Checks for imports that use the same names as builtins.	none	no
TC001	typing-only-first-party-import	Checks for first-party imports that are only used for type annotations, but aren't defined in a type-checking block.	sometimes	no
TC003	typing-only-standard-library-import	Checks for standard library imports that are only used for type annotations, but aren't defined in a type-checking block.	sometimes	no
TC004	runtime-import-in-type-checking-block	Checks for imports that are required at runtime but are only defined in type-checking blocks.	sometimes	no
F403	undefined-local-with-import-star	Checks for the use of wildcard imports.	none	no

Rule	ruff name	What it catches	Fix	Preview
PLC0415	import-outside-top-level	Checks for `import` statements outside of a module's top-level scope, such as within a function or class definition.	none	no
UP010	unnecessary-future-import	Checks for unnecessary `__future__` imports.	always	no
UP029	unnecessary-builtin-import	Checks for imports of Python 3 builtins from Python 2 compatibility shims such as `python-future` and `six`.	always	no

Rules left off

The card leaves 20 of the 28 import rules off. For each one, the write gate refused that rule's faulty or corrected sample, because the coding archetype grades the named finding high. The reason column is the card's own text.

Rule	ruff name	Reason on the card
ICN001	unconventional-import-alias	faulty: vulture unused import 'pandas'; mypy import-untyped pandas
TID252	relative-imports	faulty: vulture unused import 'foo'
TC002	typing-only-third-party-import	faulty: mypy import-untyped pandas
N811	constant-imported-as-non-constant	faulty: vulture unused import 'ConstantValue'
N812	lowercase-imported-as-non-lowercase	faulty: vulture unused import 'MyClassName'
N813	camelcase-imported-as-lowercase	faulty: vulture unused import 'myclassname'
N814	camelcase-imported-as-constant	faulty: vulture unused import 'MY_CLASS_NAME'
N817	camelcase-imported-as-acronym	faulty: vulture unused import 'MCN'
E401	multiple-imports-on-one-line	faulty: vulture unused import 'os'
E402	module-import-not-at-top-of-file	faulty: vulture unused import 'os' and 'x'
F401	unused-import	faulty: vulture unused import 'np'
F404	late-future-import	faulty: vulture unused import 'Path'
F406	undefined-local-with-nested-import-star-usage	faulty: pyright, wildcard import not allowed within a class or function
F811	redefined-while-unused	faulty: vulture unused import 'foo' and 'bar'
PLC0414	useless-import-alias	corrected: vulture unused import 'np'
PLC2403	non-ascii-import-name	faulty: vulture unused import 'bár'
PLR0402	manual-from-import	faulty: vulture unused import 'futures'
UP023	deprecated-c-element-tree	faulty: vulture unused import 'ET'; semgrep use-defused-xml
UP026	deprecated-mock-import	faulty: vulture unused import 'mock'
UP035	deprecated-import	faulty: vulture unused import 'Sequence'

CODE.COLLECTION@1

Reference. CODE.COLLECTION@1 reads the tuples, lists, sets and dictionaries in one Python file against 34 ruff rules. The Coding Archetype owns it. It is one sigil of the sigil project.

What it reads

One input line names a Python file, or one function or class in that file. Tiers 1 and 2 come from the Coding Archetype's input module. Tier 3 runs ruff with the 34 rules on the card. Three of them are preview rules, so ruff runs with preview rules on. The file goes to ruff on standard input, from the folder of the file. For a function or class, only the findings inside its lines count.

For each rule that fires, ruff's own rule page gives the rule's name, its fix class, and its published faulty and corrected forms. The card records ruff 0.16.0, and the command prints the ruff version it ran.

`dev_harness/sigils/code_collection.py, in _ruff_findings`:

```
def _ruff_findings(source: bytes, folder: Path) -> list[tuple[str, int]]:
    """Return each `RULES` code and line for `source` on stdin, run from `folder`."""
    rows = check(source, folder, RULES, preview=bool(PREVIEW))
    return [(row.code, row.line) for row in rows]
```

Tier 2 accepts an input only when Python's ast module finds a collection in the input's lines: a tuple or list written out and read as a value, a set or dictionary written out, a list, set or dictionary comprehension, or a call to tuple, list, set or dict by that name. An input that holds none is refused at tier 2.

`dev_harness/sigils/code_collection.py, in _is_collection`:

```
def _is_collection(node: ast.expr) -> bool:
    """Report whether `node` writes out, builds or calls a tuple, list, set or dict."""
    if isinstance(node, DISPLAY_NODES):
        return isinstance(node.ctx, ast.Load)
    if isinstance(node, ast.Call):
        return isinstance(node.func, ast.Name) and node.func.id in BUILDERS
    return isinstance(node, COLLECTION_NODES)
```

Outputs and flags

Output	Flag elsewhere	Flag in application source
CLEAN	PASS	PASS
FINDINGS	FIX	HOLD
REJECTED	BLOCK	BLOCK

CLEAN means no rule on the list fires. FINDINGS carries, for each finding, the rule id, the rule name, the line, the fix class, and both published forms. FINDINGS reads HOLD only when the file sits in application source and F822 fires. F822 is the pyflakes rule for an undefined name in a module's export list. Every other finding reads FIX, and the merge proceeds.

The command

The input file holds one input per line. Its test pair and sample tree sit in `harness_fixtures/sigil_code_collection/`.

```
python -m dev_harness.sigils.code_collection <file>
python -m dev_harness.sigils.code_collection --root <folder> <file>
python -m dev_harness.sigils.code_collection --gate <file>
python -m dev_harness.sigils.code_collection --card
```

Rules

Each line comes from `python -m ruff rule <code>` for ruff 0.16.0. The fix column is the fix class ruff states. A preview rule fires only with preview rules on.

Rule	ruff name	What it catches	Fix	Preview
S607	start-process-with-partial-path	Checks for the starting of a process with a partial executable path.	none	no
B006	mutable-argument-default	Checks for uses of mutable objects as function argument defaults.	sometimes	no
B013	redundant-tuple-in-exception-handler	Checks for single-element tuples in exception handlers (e.g., `except (ValueError,):`).	always	no
B033	duplicate-value	Checks for set literals that contain duplicate items.	sometimes	no
B039	mutable-contextvar-default	Checks for uses of mutable objects as `ContextVar` defaults.	none	no
COM812	missing-trailing-comma	Checks for the absence of trailing commas.	always	no
COM818	trailing-comma-on-bare-tuple	Checks for the presence of trailing commas on bare (i.e., unparenthesized) tuples.	none	no
COM819	prohibited-trailing-comma	Checks for the presence of prohibited trailing commas.	always	no
C400	unnecessary-generator-list	Checks for unnecessary generators that can be rewritten as list comprehensions (or with `list()` directly).	always	no
C401	unnecessary-generator-set	Checks for unnecessary generators that can be rewritten as set comprehensions (or with `set()` directly).	always	no
C402	unnecessary-generator-dict	Checks for unnecessary generators that can be rewritten as dict comprehensions.	always	no
C403	unnecessary-list-comprehension-set	Checks for unnecessary list comprehensions.	always	no
C404	unnecessary-list-comprehension-dict	Checks for unnecessary list comprehensions.	always	no

Rule	ruff name	What it catches	Fix	Preview
C405	unnecessary-literal-set	Checks for `set()` calls that take unnecessary list or tuple literals as arguments.	always	no
C406	unnecessary-literal-dict	Checks for unnecessary list or tuple literals.	always	no
C408	unnecessary-collection-call	Checks for unnecessary `dict()`, `list()` or `tuple()` calls that can be rewritten as empty literals.	always	no
C409	unnecessary-literal-within-tuple-call	Checks for `tuple` calls that take unnecessary list or tuple literals as arguments.	sometimes	no
C410	unnecessary-literal-within-list-call	Checks for `list()` calls that take unnecessary list or tuple literals as arguments.	always	no
C411	unnecessary-list-call	Checks for unnecessary `list()` calls around list comprehensions.	sometimes	no
C414	unnecessary-double-cast-or-process	Checks for unnecessary `list()`, `reversed()`, `set()`, `sorted()`, and `tuple()` call within `list()`, `set()`, `sorted()`, and `tuple()` calls.	always	no
C418	unnecessary-literal-within-dict-call	Checks for `dict()` calls that take unnecessary dict literals or dict comprehensions as arguments.	sometimes	no
G101	logging-extra-attr-clash	Checks for `extra` keywords in logging statements that clash with `LogRecord` attributes.	none	no
PIE800	unnecessary-spread	Checks for unnecessary dictionary unpacking operators (`**`).	sometimes	no
PERF101	unnecessary-list-cast	Checks for explicit casts to `list` on for-loop iterables.	always	no
F601	multi-value-repeated-key-literal	Checks for dictionary literals that associate multiple values with the same key.	sometimes	no
F822	undefined-export	Checks for undefined names in `__all__`.	none	no
PLC0208	iteration-over-set	Checks for iteration over a `set` literal in which each element is a literal.	always	no
PLR6201	literal-membership	Checks for membership tests on `list` and `tuple` literals.	always	yes
PLW0244	redefined-slots-in-subclass	Checks for a re-defined slot in a subclass.	none	yes
RUF022	unsorted-dunder-all	Checks for `__all__` definitions that are not ordered according to an "isort-style" sort.	sometimes	no
RUF023	unsorted-dunder-slots	Checks for `__slots__` definitions that are not ordered according to a natural sort.	sometimes	no
RUF031	incorrectly-parenthesized-tuple-in-subscript	Checks for consistent style regarding whether nonempty tuples in subscripts are parenthesized.	always	yes

Rule	ruff name	What it catches	Fix	Preview
RUF048	map-int-version-parsing	Checks for calls of the form `map(int, __version__.split("."))`.	none	no
RUF068	duplicate-entry-in-dunder-all	Detects duplicate elements in `__all__` definitions.	sometimes	no

Rules left off

The card leaves 5 of the 39 collection rules off. For each one, the write gate refused that rule's faulty or corrected sample, because the coding archetype grades the named finding high. The reason column is the card's own text.

Rule	ruff name	Reason on the card
S104	hardcoded-bind-all-interfaces	faulty: ruff S104 possible binding to all interfaces
S609	unix-command-wildcard-injection	faulty: bandit B602 and B609, ruff S602 and S609, semgrep subprocess-shell-true
F634	if-tuple	faulty: vulture redundant if-condition
RUF008	mutable-dataclass-default	corrected: mypy no-redef and pyright reportRedeclaration, class 'A'
RUF012	mutable-class-default	corrected: mypy no-redef and pyright reportRedeclaration, class 'A'

CODE.FLOW@1

Reference. CODE.FLOW@1 reads the if, try, with, return, raise, assert, pass and del statements in one Python file against 39 ruff rules. The Coding Archetype owns it. It is one sigil of the sigil project.

What it reads

One input line names a Python file, or one function or class in that file. Tiers 1 and 2 come from the Coding Archetype's input module. Tier 3 runs ruff with the 39 rules on the card. Seven of them are preview rules, so ruff runs with preview rules on. The file goes to ruff on standard input, from the folder of the file. For a function or class, only the findings inside its lines count.

For each rule that fires, ruff's own rule page gives the rule's name, its fix class, and its published faulty and corrected forms. The card records ruff 0.16.0, and the command prints the ruff version it ran.

`dev_harness/sigils/code_flow.py, in _ruff_findings`:

```
def _ruff_findings(source: bytes, folder: Path) -> list[tuple[str, int]]:
    """Return each `RULES` code and line for `source` on stdin, run from `folder`."""
    rows = check(source, folder, RULES, preview=bool(PREVIEW))
    return [(row.code, row.line) for row in rows]
```

Tier 2 accepts an input only when Python's ast module finds a flow statement in the input's lines: an if, try, try star, with, async with, return, raise, assert, pass or del statement. An input that holds none is refused at tier 2.

`dev_harness/sigils/code_flow.py, in _content_fault`:

```
def _content_fault(text: str, root: Path) -> str | None:
    """Return the tier 2 fault of `text`: `locate`'s, else `NO_FLOW`."""
    located = locate(text, root)
    if isinstance(located, str):
        return located
    _, source, (first, last) = located
    has_flow = any(
        isinstance(node, FLOW_NODES) and first <= node.lineno <= last
        for node in ast.walk(ast.parse(source))
    )
    return None if has_flow else NO_FLOW
```

Outputs and flags

Output	Flag elsewhere	Flag in application source
CLEAN	PASS	PASS
FINDINGS	FIX	HOLD
REJECTED	BLOCK	BLOCK

CLEAN means no rule on the list fires. FINDINGS carries, for each finding, the rule id, the rule name, the line, the fix class, and both published forms. FINDINGS reads HOLD only when the file sits in application source and PLE0101, PLE1205 or PLE1206 fires. These are the pylint error rules for a value returned from an init method, and for too many or too few logging arguments. Every other finding reads FIX, and the merge proceeds.

The command

The input file holds one input per line. Its test pair and sample tree sit in `harness_fixtures/sigil_code_flow/`.

```
python -m dev_harness.sigils.code_flow <file>
python -m dev_harness.sigils.code_flow --root <folder> <file>
python -m dev_harness.sigils.code_flow --gate <file>
python -m dev_harness.sigils.code_flow --card
```

Rules

Each line comes from `python -m ruff rule <code>` for ruff 0.16.0. The fix column is the fix class ruff states. A preview rule fires only with preview rules on.

Rule	ruff name	What it catches	Fix	Preview
ASYNC100	cancel-scope-no-checkpoint	Checks for timeout context managers which do not contain a checkpoint.	none	no
B029	except-with-empty-tuple	Checks for exception handlers that catch an empty tuple.	none	no
B901	return-in-generator	Checks for `return {value}` statements in functions that also contain `yield` or `yield from` statements.	none	yes

Rule	ruff name	What it catches	Fix	Preview
G201	logging-exc-info	Checks for uses of `logging.error` that pass `exc_info=True`.	none	no
G202	logging-redundant-exc-info	Checks for redundant `exc_info` keyword arguments in logging statements.	none	no
RET501	unnecessary-return-none	Checks for the presence of a `return None` statement when `None` is the only possible return value.	always	no
RET502	implicit-return-value	Checks for the presence of a `return` statement with no explicit value, for functions that return non-`None` values elsewhere.	always	no
RET505	superfluous-else-return	Checks for `else` statements with a `return` statement in the preceding `if` block.	sometimes	no
RET506	superfluous-else-raise	Checks for `else` statements with a `raise` statement in the preceding `if` block.	sometimes	no
RET507	superfluous-else-continue	Checks for `else` statements with a `continue` statement in the preceding `if` block.	sometimes	no
RET508	superfluous-else-break	Checks for `else` statements with a `break` statement in the preceding `if` block.	sometimes	no
SIM102	collapsible-if	Checks for nested `if` statements that can be collapsed into a single `if` statement.	sometimes	no
SIM105	suppressible-exception	Checks for `try-except-pass` blocks that can be replaced with the `contextlib.suppress` context manager.	sometimes	no
SIM114	if-with-same-arms	Checks for `if` branches with identical arm bodies.	sometimes	no
SIM116	if-else-block-instead-of-dict-lookup	Checks for three or more consecutive if-statements with direct returns	none	no
SIM117	multiple-with-statements	Checks for the unnecessary nesting of multiple consecutive context managers.	sometimes	no
TC005	empty-type-checking-block	Checks for an empty type-checking block.	always	no
PERF401	manual-list-comprehension	Checks for `for` loops that can be replaced by a list comprehension.	sometimes	no
E701	multiple-statements-on-one-line-colon	Checks for compound statements (multiple statements on the same line).	none	no
E722	bare-except	Checks for bare `except` catches in `try-except` statements.	none	no
PLE0101	return-in-init	Checks for `__init__` methods that return values.	none	no
PLE1205	logging-too-many-args	Checks for too many positional arguments for a `logging` format string.	none	no
PLE1206	logging-too-few-args	Checks for too few positional arguments for a `logging` format string.	none	no
PLR1708	stop-iteration-return	Checks for explicit `raise StopIteration` in generator functions.	none	no

Rule	ruff name	What it catches	Fix	Preview
PLR1711	useless-return	Checks for functions that end with an unnecessary `return` or `return None`, and contain no other `return` statements.	always	no
PLR1730	if-stmt-min-max	Checks for `if` statements that can be replaced with `min()` or `max()` calls.	sometimes	no
PLR5501	collapsible-else-if	Checks for `else` blocks that consist of a single `if` statement.	sometimes	no
FURB101	read-whole-file	Checks for uses of `open` and `read` that can be replaced by `pathlib` methods, like `Path.read_text` and `Path.read_bytes`.	sometimes	yes
FURB103	write-whole-file	Checks for uses of `open` and `write` that can be replaced by `pathlib` methods, like `Path.write_text` and `Path.write_bytes`.	sometimes	yes
FURB131	delete-full-slice	Checks for `del` statements that delete the entire slice of a list or dictionary.	sometimes	yes
FURB132	check-and-remove-from-set	Checks for uses of `set.remove` that can be replaced with `set.discard`.	always	no
RUF047	needless-else	Checks for `else` clauses that only contains `pass` and `...` statements.	always	yes
RUF050	unnecessary-if	Checks for `if` statements (without `elif` or `else` branches) where the body contains only `pass` or `...`	always	yes
RUF072	useless-finally	Checks for `finally` clauses that only contain `pass` or `...` statements	sometimes	yes
TRY004	type-check-without-type-error	Checks for type checks that do not raise `TypeError`.	none	no
TRY203	useless-try-except	Checks for immediate uses of `raise` within exception handlers.	none	no
TRY300	try-consider-else	Checks for `return` statements in `try` blocks.	none	no
TRY301	raise-within-try	Checks for `raise` statements within `try` blocks. The only `raises` caught are those that throw exceptions caught by the `try` statement itself.	none	no
TRY400	error-instead-of-exception	Checks for uses of `logging.error` instead of `logging.exception` when logging an exception.	sometimes	no

Rules left off

The card leaves 17 of the 56 flow rules off. A reason that starts with faulty or corrected names the sample. A tool and its finding mean the write gate refused that sample, because the coding archetype grades the finding high. Reads names what ruff reads on that published form. The reason column is the card's own text.

Rule	ruff name	Reason on the card
S101	assert	faulty: bandit B101 and ruff S101 use of assert
S110	try-except-pass	faulty: ruff S110 try-except-pass
S112	try-except-continue	faulty: ruff S112 try-except-continue
B012	jump-statement-in-finally	faulty: pyright, a return cannot exit a finally block
B904	raise-without-from-inside-except	faulty: vulture unreachable else block
PIE790	unnecessary-placeholder	corrected: reads PIE790
SIM103	needless-bool	corrected: reads SIM103
SIM107	return-in-try-except-finally	faulty: pyright, a return cannot exit a finally block
SIM108	if-else-block-instead-of-if-exp	corrected: reads SIM108
SIM401	if-else-block-instead-of-dict-get	corrected: reads SIM401
PERF203	try-except-in-loop	faulty: reads no PERF203, which ruff reads only below Python 3.11
F631	assert-tuple	faulty: bandit B101 and ruff S101 use of assert
F707	default-except-not-last	faulty: pyright, a named except clause after a catch-all except clause
PLE0704	misplaced-bare-raise	faulty: mypy return and pyright reportReturnType, missing return
PLR1712	swap-with-temporary-variable	faulty: bandit B101 and ruff S101 use of assert
PLW0717	too-many-statements-in-try-clause	faulty: ruff S311 pseudo-random generator
RUF051	if-key-in-dict-del	faulty and corrected: mypy var-annotated, dictionary

CODE.VARIABLE@1

Reference. CODE.VARIABLE@1 reads the names, assignments and function arguments in one Python file against 54 ruff rules. The Coding Archetype owns it. It is one sigil of the sigil project.

What it reads

One input line names a Python file, or one function or class in that file. Tiers 1 and 2 come from the Coding Archetype's input module. Tier 3 runs ruff with the 54 rules on the card. Five of them are preview rules, so ruff runs with preview rules on. The file goes to ruff on standard input, from the folder of the file. For a function or class, only the findings inside its lines count.

For each rule that fires, ruff's own rule page gives the rule's name, its fix class, and its published faulty and corrected forms. The card records ruff 0.16.0, and the command prints the ruff version it ran.

`dev_harness/sigils/code_variable.py, in _ruff_findings`:

```
def _ruff_findings(source: bytes, folder: Path) -> list[tuple[str, int]]:
    """Return each `RULES` code and line for `source` on stdin, run from `folder`."""
    rows = check(source, folder, RULES, preview=bool(PREVIEW))
    return [(row.code, row.line) for row in rows]
```

Tier 2 accepts an input only when Python's ast module finds a binding in the input's lines: an assignment, an annotated or augmented assignment, a named expression, a function argument, or a global or nonlocal

statement. An input that holds none is refused at tier 2.

`dev_harness/sigils/code_variable.py, in _content_fault`:

```
def _content_fault(text: str, root: Path) -> str | None:
    """Return the tier 2 fault of `text`: `locate`'s, else `NO_BINDING`."""
    located = locate(text, root)
    if isinstance(located, str):
        return located
    _, source, (first, last) = located
    has_binding = any(
        isinstance(node, BINDING_NODES) and first <= node.lineno <= last
        for node in ast.walk(ast.parse(source))
    )
    return None if has_binding else NO_BINDING
```

Outputs and flags

Output	Flag elsewhere	Flag in application source
CLEAN	PASS	PASS
FINDINGS	FIX	HOLD
REJECTED	BLOCK	BLOCK

CLEAN means no rule on the list fires. FINDINGS carries, for each finding, the rule id, the rule name, the line, the fix class, and both published forms. FINDINGS reads HOLD only when the file sits in application source and PLE0118, PLE0237 or PLE0605 fires. These are the pylint error rules for a name used before its global statement, an assignment to an attribute that the class slots do not list, and an export list that is not a list or a tuple. Every other finding reads FIX, and the merge proceeds.

F821 and F823 are the pyflakes rules for an undefined name and an undefined local. The card lists both in its undefined_names setting, as rules that would hold a merge. Both are left off the card, so neither can fire.

The command

The input file holds one input per line. Its test pair and sample tree sit in `harness_fixtures/sigil_code_variable/`.

```
python -m dev_harness.sigils.code_variable <file>
python -m dev_harness.sigils.code_variable --root <folder> <file>
python -m dev_harness.sigils.code_variable --gate <file>
python -m dev_harness.sigils.code_variable --card
```

Rules

Each line comes from `python -m ruff rule <code>` for ruff 0.16.0. The fix column is the fix class ruff states. A preview rule fires only with preview rules on.

Rule	ruff name	What it catches	Fix	Preview
ASYNC230	blocking-open-call-in-async-function	Checks that async functions do not open files with blocking methods like `open`.	none	no
ASYNC250	blocking-input-in-async-function	Checks that async functions do not contain blocking usage of input from user.	none	no
BLE001	blind-except	Checks for `except` clauses that catch all exceptions. This includes `except BaseException` and `except Exception`.	none	no
B003	assignment-to-os-environ	Checks for assignments to `os.environ`.	none	no
B007	unused-loop-control-variable	Checks for unused variables in loops (e.g., `for` and `while` statements).	sometimes	no
B020	loop-variable-overrides-iterator	Checks for loop control variables that override the loop iterable.	none	no
B023	function-uses-loop-variable	Checks for function definitions that use a loop variable.	none	no
B031	reuse-of-groupby-generator	Checks for multiple usage of the generator returned from `itertools.groupby()`.	none	no
A001	builtin-variable-shadowing	Checks for variable (and function) assignments that use the same names as builtins.	none	no
A002	builtin-argument-shadowing	Checks for function arguments that use the same names as builtins.	none	no
LOG002	invalid-get-logger-argument	Checks for any usage of `__cached__` and `__file__` as an argument to `logging.getLogger()`.	sometimes	no
PIE794	duplicate-class-field-definition	Checks for duplicate field definitions in classes.	always	no
PIE796	non-unique-enums	Checks for enums that contain duplicate values.	none	no
RET504	unnecessary-assign	Checks for variable assignments that immediately precede a `return` of the assigned variable.	always	no
SIM113	enumerate-for-loop	Checks for `for` loops with explicit loop-index variables that can be replaced with `enumerate()`.	none	no
SIM115	open-file-with-context-handler	Checks for cases where files are opened (e.g., using the builtin `open()` function) without using a context manager.	none	no
N806	non-lowercase-variable-in-function	Checks for the use of non-lowercase variable names in functions.	none	no
N815	mixed-case-variable-in-class-scope	Checks for class variable names that follow the `mixedCase` convention.	none	no
N816	mixed-case-variable-in-global-scope	Checks for global variable names that follow the `mixedCase` convention.	none	no

Rule	ruff name	What it catches	Fix	Preview
PERF403	manual-dict-comprehension	Checks for `for` loops that can be replaced by a dictionary comprehension.	sometimes	no
E731	lambda-assignment	Checks for lambda expressions which are assigned to a variable.	sometimes	no
E741	ambiguous-variable-name	Checks for the use of the characters 'l', 'O', or 'I' as variable names.	none	no
F405	undefined-local-with-import-star-usage	Checks for names that might be undefined, but may also be defined in a wildcard import.	none	no
F602	multi-value-repeated-key-variable	Checks for dictionary keys that are repeated with different values.	sometimes	no
F841	unused-variable	Checks for the presence of unused variables in function scopes.	sometimes	no
F901	raise-not-implemented	Checks for `raise` statements that raise `NotImplemented`.	sometimes	no
PLC0105	type-name-incorrect-variance	Checks for type names that do not match the variance of their associated type parameter.	none	no
PLC0131	type-bivariance	Checks for `TypeVar` and `ParamSpec` definitions in which the type is both covariant and contravariant.	none	no
PLC0205	single-string-slots	Checks for single strings assigned to `__slots__`.	none	no
PLC2401	non-ascii-name	Checks for the use of non-ASCII characters in variable names.	none	no
PLE0118	load-before-global-declaration	Checks for uses of names that are declared as `global` prior to the relevant `global` declaration.	none	no
PLE0237	non-slot-assignment	Checks for assignments to attributes that are not defined in `__slots__`.	none	no
PLE0605	invalid-all-format	Checks for invalid assignments to `__all__`.	none	no
PLR1704	redefined-argument-from-local	Checks for variables defined in `for`, `try`, `with` statements that redefine function parameters.	none	no
PLR6104	non-augmented-assignment	Checks for assignments that can be replaced with augmented assignment statements.	always	yes
PLW0127	self-assigning-variable	Checks for self-assignment of variables.	none	no
PLW0128	redeclared-assigned-name	Checks for declared assignments to the same variable multiple times in the same assignment.	none	no
PLW0131	named-expr-without-context	Checks for uses of named expressions (e.g., `a := 42`) that can be replaced by regular assignment statements (e.g., `a = 42`).	none	no
PLW0602	global-variable-not-assigned	Checks for `global` variables that are not assigned a value in the current scope.	none	no

Rule	ruff name	What it catches	Fix	Preview
PLW0603	global-statement	Checks for the use of `global` statements to update identifiers.	none	no
PLW0642	self-or-cls-assignment	Checks for assignment of `self` and `cls` in instance and class methods respectively.	none	no
UP001	useless-metaclass-type	Checks for the use of `__metaclass__ = type` in class definitions.	always	no
UP006	non-pep585-annotation	Checks for the use of generics that can be replaced with standard library variants based on [PEP 585].	sometimes	no
UP013	convert-typed-dict-functional-to-class	Checks for `TypedDict` declarations that use functional syntax.	sometimes	no
UP014	convert-named-tuple-functional-to-class	Checks for `NamedTuple` declarations that use functional syntax.	sometimes	no
UP019	typing-text-str-alias	Checks for uses of `typing.Text` and `typing_extensions.Text`.	sometimes	no
FURB148	unnecessary-enumerate	Checks for uses of `enumerate` that discard either the index or the value when iterating over a sequence.	sometimes	yes
FURB187	list-reverse-copy	Checks for list reversals that can be performed in-place in lieu of creating a new list.	always	no
FURB189	subclass-builtin	Checks for subclasses of `dict`, `list` or `str`.	always	yes
RUF007	zip-instead-of-pairwise	Checks for use of `zip()` to iterate over successive pairs of elements.	sometimes	no
RUF052	used-dummy-variable	Checks for "dummy variables" (variables that are named as if to indicate they are unused) that are in fact used.	sometimes	yes
RUF059	unused-unpacked-variable	Checks for the presence of unused variables in unpacked assignments.	sometimes	no
RUF070	unnecessary-assign-before-yield	Checks for variable assignments that immediately precede a `yield` (or `yield from`) of the assigned variable, where the variable is not referenced anywhere else.	always	yes
TRY401	verbose-log-message	Checks for excessive logging of exception objects.	none	no

Rules left off

The card leaves 35 of the 89 variable rules off. A reason that starts with faulty or corrected names the sample. A tool and its finding mean the write gate refused that sample, because the coding archetype grades the finding high. Reads names what ruff reads on that published form. Refused at tier 2 means the published faulty form holds no binding that tier 2 accepts, so the sigil cannot read it. The reason column is the card's own text.

Fourteen reasons name vulture. In thirteen of them, the published form leaves a function argument unused, and the coding archetype grades an unused argument high.

Rule	ruff name	Reason on the card
ANN001	missing-type-function-argument	faulty: vulture unused variable 'x'
ANN401	any-type	faulty: vulture unused variable 'x'
S509	snmp-weak-cryptography	faulty: refused at tier 2, the published form holds no binding
B025	duplicate-try-block-exception	faulty: refused at tier 2, the published form holds no binding
B032	unintentional-type-annotation	faulty: mypy valid-type and pyright reportInvalidTypeForm
ARG001	unused-function-argument	faulty: vulture unused variable 'baz'
ARG002	unused-method-argument	faulty: vulture unused variable 'arg2'
ARG003	unused-class-method-argument	faulty: vulture unused variable 'cls' and 'arg2'
ARG004	unused-static-method-argument	faulty: vulture unused variable 'arg2'
ARG005	unused-lambda-argument	faulty: vulture unused variable 'y'
PTH123	builtin-open	faulty: refused at tier 2, the published form holds no binding
N803	invalid-argument-name	faulty: vulture unused variable 'A' and 'myArg'
N804	invalid-first-argument-name-for-class-method	faulty: vulture unused variable 'data'
N805	invalid-first-argument-name-for-method	faulty: vulture unused variable 'cls' and 'data'
F402	import-shadowed-by-loop-var	corrected: vulture unused import 'path'
F633	invalid-print-syntax	faulty: refused at tier 2, the published form holds no binding
F821	undefined-name	faulty: refused at tier 2, the published form holds no binding
F823	undefined-local	faulty: pyright reportUnboundVariable, 'x' is unbound
PLE0115	nonlocal-and-global	faulty: pyright, 'counter' declared global and nonlocal
PLE0117	nonlocal-without-binding	faulty: pyright reportUndefinedVariable and no binding for nonlocal 'bar'
PLE0241	duplicate-bases	faulty: refused at tier 2, the published form holds no binding
PLE0604	invalid-all-object	faulty: mypy list-item, list items 1 and 2
PLE1141	dict-iter-missing-items	faulty: mypy str-unpack, unpacking a string
PLR1722	sys-exit-alias	faulty: refused at tier 2, the published form holds no binding
PLW0211	bad-staticmethod-argument	corrected: vulture unused variable 'sheep'
PLW0245	super-without-brackets	faulty: pyright reportIncompatibleMethodOverride, method 'speak'
PLW1514	unspecified-encoding	faulty: refused at tier 2, the published form holds no binding
UP004	useless-object-inheritance	faulty: refused at tier 2, the published form holds no binding
UP024	os-error-alias	faulty: refused at tier 2, the published form holds no binding
UP037	quoted-annotation	corrected: reads UP037
RUF013	implicit-optional	faulty: vulture unused variable 'arg'
RUF018	assignment-in-assert	corrected: reads RUF018
RUF020	never-union	faulty: refused at tier 2, the published form holds no binding
RUF036	none-not-at-end-of-union	faulty: vulture unused variable 'arg'
TRY201	verbose-raise	faulty: refused at tier 2, the published form holds no binding